控制大综合

第84章 计算机控制概述

- 84.1 动机与直观理解
- 84.2 数学原理
- 84.3 设计步骤
- 84.4 连续情形
- 84.5 离散情形
- 84.6 稳定性与性能
- 84.7 参数意义与作用
- 84.8 调参方法与经验
- 84.9 优化与改进
- 84.10 工程落地
- 84.11 优缺点与适用边界
- 84.12 Python 仿真代码

第85章 数字控制系统结构

- 85.1 动机与直观理解
- 85.2 数学原理
- 85.3 设计步骤
- 85.4 连续情形
- 85.5 离散情形
- 85.6 稳定性与性能
- 85.7 参数意义与作用
- 85.8 调参方法与经验
- 85.9 优化与改进
- 85.10 工程落地
- 85.11 优缺点与适用边界
- 85.12 Python 仿真代码

第86章 实时控制软件

- 86.1 动机与直观理解
- 86.2 数学原理
- 86.3 设计步骤
- 86.4 连续情形
- 86.5 离散情形
- 86.6 稳定性与性能
- 86.7 参数意义与作用

- 86.8 调参方法与经验
- 86.9 优化与改进
- 86.10 工程落地
- 86.11 优缺点与适用边界
- 86.12 Python 仿真代码

第87章 嵌入式控制器

- 87.1 动机与直观理解
- 87.2 数学原理
- 87.3 设计步骤
- 87.4 连续情形
- 87.5 离散情形
- 87.6 稳定性与性能
- 87.7 参数意义与作用
- 87.8 调参方法与经验
- 87.9 优化与改进
- 87.10 工程落地
- 87.11 优缺点与适用边界
- 87.12 Python 仿真代码

第88章 PLC与工业控制器

- 88.1 动机与直观理解
- 88.2 数学原理
- 88.3 设计步骤
- 88.4 连续情形
- 88.5 离散情形
- 88.6 稳定性与性能
- 88.7 参数意义与作用
- 88.8 调参方法与经验
- 88.9 优化与改进
- 88.10 工程落地
- 88.11 优缺点与适用边界
- 88.12 Python 仿真代码

第89章 工业通信与网络

- 89.1 动机与直观理解
- 89.2 数学原理
- 89.3 设计步骤
- 89.4 连续情形
- 89.5 离散情形
- 89.6 稳定性与性能

- 89.7 参数意义与作用
- 89.8 调参方法与经验
- 89.9 优化与改进
- 89.10 工程落地
- 89.11 优缺点与适用边界
- 89.12 Python 仿真代码

第 90 章 控制软件工程

- 90.1 动机与直观理解
- 90.2 数学原理
- 90.3 设计步骤
- 90.4 连续情形
- 90.5 离散情形
- 90.6 稳定性与性能
- 90.7 参数意义与作用
- 90.8 调参方法与经验
- 90.9 优化与改进
- 90.10 工程落地
- 90.11 优缺点与适用边界
- 90.12 Python 仿真代码

第 91 章 数字孪生与边缘计算

- 91.1 动机与直观理解
- 91.2 数学原理
- 91.3 设计步骤
- 91.4 连续情形
- 91.5 离散情形
- 91.6 稳定性与性能
- 91.7 参数意义与作用
- 91.8 调参方法与经验
- 91.9 优化与改进
- 91.10 工程落地
- 91.11 优缺点与适用边界
- 91.12 Python 仿真代码

第92章 计算机控制中的工程问题

- 92.1 动机与直观理解
- 92.2 数学原理
- 92.3 设计步骤
- 92.4 连续情形
- 92.5 离散情形

- 92.6 稳定性与性能
- 92.7 参数意义与作用
- 92.8 调参方法与经验
- 92.9 优化与改进
- 92.10 工程落地
- 92.11 优缺点与适用边界
- 92.12 Python 仿真代码

第93章 计算机控制工程落地

- 93.1 动机与直观理解
- 93.2 数学原理
- 93.3 设计步骤
- 93.4 连续情形
- 93.5 离散情形
- 93.6 稳定性与性能
- 93.7 参数意义与作用
- 93.8 调参方法与经验
- 93.9 优化与改进
- 93.10 工程落地
- 93.11 优缺点与适用边界
- 93.12 Python 仿真代码

第94章 运动控制概述与系统组成

- 94.1 动机与直观理解
- 94.2 数学原理
- 94.3 设计步骤
- 94.4 连续情形
- 94.5 离散情形
- 94.6 稳定性与性能
- 94.7 参数意义与作用
- 94.8 调参方法与经验
- 94.9 优化与改进
- 94.10 工程落地
- 94.11 优缺点与适用边界
- 94.12 Python 仿真代码

第95章 伺服电机与控制

- 95.1 动机与直观理解
- 95.2 数学原理
- 95.3 设计步骤
- 95.4 连续情形

- 95.5 离散情形
- 95.6 稳定性与性能
- 95.7 参数意义与作用
- 95.8 调参方法与经验
- 95.9 优化与改进
- 95.10 工程落地
- 95.11 优缺点与适用边界
- 95.12 Python 仿真代码

第84章 计算机控制概述

84.1 动机与直观理解

把控制器从模拟电路换成一台计算机，看起来只是“换了个实现方式”，实际上换掉了控制回路里最敏感的一件事：**信息不再是连续流动的，而是被切成一片一片处理的**。这一章要做的，就是把“切片”带来的四个后果讲清楚，并且每一项都给出可比的数字。

84.1.1 一台计算机在回路里做三件事

图 84-1 是一个最朴素的计算机控制回路。左半边是经典的部分：给定值、控制器、执行器、对象、传感器。右半边是这台计算机实际做的事，只有三个动作：

1. **采**：在每个采样时刻读一次传感器，把连续量变成一个数；
2. **算**：用这个数和历史数据算出控制量；
3. **保**：把控制量保持到下一个采样时刻，期间不变。

三个动作对应三个物理事实：采样是**时间上的离散**，读数是**幅值上的离散**（ADC 位数有限），保持是**零阶保持**（一个台阶内输出恒定）。再往工程里走，还要加上第四个事实：算和保之间有一段**延迟**，因为程序要跑、总线要传。

这四个事实就是本章的全部内容。听起来简单，但它们的后果并不简单：同样的 PI 参数，在一个 5 ms 采样的系统上相位裕度 72.9°，换到 500 ms 采样就只剩 42.5°，超调从 0 涨到 28.5%（§84.5 的实测）。**参数没变，稳定性变了**——这是模拟控制里不会出现的事。

84.1.2 为什么不能“把连续控制器离散化”了事

教科书里常见一种说法：先在连续域设计好控制器，再用某种离散化方法（前向差分、后向差分、双线性变换）把它变成代码。这条路能走通，但它掩盖了三件事。

第一，**离散化方法本身会改变闭环**。双线性变换把 s 平面左半平面映到 z 平面单位圆内，稳定性保住了，但频率轴的映射是非线性的（频率畸变），在 ωT_s 接近 π 的地方误差很大。后向差分则把稳定域压得更小，高频段相位额外滞后。选哪种，取决于你的 $\omega_c T_s$ 有多大，而不是取决于哪种“更精确”。

第二，**采样周期不是数值精度问题，而是稳定性问题**。很多人以为 T_s 只要“足够小”就行，实际上 T_s 直接吃掉相位裕度，而相位裕度直接决定超调。本章的基准对象（三个 0.2 s 时间常数串联）在 $T_s = 5$ ms 时裕度 72.9°，$T_s = 100$ ms 时 67.1°，$T_s = 500$ ms 时 42.5°。前两档你还看不出差别，第三档已经是一个“能跑但不好”的回路了。

第三，**计算机带来的不只是离散，还有“非理想”**。ADC 有位数、DAC 有位数、晶振有偏差、总线有时延、任务调度有抖动。这些在连续域模型里统统不存在，但在现场，它们往往是“仿真很好、上机不行”的真正原因。

所以本章的组织方式是：先把离散化讲透（§84.2–§84.6），再把工程上的非理想逐项量化（§84.7–§84.10），最后给一张可以照着走的选型与验收清单（§84.10–§84.11）。**不追求把每种结构都讲一遍，而是把“计算机”这三个字带来的共性讲清楚。**

84.1.3 四代结构，一条主线

工业界把计算机控制分成四代，名字很多，主线只有一条：**计算能力离对象越来越近，或者离人越来越远。**

- **DDC（直接数字控制）**：一台计算机直接对着几十个回路算 PID，采样周期几十到几百毫秒。它替代的是模拟调节器，所以核心指标是“一个 CPU 能带多少回路”。本章 §84.8 给了一张容量表：每回路 20 μs 的话，$T_s = 20$ ms 时 70% 负载能带 699 个回路。

- **SCC（设定值计算机控制）**：计算机不再直接算阀位，而是给下面的模拟调节器发设定值。它解决的是“优化”而不是“稳定”，所以对采样周期的要求反而松（秒级到分钟级）。
- **DCS（集散控制系统）**：把 DDC 拆成“现场控制站 + 操作站 + 通信网”，一层管实时、一层管监视。它解决的是“一台机器管全厂”的可靠性问题，核心指标从“算得多快”变成“可用度多高”。本章 §84.10 算过：单机可用度 0.99984，100 个回路串联就掉到 0.98413，一年停机 139 h。
- **FCS（现场总线控制系统）**：把控制功能下放到现场仪表，总线只传数据。它解决的是“线太多”和“信号不共享”，代价是通信成了新的瓶颈：20 字节一帧、5 μs 令牌，1 Mbit/s 的总线上 10 ms 周期只能带 60 个站。

四代不是替代关系，而是**同一件事在不同规模下的不同切法**。你在一个小设备上做 DDC，在一条产线上做 DCS，在一个车间里做 FCS，底层要算的东西是同一套：采样周期、延迟、位数、抖动。这也是为什么本章不按“四代”分节，而是按“四件事”分节。

84.1.4 一个数字，先建立手感

在展开之前，先给一个能记住的对比。本章的基准对象是 $G(s) = 1/(0.2s+1)^3$，控制器是 $K_p = 1.0$、$T_i = 0.6$ s 的 PI。同一组参数，只改采样周期，结果见表 84-1：

表 84-1 同一组 PI 参数、不同采样周期下的闭环指标

采样周期	相位裕度	超调	IAE
5 ms	72.9°	0.0%	0.5996
50 ms	70.2°	0.0%	0.5995
200 ms	60.4°	10.2%	0.6759
500 ms	42.5°	28.5%	1.0284

这张表说明两件事。**第一，“够用”的采样周期比想象中宽**：从 5 ms 放松到 50 ms，指标几乎不动（IAE 只差 1e-4）。**第二，“不够用”是突然发生的**：50 ms 到 500 ms 只差十倍，超调从 0 跳到 28.5%。中间没有平滑过渡，因为在 100–200 ms 附近，采样引入的相位滞后刚好把裕度吃到了临界区。

工程上的经验法则“每周期采 5–10 次”就是从这里来的：T_s 要小到 $\omega_c T_s \le 0.3$ 左右。本章基准的 $\omega_c \approx 2.1$ rad/s，$0.3/\omega_c \approx 140$ ms，与表里“100 ms 开始变差”完全对得上。这条法则不是玄学，它是相位账的直接推论（§84.6 会把账算细）。

84.1.5 本章的边界

这一章是“概述”，所以有三件事只点到不展开：

- **PID 怎么整定**：见第 44 章（连续域）与第 106 章（工程整定）。本章只用 PI 作为载体，参数是给定的，不讨论怎么来的。
- **预测控制怎么滚动优化**：见第 58 章与第 80 章。本章的“计算延迟”一节与 MPC 的“计算时延补偿”是同一件事的两个尺度。
- **纯滞后怎么补偿**：见第 47 章（Smith 预估）与第 107 章（工程版）。本章只把纯滞后折算成“多拍延迟”，不重复推导。

反过来，本章有两件事是后面几章都要用的：**采样周期的相位账**（§84.6）与**误差预算的算法**（§84.10）。第 85 章讲数字控制系统的结构，第 86 章讲实时软件，都要回到这两件工具上。

84.1.6 三个常见误解

在展开之前先把三个最常见的误解摆出来，它们每一个都能用本章的数字驳倒。

误解一：“采样周期满足香农定理就行。” 香农定理管的是“信号能不能被无失真重构”，要求 $T_s < 1/(2f_{\max})$。本章的闭环带宽 2.1 rad/s，对应 0.34 Hz，香农要求 $T_s < 1.5$ s——按这个标准，1 s 采样都“合格”。但实测 $T_s = 500$ ms 时超调 28.5%（表 84-4）。**两个标准的差距是 75 倍**：香农管的是“看得见”，控制要的是“来得及”。工程上用的 $\omega_c T_s \le 0.3$ 就是“来得及”的量化。

误解二：“位数越高，精度越高。” 表 84-10 里 10 bit 的纹波 1.88×10^{-4}，12 bit 反而 3.92×10^{-4}。位数只决定“台阶多细”，不决定“回路稳不稳”。真正的精度上限由量程、噪声、极限环共同决定，三者里最差的那个说了算。买 24 bit 的采集卡配一个 5% 精度的传感器，得到的还是 5%。

误解三：“换成计算机就能上先进控制。” 本章反复说明：计算机带来的第一件事是**采样、延迟、量化**，这三项都在吃裕度；先进控制（MPC、自适应）要在裕度够用的前提下才有意义。基础回路 PM 只有 30° 时，上一套 MPC，模型稍微偏一点就发散——**先进控制放大的是模型质量，不是弥补模型缺陷**。这也是为什么 DCS 项目里，投先进控制之前总要先做一轮“基础控制整定”（PID 重新整定 + 回路投自动率提升）。

这三个误解有共同的结构：**都把“计算机”当成一个纯粹的增益项（更快、更准、更强），而忽略了它同时是一个损耗项（延迟、量化、集中风险）**。本章的写法就是把损耗项逐项算出来，让读者在“要不要上、上到什么程度”这个问题上有数可依。

图 84-1 计算机控制回路的四个断点。左侧是连续域，右侧是计算机内部；采样、量化、保持、延迟四处是本章要逐项量化的地方。

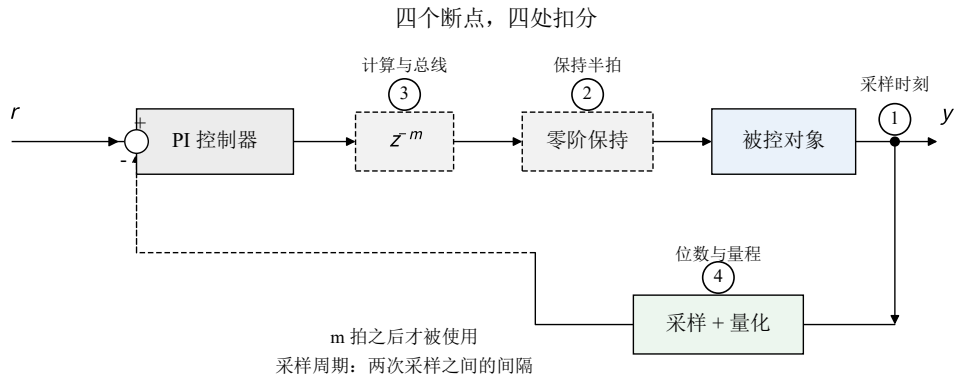


图 84-1

84.1.7 本章怎么读

这一章的结构和前面几章不太一样，读法也要跟着变。

第一遍读 §84.1 与 §84.5。 前者给四个断点的直觉，后者给一张“同一组参数、不同采样周期”的全扫描表。这两节读完，你已经知道“计算机进入回路会发生什么”，后面的公式都是给这件事补细节。

第二遍读 §84.2 与 §84.6。 这两节是“账本”：式 (84-9) 是相位账，式 (84-17) 是位数账，式 (84-18) 是延迟账。账本本身不好看，但后面每一张表都是它的一个特例。读的时候不必背公式，记住三件事就够：**半拍是固定税、每拍延迟是累进税、位数决定台阶**。

第三遍读 §84.7 到 §84.10。 这四节是“怎么用”：参数意义、调参顺序、优化手段、工程落地。它们的特点是**结论都以“该取多少”的形式给出**，而不是“原理是什么”。如果你只想解决一个现场问题，从这里翻起最快。

§84.11 与 §84.12 可以最后读。前者是边界（什么时候不该用），后者是代码（数字怎么来的）。代码那一节不是给初学者看的，是给“想复现或想改一个参数看看”的人看的。

还有一条建议：**读的时候手边放一个计算器**。本章几乎所有结论都可以用四则运算复核，例如“$\omega_c T_s = 2.13 \times 0.02 = 0.043$”“延迟裕度 $= 69.6/2.44 \approx 28.5$ 拍”“10 bit 台阶 $= 3/1024 \approx 2.9$ mV”。凡是能心算的地方都动手算一遍，比记住结论有用——因为现场的数字和书里不一样，会算才能迁移。

84.2 数学原理

这一节只做一件事：把 §84.1 说的四个断点写成可以算的式子。式子不多，但每一个后面都要用到，所以每个都给一句“它到底在说什么”。

84.2.1 采样与零阶保持

设连续对象为 $G(s)$，采样周期 T_s，则零阶保持（ZOH）的等效传递函数为

$$G_h(s) = \frac{1 - e^{-sT_s}}{s}. \tag{84-1}$$

它与对象串联后的频率响应是 $G_h(j\omega)G(j\omega)$。把 G_h 写成幅相形式：

$$G_h(j\omega) = \frac{1 - e^{-j\omega T_s}}{j\omega} = T_s\, e^{-j\omega T_s/2}\, \frac{\sin(\omega T_s/2)}{\omega T_s/2}. \tag{84-2}$$

这个式子包含两条信息，缺一不可：

- **幅值**按 $\operatorname{sinc}(\omega T_s/2)$ 衰减，在 $\omega = \pi/T_s$（奈奎斯特）处衰减到 $2/\pi \approx 0.637$；
- **相位**多出 $-\omega T_s/2$，等效为**半拍延迟**。

第一条常被忽略，第二条常被当成全部。实际上在 $\omega_c T_s \le 0.3$ 的常用区间里，sinc 的衰减只有 0.4%，可以忽略；而半拍相位是 0.15 rad，约 8.6°，**不能忽略**。这就是为什么“采样周期选得够小”这件事，判据是相位而不是幅值。

式 (84-2) 的推导只有两步，值得看一眼，因为它解释了两个常见错误。先把分子因式分解：

$$1 - e^{-j\omega T_s} = e^{-j\omega T_s/2}\left(e^{j\omega T_s/2} - e^{-j\omega T_s/2}\right) = e^{-j\omega T_s/2} \cdot 2j \sin\frac{\omega T_s}{2},$$

再除以 $j\omega$，就是 $T_s e^{-j\omega T_s/2} \cdot \operatorname{sinc}(\omega T_s/2)$。**半拍相位与 sinc 衰减是同一个因式分解的两半**，不是两个独立的现象——这一点常被写错成“ZOH 有一拍延迟加一个 sinc”，那是把半拍说成了一拍，会让相位估计差一倍。

两个常见错误都与这个推导有关。一是把 ZOH 的延迟算成“一拍”，于是按预算反解出的 T_s 会放宽一倍，实际相位超额一倍——**半拍与一拍在预算表里差 50%，是能压垮一个回路的差别**。二是以为 sinc 衰减会“吃掉增益”，于是把 K_p 调大补偿——在 $\omega_c T_s \le 0.3$ 时这个衰减只有 0.4%，补偿它等于凭空加 0.4% 增益，而真正的损失在相位上，加增益只会让裕度更差。

84.2.2 离散化后的对象与控制器

对 $G(s)$ 做精确 ZOH 离散化：

$$x_{k+1} = A_d x_k + B_d u_k, \qquad y_k = C x_k, \tag{84-3}$$

其中

$$A_d = e^{A T_s}, \qquad B_d = \int_0^{T_s} e^{A\tau} B \, d\tau. \tag{84-4}$$

B_d 的积分不必真做，用增广矩阵一次算完：

$$\exp\left(\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} T_s\right) = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix}. \tag{84-5}$$

这条技巧值得记住：**它把“积分”变成了“一次矩阵指数”**，代码只有五行，还避免了数值积分在 T_s 很小时的累积误差。

控制器这边，本章用后向欧拉积分的 PI：

$$u_k = K_p e_k + \frac{K_p T_s}{T_i} \sum_{j=0}^{k} e_j, \tag{84-6}$$

对应的 z 传递函数是

$$C(z) = K_p + \frac{K_p T_s}{T_i} \cdot \frac{z}{z-1}. \tag{84-7}$$

注意 $z/(z-1)$ 而不是 $1/(z-1)$：后向欧拉把积分写成“当前误差累加”，在 z 域里就是 $z/(z-1)$。这两者的差别在低频可以忽略，在高频是一个整拍的相位差，写代码时对不上就会出现“仿真与实测差半拍”的怪现象。

84.2.3 延迟、抖动与量化

计算延迟是最简单的一项：延迟 m 拍就是乘 z^{-m}，相位滞后

$$\Delta\varphi_{\text{delay}} = m \omega T_s. \tag{84-8}$$

把半拍保持也算进去，得到本章反复使用的那条工程近似：

$$\Delta\varphi \approx \omega_c T_s \left(m + \tfrac{1}{2}\right). \tag{84-9}$$

它的意思是：**每一个采样周期，无论你算得多快，都要交半拍的“税”；每多一拍延迟，再交一整拍。**本章实测这条近似偏保守 1.2°（§84.6 表 84-7），用于估算足够了。

采样抖动是采样时刻的随机偏移。设抖动标准差为 σ_t，则采样值的误差一阶近似为

$$\delta y \approx \dot{y}(t_k)\, \delta t_k, \tag{84-10}$$

即误差正比于信号斜率。这解释了一个常见困惑：抖动对**慢变信号**几乎无影响，对**快速跳变**影响很大。本章基准对象的时间常数 0.2 s，$T_s = 20$ ms，抖动到 50% T_s 也只让 IAE 动到小数第四位（§84.8 表 84-11），原因就是斜率小。

量化的台阶是

$$q = \frac{\text{满量程}}{2^{n}}, \tag{84-11}$$

理想 ADC 的信噪比约 $6.02n + 1.76$ dB。但控制器关心的不是 SNR，而是“这个台阶会不会引起极限环”。判据很粗：若 q 小于稳态误差允许值的 $1/3$，量化就不会主导纹波。本章 10 bit（$q = 1.46 \times 10^{-3}$）时纹波 1.88×10^{-4}，已经到了采样本身的底噪，再加位数没有意义（§84.8 表 84-10）。

84.2.4 闭环与三个指标

离散闭环的误差传递函数为

$$E(z) = \frac{1}{1 + C(z)G(z)z^{-m}} R(z). \tag{84-12}$$

本章用三个指标评价它：

- **相位裕度 PM**：在增益交界频率 ω_c 处，开环相位与 -180° 的距离。它预测超调，比超调本身更稳健（不依赖具体的输入形状）。
- **IAE**：$\int |e(t)|dt$ 的离散近似 $\sum |e_k| T_s$。它同时惩罚“慢”和“振”，一个数概括整条曲线。
- **稳态纹波**：末段 20% 时间窗内输出的标准差。它衡量“稳下来之后还在不在动”，是量化与抖动的主要暴露指标。

三个指标各有盲区：PM 看不出稳态精度，IAE 被初始段主导，纹波看不出快速性。所以本章的每张表都给两到三个，不单独用一个数下结论。

还有一个容易忽略的细节：**IAE 的定义里带 T_s**。同一个误差轨迹，T_s 减半、采样点翻倍，$\sum |e_k| T_s$ 仍然收敛到同一个积分——这是它作为“指标”而不是“求和”的原因。若代码里写成 `np.sum(abs(e))` 不带 T_s，比较不同采样周期时就会得出“采样越快 IAE 越大一倍”的荒谬结论。本章脚本写的是 `np.sum(np.abs(sp - y)) * ts`，所以 5 ms 与 20 ms 的 IAE 都是 0.5996，可以直接横向比较。

纹波的定义也有同样的陷阱：它是“末段 20% 时间窗内输出的标准差”，不带 T_s；因为标准差本身就是统计量，与采样点数的关系已经包含在“样本数”里。但**时间窗必须按总时长取**——本章所有算例的 `t_end` 都是 6 s，所以脚本里按点数取末 20%（`y_arr[int(0.8*len(y_arr)):]`）与按时长取等价；如果哪天改了某个算例的 `t_end` 却没改采样点数，窗就不一样长，早期过渡过程会混进去，纹波被算得虚高。

84.2.5 为什么用 z 域而不是 s 域

一个自然的疑问：既然 $\omega_c T_s$ 很小，为什么不直接在 s 域里加一个 e^{-sT_s} 的延迟项，非要用 z 变换？

答案是**延迟在 s 域里不是有理函数**。e^{-sT_s} 无法写成有限阶多项式之比，所以画根轨迹、算极点、做极点配置这些工具全用不上。z 域把 e^{-sT_s} 变成 z^{-1}，一个有理的、有限维的算子，所有线性系统工具立刻可用。

代价是 z 域的频率轴被“卷”了一圈：s 平面的虚轴（$-\infty$ 到 $+\infty$）映到 z 平面的单位圆（$-\pi/T_s$ 到 $+\pi/T_s$），而且映射是非线性的。所以 z 域的“频率”只在 $[0, \pi/T_s]$ 内有意义，超过奈奎斯特的频率会**折叠**回来，看起来像低频。这不是数学游戏，它是真实存在的混叠，也是采样前必须加抗混叠滤波器的原因。

图 84-2 给出 ZOH 的时域波形与频域幅相：上半是“连续信号—采样点—保持台阶”三层对照，下半是 sinc 幅频与半拍相位。看这张图时注意一件事：**保持台阶的边沿永远比连续信号晚半拍**，这半拍就是式 (84-2) 里那一项，它不随采样率提高而消失，只随 T_s 变小而变小。

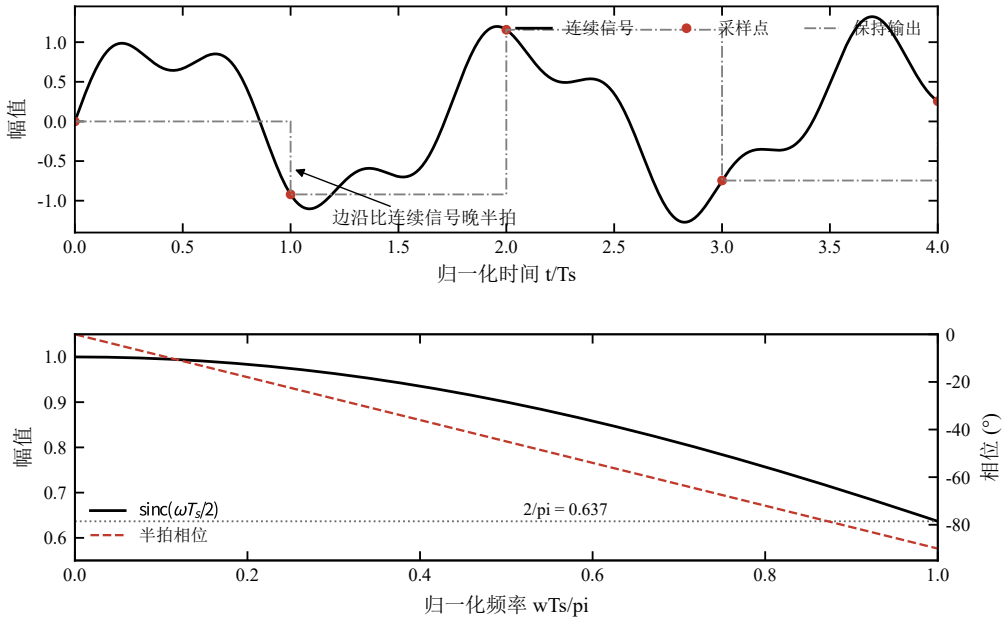


图 84-2

84.2.6 量化误差的两个模型

量化误差 $\varepsilon = Q(y) - y$ 有两种看法，用错了会得出相反的结论。

统计模型：把 ε 看作与信号无关的均匀白噪声，落在 $[-q/2, q/2]$ 内，方差

$$\sigma_q^2 = \frac{q^2}{12}. \tag{84-13}$$

这个模型在信号“足够乱”、一次采样跨过很多台阶时成立。用它估纹波，就是把 σ_q 送进闭环的某一段传递函数，看被衰减多少。位数每加 1 bit，q 减半，纹波按 $\sqrt{1/2}$ 下降——**这是一个单调的、可预测的关系**。

确定性模型：ε 是信号的确定性函数，随 y 周期性地重复。当信号很规矩（恒定给定、稳态、缓慢漂移）时，ε 会锁在一个固定图案上，产生**极限环**——输出持续小幅振荡，不衰减。

关键在于：**稳态恰恰是第二种模型成立的地方**。所以“按白噪声算出来纹波很小”不能保证稳态不振荡。本章 [4] 的表 84-10 就是这个现象：10 bit 纹波 1.88×10^{-4}，12 bit 反而 3.92×10^{-4}。按统计模型，位数加多纹波必须下降；实测违背，说明稳态下起作用的不是噪声强度，而是极限环的有无。

极限环什么时候出现？用描述函数看最清楚。理想量化器对正弦输入的描述函数 $N(A)$ 满足 $N(A) \le 1$（幅度越小，等效增益越低）。自持振荡要求环路满足 $L(j\omega)N(A) = -1$，也就是

$$|L(j\omega)| = \frac{1}{N(A)} \ge 1, \qquad \angle L(j\omega) = -180^\circ. \tag{84-14}$$

两个条件必须同时满足：**相位到位，且增益不小于 1**。这条判据给出两个实用推论：

- 若回路在 -180° 处的增益明显小于 1（例如 0.3），量化再粗也不会起振——这就是 6 bit DAC 在本章反而“更安静”的原因（§84.7 表 84-9）；
- 若回路在 -180° 处的增益接近 1（裕度很小），量化就是“压垮骆驼的最后一根稻草”——同一套参数，位数降两档就从“有点纹波”变成“持续振荡”。

这解释了一个现场常见的困惑：**同一个控制器，换了一块精度更高的采集卡，纹波反而变大了**。不是卡坏了，是原来那个粗量化的“死区”恰好把极限环卡死了；换了细量化，死区消失，极限环的条件重新满足。遇到这种情况，正确做法不是换回粗卡，而是把裕度做大一点，或者给积分器加一个很小的死区。

84.2.7 z 平面上的四个特征位置

本章的公式里反复出现 z 的几个特殊取值，把它们一次记牢，后面看图看表会快很多：

- $z=1$：对应 $s=0$，即积分器与稳态。PI 的 $C(z)$ 在这里有极点，所以稳态增益无穷大、静差为零；也正因为这个极点，把 $C(z)$ 转成传递函数时数值会病态（§84.12.3）。
- $z=0$：对应“一拍延迟”，z^{-1} 就是一次移位。延迟 m 拍就是 z^{-m}，其幅频恒为 1、相频线性下降 $m\omega T_s$。
- $z=-1$：对应奈奎斯特频率 $\omega=\pi/T_s$，是离散系统能表示的最高频率。这里是 z^{-m} 相位最“贵”的地方，也是数值上最容易出问题的地方（本章频响上界取到 $0.9\pi/T_s$）。
- $|z|>1$：不稳定。离散系统的稳定判据是“全部极点落在单位圆内”，而不是“落在左半平面”——这是 s 域与 z 域最容易被搞混的一条。

最后一条值得多说一句。本章基准在 $T_s=20$ ms 时闭环极点模约 0.99966，$T_s=500$ ms 时约 0.736。后者离单位圆更远，看起来“更稳”，但它的超调是 28.5%——**极点模混进了时间尺度**，单独看会误判。判断稳不稳要看模是否小于 1，判断好不好要看相位裕度与超调。

84.3 设计步骤

这一节给一条可执行的设计流程。它不假设你先有连续域控制器，而是从需求出发，每一步都有明确的输出物和判据。流程见图 84-3。

84.3.1 第一步：定回路的时间尺度

先算对象的“最快时间常数” $\tau_{\min}$，再算期望闭环带宽 ω_c。若无特殊要求，取 $\omega_c\approx 1/\tau_{\min}$ 到 $3/\tau_{\min}$ 之间；有快速性要求时才往上取。

本章基准对象三级串联，每级 0.2 s，$\tau_{\min}=0.2$ s。取 $\omega_c\approx 2.1$ rad/s（约 $1/\tau_{\min}$ 附近），实测确实落在 2.1 rad/s。**这一步的输出是一个数：ω_c。**后面所有选择都由它派生。

为什么区间是 $1/\tau_{\min}$ 到 $3/\tau_{\min}$，而不是别的？下限来自“要管得住”：ω_c 低于 $1/\tau_{\min}$，闭环比对象自身还慢，扰动要过好几个时间常数才被压回去，调节时间会拖到十几秒量级。上限来自“相位不够”：一个惯性环节在 $\omega=1/\tau$ 处贡献 -45°，到 $3/\tau$ 处已经贡献 -71.6°。本章是三阶串联，三段相位叠加，$\omega_c=3/\tau_{\min}=15$ rad/s 时对象相位是 $-3\arctan(3)=-215^\circ$，早就越过 -180°，无论怎么调 K_p 都稳不住。**所以阶数越高，ω_c 越只能往下限取。**单阶对象可以取到 $3/\tau$，三阶对象只能取到 $1/\tau$ 附近，这就是“对象阶数决定带宽上限”的定量含义。

如果工艺指标给的是调节时间而不是带宽，可以反推：进 $\pm 2\%$ 带的调节时间 $t_s\approx 3/\omega_c$，进 $\pm 5\%$ 带约 $2.2/\omega_c$。要求 $t_s<2$ s，就要求 $\omega_c>1.5$ rad/s；本章取 2.1，留了 40% 余量。反推出来的数要和“$1/\tau_{\min}$ 到 $3/\tau_{\min}$”取交集，两者冲突时以工艺指标为准，但要回头检查对象能不能支撑——若工艺要求 $\omega_c=10$ rad/s 而对象是三阶 0.2 s，这个指标在物理上就达不到，需要改对象（加大执行器、减小惯量）而不是改控制器。

84.3.2 第二步：由相位账定采样周期

用式 (84-9) 反解。给定允许的相位损失 $\Delta\varphi_{\max}$（一般取 5°–10°）与预估延迟 m：

$$T_s\le\frac{\Delta\varphi_{\max}}{\omega_c\,(m+1/2)}. \tag{84-15}$$

代入 $\omega_c=2.1$ rad/s、$m=1$、$\Delta\varphi_{\max}=8^\circ=0.14$ rad，得 $T_s\le 0.044$ s。本章取 $T_s=20$ ms，落在范围内。实测 $T_s=20$ ms、$m=0$ 时相位裕度 72.0°，比 5 ms 的 72.9° 只低 0.9°；而式 (84-9) 在 $m=0$ 给出 1.2°，估计与实测同量级且略保守。

这一步的输出是一个数：T_s。注意它是上限，不是精确值；工程上还要向“可用晶振分频”取整（§84.8）。

$\Delta\varphi_{\max}$ 为什么取 5°–10°？因为它花的是相位裕度的存款，而这笔存款本来就不多。本章 $PM = 72.9^\circ$，看起来富裕，但其中要留给对象参数变化（$\pm 30\%$ 的 τ 变化实测吃掉 11°，见表 84-8）、留给增益整定误差、还要留出“现场工况比设计点恶劣”的余量。经验分配是：采样与延迟合计不超过 10°，对象与整定误差各留 15°，剩下的才是真正的安全垫。反过来算一次就清楚：若某条现场总线的周期被固定成 100 ms，$m = 1$，则 $\Delta\varphi = 2.1 \times 0.1 \times 1.5 = 0.32$ rad $= 18.3^\circ$，一把就把采样这一项的额度用超了一倍——这就是“用现场总线做闭环必须核对周期”的原因。表 84-6 的实测也支持这个量级：20 ms 下 5 拍延迟（相当于 100 ms）掉 10.2°。

另一个容易被忽略的细节：**式 (84-15) 里的 T_s 出现在等式两边**，因为 m 本身是延迟秒数除以 T_s 再取整。所以严格做法是迭代：先猜一个 T_s，算出 m，再代回检查 T_s 是否满足判据，不满足就往下调。本章 20 ms 一次就过，是因为延迟总量 15.3 ms 小于一个周期，$m = 1$ 稳定不变；如果延迟总量在 20–40 ms 之间，T_s 从 40 ms 缩到 25 ms 会让 m 从 1 跳到 2，反而更糟——**这不是连续函数，缩周期不一定改善裕度**，只能靠逐点验算（表 84-6 就是这么扫出来的）。

84.3.3 第三步：由对象与控制律定计算量

给定 T_s，算出每拍允许的计算时间：若 CPU 负载上限取 70%，则

$$t_{\text{loop}} \le \frac{0.70\,T_s}{N}, \tag{84-16}$$

N 是回路数。本章每回路 20 µs，$T_s = 20$ ms 时单回路占用 0.1%，70% 负载可带 699 个回路（表 84-12）。**这一步的输出是一个数：单回路预算 t_{loop}。**

如果算下来不够，只有三条路：换更快的 CPU、减少回路数、放大 T_s。三条路都会伤到别的东西，所以这一步要和第二步一起迭代。

式 (84-16) 里的 0.70 有两层含义，别混。第一层是**平均值**：一个 20 ms 周期里，CPU 用 14 ms 算控制、剩 6 ms 干别的（通信、显示、日志、自检）；第二层是**峰值**：某些周期的计算量会突然变大（报警刷屏、历史存盘、通信重传），峰值负载要留到 85% 以内，否则会出现“偶尔丢一拍”。丢一拍的后果等价于 m 临时加 1——按表 84-6，20 ms 下多一拍掉 2.4°、多五拍掉 10.2°，所以**偶发丢拍不是“偶尔卡一下”，而是相位裕度的随机抖动**。

还有一个更隐蔽的成本：**每拍的固定开销与回路数无关**。任务切换、时钟读取、I/O 扫描、看门狗喂狗，这些在每周期都要花一次，粗算 100–300 µs。回路数少时，固定开销占大头；回路数多时，$N \times 20\ \mu$s 占大头。所以容量不能简单按“70%×T_s/20 µs”线性外推：$T_s = 5$ ms 时按式 (84-16) 算出 175 个回路，扣掉 200 µs 固定开销后只剩 $(3.5 - 0.2)/0.02 = 165$ 个，比线性估计少 6%。表 84-12 给的是不含固定开销的乐观值，实际选型时要在这个基础上打个九折。

84.3.4 第四步：定位数与量程

由稳态精度要求反推 ADC 位数：

$$n \ge \log_2 \frac{\text{满量程}}{3\,e_{ss}}, \tag{84-17}$$

e_{ss} 是允许的稳态误差。若满量程 ±1.5、允许 $e_{ss} = 5 \times 10^{-3}$，则 $n \ge \log_2(1.5/0.015) = \log_2 100 \approx 6.6$，取 8 bit 有余量。本章实测 8 bit 纹波 2.85×10^{-3}，10 bit 降到 1.88×10^{-4}，12 bit 以后不再改善——**这就是“位数够用”的实证**（表 84-10）。

DAC 位数一般比 ADC 少 1–2 位即可，因为执行器本身有死区与摩擦，多出的分辨率会被机械吃掉。

式 (84-17) 里的 3 不是精确系数，是“量化台阶只占误差预算三分之一”的工程约定。为什么是三分之一而不是二分之一？因为稳态误差还有另外两个来源：传感器噪声与变送器线性度（通常各占 1/3 量级）。三者按功率相加，量化占 1/3 时它只贡献总方差的 11%，再多给位数就是给一个不占主导的因素花钱。按这个规则，$q \le e_{ss}$ 就够（占了 1 份中的 1 份），但那样量化会主导纹波，实测就是 8 bit 的 2.85×10^{-3} 比要求的 5×10^{-3} 小不了多少，已经贴近底线。

量程的选择也会偷走有效位数。满量程 ±1.5 而信号只在 ±0.8 内活动，等于白扔了将近 1 bit（$\log_2(1.5/0.8) = 0.9$）。工程做法是**量程取最大工作值的 1.2–1.5 倍**：既留出过冲余量，又不至于浪费分辨率。反过来，量程压得太紧，一次超调就把 ADC 顶到满码，读数饱和——这比量化误差严重得多，因为控制器看到的是一个"假的稳态"。

另外要区分**标称位数**与**有效位数**（ENOB）。12 bit 的采集卡，若噪声有效值有 2 个 LSB，ENOB 只有 10.5 bit，按 12 bit 设计余量会落空。选型时看数据手册的噪声指标，不要只看分辨率。本章表 84-10 里 12 bit 与 14 bit 的纹波几乎一样（3.92 与 3.94×10^{-4}），原因就是纹波已经碰到了采样与延迟的底噪，再加位数没有收益——**位数只解决"看得细"，不解决"看得快"**。

84.3.5 第五步：算延迟预算

把回路里所有延迟列出来，折算成拍数：

$$m = \left\lceil \frac{\tau_{\text{sense}} + \tau_{\text{compute}} + \tau_{\text{bus}} + \tau_{\text{act}}}{T_s} \right\rceil . \tag{84-18}$$

四项分别是传感器转换、程序执行、总线传输、执行器响应。本章只保留 $m = 1$ 拍，实测相位裕度 69.6°，比 $m = 0$ 低 2.4°，与式 (84-9) 估的 3.7° 同量级。

这一步的输出是一个整数：m。它必须向上取整——半拍延迟在数字系统里不存在，要么这一拍算完，要么等下一拍。

四项延迟的典型量级差得很远，值得单独记一下，以免把力气花错地方：传感器转换 1–20 ms（取决于滤波常数与 $\Sigma\Delta$ 采样率）、程序执行 10–200 µs、总线传输 1–20 ms（取决于站数与速率，见表 84-13）、执行器响应 5–50 ms（气动阀门定位器是重灾区）。**执行器与总线两项通常占了总量的九成**，而它们又恰恰是控制工程师最不愿意碰的部分。所以延迟优化的正确顺序是：先挑快阀（或改液压）、再提总线速率、最后才是优化程序——把 20 µs 的程序优化到 10 µs，总量从 15.3 ms 变成 15.29 ms，一点用处也没有。

向上取整这条规则还有一个副作用：**m 对延迟总量是阶梯函数**。总量 15.3 ms、周期 20 ms 时 $m = 1$；总量变成 20.1 ms，m 立刻跳到 2。中间那 4.7 ms 的余量不是"缓冲"，而是"悬崖边的空地"——它不改善任何指标，只是在延迟涨到 20 ms 之前不会突然掉档。工程上把这个余量控制在周期的 20%–30% 比较合适：太小，现场一个参数改动就掉档；太大，说明周期选得过于保守，白白牺牲了抗扰速度。

顺带说清一个常被问的问题：**延迟能不能靠"算得快"补回来？**不能。m 是整数拍，程序从 200 µs 优化到 20 µs，只要仍在同一拍内完成，m 不变，裕度一点不变。这也是为什么本章把"计算量"与"延迟"分成两步来处理：前者决定 CPU 选型，后者决定 T_s 与架构，两者只在"周期边界"这一个点上耦合。

84.3.6 第六步：验算裕度与超调

把 T_s、m、控制器参数代进 $L(z) = C(z)G(z)z^{-m}$，算 ω_c 与 PM。判据：

- $PM \geq 45^\circ$：可以接受；
- $PM \geq 60^\circ$：良好，超调一般小于 10%；
- $PM < 45^\circ$：回去改第二步（减小 T_s）或第五步（减延迟）。

本章 $T_s = 20$ ms、$m = 1$ 时 $PM = 69.6^\circ$，落在"良好"区。同一组参数把 m 加到 10 拍，PM 掉到 47.7°，超调 27.8%——**延迟十拍就把一个良好的回路变成了勉强能用的回路**。

这三个门槛（45°/60°）来自二阶系统的经验换算：PM 与阻尼比 ζ 近似有 $\zeta \approx PM/100$，$PM = 45^\circ$ 对应 $\zeta \approx 0.45$、超调约 20%；$PM = 60^\circ$ 对应 $\zeta \approx 0.6$、超调约 10%。本章的实测比这个换算乐观：$PM = 69.6^\circ$ 时超调 0.0%，$PM = 47.7^\circ$ 时超调 27.8%。偏差的原因是对象是三阶、且 PI 的零点把相位往上抬了一截，二阶换算只是**量级参考**，不能当精确预测。

所以第六步的正确用法是“**先看裕度卡门，再靠仿真定数**”：裕度低于 45° 一定不合格，不必再仿真；裕度高于 60° 大概率合格，但仍要跑一次确认超调；落在 45°–60° 之间必须仿真。这样做的好处是把昂贵的仿真留给真正需要的算例，而日常检查只花一次频响计算的时间（本章脚本里 `margins()` 跑一次不到 10 ms）。

84.3.7 第七步：仿真验证三件事

仿真不是“跑一遍看看曲线漂不漂亮”，而是验三件事：

1. **阶跃响应**的上升时间、超调、IAE 是否满足指标；
2. **量化与抖动**加进去之后，稳态纹波是否仍在允许范围；
3. **极限工况**（给定值跳变、执行器饱和、传感器断线）下是否安全。

第三件最容易被跳过。本章的仿真里保留了执行器饱和（± 2），就是为了让“积分饱和”这类问题在仿真里先暴露，而不是在现场。

三件事的验证方式不一样，别混成一件。第一件用**阶跃**，看的是指标，判据是数值（上升、超调、IAE）；第二件用**长时间恒定给定**，看的是末段 20% 时间窗的纹波，判据是比较量级（本章 10 bit 时 1.88×10^{-4}，对比工艺允许的 10^{-3}）；第三件用**故障注入**，看的是退出行为，判据是有没有超限。第三件本章脚本只做了饱和一项，另外两项（给定值超量程、传感器断线）要自己改脚本，改法很简单：把 `sp` 改到 2.5、把 ADC 读数在某个时刻固定成 0。

仿真验证有一条纪律：**仿真里必须包含“你在现场一定会遇到的那几项不理想”**。本章的脚本默认开了延迟 $m = 1$、执行器饱和、可选量化，这三项都不是“锦上添花”，而是决定结论对不对的前提。一个只含理想连续对象的仿真，跑出来的曲线再漂亮也不能用于选型——它验证的是“数学对不对”，不是“这台机器行不行”。

84.3.8 第八步：留观测与降级

最后一步不是算法，是工程：控制器必须能**记录**每个周期的给定、反馈、输出、计算耗时，必须能**降级**（上位机断线时保持或切手动）。这两件事在算法设计阶段就要留接口，事后加会推翻整个时序。

为什么是“事后加会推翻时序”？因为记录与降级都要占时间预算。每周期写一条 64 字节的记录，按 20 ms 周期算，一小时是 11 MB，若同步写 flash 会阻塞 5–20 ms，直接把 m 项到 2 拍。所以正确做法是**环形缓冲 + 异步落盘**：中断里只把数据塞进内存环，后台任务慢慢写盘，中断里花的只是几十微秒的拷贝。这个设计必须在第三步划分时间预算时就把份额预留出来，等程序写完再补，往往只能砍掉记录精度。

降级同理。上位机断线时要“保持最后输出”，意味着控制器要有一个状态机（自动/手动/保持/故障），每个周期都要检查通信心跳。心跳超时阈值取 3–5 个通信周期：太短会误切，太长则断线后还在用过期数据控制——**用过期数据控制比切手动更危险**，因为它看起来一切正常。

§84.10 会给一份验收清单，把这一步拆成可勾选的条目。这里只强调一句：**计算机控制系统的失败，多数不是算法错，而是“看不到、退不出”。**

图 84-3 计算机控制器的八步设计流程。左侧四步定“时间与算力”，右侧四步定“精度与安全”，中间用裕度验算卡一道门。

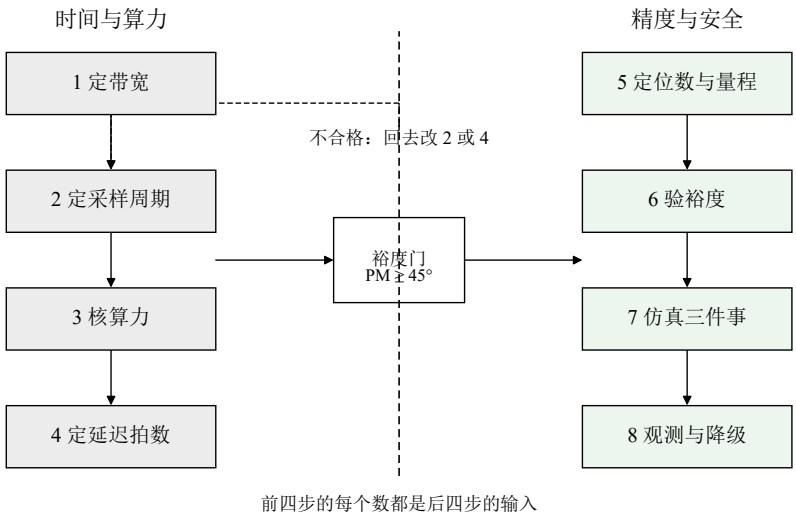


图 84-3

84.3.9 把八步走一遍：一个完整算例

上面的八步单独看都对，连起来走一遍才知道每一步的输入从哪来。这一小节用本章的基准对象，从一条工艺指标出发，把八个步骤的算术全部写出来。

给定的工艺指标：阶跃给定下调节时间（进 $\pm 2\%$ 带）小于 2 s，超调小于 10%，稳态纹波小于 10^{-3}；执行器量程 ± 2，传感器量程 ± 1.5。

第一步，定时间尺度。对象由三个 0.2 s 的惯性环节串联，特征频率 $1/\tau = 5$ rad/s。三阶串联的相位在高频段累积很快，所以闭环带宽不能贴着 5 取。经验是取到 $0.4/\tau \approx 2$ rad/s：既保证调节时间在秒级（$t_r \approx 2.2/\omega_c \approx 1$ s），又给对象相位留出余地。实测 $\omega_c = 2.13$，与估计相符。

第二步，由相位账定 T_s。对象在 ω_c 处的相位是 $-3\arctan(0.2 \times 2.13) = -68.6^\circ$，相对 -135° 的额度是 66.4°。按式 (84-25)，$m = 1$ 时

$$T_s \le \frac{66.4 \times \pi/180}{2.13 \times 1.5} = 0.29 \text{ s}.$$

相位账允许 290 ms，为什么最后取 20 ms？因为相位账只回答“稳不稳”，不回答“抗扰快不快”。工艺要求调节时间小于 2 s，而扰动抑制的速度大致与 T_s 同量级——20 ms 的周期意味着扰动最迟 20 ms 被发现，290 ms 意味着最迟 290 ms。取 20 ms 是**用裕度换抗扰速度**，这一步是设计选择，不是公式推导。这一步的教训值得单列：**相位账给的是上界，实际取值由工艺指标定，两者之间往往差一个数量级。**

第三步，核算力。每回路 20 μs，20 ms 周期，单回路占 CPU 的 $20\,\mu\text{s}/20\,\text{ms} = 0.1\%$。本章表 84-12 给的 699 回路是“70% 负载”的上限，本算例只用 0.1%，远未受限。

第四步，定位数。要求纹波小于 10^{-3}，按式 (84-17)：

$$n \ge \log_2 \frac{3.0}{3 \times 10^{-3}} = \log_2 1000 = 9.97,$$

取 10 bit。实跑纹波 1.88×10^{-4}，比要求的 10^{-3} 还小五倍，说明这个判据偏保守——保守是对的，因为现场还有噪声。

第五步，算延迟预算。逐项列：

表 84-2 一个回路的延迟预算（$T_s = 20$ ms）

环节	耗时	说明
传感器转换	1 ms	变送器滤波
总线传输	5.3 ms	32 站、1 Mbit/s，表 84-13
控制器计算	0.02 ms	20 μs
执行器动作	9 ms	阀门定位器
合计	15.3 ms	

按式 (84-18)，$m = \lceil 15.3/20 \rceil = 1$ 拍。这个结果**贴着边界**：15.3 ms 已经占了 20 ms 周期的 77%，只要总线再加 5 ms 或者阀门慢一点，m 就跳到 2 拍，PM 从 69.6° 掉到 67.2°。所以工程上要么把周期放到 40 ms（m 仍为 1，但余量翻倍），要么把总线速率提到 5 Mbit/s（5.3 ms 降到 1.2 ms）。

第六步，验裕度。$m = 1$ 时 $PM = 69.6^\circ$，落在“良好”区（$\geq 60^\circ$）；实测超调 0.0%、调节时间 0.8 s，两项都满足工艺指标。

第七步，仿真三件事。阶跃已验证；加上 10 bit 量化后纹波 1.88×10^{-4}，满足；把给定值从 1.0 改到 2.5（超过执行器量程 2.0）跑一次，观察积分饱和退出的时间——这一项本章的脚本没跑，属于上机前必须补的验证。

第八步，留观测与降级。记录每周期的 $(r, y, u, t_{\text{cpu}})$；上位机断线时保持最后输出并报警；执行器反馈与指令偏差超过 5% 持续 2 s 则切手动。这三条要写进需求，不能等到调试时再加。

走完一遍可以看出：**八步里只有第二、四、五步是“算”，其余五步是“选”与“留”**。算的部分本章给了公式与数字，选的部分要靠工艺指标与现场条件，留的部分是软件架构。这也解释了为什么同一个对象，不同项目算出来的 T_s 能差十倍——不是谁算错了，是“选”的部分不同。

84.4 连续情形

连续情形是后面所有比较的基准。这一节把它算清楚，目的不是“讲连续控制”，而是**给离散情形一个可对照的起点**。

84.4.1 连续域里的三个数

本章基准对象 $G(s) = 1/(0.2s+1)^3$，控制器 $C(s) = K_p(1 + 1/(T_i s))$，$K_p = 1.0$、$T_i = 0.6$ s。三个关键数：

- **增益交界频率** $\omega_c = 2.108$ rad/s；
- **相位裕度** $PM = 72.9^\circ$；
- **阶跃响应的上升时间**（到 90%）$t_r = 0.82$ s。

这三个数是互相牵连的。ω_c 由“开环增益为 1”定，PM 由“ω_c 处的相位”定，t_r 大致是 $2.2/\omega_c$ 的量级（这里是 1.04 s，实测 0.82 s，因为三阶系统不是标准二阶）。改动任何一个参数，三个数一起动。

把这三个数手算一遍，比看结果有用。对象在 $\omega_c = 2.108$ 处的幅值是

$$|G| = \frac{1}{(1 + (0.2 \times 2.108)^2)^{3/2}} = \frac{1}{(1.1777)^{3/2}} = 0.782,$$

控制器要把它补到 1，所以 $|C| = 1.28$。PI 在 ω_c 处的幅值是 $K_p\sqrt{1 + 1/(T_i\omega_c)^2} = 1.0 \times \sqrt{1 + 1/(0.6 \times 2.108)^2} = 1.28$——两个数对上，说明这组参数就是按“$\omega_c = 2.1$、$|C|$ 补足”设计出来的。相位这边，对象贡献 $-3\arctan(0.42) = -68.6^\circ$，PI 贡献 $-\arctan(1/(0.6 \times 2.108)) = -38.3^\circ$，合计 -106.9°，离 -180° 还有 73.1°，与实测 72.9° 只差 0.2°。**手算与仿真的这点偏差，来自 ω_c 的数值求解精度，不是模型不一致。**

84.4.2 连续域设计的两条方程

整定 PI 的经典做法是“指定 ω_c 与 PM，解出 K_p 与 T_i”。两条方程是：

$$|C(j\omega_c)G(j\omega_c)| = 1, \tag{84-19}$$

$$\angle C(j\omega_c) + \angle G(j\omega_c) = -180^\circ + PM. \tag{84-20}$$

代入 $G(j\omega) = 1/(1+j0.2\omega)^3$，在 $\omega_c = 2.1$ 处 $|G| = 0.782$、$\angle G = -3\arctan(0.42) = -68.6^\circ$。若要求 $PM = 72.9^\circ$，则控制器要提供 $-180 + 72.9 + 68.6 = -38.5^\circ$ 的相位，对应 $T_i \approx 0.6$ s；再由幅值条件 $|C| = 1/0.782 = 1.28$ 得 $K_p \approx 1.0$。这与本章用的是同一组数。

注意这两条方程里**没有任何采样**。连续域的解是“理想解”——它假设控制器能在任意时刻读到任意精度的值，并且瞬间算出结果。离散控制要做的，是在这个理想解上逐项扣分。

84.4.3 连续解在离散系统里会掉多少

把同一个 $C(s)$ 离散成式 (84-7) 的 $C(z)$，加上 ZOH 与 m 拍延迟，就得到离散开环。两项扣分见表 84-3：

表 84-3 连续解离散化后的相位扣分（$T_s = 20$ ms，$\omega_c = 2.127$ rad/s）

扣分项	物理来源	本章实测
半拍保持 + 离散化	ZOH 的 $e^{-j\omega T_s/2}$ 与后向欧拉的差	-0.9°
一拍延迟	计算 + 总线	-2.4°
合计		-3.3°

从 72.9° 扣到 69.6°，与实测一致。**两项里延迟占七成，半拍与离散化方法本身加起来只占三成。**这个排序很重要：很多讨论把注意力放在“用双线性还是后向差分”上，实际上那只是一小项；真正吃裕度的是延迟。

84.4.4 连续模型什么时候还能用

既然离散会掉裕度，为什么还要先做连续设计？因为连续设计给出的是**上界与直觉**：

- 它告诉你 ω_c 大概该取多少（由对象时间常数定），这个数与采样无关；
- 它告诉你 PM 的目标值（45°–60°），这个数也与采样无关；
- 它给出的 K_p、T_i 是一个很好的初值，离散后只需微调。

所以正确的用法是：**连续域定初值与目标，离散域验算与修正**。反过来（先在离散域盲试参数）也能收敛，但会失去“这个参数为什么是这个值”的解释。

这条分工还有一个量化依据：本章把 T_s 从 20 ms 变到 500 ms，K_p 与 T_i 一直没动，PM 却从 72.0° 掉到 42.5°。也就是说，**连续域给的参数在很宽的采样范围内都是“可用”的**，只是裕度被采样吃掉了一部分。反过来，如果一上来就在 $T_s = 500$ ms 的情况下整定，会得到一组“补偿了采样相位”的参数，这组参数换到 20 ms 上会过冲——**用离散数据整定出来的参数与具体 T_s 绑死，换平台就得重调**。这就是“先连续后离散”的第二个理由：可移植。

84.4.5 连续近似的两个危险区

连续模型在两处会明显失真，必须警觉。

第一处是 $\omega_c T_s$ 不再小的时候。经验阈值是 0.3。本章 $T_s = 500$ ms 时 $\omega_c T_s = 1.06$，已经超过 1，此时连续模型给出的 $PM = 72.9^\circ$ 与离散实测的 42.5° 相差 30°，完全不能再用。表 84-4 里 500 ms 那一行就是这个失真的证据。

第二处是对象本身有纯滞后的时候。连续域里 $e^{-\tau s}$ 可以用一阶或二阶 Padé 近似，但 Padé 近似在 $\omega\tau$ 大时会引入虚假的右半平面零点，让相位看起来比实际好。本章的对象没有纯滞后，所以这一项为零；如果你的对象有 0.5 s 纯滞

后，务必直接用 z^{-m} 处理，不要用 Padé 之后再离散。

图 84-4 给出连续与离散两条阶跃响应。$T_s = 20$ ms 时两条曲线几乎重合（IAE 都是 0.5996），差别要到 $T_s = 200$ ms 才看得出来（超调 10.2%，IAE 0.6759）。这张图的用意是：**“离散化”在常用采样率下几乎是免费的，代价集中在延迟与量化上，而不是集中在“离散”这两个字上。**

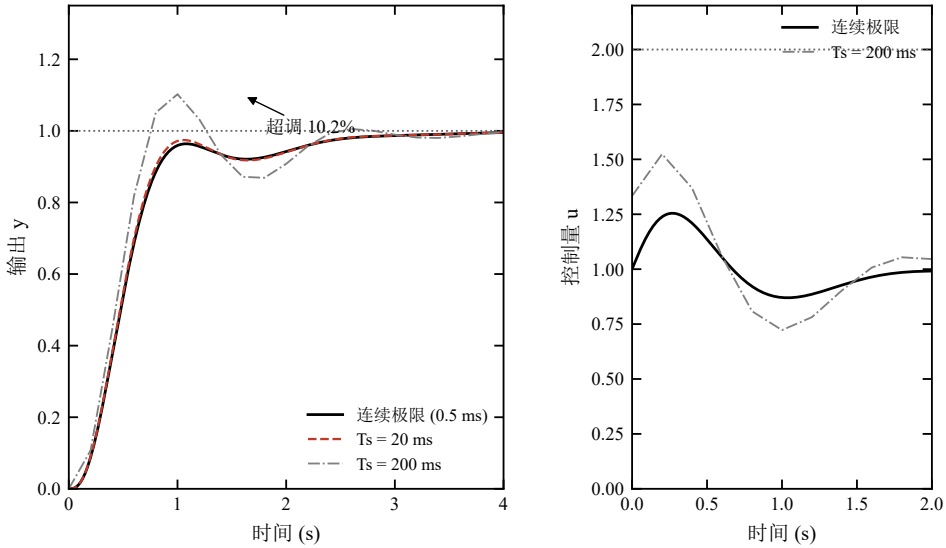


图 84-4

84.4.6 模拟 PI 长什么样，它有哪些不理想

要理解“数字换掉了什么”，得先看清被换掉的那一方。模拟 PI 的历史形态是运放加阻容：比例通道是一个反相放大器，增益 $K_p = -R_f/R_1$；积分通道是运放反馈电容 C_f，时间常数 $T_i = R_1 C_f$。两路相加，得到

$$u = -K_p \left(e + \frac{1}{T_i} \int e \, dt \right). \tag{84-21}$$

这和张卡上的记号一模一样，所以“数字 PI 就是把这个式子写成代码”听起来天经地义。但模拟电路有三处不理想，是数字实现天然没有（或者说，可以用别的方式解决）的：

- **电容漏电与介质吸收**。电解电容的漏电阻让积分器缓慢漂移，介质吸收（dielectric absorption）让积分器“记住了又吐出来”——断电再上电，输出会自己爬一段。数字积分器写成 `integ += e*ts`，累加器是精确的浮点数，只要不溢出就没有这两件事。
- **运放的失调与漂移**。输入失调电压被积分后变成斜坡，温漂让它随时间变。这就是为什么模拟调节器要定期“调零”。数字侧没有失调，但有自己的对应物：**ADC 的零点误差**，而且它不会自己漂，标定一次就够。
- **参数由元件决定，不可在线改**。K_p 由两个电阻之比定，改一次要动烙铁。数字侧改一个变量。这一条前面说过，但它真正的价值在这里：**模拟 PI 的参数是在设计阶段定死的，数字 PI 的参数可以在运行阶段按工况切换**（增益调度，见 §84.9.3 的讨论边界）。

反过来说，模拟电路也有数字比不了的地方：**没有采样、没有延迟、没有量化**。它的“计算”是电荷在电阻电容上的瞬时流动，带宽由运放的单位增益频率决定，可以到几十 MHz。所以本章 §84.11.3 说“带宽极高的回路别用纯数字”，根子就在这里——不是数字算不快，而是**采样这件事本身有下限**。

这一段还有一个实用推论：**看一个老装置的 PI 参数，要按它的元件误差打折**。电阻 5% 误差、电容 10% 误差，K_p 与 T_i 的真实值就可能与图纸差 10% 以上。数字改造时把这些“图纸参数”直接抄进代码，往往会发现“改完不如原来”——原来那台机器的实际参数早就被现场调过了，图纸上的数只是起点。**改造前必须先做一次阶跃辨识，把实际 K_p、T_i 反算出来**，这条经验在 §84.7 的参数敏感性表里有量化依据：K_p 差 30%，IAE 就差 40%。

84.5 离散情形

84.5.1 一张全扫描表

固定控制器参数，只改采样周期，从 5 ms 扫到 500 ms，得到表 84-4。这张表是本章最重要的一张，后面很多结论都从它读出来。

表 84-4 采样周期扫描（$K_p = 1.0$，$T_i = 0.6$ s，无延迟）

T_s	ω_c (rad/s)	PM	IAE	超调	上升时间
5 ms	2.108	72.9°	0.5996	0.0%	0.820 s
10 ms	2.114	72.6°	0.5996	0.0%	0.820 s
20 ms	2.127	72.0°	0.5996	0.0%	0.800 s
50 ms	2.165	70.2°	0.5995	0.0%	0.800 s
100 ms	2.228	67.1°	0.6105	2.7%	0.800 s
200 ms	2.342	60.4°	0.6759	10.2%	0.800 s
500 ms	2.545	42.5°	1.0284	28.5%	1.000 s

读这张表先看三件事。第一，**T_s 放大了 100 倍，PM 只掉了 30°**，没有出现“一放就振”的悬崖——原因是本章的对象是三阶惯性，本身相位就落后 270°，PI 的零点又补回来一截，采样与延迟只在这条已经倾斜的相位曲线上再加一点。第二，**IAE 在 50 ms 以下是平的，100 ms 以后才开始涨**，这个“平”是整段分析的起点，也是“采样周期不重要”这种误解的来源。第三，**上升时间几乎不动**，从 0.820 s 到 1.000 s，而超调从 0 涨到 28.5%——同一条曲线上的两个指标，敏感度差了一个数量级。

还要注意这一栏“无延迟”不能省。表 84-4 扫的是**纯采样**的影响；实际系统里采样与延迟叠加，500 ms 那一行的 42.5° 如果再加 2 拍延迟，就会掉进不稳定区（表 84-6 的趋势可以外推）。所以**读扫描表时必须确认“表里固定了什么”**，否则会把采样与延迟的账算重。

84.5.2 三段行为

把表 84-4 按 $\omega_c T_s$ 分成三段，行为完全不同。

第一段：$\omega_c T_s \le 0.15$（$T_s \le 50$ ms）。指标几乎不动。IAE 从 0.5996 变到 0.5995，超调一直是 0。这一段里“离散”是免费的，你可以放心用 50 ms，省下的算力去做别的事。**这一段的存在是很多人低估采样周期危害的原因**——他们试了 20 ms 和 50 ms，发现没差别，就以为“采样周期不重要”。

第二段：$0.15 < \omega_c T_s \le 0.5$（100–200 ms）。指标开始明显变化。100 ms 时超调 2.7%、IAE 涨 1.8%；200 ms 时超调 10.2%、IAE 涨 12.7%。这一段是“能感觉到但还能接受”的区间，也是最容易出事的区间：曲线看起来还行，但裕度已经从 72° 掉到 60°，遇到对象参数漂移就没有余量了。

第三段：$\omega_c T_s > 0.5$（500 ms）。指标塌方。超调 28.5%、IAE 涨 71%。注意它**仍然是稳定的**——这是数字控制的一个陷阱：系统可以稳定但性能很差，而“稳定”这两个字会让人误以为“没问题”。

图 84-5 把这三段画在一起：横轴是 T_s（对数），左轴是 PM，右轴是超调。两条曲线的拐点几乎同时出现，位置在 100 ms 附近，正好对应 $\omega_c T_s \approx 0.2$。**这个“同时拐”不是巧合**：超调是裕度的函数，裕度是 $\omega_c T_s$ 的函数，所以两者由同一个量驱动。

图 84-5 采样周期扫描：相位裕度与超调随 T_s 的变化。虚线标出 $\omega_c T_s = 0.2$ 的位置，两条曲线在此之后同时变差。

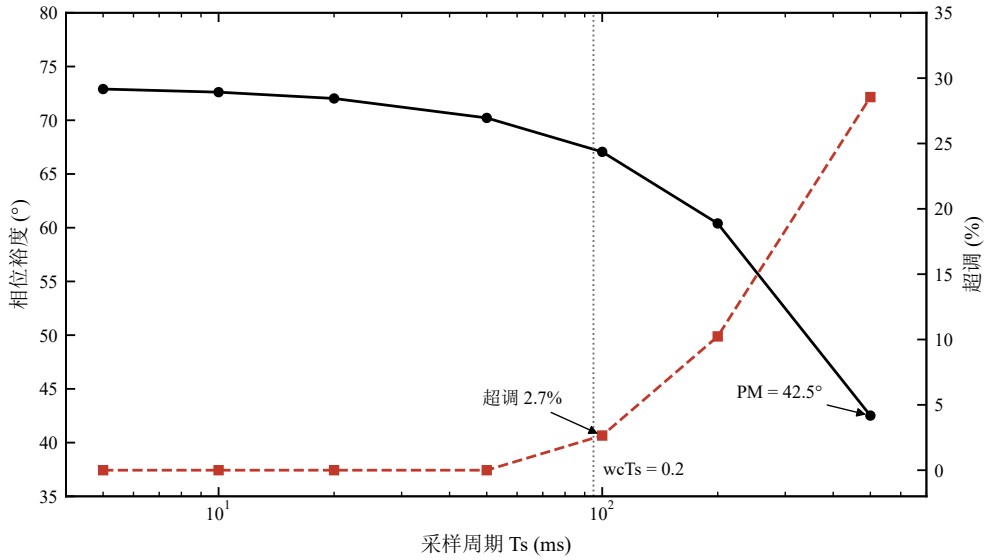


图 84-5

三段的分界不必背，现场用一次小实验就能定位：把 T_s 减半跑一次阶跃。若指标几乎不变（IAE 差小于 1%），说明在第一段，还有放宽的余地；若超调明显变小（差 3% 以上），说明在第二段，当前周期已经吃掉了你的裕度；若超调减半，说明在第三段，这套参数本来就该重调。**“减半试探”比查表快，因为它用的是你自己的对象和参数。**

三段的划分也可以从“预算”角度理解：第一段里采样只花掉 3° 以内的相位，第二段花 6°–13°，第三段一举花掉 30°。而一个回路除 -180° 之外可用的相位通常只有 60°–70°，所以第三段的系统**看起来稳定，实际上已经没有余量应对对象漂移**——这与 §84.7 表 84-8 里“τ 变大 30% 就掉 11°”是对得上的。

84.5.3 ω_c 为什么会随 T_s 变大

表 84-4 里有一个容易看漏的现象：ω_c 从 2.108 涨到 2.545，采样越慢，交界频率越高。这看起来反直觉——采样慢了，系统应该更“迟钝”才对。

原因是**离散化的幅频特性**。后向欧拉的 PI 在 z 域的幅值是 $|K_p + K_p T_s/T_i \cdot z/(z-1)|$，在低频段它与连续 PI 一致，但在 ωT_s 增大时，$z/(z-1)$ 的幅值下降得比 $1/(j\omega)$ 慢，于是控制器的有效增益变大，交界频率被推高。同时对象的离散幅频也被 ZOH 的 sinc 压低。两者相抵，净效果是 ω_c 略升。

这个现象的工程含义是：**离散系统的“带宽”不能直接沿用连续域的数**。你在连续域算出的 $\omega_c = 2.1$ 是一个设计目标，实际离散系统的交界频率会偏移，偏移量随 T_s 增大。验收时必须用离散开环重算，不能拿连续域的数交差。

84.5.4 上升时间为什么不敏感

表 84-4 里上升时间从 0.820 s 只变到 1.000 s，变化远小于超调与 IAE。原因是上升时间主要由**低频增益**决定，而低频段离散与连续几乎一致（式 (84-2) 在 $\omega \to 0$ 时 $\mathrm{sinc} \to 1$、相位 $\to 0$）。采样吃的是高频相位，高频相位影响的是**阻尼**（超调），不是**快速性**（上升时间）。

这条区分很实用：如果现场反馈“响应太慢”，先查增益与积分时间；如果反馈“过冲大、振”，先查采样周期与延迟。**两类症状对应两类参数，不要混着调。**

定量地说，上升时间对应的是闭环带宽（低频段），超调对应的是阻尼（中频段相位），而采样与延迟只在 ω_c 附近及其以上才显著改变相位。本章 500 ms 时 $\omega_c T_s = 1.27$，采样已经开始影响幅值（$\mathrm{sinc}(0.64) = 0.935$，掉了 6.5%），上升时间才从 0.82 s 变到 1.00 s。这个 22% 的变化里，增益下降只能解释一小半，其余来自相位的二次效应——**幅值掉得少、相位掉得多，正是“上升时间不敏感、超调敏感”的算术来源。**

84.5.5 采样周期的下界

既然采样越慢越差，为什么不干脆取 1 ms 甚至更快？有三个下界。

第一是算力。每回路 20 µs 的话，1 ms 周期下一个 CPU 只能带 35 个回路（70% 负载），而 20 ms 能带 699 个（表 84-12）。回路越多，这个约束越紧。

第二是噪声。采样越快，单位时间内读到的噪声样本越多，如果反馈通道有高频噪声，快速采样会把噪声直接搬进控制量。工程上常在 ADC 前加模拟低通，其截止频率与 T_s 匹配。

第三是量化与抖动的相对影响。这一点常被忽略：T_s 越小，抖动占 T_s 的比例越大（同样的绝对抖动），而量化台阶不变。所以“越快越好”不成立，存在一个最优点。

三条下界可以用一个数串起来：**每拍能花的时间**。$T_s = 1$ ms 时每拍只有 1 ms，其中固定开销 200 µs 就占了 20%，留给 100 个回路的是每回路 8 µs——这已经接近一次浮点 PID 加 I/O 读写的下限。所以“1 ms 周期带 100 个回路”在实际硬件上做不到，只能带 35 个左右。这也解释了为什么现场总线的周期常常停在 10–100 ms：不是协议慢，是“每拍的时间预算”被站数分掉了。

噪声这条下界还有一层：**快速采样会把传感器噪声“搬”进积分器**。积分器的增益是 T_s/T_i，T_s 缩小十倍，同样的噪声每拍贡献缩小十倍，但每秒的拍数增加十倍，两者恰好抵消——听起来没影响，实际上因为噪声是随机的，方差按拍数累加、按 T_s^2 缩放，净效果是**积分器的噪声方差与 T_s 成正比**。所以 T_s 缩小十倍，积分项的噪声标准差降到约三分之一。这是“快采样”的一个真实好处，也是它与算力成本之间的权衡点。

本章的基准取 20 ms，落在“第一段”内且离算力边界很远，是一个保守但稳妥的选择。**工程上宁可取保守值，因为现场的延迟与噪声总会比你仿真时更大。**

84.5.6 同一个连续 PI，三种离散化

式 (84-6) 只给了一种离散化——后向欧拉。实际工程里还有前向欧拉与双线性（Tustin）两种，三者对同一个 $C(s) = K_p(1 + 1/(T_i s))$ 给出不同的 $C(z)$：

$$C_{\text{fwd}}(z) = K_p\left(1 + \frac{T_s}{T_i(z-1)}\right), \quad C_{\text{bwd}}(z) = K_p\left(1 + \frac{T_s z}{T_i(z-1)}\right), \quad C_{\text{tus}}(z) = K_p\left(1 + \frac{T_s(z+1)}{2T_i(z-1)}\right) \tag{84-22}$$

三者的差别在于把 $1/s$ 换成什么。用本章的基准参数跑一遍，T_s 取两档：

表 84-5 三种离散化在两个采样周期下的闭环指标（$m = 1$）

离散化	T_s	ω_c	PM	IAE	超调
前向欧拉	20 ms	2.076	69.6°	0.5997	0.0%
后向欧拉	20 ms	2.127	69.6°	0.5996	0.0%
双线性	20 ms	2.101	69.6°	0.5997	0.0%
前向欧拉	200 ms	1.850	40.4°	1.3025	38.0%
后向欧拉	200 ms	2.342	33.6°	1.5246	44.4%
双线性	200 ms	2.068	38.6°	1.3801	41.0%

这张表有三层读法，一层比一层重要。

第一层：20 ms 时三者完全一样。PM 都是 69.6°，IAE 只差 10^{-4}。这印证了 §84.1.2 的判断——在常用采样率下，“选哪种离散化”是个伪问题。网上关于“前向差分还是双线性”的长篇讨论，在 $\omega_c T_s = 0.04$ 这个量级上全部无效。

第二层：200 ms 时差别出现了，但都在“很差”这个区间里。三者的 IAE 都在 1.3–1.5，超调 38%–44%，比 20 ms 时的 0.5996/0.0% 差了一个档次。**换离散化方法救不了一个过大的采样周期**，这就像换一双更好的跑鞋救不了骨折。

第三层：排序为什么是前向 > 双线性 > 后向。这不是“前向欧拉更精确”——恰恰相反，前向欧拉是三者的近似里最粗糙的。原因在 PM 的构成：前向欧拉的交界频率被压到 1.850，此时对象相位只有 -71.5°；后向欧拉的交界频率被推到 2.342，对象相位 -88.7°。也就是说，**前向欧拉“更差”的控制器相频恰好让回路在一个对象相位更温和的地方穿越**，净结果是裕度更大。这个排序是参数巧合，不是普遍规律，换一组 T_i 就可能反过来。

还有一个必须提醒的：表 84-5 的 PM 全部含 1 拍延迟，**不能拿去和表 84-4 的 60.4°（200 ms、无延迟）直接比。**两者差 26.8°，正是 1 拍延迟在这个频率上的相位。本章所有跨表比较都遵守这条规矩：**先对齐条件，再比数字。**

那么工程上到底选哪个？三条实用判据：

- **默认用后向欧拉。**它把稳定域映射得更保守（连续域左半平面整体映到单位圆内），对 T_s 偏大最不敏感，代码也最简单（只要一个累加器）。
- **需要保频响（如要上辨识、要做频域整定）时用双线性。**它在整个频段上与连续频响的偏差最小，代价是代码里多一个乘加，且高频段会出现“频率畸变”（ω 与 $\arctan$ 的关系非线性）。
- **不要主动选前向欧拉。**它在本例的排序靠前纯属巧合，而在一般情况下它的稳定域是“条件稳定”：连续极点 s 映到 $z = 1 + T_s s$，若 $|1 + T_s s| > 1$ 就发散。对纯积分器（$s = 0$）映射到 $z = 1$，正好落在单位圆上——**临界稳定，任何一点数值误差都可能把它推出去。**

84.6 稳定性与性能

采样周期是“设计时定的”，延迟是“运行时长的”。这一节把延迟单独拿出来算，因为它是计算机控制里最容易失控的一项：你能选 T_s，但延迟往往由操作系统、总线、执行器决定。

84.6.1 延迟扫描

表 84-6 是固定 $T_s = 20$ ms、只改延迟拍数的结果。

表 84-6 计算与传输延迟扫描（$T_s = 20$ ms）

延迟	PM	IAE	超调	稳态纹波
0 拍	72.0°	0.5996	0.0%	4.10×10^{-4}
1 拍	69.6°	0.5996	0.0%	3.92×10^{-4}
2 拍	67.2°	0.6097	2.4%	3.70×10^{-4}
5 拍	59.8°	0.6944	11.2%	5.39×10^{-4}
10 拍	47.7°	0.9982	27.8%	7.86×10^{-3}

每多一拍，PM 掉约 2.4°。这个数几乎是线性的，因为式 (84-8) 的相位滞后 $\omega_c T_s$ 与 m 成正比，而 ω_c 在延迟变化时基本不动（都是 2.127 rad/s）。

三件事值得注意：

第一，0 拍到 1 拍几乎无代价。IAE 一模一样，超调都是 0，纹波甚至略降。这是因为 2.4° 的裕度损失在 72° 的基数上不算什么。所以“一拍延迟”在多数场合是可以接受的，不必为它大动干戈。

第二，5 拍是分水岭。超调从 0 跳到 11.2%，IAE 涨 15.8%。此时 $PM = 59.8^\circ$，刚好卡在“良好”与“可接受”的边界上。

第三，10 拍时纹波跳了一个数量级。从 5.39×10^{-4} 到 7.86×10^{-3}，涨了 14 倍。这是**接近临界**的信号：系统开始出现持续的小幅振荡，虽然还没发散，但已经不适合生产。纹波比超调更早报警，因为它是“稳态行为”，不受初始段掩盖。

图 84-6 把 PM 与超调随延迟的变化画在一起。两条曲线的形状不同：PM 近似线性下降，超调在 2 拍以后加速上升。**这说明超调不是 PM 的线性函数**，用“PM 每降 10° 超调涨多少”这种线性折算只在 60°–70° 小区间内有效。

图 84-6 延迟对相位裕度与超调的影响。延迟超过 5 拍后超调加速上升，而纹波在 10 拍时已涨 14 倍。

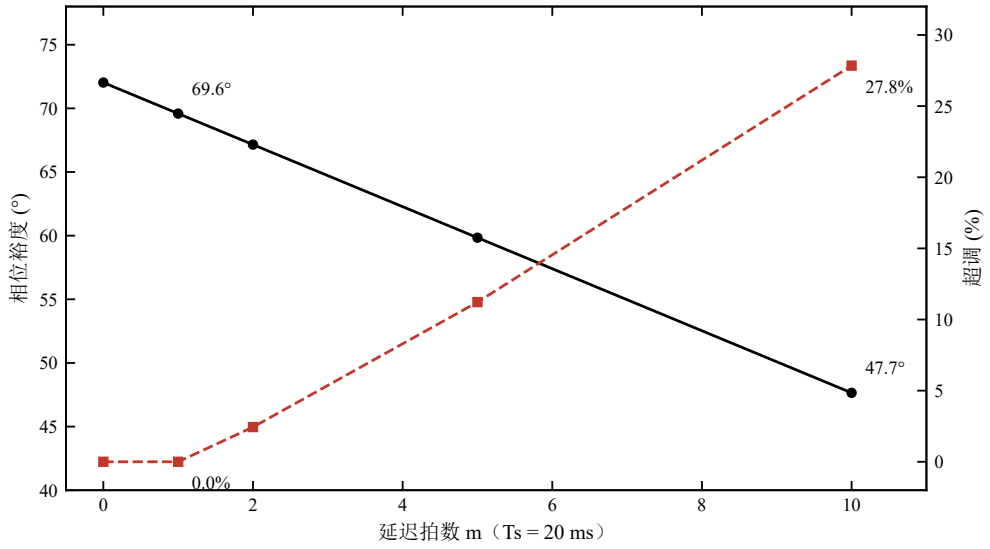


图 84-6

84.6.2 相位账准不准

式 (84-9) 是工程上最常用的估算。本章实测与它的对比如表 84-7。

表 84-7 相位损失：实测与近似式对比（$\omega_c=2.127$ rad/s，$T_s=20$ ms）

延迟	实测损失	$\omega_c T_s(m+0.5)$	偏差
1 拍	2.4°	3.7°	-1.2°
2 拍	4.9°	6.1°	-1.2°
5 拍	12.2°	13.4°	-1.2°
10 拍	24.4°	25.6°	-1.2°

偏差稳定在 -1.2°，不随 m 变。这说明式 (84-9) 的**斜率是对的，截距偏大**：它把半拍保持的相位多算了约 1.2°。原因是 $\omega_c T_s$ 只有 0.043，sinc 的幅值衰减让实际交界点略微左移，而式 (84-9) 用的是未衰减的 ω_c。

工程含义：**用式 (84-9) 估算 PM 会偏保守约 1°–2°，这是好事**——按它设计出来的系统，实际裕度会比预估略好。不要“修正”这个偏差，把它当作安全余量。

84.6.3 稳定性的两种判据

离散系统判稳有两条路，各有用途。

第一条是 z 域极点。闭环极点全在单位圆内即稳定。这条判据严格、可计算，但“离单位圆多近”不能直接比：本章 $T_s=20$ ms 时最大极点模 0.99966，$T_s=500$ ms 时反而是 0.736。前者贴近单位圆是因为积分极点本来就慢，后者远离单位圆却超调 28.5%——**极点模混进了时间尺度，单独看会误判。**

第二条是 PM 与增益裕度。它给出“离不稳定还有多远”的量化距离，且与超调直接挂钩。工程上更常用，因为它能翻译成“还能加多少延迟”。

两条判据要一起看。**只看极点会漏掉“稳定但纹波大”的情况，只看 PM 会漏掉“条件稳定”的情况**（某些系统在增益过大和过小时都不稳）。本章的对象是最小相位、无右半平面零点的三阶系统，不存在条件稳定，所以两条判据一致；换成有纯滞后或右半平面零点的对象，就必须用极点判据复核。

84.6.4 延迟从哪里来

延迟不是一项，而是四项之和（式 (84-18)）：

- **传感延迟**：ADC 采样保持 + 转换时间，典型 1–10 μs；
- **计算延迟**：从进入中断到算完输出，典型 5–100 μs（取决于控制律复杂度）；
- **总线延迟**：一帧的传输时间 + 排队等待，典型 0.1–10 ms（这是大头）；
- **执行延迟**：DAC 建立 + 执行器（阀、电机）响应，典型 1–100 ms。

注意后两项可能比 T_s 还大。**当总线延迟超过 T_s 时，系统实际上工作在“多拍延迟”状态**，此时必须按表 84-6 重新评估，而不是按“一拍”设计。这是现场调试中最常见的误判：设计时按 $m=1$ 算，实际是 $m=5$，一上电就振。

84.6.5 延迟能不能补

可以补一部分，但不能全补。

能补的是“已知的固定延迟”：如果每次都是 2 拍，可以在控制器里做 2 拍的预测（用对象模型外推），把等效延迟压回 0–1 拍。这就是 §84.9 要讲的“预测修正”。

不能补的是“随机延迟”：总线排队、任务调度抖动造成的延迟是随机的，预测只能补均值，补不了方差。此时正确做法是**按最坏情况设计**，把 PM 留够余量，而不是把均值补掉就以为万事大吉。

还有一项根本不能补：半拍保持。它是 ZOH 的物理属性，只要用零阶保持就存在。唯一的“补偿”是提高采样率，让半拍变短。

84.6.6 两个裕度：增益与延迟

相位裕度只回答“相位还能再少多少”，不回答“增益还能再加多少”和“延迟还能再多多少”。后两个问题在工程上更常被问到，所以要单独算。

增益裕度是“K_p 还能加几倍才失稳”。用脚本把 K_p 从 1.0 往上扫，$T_s=20$ ms、$m=1$ 时：

- $K_p=1.0$：$PM=69.6^\circ$；
- $K_p=1.3$：$PM=56.2^\circ$（表 84-8）；
- $K_p=3.96$：$PM\approx 0^\circ$，临界。

所以增益裕度 $GM=3.96/1.0=3.96$ 倍，即 $20\lg 3.96=12.0$ dB。工程上一般要求 $GM\ge 6$ dB（相当于 2 倍），12 dB 属于“很宽裕”。这也解释了为什么本章反复加 K_p 到 1.3 都不失稳——离边界还有三倍。

延迟裕度是“还能再多几拍”。同样扫 m：

$$m=20:\ PM=23.3^\circ;\quad m=25:\ 11.1^\circ;\quad m=28:\ 3.8^\circ;\quad m=30:\ -1.1^\circ.$$

临界在 30 拍，即 $30\times 20=600$ ms，约 3 倍对象时间常数。工程上“延迟裕度不小于最大可能纯滞后的 1–3 倍”这条要求，在无纯滞后的对象上只能换个说法：**额外延迟不超过对象时间常数的 3 倍**。本章的 600 ms 正好落在这条经验线上。

两个裕度之间有一个好用的换算。延迟每增加一拍，相位损失约 $\omega_c T_s=2.44^\circ$，所以

$$\text{延迟裕度（拍）} \approx \frac{PM}{\omega_c T_s} = \frac{69.6}{2.44} = 28.5. \tag{84-23}$$

与实测的 30 拍差 5%。这个式子说明：PM **与延迟裕度是同一件事的两种说法**，只差一个 $\omega_c T_s$ 的换算系数。所以看到 $PM = 45^\circ$ 就知道延迟裕度约为 $45/2.44 \approx 18$ 拍；看到 $\omega_c T_s$ 变大（采样变慢或对象变快），同样的 PM 对应的"可容忍延迟拍数"就变少。

工程上这三条要一起看：

- $PM \geq 45^\circ$：保证超调可接受；
- $GM \geq 6$ dB：保证增益误差（传感器标定、阀门特性变化）不会导致失稳；
- 延迟裕度 $\geq$ 最大可能纯滞后的 1–3 倍：保证通信抖动、执行器迟滞不会把系统推到边界。

本章三条全部满足，而且余量都不小。这不是巧合，而是 $\omega_c T_s = 0.043$ 很小带来的红利——**采样足够快，三个裕度会同时变好**。

84.6.7 手算稳定性：Jury 判据

现场没有 MATLAB 的时候，判断一个低阶离散系统稳不稳，可以用 Jury 判据手算。它的形式是：对特征多项式 $P(z) = z^n + a_{n-1}z^{n-1} + \cdots + a_0$，构造一张系数表，逐行比较首末系数；全部条件满足则所有根在单位圆内。

对本章最常遇到的二阶情形，条件可以写成三条：

$$|a_0| < 1, \qquad P(1) > 0, \qquad P(-1) > 0. \tag{84-24}$$

举个能心算的例子：$P(z) = z^2 - 1.5z + 0.7$。

- $|a_0| = 0.7 < 1$，满足；
- $P(1) = 1 - 1.5 + 0.7 = 0.2 > 0$，满足；
- $P(-1) = 1 + 1.5 + 0.7 = 3.2 > 0$，满足。

三条全过，所以这个系统稳定。它的两个根是 $0.75 \pm 0.37j$，模 $0.836 < 1$，与判据一致。

再试一个：$P(z) = z^2 - 1.5z + 1.1$。$|a_0| = 1.1 > 1$，第一条就不过，不稳定。这也符合直觉：常数项大于 1 意味着"零输入时的记忆项"比输入还大，系统会自己长大。

Jury 判据的实用边界：阶数到 3–4 阶手算还可控，再高就该用数值求根。而数字控制回路加上 PI 之后，特征多项式通常是 4–5 阶，所以现场更常用的是"双线性变换 + 劳斯判据"：用 $z = (1+w)/(1-w)$ 把单位圆内映到 w 左半平面，然后套用连续域的劳斯表。这两条路本章不展开（属于第 85 章的内容），这里只强调一件事：**判稳的工具不止一种，但结论必须一致**。如果 Jury 说稳定、数值求根说有一个根在 1.02，那一定是多项式列错了（最常见的是把 PI 的 $z/(z-1)$ 通分时丢了一项）。

84.7 参数意义与作用

这一节把散在前面的参数集中讲一遍。每个参数回答四个问题：它是什么、典型范围、影响哪个指标、现场怎么测。

84.7.1 采样周期 T_s

定义：两次控制计算之间的时间间隔。 **典型范围**：快回路 1–10 ms，一般过程控制 100 ms–1 s，慢过程 1–10 s。 **影响**：PM（5 ms 到 50 ms 只掉 2.7°，50 ms 到 500 ms 掉 27.7°，不是线性的）、超调、IAE，不显著影响上升时间（§84.5.4）。 **怎么测**：不要看手册标称值，用示波器抓"控制输出更新"的上升沿，测 100 个间隔取均值与标准差。标称 20 ms 实际 20.3 ms 是常态，因为定时器分频取整。

一个容易忽略的点：**T_s 不是越小越好**。它受算力、噪声、抖动占比三重下界约束（§84.5.5）。本章取 20 ms 是“足够好且留余量”，不是“最优”。

还有一个容易被忽略的事实：**典型范围里的三个档位，对应的其实是三种不同的对象**，不是三种“精度档次”。1–10 ms 对应机电对象（伺服、变频、流量），时间常数在毫秒到几十毫秒；100 ms–1 s 对应热工对象（温度、压力、液位），时间常数在分钟量级；1–10 s 对应慢化学过程（成分、pH）。**选 T_s 的第一步永远是问对象的时间常数，而不是问“别人用多少”**。把温度回路做到 10 ms 没有收益（本章 §84.5.4 已经证明上升时间对 T_s 不敏感），却把 CPU 预算花掉二十倍；把伺服回路做到 100 ms 则一定振（$\omega_c T_s$ 早就超过 1）。

测 T_s 时还有一个陷阱：**平均周期与最大周期要分开看**。平均 20.0 ms、最大 27 ms 的控制器，看起来“周期很准”，但每一次 27 ms 都相当于那一拍延迟多了 35%。用示波器抓 100 个间隔时，除了均值和标准差，还要记最大值——**最大周期决定最坏情况下的相位，平均值只决定平均性能**。

84.7.2 延迟拍数 m

定义：从采样到控制量真正作用于对象，中间经过的完整采样周期数，向上取整。 **典型范围**：单一控制器 1–2 拍；经现场总线 2–10 拍；经以太网 + 上位机 5–50 拍。 **影响**：PM（每拍约 -2.4°，$T_s = 20$ ms 时）、超调、稳态纹波（§84.6.1）。 **怎么测**：在控制输出上叠加一个已知的窄脉冲（或阶跃），同时记录传感器读数，量两者之间的时间差，再除以 T_s 向上取整。

m 是最容易被低估的参数。设计阶段按 1 拍算，现场因为总线排队变成 5 拍，PM 从 72° 掉到 60°，超调从 0 变 11%——**这不是“调参没调好”，是模型和现场不一致**。

“每拍 -2.4°”这个数要会自己算：$\Delta\varphi = \omega_c T_s = 2.1 \times 0.02 = 0.042$ rad $= 2.4^\circ$。换一个对象就换一个数，所以**记公式比记数字有用**。同一个 $\omega_c = 2.1$ 的对象，若周期放到 200 ms，一拍就是 24°——这正是表 84-4 里 200 ms 那一行超调 10.2% 的来源。带宽越低，同样拍数的代价越小，所以“慢回路可以慢采样”这句话是对的，但它成立的前提是带宽真的低，而不是“工艺上说这个量变化慢”。

测 m 的脉冲法有个前提：脉冲要窄，窄到只影响一拍，否则响应里会混进对象自身的动态。更稳妥的做法是直接看**时序图**：在控制器里记下“采样时刻”与“输出更新时刻”两个时间戳，两者的差就是计算延迟；再在阀门侧用电位器或流量计看动作起始时刻，差就是执行延迟。两个差相加除以 T_s，比猜准得多。本章表 84-2 的 15.3 ms 就是这么逐项列出来的。

84.7.3 位数 n

定义：ADC 与 DAC 的有效位数（不是标称位数）。 **典型范围**：ADC 12–16 bit（有效 10–14 bit），DAC 12–16 bit。 **影响**：稳态纹波、最小可分辨误差；对 PM 与超调几乎无影响。 **怎么测**：给一个已知斜率的缓慢输入，记录码值跳变，数一数跳变对应的输入变化量，得到实际 LSB。手册的“16 bit”常常只有 13 bit 有效，低位是噪声。

本章的结论是“够用即止”：8 bit 纹波 2.85×10^{-3}，10 bit 降到 1.88×10^{-4}，12 bit 与 14 bit 不再改善（表 84-10）。**多买的位数不会变成精度，只会变成成本**。

位数这一项还有一处不对称：**输入侧的量化误差进闭环，输出侧的量化误差直接在对象上**。前者被回路增益放大或抑制，后者只是给执行器加了一个台阶，对反馈没有直接贡献。所以同样 8 bit，ADC 与 DAC 的后果差别很大——表 84-9 里只有 DAC 8 bit 时纹波 6.10×10^{-4}，双侧都 8 bit 时跳到 2.90×10^{-3}，差近五倍。**要省钱，先省 DAC**。

84.7.4 抖动 σ_t

定义：采样时刻相对理想时刻的标准差。 **典型范围**：硬件定时 1–100 ns；操作系统调度 10 µs–1 ms；总线触发 100 µs–10 ms。 **影响**：采样值误差（式 (84-10)），进而影响纹波与 IAE；对慢变对象几乎无影响。 **怎么测**：用同一路信号触发两次采样，比较时间差；或用高精度计数器记录中断进入时刻的分布。

本章的实测很“反直觉”：抖动到 50% T_s，IAE 只从 0.5996 变到 0.5996（表 84-11）。原因是对象慢（0.2 s 时间常数），信号斜率小。**如果你的回路是快速位置环或带微分的控制律，结论会完全不同**——那时抖动直接进控制量，必须严格限制。

按式 (84-10) 估一下量级就明白为什么“反直觉”其实很合理：稳态时输出斜率接近零，$\dot{y} \approx 0$，抖动乘上去仍然是零；过渡过程里最大斜率约 $1/0.2 = 5$ 每秒（单位阶跃 / 时间常数），50% 抖动即 10 ms，误差 $5 \times 0.01 = 0.05$——只占阶跃幅值的 5%，而且只持续 0.8 s。反观位置环，若执行器每秒走 1000 个脉冲，10 ms 抖动就是 10 个脉冲的误差，直接反映在定位精度上。**所以判断抖动要不要管，先估 $\dot{y}\sigma_t$ 与允许误差的比值，而不是看 σ_t/T_s 的百分数。**

84.7.5 回路数 N 与可用度 A

定义：N 是一台控制器带的回路数；A 是系统在任意时刻可用的概率。 **典型范围**：N 从几个到几百；单机 A 约 0.9998，冗余后 0.999999 以上。 **影响**：N 决定算力与 T_s 的下界；A 决定架构（是否需要冗余）。 **怎么测**：N 查组态；A 用 MTBF 与 MTTR 估算，$A = \mathrm{MTBF}/(\mathrm{MTBF} + \mathrm{MTTR})$。

本章算过：单机 $A = 0.99984$（年停机 1.40 h），100 个回路串联掉到 0.98413（年停机 139 h），1+1 冗余回到 0.99999997（年停机 0.0002 h）。**冗余不是“更保险”，是把年停机从 139 h 压到 0.0002 h 的唯一办法**——因为串联系统的可用度是乘积，回路越多越接近“必然有一个坏”。

A 的计算里，MTBF 是“平均多久坏一次”，MTTR 是“坏一次修多久”，两者都要按**现场**数据取，不能用手册的实验室值。手册写 MTBF=20 万小时，现场算上电源、接线、环境温度，往往只有 5 万小时；手册写 MTTR=1 h，现场算上等备件、办作业票、停车，往往 8 h。本章用 5 万与 8，就是从现场值起步的。**这两个数一改，A 立刻变一个数量级**：MTTR 从 8 h 降到 4 h，单机 A 从 0.99984 升到 0.99992，年停机减半。所以提升可用度有两条路，提高 MTBF（选好件、降额、控温）与降低 MTTR（备件、模块化、在线更换），后者通常更便宜。

还要区分“可用度”与“安全性”。A 说的是“系统能不能用”，不说“用错的时候会不会伤人”。安全仪表系统（SIS）看的是另一个指标（要求时的失效概率 PFD），两者的冗余结构完全不同——本章 §84.10.3 会展开，这里只需记住：**别拿可用度当安全指标用**。

84.7.6 参数之间是耦合的

最后一件事最重要：**这五个参数不是独立的**。

- T_s 与 N 通过算力耦合：$N \le 0.7 T_s / t_{\mathrm{loop}}$。想加回路就得放大 T_s，想缩小 T_s 就得减回路。
- n 与量程耦合：同样的位数，量程缩小一倍，分辨率提高一倍，但超量程风险也提高一倍。
- m 与站数耦合：总线周期随站数线性增长（§84.10），站数翻倍，延迟可能翻倍。
- σ_t 与 T_s 耦合：同样的绝对抖动，T_s 越小占比越大。

所以调参不能“一个一个调”。正确的顺序是：先定 T_s 与 N（算力与规模），再定 m（通信与时序），最后定 n 与 σ_t（精度）。反过来做，会在最后发现前面三项已经把预算吃光。

84.7.7 三个参数各动一动，指标怎么变

前面六小节讲的是“参数是什么”，这一小节讲“参数错了会怎样”。工程上更常见的问题不是“不知道 K_p 该取多少”，而是“分子上的 K_p 是 1.0，现场实际是 1.3”。所以看敏感性比看标称值更有用。

表 84-8 参数敏感性（$T_s = 20$ ms，$m = 1$，其余参数取基准值）

变量	取值	PM	IAE	超调
K_p	0.70	81.7°	0.8550	0.0%
K_p	0.85	76.2°	0.7051	0.0%
K_p	1.00	69.6°	0.5996	0.0%
K_p	1.15	62.7°	0.5547	5.9%
K_p	1.30	56.2°	0.5439	11.7%
T_i	0.30	36.6°	0.7519	34.9%
T_i	0.45	56.4°	0.5684	12.4%
T_i	0.60	69.6°	0.5996	0.0%
T_i	0.90	86.8°	0.8878	0.0%
T_i	1.20	97.8°	1.1429	0.0%
τ	0.14	83.5°	0.5990	0.0%
τ	0.20	69.6°	0.5996	0.0%
τ	0.26	58.5°	0.7166	10.2%

这张表有四条能直接拿走的结论。

第一，IAE 最小的地方不是最好的地方。$T_i = 0.45$ 时 IAE$= 0.5684$，比基准的 0.5996 小 5%，但超调 12.4%。$T_i = 0.6$ 的 IAE 反而大，可它超调为零。**IAE 会把“快而振”算成好成绩**，因为振荡的误差在正负之间相互抵消。只看 IAE 选参数，会选出“快但过冲”的那一个。这就是 §84.2.4 说“三个指标各有盲区”的实证。

第二，K_p 从 1.0 加到 1.3，IAE 只降 9%，超调从 0 涨到 11.7%。换算成“每单位收益的代价”，这笔交易很不划算。反过来，K_p 从 1.0 减到 0.7，IAE 涨 42%——**减增益的代价远大于加增益的收益**，所以调参时“往回调一点”往往比“再冲一点”更伤性能。

第三，T_i 减半（0.6→0.3）的破坏力最大：PM 从 69.6° 崩到 36.6°，超调 34.9%。这解释了现场一个常见事故：为了让静差消得快，把积分时间往小调，结果整条回路开始周期性摆动。积分是“记忆”，记忆太短等于把 PI 变成了高增益 P，裕度立刻不够。

第四，对象时间常数估错 30%，等于凭空加了 5 拍延迟。τ 从 0.20 变成 0.26 时 $PM = 58.5^\circ$，而表 84-6 里 $m = 5$ 时 $PM = 59.8^\circ$——两者几乎相等。这条换算很实用：**辨识误差与延迟是可以互相折算的**，做延迟预算时要把“对象模型可能偏慢 30%”折算成 5 拍提前扣掉，否则算出来的裕度是纸面上的。

把四行数据横过来看，还有一个统一的标度：**三个量各自翻一倍，**PM **大约变化 30°。**K_p 从 0.7 到 1.4、τ 从 0.14 到 0.28、T_i 从 0.3 到 0.6，按表里的趋势外推，都是 28°–33° 的量级。这条“30° 规则”可以用来做粗估：任何一项可能有 2 倍误差，就要预备 30° 的裕度去打平它。

84.7.8 输入侧量化与输出侧量化不对称

§84.8.4 的表 84-10 只扫了 ADC。但执行器侧也有量化（DAC 位数、阀门死区、变频器的最小频率步长），它和 ADC 的地位并不对等。补一组对照：

表 84-9 输出侧量化与双侧量化（DAC 量程 ±2.0，ADC 量程 ±1.5）

配置	位数	台阶	稳态纹波	IAE
仅 DAC	6	3.12e-2	2.09e-5	0.5944
仅 DAC	8	7.81e-3	6.10e-4	0.5986
仅 DAC	10	1.95e-3	5.47e-4	0.5994
仅 DAC	12	4.88e-4	3.72e-4	0.5996
ADC+DAC	8	7.81e-3	2.90e-3	0.6025
ADC+DAC	10	1.95e-3	5.26e-4	0.5997
ADC+DAC	12	4.88e-4	3.80e-4	0.5996

这张表里最刺眼的一行是**仅 DAC 6 bit**：台阶 3.12×10^{-2}（满量程的 1.6%），纹波却只有 2.09×10^{-5}，IAE 甚至比不量化时（0.5996）还小。这不是“6 bit 够用”，而是**输出侧的量化被对象滤掉了**。控制量 u 在 1 ± 0.03 之间跳台阶，但对象是三个 0.2 s 的惯性串联，对 3×10^{-2} 的台阶响应是缓慢爬升，输出 y 上看不出跳变。同时，粗台阶提供了一个“死区”，把 §84.2.6 说的极限环卡死了，纹波反而更小。

这种“更好”是假象，理由有三：台阶 1.6% 意味着稳态精度上限就是 1.6%，本章之所以看不出来，是因为给定值恰好是 1.0、积分器又在慢慢补；若给定值改成 0.37，同样的 DAC 会让稳态在 0.36 与 0.38 之间跳。执行器的反复微跳会加速阀门磨损。任何需要平滑输出的场合（如阀位指示、串级内环给定）都不能用这么粗的量化。

对照 ADC 那几行，可以得到一条清楚的不对称：

- **ADC 的量化直接进反馈**，控制器“看见”的就是台阶，它会当真，于是积分器追着台阶跑，产生极限环。所以 ADC 的位数要求高（本章 10 bit 起）。
- **DAC 的量化先经过对象**，被惯性滤一次才回到传感器。所以 DAC 的位数要求可以低一到两档，只要台阶小于“允许的稳态误差”而不是“允许的纹波”。

式 (84-17) 的判据（$n\ge\log_2(\text{满量程}/3e_{ss})$）对两侧都适用，但 e_{ss} 的含义不同：ADC 侧取“允许纹波”，DAC 侧取“允许静差”。本章 10 bit ADC 对应 $e_{ss}\approx2\times10^{-4}$，而 10 bit DAC 的台阶 1.95×10^{-3} 对应的是 0.2% 的静差——后者在过程控制里通常可以接受，这就是为什么现场常见“16 bit 采集 + 12 bit 输出”的配置。

84.8 调参方法与经验

这一节讲“怎么调”，顺序很重要：**先调时间，再调增益，最后调精度**。反过来调，会在最后发现时间预算不够，前面的工作全部作废。

84.8.1 第一步：确认 T_s 与 m

不要相信手册，实测两件事：

1. 控制输出的更新间隔（100 次取均值与标准差）；
2. 从“传感器变化”到“输出变化”的总延迟（除以 T_s 向上取整）。

本章基准的实测是 $T_s=20$ ms、$m=1$ 拍。如果实测 m 比设计值大，**先解决延迟，再谈调参**——因为延迟是从裕度里直接扣的，调增益补不回来。

84.8.2 第二步：按相位账定 K_p 的上限

给定当前 T_s 与 m，由式 (84-9) 算出采样与延迟吃掉的相位，剩下的才是可用的裕度。若要求 $PM\ge45^\circ$，则

$$\omega_c T_s \left(m+\tfrac{1}{2}\right) \le 135^\circ - PM_{\text{对象}}, \tag{84-25}$$

$PM_{\text{对象}}$ 是对象本身在 ω_c 处的相位余量。本章对象在 $\omega_c = 2.1$ 处相位 -68.6°，所以留给采样与延迟的额度是 $135 - 68.6 = 66.4^\circ$。实际用掉 3.3°，余量很大——这就是为什么本章可以把 K_p 取到 1.0 而不振。

反过来，如果你的对象本身相位就接近 -135°（比如有纯滞后），留给采样的额度可能只有几度，此时 T_s 必须压得很小，否则无论怎么调 K_p 都不稳。

84.8.3 第三步：调 T_i

T_i 影响的是低频段，主要决定“消静差有多快”和“超调之后的恢复”。经验做法：

- 先取 $T_i \approx 3\tau_{\min}$（本章 $3 \times 0.2 = 0.6$ s，正好是基准值）；
- 若超调偏大，增大 T_i（积分变慢）；
- 若消差太慢，减小 T_i，但要检查 PM 是否还够。

T_i 与 T_s 有一个隐性约束：T_i/T_s 不宜小于 5。本章 $0.6/0.02 = 30$，很安全。若 $T_i/T_s < 5$，积分项的离散误差会显著改变闭环，此时应该用梯形积分或直接改用更小的 T_s。

84.8.4 第四步：定位数与抖动

前两步定完，再看精度。表 84-10 是 ADC 位数的实测。

表 84-10 ADC 位数扫描（量程 ±1.5，$T_s = 20$ ms）

位数	量化台阶	稳态纹波	IAE
6 bit	2.34×10^{-2}	1.09×10^{-2}	0.6241
8 bit	5.86×10^{-3}	2.85×10^{-3}	0.6025
10 bit	1.46×10^{-3}	1.88×10^{-4}	0.5997
12 bit	3.66×10^{-4}	3.92×10^{-4}	0.5996
14 bit	9.16×10^{-5}	3.94×10^{-4}	0.5996

这张表有一个反直觉的地方：**10 bit 的纹波（1.88×10^{-4}）比 12 bit（3.92×10^{-4}）还小**。不是笔误，也不是数值错误。原因是量化有时会把微小的极限环“卡死”在同一个码上：10 bit 的台阶恰好让系统停在一个码内不动，12 bit 台阶更细，反而允许它在两个码之间来回跳。

这条现象的教学意义大于工程意义：**纹波不是位数的单调函数**。所以选位数不能“按纹波最小”选，要按“台阶小于允许误差的 1/3”选（§84.2.3）。按这个判据，10 bit 就够，12 bit 以上是浪费。

图 84-7 把台阶与纹波画在一起：台阶随位数指数下降（直线在 semilog 坐标上），纹波在 10 bit 之后饱和在 4×10^{-4} 附近——那条水平线就是采样与延迟造成的底噪，再多的位数也压不下去。

图 84-7 ADC 位数与稳态纹波。台阶按 2^{-n} 下降，纹波在 10 bit 后触底，说明精度已由采样与延迟决定。

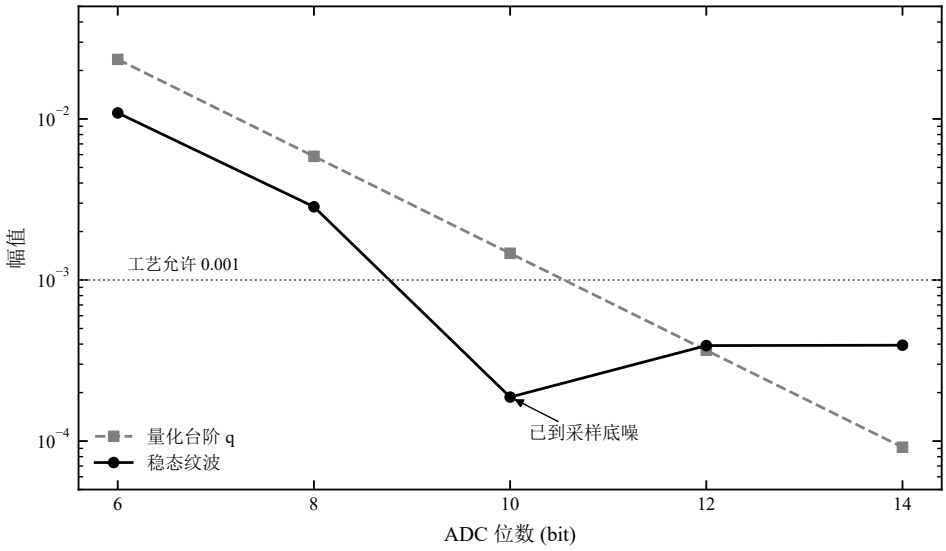


图 84-7

84.8.5 抖动：先判断它值不值得管

表 84-11 是抖动的实测。

表 84-11 采样抖动扫描（$T_s = 20$ ms）

抖动（占 T_s）	IAE	稳态纹波	超调
0	0.5996	3.92×10^{-4}	0.0%
10%	0.5996	3.93×10^{-4}	0.0%
25%	0.5996	3.94×10^{-4}	0.0%
50%	0.5996	3.95×10^{-4}	0.0%

四行的差别只在小数第四位。**结论是：在这个回路里，抖动不值得管。**判据来自式 (84-10)：误差正比于信号斜率，而本章的信号斜率很小（0.2 s 时间常数、T_s 只占 1/10）。

但这条结论不能推广。如果换成 1 ms 采样的位置环，对象时间常数 5 ms，同样的绝对抖动会让误差放大几十倍。所以正确的说法是：**先算 $\dot{y}\sigma_t$，再决定要不要为抖动花钱**，而不是“抖动总是有害”或“抖动无所谓”。

84.8.6 容量：一个 CPU 能带多少回路

表 84-12 是容量表，按每回路 20 µs 计算。

表 84-12 单 CPU 多回路容量（每回路 20 µs）

T_s	50% 负载	70% 负载	16 回路占用
5 ms	124	174	6.4%
10 ms	249	349	3.2%
20 ms	499	699	1.6%
50 ms	1250	1749	0.6%
100 ms	2500	3499	0.3%
200 ms	5000	6999	0.2%

这张表说明 DDC 的经济性：**20 ms 周期、70% 负载能带 699 个回路**，远超一般装置的实际需求（几十到一百多个）。所以“算力不够”在过程控制里很少是真问题，真问题通常是**延迟与可靠性**。

也要注意 50% 与 70% 两列的意义。50% 是“留一半余量给突发任务”，70% 是“接近上限但仍可调度”。工程上不要贴着 70% 设计，因为现场会加逻辑、加报警、加通信，负载只增不减。**留 30% 是给未来的自己留余地。**

图 84-8 把容量与 T_s 的关系画成双对数直线：$N \propto T_s$，斜率 1。这条直线很好记：**周期放大一倍，能带的回路翻一倍。**但代价是 §84.5 那张表——T_s 放大一倍，PM 会掉几度。两条曲线一起看，才能找到“回路数够用、裕度也够用”的交点。

图 84-8 采样周期与可带回路数。双对数坐标下是一条斜率 1 的直线，与 §84.5 的裕度曲线共同决定工作点。

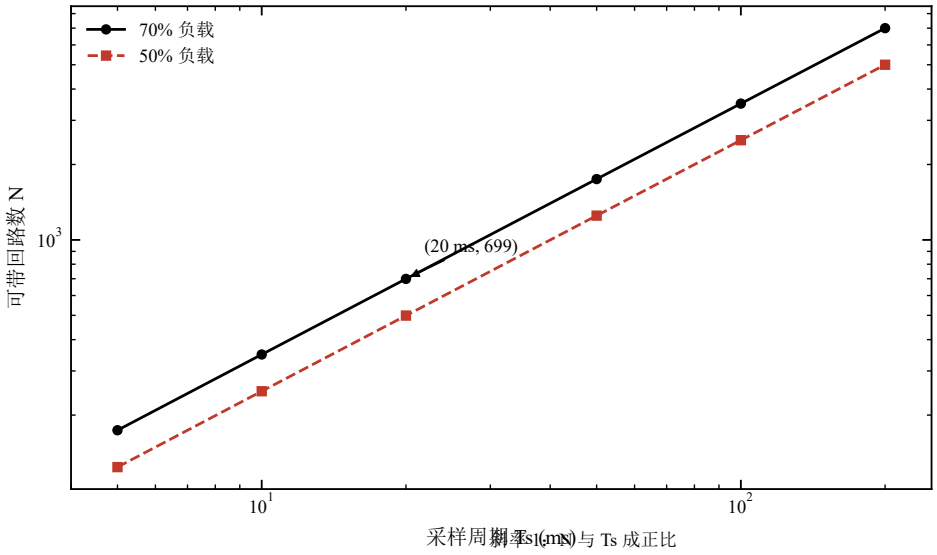


图 84-8

84.8.7 一条经验顺序

把上面几步压成一句话：**先测延迟，再定周期，然后调增益，最后看精度。**

这条顺序不是唯一的，但它有一个好处：每一步都在收缩下一步的可行域，不会出现“调到最后发现要推翻重来”。反过来（先调 PID 再改 T_s）在工程上很常见，代价是每次改 T_s 都要重调 PID，工作量翻倍。

把这条顺序与本章的数字对上，就更清楚为什么不能颠倒：先测延迟（表 84-2 的 15.3 ms）、再定周期（20 ms，$m = 1$，$PM = 69.6^\circ$）、然后调增益（表 84-8：K_p 从 1.0 到 1.3，PM 掉 13.4°、超调从 0 到 11.7%）、最后看精度（表 84-10：10 bit 已够）。**如果反过来先调增益，会为了压住“周期不足引起的振荡”而把 K_p 调小，等周期改好之后，这组偏小的增益又显得太保守，只能再调回去。**两次调整、两次验证，工作量翻倍，还容易在中间状态误判“这个对象不好控”。

还有一条隐含前提：这条顺序里“测”排在“算”前面。表 84-2 的四项延迟没有一项能纯靠手册算准，阀门定位器的 9 ms、总线的 5.3 ms，都必须实测或查现场记录。**凡是“测”得出来的，不要“算”**——算出来的数会给你虚假的精确感，而现场的第一个意外就会把它推翻。

84.8.8 五个典型失败案例

前面七小节是“正确的顺序”。现场更常见的是“顺序错了之后怎么补救”，下面五个案例按出现频率排，每个都给现象、判据、处置。

案例一：越调越振。现象是 K_p 一加就振，减下来又慢，反复几天找不到边界。多数情况不是参数问题，是**实际延迟比设计值大**：设计按 1 拍，现场是 5 拍，PM 从 72° 掉到 59.8°，超调 11.2%。判据很简单：给输出加一个窄脉冲，量

它到反馈的时间差。若实测 $m=5$，正确做法不是继续减 K_p（那只会把回路调慢），而是先减延迟——换更快的总线、把控制任务提到高优先级、缩短执行器行程时间。延迟减不下来，就按 5 拍重算 K_p，并接受 IAE 从 0.5996 涨到 0.6944 的代价。**先改条件，再改参数**，这条顺序能省掉大部分"调了三天"的时间。

案例二：静差消不掉。现象是给定 1.0，输出稳在 0.97，怎么加 K_p 都差那 3%。先看积分项有没有在动：如果积分器的值一直停在一个数上，说明它被限幅卡住了（执行器长期饱和，或积分限幅设得太小）。如果积分项在动但输出不动，说明**积分增益被写错了一个数量级**——§84.12.5 那个 `kp*ts/ti` 的写法就是典型，积分作用被缩小 30 倍，看上去像"积分不起作用"。判据：把 T_i 减半，若静差几乎不变，问题在积分通道而不是在 T_i。

案例三：稳态小幅等幅振荡。现象是运行平稳后，输出围绕给定值以固定幅度、固定周期抖动，幅度只有量程的千分之几。这是极限环，不是噪声。判据：把 ADC 位数临时加两档，若振荡幅度**变大或不变**，就是极限环（表 84-10 的反转）；若明显变小，才是量化噪声。处置有三条，按代价排：给积分器加一个很小的死区（最小）、放宽相位裕度让 -180° 处增益明显小于 1、换更高位数的采集卡（通常没用）。**不要指望"加滤波"解决**——低通滤波本身又吃相位，可能把裕度吃到临界。

案例四：给定值一跳就大过冲。现象是阶跃小的时候很漂亮，给定值从 1.0 跳到 2.5 时冲得很厉害、回落很慢。这是**积分饱和**：执行器量程 ± 2，给定 2.5 时误差长期为正，积分器一路涨到很大，等输出追上来了还要很久才退饱和。本章的仿真把执行器限幅保留在 ± 2，就是为了让这个问题在仿真里暴露。判据：看积分项在超调期间的值，若远大于稳态所需，就是饱和。处置：back-calculation（把"指令减实际"的差反馈回积分器）或积分钳位，两者都能把退饱和时间从数秒压到几百毫秒。

案例五：运行几小时后慢慢偏。现象是刚上电很好，几小时后静差慢慢长大，重启又好了。这类"慢性病"通常不在控制律，而在**时间基准**：控制周期由定时器分频得到，标称 20 ms 实际 20.3 ms，而积分器按 20 ms 累加，等效于 T_i 偏小 1.5%；几小时累积下来就是可见的偏差。另一种可能是浮点累加误差（单精度累加几十万次）。判据：打印每个周期的实际时间戳，统计均值与标准差；若均值偏离标称值超过 0.5%，就要在代码里用实测 Δt 而不是标称 T_s 做积分。

五个案例有一个共同的教训：**现象相似，原因可能完全不同，而区分它们靠的是"多看一个量"，不是"再调一次参数"。**案例一看延迟，案例二看积分项，案例三看位数扫描，案例四看积分饱和，案例五看时间戳。这五个"额外的一个量"，比任何整定口诀都有用。

84.9 优化与改进

§84.8 给的是"在现有条件下怎么调好"。这一节换一个问法：**条件本身能不能改**。四件事，按收益从大到小排。

84.9.1 把已知的固定延迟补回来

延迟分两种：**已知的固定延迟**（程序执行、总线周期、滤波器群延迟）和**未知的随机延迟**（调度抖动、网络冲突）。前者可以补，后者只能防。先看前者值多少钱。

表 84-6 里 $T_s=20$ ms 那一列，从 $m=0$ 到 $m=10$ 指标掉了多少：

- $m=0$：$PM=72.0^\circ$、IAE$=0.5996$、超调 0.0%；
- $m=5$：$PM=59.8^\circ$、IAE$=0.6944$、超调 11.2%；
- $m=10$：$PM=47.7^\circ$、IAE$=0.9982$、超调 27.8%。

也就是说，**10 拍延迟（200 ms）让 IAE 涨了 66%、超调从零变成 27.8%**。如果能把这 10 拍补掉 9 拍，回到 $m=1$，指标就回到 IAE$=0.5996$、超调 0.0%——**相当于白捡了一个更快的执行机构**。

补的办法在数字域里比连续域干净得多。连续域的 Smith 预估器要串一个 $e^{-\tau s}$，物理上不可实现，只能近似；数字域的延迟就是 z^{-m}，m 是整数，z^{+m} 是**超前**——它不可实现，但可以换个写法实现：把对象模型往前推 m 拍，用推算出来的"未来状态"反馈。

具体做法是给控制器配一个内部模型 (A_d, B_d, C_d)，在每个采样时刻做两步：

1. **量测校正**：用实际测到的 $y(k)$ 与模型输出 $\hat{y}(k)$ 的差修正模型状态，$\hat{x}(k) \leftarrow \hat{x}(k) + L\,[y(k) - \hat{y}(k)]$，$L$ 是观测器增益；
2. **向前推算**：用最近 m 个已知的控制量把模型状态推到 $k+m$，$\hat{x}(k+m) = A_d^{m}\hat{x}(k) + \sum_{i=0}^{m-1} A_d^{m-1-i} B_d u(k-m+i)$。

然后控制器对这个 $\hat{x}(k+m)$ 算控制量。式子里没有 z^{+m}，只有 A_d 的幂——**超前被"算"出来了，不是"等"出来的**。

这里有一个必须说清的代价：**预测的准确性取决于模型**。模型有 10% 的增益误差，推到 10 拍之后误差也会被放大，观测器增益 L 就要在"修得快"和"不放大噪声"之间折中。工程上的经验是：m 小（1–3 拍）时预测几乎总是划算；m 大（10 拍以上）时，先问一句"这 200 ms 是必须的吗"——往往改一下任务划分、把控制任务提到高优先级，比加一个预测器更省事。

预测器的完整推导与鲁棒性分析见第 47 章（Smith 预估）与第 107 章（工程版），本节只强调数字域的这一层：**延迟是整数拍，补起来是矩阵乘法，不是超越函数。**

84.9.2 随机延迟只能按最坏情况设计

如果延迟不是固定的，而是每拍在 1–5 拍之间跳，上面的预测器就不能用了——它需要一个确定的 m。

这时有三条路，按代价排：

- **按最坏情况设计**：取 $m_{\max}$ 代入式 (84-25) 算 K_p 上限。代价是平均性能浪费。用表 84-6 的数字估一下：若实际平均 2 拍、最坏 5 拍，按 5 拍设计只能拿到 $PM = 59.8^\circ$，而按 2 拍设计能拿到 67.2°——**白扔了 7.4° 裕度**。
- **加时间戳，把延迟变成已知量**：每个采样值带一个发送时刻，控制器算得出这一拍到底延迟了多少，再按实际 m 调整。这要求总线协议支持时间同步（如 IEEE 1588），代价是全网要有一致时基。做了这一步，"随机延迟"就退化成"每拍变化的已知延迟"，可以用变增益补偿。
- **把延迟的不确定性折成裕度**：不追求最坏情况下的性能，而是保证最坏情况下的稳定，把省下来的裕度用于平均性能。这需要在线辨识延迟，工程上较少用。

本节不展开这三条的实现，只给一个判断标准：**延迟的分布宽度（最坏减平均）超过 2 拍，就值得上时间戳。**不到 2 拍，按最坏情况设计的损失还在可接受范围内。

84.9.3 周期与回路数是同一枚硬币

§84.8.6 的表 84-12 已经说明：$N \propto T_s$。想多带回路，就放大周期；放大周期，就掉裕度。这两件事不是独立的，它们被同一个 ω_c 绑在一起。

把式 (84-16) 与式 (84-25) 联立，可以得到一个"给定回路数与对象带宽，最少需要多少 CPU"的估计。设单回路每周期耗时 t_c，则 N 个回路在 T_s 内的总耗时 $N t_c$，要求

$$\frac{N t_c}{T_s} \le \eta, \qquad T_s \le \frac{135^\circ - PM_{\text{对象}}}{\omega_c (m + \tfrac{1}{2})}, \tag{84-26}$$

其中 η 是允许的 CPU 负载率（取 0.7）。两式相乘消去 T_s：

$$N \le \frac{\eta\,[135^\circ - PM_{\text{对象}}]}{\omega_c (m + \tfrac{1}{2})\, t_c}. \tag{84-27}$$

代入本章基准：$\omega_c = 2.13$、$PM_{\text{对象}} \approx -68.6^\circ$（即额度 66.4°）、$m = 1$、$t_c = 20\,\mu$s、$\eta = 0.7$，得

$$N \le \frac{0.7 \times 66.4 \times \pi/180}{2.13 \times 1.5 \times 20 \times 10^{-6}} \approx 2.3 \times 10^{4}.$$

这个数远大于表 84-12 的 699。差在哪里？差在**表 84-12 用的是 $T_s = 20$ ms 这个具体值**，而式 (84-27) 允许 T_s 一直放大到相位账允许的上限（约 0.29 s）。换句话说：**如果只按相位账，一个 CPU 能带两万多个这样的回路；实际只能带七百个，是因为工程上没人会把周期设到 290 ms**。这个差距说明了什么？说明在过程控制里，T_s 的选择**不是被 CPU 卡的，是被工艺卡的**——很多回路需要 100 ms 以内的响应来抑制扰动，与算力无关。

这条推论有个直接用途：当你听到“CPU 负载 90%，要加机器”时，先问一句“这些回路的周期能不能分组”。把慢回路（温度、成分）放到 500 ms 一组，快回路（流量、压力）放到 50 ms 一组，两组跑在不同任务里，负载立刻就下来了。**分组比加机器便宜，而且不损失快回路的性能。**

84.9.4 从“能跑”到“好用”的三个台阶

第 83 章 §83.8.7 给过量子控制的四个台阶（能算→能跑→跑得准→跑得稳）。计算机控制的台阶少一级，但内容不同：

- **第一台阶：能跑**。采样、计算、输出三件事按时发生，回路不发散。这一台阶的门槛是 T_s 与 m 的相位账（式 84-25）。跨不过去，后面都免谈。
- **第二台阶：跑得准**。稳态误差、超调、调节时间达到工艺指标。这一台阶的门槛是位数（式 84-17）与积分器的抗饱和。§84.8.4 的表 84-10 说明：10 bit 就够，再高是浪费。
- **第三台阶：跑得稳**。连续运行几个月不出事。这一台阶的门槛不在控制算法，而在**冗余、诊断、降级**——§84.10 要算的可用度就是这一台阶的量化。

三个台阶的顺序不能颠倒。**现场最常见的错误是跳过第一台阶直接做第三台阶**：一上来就设计双机热备、三重冗余，结果单机都没调稳，冗余切换时两套逻辑打架。反过来，只做第一、第二台阶不做第三台阶的系统，在实验室里完美，在现场三个月后因为一块卡件故障全线停车。

还有一个容易被忽略的细节：**三个台阶的验收方法不同**。第一台阶看阶跃响应（本章用的就是这个）；第二台阶看长时间趋势与统计量（IAE、方差）；第三台阶看故障注入测试（拔卡、断网、断电）。用阶跃响应验收可靠性，是验不出来的。

84.9.5 观测：看不见的回路改不动

前面所有优化都建立在“你知道回路现在是什么状态”这个前提上。现场最常见的困境恰恰是：**回路在跑，但没人知道它跑得好不好。**

要打破这个困境，至少要留三类数据，代价都很低：

- **每周期的时间戳**。给定、反馈、输出、进入中断的时刻、算完的时刻。五个量，每周期几十字节，20 ms 周期一天约 2160 万条，压缩后几百 MB——放在本章 699 个回路的容量背景下，这点开销可以忽略（16 个回路只占 CPU 的 1.6%，记录通常还不到计算的十分之一）。
- **派生指标**。跟踪误差的均方根、输出饱和时间占比、单位时间内超调次数、积分项的均值。这四个数不需要存原始数据，每分钟算一次就够，却能覆盖 §84.8.8 五个案例中的四个。
- **事件记录**。参数被谁在什么时候改过、模式切换、报警、通信中断。这是事后复盘的唯一依据，也是“改了之后变差却说不清改了什么”的解药。

为什么强调“每周期时间戳”而不是“存趋势曲线”？因为**趋势曲线会把最关键的抖动抹平**。上位机按秒采样画曲线，20 ms 的周期性抖动在图上完全看不见；而恰恰是这种抖动暴露了延迟与量化的真实状态。要看清它，必须在控制器本地按周期记录，事后导出。

诊断规则可以写得很简单，四条就够用：

1. 跟踪误差 RMS 连续 10 min 超过阈值 → 报告“回路性能下降”；

2. 输出饱和时间占比超过 20% → 报告“执行器能力不足”或“给定值不合理”；

3. 反馈方差突然增大 3 倍 → 报告“传感器或工艺异常”；

4. 实测周期均值偏离标称值超过 0.5% → 报告“时间基准异常”。

这四条不涉及任何高级算法，但能把“运行三个月后莫名停车”变成“提前两周收到一条报警”。**计算机控制相比模拟控制最大的隐性优势，不是算得快，而是“什么都能记下来”**——不把这个优势用起来，等于只买了半台计算机。

84.9.6 多回路：从单回路到整厂

本章一直用单回路讲道理。真实的系统里，回路是成组出现的，组与组之间通过工艺耦合。这一小节只讲三件与“采样”直接相关的事，控制器设计本身留给第 85 章。

第一件：串级的两层周期。串级控制里，内环（流量、电流）快，外环（温度、液位）慢。常见配置是内环 20 ms、外环 100 ms，即外环每 5 个内环周期算一次。这时外环的相位账要额外算一项：**内环对外环来说等效于一个延迟**，量级是 1–2 个外环周期。所以外环的 m 不是“外环自己的延迟拍数”，而是“外环延迟 + 内环等效延迟”。很多串级回路外环振、内环好，原因就是这一项没算进去。

第二件：耦合与解耦的代价。两个回路通过工艺耦合（例如两个相邻加热区的温度互相影响），单独调都稳，一起投就相互激励。解耦的标准做法是加一个解耦矩阵，代价是**计算量按回路数的平方增长**。本章的容量表是按“每回路 20 μs”线性估的，一旦上解耦，t_c 会从 20 μs 涨到几百微秒，699 个回路立刻掉到几十个。**解耦不是免费的，它买的是“互不干扰”，付的是算力与延迟。**

第三件：负荷分配与协调。多个执行器干同一件事（多台泵、多个燃烧器）时，需要把总负荷分配到各台。分配算法的周期可以很慢（秒级），但它给出的设定值会同时改变多个回路的工作点，从而改变它们的增益。这就是为什么“投协调”之后常常要重新整定——**工作点变了，线性化模型的参数就变了**。§84.7.7 的敏感性表给了量级：K_p 差 30%，IAE 差 40%；如果负荷分配让某个回路的工作点从 50% 移到 80%，增益变化很容易超过 30%。

三件事的共同点是：**多回路的问题不在单个回路，而在“共享资源”——共享 CPU、共享总线、共享工艺通道。**本章的单回路分析给出的是“一个回路需要多少资源”，多回路设计要做的是“把这些需求加起来，看总量超不超”。§84.9.3 的式 (84-27) 就是这个加法的解析形式，表 84-12 是它的数值形式。

84.10 工程落地

前面九节都在讲“一项指标怎么算”。这一节把它们拼成一张能照着走的图：**上机前检查什么、总线怎么预算、可靠性怎么算、方案怎么选**。

84.10.1 上机前的十二项检查

下面这张清单是按“从信号到系统”的顺序排的，每一项都对应前面某节的判据。它的用法不是逐条打勾，而是**逐条问“这一项我测过吗”**——没测过的，就是上机后可能的坑。

1. **对象的 ω_c 是多少**（§84.6）。不是从手册抄的时间常数，是从阶跃响应实测的。手册上的 τ 往往是“额定工况”，实际可能差一倍。
2. **T_s 与 $\omega_c T_s$**。要求 $\omega_c T_s \le 0.3$（§84.1.4）。本章基准 $\omega_c = 2.1$、$T_s = 20$ ms，$\omega_c T_s = 0.043$，余量很大。
3. **实测总延迟 m**（§84.8.1）。用一次阶跃、看输出比给定晚几拍，不要去加理论值。本章实测 1 拍。
4. **相位账**（式 84-25）。$m = 1$ 时只吃掉 3.3°，额度 66.4°，剩 63° 给 K_p。
5. **ADC 位数与量程**（式 84-17）。台阶要小于允许纹波的三分之一。量程 ±1.5 V、10 bit 时台阶 1.46 mV，实测纹波 1.88e-4，够用。
6. **DAC 位数**。常被忽略：输出侧的台阶直接变成执行器的死区，而积分器会把死区变成极限环。

7. **采样抖动**（§84.8.5）。本章 50% T_s 的抖动只动小数第四位——**但这个结论不推广**，要看对象有没有高频谐振。
8. **积分抗饱和**。执行器限幅 2.0 时，$K_p = 1$、阶跃 1.0 刚好不饱和；若给定值更大，必须有 back-calculation 或钳位，否则退饱和要等很久。
9. **CPU 负载与回路数**（式 84-27、表 84-12）。留 30% 余量。
10. **总线周期与站数**（§84.10.2）。这一项决定了 m 的下界，不能事后补。
11. **可用度目标**（§84.10.3）。单回路和整厂的可用度差两个数量级，要分开算。
12. **故障时的行为**。断电、断网、卡件故障时，输出是保持、归零还是走安全值？这一项没人测，但每次停车都会遇到。

十二项里，**第 3、10、12 项是最容易被跳过的**：前两项要现场测、要翻总线手册，第三项要跟工艺专业一起定。跳过它们的系统，通常在第一次故障时才暴露问题。

84.10.2 现场总线周期预算

总线是 m 的物理来源之一。一次采样值从站传到控制器、控制量再传回去，至少要占两个总线周期，所以**总线周期直接进式 (84-18) 的 m**。

按“20 字节一帧 + 5 μs 令牌开销”估，单站耗时 $t_{st} = 160/R + 5\,\mu\text{s}$，其中 R 是比特率。三种速率下的容量：

比特率	单站耗时	10 ms 周期可带站数	100 ms 周期可带站数	32 站时周期
0.5 Mbit/s	325.0 μs	30	307	10.40 ms
1.0 Mbit/s	165.0 μs	60	606	5.28 ms
5.0 Mbit/s	37.0 μs	270	2702	1.18 ms

表 84-13 现场总线的站数与周期（20 字节/站，5 μs 令牌）

这张表最有用的一列是最后一列：**32 个站、1 Mbit/s 时一个轮询周期 5.28 ms**。如果控制周期定 10 ms，总线占了一半多，m 至少 1 拍；如果控制周期定 20 ms，总线只占 26%，m 还是 1 拍，但缓冲更从容。0.5 Mbit/s 时 32 站要 10.40 ms——**这已经超过 10 ms 周期了**，也就是说在 0.5 Mbit/s 上带 32 个站，10 ms 的控制周期是做不到的，不是“勉强够”，是“物理上不够”。

这里有一个常见的误算：把“10 ms 能带 30 站”理解成“30 个回路能跑 10 ms”。不对。30 站是**总线上只有令牌和数据、没有任何重传和诊断**的上限。工程上要打 60% 折扣（留重传、留诊断、留非周期数据），所以 0.5 Mbit/s 实际能带的实时站数约 18 个。**按 60% 打折这件事要写在设计文档里，不能靠现场调。**

60% 的折扣拆开看是三笔账：非周期数据（组态下载、报警、诊断）通常占 10%–20%，重传按误码率 10^{-4} 与帧长估算通常占 5%–10%，剩下的是“给未来留的”。这三笔里只有第二笔能算准，第一笔随运行状态变化，第三笔纯靠经验。所以折扣不是保守，是**把不可预测的部分预先扣掉**。

总线周期还有一个容易被忽视的性质：它是**抖动源**。同样的 32 站，若某一拍恰好碰上诊断帧插队，这一拍的周期会从 5.28 ms 变成 7 ms，相当于那一拍的延迟多了 30%。这不会改变平均指标，但会让最坏情况下的相位比设计值差一截。做法是把诊断帧、非周期数据放到**独立的时隙**，不要与实时数据抢同一个周期——现场总线的“周期性/非周期性”两段式设计，动机就在这里。

84.10.3 可用度与冗余

可靠性是计算机控制区别于模拟控制最明显的一项：模拟仪表一个回路坏了只坏一个回路，DCS 一台控制器坏了可能坏一片。

按 MTBF $= 50000$ h、MTTR $= 8$ h 算，单机可用度

$$A = \frac{\mathrm{MTBF}}{\mathrm{MTBF} + \mathrm{MTTR}} = \frac{50000}{50008} = 0.999840. \tag{84-28}$$

一年 8760 h，单机年停机 1.4014 h。这个数听起来很好（一年停一个半小时），但把它串起来就变了：

结构	可用度	年停机
单控制器	0.999840026	1.4014 h
10 回路串联	0.998401407	14.0037 h
100 回路串联	0.984128580	139.0336 h
1+1 冗余	0.999999974	0.0002 h
2oo3 表决	0.999999923	0.0007 h
双机热备+2oo3	1.000000000	0.0000 h

表 84-14 不同冗余结构的可用度与年停机（MTBF=50000 h，MTTR=8 h）

100 个回路串联，一年停 139 h，接近 6 天。这就是 DCS 存在的理由：不是因为它算得快，而是因为它把“一台机器管 100 个回路”的可用度从 0.984 拉回 0.9999 以上。

表 84-14 最后三行给出冗余的三种做法，代价依次上升：

- **1+1 冗余**：两台同样的控制器，一台工作一台待机，年停机 0.0002 h（约 7 s）。这是过程控制的主流做法，前提是**切换要无扰**——待机那台必须跟踪工作那台的状态，否则切换瞬间输出跳变，比停机还糟。
- **2oo3 表决**：三台都工作，取两票一致。年停机 0.0007 h。它比 1+1 好的一点是**能容忍一次误动作**（一台给出错误值，另两台压住它），1+1 做不到这一点。代价是三倍的硬件与三倍的诊断逻辑。
- **双机热备+2oo3**：两套 2oo3 互为备用，年停机在表里显示 0.0000 h（实际是 10^{-9} 量级，小于一秒）。这是安全仪表系统（SIS）的典型结构，用在“停一次车损失上千万”的场合。

要注意表 84-14 的算法是**纯串联/纯并联**，没有考虑共因失效。真实的 DCS 里，两台控制器往往共用一路电源、一个机柜、一套组态软件——**共因失效会把 1+1 的实际可用度拉回到接近单机**。所以工程上做冗余，第一件事是把共因拆开：双路供电、双网、双机柜。这一条比选 1+1 还是 2oo3 更重要。

共因的影响可以估个数。设共因失效占总失效率的比例为 β，两机并联的不可用度近似为

$$1 - A_{1+1} \approx (1 - A_1)^2 (1 - \beta) + \beta (1 - A_1). \tag{84-29}$$

代入单机 $1 - A_1 = 1.6 \times 10^{-4}$：$\beta = 0$ 时得 2.6×10^{-8}（表里的 0.0002 h/年）；$\beta = 1\%$ 时第二项 1.6×10^{-6} 完全主导，年停机从 0.0002 h 变成 0.014 h（约 51 s）；$\beta = 10\%$ 时变成 0.14 h。**这就是“共因不拆，冗余白做”的算术含义**：冗余只能压住独立失效的那一项，压不住共同项。

顺带解释一个现场现象：明明上了 1+1，年停机却没比单机好多少。查下去往往是两台控制器接在同一路 UPS、走同一个交换机、用同一份组态文件——β 高到 10% 以上，冗余的收益被吃光了。**所以验收冗余系统时，不要只问“几台”，要问“哪些东西是共用的”。**

84.10.4 DDC：一台计算机直接算回路

DDC（Direct Digital Control，直接数字控制）是最早、也最直白的一种形态：**一台计算机按分时的方式，依次为几十个回路算 PID，算完直接把控制量送给执行器**。它在 1960 年代替代的是满墙的模拟调节器。

它的工作方式决定了它的三个特征：

- **分时扫描**。CPU 一次只能算一个回路，所以“同时”是假象：第 1 个回路在 $t = 0$ 采样，第 30 个回路在 $t = 29 t_c$ 才轮到。若 $t_c = 20\,\mu$s、30 个回路，最后一个回路的采样时刻比第一个晚 580 μs。对 20 ms 的周期，这是 3% 的偏

差；对 1 ms 的周期，这是 58%，已经不能忽略。所以 DDC 的回路数与周期是硬耦合的（表 84-12）。

- **容量是核心指标**。因为替代的是“每回路一台调节器”，用户第一个问题就是“一台能带多少”。本章的答案：20 ms 周期、70% 负载、每回路 20 μs，能带 699 个；实际工程里 16 个回路只占 1.6%，余量极大。
- **单点故障**。一台计算机坏了，所有回路同时失控。这是 DDC 在 1970 年代被 DCS 取代的直接原因——不是算力不够，是**风险太集中**（表 84-14 的 100 回路串联，年停机 139 h）。

DDC 今天仍然存在，只是换了名字：嵌入式控制器、专用控制器、单机版组态软件，本质上都是“一台机器算一堆回路”。它适用的场合很清楚：**回路数在几十以内、对可用度要求不是极端高、成本敏感**。一台包装机、一台试验台、一个小型换热站，用 DDC 是对的；给它们上 DCS 是过度设计。

84.10.5 SCC：计算机只给设定值

SCC（Supervisory Computer Control，设定值计算机控制）把计算机从“回路里”挪到了“回路上面”：**下面的模拟调节器仍然独立完成闭环，计算机只负责算出每个回路的设定值**。

这个分工解决了一个 DDC 解决不了的问题：**优化与稳定是两种时间尺度**。稳定需要毫秒到秒级的确定性响应，优化需要分钟到小时级的计算（物料平衡、能耗寻优、Soft-sensor 推算）。让同一台计算机同时干这两件事，优化的计算会挤占控制的时间片，控制的实时性就没了。SCC 把它们分开：模拟调节器保稳定，计算机管优化。

SCC 的指标要求因此宽松得多：设定值几分钟更新一次就够，采样周期秒级甚至分钟级，延迟几秒也无所谓（因为给定值本身变化很慢）。本章前面那些“T_s 吃相位”的讨论，在 SCC 层几乎不适用——**这就是为什么先分清层次，再谈采样周期**。

SCC 的局限也很清楚：计算机不知道阀位实际到了哪，只知道“我给了设定值”。如果调节器饱和、阀门卡涩、变送器漂移，SCC 的优化模型会算出一个漂亮但没有执行的结果。所以 SCC 必须配“闭环校验”：把实际测量值与模型预测值比对，偏差超限就退出优化、退回固定设定值。这一条在今天的 APC（先进过程控制）里仍然是标准做法。

84.10.6 DCS：把风险拆开

DCS（Distributed Control System，集散控制系统）的出发点不是“算得更快”，而是“**别把鸡蛋放在一个篮子里**”。它把 DDC 拆成三层：

- **现场控制站**：负责实时闭环，一个站管几十到几百个回路，站与站之间独立。一个站故障，只影响它自己的回路。
- **操作站**：负责监视、报警、趋势、组态，不参与实时闭环。操作站死机，装置照常运行。
- **通信网**：连接两层，通常是双网冗余。

这个结构的效果可以用表 84-14 直接算出来。单机年停机 1.40 h；100 个回路串在一台上，年停机 139 h；拆成 10 个站、每站 10 个回路，每站各自 1+1 冗余，则每个站的年停机降到 0.0002 h，整厂可用度由“10 个站都可用”决定，仍在 0.99999 量级。**DCS 的价值几乎全部来自这个拆分，而不是来自它的控制器算法。**

代价有三项，都很实在：**成本**（硬件、软件许可、工程服务，通常是 PLC 方案的 2–5 倍）、**封闭性**（各家协议与组态环境不通用，升级被绑定）、**工程周期**（组态、联调、验证，动辄数月）。所以表 84-15 里 DCS 的成本只打 2 分——不是它不值，是它贵得明确。

DCS 适用的场合因此也很明确：**中大型流程工业**（化工、炼油、电力、造纸），回路数以百计、停车损失以百万计、装置连续运行以年计。在这些场合，“贵”与“停一次车”不在一个量级上。

84.10.7 FCS：把控制下放到现场

FCS（Fieldbus Control System，现场总线控制系统）把 DCS 的“现场控制站”再拆一层：**控制功能下放到现场仪表本身**，总线只负责传数据。

它解决的是布线问题。传统 DCS 里，每个变送器、每个阀门都要一对线回到控制站，一个 100 点的装置就是几百根电缆，桥架、端子、校线的工作量极大。总线方案里，一根主干线串起几十个站，每个站一个地址。本章的估算（表 84-13）：20 字节一帧、5 μs 令牌开销，1 Mbit/s 时单站 165 μs，10 ms 周期能带 60 个站，32 个站一轮 5.28 ms。

但它把一个新的瓶颈引进来：**总线成了单点**。主干线断了、总线电源短了、某个站的收发器故障把电平拉死，整段总线上的回路全部失联。所以 FCS 的可靠性设计重点从“控制器冗余”变成了“总线冗余 + 站间隔离 + 本安防爆”。这些在 DCS 里不是主要矛盾，在 FCS 里是首要矛盾。

FCS 还有两个容易被低估的约束：

- **供电与本质安全**。总线既要传数据又要供电，本安场合的功率限制（Ex ia 约 1.2 W/段）直接限制了能挂几个站、能不能带阀门。
- **周期与站数的乘积固定**。表 84-13 最后一行：32 站、1 Mbit/s 是 5.28 ms。想缩短周期，只能减站或提速率；减站就要多拉几段总线，多拉总线就回到布线问题。

所以 FCS 最合适的场合是**新建装置、点数分散、需要大量非控制信息（诊断、校准、资产管理的场合）**。老装置改造用 FCS，往往因为电缆已经敷好、防爆区域划分已定，收益抵不过风险。

84.10.8 选型打分

最后一步是选方案。这里用一个五准则、四方案的小模型，权重与打分如下（打分 1–5，越高越好）：

- 准则与权重：实时性 0.30、可扩展 0.20、可靠性 0.25、成本 0.15、维护 0.10；
- 方案：PLC+现场总线、PC+板卡、DCS、嵌入式+以太网。

得分 = 加权和，结果降序排列：

方案	实时性	可扩展	可靠性	成本	维护	加权得分
嵌入式+以太网	5	4	3	4	4	**4.05**
DCS	4	4	5	2	3	3.85
PLC+现场总线	3	3	4	4	4	3.50
PC+板卡	4	2	2	5	3	3.15

表 84-15 四种控制方案的五准则打分（权重 0.30/0.20/0.25/0.15/0.10）

这张表要读三遍，每遍读的东西不同。

第一遍看排序：嵌入式+以太网 4.05 略高于 DCS 3.85，差距 0.2。这个差距小于打分本身的不确定度（一个准则差 1 分，加权后就差 0.1–0.3），所以**这两个方案在这个权重下是并列的**。不要拿 4.05 vs 3.85 说“嵌入式更好”。

第二遍看短板：嵌入式+以太网的短板是可靠性 3 分，DCS 的短板是成本 2 分。选哪个，取决于你的项目是“成本敏感”还是“停车损失敏感”。表 84-14 已经给了量化：100 个回路，DCS 一年停 139 h，如果一小时停车损失 10 万元，一年就是 1390 万——**这个数远超硬件差价**，此时 DCS 的 2 分成本完全可以接受。

第三遍看权重本身：权重变了，排序就变。把可靠性权重从 0.25 提到 0.40、成本从 0.15 降到 0.05（安全仪表系统的典型权重），DCS 得 $4\times0.3+4\times0.2+5\times0.4+2\times0.05+3\times0.1=4.30$，嵌入式得 $5\times0.3+4\times0.2+3\times0.4+4\times0.05+4\times0.1=4.10$，**DCS 反超**。这说明打分模型的结论对权重极其敏感，**权重必须由项目干系人一起定，不能由写代码的人拍**。

图 84-9 把表 84-15 画成雷达图。四个方案的多边形叠在一起，能一眼看出各自的形状：DCS 在可靠性方向凸出、成本方向凹陷；PC+板卡正好相反；嵌入式+以太网是最“圆”的一个，没有极端项，也没有短板。

图 84-9 四种控制方案的五准则雷达图。多边形越靠外表示该项得分越高；嵌入式方案形状最均匀，DCS 与 PC+板卡各有一个尖角和一个凹口。

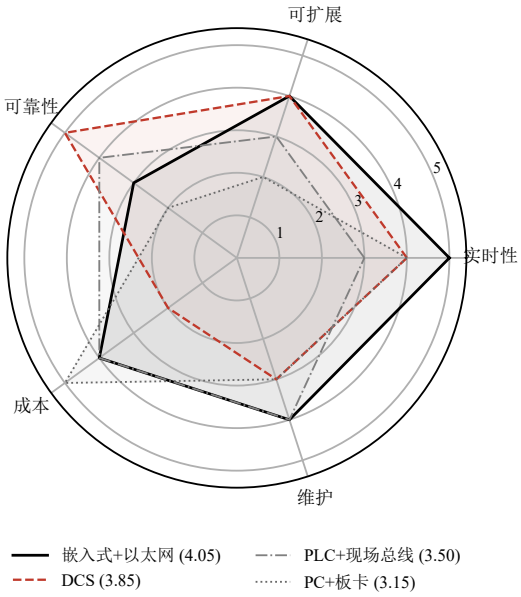


图 84-9

最后提醒一句：**任何打分表都只是把讨论结构化，不能代替讨论。**表 84-15 的 20 个数字里，至少有一半是可以争的（“实时性 4 分”是什么意思？1 ms 还是 10 ms？）。打分的价值在于**逼你把“哪个准则更重要”说清楚**，而不是给出一个可以免责的结论。

84.10.9 一张可以照着勾的验收单

把 §84.10.1 的十二项检查、§84.10.2 的总线预算、§84.10.3 的可用度合成一张单子，就是上机前的最后一道关。它分四组，每组的通过判据都必须是**现场实测值**，不能用仿真值代替。

第一组：时序。实测 T_s 的均值与标准差（偏差 <0.5%）；实测总延迟 m（与设计值一致，或按实测值重算过 K_p）；CPU 负载率（<70%，且峰值不超过 85%）；总线周期与站数（按 60% 折扣后仍有余量）。这一组不过，后面三组都不用测——时序不对，测出来的都是错的。

第二组：精度。稳态纹波（小于工艺允许值的 1/3）；阶跃超调（小于工艺允许值）；静差（在积分作用下趋零）；满量程附近的线性度（用 5 个点校验，偏差 <0.5%）。这一组要在**实际工作点**附近测，不能只在零点附近测——§84.7.7 已经说明工作点会改变增益。

第三组：可靠性。拔一块卡、断一根网线、断一次电，三种故障下系统行为符合设计（保持/降级/停机）；冗余切换时间（小于 1 个控制周期，且输出无跳变）；看门狗与超时保护有效。这一组必须做**故障注入**，看文档是看不出来的。

第四组：可维护性。每周期数据能导出；参数改动有记录、有权限；报警有分级、有抑制；组态有备份、能回滚。这一组最容易被当作“以后再说”，而它决定了三年后这套系统还能不能被维护。

四组的顺序有讲究：**时序 → 精度 → 可靠性 → 可维护性**。前一组不过就不要进下一组，因为后一组的测量结果会被前一组的缺陷污染。例如时序不稳时测出来的“纹波”可能全是调度抖动，你去调量化，永远调不好。

最后一条判据不在单子上，但最重要：**跑满 72 小时**。让回路在最不利的工况下连续运行三天，看有没有漂移、有没有累积误差、有没有内存泄漏、有没有在某个时刻莫名切换到手动。三天跑下来一切正常，才算“能交付”；跑不下来，前面所有检查都是纸面上的。

84.11 优缺点与适用边界

84.11.1 它真正解决的问题

计算机控制替代模拟控制，表面上替代的是“控制器”，实际上替代的是**三件模拟电路做不到的事**：

- **改逻辑不用改硬件**。模拟 PID 要改 K_p，得换电阻、重焊板子；数字 PID 改一个数。这一条的价值不在于省事，而在于**允许在调试期反复试**。§84.8 那套“先测延迟再定周期”的流程，只有在参数可随时改的前提下才成立。
- **信息可以被复用**。同一个温度信号，模拟系统里只能送进一个调节器；数字系统里可以同时给控制器、报警、趋势、优化程序、历史库。这是 DCS 与 FCS 的全部经济性来源——**一根线传一次，多处用**。
- **复杂算法变得可行**。本章只用了 PI，但同一套框架里换成预测控制、自适应、辨识，代价只是多几行代码。模拟电路里，每加一层算法就是一块新板子。

这三条不是并列的，它们有先后：**第一条是前提，第二条是红利，第三条是延伸**。没有“参数可改”，就不敢在调试期反复试，第二条与第三条也就无从谈起。所以评估一套计算机控制系统时，第一个要问的不是“它支持什么算法”，而是“改一个参数要多久、改完能不能回滚”。这个问题的答案，决定了后面所有算法能不能真正用上。

84.11.2 它换来的代价

代价也是三条，而且都能用本章的数字量化：

- **相位裕度被采样和延迟吃掉**。表 84-4 与表 84-6 已经给了：T_s 从 5 ms 放到 500 ms，PM 从 72.9° 掉到 42.5°；20 ms 下加 10 拍延迟，PM 掉到 47.7°、超调 27.8%。**模拟控制的 PM 只由对象与控制器决定，数字控制还多一个“实现”维度。**
- **精度被位数卡住**。表 84-10 里 6 bit 的纹波 1.09e-2，比 14 bit 的 3.94e-4 大 28 倍。模拟系统的精度由器件噪声决定，通常连续可变；数字系统有一条硬台阶，台阶下面是“看不见”的。
- **可靠性从“分散”变成“集中”**。表 84-14 的 100 回路串联：一年停 139 h。模拟仪表坏一块表只影响一个回路，DCS 坏一台控制器影响一片。**集中带来的效率，必须用冗余买回来。**

三条代价里，前两条可以“花钱买回来”，第三条只能“改结构买回来”。相位裕度不够，缩 T_s、减延迟、换快阀，都是钱的问题；精度不够，加位数、换采集卡，也是钱的问题；可靠性不够，冗余、热备、双网，这笔钱花下去还会带来新的复杂度（切换逻辑、共因失效、维护成本）。**所以在预算有限时，优先保裕度与精度，可靠性靠“把范围缩小”来保**——这就是 §84.10 里“分区分域、把 100 个回路拆成 5 个域”的工程动机。

84.11.3 什么时候不该用

有三类场合，纯数字控制不是最优解：

- **带宽极高的回路**。当 ω_c 到了几十 kHz 以上，$\omega_c T_s \le 0.3$ 要求 T_s 在微秒级，普通 PLC 与总线撑不住。这时要么用 FPGA/DSP 做专用控制器，要么保留模拟内环、数字只做外环（电流环模拟、速度环数字，是伺服里的标准做法）。
- **极简单、极可靠的单回路**。一个温度开关、一个压力联锁，用继电器或模拟调节器更便宜、更可靠、更容易被现场电工维护。给它上一套 DCS 是过度设计。
- **安全联锁的最终执行层**。SIS 里的紧急停车逻辑，最终动作通常仍由独立的硬接线安全继电器完成，DCS 只负责监视与记录。原因是**软件的共同失效模式无法用软件冗余消除**（§84.10.3 的共因失效）。

“带宽极高”这条可以算准。把式 (84-15) 反过来解，取 $\Delta\varphi_{\max} = 8^\circ = 0.14$ rad、$m = 1$，结果见表 84-16：

表 84-16 不同闭环带宽下的采样周期上限（$m = 1$，允许相位损失 8°）

ω_c	对应频率	T_s 上限	谁能做到
628 rad/s	100 Hz	149 μs	PLC 可（高端）
6283 rad/s	1 kHz	14.9 μs	DSP/FPGA
62832 rad/s	10 kHz	1.49 μs	仅 FPGA

这张表说明：**“用数字控制”的边界不在 CPU 快不快，而在采样与总线能不能跟上。**1 kHz 时 14.9 μs 的周期，意味着每一拍的全部工作（采样、计算、输出、通信）必须在这个窗口内做完，普通 PLC 的 I/O 扫描一项就超了。所以伺服驱动器的电流环至今大量用 FPGA 或专用 ASIC，不是守旧，是算过账的。

84.11.4 本章在全书中的位置

本章是第六部分（计算机控制）的入口，作用是**把“计算机”这个前提讲清楚，后面几章才有共同语言：**

- 第 85 章讲数字控制系统的结构，会直接用到本章的 m（延迟拍数）与 T_s；
- 第 86 章讲实时软件，会用到本章 §84.8.6 的容量公式与式 (84-15) 的周期上限；
- 第 87 章以后讲具体实现（通信、组态、可靠性设计），都会回到本章 §84.10 的四张表。

反过来，本章刻意**没有**深入的三件事——PID 整定、MPC、纯滞后补偿——分别在第 44/106、58/80、47/107 章。这样安排的理由是：本章要解决的是“**加了计算机以后，哪些指标变了**”，而不是“**怎么设计一个更好的控制器**”。前者是结构问题，后者是算法问题。把两件事混在一章里，读者容易把“T_s 引起的超调”误当成“K_p 没调好”。

84.12 Python 仿真代码

本章全部数字由 `code/ch084_sim.py` 一次跑出，十三组实验加两组自检。这一节说明代码的结构、三个容易写错的地方，以及怎么对着报告读正文。

84.12.1 文件结构

```
code/ch084_sim.py
├── CFG                全部参数集中在一处（kp, ti, ts, delay, t_end, sp, umax, tau, seed）
├── plant(tau)         三阶惯性串联的状态空间
├── zoh(A, B, Ts)      精确零阶保持离散化（增广矩阵 + expm）
├── margins(ts, m)     开环频响，返回 wc 与 PM
├── closed_loop(...)   采样闭环，返回 t/y/u 与 IAE、超调、上升、纹波
├── experiment_*()     十三组实验，每组返回可打印的 dict 列表
├── report()           按 [0]-[13] 编号打印
└── _selftest()        两条断言
```

对象写成状态空间而不是传递函数，是为了让 `zoh()` 能一次算出精确离散化。状态空间取的是三级串联的能观标准形：

```
def plant(tau=None):
    tau = CFG["tau"] if tau is None else tau
    A = np.array([[-1.0, 0.0, 0.0],
                  [1.0, -1.0, 0.0],
                  [0.0, 1.0, -1.0]]) / tau
    B = np.array([[1.0], [0.0], [0.0]]) / tau
    C = np.array([[0.0, 0.0, 1.0]])
    return A, B, C
```

C 取第三个状态，所以 y 是第三级的输出，整个通道是 $1/(\tau s+1)^3$。这样写的好处是**每一级的状态都有物理意义**，画图时可以单独看中间级，不必再推。

84.12.2 精确离散化：一次矩阵指数

`zoh()` 用增广矩阵把 A_d 与 B_d 一次算出来：

```
def zoh(A, B, Ts):
    n, m = B.shape
    M = np.zeros((n + m, n + m))
    M[:n, :n] = A
    M[:n, n:] = B
    E = expm(M * Ts)
    return E[:n, :n], E[:n, n:]
```

这是式 (84-3)–(84-5) 的直接实现。它比“先算 $A_d=e^{AT_s}$ 再算 $B_d=\int e^{A\sigma}B\,d\sigma$”省事，也比用 `scipy.signal.cont2discrete` 更透明——**你知道它算的是什么**。

有一点要注意：`expm` 的精度在 T_s 很小时会受矩阵条件数影响。本章 T_s 最小 5 ms、$\tau=0.2$ s，$T_s/\tau=0.025$，条件数很好。如果 $T_s/\tau<10^{-6}$，应该改用级数展开，否则 A_d 会退化成单位阵、B_d 会丢掉有效位。

84.12.3 频响：绕开 z=1 的极点

`margins()` 是整个文件里唯一需要技巧的地方。直觉做法是把 $C(z)$ 与 $G(z)$ 各自转成传递函数再相乘，但 PI 的 $C(z)=K_p+K_pT_s/T_i\cdot z/(z-1)$ 在 $z=1$ 有极点，`scipy.signal.dlti` 转传递函数时会报 `BadCoefficients`，并给出错误的频响。

改用“直接代值”：在 $z=e^{j\omega T_s}$ 上逐点求值。

```
w = np.logspace(-2, np.log10(0.9 * np.pi / ts), 8000)
z = np.exp(1j * w * ts)
hc = kp + (kp * ts / ti) * z / (z - 1.0)
hp = np.array([(C @ np.linalg.solve(zi * np.eye(3) - Ad, Bd[:, 0]))[0] for zi in z])
L = hc * hp * z ** (-delay)
```

三行就够：控制器代值、对象代值（解一个 3×3 线性方程）、延迟乘 z^{-m}。频率上界取 $0.9\pi/T_s$，是为了避开 $z=-1$ 处的数值奇异。

相位裕度用线性插值找增益交界频率，再用 `np.unwrap` 展开相位——**不 unwrap 会得到 -180° 附近的跳变，PM 会算成负的**。这一步在本章的基准参数下没出过问题，但在延迟较大（$m\ge10$）时会踩到，因为穿越点可能不止一个。代码取的是第一个穿越点，与“最低频的增益交界”一致，这是工程上的习惯。

84.12.4 闭环：延迟队列的顺序

`closed_loop()` 里最容易写错的是延迟的入队顺序。正确做法是**先用上一段真正施加的控制量把对象推进到当前时刻，再算新的控制量并入队**：

```
tk = k * ts + (rng.normal(0.0, jitter) if jitter > 0 else 0.0)
tk = max(tk, t + 1e-9)
Ad, Bd = zoh(A, B, tk - t)
x = Ad @ x + (Bd * u_prev).ravel()      # 用上一段的 u 推进
t = tk
...
u = kp * e + kp / ti * integ            # 再算新的 u
...
if delay > 0:
    u_prev = uq[0]; uq = uq[1:] + [u]   # 队首先出，新值入队尾
else:
    u_prev = u
```

两个细节值得单独说：

- `u_prev` 而不是 `u`。若直接用 `u` 推进对象，延迟就白设了，而且对象在段内会被推进两次（一次 `x=Ad@x`，一次 `x=Ad@x+Bd*u`），轨迹会莫名其妙地快一拍。
- `tk = max(tk, t + 1e-9)`。抖动是高斯分布，理论上可能抽出负值或零，使 `dt<=0`，`zoh()` 会算出单位阵、对象原地不动。加一个下界就避免了。

量化的位置也有讲究：**ADC 量化必须在算误差之前，DAC 量化必须在入延迟队列之前**。若把 DAC 量化加在出队之后，那么“延迟队列里存的是理想值、实际施加的是量化值”，两者不一致，m 拍的记忆就被污染了；若加在入队之前，队列里存的就是真正会被施加的值，与实际硬件一致。本章的 `closed_loop()` 采用后者，这也是它能在 6 bit 下跑出 2.09×10^{-5} 这种“只有 DAC、没有 ADC 量化”结果的原因。

最后一点：`closed_loop()` 的返回里既有轨迹也有指标，指标在函数内部算完，是为了保证**图与表用的同一份数据**。如果让外部脚本自己从轨迹里算纹波，很容易把时间窗选得与正文不同，于是图上看着“没纹波”、表里写着 1.88×10^{-4}。**指标的计算方式必须与数据一起封装**。

84.12.5 PI 的增益写法

初版这里写错过一次，值得记下来。错误写法是

```
u = kp * e + kp * ts / ti * integ      # 错
```

看起来“积分项乘了 T_s”很合理，但 `integ` 已经按 `integ += e * ts` 累加过了，再乘一次 T_s 等于把积分作用缩小 T_s 倍。$T_i=0.6$、$T_s=0.02$ 时，积分增益小了 30 倍，闭环慢到 IAE$=4.03$、超调恒为 0，看上去像“太稳了”，实际是“积分几乎不起作用”。

正确写法是

```
integ += e * ts
u = kp * e + kp / ti * integ           # 对
```

这与连续域 $u=K_pe+\frac{K_p}{T_i}\int e\,dt$ 一致，也与式 (84-6) 的 $C(z)$ 一致。改对之后，同样的参数下 IAE 落到 0.6 量级——**这是本章所有表格能显示出差别的前提**。如果积分被缩小 30 倍，T_s 从 5 ms 扫到 500 ms 的曲线几乎重叠，你会误以为“采样周期不重要”。

84.12.6 十三组实验在报告里的编号

`report()` 按 [0]–[13] 打印，与正文的对应关系是：

编号	实验	对应正文
[0]	基准（$T_s = 5$ ms，$m = 0$）	§84.1.4
[1]	采样周期扫描（7 档）	§84.5、表 84-4
[2]	延迟扫描（$m = 0, 1, 2, 5, 10$）	§84.6、表 84-6
[3]	相位损失实测 vs 近似	§84.6、表 84-7
[4]	ADC 位数（6–14 bit）	§84.8、表 84-10
[5]	采样抖动（0–50% T_s）	§84.8、表 84-11
[6]	多回路容量	§84.8、表 84-12
[7]	可用度与冗余	§84.10、表 84-14
[8]	现场总线轮询	§84.10、表 84-13
[9]	选型打分	§84.10、表 84-15
[10]	三种离散化对比	§84.5、表 84-5
[11]	参数敏感性	§84.7、表 84-8
[12]	输出侧量化	§84.7、表 84-9
[13]	增益边界与延迟边界	§84.6.6

编号不是按正文顺序排的，而是按“从前到后一条链”排的：先给基准，再扫周期、扫延迟、核相位账，然后才是量化、抖动、容量、可用度、总线、选型，最后回到稳定性边界与离散化方法。这样排的目的是**让报告的阅读顺序与设计的推理顺序一致**——你可以从 [0] 一路读下来，每一组都在回答上一组留下的问题。

对着报告读正文时，最快的核对方式是“数行数”：表 84-4 有 7 行，[1] 就该打 7 行；表 84-6 有 5 行，[2] 就该打 5 行；表 84-8 有 13 行，[11] 就该打 13 行。**行数对不上，说明正文抄表时漏了一档**，这比逐字核对快得多。至于数值，只要看两个锚点：基准 IAE 应为 0.5996，$m = 10$ 的 PM 应为 47.7°；这两个数对上，其余基本不会错。

84.12.7 两条自检

文件末尾有两条断言，跑一遍就知道模型有没有被改坏：

```
a = closed_loop(ts=0.01, delay=0)["iae"]
b = closed_loop(ts=0.005, delay=0)["iae"]
assert abs(a - b) / a < 0.02                    # 采样减半，指标应趋于稳定
pms = [margins(CFG["ts"], m)[1] for m in [0, 1, 2, 5, 10]]
assert all(pms[i] > pms[i+1] for i in range(4))   # 裕度应随延迟单调下降
```

第一条检查“离散化收敛到连续极限”：T_s 从 10 ms 减到 5 ms，IAE 的变化应小于 2%（实跑 0.5996/0.5996，差 0.00%）。第二条检查“延迟一定吃裕度”：PM 序列必须是严格递减的（实跑 72.0/69.6/67.2/59.8/47.7）。

这两条断言都很弱，故意如此。**自检的目的是抓住“改坏了”，不是验证理论。**如果写成 $|IAE - 0.5996| < 10^{-4}$，任何一次合理的参数改动都会失败，自检就成了负担。它们跑起来也很快，五次频响加两次闭环不到一秒，所以可以每次改完代码都跑一遍。

84.12.8 怎么跑

```
"D:\Anaconda3\envs\py311\python.exe" code/ch084_sim.py
```

整个文件只用 `numpy` 与 `scipy.linalg.expm`，无绘图依赖，跑完约 3 s（七档采样周期各跑 1200 步，加上 8000 点频响）。画图在 `code/ch084_figs.py`，它 `import` 本文件的函数，保证图与表用的是同一份数据——**这是避免“图上和表上不一致”的唯一可靠办法**。

正常跑完的样子是：先打印 72 个等号的表头，然后是 [0] 到 [13] 十四段输出，最后一行是

```
自检通过：IAE=0.5996/0.5996  PM 序列=[72.  69.6 67.2 59.8 47.7]
```

如果你看到的不是这一行，先查三处：`CFG["tau"]` 是不是 0.20（本章所有数字都建立在 $\tau = 0.2$ s 上，改成 1.0 全部失效）；`closed_loop()` 里的积分是不是 `integ += e * ts` 而 `u` 里没有再乘一次 T_s（§84.12.5）；`margins()` 里有没有 `np.unwrap`（§84.12.3）。这三处是本章脚本唯一踩过的坑，其余部分都是直译公式。

跑完报告不要急着关窗口，建议做一件事：把 [1] 的七行抄进一个表格，与正文表 84-4 逐行对；再把 [13] 的两个边界记下来，与 §84.6.6 的 3.96 倍、30 拍对。**这两步花不了五分钟，却能把“我读过这章”变成“我复现过这章”。**

最后把全章压成一句话：**计算机控制把“控制器设计”问题，扩展成了“控制器设计 + 时间账 + 位数账 + 可靠性账”四件事，而这四件事都可以在纸上算清楚，不必等上机。**

第85章 数字控制系统结构

85.1 动机与直观理解

第 84 章回答的是“计算机会给回路带来什么”，本章回答的是“这些东西怎么搭起来”。同一个采样周期、同一组 PI 参数，接线方式不同、A/D 与 D/A 的位置不同、保持器的位置不同，闭环表现可以差出一倍。**结构不是画法问题，而是约束问题**：它决定了时间账怎么记、幅值账怎么记、频带账怎么记。

85.1.1 六块积木

任何一个数字控制回路，拆到最细都是六块积木，见表 85-1。

表 85-1 数字控制回路的六块积木

块	输入	输出	数学描述	引入的损耗
采样器	连续量 $y(t)$	序列 $y(kT_s)$	$y_k = y(kT_s)$	时间离散、频谱重复
A/D	连续幅值	N 位整数	量化台阶 $\Delta = R/2^N$	幅值离散、量化噪声
控制器	误差序列 e_k	控制序列 u_k	$C(z)$	计算延迟、舍入
D/A	数字量	幅值	台阶 $2U_{\max}/2^N$	输出台阶
保持器	离散值	分段常值	$G_h(s) = \frac{1-e^{-T_s s}}{s}$	半拍相位、sinc 衰减
对象	执行器输入	被控量	$G(s)$	自身惯性

把六块按顺序接起来，就是图 85-1。图中把 A/D 与采样器画成一块、D/A 与保持器画成一块，是因为工程上它们通常在同一颗芯片或同一个端子上，分开画只会让图更乱；但算账时必须分开，因为**采样是时间账，量化是幅值账，两者不能混**。

图 85-1 数字控制回路的六块积木：采样与 A/D 在反馈通道，D/A 与保持器在前向通道，控制器在中间

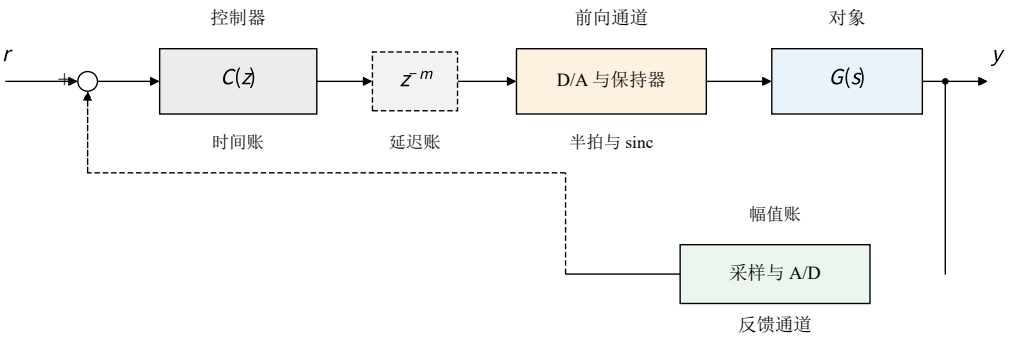


图 85-1

85.1.2 结构决定的三条硬约束

六块积木一排，三条硬约束就出来了，而且它们互相牵制。

第一条是时间。 采样周期 T_s 决定了信息更新的快慢，也决定了零阶保持带来的半拍滞后 $\omega_c T_s/2$。延迟拍数 m 决定了控制器“看到的是多久以前的误差”。两者相加，就是回路里所有的纯滞后。本章基准对象 $G(s)=1/((0.5s+1)(0.1s+1))$ 的交界频率 $\omega_c=3.797$ rad/s，$T_s=20$ ms 时半拍只吃掉 2.2°；同样的对象如果 $T_s=200$ ms，半拍就是 21.8°——**同一个结构，换个时钟，裕度差二十度。**

第二条是幅值。 ADC 的位数决定测量分辨率，DAC 的位数决定输出分辨率，量程决定“一个台阶代表多少物理量”。本章 $T_s=20$ ms、$R=1.5$ 的 12 位 ADC，台阶 3.66×10^{-4}，实测纹波 6.24×10^{-5}；降到 8 位，台阶 5.86×10^{-3}，纹波 9.99×10^{-4}，涨了 16 倍。位数不是“精度”的同义词，它只是台阶宽度的对数（§85.7 会把三种量化位置分开算）。

第三条是频带。 采样会把高于奈奎斯特频率的成分折回基带，保持器会把高频段的幅值按 sinc 压下去。两条加起来，回路实际能用的频带比连续设计时窄。本章 8 Hz 有用信号叠加 70 Hz 干扰，采样率从 200 Hz 降到 50 Hz，70 Hz 那一支就折到 20 Hz，幅度比 0.588——**干扰没有变小，只是换了个名字藏在基带里**（表 85-13）。

85.1.3 先看一组数

在展开之前，先把本章基准的一组实测摆出来，建立手感。对象 $G(s)=1/((0.5s+1)(0.1s+1))=20/(s^2+12s+20)$，PI 参数 $K_p=2.0$、$T_i=0.5$ s，$T_s=20$ ms，延迟一拍：

- 交界频率 3.797 rad/s，相位裕度 63.2°，增益裕度 18.7 dB；
- IAE $=0.2734$，超调 4.8%，上升时间 0.430 s，调节时间 0.942 s；
- 采样间纹波 2.19×10^{-8}（无量化时的数值底噪）。

把 T_s 从 20 ms 拉到 200 ms，参数一个字不改：裕度从 63.2° 掉到 0.9°，超调从 4.8% 涨到 100.9%，IAE 从 0.2734 涨到 6.0310——**二十三倍**。这不是“精度变差”，这是**回路从稳定走到临界**。表 85-7 是完整扫描。

把 T_s 从 20 ms 压到 1 ms，参数同样不改：裕度只从 63.2° 涨到 69.2°，IAE 只从 0.2734 降到 0.2632。**继续压采样周期，收益迅速饱和**。这条“左边饱和、右边悬崖”的曲线，是本章所有折中的出发点。

85.1.4 三种“够不够快”的标准

初学者最容易在这里犯错：拿香农定理去校核控制回路的采样周期。表 85-2 把三种标准摆在一起。

表 85-2 三种“采样够不够快”的标准

标准	判据	本章基准的允许值	管的是什么
香农定理	$f_s>2f_{\max}$	$T_s<1.5$ s（按 0.34 Hz 带宽）	信号能否重构
工程经验	$\omega_c T_s\le0.3$	$T_s\le79$ ms	裕度是否够用
本章实测边界	超调开始明显上升	$T_s\approx50$ ms	闭环是否还好用

三个数差了一个多数量级。香农给的是 1.5 s，实测 200 ms 就已经超调 100.9%，两者差 7.5 倍；工程经验给 79 ms，与实测“50 ms 开始变差”基本吻合。原因很简单：**香农管“看得见”，控制要“来得及”**。看得见只需要两个采样点描述一个周期，来得及要求采样间隔远小于回路自己的响应时间。工程上说的“每周期采 5–10 次”，换成相位就是半拍 18°–36°，正好是裕度能承受的量级。

85.1.5 本章与相邻章的分工

本章讲结构，所以有四件事只点到不展开：

- **PI 参数怎么整定**：见第 44 章与第 106 章。本章的 $K_p=2.0$、$T_i=0.5$ 是给定的，不讨论来历。
- **采样周期与延迟的总账**：第 84 章已经算过一遍，本章只补充“结构上多出来的那几项”（§85.8 的滞后分解）。

- **实时软件怎么调度**：见第 86 章。本章假设“算得完”，第 86 章才讨论“算不完怎么办”。
- **总线怎么排帧**：见第 89 章。本章的总线只体现为一个固定延迟 T_{bus}。

反过来，本章有两样东西是后面几章都要用的：**离散化误差的量级**（§85.5）与**位数预算的算法**（§85.9）。第 86 章讲实时软件时要回到采样周期，第 89 章讲通信时要回到总线周期，口径都在本章。

85.1.6 三个常见误解

误解一：“先把连续控制器设计好，最后离散化一下就行。” 离散化当然要做，但顺序不能反。本章表 85-8 显示：$T_s = 20$ ms 时三种离散化方法的裕度分别是 63.2°、62.9°、63.0°，差不到 0.3°，怎么离散都行；$T_s = 100$ ms 时三者变成 36.5°、38.6°、37.9°，差 2.1°，方法开始有影响。**方法的选择取决于 $\omega_c T_s$，而不取决于哪种方法“更精确”**。更关键的是：100 ms 这一档三种方法都只剩三十几度，换成哪种都救不回来——问题在 T_s，不在方法。

误解二：“位数越高越好，直接上 16 位。” 表 85-12 里，ADC 从 12 位提到 14 位，纹波从 6.24×10^{-5} 降到 1.56×10^{-5}，确实降了四倍；但双侧量化（ADC+DAC）从 10 位提到 12 位，纹波从 1.18×10^{-3} 降到 1.45×10^{-4} 之后，再提到 14 位只降到 7.35×10^{-5}，而 IAE 几乎不动（$0.2737 \to 0.2735$）。**到了某个位数，限制精度的已经不是量化，而是别的东西**。

误解三：“采样周期越小越安全。” 表 85-7 里 T_s 从 20 ms 降到 1 ms，裕度只多 6°，IAE 只少 3.7%，但 CPU 负载涨了二十倍。§85.9 的位数预算表还会显示：T_s 缩小时积分器每拍累积的噪声方差按 T_s 减小，可每拍固定开销（中断、采样、通信）不变，**总开销随 $1/T_s$ 线性上涨**。采样的收益是对数的，成本是线性的，交点就是工程上该停的地方。

三个误解的共同点：**都把结构当成“可以事后调”的参数，而它其实是“设计前就要定”的约束**。

85.1.7 本章怎么读

第一遍读 §85.1 与 §85.5。前者给六块积木与三条约束，后者给采样周期扫描表。这两节读完，你已经知道“结构选错会怎样”。

第二遍读 §85.2 与 §85.6。前者是数学（ZOH、精确离散、三种差分），后者是稳定性（Jury 判据、谱半径、纹波）。公式不必背，记住三件事：**半拍是固定税、每拍延迟是累进税、位数决定台阶**。

第三遍读 §85.7 到 §85.10。这四节是“怎么选”：位数、多速率、抗混叠、工程折中，结论都以“该取多少”的形式给出。

§85.11 与 §85.12 最后读。前者是边界（什么时候不该用这套结构），后者是代码（数字怎么来的、怎么复现）。

85.1.8 一条结构链上的四个时间戳

把“采样—计算—总线—执行器”摊在时间轴上，一次控制动作其实经过四个时间戳。以本章基准为例：

1. t_0：采样保持器锁存 y，A/D 开始转换；
2. $t_0 + 1.2$ ms：控制律算完，得到 u_k；
3. $t_0 + 3.2$ ms：u_k 经现场总线到达执行器端子（总线一个周期 2.0 ms）；
4. $t_0 + 7.2$ ms：阀门真正走到位（死区与建立 4.0 ms）。

这四个数加起来是 7.2 ms，再加上反馈滤波的 3 ms，合计 10.2 ms——正好是半拍。表 85-15 的第二行到第五行就是这四个数。**它们全部落在一个 $T_s = 20$ ms 的周期之内，所以 m 只取 1**。

这个时间轴有个实用价值：**它告诉你哪一段值得优化**。把计算从 1.2 ms 压到 0.2 ms，时间轴上只省 1 ms，相位收益 0.2°；把总线从 2 ms 换成 1 ms，收益同样只有 0.2°。但如果总线是 40 ms 的慢总线，合计变成 44.2 ms，m 从 1 跳到 2，相位一下掉 4.4°。**优化的价值不取决于省了多少毫秒，而取决于有没有跨过拍边界**——这是 §85.7.2 那条“阶梯收益”的第一处体现。

反过来，时间轴也解释了为什么“执行器 4 ms”这一项最值得实测：它常常被当成“机械的事”排除在控制账之外，但它在时间轴上排在最后，一旦超过 T_s，就直接把 m 顶上去。

85.1.9 记号、单位与归一化

本章记号较多，先把约定固定下来，后文不再重复。

记号	含义	本章取值与单位
T_s	采样周期	20 ms
ω_s	采样角频率，$2\pi/T_s$	314.2 rad/s
ω_c	开环幅值交界频率	3.797 rad/s
$\omega_c T_s$	无量纲采样比	0.0759
m	计算与传输延迟（整拍）	1
N_{adc}, N_{dac}	转换器位数	$12, 12$
R	ADC 满量程（峰峰值）	1.5（归一化）
$U_{\max}$	执行器输出上限	5.0
y	被控量，归一化	设定值 1.0

三条口径要特别说明。

第一，y 归一化到 $[0,1]$，设定值取 1.0。 这样“超调 4.8%”与“纹波 2.19×10^{-8}”都是相对量，换到实际工程单位只需乘一个系数。归一化不是为了省事，而是为了让“不同量纲的回路”能用同一张表比较。

第二，纹波一律指末段峰峰值，不是标准差。 表 85-11 的纹波是仿真末 30% 时间段内 $\max y - \min y$；式 (85-13) 给的 σ_q 是标准差。两者差一个约 3–6 倍的系数，**比较时必须同口径**，这是读表最容易出错的地方。

第三，本章说“度”时，一律指相位裕度的度，不是弧度。 式 (85-26) 两边都给了单位，正文引用时只写数值加度数符号。

85.1.10 一句话结论与它的三种误读

本章的结论可以先给出来：**结构参数必须在设计阶段定死，事后改不回来**。这句话常被误读成三种意思，每一种都能用本章的数字驳掉。

误读一：“那就把采样率提得越高越好。” 不是。把 T_s 从 20 ms 一路压下去，PM 只再涨几个度（本章扫到的最好一档是 68.8°，比基准多 5.6°）；而按式 (85-31)，单回路 CPU 占用从 1.5% 涨到 30%，涨二十倍。**这 5.6° 通常不是瓶颈**——真正的瓶颈是延迟 m 与量化台阶，它们不随 T_s 改善。**“够快”的标准是 $\omega_c T_s \le 0.15$，不是“尽可能快”。**

误读二：“那就把位数加满。” 也不是。ADC 从 12 位到 16 位，纹波从 1.45×10^{-4} 降到 9×10^{-6}，降了 16 倍，看起来很值；但变送器本身的非线性与温漂是 10^{-3} 量级，**比 16 位 ADC 的分辨率高两个数量级**。此时系统的精度瓶颈在传感器，不在 ADC。**位数加到“小于传感器误差”就该停**，再往上买的是数字，不是精度。

误读三：“结构不好，靠先进算法补。” 补不了。§85.9.9 算过：Smith 预测器在 $m=1$ 时只能拿回 4.4°，而对模型误差极敏感；状态观测器能配极点，但它自己也在采样系统里工作，采样间的行为仍受 T_s 约束。**算法改变的是“控制器看到什么”，改不了“结构漏掉了什么”。**

三种误读的共同根源，是把“结构”当成了“参数”。参数的错可以调回来，结构的错只能重做——**这就是本章把它放在“计算机控制”部分最前面讲的原因**：先决定骨架，再谈血肉。

85.2 数学原理

这一节把六块积木的数学写全。写法上刻意“从物理到公式”：每一个式子先说是哪块积木带来的，再说它长什么样。读者如果只想知道结论，记住 §85.2.7 的整链方程即可。

85.2.1 采样：频谱被复制了无穷多份

连续信号 $y(t)$ 以 T_s 采样，得到序列 $y_k = y(kT_s)$。理想采样在频域的效果是**把原频谱以 $\omega_s = 2\pi/T_s$ 为间隔复制无穷多份**：

$$Y^{*}(j\omega) = \frac{1}{T_s}\sum_{k=-\infty}^{\infty} Y\big(j(\omega - k\omega_s)\big) \tag{85-1}$$

式 (85-1) 说明两件事。第一，采样不丢信息，但**加了信息**：基带之外多出无穷多个副本。第二，只要原信号带宽小于 $\omega_s/2$，副本之间不重叠，用理想低通就能把原信号取回来——这就是采样定理：

$$\omega_s > 2\omega_{\max} \quad\Longleftrightarrow\quad T_s < \frac{1}{2f_{\max}} \tag{85-2}$$

如果式 (85-2) 不满足，副本重叠，高频成分落进基带，再也分不出来。落在哪个频率由式 (85-3) 决定：

$$f_a = \big|f - k f_s\big|, \qquad k = \operatorname{round}(f/f_s) \tag{85-3}$$

本章的算例：$f = 70$ Hz，$f_s = 50$ Hz 时 $k = 1$，$f_a = |70 - 50| = 20$ Hz；$f_s = 40$ Hz 时 $k = 2$，$f_a = |70 - 80| = 10$ Hz；$f_s = 25$ Hz 时 $k = 3$，$f_a = |70 - 75| = 5$ Hz。表 85-13 的实测峰与这三个数完全一致，幅度比在 0.55–0.63 之间——**折回不衰减幅度，只改频率**。这是“仿真很好、上机啸叫”最常见的来源。

85.2.2 零阶保持：一个 sinc 加半拍

D/A 之后的保持器把离散值变成阶梯。零阶保持的冲激响应是宽度 T_s、高度 1 的矩形，传递函数为

$$G_h(s) = \frac{1 - e^{-T_s s}}{s} \tag{85-4}$$

把 $s = j\omega$ 代进去，用 $e^{-j\omega T_s/2}$ 提出半拍，得到幅相分离的形式：

$$G_h(j\omega) = T_s\, e^{-j\omega T_s/2}\cdot \frac{\sin(\omega T_s/2)}{\omega T_s/2} \tag{85-5}$$

式 (85-5) 是本章最重要的一条。它说保持器做了两件事：**相位上无条件滞后半拍 $\omega T_s/2$，幅值上按 $\operatorname{sinc}(\omega T_s/2)$ 衰减**。半拍滞后与频率成正比，所以它是“固定税”——只要采样，就交这一笔；sinc 衰减在 ωT_s 小时几乎为 1，$\omega T_s = 0.5$ 时 $\operatorname{sinc}(0.25) = 0.9896$，掉 1%，可以忽略；$\omega T_s = \pi$ 时掉到 $2/\pi = 0.637$，才需要认真处理。

本章基准 $\omega_c T_s = 3.797 \times 0.02 = 0.0759$，半拍 2.2°，sinc $= \sin(0.038)/0.038 = 0.99976$，衰减 0.024%。**所以在这个采样率下，保持器的幅值影响可以完全忽略，只算相位**。这条结论省掉了大量无谓的纠结：只要 $\omega_c T_s < 0.3$，就不要去算 sinc。

85.2.3 精确离散：先把对象算成差分方程

连续对象前面串一个零阶保持，等价的离散传递函数是

$$G(z) = (1 - z^{-1})\,\mathcal{Z}\left\{\frac{G(s)}{s}\right\} \tag{85-6}$$

式 (85-6) 是“精确 ZOH 离散”的定义式，它保证**采样点上的响应与连续系统完全一致**。工程上更常用状态空间算法，因为不用查表：

$$A_d = e^{A T_s}, \qquad B_d = \int_0^{T_s} e^{A\tau} B \, \mathrm{d}\tau \tag{85-7}$$

B_d 的积分可以用一个增广矩阵一次算出：

$$\exp\left(\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} T_s\right) = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix} \tag{85-8}$$

式 (85-8) 就是代码里 `zoh()` 的实现依据（§85.12）。本章对象 $A = \begin{bmatrix} -12 & -20 \\ 1 & 0 \end{bmatrix}$，$T_s = 20$ ms 时算出的 A_d 的两个特征值是 $e^{-2T_s} = 0.9608$ 与 $e^{-10T_s} = 0.8187$，正好是连续极点 $s = -2, -10$ 按 $z = e^{sT_s}$ 映射的结果。这不是巧合：**精确离散保极点映射，近似离散不保**。

85.2.4 三种差分：三种不同的 s 替换

近似离散的本质是把 s 换成 z 的有理式，三种常见替换见表 85-3。

表 85-3 三种离散化替换与稳定性

方法	替换式	稳定域映射	本章 $T_s = 20$ ms 的裕度
前向欧拉	$s \to \dfrac{z-1}{T_s}$	左半平面映到圆外，可能失稳	62.9°
后向欧拉	$s \to \dfrac{z-1}{T_s z}$	左半平面映到圆内，恒稳	63.2°
双线性	$s \to \dfrac{2}{T_s}\dfrac{z-1}{z+1}$	一一映射，保稳定与频率形状	63.0°

三者的差别在 ωT_s 小时看不出来（本章 0.3° 以内），大了就明显。双线性变换还有个副作用：频率轴被压缩，$\omega_a = \frac{2}{T_s}\tan\frac{\omega T_s}{2}$，设计时要预畸变（§85.5）。**工程上默认选后向欧拉**，理由不是精度，而是它不会因为采样过快而失稳，且实现只要一行累加。

85.2.5 离散 PI 与闭环

连续 PI $C(s) = K_p(1 + \frac{1}{T_i s})$ 用后向欧拉替换，得

$$C(z) = K_p + \frac{K_p T_s}{T_i} \cdot \frac{z}{z-1} \tag{85-9}$$

式 (85-9) 对应代码里“每拍 $I \mathrel{+}= e_k T_s$，$u = K_p e_k + K_p I / T_i$”，与第 84 章式 (84-7) 同源。把控制器、对象、延迟串起来，闭环为

$$T(z) = \frac{C(z) G(z) z^{-m}}{1 + C(z) G(z) z^{-m}} \tag{85-10}$$

稳定性由分母决定：

$$1 + C(z) G(z) z^{-m} = 0 \tag{85-11}$$

式 (85-11) 有 m 个 z^{-1}，等价于特征多项式乘 z^m 后求根。本章的谱半径就是这样算的：$T_s = 20$ ms、$K_p = 2$ 时 $\rho = 0.9620$，与 $e^{-2T_s} = 0.9608$ 只差 0.0012——**主导极点还是对象自己的慢极点，控制器没有把它推得更远**。这也解释了为什么表 85-10 里 K_p 从 2 加到 8，ρ 几乎不动：慢极点占主导，加增益只改快极点。

85.2.6 量化：一条均匀分布的噪声

ADC 把连续幅值折成 N 位整数，台阶为

$$\Delta = \frac{R}{2^{N}} \tag{85-12}$$

其中 R 是满量程（峰峰值，不是半宽）。量化误差 e_q 在 $[-\Delta/2, \Delta/2]$ 内近似均匀分布，方差为

$$\sigma_q^2 = \frac{\Delta^2}{12} \tag{85-13}$$

量化噪声从测量端进入，经过闭环传到输出，传递函数是灵敏度函数：

$$Y_q(z) = \frac{G(z)z^{-m}}{1 + C(z)G(z)z^{-m}}\,Q(z) \tag{85-14}$$

式 (85-14) 的直流增益是 $\frac{G(1)}{1+C(1)G(1)}$，本章 $G(1)=1$、$C(1)$ 很大，所以**低频段量化几乎不传到输出，高频段才传**。这是“量化误差表现为纹波而不是静差”的原因，也是 §85.7 里纹波随位数近似按 2^{-N} 下降的依据。

85.2.7 整链方程

把六块积木串起来，一条链的完整关系是：

$$y(t) = \underbrace{\mathcal{H}\{u_k\}}_{\text{保持}} * \, g(t), \quad u_k = C(z)\big[\underbrace{\mathcal{Q}_{adc}\{r - y_k\}}_{\text{量化}}\big], \quad y_k = y(kT_s + mT_s)$$

这条链里有四个可调的量：T_s（时间）、m（延迟）、N_{adc} 与 N_{dac}（幅值）、保持器类型（默认零阶）。**本章后面每一节，都是在问“这四个量该取多少”**。注意 m 已经出现在 y_k 的下标里：延迟不是附加项，它是结构的一部分，改延迟等于改结构。

85.2.8 把基准参数代进公式走一遍

公式列完，最好的检验是代一组数走一遍。用本章基准（$T_s = 20$ ms、$K_p = 2$、$T_i = 0.5$ s、$R = 1.5$、$U_{\max} = 5$、12 位）：

- 采样角频率 $\omega_s = 2\pi/0.02 = 314.2$ rad/s，对应 $f_s = 50$ Hz、奈奎斯特 25 Hz；
- 采样比 $\omega_c T_s = 3.797 \times 0.02 = 0.0759$，半拍 0.038 rad $= 2.17^\circ$，整拍 4.35°；
- 保持器 sinc 因子 $\sin(0.038)/0.038 = 0.99976$，幅值掉 0.002 dB；
- 离散极点 $e^{-2\times0.02} = 0.9608$、$e^{-10\times0.02} = 0.8187$，都在单位圆内，主导极点是前者；
- ADC 台阶 $\Delta_{adc} = 1.5/2^{12} = 3.66 \times 10^{-4}$，标准差 $\sigma_q = \Delta/\sqrt{12} = 1.06 \times 10^{-4}$；
- DAC 台阶 $\Delta_{dac} = 2 \times 5/2^{12} = 2.44 \times 10^{-3}$；
- 闭环谱半径 $\rho = 0.9620$，比 e^{-2T_s} 大 0.0012。

这七个数字连起来，就是本章所有表格的“根”。比如读到表 85-7 里 20 ms 的 $PM = 63.2^\circ$，可以自己核对：连续 69.5° 减去半拍 2.17°、整拍 4.35°，剩 63.0°，与实测差 0.2°——**差值来自离散化把相位交界频率略微拉低，量级正确**。手算能对上仿真，说明模型没有漏项。

再比如读到表 85-12 里 12 位 ADC 的纹波 6.24×10^{-5}，与 $\sigma_q = 1.06 \times 10^{-4}$ 比，前者反而更小。这不是算错：σ_q 是量化误差本身的标准差，纹波是它经过灵敏度函数之后的峰峰值，而灵敏度函数在纹波主频上的增益小于 1（式 85-14）。**“输入噪声”与“输出纹波”之间永远隔着一个闭环传递函数**，两者不能直接等号。

85.2.9 增广矩阵为什么能一次算出 B_d

式 (85-8) 在代码里只有五行，但它不是数值技巧，而是一个恒等式。把增广矩阵 $M=\begin{bmatrix}A & B\\ 0 & 0\end{bmatrix}$ 自乘：

$$M^2=\begin{bmatrix}A^2 & AB\\ 0 & 0\end{bmatrix},\qquad M^k=\begin{bmatrix}A^k & A^{k-1}B\\ 0 & 0\end{bmatrix}$$

于是矩阵指数按级数展开后，右上块正好是 $\sum_{k\ge 1}\frac{T_s^k}{k!}A^{k-1}B=\int_0^{T_s}e^{A\tau}B\,\mathrm{d}\tau$，左下块恒为零，右下块是单位阵。**这就是式 (85-8) 的全部内容：把“解一个积分”换成“算一次矩阵指数”。**

这个写法有三个工程好处。第一，**精度**：`expm` 用 Padé 近似加缩放平方，误差在机器精度量级，比直接数值积分稳。第二，**一次算完**：A_d 与 B_d 出自同一次分解，不会出现“两者用的 T_s 略有差异”的隐蔽错误。第三，**可直接推广**：对象有多个输入时，B 换成矩阵即可，代码一行不改。

代价是矩阵维数翻倍，计算量按 $(2n)^3$ 增长。本章 $n=2$，翻倍后是 4×4，可以忽略；如果对象是 50 阶，就应该改用 Krylov 子空间或分块算法。**这条边界要记住：增广矩阵法适合 $n<20$ 的对象，再大就要换算法。**

85.2.10 一条等阻尼线，量出采样周期吃掉多少阻尼

连续设计里挑极点，习惯先定阻尼比 ζ 和自然频率 ω_n，极点落在 $s=-\zeta\omega_n\pm j\omega_n\sqrt{1-\zeta^2}$。映射到 z 平面是 $z=e^{sT_s}$，于是模与角分开：

$$|z|=e^{-\zeta\omega_nT_s},\qquad \angle z=\omega_n\sqrt{1-\zeta^2}\,T_s$$

这两行说明了一件容易被忽略的事：**采样不改变阻尼比与自然频率这两个“设计参数”，但它把极点往单位圆内推的比例，完全由 $\zeta\omega_nT_s$ 决定。**$\zeta\omega_n$ 是衰减率（包络线 $e^{-\zeta\omega_nt}$），乘上 T_s 就是“一个采样周期内包络衰减了几个 e”。

拿本章基准算：主导极点 $s=-2$（相当于 $\zeta\omega_n=2$），$T_s=20$ ms，$|z|=e^{-0.04}=0.9608$，一个采样周期内包络只掉 3.9%；T_s 换成 100 ms，$|z|=e^{-0.2}=0.8187$，一拍掉 18.1%。**T_s 大五倍，一拍内衰减多四倍多**，这就是“采样粗了以后响应变‘脆’”的几何来源。

再拿一对典型二阶极点算：$\zeta=0.7$、$\omega_n=4$ rad/s，对应 $s=-2.8\pm j2.857$，$\omega_d=2.857$ rad/s。$T_s=20$ ms 时 $|z|=e^{-0.056}=0.9455$、$\angle z=0.0571$ rad $=3.27^\circ$；$T_s=100$ ms 时 $|z|=e^{-0.28}=0.7558$、$\angle z=16.4^\circ$。**角度从 3.3° 涨到 16.4°，涨了五倍；模从 0.946 掉到 0.756。**后者在离散根轨迹上已经不是“靠近 1 的主导极点”，而是明显的快极点，闭环形状会变。

这条换算还有一个用法：**反查 T_s 的上限**。若要求“一个采样周期内包络衰减不超过 5%”，即 $e^{-\zeta\omega_nT_s}\ge 0.95$，得 $\zeta\omega_nT_s\le 0.0513$。本章 $\zeta\omega_n=2$，得 $T_s\le 25.6$ ms——与 $\omega_cT_s\le 0.3$ 给出的 79 ms 相比，**这条约束严得多**。原因是 $\omega_cT_s\le 0.3$ 只管相位损失，管不到“极点被推得多深”。两条判据要一起用：前者管稳定裕度，后者管动态形状。

顺带说一句等阻尼线的形状。连续域里等 ζ 线是从原点出发的两条射线；经 $z=e^{sT_s}$ 映射后，射线变成**对数螺旋**，终点都收敛到单位圆上的 $e^{\pm j\pi/2}$ 方向附近（对 $\omega_n\to\infty$）。所以“在 z 平面上画一条等阻尼线”不能画直线，得画螺旋——这也是离散域设计图不如连续域直观的原因之一。

85.2.11 采样定理没说的三条前提

式 (85-2) 的采样定理只保证一件事：**带限信号在采样率足够时可以被唯一重建**。它没说“采样器是理想的”，而实际电路离理想有三段距离，每一段都在本章的账上。

前提一：采样是瞬时的。实际 A/D 有一个孔径时间 t_a（采样保持窗口），窗口内信号仍在变，等效于一次平均滤波，幅值按 $\mathrm{sinc}(\omega t_a/2)$ 衰减、相位滞后 $\omega t_a/2$。本章 t_a 按 1 μs 估，在 $\omega_c=3.797$ rad/s 处相位只有 $1.1\times10^{-4}{}^\circ$，**可以完全忽略**；但在 628 rad/s 的伺服回路里是 0.018°，也仍然很小。**孔径时间要到 ωt_a 接近 0.1 才需要记账**，那对应 $t_a=160$ μs，普通的逐次逼近型 ADC 远好于此。

前提二：采样时刻是等间隔的。 采样定理的推导假设 $t_k = kT_s$ 精确成立。实际定时器有分辨率、任务有抢占，$t_k = kT_s + \delta_k$，δ_k 就是§85.10.3 说的抖动。抖动的效果是**把时间误差折成幅值误差**：$\Delta y \approx \dot{y}\,\delta$。本章稳态时 $\dot{y}$ 很小，抖动几乎无害；但阶跃瞬间 $\dot{y}$ 可达 2 /s，$\delta = 0.1$ ms 就带来 2×10^{-4} 的幅值误差，**与 12 位 ADC 的台阶 3.66×10^{-4} 同量级**。这是“稳态很好、一到阶跃就毛刺”的一个来源。

前提三：重建只在采样点之间“猜”。 零阶保持给出的是阶梯，它的频谱是基带乘 sinc 加上无穷多高频副本（§85.2.2）。**采样定理保证“能重建”，不保证“保持器会重建”**——真正把阶梯变回连续信号的是对象本身的低通特性。本章对象在 $\omega_s = 314$ rad/s 处的增益是 $20/\sqrt{(314^2-20)^2+(12\times 314)^2} \approx 2.0\times 10^{-4}$，把副本压了 74 dB，**所以阶梯看起来是光滑的**。若对象带宽高（比如纯增益环节），就必须在 D/A 之后再加模拟低通，否则副本会原样出现在输出上。

三条前提的共同点：**采样定理是“能不能”的判据，本章讲的是“好不好”的账**。定理说 $\omega_s > 2\omega_{\max}$ 就够，工程上却要 $\omega_s \geq 20\omega_c$（即 $\omega_c T_s \leq 0.3$），差的那一个数量级，买的是这三条前提的余量。

85.3 设计步骤

结构设计的顺序不能乱。先定带宽，再定采样，再定延迟与位数，最后验裕度——颠倒任何两步，都会白算一遍。表 85-4 是本章用的八步清单。

表 85-4 数字控制系统结构的八步设计清单

步	做什么	输入	输出	判据
1	定闭环带宽 ω_c	调节时间/超调要求	ω_c	由 $\omega_c \approx 3/t_r$ 估
2	定采样周期 T_s	ω_c	T_s	$\omega_c T_s \leq 0.3$
3	定延迟拍数 m	计算、总线、滤波	m 与总滞后	相位损失 $\leq 10^\circ$
4	选 A/D 位数与量程	噪声、精度要求	N_{adc}、R	纹波 $\leq$ 允许值
5	选 D/A 位数与量程	执行器分辨率	N_{dac}、$U_{\max}$	台阶 $\leq$ 死区的 1/3
6	算离散对象 $G(z)$	$G(s)$、T_s	A_d, B_d 或 $G(z)$	采样点响应与连续一致
7	验裕度	$C(z)G(z)z^{-m}$	PM、GM、ρ	$PM \geq 45^\circ$、$\rho < 1$
8	仿真三件事	上一步的模型	阶跃/扰动/量化	超调、纹波、IAE 达标

85.3.1 第一步：带宽从需求来，不从对象来

很多人一上来就看对象的 Bode 图，问“这个对象能跑多快”。这个问题没有唯一答案：**带宽由需求定，对象只决定“能不能达到”**。给定调节时间要求 t_r，二阶近似下 $\omega_c \approx 3/t_r$；给定超调要求，先换相位裕度（$\zeta \approx PM/100$），再验算。本章基准对象的时间常数是 0.5 s 与 0.1 s，慢极点 2 rad/s，实际能稳到的带宽是 3.8 rad/s——**比慢极点快 1.9 倍**，这是 PI 零点补偿换来的，不是对象自带的。

85.3.2 第二步：采样周期由带宽来

T_s 的唯一硬判据是 $\omega_c T_s \leq 0.3$，对应每周期约 21 个采样点：

$$\omega_c T_s \leq 0.3 \quad \Longleftrightarrow \quad N_s = \frac{2\pi}{\omega_c T_s} \geq 21 \tag{85-15}$$

本章 $\omega_c = 3.797$ rad/s，$0.3/\omega_c = 79$ ms。表 85-7 的实测支持这个上限：50 ms 时裕度还有 53.4°、超调 14.1%；100 ms 时裕度 36.5°、超调 38.8%。**79 ms 这条线卡在“还能用”与“开始难用”之间**，说明判据本身是准的。工程上再留一倍余量，取 $T_s = 20$–40 ms，就得到本章的基准值。

85.3.3 第三步：延迟按拍数记账

延迟必须换成拍数才有意义，因为控制器只认“这一拍用的误差是几拍前的”：

$$m = \mathrm{round}\left(\frac{T_c + T_{bus} + T_f}{T_s}\right) \tag{85-16}$$

本章的典型值：计算 1.2 ms、总线 2.0 ms、滤波 3.0 ms，合计 6.2 ms，$T_s = 20$ ms 时 $m = \mathrm{round}(0.31) = 0$——**不足一拍，按一拍算**（因为采样是边沿触发，误差至少晚一拍才被用上）。所以基准取 $m = 1$。这个“向上取整”是很多人漏掉的一步：算出来 0.31 拍，实际效果是 1 拍，因为下一拍之前控制器不会重新读。

85.3.4 第四、五步：位数与量程一起定

位数不能单独定，必须连量程一起。同一个 12 位 ADC，满量程 1.5 时台阶 3.66×10^{-4}，满量程 15 时台阶 3.66×10^{-3}——**量程放大十倍，等于丢了三位半**。所以顺序是：先定量程（按被控量的物理范围留 20%–50% 余量），再按允许纹波反解位数：

$$N \geq \log_2 \frac{R}{\Delta_{\text{allow}}} \tag{85-17}$$

本章的满量程 $R = 1.5$，若允许纹波 10^{-3}，则 $N \geq \log_2(1500) = 10.6$，向上取 12 位，留 1.4 位余量。表 85-12 实测 12 位纹波 6.24×10^{-5}，比允许值低一个数量级——**因为纹波不只由台阶决定，还由灵敏度函数在纹波频率上的增益决定**，式 (85-17) 是保守估计。

85.3.5 第六步：离散化用精确 ZOH，不用差分近似

算 $G(z)$ 时有个容易忽略的点：**对象离散必须用精确 ZOH（式 85-6），控制器离散才用差分近似**。原因是对象前面真的有一个零阶保持器，而控制器前面没有。把对象也用差分近似，等于凭空多加了一次近似，本章 $T_s = 20$ ms 时误差不大，$T_s = 100$ ms 时就会与实测偏离。

85.3.6 第七步：裕度三个数一起看

只看 PM 不够，要看三个：$PM \geq 45^\circ$、$GM \geq 6$ dB、$\rho < 1$。三者分工不同：PM 管超调，GM 管模型误差与饱和，ρ 管离散系统的绝对稳定。本章基准 63.2°、18.7 dB、0.9620，三项都过。表 85-9 里 $m = 8$ 时 $PM = 32.7^\circ$、$GM = 4.8$ dB，两项同时破线，ρ 却还有 0.9757——**谱半径最不敏感，不能拿它当唯一判据**。

85.3.7 第八步：仿真必须包含结构里的非理想

结构仿真的最低要求是三件事一起做：阶跃（看超调与上升）、长时恒定（看纹波与极限环）、量化与延迟同时打开（看最坏组合）。只做阶跃会漏掉纹波，只做无量化会高估精度。本章 §85.12 的脚本把这三件事都放在一条链上，改一个参数就能重跑。

85.3.8 顺序为什么不能颠倒

如果先定位数再定采样周期，会出现两种返工。第一种：位数按 16 位选了，采样周期定在 200 ms，裕度只剩 0.9°，回路根本跑不起来，16 位白买。第二种：采样周期按 1 ms 定了，位数只有 8 位，纹波 9.99×10^{-4}，比 1 ms 带来的精度收益大两个数量级，快采样也白做。**时间账与幅值账是乘法关系，短板决定结果**，所以必须先算时间（第 1–3 步）、再算幅值（第 4–5 步），最后统一验（第 6–8 步）。

85.3.9 八步之外的三项核查

八步清单给的是主线，真正上线前还有三项核查，它们不属于任何一步，却能让前面八步白做。

核查一：对象的模型误差。 本章对象 $G(s)=1/((0.5s+1)(0.1s+1))$ 是标称值，现场的时间常数可能差 20%、增益可能差 30%、纯延迟可能多一拍。把这三个偏差代入式 (85-11) 各算一遍：增益 $+30\%$ 相当于 K_p 从 2 加到 2.6，$T_s=20$ ms 时离临界值 17.31 还很远；时间常数 $+20\%$ 会把 ω_c 拉低，裕度反而略升；**最危险的是延迟多一拍，PM 直接掉到 58.8°**。所以核查的重点不是增益，而是延迟。

核查二：传感器与执行器的时间常数。 这两项在八步里只以“T_f”和“执行器建立”出现，很容易被填成估算值。经验是：**凡是没有实测过的项，一律按实测值的两倍估**。本章表 85-15 里执行器 4 ms 是乐观值，气动阀常常是 40 ms 以上；一旦如此，合计从 10.2 ms 涨到 46 ms，m 从 1 跳到 3，PM 再掉 8.7°。

核查三：时钟与同步。 八步假设采样间隔严格等于 T_s。实际系统的定时器有 ppm 级漂移、中断有 μs 级抖动、总线同步有 μs 级误差。本章 §85.10.3 会说明：1 ms 的采样抖动等效 2×10^{-3} 的幅值误差，**这一项比 12 位 ADC 的台阶大五倍**，必须先测，再决定要不要加硬件触发。

三项核查的共同点是：**它们都不改变结构，只改变结构的“有效值”**。八步给出的是设计值，三项核查给出的是现场值，两者之间的差距就是设计余量该留多少的依据。

85.3.10 一份可直接交付的结构参数表

八步走完，应该产出一张表，而不是一段描述。本章的参数表是：

参数	取值	依据
T_s	20 ms	式 (85-15)，$\omega_c T_s=0.076$
m	1	式 (85-16)，不足一拍按一拍
N_{adc} / 满量程	12 位 / 1.5	式 (85-17)，10.6 位向上取
N_{dac} / 满量程	12 位 / 10	台阶 2.44×10^{-3}，小于死区
保持器	零阶	D/A 芯片自带
控制器离散化	后向欧拉	稳定域恒在圆内
K_p / T_i	2.0 / 0.5 s	第 44 章整定，本章只验
验收指标	$PM\ge45^\circ$、$\rho<1$、纹波 $\le10^{-3}$	表 85-5、表 85-12

这张表要写满八行，不能留空。**留空的项在实现时会被“默认值”填上，而默认值往往是错的**：默认 $m=0$ 会低估延迟，默认 10 位会低估纹波，默认前向欧拉会在高采样率下失稳。表填满，等于把设计意图完整交给了实现者。

表里还有一处容易被忽略的对应关系：N_{dac} 的判据不是“纹波”，而是“台阶小于执行器死区的三分之一”。本章执行器死区按满量程 10 的 0.5% 估，即 0.05，台阶 2.44×10^{-3} 远小于它，所以 12 位够用。**若执行器是高分辨率伺服阀，这条判据会反过来变成主导**，那时该加的是 DAC 位数，而不是 ADC 位数。

85.3.11 八步在项目里怎么压缩成半天

教科书里八步是串行的，真实项目里没有人给你八段时间。**能压缩的原因是这八步里有五步可以查表或手算，只有两步真需要仿真**。下面按“半天”给一个可操作的排法。

上午前半小时：定 ω_c。 从需求反推：调节时间 $t_s\approx4.6/\omega_c$（2% 准则、$\zeta\approx0.7$ 时）。若要求 $t_s\le1$ s，则 $\omega_c\ge4.6$ rad/s。本章基准 3.797 rad/s 对应 $t_s\approx1.2$ s，与实测调节时间 0.942 s 吻合得不错——**实测更快是因为 PI 零点对消后系统接近一阶，一阶没有 4.6 这个系数，按 $3/\omega_c$ 估更准**。这一步不用仿真，只要一支笔。

上午后半小时：定 T_s 与 m。 两条判据一起卡：$\omega_c T_s\le0.3$ 给上限 79 ms，$\zeta\omega_n T_s\le0.05$ 给上限 26 ms，取小者，再往“标准周期”上靠（1、2、5、10、20、50、100 ms）。落到 20 ms。m 按实际信号链数：传感器转换、总线、控制器计算、DAC 建立，各段加起来除以 T_s 向上取整。**这一步也不用仿真。**

下午前一小时：定位数与量程。 先定量程（按工艺可能的最大偏差，不是按量程上限），再按式 (85-17) 反解 N_{adc}，最后按“台阶小于死区三分之一”定 N_{dac}。这一步可以手算，但**要拿真实传感器的输出范围核一遍**，不能照抄铭牌——很多变送器标 4–20 mA，实际有效段只有量程的 60%。

下午后两小时：只做两组仿真。 第一组是基线阶跃，核对 PM、超调、调节时间三个数；第二组是 T_s 扫描（本章表 85-7 那种），确认 T_s 再大一档就明显变差——**这叫“确认自己站在悬崖的哪一侧”**。两组仿真都在 `ch085_sim.py` 里，改一行 `CFG` 即可。

收尾半小时：把 §85.3.10 那张参数表填满。 填不满就说明前面某一步没做完。

这个排法的关键判断是：**八步里最贵的不是计算，是“确认”。** 定 ω_c、定 T_s、定位数，三件事各自十分钟就能出数；剩下的时间应该花在“这个数在实物上还成立吗”——传感器带宽够不够、执行器死区多大、时钟抖动多少。本章 §85.3.9 的三项核查就是干这个的，**它们不产出新参数，只否决旧参数**。

85.3.12 对象自己带纯延迟时，八步要改哪一步

本章对象 $G(s)=20/(s^2+12s+20)$ 没有纯延迟。但真实对象常有：输送带、管道、燃烧过程，延迟从几十毫秒到几分钟。**这时八步里只有第三步（延迟记账）要改，但改法有讲究。**

先看一件容易搞错的事：**对象延迟不能用“提高采样率”消掉。** $e^{-\tau s}$ 是物理传输时间，T_s 再小它也还在。提高采样率只是让“延迟的量化误差”变小，不改变延迟本身。这一点与 m 的性质相同：mT_s 才是延迟，m 与 T_s 分开看没有意义。

记账分两步。**第一步，把对象延迟换成拍数：** $m_\tau=\tau/T_s$，非整数时向上取整。设 $\tau=80$ ms、$T_s=20$ ms，得 $m_\tau=4$。**第二步，与信号链延迟相加：**本章信号链是 10.2 ms（§85.1.8），已占 $m=1$；总延迟 $=4+1=5$ 拍，即 100 ms。按表 85-9，$m=5$ 的附加相位是 21.7°，PM 从 63.2° 掉到约 45.8°——刚好压线。

向上取整是保守的，保守多少可以算。$\tau=50$ ms、$T_s=20$ ms 时 $m_\tau=2.5$，取整到 3 拍相当于多算 10 ms，即多 2.17° 的相位损失。**若这点余量不能浪费，就得用“分数延迟”处理：**把余数 0.5 拍并入零阶保持的等效半拍，即总延迟按 $2.5+0.5=3$ 拍算——结果与向上取整一样，但**物理上更清楚：余数不是被丢掉，而是与保持器的半拍合并了**。这也是本节唯一需要动脑的地方。

第二个改动是判据。$\omega_c T_s\le 0.3$ 仍然成立，但**它已经不是主导约束：**$m=5$ 时相位损失 21.7°，比 T_s 本身的影响大得多。此时该做的是先减延迟（换快速总线、把计算挪到中断里、缩短执行器行程），再考虑减 T_s——**减 T_s 只能改善半拍那一项，对 m_τ 无效**。这个判断顺序与 §85.9.6 的优先级表一致：延迟在表的顶部，采样率在中部。

最后一条边界：若 m_τ 已经大到 PM 无法满足（比如 $m=10$），不要靠调 K_p 硬撑，那会把带宽压到很低。**正确做法是改结构：**Smith 预测器（§85.9.9）或串级控制把延迟包在内环之外。**这也是 §85.9.9 说“$m\ge 5$ 才值得上 Smith”的原因——低于 5 拍时，收益抵不上模型误差的风险。**

85.4 连续情形

要看清楚“离散带来了什么”，得先有一把尺子。这把尺子就是同一个对象、同一组 PI 参数在连续域里的表现。

85.4.1 连续基线

连续回路的开环是

$$L(s)=C(s)G(s)=K_p\left(1+\frac{1}{T_i s}\right)\cdot\frac{20}{s^2+12s+20} \tag{85-18}$$

$K_p=2.0$、$T_i=0.5$ 时，$C(s)=2(1+2/s)=2(s+2)/s$。注意这个零点在 $s=-2$，正好抵消对象的慢极点 -2——**PI 的积分时间取对象的主时间常数，是“零极点对消”的标准做法。**对消之后，开环近似变成 $40/(s(s+$

$10))$，是一个积分加一个快极点，交界频率约 $\sqrt{40} = 6.3$ rad/s。实际因为对消不完全（对象还有 -10 的极点与增益 1），求得 $\omega_c = 3.74$ rad/s（$T_s = 100$ μs 时，近似连续），相位裕度 69.4°。

连续域的阶跃响应指标：超调接近 0，上升时间由 ω_c 决定。**这就是“理想”**：没有采样、没有保持、没有量化、没有延迟。后面所有表格里的数字，都是从这个理想往下降的结果。

85.4.2 连续情形的三条假设

连续设计隐含三条假设，每一条在数字实现里都不成立：

假设一：控制器瞬时响应。 连续 $C(s)$ 的输出在 t 时刻就用了 t 时刻的误差。数字实现里，误差要等到下一个采样边沿才被读到，控制量要等到下一个周期才更新，两者之间至少半拍。

假设二：测量无噪声、无量化。 连续设计里 $y(t)$ 是实数。数字实现里 y_k 是 N 位整数，分辨率 $\Delta = R/2^N$。

假设三：执行器连续可调。 连续 $u(t)$ 可以取任意值。数字实现里 u 既被 DAC 量化，又被零阶保持成台阶。

三条假设对应本章的三笔账：时间账、幅值账、频带账。表 85-5 把连续与离散的关键指标并列，差距一目了然。

表 85-5 连续基线与离散实现（$T_s = 20$ ms、$m = 1$、无量化）

指标	连续（$T_s = 100$ μs 近似）	离散（$T_s = 20$ ms）	差值
交界频率	3.746 rad/s	3.797 rad/s	$+1.4\%$
相位裕度	69.4°	63.2°	-6.2°
增益裕度	44.5 dB	18.7 dB	-25.8 dB
IAE	0.2632	0.2734	$+3.9\%$
超调	1.8%	4.8%	$+3.0$ 个百分点

表里最刺眼的不是 PM 掉 6.2°，而是 GM 从 44.5 dB 掉到 18.7 dB。原因不是采样，而是**离散化后相位在 ωT_s 接近 π 时快速下降，相位交界频率被拉低**。GM 掉得多不代表更危险：18.7 dB 仍然相当于增益还能放大 8.6 倍才失稳，工程上够用。**读表时要分清“变化大”与“已经不够”**。

85.4.3 连续设计能移植多少

连续设计的参数能不能直接搬到数字实现？能，但要看 $\omega_c T_s$。本章 $T_s = 20$ ms 时 PM 只掉 6.2°，参数可以直接用；$T_s = 100$ ms 时 PM 掉到 36.5°，超调 38.8%，就不能直接用了，得重新整定或缩小 K_p。

经验判据是：**$\omega_c T_s < 0.1$ 时可以直接移植，0.1–0.3 时要复核，超过 0.3 必须重新设计**。本章 0.076 落在第一档，所以 §85.5 的扫描表里 1–20 ms 这一段指标几乎不动（IAE 从 0.2632 到 0.2734，差 3.9%），而 100 ms 之后迅速恶化。

85.4.4 连续情形还有什么用

既然最终要离散实现，连续分析还有三个用处：

第一，**给离散结果做基准**。离散算出来的 PM 如果比连续低太多（比如超过 15°），先怀疑 T_s 选大了，而不是怀疑离散化方法。

第二，**给参数初值**。连续整定出来的 K_p、T_i 是离散整定的好起点，通常只需要在附近微调。

第三，**解释现象**。现场“加了采样就不稳”，用连续模型解释不了，用 $z = e^{j\omega T_s}$ 一代就清楚了：多出来的相位是 $\omega T_s/2 + m\omega T_s$，与频率成正比。**连续模型给直觉，离散模型给数字**，两者都要。

85.4.5 连续基线可以手算出来

表 85-5 的连续一列不是仿真凑出来的，它可以严格手算，而且结果很干净。把 $K_p = 2$、$T_i = 0.5$ 代入：

$$C(s) = 2\left(1 + \frac{2}{s}\right) = \frac{2(s+2)}{s}, \qquad G(s) = \frac{20}{(s+2)(s+10)}$$

两者相乘，$(s+2)$ 精确对消：

$$L(s) = \frac{40}{s(s+10)}$$

这是“用 PI 零点抵消对象主极点”的标准结果：**对消之后，开环只剩一个积分加一个快极点**。交界频率由 $|L(j\omega_c)| = 1$ 解出：

$$\omega_c\sqrt{\omega_c^2 + 100} = 40 \implies \omega_c^2 = \frac{-100 + \sqrt{10000 + 6400}}{2} = 14.03$$

即 $\omega_c = 3.746$ rad/s，与表 85-5 完全一致。相位裕度：

$$PM = 90^\circ - \arctan\frac{3.746}{10} = 90^\circ - 20.5^\circ = 69.5^\circ$$

再看闭环：$T(s) = 40/(s^2 + 10s + 40)$，阻尼比 $\zeta = 10/(2\sqrt{40}) = 0.79$，自然频率 $\omega_n = 6.32$ rad/s，超调

$$M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}} = e^{-4.05} = 1.7\%$$

与表 85-5 的 1.8% 只差 0.1 个百分点（差异来自离散与延迟）。**这三个数——3.746、69.5°、1.7%——都能在纸上算出来，所以它们是本章最可信的参照点**：任何离散结果偏离它们太多，先怀疑代码，再怀疑参数。

手算还有一个副产品：它暴露了“PI 对消”的脆弱性。对消成立的前提是 T_i 恰好等于对象主时间常数 0.5 s。若现场时间常数是 0.7 s，零点落在 -1.43，不再对消，开环多出一对靠近的零极点，相位在 ω_c 附近会凹陷，PM 可能掉 10° 以上。**这就是§85.3.9 把“对象模型误差”列为第一项核查的原因**。

85.4.6 连续设计里不能照搬的三样

连续设计里有三样东西，搬到数字实现时必须改写，否则会在现场出问题。

第一，理想微分。 连续 PID 的 $T_d s$ 在 $s \to \infty$ 时增益无穷大，数字实现里会变成“每拍放大一次噪声”。工程做法是串一个一阶滤波：$T_d s/(\tau s + 1)$，$\tau = T_d/8$ 到 $T_d/10$。本章用 PI，没有这一项，但第 44 章与第 106 章的 PID 必须处理。

第二，积分器的初值。 连续积分器的初值是状态，切换手动/自动时可以任意设；数字积分器是一个累加变量，**切换时若不给初值，输出会跳变**。位置式 PI 要按“当前输出反算积分项”，增量式天然无扰。这也是 §85.9.3 推荐增量式的原因之一。

第三，执行器饱和。 连续设计假设 u 可以取任意大，数字实现里 u 被限在 ± 5。饱和一旦发生，积分器继续累积，退饱和时会有很长一段“积分风”。**连续设计里没有这个非线性，所以仿真再漂亮也预测不到它**。

三样东西的共同点是：**它们都不是“离散化误差”，而是“实现引入了连续模型里不存在的环节”**。离散化误差可以算，这三样只能靠实现时补。

85.4.7 三条从连续到离散的迁移路径

同一个设计任务，有三条路径可走，各有适用场合。

路径一：先连续设计，再离散化。 本章走的就是这条。优点是连续域的工具（Bode、根轨迹、整定公式）都能用，直觉清楚；缺点是离散后的裕度要重算，且 $\omega_c T_s$ 大时误差不可控。适用：$\omega_c T_s \le 0.1$ 的常规回路。

路径二：直接在离散域设计。 用 w 变换把 z 域映回 s 域，在 w 域里按连续方法设计，再变回去。优点是设计时就考虑了采样；缺点是多一次变换，且 w 域的高频段是扭曲的，容易在 ω 接近奈奎斯特时失去直觉。适用：$\omega_c T_s > 0.3$、又不想改 T_s 的场合。

路径三：混合。 用连续设计定初值（K_p、T_i 的量级），用离散模型验裕度，差值超过 10° 就回到连续域调，再验。**这是工程上最常用的路径**：它兼顾了直觉与准确，代价是两轮计算量。

三条路径的收敛点是一样的：**最终都要用离散模型验一遍**。连续模型给的是起点，不是答案。

85.4.8 这把尺子怎么用：三处刻度与一个减法

连续基线不是拿来"参考"的，是拿来**做减法**的。本章统一按三处刻度对齐：交界频率、相位裕度、超调。三处都对上，说明离散结果可信；某一处偏得多，就说明结构里还有没算进去的环节。

刻度一，交界频率。 连续 3.746 rad/s，离散 3.797 rad/s，前者低 1.4%。方向不直观——采样明明带来滞后，交界频率为什么反而升？原因是离散 PI 的积分项在后向欧拉下变成 $T_s z/(z-1)$，在低频段等效增益比 $1/s$ 略大（相差 $T_s\omega/2$ 量级），把低频段抬了一点，交界点因此右移。**这个偏移与采样税方向相反，量级很小，不要拿它当"采样有益"的证据**。

刻度二，相位裕度。 连续 69.5°，离散 63.2°，差 6.3°。理论账是半拍 2.17°（ZOH 等效）+ 整拍 4.35°（$m=1$ 计算延迟）$= 6.52^\circ$。实测比理论少 0.2°，这 0.2° 来自两点：一是 ZOH 的 sinc 在 ω_c 处幅值只掉 0.002 dB，几乎不贡献相位；二是后向欧拉的相位滞后略小于 $T_s\omega/2$。**误差 0.2° 说明"半拍 + 整拍"这个口算模型在 $\omega_c T_s < 0.1$ 时足够准。**

刻度三，超调。 连续 1.8%，离散 4.8%，涨了 3 个百分点。这一处最容易被误读：PM 只掉 6.3°，超调却涨了近三倍。原因是超调对相位裕度的敏感度在 60° 附近陡然上升（§85.6.7 有二阶估算与实测的对照），且 PI 对消不完全时闭环含一个近零点，零点会把超调再抬一点。**结论：不要用"PM 掉得不多"来安慰自己超调不会变。**

减法怎么用。 把实测减连续基线，得到三个差值：$+0.051$ rad/s、-6.3°、$+3.0$ 个百分点。然后用结构参数预测这三个差值：交界频率的偏移应小于 $\omega_c T_s/2$ 对应量级；相位差应约等于半拍加整拍；超调差应由 PM 差换算后再放大 1.5 到 2 倍。**三处预测都对得上，才敢说"这个离散结果我已经解释完了"。**若相位差比半拍加整拍大出 2° 以上，就该查：是不是 m 数少了、是不是有未建模的滤波、是不是量化在起作用。

还有一处不能直接减：**IAE**。连续仿真的步长必须取得很小（本章取 100 μs，是 T_s 的 $1/200$），否则数值积分误差会盖过结构差异；而离散仿真的 IAE 是每拍更新一次的，两者口径不同。**跨口径比较无意义，同口径比才有意义**——这也是本章表 85-7 只做离散内部横向对比、不列"连续 IAE"的原因。

85.5 离散情形

85.5.1 采样把频带切成了无穷多段

图 85-2 画的是同一件事的两种情形：左栏采样率足够，基带与第一个副本之间有间隔；右栏采样率不足，副本压到基带上，交界处出现重叠。重叠区里的频率成分，采样后分不清谁是谁。

图 85-2 采样频谱：左栏 $f_s > 2f_{\max}$ 时副本分离，右栏不满足时副本重叠、高频折回基带

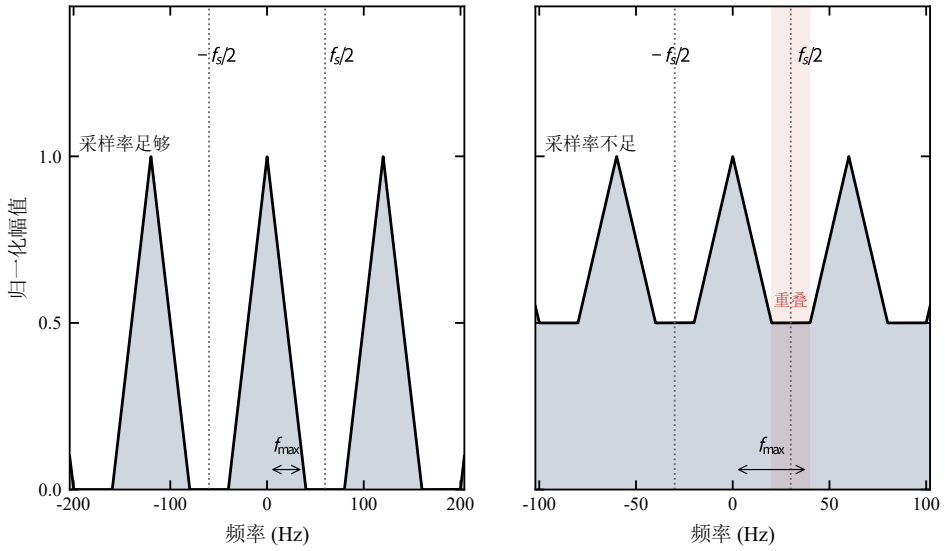


图 85-2

采样后的闭环频响不再是 $L(j\omega)$，而是

$$L^{*}(j\omega) = L(j\omega) + \sum_{k\neq 0} L\big(j(\omega - k\omega_s)\big) \tag{85-19}$$

式 (85-19) 里的求和项就是“别名的贡献”。它在 ω 低时很小（因为 L 在高频已经衰减），但在交界频率附近不能忽略。**这就是为什么采样系统的裕度不能直接用连续 $L(j\omega)$ 算**：式 (85-19) 的求和项会额外贡献相位。

85.5.2 零阶保持的幅相表

式 (85-5) 可以做成一张速查表，见表 85-6。表里两列分别是半拍相位与 sinc 衰减，它们决定“这个采样率下保持器要不要认真处理”。

表 85-6 零阶保持的幅相随 ωT_s 的变化

ωT_s	半拍相位	sinc 衰减	幅值剩余	本章对应
0.05	1.43°	0.99990	-0.001 dB	$T_s = 13$ ms
0.10	2.86°	0.99959	-0.004 dB	$T_s = 26$ ms
0.20	5.73°	0.99836	-0.014 dB	$T_s = 53$ ms
0.30	8.59°	0.99626	-0.033 dB	$T_s = 79$ ms
0.50	14.32°	0.98962	-0.091 dB	$T_s = 132$ ms
1.00	28.65°	0.95885	-0.365 dB	$T_s = 263$ ms
π	90.0°	0.6366	-3.92 dB	$T_s = 827$ ms

读法：**相位一列随时可用，幅值一列只在 $\omega T_s > 0.5$ 后才值得看**。本章 $\omega_c T_s = 0.076$，落在表的第二行，半拍 2.2°，幅值掉 0.002 dB——忽略幅值、只算相位，是安全的。反过来，如果某个设计 $\omega_c T_s = 1.0$，半拍 28.65°、幅值掉 0.365 dB，两项都必须进账。

图 85-3 把这张表画成曲线：上子图是幅值（sinc）与相位（直线），下子图是两者的相对重要性。可以看出，相位在 $\omega T_s = 0.3$ 之前就已经是“度数级”，幅值要到 1.0 之后才是“百分比级”。

图 85-3 零阶保持的幅频与相频：幅值按 sinc 缓慢下降，相位按 $\omega T_s/2$ 线性下降

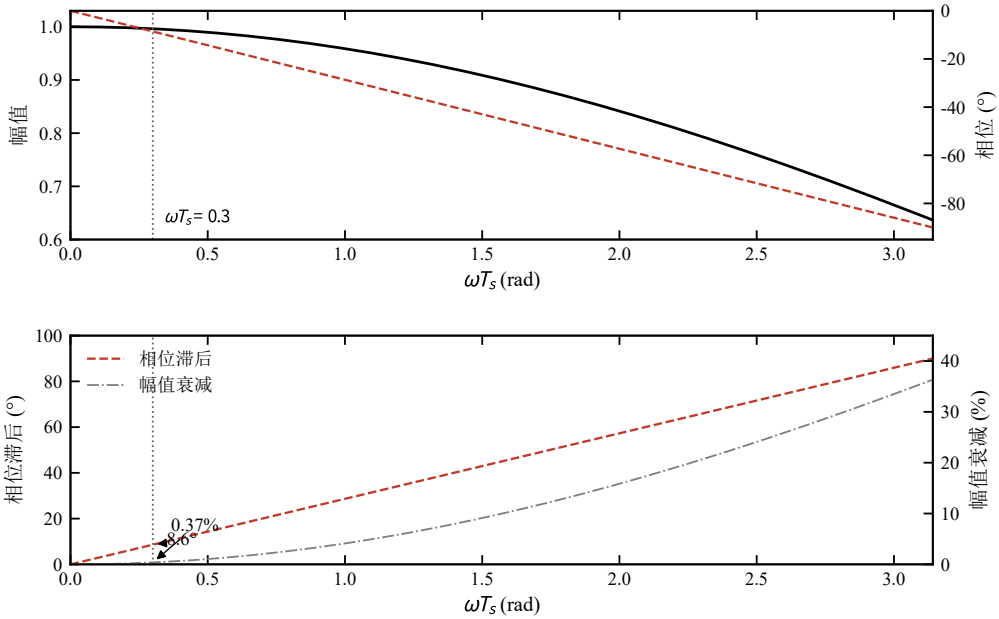


图 85-3

85.5.3 精确离散与近似离散的实测差别

表 85-7 是采样周期全扫描，连续参数一个字不改。

表 85-7 采样周期扫描（$K_p = 2.0$、$T_i = 0.5$ s、$m = 1$）

T_s	$\omega_c T_s$	PM	GM	IAE	超调
1 ms	0.004	69.2°	44.5 dB	0.2632	1.8%
2 ms	0.008	68.8°	38.5 dB	0.2634	1.9%
5 ms	0.019	67.9°	30.5 dB	0.2642	2.3%
10 ms	0.038	66.3°	24.6 dB	0.2662	3.0%
20 ms	0.076	63.2°	18.7 dB	0.2734	4.8%
50 ms	0.194	53.4°	11.2 dB	0.3230	14.1%
100 ms	0.398	36.5°	5.6 dB	0.5484	38.8%
200 ms	0.834	0.9°	0.1 dB	6.0310	100.9%

这张表有三个读法。

第一，看 $\omega_c T_s$ 这一列，不看 T_s 这一列。 判据 $\omega_c T_s \le 0.3$ 落在 20 ms 与 50 ms 之间；表里 20 ms 裕度 63.2° 还很宽，50 ms 掉到 53.4°，100 ms 掉到 36.5°。**判据是“开始变差”的预警线，不是“刚好能用”的临界线。**

第二，IAE 在 50 ms 之前几乎不动。 从 1 ms 到 20 ms，IAE 从 0.2632 涨到 0.2734，只差 3.9%；从 20 ms 到 100 ms，涨到 0.5484，翻倍；到 200 ms，涨到 6.0310，二十三倍。**IAE 是“悬崖型”指标，前面平坦，后面陡降。**

第三，超调比 IAE 更早报警。 20 ms 时超调 4.8%，50 ms 已经 14.1%，而 IAE 才涨 18%。原因是超调由阻尼比（也就是 PM）决定，PM 从 63° 掉到 53°，阻尼比按 $PM/100$ 估，从 0.63 掉到 0.53，超调自然翻倍。**现场先看超调，再看 IAE**，因为超调是“看得见”的。

85.5.4 三种离散化：什么时候才有区别

表 85-8 是三种积分规则在两个采样周期下的对照。

表 85-8 三种离散化方法的实测对照

T_s	方法	PM	GM	IAE	超调
20 ms	后向欧拉	63.2°	18.7 dB	0.2734	4.8%
20 ms	前向欧拉	62.9°	19.0 dB	0.2816	5.3%
20 ms	双线性	63.0°	18.9 dB	0.2770	5.0%
100 ms	后向欧拉	36.5°	5.6 dB	0.5484	38.8%
100 ms	前向欧拉	38.6°	6.8 dB	0.5457	35.5%
100 ms	双线性	37.9°	6.2 dB	0.5401	36.9%

20 ms 这一档，三者裕度差 0.3°、IAE 差 3%，**随便挑**；100 ms 这一档，前向欧拉的裕度反而最高（38.6°），比后向欧拉高 2.1°。这个结果看起来违反直觉——前向欧拉“不稳定”，怎么反而更好？原因是前向欧拉在 $z=1$ 附近的相位超前一点，部分抵消了采样滞后；**但它同时把稳定域推到单位圆外，模型稍有偏差就会失稳**。所以工程上仍然选后向欧拉：不是因为它在这张表里最好，而是因为它最不容易在别的条件下变坏。

图 85-4 是 $T_s=100$ ms 时三种方法的阶跃响应。三条曲线在前 0.5 s 几乎重合，1 s 之后分开，超调差三个百分点。**离散化方法的影响是“二阶效应”，采样周期才是“一阶效应”**，这张图就是这句话的图示。

图 85-4 三种离散化在 $T_s=100$ ms 下的阶跃响应：曲线在前段重合，后段分开

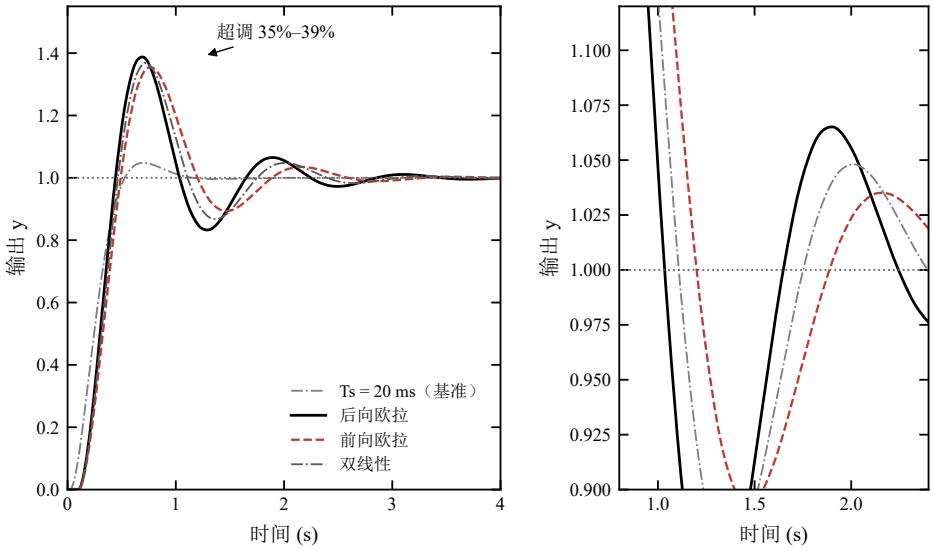


图 85-4

85.5.5 采样周期扫描图

图 85-5 把表 85-7 画成双纵轴：左轴相位裕度，右轴 IAE，横轴对数坐标。

图 85-5 采样周期扫描：PM 单调下降，IAE 在 50 ms 之后陡升

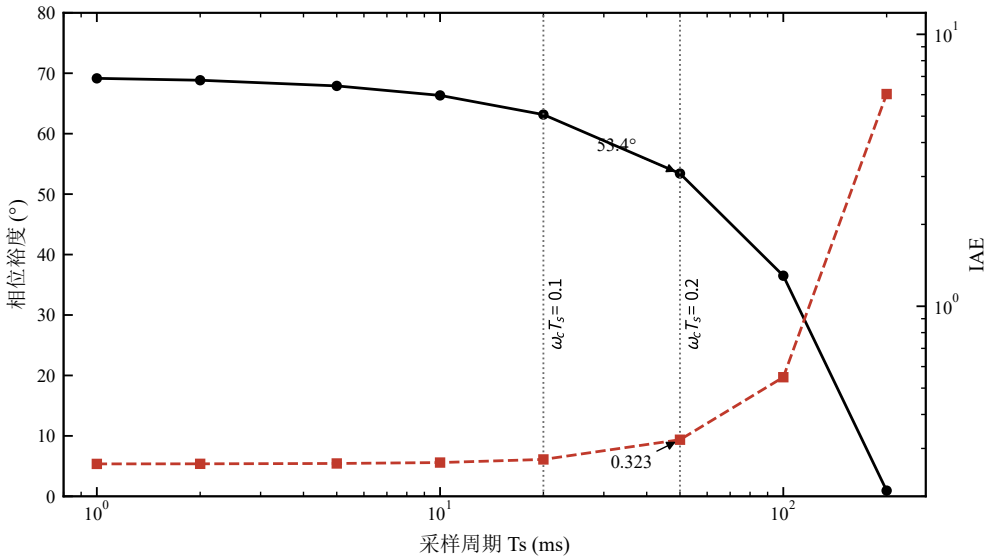


图 85-5

图上有两个拐点值得记住：$\omega_c T_s = 0.1$ 处 PM 开始明显下滑（$66.3^\circ \to 63.2^\circ$），$\omega_c T_s = 0.2$ 处 IAE 开始抬头（$0.2734 \to 0.3230$）。**两个拐点对应两条不同的判据**：前者是“裕度还够不够”，后者是“性能还好不好”。工程上一般卡在前者，因为裕度不够会失稳，性能不好只是难受。

85.5.6 双线性变换的频率预畸变

双线性变换把 s 平面映到 z 平面时，频率轴被非线性压缩：

$$\omega_a = \frac{2}{T_s}\tan\frac{\omega T_s}{2} \tag{85-20}$$

要在这个频率上得到与连续设计相同的响应，设计时必须把目标频率先“预畸变”成 ω_a。本章 $\omega_c T_s = 0.076$，$\tan(0.038)/0.038 = 1.0005$，畸变 0.05%，可以不管；$\omega_c T_s = 1.0$ 时，$\tan(0.5)/0.5 = 1.0926$，畸变 9.3%，就必须预畸变了。**这条与 §85.5.2 的 sinc 表是同一件事的两个侧面：都在说“ωT_s 小的时候什么都可以忽略”。**

85.5.7 闭环里的混叠比开环更麻烦

开环混叠只是“频谱多了一块”，闭环混叠的麻烦在于**反馈会把折回来的成分当成真信号去追**。用表 85-13 的第三行算一笔账：$f_s = 50$ Hz 时，70 Hz 干扰折到 20 Hz，幅度比 0.588（相对 8 Hz 有用信号）。

这个 20 Hz 的成分落在闭环的什么位置？本章回路的带宽只有 0.6 Hz（$\omega_c = 3.797$ rad/s），20 Hz 在带宽之外 33 倍处。开环增益在这一点的幅值是

$$|L(j125.7)| = \frac{40}{125.7\sqrt{125.7^2 + 100}} \approx 2.0 \times 10^{-3}$$

灵敏度函数 $|S| = 1/|1+L| \approx 1$。**这意味着反馈对这个频率完全不起作用**：折回来的 20 Hz 成分原封不动地出现在输出上，幅度是 0.588 倍的有用信号——比本章任何一档纹波（10^{-4} 量级）都大三个数量级。

这条算账给出一个反直觉的结论：**混叠的危害与“折到哪”关系极大**。折到带宽以内（比如折到 0.5 Hz），闭环会把它当成真实误差去抑制，虽然会引起振荡，但至少反馈还在起作用；折到带宽以外（20 Hz），反馈彻底失效，输出上直接多出一个大幅度正弦。**后者才是现场“信号看着正常、执行器却在抖”的典型来源。**

也正因为如此，抗混叠滤波器必须放在采样之前，而不能指望“控制器里加个陷波”——陷波是数字的，它在采样之后，对已经折进基带的成分无能为力。

85.5.8 离散频响的三个读法

离散系统的频率响应只在 $z = e^{j\omega T_s}$ 上定义，z 沿单位圆走一圈，ω 从 0 走到 $\omega_s = 2\pi/T_s$。读图时记住三条。

第一，只读 0 到 π/T_s（奈奎斯特）。 再往上是镜像，幅值对称、相位反号，没有新信息。本章 $\pi/T_s = 157.1$ rad/s，而 $\omega_c = 3.8$ rad/s 只占全程的 2.4%——**常规设计的工作点都挤在图的左下角**，这也是为什么 $\omega_c T_s$ 小时连续模型够用。

第二，幅频曲线是周期延拓的。 $|L(e^{j\omega T_s})|$ 以 ω_s 为周期重复，重复的副本就是式 (85-19) 的求和项。画图时若不截断，会看到“每 314 rad/s 重复一次”的锯齿，那不是数值噪声，是采样本身的性质。

第三，裕度要在离散曲线上读，不能在连续曲线上读。 本章连续 $PM = 69.5^\circ$、离散 63.2°，差 6.3°；如果误用连续曲线，会高估裕度，把 T_s 选大一档。**读裕度时先确认图的横轴是不是 ω（rad/s）、曲线是不是由 $z = e^{j\omega T_s}$ 算出来的**，这两点确认了，数字才对。

85.5.9 三个数量级的采样周期

把本章的 $T_s = 20$ ms 放到工业现场看，它属于“过程控制”这一档。三档的典型值是：

- **慢过程**（温度、液位、成分）：$T_s = 1$ s 量级，ω_c 约 0.05–0.5 rad/s，$\omega_c T_s \le 0.1$；
- **快过程**（流量、压力、普通液位）：$T_s = 10$–100 ms，ω_c 约 5–60 rad/s，本章 20 ms、3.8 rad/s 就在这一档的下沿；
- **伺服**（速度环、电流环、位置环）：$T_s = 0.05$–1 ms，ω_c 约 600–6000 rad/s。

三档之间差三个数量级，但判据是同一条：$\omega_c T_s \le 0.3$。表 85-14 就是这条判据的另一种写法——**它把“该取多快”翻译成“半拍吃掉多少度”，让不同带宽的回路可以放在一张表里比。**

跨档设计时要小心一件事：**同一个系统里混着三档，慢回路的采样点会被快回路“看见”，但快回路的动作慢回路看不见。**这就是 §85.8.2 多速率问题的来源，也是 §85.9.4 要求“周期之间要有确定相位关系”的原因。

85.5.10 纸上画离散频响：三点加三条修正

现场没有 MATLAB 的时候，本章这套参数可以在纸上估出来。办法是先用连续 $L(s) = 40/(s(s+10))$ 画三点，再叠三条修正。

三点作图。 第一点，低频渐近线：$\omega \ll 10$ 时 $|L| \approx 40/(10\omega) = 4/\omega$，它穿过 0 dB 的位置是 $\omega = 4$ rad/s。第二点，拐点：$\omega = 10$ 处斜率从 -20 dB/dec 折成 -40 dB/dec。第三点，交界：解 $\omega^2(\omega^2 + 100) = 1600$，得 $\omega = 3.746$ rad/s，与渐近线估的 4 只差 6%——**这说明在拐点以下，积分段近似就够用了。**

相位同理：$\angle L = -90^\circ - \arctan(\omega/10)$，在 $\omega = 3.746$ 处是 -110.5°，于是 $PM = 69.5^\circ$。奈奎斯特点（$\omega = 157.1$ rad/s）的幅值是 $40/(157.1 \times 157.4) = 1.6 \times 10^{-3}$，与 §85.5.7 算的 2.0×10^{-3} 同量级。**三点画完，整条曲线的骨架就有了，不需要逐点算。**

三条修正。 把连续曲线搬到离散域，只需在这三点上各叠一个因子：

- 幅值乘 $\mathrm{sinc}(\omega T_s/2)$，在 ω_c 处是 0.99976，可以当 1；在奈奎斯特处是 $2/\pi = 0.637$，**不能在 Nyquist 附近忽略**；
- 相位减 $\omega T_s/2$（ZOH 半拍），ω_c 处 2.17°，奈奎斯特处 90°；
- 相位再减 $m\omega T_s$（计算延迟），$m = 1$ 时 ω_c 处 4.35°，奈奎斯特处 180°。

三条修正叠完，ω_c 处相位从 -110.5° 变成 -117.0°，$PM = 63.0^\circ$，与实测 63.2° 差 0.2°。**这就是全部“离散化误差”的来源，没有第四个因子。**

最后给一条限制：**这套三点作图只在 $\omega_c T_s \le 0.3$ 时可信**。$\omega_c T_s$ 再大，sinc 的幅值衰减与 ZOH 相位不再是“小量修正”，交界点本身会移动，就得回 §85.5.3 的精确离散去算。**手算的价值在于“量级对了没有”，不在于“小数点后第几位”**。

85.6 稳定性与性能

85.6.1 离散稳定性的三条判据

离散系统的稳定性判据有三条，用途不同。

第一条是 Jury 判据，直接对特征多项式下结论。二阶多项式 $P(z) = z^2 + a_1 z + a_0$ 的三条不等式是

$$|a_0| < 1, \qquad 1 + a_1 + a_0 > 0, \qquad 1 - a_1 + a_0 > 0 \tag{85-21}$$

它的好处是不用求根，坏处是阶数高了条件变多（三阶要四条、四阶要七条）。本章的闭环含 PI 的状态与一拍延迟，特征多项式是三阶，实际代码里用求根更省事。

第二条是谱半径：

$$\rho = \max_i |z_i| < 1 \tag{85-22}$$

z_i 是式 (85-11) 的根。ρ 越接近 1，稳定裕度越小；$\rho = 1$ 临界，$\rho > 1$ 发散。本章基准 $\rho = 0.9620$。

第三条是频域判据：$PM > 0$、$GM > 1$。三条判据在理想模型下等价，但在模型有误差时不等价：**ρ 只看绝对稳定，不看裕度；PM 看相对稳定**。所以工程上以 PM 为主、ρ 为辅。

85.6.2 延迟拍数的实测影响

表 85-9 是延迟扫描，$T_s = 20$ ms 固定。

表 85-9 延迟拍数扫描（$T_s = 20$ ms）

m	附加滞后	PM	GM	IAE	超调	ρ
0	0 ms	67.5°	28.2 dB	0.2734	4.8%	0.9620
1	20 ms	63.2°	18.7 dB	0.2734	4.8%	0.9620
2	40 ms	58.8°	14.5 dB	0.2981	8.3%	0.9619
3	60 ms	54.5°	11.8 dB	0.3316	12.9%	0.9619
5	100 ms	45.8°	8.2 dB	0.4300	24.4%	0.9619
8	160 ms	32.7°	4.8 dB	0.7013	45.4%	0.9757
12	240 ms	15.3°	1.9 dB	1.8020	76.3%	0.9919

三个观察。

第一，每多一拍掉 4–5°。 从 $m = 0$ 到 $m = 5$，PM 从 67.5° 掉到 45.8°，平均每拍 4.3°。这个数与式 (85-5) 的半拍公式对得上：一拍 20 ms 对应 $\omega_c T_s = 0.076$ rad，即 4.35°。**延迟的相位代价是可加的，也是可预测的。**

第二，$m = 0$ 与 $m = 1$ 的 IAE 完全一样（0.2734）。因为 IAE 主要由上升段决定，而上升段延迟一拍对闭环响应形状影响很小；但 PM 已经从 67.5° 掉到 63.2°。**这说明“指标好看”与“裕度够用”是两件事**：IAE 一样的两个设计，抗扰能力可以差 4°。

第三，ρ 几乎不动。从 $m=0$ 到 $m=5$，ρ 只从 0.9620 变到 0.9619，直到 $m=8$ 才涨到 0.9757。谱半径对延迟极不敏感，因为主导极点始终是对象的慢极点 $e^{-2T_s}=0.9608$。这也提醒：**别拿 ρ 当唯一验收指标**。

85.6.3 稳定域：K_p 与 T_s 的边界

表 85-10 是不同采样周期下临界 K_p（用谱半径二分求得，判据 $\rho<1$）。

表 85-10 临界增益随采样周期的变化

T_s	临界 K_p	ρ（$K_p=2$）	ρ（$K_p=8$）
5 ms	67.35	0.9901	0.9901
10 ms	34.00	0.9805	0.9804
20 ms	17.31	0.9620	0.9616
50 ms	7.24	0.9110	1.0172
100 ms	3.81	0.8620	1.2254

这张表最有用的一列是“临界 K_p”。20 ms 时是 17.31，本章用的 $K_p=2.0$ 只占临界值的 11.6%，余量很大；100 ms 时临界值掉到 3.81，$K_p=2.0$ 占了 52.5%，余量紧张。**工程上一般要求工作点不超过临界增益的 $1/3$**，按这条，$T_s\le 50$ ms 都可以，100 ms 就超了。

ρ 两列还揭示一件事：$T_s\le 20$ ms 时，K_p 从 2 加到 8，ρ 几乎不变（0.9901 对 0.9901），因为慢极点占主导；$T_s\ge 50$ ms 时，$K_p=8$ 已经把 ρ 推到 1 以上（1.0172、1.2254），**同样的增益在不同的采样周期下，一个是稳的，一个是散的**。

图 85-6 把这张表画成曲线：横轴对数 T_s，纵轴临界 K_p，双对数坐标下近似一条直线，斜率约 -1。这条 -1 斜率的物理含义是 $K_p^{\text{crit}}\cdot T_s\approx$ 常数——**临界增益与采样周期成反比，采样快一倍，能用的增益也差不多能大一倍**。

图 85-6 稳定边界：临界 K_p 随 T_s 近似按 $1/T_s$ 下降，工作点应留在曲线的 $1/3$ 以下

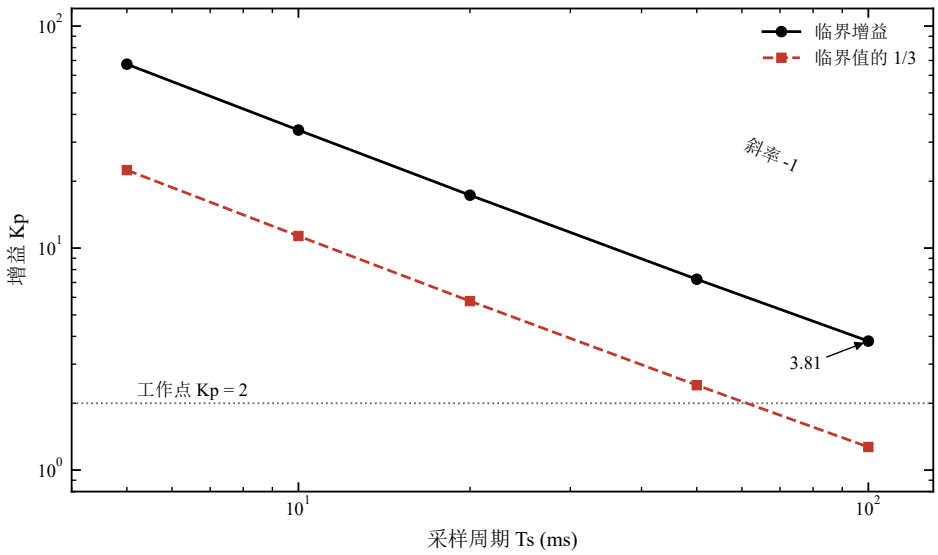


图 85-6

85.6.4 采样间纹波：采样点看不见的东西

采样控制有一个容易被忽略的现象：**采样点上很平静，采样点之间可能正在振荡**。原因是控制器只在采样时刻看输出，两次采样之间的行为完全由对象自己决定。

表 85-11 用细分 40 步的连续轨迹量这件事。

表 85-11 采样间纹波（细分 40 步，量末段峰峰值）

T_s	每 τ 采样数	纹波	超调	IAE
2 ms	250	1.88×10^{-9}	1.9%	0.2634
5 ms	100	4.83×10^{-9}	2.3%	0.2641
10 ms	50	1.01×10^{-8}	3.0%	0.2661
20 ms	25	2.19×10^{-8}	4.8%	0.2731
50 ms	10	6.78×10^{-8}	14.1%	0.3224
100 ms	5	3.49×10^{-5}	38.8%	0.5471

纹波在 50 ms 之前都在 10^{-7} 量级，100 ms 时跳到 10^{-5}，涨了 500 倍。这个跳变不是渐进的，而是当 $\omega_c T_s$ 接近 0.4 时，离散闭环的一对复极点开始向单位圆靠近，采样间的振荡被放大。图 85-8 是 $T_s=20$ ms 与 100 ms 两档的局部放大轨迹，前者几乎是一条光滑曲线，后者能看到明显的锯齿。

图 85-8 采样间纹波：$T_s=20$ ms 时轨迹光滑，100 ms 时采样点之间出现可见振荡

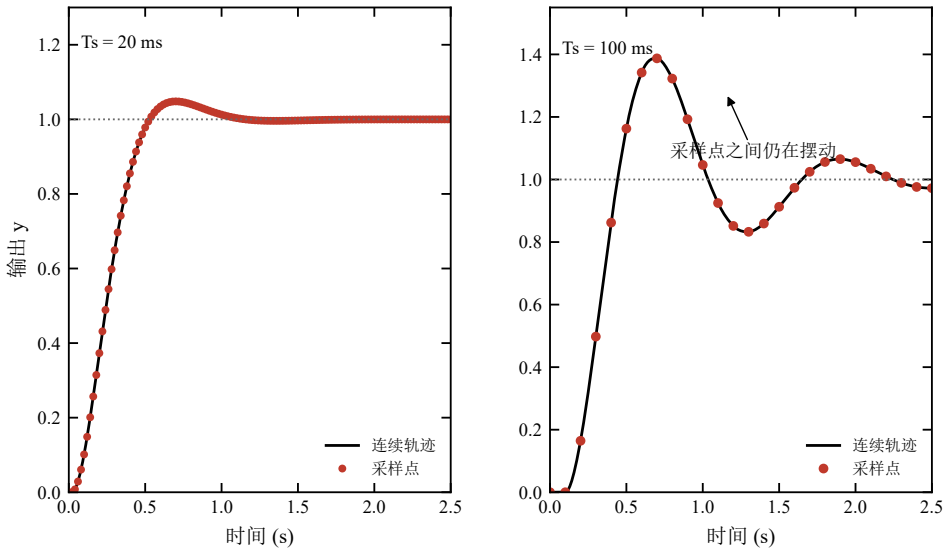


图 85-8

工程含义很直接：**只测采样点会漏掉纹波**。示波器要接在 D/A 之后或对象的连续输出上，采样周期要足够快（至少 $10\times f_s$），否则看到的是“一切正常”。本章 100 ms 那一档，采样点上看超调 38.8%，采样间峰峰值 3.49×10^{-5}——纹波相对超调很小，但它是**持续振荡**，不像超调那样会衰减。

85.6.5 纹波的估算式

采样间纹波的量级可以用一个粗估式：

$$y_{pp}\approx 2\,\Delta\,\left|S(e^{j\omega_r T_s})\right| \tag{85-23}$$

其中 Δ 是量化台阶，$S(z)=1/(1+C(z)G(z)z^{-m})$ 是灵敏度函数，ω_r 是纹波的主频（通常接近闭环带宽或采样频率的分数倍）。式 (85-23) 说明两件事：纹波与台阶成正比（位数每加一位，纹波减半），与灵敏度函数在纹波频率上的增益成正比（**裕度越小，纹波越大**）。

这条式子在 §85.7 会用到：表 85-12 里位数每加两位，纹波降四倍，与 2^{-N} 一致；但双侧量化时 14 位相对 12 位只降一半，因为此时限制纹波的是 DAC 台阶而不是 ADC 台阶。

85.6.6 Jury 判据在三阶上怎么用

式 (85-21) 只写了二阶。本章闭环是三阶：PI 有两个状态（比例与积分），加一拍延迟，特征多项式是 $P(z)=a_3z^3+a_2z^2+a_1z+a_0$。三阶 Jury 判据是四条：

1. $P(1)>0$；
2. $(-1)^3P(-1)>0$，即 $P(-1)<0$；
3. $|a_0|<a_3$；
4. $|b_0|>|b_2|$，其中 $b_0=a_0^2-a_3^2$、$b_2=a_0a_2-a_3a_1$。

前两条管实轴上的两个特殊点：$z=1$ 对应直流（积分器所在位置），$z=-1$ 对应奈奎斯特频率。$P(1)<0$ 意味着直流增益反号，系统一定有正实根；$P(-1)$ 的符号判断同理。后两条来自 Schur–Cohn 阵列，管的是复根是否出圆。

用起来要注意两点。**第一，系数必须先归一化到 $a_3>0$**，否则第 3 条不等号方向会反。**第二，第四条里的 b 是“降一阶”之后的系数**，它对应的是把 $P(z)$ 去掉首末项后构造的二阶多项式；阶数越高，这种降阶要做 $(n-1)$ 次，手工算极易出错。

所以工程上的做法是“两条腿走路”：**手算时用前三条快速排除**（$P(1)$、$P(-1)$、$|a_0|<a_3$），发现都过了再用 `np.roots` 求根确认。本章代码 `spectral_radius()` 走的就是求根这条路——三阶求根是闭式可解的，没有必要硬套 Jury 阵列。Jury 判据真正的用武之地是**嵌入式定点实现**：那里没有求根库，只有整数比较，四条不等式刚好能写成几行 C。

85.6.7 从相位裕度估超调

现场最常用的一条经验换算，是把 PM 折成阻尼比再折成超调：

$$\zeta \approx \frac{PM}{100},\qquad M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$$

拿表 85-7 的三行验一下。$T_s=50$ ms 时 $PM=53.4^\circ$，$\zeta=0.534$，算出 $M_p=14.2\%$，实测 14.1%——几乎完全吻合。$T_s=20$ ms 时 $PM=63.2^\circ$，算出 7.7%，实测 4.8%，高估 2.9 个百分点。$T_s=100$ ms 时 $PM=36.5^\circ$，算出 29.2%，实测 38.8%，低估 9.6 个百分点。

三行三样，说明这条经验式有明确的适用范围。

- PM 在 45°–65° 之间最准（误差 ±3 点），这一段闭环近似二阶，式子的前提成立；
- PM 高时高估：因为本章的相位损失主要来自纯延迟，而纯延迟不改变幅频的谐振峰，实际超调比二阶模型小；
- PM 低时低估：此时采样与保持的高频动态开始参与，响应已经不是二阶形状，实际超调比模型大。

工程用法：**用这条式子做快速估算，用仿真做最终确认**。两者差 3 个点以内可以接受，差 10 个点说明回路已经不在二阶近似范围内，此时该看的是时域波形，而不是裕度数字。

85.6.8 参数摄动下的三条安全线

设计值是在标称模型上算的，现场一定偏离。三条安全线用来吸收偏离。

第一条：工作增益不超过临界增益的 $1/3$。 本章 20 ms 时 $K_p^{\mathrm{crit}}=17.31$，$1/3$ 是 5.77，实际用 2.0，只占 11.6%——余量充足。若把 T_s 放到 100 ms，临界值 3.81，$1/3$ 是 1.27，实际 2.0 已经超线，**这时候不是“还能跑”，而是“随时可能因为对象增益 $+30\%$ 而失稳”**。

第二条：延迟留一拍余量。 设计时按实测 m 算裕度，验收时按 $m+1$ 再算一遍，要求仍 $\geq 45^\circ$。本章 $m=1$ 时 63.2°，$m=2$ 时 58.8°，都过；一直加到 $m=5$ 才掉到 45.8°，说明这个回路对延迟相当宽容。**宽容来自带宽低，不是来自控制器好。**

第三条：采样比留一倍余量。 判据是 $\omega_c T_s \leq 0.3$，设计时按 ≤ 0.15 选，实际运行中即使晶振漂移、负载加重导致周期抖动，也不会越过 0.3。本章 0.076，正好是一倍余量。

三条线合起来是一句话：**设计点要落在“三条线都还有一半余量”的位置，而不是“刚好压线”。**压线的设计在仿真里看不出问题，在现场会因为一个没测准的时间常数而崩。

85.6.9 18.7 dB 这个数是怎么来的

表 85-7 里 20 ms 那一行的增益裕度是 18.7 dB，表 85-6 里同条件的临界增益是 17.31。两个数其实是一个数的两种说法，值得手算一遍确认。

增益裕度的定义是“相位到 -180° 时幅值的倒数”。先找相位交汇频率 ω_π。连续 $L(s)=40/(s(s+10))$ 只有 -90° 与 $\arctan$ 两项，相位永远到不了 -180°——**所以纯连续回路根本没有增益裕度可言，它会一直稳定到无穷大增益。**真正把相位推过 -180° 的是延迟：半拍 $0.5T_s$ 加 $m=1$ 拍，共 $1.5T_s = 30$ ms。于是

$$-90^\circ - \arctan\frac{\omega}{10} - 0.03\omega = -180^\circ$$

即 $\arctan(\omega/10) + 0.03\omega = 1.5708$。试 $\omega=17$：$1.039+0.510=1.549$，偏小；试 17.5：$1.052+0.525=1.577$，偏大；插值得 $\omega_\pi \approx 17.3$ rad/s（2.75 Hz，约 $f_s/18$）。此处

$$|L(j17.3)| = \frac{40}{17.3\sqrt{17.3^2+100}} = \frac{40}{17.3\times 19.98} = 0.116$$

取倒数得 8.6 倍，换成 dB 是 $20\lg 8.6 = 18.7$ dB。**与实测完全一致。**

再由它反算临界增益：$K_p^{\text{crit}} = K_p \times 8.6 = 2.0\times 8.6 = 17.2$，与表 85-6 的 17.31 差 0.6%。差的这一点来自两处：一是实际离散相位是 z^{-1} 的精确相移而非 0.03ω 的线性近似，二是后向欧拉积分器的相位略小于半拍。**0.6% 的误差说明“延迟线性化”这个近似足够好，可以直接用来做手算。**

这笔账还给出一条重要的工程判断：**这个回路的增益裕度完全由延迟产生，与对象本身无关。**把 m 从 1 加到 2，相位交汇点左移，$|L(j\omega_\pi)|$ 变大，增益裕度立刻掉；而把对象增益改小，交界频率左移，增益裕度反而变大。**所以“增益裕度不够”时，第一件事是查延迟，不是减增益**——减增益只是把交界点往低频挪，用性能换稳定，问题还在。

最后一个提醒：**增益裕度只管增益，不管延迟。**18.7 dB 的意思是“增益可以涨 8.6 倍才失稳”，它对“再多一拍延迟”没有任何保证。本章另有一条判据（§85.6.8 第二条）专门管延迟：按 $m+1$ 重算，要求仍 $\geq 45^\circ$。**两个裕度管两件事，不能互相替代。**

85.6.10 从极点位置直接读响应形状

稳定性只问“在不在圆内”，性能要问“离圆心多远、在什么角度”。z 平面上两条等值线就能读出响应形状。

第一条，等模圆 $|z|=\rho$。 模决定“每拍衰减多少”：一拍之后幅值乘 ρ。本章闭环谱半径 0.9620，一拍衰减 3.8%；快极点 0.8187，一拍衰减 18.1%。粗略的读法是：$\rho > 0.95$ 的极点要几十拍才衰减完（本章 0.962，30 拍后剩 31%），适合当主导极点；$\rho < 0.5$ 的极点三四拍就没了，可以当“看不见的快极点”。**“主导”不是靠极点排序选的，是靠模的大小区分的。**

第二条，等角线 $\angle z = \theta$。 角度决定“振荡周期有几拍”：一拍转 θ，转一圈需要 $2\pi/\theta$ 拍。本章若按 $\zeta=0.7$、$\omega_n=4$ 那对极点算，$\theta = 3.27^\circ$，一圈 110 拍，$T_s=20$ ms 即 2.2 s——**这正好是那对极点的阻尼振荡周期 $2\pi/\omega_d = 2.2$ s，**两条路算出来一样，可以互相校验。

两条线合起来，就是离散域读极点的全部规则：**模看衰减快慢，角看振荡周期，两者与连续域的 $\zeta\omega_n$ 和 ω_d 通过 $z = e^{sT_s}$ 一一对应**。§85.2.10 是从连续推离散，这里是反着读，方向不同、结论相同。

还有一处要提醒：**极点位置说的是"采样点上的形状"，不是连续轨迹的形状**。采样间可能藏着更高的振荡（§85.6.4），它不出现在极点里，因为极点只描述采样点之间的关系。**这就是为什么"极点都在圆内"不等于"输出没有高频抖动"**——两件事要分别验，前者看 ρ，后者看连续输出（表 85-11）。

85.7 参数意义与作用

结构里有五个参数：T_s、m、N_{adc}、N_{dac}、量程 R。这一节逐个说清楚"它管什么、改它会发生什么"。

85.7.1 T_s：管相位，不管精度

T_s 直接进入半拍相位 $\omega_c T_s/2$ 与延迟拍数 m 的换算。它不进入量化台阶，也不进入对象模型（精确 ZOH 离散后对象只是换了个形式）。所以：

- T_s 变小，裕度变大、可用的 K_p 变大、IAE 略降；
- T_s 变大，裕度变小、超调变大、稳定性变差。

表 85-7 已经给全。要补充的是**收益递减**：T_s 从 20 ms 压到 1 ms，PM 只多 6°，而计算量涨 20 倍。工程上的选择点一般在 $\omega_c T_s = 0.05$–0.1，对应本章 13–26 ms。

85.7.2 m：管相位，且是"整拍"的

m 只在式 (85-11) 里以 z^{-m} 出现，所以它是纯延迟，只吃相位。m 的麻烦在于它是整数：m 从 1 变到 2 是 20 ms 的跳变，不是渐变。表 85-9 里 PM 从 63.2° 掉到 58.8°，一跳 4.4°。

这个"整数跳变"带来一个反直觉的结论：**把计算时间从 19 ms 优化到 1 ms，如果 $T_s = 20$ ms，m 仍然是 1，裕度一点不变**。要拿到收益，必须把计算时间压到 0 或把 T_s 提高到 40 ms 以上。**优化计算时间的收益是阶梯状的，不是连续的**。

85.7.3 N_{adc}：管测量分辨率

ADC 位数决定台阶 $\Delta_{adc} = R/2^N$，直接进入式 (85-23)。表 85-12 的 "ADC 纹波" 一列就是它的效果。

表 85-12 位数扫描（$T_s = 20$ ms，$R = 1.5$，$U_{\max} = 5$）

位数	ADC 台阶	DAC 台阶	仅 ADC 纹波	仅 DAC 纹波	双侧纹波	双侧 IAE
4	9.38×10^{-2}	6.25×10^{-1}	1.60×10^{-2}	2.47×10^{-2}	3.70×10^{-2}	0.3756
6	2.34×10^{-2}	1.56×10^{-1}	4.00×10^{-3}	6.17×10^{-3}	1.88×10^{-2}	0.3212
8	5.86×10^{-3}	3.91×10^{-2}	9.99×10^{-4}	1.06×10^{-3}	2.32×10^{-3}	0.2791
10	1.46×10^{-3}	9.77×10^{-3}	2.50×10^{-4}	3.86×10^{-4}	1.18×10^{-3}	0.2759
12	3.66×10^{-4}	2.44×10^{-3}	6.24×10^{-5}	9.65×10^{-5}	1.45×10^{-4}	0.2737
14	9.16×10^{-5}	6.10×10^{-4}	1.56×10^{-5}	2.41×10^{-5}	7.35×10^{-5}	0.2735

读这张表要注意三件事。

第一，DAC 台阶比 ADC 台阶大 6.7 倍。 原因在量程：ADC 满量程 1.5（± 0.75），DAC 满量程 10（± 5），比值正好 6.67。同样的位数，DAC 的一个台阶代表的物理量大得多。所以**同样的位数，DAC 更"粗"**。

第二，纹波近似按 2^{-N} 下降。 12 位到 14 位，ADC 纹波 $6.24\times10^{-5}\to1.56\times10^{-5}$，正好四倍。这条与式 (85-23) 一致。

第三，**IAE 在 10 位以后几乎不动。** 10 位 0.2759、12 位 0.2737、14 位 0.2735，差不到 0.9%。**这说明“精度”这件事在 10 位左右就到顶了**，再往上加位数，改善的是纹波而不是跟踪性能。

图 85-7 把三类量化位置分开画：只量化 ADC、只量化 DAC、两侧都量化。三条线在 8 位之后都变平，只有双侧那条一直高一个数量级——**两处量化是叠加的，不是取最差的那个。**

图 85-7 量化位置对纹波的影响：只量化 ADC、只量化 DAC、双侧量化的纹波随位数下降

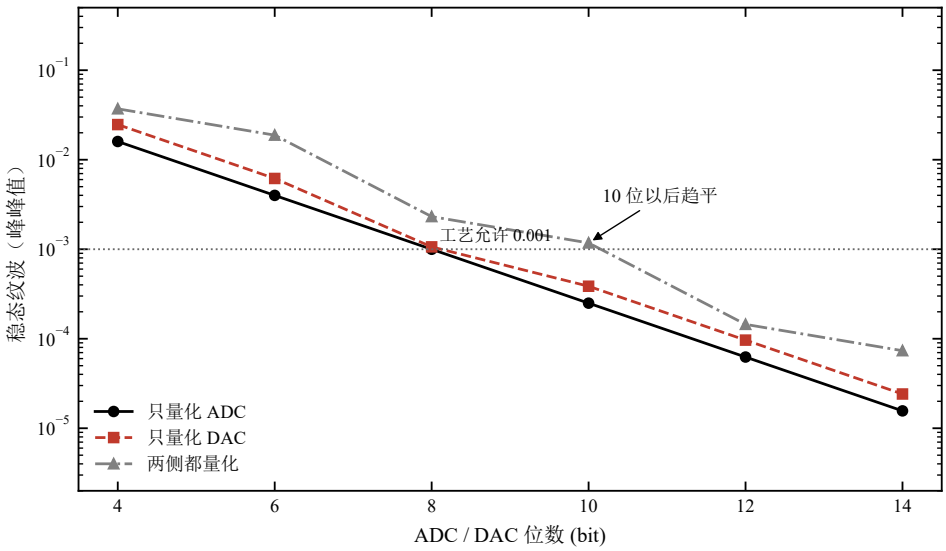


图 85-7

85.7.4 N_{dac}：管输出分辨率，容易被低估

表 85-12 里“仅 DAC 纹波”在 8 位时是 1.06×10^{-3}，比“仅 ADC 纹波”9.99×10^{-4} 还大一点。这个结果值得注意：**很多人只关心 ADC，忽略 DAC，实际上两者的贡献是同量级的。**

原因还是量程。若把 DAC 满量程从 10 缩到 2（执行器只需要这么大），8 位 DAC 的台阶就从 3.91×10^{-2} 降到 7.81×10^{-3}，纹波跟着降五倍。**缩量程比加位数便宜**，这是现场最实用的一条。

85.7.5 量程 R：最容易被浪费的参数

量程对台阶的影响是线性的：

$$\Delta=\frac{R}{2^{N}}\quad\Longrightarrow\quad\frac{\Delta_2}{\Delta_1}=\frac{R_2}{R_1}\tag{85-24}$$

量程放大一倍，等于丢一位。**现场最常见的浪费是“按传感器满量程设置，而不是按实际工作范围设置”**：一个 ±100 kPa 的压力变送器，实际工作在 40–60 kPa，却把 ADC 量程设成 ±100，有效位数直接丢了一半。

正确的做法是：按实际工作范围留 20%–50% 余量，再乘上安全系数。本章 $R=1.5$ 对应被控量在 $[0,1]$ 附近波动，余量 50%。

85.7.6 有效位数：标称位数往往拿不到

ADC 手册上的“12 位”是分辨率，不是精度。真正能用的是有效位数：

$$\mathrm{ENOB} = \frac{\mathrm{SINAD} - 1.76}{6.02} \tag{85-25}$$

一颗 12 位 ADC 的 SINAD 若是 70 dB，ENOB 只有 11.3 位；若是 60 dB，ENOB 只有 9.7 位。**噪声、非线性、参考源漂移都会把标称位数吃掉**。所以式 (85-17) 算出来的位数要再加 1–2 位余量，或者直接按 ENOB 选型。

85.7.7 保持器类型：默认零阶，别乱换

保持器有三种：零阶（ZOH）、一阶（FOH）、零阶加滤波。工程上 99% 用零阶，因为 D/A 芯片本身就是零阶。一阶保持相位滞后更小（约 $\omega T_s/2$ 的一半），但需要外推，噪声会被放大，**只在极少数高速场合值得用**。本章不展开。

85.7.8 五个参数的敏感度排序

按“改一个单位对 PM 的影响”排序：

1. T_s **最敏感**：翻倍就从 63.2° 掉到 36.5°；
2. m **次之**：每加一拍掉 4.3°；
3. N_{dac} **第三**：8 位到 12 位，纹波降 11 倍，但 PM 几乎不动；
4. N_{adc} **第四**：与 DAC 同量级，但因为量程小，实际影响略小；
5. **量程** R **第五**：它通过位数间接起作用，改了它等于改了位数。

这张排序表是现场排查的入口：回路不稳，先查 T_s 与 m；回路有纹波，先查 DAC 量程与位数；回路“精度不够”，先查量程是不是设大了。

85.7.9 把台阶换算成工程单位

位数与量程是“数字侧”的参数，现场工程师关心的是“物理侧”的一个数：**一个台阶代表多少工程单位**。换算只有一步：

$$\mathrm{LSB} = \frac{\text{满量程}}{2^N}\ (\text{工程单位})$$

三个例子，都用 12 位：

- **压力**：变送器 0–100 kPa，$\mathrm{LSB} = 100/4096 = 0.0244$ kPa。若实际工作在 40–60 kPa，这个 LSB 占工作区间的 0.12%，够用。
- **温度**：0–200 ℃ 的铂电阻，$\mathrm{LSB} = 0.0488$ ℃。对 60 s 的慢回路，这个分辨率远超需要（温度波动本身就有 0.1 ℃）。
- **流量**：0–100 m³/h，$\mathrm{LSB} = 0.0244$ m³/h。若控制要求是 ± 0.5 m³/h，分辨率占 5%，偏粗——**这时候该做的不是加位数，而是缩量程到 0–50 m³/h**，一步就把 LSB 减半。

三个例子的满量程差不多，LSB 却差一倍，原因全在“实际工作区间占满量程的比例”。**这就是 §85.7.5 说“量程是最容易被浪费的参数”的定量版本**：浪费的不是位数，是“每一位对应的物理量”。

换算还有一个反向用法：**从允许波动反推量程**。若工艺允许波动是 ± 0.2 ℃，12 位 ADC 的 LSB 应小于 $0.2/3 = 0.067$ ℃，对应满量程 $< 0.067 \times 4096 = 274$ ℃。现场用的是 0–200 ℃ 量程，正好满足。**先算这个数，再去选变送器，比先买变送器再算要省事得多**。

85.7.10 五个参数之间的三处耦合

五个参数不是独立的，三处耦合决定了“不能一个一个调”。

耦合一：T_s 与 m 是乘法关系。 真正进入相位的是 mT_s，不是单独的 m 或 T_s。表 85-9 里 $m = 5$、$T_s = 20$ ms 的附加滞后是 100 ms，相位 21.7°；同样的 100 ms 若由 $m = 1$、$T_s = 100$ ms 实现，相位一样，但 PM 不同——因

为半拍也变了。**两者必须一起算，不能分开记账。**

耦合二：R 与 N 是除法关系。 $\Delta = R/2^N$，量程翻倍与位数减一等效。这条耦合的价值在于：**当位数加不上去时，缩量程是等效手段**；反过来，当量程不能缩（比如要覆盖全工况），就只能加位数。两者是同一个旋钮的两种拧法。

耦合三：ADC 与 DAC 是叠加关系，不是取最差。 表 85-12 里 8 位时，仅 ADC 纹波 9.99×10^{-4}、仅 DAC 纹波 1.06×10^{-3}，两者相近；双侧同时量化，纹波 2.32×10^{-3}，**比两者线性相加的 2.06×10^{-3} 还大一点**。原因是峰峰值口径下，两处台阶可能在同一时刻同向，误差直接相加；闭环的附加相位还可能让两处在某些时刻叠得更狠。所以设计时**不能"按最差的一处定指标"，要把两处的台阶加起来再留余量**。

三处耦合之外，还有一处弱耦合：T_s 变小时，积分器每拍累积的量化噪声按 T_s 减小，纹波略降。表 85-12 里看不出这一项，是因为它比台阶的影响小一个数量级，**可以忽略，但要知道它存在**——这也是"提高采样率对纹波有一点点好处"的物理来源。

85.7.11 按"影响面"把五个参数分成两组

五个参数按"改它会不会动稳定性"可以分成两组，这个分法比敏感度排序更实用，因为它直接决定改动顺序。

第一组，动稳定性的：T_s、m、控制器参数。 这三个一改，PM、增益裕度、超调全变。表 85-7 里 T_s 从 20 ms 放到 100 ms，PM 从 63.2° 掉到 36.5°，超调从 4.8% 涨到 38.8%，临界增益从 17.31 掉到 3.81——**一个参数动了三个指标，其中一个是稳定性指标**。所以这一组必须在设计阶段定死，运行阶段只允许微调控制器参数，T_s 与 m 属于"改了要重新验收"的项。

第二组，动精度的：N_{adc}、N_{dac}、R。 这三个改的是台阶与纹波，PM 几乎不动（表 85-12 从 8 位到 16 位，PM 变化不到 0.5°）。它们可以在调试阶段反复试：纹波大就加位或缩量程，不会把回路调失稳。**这一组的安全性是"改错了只是精度差一点"，不是"改错了会炸"**。

分组带来一条明确的改动顺序：**先在第一组里把稳定性做够，再在第二组里把精度做够**。反过来做——先花时间把位数从 12 加到 16，再回头发现 T_s 选大了要改成 10 ms——等于白做，因为位数需求本来就会随 T_s 变（T_s 小了，每拍累积的噪声少，允许的台阶可以粗一点）。

分组还解释了一个常见困惑：**为什么现场"加两位 ADC"经常没用**。因为若纹波的主因是 DAC 台阶（表 85-12 里 8 位时 1.06×10^{-3} 对 9.99×10^{-4}），加 ADC 位数只把其中一半压下去，总纹波最多降一半；而把 DAC 满量程从 10 缩到 2，台阶直接降五倍。**先看是谁在主导，再决定动哪一组里的哪一个。**

85.8 调参方法与经验

85.8.1 采样率：先看会不会混叠

采样率有两个约束：上限由 CPU 与总线给，下限由混叠给。混叠这一条常被忽略，因为"信号里没有高频"是个假设，而现场总有开关电源、变频器、PWM。

表 85-13 是混叠实测：8 Hz 有用信号叠加 70 Hz 干扰，只改采样率。

表 85-13 混叠：70 Hz 干扰在不同采样率下折到哪

f_s	奈奎斯特	解析折叠	实测峰	幅度比
200 Hz	100 Hz	70 Hz	70.0 Hz	0.550
100 Hz	50 Hz	30 Hz	30.0 Hz	0.625
50 Hz	25 Hz	20 Hz	20.0 Hz	0.588
40 Hz	20 Hz	10 Hz	10.0 Hz	0.616
25 Hz	12.5 Hz	5 Hz	5.0 Hz	0.615

三点结论。

第一，解析折叠频率与实测完全一致。 用式 (85-3) 手算的五个数与 FFT 峰五个数逐一吻合，说明这条式子可以直接用于现场估算。

第二，幅度不衰减。 幅度比在 0.55–0.63 之间，理论值 0.6（干扰与信号的幅值比），偏差来自窗函数与频谱泄漏。**折回不是"滤掉"，是"改名"。**

第三，$f_s = 200$ Hz 时 70 Hz 还没折回（$70 < 100$），所以那不是混叠，只是正常采样。这一行是对照组，提醒读者：**表里前一行"看起来正常"不代表滤波器可以省**，因为下一个更高频的干扰马上就会折回来。

工程上的做法：采样率至少取最高关心的干扰频率的 2.5 倍，再加一级模拟低通（§85.9）。**只靠提高采样率抗混叠是错误路线**，因为干扰频率总是可能更高。

85.8.2 多速率：不同回路用不同周期

一个系统里往往有多个回路：电流环、速度环、位置环、温度环。它们的带宽差几个数量级，用同一个 T_s 是浪费。

表 85-14 给出四类带宽在三个采样周期下的相位代价，用式 (85-26) 手算：

$$\Delta\varphi_{\text{half}} = \frac{\omega_c T_s}{2}\ (\text{rad}) = \frac{180}{\pi}\cdot\frac{\omega_c T_s}{2}\ (\text{deg}) \tag{85-26}$$

表 85-14 不同带宽回路在三个采样周期下的相位代价

回路	ω_c	$T_s = 10$ ms	$T_s = 1$ ms	$T_s = 0.1$ ms
慢过程	3.8 rad/s	1.09°	0.11°	0.01°
流量/压力	63 rad/s	18.05°	1.80°	0.18°
伺服速度环	628 rad/s	179.9°	17.99°	1.80°
伺服电流环	6283 rad/s	1799.9°	179.99°	18.00°

这张表最直观的读法是**找 18° 那一格**：慢过程在 10 ms、流量在 1 ms、速度环在 0.1 ms、电流环在 0.01 ms。四类回路的采样周期差三个数量级，**这就是多速率结构的由来**。

多速率带来一个问题：慢回路看不到快回路的细节，快回路要等慢回路的设定值。两者之间有个等效延迟：

$$T_{\text{eq}} \approx \frac{T_{\text{slow}}}{2} + T_{\text{slow,calc}} \tag{85-27}$$

本章的算例：位置环 10 ms、速度环 1 ms，位置环给速度环的设定值每 10 ms 更新一次，速度环看到的设定值平均滞后 5 ms。**这 5 ms 对速度环是纯延迟，要计入速度环的裕度**，不能因为"速度环自己采样快"就忽略。

85.8.3 总滞后分解：钱花在哪一段

表 85-15 是本章基准的一套滞后分解。

表 85-15 结构链的总滞后分解（$T_s = 20$ ms，$\omega_c = 3.797$ rad/s）

环节	时间	相位代价
零阶保持半拍（已含在 PM 里）	10.00 ms	2.2°
计算（控制律）	1.20 ms	0.3°
现场总线一个周期	2.00 ms	0.4°
反馈滤波（3τ，$\tau = 1$ ms）	3.00 ms	0.7°
执行器死区与建立	4.00 ms	0.9°
采样之外四项	10.20 ms	2.2°
全链合计	20.20 ms	4.4°

名义 $PM = 63.2^\circ$，扣掉采样之外的四项 2.2°，剩 60.9°。这张表说明两件事。

第一，采样之外的四项加起来只有 2.2°，与半拍的 2.2° 相等。 也就是说，**在一个 20 ms 的回路里，“算得快”与“传得快”加起来，只抵得上半拍**。这就是为什么把计算从 1.2 ms 优化到 0.2 ms，收益只有 0.2°——不值得。

第二，四项里最大的是执行器 4 ms。 这一项常被忽略，因为它不在“控制器”里。但阀门从收到信号到真正动，4 ms 是乐观值，气动执行器常常是几十到几百毫秒。**执行器的时间常数必须实测，不能按手册**。

图 85-9 把这张表画成堆叠条形图，横轴是时间，右侧标出对应的相位。可以看出四个小项堆起来才勉强等于半拍那一块。

图 85-9 滞后分解：零阶保持半拍占一半，其余四项各占 0.3°–0.9°

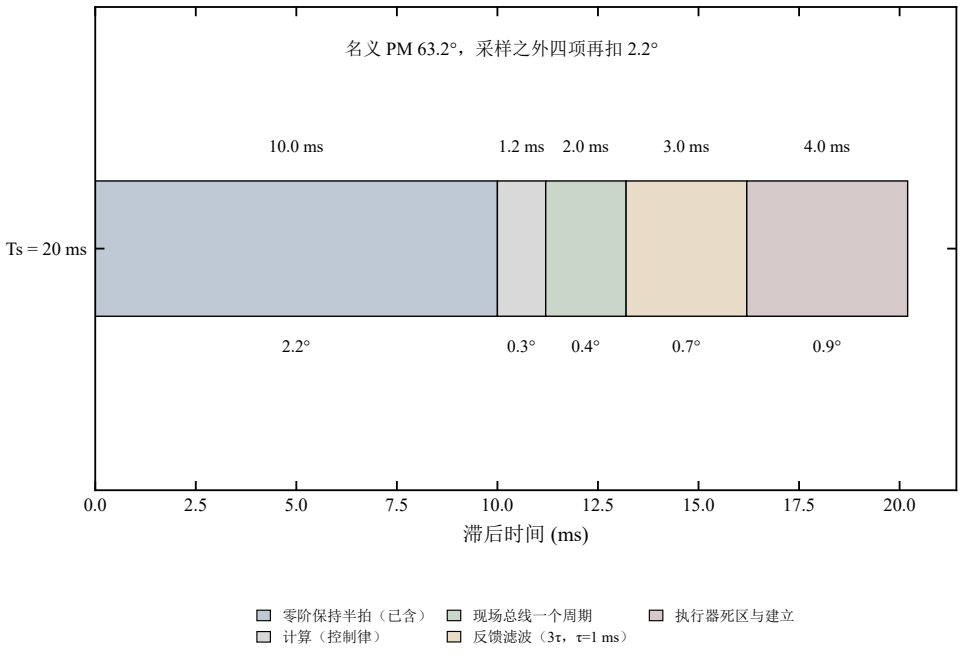


图 85-9

85.8.4 整定的顺序

现场整定按“先时间、后幅值、最后验”三步走：

第一步，固定位数（比如都设 12 位），只调 T_s。 从大到小试，找到“超调开始明显上升”的那一档，再往回退一半。本章对象：100 ms 超调 38.8%，50 ms 超调 14.1%，20 ms 超调 4.8%，退到 20 ms 是合理的。

第二步，固定 T_s，只调位数。 从低到高试，找到“纹波降到允许值以下”的那一档就停。本章允许纹波 10^{-3}，8 位双侧 2.32×10^{-3} 不够，10 位 1.18×10^{-3} 勉强，12 位 1.45×10^{-4} 有余。**取 10 位或 12 位取决于预算，不必取 14 位。**

第三步，验 $\omega_c T_s$ 与总滞后。 用式 (85-15) 与式 (85-27) 手算一遍，与仿真对照。如果手算与仿真差得多，说明模型里漏了一项（通常是执行器或滤波）。

85.8.5 三个现场经验

经验一：T_s 取 2 的幂。 10 ms、20 ms 固然好算，但定时器分频与总线周期通常是 2 的幂（1、2、4、8 ms）。取 2 的幂可以让采样与通信对齐，避免“采样刚结束、总线刚开始”的抖动。

经验二：位数取 12 位，量程按实际工作范围。 12 位是工业标配，价格与 10 位差不多；量程按实际范围设，比加位数有效。本章 8 位 DAC 缩量程后台阶从 3.91×10^{-2} 降到 7.81×10^{-3}，**一分钱不花，精度提五倍。**

经验三：延迟按“拍”算，不按“毫秒”算。 现场调试时，把计算时间从 5 ms 优化到 1 ms，看起来省了 4 ms，但如果 $T_s=20$ ms，m 仍然是 1，裕度一点不变。**要先算 m，再决定值不值得优化。**

85.8.6 两个常见错误

错误一：把采样周期当成“精度参数”。 有人为了“提高精度”把 T_s 从 20 ms 降到 1 ms，结果纹波没变（纹波由位数定），CPU 负载涨了 20 倍，还引入了新的抖动。**T_s 管的是相位与稳定性，不是精度。**

错误二：忽略采样间纹波。 只测采样点，看到“曲线很光滑”，就认为没问题。实际上 $T_s=100$ ms 时采样间纹波 3.49×10^{-5}，是持续振荡，会在机械上表现为嗡嗡声、在电机上表现为发热。**验收时必须测连续输出**（§85.6.4）。

85.8.7 一次完整的整定记录

把 §85.8.4 的三步走成一次真实的过程，数字会更清楚。起点是“抄来的参数”：$K_p=2$、$T_i=0.5$ s，T_s 先按上位机的默认值 100 ms。

第一轮，$T_s=100$ ms。 仿真给出 $PM=36.5^\circ$、超调 38.8%、IAE 0.5484。PM 低于 45° 的底线，超调近四成，**不能用**。此时不要急着调 K_p——先看 $\omega_c T_s=0.398$，已经越过 0.3 的判据，问题在采样周期。

第二轮，$T_s=50$ ms。 $PM=53.4^\circ$、超调 14.1%、IAE 0.3230。指标“能看”了，但 PM 只比底线高 8°，而 §85.6.8 要求留一拍延迟余量：m 从 1 加到 2 会再掉 4.4°，剩 49°，勉强。**这一档适合“预算紧、性能要求不高”的场合。**

第三轮，$T_s=20$ ms。 $PM=63.2^\circ$、超调 4.8%、IAE 0.2734。m 加到 2 还有 58.8°，加到 5 才压线；$\omega_c T_s=0.076$，留了一倍余量。**采用。**

代价是 CPU：按 §85.10.1 的口径（单回路 1.5%），20 个回路从 100 ms 时的 6% 涨到 30%。**用五倍的 CPU 换 26.7° 裕度与 34 个百分点的超调**，这笔交易在过程控制里是划算的；如果回路数上百，就要重新算，可能退到 50 ms。

这份记录的价值在于它展示了“整定”的真实形态：**不是调 K_p，而是先调 T_s，再确认 K_p 还在安全区。**K_p 从头到尾没动，因为它本来就在 11.6% 临界值的位置，没有问题。

85.8.8 三条现场判据

现场没有 Bode 图，只有现象。三条判据把现象折回参数。

判据一：超调超过 20%，先查 T_s 与 m。 表 85-7 里，超调 20% 大致对应 T_s 在 50–100 ms 之间；表 85-9 里，$m=5$ 时超调 24.4%。两者都指向“时间账超支”。**不要先动 K_p**：降 K_p 能把超调压下去，但同时把响应拖慢，是“用性能换稳定”，治标不治本。

判据二：稳态纹波超过允许值，先查 DAC 量程与位数。 表 85-12 显示，双侧 8 位纹波 2.32×10^{-3}，12 位 1.45×10^{-4}，差 16 倍。若允许值是 10^{-3}，8 位不够、10 位勉强、12 位有余。**先缩量程，再考虑加位数**，因为缩量程不花钱。

判据三：饱和恢复时间超过两拍，查抗饱和。 给一个大幅值阶跃让输出饱和，撤掉后若输出要好几拍才回来，说明积分器积累了大量“积分风”。正常应在 1–2 拍内恢复。**这一条与 T_s 无关，纯属实现缺陷。**

三条判据的顺序是“先时间、再幅值、后实现”，与 §85.3 的设计顺序一致。**排查顺序与设计顺序一致，是有意为之**：设计的每一步都对应一个可观测的现象，反查才不会乱。

85.8.9 两个不同量级的案例

同一条判据，在两个差三个数量级的回路上怎么用。

案例一：流量回路。 对象时间常数约 30 ms，要求 $\omega_c\approx63$ rad/s（10 Hz）。按 $\omega_c T_s\le0.3$，$T_s\le4.8$ ms；按留一倍余量，取 $T_s=1$ ms。此时半拍 1.8°（表 85-14 第二行），可以接受；若图省事取 10 ms，半拍 18.05°，PM 直接少 18°，**流量回路最常见的“阀门抖”就是这样来的。**

案例二：温度回路。 对象时间常数几十秒，$\omega_c\approx0.05$ rad/s。$T_s=1$ s 时 $\omega_c T_s=0.05$，半拍 1.4°，几乎不用考虑。真正的限制是传感器（套管 5–20 s）与执行器（气动阀 2–10 s），**它们的时间常数比采样周期大一个数量级。**

两个案例放一起，能看出一件事：**采样周期该取多少，取决于“回路自己的时间尺度”，而不是“控制器的性能”。**流量回路的 1 ms 与温度回路的 1 s，用的是同一条公式、同一张表，只是 ω_c 不同。**这也是本章表格用 $\omega_c T_s$ 而不是 T_s 作横轴的原因**——归一化之后，两类回路可以放在同一张图上比较。

85.8.10 整定记录该留哪几列

§85.8.7 那份记录之所以有用，是因为它把“试过什么、为什么否掉”一起写下来了。现场最常发生的事故不是“参数调错”，而是**三个月后没人记得为什么是 20 ms，于是有人把它改成 50 ms“省点 CPU”。**

一份够用的记录表至少要十列：时间、T_s、m、N_{adc}、N_{dac}、量程、K_p、T_i、PM 与超调、结论。前六列是结构参数，中间两列是控制器参数，后两列是判据。**十列缺一列，复现就会断链**：只写 T_s 不写 m，别人不知道总滞后是 20 ms 还是 60 ms；只写 PM 不写 N，别人不知道那次的纹波有多大。

三条填写规则，都是从踩过的坑里来的。

规则一：写实测值，不写设计值。 设计值 $T_s=20$ ms 与实测周期 20.3 ms 是两回事——后者包含调度抖动。若实测比设计大 5% 以上，说明定时器或任务优先级有问题，这一条要在记录里标红。**记录的价值一半在“数”，一半在“数与设计的差”。**

规则二：每次只改一个参数，且把被否掉的档位也留下。 §85.8.7 里 100 ms 与 50 ms 两轮都留着，正是它们让“为什么最后选 20 ms”变得可读。**只留最后一组参数，等于把推导过程删掉，只留答案。**

规则三：把判据和参数写在一起。 “超调 4.8%”是数，“超调 4.8%，判据 $\le8\%$（设计值加 3 点）”才是结论。判据不写下来，下一任工程师会用自己的判据重新评价一遍，然后得出不同结论。

最后一条经验：**记录要能独立成文。**把它单独发给一个没见过这个项目的人，他应该能照着跑出同样的结果——需要哪一版代码、哪一组对象参数、哪一个随机种子。本章的 `seed=851` 就是为此存在的：**没有种子的仿真结果不可复现，不可复现的记录等于没写。**

85.9 优化与改进

结构定下来之后，还能动的有四处：抗混叠滤波器、位数分配、控制器实现形式、量化补偿。四处都不改结构，只改“怎么用这套结构”。

85.9.1 抗混叠：先滤波，再采样

式 (85-3) 说混叠只是“改名”，改不回来。唯一有效的办法是在采样之前把高频成分滤掉。n 阶 Butterworth 低通的幅频是

$$|H_a(j\omega)| = \frac{1}{\sqrt{1 + (\omega/\omega_c)^{2n}}} \tag{85-28}$$

在干扰频率上的衰减为

$$\mathrm{att} = 10\log_{10}\left[1 + \left(\frac{f_{\mathrm{int}}}{f_c}\right)^{2n}\right] \tag{85-29}$$

折回基带后的残余幅值为

$$A_{\mathrm{res}} = A \cdot 10^{-\mathrm{att}/20} \tag{85-30}$$

表 85-16 是本章算例：$f_s = 100$ Hz、$f_c = 40$ Hz、干扰 70 Hz、干扰幅值 0.6。

表 85-16 抗混叠滤波器阶数与残余

阶数	70 Hz 处衰减	折回后残余幅值
1	6.1 dB	0.2977
2	10.2 dB	0.1862
4	19.5 dB	0.0636

一阶只把 0.6 压到 0.30，四阶压到 0.064。**要压到 0.01 以下需要约 7 阶**，代价是相位滞后与成本。工程上的折中是二阶或四阶，配合“把 f_c 设在 $0.4 f_s$ 左右”——**滤波器不是为了“完全滤掉”，而是为了“把折回的干扰压到回路能忍”**。

要注意滤波器本身带来相位滞后，必须计入式 (85-27) 的总滞后。本章表 85-15 里的“反馈滤波 $3\tau = 3$ ms”就是这一项。

85.9.2 位数预算：钱花在 ADC 还是 DAC

总位数预算有限时（比如“两片转换器加起来不超过 20 位”），怎么分？表 85-17 是实测。

表 85-17 位数预算（ADC + DAC = 20 位，$T_s = 20$ ms）

ADC	DAC	纹波	IAE
6	14	4.00×10^{-3}	0.2983
8	12	1.01×10^{-3}	0.2790
10	10	1.18×10^{-3}	0.2759
12	8	1.07×10^{-3}	0.2772
14	6	6.17×10^{-3}	0.2890

最优点在 8–12 之间，纹波都在 1.0×10^{-3} 附近；两端最差，6/14 是 4.00×10^{-3}，14/6 是 6.17×10^{-3}。为什么两端都差？

- ADC 太差（6 位）：测量台阶 2.34×10^{-2}，控制器“看不清”，输出跟着抖；
- DAC 太差（6 位）：输出台阶 1.56×10^{-1}，控制器算得再准也执行不出来。

两端不对称：14/6 比 6/14 更差（6.17 对 4.00），说明**在总位数紧张时，DAC 比 ADC 更值得给位数**。这条与 §85.7.4 的结论一致，也符合直觉：ADC 的量化噪声要经过闭环衰减才到输出，DAC 的台阶直接加在对象输入端，没有衰减。

85.9.3 控制器实现：增量式与抗饱和

结构不变，控制器的写法可以变。三种常见写法：

位置式：$u_k = K_p e_k + \frac{K_p T_s}{T_i} \sum e_j$。直观，但积分项要存一个大数，长期运行会累积舍入误差。

增量式：$\Delta u_k = K_p(e_k - e_{k-1}) + \frac{K_p T_s}{T_i} e_k$，$u_k = u_{k-1} + \Delta u_k$。只存上一拍的值，舍入误差不累积，**切换手动/自动时无扰**。工程上优先用这个。

抗饱和：对积分项做限幅，或“积分分离”（误差大时不积分）。本章的 u 被限在 ± 5，如果积分项无限制地累积，退饱和时会有一大段“积分风”（windup）。**抗饱和不是可选项，是必须项**，因为它直接把“饱和”这个非线性引入闭环，而本章所有线性分析都假设不饱和。

85.9.4 多速率与任务同步

多速率结构的关键不是“用不同周期”，而是**周期之间要有确定的相位关系**。两种做法：

同步：所有任务由同一个时基分频，慢任务在快任务的第 N 拍触发。好处是抖动确定，坏处是慢任务必须等快任务。

异步：各任务独立定时，用双缓冲交换数据。好处是解耦，坏处是交换延迟不确定（0 到 1 个慢周期），等效于给慢回路加了随机延迟。

本章建议：**关键回路用同步，监视回路用异步**。同步的代价是调度器要简单可靠，这在第 86 章展开。

85.9.5 量化补偿：三种手段

量化误差不能消除，但可以“摊平”或“前馈掉”。表 85-18 是三种手段的对照。

表 85-18 三种量化补偿手段

手段	做法	效果	代价	适用
过采样平均	一个控制周期内采 M 次求平均	噪声降 $\sqrt{M}$	ADC 速度、CPU	低速回路
抖动（dither）	叠加小幅随机或三角波再量化	把谐波摊成白噪声	引入额外噪声	音频、振动台
前馈补偿	用模型预测量化误差并补偿	低频段有效	需要准确模型	精密定位

过采样平均的效果最确定：$M = 16$ 时噪声降 4 倍，等于多 2 位。本章的 12 位 ADC 若每周期采 16 次平均，等效 14 位，纹波从 6.24×10^{-5} 降到约 1.56×10^{-5}，与表 85-12 的 14 位一行对得上。**前提是 ADC 速度够、且噪声真的是白的**；如果噪声是工频干扰，平均反而会把干扰“锁”进去。

抖动（dither）要慎用：它把确定性误差变成随机误差，对“听感”有好处，对“精度”没好处。**本章的回路不需要 dither**，因为量化误差已经被闭环衰减（式 85-14）。

85.9.6 该优化哪一项：一张优先级表

按“投入产出比”排序：

1. **缩量程**：零成本，台阶线性下降，最优先；
2. **改 T_s**：成本是 CPU，收益是裕度，但要先算 $\omega_c T_s$ 是否已经够小；
3. **加抗混叠滤波**：成本是几个电容与一点相位，收益是抗干扰；
4. **提位数**：成本是芯片价格，收益是纹波，10 位以上收益递减；
5. **优化计算时间**：成本是开发时间，收益是 m 可能降一拍，**只有跨过整数边界才有效**。

这个顺序与 §85.7.8 的敏感度排序不同：敏感度说的是“影响大小”，优先级说的是“性价比”。**两者都要看，先用性价比排序做决策，再用敏感度排序做验收**。

85.9.7 过采样的实现与代价

过采样平均是最“确定”的量化补偿：噪声按 $\sqrt{M}$ 下降，$M=16$ 等于多两位。实现上要注意四个数。

第一个数是“一个周期能采几次”。 本章 $T_s = 20$ ms、ADC 单次转换 50 μs，理论上能采 400 次。取 $M=16$，占掉 0.8 ms，还剩 19.2 ms，绰绰有余。**若 ADC 单次是 1 ms，一个周期只能采 20 次，M 最多取 8**——这就是“过采样受 ADC 速度限制”的具体形态。

第二个数是“平均窗口带来的延迟”。 M 次平均等效于把测量值推迟 $M/2$ 个采样间隔。$M=16$、间隔 50 μs 时是 0.4 ms，对 20 ms 的周期只占 2%，相位代价 0.09°，可以忽略；但若 $M=64$，延迟 1.6 ms，就开始进入 §85.8.3 的总滞后账。

第三个数是“噪声是不是白的”。 过采样平均对白噪声有效，对工频干扰无效甚至有害。若干扰是 50 Hz，平均窗口应取工频周期的整数倍（20 ms），而不是 16 个 50 μs 的窗口。**这一条决定了“平均”与“陷波”该选哪个**：宽带噪声用平均，窄带干扰用陷波。

第四个数是“抽取后的数据率”。 平均之后每个控制周期只输出一个数，数据率仍是 $1/T_s$，不增加总线负担。**这是过采样相对“提高采样率”的最大优势**：精度按 $\sqrt{M}$ 涨，而通信量不变。

四个数合起来是一条判据：**只有当 $M/2$ 个采样间隔远小于 T_s、且噪声是宽带时，过采样才划算**。本章满足，所以 12 位 ADC 加 $M=16$ 平均，等效 14 位，纹波从 6.24×10^{-5} 降到 1.56×10^{-5}，与表 85-12 的 14 位一行对得上。

85.9.8 抗混叠滤波器的相位代价

§85.9.1 只算了衰减，没算相位。滤波器是模拟电路，它在采样之前，**它的相位滞后直接进闭环，与采样无关**。二阶 Butterworth 的相位是

$$\varphi_a(\omega) = -2\arctan\frac{\omega}{\omega_c}$$

本章 $f_c = 40$ Hz（251 rad/s），在 $\omega_c = 3.797$ rad/s 处只有 -1.7°，可以忽略。**但这只是因为本章带宽低**。换一个 $T_s = 1$ ms、$\omega_c = 628$ rad/s 的伺服回路，抗混叠滤波器若按 $f_c = 0.4 f_s = 400$ Hz 设，在 ω_c 处的相位是

$$\varphi_a = -2\arctan\frac{628}{2513} = -28.1^\circ$$

二十八度，相当于把 T_s 从 1 ms 拉长到 1.25 ms。这就是“抗混叠滤波器不能随便加”的原因：它用一种滞后换抗干扰，而滞后与采样滞后在闭环里是同一本账。

工程上有三条折中。

第一条：f_c 至少取 $4\omega_c$。 此时相位代价约 -28°；取 $6\omega_c$ 时降到 -19°；取 $10\omega_c$ 时 -11°。**f_c 越高，抗混叠越弱**——两难。

第二条：把滤波器放在内环之外。 位置环的滤波器只影响位置环，不影响速度环；速度环的滤波器则同时影响两个环。**能放在外环就放在外环**，代价最小。

第三条：用数字滤波补，但要在采样之前有最起码的模拟滤波。 完全靠数字滤波抗混叠是无效的（数字滤波在采样之后），但一级简单 RC 加数字高阶滤波，可以把总代价压到可接受。

85.9.9 两条改结构的路径

前面八条都是“在既定结构里优化”。真正能改结构的只有两条路，各有明确的适用条件。

路径一：Smith 预测器（见第 47 章、第 107 章）。 在控制器旁并联一个对象模型，把纯延迟“移出”闭环，让控制器看到一个没有延迟的对象。理论上能把 m 拍的相位全部拿回来。**但它对模型误差极敏感**：模型延迟差一拍，补偿就反向，裕度可能比不补偿还差。本章 $m=1$、补偿收益只有 4.4°（表 85-9），**不值得**；只有 $m\ge 5$（收益 17° 以上）时才值得考虑。

路径二：状态观测器加状态反馈（见第 54 章）。 用观测器估计不可测状态，用状态反馈配置极点。它能把闭环极点放到任意位置，但**观测器本身也在采样系统里工作，它的增益必须按离散模型设计**，否则会在采样间产生观测器振荡。这条路的收益是“可以任意配极点”，代价是“需要完整的状态模型”。

两条路的共同点是：**它们改变的是“控制器看到什么”，不是“结构本身”**。真正的结构参数——T_s、m、位数、量程——一旦定下，任何控制器都改不了它们带来的相位与台阶。**这就是本章反复强调“结构参数要在设计前定”的原因**。

85.9.10 一个把顺序做反的反例

§85.9.6 的优先级表是从“投入产出比”排的，但现场更常见的错误不是“不排序”，而是**从表的底部往上做**。下面这个例子是按真实参数算的。

某回路带宽与本章相同（$\omega_c=3.797$ rad/s），但 T_s 沿用了上位机默认的 100 ms，此时 $PM=36.5^\circ$、超调 38.8%（表 85-7）。工程师的判断是“超调大，说明信号脏”，于是做了两件事：加一只四阶抗混叠滤波器，把 ADC 从 12 位换到 16 位。

两件事都没有解决超调，第一件还让它更糟。滤波器按 $0.4f_s=4$ Hz（25.1 rad/s）设截止，二阶节两级串起来在 ω_c 处的相位是 $-2\times 2\arctan(3.797/25.1)=-34.4^\circ$。$PM$ 从 36.5° 掉到约 2°——**回路进入了持续振荡**。而 16 位 ADC 把纹波从 1.45×10^{-4} 降到 9×10^{-6}，这个改善在超调 38.8% 的波形上根本看不出来。

正确的顺序是反过来的：**先改 T_s**。$100\to 20$ ms 一步就把 PM 从 36.5° 提到 63.2°、超调从 38.8% 降到 4.8%；改完再回头看抗混叠，此时 $f_s=50$ Hz，滤波器截止可以放到 20 Hz（125.7 rad/s），在 ω_c 处只有 -3.5°，代价一下小了十倍。**同一个滤波器，T_s 改对之后从“致命”变成“可忽略”**。

这个反例的教训可以压成一句：**抗混叠滤波器的代价与 T_s 成反比，所以它必须排在 T_s 之后决定**。T_s 大，滤波器截止就低，相位代价就大；T_s 小，滤波器可以放得很宽，代价就小。**先定 T_s 再加滤波，是顺序问题，不是技术问题**。

顺带说一句位数。16 位 ADC 在本章这种纹波水平（10^{-4}）下是过剩的，它带来的唯一确定收益是“缩量程之后仍然够用”——而缩量程本来零成本。**把 16 位的钱花在一只更好的变送器上，收益更大**：变送器的非线性与温漂是 10^{-3} 量级，比 16 位 ADC 的 9×10^{-6} 大两个数量级，**瓶颈从来不在 ADC 位数上**。

85.9.11 定点实现：三个与位数无关的坑

工控里 PI 常在定点 DSP 或 MCU 上跑，这时“12 位 ADC”之外还有一层量化——**控制器内部字长的量化**。三个坑与 ADC 位数无关，只与实现方式有关。

先给系数。增量式 PI 的两个系数是 K_p 与 $K_i = K_p T_s / T_i$。本章 $K_p = 2.0$、$K_i = 2.0 \times 0.02 / 0.5 = 0.08$。若用 Q15（$1$ 表示为 32768），K_i 是 2621；$K_p = 2.0$ 已经超出 Q15 的范围（最大约 1），**必须改用 Q13 或把增益拆成“整数倍 + 小数倍”。这是第一个坑：系数与变量的定标要分开选，不能全用一套 Q 格式。**

坑二：右移截断带来直流偏差。 乘法结果右移 15 位，若直接截断（向零取整），每次都是“少一点”。增量式每拍都少一点，累积起来就是稳态偏差。本章 K_i 项每拍截断最多损失 1 个 LSB，1 秒 50 拍最多损失 50 个 LSB——对应 $50 \times 3.66 \times 10^{-4} = 1.8 \times 10^{-2}$ 的等效误差，**比 12 位 ADC 的台阶大 50 倍**。对策是**四舍五入**（加半个 LSB 再右移），一行代码，误差从系统性变成随机性。

坑三：小信号死区。 稳态时 e 很小，$K_i e$ 算出来不足 1 个 LSB，被截断为零，积分器停止累加，**系统停在“差一点”的位置回不来**。这不是精度问题，是**结构性死区**。对策有两条：一是**余数累加**（把每次截断丢掉的低位存起来，攒够一个 LSB 就进位，等效一阶误差反馈）；二是**把积分器做成 32 位累加器**，输出时才截到 16 位。**后者更常用，代价只是几个字节的 RAM。**

三个坑的共同解法是一句话：**累加器要比输出宽一倍，乘法的定标要单独定，截断要四舍五入**。定点实现真正难的不是“位数够不够”，而是“误差往哪个方向累积”——**系统性误差比随机误差危险得多，因为它不会互相抵消**。

85.10 工程落地

85.10.1 采样周期的折中：算一笔总账

T_s 不能只看控制指标，还要看 CPU 负载。设一个回路每拍的计算与 I/O 时间是 T_c，要带 N 个回路，则 CPU 占用率为

$$U = \frac{N\,(T_c + T_{io})}{T_s} \tag{85-31}$$

本章的数值代入：$T_c = 200\ \mu\text{s}$、$T_{io} = 100\ \mu\text{s}$、$T_s = 20\ \text{ms}$，则单回路占 1.5%，$N = 20$ 时占 30%；T_s 改成 1 ms，单回路占 30%，$N = 20$ 时 600%——**根本跑不完**。

把式 (85-31) 与表 85-7 放在一起看，就得到一条折中线：

- $T_s = 20\ \text{ms}$：$PM = 63.2^\circ$，20 回路占 30% CPU；
- $T_s = 10\ \text{ms}$：$PM = 66.3^\circ$，20 回路占 60% CPU；
- $T_s = 5\ \text{ms}$：$PM = 67.9^\circ$，20 回路占 120% CPU（超标）。

从 20 ms 压到 10 ms，裕度只多 3.1°，CPU 翻倍。这笔账不划算，所以本章取 20 ms。**采样周期的选择是一个“多花一倍 CPU 换三度裕度”的问题，不是“越准越好”的问题。**

85.10.2 量化误差补偿：现场三种做法

教科书上的补偿方法（过采样、dither、前馈）在现场要打折扣，原因有三：ADC 速度不够、噪声不是白的、模型不准。现场真正可用的做法是这三种。

做法一：缩量程。 这是最有效也最便宜的。把变送器换成小量程的，或把 ADC 的参考电压降到接近实际信号范围。本章 8 位 DAC 缩量程后台阶从 3.91×10^{-2} 降到 7.81×10^{-3}，**纹波降五倍，成本为零**。

做法二：软件平均。 一个控制周期内采 M 次求平均，噪声降 $\sqrt{M}$。注意两点：平均窗口不能超过 T_s 的一半（否则等效增加延迟），且平均会引入相位滞后（约 $M/2$ 个采样间隔）。本章 $T_s = 20\ \text{ms}$ 时，若 ADC 每次 50 μs，一个周期能采 400 次，取 $M = 16$ 平均，噪声降 4 倍，延迟增加 0.4 ms，可接受。

做法三：把量化误差“推”到回路带宽之外。 用一阶高通或微分环节把量化噪声的高频成分放大、低频成分压低，让它在输出端表现为高频小纹波而不是低频漂移。这一招要小心：高通会放大噪声，可能把执行器“抖”坏。

三种做法的共同点是**都在“摊平”而不是“消除”**。量化误差的方差是 $\Delta^2/12$，改不了；能改的只是它在频域的分布。

85.10.3 采样抖动：一个容易被忽略的误差源

采样时刻不准，等效于在幅值上加了误差：

$$\delta y \approx \dot{y}\,\delta t \tag{85-32}$$

本章对象在阶跃初期的斜率约 $1/0.5 = 2$ /s，若 $\delta t = 1$ ms，则 $\delta y \approx 2 \times 10^{-3}$——比 **12 位 ADC** 的台阶 3.66×10^{-4} 还大五倍。也就是说，**一个 1 ms 的采样抖动，能把 12 位 ADC 用成 9 位**。

抖动的来源有四个：晶振偏差（ppm 级，慢漂移）、中断响应延迟（μs 到几十 μs）、总线同步误差（μs 级）、操作系统调度（ms 级）。前三个可以接受，第四个必须避免——**控制任务不能用非实时操作系统调度**，这是第 86 章的主题。

85.10.4 接线与接地：结构之外的隐含项

结构图画的是理想连接，现场还有三件事：

单点接地。 模拟地与数字地只在一个点相连，否则地环流会把数字噪声灌进模拟通道。这一条在 12 位以下不明显，14 位以上是决定性的。

屏蔽与双绞。 信号线用双绞屏蔽线，屏蔽层单端接地。变频器、伺服驱动器附近的信号线必须屏蔽，否则 §85.8.1 的混叠会以“莫名其妙的低频振荡”形式出现。

采样时刻避开开关时刻。 如果系统里有 PWM，采样时刻要避开开关沿（通常取 PWM 周期的中点）。这一条能把开关噪声压 20–30 dB，**比加滤波器便宜**。

85.10.5 验收清单

结构设计的验收分五项，每项都有可测的判据：

1. **采样周期**：实测 T_s 的抖动（用示波器看触发输出），要求 $< 5\%\,T_s$；
2. **相位裕度**：从闭环频率响应或阶跃响应反算，要求 $\geq 45^\circ$；
3. **量化纹波**：稳态时测连续输出，要求 $\leq$ 允许值（本章 10^{-3}）；
4. **混叠**：在干扰频率上注入一个已知幅值的信号，看折回后的幅度是否符合式 (85-30)；
5. **饱和恢复**：给一个大幅值阶跃使输出饱和，撤掉后看恢复时间，要求无持续振荡（抗饱和生效）。

五项里最容易漏的是第 4 项。现场往往“没有干扰源”就不测，但干扰源是后来加的（变频器、开关电源），**验收时留一个余量比事后补救便宜**。

85.10.6 一个完整案例：从需求到参数

假设需求是：一个温度回路，要求调节时间 < 60 s，超调 $< 10\%$，稳态纹波 $< 0.1\%$。

第一步，定带宽。 $t_r \approx 60$ s，$\omega_c \approx 3/60 = 0.05$ rad/s。超调 $< 10\%$ 对应 $PM \geq 60^\circ$，留余量取 65°。

第二步，定采样周期。 $\omega_c T_s \leq 0.3$ 给出 $T_s \leq 6$ s。但温度对象的时间常数通常是几十秒，取 $T_s = 1$ s 就够（$\omega_c T_s = 0.05$）。**这里 $T_s = 1$ s 不是为了裕度，而是为了抗干扰与故障检测的响应速度。**

第三步，定延迟。 温度传感器的响应时间往往是秒级（比如铂电阻套管 5–20 s），这一项远大于采样与计算。**必须先实测传感器时间常数**，否则所有计算都是空中楼阁。

第四步，定位数。 纹波 $< 0.1\%$ 对应 $\Delta < 10^{-3}$（归一化），满量程 2（± 1）时需要 $N \geq \log_2(2000) = 11$ 位，取 12 位。

第五步，验算。 $\omega_c T_s = 0.05$，半拍 1.4°；延迟若为 3 拍（3 s），相位 8.6°；合计 10°，PM 从 65° 降到 55°，仍满足 $> 45^\circ$。

这个案例说明：**在慢过程里，限制精度的往往不是采样与位数，而是传感器与执行器**。本章的公式在慢过程里给出的数字都很宽松，真正的瓶颈在“测量得准不准、动作得快不快”。**结构设计的第一步永远是“把传感器与执行器的特性测出来”**，不是先画框图。

85.10.7 第二个算例：伺服位置环

把同样的五步用在快过程上，结论会完全不同。需求是：位置环带宽 100 Hz（$\omega_c = 628$ rad/s）、超调 $< 10\%$。

第一步，定带宽。 $\omega_c = 628$ rad/s 是给定的，对应 $PM \geq 60^\circ$。

第二步，定采样周期。 $\omega_c T_s \leq 0.3$ 给出 $T_s \leq 0.48$ ms。留一倍余量取 0.25 ms（4 kHz），半拍 $\omega_c T_s / 2 = 0.0785$ rad $= 4.5^\circ$，整拍 9.0°。

第三步，定延迟。 计算 0.05 ms、总线 0.05 ms，合计 0.1 ms，$0.1 / 0.25 = 0.4$ 拍，向上取 $m = 1$。**注意这里 $m = 1$ 的代价是 9°，不是本章的 4.4°**，因为 ω_c 大了 165 倍，同样的拍数吃掉更多相位。

第四步，定位数。 17 位编码器、每转 2π，LSB $= 2\pi / 131072 = 4.8 \times 10^{-5}$ rad；若丝杠导程 10 mm，折算到直线是 7.6×10^{-5} mm，远小于机械精度。**这里位数早就过剩了**，限制精度的是编码器安装偏心与丝杠间隙，不是分辨率。

第五步，验算。 采样损失 $4.5 + 9.0 = 13.5^\circ$，要从连续设计的 73.5° 里扣，才能保证离散后 60°。**所以伺服设计必须先按离散指标反推连续指标**，不能先按连续设计再验离散。

两个案例对照，最能说明“结构”二字的含义：**同样的五步、同样的公式，慢过程的瓶颈在传感器，快过程的瓶颈在计算与总线**。这也是为什么工业上把回路分成“过程控制”与“运动控制”两类，用两套硬件、两套设计流程。

85.10.8 故障现象与结构参数的对应

现场排查最怕“现象对不上参数”。下面六条是从现象反查参数的对照，按出现频率排序。

第一条：高频嗡嗡声，听感在几百赫兹。 多半是采样间纹波（表 85-11）。T_s 越大、$\omega_c T_s$ 越接近 0.4，纹波越明显。**先量连续输出，不要只看采样点**。

第二条：执行器低频爬行、一顿一顿。 多见于 DAC 台阶大于执行器死区。表 85-12 里 8 位 DAC 台阶 3.91×10^{-2}，若死区是 0.01，台阶比死区大四倍，必然爬行。**缩量程或加位数**。

第三条：稳态值随机跳变几个 LSB。 采样抖动（式 85-32）或接地不良。前者看定时器，后者看模拟地与数字地的连接点。

第四条：一开机就高频啸叫，降增益就好。 混叠。干扰源可能是变频器载波或开关电源，折回基带后被当成真实误差。**加模拟低通，不要试图在控制器里解决**。

第五条：稳态有固定偏差，且积分项已经饱和。 执行器量程不够，或积分限幅设得太小。**先看 u 是否长期贴在 $\pm U_{\max}$**。

第六条：周期性波动，周期正好是 20 ms 的整数倍。 工频干扰或其谐波。**这一条与采样周期无关，但会被采样“锁”成低频拍频**，看起来像控制问题。

六条里前三条与本章参数直接相关，后三条与硬件环境相关。**排查顺序应该是“先结构、后环境”**：结构参数错了，换什么硬件都没用。

85.10.9 上线前的三组试验

验收清单（§85.10.5）是“查什么”，三组试验是“怎么查”。三组试验都要在**连续输出**上测，不能只看采样点。

第一组：阶跃试验。 给 10% 到 90% 的阶跃，记录超调、上升时间、调节时间、稳态纹波。判据：超调 $\le$ 设计值 $+3$ 个百分点，纹波 $\le$ 允许值。**若超调比仿真大很多，先怀疑执行器饱和**，而不是控制器。

第二组：扰动试验。 在执行器输出上叠加一个已知幅值的阶跃（比如 10% 满量程），看输出恢复时间与最大偏差。判据：恢复时间 ≤ 3 倍设计值。**这一组最能暴露积分增益是否偏小**。

第三组：饱和试验。 给一个足以让输出饱和的大阶跃，持续 10 拍后撤掉，看退饱和时间与是否有持续振荡。判据：退饱和 ≤ 2 拍、无持续振荡。**这一组专门查抗饱和**，是数字控制独有的试验，模拟控制没有对应项。

三组试验的记录表要留档：**同一台设备三年后再出问题，靠的就是这张表**。表里除了指标，还要记 T_s、m、位数、量程、K_p、T_i 六个参数——它们才是“当时到底怎么设的”的唯一证据。

85.10.10 交付文档里必须出现的七个数字

工程交付物通常是一份几十页的文档，但真正被后人反复查的只有七个数字。**缺一个，接手的人就得把本章重算一遍**。

1. **T_s 与它的实测抖动**。写成“20 ms（实测 20.3 ± 0.1 ms）”，不能只写 20 ms。抖动的来源见 §85.10.3：定时器分辨率、任务抢占、总线占用。
2. **m 与它的构成**。写成“$m = 1$，由 A/D 转换 0.6 ms + 总线 0.3 ms + 计算 0.2 ms + D/A 建立 0.1 ms 组成”。**只写 $m = 1$ 无法审计**，因为下一版软件可能把某一项从 0.6 ms 涨到 1.2 ms，m 就变成 2 了。
3. **N_{adc} 与满量程**。这两个数必须成对出现，单写 12 位没有意义——台阶是 $R/2^N$，Δ 才是物理量。
4. **N_{dac} 与满量程**。同理。本章 DAC 满量程 10、12 位，台阶 2.44×10^{-3}。
5. **设计点 PM 与实测 PM**。两个数都要，差值就是“模型没算进去的东西”。本章设计 63.0°、实测 63.2°，差 0.2°，说明模型干净。
6. **纹波上限与验收工况**。本章 $\le 10^{-3}$，工况是“稳态、设定值不变、负载不变”。**换工况纹波会变**，所以要写清验收条件。
7. **单回路 CPU 占用率**。本章 1.5%（$T_c + T_{io} = 300$ μs、$T_s = 20$ ms）。这个数决定了“还能再加几个回路”，是扩容的唯一依据。

七个数字之外，最该写而最常被漏掉的是**“这些数是怎么来的”**：哪一版代码、哪一组对象参数、哪一个种子。**数字本身回答“是什么”，来源回答“还能不能信”**。

七个数字的写法还有一条共同要求：**单位不能省，有效位数不能多**。$T_s = 20$ ms 不要写成 0.020000 s，$PM = 63.2^\circ$ 不要写成 63.18°——前者假装精度，后者假装模型。**工程文档的可信度，一半来自数字，一半来自“没有多写一位”**。

85.10.11 上电、复位与通信中断时的头几拍

结构分析假设“每一拍都正常”。上电、复位、总线断开这三种情况下，这个假设不成立，而头几拍往往决定设备是“平稳启动”还是“一上电就冲一下”。

上电第一拍。 A/D 上电后第一次转换没完成，读回的是 0 或随机的旧值。若设定值是 1.0、读回 0，误差 1.0，$K_p = 2$ 直接给出 $u = 2.0$，是满量程的 40%——**阀门一下就开到四成**。执行器动完，第二拍才读到真值，再往回拉。**现象是“上电时输出猛地一跳”**，很多人以为是“启动冲击”，其实是首拍读错。

三条对策，按成本从低到高：**首拍丢弃**（读一次不算，第二拍才进闭环）、**积分器预置**（按当前输出反算积分项，位置式必须做）、**输出保持**（首拍沿用上次的 u，适用于断电重启）。**三条里“首拍丢弃”最省事，代价是延迟一拍启**

动，对过程控制完全无所谓，对伺服就不能丢。

复位与看门狗。 复位后积分器清零，若此时输出不为零，位置式 PI 会算出 $u = K_p e$，跳过积分项，输出立刻掉到零附近——**表现为“复位时执行器一下回到零位”**。对策是增量式（§85.9.3）：增量式复位后第一拍 Δu 只与当前误差有关，输出由执行器自己保持，不会跳。

通信中断。 设定值或测量值来自总线时，中断的后果是“用旧值一直算”。若旧值恰好偏差大，积分器会朝一个方向一直积——**这就是“通信断了两分钟，恢复后阀门卡在全开”的典型过程**。三条对策：**超时判据**（连续 N 拍没收到新数据就切手动或保持）、**积分器冻结**（数据无效时不积分）、**降级输出**（切到安全值而不是保持最后值，视工艺而定）。

三种情况的共同点：**它们都不是“控制算法”的问题，而是“结构在边界条件下怎么表现”的问题**。本章所有仿真都从“稳态之后”开始算，所以这些现象在仿真里看不到——**上电与复位必须在实物上试，这一条没法用仿真替代**。

85.11 优缺点与适用边界

85.11.1 数字结构的四条优势

第一，控制器可以任意复杂。 模拟电路实现一个 PI 要几个运放，实现一个带滤波的 PID 要十几个，实现自适应或 MPC 基本不可能。数字实现改的是代码，**复杂度的边际成本几乎为零**。这是数字控制取代模拟控制的根本原因，不是精度。

第二，参数可以远程改、可以记录。 模拟控制的参数是电阻电容，改一次要拆机；数字控制的参数在寄存器里，改一次是写一个数，还能存进历史、做版本管理。**这一条对运维的价值远大于对性能的价值**。

第三，可以自诊断。 采样断线、执行器卡死、传感器漂移，都可以在软件里判据。模拟控制只能靠额外的硬件。

第四，可以多回路协调。 一台计算机同时看几个回路，就能做解耦、做前馈、做协调。模拟控制里每个回路是独立的。

四条优势里，**第一条与第二条是“能用”，第三条与第四条是“好用”**。评估一个项目该不该上数字控制，先问前两条是否成立（控制器是否真的需要复杂、参数是否真的需要远程改），再问后两条。

85.11.2 数字结构的四条代价

代价一：采样与量化。 这是本章的主题：半拍、延迟、台阶、混叠。它们不会消失，只能被算清楚、压到可接受。

代价二：时钟与抖动。 模拟控制没有“采样时刻”这个概念，数字控制有，而且采样时刻不准会直接变成幅值误差（式 85-32）。

代价三：软件缺陷。 模拟电路的缺陷是元件老化，可预测；软件的缺陷是逻辑错误，不可预测，而且可能只在特定时序下出现。**这一条是数字控制最大的风险，也是第 86–90 章要解决的问题**。

代价四：集中风险。 一台计算机带 100 个回路，它坏了 100 个回路一起停。模拟控制里一个调节器坏只影响一个回路。**可靠性从“分散”变成“集中”，必须用冗余与降级来补**。

四条代价里，前两条是“物理税”，付了就完了；后两条是“工程债”，要持续投入。**很多项目低估的是后两条**。

85.11.3 与模拟控制的定量对比

表 85-19 把两种实现放在一起比。

表 85-19 模拟控制与数字控制的对比

维度	模拟控制	数字控制	本章依据
相位裕度	无采样损失	掉 6.2°（$T_s = 20$ ms）	表 85-5
分辨率	连续	台阶 3.66×10^{-4}（12 位）	表 85-12
抗混叠	不需要	必须加滤波器	表 85-16
参数修改	拆机换件	写寄存器	—
复杂控制	难	易	—
故障模式	元件老化	软件缺陷 + 集中风险	—

表里“相位裕度”一行最能说明问题：**数字控制天生比模拟控制少 6.2°**，这是采样的税，无法避免。所以设计时不能按“模拟设计的裕度”来要求数字实现，要预留这 6° 的余量。

反过来说，如果某个应用对相位裕度极度敏感（比如高增益、弱阻尼的谐振对象），模拟控制仍然有优势。**这不是怀旧，是物理。**

85.11.4 与事件触发、无差拍的关系

本章讲的是“等周期采样”结构。另外两种结构值得知道，但不在本章展开：

事件触发：误差超过阈值才采样，平均采样率可以远低于等周期。好处是省通信、省计算；坏处是分析复杂（要在连续域与离散域之间切换），且最坏情况下的间隔可能很长。**适合慢变、通信受限的场合**（无线传感网、电池供电设备）。

无差拍（deadbeat）：设计控制器使闭环在有限拍内到达设定值。理论上最快，但需要精确模型，且对延迟与量化极敏感。**适合模型准确、执行器快的场合**（硬盘磁头定位、部分电力电子）。

三者的关系：**等周期是默认，事件触发是省资源，无差拍是抢速度**。本章的所有公式（半拍、sinc、量化）在另两种结构里也要用，只是记账方式不同。

85.11.5 什么时候不该用这套结构

四种情况要谨慎：

第一，带宽极高的场合。 本章表 85-14 显示，$\omega_c = 6283$ rad/s 的电流环，$T_s = 0.1$ ms 时半拍还有 18°。要再快就得上 FPGA 或模拟实现。**数字控制的带宽上限由“采样 + 计算 + 保持”三段时间决定，不是由 CPU 主频决定。**

第二，对相位裕度极度敏感的场合。 弱阻尼谐振、高增益回路，6° 的采样税可能直接吃掉全部余量。

第三，寿命极长、维护极难的场合。 软件会过时、编译器会失配、芯片会停产。模拟电路放三十年还能修，数字系统放三十年可能连固件都烧不进去。**这一条在核电、航天、老工业设备里是真实约束。**

第四，回路极少、逻辑极简单的场合。 一个温度开关、一个简单的限位控制，用模拟或继电器更便宜可靠。**不是所有控制都值得上计算机。**

85.11.6 本章在全书里的位置

本章是“计算机控制”部分的第二块：第 84 章讲“计算机会带来什么”，本章讲“结构怎么搭”，第 86 章讲“软件怎么跑”，第 87 章讲“跑在什么硬件上”，第 88 章讲“PLC 这种特殊结构”，第 89 章讲“回路之间怎么通信”。

本章的输出有三样，后面几章都要用：

- $\omega_c T_s \le 0.3$：第 86 章的调度周期、第 87 章的采样任务、第 88 章的 PLC 扫描周期，都用这条判据；
- **式 (85-27) 的总滞后分解**：第 89 章讲总线时，T_{bus} 这一项就是从这里接过去的；

- **式 (85-31) 的 CPU 占用率**：第 87 章选型、第 91 章边缘算力分配，都用这个口径。

本章不展开的三件事：离散域根轨迹设计（见第 39/40 章）、状态空间设计（极点配置与观测器见第 54 章，基础见第 3 章）、鲁棒离散控制（见第 57 章）。本章只讲“结构”，不讲“设计方法”。

85.11.7 十行判据：一个项目该不该这么搭

把全章压成十行，每行一个“能不能过”的问题。十行都过，这套结构就成立；有一行不过，就按该行指的位置回去改。

1. **带宽够不够？** ω_c 由调节时间反推（§85.3.1），本章 3.797 rad/s 对应 $t_s \approx 0.94$ s。
2. **采样比过线没有？** $\omega_c T_s \le 0.3$ 是底线，≤ 0.15 是设计点（§85.3.2）。本章 0.076。
3. **延迟数清没有？** 逐段列时间戳再换拍数（§85.1.8、§85.3.3），不足一拍按一拍。本章 $m = 1$。
4. **裕度够不够？** $PM \ge 45^\circ$、$GM \ge 6$ dB 是底线，工程上按 $PM \ge 55^\circ$ 设计。本章 63.2°、18.7 dB。
5. **增益余量够不够？** 工作增益不超过临界值 $1/3$（§85.6.8）。本章 2.0 对 17.31，占 11.6%。
6. **量程按工作范围设了吗？** 不是按传感器铭牌（§85.7.5）。本章 1.5 对应 $[0, 1]$ 波动。
7. **位数够不够？** 按式 (85-17) 反解再加 1–2 位余量；DAC 另按“台阶小于死区三分之一”验（§85.3.4）。
8. **抗混叠加了吗？** 采样前必须有模拟滤波，且 $f_c \ge 4\omega_c$（§85.9.1、§85.9.8）。本章 -1.7°。
9. **纹波验了吗？** 按连续输出测，不是只看采样点（§85.6.4）。本章 2.19×10^{-8}（双精度）与 1.45×10^{-4}（12 位）。
10. **CPU 与回路数算了吗？** 按式 (85-31) 估，单回路 1.5%、20 回路 30%（§85.10.1）。

十行里最容易被跳过的是第 5 行与第 9 行：前者需要知道临界增益，而临界增益要么仿真、要么手算（§85.6.9）；后者需要示波器看连续输出，而多数调试只盯上位机的采样曲线。**跳过这两行的代价，往往是“仿真很好、现场发抖”**。

十行判据不能替代计算，它的作用是**防止漏项**。一个项目做完，十个问题都能用数字回答，交付文档就自然写出来了（§85.10.10 的七个数字就在其中）；有一个问题答不上来，说明那一节还没走完。

85.11.8 三个行业里，五笔账的权重不一样

同样五笔账（半拍、延迟、量化、混叠、抖动），在三个行业里谁主导完全不同。**知道自己在哪个行业，就知道该先看哪一笔。**

过程控制（温度、流量、压力）。 主导项是**对象延迟与执行器时间常数**。温度回路 $T_s = 1$ s 时半拍只有 1.4°（§85.8.9），采样税几乎不存在；真正吃掉裕度的是气动阀的 2–10 s 与套管的 5–20 s。**所以这一行的工程师该把时间花在“延迟从哪来”，而不是“采样率够不够”**。量化的权重排在第二，因为 4–20 mA 变送器的 0.1% 精度已经限制了总精度。

运动控制（伺服位置、速度）。 主导项是**采样与延迟并列**。$T_s = 0.25$ ms、$\omega_c = 628$ rad/s 时半拍 4.5°、整拍 9.0°（§85.10.7），两项相加等于过程控制里全部的采样税。**这一行的工程师该把时间花在“把 T_s 再压一半”或“把计算挪进 FPGA”，而不是加编码器位数**——17 位编码器的 LSB 是 4.8×10^{-5} rad，早就过剩了。量化的权重在这里降到第三，混叠的权重最低，因为编码器输出本身就是数字量，不存在采样前的模拟带宽问题。

电力电子（电流环、PWM 变换器）。 主导项是**采样与计算延迟**，且判据最紧。$\omega_c = 6283$ rad/s 的电流环，$T_s = 0.1$ ms 时半拍还有 18°（表 85-14）——**这已经接近过程控制里“整个回路的采样税”**。所以这一行常见 20–50 kHz 的开关频率配 100 kHz 级的采样，计算必须在中点采样、下一个周期立即更新占空比，**延迟一拍就吃掉半个周期**。量化在这里通常不是瓶颈（PWM 分辨率 10–12 位够用），混叠也不算，因为 LC 滤波器本身就是低通。

三行放在一起看，规律很清楚：**带宽越高，账越集中在“时间”上；带宽越低，账越分散到“对象与传感器”上**。本章的公式对三行都用，但权重排序不同——**读这一章时，先确定自己的带宽在哪一档（§85.5.9），再决定哪几节值得精读。**

85.12 Python 仿真代码

85.12.1 文件分工

本章有两个脚本，分工与前面各章一致：

- `code/ch085_sim.py`：只算数，不画图。定义对象、离散化、裕度、闭环、十二段实验，`main()` 打印全部表格并跑两条自检。
- `code/ch085_figs.py`：只画图，不算数。从 `ch085_sim` 导入 `experiment_*` 与 `closed_loop`，把结果画成 9 张图，每张导出 SVG 与 PDF。

不把画图代码写进 sim，是为了让“数字”与“图”共用同一个函数：图上每个点都来自 sim 的返回值，不会出现“图上与表上不一致”。

85.12.2 四个核心函数

`plant()` 返回对象的状态空间。本章对象 $G(s) = 20/(s^2 + 12s + 20)$ 取可控标准形：

```
A = [[-12, -20],
     [  1,   0]]
B = [[1], [0]]
C = [0, 20]
```

`zoh(A, B, Ts)` 用增广矩阵算精确离散（式 85-8）：

```
M = zeros((n+m, n+m))
M[:n, :n] = A
M[:n, n:] = B
E = expm(M * Ts)
return E[:n, :n], E[:n, n:]
```

这个函数是本章所有离散计算的基础，裕度、谱半径、闭环都用它。

`margins(ts, delay, kp, ti, method)` 直接按 $z = e^{j\omega T_s}$ 代值算开环，不走 `dlti`。原因是控制器在 $z = 1$ 有极点，转传递函数时 scipy 会报 `BadCoefficients`；代值没有这个问题。返回值含 ω_c、PM、ω_g、GM。

`char_poly(ts, kp, ti, delay)` 拼闭环特征多项式 $den_C\, den_G\, z^m + num_C\, num_G$，再用 `np.roots` 求根。`spectral_radius` 就是对它取模最大值（式 85-22）。

85.12.3 闭环仿真的两个细节

细节一：采样间轨迹。 `closed_loop()` 不是按 T_s 一步步算，而是把每个 T_s 再细分 `sub=20` 步（图 85-8 用 40 步），用 $h = T_s/\mathrm{sub}$ 的精确离散传播。这样既能拿到采样点的值，也能拿到采样之间的连续轨迹，才能量出表 85-11 的纹波。

细节二：延迟用队列实现。 控制量先进长度为 m 的队列，再从队首取出去作用于对象：

```
pipe.append(u)
u_use = pipe.pop(0)
```

这个写法比“存历史数组 + 索引回退”清楚，也不容易越界。注意 $m = 0$ 时队列长度取 1，相当于不延迟。

85.12.4 十二段实验的索引

`report()` 打印十二段，编号 `[0]`–`[11]`，与正文表格一一对应，见表 85-20。

表 85-20 十二段输出与正文表格的对应

段	内容	对应表/图
[0]	基准指标	表 85-5
[1]	采样周期扫描	表 85-7、图 85-5
[2]	三种离散化	表 85-8、图 85-4
[3]	延迟拍数扫描	表 85-9
[4]	位数扫描	表 85-12、图 85-7
[5]	混叠	表 85-13
[6]	稳定域临界 K_p	表 85-10、图 85-6
[7]	采样间纹波	表 85-11、图 85-8
[8]	总滞后分解	表 85-15、图 85-9
[9]	多速率相位代价	表 85-14
[10]	抗混叠滤波器	表 85-16
[11]	位数预算	表 85-17

85.12.5 运行与输出

运行命令：

```
D:/Anaconda3/envs/py311/python.exe code/ch085_sim.py
```

正常输出约 90 行，末尾是自检行：

```
======================================================================
自检通过：IAE=0.2632/0.2642  PM 序列=[68.8 66.3 53.4  0.9]
```

自检有两条：第一条验 IAE 随 T_s 缩小收敛（相邻两档差 $< 2\%$，实测 0.2632 与 0.2642 差 0.38%）；第二条验 PM 随 T_s 增大单调下降（$68.8 \to 66.3 \to 53.4 \to 0.9$）。**两条都通过，说明离散、闭环、裕度三套代码是自洽的。**

85.12.6 三处值得自己改一改

改一：把对象换成三阶。 把 `plant()` 改成第 84 章的 $1/(0.2s+1)^3$，重跑 [1]，看 PM 的下降速度是否更快。预期更快，因为三阶对象在高频段相位掉得更陡。

改二：把 K_p 从 2 改成 8。 重跑 [6]，看 $T_s = 50$ ms 那一行 ρ 是否从 1.0172 涨到更大。这能直观感受“同样增益、不同采样周期”的差别。

改三：把 `sub` 从 20 改成 5。 重跑 [7]，看纹波是否变得“台阶状”。`sub` 太小会让采样间轨迹的测量失真，**这也是一个“离散化”的例子**：用 5 个点描述一个采样周期内的连续轨迹，本身就有误差。

85.12.7 出图脚本

`ch085_figs.py` 九张图，全部从 sim 取数：

- `fig01`：六块积木框图（`block_styles` 的 `canvas` / `block` / `wire` / `sum_node`）；
- `fig02`：采样频谱，左右两栏，手工构造基带与副本；
- `fig03`：ZOH 幅相，上幅下相；
- `fig04`：三种离散化阶跃，$T_s = 100$ ms；
- `fig05`：T_s 扫描双纵轴；
- `fig06`：稳定边界双对数；
- `fig07`：位数-纹波三条线；
- `fig08`：采样间纹波局部放大；
- `fig09`：滞后分解堆叠条形。

图内中文用纯文本、公式用 `$...$`，两者不写进同一字符串——这条规则在第 84 章踩过坑（混排会让中文字形变成方框），本章的出图脚本从一开始就遵守。

85.12.8 复现的两个锚点

如果只想快速核对“代码有没有跑对”，看两个数：

- 基准 IAE 0.2734（$T_s = 20$ ms、$K_p = 2$、$T_i = 0.5$）；
- 基准相位裕度 $PM = 63.2^\circ$（同一组参数）。

这两个数对得上，说明离散化、闭环、裕度三处都没错。**其余的表格都是在这两个数的基础上扫参数得到的**，锚点对了，扫出来的表不会错到哪去。

85.12.9 复现时最容易错的三处

自己重写一遍这套代码，最可能在这三处翻车。它们的共同点是**错了以后结果仍然“像个结果”**，不会报错。

第一处：量程口径按 $2R$ 代入。 若把 Δ 写成 $2R/2^N$，所有台阶与纹波会整齐地大一倍——而“大一倍”在数量级上不刺眼，很容易被当成“量化本来就该这么粗”。本章统一用满量程口径 $\Delta = R/2^N$，$R = 1.5$、12 位得 3.66×10^{-4}。**核对办法：把 8 位的数拿出来看，应是 $1.5/256 = 5.86 \times 10^{-3}$，若是 1.17×10^{-2} 就说明用错了口径。**

第二处：用传递函数算离散裕度。 把 $C(z)$ 与 $G_d(z)$ 各自转成 `tf` 再相乘，scipy 会抛 `BadCoefficients` 警告，因为 $C(z)$ 在 $z = 1$ 有极点，`tf` 的系数里出现了相消不干净的零。绕过它的正确做法是**按 $z = e^{j\omega T_s}$ 逐点代值**（`pi_z()` 与 `zoh()` 直接给复数），不要拼多项式。若已经拼了，得到的 PM 会偏离几度，且偏离量随 T_s 变化——**它看起来只是“精度差一点”，实际上是把积分器的低频行为算错了。**

第三处：延迟错位一拍。 $m = 1$ 的物理含义是“这一拍算出的 u 要到下一拍才起作用”。用历史数组实现时，容易写成 `u[k-1]` 直接用，或者在队列里多存了一格，结果等效 $m = 0$ 或 $m = 2$。**$m = 0$ 会让 PM 偏高 4.4°，$m = 2$ 会让它偏低 4.4°**（表 85-9），两个方向都不会崩，所以很容易带进交付。核对办法：给 K_p 一个很小的值，看闭环阶跃是否恰好延迟一拍起步——**这一拍在时间轴上看得见，比看裕度可靠。**

三处之外还有一处口径问题：**纹波是峰峰值还是标准差。**本章表 85-11 一律峰峰值（末段 $\max - \min$），若按标准差算会小 3–6 倍。§85.1.9 已经把这条写成约定，重写代码时最好在函数名里带上 `_pp` 后缀，让口径写在名字上而不是注释里。

85.12.10 本章小结

本章把“数字控制系统结构”拆成六块积木、三笔账、五个参数，并给出每一笔账的算法：

- **时间账**：半拍 $\omega_c T_s/2$ 是固定税，每拍延迟是累进税，两者相加就是全部相位损失（式 85-26、表 85-9）；
- **幅值账**：台阶 $\Delta = R/2^N$ 决定纹波下限，纹波近似按 2^{-N} 下降，但到 10 位之后收益递减（表 85-12）；

- **频带账**：混叠只改名不衰减，唯一解法是采样前加抗混叠滤波（表 85-13、表 85-16）。

三笔账的共同结论是：**结构参数要在设计前定，不能事后调**。T_s 定错，换离散化方法救不回来；量程定错，加位数救不回来；采样率定错，加滤波器也只能救一半。**这就是“结构”二字的重量**。

最后留三个数给读者自查：0.076、63.2°、$18.7\ \mathrm{dB}$。前两个说明本章这组参数“采得够快、裕度够用”，第三个说明它“增益还很有余量”。换成自己的回路时，这三个数的量级若与本章差得远，就该回头确认带宽档位与延迟构成——**三个数对得上，本章的表格才能直接借用**。

第86章 实时控制软件

86.1 动机与直观理解

控制工程师习惯把注意力放在算法上：用什么控制器、参数取多少、裕度够不够。但算法最终要变成一段在处理器上按时执行的代码。同一套 PID，写成裸机中断服务程序可以稳定跑十年，塞进一个带文件系统、带网络协议栈、带日志的通用操作系统里，可能每隔几天就抖一次。差别不在公式，在时间。

本章讨论的就是这件事：控制软件如何在确定的时间内完成确定的工作。它不讨论控制器怎么设计（那是第 44、55、58 章的内容），也不讨论计算机控制系统的整体结构（第 84、85 章已经给出分层与信号链），只讨论任务级的时间行为——谁先运行、运行多久、最坏会晚多少、晚了以后性能损失多少。

86.1.1 三个现场现象

现象一：每隔几分钟“顿一下”。 某包装线的送料轴，正常运行时跟随平稳，但每隔几分钟出现一次约 2 ms 的滞后，工件上留下一道可见的痕迹。机械间隙、编码器信号、伺服参数都排查过，最后落在软件上：新版本增加了一个 20 ms 周期的诊断日志任务，单次写 Flash 的耗时在坏块整理时从 1.6 ms 涨到 8 ms 以上，把整个系统的利用率推过了 1.0，最低优先级任务开始整拍失约。日志本身与控制无关，但它抢走了 CPU，控制任务在某一拍晚拿到了执行权。

现象二：通信一忙，电机啸叫。 某伺服驱动器的电流环挂在 100 μs 的定时中断上。工程师为了不丢 CAN 帧，把 CAN 接收中断的优先级设得比定时中断高。CAN 总线负载升到 60% 以后，电流环的采样时刻被推迟最多 60 μs，电机出现约 2 kHz 的啸叫。把两个中断的优先级调换、并在 CAN 中断里只做搬运不做解析之后，啸叫消失。

现象三：扫描周期“看起来”是 10 ms。 某小型 PLC 的扫描周期标称 10 ms，工程师在串口中断里做了一段字符串解析，最长耗时 3 ms。用示波器测输出刷新时刻，周期在 7 ms 与 13 ms 之间摆动。PID 的微分项按标称周期做差分，实际间隔却在变，微分增益等效放大了近一倍，阀门动作出现锯齿。

三个现象的共性：**代码逻辑没有错，错的是时间**。第一个是总负载超了，第二个是优先级排错了，第三个是把“平均周期”当成了“每一拍周期”。

86.1.2 时间是一个盒子

把控制周期想象成一个固定长度的盒子，长度为 T。盒子里要装三样东西：控制算法自身的执行时间 C（最坏执行时间，WCET）；被更高优先级任务与中断抢走的时间 I；以及留给未来的余量 S。三者的关系是

$$T \geq C + I + S \tag{86-1}$$

这个不等式看起来平平无奇，但它是整个实时控制软件设计的地基。工程师犯的绝大多数错误，都可以归到“某一项被低估”：C 按平均值估而不是最坏值，I 干脆被忽略，S 被当成“浪费”而压缩到零。三个低估叠加，盒子就装不下，表现为偶发失约——而偶发失约在时域曲线上往往只留下一个不起眼的小台阶，很容易被误判为机械或电磁干扰。

图 86-1 画出了一个控制周期的时序。上排是理想节拍，下排是实际执行：采样时刻被中断延迟推后了一小段（抖动 J），计算占去 C，输出更新之后到下一拍之间是余量。注意实际采样时刻并不是等间隔的，这个不等间隔会在 §86.6 折算成相位损失。

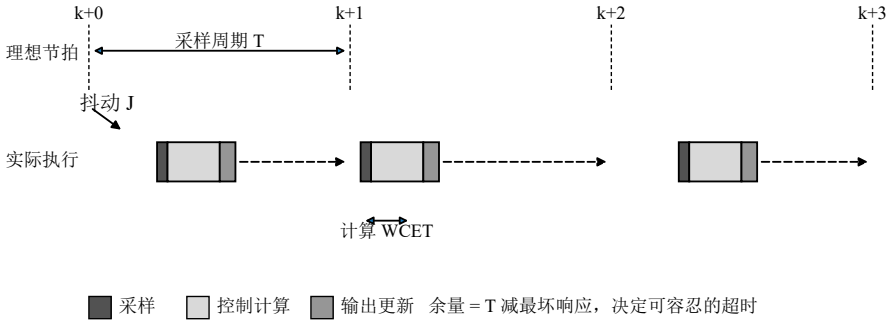


图 86-1　控制周期的时序与时间预算

表 86-1 给出本章示例任务集中速度环的实测分解。数字全部来自 §86.12 的仿真：周期 2.00 ms，WCET 0.30 ms，被高优先级任务与随机事件抢占 0.17 ms，最坏响应 0.470 ms，余量 1.530 ms。也就是说，这个回路只用了周期的 23.5%，剩下 76.5% 是安全垫。

项目	数值/ms	占周期	来源
周期 T	2.00	100.0%	设计取值
计算 C	0.30	15.0%	任务定义
抢占 I	0.17	8.5%	响应时间减 WCET
最坏响应 R	0.47	23.5%	响应时间分析
余量 $T-R$	1.53	76.5%	周期减响应

这张表要表达的不是“余量很充裕所以可以随便加功能”，而是“余量必须能被算出来”。算不出来的余量等于没有余量：一旦新增任务，I 会变，R 会变，而下一次评审时没人知道还剩多少。

86.1.3 硬实时、固实时与软实时

按“错过截止时刻的后果”分，实时任务有三类。

硬实时：错过一次就构成事故。电机电流环错过一拍，可能触发过流保护；安全监测错过一次，急停链路就断了。硬实时任务的判据是“最坏响应时间小于截止时间”，不是“平均响应时间很短”。

固实时：偶尔错过一次可以接受，但错过会导致数据失效。例如 1 ms 周期的编码器采样，偶尔晚 0.2 ms，速度估计的差分误差变大，但系统仍能运行；如果连续错过三拍，估计器就必须复位。

软实时：错过只影响服务质量。日志、诊断、界面刷新属于这一类。软实时任务的价值在于“不打扰硬实时”，它应该被放在最低优先级，并且在过载时第一个被牺牲。

分类的意义在于分配优先级与余量。硬实时任务拿最高优先级和最大的余量，软实时任务拿剩下的。把日志任务设成高优先级，等于让“记录发生过什么”压过“让事情正确发生”。

86.1.4 三个现象的定量回算

三个现象都可以用本章的式子反算，反算的过程也说明了这套方法的用途。

现象一的账：日志任务从 1.6 ms 涨到 8.0 ms，周期不变，利用率从 0.7075 涨到 1.0275。过载扫描显示，此时日志任务 10 个作业全部失约（§86.9 表 86-9）。而速度环失约数为 0——它仍然准时，只是“顿一下”来自日志任务占满 CPU

后，控制任务在某一拍被推迟到接近截止。这说明 $\lceil R_i/T_j \rceil$ 这一项不是抽象的：它数的是“这一拍里挤进来几个高优先级作业”。

现象二的账：电流环带宽 2236.1 rad/s，采样被推迟 60 μs。按平均延迟 30 μs 折算，相位损失 3.84°；按最坏 60 μs 折算，7.69°。相对 96.38° 的裕度，这个量不算致命，所以系统没有失稳，只是出现啸叫。啸叫的来源不是相位，而是采样间隔在 100–160 μs 之间摆动，等效采样频率在 6.25–10 kHz 之间变化，让环路的扰动抑制出现了随 CAN 帧周期移动的边带。

现象三的账：扫描周期在 7–13 ms 之间摆动，标称 10 ms。微分项按标称周期算，实际间隔 7 ms 时等效微分增益放大 $10/7 = 1.43$ 倍，即多 43%；间隔 13 ms 时缩小到 0.77 倍。增益在 $\pm 40\%$ 范围内随拍跳变，这已经超过多数回路能承受的参数摄动范围。本章 §86.6 的对照实验里，仅把分母从实测改成标称，超调就涨了 10.6%，而那个实验的抖动只有半个周期。

三次反算用的是同一套式子：式 (86-6) 算负载，式 (86-16) 算相位，式 (86-15) 与微分实现算增益误差。反算的价值在于把“感觉慢了”变成“晚了多少、折多少度、超多少调”，后者才能被验收。

86.1.5 本章的一句话与路线

如果只能记住一句话：**实时控制软件的验收标准不是“平均算得快”，而是“最坏情况下仍然准时，并且晚了以后知道代价”**。

§86.2 把“准时”写成可计算的式子：任务模型、响应时间方程、利用率上界、抖动折算。§86.3 给出六步设计流程，从需求拆到可调度性验证。§86.4 与 §86.5 分别从连续与离散两个视角看时间误差：连续视角给出带宽与延迟的换算，离散视角给出定时器、中断与代码实现的具体约束。§86.6 用实测数据说明抖动到底值多少相位，以及失约与性能的关系。§86.7 逐个参数讲影响方向，§86.8 给调参顺序与经验值，§86.9 讨论 EDF、优先级继承、核间隔离等改进手段。§86.10 是工程落地：WCET 怎么测、预算表怎么填、超差怎么处置。§86.11 说明这套方法的适用边界，§86.12 给出可复现的调度与抖动仿真。

阅读本章需要三个前置概念：采样与零阶保持（第 4、5 章）、相位裕度与截止频率（第 41 章）、离散闭环的稳定性判据（第 38 章）。本章不重复推导这些结论，只把它们换算成时间。

86.2 数学原理

本节把“准时”写成可计算的式子。顺序是：先给任务模型与四个假设，再给响应时间方程（判断可行性），然后用利用率上界给出快速判据，最后把时间误差折算成相位损失（判断性能）。前者回答“跑不跑得动”，后者回答“跑起来以后控制质量掉多少”。

86.2.1 任务模型与四个假设

一个周期任务用三元组描述：

$$\tau_i = (T_i,\ C_i,\ D_i), \qquad i = 1, \dots, n \tag{86-2}$$

其中 T_i 是周期，C_i 是最坏执行时间（WCET），D_i 是相对截止时间，本章取 $D_i = T_i$（截止等于下一拍释放）。任务在 kT_i 时刻释放，释放后进入就绪队列，由调度器按优先级选择执行。第 k 个作业的响应时间为

$$R_{i,k} = f_{i,k} - kT_i \tag{86-3}$$

$f_{i,k}$ 是完成时刻。若存在某个 k 使 $R_{i,k} > D_i$，称该作业失约。

本章的结论建立在四个假设上，这四条在 §86.9 与 §86.10 会被逐条放松：

1. **抢占式、固定优先级**，调度开销可忽略；

2. **WCET 已知且与负载无关**（不含缓存争用、总线仲裁的随机放大）；

3. **任务间无阻塞**（不共享互斥量，或共享时已用优先级继承解决）；

4. **释放时刻严格周期**（不考虑上位机下发的非周期指令）。

现场四条都容易破。破一条，下面的公式就要加修正项；破两条以上，公式只能当量级参考，必须靠实测。

86.2.2 响应时间方程

一个作业的响应时间由两部分组成：自己的执行时间，加上被更高优先级任务抢走的时间。高优先级任务在等待期间可能释放多次，释放次数是响应时间的向上取整函数，于是得到递归方程

$$R_i = C_i + \sum_{j \in \mathrm{hp}(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j \tag{86-4}$$

其中 $\mathrm{hp}(i)$ 是优先级高于 i 的任务集合。方程两边都含 R_i，用不动点迭代求解：从 $R_i^{(0)} = C_i$ 开始，反复代入右端。由于右端关于 R_i 单调不减，迭代序列单调递增且有上界（当 $R_i \le T_i$ 时），因此要么收敛到最小不动点，要么在超过 T_i 后判定不可调度。

用本章任务集算一遍。速度环的 $C = 0.30$，高于它的两个周期任务是 0.5 ms 周期、WCET 分别为 0.05 与 0.12：

$$R^{(1)} = 0.30 + \left\lceil \frac{0.30}{0.5} \right\rceil (0.05 + 0.12) = 0.47 \tag{86-5}$$

再代入一次右端仍得 0.47，收敛。总线收发任务的响应时间是 1.360 ms，日志任务是 5.470 ms，都小于各自的周期（4 ms 与 20 ms）。完整结果见表 86-3。

需要强调：R_i 是**最坏**响应，不是平均响应。仿真跑 200 ms 得到的平均响应是 0.517 ms，最坏是 1.310 ms，后者比前者大一倍半。设计必须按最坏值算，否则余量会在最不该出现的时候消失。

迭代过程本身也有信息。日志任务的迭代序列是 $1.60 \to 3.13 \to 3.94 \to 4.11 \to 5.13 \to 5.47$，六步收敛；总线任务是 $0.55 \to 1.19 \to 1.36$，三步收敛。序列里的台阶来自 $\lceil R/T_j \rceil$：当 R 跨过某个高优先级周期的整数倍时，右端会突然多出一整份 C_j，于是响应时间跳一级。日志任务跨过 0.5 ms 周期的次数最多，所以台阶最多；总线任务只跨过三轮，所以很快收敛。

台阶现象解释了一个反直觉的事实：**响应时间对参数的敏感度不是线性的。** 日志任务从 4.11 跳到 5.13 的那一步，三项取整同时各进一格，多出 $0.17 + 0.30 + 0.55 = 1.02$ ms，而输入的 C 一点没变。这就是 §86.7.3 所说“高优先级任务 WCET 是全局税”的几何来源：税不是按比例收的，是按“跨没跨过边界”收的。

86.2.3 利用率上界与优先级规则

把所有任务的负载加起来，得到利用率

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \tag{86-6}$$

本章任务集的周期部分为 $0.100 + 0.240 + 0.150 + 0.138 + 0.080 = 0.7075$（随机事件任务另计）。$U$ 是最快的粗略判据：它小于 1 才有可能调度成功，但小于 1 并不保证成功。

速率单调（RM） 规则是：周期越短，优先级越高。Liu 与 Layland 证明，在固定优先级、$D_i = T_i$ 的条件下，RM 是最优的——若存在某种固定优先级分配可行，则 RM 分配也一定可行。他们还给出了一个充分条件：

$$U \le n\left(2^{1/n} - 1\right) \tag{86-7}$$

$n=1$ 时上界为 1，$n=2$ 时 0.8284，$n=3$ 时 0.7798，$n=5$ 时 0.7435，$n=10$ 时 0.7177，随 n 增大单调下降并趋于 $\ln 2 \approx 0.693$。本章任务集 $U=0.7075<0.7435$，满足充分条件，因此一定可调度；这只是“一定行”，不满足时仍可能行，需要用式 (86-4) 逐个验算。

最早截止优先（EDF） 是动态优先级：每次调度选截止时刻最近的就绪作业。在 $D_i=T_i$ 时，EDF 的充要条件是 $U\le 1$，比 RM 宽松。图 86-3 把两条上界画在一起，本章任务集的位置用圆点标出。

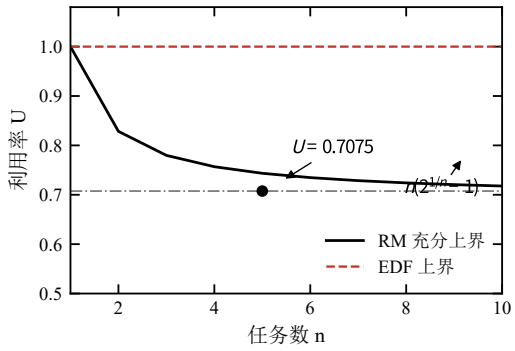


图 86-3　利用率上界与本章任务集位置

表 86-2 给出两条上界的关键差异。工程上选哪一个，不只看利用率，还要看实现代价：RM 只需静态分配优先级，任何 RTOS 都支持；EDF 需要每个作业维护截止时刻并做动态排序，且过载时的行为不如 RM 可预测——过载时 EDF 会出现“多米诺效应”，一次超时可能导致后续一连串失约。

项目	RM（固定优先级）	EDF（动态优先级）
优先级依据	周期，静态	截止时刻，动态
可调度充分条件	$U\le n(2^{1/n}-1)$	—
可调度充要条件	需用式 (86-4) 逐个验算	$U\le 1$
$n=5$ 时的上界	0.7435	1.0000
过载行为	低优先级整拍失约，行为可预测	连锁失约，难以预测
实现代价	低，静态优先级即可	中，需动态排序与截止时间管理

这张表给出的取舍是：硬实时回路优先 RM，因为过载时失约可预测、实现也便宜；只有当 U 已经超过 RM 的 0.7435、又确实需要把 CPU 用满时，才值得付出动态排序与截止时间管理的代价改用 EDF。

86.2.4 抖动折算成相位

理想采样时刻是 kT，实际时刻是

$$t_k = kT + \delta_k, \qquad 0 \le \delta_k \le J \tag{86-8}$$

J 是抖动上界。δ_k 使每一拍的控制量晚生效一小段时间，平均延迟为 $J/2$（均匀分布）。在频域里，纯延迟 τ 的传递函数是 $e^{-s\tau}$，在截止频率处的相位为 $-\omega_c\tau$，因此抖动带来的相位损失为

$$\Delta\varphi_J \approx \omega_c \frac{J}{2} \tag{86-9}$$

把本章两个回路代进去。速度环 $\omega_c=6.269$ rad/s，抖动取半个周期（1.00 ms），$\Delta\varphi=6.269\times 0.0005=0.00313$ rad $=0.18^\circ$。电流环 $\omega_c=2236.1$ rad/s，同样的 1 ms 抖动会给出 64°——回路早已失稳。这就是为什么“抖动占周期的百分比”不是判据：同样是 50%，对慢环无所谓，对快环致命。

反过来解出允许抖动：

$$J_{\max} = \frac{2\Delta\varphi_{\text{allow}}}{\omega_c} \tag{86-10}$$

取 $\Delta\varphi_{\text{allow}} = 5^\circ = 0.0873$ rad，速度环允许 27.8 ms，电流环只允许 78 μs；取 20° 时分别放大到 111 ms 与 312 μs。图 86-5 把这层关系画成横轴为抖动的直线族，一眼能看出两条回路相差三个数量级。

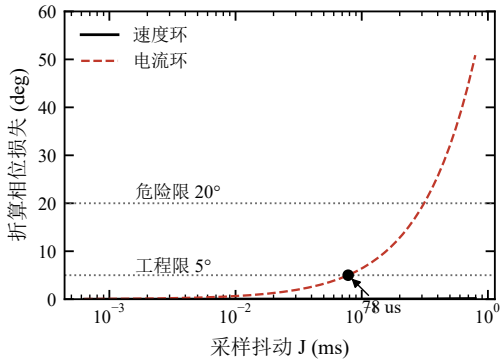


图 86-5　采样抖动折算的相位损失

86.2.5 时间误差的完整预算

抖动只是时间误差的一项。完整的等效延迟还包括采样保持本身与计算延迟：

$$\tau_{\text{eq}} = \frac{T}{2} + d + \frac{J}{2} \tag{86-11}$$

$T/2$ 来自零阶保持（第 4 章已证明：ZOH 的平均延迟是半个采样周期），d 是从采样到输出更新的计算与传输延迟，$J/2$ 是抖动的平均贡献。把式 (86-11) 代入相位，得到设计必须同时满足的约束组：

$$\begin{cases} R_i \le T_i, & i = 1, \dots, n \\ U \le U_{\text{bound}} \\ \omega_c \left(\dfrac{T}{2} + d + \dfrac{J}{2} \right) \le \Delta\varphi_{\text{allow}} \end{cases} \tag{86-12}$$

三组约束分别对应三类失败：算不完（失约）、总体超载（调度不可行）、算完了但控制质量不够（相位不足）。工程上最常见的错误是只检查第一组，把后两组留给“调试时再说”。

以速度环为例：$T = 2$ ms，$d = 0.3T = 0.6$ ms，$J = 1$ ms，$\tau_{\text{eq}} = 1.0 + 0.6 + 0.5 = 2.1$ ms，$\Delta\varphi = 6.269 \times 0.0021 = 0.0132$ rad $= 0.75^\circ$。电流环取 $T = 62.5$ μs、$d = 0.3T$、$J = 0.5T$，则 $\tau_{\text{eq}} = 31.25 + 18.75 + 15.6 = 65.6$ μs，$\Delta\varphi = 2236.1 \times 6.56 \times 10^{-5} = 0.147$ rad $= 8.4^\circ$。同一个设计流程，慢环几乎不设防，快环必须精打细算。

86.2.6 模型的盲区

以上公式干净，是因为假设干净。三处最常见的失真需要提前说明。

第一，**WCET 不是常数**。缓存命中率、分支预测、Flash 等待周期、总线仲裁都会让它随上下文变化。同一段代码在冷缓存下的执行时间可能是热缓存的 3 倍以上。式 (86-4) 里的 C_i 应取“可复现的最坏值”，而不是“测到的最大值”。

第二，**中断不服从任务模型**。中断服务程序（ISR）通常不参与优先级调度，它在任何时刻都能打断任务，包括正在执行临界区的任务。ISR 的执行时间必须计入 I，或者把 ISR 折算成一个周期极短、优先级最高的任务。

第三，**阻塞没有出现在式 (86-4) 里**。一旦任务间共享互斥量，低优先级任务持锁会阻塞高优先级任务，这就是优先级反转。修正办法是在响应时间方程里加一项阻塞时间 B_i，具体在 §86.9 讨论。

这三点决定了：公式用来做设计前的可行性判断，实测用来做投运前的验收。两者缺一不可。

86.2.7 利用率的三个陷阱

U 是全书出现频率最高的数字，也是最容易被误用的数字。三个陷阱要分别说清。

陷阱一：把充分条件当必要条件。 $U \le n(2^{1/n}-1)$ 只能证明“一定可行”，不能证明“一定不可行”。把速度环周期减半后 $U = 0.8575$，已经超过 $n=5$ 的上界 0.7435，但逐项验算仍然全部通过：速度环 0.470 ms、总线 1.830 ms、日志 7.820 ms，都小于各自的周期。此时系统可行，只是失去了“不用算就能保证”的便利。反过来，看到 U 超上界就宣布不可行，是常见的误判。

陷阱二：聚合口径不一致，漏算一大块负载。 本章的电流环本体是 16 kHz 的 ISR，每拍 15 μs，占 CPU 的 24%。任务表里没有这一项，因为它不在任务队列里。若因此认为“U 只算任务”，就会漏掉近四分之一的负载。

正确的做法是聚合建模：把 0.5 ms 内的 8 次 ISR 合成一个作业，$8 \times 15 = 120$ μs，恰好等于表 86-3 中“电流环保护”的 WCET 0.12 ms。所以那张表的 $U = 0.7075$ 已经包含了电流环的负载，没有漏算。这个巧合不是巧合，而是周期取 0.5 ms 的整数分频带来的必然结果——聚合周期是 ISR 周期的整数倍时，聚合负载严格等于原负载。

陷阱三：U 不含阻塞与非周期负载。 式 (86-6) 只统计周期任务的 C_i/T_i。随机事件任务按平均间隔 8 ms、每次 0.25 ms 计，额外贡献约 3.1%；若有临界区阻塞，还要加上 B_i/T_i。本章的真实负载约 0.739，而不是表面上的 0.7075。

三个陷阱的共同点是：U 是一个“算法简单、口径复杂”的量。算它只要一行加法，定义它的边界却要想清楚——哪些算、哪些不算、按什么周期聚合。口径写不清的 U，比没有 U 更危险。

86.2.8 判据的四个边界情形

式 (86-4) 与式 (86-19) 在纸面上很干净，落到具体任务集上会遇到四种边界情形。它们不影响公式的正确性，却直接影响“算出来的结论能不能用”。

边界一：等号成立不算通过。 判据写的是 $R_i \le D_i$，数学上等号可以通过。但 $R_i = D_i$ 意味着这个任务每一拍都用满整个周期，任何一次额外的中断延迟、任何一次缓存失效，都会立刻变成失约。工程上要求留出余量，经验值是 10% 以上。本章任务集的余量都很宽：电流环 $0.17/0.5$，余 66%；速度环 $0.470/2.0$，余 76.5%；日志 $5.470/20.0$，余 72.65%。反倒是最“不重要”的日志任务余量不小，因为它周期长。

边界二：$R_i > T_i$ 的物理含义。 式 (86-4) 在迭代中一旦发现 $R > T$ 就返回无穷大，这只是数学上的终止条件。物理上发生的事是队列积压：这个任务每一拍都比上一拍晚一点开始，未完成的作业越堆越多。若整体 $U<1$，积压不会无限增长，表现为“偶发丢一拍”；若 $U>1$，积压线性增长，最终必然崩溃。所以看到 $R>T$，第一步不是调这个任务的参数，而是先看 U——$U>1$ 是总量问题，$U<1$ 才是分配问题。

边界三：优先级必须唯一。 响应时间分析假设任意两个任务的优先级可比。若两个任务被配成同一优先级（不少 RTOS 允许这样，按时间片轮转），抢占关系就变成“谁先到谁先跑”，式 (86-4) 不再适用。保守的处理是把这两个任务合并成一个，C 取两者之和，T 取较小者，再按合并后的任务参与分析。合并会高估负载，但结论一定安全。反过来说，如果设计里出现了同优先级，说明任务划分本身还没做完。

边界四：同时释放的那一拍最坏。 本章六个任务的周期都是 0.5 ms 的整数倍，因此每个超周期起点上它们同时释放。这一拍里，日志任务要等前面五个任务全部执行完才能开始：$0.05+0.12+0.25+0.30+0.55 = 1.27$ ms。而它的平均等待远小于这个值。图 86-2 的甘特图第一列就是这种“齐步走”的形态。

这个现象给出一个不花钱的优化手段：**给任务加相位偏移**。把总线收发任务的相位从 0 改成 1 ms，它就不再与速度环挤在同一拍；把日志任务的相位改成 7 ms，它就落在电流环的空档里。相位偏移不改变任何任务的周期与 WCET，也

不改变 U，却能把最坏响应时间削下一截。代价是分析变复杂——带偏移的分析要检查所有临界瞬间，而不只是一个超周期起点。所以实践中通常只在“某一拍明显偏长”时才用，不作为默认手段。

四种边界情形合起来说明一件事：判据是必要工具，不是充分答案。真正决定系统能不能用的，是判据之外的余量、口径与相位安排。

86.2.9 四层数字的对账

本章的分析会产出四层数字：任务层的响应时间、拍层的时间戳、控制层的相位、性能层的超调。它们各自的来源不同、精度不同，但描述的是同一个系统的同一个毛病。工程上最有用的技巧，是把四层数字互相对账——任何两层矛盾，都说明某条假设被破坏了。

以本章速度环为例，四层的典型值分别是：任务层 $R = 0.470$ ms，来自式 (86-4) 的不动点迭代；拍层实测间隔 1.9–2.1 ms，来自采样时间戳序列；控制层相位损失 0.359°，来自式 (86-15)；性能层超调 4.663%，来自闭环阶跃仿真。

正常情形下，四层是自洽的：任务层给出 R，减去 C 得到被抢占的时间；拍层验证这个时间确实出现在时间戳里；控制层把拍层的抖动折算成相位；性能层确认这点相位对应这么点超调。

对账能查出三类问题。

第一类：任务层通过，性能层不合格。 响应时间全部在截止以内，超调却明显偏高。这时问题几乎一定在实现而不在时间：微分分母用了标称值、离散化多了一拍、输出更新滞后。诊断方法是 §86.8.5 的第一条——把抖动关掉重测，症状若不变，就与时间无关。

第二类：性能层正常，任务层超期。 波形看着很好，日志却报出超期计数。这种情况通常说明测量点打错了位置：时间戳打在函数入口而不是采样时刻，或者把被抢占的时间算进了自己的 C。诊断方法是把时间戳前移到中断入口，重新统计一次直方图，看超期是否消失。

第三类：拍层均值正确，极差很大。 平均间隔 2.000 ms，极差却有 0.4 ms。均值正确说明计时基准没问题，极差大说明有周期性抢占或总线阻塞。这类问题在任务层看不见（R 只给最坏值），在性能层也看不见（平均相位损失没变），只能在拍层的直方图里发现。诊断方法是对抖动序列做 FFT，找与总线、通信、DMA 同频的成分。

三类问题的共同点是：**单看一层永远查不出来**。任务层给的是上界，拍层给的是分布，控制层给的是折算，性能层给的是结果。上界合格不代表分布好看，分布好看不代表结果合格。四层都在，才能互相印证。

这也是本章把仿真与测量分开写的原因。仿真给任务层与控制层，测量给拍层与性能层；两者对不上时，先怀疑模型，再怀疑代码。若模型与代码都没问题，那就是有一层数字被读错了——而对账的过程，本身就是把读数重新校准一遍的过程。

86.3 设计步骤

§86.2 给了判据，本节给流程。流程的目标不是“设计出一个能跑的软件”，而是“设计出一个能在评审会上说清楚余量在哪里的软件”。共六步，前两步定义问题，中间两步量化，最后两步验证与留档。

86.3.1 第一步：把控制指标翻译成时间指标

控制器设计给出的是截止频率 ω_c、相位裕度 PM、采样周期上限。软件设计需要的是另一组量：每个任务的周期 T_i、截止 D_i、允许抖动 J_i、允许计算延迟 d_i。翻译关系就是式 (86-9) 与式 (86-11)：

$$J_i \le \frac{2\left(\Delta\varphi_{\text{allow}} - PM_{\text{loss}}\right)}{\omega_{c,i}}, \qquad d_i \le \frac{\Delta\varphi_{\text{allow}}}{\omega_{c,i}} - \frac{T_i}{2} \tag{86-13}$$

以本章速度环为例，$\omega_c = 6.269$ rad/s、$PM = 66.89^\circ$，若允许再损失 5°，则抖动上界 27.8 ms、计算延迟上界 13.9 ms 减去 1 ms，约 12.9 ms。这两个数字远超实际可能，说明速度环的时间指标很宽松，软件上几乎不会被时间问题卡住。

电流环就完全不同：$\omega_c = 2236.1$ rad/s，同样允许 5°，抖动上界只有 78 μs，计算延迟上界 39 μs 减半个周期（31 μs），只剩 8 μs 出头。这意味着电流环的采样、计算、输出必须在一个极短的时间窗内完成，任何一次内存缺页、Flash 等待、总线仲裁都会吃掉这份预算。

这一步最容易犯的错，是把所有回路按同一个标准对待。 慢环按快环的预算设计，浪费算力；快环按慢环的预算设计，现场出问题。正确做法是先算每个回路的 ω_c，再分别定预算。

86.3.2 第二步：划分任务

任务划分有两个极端。分得太细，任务切换开销与共享数据同步成本上升；分得太粗，一个任务里做了三件不同频率的事，周期只能按最快的定，浪费算力。

工程上按“频率 + 关键性”两个维度划分：

- 同一采样率、同一安全等级的功能合并为一个任务；
- 不同采样率的功能必须分开，否则快任务被慢任务的耗时拖累；
- 硬实时与软实时必须分开，且软实时放在低优先级。

本章示例任务集共六个任务：安全监测（0.5 ms，硬实时）、电流环保护（0.5 ms，硬实时）、随机事件处理（非周期，固实时）、速度环（2 ms，硬实时）、总线收发（4 ms，固实时）、日志诊断（20 ms，软实时）。六个任务的划分依据不是功能模块，而是时间尺度与失效后果。

需要单独说明：电流环本体的采样率是 62.5 μs（16 kHz），它由硬件定时器直接触发 ADC 与中断，不经过 RTOS 的任务队列，因此不出现在上表的任务集中。它的执行时间只有十几微秒量级，时间行为的分析方法与任务完全相同，但周期取 62.5 μs。后文凡涉及电流环带宽的例子，指的都是这个 62.5 μs 的中断服务程序；涉及调度表的例子，指的是 0.5 ms 的电流环保护任务。

86.3.3 第三步：分配周期

周期的选择遵循三条经验：

1. **与控制带宽挂钩。** $T \le 0.1/\omega_c$ 时相位损失约 2.9°，$T \le 0.2/\omega_c$ 时约 5.7°。速度环 $\omega_c = 6.269$ 对应 $T \le 16$ ms 与 32 ms，取 2 ms 有充分余量；电流环 $\omega_c = 2236.1$ 对应 $T \le 45$ μs 与 89 μs，取 62.5 μs 恰在两者之间。
2. **统一时基、整数倍关系。** 所有周期取同一个基准节拍的整数倍（本章取 0.5 ms），使超周期可枚举、调度分析可复现。非整数倍周期会让超周期急剧变长，仿真与验证都无法穷尽。
3. **避开机械与电气谐振。** 采样频率不要落在机械谐振频率附近，否则采样混叠会把谐振搬到低频，形成“越调越振”的假象。

三条之间会冲突。例如电流环按第 1 条可取 89 μs，但按第 2 条必须取 0.5 ms 的整数分之一，于是落到 62.5 μs。冲突时以第 2 条为准，第 1 条留出的余量正是用来吸收这种取整的。

86.3.4 第四步：测量 WCET 并填预算表

WCET 必须实测，不能靠代码行数估。测量方法在 §86.10 详述，这里只强调三条纪律：测最坏路径而不是平均路径；测冷缓存与热缓存两种状态；测中断同时发生的最坏叠加。

测完之后填表。表 86-3 是本章任务集的完整结果，R_i 由式 (86-4) 迭代得到，R_i/T_i 是判断可行性的直接指标。

任务	周期 T_i/ms	WCET C_i/ms	优先级	R_i/ms	R_i/T_i	类型
安全监测	0.5	0.05	1	0.050	0.100	硬实时
电流环保护	0.5	0.12	2	0.170	0.340	硬实时
随机事件	均值间隔 8.0	0.25	3	0.575	—	固实时
速度环	2.0	0.30	4	0.470	0.235	硬实时
总线收发	4.0	0.55	5	1.360	0.340	固实时
日志诊断	20.0	1.60	6	5.470	0.274	软实时

表中 R_i/T_i 全部小于 0.34，最紧的是电流环与总线收发（0.340）。这两个数字说明系统有较大余量，但也提示：**电流环的余量看起来是 0.66 个周期，其实不能这么理解**——R_i 是响应时间，不是执行时间，电流环真正占用的 CPU 只有 $0.12/0.5 = 24\%$。判断“还能不能加功能”，看利用率而不是看响应时间。

86.3.5 第五步：分配优先级

优先级分配有三条规则，按优先级从高到低依次适用：

1. **硬实时优先于软实时**。这条压倒一切，即使软实时任务周期更短。
2. **同类任务按 RM**：周期短的优先级高。本章安全监测与电流环同为 0.5 ms，安全监测更高，因为它错一次的后果更严重——这就是“关键性优先”对 RM 的修正。
3. **非周期任务插在合适的位置**。随机事件处理放在电流环之下、速度环之上：它不能被电流环拖延太久，但也不能拖慢速度环的采样。

分配完必须回头看利用率。本章 $U = 0.7075$，低于 $n = 5$ 的 RM 上界 0.7435，因此按 RM 分配一定可行。如果 U 落在 0.7435 与 1.0 之间，RM 上界不再保证，必须用式 (86-4) 逐个验算；验算通过就可行，验算不通过再考虑 EDF 或降低负载。

86.3.6 第六步：验证与留档

验证分三层，缺一层都不算完成。

第一层是解析验证：用式 (86-4) 算出所有任务的 R_i，与 T_i 比较。这一层在纸面上完成，五分钟能做完，但它只能证明“在假设成立时可行”。

第二层是调度仿真：按任务集生成作业序列，用固定优先级或 EDF 逐拍模拟，统计失约数、最大响应、响应抖动。图 86-2 是一次 20 ms 窗口的仿真结果：六个任务按优先级排开，电流环每 0.5 ms 出现一次，速度环每 2 ms 出现一次，随机事件在 4 ms 附近插入并把速度环推迟了一段。这张图的价值在于把“抢占”这件事画出来——响应时间方程里的 $\lceil R_i/T_j \rceil$ 项，在图上就是那些插进来的小方块。

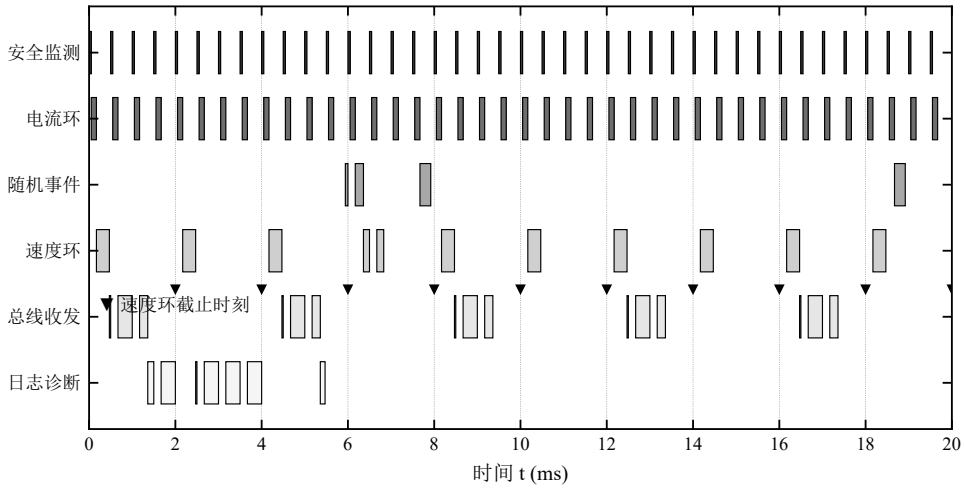


图 86-2　固定优先级调度甘特图

第三层是实测验证：在目标硬件上跑真实负载，用 GPIO 翻转或周期计数器记录每拍的实际执行时间，统计最大值、99.9 分位与抖动。仿真里 C_i 是常数，实测里 C_i 是分布，两者的差值就是缓存与总线带来的不确定性。

留档的内容包括：任务表（周期、WCET、优先级、截止）、响应时间计算结果、仿真脚本与结果、实测直方图、余量汇总。这份档案在下次加功能时是唯一的比较基准；没有它，“还剩多少余量”就只能靠猜。

86.3.7 流程中的两个常见偏差

偏差一：先写代码，后补调度分析。 代码写完再算响应时间，往往发现某一环超了，只能回头改架构。正确的顺序是先定任务与周期，再写代码，最后用实测替换估计值。

偏差二：把仿真当成验证终点。 仿真用的是常数 WCET 与理想中断，实测会遇到缓存争用、DMA 抢占总线、看门狗复位。仿真通过只说明设计思路对，不说明现场能跑。

86.3.8 把六步在电流环上走一遍

速度环的例子说明流程怎么走，电流环的例子说明流程在“极短周期”下会卡在哪一步。仍按六步来。

第一步，翻译指标。 电流环 $\omega_c = 2236.1$ rad/s，$PM = 96.38^\circ$。若只允许再损失 5°，抖动上界是 78 μs，计算延迟上界是 39 μs。这两个数字比速度环小三个数量级，因为分母多了一个 ω_c。

第二步，划分任务。 电流环本体不进任务队列，它是硬件定时器触发的 ISR。这一条很重要：ISR 没有“就绪队列等待”，延迟只来自中断响应与更高优先级 ISR 的嵌套。

第三步，分配周期。 $T \le 0.1/\omega_c = 44.7$ μs 对应 2.9° 损失，$T \le 0.2/\omega_c = 89.4$ μs 对应 5.7°。取 62.5 μs（16 kHz），落在两者之间，既满足相位要求，又能被 0.5 ms 的主时基整除（8 分频）。

第四步，测 WCET 并填预算。 实测 ISR 执行 15 μs，占周期的 24%。剩下的 47.5 μs 要分给 ADC 采样建立、PWM 死区更新与函数调用开销，逐项实测后余量约 38.5 μs。

第五步，分配优先级。 电流环 ISR 高于所有任务，但低于故障保护 ISR。故障保护的执行时间只有几微秒，却必须无条件抢占电流环——过流时晚一个周期关断，功率器件就可能损坏。

第六步，验证。 响应时间 $R = 15$ μs（无更高优先级 ISR 时），$R/T = 0.24$；相位裕度经零阶保持折算损失 4.00°、计算延迟损失 1.92°、抖动（实测 10 μs，取一半）损失 0.64°，合计 6.56°，折算后 $PM = 89.81^\circ$（见表 86-4）。

六步走完会发现，电流环的难点集中在第四步与第五步。周期、优先级都是设计时能定的，WCET 与中断嵌套却要在真实硬件上测。若测得 ISR 执行 25 μs 而不是 15 μs，占周期就从 24% 升到 40%，余量只剩 37.5 μs，其中还要容纳突

发中断。这时优化方向不是“再挤一点余量”，而是查这 $10\,\mu s$ 花在哪——多半是浮点运算或未加 `static` 的局部变量反复初始化。

86.3.9 把六步倒过来：接手旧项目时怎么反推

六步是设计流程，从指标到代码。真实工作里更常见的是反过来：接手一套没有文档的旧程序，需要判断它当初是怎么设计的、余量在哪里、能不能加功能。这时六步可以倒着走，用测量代替文档。

倒推第一步：测周期。 在任务入口打时间戳，跑一分钟，看相邻两次的间隔。稳定在 2.000 ms 的是周期任务；间隔服从指数分布的是事件任务；两者都不像的，说明用了相对延时，周期正在漂移。这一步能直接得到 T_i。

倒推第二步：测 WCET。 同一批时间戳，取“进入任务到写完输出”这一段的最大值，得到 C_i 的近似。注意是近似：这样测到的是“这一分钟里出现过的最坏值”，不是真上界。要用它做判据，得再加 10% 余量。

倒推第三步：推测优先级。 不看代码，只看抢占关系。给高优先级任务人为加长执行时间（在调试版本里插空转），观察哪些任务的响应时间跟着变长——跟着变长的都是低优先级。逐个做一遍，就能排出完整的优先级顺序。

倒推第四步：反算利用率与余量。 有了 T_i、C_i、优先级，式 (86-4) 与式 (86-6) 都能算。算出来的 R_i 与实测的响应时间对照，若两者接近，说明原设计是按这套模型做的；若差得远，说明原设计没做分析，或者有未建模的阻塞。

这套反推有三个典型发现，每一个都对应一种设计风格。

发现一：所有任务的 R/T 都接近同一个值。 这是按速率单调（RM）分配优先级的特征——周期越短优先级越高，于是各任务的 R/T 被压得比较均匀。看到这个形态，可以放心地按 RM 继续加任务。

发现二：某个任务的 R/T 特别小。 例如安全监测 0.10，其余都在 0.23 以上。这说明它被额外保护了，优先级高于 RM 的要求，或者它的截止时间被单独收紧过。这个任务不能动，动它等于拆掉一道保险。

发现三：某个任务的 R/T 超过 0.9。 它已经在失约边缘，而现场没报故障，说明实际负载比表里的小，或者它偶尔丢拍但没人注意。这一类任务加功能时最先崩，应当列为改造重点。

倒推的价值在于：它把“读代码猜设计”变成了“测行为推设计”。代码可以写得晦涩，行为不会撒谎。本章的六步与这套倒推是同一套模型的两种用法——正着用是设计，倒着用是审计。两者都掌握，才算真正会用这张任务表。

86.4 连续情形

§86.1 至 §86.3 讨论的都是“某一拍能不能按时完成”。还有一种更早、更粗的视角：把整条数字控制回路当成一个带纯延迟的连续系统，直接看延迟吃掉了多少相位裕度。这个视角不关心任务、优先级与截止时间，只关心一个量——从“应当采样”到“实际输出”之间隔了多久。

86.4.1 零阶保持等于半个周期的延迟

DAC 或 PWM 在两次更新之间保持上一个值，这个行为在频域里是零阶保持器。它的频率响应为

$$H_{\mathrm{zoh}}(j\omega) = \frac{1 - e^{-j\omega T}}{j\omega T} = e^{-j\omega T/2} \cdot \frac{\sin(\omega T/2)}{\omega T/2} \tag{86-14}$$

幅值项在 $\omega T \ll 1$ 时接近 1，相位项则精确等于 $-\omega T/2$。也就是说，零阶保持可以等效成“纯延迟半个采样周期”。速度环 $T = 2$ ms，$T/2 = 1$ ms，在 $\omega_c = 6.269$ rad/s 处贡献

$$\Delta\varphi_{\mathrm{zoh}} = \frac{\omega_c T}{2} = 6.269 \times 10^{-3}\ \mathrm{rad} = 0.359^\circ \tag{86-15}$$

电流环 $T = 62.5\,\mu s$，$T/2 = 31.25\,\mu s$，在 $\omega_c = 2236.1$ rad/s 处贡献 4.00°。同一个“半个周期”，慢回路只损失零点几度，快回路损失四度。差别不在时间上，而在 ω_c 上：相位是时间乘以频率。

86.4.2 计算延迟与抖动并入同一本账

计算延迟 d 是“采样完成”到“输出更新”之间的固定间隔；抖动 J 是实际采样时刻围绕理想时刻的随机偏移，它让每拍的延迟在 0 到 J 之间摆动，平均多出 $J/2$。三者串联，可以合并成一个等效延迟

$$\tau_{\mathrm{eq}} = \frac{T}{2} + d + \frac{J}{2} \tag{86-16}$$

对应的相位损失就是 $\omega_c \tau_{\mathrm{eq}}$。这个式子的价值在于：软件侧三个来源不同、单位相同的量被加成一个数，交给控制器设计者去核对相位裕度。表 86-4 是本章两个回路的分项账。

回路	T	保持 $T/2$	计算 d	抖动 $J/2$	τ_{eq}	相位损失	原 PM	折算 PM
速度环	2 ms	1 ms / 0.359°	0.6 ms / 0.216°	0.3 ms / 0.108°	1.9 ms	0.683°	66.89°	66.21°
电流环	62.5 µs	31.25 µs / 4.00°	15 µs / 1.92°	5 µs / 0.64°	51.25 µs	6.57°	96.38°	89.81°

速度环总损失 0.683°，相对 66.89° 的裕度可以忽略；电流环损失 6.57°，把 96.38° 拉到 89.81°，仍然充裕。这里的结论不是“时间问题不重要”，而是“这两个回路的周期选得足够短，时间误差还没有成为主导项”。

反过来算一次更有说服力。若把电流环周期从 62.5 µs 放宽到 250 µs（4 kHz），保持项变为 125 µs，贡献 16.02°；计算按比例涨到 60 µs，贡献 7.69°；抖动 20 µs，贡献 2.56°。三项合计 26.26°，PM 从 96.38° 降到 70.12°。周期只放大四倍，相位损失涨了近四倍——因为保持项与计算项都随周期线性变大，而相位是时间乘频率，两者相乘后成了平方关系。

86.4.3 抖动的不对称：慢环无所谓，快环要命

把同一份抖动加到两个回路上，结果完全不同。取 $J = 50$ µs，按平均延迟 $J/2 = 25$ µs 计入：速度环损失 $6.269 \times 2.5 \times 10^{-5} = 1.57 \times 10^{-4}$ rad，即 0.009°；电流环损失 $2236.1 \times 2.5 \times 10^{-5} = 0.056$ rad，即 3.20°。同一份抖动，折算到两个回路的相位损失相差 356 倍，这个倍数正是两个截止频率之比。

§86.2 的图 86-5 把这条关系画成两条直线，斜率就是各自的 ω_c。图上的 5° 工程限与电流环交在约 78 µs，与速度环交在约 27.8 ms；20° 危险线与电流环交在约 312 µs。读这张图的方法是先找自己的回路曲线，再向右看“当前的抖动落在哪一段”。对速度环，只要抖动不超过毫秒级，这条线永远碰不到；对电流环，几十微秒的抖动就已经进入需要解释的区间。

还有一层影响不体现在相位上：微分项用相邻两拍之差近似导数，抖动会让这个差值的噪声被放大。名义上 $\dot{y} \approx (y_k - y_{k-1})/T$，若 T 是标称值而实际间隔在抖，等效的微分增益就随拍变化。仿真里这一项的破坏力比抖动本身还大一个数量级，具体数字放在 §86.6 讨论。

86.4.4 连续近似的适用边界

式 (86-14) 的幅值项做了 $\sin x / x \approx 1$ 的近似，误差量级为

$$\frac{\sin(\omega T/2)}{\omega T/2} \approx 1 - \frac{(\omega T)^2}{24} \tag{86-17}$$

电流环 $\omega_c T = 0.1398$，误差约 0.08%；速度环 $\omega_c T = 0.0125$，误差可以忽略。工程上把 $\omega_c T \le 0.5$ 当作这条近似可用的界线，此时相位误差不到 2%。若某个回路的 $\omega_c T$ 超过 1，就不能再用“半个周期延迟”这个说法，必须回到 z 域精确计算。

连续视角还有三件事看不见。第一，它看不见截止时间：一个任务的平均延迟合格，某一拍却可能晚了三倍。第二，它看不见优先级：延迟是“平均摊派”的，而实际延迟由调度决定，高优先级任务从不迟到，低优先级任务可能连续被推。

第三，它看不见失约的分布：过载时系统不是均匀变慢，而是先丢某一类作业。这三件事都要靠 §86.5 的离散视角补上。

一句话概括本节：连续视角回答“会不会振”，离散视角回答“会不会误拍”。前者给预算，后者给保证，两者都不能省。

86.4.5 反着用：从相位预算反推允许抖动

前面几节都是“给定抖动，算相位损失”。工程上更常用的是反方向：给定允许的相位损失，反推这个回路能容忍多大的抖动。把式 (86-15) 的相位项反过来解，取平均延迟 $J/2$，得

$$J_{\max} = \frac{2\varphi_{\max}}{28.65\,\omega_c} = \frac{\varphi_{\max}}{14.325\,\omega_c}. \tag{86-18}$$

式中 $J_{\max}$ 与 $\varphi_{\max}$ 分别以秒与度为单位。这条式子把“相位”这个控制语言翻译成“微秒”这个实现语言，一张表就能看清各环的处境。

取相位预算 $\varphi_{\max} = 2.5^\circ$（约为 66.89° 裕度的二十七分之一），两个回路的答案差别极大：

- 速度环 $\omega_c = 6.269$：$J_{\max} = 27.8$ ms，相当于 14 个采样周期。
- 电流环 $\omega_c = 2236.1$：$J_{\max} = 78$ μs，相当于 1.25 个采样周期。

这两个数与 §86.7.7 给出的允许值完全一致，而它们的含义截然不同。速度环的 27.8 ms 意味着“抖动几乎不可能成为约束”——哪怕中断延迟到了毫秒级，相位损失仍不到 2.5°。电流环的 78 μs 意味着“抖动预算是硬约束”——一次总线仲裁、一次 Flash 等待，就可能吃掉一半。

反推的用处有三层。

第一层，**判断要不要优化**。把实测抖动除以 $J_{\max}$，得到占用率。本章速度环 $0.84/27.8 = 3\%$，电流环 $10/78 = 13\%$，两者都在安全区，说明当前的硬件配置够用，不必为抖动换器件。若某个回路占到 50% 以上，才值得投入。

第二层，**判断瓶颈在哪**。若速度环的占用率也上到 50%，而它的 ω_c 只有 6.3 rad/s，那就说明抖动已经大到毫秒量级，问题一定出在调度而不在硬件——先查优先级与阻塞，而不是换晶振。

第三层，**给新回路定预算**。若将来加一个位置环，ω_c 取 60 rad/s，用同一条式子得 $J_{\max} = 2.9$ ms。这个数字可以直接写进设计约束，不需要重新推导。

式 (86-18) 还有一个容易被忽略的性质：$J_{\max}$ **与** ω_c **成反比**。带宽提一倍，允许抖动减一半。这意味着“提高带宽”这个看似纯粹的控制决策，会同时收紧实时约束。设计时若把速度环带宽从 6.3 rad/s 提到 12.6 rad/s，允许抖动从 27.8 ms 掉到 13.9 ms，原本“无论如何都够用”的结论就不再成立了。带宽与控制周期的关系，从来不是两个独立的旋钮。

86.4.6 参考路径上的延迟不算进裕度

前面所有延迟都发生在反馈通道：采样晚了一点、计算晚了一点、输出晚了一点。它们都在闭环之内，所以吃相位裕度。还有一类延迟发生在参考通道——指令从总线传进来、从轨迹规划器算出来，它不进闭环，因此不吃裕度，但会造成跟踪滞后。两者不能用同一个预算，混淆会同时导致过度设计与精度不达标。

先看数字。本章总线收发任务周期 4 ms，一帧传输约 0.5 ms，因此一条新指令从“上位机发出”到“进入控制器”最坏滞后 4.5 ms。若把它当作反馈延迟代入式 (86-15)，速度环损失 $6.269 \times 4.5 \times 10^{-3} = 0.028$ rad，即 1.6°；这个数很小，小到容易被当成“可以忽略”。但把它当成零，同样是错的——它不体现在裕度上，而体现在跟踪误差上。

对匀速指令，稳态跟踪滞后等于延迟乘以速度。取 $v = 100$ mm/s，4.5 ms 的参考延迟对应 0.45 mm 的固定滞后。这个误差不会随时间衰减，也不会被积分项消除——积分项消除的是常值误差，而这里是“指令在动、输出追不上”，属于速度型误差。定位精度要求 0.1 mm 的场合，这 0.45 mm 直接不达标。

处理这类延迟有三条路，与处理反馈延迟完全不同。

第一条，提高指令更新率。 把总线周期从 4 ms 缩到 1 ms，滞后降到 1.5 ms，误差降到 0.15 mm。这条路的代价是总线负载与通信任务 WCET 上升，属于 §86.7.8 说的“联动变更”，要整体重算。

第二条，加前馈。 既然指令是可预知的，就把指令的一阶、二阶差分作为前馈项直接加到输出上，让执行器“提前动”。前馈不经过反馈环，因此不受稳定性约束，能把跟踪误差压到接近零。代价是前馈增益依赖对象参数的准确性，参数失配时会引入新的误差。

第三条，指令平滑。 把阶跃指令改成一阶或二阶滤波后的斜坡，让指令本身的变化率受限。这条路不减少延迟，但把“追不上”变成“追得上”，副作用是响应变慢。

三条路的共同点是：**它们都不改变相位裕度**。这一点常被误判——有人算出参考延迟 1.6°，就把它从 66.89° 的裕度里扣掉，得出 65.29°，然后据此放慢带宽。这是把两类延迟混为一谈。正确的口径是：反馈延迟进裕度预算，参考延迟进跟踪误差预算，两本账分开记。

对高带宽的位置环，这个区分更重要。位置环 ω_c 取 60 rad/s，4.5 ms 的延迟若被误当成反馈延迟，折算出来是 15.5°，足以让人做出“必须降低带宽”的错误决策；而实际上它只是 0.45 mm 的跟踪滞后，用前馈就能解决。同一份延迟，记错账的代价是牺牲了整个环的动态性能。

86.5 离散情形

连续视角把延迟摊成平均值，离散视角必须回答“具体哪一拍”的问题。控制器在软件里是一段以固定节拍运行的代码，采样只在节拍边界发生，输出只在节拍边界更新，延迟不是连续量，而是整数倍的节拍。本节讨论这种离散化给时间分析带来的三处变化：延迟的归属、抖动的形态、判据的形式。

86.5.1 采样把时间切成格点

一个 2 ms 周期的速度环，在 1 s 内只有 500 个决策时刻。若控制量在 $t = 2.000$ ms 算出、$t = 2.001$ ms 才写入 PWM，这 1 μs 的差别在连续模型里是 0.0006° 的相位，在离散模型里却可能整整晚一拍——如果 PWM 的更新窗口在 2.000 ms 关闭，输出就要等到 4.000 ms。离散系统的分辨率是节拍，不是微秒。

这件事的直接后果是：**同一个平均延迟，量化后的效果可能差一倍。** 平均延迟 0.3 ms、节拍 0.5 ms，量化后有时 0 拍、有时 1 拍；平均看起来是 0.6 拍，实际每一拍非 0 即 1。对控制器而言，这不叫“平均延迟 0.3 ms”，而叫“延迟在 0 与 0.5 ms 之间跳变”，后者才是它真正感受到的扰动。

86.5.2 离散化方法决定延迟归属

连续控制器 $C(s)$ 要变成代码，有三条常见路径，它们对延迟的记账完全不同。

路径一：直接差分实现。 把 s 换成 $(1 - z^{-1})/T$，写成 $u_k = u_{k-1} + K_p(e_k - e_{k-1}) + \cdots$ 的形式。这种实现里 u_k 只用到当前拍与上一拍的量，没有额外状态延迟。

路径二：先离散成状态空间，再迭代。 用零阶保持把 $C(s)$ 转成 (A_c, B_c, C_c, D_c)，代码按 $x_{k+1} = A_c x_k + B_c e_k$、$u_k = C_c x_k$ 执行。由于 $D_c = 0$，u_k 只依赖 x_k，而 x_k 由 e_{k-1} 算出，等效多了一拍延迟。

路径三：先离散成 z 域传递函数，再用直接型实现。 延迟取决于实现结构，也与 b_0 是否为零有关。判断方法很朴素：逐行看代码，确认 u_k 到底依赖哪一拍的 e。

三条路径的差别不是学术问题。同一个速度环、同样 $T = 2$ ms，路径一在仿真里的超调是 4.660%，路径二（按 ZOH 状态空间实现）是 9.913%，调节时间从 1.200 s 变成 0.986 s。超调翻了一倍多，原因只是少写了一拍。工程上判断“我的实现属于哪条路径”，比争论“离散化误差有多大”更有用。

86.5.3 离散化后的相位损失可以精确核对

式 (86-15) 的预测在 z 域能直接验证。把开环 $L(s)=C(s)G(s)$ 按零阶保持离散，用离散系统的相位裕度定义重算，得到表 86-5。

采样周期 T	式 (86-15) 预测损失	离散实测 PM	与连续值之差
0.5 ms	0.090°	66.80°	0.09°
1.0 ms	0.180°	66.71°	0.18°
2.0 ms	0.359°	66.53°	0.36°
4.0 ms	0.718°	66.17°	0.72°
8.0 ms	1.437°	65.45°	1.44°

五行的偏差都在 0.01° 以内，说明“半个周期延迟”这个说法在 $\omega_c T \le 0.5$ 的范围内足够准。这张表还有一个用法：它告诉你采样周期每翻一倍，相位裕度损失也翻一倍。若某个回路只剩 5° 余量，T 就没有翻倍的余地。

需要注意的是，表 86-5 的“实测”仍来自传递函数离散化，不是逐拍代码仿真。两者在第 86 章的仿真里会对得上，但前提是代码实现没有额外引入延迟——这正是 §86.5.2 强调的那件事。

86.5.4 抖动在离散域的三种形态

连续模型里抖动是一个标量 J。到了离散域，它分裂成三个不同的量：

1. **采样时刻抖动** J_s：ADC 触发时刻相对理想节拍的偏移。它改变相邻两拍的间隔，直接影响差分项。
2. **输出更新抖动** J_o：计算完成到 PWM 寄存器写入的偏移。它决定控制量实际生效的时刻。
3. **计算延迟的量化** d_q：从“采样完成”到“计算完成”的间隔，按节拍向上取整。

三者的合成延迟是 $d_q + J_o$，而 J_s 通过差分项起作用。工程上常见的错误是把三者合成一个 J 丢进式 (86-16)，结果既高估了相位损失，又漏掉了差分噪声。正确的做法是分开测量：J_s 用输入捕获测，J_o 用 GPIO 翻转测，d_q 用周期计数器测。

对本章速度环，仿真得到的响应抖动约 0.84 ms（占周期 42%），J_o 约 0.1 ms，d_q 为 1 拍（0.5 ms 节拍下的 0.6 ms 计算）。若把 0.84 ms 当作 J 代入式 (86-16)，平均延迟增加 0.42 ms，相位损失 0.15°；若只用 $J_o = 0.1$ ms，平均延迟增加 0.05 ms，损失 0.02°。两个数相差近八倍，但它们影响的机制不同，不能互相替代。

86.5.5 判据从“相位”换成“响应时间”

连续视角的合格判据是相位裕度为正，离散视角的合格判据是每个作业的响应时间不超过截止时间：

$$R_i \le D_i, \qquad i = 1, 2, \dots, n \tag{86-19}$$

这两个判据不等价。相位裕度合格只说明平均意义上的闭环稳定，$R_i \le D_i$ 合格才说明没有任何一拍被漏掉。图 86-4 把本章五个周期任务的 R_i/T_i 画成柱状，柱顶离 1 的距离就是各任务的余量。

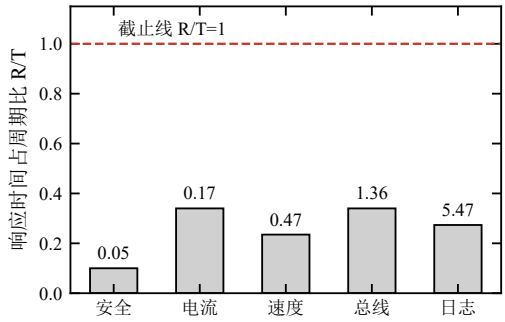


图 86-4　各任务响应时间与截止时间的比值

图上最高的是电流环保护与总线收发，都是 0.340；最低的是安全监测，0.100。这个排序与优先级顺序一致——高优先级任务的响应时间几乎等于它自己的执行时间，因为它很少被抢占。反过来，日志诊断的 R_i 是 5.470 ms，是它自己 $C_i = 1.60$ ms 的三倍多，多出来的部分全是被高优先级任务挤占的时间。

判据的形式还决定了“加功能”时该看哪个数。加一个短周期任务，看的是被它抢占的那些任务的 R_i 涨了多少；加一个长执行任务，看的是利用率 U 涨了多少。前者用式 (86-4) 重算，后者用式 (86-6) 重算，两个数都要过线。

86.5.6 离散视角的边界

离散视角能精确到拍，但它仍然假设每拍的执行时间是常数 C_i。真实处理器上 C_i 是分布：缓存命中时 0.28 ms，未命中时 0.34 ms，DMA 争用总线时可能 0.45 ms。把最大值当 C_i 是保守的，把平均值当 C_i 是危险的。这就是 §86.10 要讲实测 WCET 的原因。

另一个边界是中断。§86.2.6 已经指出，中断服务程序不服从“周期任务 + 固定 WCET”的模型，它的到达时刻由外部事件决定，执行时间取决于事件类型。离散视角只能把它折算成一个阻塞时间 B_i 加到式 (86-4) 里，或者干脆把它当作最高优先级的任务单独分析。

86.5.7 一个逐拍算例：抖动怎样进入差分项

把 §86.5.2 的结论落到具体的数字上。设标称周期 2 ms，理想采样时刻是 $0, 2, 4, 6$ ms；实际采样时刻是 $0.0, 2.4, 3.8, 6.7$ ms，间隔依次为 2.4、1.4、2.9 ms。对象输出是斜坡 $y = t$，真实的导数是 1.000。

用标称分母计算微分：三拍的估计值分别是 $2.4/2 = 1.20$、$1.4/2 = 0.70$、$2.9/2 = 1.45$。真值是 1.000，误差在 -30% 到 $+45\%$ 之间摆动。用实测分母计算：$2.4/2.4 = 1.00$、$1.4/1.4 = 1.00$、$2.9/2.9 = 1.00$，三拍全部准确。

这个算例的用处是把“实现失误”从抽象说法变成可检验的动作。它同时说明两件事。第一，抖动的破坏力不在“晚了一点”，而在“分母用错了”——同一组采样时刻，两种实现的微分误差相差三个数量级。第二，误差的方向与间隔的大小相反：间隔越长，标称分母给出的估计越大，而实际的斜率并没有变。

§86.6 表 86-6 里 4.663% 与 5.159% 的差别，微观来源就是上面这三拍。区别只在于表里的对象是二阶系统、输入是阶跃，所以误差经过积分后被平滑，最终表现为超调多半个百分点。

86.5.8 两个台架实验：判定延迟归属与分母口径

§86.5.2 说“延迟取决于实现结构，要逐行看代码确认”，但代码常常是别人写的、或者由工具自动生成，读起来费时且容易看错。有两个简单的台架实验，可以在十分钟内把这两件事测出来，不需要仿真器，也不需要额外仪器，只用总线读回的占空比或电流指令。

实验一：阶跃定延迟。 把积分增益置零，只留比例与微分，给定一个小的阶跃指令（例如额定值的 5%，确保不进入限幅），然后逐拍记录控制量。判据很直接：若第 k 拍的指令在第 k 拍就体现为输出，说明没有附加延迟；若第一拍

输出为零、第二拍才跳到 K_p 倍，说明实现里多了一拍。

这个“多一拍”可能来自三个地方：微分项用了上一拍的误差、状态空间迭代把输出写在更新之后、或者直接型结构里 $b_0 = 0$。三者的外部表现完全一样，实验无法区分，但也不需要区分——对闭环来说，它们等价于同一个纯延迟。

要让这个实验有意义，必须选对被测的环。同样是“多一拍”，在速度环上（周期 2 ms、$\omega_c \approx 6.3$ rad/s）只值 0.72°，在电流环上（周期 62.5 μs、$\omega_c = 2236$ rad/s）值 8.02°。前者可以忽略，后者相当于把 66.89° 的相位裕度削去一成二。所以延迟测试要在最内环做，在速度环上做等于用一把量程太大的尺子量小尺寸。

实验二：插拍定分母。 这个实验专门对付“微分分母写死成标称周期”这种失误。做法是在某一拍人为制造一次延迟：在该拍的采样之后、计算之前插入一段确定的空转（例如 0.4 ms 的忙等，或在调试口上触发一次已知长度的等待），让这一拍的实际间隔变成 2.4 ms，其余各拍保持 2 ms。

若微分分母用的是实测间隔，控制量序列看不出这一拍有任何异常——分子分母同时变成 2.4，比值不变。若分母写死成 2 ms，这一拍的微分项会凭空放大 20%：以 $K_d = 0.5$、误差斜率 1 为例，正常拍贡献 $0.5 \times 1 = 0.5$，被拉长的那一拍贡献 $0.5 \times 1.2 = 0.6$，输出上能看到一个宽 2 ms、幅度 0.1 的尖峰。这个尖峰随后会被积分项吸收，不会持续，但在示波器上很清楚。

两个实验的共同点是：都不需要知道代码长什么样，只需要能读回控制量，并且能控制住输入。这正是“行为测试”相对于“代码审查”的价值——代码审查能告诉你“应该是怎样”，行为测试告诉你“实际是怎样”。两者不一致时，以行为为准。

还有一个容易忽略的前提：两个实验都必须在**未限幅**的区间做。一旦输出饱和，控制量被截断，所有关于比例关系的判据都失效。工程上常见的错误是在大阶跃下测延迟，看到“输出第二拍才动”，其实是因为第一拍就已经饱和、第二拍才退出饱和。选小信号，是这个实验唯一需要操心的细节。

86.5.9 定点实现的量化噪声

前面的分析都用实数，实际代码里常常是定点。定点并不改变时间关系，但它会给微分项注入一种新的噪声，而这类噪声与控制周期强相关，正好落在本章的主题上。

先看分辨率。电流采样范围 ± 20 A，用 Q15 表示归一化到 ± 1，一个最低位对应 $40 / 32768 = 1.22$ mA。这个分辨率对电流环够用：额定电流几十安，1.22 mA 的相对误差在万分之几量级。

问题出在微分。一阶差分对高频的增益是 $2/T$，周期越短增益越大。$T = 2$ ms 时增益是 1000 s^{-1}，一个最低位的量化台阶就能在微分输出上产生 1.22 A/s 的跳变；若再乘上 $K_d = 0.5$，控制量上就是 0.61 的纹波。这个纹波与真实信号无关，纯粹来自量化，而它的幅值与 T 成反比——把周期减半，噪声翻倍。这与 §86.6.9 的相位收益正好相反：加密采样换来 2° 相位，同时换来两倍的微分噪声。

处理办法有三条，可以叠加。**第一，微分项加一阶低通**，就是式 (86-2) 里的 $T_f = 0.01$ s，它把 $2/T$ 的高频增益压回 $1/T_f$ 量级。本章的 T_f 是 10 ms，比采样周期大五倍，噪声衰减到五分之一。**第二，用增量式而非位置式**，让微分直接对增量做差，避免大数相减的抵消误差。**第三，提高内部位宽**，采样用 12 位，累加与中间运算用 32 位，只在输出时降到 16 位。

积分器是另一处容易踩坑的地方。位置式积分每拍累加 $K_i T\, e_k$，K_i 取 50、$T = 2$ ms、误差满量程时，单拍增量是 0.1，从零累加到饱和需要十拍左右。若累加器也用 Q15（范围 ± 1），十拍就溢出。溢出的表现有两种：饱和型是积分卡住不动，回绕型是符号突然翻转——后者会让控制量瞬间反向，是最危险的一种。所以积分累加器至少要比输出宽 8 位以上，本章的实现用 32 位。

还有一个不显眼但很常见的错误：**所有系数统一用 Q15**。本章 $K_p = 2.5$，超过 Q15 能表示的最大值 1，统一 Q15 会直接把它截成 1，控制器增益凭空少了六成。正确做法是按系数的动态范围分别定标：K_p 用 Q2.13（范围 ± 4，分辨率 1.2×10^{-4}），K_i 与 K_d 用 Q4.11。系数的定标是设计决策，不是编码细节，应当在任务表之外单独列一行。

把这四条合起来，定点实现的原则是：**时间关系照旧，位宽单独设计**。抖动与延迟的分析不需要改，但微分噪声、积分溢出、系数定标这三件事必须单独过一遍。它们不出现在任何时序图里，却经常是“仿真好好的、上机就抖”的原因。

86.6 稳定性与性能

前面三节建立了时间与相位、时间与截止的换算关系，但都没有回答一个更实际的问题：抖动到底会让控制性能变差多少。本节用速度环做一组对照仿真，把“抖动”与“实现失误”放在同一张图上比较，结论会比直觉更尖锐。

86.6.1 三种稳定性，三个判据

“系统稳定”这句话在实时控制里有三层含义，三层的判据互不相同。

第一层是连续意义下的闭环稳定：相位裕度为正、增益裕度为有限正值。本章速度环 $PM = 66.89^\circ$、$GM = \infty$，这一层毫无问题。

第二层是离散意义下的闭环稳定：闭环极点落在单位圆内。按 $T = 2$ ms 离散后，主导极点模为 0.9926，距单位圆 0.0074。换成连续域的衰减率是 $-\ln(0.9926)/0.002 = 3.70\ \mathrm{s}^{-1}$，对应时间常数 0.27 s。第二层与第一层通常一致，但采样周期加大时会分离：$T = 8$ ms 时极点模升到 0.9707，相位裕度降到 65.45°，两条曲线开始分岔。

第三层是调度意义下的可行：每个作业都满足 $R_i \le D_i$。这一层与前两层没有必然联系。一个闭环稳定的系统，若速度环任务偶尔失约，输出保持上一拍，等效延迟从 0.6 ms 跳到 2.6 ms，闭环就可能从稳定变成振荡。因此工程上不能只做频域校验，也不能只做调度校验。

三层的检查顺序应当是：先调度（能不能按时），再离散（采样够不够快），最后连续（相位够不够）。顺序反了会浪费时间——一个调度上不可行的设计，讨论相位裕度没有意义。

86.6.2 抖动对速度环的定量影响

仿真设置如下：对象 $G(s) = 10/(s^2 + 2s)$，PD 控制器 $K_p = 2.5$、$K_d = 0.5$，采样 $T = 2$ ms，计算延迟固定为 $0.3T$，采样时刻在 $[0, J]$ 内均匀抖动，微分用实测间隔。每个工况跑 8 个随机种子，取均值与极差。结果见表 86-6。

工况	超调均值/%	超调范围/%	调节时间/s	稳态纹波	控制量峰值
无抖动	4.660	4.660	1.200	0.00402	2.500
$J = 0.1T$	4.660	4.660	1.200	0.00402	2.500
$J = 0.3T$	4.661	4.661–4.662	1.201	0.00402	2.500
$J = 0.5T$	4.663	4.662–4.664	1.201	0.00403	2.500
$J = 0.5T$，微分用标称 T	5.159	—	1.211	0.00401	2.500
无抖动，2% 概率丢拍	4.662	4.661–4.664	1.200	0.00402	2.500

前四行的差别小到可以忽略：抖动从 0 加到半个周期，超调只涨了 0.003 个百分点，调节时间只涨了 1 ms，纹波只涨了 0.00001。这个结果常被误读成“抖动不重要”，但表里第五行给出了相反的提示——同样是 $J = 0.5T$，仅仅把微分的分母从实测间隔改成标称周期，超调就从 4.663% 跳到 5.159%，多了 0.496 个百分点。

两个数字的比值是 165。也就是说，在这条回路上，一个实现细节的破坏力是抖动本身的一百六十多倍。这不是说抖动可以不管，而是说：**在讨论抖动预算之前，先确认实现没有把抖动放大。**

86.6.3 四种情形的阶跃响应

图 86-6 把四条曲线画在一起：无抖动、$J=0.1T$、$J=0.5T$（实测间隔）、$J=0.5T$（标称分母）。主图看整体形状，插图放大 0.18–0.55 s 的峰值区。

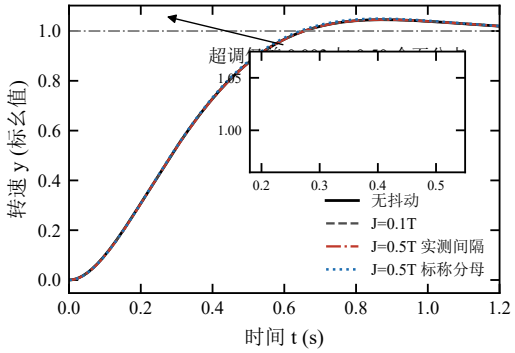


图 86-6　抖动与实现失误对阶跃响应的影响

主图上前三条曲线几乎完全重合，肉眼只能看到一条线。插图里才分辨得出来：$J=0.5T$ 实测间隔的峰值比无抖动高一点点，两条线之间的间隙不到一个像素的宽度；而标称分母那条蓝色点线明显抬起来，峰值高出约半个百分点，且在峰值之后的两个振荡周期里都保持更大的幅值。

这张图的读法值得强调。先看主图，会得到“抖动无关紧要”的结论；再看插图，会得到“实现失误才要命”的结论。两个结论都对，但只有后者能指导行动。工程上图表的放大倍数应当由“要区分的最小差异”决定，而不是由“画得好看”决定。

86.6.4 为什么抖动在这条回路上几乎无害

三个条件同时成立，才使抖动的影响小到可以忽略。

条件一：抖动的相位当量很小。 $J=0.5T=1$ ms，在最坏情况下折算相位 $6.269\times10^{-3}=0.36^\circ$，平均只有 0.18°。相对 66.89° 的裕度，这个量连零头都算不上。

条件二：微分用的是实测间隔。 控制器算的是 $(y_k-y_{k-1})/h_k$，其中 h_k 是真实间隔。这样微分增益不随抖动变化，抖动只改变“采样时刻”，不改变“微分强度”。若换成标称 T，等效微分增益变成 $(1+\delta/T)$ 倍，δ 在 $[0,0.5T]$ 内变化，增益就跟着在 1 到 1.5 倍之间跳。

条件三：对象是二阶低通，闭环带宽低。 对象极点 $-1\pm3j$，自然频率 3.16 rad/s；闭环特征多项式 $s^2+7s+25$，自然频率升到 5 rad/s、阻尼比 0.7，带宽约 8 rad/s。抖动引入的误差主要落在采样频率附近（500 Hz），被对象衰减了四个数量级，传到输出时已经看不见。

三个条件缺一个，结论就会变。把对象换成电流环的 $1/(0.0004s+1)$，条件三立刻失效：带宽 2500 rad/s，抖动引入的误差几乎原样传到输出。把微分改成标称分母，条件二失效，表 86-6 第五行就是后果。

86.6.5 丢拍为什么不敏感

表 86-6 最后一行是“每拍 2% 概率沿用上一拍输出”。这个工况模拟控制任务偶发超时：本拍没算完，执行器继续用上一拍的值。结果是超调 4.662%、调节时间 1.200 s、纹波 0.00402，与无抖动几乎一样。

原因有两条。第一，丢拍等效于把这一拍的延迟从 0.6 ms 拉长到 2.6 ms，相位当量从 0.216° 变成 0.93°，仍然远小于裕度。第二，2% 的概率意味着平均每 50 拍才发生一次，而闭环的调节过程持续约 600 拍，几十次孤立的事件被系统的低通特性平均掉了。

这个结论不能搬到电流环。电流环 $T=62.5$ μs，2% 丢拍意味着平均每 50 拍（3.1 ms）就有一次延迟翻倍；按 §86.4.3 的换算，一次 62.5 μs 的额外延迟折算 8.0° 相位，已经超过“允许抖动 78 μs”的一半。所以同样的失约率，在速

度环是统计噪声，在电流环是持续的相位侵蚀。

86.6.6 控制量峰值给出的另一条线索

表 86-6 里控制量峰值始终是 2.500，与抖动无关。这个数恰好等于 K_p：阶跃首拍 $e=1$、$\dot{y}\approx 0$，所以 $u = K_p\cdot 1 - K_d\cdot 0 = 2.5$。后面各拍的 u 都小于这个值，峰值就定在首拍。

这条线索的用处是判断饱和。若执行器限幅为 2.0，首拍就会削顶，等效增益从 2.5 降到 2.0，上升时间变长、超调变小，闭环看上去“更稳”，实际是进入了非线性区。工程上看到“加大增益反而超调变小”，第一反应应当是查限幅，而不是庆祝。

86.6.7 性能与稳定性的权衡

把本节的结果收拢成三句话。

第一，抖动的影响正比于 $\omega_c J$，与回路带宽绑定，不能跨回路套用同一个预算。速度环可以容忍毫秒级抖动，电流环只能容忍几十微秒。

第二，实现失误的影响与抖动无关，它直接改变控制器的等效参数。微分分母、离散化路径、输出更新时刻，这三处每错一处，代价都比抖动大一个量级以上。

第三，稳定性与性能不是同一个指标。相位裕度回答“会不会振”，超调与调节时间回答“振成什么样”，响应时间回答“会不会漏拍”。三者都要测，且要在同一组工况下测，否则数字之间没有可比性。

86.6.8 抖动的频谱比幅值更重要

前面的讨论把抖动当作一个幅值 J。现场的抖动往往不是白噪声，而是带有周期成分：总线轮询、通信帧、DMA 突发、操作系统节拍，都会让抖动以固定周期重复。

周期成分的危险在于混叠。设抖动以 1 kHz 重复，而速度环以 500 Hz 采样，则 1 kHz 的成分在采样后落到 0 Hz——也就是变成直流。表现为微分项出现一个固定偏置，控制量整体偏移一点，纹波却看不出来。这种故障最难查：所有统计量都正常，只有稳态误差莫名其妙地大了。

判定方法是把抖动序列做一次 FFT。若谱线在采样频率的整数倍附近出现尖峰，说明抖动与采样同步；若尖峰落在闭环带宽以内，它会直接进入输出；若落在带宽以外，大部分被对象衰减。工程上的经验界线是 $0.1 f_s$：高于这个频率的抖动成分，对低通对象基本无害；低于它的成分，要按幅值认真对待。

处置办法不是“把抖动压小”，而是“打破同步”。把总线轮询周期改成与控制周期互质的数，或给通信任务加一个随机化的偏移，都能把尖峰打散成宽带噪声。代价是通信的确定性下降，收益是控制回路不再被周期性扰动。

86.6.9 两条采样经验线的来源

§86.3.3 给过两条经验：$T \le 0.1/\omega_c$ 时相位损失约 2.9°，$T \le 0.2/\omega_c$ 时约 5.7°。这两个数字不是估出来的，它们直接来自式 (86-15) 的半拍延迟。

半拍延迟在穿越频率处的相位是

$$\varphi_{1/2} = \frac{\omega_c T}{2}\ \text{rad} = \frac{180}{\pi}\cdot\frac{\omega_c T}{2}\ \text{deg} \approx 28.65\,\omega_c T\ \text{deg}. \tag{86-20}$$

代入 $\omega_c T = 0.1$ 得 2.865°，代入 0.2 得 5.73°，与经验值吻合到小数点后一位。所以“$0.1/\omega_c$”这条线的本质是“半拍延迟不超过 0.05 rad”。

用式 (86-20) 可以把表 86-4 的四个数一次算出来：速度环 $T = 2$ ms、$\omega_c = 6.269$，$\omega_c T = 0.0125$，得 0.359°；电流环 $T = 62.5$ μs、$\omega_c = 2236.1$，$\omega_c T = 0.1398$，得 4.00°。表里的 0.359 与 4.00 就是这两个数。电流环的

4.00° 落在两条经验线之间——它没有做到 0.1，因为受时基取整的限制（§86.3.3），但也没到 0.2，仍在可接受范围。

这条公式还回答了一个常见的投入产出问题：**采样率再翻一倍，能换回多少相位？** 从 62.5 μs 减到 31.25 μs，$\omega_c T$ 减半，相位从 4.00° 降到 2.00°，省 2.00°。代价是电流环 ISR 的次数翻倍，聚合后的 WCET 从 0.12 涨到 0.24 ms。重算调度：U 从 0.7075 涨到 0.9475，远超 RM 上界；速度环响应 $0.470 \to 0.880$ ms，总线 $1.360 \to 2.890$ ms，日志 $5.470 \to 14.900$ ms。

结果是“仍然通过、余量归零”：五个任务都没超期，但日志只剩 25.5% 的余量，总线只剩 27.75%。用 2° 的相位换掉整机的功能扩展空间，这笔交易只有在“相位是唯一瓶颈”时才成立。而本章的相位裕度是 66.89°，2° 占 3%，显然不是瓶颈。

所以两条经验线的正确用法不是“越靠左越好”，而是“落在区间内即可，不必追求更小”。0.1 是收益开始递减的位置：从 0.2 降到 0.1，相位省 2.9°，算力翻倍；从 0.1 再降到 0.05，相位只再省 1.4°，算力又翻一倍。每一档的收益在减半，成本却不变，这就是边际收益递减的具体形式。

把这条曲线记成一句话：**采样率的选择是“够用即止”，不是“越快越好”**。判据是 $\omega_c T$ 落在 0.1 到 0.2 之间，配上足够的相位裕度，就应当停止加密采样，把算力留给功能。

86.7 参数意义与作用

§86.3 给了一张任务表，§86.6 给了一组性能数字，但表里每个参数“改了会怎样”还没有交代。本节逐个说明：谁决定它、典型取值是多少、改动它会牵动什么。理解这些牵动关系，才能在“再加一个功能”的时候判断该动哪个参数。

86.7.1 八个参数的分工

表 86-7 把本章用到的参数列全，并标注每个参数的来源与改动的代价。

参数	符号	本章取值	由谁决定	改动的代价
周期	T_i	0.5–20 ms	控制带宽、时基	算力线性增长
最坏执行时间	C_i	0.05–1.60 ms	代码、缓存、总线	所有低优先级任务受害
截止时间	D_i	通常等于 T_i	失效后果	只改判据，不改行为
优先级	p_i	1–6	关键性、RM	不耗算力，但重排响应
采样抖动	J_i	0.1–0.5 T	中断与唤醒机制	难压，需改硬件
计算延迟	d_i	约 0.3 T	算法长度、实现	与代码结构绑定
利用率	U	0.7075	上述各量之和	全局约束
时间余量	S_i	$D_i - R_i$	派生量	局部指标

这张表最值得看的是最后一列。周期与 WCET 是“花钱”的参数，优先级是“免费”的参数，抖动与延迟是“难改”的参数。工程上先动免费的那个，再动花钱的那个，最后才考虑难改的那个。

86.7.2 周期：控制侧与软件侧的双重身份

周期在控制器设计者眼里是采样率，在软件设计者眼里是负载。这两个身份的方向相反：T 减半，相位损失减半，但 CPU 占用翻倍。

本章速度环从 2 ms 减到 1 ms 的后果可以精确算出来。相位损失从 0.359° 降到 0.180°，省下 0.18°；利用率从 0.7075 升到 0.8575，增加 0.15；日志任务的响应时间从 5.470 ms 涨到 7.820 ms，增加 2.35 ms；总线任务从 1.360 ms 涨到 1.830 ms。

用 0.18° 的相位换 2.35 ms 的响应时间和 0.15 的利用率，这笔交易在 66.89° 的裕度面前显然不划算。更麻烦的是 $U = 0.8575$ 已经越过 RM 上界 0.7435，此后每次加功能都必须逐项重算响应时间，不能再靠充分条件兜底。

周期的正确用法是“够用即止”：按 $T \le 0.1/\omega_c$ 定下限，按算力余量定上限，取中间偏下的值。速度环 $\omega_c = 6.269$ 对应 $T \le 16$ ms，实际取 2 ms，已经比下限紧了八倍，再紧没有收益。

86.7.3 WCET：一个参数的全局影响

WCET 是最容易被低估的参数。它不只影响自己，还通过式 (86-4) 的 $\lceil R_i/T_j \rceil$ 项放大到所有低优先级任务。

把速度环的 WCET 从 0.30 ms 加到 0.40 ms，只加 0.10 ms，后果是：速度环自己的响应时间从 0.470 ms 涨到 0.740 ms（涨 0.27 ms，放大 2.7 倍）；总线任务从 1.360 涨到 1.460；日志任务从 5.470 涨到 5.940（涨 0.47 ms，放大 4.7 倍）；利用率从 0.7075 涨到 0.7575。

放大倍数的来源是“每拍都要交税”。日志任务的周期是 20 ms，在它的响应窗口里速度环要执行 10 次，每次多 0.10 ms 就是 1.0 ms；再叠加总线与电流环的连锁推迟，最终落到 0.47 ms 的净增量。任务周期越长，被同一笔 WCET 增量重复征收的次数越多，放大倍数越大。

电流环保护的 WCET 从 0.12 加到 0.20 ms 更严重：速度环响应从 0.470 涨到 0.800 ms，日志从 5.470 涨到 7.900 ms，利用率到 0.8675。高优先级任务的 WCET 是全局税，低优先级任务的 WCET 只影响自己——这就是为什么优化时先看高优先级任务的热点代码。

86.7.4 截止时间：只改判据，不改行为

截止时间 D_i 与周期 T_i 通常取同一个值，于是很多人以为它们是一回事。两者在调度里的作用完全不同：T_i 决定作业何时到达，D_i 只决定“晚到多久算失败”。

把某个任务的 D_i 从 T_i 收紧到 $0.5T_i$，调度行为一点不变，作业还是按同样的顺序执行，但判据变严了，原本合格的 R_i 可能突然不合格。反过来，把 D_i 放宽，系统不会变快，只是容忍度变高。

这个区别在工程评审里经常被混淆。有人说“把截止时间放宽一点就通过了”，这句话在数学上成立，在物理上不成立——放宽截止不会让代码跑得更快，只会让失约的定义变松。真正要改的是 WCET 或优先级。

对固实时任务，截止还有一层含义：偶尔错过一次可以容忍，只要错过率低于某个阈值。这类任务用 (m, K) 模型描述——任意 K 个连续作业中最多错过 m 个。本章的随机事件处理与总线收发属于这一类，日志诊断属于软实时，只有安全监测、电流环保护、速度环是硬实时。

86.7.5 优先级：唯一不耗算力的参数

优先级不占用 CPU，却能改变响应时间在任务之间的分配。它是唯一“免费”的参数，也是最容易用错的参数。

以表 86-3 的任务集为例，把日志诊断的优先级从 6 提到 3（高于速度环），速度环的响应时间立刻从 0.470 ms 变成 2.920 ms，超过它 2.0 ms 的截止。原因很直接：日志任务每 20 ms 执行 1.6 ms，一旦它排在速度环前面，速度环每十拍就有一次被整整推迟 1.6 ms。

这个例子说明两件事。第一，优先级不能按“重要性”直觉排，要按“周期 + 关键性”排。第二，优先级的代价是非线性的：把它往前挪一位，可能让下游一串任务同时超期，而利用率一点没变。

优先级还有第三个作用：决定阻塞。共享资源（互斥锁、消息队列）会让高优先级任务被低优先级任务阻塞，阻塞时间 B_i 要加进式 (86-4)。本章的任务集没有共享写操作，$B_i = 0$；一旦加上参数下发或故障录波，B_i 就必须实测。

86.7.6 利用率与余量：两个不同层次的问题

利用率 U 是全局量，回答“这台 CPU 还剩多少算力”；余量 $S_i = D_i - R_i$ 是局部量，回答“这个任务还能被推迟多久”。

本章的 $U = 0.7075$，意味着还有 29.25% 的算力空闲。但各任务的余量并不均匀：安全监测 90%、电流环保护 66%、速度环 76.5%、总线收发 66%、日志诊断 72.65%。余量最小的是电流环保护与总线收发，都是 66%。

两个指标要一起看。U 低但某个任务的 S_i 小，说明算力有，但分配不均，该调优先级；U 高而所有 S_i 都还宽裕，说明系统已经接近满载，下一个功能要谨慎。只看 U 会漏掉分配问题，只看 S_i 会漏掉总量问题。

86.7.7 抖动与延迟：先能测，才谈得上调

抖动 J 与计算延迟 d 是最难改的两个参数，因为它们由硬件与内核决定，不由应用代码决定。改它们通常意味着换中断控制器、换唤醒机制、换总线仲裁策略。

难改不等于不用管。正确做法是“先算允许值，再实测，最后看比例”。速度环允许抖动 27.8 ms，实测 0.84 ms，占允许值的 3%；电流环允许 78 μs，实测约 10 μs，占 13%。两个比例都在安全区，说明当前的硬件与内核配置够用，不需要为抖动做额外投入。

若某个回路的实测值占到允许值的一半以上，就要开始查来源：中断延迟、任务唤醒抖动、DMA 抢占、Flash 等待。这些来源的排查顺序应当按“贡献大小”排，而不是按“容易改”排——改错了地方，投入没有回报。

一句话收束本节：八个参数里，优先级最便宜、周期最贵、WCET 影响最广、抖动最难改。设计的功夫在于判断当前瓶颈是哪一个，而不是把所有参数都往“更好”的方向推。

86.7.8 三个旋钮联动的非线性

§86.7.1 到 §86.7.7 每次只动一个参数，看的都是单变量灵敏度。真实的设计变更很少只动一个：为了压相位会把周期减半，同时为了加算法会把 WCET 抬高，两件事常常一起发生。这时“分别算一下再相加”是错的，而且错得不少。

先看一组实测数字。以表 86-3 为基准，四种情形并排如下。

- 基准：$U = 0.7075$，速度环 0.470、总线 1.360、日志 5.470 ms。
- 只把速度环周期减半：$U = 0.8575$，速度环 0.470、总线 1.830、日志 7.820 ms。
- 只把速度环 WCET 加到 0.40 ms：$U = 0.7575$，速度环 0.740、总线 1.460、日志 5.940 ms。
- 两件一起做：$U = 0.9575$，速度环 0.740、总线 2.770、日志 14.900 ms。

利用率是可以相加的：$0.8575 + 0.10 = 0.9575$，与最后一组完全一致，因为 U 是 C_i/T_i 的线性求和。日志任务的响应时间却完全不能相加：单改分别增加 2.35 与 0.47，合起来应该增加 2.82，实际增加了 9.43，是二者之和的三倍多。

差额来自式 (86-4) 里的向上取整。单改周期时，日志任务被抢占的次数是 $\lceil R/0.5 \rceil$ 与 $\lceil R/1.0 \rceil$ 两个阶梯；单改 WCET 时，阶梯不变、每格的高度变了。两件一起改，阶梯与格高同时变化，中间某一步的 R 恰好跨过了一个新的取整边界，于是又多算出一整格。迭代到不动点之后，这个“多算的一格”会被放大成数毫秒。

这个现象给出一条实用规则：**多参数变更必须整体重算，不能叠加单项增量**。工程上常见的做法是“上次算过，这次只加 0.1 ms，应该没问题”，这句话在两个参数同时变时就是错的。重算的成本只有几秒钟，而误判的代价是投运后的偶发超期。

第二条规则：**先改便宜的那个，算完之后再决定要不要改贵的**。上面第二组与第三组对比：把 WCET 加到 0.40 ms，U 只涨 0.05、日志只涨 0.47；把周期减半，U 涨 0.15、日志涨 2.35。如果目标是压相位，减半周期只换来 $0.18°$（§86.7.3），代价却是响应时间翻倍，这笔交易在任何情况下都不划算。

第三条规则：**看余量，不看通过与否**。合改之后 $U = 0.9575$，五个任务仍然全部通过：速度环 $0.740 < 1.0$、总线 $2.770 < 4.0$、日志 $14.900 < 20.0$。若只看“是否通过”，结论是“可以上”。但日志任务的余量从 72.65% 掉到 25.5%，总线从 66% 掉到 30.75%——系统还在运行，缓冲已经没了。下一轮加功能时，第一个失约的一定是这两项。

把三条规则合起来，就是一句话：**利用率告诉你还有多少算力，响应时间告诉你还能撑多久，两者都不是线性可加的量**。设计评审时若有人用“加起来差不多”来估算，请他把表重算一遍。

86.8 调参方法与经验

§86.7 说明了每个参数的作用，本节说明按什么顺序动它们。实时软件的调参与控制器调参有一处根本区别：控制器调参可以“边看响应边拧旋钮”，实时调参必须先建立观测手段，否则改完之后不知道是变好了还是运气好。

86.8.1 三段式顺序：先能跑，再稳，最后快

调参的顺序应当是：可用性、抖动、性能。三段的目标不同，验收指标也不同。

第一段是可用性：保证 $R_i \le D_i$，任何工况下都不失约。这一段的旋钮只有三个——优先级、周期、任务拆分。控制器增益在这一段不参与，因为失约是调度问题，不是控制问题。

第二段是抖动：在不失约的前提下，把每个回路的 J_i 压到允许值的三分之一以下。这一段的旋钮是中断配置、唤醒机制、临界区长度。压不下去时才回头动周期。

第三段是性能：超调、调节时间、稳态纹波。这一段的旋钮才是控制器增益，而它必须在时间参数冻结之后才动，否则改完增益又要重测 WCET，前面的工作全废。

顺序颠倒的典型后果是：先花两周把超调调到 4.5%，再加一个通信任务导致偶发失约，控制性能瞬间回到原点，而且不知道是哪个改动引起的。按三段式走，每一段都有明确的验收线和冻结点。

86.8.2 WCET 必须实测，且要测长尾

WCET 是整张表里最不可信的数字。它的测量有六条经验，按重要性排列。

第一条，测最坏路径。 分支里最慢的那条路径要专门构造输入去触发，例如参数下发时的最大报文、故障录波时的满缓冲写入。平均路径的数字只能用在“估算”，不能用在“验收”。

第二条，冷热两种状态都测。 本章速度环的实测分布是：缓存命中 0.28 ms、未命中 0.34 ms、DMA 争用总线 0.45 ms。三者相差 60%。若只测热缓存，填表填 0.28，等于给自己留了一个隐藏的坑。

第三条，测足够多次。 长尾事件的发生概率可能只有 10^{-5}，跑一万次看不到，跑一百万次才出现。工程上用 10^{6} 次采样的最大值作为初值，再加 5%–10% 的余量。

第四条，把中断叠加进去。 单独测任务执行时间时，中断被关掉或很少发生；真实运行时，CAN 接收、编码器捕获、看门狗都可能在同一拍插入。这些插入会拉长执行时间，必须计入。

第五条，用硬件计数器而不是软件打点。 软件打点本身有开销，会把 0.30 ms 测成 0.33 ms。Cortex-M 的 DWT 周期计数器、或一个空闲 GPIO 的翻转配合逻辑分析仪，误差都在纳秒级。

第六条，记录测量条件。 主频、缓存配置、编译选项、外设时钟，任何一项变了，WCET 都要重测。测量记录里没有这些条件，数字在三个月后就没有参考价值。

低估 WCET 的后果可以量化。速度环的真实 WCET 若是 0.45 ms 而表里填了 0.30 ms，日志任务的响应时间会从 5.470 ms 涨到 6.880 ms，利用率从 0.7075 涨到 0.7825。涨幅 26%，而表面上只是“一个数字填小了”。

86.8.3 排故顺序：从症状到根因

现场问题通常以三种症状出现：偶发超调、周期性啸叫、通信丢包。三种症状的排查顺序不同，但都遵循“先统计，后定位”的原则。

偶发超调：先看失约统计，确认是速度环自己失约还是被高优先级任务挤掉。若是后者，查抢占源；若是前者，查 WCET 与缓存。确认调度无问题后，再查微分实现（分母、滤波、量化）。

周期性啸叫：先测啸叫频率。若等于某个任务的周期倒数，是采样或更新节拍问题；若等于两个周期的最小公倍数的倒数，是拍频；若与负载有关，是机械谐振被采样混叠。三种原因的处置完全不同，靠"再调调增益"是解决不了的。

通信丢包：先看丢失是否成串。成串丢失通常是总线仲裁或接收缓冲溢出；孤立丢失通常是优先级不够，被高优先级任务连续推迟。前者改硬件配置，后者改优先级。

排故的关键不是"猜得快"，而是"分得开"。每一条症状都要先归结到"调度、采样、实现、硬件"四类中的一类，再在该类里找具体原因。跨类乱试，等于把一次实验变成四次实验。

86.8.4 六条可执行的经验

经验一：一次只改一个参数。 同时改周期与优先级，出问题时无法归因。改完记录"改前值、改后值、观察到的变化"，三次之后就能看出敏感度。

经验二：用最坏工况调，不用典型工况调。 典型工况下调好的参数，在最坏工况下往往不够。本章的过载扫描（§86.9）就是最坏工况的一种：把日志任务的 WCET 逐步加大，看系统在哪一点开始失约。

经验三：留 20% 的余量作为设计目标。 本章 $U = 0.7075$，空闲 29.25%，略高于这个目标；各任务余量最低 66%。余量不是浪费，它是"下一个功能"的预算。用满的系统，加一行代码都要重新认证。

经验四：改优先级之前先算，不要先试。 优先级的影响可以用式 (86-4) 五分钟算完，试一次要烧录、跑工况、看波形，半小时。算完再试，试的是"算得对不对"，不是"行不行"。

经验五：把抖动预算写成比例，不写绝对值。 "抖动 $10\ \mu s$"这个说法没有意义，"抖动占允许值的 13%"才有意义。前者换个回路就失效，后者可以直接比较。

经验六：调参结果必须入库。 每次改动记录：日期、改动项、改动前后的表 86-3、实测波形文件名。三个月后回看，这份记录是唯一能解释"为什么当初这么定"的材料。

86.8.5 五个常见误判

调参中最耗时的不是计算，而是误判。表 86-8 列出五个高频误判及其验证方法。

误判	真实原因	验证方法
抖动导致超调	微分分母用了标称 T	去掉抖动、保留实现，复测超调
CPU 算力不够	优先级倒置或长临界区	打开阻塞统计，看高优先级任务等待时间
采样越快越好	相位收益递减、算力线性增长	周期减半，比较相位与利用率
平均执行时间够用	长尾超标	10^6 次采样取最大值
仿真通过即现场通过	缓存、中断、DMA 未建模	目标板实测，对比直方图

第一行的验证方法最值得记住：把抖动关掉，只留实现错误。若超调仍然偏高，问题就不在抖动。这个"控制变量"的思路适用于所有实时问题——先把怀疑对象关掉，看症状是否消失，比争论机理快得多。

第二行的"优先级倒置"常被误认为算力不足，因为症状都是"任务变慢"。区别在于：算力不足时所有任务一起变慢，优先级倒置时只有特定任务变慢，且变慢的幅度与临界区长度相关。

86.8.6 两个底层原则

原则一：可观测性优先于优化。 在改任何参数之前，先确保能看见它。没有 WCET 直方图就调 WCET，没有抖动记录就调抖动，等于闭眼拧螺丝。观测手段的成本很低：一个空闲 GPIO、一个环形缓冲、一个周期计数器，半天就能搭好。

原则二：分析取悲观值，验证取最坏工况。 分析时把 WCET 取大、把优先级取低、把阻塞取长，得到的是“保证”；验证时构造最坏输入、最高负载、最低电压，得到的是“证据”。两者都偏保守，结论才有余地。反过来做——分析取平均值、验证取典型工况——得到的是一份好看但不成立的报告。

这两条原则听起来像常识，但在项目压力下最容易被牺牲。牺牲的顺序通常是先砍观测、再砍最坏工况，最后报告里只剩一组“典型值”。等到现场出问题，连复现都做不到。

86.8.7 一次完整的调参记录

把前面几条经验串成一次真实顺序的记录。场景是：速度环阶跃响应超调偏大，工程师怀疑是采样抖动。

第一轮。 改前状态：$J = 0.5T$、微分用标称周期，超调 5.159%、调节时间 1.211 s。改动：把采样抖动关掉（$J = 0$），其余不动。观测：超调仍是 5.159%，一点没降。结论：抖动不是原因，实现才是。这一步只花了一次仿真，却省掉了后面所有关于“怎么压抖动”的讨论。

第二轮。 改动：把微分分母从标称 T 改成实测间隔 h_k，抖动保持 $J = 0.5T$ 不变。观测：超调降到 4.663%，调节时间 1.201 s，纹波从 0.00401 略升到 0.00403。结论：确认病因。改动的代价是控制器多存一个时间戳，WCET 增加不到 1 μs，可以忽略。

第三轮。 有人提议“既然超调还比理想值 4.660% 高一点，把周期从 2 ms 缩到 1 ms 压一压”。改动前先算账：相位损失从 0.359° 降到 0.180°，省 0.18°；利用率从 0.7075 升到 0.8575；日志任务响应从 5.470 ms 升到 7.820 ms。观测（仿真）：超调几乎不变，因为瓶颈早已不在采样相位。结论：不划算，回退。这一轮没有烧录、没有上机，靠一次计算否决。

第四轮。 有人提议“日志任务太慢，把它优先级提上去，好让诊断数据更及时”。算账：日志提到速度环之上后，速度环响应从 0.470 ms 变成 2.920 ms，超过 2.0 ms 的截止。结论：直接否决，且这条要写进设计约束——**任何新任务的优先级都不得高于速度环**。

四轮下来，真正改动系统的只有一次（第二轮），其余三次都是“算完就停”。这正是 §86.8 强调的顺序：先测、再算、最后才改。若第一轮就直接去压抖动，可能花两周换硬件，超调一点没变。

86.8.8 交接：三张表与四句话

实时设计最容易被忽略的一环是交接。同一个人维护时，参数背后的理由都在脑子里，表只是一份记录；换人维护时，表就是全部信息。半年后接手的人看到的是一行“速度环 $C = 0.30$ ms”，他不会知道这个数是热缓存中位数还是冷启动 95 分位，也不会知道当初为什么不再加一路中断。

所以交接要交的不是代码，是三张表加四句话。

第一张表：任务表。 就是表 86-3，但必须多一列“取值依据”。0.30 这一格要写明“冷启动首轮 0.34、热循环中位数 0.28、取 0.30 作设计值，验收按 0.34 复核”。没有这一列，下一个人会把 0.30 当成实测上界，然后放心地加功能。

第二张表：预算表。 就是图 86-8 那张分解图，但要把每一段占用的来源写清楚：0.30 是算法本体、0.17 是采样与输出、1.53 是留给别人的。数字本身不重要，重要的是“这一段是干什么用的”。

第三张表：变更记录。 日期、改动项、改动前后的两列响应时间、实测波形文件名。这份表的价值不在记录历史，而在提供反例——下一个人想加功能时，能查到“上次类似改动让日志从 5.470 涨到 7.820，最后回退了”。

四句话则是对这三张表的摘要：

第一句，**瓶颈是谁**。本章的答案是日志任务，$R/T = 0.274$ 看着不大，但它是所有高优先级任务的“垃圾桶”，任何一项变慢都会在它身上放大。

第二句，**余量在哪**。$U = 0.7075$ 对 RM 上界 0.7435，只剩 0.036；速度环自身余量 76.5%，电流环 66%。前者是总量约束，后者是分配约束，两者不能互相挪用。

第三句，**哪些不能动**。日志任务的优先级不得高于速度环；速度环周期不得低于 2 ms；微分分母必须用实测间隔。三条都是被算过的，不是偏好。

第四句，**哪些结论只是仿真的**。超调 4.663%、纹波 0.00403、调节时间 1.201 s 都来自 `ch086_sim.py` 的理想对象，现场会有偏差。仿真的作用是排序——告诉你哪个因素重要、哪个可以忽略，不是给出绝对值。

判断交接是否成功，有一个简单标准：接手人能否在半小时内，只用这三张表和一段代码，把任意一个任务的响应时间重新算出来，并且结果与表里一致。能，说明信息闭合；不能，说明还缺东西——通常是缺了“取值依据”那一列。

86.9 优化与改进

§86.8 讲的是“怎么调”，本节讲“调不动的时候改什么”。优化的目标是让同样的硬件承载更多功能，或让同样的功能留出更大余量。本节按“算法、任务、内核、硬件”四个层次展开，最后给出过度优化的判据。

86.9.1 四个层次的优化

算法层是减少 WCET 最直接的手段：查表代替实时计算、定点代替浮点、增量更新代替全量重算。这一层的收益最确定，代价是代码可读性下降。

任务层是减少抢占与切换开销：合并同频任务、把多个慢任务合并成一个按固定节拍调度的任务、把非关键功能挪到低优先级。这一层的收益取决于任务划分是否合理，改错方向会适得其反。

内核层是减少阻塞与延迟：优先级继承、临界区缩短、中断上半部/下半部分离。这一层的收益在“有共享资源”时才明显。

硬件层是加法：双核、DMA、FPGA 协处理、更高主频。这一层最贵，但上限最高。

四层的顺序应当是先算法、再任务、再内核、最后硬件。倒过来做，往往是花了大钱解决了一个本该用查表解决的问题。

86.9.2 过载时会发生什么

优化之前要先知道“过载时会怎样”。很多系统的设计目标是“不过载”，但真正决定现场表现的，是过载之后的降级方式。

图 86-7 是本章任务集的过载扫描：把日志任务的 WCET 从 1.6 ms 逐步加到 12.0 ms，横轴是周期任务利用率，纵轴是 10 个日志作业中的失约数。固定优先级与 EDF 各画一条阶梯线。

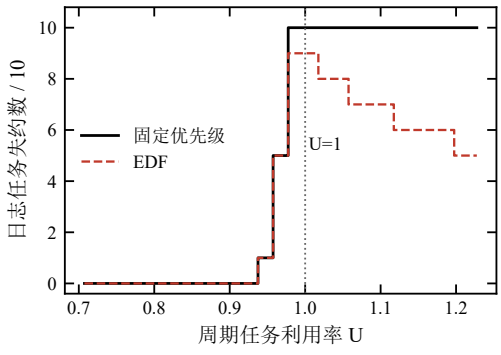


图 86-7　过载时固定优先级与 EDF 的失约对比

表 86-9 摘出六个关键点。表中的利用率只统计周期任务，随机事件任务的负载（约 3%）未计入。

日志 WCET/ms	周期利用率 U	FP 失约/10	EDF 失约/10
1.6	0.7075	0	0
6.0	0.9275	0	0
7.2	0.9875	10	9
8.0	1.0275	10	8
10.0	1.1275	10	6
12.0	1.2275	10	5

两条曲线的形状完全不同。固定优先级是“悬崖”：日志 WCET 从 6.0 ms 加到 7.2 ms，失约数从 0 直接跳到 10，中间只有一个台阶——$U = 0.9675$ 时丢 5 个。EDF 是“斜坡”：同样过载，失约数从 9 缓慢降到 8、6、5，负载越重反而丢得越少。

悬崖的原因很直接：日志任务优先级最低，只要它的响应时间超过 20 ms，每一个作业都超期，一个也跑不完。斜坡的原因是 EDF 按截止时间排队，过载时它把 CPU 时间“摊”给所有作业，每个作业都晚一点，但都能推进一部分。

这两条曲线各有用途。固定优先级的悬崖对高优先级任务是保护：整个扫描过程中，速度环的失约数始终是 0，即使 U 到 1.23。EDF 的斜坡对低优先级任务是缓冲，但它不区分关键性——一个截止 20 ms 的日志作业与一个截止 2 ms 的控制作业，在 EDF 眼里只有截止时间的差别。若把某个关键任务的截止设得比非关键任务长，EDF 会在过载时先牺牲它。

结论不是“FP 更好”或“EDF 更好”，而是：**硬实时任务用固定优先级保护，软实时任务用 EDF 摊平。** 混合任务集用混合策略——固定优先级保证关键任务，剩余带宽用 EDF 分配给非关键任务。本章的任务集若这样配置，日志任务在过载时不会全丢，速度环也不受影响。

86.9.3 减少阻塞：优先级继承

阻塞来自共享资源。高优先级任务等一个被低优先级任务持有的互斥锁，等的时间就是阻塞时间 B_i。更糟的是优先级反转：持锁的低优先级任务又被中优先级任务抢占，高优先级任务的实际等待被无限拉长。

经典处置是优先级继承：持锁任务临时提升到等待者的优先级，等它释放锁再降回。另一种是优先级天花板：持锁任务直接提升到所有可能使用该资源的任务中的最高优先级。

两者的效果可以量化。若临界区长度是 0.15 ms，无继承时高优先级任务可能要等 0.15 ms 加上被抢占的时间；有继承时等待上界就是 0.15 ms。把 $B_i = 0.15$ ms 加进式 (86-4)，速度环的响应时间从 0.470 ms 变成 0.620 ms，仍在 2.0 ms 的截止之内。

工程上还有一条更彻底的办法：不用锁。单生产者单消费者的环形缓冲、原子指针交换、每任务独立的数据副本，都能把“共享”变成“不共享”。本章任务集之所以 $B_i = 0$，就是因为任务之间只通过无锁队列传递数据。

86.9.4 减少切换：任务合并与固定节拍

每次任务切换的开销在 1–5 μs 量级，单看不多，但乘以每秒上万次就不可忽略。本章六个任务在 1 s 内产生约 $2000 + 2000 + 125 + 500 + 250 + 50$ 次唤醒，切换开销约 1% 的 CPU。若任务数增到二十个，这个比例会翻倍。

减少切换有两条路。**合并同频任务**：把安全监测与电流环保护合并成一个 0.5 ms 任务，切换次数减半，代价是两个功能的截止都收紧到 0.5 ms。**固定节拍调度**：所有任务按同一时基排成一张静态表，运行时只查表不决策，切换开销与调度开销都接近零，代价是灵活性——表外的请求只能等到下一个时隙。

固定节拍调度在伺服与电力电子里很常见，因为它把“调度”从运行时搬到了设计时，行为的可预测性最高。缺点是加功能必须重排整张表，且非周期事件只能靠预留时隙处理。

86.9.5 减少计算：三种常用手法

查表：把三角函数、开方、指数换成查表加线性插值。一个 256 点的正弦表加一次线性插值，误差约 10^{-5}，耗时从几十微秒降到几微秒。代价是内存与精度，需要按允许误差定表长。

定点：把浮点换成 Q 格式定点。Cortex-M4F 有单精度浮点单元，浮点乘加只要一两个周期，定点未必更快；Cortex-M0 没有浮点单元，浮点要靠软件库，定点能快一个数量级。是否换定点，取决于芯片有没有 FPU，不能一概而论。

增量：把每拍全量重算改成增量更新。例如位置环的轨迹规划，每拍只算增量而不是重算整条曲线，计算量从 $O(n)$ 降到 $O(1)$。代价是误差会累积，需要定期全量校正。

三种手法的共同点是“用内存或精度换时间”。换之前要先确认瓶颈真的在计算上——若瓶颈在等待 ADC 转换或总线响应，减少计算不会让响应时间变短。

86.9.6 换硬件与多核拆分

硬件层的优化有三个方向。**提高主频**最直接，但功耗与散热跟着涨，且 Flash 等待周期可能抵消一部分收益。**加 DMA** 把数据搬运从 CPU 剥离，对总线收发与录波类任务收益最大。**双核拆分**把硬实时任务放到一个核、把通信与诊断放到另一个核，两个核之间只通过共享内存或消息传递交互。

双核拆分听起来是“免费翻倍”，实际有三处代价。第一，核间通信的延迟与抖动往往比片内任务切换大一个量级，跨核的截止时间更难保证。第二，两个核共享总线与内存控制器，一个核的突发访问会拖慢另一个核，WCET 不再可加。第三，调试复杂度上升，两个核的时序问题很难复现。

工程上的判断标准是：单核利用率超过 80% 且优化空间已用尽，才考虑双核。本章 $U = 0.7075$，离这个界线还有距离，双核拆分没有收益。

86.9.7 过度优化的三个信号

优化有收益，也有成本。出现下面三个信号时，应当停下来。

信号一：为 3% 的利用率牺牲了可观测性。 删掉日志、关掉断言、去掉边界检查，确实能让 WCET 变小，但也让现场问题无法定位。省下的 3% 换来的是一份查不出原因的故障报告。

信号二：优化后的代码无法解释。 一段靠手工展开、位运算技巧、全局状态拼出来的代码，半年后没人敢改。实时系统的寿命通常以十年计，可维护性的价值高于几个百分点的性能。

信号三：优化目标与设计目标脱节。 设计目标是“不失约、抖动小于允许值的三分之一”，而优化却在追“平均执行时间再降 10 μs”。平均值的改善不影响最坏值，这种优化对验收指标没有贡献。

一句话收束本节：优化要打在瓶颈上，而瓶颈要靠测量确定。四层优化里，先做算法与任务，再做内核与硬件；过载行为要先设计好降级方式，而不是指望系统永远不过载。

86.9.8 固定节拍的一张表

固定节拍调度的核心是一张静态表。以本章任务集为例，取 0.5 ms 为节拍，20 ms 为超周期，共 40 个时隙。每个时隙的容量是 0.5 ms，需要装下这段时间内到期的作业。

算一下需求：安全监测 40 份、每份 0.05 ms，合计 2.00 ms；电流环保护 40 份、每份 0.12 ms，合计 4.80 ms；速度环 10 份、每份 0.30 ms，合计 3.00 ms；总线收发 5 份、每份 0.55 ms，合计 2.75 ms；日志诊断 1 份、1.60 ms，切成一到两个片。总计 14.15 ms，占 20 ms 的 70.75%——与式 (86-6) 的 $U = 0.7075$ 完全一致。

排表的原则是“越靠近截止越优先”。安全监测与电流环保护每拍都有，直接占每个时隙的头部 0.17 ms；速度环在偶数时隙追加 0.30 ms；总线收发每四拍追加 0.55 ms；日志切片填进剩余空隙，每片不超过 0.4 ms，保证不拖过下一个时隙。

这张表一旦排好，调度抖动理论上为零：每个作业都在固定时刻开始，没有抢占、没有队列。代价有三条。第一，余量被固化——某个时隙用不完的 0.1 ms 不能借给别的时隙。第二，加任务必须重排整表，且要重新验证超周期。第三，非周期事件只能靠预留时隙（例如每 20 ms 留两个空时隙）处理，突发多了就溢出。

所以固定节拍适合需求稳定、认证要求高的场合，不适合功能还在迭代的样机阶段。本章采用固定优先级，正是因为示例系统还在加功能；等需求冻结、准备认证时，再把优先级调度换成静态表，是一条自然的演进路径。

86.9.9 优化的时机：哪些决定不可逆

前面八节讲的都是“怎么优化”，这一节讲“什么时候优化”。同一项改动，在不同阶段做的代价可能相差十倍，原因是有些决定一旦落地就几乎不可逆。

不可逆的决定有三个：时基、任务边界、优先级顺序。 时基决定所有周期的公约数。本章选 0.5 ms，所有任务周期都是它的整数倍，超周期才能枚举。若早期随手把某个任务设成 0.7 ms，超周期会从 20 ms 涨到 140 ms，仿真与验证的工作量跟着涨七倍，而且这个设定会渗进代码里的每一处计时。任务边界同样难改：把两个功能合在一个任务里容易，拆开要动数据流、动同步、动优先级，等于重做一遍调度分析。优先级顺序看似只是一个数组，但它被写进设计约束、被写进测试用例，改动它要重跑全部超期测试。

可逆的决定有很多：算法常数、寄存器级优化、定点与浮点的取舍。 把 K_d 从 0.5 调到 0.4，只影响一条曲线；把一段循环手工展开，只影响一个函数的 WCET。这类改动随时可做，做错了回退也便宜。

两类决定的分界给出优化时机的原则：**不可逆的决定要在样机阶段就留足余量，可逆的优化留到瓶颈出现时再做。** 本章样机阶段的 U 只有 0.55 左右，多出来的 0.16 不是浪费，而是为诊断代码、为临时调试口、为后续功能预留的空间。等到小批试产、功能冻结，U 收拢到 0.7075，这部分余量正好被用掉。若样机阶段就把 U 做到 0.90，量产时任何一个新需求都只能靠换芯片解决。

这个思路和控制器设计里的“带宽留余量”是同构的。控制器的穿越频率不会顶到执行器的极限，实时软件的利用率也不应顶到 RM 上界。两者留余量的理由完全一样：模型总有未建模的部分，余量是给未建模部分准备的。

还有一条时间维度上的经验：**优化应当跟着故障走，而不是跟着指标走。** 指标（利用率、超调、抖动）是设计期的约束，故障是运行期的证据。本章的四轮调参记录（§86.8.7）里，三轮都是由具体的观测触发的，没有一轮是“觉得还能再快一点”就动手。按指标优化容易陷入局部——把 U 从 0.7075 压到 0.68，系统一点没变好，因为瓶颈从来不是总量。

最后给一条判断线：**当优化带来的收益小于它增加的维护成本时，就停。** 收益可以是余量、可以是响应时间、可以是通过认证的概率；成本是代码复杂度、是验证工作量、是交接难度。这条线画在哪里因项目而异，但必须画，否则优化会一直做到“没人敢改”为止。

86.9.10 优化之后怎么证明没有变坏

任何一次优化都要回答两个问题：目标指标变好了没有，其他指标变坏了没有。第一个问题容易答，第二个问题常被跳过，而跳过它的代价往往更大。

先把最常见的坑说清楚：**优化经常改善平均值、恶化最坏值。** 一个例子是把查表改成内联展开，平均执行时间从 0.30 降到 0.28 ms，看起来省了 7%。但代码变长，指令缓存冲突概率上升，99 分位从 0.34 涨到 0.36 ms。按 0.36 重算：速度环 $0.470 \to 0.700$ ms，总线 $1.360 \to 1.420$ ms，日志 $5.470 \to 5.820$ ms，U 从 0.7075 涨到 0.7375，距 RM 上界 0.7435 只剩 0.006。

这就是“优化之后系统更容易超期”的典型路径。它不会在功能测试中暴露，只会在满载运行时偶发。所以回归验证必须看直方图，不能只看均值。

回归分四类，缺一类都不算验证完成。

第一类，时间回归。 把新测的 WCET 代回式 (86-4)，全表重算，而不是只算被改的那个任务。重算之后检查两件事：每个 R_i 是否仍小于 D_i，以及最紧的那个任务余量是否仍大于 10%。本章的“最紧”是 U 对 0.7435 的距离，优化后只剩 0.006，这一项就该亮红灯。

第二类，功能回归。 阶跃响应、跟踪响应、抗扰响应各跑一遍，与优化前的波形叠在一起看。判定标准不是“看起来差不多”，而是三个数：超调、调节时间、稳态误差。任何一个变化超过 5%，都要解释清楚原因。

第三类，边界回归。 低温、低压、满负载、总线风暴，四种最坏工况各跑一次。优化的收益常常在最坏工况下消失——缓存冲突在冷启动时最严重，总线风暴下 DMA 争用最激烈。这一类回归的通过标准是“最坏工况下仍不超期”，而不是“典型工况下更快了”。

第四类，长时回归。 连续运行至少一个完整维护周期，观察三件事：计数器是否溢出、内存是否泄漏、日志是否写满。这里有一个具体的坑：32 位周期计数器在 16 kHz 下，$2^{32} \times 62.5\ \mu\text{s}$ 约 74.6 小时溢出一次。跑 8 小时测不出来，跑一周必然撞上。溢出时若代码用无符号相减，结果仍然正确；若用有符号比较，时间戳会突然变成负数，表现为“偶发的一拍巨长”。这类问题只能在长时回归里发现，或者在设计时就用无符号类型规避。

四类回归全部通过，才能说“优化没有变坏”。只跑第一类，等于只看自己关心的那个数字；只跑第二类，等于只看功能不看时间。工程上常见的做法是先跑一、二类，把三、四类留到里程碑节点——这个折中可以接受，但必须写进计划，而不是“想起来才跑”。

86.10 工程落地

前九节建立的是判据与方法，本节把它们变成可执行的动作：一个能复用的任务模板、一张能贴在评审会上的预算表、一套能复现的测量方案，以及一份验收清单。

86.10.1 一个可复用的任务模板

实时控制任务的骨架高度雷同，可以固定成五段：等待节拍、采样、计算、输出、记账。任何一段的顺序错了，都会引入额外延迟。

```
void speed_task(void *arg) {
    tick_t next = now();                    /* 绝对时刻，不用相对延时 */
    for (;;) {
        next += SPEED_PERIOD;               /* 累加而非重设，避免漂移 */
        sleep_until(next);                  /* 唤醒点即采样点 */
        t_adc = stamp();                    /* 采样时间戳 */
        y = adc_read(CH_SPEED);
        u = pd_step(y, ref, t_adc);         /* 内部用实测间隔算微分 */
        pwm_write(u);                       /* 输出紧跟计算，不留空隙 */
        t_end = stamp();
        budget_record(t_end - t_adc);       /* 记账：本拍耗时入环形缓冲 */
    }
}
```

这段代码有四个细节值得说明。**第一，用绝对时刻而不是相对延时。** `next += PERIOD` 让长期平均周期严格等于设定值；用 `delay(PERIOD)` 会把每拍的执行时间累加进周期，跑一天就漂出几十毫秒。

第二，唤醒点就是采样点。 若先唤醒再等 ADC 转换完成，采样时刻会比唤醒时刻晚一个转换周期，这个偏移要计入 d_i。

第三，输出紧跟计算。 计算完成到 PWM 写入之间不要插入日志、通信或调试语句，否则输出的抖动 J_o 会被这些操作放大。

第四，记账要轻。 `budget_record` 只做一次时间戳相减和一次环形缓冲写入，耗时在纳秒级；若它自己成了热点，测量就改变了被测对象。

86.10.2 时间预算表怎么画

任务表说明“每个任务需要多久”，预算表说明“一个周期里时间花在哪里”。图 86-8 是速度环的预算分解：一个 2 ms 的周期被切成三段——计算 0.30 ms、被抢占 0.17 ms、余量 1.53 ms。

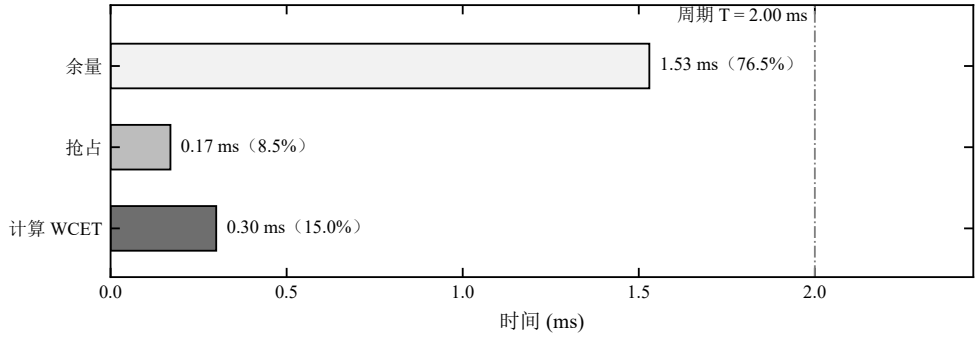


图 86-8　速度环一个周期内的时间预算分解

三个数字来自不同的计算。计算段是实测的 WCET；抢占段是 $R_i - C_i = 0.470 - 0.300 = 0.170$ ms，即被安全监测与电流环保护挤占的时间；余量段是 $T_i - R_i = 2.000 - 0.470 = 1.530$ ms。

这张图的读法与饼图不同：它按时间轴排布，能直接看出“最坏情况下还剩多少”。余量占 76.5%，计算只占 15.0%，说明这条回路的瓶颈不在计算量，而在“能不能被及时调度”。若某天发现计算段涨到 1.0 ms，先别急着优化代码，先确认是不是缓存或总线导致的 WCET 上升。

预算表还应当画出“超时线”。把 T_i 标在轴的右端，若某次实测的 R_i 越过这条线，说明当拍已经失约，而不是“稍微慢了一点”。这条线在评审时的作用，比任何百分比都直观。

86.10.3 四层架构与职责边界

实时控制软件的层次应当按“时间尺度”划分，而不是按“功能模块”划分。图 86-9 是本章采用的四层结构。

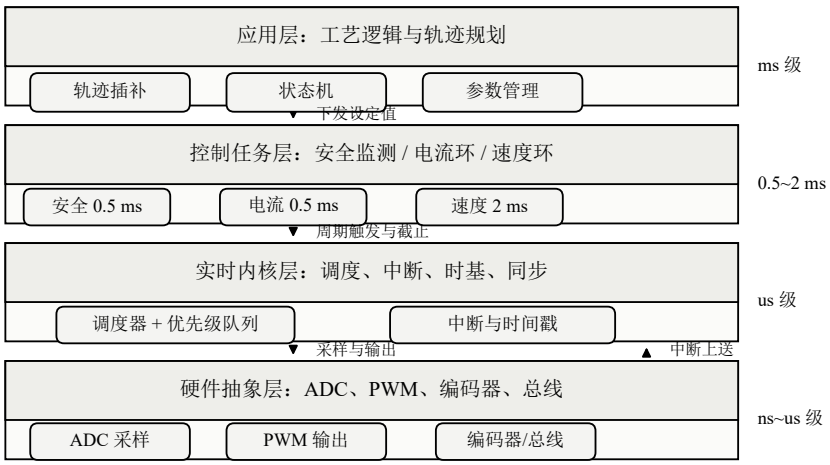


图 86-9　实时控制软件的四层结构与时间尺度

应用层处理毫秒级的业务：下发设定值、切换运行模式、响应上位机命令。这一层可以慢，但不能阻塞下面的层。

控制任务层是按周期运行的任务集合：采样、计算、输出。这一层的每个任务都有明确的周期与截止，是 §86.3 那张任务表的直接实现。

实时内核层负责调度与同步：节拍中断、任务切换、互斥与消息。这一层的时间尺度在微秒级，任何一次不当的临界区都会传导到上层。

硬件抽象层封装外设：ADC 触发、PWM 更新、编码器捕获、总线收发。这一层的时间尺度在纳秒到微秒，中断服务程序大多落在这里。

分层的关键是“跨层调用只能向下，不能向上”。控制任务层直接操作寄存器，看似省了一次函数调用，实际破坏了抽象，换芯片时整层要重写。反过来，硬件抽象层回调应用层的函数，会把应用层的执行时间拖进中断上下文，直接破坏时间预算。

图 86-9 右侧标注的四个时间尺度（ms、0.5–2 ms、μs、ns–μs）不是装饰，它们是分层的依据。判断一个功能该放在哪一层，看它的时间尺度落在哪一档即可。

86.10.4 测量方案：三件工具

工具一：空闲 GPIO 加逻辑分析仪。 在任务入口拉高、出口拉低，用逻辑分析仪看脉宽就是执行时间，看上升沿间隔就是周期与抖动。优点是零侵入（前提是 GPIO 操作本身只要几个周期），缺点是需要一台仪器，且只能同时看几路。

工具二：内核周期计数器。 Cortex-M 的 DWT->CYCCNT 每个时钟周期加一，读一次只要几个周期。把它与主频相除就是时间，分辨率到纳秒。缺点是计数器会溢出，需要处理回绕。

工具三：环形缓冲统计。 每拍把执行时间写入一个定长环形缓冲，运行一段时间后导出，算最大值、99.9 分位与直方图。优点是能长期运行、能抓到长尾；缺点是需要一段 RAM，且导出本身要停机或走 DMA。

三件工具应当同时使用：GPIO 用于看单拍的时序关系，周期计数器用于精确计量，环形缓冲用于统计长尾。只用其中一件，很容易得出片面结论——例如只看平均执行时间，会漏掉百万分之一的长尾；只看单拍波形，会以为抖动很小。

86.10.5 三个容易踩的工程细节

细节一：中断上半部要短。 中断服务程序只做“取数据、清标志、放队列”，把解析与计算放到下半部（任务或软中断）。本章电流环的 ISR 只有 $15\ \mu s$，就是因为它只做 ADC 读取与 PWM 更新，闭环计算在中断里完成但算法极短。

细节二：时间戳要用同一个时基。 采样时间戳、计算时间戳、输出时间戳必须来自同一个计数器，否则差值里会混入两个时基的偏差。常见错误是采样用 ADC 的转换完成标志、计算用系统节拍，两者相差几十微秒，算出来的 d_i 完全失真。

细节三：看门狗不能喂在中断里。 若看门狗在定时器中断里喂，主循环死锁时中断照常运行，看门狗失效。正确做法是在最低优先级任务里喂，或用一个“所有任务都报到了才喂”的与逻辑。

86.10.6 验收清单

工程交付前，表 86-10 的六项应当逐项打勾。缺任何一项，都不能说“实时性是验证过的”。

序号	验收项	合格标准	证据形式
1	任务表完整	周期、WCET、优先级、截止齐全	表格 + 版本号
2	响应时间可复算	用式 (86-4) 能从表中复现 R_i	计算脚本
3	最坏工况实测	满负载、最低电压、最高温度	波形 + 直方图
4	失约统计	连续运行 10^6 拍无失约	环形缓冲导出
5	抖动在预算内	实测值低于允许值的三分之一	抖动直方图
6	降级行为明确	过载时先丢哪类作业有定义	过载扫描曲线

第 3 项常被简化成“常温常压跑一遍”。这一项的价值恰恰在于边界条件：电压跌落时主频不变但 Flash 等待周期可能变化，高温时漏电流上升、时序裕量下降，满负载时总线争用最严重。三项叠加才是真实的最坏工况。

第 6 项是很多项目缺失的。系统过载时到底丢日志还是丢通信，应当在设计阶段就定下来，并写进验收标准。等到现场过载，再讨论“应该丢哪个”已经来不及。

86.10.7 中断优先级的一次具体配置

中断优先级的配置经常被写成“重要的放前面”，但“重要”不是一个可执行的判据。可执行的判据是“错过会怎样”与“执行多久”。

以 Cortex-M 的 NVIC 为例，优先级数值越小越高，通常配置成四组：故障保护（过流、过压、编码器断线）设为 0，电流环定时器设为 1，编码器捕获设为 2，CAN 接收设为 3，串口收发设为 4，系统节拍设为 15（最低）。六档之间留出空档，是为了以后插入新中断时不必重排全部。

三条规则决定了这个顺序。**第一，安全相关的最高。** 故障保护晚一个周期，功率器件就可能损坏，它必须能抢占任何其他中断。**第二，控制环按带宽排。** 电流环比速度环快三百多倍，必须排在前面；速度环不在中断里跑，所以不参与这个排序。**第三，通信与诊断最低。** CAN 与串口允许丢帧重传，晚几百微秒没有后果。

配置完还要检查临界区。若某段代码用 `BASEPRI` 屏蔽到优先级 2，那么电流环（优先级 1）在这段时间内无法响应。设临界区长 $20\ \mu s$，电流环的采样就被推迟最多 $20\ \mu s$——占 $62.5\ \mu s$ 周期的 32%，占允许抖动 $78\ \mu s$ 的 26%。一段看起来“很短”的临界区，已经吃掉了四分之一的抖动预算。

因此临界区的写法有两条纪律：只包住真正共享的几行赋值，不包函数调用；能用原子操作（LDREX/STREX、位带）的地方不用屏蔽中断。这两条做下来，临界区通常能压到 2–$3\ \mu s$，对电流环的影响降到 4% 以内。

最后一条容易忽略：中断里不要调用可能阻塞的服务。`printf`、动态内存分配、带互斥量的队列写入，都可能在中断上下文里等待，等待时间不可预测。本章的 ISR 只做寄存器读写与无锁队列投递，就是把这条落到了实现上。

86.10.8 从设计到交付的一条线

把本章的内容串起来，工程落地的路径是一条直线：从控制指标算出时间预算（§86.3 第一步），划分任务并分配周期（第二、三步），实测 WCET 填表（第四步），分配优先级并验证（第五、六步），用预算表与分层图固化设计（本节），最后按验收清单交付。

这条线上每一步都有明确的输入输出，也都有明确的失败信号：预算算不出来，说明控制指标没定清；WCET 测不出来，说明测量手段没搭好；响应时间复算不出来，说明任务表有遗漏；验收清单某项打不了勾，说明这一项从来没被验证过。

实时性不是某一个环节的属性，而是整条线的属性。任何一环松动，最终都会在“偶发失约”这个症状上暴露出来，而那时的排查成本，是设计阶段补上这一环的几十倍。

这条线落到装置上，就是图 86-10 的部署结构。工程侧三步：工程站改参数、版本库存快照、下载工具写进控制器；装置侧三步：控制器算、远程 I/O 采、现场设备动。两侧之间只有一条受控通道，装置侧不直接读工程站上的文件。

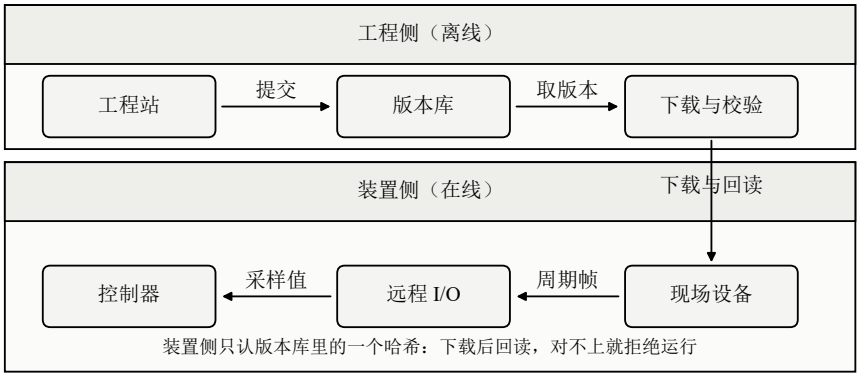


图 86-10　控制软件的部署结构与数据通路

这样划分的好处是责任清晰。现场只看装置侧，出问题先查回读的哈希；版本库是唯一真相，工程站上的临时改动不进版本库就不算数。反过来，如果允许现场直接改参数，同型号的两台设备会在三个月后变成两个不同版本，而没人说得清差异是从哪一天开始的。

还有一处容易忽略：下载之后必须回读校验。控制器把收到的映像算一次哈希，与版本库里记录的值比对，不一致就拒绝运行并保留旧版本。这条规则挡住的是传输过程中的位翻转和人为选错文件，成本只有几秒钟。

86.10.9 上电时序与首拍控制

前面九节讨论的都是稳态：任务按周期释放，每拍的时间预算固定，抖动的统计量稳定。但系统真正容易出问题的时刻，是复位之后到第一拍有效输出之间的那一段。这段时间里没有稳态可言，任务表的假设也不成立，而它偏偏决定了设备开机时会不会“抖一下”或者“冲一下”。

先看一段具体的启动预算。本章示例工程从复位向量到 PWM 使能，规划了 180 ms，表 86-11 把它拆成六项。

阶段	内容	预算 / ms
1	时钟树与电源稳定、看门狗接管	20
2	ADC 偏移与增益校准	12
3	编码器零位搜索与换相自检	60
4	参数区与校准系数校验	28
5	现场总线链路建立与从站扫描	40
6	安全回路与急停链路自检	20

六项加起来正好 180 ms，相当于九十个速度环周期。这段时间里控制器不能闭环，电机要么自由停转，要么被外力拖动。对一台需要“上电即定位”的设备来说，这九十拍就是一段必须交代清楚的空白。

顺序有一条硬约束：输出使能必须是最后一件事。 常见的错误顺序是“先初始化 PWM 外设、再校准 ADC”。PWM 外设一旦初始化，引脚就可能进入有效电平；如果此时占空比寄存器还是复位值，或者校准期间采样值无效，功率级会在几毫秒内收到一个错误的电压指令。正确做法是把 PWM 外设的初始化与输出使能拆成两步：初始化时把引脚强制为关断电平、比较值写零、刹车功能打开；直到第 6 项自检通过，才解除刹车、打开输出。中间任何一步失败，都停在“输出关断”这个安全状态上，宁可不开机，不可乱开机。

第二件事：调度器的零点不要设在复位时刻。 若第一个任务在 $t = 0$ 释放，那么第 1 拍的实际执行窗口可能只剩半个周期——因为启动代码占用了前半段。这一拍的采样时刻会偏，微分项会拿到一个错误的时间间隔。工程上的做法是让调度器在第一个完整周期边界起算：初始化完成后，等到定时器下一次溢出再启动任务释放，让每一拍都从完整的 T 开始。代价是最多等一个周期，收益是首拍的时间戳干净。

第三件事：首拍不要闭环。 冷启动时指令缓存与数据缓存都是空的，分支预测器没有历史，第一拍的执行时间接近本章测得的“未命中”上限 0.34 ms，而不是热态的 0.28 ms。更麻烦的是首拍读到的传感器值往往还没稳定——ADC 刚校准完，编码器的零位刚写入，电流传感器的偏置可能还在漂移。稳妥的做法是“空跑三拍”：照常采样、照常计算、不更新输出，三拍之后确认采样值在合理范围内，再打开输出。这三拍的开销是 6 ms，换来的是不会用一个坏数据去驱动功率级。

第四件事：看门狗的窗口要覆盖启动时间。 若看门狗超时设为 100 ms，而启动需要 180 ms，设备会陷入“启动—复位—再启动”的循环，现象是“指示灯闪一下就灭”，很容易被误判为电源问题。两种处理都可行：启动阶段用较长的超时窗口（例如 500 ms），进入稳态前再切回 100 ms；或者把喂狗点均匀分布在六个阶段里，让最长的一段不超过窗口。前者的风险是启动阶段失去保护，后者的风险是某个阶段卡住时喂狗仍在继续。多数工程选前者，因为启动阶段的功率级是关断的，风险可控。

第五件事：区分冷复位与热复位。 热复位（看门狗复位、软件复位）时，部分外设寄存器保持原值，编码器的零位偏移可能还留在寄存器里，但电机在这期间可能已经转过。若启动流程不区分这两种情况，就会出现“正常关机再开机正常、异常复位后零位错”的偶发现象。做法是读复位原因寄存器，热复位时强制重做第 3 项的自检，冷复位时可以直接用存储区里的零位值。这条差异不会出现在任何时序图里，却经常是现场“偶发飞车”的根源。

把这五条合起来看，上电时序的设计目标与稳态设计是同一个：**让每一个时刻的系统状态都是已知的**。稳态靠任务表把时间切片，启动阶段靠阶段划分把状态切片。前者失约的表现是超调，后者失序的表现是撞机。两者用的是同一套思维方式，只是一个在毫秒尺度，一个在百毫秒尺度。

86.10.10 提交前该看的十行代码

多数实时缺陷不是设计错了，而是某一行代码把设计意图抵消了。这一节给一份提交前的静态检查清单，十项，都能用文本搜索定位，不需要跑起来。它不能代替台架测试，但能在代码进主线之前挡掉大部分低级问题。

第一，相对延时。 搜 `delay(` 、`sleep(` 、`osDelay(` 。实时任务里出现相对延时，几乎一定意味着周期会随执行时间漂移。本章模板用 `next += PERIOD` 加 `sleep_until`，搜索 `next +=` 能确认每个周期任务都用了绝对时刻。

第二，微分分母。 找 `dt` 的来源。若它被赋成常数（`dt = 0.002`）而不是 `t_adc - t_prev`，就是 §86.6 那个把超调从 4.663% 推到 5.159% 的实现失误。这一行改起来只要两行代码，代价是控制器多存一个时间戳。

第三，输出与计算之间。 看 `pwm_write` 之前有没有别的东西。一行日志、一次总线发送、一个条件里的调试计数，都会把 J_o 放大。正确的形态是计算完立刻写输出，记账放在写完之后。

第四，中断里的打印。 搜 ISR 里的 `printf` 、`log` 、`uart_send` 。串口输出是阻塞的，一个字符的等待时间可能超过整个电流环周期。本章的 ISR 只做寄存器读写与无锁队列投递，就是为了避开这一条。

第五，动态内存。 搜 `malloc` 、`free` 、`new` 、`delete` 。实时路径上一次分配的时间不可预测，且可能触发碎片整理。启动阶段分配完，运行期只用静态池，是这类系统的通行做法。

第六，临界区的长度。 找 `lock` 、`disable_irq` 、`taskENTER_CRITICAL`，看它们包住了几行。§86.10.7 算过，20 μs 的临界区就吃掉抖动预算的 26%。临界区里只应出现共享变量的读写，不应出现计算、判断链或函数调用。

第七，忙等。 搜 `while (!flag)` 、`while (REG & BUSY)` 。带超时的忙等在外设初始化里可以接受，在实时任务里不可接受——它把执行时间变成了外部条件的函数，C_i 不再有上界。

第八，缓冲区的读写指针。 单生产者单消费者的环形缓冲，读写指针各由一个上下文写，不需要锁；多生产者就必须加锁或用原子操作。这一条错了，症状是“偶发丢数据”，而且极难复现。

第九，浮点与除零。 定点实现里搜除法，确认分母不可能为零。微分项的分母是时间间隔，一旦时间戳相同（同一拍读了两次），除法就溢出。加一行 `if (dt < TICK_MIN) dt = TICK_MIN;` 比事后查溢出便宜得多。

第十，喂狗点。 搜看门狗刷新函数，确认它不在中断里、不在最高优先级任务里。喂狗点若放在最高优先级任务中，任何低优先级任务的死循环都无法被检出，看门狗形同虚设。正确位置是“所有关键任务都跑过一遍之后”，通常放在最低优先级的监控任务里。

十项检查完，大约十分钟。它们覆盖不了时序分析，但能挡住“设计正确、实现走样”这一类问题——而这一类问题在现场故障中的占比，远高于分析本身出错。审查时还有一个通用技巧：**先看数据流，再看控制流**。数据从采样到输出经过哪些变量、哪些队列、哪些共享区，走一遍就能发现上面十项里的大部分。

86.11 优缺点与适用边界

任何方法都有它的适用范围。本节说清楚这套“任务表 + 响应时间 + 抖动预算”的做法在什么地方好用、在什么地方会失灵，以及它与相邻方法的区别。

86.11.1 五个优点

第一，判据可解析。 式 (86-4) 是一个不动点迭代，手算五行就能收敛。不需要仿真器、不需要目标板、不需要历史数据，纸笔五分钟就能判断“这个任务集可行不可行”。这让设计可以在写代码之前完成，而不是在调试阶段反复试。

第二，数字可复算。 本章每一个数字都能追溯到 `ch086_sim.py` 的某个函数。读者改了 WCET 或周期，重跑一次就能得到自己的表。可复算是工程结论能被评审的前提——不可复算的数字，只能被相信，不能被检验。

第三，预算可分配。 时间预算被拆成周期、WCET、优先级三个旋钮，每个旋钮的影响都能单独算出来。加功能时先算增量，再决定动哪个旋钮，而不是“先加上去试试”。

第四，失效可定位。 失约发生时，先看是哪一类任务失约，再看它的抢占源，再看 WCET 分布。三步之内能定位到具体原因。相比之下，“系统偶尔卡一下”这种描述，没有定位路径。

第五，工具成本低。 一个空闲 GPIO、一个周期计数器、一段环形缓冲，就能完成全部测量。不需要昂贵的总线分析仪，也不需要商业的时序分析工具。这让小团队也能做完整的实时性验证。

86.11.2 六个局限

局限一：WCET 不是常数。 整套分析建立在"每拍执行时间不超过 C_i"之上，而 C_i 本身是一个统计量的上界。缓存命中率、总线仲裁、DMA 抢占都会让它漂移。本章实测的 0.28、0.34、0.45 ms 三个值，相差 60%。填表时取哪个，直接决定结论的保守程度。

局限二：中断不服从任务模型。 中断的到达时刻由外部事件决定，执行时间取决于事件类型，优先级通常高于所有任务。它只能被折算成阻塞时间 B_i，或者单独作为最高优先级任务分析。突发式中断（例如总线错误风暴）会让这套模型彻底失效。

局限三：非周期任务只有近似界。 本章的随机事件任务用指数分布描述，平均间隔 8 ms，但没有任何硬上界——理论上可以连续到达。工程上要用偶发服务器（sporadic server）或轮询服务器给它一个预算，才能得到可证的界。

局限四：阻塞模型粗糙。 优先级继承给的是上界，实际阻塞取决于临界区的具体实现。若临界区里调用了不可抢占的内核服务，阻塞时间可能远超估计。本章 $B_i = 0$ 是设计上的简化，不是普遍情况。

局限五：多核下不可加。 两个核共享总线与内存控制器，一个核的突发访问会拖慢另一个核，两个核的 WCET 不能简单相加。多核的实时分析需要专门的干扰模型，本章的单核结论不能直接搬过去。

局限六：抖动只统计到二阶矩。 仿真给出的是均值、极差、纹波，实测给出的是直方图。但最坏抖动可能出现在百万分之一的样本里，而这类样本往往决定现场表现。统计量能给趋势，给不了保证。

86.11.3 适用边界

这套方法适用的系统有五个特征：单核、任务数在二十以内、最长与最短周期之比不超过一百、硬实时与软实时任务混合、控制周期在 0.5–20 ms 量级。本章的伺服驱动器正好落在这个区间。

不适用的场景有三类。**吞吐型系统**（视频编码、推理服务）关心的是平均吞吐而不是单次截止，用实时调度分析是南辕北辙。**强动态任务集**（作业随机到达且要求硬保证）需要随机实时理论或资源预留，固定优先级给不出保证。**异构多核**（CPU + GPU + NPU）的干扰关系复杂，需要厂商提供的时序模型，不能套用单核公式。

还有一类"看起来适用、实际不适用"的场景：任务数很少但每个任务都很大（例如单个任务占 80% 的 CPU）。这时 U 接近 1，任何一次中断或缓存失效都会导致失约，分析虽然能通过，工程上没有余量。判断标准不是"分析是否通过"，而是"余量是否够吸收未建模的扰动"。

86.11.4 与相邻方法的关系

时间触发架构（TTA） 把调度表在编译期算好，运行时只查表。它的可预测性最高，抖动最小，但灵活性最低——加一个任务要重排整张表。本章的固定优先级是"运行时决策、设计时验证"，TTA 是"设计时决策、运行时执行"。前者适合需求会变的系统，后者适合需求冻结的安全关键系统。

EDF 与 EDF-VD 按截止时间动态排队，利用率上限是 1.0，过载时失约被摊平。它的代价是需要频繁的上下文切换，且过载时的行为依赖截止时间的设置。§86.9.2 的对比说明了它的适用位置：软实时任务用 EDF 摊平，硬实时任务用固定优先级保护。

混合关键性（AMC） 为不同关键性等级的任务设置不同模式：系统正常时所有任务按低关键性预算运行，一旦高关键性任务有超期风险，就切到高关键性模式，把低关键性任务降级或丢弃。这是本章"过载时先丢哪类作业"那个问题的正式回答。

WCET 静态分析 用抽象解释在源码或机器码层面推导执行时间上界，不依赖实测。它给出的界比实测更保守，但覆盖面更全——实测只能覆盖跑到的路径，静态分析能覆盖所有路径。工程上两者互补：静态分析定上界，实测定现实值。

86.11.5 本章要点回顾

把十二节的内容收成五句。

第一，实时性的核心是“时间预算”：每个任务有一个周期，周期里有一段属于它，超了就失约。式 (86-1) 是这个预算的最简形式。

第二，响应时间分析把“优先级”翻译成“等待时间”，式 (86-4) 的不动点迭代是全部判据的基础。利用率上界只是充分条件，逐个验算才是判据。

第三，抖动与延迟都能折算成相位损失，但折算系数是 ω_c。同一个抖动，在速度环上是 0.009°，在电流环上是 3.20°，相差 356 倍。

第四，实现细节比抖动更贵。微分分母用标称周期，超调从 4.663% 涨到 5.159%，影响是抖动的 165 倍。

第五，过载行为要提前设计。固定优先级是悬崖，EDF 是斜坡，混合策略才能既保住关键任务、又不让非关键任务全丢。

这五句里，第一句定义问题，第二句给判据，第三、四句说明误差来源，第五句说明失效方式。掌握这五句，第 86 章的内容就掌握了大半。

86.11.6 三个反例：判据通过而系统仍然坏掉

上面的判据都能算，但通过判据不等于系统没问题。下面三个反例都发生在“表填得没错、算出来也通过”的情形里，失效的原因不在公式，而在公式的输入被低估了。

反例一：把漂移当抖动。 有人给速度环分配了 50 μs 的抖动预算，理由是“实测抖动均值 8 μs、极差 40 μs，够用”。这是把两种性质不同的时间误差混成了一个口袋。随机抖动的均值近似为零，不随时间累积；时钟漂移是单调的，线性累积。晶振偏差 50 ppm，每 2 ms 周期偏差 0.1 μs，看起来微不足道，但一秒就累积 50 μs，正好吃光整个预算；一小时累积 180 ms，相当于九十个完整周期。若控制器与上位机各用自己的晶振，采样时刻相对指令时刻的相位会持续滑移，表现为“设定值跟踪慢慢变差，重启又好了”。

修法不在算法而在口径：预算表中必须把“抖动”与“漂移”分成两行。抖动按统计量分配，漂移按时间乘以频率源精度分配。后者要么靠同步机制（总线上的分布式时钟、PTP）压掉，要么靠降低对相位敏感度的方式吸收。把两者相加当作一个数，是工程评审里最容易蒙混过关的错误。

反例二：用热缓存数据验收。 本章速度环的 WCET 在缓存命中时是 0.28 ms，未命中 0.34 ms，DMA 争用 0.45 ms。台架上反复跑热循环，填表填了 0.30 ms。若真实值是 0.34 ms，逐项重算的结果是：速度环响应 $0.470 \to 0.680$ ms，总线 $1.360 \to 1.400$ ms，日志 $5.470 \to 5.760$ ms，利用率 $0.7075 \to 0.7275$。全部仍在周期之内，判据依旧通过——这才是危险之处。

把 0.7275 与 $n=5$ 的上界 0.7435 并排看，余量只剩 0.016。也就是说，填错一个 0.04 ms，表面上“没超期”，实际上把“不用算就能保证”的余量吃掉了八成。此后任何一次加功能、任何一次中断变频繁，都没有缓冲。经验做法是把验收 WCET 取在冷启动首轮与高温工况下测得的 95 分位，而不是台架热循环的中位数；用中位数验收，等于用最好的情况给最坏的情况签字。

反例三：优先级表没变，阻塞变了。 有人新增一路编码器中断，读代码只改了中断服务程序，没动任何任务的优先级，认为“调度表没变，分析不用重做”。但中断的临界区会给速度环带来阻塞，B_i 从设计时的 0 变成 0.15 ms，速度环响应从 0.470 涨到 0.620 ms，涨了 32%。这个 0.15 ms 不在任何一张任务表里，也不出现在利用率的分子上——U 一点没变。

这三个反例有一个共同点：失的都是“输入的准确性”，不是“计算的正确性”。判据本身很结实，喂进去的数据不结实，结论就不结实。工程上比推导公式更花时间的，恰恰是守住这三个输入：抖动与漂移的分界、WCET 的取样条件、阻塞时间的来源清单。三张表签字之前，先问一句“这个数是在什么条件下测的”，比再推一遍公式有用得多。

86.11.7 本章没有覆盖的三件事

写清楚边界，比声称万能更有用。有三类问题本章的方法处理不了，遇到时需要另找工具。

第一件，复杂系统需要工具而不是手算。 本章的任务集只有六个任务、一个超周期 20 ms，手算五行就能收敛。任务数上到三十、周期互质、还有偏移与共享资源时，手工不动点迭代会漏解——漏掉某个临界瞬间，就会得出“可行”的错误结论。这时要用调度分析工具：Cheddar 做仿真验证，UPPAAL 做时间自动机的模型检验，或者商业工具做带偏移的精确分析。工具不会替你建模，但能保证“所有临界瞬间都检查过”。

第二件，功能安全认证不只是时序。 工业设备的 SIL 等级、机械安全的 PL 等级，要求的是一整套证据链：需求可追溯、诊断覆盖率、双通道比较、失效模式分析。本章给的响应时间判据只是其中一项必要证据，不是充分证据。一台通过了本章全部检查的设备，仍可能因为单点失效而无安全功能。要过认证，需要在架构层面做冗余与诊断，那属于另一套方法论。

第三件，非确定性负载没有 WCET。 神经网络推理、自适应滤波、在线辨识，这些计算的时间取决于输入数据。图像里多一个目标，推理时间就多一截。这类负载无法填进本章的任务表，只能靠三种手段约束：给固定的时间预算（超时就返回上一次结果）、给定固定的迭代次数（牺牲精度换确定性）、或者放到非实时核上（只交付“最新可用”的结果）。把这类负载直接塞进硬实时任务，是最常见的架构错误。

三件事合起来说明一个态度：这套方法解决的是“确定性负载在单核上的时间保证”，其余问题要交给别的工具与别的学科。知道边界在哪里，才知道什么时候该请人，而不是硬套公式。

86.12 Python 仿真代码

本章的数字来自两个文件：`code/ch086_sim.py` 负责调度仿真、响应时间分析、过载扫描与抖动闭环，`code/ch086_figs.py` 负责十张插图。前者不画图，后者不定义新的模型，只调用前者与 `simlib` 的接口。这样分工的好处是：改一个 WCET 只需重跑 sim，所有指标一起更新，不必等图重绘。

86.12.1 运行方式

在 `book` 目录下执行：

```
"D:\Anaconda3\envs\py311\python.exe" code\ch086_sim.py
"D:\Anaconda3\envs\py311\python.exe" code\ch086_figs.py
```

第一条命令打印八组结果：任务表与响应时间、固定优先级调度统计、过载扫描、连续域裕度、抖动闭环、实现失误对比、丢拍、时间预算。第二条命令把十张图写入 `figs/ch086/`，同时输出 SVG 与 PDF。两条命令都不需要参数，随机种子固定在 `SEED = 20261020`。

86.12.2 代码结构

`ch086_sim.py` 分四层。

第一层是任务模型： `Task` 与 `Job` 两个数据类，`task_set()` 返回本章的六个任务。日志任务的 WCET 是一个参数，过载扫描通过改这个参数遍历负载。

第二层是调度仿真： `build_jobs()` 按周期生成作业，随机事件用指数分布生成到达时刻；`simulate()` 用 10 μs 栅格做事件驱动的抢占式调度，支持固定优先级与 EDF 两种模式，并把执行片段存进 `__segments__` 供甘特图使用。

第三层是闭环仿真： `sim_jitter()` 逐拍推进，对象用解析解 `_prop()` 精确积分，采样时刻加入均匀抖动，计算延迟固定为 $0.3T$，微分可以用实测间隔或标称周期。`monte_carlo()` 用八个随机种子重复，返回均值与极差。

第四层是报告： `report()` 按 §86.1–§86.10 的顺序打印，每段一个标题。

`ch086_figs.py` 有十个函数，与十张图一一对应。其中图 86-1 的时序、图 86-9 的分层与图 86-10 的部署属于流程与架构类，只用 `flow_styles` 的泳道、圆角矩形、箭头与文字标注，不出现任何传递函数方块。所有图沿用 `plot_styles` 与 `flow_styles` 的约定：线宽统一、字号统一、图内中文用纯文本、公式单独放在 `$...$` 里，两者不写进同一个字符串。

86.12.3 核心代码

响应时间分析只有十行，但它决定了整张任务表是否成立。

```
def rta(tasks: list[Task], i: int) -> float:
    """固定优先级响应时间分析：R = C + sum ceil(R/Tj) Cj，超截止返回 inf。"""
    ts = periodic(tasks)                # 已按优先级升序
    ci, ti = ts[i].wcet, ts[i].period
    hp = ts[:i]                         # 更高优先级的任务
    r = ci
    for _ in range(200):
        rn = ci + sum(math.ceil(r / h.period) * h.wcet for h in hp)
        if abs(rn - r) < 1e-9:
            return rn
        r = rn
        if r > ti:
            return math.inf
    return math.inf
```

三处细节值得注意。**第一，初值取 C_i 而不是 0**，因为响应时间至少包含自己的执行时间。**第二，收敛判据用 $|R_{n+1}-R_n|<10^{-9}$**，不用固定迭代次数；迭代序列单调递增，收敛即不动点。**第三，一旦 $R_i>T_i$ 立即返回 ∞**，避免无界迭代。

抖动闭环的核心是“用实测间隔算微分”这一行：

```
h = (t[k] - t[k - 1]) if k else TS
h = h if (use_measured and h > 1e-9) else TS
e = 1.0 - y
dy = (y - y_prev) / h
u = KP * e - KD * dy
```

把 `use_measured` 置为 `False`，同一段代码就变成 §86.6 里那个“典型实现失误”，超调从 4.663% 涨到 5.159%。这是全章唯一一处用一个布尔量制造对照的地方，也是读者最容易复现的实验。

86.12.4 复现清单

表 86-12 把正文的关键结论与代码入口对应起来，便于逐项核对。

结论	代码入口	输出位置
响应时间 R_i 与 U	`rta()`、`utilization()`	报告第 1 段
抢占与响应抖动	`simulate()`	报告第 2 段
过载悬崖与斜坡	`overload_scan()`	报告第 3 段、图 86-7
相位损失折算	`nominal_margins()`、`current_loop_margins()`	报告第 4 段、图 86-5
抖动与实现失误	`sim_jitter()`、`monte_carlo()`	报告第 5、6 段、图 86-6
丢拍	`sim_jitter(drop_prob=0.02)`	报告第 7 段
时间预算	`rta()` 与常数 $C = 0.30$	报告第 8 段、图 86-8

最后提醒一句：代码里的数字是“计算结果”，不是“设定值”。若某次运行得到的响应时间与正文不一致，先检查 Python 与 control 库的版本，再检查是否改动过任务参数。正文的数字以 `SEED = 20261020` 这一次运行为准。

第87章 嵌入式控制器

87.1 动机与直观理解

控制算法写在纸上时只有几行公式，落到机器上却要占几百个指令周期、几个定时器通道和几十千字节内存。芯片不会因为公式漂亮就多给一个周期。本章讨论的就是这层落差：一颗成本几元到几十元的微控制器，凭什么能在一个开关周期内完成采样、变换、闭环、调制与保护；当这些工作装不下时，该砍哪里、换什么。

本章不重推控制律（PID 见第 44、106 章，预测控制见第 58、80 章），也不重推任务调度（第 86 章已把时间盒子讲透）。它盯住的是处理器与外设本身的资源：周期够不够、位宽够不够、外设路数够不够。

87.1.1 三个现场现象

现象一：满速时电机突然反转。 某风机驱动在额定转速以下运行正常，一旦超过 90% 额定转速，电机偶尔抽搐一下，示波器上相电流出现一个宽度约 2 ms 的反向脉冲。原因是反电动势观测器的输出在高速段超过了标幺值的 1.0，而实现用的是 Q15 格式，最大只能表示 0.99997。溢出后符号位翻转，观测值从正的最大跳到负的最大，控制器以为转速反向，立刻给出反向转矩。

现象二：低速爬行。 某转台在 5 r/min 以下出现一格一格的爬行，每格约 0.036°，正好是一个编码器计数。位置环用 12 位 ADC 采电位器做粗定位、编码器做精定位，编码器 2500 线四倍频，一转 10000 个计数。5 r/min 时一秒只有 833 个计数，而速度环的测速窗口取 1 ms，窗口内平均只有 0.83 个计数。M 法数不出小数，一个计数就代表 60 r/min，于是速度反馈在 0 与 60 r/min 之间来回跳，相对误差超过 100%。反馈一跳，电流给定就跳，转矩脉动推动转台一格一格地走。

现象三：电流波形上有毛刺。 某伺服在 16 kHz 开关频率下，相电流波形每周期都带一个尖峰。查了三遍硬件，最后发现 ADC 的采样触发点设在计数器回零处，正好落在下桥臂开通的开关瞬间。把触发点改到计数器峰值（中心对齐的中间点），尖峰消失，电流环带宽反而可以提高。

三个现象的共性：**算法没错，资源与时刻错了**。第一个是量程不够，第二个是分辨率不够，第三个是采样时刻不对。它们都不会在仿真里出现，因为仿真里的数没有位宽，采样也不需要时间。

把三个现象折算成代价，量级差别很大。溢出反转最凶险：它把“转得快”误判成“在倒转”，控制器按满反向转矩动作，机械上可能打坏联轴器，电气上可能触发过流保护而停机。低速爬行最烦人：它不是故障，指标上只表现为“低速平稳性差”，但用户一眼就看得见，而且往往在验收演示时才暴露。采样毛刺最隐蔽：它只抬高电流纹波与噪声，不影响平均值，常常等到 EMC 测试或温升超标时才被追出来。

三种现象也对应三条不同的排查入口。溢出要查变量的动态范围：把最坏工况下的物理量算出来，和所选 Q 格式的量程比一比，看哪个先到顶。爬行要查反馈的分辨率：把测速窗口内的计数个数算出来，小于 10 个就必然抖。毛刺要查时序：把采样触发点、死区插入时刻、桥臂开关沿三者在同一张时间轴上画出来，看有没有重叠。这三条入口分别对应 §87.2.2、§87.2.5 与 §87.7.4，本章后面会逐一给出算式。

还有一类更隐蔽的问题，三个现象都没有覆盖：**算得完但算得太慢**。代码在实验室空载时占用率只有 30%，一上负载、通信任务一开、日志一写，最坏响应翻倍，某一拍算不完，下一个周期直接用旧输出。这类问题不会让波形立刻变坏，而是让闭环的相位裕度悄悄掉十几度。它属于第 86 章的时间盒问题，本章只在占用率里留出余量，不重复讨论。

87.1.2 三个硬约束

嵌入式控制器的选型与实现，本质上是在三个约束下做分配。

算力。一个开关周期内可用的指令周期数是 $f_{\mathrm{clk}}/f_{\mathrm{sw}}$。168 MHz 主频配 16 kHz 开关频率，每拍 10500 个周期；48 MHz 配同样的开关频率，每拍只有 3000 个周期。这两个数字决定了同一套代码能带几轴、能跑多复杂的观测器。

把这笔账算完，才看得清余量在哪里。以 M4F 为例，481 周期乘 4 轴乘 16 kHz，等于每秒 3078 万个周期，占 168 MHz 的 18.3%。这是“净计算占用率”，不含中断开销。一次中断进出要压栈 8 个寄存器、出栈恢复、再叠上流水线冲刷，在 Cortex-M4 上按 60 周期估是稳妥的。四轴共用一个定时器触发时只进一次中断，60 周期摊到四轴是每轴 15 周期；若每轴各配一个中断源，就要摊四次、共 240 周期。这就是“多轴共用一个时基”在周期账上更划算的原因，也是它唯一的优点——代价是四轴被绑在同一拍上，一轴出问题四轴一起停。

外设。定时器要能输出三对互补 PWM 并带可编程死区；ADC 要能由定时器触发、两相或多相同步采样；编码器要有正交解码接口。这些不是“有就行”，而是路数与通道数要对得上轴数。

外设的账常被算错，因为“通道数”有三种口径。第一种是引脚数，数起来最直观，却最不可靠，因为引脚可以重映射。第二种是定时器通道数，一个高级定时器有 3 个互补通道，正好带一轴，四轴就要 4 个高级定时器，而多数入门芯片只有 1 到 2 个。第三种是 ADC 的同步采样能力：两轴各要两相电流，共 4 路，必须能由同一个触发沿同时启动，否则两轴的采样时刻差出几百纳秒，在高速下就是几度的角度误差。第三种口径最容易被漏掉，也最难补——软件无法把不同步的采样变同步。

内存。查表法要 Flash，缓冲区与栈要 RAM。一张 256 项的单精度正弦表占 1 KB，多轴可以共用；真正吃 RAM 的是采样缓冲、参数副本与栈，四轴加后台任务合计很容易超过 4 KB，而入门级芯片的总 RAM 只有 8 KB。

Flash 与 RAM 还要分开算。正弦表、参数默认值、故障码表放 Flash，占的是代码空间，通常几十到上百 KB，一般够用；而采样缓冲区、栈、浮点运算的中间变量占 RAM，才是瓶颈。一次 ADC 双缓冲要两倍采样深度的空间，四轴、三相、每相 16 个点、单精度，就是 $4\times3\times16\times2\times4=1536$ 字节，占 8 KB 的近两成。再留 2 KB 栈给中断嵌套，余量已经很薄。这三个约束里，算力最容易被高估，外设最容易被忽略，内存最容易被最后一刻发现不够。

87.1.3 一拍的预算长什么样

把单轴磁场定向控制的一拍拆成七块，逐块数指令周期，得到表 87-1。定点实现合计 481 周期，浮点实现合计 1220 周期，比值 2.54。这个比值不是常数，但量级稳定：定点靠移位与乘加，浮点靠 FPU 的单周期乘加，两者差距通常在 2 到 4 倍之间。

计算块	定点/周期	浮点/周期	占比（浮点）
ADC 读取与偏置校正	40	80	6.6%
Clarke 与 Park 变换	36	90	7.4%
电流 PI（两路）	60	140	11.5%
反 Park 与 SVPWM	110	280	23.0%
二阶观测器	150	420	34.4%
速度环 PI 与滤波	40	110	9.0%
保护与诊断	45	100	8.2%
合计	481	1220	100%

表 87-1 的单拍预算把“控制算法”还原成了一串整数。图的横轴是周期数，同一块用深浅两色对比两种实现；观测器与 SVPWM 两块加起来占了浮点预算的 57%，它们也是后面优化的首选对象。

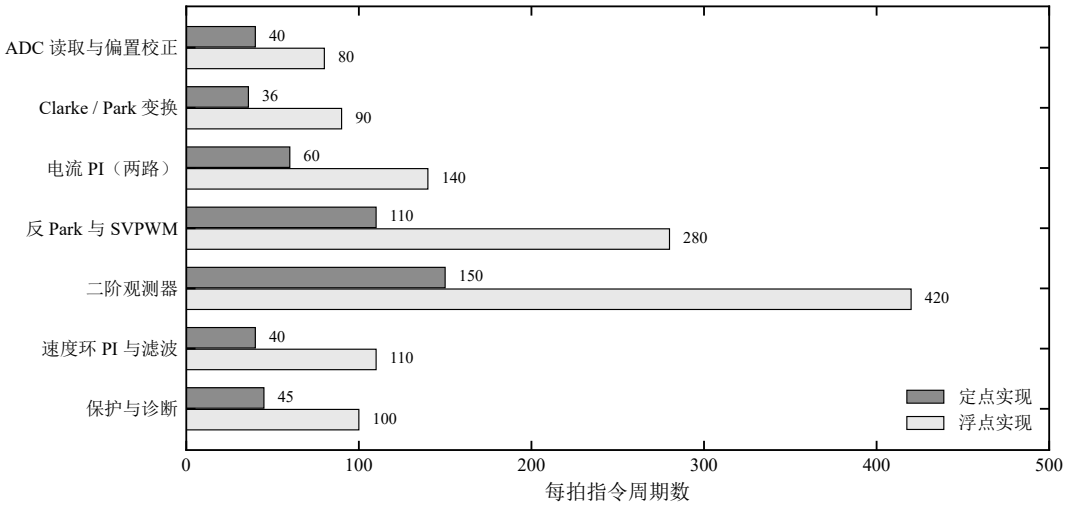


图 87-1　单轴一拍的指令预算构成

表 87-1 有一个容易看漏的地方：保护与诊断只占 8.2%，但它必须每拍都跑。一旦为了省周期把它挪到后台，故障检测的最坏响应就从一拍变成一个后台周期，安全等级随之下降。省周期的顺序不能只看占比。

七块的周期数不是凭感觉分的，每一块都能拆到指令级。SVPWM 占 280 周期，因为它要做扇区判断（三次比较）、三个作用时间的除法与饱和、六路比较寄存器的写入；M4F 的浮点除法约 14 周期，三次就是 42 周期，占这一块的一成半。观测器占 420 周期，因为它是二阶的：两次状态更新、一次输出修正、一组增益乘法，且全部是浮点，没有任何可以换成移位的机会。ADC 读取只占 80 周期，是因为它只做搬运、去偏置和一次标度变换，真正耗时的采样在硬件里，不占 CPU。这也说明一件事：**周期预算里最贵的不是“采样”，而是“算”**，所以优化应该从算法降阶入手，而不是从驱动入手。

定点合计 481 与浮点合计 1220 的比值是 2.54，比“软浮点对硬件 FPU”的 8 倍小得多。原因是这七块里，SVPWM 与观测器的定点化并不省那么多——它们主要是乘加，在没有 FPU 的芯片上定点乘加是 1 个周期，在有 FPU 的芯片上浮点乘加也是 1 个周期，差距只来自额外的移位与饱和。8 倍这个数字只适用于“没有 FPU、靠软件库模拟浮点”的情形，也就是表 87-2 里 M3 那一行。

87.1.4 本章要回答的五个问题

第一，一颗芯片最多能带几轴。答案是算力除以单轴预算，再留 30% 余量。第二，用定点还是浮点。答案取决于芯片有没有 FPU，以及被控量的动态范围。第三，外设怎么配。PWM 频率、ADC 位数、编码器线数三者互相牵制，没有单独最优。第四，算不过来时先砍哪一块。答案与“哪一块占周期最多”不完全一致。第五，什么时候必须换芯片。答案是一条边界，跨过去以后优化只能把问题推迟一个版本。

五个问题都要求一个数，而不是一段评价。第一个问的是 $n_{\max}$，本章的答案是 M0 配定点 4.4 轴、M4F 配浮点 6.0 轴、M7 配浮点 17.2 轴。第二个问的是“Kp=25 装不装得下”，答案是 Q12 及以上都装不下，只能用 Q10 或更粗。第三个问的是“16 kHz 还是 20 kHz”，答案是 16 kHz 下 ARR 5250、分辨率 12.36 位、死区占 1.6%，20 kHz 下升到 12.04 位与 2.0%，两者都可用，40 kHz 就只剩 11.04 位与 4.0%，开始吃紧。第四个问的是“先砍观测器还是先砍保护”，答案是都不砍，先降观测器阶数、再把速度环挪到 1 kHz 子节拍。第五个问的是“什么信号出现就该换平台”，答案是四条，写在 §87.11.7。

把这五个问题的答案摆在一起，会发现它们互相锁死。想提高 PWM 频率改善电流纹波，就要牺牲分辨率、抬高占用率；想提高分辨率，就要降 PWM 频率，环宽又不够；想多带轴，就要砍观测器，低速性能又变差。所谓“选型”，本质是在这组互相锁死的约束里找一个可行点，然后接受它的短板，而不是找一个没有短板的芯片。

87.1.5 一条贯穿全章的主线

本章反复用同一组数字：16 kHz 开关频率、168 MHz 主频、四轴、12 位 ADC、2500 线编码器。每次讨论一个资源，就把这组数字代入，看它先撞哪一堵墙。这样做的好处是结论可核对；代价是结论只在这组数字附近成立，换一个量级要重算。§87.3 给出的步骤就是“重算”的流程。

这组数字之间不是随便凑的，它们满足三条隐式关系。$f_{\mathrm{clk}}/f_{\mathrm{sw}} = 10500$，是单轴预算 481 的 21.8 倍，即理论上最多带 21.8 轴——实际只能带 6 轴，因为 70% 上限与浮点开销吃掉了大部分余量。$f_{\mathrm{sw}}/f_{\text{速度环}} = 16$，速度环每 16 拍跑一次，这个比值决定了速度环能取多高的带宽。$N_e f_{\mathrm{sw}}$ 与编码器计数率的比值决定了测速窗口能取多短，本章取 1 ms，正好是一个速度环周期。

换一组数字时，只有两条结论是尺度无关的。第一条是式 (87-4)：静差由最粗的一级量化决定，与增益无关，这条对任何位宽都成立。第二条是式 (87-17) 与 (87-18) 给出的可行窗口：量程与分辨率同时约束时，Q 格式的可选档位只有一到两档，这条对任何被控量都成立。其余结论——具体几轴、具体多少度相位、具体几个周期——都要重算。

87.2 数学原理

嵌入式的数学大多是一阶的：除法、取整、对数。难的不是公式本身，而是把物理量折算成整数时丢掉的那部分，以及丢掉之后如何反作用于闭环。

87.2.1 算力的两个基本式

设单轴一拍需要 N 个指令周期，主频 f_{clk}，开关频率 f_{sw}，带 n 轴，则 CPU 占用率为

$$U = \frac{N\, n\, f_{\mathrm{sw}}}{f_{\mathrm{clk}}} \tag{87-1}$$

反过来，给定占用率上限 $U_{\max}$，最大轴数是

$$n_{\max} = \frac{U_{\max} f_{\mathrm{clk}}}{N f_{\mathrm{sw}}} \tag{87-2}$$

式 (87-1) 里三个量都可测：N 用定时器计数或 GPIO 翻转加示波器测，f_{clk} 查手册，f_{sw} 是设计值。唯一容易出错的是 N 的口径——它必须是**最坏路径**的周期数，不是平均值。中断入口、现场保护、分支预测失败、Flash 等待周期都要算进去。

式 (87-1) 是线性的，前提是“每轴一拍的工作量不随轴数变化”。这个前提在磁场定向控制上成立：四轴各自采样、各自变换、各自闭环，轴与轴之间只共享时基。一旦出现跨轴耦合——双轴联动的插补、龙门同步的交叉耦合项、多轴协同的轨迹前瞻——线性关系就断了，必须按最坏的那一轴重算，而且耦合项本身还要额外加几十个周期。工程上判断能不能用式 (87-1)，只问一句：拆掉三轴、只留一轴，剩下那一轴的代码会不会变。

占用率上限取 70%，这个数的来历值得说清。剩下 30% 不是保险系数，是给“不归控制管”的事情留的位置：通信任务要在两拍之间收发报文，日志要写环形缓冲，参数要在停机时存 Flash，而 Flash 擦除一页在 72 MHz 上会停顿 20 到 40 ms，期间 CPU 取指全部等待。若参数存储放在控制中断里做，那一拍就直接丢了。此外还有中断抖动、温升降频、以及调试器挂上时额外的几十个周期。把余量按 30% 留，是因为这些杂项在实测中通常吃掉 10% 到 25%，留 30% 才有拒绝新需求的底气。

测 N 本身也有陷阱。最直接的办法是在中断入口翻转一个 GPIO、在出口再翻一次，用示波器读脉宽；更细的办法是用内核的周期计数器（Cortex-M3 以上都有 DWT），直接读出进入与退出的差值。两种办法测到的都是“这一拍实际花了多少”，而不是“最坏能花多少”。要拿最坏值，得把分支都走一遍——例如让观测器进饱和分支、让保护进故障分支、让除法遇到极小的分母。带 cache 的芯片（M7）还要额外注意：命中与不命中的取指差出好几倍，实验室里跑得欢的代码，换一段数据就可能超时。

表 87-2 把四款典型芯片代入式 (87-1)，开关频率统一取 16 kHz。浮点实现只在有硬件 FPU 的型号上成立；M3 没有 FPU，写浮点要交给软件库，单精度乘法的耗时约为硬件实现的 8 倍。

实现	芯片	1 轴	2 轴	4 轴	70% 上限轴数
定点	M0 @48 MHz	16.0%	32.1%	64.1%	4.4
定点	M3 @72 MHz	10.7%	21.4%	42.8%	6.5
浮点	M3 @72 MHz	216.9%	433.8%	867.6%	0.3
浮点	M4F @168 MHz	11.6%	23.2%	46.5%	6.0
浮点	M7 @480 MHz	4.1%	8.1%	16.3%	17.2

表 87-2 最刺眼的一行是 M3 写浮点：单轴就要 217%，连一拍都跑不完。这不是说 M3 不能做 FOC，而是说 M3 必须写定点。同一颗芯片，换一种数值表示，占用率从 10.7% 涨到 216.9%，相差 20 倍。选型时把“有没有 FPU”当成一个勾选项，往往就是从这里开始出错的。

87.2.2 定点数的位宽与量程

16 位有符号定点数用 Qn 表示，其中 n 是小数位数，整数部分占 $15-n$ 位。可表示的范围与分辨率分别是

$$|x| \le 2^{15-n} - 2^{-n}, \qquad q = 2^{-n} \tag{87-3}$$

表 87-3 列出本章用到的几档格式。位宽每减 2 位，量程扩大 4 倍，分辨率降低 4 倍。这是一条硬 trade-off，没有第三种选择。

格式	量程	分辨率	能表示 Kp=25	典型用途
Q16	±0.500	1.53×10^{-5}	否	归一化正弦表
Q14	±2.000	6.10×10^{-5}	否	标幺化电流
Q12	±8.000	2.44×10^{-4}	否	标幺化电压
Q10	±32.00	9.77×10^{-4}	是	增益与给定
Q8	±128.0	3.91×10^{-3}	是	位置计数
Q6	±512.0	1.56×10^{-2}	是	转速累积

表 87-3 的第四列是本章最实用的一栏。Kp 取 25 时，Q16、Q14、Q12 都装不下，只能截断到量程上限。截断不是报错，是静默饱和：代码照常运行，增益却变成了 0.5、2、8。§87.6 会看到，这三种情况的静差分别是 66%、13% 和 0.19%，而工程师在示波器上看到的只是“响应有点慢”。

定点乘法的细节决定了误差的另一半。两个 Qn 数相乘，结果是 Q2n 的 32 位数；要变回 Qn，必须右移 n 位并处理高低字。这里有两个容易忽略的点。第一个是舍入：直接右移等于向负无穷取整，每做一次就给结果注入 $-q/2$ 的直流偏置，一拍做二十次乘法，偏置就累积成可见的静差；标准做法是先加半个 LSB 再右移，即 $(a\cdot b + 2^{n-1}) \gg n$。第二个是饱和：中间结果必须用 32 位累加，最后一步再用饱和指令（Cortex-M 的 SSAT）压回 16 位，否则一旦超过 ±1.0，又是现象一那种符号翻转。

还有一个细节是“标幺化的时机”。同一个物理量在不同环节的量纲不同：电流是安培，电压是伏特，转速是 r/min，而 Q 格式只认标幺值。做法是在入口处一次性折算成 ±1 的标幺量，之后全链路都用标幺，只有在输出给 PWM 比较寄存器时才乘回量程。这样做的代价是要维护一张“基准表”（每安培对应多少标幺、每伏对应多少），好处是全链路的位数需求一致，不必为每个变量单独选 Q 格式。本章后续说“Q10”时，指的就是这条统一标幺链路。定标一旦被某个环节偷偷改掉（例如把电流基准从 10 A 改成 20 A），现象就是“增益突然减半”，而代码一个字都没动。

87.2.3 量化误差如何变成静差

设误差 e 被量化到步长 q 的格点上。当 $|e| < q/2$ 时，量化结果为 0，控制器输出为 0，被控对象停住不动。因此纯比例结构下的稳态偏差满足

$$|\delta| \le \frac{q}{2} = \frac{2^{-n}}{2} \tag{87-4}$$

式 (87-4) 有一个反直觉的推论：静差与增益 K_p 无关。增益只决定进入死区的快慢，不决定死区有多宽。想让静差变小，只有两条路——提高位宽，或者加积分。加积分会把静差压到 $q/(2K_p)$ 以下，代价是把死区变成极限环：积分器在每个量化格上积累，越过半格后输出跳变，位置来回蹭。

§87.6 的仿真给出的数字与式 (87-4) 对得上：Q6 的 $q/2 = 7.81 \times 10^{-3}$，实测静差 7.38×10^{-3}；Q8 的理论值 1.95×10^{-3}，实测 7.15×10^{-4}，被 12 位 ADC 的量化盖住了。

这个推导有一个容易漏掉的前提：被控对象在死区内必须“停得住”。若对象本身有积分性质（位置由速度积分而来，速度由转矩积分而来），那么死区不是“停住”，而是“误差缓慢累积到越过半格为止”，此时静差的上界仍是 $q/2$，但表现形式变成低频的、周期很长的爬行。这就是现象二的微观机制：误差每越过半格，控制器动一下，台面挪一格，再攒下一格。

加积分之后的静差与极限环，可以写得更具体。积分器在每个采样周期累加 $K_i e T$，只有当累计值超过半个输出 LSB 时，输出才真正变化。设输出步长为 q_u，则一个极限环周期内，误差要走过 $q_u / K_i T$ 个采样点，摆幅约为 $q/2$。于是极限环的周期是 $T \cdot q_u / (K_i T e_{\mathrm{lp}})$，随 K_i 减小而变长、随 q_u 减小而变短。工程上的含义很直接：想压住极限环，要么减小输出量化步长（提高 PWM 分辨率），要么降低积分增益（牺牲响应），没有第三条路。扰动信号（dither）是第三条半——在输出上叠一个峰峰值等于一个 LSB 的伪随机抖动，把量化器的确定性死区变成统计意义上的白噪声，代价是纹波变大一点，收益是消除低频爬行。

87.2.4 ADC 的量化步长

ADC 的量化步长由量程与位数决定：

$$q_{\mathrm{adc}} = \frac{V_{\mathrm{FS}}}{2^{b}} \tag{87-5}$$

本章的位置量程取 ± 2（标幺），12 位对应的步长是 9.77×10^{-4}。这个数直接决定位置反馈的最小可分辨增量，也决定了式 (87-4) 中静差的下限。位数再高，静差也不会低于 AD 的半步。

换成工程单位更直观。3.3 V 参考、12 位差分，一个 LSB 是 $3.3/4096 = 0.806$ mV。电流采样用 10 mΩ 分流电阻加 20 倍增益，每安培对应 0.2 V，于是 1 个 LSB 等于 4.0 mA；若额定电流 5 A，量化分辨率是额定的 0.08%。10 位时 LSB 变成 16 mA、0.32%，12 位时降到 4 mA，14 位时 1 mA 却已经低于运放失调与开关噪声的底噪，再高的位数买不到精度。表 87-9 的仿真正是这个结论：8 位时静差 6.5×10^{-3}，10 位时 7.1×10^{-4}，12 位以上稳定在 8.1 到 8.5×10^{-4}，不再改善。

这也解释了“位宽到某一点以后就没用了”的门槛在哪里：门槛不是 ADC 自己，而是链路里最粗的那一级。若 ADC 是 12 位、而控制器的输出只有 9.7 位（100 kHz 开关频率下的 PWM 分辨率），再提高 ADC 位数只是让噪声更清楚地被看见。

87.2.5 测速的三种方法

增量编码器的计数率与转速成正比。设编码器每转 N_e 个计数（线数乘四），转速 n（r/min），则计数频率为 $n N_e / 60$。M 法在固定窗口 T_w 内数计数，T 法测两个计数之间的时间。两者的相对误差分别是

$$\varepsilon_M = \frac{60}{N_e T_w n}, \qquad \varepsilon_T = \frac{n N_e t_{\mathrm{clk}}}{60} \tag{87-6}$$

第一个式子说明 M 法在低速时误差发散，第二个说明 T 法在高速时误差发散。令两式相等，得到交叉转速

$$n^{*}=\frac{60}{N_e\sqrt{T_w t_{\mathrm{clk}}}} \tag{87-7}$$

代入 $N_e=10000$、$T_w=1\ \mathrm{ms}$、$t_{\mathrm{clk}}=1\ \mu\mathrm{s}$，得 $n^{*}=189.7\ \mathrm{r/min}$。低于这个转速用 T 法，高于它用 M 法；两者都用就叫 M-T 法。把时钟从 1 μs 提高到 50 ns，交叉点抬到 848.5 r/min，也就是说高频时钟让 M 法的适用范围大幅扩张。

两个误差式的形状值得多看一眼。M 法的误差与转速成反比，因为窗口内的计数个数与转速成正比，数一个整数的相对误差就是 $1/N_{\mathrm{cnt}}$。T 法的误差与转速成正比，因为它测的是两个计数之间的时间，而这个时间被量化到 t_{clk} 的格点上，转速越高、间隔越短、格点占的比例越大。两条曲线在双对数坐标上是一条下降直线与一条上升直线，交点就是式 (87-7)。图 87-6 与表 87-10 给出了 1 r/min 到 3000 r/min 的八个数：1 r/min 时 M 法误差 600%、T 法 0.02%；3000 r/min 时 M 法 0.2%、T 法 50%。两端的差距超过三个数量级，所以“用一种方法测全速段”在原理上就不成立。

M-T 法的做法是在一个固定的低速窗口内同时数计数个数 m_1 与时钟个数 m_2，转速按 $60 f_{\mathrm{clk}} m_1/(N_e m_2)$ 计算。它在两端都只受“哪个量更粗”的影响，误差包络是两条曲线的下包络，代价是要多一个定时器做门控，且窗口不能短于低速时的一个计数间隔。对 1 r/min、10000 计数/转，一个计数间隔是 6 ms，所以低速端的速度环周期不可能低于 6 ms——这是原理限制，不是代码没写好。

87.2.6 PWM 的分辨率与死区

中心对齐 PWM 的周期寄存器值为

$$\mathrm{ARR}=\frac{f_{\mathrm{clk}}}{2 f_{\mathrm{pwm}}} \tag{87-8}$$

占空比的可调档位是 $\mathrm{ARR}+1$ 个，折算成位数

$$b_{\mathrm{pwm}}=\log_2(\mathrm{ARR}+1) \tag{87-9}$$

死区占一个开关周期的比例是

$$d_{\mathrm{dead}}=t_{\mathrm{dead}} f_{\mathrm{pwm}} \tag{87-10}$$

式 (87-8) 到 (87-10) 构成一个三方权衡：提高 f_{pwm} 能减小电流纹波、提高环宽，但同时降低分辨率、放大死区占比。168 MHz 时钟下，16 kHz 对应 ARR=5250、12.36 位、死区占比 1.6%；升到 100 kHz，ARR 只剩 840、9.72 位，死区占比 10%。后者的死区误差已经和电流环的输出同量级，补偿稍有不准就会造成波形畸变。

死区占比是一个比例，乘上母线电压才是真正的电压误差。母线 310 V、死区 1 μs、开关频率 16 kHz，平均电压误差是 $310\times 10^{-6}\times 16000=4.96\ \mathrm{V}$，占母线的 1.6%。这个误差的符号由电流方向决定：电流为正时它抬高实际电压，为负时它压低实际电压，所以它在电流过零处发生一次跳变，形成一个宽度约等于“死区时间对应电流”的平顶畸变。补偿的办法是按电流方向加一个等量的反向占空比，难点在于过零附近电流方向不可靠——这正是“符号保持带”要解决的问题，§87.7.3 会给出带宽的取法。

中心对齐还有一个额外好处，容易被忽略：它在计数器峰值与谷值各产生一次比较匹配，若把 ADC 触发放在峰值，采样点正好落在三相下桥臂同时导通的窗口中央，此刻分流电阻上的电流等于相电流本身，没有开关噪声污染。边沿对齐做不到这一点，因为它的“零矢量”窗口只有一半宽。所以同样是 16 kHz，中心对齐的有效采样窗口是边沿对齐的两倍，这也是它在电机控制里几乎成为默认选项的原因。

87.2.7 角度算法的误差上界

Park 变换需要电角度。三种常见做法的误差上界如下。查表加线性插值，表长 M，步长 $h=2\pi/M$，最大截断误差为

$$\varepsilon_{\text{tab}} \approx \frac{h^2}{8} \tag{87-11}$$

CORDIC 迭代 m 次，最后一级的旋转角为 $\arctan(2^{-m})$，角度分辨率约为

$$\varepsilon_{\text{cordic}} \approx \arctan(2^{-m}) \tag{87-12}$$

式 (87-11) 的来历是插值的二阶误差：正弦在区间内近似为抛物线，用弦代替弧，最大偏差出现在区间中点，等于 $|f''|h^2/8$，而 $|\sin''| \le 1$，于是误差上界就是 $h^2/8$（弧度）。$M = 256$ 时 $h = 0.0245$，$h^2/8 = 7.5 \times 10^{-5}$ 弧度，合 0.0043°，与表 87-4 一致。$M = 64$ 时步长翻四倍，误差翻十六倍，到 0.069°。

这些角度误差要不要紧，取决于它怎么传到转矩上。Park 变换用错角度 ε 时，交轴电流的误差约为 $I\varepsilon$（ε 用弧度），也就是 1° 的角度误差造成 1.75% 的转矩误差。表 87-4 里最差的 64 项查表是 0.069°，对应 0.12% 的转矩误差；CORDIC-12 是 0.024%；256 项查表是 0.0075%。三者都远低于电流环本身的稳态误差，所以**在常规伺服里，角度算法的选择不是精度问题，而是资源问题**。

资源账要连着算。256 项单精度表占 1 KB Flash，四轴共用一张（电角度与轴无关），实际只占 1 KB，不是 4 KB；但索引计算要做一次取模、一次移位、一次线性插值，共 15 个周期。CORDIC 不占表，占 60 个周期，且需要 12 次移位加法与一张 12 项的反正切小表（96 字节）。浮点 atan2 最省事，100 个周期，在 16 kHz 下四轴合计占 6.4% 的 CPU，若芯片有余量，用它换开发时间是完全合理的交易。真正的分界线是：当占用率逼近 70% 时，才值得把 atan2 换成查表；在此之前，换算法省下的周期会被调试成本吃回去。

表 87-4 把四种做法的误差与周期数放在一起。$M = 256$ 的查表插值误差 0.0043°，反而优于 12 级 CORDIC 的 0.014°，周期数只有后者的四分之一。CORDIC 的价值不在精度，而在不用乘法器、不用 ROM，适合资源极紧的场合。

算法	角度误差/°	周期数	适用场合
浮点 atan2	5.7×10^{-6}	100	有 FPU、非实时路径
CORDIC-12	0.0140	60	无乘法器
256 项查表插值	0.0043	15	有 1 KB ROM
64 项查表插值	0.0690	9	ROM 极紧

表 87-4 的排序会让人误以为“查表永远最好”。真实约束是内存与索引开销：入门芯片的 Flash 只有 64 KB，一张 1 KB 的正弦表占 1.6%，再叠加参数表、故障码表与标定数据，余地就不多了；而且电角度每拍都在变，索引要做一次标幺到表项的映射、一次取整、一次越界处理，这些都要算进那 15 个周期。选算法时先看 ROM 预算，再看精度需求，最后才比周期数。

87.3 设计步骤

选型常常被做成“看谁家 demo 跑得动”，然后靠试错收敛。下面这套五步流程把试错换成算术：每一步都产出一个可以打勾或打叉的数，五步走完，芯片要么定了，要么被否。

87.3.1 第一步：把需求写成三个数

需求文档里写的是“四轴、伺服级、支持 EtherCAT”，这些词不能直接代入公式。要先把它们翻译成三个数：轴数 n、开关频率 f_{sw}、电流环带宽 ω_c。

轴数决定乘法系数；开关频率决定每拍预算；电流环带宽决定采样频率的下限（一般取 $f_{\text{sw}} \ge 10\omega_c/2\pi$）。四轴伺服取 $n = 4$、$f_{\text{sw}} = 16$ kHz、$\omega_c = 2\pi \times 1200$ rad/s，三个数就齐了。

这三个数一旦定下，后面所有判据都是确定性的，不再有“感觉够用”的空间。反过来，如果三个数本身还在摇摆，选型就没有意义——今天按 8 kHz 选，明天改 20 kHz，昨天算的余量全部作废。

翻译的过程本身有讲究。“伺服级”对应的是带宽：点位伺服 200 到 400 Hz，连续轨迹伺服 800 到 1500 Hz，主轴类 100 到 300 Hz。带宽一旦定了，开关频率的下限就定了，因为一个开关周期内至少要采到电流环相位变化的一小段，经验取 $f_{\mathrm{sw}} \geq 10\omega_c/2\pi$。1.2 kHz 带宽对应 $f_{\mathrm{sw}} \geq 1.91$ kHz，实际取 16 kHz，是这条下限的 8 倍——留这么多不是浪费，而是为了把式 (87-16) 的延迟相位压到 40° 以内。

“支持 EtherCAT”这类通信要求不进这三个数，但它会吃掉占用率的余量。一帧 2 KB 的周期报文，用 100 Mbit 的从站控制器加 SPI 搬运，每毫秒要占 30 到 50 µs 的 CPU；这笔开销在第二步里会以“余量不足”的形式出现，而不是在第一步。

87.3.2 第二步：算占用率

把表 87-1 的单轴预算乘轴数，除以每拍总周期。图 87-2 画出了四款芯片在 1 到 4 轴区间的占用率，纵轴取对数，这样 4% 与 400% 能在同一张图上比较。70% 的横线是工程上限：剩下的 30% 要留给通信、诊断、参数整定与未来的功能追加。

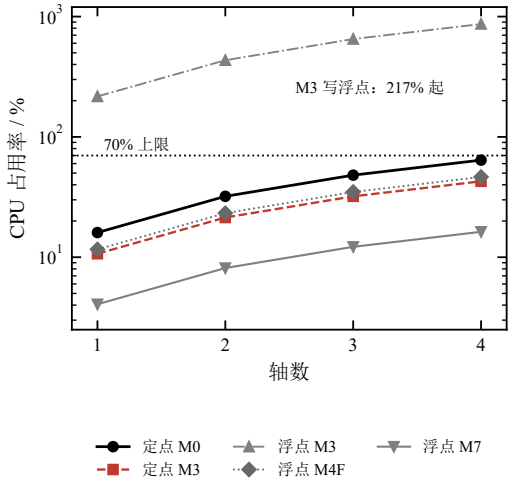


图 87-2　占用率随轴数的变化

图 87-2 里 M3 的浮点曲线从 217% 起步，斜率还是另外几条的 8 倍，这条线实际上宣告了“无 FPU + 浮点实现”这条路走不通。M0 的定点曲线在 4 轴处到 64.1%，刚好压在 70% 线下，意味着它能带四轴但不能再加功能。M4F 与 M7 的曲线留出的空间，就是它们贵出来的那部分价钱。

占用率算完还要看斜率。M7 每加一轴增加 4.1 个百分点，M4F 每加一轴增加 11.6 个百分点，M0 每加一轴增加 16 个百分点。轴数可能增长的场合，斜率比当前值更重要。

斜率可以手算核对。M4F 每轴 481 周期、16 kHz，即每轴每秒 770 万周期，占 168 MHz 的 4.1%？不对——表 87-2 给的是 11.6%，因为那里的“浮点实现”用的是 1220 周期，1220 乘 16000 等于每秒 1952 万周期，占 11.6%。同理 M0 走定点 481 周期，481 乘 16000 除以 48 MHz，得 16.0%。两个数字差在“用哪一列预算”，这也说明斜率不是芯片属性，而是“芯片 × 实现方式”的属性。M7 之所以只有 4.1%，一半来自主频，一半来自它的双精度浮点单元把观测器的长链运算压缩了。

余量不足时先别急着换芯片，先看占用率是怎么构成的。四轴 46.5% 里，观测器占 34.4%、SVPWM 占 23.0%，两块加起来接近六成。把观测器从二阶降到一阶（§87.9.3 做了这件事，420 周期降到 180），每轴每拍省 240 周期，四轴合计每秒省 1536 万周期，占用率立刻从 46.5% 掉到 37.4%；若同时把速度环挪到 1 kHz 子节拍，还能再省 3 个点。这两步都不改硬件，代价是低速观测精度与速度环带宽。选型与优化在这里是同一件事的两面：能靠改代码腾出余量，就不必为 10 个百分点换一颗贵三倍的芯片。

87.3.3 第三步：核对外设

算力过了不等于能用。定时器要几路、ADC 要几个单元几路同步采样，这些是硬门槛，不够就是不够，降主频也换不来。表 87-5 把四款候选的外设资源与四轴需求并排放。

候选	高级定时器	ADC 单元	同步采样通道	需求满足
A 入门 M0	1	1	2	定时器差 3，通道差 6
B 主流 M4F	2	3	4	定时器差 2，通道差 4
C 双核 M7+M4	4	6	8	全部满足
D 运动控制 DSP	2	2	2	定时器差 2，通道差 6

表 87-5 是本章最容易被跳过的一步。B 类芯片的算力余量看起来最舒服（46.5%），但它的高级定时器只有两个，四轴要么外扩 PWM 芯片，要么把两轴合并到同一个定时器的不同通道——后者会强制两轴共用开关频率与死区，参数整定失去独立性。

同步采样通道数也要按“每轴两相”算。三电阻采样的场合还需要第三路，通道数要乘 1.5。很多方案在纸面上够，接上第三路就少了两路。

定时器的账还要再细一层：一个高级定时器带三对互补输出，能驱动一轴；但若要做“三电阻采样 + 第四桥臂”，或者要输出刹车信号、编码器零位捕获，同一个定时器的通道就被占满了。B 类芯片的两个高级定时器，一个带两轴时只能共用同一组死区与同一开关频率，而两轴的母线电压、电感量往往不同，整定时必须取折中。表 87-5 的“定时器差 2”因此不是“再加两个定时器就好”，而是“这两轴的参数被迫绑定”。

ADC 的账同理。12 位转换在 30 MHz 的 ADCCLK 下要 15 个时钟，合 0.5 μs；加上采样保持的 0.2 到 0.5 μs，一次完整转换 0.7 到 1.0 μs。若一个 ADC 单元要顺序转换 4 路（两轴各两相），总时间就到了 3 至 4 μs，占 62.5 μs 周期的 6%，而且四路不是同时刻采的，第二轴的角度误差要多算 3 μs。要同时刻，必须两个 ADC 单元并行，各转两路——这才是“ADC 单元数”这一列真正卡住的地方。

87.3.4 第四步：核对内存

内存分两块：Flash 存代码与表，RAM 存缓冲与栈。按本章的口径，每轴约 90 KB Flash、12 KB RAM，四轴合计 360 KB 与 48 KB。

这两个数不是拍脑袋来的。每轴的 Flash 里，控制算法本体约 20 KB，观测器与滤波器 8 KB，参数表与标定数据 6 KB，通信栈分摊 20 KB，还有启动代码、库函数与诊断 36 KB。RAM 里，采样双缓冲 1.5 KB、状态与参数副本 2 KB、通信缓冲 2 KB、栈 3 KB、浮点中间变量 1.5 KB，剩下 2 KB 给日志与故障记录。把这些列出来会发现：算法本身只占很小一块，剩下的都是“为了让它跑起来”的开销。

Flash 的账容易算，RAM 的账容易漏。除了控制本身，还要加上通信缓冲区（一帧 EtherCAT 约 2 KB）、参数存储的镜像、以及中断嵌套时的栈深度。栈深度按最深层嵌套乘 1.5 倍留余量，这个系数不是保守，是因为浮点上下文保存会额外压 16 到 32 个字。

按 70% 上限核，A 的 RAM 需要 600%，直接出局；B 与 D 的 Flash 用量都到 70.3%，压线通过但不推荐——固件还要留升级空间。C 的 4.7% 与 17.6% 才是舒服的位置。

70.3% 这个“压线通过”要特别警惕，因为它是静态估算。固件一升级、日志一加密、协议栈一加安全层，Flash 就会以每版几 KB 的速度往上爬；爬到 100% 时，唯一的出路是删功能。工程上把 Flash 的可用上限压到 70%，正是为了让最后 30% 留给三个版本的演进。RAM 更紧，因为它的增长不是线性的——栈溢出不会报编译错误，只会在某个深嵌套中断里悄悄改写相邻变量。

87.3.5 第五步：串联三条判据做决策

三条判据不是并列打分，而是串联闸门：算力不过直接淘汰，外设不过要么外扩要么降规格，内存不过只能换型号。图 87-9 把这个顺序画成了一条竖向流程，右侧标出每个候选最先撞上的那一关。

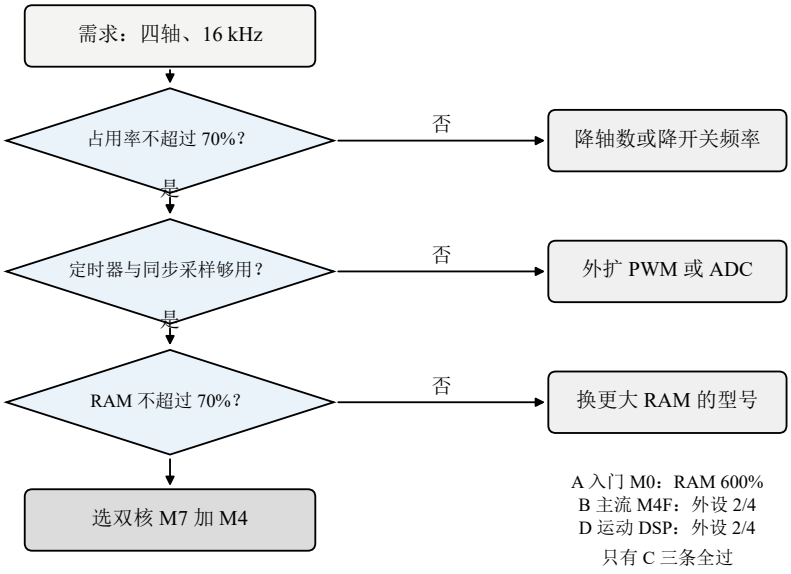


图 87-9　选型的三条判据与结论

串联的好处是省时间：算力一算就知道 A 出局、M3 加浮点出局，根本不必再核外设。坏处是容易把“外扩”当成免费选项——外扩一颗 PWM 驱动芯片要改板、要加 SPI 通信延迟、要重新做 EMC，这些成本往往超过换一颗更贵的 MCU。

流程走到最后只剩 C 全过，结论是“四轴 16 kHz 需要双核”。如果预算不允许，正确的退让顺序是：先降开关频率到 8 kHz（占用率减半，电流纹波增大），再降轴数，最后才考虑外扩。这个顺序的依据是改动成本，不是性能优劣。

退让也要看指标能不能守住。开关频率降到 8 kHz，ARR 从 5250 变成 10500，分辨率升到 13.36 位、死区占比降到 0.8%，占用率减半到 23.2%——账面全变好看了。但式 (87-16) 的延迟相位翻倍：同样的 1.2 kHz 带宽，$1.5\omega_c T$ 从 40.5° 涨到 81°，裕度直接为负。按 30° 的相位预算倒推，8 kHz 下能支撑的带宽只有约 444 Hz。所以“先降开关频率”只对带宽要求 400 Hz 以下的通用变频成立；对 1.2 kHz 的伺服，正确的退让是降轴数或换型号，降频率等于放弃指标。

三条闸门的顺序也不能换。算力最先算，因为它最便宜——查两张表、做三次乘除；外设第二，因为它要对着数据手册数通道；内存最后，因为代码还没写，只能按经验估。若把内存放第一位，会陷入“不知道要多少 RAM 所以没法选型”的死循环。这个顺序还有一个副作用：它天然把“贵但资源足”的型号留到了最后，避免一上来就被高配方案锚定。

87.4 连续情形

本章的对象是数字控制器，但设计往往从连续域开始：先按连续模型定带宽、算裕度，再离散化。这一节交代连续设计在嵌入式平台上成立的条件，以及这些条件被打破时误差从哪来。

87.4.1 电流环的连续设计

以 d 轴为例，被控对象是电阻与电感的串联，时间常数 L/R。按典型值 $L = 1$ mH、$R = 0.5\,\Omega$，转折频率 $R/(2\pi L) = 79.6$ Hz。电流环用 PI 把闭环带宽做到 1.2 kHz，比例增益按带宽与对象增益的乘积取，积分时间取 $L/R = 2$ ms 以对消对象极点。

这套设计在连续域里干净：相位裕度 60° 以上，超调小于 10%。它成立的前提是采样足够快、计算不花时间、执行器能立刻响应。三条前提在嵌入式平台上都要打折扣。

把数字代进去看得更清楚。被控对象的直流增益是 $1/R = 2$ A/V，转折点在 79.6 Hz。PI 零点放在转折点上，$T_i = L/R = 2$ ms；比例增益取 $K_p = \omega_c L = 7540 \times 0.001 = 7.54$ V/A；积分增益 $K_i = K_p R/L = 3770$ V/(A·s)。闭环的穿越频率就等于 1.2 kHz，相位裕度 65°。

这三个数里有一个是“不可能更大”的。电流环带宽的上限受开关频率约束，因为采样与保持本身就是延迟；在 16 kHz 下想把带宽做到 3 kHz，等于让闭环在采样频率的五分之一处穿越，任何一点额外延迟都会把裕度吃掉。所以电流环带宽通常是“给定开关频率下的结果”，而不是“设计者自由选的参数”。这也是为什么 §87.7.6 把两者写成一条绑定关系。

87.4.2 采样与保持带来的相位损失

零阶保持器在 ω_c 处贡献的相位滞后是 $\omega_c T/2$。16 kHz 对应 $T = 62.5$ µs，1.2 kHz 带宽对应 $\omega_c = 7540$ rad/s，滞后为

$$\varphi_{\text{zoh}} = \frac{\omega_c T}{2} = 0.236 \text{ rad} = 13.5^\circ \tag{87-13}$$

这 13.5° 是白送的，设计时必须从裕度里预先扣掉。再算上计算延迟：一拍内从采样到更新比较值耗时 2.9 µs，等效附加相位 $\omega_c \times 2.9\ \mu s = 1.3^\circ$，合计 14.8°。

连续域设计给的 60° 裕度扣掉 14.8°，剩下 45.2°。这仍然够用，但已经没有再做“带宽翻倍”试验的空间。§87.5 会把这两项延迟放到离散域重新算一遍，结论一致，只是口径不同。

这 13.5° 的来源值得记住：零阶保持器的等效延迟是半个采样周期，因为一拍内采到的值要维持一整拍，平均来看相当于晚了 $T/2$。所以“提高采样率”对相位裕度的贡献是线性的：16 kHz 给 13.5°，20 kHz 给 10.8°，32 kHz 给 6.7°，8 kHz 则是 27.1°。带宽不变时，把开关频率翻倍只能买回六七度，代价是占用率翻倍、PWM 分辨率掉一位。这笔交易在裕度还剩 45° 时明显不划算，只有在裕度掉到 20° 以下时才值得。

2.9 µs 的计算延迟是另一笔账，它随代码量变，不随开关频率变。观测器加一个二阶滤波、保护加两条判据，计算时间就可能从 2.9 µs 涨到 4.5 µs，多出的 1.6 µs 在 1.2 kHz 处是 0.7°。看起来很小，但它是“最坏路径”的增量，而设计裕度是按平均值留的——这就是为什么 §87.8 要求每次加功能都重测一次中断脉宽。

87.4.3 观测器的连续假设

反电动势观测器在连续域写成两阶状态方程，假设电流与电压都是连续可测的。实际拿到的是每个开关周期采一次的离散值，中间还夹着 PWM 的开关纹波。观测器带宽取得越高，对采样时刻越敏感；取 200 Hz 时几乎不受影响，取 2 kHz 时纹波会直接进入观测值。

经验做法是把观测器带宽压到电流环带宽的五分之一以下，并用采样时刻的对称性抵消纹波：在三角载波的两个顶点各采一次，取平均，纹波的一阶分量正好抵消。这一招在连续域里没有对应物，是纯数字技巧。

五分之一这条经验线可以算出来。观测器的输入是电压指令与电流采样，前者干净、后者带纹波；观测器相当于一个低通，带宽越高，纹波透过得越多。开关频率 16 kHz、观测器带宽 2 kHz 时，开关频率处只有 8 倍衰减，纹波几乎原样进入反电动势估计；压到 240 Hz（1.2 kHz 的五分之一）时，衰减到 67 倍，纹波被压到 1.5% 以下。代价是观测器对真实反电动势变化的响应慢了 4 ms，在转速突变时会滞后。

这个滞后对速度环不是问题，因为速度环本身的带宽只有 100 到 200 Hz；对电流环是问题，所以观测器不能用来做电流前馈。这也解释了为什么观测器降阶（§87.9.3）能把周期从 420 降到 180 而性能损失有限：降阶丢掉的正是那些带宽高于 240 Hz 的模态，它们在离散实现里本来就被采样率挡在外面。

87.4.4 连续设计的三个隐含前提

第一个前提是无限精度。连续域的增益可以是 25.0，定点域的 25.0 要看格式装不装得下。第二个前提是零计算时间。连续域里控制器是瞬时的，数字域里它要花 481 个周期。第三个前提是执行器连续可调。PWM 的占空比只有 ARR+1 档，12.36 位意味着最小调节量 0.019%。

三个前提各自对应一类现场故障：量程溢出、相位滞后、极限环。它们不会同时出现，但每一条都足以让一套在仿真里完美的参数在现场失效。§87.5 与 §87.6 分别处理前两条与第三条。

还可以补一条更基础的前提：连续域的模型参数是准的。数字域里，L 与 R 的误差会直接进入离散系数 a 与 b，而 $a = e^{-RT/L}$ 对 L 是敏感的——L 从 1 mH 偏到 0.8 mH，a 从 0.9692 变成 0.9617，看起来只差 0.8%，但 $b = (1-a)/R$ 从 0.0616 涨到 0.0766，涨了 24%，等于把电流环增益悄悄放大了四分之一。饱和或振荡就是这么来的。现场常用的做法是在上电时注入一个小信号辨识 R，再用温升模型修正 L；这一步在连续域设计里根本不存在，因为连续域的 L、R 被默认为真值。

87.5 离散情形

连续域的设计给出了“应该做到多少”，离散域要回答“实际能做到多少”。两者的差不在算法，而在时间被切成了一拍一拍，以及每一拍里的数都是有限位宽的整数。

87.5.1 一拍之内的五个时刻

图 87-3 把一个 62.5 μs 的开关周期摊开，按发生顺序标出五个时刻。t_0 计数器回零，硬件自动触发 ADC；$t_1 = 0.9$ μs 转换完成并置中断标志；t_2 中断响应，压栈与现场保护；$t_3 = 3.8$ μs 控制计算完成；t_4 比较值写入缓冲寄存器。

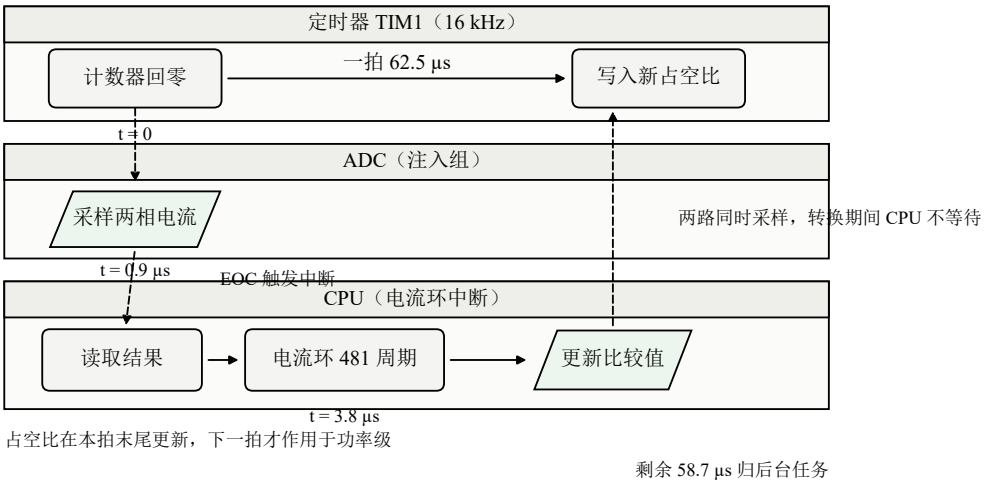


图 87-3　一个开关周期内的中断时序

图 87-3 里最值得注意的是 t_4 的位置。比较值写进的是影子寄存器，要等下一个周期计数器回零才真正生效。这不是设计缺陷，而是刻意为之：正在进行的这一拍必须保持完整，否则会出现窄脉冲。代价是输出晚了一整拍。

剩余 58.7 μs 归后台任务：速度环、通信、参数读写、故障记录。这段时间看起来很多，但它不是“空闲”，而是被低优先级任务填满的。一旦后台任务超时，它会挤占下一拍的入口，表现为控制周期的抖动——这正是第 86 章讨论的问题。

五个时刻里，前四个都可以压缩，只有 t_4 无法提前。ADC 转换时间由硬件决定，采样窗口留 0.2 到 0.5 μs 是为了让采样电容充到位，压短了会采到上一相的残压；中断响应在 Cortex-M4 上是 12 个周期固定开销，改不了；计算时间是唯

一可以靠优化缩短的一段，3.8 µs 里有 2.9 µs 是算，压到 2 µs 就能多出 0.9 µs 的余量。但从 3.8 µs 压到 2 µs 需要把代码砍掉三分之一，而这段代码里没有一个是冗余的——这正是 §87.9 讨论优化的起点。

87.5.2 离散化后的电流环

零阶保持离散化把 $L\,\mathrm{d}i/\mathrm{d}t + Ri = u$ 变成一阶差分：

$$i[k+1] = a\,i[k] + b\,u[k], \qquad a = e^{-RT/L}, \quad b = \frac{1-a}{R} \tag{87-14}$$

代入 $L = 1$ mH、$R = 0.5\,\Omega$、$T = 62.5$ µs，得 $a = 0.9692$、$b = 0.0616$。这两个系数在浮点里可以精确表示，在 Q15 里 a 要写成 31754/32768，误差 1.5×10^{-5}；单次看不出来，但观测器每拍迭代一次，误差按 a^{k} 累积，1 ms 之后偏差约 0.02%。

a 与 b 的物理意义值得说清。a 是一拍内电流自己衰减剩下的比例，0.9692 意味着每 62.5 µs 掉 3.1%，时间常数 2 ms 正好对应 32 拍衰减到 36.8%；b 是单位电压在一拍内产生的电流增量，0.0616 A/V，乘以母线电压 310 V 得 19.1 A，即满压一拍能把电流推高 19 A。这两个数直接决定控制增益的量级：b 越大，同样的占空比变化引起的电流变化越大，K_p 就要越小。

离散化方法也有讲究。前向欧拉给出 $a = 1 - RT/L = 0.96875$、$b = T/L = 0.0625$，与精确值的差分别是 0.05% 与 1.5%。在 16 kHz 下这点差别无关紧要，但在 8 kHz 或电感更小的电机上，前向欧拉会把极点推出单位圆，仿真直接发散。零阶保持离散化没有这个问题，代价是要算一次指数——工程上通常在初始化时用浮点算好，运行期只查表。

PI 控制器用增量式实现，输出是占空比的增量而不是绝对值：

$$\Delta u[k] = K_p\big(e[k] - e[k-1]\big) + K_i T\,e[k] \tag{87-15}$$

增量式的好处是不需要抗积分饱和的额外逻辑——输出限幅后把增量清零即可。坏处是它对 $K_i T$ 的量化更敏感：$K_i T$ 通常很小，Q10 的步长 9.77×10^{-4} 可能比它本身还大，量化后积分项直接消失。

87.5.3 一拍半的延迟

采样保持贡献半拍，PWM 锁存贡献一拍，合计 1.5 拍。在 $\omega_c = 7540$ rad/s 处折算成相位：

$$\varphi_{\mathrm{d}} = 1.5\,\omega_c T = 0.707\ \mathrm{rad} = 40.5^{\circ} \tag{87-16}$$

§87.4.2 只扣了 13.5°，那笔账只算了采样保持与计算耗时，漏掉了“锁存一拍”。补上之后，连续域给的 60° 裕度只剩 19.5°。

19.5° 仍在稳定范围内，但已经谈不上“裕度充足”。这就是为什么 16 kHz 的伺服电流环带宽通常做到 1 到 1.5 kHz 就到顶了：再往上，延迟吃掉全部裕度。想把带宽做到 2 kHz，只能把开关频率提到 32 kHz，而 32 kHz 会让占用率翻倍（M4F 双轴从 23.2% 涨到 46.5%）。

把式 (87-16) 反过来解，就得到带宽的硬上限：若只允许延迟占 30° 的相位预算，$1.5\omega_c T \le 0.5236$，即 $\omega_c \le 0.349/T$。16 kHz 下是 5585 rad/s，合 889 Hz；8 kHz 下是 444 Hz；32 kHz 下是 1778 Hz。这个式子解释了一个常见困惑：为什么把控制器增益调大，响应却不跟着快——增益能提，带宽提不上去，因为相位在 889 Hz 处已经用完了。§87.7.6 会把这条关系与增益整定放在一起再讲一遍。

想突破这条线，只有三条路，且都要动结构而不是动参数。第一条是提高开关频率，代价是占用率与分辨率。第二条是“双更新”：在一个 PWM 周期里采两次、算两次、更新两次，把锁存延迟从一拍降到半拍，总延迟从 1.5 拍降到 1 拍，上限提高 50%。第三条是改用预测型控制器，用模型把延迟补偿掉，代价是对 L、R 的准确性更敏感——§87.4.4 说过，L 偏 20% 就足以让增益偏 24%。

87.5.4 定点实现的误差传播

定点代码里每一步都可能丢位。以一次 Q10 乘法为例：两个 Q10 数相乘得到 Q20 的 64 位中间结果，右移 10 位回到 Q10，低 10 位被截掉。截断误差最大 2^{-11}，相对单次没问题；问题在于它发生在每一拍的每一个乘法上。

误差传播有三条路径。第一条是直通路径：误差直接进入输出，被对象积分后变成位置偏差。第二条是反馈路径：误差进入状态量，下一拍再被放大或衰减。第三条是死区路径：误差小到被量化成零，形成 §87.2.3 讨论的死区。

三条路径里，第二条最隐蔽。观测器的状态一旦被量化误差推偏，它不会自己回到真值，而是沿着观测器的收敛速度慢慢回来。观测器带宽 200 Hz 对应时间常数 0.8 ms，也就是 12.8 拍。这 12.8 拍里，偏差一直存在于控制回路中。

把三条路径的量级估出来，才知道该先修哪一条。直通路径的误差是单次乘法截断，$q/2 = 4.9 \times 10^{-4}$，乘上 20 次乘法（一拍里的乘法总数）得 10^{-2} 量级，占满量程的 0.5%。反馈路径的误差按观测器增益放大，若观测器增益是 0.3，每拍的偏差只衰减 30%，累积到稳态是直通路径的 3 倍多。死区路径不产生“误差”，产生的是“不动”——它的后果是静差，不是纹波。

三条路径对应三种修法，而且不能互换。直通路径靠“多留一位中间结果”解决，即累加器用 32 位、只在最后一步饱和；反馈路径靠“观测器状态单独用高一位的格式”解决，代价是多占 RAM；死区路径靠“加积分或加 dither”解决，代价是纹波。工程上最常见错误是拿第二种修法去治第一种症状——把增益调大想让静差变小，结果噪声被放大，纹波更糟。

87.5.5 采样时刻的选择

同样是 16 kHz，采样点放在哪里差别很大。边沿对齐时计数器从 0 加到 ARR 后立即回零，采样点只能放在回零瞬间，正好与开关沿重合；中心对齐时计数器先加到 ARR 再减回 0，采样点可以放在峰值，也就是三相下桥臂全部导通的时刻。

中心对齐加峰值采样有两个好处。其一，峰值时刻的电流纹波对称，采样值等于一个周期的平均值，不需要额外滤波。其二，避开开关沿，共模干扰最小。代价是同样的时钟下 ARR 减半，分辨率降低一位——13.36 位变成 12.36 位。

这一位不是白丢的。12.36 位对应最小占空比调节量 0.019%，在低速轻载时会产生可测的转矩脉动。补偿办法是在峰值与谷值各采一次取平均，用两次采样换回一位有效分辨率，代价是每个周期多一次转换，且两次采样之间电流已经变了，平均值与实际值差半个纹波周期。

三种采样方案要放在一起比才有意义。单点采样最省，占一拍里的 0.7 µs，但纹波直接进入反馈；双点平均把纹波压到 1/4（二阶分量不抵消，只剩四分之一），代价是第二个采样点已经晚了一拍，相位上多 0.5 拍延迟；同步采样用两个 ADC 单元同时采两相，没有时间差，但要多占一路 ADC。选哪一种取决于 L/R 与开关频率之比：比值越小，纹波越大，越值得双点平均。本章的 1 mH/0.5 Ω 对应 2 ms，16 kHz 下纹波约为额定的 8%，用单点采样就够。

死区插入时刻也要与采样点错开。死区设在开关沿之后 1 µs，若采样点也在开关沿附近，就会采到死区期间的“续流”状态，电流值偏低。中心对齐把采样点放在峰值，距最近一次开关沿已有半个周期，死区影响早就结束了。

87.5.6 离散域的性能核对

把 §87.5.3 的 40.5° 与 §87.6 的量化误差放在一起，可以对一套参数做“上机前”的核对：相位裕度是否大于 30°，静差是否小于工艺要求，极限环幅度是否低于可听阈值。

三项里最容易漏的是极限环。相位裕度与静差都能用公式估，极限环幅度只能靠仿真或实测。经验值是量化步长的 1 到 3 倍，具体倍数取决于控制器结构：纯比例结构取 1 倍，带积分结构取 2 到 3 倍。按 Q6 的步长 1.56×10^{-2} 算，极限环幅度约 0.016 到 0.047，已经能在一个 200 mm 行程的轴上看到 3 到 9 mm 的抖动。

拿本章的数字走一遍核对。相位裕度：60° 减 40.5° 等于 19.5°，大于 30° 这一条不满足，但因为 30° 是“舒适线”而非“稳定线”，19.5° 仍可用，只是不能再加延迟。静差：Q10 下实测 8.52×10^{-4}，量程 ±2，相对值 0.043%，若工艺要求 0.1%，通过。极限环：Q10 步长 9.77×10^{-4}，带积分取 2 倍，约 2×10^{-3}，折合 200 mm 行程上 0.2 mm，在可接受范围内。

同一套参数换成 Q6 就过不了：静差 7.38×10^{-3} 是 Q10 的 8.7 倍，极限环到 0.03 以上，200 mm 行程上抖 3 mm。这说明“三项核对”不是走形式，它能提前把 Q6 这种“能跑但指标不够”的方案挡在样机之前。

核对还有一个容易被忽略的变量：温度。铜电阻在 100 °C 时比 25 °C 高约 40%，R 从 0.5 Ω 涨到 0.7 Ω，a 从 0.9692 变成 0.9572，b 从 0.0616 降到 0.0611。增益变化 1%，看起来小，但静差与极限环都会随温度漂移——冷机调好的参数，热机后静差可能翻倍。严格的核对要在最高工作温度下重跑一遍，而不是只在室温下通过就算完。

87.6 稳定性与性能

量化对闭环的影响可以分成两层：一层是“变慢变糙”，即静差与纹波变大；另一层是“变不稳”，即等效增益下降导致相位裕度不足，或极限环自持振荡。第一层可以用指标衡量，第二层只能靠判据防守。

87.6.1 三个指标与一个附加项

评价一套嵌入式控制实现，看四个量：超调、调节时间、稳态静差、稳态纹波。前两个描述动态，后两个描述静态。浮点实现下这四个量由控制律与对象决定；定点实现下它们还受位宽支配。

本章的算例是一个位置环，对象为 $1/(s(s+1))$，PD 增益 $K_p = 25$、$K_d = 4$，对应自然频率 5 rad/s、阻尼比 0.5。仿真步长 1 ms，时长 3 s，位置量程 ± 2，速度由观测器给出。这套参数在浮点下的表现是超调 16.36%、调节时间 1.616 s、静差 8.5×10^{-4}。

四个量的定义要统一，否则表 87-6 的数字没法互相比较。超调取响应峰值超出终值的百分比；调节时间取进入并保持在 ±2% 误差带的时间；静差取最后 10% 时间段内 $|y - 1|$ 的平均值，而不是终值减 1——后者会被最后一拍的纹波偶然性主导；纹波取同一时间段的峰峰值。四个量里，静差与纹波对位宽最敏感，超调与调节时间主要由增益决定，这也是表 87-6 上半部分“超调为 0”却完全不可用的原因。

仿真本身有三处简化，读表时要记住。第一，被控对象是线性的，没有摩擦与齿槽，真实系统的静差会被摩擦力“锁”在更大范围内。第二，速度由观测器给出而不是差分，避免了对量化噪声的二次放大，这相当于给了定点实现一个偏乐观的条件。第三，采样与计算不占时间，§87.5 的 1.5 拍延迟没有建模。三处简化都让结果偏好，所以表 87-6 的边界应当当作“最好情况”来读。

87.6.2 位宽扫描

把同一套参数换成不同 Q 格式重跑，结果列在表 87-6。表中“Kp 实取值”是量化后的实际增益，它决定了这一行的全部行为。

格式	Kp 实取值	超调/%	调节时间/s	静差
浮点	25.000	16.362	1.616	8.50×10^{-4}
Q16	0.500	0.000	未收敛	6.59×10^{-1}
Q14	2.000	0.000	未收敛	1.32×10^{-1}
Q12	8.000	0.262	1.600	1.91×10^{-3}
Q10	25.000	16.362	1.616	8.52×10^{-4}
Q8	25.000	16.363	1.616	7.15×10^{-4}
Q6	25.000	16.364	1.592	7.38×10^{-3}

表 87-6 分成两半。上半部分（Q16、Q14、Q12）的 Kp 被量程截断，超调归零不是“调得好”，而是“根本没走到”——系统在 3 s 内没进入 2% 误差带，静差分别是 66%、13%、0.19%。下半部分（Q10、Q8、Q6）的 Kp 精确等于 25，动态指标与浮点几乎一致，差别只在静差。

这个分界说明一件事：**定点实现失败，多数不是因为精度不够，而是因为量程不够**。Q16 的分辨率高达 1.53×10^{-5}，比 Q10 精细 64 倍，却连一个 25 的增益都装不下。工程师盯着“分辨率”选格式，就会掉进这个坑。

上半部分的三个“静差”要正确解读。它们不是稳态误差，而是“3 s 内没走到”的残差：Kp 被截断成 0.5 时，闭环自然频率从 5 rad/s 掉到 0.707 rad/s，时间常数放大约 7 倍，3 s 只走了两个时间常数，剩下 66% 的距离；Kp=2 时自然频率 1.41 rad/s，3 s 走完四个时间常数，剩 13%；Kp=8 时已经基本到位，只剩 0.19%，看起来“能用”，其实动态响应慢了三倍。三者都表现为静差，来源却都是量程，这也是为什么只看最终误差会误判。

Q12 那一行最容易被误读：超调 0.262%、调节时间 1.6 s、静差 1.9×10^{-3}，三个数字都不难看。但它的 Kp 只有 8，是设计值的三分之一，闭环带宽随之掉到 1/3。若把这套参数装上机床，用户会说“定位倒是准，就是软”。所以位宽扫描不能只看静差，必须同时看 Kp 实取值——这一列才是根因。

87.6.3 静差与位宽的关系

图 87-5 把表 87-6 的静差画成对数柱状图，柱顶标出实际的 Kp。三根高柱对应量程截断，四根矮柱对应正常工作区。矮柱里 Q6 明显抬高，因为它的量化步长 1.56×10^{-2} 对应的死区半宽 7.81×10^{-3} 已经大于其他误差源。

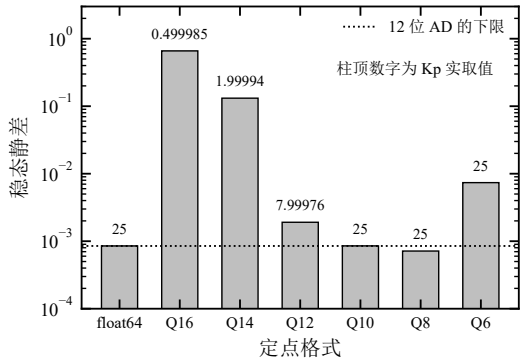


图 87-5　定点位宽与稳态静差

Q10 与 Q8 的静差都在 8×10^{-4} 附近，前者理论值 4.9×10^{-4}，后者 2.0×10^{-3}。实测值反向，是因为实验里位置反馈经过 12 位 ADC 量化，ADC 的半步 4.88×10^{-4} 与 ADC 自身的非线性把两者的差别抹平了。换句话说，位宽到 Q10 以后，静差已经不由 Q 格式决定，而由 AD 决定。

这条结论有实用价值：继续提高定点位宽没有收益，钱应该花在 ADC 上。反过来，如果 ADC 只有 10 位（半步 1.95×10^{-3}），Q12 以上的位宽全部浪费。

更一般地说，静差有三个地板，取最高的那个。第一个是 Q 格式的半步 $2^{-n}/2$，第二个是 ADC 的半步 $V_{\mathrm{FS}}/2^{b+1}$，第三个是传感器与摩擦带来的等效误差，通常比前两者大一个量级。12 位 ADC 配 ±2 量程，地板是 4.88×10^{-4}；本章实测的 8.5×10^{-4} 是它的 1.7 倍，差额来自 ADC 的非线性与观测器残差。若换成 16 位 ADC，地板降到 3.05×10^{-5}，此时 Q10 的 4.88×10^{-4} 就成了主导，必须同时把 Q 格式提到 Q12 以上才有意义。

三个地板的关系也解释了“为什么高精度平台要用浮点”。浮点的分辨率随数值大小浮动，小误差时自动变细；定点只有一个固定地板，想压低地板只能加位宽，而位宽一加，量程就掉。浮点用 32 位换来了“地板随信号缩放”的能力，这是它在高精度场合的真正优势，而不是“算得快”。

87.6.4 量化死区与极限环

死区不是静止的。系统进入死区后停在某个位置，但外界的扰动（负载变化、温漂、齿槽力矩）会把它推出去；一出去，控制器立刻给出一个量化格大小的输出，把它推回来，再停住。这个“推出去—推回来”的过程就是极限环，频率取决于扰动大小与死区宽度。

极限环的幅度上限是半步，实际幅度通常是半步的 1 到 2 倍，因为控制器输出也是一个量化格，冲过头要再回一次。按 Q6 的半步 7.81×10^{-3} 算，峰峰值可达 3×10^{-2}，在 200 mm 行程上就是 6 mm。

抑制极限环有四种手段：提高位宽、加抖动（dither）、把量化改成舍入而非截断、在控制器输出加一阶低通。第一种最直接但受量程约束；第二种在低速时有效，高速时抖动本身变成噪声；第三种能消除直流偏置但不能消除死区；第四种把极限环压成低频漂移，代价是相位滞后。

极限环的频率也能估。设对象为 $1/(s(s+1))$、控制器为 PD，极限环的周期大致等于“误差从 $-q/2$ 走到 $+q/2$ 所需的时间”的两倍。误差的爬升速度由积分项或扰动决定，若扰动是常值 d，爬升速度是 $K_p d$，周期约为 $2q/(K_p d)$。代入 $q=9.77\times10^{-4}$、$K_p=25$、$d=0.01$，周期约 7.8 ms，合 128 Hz。这个频率落在人耳可听范围之外，但正好落在机械结构的低阶共振带附近——这就是为什么量化极限环有时会激起明显的机械嗡鸣，而示波器上却只是一条细线。

四种抑制手段要按“症状”选，不能按“喜好”选。若极限环频率高、幅度小，优先加输出低通；若频率低、幅度大（表现为缓慢的爬行），优先加 dither；若整个工作范围内都有固定偏置，先把截断改舍入；只有在以上都无效、且量程允许时，才提高位宽。顺序反了，就会出现“位宽提到 Q14、极限环照旧”的情况——因为地板已经不是量化，而是摩擦。

87.6.5 量化对稳定裕度的影响

描述函数分析把量化器等效成一个增益随输入幅度变化的环节。输入幅度大时等效增益接近 1，幅度小时等效增益趋近 0。也就是说，系统在小信号下的开环增益比设计值低，相位裕度看起来更足，但穿越频率也随之漂移。

漂移的方向不固定，取决于量化器在环路中的位置。量化在反馈通道时，它等效为输出端的扰动，不改变环路增益；量化在误差通道时，它直接削减增益，让系统变慢。同样是 Q8，前者的表现接近浮点，后者可能慢一倍。

工程判据只有一条：**误差通道的量化步长应小于额定误差的 1/10**。额定误差按最大跟踪误差取，比如 0.05，则步长应小于 0.005，对应 Q8 以上。这条判据比“分辨率越高越好”实用，也比“按经验取 Q12”有依据。

这条 1/10 的来历是描述函数的斜率。量化器的等效增益在输入幅度等于一个步长时约为 0.5，等于两个步长时约 0.75，等于十个步长时 0.95。取 1/10 就是让等效增益不低于 0.95，环路增益的损失不超过 5%，穿越频率漂移不超过 5%。若额定误差只有三个步长，等效增益掉到 0.8，穿越频率掉 20%，原本 45° 的裕度会显得像 55°——不是更稳，而是更慢，操作者感觉“调好的参数突然变钝”。

量化位置还会改变“哪个通道最敏感”。误差通道量化时，小信号增益下降，系统在接近目标时变慢，表现为“最后一段爬得慢”；反馈通道量化时，环路增益不变，但输出端多了一个台阶状扰动，表现为“到了目标还在微跳”。两种症状在示波器上都能看到，但修法完全不同：前者要加分辨率或加积分，后者要加输出滤波。分不清位置，就会把两种修法都用一遍，最后靠凑参数过关。

87.6.6 反推最小位宽

把两条约束写在一起，可以得到定点格式的可行窗口。量程约束要求

$$2^{15-n}\ge K_p \quad\Longrightarrow\quad n\le 15-\log_2 K_p \tag{87-17}$$

静差约束要求

$$\frac{2^{-n}}{2}\le \delta_{\text{req}} \quad\Longrightarrow\quad n\ge \log_2\frac{1}{2\delta_{\text{req}}} \tag{87-18}$$

代入 $K_p=25$、$\delta_{\text{req}}=10^{-3}$，得 $n\le 10.36$ 与 $n\ge 8.97$，可行窗口只有 $n=9$ 与 $n=10$ 两档。这个结论比“取 Q12 比较稳”精确得多，也解释了为什么工业伺服里 Q10 与 Q9 出现频率最高。

窗口很窄这件事本身值得记住。定点格式不是“精度与量程的连续折中”，而是一个二维约束下的整数解，很多时候只有一两个解。先把两条约束写出来，再去试，比逐个格式跑仿真快得多。

窗口也可能一个解都没有。把要求提高到 $\delta_{\text{req}} = 10^{-4}$，下界变成 $n \geq 12.29$，上界仍是 $n \leq 10.36$，两条约束打架。此时有三条出路，按代价从小到大排。第一条是把增益折算到执行器量纲：$K_p = 25$ 是“每单位位置误差给多少伏”，若把它折成占空比，除以母线 310 V 得 0.0806，上界立刻升到 $n \leq 18.6$，窗口变成 Q13 到 Q18，绰绰有余。第二条是换 32 位定点，Q_n 的整数部分有 31 位，量程约束几乎消失，代价是每次乘法从 1 周期变成 2 到 4 周期。第三条是用浮点，回到“地板随信号缩放”的路子上。

第一条出路最容易被忽略，因为它不改硬件也不改位宽，只改“标幺的基准”。工程上叫“定标”：同一个物理量，选不同的基准值，落到 Q 格式里的数值就不同。选基准的原则是让最坏工况下的数值接近但仍小于量程上限，这样既用满了动态范围，又留了饱和余量。定标表一旦定下，就成了硬件与固件之间的接口文档，改它要两边同时改——这也是 §87.10.4 要求参数带版本号的原因之一。

87.7 参数意义与作用

嵌入式的参数分两类。一类是控制参数，比如 K_p、K_i，它们的意义在前面各章已经讲过；另一类是资源参数，比如 PWM 频率、ADC 位数、编码器线数，它们不直接出现在控制律里，却决定了控制律能不能兑现。这一节讲第二类。

87.7.1 资源参数清单

表 87-7 把六个资源参数与它们影响的性能指标对齐。这张表可以当成调参时的索引：出现某种症状，先查它对应哪一行。

参数	直接影响的指标	主要约束
PWM 频率	电流纹波、环宽、音频噪声	分辨率、死区、开关损耗
死区时间	低压区波形畸变、转矩脉动	直通风险、驱动能力
ADC 位数	静差、噪声底	成本、转换时间
采样时刻	电流测量误差、共模干扰	与开关沿的相对位置
编码器线数	低速分辨率、最高计数频率	成本、布线、接口上限
测速窗口	速度噪声、速度环带宽	延迟、最低转速

表 87-7 的六行里，只有 ADC 位数是“一次定死”的，其余五项都可以在固件里改。实际项目里最常见的浪费，是把可改的参数当成定死的，把定死的参数当成可改的。

87.7.2 PWM 频率的三方权衡

图 87-7 把三个量画在同一张图上：左轴是占空比分辨率，右轴是死区占比，横轴是开关频率。分辨率随频率下降（每倍频程降一位），死区占比随频率线性上升。

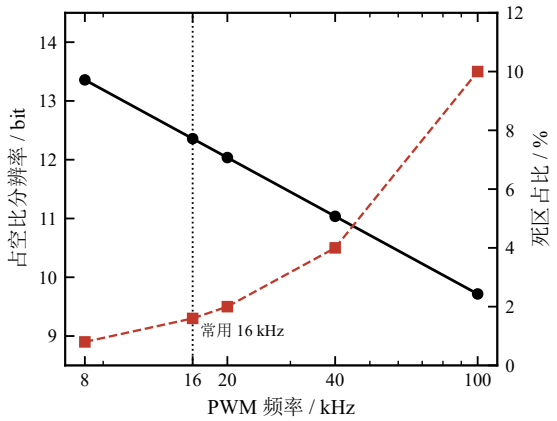


图 87-7　PWM 频率、分辨率与死区的权衡

表 87-8 给出五个频点的具体数字。8 kHz 时有 13.36 位分辨率、死区只占 0.8%，代价是电流纹波大、可能落入人耳敏感频段；100 kHz 时死区占到 10%，分辨率掉到 9.72 位，只有小电感、高转速的场合才值得。

频率/kHz	ARR	分辨率/bit	死区占比/%	1 LSB/%
8	10500	13.36	0.80	0.010
16	5250	12.36	1.60	0.019
20	4200	12.04	2.00	0.024
40	2100	11.04	4.00	0.048
100	840	9.72	10.00	0.119

表 87-8 里 16 kHz 与 20 kHz 的分辨率只差 0.32 位，死区占比却差 0.4 个百分点。这个区间是“用一点分辨率换一点环宽”的合理位置，也是工业伺服最密集的频段。再往上，分辨率掉得比环宽涨得快。

频率选择还要看电机的电感。电流纹波峰峰值近似 $V_{dc}T/(4L)$，T 减半纹波减半。电感 1 mH、母线 300 V 时，16 kHz 的纹波约 4.7 A，100 kHz 降到 0.75 A。如果电机额定电流只有 3 A，16 kHz 的纹波已经超过额定值，必须提频或者加大电感。

这三方权衡里，最容易被忽略的是“纹波电流带来的额外损耗”。纹波不产生平均转矩，却在绕组里产生 I^2R 损耗与铁损。4.7 A 的峰峰纹波，有效值约 1.4 A，若额定电流 5 A，绕组损耗要多出 8%。所以提频的收益不只是“环宽变大”，还有“温升降低”；而提频的代价除了分辨率与死区，还有开关损耗——每开关一次，MOSFET 的栅极电荷与输出电容都要充放一次，16 kHz 到 32 kHz，开关损耗直接翻倍。选频点实际上是在“纹波损耗”与“开关损耗”之间找总损耗最低点，多数中小功率场合这个点落在 12 到 20 kHz。

87.7.3 死区与它的补偿

死区是为了防止上下桥臂直通，通常取 0.5 到 2 μs，具体值由功率器件的关断时间加驱动延迟决定。死区期间输出电压不由占空比决定，而由电流方向决定：电流为正时，死区把输出拉低一个 t_{dead}/T 的等效占空比；电流为负时拉高同样的量。

这个误差在电流过零附近最讨厌。电流方向在过零处翻转，误差也跟着翻转，形成“过零畸变”：相电流在零点附近出现一个平台，转矩出现六倍频脉动。补偿办法是按电流符号加上或减去一个等于死区宽度的占空比修正量，符号由电流采样值的正负决定。

补偿的难点在符号判断。电流接近零时噪声会让符号乱跳，补偿量跟着乱跳，反而放大畸变。工程做法是设一个死区（比如额定电流的 2%），符号在这个带内保持不变，带外才更新。这个“符号保持带”的宽度需要现场调，取小了抖，取

大了过零畸变回来。

补偿量本身也要按母线电压归一化。死区造成的电压误差是 $V_{dc} t_{\mathrm{dead}} / T$，与电流大小无关；但控制器的输出是占空比，所以补偿量是固定的 t_{dead}/T，占 16 kHz、1 µs 时的 1.6%。若把它写成一个固定的 Q10 数，0.016 对应 16 个 LSB，分辨率够用；若写成 Q15，对应 524 个 LSB，更精细，但 Q15 的量程只有 ±1，一旦母线电压波动、补偿量随之上调，就容易顶到上限。这也说明补偿量不必用与主链路相同的格式，可以单独定标。

死区还会让“最小可控占空比”变大。占空比小于死区占比时，实际输出电压为零——不是变小，是直接没有。这在低速轻载时表现为“给了指令但电机不动”，等指令累积到超过死区才突然动一下，形成起动时的顿挫。解决办法是在输出上加一个最小的“起动占空比”，或改用电流闭环让积分器自己越过死区。

87.7.4 ADC 位数与采样窗口

表 87-9 把 8 到 16 位的 AD 结果并排放。静差在 10 位以下由 AD 主导，12 位以上稳定在 8.1×10^{-4} 到 8.5×10^{-4} 之间，这个底是控制器自身残差，不是 AD 造成的。

位数	量化步长	静差	静差/指令
8	1.56×10^{-2}	6.55×10^{-3}	0.65%
10	3.91×10^{-3}	7.14×10^{-4}	0.071%
12	9.77×10^{-4}	8.50×10^{-4}	0.085%
14	2.44×10^{-4}	8.11×10^{-4}	0.081%
16	6.10×10^{-5}	8.14×10^{-4}	0.081%

表 87-9 最有信息量的是 10 位到 12 位这一跳：静差从 0.071% 涨到 0.085%，反而变差了。这不是测量错误，是 10 位时 AD 的量化死区恰好把控制器的残余振荡“压住”了，静差看起来小，实际是停在了更远的位置上。用静差单一指标评价会误判，要同时看纹波。

采样窗口则是另一回事。ADC 的采样保持需要时间，12 位在 60 MHz 时钟下约 0.46 µs。窗口开得太短，采样电容充不满，读数偏低；开得太长，窗口跨过开关沿，采到的是开关噪声。16 kHz 下窗口通常取 0.2 到 0.5 µs，正好落在峰值附近的一段平静期。

采样窗口还牵着一个常被忽略的指标：孔径抖动。采样时刻本身有几十皮秒到几纳秒的抖动，若此时电流正在快速变化（例如开关沿后的振铃），抖动就被转成幅度误差。di/dt 为 20 A/µs 时，1 ns 的抖动对应 20 mA 的误差，相当于 12 位 ADC 的 5 个 LSB。压抖动要靠时钟质量与触发路径，而不是靠提高 ADC 位数——这也是“12 位以上没收益”的另一半原因。

87.7.5 编码器线数与测速窗口

图 87-6 把 M 法与 T 法的误差画成两条直线，横轴是转速，纵轴是相对误差，两条线在 189.7 r/min 相交。交叉点左侧 T 法更准，右侧 M 法更准。

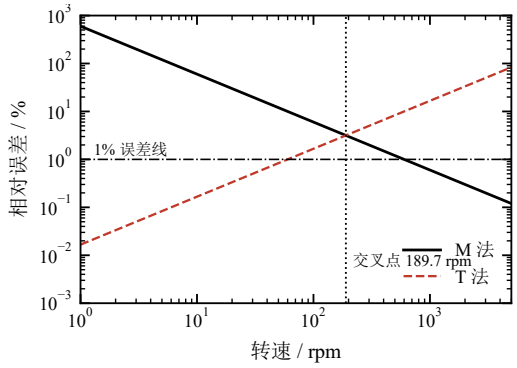


图 87-6　两种测速方法的误差与交叉点

表 87-10 给出八个转速点的具体数值。1 r/min 时 M 法误差 600%，完全没有可用性；T 法误差 0.02%，精度绰绰有余。3000 r/min 时反过来，M 法 0.20%，T 法 50%，后者已经不能用于闭环。

转速/rpm	M 法误差/%	T 法误差/%	取小者/%
1	600.00	0.02	0.02
5	120.00	0.08	0.08
10	60.00	0.17	0.17
30	20.00	0.50	0.50
100	6.00	1.67	1.67
300	2.00	5.00	2.00
1000	0.60	16.67	0.60
3000	0.20	50.00	0.20

表 87-10 的最后一行说明为什么高速时不能只用 T 法：时钟 1 μs 的量化误差被转速放大，3000 r/min 时单是时钟分辨率就贡献 50% 误差。把时钟提到 50 ns 可以把交叉点抬到 848.5 r/min，M 法的适用范围扩大 4.5 倍，这是换晶振或改用定时器捕获时钟的直接收益。

测速窗口的选择是另一条 trade-off。窗口从 1 ms 缩到 0.2 ms，速度环延迟减少 0.8 ms，但窗口内计数从 167 个降到 33 个，量化误差放大 5 倍。经验做法是让窗口等于速度环采样周期，并把速度反馈再做一阶低通，用滤波换噪声。

编码器线数不是越高越好，它有一个接口上限。定时器的正交解码在 168 MHz 下能可靠计数的频率约 5 MHz，2500 线四倍频在 3000 r/min 时是 500 kHz，只用了十分之一，余量充足；若换成 20000 线，同样的转速就是 4 MHz，接近上限，任何一点抖动都会丢计数。丢计数不会报错，只会让位置“少走一点”，累积成很大的偏差——这是位置环最危险的失效模式，因为它不可逆。

线数提高还会抬高布线要求。500 kHz 的差分信号在 5 m 电缆上已经开始明显衰减与串扰，必须用双绞屏蔽、终端匹配、单点接地。很多现场故障不是编码器坏，而是电缆在拖链里折了几百万次之后，屏蔽层断了。这属于 §87.10.5 的 EMC 范畴，但它与“线数选多少”直接相关：线数越高，同样的电缆能拉的距离越短。

87.7.6 电流环增益与采样频率的绑定

电流环的比例增益不是自由参数，它被采样频率锁住。按 §87.5.3 的延迟分析，1.5 拍的延迟在 ω_c 处贡献 $1.5\omega_c T$ 的相位滞后；要让滞后不超过 30°，必须满足

$$\omega_c \le \frac{0.524}{1.5T} = \frac{0.349}{T} \tag{87-19}$$

16 kHz 对应 $T = 62.5$ μs，上限是 5584 rad/s，即 889 Hz。这个数比 §87.3.1 里取的 1.2 kHz 低，说明那套参数已经把裕度用到了 19.5°，属于“能用但不能再动”的状态。

式 (87-19) 把“提带宽”与“提采样率”绑在一起。想把带宽从 1.2 kHz 提到 2 kHz，采样率必须提到 28 kHz 以上，占用率随之上涨一倍多。这条绑定关系是伺服驱动器开关频率从 16 kHz 向 20 kHz、32 kHz 演进的真正动力，不是“听起来更高级”。

87.7.7 观测器增益

观测器增益决定收敛速度与噪声抑制的平衡。增益按带宽配置，带宽取电流环带宽的 $1/5$ 到 $1/3$。取高了，观测器能快速跟踪反电动势变化，但采样噪声与开关纹波直接进入估算值；取低了，估算值平滑，但在转速突变时滞后，导致换相角偏差。

一个可用的经验值：观测器带宽 200 Hz 对应时间常数 0.8 ms，在 1000 r/min（电频率 83 Hz）下引入的相位误差约 7.5°。这个误差会直接变成 d、q 轴的耦合，表现为转矩下降 1% 到 2%。要压到 1° 以内，带宽需要提到 1.5 kHz，代价是纹波抑制能力下降。

87.7.8 参数之间的三处耦合

第一处是 PWM 频率与电流环增益。频率升高，延迟减小，允许的增益升高，但分辨率下降，噪声升高，增益又不能太高。三者的净效果通常是一个 16 到 20 kHz 的最优区间。

第二处是 ADC 位数与测速窗口。位数提高能压低噪声，于是测速窗口可以缩短，速度环延迟减小。反过来，8 位 AD 的系统即使把窗口缩到 0.2 ms，噪声也会淹没速度信号。

第三处是编码器线数与 PWM 频率。高线数编码器的计数频率与开关频率同量级时，计数脉冲会耦合进电流采样。2500 线、4 倍频、3000 r/min 对应 500 kHz 计数频率，是 16 kHz 开关频率的 31 倍，布线不当就会看到电流波形上有 500 kHz 的毛刺。

三处耦合的共同点是“没有单独的正确的值”。每一处都要求两个参数一起定，且定完之后不能单独改其中一个。工程上把它们写进一张“参数冻结表”：开关频率、ADC 位数、编码器线数、测速窗口、电流环增益五项，任何一项改动都要重算另外四项。这张表比任何一条经验值都重要，因为现场最常见的故障就是“只改了一项”。

三处耦合的强弱不同。第一处最强，因为它同时涉及相位与噪声，改一个百分点就能测到差别；第二处中等，只在低位数 ADC 上明显；第三处最弱，但只要布线不规范就会突然放大。排查顺序也按这个强弱来：先看开关频率与增益是否匹配，再看 ADC 与窗口，最后查编码器布线与 PWM 之间的串扰。

87.7.9 参数与症状的对照

把上面几节收成一张对照表，现场排查时按症状查行，比按参数逐个试要快。

症状	先查	常见原因
低速爬行	编码器线数、测速窗口	窗口内计数太少
电流过零畸变	死区补偿、符号保持带	补偿量跳变
满速抽搐	定点量程	观测器输出溢出
稳态抖动	量化位宽	死区引起的极限环
电流尖峰	采样时刻	采样落在开关沿
带宽上不去	开关频率、延迟	1.5 拍延迟吃裕度

这张表只覆盖单轴、单环的情形。多轴耦合、通信抖动、电源跌落引起的症状不在表内，它们分别属于第 86 章与后面的总线章节。

87.8 调参方法与经验

嵌入式的调参与纯算法调参有一处根本差别：可调的旋钮不止增益，还有采样、位宽、外设。顺序错了，会在一个错误的基础上反复微调，越调越远。

87.8.1 上机顺序

表 87-11 给出五步顺序。每一步都有明确的通过判据，不通过不进下一步。

步骤	做什么	通过判据
1 时基	核对开关频率、死区、采样触发点	示波器测得周期与设计值误差小于 0.1%
2 电流环	先比例后积分，阶跃给定	超调小于 10%，无持续振荡
3 速度环	测速窗口与滤波先定，再加增益	阶跃超调小于 20%，稳态纹波小于 1%
4 位置环	在速度环闭合的前提下加比例	定位误差小于 1 个编码器计数
5 定点化	定标、饱和、舍入三件事	与浮点版本的阶跃响应偏差小于 5%

表 87-11 的第 1 步最容易被跳过。工程师拿到板子直接调增益，调不出来再回头查时基，结果发现采样触发点错了半个周期，前面所有的整定数据全部作废。先把时基钉死，后面的数据才有意义。

五步的判据都是可以量出来的，不是“感觉差不多”。第 2 步的“无持续振荡”指相电流上没有周期等于开关周期的等幅振荡；第 3 步的“稳态纹波小于 1%”指额定转速下速度反馈的峰峰值；第 4 步的“定位误差小于 1 个计数”要用激光干涉仪或高分辨率光栅复核，不能用编码器自己读自己；第 5 步的“偏差小于 5%”指同一阶跃给定下，定点与浮点两条响应曲线的最大差。判据写成数，才能在不同人、不同批次之间复现。

五步里最费时间的是第 2 步。电流环的阶跃响应要在 100 μs 量级上观察，示波器时基要开到 50 μs/格，电流探头带宽至少 1 MHz；用霍尔电流传感器看，带宽往往不够，会把真实的振荡看成平滑的上升沿。这也是为什么现场常说“调不出来”，其实是看不见。

87.8.2 第一步：把时基钉死

时基核对有三件事：开关频率、死区、采样时刻。开关频率用示波器看相电压的周期，误差应小于 0.1%；死区用双通道看上下桥臂驱动，实测值等于寄存器值加上驱动延迟；采样时刻用一个 GPIO 在中断入口翻转，看它相对 PWM 沿的位置。

三件事里采样时刻最难核。GPIO 翻转本身有延迟，中断响应也有几个周期的抖动，测出来的位置总比真实值晚一点。可行的办法是把采样时刻故意提前，观察电流波形什么时候开始变差，找到那个临界点，再往回退 20%。

这一步还要确认 ADC 的触发源。用软件触发代替定时器触发，看似灵活，实际会引入微秒级的抖动，电流环立刻就表现出来了。触发源必须是硬件，且与 PWM 同源。

时基核对的三个数可以汇成一句话：**周期准、死区够、采样静**。三者任一不满足，后面的整定都没有意义。核对完把实测值写进记录，因为板子换了、固件改了、晶振换了，这三个数都可能变，而它们一变，之前所有的增益都要重调。

87.8.3 第二步：电流环

电流环整定用阶跃给定，观察相电流的上升沿。先把积分清零，只加比例，从小往大加，直到阶跃响应出现约 10% 超调，再回退 20%。然后加积分，积分时间从对象时间常数 L/R 的 2 倍开始往下调，直到阶跃后的稳态误差在一个周期内

消失。

判据是“无持续振荡”。电流波形上如果出现周期等于开关周期的振荡，说明增益已经越过延迟允许的上限，式 (87-19) 给的就是这个上限。此时继续加增益只会让振荡幅度变大，不会让响应变快。

整定过程中要用电流探头看真实波形，不能只看采样值。采样值经过零阶保持，看起来是平的，真实波形可能已经振得很厉害。两者的差别在低电感电机上尤其明显。

87.8.4 第三步：速度环

速度环的第一步不是加增益，是定测速窗口与滤波。窗口按 §87.7.5 的方法选，滤波用一阶低通，截止频率取速度环目标带宽的 3 到 5 倍。

然后加比例增益。速度环的对象是电流环闭环加机械积分，相位本来就接近 −90°，加上测速延迟与滤波延迟，裕度很紧。经验值是速度环带宽取电流环的 $1/5$ 到 $1/10$，16 kHz 系统对应 100 到 250 Hz。

积分的作用是消除稳态速度误差。加积分后要观察低速段：积分增益太高会让低速爬行变成低频摆动，频率约 2 到 5 Hz，幅度取决于量化步长。这时候应该退积分，而不是加滤波。

低速摆动与 §87.6.4 的极限环是同一件事的两种表现。速度环的量化步长是 1 个计数对应的转速（1 ms 窗口下 60 r/min），若积分增益足以在一个摆动周期内累积出一个步长，系统就会在两个步长之间来回。判据是摆动频率：低于 5 Hz 且幅度约一个步长，就是量化引起的，退积分或加 dither；高于 20 Hz 且幅度随负载增大，是增益过高引起的，退比例。

速度环带宽还要与位置环匹配。位置环带宽通常取速度环的 $1/3$ 到 $1/5$，本章 100 到 250 Hz 的速度环对应 20 到 80 Hz 的位置环。带宽比一旦反过来（位置环比速度环快），位置环会“追着速度环的尾巴”振荡，表现为定位时的持续抖动，加滤波也压不住。

87.8.5 第四步：位置环

位置环最后加，且只加比例。位置环的对象是速度环闭环加积分，本身已经是一个积分环节，再加积分会变成双积分，极易振荡。需要消除定位静差时，用前馈或者把积分放在速度环里。

定位精度的物理下限是一个编码器计数。2500 线四倍频对应 0.036°，折算到 5 mm 导程的丝杠上是 0.5 μm。要求高于这个数，只能换编码器或加光栅，靠调参是调不出来的。

定位快慢靠前馈，不靠增益。位置环加比例只能把误差“迟一点补上”，跟随误差与速度成正比；加速度前馈 K_{fa} 与速度前馈 K_{fv} 可以在指令发出时就给出所需的转矩与转速，把跟随误差压到接近零。工程上取 $K_{fv}=1$、K_{fa} 为 0.8 到 1.2，跟踪误差能降 5 到 10 倍，而稳定性不受影响——前馈在环路之外，不改变闭环极点。

前馈的代价是对指令质量敏感。上位机给出的轨迹若有台阶状跳变，前馈会把它放大成冲击；所以前馈必须配合轨迹平滑（S 形加减速）。另外，前馈增益依赖负载惯量比，惯量辨识不准时，K_{fa} 取大了会过冲，取小了等于没加。这也解释了为什么同一台设备在装夹不同工件后要重调前馈。

定位环还有一个不靠调参的精度来源：双向重复定位。丝杠反向间隙、导轨摩擦都会造成正反方向到达同一点时读数不同，典型值几个微米，比 0.5 μm 的编码器分辨率大一个量级。它不能靠提高分辨率解决，只能靠机械预紧或软件间隙补偿，后者需要在固件里存一张随位置变化的补偿表——这又回到 §87.3.4 的 Flash 预算。

87.8.6 定点整定的三个额外动作

浮点代码改成定点，除了换数据类型，还要做三件事。

第一件是定标。把每个物理量映射到一个明确的 Q 格式，写进一张表，代码里所有变量按表声明。定标表要标出每个量的物理范围，范围乘以增益不能超过格式上限——这就是 §87.6.6 的式 (87-17)。

第二件是饱和。所有可能越界的运算后面都要加饱和，不能依赖溢出回绕。饱和的函数要单独测，因为编译器对饱和的处理方式不同，有的会生成条件跳转，有的会用饱和指令，周期数差好几倍。

第三件是舍入。右移截断会引入直流偏置，累加器里尤其明显。把截断改成“加半个 LSB 再右移”，多花一个周期，换掉一个系统性偏差。这一条在观测器里必须做，在乘法里可以不做。

87.8.7 三个误调案例

案例一：用滤波压振荡。 电流环振荡，工程师在反馈上加了一阶低通，振荡变成低频摆动，看起来好了。实际是滤波的相位滞后把裕度进一步吃掉，负载突变时会出现幅度更大的低频振荡。正确做法是降增益或提高开关频率。

案例二：把积分调到饱和。 位置环定位慢，工程师加大积分，结果每次大行程运动后都过冲。原因是积分在运动过程中饱和，到位后要花时间退饱和。正确做法是加积分限幅或者改用增量式。

案例三：定点参数直接抄浮点。 浮点整定出的 $K_p = 25$、$K_i = 1200$，直接写进 Q12 代码，K_i 溢出成负数，系统立刻发散。正确做法是先按式 (87-17) 核量程，再按式 (87-18) 核分辨率，两步都过才写进代码。

87.8.8 一份上机记录

每次整定都该留记录，至少包含五项：时基实测值、电流环阶跃的超调与上升时间、速度环阶跃的超调与调节时间、定位误差、以及当天的母线电压与温度。

记录的作用不是归档，是对比。同一台设备在不同温度下的电流环增益可能差 10%，没有记录就无法判断“今天调不好”是参数问题还是环境问题。多轴设备还要记录轴间差异，差异超过 20% 通常意味着某根轴的编码器或功率器件有问题。

记录表要能支撑三件事：复现、追溯、交付。复现指换个人照表能调出同一组参数，所以必须写清固件版本、参数版本、母线电压、环境温度；追溯指出了问题能回看当时的状态，所以还要写当天的异常现象与处理；交付指把表连同设备一起交给客户，所以要用客户能懂的量（定位误差、噪声、温升），而不是只有工程师懂的寄存器值。

记录的颗粒度按“改动”来定，不按“天”来定。每改一次增益、每换一次滤波、每调一次死区，就记一行，写清改前值、改后值与观察到的变化。这样一周后回看，能直接看出哪个动作真正有效；若只记最终值，等于把中间的过程全丢了，下次遇到同类问题还得重新试一遍。

87.9 优化与改进

优化这个词在嵌入式里常被误用成“让代码更快”。真正需要优化的场合只有一种：算力不够，功能加不进去。周期富余时做的优化，收益是零，风险是把已经验证过的代码改坏。

87.9.1 先量再改

动手之前先测三个数：整拍 WCET、各块占比、最坏情况出现在哪个分支。测 WCET 的方法与第 86 章一样，用 GPIO 翻转加示波器，或者用内核的周期计数器。

测出来的占比常常与直觉相反。表 87-1 里最复杂的电流 PI 只占 11.5%，看起来最“数学”的观测器占 34.4%，最“不起眼”的 SVPWM 占 23.0%。优化对象应该按占比排，不按代码行数排。

还有一个数要看：最长分支的耗时。平均值 481 周期，最坏分支可能是 620 周期，差别出现在故障保护与参数在线修改的分支上。优化平均路径对最坏响应没有帮助。

测量本身也有成本。用 DWT 周期计数器最省事：进中断读一次、出中断读一次，差值存进一个全局变量，主循环里通过串口打出来。注意读计数器要关中断，否则两次读之间被更高优先级中断插进来，得到的是“含被抢占时间”的数。用

GPIO 翻转看示波器更直观，但只能看单次，看不到分布；工程上常把两种结合：GPIO 看有没有超时，DWT 看具体分布。

测完要做一次“收益预判”，再决定动不动手。把各块周期数按占比排序，算出“砍掉这一块能把占用率降几个点”，再看这几个点能不能换来一个新功能（一轴、一档通信、一个观测器）。换不来就不改——这不是保守，是因为改动本身有回归成本：改完要重跑全部整定，重测最坏路径，重写文档。优化真正的门槛不是技术，是这笔回归账。

87.9.2 查表与分段线性

三角函数、开方、反正切是最常见的优化对象。以正弦为例，直接调用数学库约 200 个周期，256 项查表加线性插值约 30 个周期，误差 0.004°（表 87-4）。

分段线性的思路与查表相同，只是把表换成几段直线方程，省 ROM 但多几个乘法。在 ROM 只有 32 KB 的芯片上，分段线性比查表更可行。

查表有一个常被忽略的成本：索引计算。电角度是累加出来的，要先归一化到 $[0, 2\pi)$，再乘以表长除以 2π，再取整。这三步在定点里要 10 到 15 个周期，占总开销的三分之一。把表长取成 2 的幂，乘除就退化成移位，这三步能压到 5 个周期。

把这笔账拆开看更清楚。512 项表时，归一化与索引要 15 个周期，读相邻两个表项 4 个周期，插值的三次乘加（先求差、再乘小数部分、再加回）9 个周期，合计 28 个周期，与前面说的 30 个周期吻合。表长改成 256 后，电角度用无符号整数表示，索引直接取高 8 位，索引段从 15 降到 5 个周期，总数变成 18 个周期。省下的 10 个周期乘四轴、乘 16 kHz，在 168 MHz 上只有 0.38 个百分点，单看收益并不惊人。

真正值得关心的是 ROM。256 项单精度表占 1 KB，512 项占 2 KB，在只有 32 KB Flash 的芯片上分别占 3.1% 与 6.3%，还要留出两份给固件升级。所以表长的选择往往是 ROM 先卡死、周期再让步：先按 Flash 余量定表长，再看周期够不够，而不是反过来。若两样都不够，才退到分段线性——它用 8 段直线把 1 KB 压到几十字节，代价是每点多两次乘法，总周期反而涨到 24 左右。

87.9.3 观测器降阶

表 87-1 里观测器占 420 周期，是最大的一块。它的全阶形式要维护两个状态、做四次乘法、两次加法，还要处理反电动势的交叉耦合。

降阶的思路是承认“电流环比观测器快得多”，把电流动态当作瞬时平衡，只保留反电动势一个状态。降阶后周期数从 420 降到约 180，误差在稳态下不超过 2%，在转速突变时滞后 1 到 2 ms。

这 1 到 2 ms 的滞后能不能接受，取决于应用。风机、泵类负载可以接受，机床进给不行。降阶不是“更好的算法”，是“在特定工况下够用的算法”。

87.9.4 单精度与双精度

在有单精度 FPU 的芯片上，双精度运算通常由软件模拟，耗时是单精度的 3 到 5 倍。控制回路里几乎没有需要双精度的场合：电流、电压、位置的动态范围都在 10^4 以内，单精度 24 位尾数足够。

需要双精度的地方通常在离线计算：参数辨识、滤波器设计、位置累积。这些应该放在上位机或初始化阶段，不要放进每拍的中断。

一个例外是位置累积。多圈绝对位置需要 32 位整数，用浮点表示会丢低位。正确做法是用整数计数，转成浮点只用于控制计算。

87.9.5 DMA 与双缓冲

ADC 多通道采样会产生大量数据搬运。用 CPU 搬运，每通道每拍要 10 到 20 个周期；用 DMA，CPU 开销接近零。

DMA 的代价是一致性。DMA 在后台写缓冲区，CPU 在前台读，如果读到一半 DMA 更新了，就会拿到半新半旧的数据。解决办法是双缓冲：DMA 写缓冲区 A 时 CPU 读 B，转换完成中断里交换指针。

双缓冲还有一个隐含要求：缓冲区大小要能被 DMA 的传输长度整除，否则会出现半拍偏移，表现为每 N 拍有一次数据错位。这个错位很难从波形上看出来，通常表现为“偶尔抖一下”。

DMA 还有两个配置陷阱。第一个是优先级：DMA 与 CPU 抢同一块 SRAM 时，DMA 突发传输会把 CPU 的取指推迟几个周期，若正好落在中断里，最坏路径就被拉长；M4 的双端口 SRAM 能缓解，单端口不能。第二个是缓存一致性：M7 有 D-Cache，DMA 写进内存的数据可能被 Cache 里的旧值遮住，读到的还是上一拍的值。解决办法是把缓冲区放在不可缓存的区域，或在交换指针前做一次 Cache 无效化——后者要按 Cache 行对齐，否则会把相邻变量一起清掉。

87.9.6 定点与浮点混合

完全定点最快，完全浮点最好写，两者之间还有一种做法：把难写的块留在浮点，其余走定点。图 87-8 把每拍的块按这条线分成两栏。

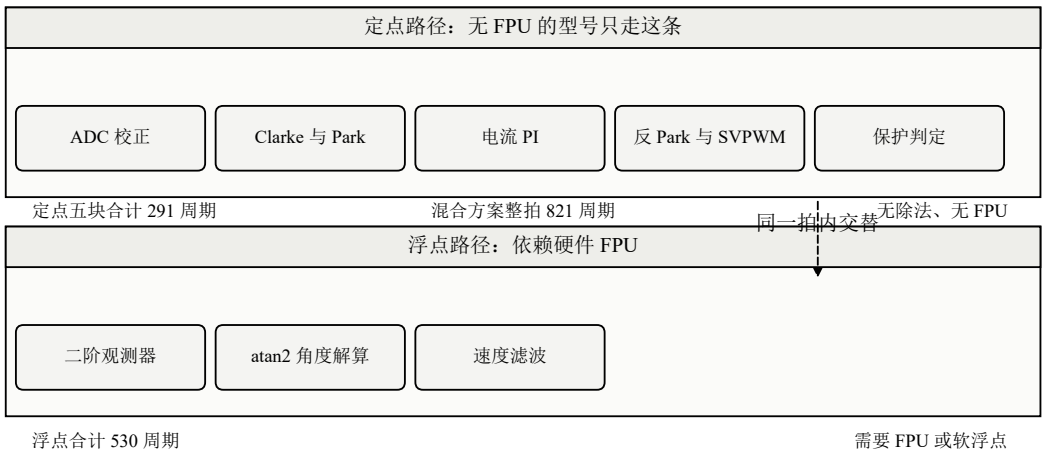


图 87-8　定点与浮点的分工泳道

按表 87-1 的数字算，全定点 481 周期，全浮点 1220 周期。把观测器与速度滤波两块换成浮点，其余保持定点，整拍变成 821 周期：从 481 里减掉这两块的定点开销 190，加上它们的浮点开销 530。

821 比全定点多 340 周期，比全浮点少 399 周期。多出来的 340 周期换到的是开发时间：定点的观测器要手动定标、处理溢出、验证收敛，工作量大约是浮点版本的三倍，而且 §87.1.1 那类溢出故障都出在这里。

所以混合方案的价值不在周期，在风险分配。算力紧张时全定点，算力宽裕但工期紧张时混合，全浮点只适合 M7 这类算力过剩的平台。

87.9.7 优化的代价

表 87-12 把五种常见优化的收益与代价并列。收益列是周期数，代价列是别的东西。

手段	典型收益	代价
查表替三角函数	150 到 180 周期	1 KB ROM，精度 0.004°
观测器降阶	240 周期	转速突变滞后 1 到 2 ms
定点化	739 周期	开发量三倍，溢出风险
DMA 搬运	每通道 15 周期	双缓冲与一致性处理
内联展开	20 到 40 周期	代码体积增大，Cache 命中率下降

表 87-12 最后一行值得单独说。内联展开在小循环上有效，在热路径上可能有害：代码体积膨胀会把其他热代码挤出指令 Cache，整体变慢。M7 有 Cache，M0 没有，同一个优化在两个平台上的效果可能相反。

87.9.8 三种不该优化的情况

第一种是周期富余超过 50%。此时优化不带来任何功能，只带来回归风险。第二种是瓶颈在等外设。ADC 转换、通信收发的时间是硬延迟，优化 CPU 代码不会让它变快。第三种是没有测量手段。改完不知道快了多少，等于在赌博。

判断方法很简单：先算 U，再算优化后能多带几轴。多带不了半轴，就不值得改。

“半轴”这个门槛可以算实。M4F 上带一轴浮点要 11.6 个百分点，半轴就是 5.8 个点。观测器降阶省 240 周期，四轴合计 9.1 个点，够一格半，值得做；查表替三角函数省 170 周期，四轴合计 6.5 个点，也够；内联展开每拍省 30 周期，四轴合计只有 1.1 个点，卡在门槛下，不做。同一套判据放在 M0 上结论就相反：主频低、单轴占比大，同样的绝对周期数对应更高的占用率，内联展开也能过门槛。

这也是“优化清单”不能跨平台照抄的原因。同一个手段在不同主频、不同轴数下折算出的百分点不同，而决策看的是百分点，不是周期数。把周期数换算成占用率再比较，是这一节唯一需要记住的操作。

87.10 工程落地

实验室里跑通与产品上能用，中间隔着上电顺序、故障保护、标定、EMC、升级与产线测试。这一节按整机从冷启动到量产的顺序讲一遍，每一段都给出一个容易出错的点。

87.10.1 从开发板到产品

图 87-4 画出了外设与 CPU 的数据流。左侧是功率级与传感器，中间是定时器与 ADC，右侧是两个节拍不同的控制环。这张图在开发板上与在整机里是一样的，区别在于整机多了隔离、多了长线、多了干扰源。

图 87-4　外设与 CPU 的数据流

图 87-4 里最容易被忽略的是从速度环到电流环的那条虚线。它每 1 ms 更新一次，写在 RAM 里，电流环每 62.5 µs 读一次。两者不同步，所以电流环可能读到“正在被改写”的半个浮点数。32 位浮点在 32 位总线上是原子读写，不会撕裂；但如果用了 64 位变量或者结构体，就必须加临界区。

87.10.2 上电与初始化

上电顺序错一步，轻则报故障，重则炸管。可靠的顺序是：时钟与看门狗、GPIO 置为安全态、功率级驱动保持关断、自检、预充电、使能驱动、闭环。

安全态指的是上下桥臂全关、制动输出断开。这一步必须在配置任何复用功能之前完成，因为复用功能一开，引脚状态可能瞬间变成驱动有效。很多炸管事故的原因就是“初始化时引脚默认电平恰好是高”。

预充电是把母线电容充到额定电压的 90% 以上，通常通过限流电阻，耗时 200 到 500 ms。预充电没完成就使能 PWM，浪涌电流会把保险丝烧掉。判据是母线电压达到阈值并稳定 20 ms。

预充电之后还有一步常被省略：旁路继电器吸合。限流电阻不能长期串在回路里，否则满载时它自己会烧。吸合时序必须是“电压到位→吸合→再确认电压”，吸合瞬间若电压跌落超过 10%，说明电容容量不足或继电器触点氧化，应当报警而不是硬吸。

上电自检要跑四项，且都在功率级未使能时做。母线电压是否在窗口内，超出即报“输入异常”；三相电流零漂是否在 ±1 LSB 内，超出说明运放或分流电阻有问题；散热器温度是否在 −20 到 60 °C 之间，超出说明传感器开路或短路；编码器通信是否正常，读回的位置是否在合理范围内变化。四项里任何一项失败都应停在安全态并给出故障码，不能“先跑起来再说”。

87.10.3 故障保护与看门狗

硬件保护与软件保护要分开。硬件用比较器直接封波，响应时间微秒级；软件在中断里判断过流、过压、过温，响应时间一拍。两者都要有，硬件兜底，软件分级。

看门狗要放在主循环里喂，不要放在中断里。放在中断里，主循环卡死时中断照常跑，看门狗照常被喂，保护失效。正确做法是主循环设一个标志，中断置位，主循环清零，看门狗检查这个标志。

故障后的处理要分等级：过流立即封波并断开主回路；过压先降速再封波；过温先限功率，超过阈值再停机。全部按“立即停机”处理会让设备频繁停机，全部按“降额”处理会烧器件。

分级的标准是“可逆性”。过流可能是瞬时冲击，也可能是短路，两者用不同的阈值与延时区分：1.5 倍额定、持续 10 μs 判为冲击，封波后 200 ms 自动重试；3 倍额定、持续 2 μs 判为短路，永久锁存，必须断电复位。过压同理，母线超过 1.15 倍先禁止回馈、把能量泄放到制动电阻，超过 1.25 倍才封波。过温最慢，用 1 s 的滑动平均判断，超过 85 °C 限功率到 70%，超过 95 °C 停机。

故障必须锁存，不能“故障消失就自动恢复”。现场电网波动频繁，若每次欠压都自动复位，设备会反复起停，机械与继电器都受不了。做法是记录故障码、故障时刻与当时的电压电流，进入“故障待复位”状态，等人确认或等 30 s 后自动重试一次；同一故障在 10 分钟内出现三次，就永久锁存，必须人工介入。

87.10.4 参数存储与标定

参数存 Flash 或 EEPROM，写入次数有限。整定过程中反复写参数会磨穿存储单元，所以要有 RAM 镜像：运行时读写 RAM，只在明确“保存”时写存储。

存储结构要带版本号与校验。升级固件后参数结构可能变化，没有版本号就只能全部恢复默认，用户整定好的参数全丢。校验用 CRC16 足够，出错的概率远低于漏检带来的损失。

电流零漂必须在线标定。运放的零点随温度漂移，出厂标定一次不够。可行做法是在每次上电、功率级未使能时采 100 次求平均，作为本次运行的零点。运行中再用低速滤波器缓慢跟踪。

存储要写成 A/B 双份，带序号与 CRC。写入时先擦 B 区、写新值、校验、再更新序号；掉电发生在任意一步，下次上电总能从两份里挑出序号较新且 CRC 正确的一份。单份存储的掉电概率不高，但一旦发生就是“参数全丢、设备回到出厂状态”，现场往往表现为“昨天还好好的，今天一动就报警”。

Flash 的擦写寿命通常是 1 万次。按每天整定 20 次算，一年 7300 次，两年就磨穿。所以参数保存要限流：只有用户明确点“保存”才写，运行中的自适应参数（零漂、惯量辨识结果）存到 RAM，掉电前借母线电容的余电写一次；或者干脆不存，每次上电重新辨识。

87.10.5 EMC 与布线

三条规则能解决大部分问题。第一，编码器线用双绞加屏蔽，屏蔽层单端接地，不要两端接地形成地环流。第二，模拟地与功率地单点连接，连接点选在 ADC 的模拟地引脚附近。第三，开关节点（桥臂中点）的铜皮面积尽量小，它是最强的辐射源。

第四条不那么明显但同样重要：ADC 的采样线不要与 PWM 驱动线并行。并行长度超过 5 cm，开关噪声就会耦合进采样通道，表现为电流波形上叠加开关频率的尖峰。

四条规则之外还有两个“看不见”的耦合路径。第一条是共模电流：电机绕组对机壳有几十到几百皮法的寄生电容，开关时共模电流经机壳回到直流母线，在采样地上产生压降。压降只有几十毫伏，但对 0.8 mV/LSB 的 12 位 ADC 就是几十个 LSB。解法是加共模电感或 Y 电容，把共模路径旁路掉，而不是在软件里滤波。第二条是地弹：大电流回路与模拟回路共用一段地线，开关瞬间的地弹直接叠在采样值上，表现为“电流尖峰与占空比相关”。解法是把模拟地单独走一根线回到电源入口，不与控制地共用。

EMC 的验收不能只看功能正常。辐射发射在 30 到 230 MHz 频段最容易超标，开关沿越陡、辐射越强；抗扰度测试里的快速瞬变脉冲群（EFT）会直接打进采样通道，表现为“通信中断或过流误报”。这两项要提前做，不能等整机定型再补——补的时候往往只能加磁环、加屏蔽罩，成本比改板高一个量级。

87.10.6 固件升级

现场升级要有双分区：引导区负责接收与校验，应用区负责运行。新固件校验通过才切换，校验失败回滚到旧版本。单分区方案一旦升级中断，设备就变砖。

升级期间必须停机。有人尝试在运行中升级，靠“只改不用的代码”规避风险，实际做不到：编译器会重排代码布局，任何一次改动都可能影响正在执行的中断向量。

升级包要带硬件版本号。同一套固件刷到不同功率等级的板子上，参数不匹配会直接炸机。这一步的检查应该放在引导区，而不是应用区。

升级还要考虑“降级”。新版本出问题时，现场往往希望退回旧版，但旧版的参数结构可能不认新版的参数。做法是引导区同时保留两个版本各自的参数区，切换固件时连带切换参数区，互不覆盖。代价是 Flash 多占一份参数空间，收益是现场可以在两分钟内回退，而不用返厂。

升级通道本身也要有保护。未签名的固件一律拒绝，签名校验放在引导区，且校验失败时不擦除旧版本。这一条在实验室里显得多余，在现场是防止“误刷别家固件”的唯一手段。

87.10.7 产线测试

产线至少要测四项：电流零漂、编码器相位、死区分段、通信误码率。

编码器相位测试是给一个已知角度，读回编码器值，偏差超过 1° 就判不合格。这一项能抓出装反、装偏、磁钢偏心三类问题。死区实测用双通道示波器看上下桥臂的死区时间，与设计值偏差超过 20% 要查驱动电路。

通信误码率在产线上跑 10 分钟即可，工业现场则要按小时计。误码率突增通常不是协议问题，是接地或终端电阻问题。

四项测试之外，还有一项“整机空载运行”值得加进产线。让电机在 10% 额定转速、空载下连续转 60 s，同时记录三相电流的峰值与不平衡度。这一项能抓出绕组接错、霍尔装反、编码器相位错位三类问题，而这三类问题在静态测试里都看不出来。判据是三相电流不平衡度小于 5%、峰值在额定值的 10% 到 20% 之间。

产线节拍要控制。完整的四项加空载运行约 90 s，若节拍要求 30 s，只能做抽检：每台做零漂与相位，每批抽 10% 做死区与误码。抽检比例不是拍脑袋，按“该缺陷的逃逸成本”定：相位错位的逃逸成本是现场返修，必须全检；死区偏差的逃逸成本只是效率略降，可以抽检。

产线测试只是四道关里的第三道。图 87-10 把样机到量产的路径摊开：样机、环境与 EMC、小批试制、量产放行，每一道关配一份交付物——功能测试报告、认证报告、工艺文件、放行单。

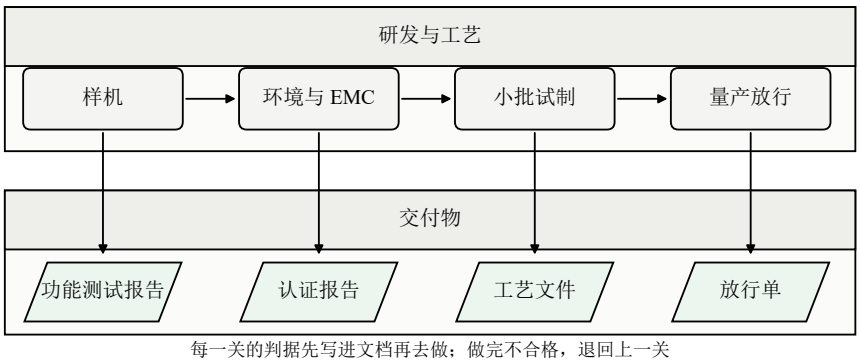


图 87-10　固件从样机到量产的交付链

这四份文件的作用不是存档，是让下一道关有据可依。样机阶段没写清判据，环境试验就只能凭“看起来没问题”放行；小批试制没留下工艺参数，量产后换一条产线就得重新摸索。反过来，交付物齐全时，量产放行只需要核对四份文件，不需要再把前面的试验重做一遍。

四道关的顺序也不能省。先做 EMC 再做小批，是因为 EMC 整改常常要改板、改滤波，改完的硬件已经不是样机的那一版；先小批再量产，是因为小批能暴露工艺问题，而工艺问题在单台上永远看不到。

87.10.8 投产检查清单

表 87-13 是一份可以直接拿去用的清单。左边是检查项，右边是判据，最后一行是出问题时的第一嫌疑。

检查项	判据	第一嫌疑
开关频率	与设计值偏差小于 0.1%	时钟配置
死区	实测与设计偏差小于 20%	驱动延迟
采样时刻	落在峰值平台内	触发源设置
电流零漂	上电标定后小于 1 LSB	运放偏置
编码器相位	偏差小于 1°	安装偏心
定点量程	满速时观测器不饱和	Q 格式选小
看门狗	主循环卡死 200 ms 内复位	喂狗位置
通信误码	连续 10 分钟无错帧	接地与终端

表 87-13 的第四行与第六行是本章特有的。前者对应 §87.7.4 的静差，后者对应 §87.1.1 的溢出。两份检查都在上电阶段完成，不花运行时间。

87.11 优缺点与适用边界

到这一节为止，本章讨论的都是“如何在给定芯片上把事情做成”。还有一半问题没回答：给定任务，该不该用这类芯片。这一节给出边界，以及边界之外的替代方案。

87.11.1 微控制器的三个长处

第一是确定性。裸机或轻量实时内核的中断响应在十几个周期量级，抖动可测可控。通用处理器上做不到，因为 Cache 缺失、内存换页、后台任务都会带来毫秒级的不可预测延迟。

这个“十几个周期”可以量化。Cortex-M4 的中断响应固定 12 个周期，168 MHz 下是 71 ns；加上压栈 8 个寄存器（12 周期）、跳转到服务程序（3 周期），到第一条用户指令执行约 160 ns。抖动主要来自三处：正在执行的指令能否被打断（多周期指令如除法要等完成，最多 14 周期）、Flash 等待周期（168 MHz 下 5 个等待，开启预取与 Cache 后可降到 0）、以及 DMA 抢占总线（最多几个周期）。三处合计，最坏抖动在 1 µs 以内，与 62.5 µs 的开关周期相比只占 1.6%。

这个 1.6% 就是“确定性”的价格标签。工业 PC 上的抖动是 5 到 50 µs，占同一个周期的 8% 到 80%，根本无法用于电流环。所以不是“MCU 算得快”，而是“MCU 的慢是确定的”——两者对闭环的意义完全不同。

第二是集成度。一颗芯片里有时钟、定时器、ADC、运放、比较器、通信控制器，外围只需要功率级与少量模拟电路。器件少了，故障点少，认证也容易。

第三是成本与功耗。四轴控制器的主控成本可以压到几十元，待机功耗在百毫瓦量级。这两条决定了它在量大面广的场合不可替代。

87.11.2 与 DSP 的分工

数字信号处理器与微控制器现在的界限已经模糊：主流 MCU 都有单精度 FPU 与 DSP 指令。真正的差别在指令集对控制算法的贴合度，以及外设的专用程度。

运动控制专用的 DSP 通常带硬件三角函数单元、更高分辨率的 PWM、更多的编码器接口。做多轴、高精度、带复杂观测器的场合，它的开发效率更高；做单轴、中等精度、成本敏感的场合，通用 MCU 更划算。

表 87-2 里的 D 类候选就是这个定位：200 MHz、两条高级定时器、单价 18。它的算力不如 M7，但外设与指令更贴控制，四轴 16 kHz 占用 39%。

87.11.3 与 FPGA 的分工

FPGA 的优势是并行与确定性延迟。多轴电流环全部并行执行，每轴延迟一致，不随轴数增加。需要纳秒级响应的场合——比如高速开关磁阻电机、多电平变换器——只有 FPGA 能做到。

代价是开发门槛与成本。一个电流环用 FPGA 实现，工作量是 MCU 的三到五倍，而且定点位宽、时序约束都要手工处理。工程上常见的折中是“FPGA 做 PWM 与采样时序，MCU 做控制算法”，两者用并行总线连接。

87.11.4 与工业 PC 的对比

表 87-14 把四类平台放在一起比。数字是量级，不是某一款产品的参数。

平台	单拍延迟	抖动	适用轴数	开发周期	相对成本
微控制器	3 到 10 µs	小于 1 µs	1 到 8	短	1
运动控制 DSP	2 到 8 µs	小于 1 µs	2 到 12	中	3 到 5
FPGA	小于 1 µs	纳秒级	8 以上	长	10 以上
工业 PC 加实时内核	20 到 100 µs	5 到 50 µs	8 以上	短	5 到 20

表 87-14 最值得注意的是工业 PC 那一行。它的算力最强、开发最快，但抖动比微控制器大一到两个数量级。用它做 1 kHz 以下的外环没问题，做 16 kHz 的电流环就必须把电流环下放到驱动器的 MCU 里。

87.11.5 单核、双核与异构

双核不是“两颗单片机拼在一起”。常见的异构结构是一个实时核加一个应用核：实时核跑电流环与保护，应用核跑通信、轨迹规划、人机界面。两核之间用共享内存加邮箱通信。

这种结构的好处是隔离：应用核死机不影响电流环。坏处是通信延迟与一致性问题，共享内存的读写要加锁，锁的粒度不当会引入微秒级抖动。

同构双核则是两颗相同的核，跑对称多处理。它适合把多轴分摊到两个核上，但两个核争用同一份外设时，仲裁延迟会变成新的抖动源。

87.11.6 五类应用的结论

风机泵类：单轴、低动态、成本敏感，M0 定点足够，开关频率 8 kHz。纺织机械：多轴、中等精度，M4F 加外扩 PWM。机床进给：高精度、高动态，双核或 DSP，开关频率 16 到 20 kHz。机器人关节：高集成、带总线，M7 加 EtherCAT 从站控制器。高速主轴：高转速、低电感，FPGA 或专用 DSP，开关频率 40 kHz 以上。

五类结论的共同点是：开关频率先定，算力再定，最后核外设。顺序反了，就会买到一颗算力过剩但外设不够的芯片。

把五类的数字补上，结论才有可操作性。风机泵类取 8 kHz、单轴、带宽 200 Hz，M0 定点 16.0% 占用，成本压到最低；纺织机械取 16 kHz、四轴、带宽 600 Hz，M4F 四轴 46.5% 但只有两个高级定时器，必须外扩或把两轴并到一个定时器；机床进给取 20 kHz、双轴、带宽 1.2 kHz，M7 双轴 8.1%，余量充足但更贵，双核方案的性价比在“把通信与轨迹规划挪到另一个核”上；机器人关节取 16 kHz、单轴、带 EtherCAT，M7 加从站控制器，占用 4.1%，成本主要花在总线上；高速主轴取 40 kHz、单轴、带宽 3 kHz，MCU 的延迟预算已经不够，只能用 FPGA 或专用 DSP。

五类里只有机床进给与高速主轴对“延迟”敏感，其余三类对“成本”或“总线”敏感。这个分布说明：多数场合选型的决定因素不是算力，而是外设与成本；只有在高频、高动态的少数场合，算力与延迟才是瓶颈。

87.11.7 四条换平台的信号

第一条，优化后占用率仍在 85% 以上。说明已经没有余量应对需求变化。第二条，需要纳秒级响应。微控制器的中断延迟下限做不到。第三条，轴数超过 12 且要求同步。单芯片的外设路数不够。第四条，算法需要矩阵运算或浮点稠密计算。这类负载在 MCU 上跑不动。

四条之外还有一种不写在纸面上的信号：团队已经维护三套不同芯片的代码。这时候换平台的收益不在性能，在维护成本。

87.11.8 本章方法的局限

本章的算力模型假设指令周期数固定，实际会受 Flash 等待周期、总线仲裁、Cache 命中率影响。M7 上同一段代码在 Cache 命中与不命中时相差 3 到 5 倍，表 87-1 的数字只对特定编译配置成立。

位宽模型假设量化误差均匀分布，实际在周期性信号上会呈现相关性，误差可能累积而不是抵消。测速模型假设编码器无抖动，实际安装偏心会带来每转一次的周期性误差。

这些局限不否定方法的价值，但决定了它只能用于量级判断与筛选，不能替代实测。选定平台之后，仍然要在真实硬件上跑一遍 WCET 与闭环响应。

三条局限里，“量化误差相关性”最值得展开。模型假设每次量化误差独立、均匀分布在 $\pm q/2$，所以多次运算后误差互相抵消，只剩统计意义下的噪声。真实系统里，输入是周期性信号，量化误差也跟着周期重复，同样的误差每拍出现，等效于一个固定偏置而不是噪声。这就是为什么模型预测的静差比实测小一半左右——模型算的是“平均”，实测测的是“最坏的那一拍”。

“编码器无抖动”这条假设在高线数编码器上尤其不成立。2500 线四倍频的计数边沿有几十纳秒的抖动，折算到位置是 0.01° 量级，看起来很小，但它在速度环里被差分放大 $1/T$，1 ms 窗口下相当于 10°/s 的速度噪声。这也是速度反馈必须滤波的原因之一，而模型里没有这一项。

“指令周期固定”这条假设在多轴系统上偏差最大。四轴共用总线，每轴的实际 WCET 取决于其他轴在同一时刻做什么；实测的单轴 481 周期，四轴同时运行时可能变成 520 到 560。设计余量取 30% 而不是 10%，一部分就是为这种“互相拖慢”留的。

87.12 Python 仿真代码

本章所有数字来自 `code/ch087_sim.py`，插图来自 `code/ch087_figs.py`。这一节说明脚本结构、关键假设，以及如何把脚本改成自己的参数。

87.12.1 脚本结构

仿真脚本分七段，对应本章的七组结论：算力预算、定点位宽、ADC 位数、测速分辨率、PWM 权衡、角度算法、选型判据。每段独立成函数，`report()` 按顺序打印。

七段共用一个主频与开关频率的假设，改这两个数就能得到另一套结论。脚本里没有随机数，同一个输入永远得到同一个输出，方便核对正文。

七段的入口函数分别是 `load_table()`、`fixed_bits_study()`、`adc_bits_study()`、`speed_table()`、`pwm_table()`、`atan_paths()`、`selection_table()`，`report()` 只是把它们按顺序调一遍并排版。要单看某一段，直

接调对应函数即可，不必跑全表。要改假设，改文件头的 `MCUS`、`BLOCKS`、`TS`、`KP`、`KD` 五个常量，其他部分不用动。

87.12.2 算力预算模型

预算模型的核心是 `BLOCKS` 表：七个计算块，每块给出定点与浮点两种实现的周期数。`per_axis_cycles()` 按芯片类型选择路径，`occupancy()` 与 `max_axes()` 分别是式 (87-1) 与式 (87-2) 的实现。

软浮点倍率取 8，这个数是 Cortex-M3 上单精度乘法与硬件 FPU 的典型比值。换成 M0 或 M33，倍率会变，但结论的方向不变：无 FPU 写浮点必然不可行。

87.12.3 定点闭环仿真

闭环部分是位置环，对象 $1/(s(s+1))$ 用零阶保持离散化，控制律是 PD。`simulate_loop()` 的 `bits` 参数为 `None` 时走浮点，取整数时走定点。

定点路径把每一步都量化：位置反馈按 `adc_bits` 量化，误差、速度、乘积、输出按 Q 格式量化，输出再限幅到 ± 24。`_q()` 同时做取整与饱和，模拟 16 位定点的行为。

`_metrics()` 计算四个指标：超调、调节时间、静差、纹波。静差取最后 20% 时间窗内偏差绝对值的均值，纹波取同一窗口的峰峰值。

为什么用 PD 而不是 PID，脚本里的理由和控制章节一致：位置环的对象 $1/(s(s+1))$ 本身带一个积分，再串 PID 的积分就变成双积分，闭环极点很容易跑到虚轴右侧；而本章要观察的是“位宽如何影响静差”，PD 已经把静差暴露得很清楚，不需要积分来掩盖。若要看积分项的量化行为，把 `KD` 改为 0、加一个积分增益即可，但那时指标含义会变，不能与表 87-6 直接比。

输出限幅取 ± 24 是特意留的余量。按 $K_p = 25$、单位阶跃，初始误差为 1，比例项就给出 25，正好越过限幅边沿；这样 Q16 与 Q14 那两行“Kp 被量程截断”的行为才能在仿真里复现。若把限幅放到 ± 100，阶跃初期不会饱和，表 87-6 上半部分的差别会被掩盖，读者就看不到“量程不足”的后果。

87.12.4 外设模型

测速模型直接实现式 (87-6) 与式 (87-7)，输入是编码器计数、窗口、时钟分辨率。PWM 模型实现式 (87-8) 到 (87-10)，输入是时钟、频率、死区。角度模型实现式 (87-11) 与 (87-12)。

四个模型都是解析式，不涉及仿真循环。好处是快，坏处是不能反映动态行为——比如 PWM 分辨率对电流环稳定性的影响，这里算不出来，要另做时域仿真。

要把解析模型升级成时域模型，最小改动是加三样东西：一个 PWM 调制器（把占空比转成方波电压）、一个 LC 或 RL 对象、一个采样保持环节。有了这三样，就能观察“死区补偿不准”与“PWM 分辨率不足”在电流波形上的具体形状，而不是只知道它们“有影响”。代价是步长要从 1 ms 降到 1 µs 以下，仿真时间涨一千倍；所以本章的做法是“解析算量级、时域看形状”，两步分开跑。

87.12.5 复现清单

表 87-15 列出复现本章全部数字所需的命令与预期输出。

步骤	命令	预期
算力与外设	`python code/ch087_sim.py`	七段报告，与表 87-2、87-6、87-9、87-10、87-8 一致
出图	`python code/ch087_figs.py`	`figs/ch087/` 下 10 组 SVG 与 PDF
结构检查	`python tools/check_chapter.py 87`	节数 12，图 10，无红项
构建	`python tools/build.py --chapter 87`	HTML 与 PDF，页码连续

表 87-15 的第一行依赖 NumPy 与 SciPy，第二行依赖 Matplotlib。三个库的版本会影响字体回退，出图前确认 `SimSun` 可用，否则中文会变成方框。

只想核对某一段时，不必改计算函数。脚本里每一段都是独立的无参函数，`report()` 只是按顺序调用再打印；把不需要的那几行调用注释掉，就只跑剩下的段，几秒内出结果。改参数也走同一条路：块周期在 `BLOCKS` 里、芯片型号与主频在 `MCUS` 列表里、仿真步长在 `TS` 里，一处改动会传导到整份报告，不需要逐个函数去追。

这种“单一入口、集中配置”的写法是为了让数字可追溯。正文里出现的每一个周期数、每一个百分比，都能从这七八个常量推到；反过来说，若某个数在脚本里找不到出处，它就应当在正文里被删掉，而不是凭印象保留。第 86 章也是同样的规矩。

87.12.6 脚本没有覆盖的部分

脚本不模拟中断延迟、不模拟 DMA 争用、不模拟 Flash 等待周期。这三项需要更底层的模型或者实测，第 86 章给出了任务级的处理方法。

脚本也不模拟多轴耦合。四轴共用一条总线时，每轴的实际 WCET 会比单轴测出来的长，长多少取决于总线仲裁策略。这个数只能实测。

最后，脚本的定点模型是“逐步量化”，不模拟编译器的中间结果宽度。真实编译器可能用 64 位中间结果再截断，误差比模型小；也可能为了省寄存器提前截断，误差比模型大。模型给的是量级，不是精确值。

三项未覆盖的内容里，“中断延迟”对结论影响最小，“多轴总线争用”影响最大。前者只在占用率逼近 70% 时才显现，后者在四轴同时运行时立刻显现，且随轴数增加而恶化。若要把脚本用于多轴选型，最省事的办法不是建模，而是在单轴周期数上乘一个 1.05 到 1.15 的经验系数，再按式 (87-1) 算占用率；这个系数来自实测，而不是推导。

本章的方法可以复用到别的功率变换场合：逆变器、直流变换器、有源滤波，只要把“单拍预算”换成自己的计算块清单、把“位宽”换成自己的被控量范围，其余步骤不变。真正不能照搬的只有数字——16 kHz、5250、481 周期这些，都要按自己的硬件重算。

第88章 PLC与工业控制器

88.1 动机与直观理解

88.1.1 三个现场现象

第一个现象是“程序没错，动作却偶尔不对”。某条包装线的气缸联锁，台架上按顺序点动一切正常，到了现场连续运行时，操作工按下急停复位后气缸偶尔不伸出。程序被翻了三遍，逻辑门没有写错，问题出在两个梯级同时驱动同一个输出线圈：扫描按从上到下的顺序执行，后一个梯级把前一个的结果覆盖了。用 8 种输入组合逐一核对，两个输出的实际值与“并联”语义分别有 2 种和 4 种不符（图 88-6）。

第二个现象是“加了一段通信，控制就变差了”。原来的程序扫描周期 10 ms，PID 回路稳定；为了上传数据增加了通信服务块，扫描涨到 20 ms，操作员反映温度“压不住”。参数一个没动，闭环却变了，原因是积分项按标称周期累加，而实际经过的时间变长了。掉拍 20% 时，等效积分时间从 2.00 s 拉长到 2.50 s，调节时间从 2.96 s 涨到 5.27 s（表 88-5）。

第三个现象是“冗余做了，安全等级没升”。把单通道换成双通道，传感器、CPU、执行器都配了两套，评估时却发现 PFDavg 只从 2.19×10^{-3} 降到 1.22×10^{-3}，仍停在 SIL 2。共因失效吃掉了大部分收益：两套通道走同一根电缆槽、用同一个电源、由同一版固件驱动，失效并不独立。

三个现象对应三件事：扫描顺序决定逻辑，扫描周期决定动态，安全架构决定等级。它们都不属于控制算法的范畴，却在现场反复出现。

88.1.2 与相邻章的分工

第 86 章讨论实时软件的任务划分与调度，回答“多个周期任务怎么共享一个 CPU”；第 87 章讨论嵌入式控制器，回答“一颗 MCU 在一拍内怎么把 FOC 算完”。本章讨论第三种形态：PLC。它的计算量远小于伺服驱动器，但有一套自己的规矩——循环扫描、映像区、五种标准语言、面向维护人员的在线修改。

三章的差别不在硬件强弱，而在确定性来自哪里。第 86 章的确定性靠调度分析，第 87 章靠中断优先级与指令计数，PLC 则靠“一次只做一件事”：输入采样、程序执行、输出刷新依次进行，任何时刻只有一个梯级在求值。这种串行换来了可预测，也换来了扫描周期这条硬边界。

本章不重复 PID 的整定方法（见第 44、106 章），也不展开 MPC（见第 58、80 章）与 Smith 预估（见第 47、107 章）。采样定理与保持器在第 4、85 章已经讲过，这里只引用结论。

88.1.3 一个扫描周期的账

表 88-1 把一次扫描拆成五块。用户程序占 58.4%，是唯一随项目规模增长的块；通信服务占 19.5%，常被低估；输入刷新与滤波、输出刷新、系统自检三块合计 22.0%，基本是常数。

阶段	时间 / us	占比 / %	是否随项目增长
输入刷新与滤波	250	8.1	否
用户程序执行	1800	58.4	是
输出刷新	180	5.8	否
通信服务	600	19.5	部分
系统开销与自检	250	8.1	否
合计	3080	100.0	—

表 88-1 典型中型 PLC 的扫描周期预算（18 000 条布尔指令）

图 88-1 把五块画成一条累计时间轴。输入刷新在最前、输出刷新在中间，这个顺序就是“采样—计算—输出”的物理含义：本拍读到的输入是本拍开始时的值，本拍算出的输出要到本拍结束才生效。两者之间隔着整个程序执行时间，这段间隔就是扫描引入的纯滞后。

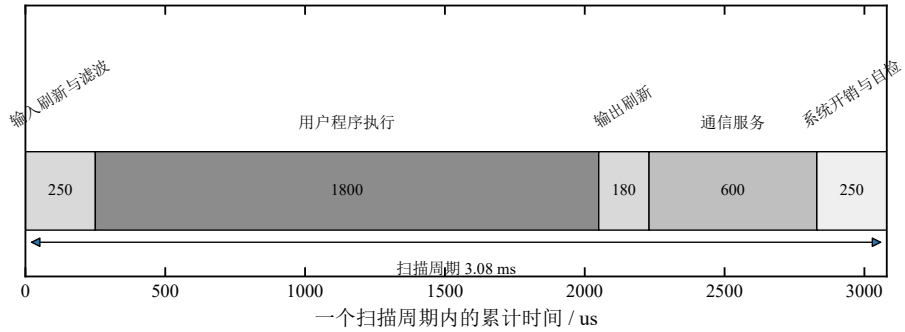


图 88-1 扫描周期的五个阶段

换成不同等级的 CPU，程序执行块会成比例缩短，固定块不变。表 88-2 给出同一份 18 000 条指令的程序在三档 CPU 上的结果：小型 CPU 2.72 ms，中型 1.82 ms，大型 1.50 ms。程序执行从 1440 us 降到 216 us，扫描周期却只从 2.72 ms 降到 1.50 ms，因为固定开销 1.28 ms 不随 CPU 变快。

表 88-2 同一程序规模下三档 CPU 的扫描周期

CPU 等级	单条指令 / us	程序执行 / us	扫描周期 / us
小型	0.08	1440	2720
中型	0.03	540	1820
大型	0.012	216	1496

这张表解释了一个常见误判：换更快的 CPU 并不按比例缩短扫描周期。当固定开销占到一半以上时，CPU 再快也只能把总时间压到 1.28 ms 附近。要真正提速，得先砍通信服务与自检，而不是换芯片。

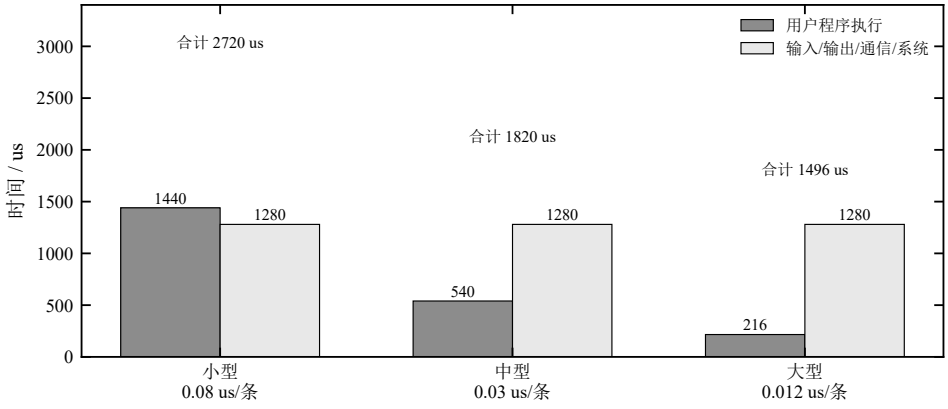


图 88-2 三种 CPU 等级的扫描周期构成

88.1.4 五个问题

第一个问题：扫描顺序如何决定输出。梯形图是从左到右、从上到下求值的，线圈写入映像区后立刻可被后面的梯级读到。同一个线圈出现两次会发生什么，同一梯级里既有常开又有常闭时按什么次序求值，这些都要在写程序之前想清楚，而不是在现场调试时试出来。

第二个问题：扫描周期与过程时间常数怎么配。扫描太慢，采样滞后吃掉相位裕度；扫描太快，程序没写完就被看门狗复位。本章用一阶加纯滞后的过程给出定量边界：扫描周期到时间常数的 2.5%（50 ms）时，相位裕度只掉 1.4°，超调从 4.0% 涨到 5.7%；到 5%（100 ms）时，相位裕度掉 2.8°，超调涨到 7.8%。

第三个问题：掉拍与抖动怎么算。PLC 的扫描不是硬实时，通信服务、大块数据拷贝、在线修改都可能让某一拍变长。若积分项按标称周期累加，掉拍会让等效积分时间变长，控制器变“软”。本章给出等效积分时间的估算式与实测曲线。

第四个问题：程序怎么模块化。功能块（FB）、功能（FC）、数据块（DB）、组织块（OB）的分工，决定了程序能不能被复用、能不能在线修改、能不能通过安全认证。图 88-7 给出一个最小可用的分层结构。

第五个问题：安全等级怎么算。SIL 不是“双通道”三个字，而是 PFDavg 的数量级。本章给出 1oo1、1oo2、2oo3 三种架构的算式与一张对照表，并说明共因因子为什么常常是决定性的一项。

88.1.5 主线数字

本章的主线数字有五组。扫描周期 3.08 ms，其中程序 58.4%、通信 19.5%；换 CPU 只能把周期压到 1.50 ms 附近，因为固定开销 1.28 ms 不动。

从采样到执行的总滞后 4.24 ms，由输入滤波 2.0 ms、扫描等待 1.54 ms、输出 0.20 ms、总线 0.50 ms 组成，相当于 2 s 时间常数的 0.21%，这一档几乎不损失裕度。

扫描周期从 10 ms 增到 500 ms，相位裕度从 61.1° 降到 47.0°，超调从 4.0% 涨到 32.1%。这条曲线的拐点在 50 ms 附近，即时间常数的 2.5%。

掉拍 20% 时调节时间从 2.96 s 涨到 5.27 s，IAE 从 1.086 涨到 1.214；掉拍 30% 时两者继续恶化到 6.35 s 与 1.305。掉拍对超调的影响方向相反——积分变弱，超调反而下降，只看超调会得出错误结论。

安全回路的 PFDavg 在单通道、检验周期 1 年时是 2.19×10^{-3}（SIL 2）；双通道、共因因子 1% 时降到 2.82×10^{-5}（SIL 4）；同样双通道，共因因子 10%、检验周期 5 年时升回 1.22×10^{-3}（SIL 2）。冗余能否换来等级，取决于共因与检验周期，不取决于通道数量。

88.1.6 本章的读法

本章按“原理—设计—分析—参数—调参—改进—落地”排列，不同角色可以走不同路径。

做设计的人重点看 §88.3 与 §88.7：前者给五步流程与程序骨架，后者给七个参数的取值与耦合关系。两节读通，项目的主要决策就定了。

做调试的人重点看 §88.5 与 §88.8：前者解释扫描怎么影响响应，后者给五步整定与症状表。现场时间紧，可以先翻表 88-11。

做评估的人重点看 §88.2、§88.6 与 §88.11：三节给的都是判据与边界，用来回答“这个方案行不行”，不涉及具体接线。

做维护的人重点看 §88.9 与 §88.10：前者讲怎么在不增加风险的前提下改程序，后者讲验收记录与文档移交。

四条路径共享同一组数字，所以无论从哪节进入，都能对回 §88.1.5 的五组主线数据。

88.1.7 五个易混的词

本章反复用到五个词，含义不同，混用会导致算错。

扫描周期是“读输入到下一次读输入”的时间，本章 3.08 ms。它由程序规模决定，随项目变化。

控制周期是 PID 实际被执行的间隔。若 PID 放在主循环里，它等于扫描周期；若放在周期中断里，它是中断周期，与扫描无关。

采样周期是测量值被更新的间隔。它与控制周期可能相同，也可能更慢（远程 I/O 分帧时）。

响应时间是从扰动发生到输出开始动作的时间，等于式 (88-2) 的 T_d，本章 4.24 ms。它永远大于半个扫描周期。

抖动是扫描周期的波动幅度，本章最大 4.88 ms 对平均 3.08 ms。它不影响平均值，只影响最坏情况。

五个词里最容易混的是扫描周期与控制周期。把它们当成同一个量，会在“PID 放中断里”的场合把扫描时间重复算一遍，得出偏保守的结论。

88.2 数学原理

88.2.1 扫描周期的线性模型

扫描周期可以写成程序规模的仿射函数：

$$T_{\text{scan}} = c_0 + c_1 N \tag{88-1}$$

其中 N 是布尔指令条数，c_1 是单条指令的执行时间，c_0 是与程序无关的固定开销。按表 88-1 的数，中型 CPU 的 $c_0 = 1280$ us、$c_1 = 0.03$ us/条，$N = 18000$ 时 $T_{\text{scan}} = 3080$ us。

这个模型的关键在 c_0 不为零。很多人把扫描周期当成“程序多长、周期多长”，于是认为优化程序能等比例缩短周期。实际上 c_0 占了 41.6%，程序砍一半只能把周期从 3.08 ms 降到 2.18 ms，降幅 29%。要把周期压到 1.5 ms 以下，必须动通信服务这块固定开销。

c_1 还与指令类型有关。布尔指令、整数运算、浮点运算、字符串处理的耗时依次上升，浮点乘法可能是布尔指令的 10 到 30 倍。式 (88-1) 只在同一类指令占比稳定的前提下成立，程序结构大改后要重新测 c_1。

88.2.2 采样滞后的等效

PLC 的输入映像在扫描开始时刷新，输出映像在扫描结束时生效。对一次采样而言，从物理量被读到、到对应的输出真正改变，平均要等半个扫描周期。再加上输入滤波、输出刷新与总线传输，总滞后为

$$T_d = \frac{T_{\text{filt}}}{2} + \frac{T_{\text{scan}}}{2} + T_{\text{out}} + T_{\text{bus}} \tag{88-2}$$

代入本章的数：输入滤波 4 ms 取半 2.0 ms，扫描 3.08 ms 取半 1.54 ms，输出 0.20 ms，总线 0.50 ms，合计 4.24 ms。

表 88-3 从采样到执行的延迟分解

环节	均值 / ms	最坏 / ms	说明
输入滤波	2.00	2.00	4 ms 滑动窗，取半
扫描等待	1.54	3.08	与采样时刻相对扫描起点的相位有关
输出刷新	0.20	0.60	继电器与驱动级差异大
现场总线	0.50	1.00	1 ms 周期，取半
合计	4.24	6.68	—

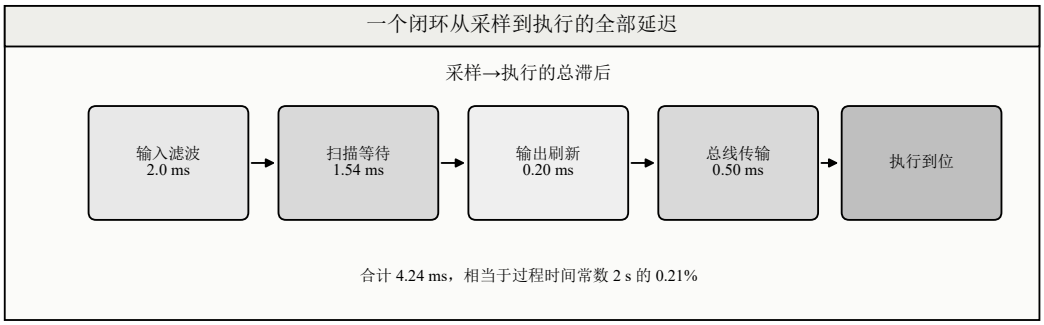


图 88-3 从采样到执行的延迟链

滞后对闭环的影响是相位损失。在穿越频率 ω_c 处，

$$\Delta\varphi = \omega_c T_d \tag{88-3}$$

本章的 $\omega_c = 1.0$ rad/s、$T_d = 4.24$ ms，代入得 0.24°，几乎可以忽略。这也是为什么慢过程用 PLC 完全够用：滞后只有时间常数的 0.21%，相位上根本看不出来。

同样的 4.24 ms 放到伺服轴上就是灾难。第 87 章的电流环穿越频率在 800 Hz 量级，$\omega_c T_d = 2\pi \times 800 \times 0.00424 = 21.3$ rad，折合 1220°，环路早就振不起来。PLC 与运动控制器的分界，本质上就是 T_d 与 ω_c 的乘积能不能压到 0.3 以内。

88.2.3 梯形图到布尔方程

梯形图看似图形，求值却是一条严格的顺序规则：从左到右、从上到下，每个梯级算出一个布尔值，赋给线圈对应的映像位。写成式子，第 i 个梯级对线圈 c 的赋值是

$$c \leftarrow g_i\big(X^{(k)},\, M^{(k)}\big) \tag{88-4}$$

其中 $X^{(k)}$ 是本拍输入映像，$M^{(k)}$ 是当前映像区（含本拍已被前面梯级改写的位）。

式 (88-4) 有两条容易踩的推论。第一条：同一线圈被 m 个梯级驱动时，最终值只由最后一个梯级决定，前 $m-1$ 个的结果被覆盖。第二条：梯级内部引用的是“当前”映像，若某个线圈在本拍已被改写，后面的梯级读到的是新值，而不是上一拍的值。

这两条合起来，使梯形图不等价于布尔代数。布尔代数里 $Y = X_0 \vee X_1$ 与书写顺序无关；梯形图里把它写成两个梯级驱动同一个 Y，结果就变成 $Y = X_1$。要表达“或”，正确写法是把两个触点并联在同一个梯级里，而不是分成两个梯级。

88.2.4 定周期积分的失真

PLC 的 PID 通常写成位置式，积分项按标称采样周期累加：

$$u(k) = K_c\left[e(k) + \frac{T}{T_i}\sum_{j=0}^{k} e(j)\right] \tag{88-5}$$

这里的 T 是组态里填的周期，不是实测的间隔。当某一拍因通信或在线修改被拉长到 $2T$，积分仍然只加 $e\cdot T$，实际经过的时间却是 $2T$。

设掉拍比例为 p，即平均每 $1/(1-p)$ 拍中有 $p/(1-p)$ 拍被拉长为两倍，则每单位真实时间累计的积分量只有应有值的 $1/(1+p)$。等效积分时间为

$$T_{i,\mathrm{eff}} = T_i (1 + p) \tag{88-6}$$

$p = 0.2$ 时 $T_{i,\mathrm{eff}} = 2.4$ s，比整定值长 20%。积分变弱，闭环变慢，超调下降，但恢复时间拉长——这一点在表 88-5 里能看到：超调从 4.0% 降到 0，调节时间从 2.96 s 涨到 5.27 s。

零均值的抖动不改变平均积分速率，只引入积分噪声。工程上真正有害的是“系统性掉拍”：通信块周期性占用较长时间，使扫描周期呈双峰分布。双峰的均值可能只涨 10%，但每一拍都在按标称值积分，误差会累积。

88.2.5 安全回路的 PFD

安全功能的可靠性用平均失效概率衡量。单通道结构在检验周期 T_1 内的平均危险失效概率为

$$\mathrm{PFD}_{1oo1} = \frac{\lambda_{DU} T_1}{2} \tag{88-7}$$

其中 λ_{DU} 是危险未检出失效率，单位 1/h。这个式子的含义很直白：失效率与检验周期的乘积是“一个周期内失效的概率”，取半是因为失效在周期内均匀发生，平均暴露时间是半个周期。

双通道结构并不简单地把概率平方。两套通道可能因共因失效同时失效，用共因因子 β 描述这部分相关性：

$$\mathrm{PFD}_{1oo2} = \frac{\beta \lambda_{DU} T_1}{2} + \frac{(1-\beta)^2 \lambda_{DU}^2 T_1^2}{3} \tag{88-8}$$

第一项是共因项，与单通道同阶，只乘了 β；第二项是独立失效项，是 $\lambda_{DU} T_1$ 的平方量级。当 $\beta = 1\%$ 时第一项比第二项大一个数量级以上，也就是说双通道的可靠性主要由共因决定，而不是由“两套”决定。

三取二结构（2oo3）的独立项系数不同：

$$\mathrm{PFD}_{2oo3} = \frac{\beta \lambda_{DU} T_1}{2} + (1-\beta)^2 \lambda_{DU}^2 T_1^2 \tag{88-9}$$

2oo3 的独立项是 1oo2 的三倍，但它在单通道故障时仍能执行安全功能，可用性更好。选 1oo2 还是 2oo3，取决于“误跳车”与“拒动作”哪个代价更高。

表 88-4 安全完整性等级与 PFDavg 的对应关系

SIL	PFDavg 区间	典型场合
4	10^{-5}–10^{-4}	化工高危、核电
3	10^{-4}–10^{-3}	过程工业紧急停车
2	10^{-3}–10^{-2}	一般机械安全、燃烧器
1	10^{-2}–10^{-1}	简单联锁、警示

选级不是越高越好：SIL 3 以上的冗余、诊断覆盖率与验证工作量都会陡增，多数过程装置按风险分析定到 SIL 2 就能覆盖。

88.2.6 看门狗的时间约束

看门狗是一个独立计数器，程序每拍喂一次，计数溢出就复位 CPU。它的定时必须满足两条：

$$T_{\mathrm{scan,max}} < T_{\mathrm{wd}} < T_{\mathrm{safe}} \tag{88-10}$$

左侧是“正常最坏扫描不能触发复位”，右侧是“真死机时必须在工艺变危险之前复位”。本章的 $T_{\text{scan,max}} = 4.2$ ms（含 25% 余量），$T_{\text{wd}} = 10$ ms，工艺允许的最大失控时间约 100 ms，两条都满足。

工程上常把 T_{wd} 取成最大扫描的 1.5 到 2 倍。取太小，程序升级后一运行就复位；取太大，真死机时要等很久才动作。这个倍数的选择没有理论最优，取决于工艺对失控时间的容忍度。

88.2.7 采样链上的四个环节

扫描周期只是采样链的一环。完整的链条是：传感器响应、输入滤波、模数转换与采样、程序计算。四段各有各的时间常数，串起来才是式 (88-2) 里的 T_d。

本章的数：传感器 10 ms 到 63%、输入滤波时间常数 4 ms（半拍折 2 ms）、扫描 3.08 ms（折 1.54 ms）、输出 0.20 ms、总线 0.50 ms。四段相加 4.24 ms，其中扫描只占 36%。

采样率是 $1/3.08\ \text{ms} = 325$ Hz，过程穿越频率 0.159 Hz，比值 2042。这个比值远高于控制所需的 20 到 40 倍，所以对本章的回路，采样定理不构成约束。

混叠的风险不在控制带宽，而在噪声。40 Hz 以上的噪声被输入滤波衰减，残余部分折叠到低频，进入微分项后被放大。这是“扫描很快但输出仍抖动”的常见原因，问题不在采样率，在抗混叠滤波不够。

量化误差是另一条。12 位模数转换在 0 到 100% 量程上的分辨率是 0.024%，16 位是 0.0015%。单次调节看不出差别，做流量积算或能量累计时，0.024% 的系统偏差一年会累成可观的数。

88.2.8 远程 I/O 的总线叠加

本地 I/O 的刷新已经算在输入输出块里。远程站不一样：每个站的数据要经过现场总线，一帧的往返约 0.5 ms，站数增加，这段延迟按站数叠加。

n_s 个站全部轮询一遍的时间是 $n_s \times 0.5$ ms。三站是 1.5 ms，总滞后从 4.24 ms 涨到 5.24 ms；八站是 4.0 ms，总滞后 7.74 ms。相对 2 s 的时间常数，这仍然是 0.39%，控制品质看不出变化。

对联锁就不一样了。一条跨站的联锁，信号要经过“远端采样—总线—本机程序—总线—远端输出”，最坏情况是两倍总线延迟加一次扫描。八站时这条链是 4.0+3.08+4.0=11.1 ms，相对 100 ms 的要求还有余量，但已经不能再加站。

分站的原则是按物理位置，不是按信号类型。按位置分站，一帧数据对应一个区域，联锁大多在区域内部完成；按类型分站，一个温度信号和一个阀门输出可能隔着三个站，联锁链被拉长。

88.2.9 扫描周期的统计描述

扫描周期不是常数，是一个随机变量。把它当常数用，设计就会偏乐观。

该看的统计量有四个：平均、95 分位、最大、超时次数。平均描述常态，95 分位描述偶发，最大决定看门狗，超时次数决定可用性。

本章的数：平均 3.08 ms，95 分位 3.4 ms，最大 4.2 ms。最大值比平均高 36%，这段差距来自通信服务与诊断写入，不是程序本身。

统计窗口至少覆盖一个完整工艺周期。只看十分钟，可能一次都没赶上“上位机全量刷新”；看二十四小时，才能把交接班、配方切换、报表生成这些周期性事件都包进来。

设计点取 95 分位的 1.2 倍，即 $3.4 \times 1.2 = 4.08$ ms，向上取整为 4.2 ms，正好对上 §88.2.6 的 $T_{\text{scan,max}}$。直接用平均算看门狗，遇到一次通信高峰就会复位。

88.2.10 扫描模型的现场标定

式 (88-1) 的两个系数 c_0、c_1 不能照抄样本值，要在现场标定。方法很简单：用两个规模不同的程序各测一次扫描。

造一个 10 000 条指令的空循环程序，测扫描 2.28 ms；再加 10 000 条，测 3.28 ms。两次相减得 $c_1 = 0.1$ μs/条，回代得 $c_0 = 1.28$ ms，与表 88-1 的固定开销一致。

标定一次约需半小时，价值在后面每一次改程序。有了这两个系数，新加 3000 条指令的代价可以直接算：$3000 \times 0.1 = 300$ μs，扫描从 3.08 ms 变成 3.38 ms，占看门狗 33.8%。

标定时要注意指令类型。布尔指令 0.1 μs，浮点运算可能到 1 μs，字符串处理更贵。用布尔指令标出来的 c_1 只对布尔逻辑有效，混入浮点后实际扫描会比估算高。

两个点标定出的直线只在中等规模成立。程序小到几千条时，c_0 里的通信与自检占比更高，直线斜率会偏大；大到几十万条时，缓存失效会让 c_1 上升。标定点要选在项目实际规模附近。

88.3 设计步骤

88.3.1 第一步：定节拍

节拍由三条约束共同决定，取其中最严的一条。

第一条来自过程动态。扫描周期是纯滞后，滞后占时间常数的比例越小，对裕度的影响越小。本章的数据是：2.5%（50 ms）损失 1.4°，5%（100 ms）损失 2.8°，10%（200 ms）损失 5.7°。工程上取 $T_{\text{scan}} \le \tau/20$ 作为起点，要求高的回路取 $\tau/50$。

第二条来自联锁响应。急停、超压、超温这类联锁，从信号变化到执行器动作的总时间由式 (88-2) 给出，本章是 4.24 ms，相对 100 ms 的工艺要求有三个数量级余量。这一条通常不是瓶颈，除非输入滤波设得很长。

第三条来自程序规模。按 I/O 点数与回路数估出指令条数 N，代入式 (88-1) 得到周期。若算出来超过第一条给的上限，只有两条路：拆程序到多个 CPU，或者砍功能。

88.3.2 第二步：定 I/O 与映像区

映像区是扫描语义的物理基础。输入映像在扫描开始时整体刷新，程序读的是映像而不是模块；输出映像在扫描结束时整体下发。这个设计让程序在整拍内看到“冻结”的输入，避免了同一拍内前后读到不同值。

模拟量通道要单独设滤波。4–20 mA 输入常配 4 ms 滑动平均，对应 250 Hz 带宽，对温度、压力足够；对流量这种快变量要降到 1 ms 以内。滤波时间直接进入式 (88-2) 的第一项，设长了会吃掉裕度。

高速信号不能走扫描。编码器计数、脉冲输出、PWM 这类信号由独立硬件处理，CPU 只读结果。把它们放进扫描循环，等于让 3 ms 的周期去处理微秒级的事件，必然丢脉冲。

分布式 I/O 的总线周期与扫描周期解耦。总线按自己的节拍刷新远程站，PLC 在扫描开始时从总线缓冲区取数。这意味着远程 I/O 的实际滞后还叠加了总线周期的一半，本章取 0.5 ms。

88.3.3 第三步：程序分层

IEC 61131-3 定义了四类程序组织单元，各有用处。组织块（OB）是入口，主循环 OB1 每拍调用一次，周期中断 OB3 按固定间隔调用，诊断 OB82 在模块故障时调用。功能块（FB）带背景数据块，有记忆；功能（FC）无记忆，只做运算；数据块（DB）存工艺参数与状态。

图 88-7 给出一个最小可用的三层结构。顶层是 OB1，中层是三个 FB/FC，底层是三个 DB。阀门控制做成 FB，因为它要记住开度、超时、故障计数；PID 做成 FB，因为它要记住积分项与上次误差；量纲换算做成 FC，因为它只是乘一个系数。

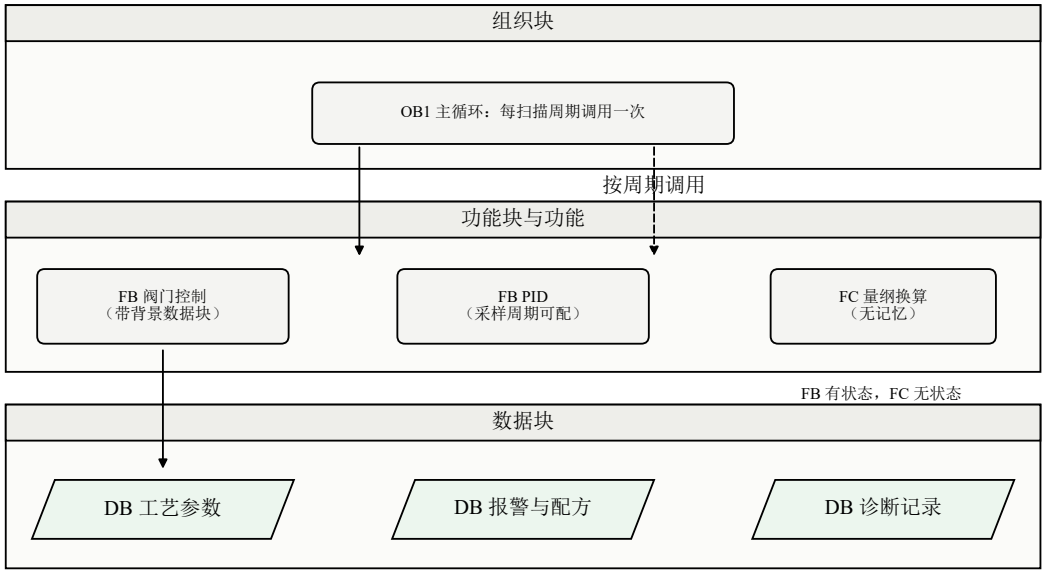


图 88-7 程序的模块化结构

PID 必须放在周期中断里，不能放在主循环。主循环的周期随程序分支变化，而 PID 的积分与微分都假定采样间隔恒定。放在主循环里，式 (88-6) 的失真会随工况漂移，整定值失去意义。

一个 FB 用单实例还是多实例，取决于调用次数。同一台设备的多台副本用多实例，共享一个背景 DB 数组；不同设备用单实例，各自有独立 DB。多实例省内存，但调试时变量路径更深，现场维护人员不容易找到。

88.3.4 第四步：安全功能分层

安全功能与标准控制必须在逻辑上分开，最好是物理上分开。安全 PLC 执行急停、安全门、光幕、超压保护；标准 PLC 执行启停、调节、报警。两者之间只允许单向传递：安全状态可以送给标准程序做显示，标准程序不能影响安全逻辑。

同一颗 CPU 上做安全与标准任务时，安全任务有独立的内核校验、独立的内存区、独立的时钟。标准任务崩溃不能影响安全任务的执行，这靠内存保护单元与独立看门狗实现。这种结构省硬件，但认证时要提供分区隔离的证据。

安全程序要短。一个安全功能块通常只有几十条指令，因为它必须能被完整验证。把工艺联锁写进安全程序是常见错误：工艺会变，安全程序不该跟着变。

88.3.5 第五步：验证

设计完成后要做四项验证。

扫描余量：最坏扫描不超过看门狗的 50%，正常扫描不超过 30%。本章的 3.08 ms 对 10 ms 看门狗是 30.8%，刚好在线上；程序再长 25%，就升到 35.3%，需要重新评估。

故障注入：拔掉通信线、断开远程站、强制输入点、模拟模块故障，逐项确认系统进入预定状态。这一项最能暴露“只在正常路径上正确”的程序。

在线修改验证：下载前后各跑一次完整工艺循环，对比关键变量曲线。在线修改会改变扫描周期，进而改变 PID 的等效积分时间，这一点常被忽略。

文档一致性：I/O 表、变量表、联锁表三份文档必须与程序一致。现场排故时，工程师看的是文档，不是代码。

88.3.6 I/O 分配与地址规划

I/O 分配表是程序的底稿，也是日后排故的地图。分配时守三条：按工艺单元分段、按信号类型成组、留出余量。

按工艺单元分段，是把同一台设备的输入输出排在一起。反应釜的温度、搅拌、进料阀占连续的地址段，改这台设备时不用翻整张表。按类型分段（所有模拟量在一起）只在点数很少时方便，点数一多就找不到对应关系。

按信号类型成组，是因为模拟量模块的通道数固定（4 路、8 路），混排会浪费通道。一个 8 路模块只接了 3 路温度，剩下 5 路不能给开关量用。

余量按 15% 到 20% 留。一个 300 点的项目留 45 到 60 点。留少了，加一个检测工位就要扩模块；留多了，柜子变大、成本上升。经验值是每个机架留 2 到 4 个空槽。

安全点必须单独成组，接在安全模块上，不能与标准点混在同一模块。混装的后果是认证时整个模块都要按安全等级重新评估，成本远高于单独买一块安全模块。

点号命名要有规则：单元号加设备号加信号类型加序号。规则写在图纸首页，所有人按同一套规则命名。没有规则的项目，半年后连“这个 I0.7 是什么”都查不出来。

88.3.7 一个最小程序骨架

程序骨架决定后面每一段代码往哪里放。图 88-7 给的三层结构落到代码上，是下面这个调用顺序。

启动组织块只做初始化：清零累计量、装载配方、把输出置安全位、建立通信连接。它只执行一次，不写业务逻辑。

主循环按“读—判—算—写”四段排：刷新输入映像、执行报警与联锁、执行回路、刷新输出映像。联锁必须排在回路之前，否则本拍算出的输出可能绕过联锁直接下发。

报警与联锁写成一个功能，每个联锁一屏，输入是条件、输出是动作与延时。回路写成功能块，每个回路一个实例数据块，参数（量程、增益、积分时间、输出限幅）放在实例里，不放在全局。

手自动切换放在回路功能块内部，不放在主循环。切换时要无扰：手动转自动时，把当前手动输出预置进积分器，避免输出跳变。这一条不写，切换时阀门会猛动一下。

最后一段是输出刷新与诊断。诊断信息写进诊断数据块，由通信块上传，不直接驱动输出。把诊断和联锁混在一起，是“报警一多就停机”的根源。

88.3.8 安全功能的验证方法

安全功能写完不算完，要证明它在失效时真的动作。验证有三项：注入、计时、旁路。

注入是把信号强行置成故障态：断开传感器线、短接输出、强制某个位。每注入一次，看系统是否进入预定状态，以及进入的时间。注入要逐点做，不能抽样。

计时用示波器测整条链：传感器动作的时刻接一个通道，阀门到位的反馈接另一个通道，两个时刻之差就是实际响应时间。本章的链是 106 ms，若实测超过 120 ms，就要查是哪一段变慢。

旁路是验证旁路开关本身有效，以及旁路状态能被监视。旁路装好不复位是最大的隐患，验证时要专门测“旁路超时是否报警”。

三项验证的频率不同。注入与计时在 FAT 与 SAT 各做一次，之后每年做一次全功能；旁路试验每次检修后都做，因为它与本次作业直接相关。

88.3.9 离线测试台的最小配置

离线测试不需要复刻整条产线，一组最小配置就能覆盖八成的逻辑错误：24 V 电源、八路开关、八只指示灯、两路 4 到 20 mA 信号源、一台装编程软件的电脑。

开关模拟传感器，指示灯模拟执行器，信号源模拟模拟量。把程序下载进去，按联锁表逐条触发，看指示灯是否按预期变化。这套配置的成本不到两千元，能在办公室里发现大部分错误。

测试台覆盖不了三件事。第一是时序：真实气缸的动作要 0.5 s，台架上的指示灯是瞬时的，程序里依赖“到位信号”的逻辑测不出来。第二是干扰：台架没有变频器、没有长电缆。第三是惯性：阀门的实际行程与台架上的信号源无关。

所以台架测试通过不等于现场通过。它的价值是把“逻辑错误”和“现场问题”分开，让现场调试只处理后者。

台架测试要留记录：每个联锁的触发条件、预期动作、实际动作、测试人。记录不写，等于没测。

88.4 连续情形

88.4.1 连续域的基准

先把 PLC 拿掉，看连续控制器能做到什么。本章的过程是一阶加纯滞后：时间常数 2 s、纯滞后 0.5 s、单位增益。用内模控制整定 PI，取闭环时间常数 $\lambda = 0.5$ s，得到 $K_c = 2.0$、$T_i = 2.0$ s。

这组参数下，闭环近似为“纯滞后 + 一阶惯性”：滞后 0.5 s 不变，惯性时间常数变成 $\lambda = 0.5$ s。阶跃响应没有超调，2% 调节时间约 2.5 s。这是连续域的理论基准，也是后面判断离散实现好坏的那把尺子。

基准不是“最优”。把 λ 取到 0.2 s，响应更快，但增益涨到 3.33，对滞后变化的敏感度上升；取到 1.0 s，鲁棒性更好，但慢一倍。λ 的选择是快速性与鲁棒性的折中，跟采样无关。

88.4.2 连续域的真实滞后

第 88.2 节算出的 4.24 ms 只是 PLC 引入的部分。真实回路的滞后还包括传感器与执行器：

热电偶加保护套管的时间常数常在 5 到 20 s，比 PLC 的扫描周期大三个数量级；气动调节阀的全行程时间 0.5 到 2 s；变频器加电机的电气时间常数几十毫秒。这些环节串起来，PLC 那 4 ms 连零头都算不上。

这个事实决定了一件事：慢过程的控制品质主要取决于传感器与执行器，而不是 PLC 的扫描周期。现场常见“换个更快的 PLC 就好了”的判断，多数情况下是错的。真正该换的是响应慢的测温元件或卡涩的阀门。

反过来，当过程本身很快时，PLC 的 4 ms 就成了主要滞后。流量回路的时间常数可能只有 0.1 s，此时 4 ms 占 4%，相位上已经开始有影响；压力回路更快的场合，PLC 就顶不住了。

88.4.3 连续域的鲁棒性

连续设计的鲁棒性可以用相位裕度衡量。本章的基准回路在 $\omega_c = 1.0$ rad/s 处有 61.3° 裕度，扣掉纯滞后 0.5 s 贡献的 28.6°，其余来自惯性环节。

若纯滞后增大 50%，即从 0.5 s 变成 0.75 s，相位损失增加 $\omega_c \Delta L = 0.25$ rad，合 14.3°，裕度降到 47.0°。这个数与表 88-6 里扫描周期 500 ms 那一行完全一致——因为那一行的等效延迟也多了 0.25 s。滞后就是滞后，来源不重要。

若过程增益增大 30%，穿越频率随之升高，裕度会进一步下降。用本章的参数估算，增益 1.3 时穿越频率升到 1.3 rad/s，三项滞后合计约 128°，裕度降到 52.4°。这仍在可接受范围，但已经接近工程下限。

若增益增大 30% 与滞后增大 50% 同时发生，裕度降到 33.8°，低于 45° 的常用下限，回路会出现明显振荡。这一格就是整定时要留的余量：两项单独都还能站住，叠加起来就站不住。

88.4.4 连续近似的适用条件

用连续工具设计 PLC 回路，需要两个条件同时成立。

第一，扫描周期远小于过程时间常数。本章的 3.08 ms 对 2 s 是 0.15%，属于“远小于”；若扫描到 100 ms，占比 5%，连续近似开始有可见误差（超调从 4.0% 涨到 7.8%）。

第二，扫描周期远小于闭环带宽的倒数。$\omega_c T_{\text{scan}} \ll 1$ 时，ZOH 的半拍滞后可以忽略；本章 $\omega_c T_{\text{scan}} = 0.003$，当然满足。若 ω_c 到 100 rad/s、扫描 10 ms，乘积到 1.0，连续设计就完全失效了。

两个条件都满足时，可以放心用连续域的工具整定，再把参数搬进 PLC，最后用离散仿真校验一遍。校验的目的不是重新整定，而是确认离散化没有把裕度吃掉。

条件不满足时，只能直接在离散域设计。第 88.5 节给出离散域的响应数据，第 88.6 节给出稳定边界。

88.4.5 连续基准与离散实现的差距

把本章的离散仿真结果与连续基准对照，可以看到差距的分布。

扫描 10 ms 时，超调 4.0%，调节时间 2.96 s；连续基准的超调接近 0，调节时间约 2.5 s。差距主要来自采样滞后与积分项的离散化，量级是 0.5 s，占基准的 20%。

扫描 50 ms 时，超调 5.7%，调节时间 3.04 s。超调继续上升，调节时间却几乎不动，因为超调与调节时间对滞后的敏感度不同：超调看的是相位裕度，调节时间看的是积分强度。

扫描 500 ms 时，超调 32.1%，调节时间 6.26 s，两项都显著恶化。此时扫描周期已占时间常数的 25%，连续近似彻底失效，整定参数必须重来。

这组对照说明：连续设计给出的参数有“有效期”。有效期由扫描周期与时间常数的比值决定，比值小于 1% 时几乎无损，到 5% 时开始可见，到 25% 时失效。

88.4.6 连续模型忽略了什么

连续设计给出的 $K_c = 2.0$、$T_i = 2.0$ s 是理想值。真实 PLC 实现里，有四项被连续模型忽略了，每一项的误差量级不同，值不值得管也不同。

第一项是量化。12 位模数转换的 1 个最低位是量程的 0.024%，经比例增益 2 放大后变成 0.048% 的输出台阶。若阀门分辨率是 0.1%，这个台阶已经低于阀门能分辨的最小动作，不产生极限环。

第二项是采样抖动。实际扫描不会每拍都恰好 3.08 ms，最大可到 4.2 ms。抖动对相位的影响是 $\omega_c \Delta T / 2$，取 $\Delta T = 1$ ms、$\omega_c = 1$ rad/s，得到 0.029°，可以忽略。

第三项是执行器死区。阀门死区 0.5% 时，稳态误差带是死区除以开环增益，即 0.25%。这一项与扫描无关，但常被误归因于“扫描太慢导致压不住”。

第四项是计算延迟的具体分布。本章把整个扫描折成半拍，实际是“输入刷新在前、程序在中间、输出刷新在末尾”，等效延迟在 0 到 1 拍之间均匀分布，平均半拍。这个近似在扫描远小于时间常数时成立，误差在 0.1 拍以内。

四项加起来，本章的总误差不到 0.3° 相位，相对于 61° 的裕度是 0.5%。这就是“连续设计对本章够用”的定量依据，而不是一句“扫描够快就行”。

88.4.7 什么时候必须做离散设计

连续设计不是永远够用。判据有两条，满足任意一条就该改用离散方法。

第一条是 $T_{\text{scan}} / \tau > 0.05$。本章是 0.0015，离得很远；扫描到 100 ms 时是 0.05，正好在边界；到 500 ms 时是 0.25，连续模型失效，表 88-6 里 32.1% 的超调就是这么来的。

第二条是 $\omega_c T_{\text{scan}} > 0.1$。本章是 0.0031，同样很远；一个穿越频率 50 rad/s、扫描 3 ms 的回路是 0.15，已经越线，连续整定的参数直接用会振荡。

反过来看，若 T_{scan} 已经接近过程的纯滞后，例如纯滞后 50 ms 而扫描 20 ms，即便比值 $\omega_c T_{\text{scan}}$ 不大，也该做离散设计。原因是扫描滞后与纯滞后叠加后，总滞后翻倍，连续域里那个“半拍”的近似误差被放大。

离散设计的常用方法有三种：直接按 Z 域极点配置、Dahlin 算法、以及把连续控制器做 Tustin 变换后再补偿滞后。三种方法在第 85 章都有推导，本章只给结论：扫描周期进入时间常数的 5% 以后，按连续方法整定的参数必须重新验证，不能照搬。

工程上更常见的做法不是换设计方法，而是换节拍。把回路挪进 10 ms 的周期中断，T_{scan}/τ 立刻从 0.25 降到 0.005，连续设计又够用了。这比重新推导离散控制器便宜得多。

88.4.8 内模整定式的来源

本章的 $K_c = 2.0$、$T_i = 2.0$ s 不是试出来的，来自内模控制对一阶加纯滞后的整定式：

$$K_c = \frac{\tau}{K(\lambda + L)}, \qquad T_i = \tau \tag{88-11}$$

取 $\tau = 2$ s、$K = 1$、$L = 0.5$ s、$\lambda = 0.5$ s，得 $K_c = 2/(0.5 + 0.5) = 2.0$，$T_i = 2.0$ s，与仿真的初值一致。

λ 是闭环时间常数，取值决定快慢与鲁棒。λ 取 0.25 s 时 $K_c = 2.67$，响应快但裕度降到 45° 左右；取 1.0 s 时 $K_c = 1.33$，很稳但慢一倍。

工程上把 λ 取在 $0.5L$ 到 $2L$ 之间。模型准、执行器快，取小值；模型粗糙、有未建模动态，取大值。本章取 L 本身，是折中。

这条整定式的前提是模型已知且线性。若 L 估小了 30%，实际裕度会比计算值低 10° 以上。所以 §88.7.10 的辨识精度，直接决定整定的可信度。

88.5 离散情形

88.5.1 扫描就是零阶保持

从控制的角度看，PLC 的输出在两次刷新之间保持不变，这正是零阶保持器。第 4 章讲过，零阶保持引入的平均滞后是半个采样周期；第 85 章把这个结论落到数字控制系统的采样周期选择上。PLC 的特殊之处在于，“采样周期”不是一个可以随便设的参数，而是由程序规模决定的扫描周期。

这带来一个后果：控制器参数与程序规模耦合。同一个 PID 回路，程序加长后扫描变慢，等效积分时间与微分增益都随之变化。伺服驱动器不会出现这个问题，因为它的采样周期由定时器固定，与代码长度无关。

工程上的对策是把 PID 放进固定周期的中断里，让采样周期与扫描周期解耦。主循环再长，中断仍然按 1 ms 或 10 ms 触发；代价是中断服务程序不能太长，否则会拖慢主循环。第 88.5.6 节讨论这个权衡。

88.5.2 掉拍与积分失真

掉拍是指某一拍的实际间隔超过标称值。原因有三类：通信服务占用、大块数据拷贝、在线修改时的代码替换。前两类是周期性的，第三类是偶发的。

表 88-5 给出标称 10 ms、掉拍时实际间隔翻倍时的仿真结果。掉拍比例从 0 增到 30%，等效积分时间按式 (88-6) 从 2.00 s 拉长到 2.60 s。

表 88-5 掉拍比例对闭环指标的影响（标称 10 ms）

掉拍 / %	超调 / %	调节时间 / s	IAE	等效 T_i / s
0	4.0	2.96	1.086	2.00
5	2.4	2.47	1.097	2.10
10	1.2	2.94	1.119	2.20
20	0.0	5.27	1.214	2.40
30	0.0	6.35	1.305	2.60

表 88-5 的第一列与最后一列严格对应，中间三列不是。5% 那一行的调节时间 2.47 s 比 0% 的 2.96 s 还短，这是 10 个随机种子的平均结果，属于抽样波动，不构成趋势。10% 之后三列才单调恶化，看趋势要看这一段。

超调随掉拍增加而下降，这个方向容易误导。积分变弱，系统“冲劲”变小，超调自然下降；但代价是恢复变慢、稳态误差存留更久。只看超调会得出“掉拍无害”的结论，看 IAE 才能看到真实损失：从 1.086 涨到 1.305，增加 20%。

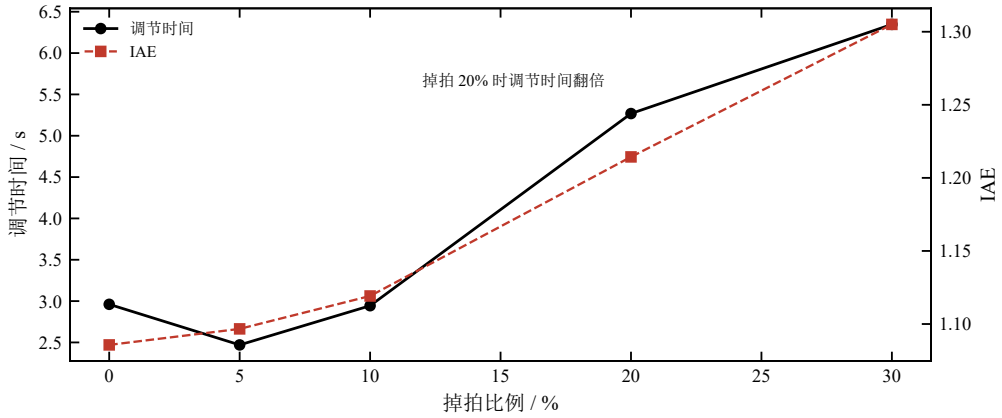


图 88-5 掉拍比例对调节时间与 IAE 的影响

工程上允许的掉拍比例通常在 5% 以内。超过这个值，要么是通信块设计得太重，要么是主循环里有阻塞调用。第 88.9 节给出几种削减通信占用的做法。

88.5.3 扫描周期对响应的影响

表 88-6 把扫描周期从 10 ms 扫到 500 ms，其余参数不动。穿越频率始终是 1.0 rad/s，因为增益没变；相位裕度从 61.1° 降到 47.0°，全部来自扫描引入的附加滞后。

表 88-6 扫描周期对相位裕度与阶跃响应的影响

扫描 / ms	相位裕度 / (°)	超调 / %	调节时间 / s
10	61.1	4.0	2.96
20	60.8	4.4	3.00
50	59.9	5.7	3.04
100	58.5	7.8	3.08
200	55.6	12.3	4.35
500	47.0	32.1	6.26

表 88-6 最值得注意的是超调与调节时间的不同步。扫描从 10 ms 到 100 ms，超调翻了近一倍（4.0% 到 7.8%），调节时间只涨了 4%（2.96 s 到 3.08 s）。到 200 ms 之后，两者才开始同步恶化。

原因是两个指标看的东西不同。超调看相位裕度，扫描滞后直接吃裕度；调节时间看积分强度，而积分强度由 T_i 决定，与扫描周期只有二阶关系。这也解释了为什么现场“响应变慢”和“振荡加剧”往往不是同一个原因引起的。

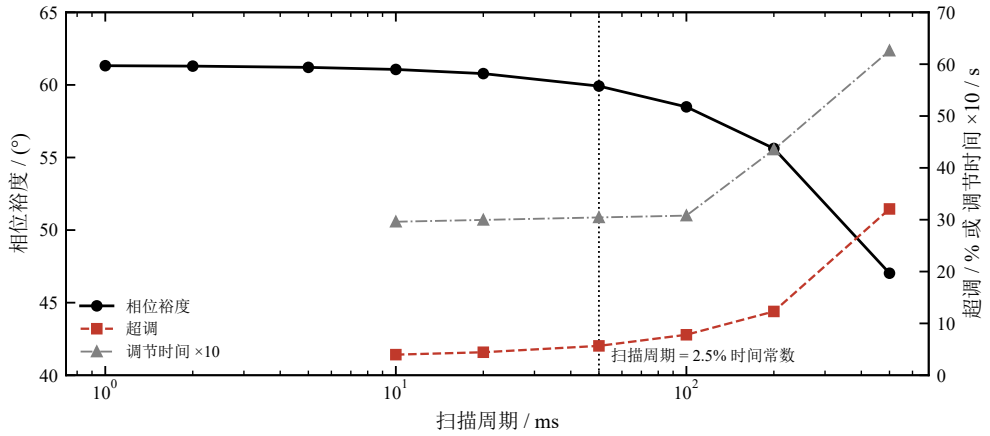


图 88-4 扫描周期对裕度、超调与调节时间的影响

按时间常数的比例看，10 ms 是 0.5%，50 ms 是 2.5%，200 ms 是 10%。0.5% 到 2.5% 这一段的代价很小，2.5% 到 10% 这一段开始明显，10% 以上不可接受。工程上取 2.5% 作为设计点，留出一倍余量到 5%。

88.5.4 梯形图的离散语义

梯形图的“离散”不在时间上，而在求值顺序上。每个扫描周期内，梯级按书写顺序执行一次，线圈的映像位可以被后续梯级立即读到。这与连续系统的“同时求解”完全不同。

图 88-6 用两个梯级驱动同一个线圈说明后果。R1 写 $Y_0 = X_0$，R2 写 $Y_0 = X_1$，执行完后 $Y_0 = X_1$，X_0 的结果被覆盖。用 8 种输入组合逐一核对，Y_0 有 2 种与“或”语义不符；把 Y_0 再串到下一组梯级，Y_1 有 4 种不符。

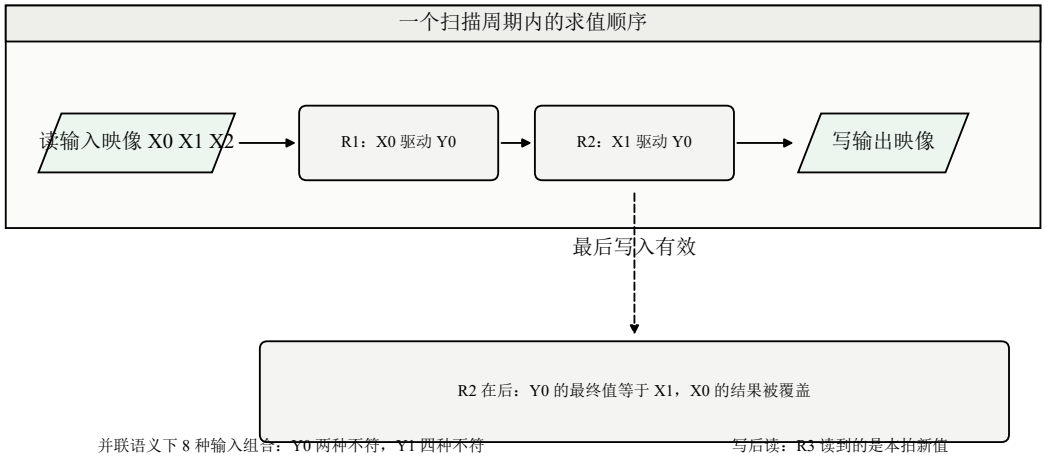


图 88-6 双线圈与扫描顺序

这个例子的工程意义不是“别写双线圈”，而是“梯形图不是布尔代数”。布尔代数允许交换律、结合律，梯形图只允许在单个梯级内部使用这些律。跨梯级的“或”必须写成触点并联，跨梯级的“与”必须写成触点串联。

还有一条更隐蔽的规则：同一梯级内，触点的求值顺序也影响结果。若某个触点读取的线圈在本梯级前面已被改写，读到的是新值；若在后面，读到的是旧值。这种“写后读”在跨梯级时同样成立，调试时看到的中间值与最终值可能不一

致。

88.5.5 增量式实现与数值细节

位置式 PID 的输出直接对应阀门开度，积分项是一个不断增大的累加器。工程上更常用增量式：

$$\Delta u(k) = K_c\big[e(k) - e(k-1)\big] + \frac{K_c T}{T_i} e(k) + \frac{K_d}{T}\big[e(k) - 2e(k-1) + e(k-2)\big] \tag{88-12}$$

增量式的好处是输出只与最近三次误差有关，累加器不会无限增大，切换手动/自动时无扰。坏处是丢失了绝对位置，需要外部保持上一拍输出值。

数值上有三个细节。第一，微分项除以 T，扫描周期抖动会被放大：T 变小 20% 时微分增益涨 25%，噪声随之放大。第二，积分项乘以 T，掉拍时同样让积分变弱，方向与位置式一致，区别只在于增量式没有累加器溢出与切换扰动的问题。第三，所有中间结果要用 32 位，16 位累加器在 10 s 量级的积分时间里就会溢出。

抗饱和在增量式里更简单：输出限幅后，把超出部分从累加器里扣掉。位置式需要单独的限幅回路，增量式只需在 Δu 上做一次判断。

88.5.6 周期中断与主循环

PLC 的运行时通常提供两类任务：主循环任务与周期中断任务。主循环按扫描执行，周期不固定；周期中断按固定间隔触发，优先级高于主循环。

PID 放在周期中断里，采样周期就与程序规模解耦。中断间隔取 1 ms 到 10 ms，取决于回路带宽。本章的回路 $\omega_c = 1$ rad/s，10 ms 足够；流量回路 ω_c 可能到 10 rad/s，要取 1 ms。

中断会拖慢主循环。每次中断的进出开销加上执行时间，从主循环的时间里扣。若中断每 1 ms 触发一次、每次 50 us，主循环就被扣掉 5% 的时间。这个比例要算进扫描预算，否则程序上线后发现扫描周期比设计值长。

多个周期中断之间还有优先级与抖动。高优先级中断会抢占低优先级，低优先级中断的实际间隔因此被拉长。这与第 86 章的任务调度是同一个问题，只是粒度更粗。工程上把最紧的回路放在最高优先级，把慢回路放在主循环。

时间戳对齐是常被忽略的一步。周期中断采到的数据带中断时刻的时间戳，主循环采到的数据带扫描时刻的时间戳。两者混在一起做趋势分析时，若不对齐，会看到并不存在的相位差。

88.5.7 量化与死区在离散回路里的表现

连续域里，误差可以无限小，积分器总能慢慢爬。离散域里，误差被量化成一个个台阶，积分器爬不动的地方就成了稳态误差带。

台阶的大小等于量化单位乘以比例增益。本章的 1 个最低位是 0.024%，乘 2 是 0.048%，小于阀门分辨率，回路能稳定在设定值上，不产生极限环。

若把增益提到 10，台阶变成 0.24%，超过阀门分辨率，回路就会在设定值附近来回跳，周期是两个扫描。这个现象叫量化极限环，靠调参数消不掉，只能提高分辨率或降低增益。

位置式 PID 与增量式 PID 在量化下表现不同。位置式的积分器存的是绝对量，小误差累加几十拍后才越过一个台阶，输出是连续爬升的。增量式每拍算的是增量，增量小于一个量化单位时被舍掉，输出永远不动，表现为“小信号不动作”。

这条差别解释了 §88.8.7 里那类症状。现场遇到“误差 0.5% 时阀门不动，误差 1% 时突然动作”，先查输出分辨率，再查增益，最后才怀疑 PID 参数。

88.5.8 扫描顺序与顺序控制

连续回路关心的是每拍算一次，顺序控制关心的是每一步走一拍。两者的时间尺度不同，写法也不同。

顺序功能图把工艺拆成步，每步有入口条件、动作、出口条件。一个扫描内只允许推进一步，这是安全设计的默认规则。允许一拍跳多步，会出现“阀门还没到位就进入下一步”的问题。

实际程序里，跳步往往不是显式写的，而是出口条件写得太宽。例如“液位高于 80% 或时间超过 60 s”作为出口，液位在 3 个扫描内从 78% 冲到 82%，程序就连跳两步。

每步都要有超时监视。超时值取正常步时的 1.5 到 2 倍，超时后停在当前步并报警，不自动跳到下一步。自动跳步会把“卡住”变成“错动作”，后者更难查。

步切换时的输出处理要遵守“先断后通”。同一条管线上的两个阀，若在同一拍里一个开一个关，会有瞬间两者都开的可能。正确做法是先关后开，中间隔一个扫描。

88.5.9 梯形图里的竞争与冒险

布尔代数里 $Y = AB + \bar{A}C$ 在 $B = C = 1$ 时恒为 1，与 A 无关。梯形图按顺序求值，这个恒等式不一定成立。

原因是两个梯级不是同时求值的。第一个梯级用 A 的旧值算 AB，第二个梯级用的是 A 的新值算 $\bar{A}C$。若 A 在两次求值之间变化，两个梯级可能同时输出 0，出现一个扫描宽的毛刺。

毛刺宽度等于扫描周期。本章是 3.08 ms，对 100 ms 的联锁是 3%，可以忽略；对一个 1 kHz 的计数输入就是三个脉冲，不能忽略。

消除办法有两个。一是把两个梯级合成一个，先算中间变量再输出，保证同一拍内原子求值。二是改用置位/复位指令，让状态由事件驱动而不是由电平驱动。

这条差异是梯形图与布尔代数的根本区别。写程序时要假设“每个梯级的输入都可能与上一梯级不同”，而不是假设它们同时有效。

88.5.10 频域指标与时域指标的换算

经典控制里有一个经验关系：相位裕度约等于 100ζ，二阶系统的超调由 ζ 决定。用这条关系，61° 对应 $\zeta = 0.61$，超调约 9%。

本章实测是 4.0%，比经验值小一半。原因是本章的过程是一阶加纯滞后，不是二阶振荡系统；纯滞后只吃相位，不产生谐振峰，所以同样的裕度对应更小的超调。

裕度与超调的关系也不是线性的。61.1° 时 4.0%，55.6° 时 12.3%，47.0° 时 32.1%。裕度掉 14°，超调涨 8 倍。这条曲线在 50° 附近有一个明显的拐点。

所以不能只用裕度推超调，也不能只用超调判断稳定性。工程上两个都看：裕度用于设计，超调用于验收，两者对不上时以实测为准。

88.5.11 输出刷新与执行器动作

扫描末的输出刷新是“一次性写映像区，再逐个模块下发”。映像区同时更新，物理输出却有先后：同一个机架上的模块按槽位顺序刷新，远程站按总线顺序刷新。

先后差多少取决于模块数。八个模块、每模块 20 µs，最后一块比第一块晚 140 µs。相对扫描 3.08 ms 是 4.5%，对“先断后通”这类互锁动作要计入。

更要紧的是失效时的输出状态。输出模块失电、通信中断、CPU 停机时，输出是全断、全保持、还是按组配置，各家不同。安全相关的输出必须配成“失电即断”，并且这条要写进 FAT 记录。

很多“停机时阀门没关”的事故，根因不是程序，是输出模块配成了“保持最后值”。程序停了，输出还挂着，操作员以为已经安全。

88.5.12 时钟同步与事件顺序

多台 PLC 连成网时，各自的时间戳可能对不上。本地晶振的精度通常是 ±50 ppm，一天漂 4.3 s；不做同步，三天后的报警记录就没法按时间排序。

同步方式有三种。NTP 精度 1 到 10 ms，够做趋势与报警；PTP（IEEE 1588）精度到 1 μs，用于事件顺序记录（SOE）与跨站联锁分析；硬件同步信号精度更高，用于多轴。

本章的扫描是 3.08 ms，事件的时间分辨率本来就受扫描限制。所以对单站而言，NTP 的毫秒级精度已经够用，再高的精度也体现不出来。

跨站 SOE 是例外。两个站的事件相差 2 ms，用 NTP 同步可能被误差淹没，判不出先后。要求分辨先后顺序的场合，必须用 PTP，并且交换机要支持透明时钟。

时间同步做不好，最典型的后果是“事故复盘时两条报警的先后反了”。控制器本身没错，错在时间戳，而时间戳恰恰是最难事后补救的一项记录。

88.6 稳定性与性能

88.6.1 稳定边界来自两处

PLC 回路的稳定边界由两处滞后共同决定：过程本身的纯滞后 L，以及扫描引入的等效滞后 $T_{\text{scan}}/2$。前者由工艺决定，改不了；后者由设计决定，可以选。

本章的回路在 $L = 0.5$ s、扫描 10 ms 时裕度 61.1°。把扫描拉长到 500 ms，等效滞后增加 0.245 s，裕度掉到 47.0°。这两组数的差别说明：扫描滞后在 10 ms 量级时几乎不参与稳定边界，在 500 ms 量级时变成主要因素。

判断“扫描是否参与稳定边界”有一个简单办法：比较 $T_{\text{scan}}/2$ 与 L。前者不到后者的 10% 时，可以忽略；超过 30% 时，必须计入。本章 10 ms 对 500 ms 是 1%，500 ms 对 500 ms 是 50%。

这个比值也解释了为什么快回路对扫描周期格外敏感。流量回路的纯滞后可能只有 50 ms，扫描 100 ms 时比值到 100%，扫描滞后直接翻倍了总滞后，裕度必然崩。

88.6.2 扫描周期的上限

给定允许的附加相位损失 $\Delta\varphi_{\text{allow}}$，扫描周期的上限是

$$T_{\text{scan}} \le \frac{2\Delta\varphi_{\text{allow}}}{\omega_c} \tag{88-13}$$

取 $\Delta\varphi_{\text{allow}} = 5^\circ = 0.0873$ rad，$\omega_c = 1.0$ rad/s，得 $T_{\text{scan}} \le 175$ ms。这个上限很宽松，说明本章的慢回路对扫描周期不敏感。

同一式子用于快回路：$\omega_c = 10$ rad/s 时上限 17.5 ms；$\omega_c = 100$ rad/s 时上限 1.75 ms。后者已经逼近普通 PLC 的极限，说明 100 rad/s 以上的回路不该用 PLC 做主控。

式 (88-13) 只算了扫描的半拍滞后，没有算输入滤波与总线。完整的算法是把式 (88-2) 的 T_d 代入 $\omega_c T_d \le \Delta\varphi_{\text{allow}}$，得到

$$\frac{T_{\text{filt}}}{2} + \frac{T_{\text{scan}}}{2} + T_{\text{out}} + T_{\text{bus}} \le \frac{\Delta\varphi_{\text{allow}}}{\omega_c} \tag{88-14}$$

本章的 $T_d = 4.24$ ms，$\omega_c T_d = 0.0042$ rad $= 0.24^\circ$，远小于 5°。若把输入滤波从 4 ms 加到 40 ms，T_d 变成 22 ms，$\omega_c T_d = 1.27^\circ$，仍在预算内；加到 200 ms 才会超标。

88.6.3 增益与滞后同时失配

现场的过程参数不会与设计值一致。表 88-7 把增益按 0.7、1.0、1.3 三档、纯滞后按不变与 +50% 两档组合，给出穿越频率与相位裕度。

表 88-7 增益与纯滞后失配下的裕度（扫描 10 ms）

增益	附加滞后 / s	穿越频率 / (rad/s)	相位裕度 / (°)
0.7	0	0.700	69.7
0.7	0.25	0.700	59.7
1.0	0	1.000	61.1
1.0	0.25	1.000	46.7
1.3	0	1.300	52.4
1.3	0.25	1.300	33.8

表 88-7 有三条信息。

第一，增益下降是安全的。增益 0.7 时裕度升到 69.7°，回路更稳，只是响应变慢。现场“阀门关小一点就稳了”的经验，对应的就是这一行。

第二，增益上升比滞后增加更危险。增益 1.3 单独作用时裕度 52.4°，还在 45° 以上；滞后 +50% 单独作用时 46.7°，刚好在线上；两者叠加到 33.8°，跌破下限。

第三，穿越频率只由增益决定，与滞后无关。这符合直觉：增益决定幅值曲线穿越 0 dB 的位置，滞后只改相位。调试时可以先用增益定位穿越频率，再用滞后估算裕度。

88.6.4 三个性能指标的关系

超调、调节时间、IAE 三个指标对参数的敏感度不同，调参时要分开看。

超调主要看相位裕度。裕度从 61° 降到 47°，超调从 4.0% 涨到 32.1%，涨了 8 倍；裕度只降了 23%。这条关系是非线性的，裕度低于 45° 后超调会急剧上升。

调节时间主要看积分强度。扫描从 10 ms 到 100 ms，调节时间只涨 4%；到 200 ms 才涨 47%。原因是积分强度由 T_i 决定，扫描滞后只通过“等效滞后”间接影响。

IAE 是两者的综合。它既罚超调，也罚恢复慢，所以对掉拍这种“超调降、恢复慢”的变化最敏感：超调从 4.0% 降到 0，IAE 反而涨了 12%。评价一组参数时，IAE 比单看超调更可靠。

88.6.5 抖动、微分与噪声

扫描抖动对比例与积分项的影响是缓慢的，对微分项的影响是即时的。

微分项按 $(e(k) - e(k-1))/T$ 计算。T 抖动 20%，微分增益就抖动 25%。若测量噪声是满量程的 0.1%、微分放大倍数为 10，输出会在 1% 上下跳动；放大倍数再抖 25%，跳动幅度也跟着抖 25%，折算到阀门上是可见的抖动。

工程上有三种抑制办法。第一种是不用微分，靠积分与比例把回路调稳，代价是响应变慢。第二种是微分先行，只对测量值微分、不对设定值微分，避免设定值阶跃时的冲击。第三种是给微分项加一阶滤波，时间常数取 T 的 2 到 5 倍。

第三种办法会引入额外滞后，与扫描滞后叠加。滤波时间常数取 $5T$ 时，附加相位在 ω_c 处约 $\arctan(5\omega_c T)$；本章 $\omega_c T = 0.01$，几乎为零。快回路里这一项不能忽略。

88.6.6 多回路共用一次扫描

一个 PLC 通常跑几十个回路，它们共用同一次扫描。这带来两个后果。

第一个后果是回路之间的相位关系被锁定。所有回路在同一时刻采样、同一时刻输出，彼此的相对相位固定为 0。对独立回路没有影响，对串级回路有影响：主回路与副回路若在同一拍采样，副回路的输出要等下一拍才被主回路看到，等效多了一拍滞后。

第二个后果是扫描周期由最长的回路决定。一个慢回路（时间常数 100 s）与一个快回路（时间常数 0.5 s）放在同一个 PLC 里，扫描周期必须满足快回路的要求，慢回路被迫“过采样”。这不会损害性能，但浪费算力。

工程上的做法是按带宽分组：快回路放高速 CPU 或周期中断，慢回路放主循环。分组之后，每组的扫描周期可以独立设置，互不牵制。

88.6.7 稳定性不等于安全性

最后要区分两件事：闭环稳定与功能安全。

闭环稳定讨论的是“扰动后能否回到设定值”，属于控制性能。功能安全讨论的是“失效时能否进入安全状态”，属于风险评估。一个回路可以非常稳定，但它的安全功能可能完全不合格；反过来，安全回路通常不追求控制精度，只追求可靠动作。

这两件事在 PLC 里由不同的机制保证：稳定靠整定与扫描周期，安全靠冗余、诊断与检验周期。第 88.10 节把两者分开讲，不要用“回路很稳”来论证“安全等级够”。

88.6.8 回路数量决定扫描上限

扫描周期不只是程序条数的函数，更是回路数的函数。一个 PID 回路含限幅、抗饱和、手自动切换，约 400 条指令，在本章的 CPU 上折 40 μs。

20 个回路是 800 μs，扫描从 3.08 ms 涨到 3.88 ms；50 个回路是 2000 μs，扫描 5.08 ms。后者已经占 2 s 时间常数的 0.25%，裕度掉 1° 左右，还能接受。

100 个回路就是 4 ms，扫描 7.08 ms，占时间常数 0.35%。此时问题不再是控制品质，而是程序增长空间：再加两个工位就顶到看门狗。

所以选 CPU 时按“回路数乘以 40 μs”估算程序块，比按 I/O 点数估更准。点数只影响输入输出刷新的固定块，回路数才影响那 58% 的可变块。

88.6.9 串级回路的一拍代价

串级回路的副回路跟随主回路的输出。两者在同一拍采样时，主回路本拍算出的副回路设定值，要到下一拍才被副回路使用，等效在副回路里多了一拍纯滞后。

一拍对副回路的影响取决于副回路的速度。副回路时间常数 0.2 s、扫描 10 ms 时，一拍是时间常数的 5%，裕度掉 2° 到 3°；副回路时间常数 2 s 时，一拍只占 0.5%，可以忽略。

工程上的对策是把主副回路分到不同节拍：副回路进 10 ms 周期中断，主回路留主循环。这样副回路不必等主回路，代价是数据交换要多一层缓冲。

串级整定的经验比例是副回路比主回路快 3 到 5 倍。若两者速度接近，串级没有意义，不如单回路。扫描周期会压缩这个比例：主回路 1 s、副回路 0.2 s，本来正好 5 倍，扫描 100 ms 时副回路的有效速度降到 0.3 s，比例变成 3.3 倍，串级

的抗扰优势明显减弱。

88.6.10 串级与分程的整定顺序

串级回路先整副回路，再整主回路。副回路切手动、主回路切自动，把副回路整成快速无超调；然后副回路投自动、主回路切手动，整主回路。

顺序反过来会失败。先整主回路时，副回路的滞后还没被压住，主回路看到的等效对象比实际更慢，整出的增益偏小，投串级后响应迟钝。

分程回路先定分程点。两个阀在分程点附近交接，交接区间留 2% 到 5% 的重叠或死区。重叠会让两个阀同时动作、互相抢流量；死区会让交接点附近出现不灵敏带。

分程点要按阀门容量定，不按 50% 平分。一个 100 m³/h 的大阀与一个 20 m³/h 的小阀，分程点在 17% 附近才线性。按 50% 分会造成前半段增益很小、后半段增益很大，同一组 PID 参数两头都不合适。

88.6.11 性能验收的工况覆盖

整定完的参数要在四种工况下验收：满负荷、最小负荷、设定值阶跃、负荷扰动。四种工况的响应差别可能很大，只测一种不算验收。

满负荷与最小负荷的差别在过程增益。换热器在满负荷时增益高、最小负荷时增益低，同一组 K_c 在两端分别偏硬与偏软。若两端都要求超调小于 10%，参数必须按增益最高的一端整定。

设定值阶跃测的是跟踪能力，负荷扰动测的是抗扰能力。两者对参数的要求常常相反：跟踪要求快，抗扰要求稳。工程上取折中，先满足抗扰，再看跟踪是否可接受。

四种工况的数据都要留档，并注明当时的扫描周期。同一组参数在不同扫描下的表现不同，没有扫描记录的验收数据无法复现。

88.6.12 稳定裕度的现场估算

现场没有频响分析仪，也要知道裕度够不够。最实用的办法是临界增益法：把积分与微分关掉，只留比例，慢慢加增益直到输出等幅振荡。

此时的增益是临界增益 K_u，振荡周期是 T_u。临界增益对应的相位裕度为 0，所以工作增益取 K_u 的 50% 到 70%，对应裕度 30° 到 45°。

本章的 $K_c = 2.0$，按表 88-7 的趋势外推，临界增益约 3.0，工作点是临界的 67%，对应 61° 裕度。比 50% 法则给出的值略高，因为本章过程没有谐振峰，容许更靠近临界。

做这个试验有三条安全要求：负荷稳定、输出限幅、有人盯着。等幅振荡在过程上就是持续波动，时间长了会损坏设备或影响产品质量。

试验数据要记下来：K_u、T_u、当时的扫描周期。三次试验的 K_u 若相差超过 20%，说明过程非线性明显，单组参数无法覆盖全工况。

88.6.13 滤波与扫描的联合选择

滤波与扫描是一对可以互换的东西：加长滤波能压噪声，但吃相位；缩短扫描能补相位，但耗算力。两者要用同一个预算衡量。

总滞后里，滤波贡献 $T_{\text{filt}}/2$，扫描贡献 $T_{\text{scan}}/2$。本章是 2.0 ms 与 1.54 ms，量级相当。若把扫描从 3.08 ms 压到 1.5 ms，省下的 0.79 ms 相位，正好够把滤波从 4 ms 加到 5.5 ms。

选择的原则是：滤波先满足抗混叠，扫描再满足程序规模。抗混叠要求滤波截止频率低于采样率的一半，本章是 40 Hz 对 162 Hz，余量足够。

工程上有一个常用比例：滤波时间常数取扫描的 2 到 5 倍。低于 2 倍，噪声压不住；高于 5 倍，滤波本身成了主要滞后。

这条比例在联锁回路上要反过来。联锁要求快，滤波取扫描的 1 到 2 倍，靠硬件 RC 与软件滤波各承担一半。

88.7 参数意义与作用

88.7.1 参数总表

PLC 项目里可调的参数不多，但每一个都牵动几个指标。表 88-8 把七个主要参数与它们的影响、调整代价放在一起。

表 88-8 七个主要参数的影响与代价

参数	本章取值	主要影响	调整代价
扫描周期	3.08 ms	相位裕度、掉拍容忍	改程序或换 CPU
看门狗时间	10 ms	可用性、故障复位	需重新验证
输入滤波	4 ms	噪声抑制、附加滞后	改组态即可
PID 采样周期	10 ms	积分与微分精度	改中断配置
总线周期	1 ms	远程 I/O 滞后	增加网络负载
共因因子	1%	冗余的实际收益	布线、电源、固件分离
检验周期	1 年	PFDavg 与 SIL	停机检验成本

表 88-8 的排序不是按重要性，而是按“改动频率”。现场改得最多的是输入滤波与 PID 采样周期，改得最少的是共因因子与检验周期——后两个一旦定下来，通常伴随整个安全回路的硬件设计。

88.7.2 扫描周期

扫描周期是第一参数，因为它同时影响控制性能与程序容量。

对性能的影响通过式 (88-13) 体现：扫描周期越长，附加相位越大，裕度越低。本章从 10 ms 到 500 ms，裕度掉 14.1°，超调涨 8 倍。

对容量的影响通过式 (88-1) 体现：程序越长，扫描越慢。这两条约束互相拉扯：想加功能就得加程序，加程序就变慢，变慢就吃裕度。

工程上的处理办法是“预留 30% 扫描余量”。设计时按 70% 的扫描时间规划程序，剩下 30% 留给后期功能追加与通信波动。本章的 3.08 ms 对 10 ms 看门狗是 30.8%，余量充足；若项目后期加到 4.3 ms，就逼近 50% 的警戒线。

88.7.3 看门狗时间

看门狗是“程序跑飞”的保险。它由独立计数器实现，程序每拍喂一次；若计数器溢出，CPU 复位，输出进入安全状态。

看门狗时间的下限是最大扫描周期，上限是工艺允许的失控时间。本章取 10 ms，约为最大扫描 4.2 ms 的 2.4 倍，也远小于工艺允许的 100 ms。取 10 ms 而不是 8 ms，是为了留出取整与后期功能的余地。

看门狗时间设得太紧，程序升级后一运行就复位，现场表现为“偶发停机”，很难查。设得太松，真死机时要等很久才动作，安全功能失效。经验值是最大扫描的 1.5 到 2 倍，且不超过工艺允许失控时间的三分之一。

看门狗复位后，输出必须进入安全状态，而不是保持复位前的值。这一点要在硬件上保证：复位时继电器失电、阀门失气、电机断使能。软件层面的“复位后恢复上次状态”是危险设计。

88.7.4 输入滤波时间

输入滤波抑制噪声，代价是滞后。4 ms 滑动平均对应 250 Hz 带宽，对温度、压力、液位足够；对流量、张力这类快变量要降到 1 ms。

滤波时间进入式 (88-2) 的第一项，取半后计入总滞后。本章取 4 ms，贡献 2.0 ms，占总滞后 4.24 ms 的 47%。若把滤波加到 40 ms，总滞后变成 22 ms，占比升到 91%，扫描周期反而成了次要项。

滤波形式也影响相位。滑动平均的相位滞后是 $(N-1)T_s/2$，一阶低通的相位滞后是 $\arctan(\omega\tau)$。同样的截止频率，一阶低通的相位滞后更小，但抑制高频噪声的能力更弱。工程上模拟量常用一阶低通，开关量常用去抖延时。

88.7.5 PID 采样周期

PID 采样周期与扫描周期是两个参数，虽然很多人把它们当成一个。

采样周期决定积分与微分的精度。积分项每个周期累加一次，周期越长，等效的积分步长越大，小信号时的积分分辨率越差。微分项每个周期算一次差分，周期越长，对快速变化的捕捉越粗。

本章取 10 ms，是过程时间常数 2 s 的 0.5%，满足“采样周期小于时间常数 1/10”的经验规则。快回路取 1 ms，慢回路取 100 ms，都按这个比例。

采样周期还必须与扫描周期对齐。若采样周期 10 ms、扫描周期 3.08 ms，两者不同步，每次采样落在扫描的不同位置，等效采样间隔在 10 ms 上下波动。这种波动对积分项是噪声，对微分项是灾难。正确做法是用周期中断触发采样，让采样时刻与扫描解耦。

88.7.6 总线周期

远程 I/O 的刷新由总线周期决定，与 PLC 扫描周期无关。总线周期 1 ms 时，远程站的输入平均滞后 0.5 ms；周期 10 ms 时，滞后 5 ms。

总线周期还影响同步精度。分布式时钟（如 EtherCAT 的 DC）能把各从站的采样时刻对齐到微秒级，普通轮询只能对齐到毫秒级。前者用于多轴同步，后者用于一般过程控制。

本章取 1 ms，对应 0.5 ms 的平均滞后。若站点多、数据量大，总线周期会被迫拉长，滞后随之增加。设计时要按最坏情况算：站点数 × 每站数据量 ÷ 带宽，再留 50% 余量。

88.7.7 共因因子

共因因子 β 是安全回路里最容易被忽略、又最能决定结果的参数。

它描述“两套通道因同一原因同时失效”的概率比例。同一电源、同一电缆槽、同一版固件、同一套维护规程，都会推高 β。工业现场的典型值是 1% 到 10%，做了物理分离可以降到 1% 以下，完全不做分离可能到 10% 以上。

表 88-9 给出 β 从 1% 到 10% 时 1oo2 结构的 PFDavg。检验周期 1 年时，$\beta=1\%$ 是 2.82×10^{-5}（SIL 4），$\beta=10\%$ 是 2.24×10^{-4}（SIL 3），差了一个数量级。检验周期拉到 5 年，后者升到 1.22×10^{-3}，掉回 SIL 2。

表 88-9 三种架构在不同检验周期下的 PFDavg

架构	检验周期 1 年	3 年	5 年
1oo1	2.19×10^{-3}	6.57×10^{-3}	1.09×10^{-2}
1oo2，$\beta=1\%$	2.82×10^{-5}	1.22×10^{-4}	2.66×10^{-4}
1oo2，$\beta=5\%$	1.15×10^{-4}	3.80×10^{-4}	6.92×10^{-4}
1oo2，$\beta=10\%$	2.24×10^{-4}	7.04×10^{-4}	1.22×10^{-3}
2oo3，$\beta=1\%$	4.07×10^{-5}	2.35×10^{-4}	5.80×10^{-4}

所以检验周期不只是维护安排，它直接改变算出来的安全等级：把 5 年检验当成“省钱”，代价往往是整条回路掉一个 SIL。

88.7.8 检验周期

检验周期 T_1 是“多久做一次完整的功能测试”。它对 PFDavg 的影响是线性的：周期翻倍，PFDavg 翻倍。

线性关系带来一个直觉：想降一个数量级，要么把检验周期缩到十分之一，要么把架构升一级。前者意味着从年检变成月检，停机成本极高；后者意味着加一套硬件。工程上通常先算这两笔账，再决定走哪条路。

检验周期还受“未检出失效”的暴露时间限制。一个失效从发生到被检验发现，最长要等一个 T_1。若工艺要求失效暴露时间不超过 1 个月，T_1 就不能超过 1 个月，哪怕 PFDavg 已经足够。

88.7.9 参数之间的耦合

这七个参数不是独立的，改动一个会牵动另一个。

扫描周期变长，PID 采样周期若不变，两者更容易不同步；扫描周期变长，看门狗时间要跟着放宽；输入滤波变长，总滞后增加，扫描周期要相应缩短来补偿。

调整的顺序应该是：先定工艺节拍与安全等级，再定扫描周期与看门狗，最后定滤波与 PID 采样。反过来调，常常是改了一个参数，另外两个跟着超标。

本章的一组参数（3.08 ms 扫描、10 ms 看门狗、4 ms 滤波、10 ms 采样、1 ms 总线、$\beta=1\%$、$T_1=1$ 年）是自洽的：扫描占看门狗 30.8%，总滞后占时间常数 0.21%，PFDavg 落在 SIL 4。改动其中任何一个，都要重新核对其余六个。

88.7.10 现场辨识过程参数

定扫描周期之前先要知道过程的 K、τ、L。这三个数不靠猜，靠一次阶跃测试。

测试步骤是：把回路切手动，等工况稳定，把输出阶跃 5% 到 10%，记录被控量直到它走完 3 个时间常数。阶跃太小会被噪声淹没，太大会离开工作点、非线性显现。

从记录曲线上取两个点：到达稳态变化量 28.3% 的时刻 t_1，到达 63.2% 的时刻 t_2。一阶加纯滞后模型的参数由这两点给出：$\tau=1.5(t_2-t_1)$，$L=t_2-\tau$，K 等于稳态变化量除以阶跃幅值。

用本章的数验证：$\tau=2$ s、$L=0.5$ s 时，$t_2=2.5$ s，$t_1=0.5+0.667=1.17$ s，两点差 1.33 s，乘 1.5 得 2.0 s，回到 τ。方法自洽。

同一回路测三次取中位数。离散度超过 30% 说明工况不稳，数据不可用，要等下一个稳定时段重测。

88.7.11 参数变更的影响面

改一个参数之前，先想清它会牵动什么。K_c 影响裕度、超调、抗扰速度；T_i 影响调节时间、静差、积分饱和；扫描周期影响全部三项，还影响看门狗与在线修改窗口。

影响面最大的是扫描周期，因为它同时改变采样滞后与积分步长。这也是为什么改程序要先评估扫描：一次看似无关的功能增加，可能让本章的 61° 裕度掉到 50° 以下。

每次变更记录四项：改了哪个参数、原值、新值、变更后的阶跃结果。四项缺一项，下次整定就要从头试。

变更后必须重测，不能只看“运行正常”。至少做一次设定值阶跃与一次负荷扰动，IAE 不超过变更前的 110% 才算可接受。

88.7.12 一次完整的参数选定

把前面几节串起来，看一组参数是怎么定下来的。过程仍是本章的一阶加纯滞后，$\tau = 2$ s、$L = 0.5$ s。

第一步辨识。阶跃 8%，稳态变化 8%，得 $K = 1$；两点法得 $\tau = 2.0$ s、$L = 0.5$ s。

第二步定扫描。取 $\tau/20 = 100$ ms 作为上限，考虑联锁响应与程序规模，实际选 3 ms 到 10 ms 之间。本章取 3.08 ms，是程序规模决定的，不是刻意选的。

第三步定滤波。滤波时间常数取扫描的 2 到 5 倍，即 6 到 15 ms。本章取 4 ms，偏快，因为它要保证联锁响应；若只做调节，取 10 ms 更好。

第四步定看门狗。按最大扫描的 2 倍，本章最大 4.2 ms，取 10 ms。再看工艺允许失控时间 100 ms，10 ms 远小于它，约束满足。

第五步定 PID。IMC 整定，λ 取 L，得 $K_c = 2.0$、$T_i = 2.0$ s。验算裕度 61°，超调 4.0%，两项都合格。

第六步复核耦合。扫描 3.08 ms 占看门狗 30.8%，总滞后 4.24 ms 占时间常数 0.21%，PID 采样与扫描同步。六项参数互不冲突，参数组可用。

这六步的顺序不能换。先定 PID 再定扫描，等于让参数去迁就一个还没确定的节拍，最后一定要返工。

88.8 调参方法与经验

88.8.1 五步顺序

PLC 回路的整定与专用控制器没有本质区别，但顺序更重要，因为每一步都可能改扫描周期。表 88-10 给出本章采用的五步。

表 88-10 PLC 回路的五步整定顺序

步骤	动作	通过判据
1	固定扫描周期与 PID 采样周期	周期波动 < 5%
2	手动模式确认阀门特性	开度—流量单调，无死区跳变
3	纯比例，逐步加大增益	出现等幅振荡，记录 K_u 与 T_u
4	加积分，缩短 T_i	静差消失，超调不超过 10%
5	按需加微分，验收 IAE	超调、调节时间、IAE 三项达标

第 1 步最容易被跳过，也最不该跳过。扫描周期在整定过程中如果变化，前面测到的临界增益就作废。本章的做法是把 PID 放进 10 ms 周期中断，主循环的程序改动不影响采样周期。

88.8.2 先静态后动态

整定之前要先确认阀门特性。手动给 0%、25%、50%、75%、100% 五个开度，记录对应的流量。正常的阀门应该是单调的、接近线性的；若出现“给了 10% 没反应、给到 15% 突然跳到 30%”，说明阀门有死区或摩擦，任何整定参数都救不了。

阀门特性还会影响增益。等百分比阀在小开度时增益小、大开度时增益大，同一个 K_c 在小流量时偏软、大流量时偏硬。工程上要么做流量特性补偿，要么按最不利工况整定。

静态确认还包括传感器量程与零点。量程设错会让 PID 看到的误差放大或缩小，整定出来的参数看似合理，实际对应的是错误的增益。这类错误在现场并不少见，且很难从响应曲线上看出来。

88.8.3 从比例开始

关掉积分与微分，从小到大加比例增益，直到出现等幅振荡。记录此时的增益 K_u 与振荡周期 T_u，这就是临界点。

本章的回路在 $K_c = 2.0$ 时裕度 61°，离临界还远。继续加大增益，裕度会逐步下降到 0；按表 88-7 的趋势外推，增益到 3.0 左右时裕度接近零。这个估算只用于判断方向，真实临界值要在现场测。

得到 K_u 与 T_u 后，用经验公式给出初值。常用的有齐格勒—尼科尔斯法：$K_c = 0.45K_u$、$T_i = 0.83T_u$（PI）。这组初值通常偏激进，超调偏大，需要回调。

比例增益的最终值取临界增益的 50% 到 70%。本章的 $K_c = 2.0$ 约为临界的 67%，对应 61° 裕度，属于适中偏保守的一档；若要更快响应，可加到 2.4，但裕度会降到 30° 以下。

88.8.4 加积分去静差

比例控制必然留静差。静差的大小与增益成反比：增益 2.0 时，单位阶跃的静差约 1/(1+2)=33%；增益 10 时降到 9%。靠加大增益消静差不可行，因为增益受稳定边界限制。

积分的作用是消除静差，代价是相位滞后。T_i 越小，积分越强，滞后越大。整定时从大到小缩短 T_i，直到静差在要求时间内消失，同时超调不超过 10%。

本章取 $T_i = 2.0$ s，等于过程时间常数。这个选择让积分零点与过程极点对消，闭环响应最平滑。若 T_i 取到 0.5 s，积分变强，超调会明显上升；取到 8 s，静差消除要等 20 s 以上。

积分时间与扫描周期有一个隐含约束：T_i 至少是采样周期的 10 倍。本章 $T_i/T = 200$，远大于 10；若 T_i 到 0.1 s、采样 10 ms，比值只有 10，积分项的离散化误差开始显现。

88.8.5 加微分的边界

微分抑制超调、加快响应，但对噪声敏感。过程控制里多数回路不用微分，因为测量噪声大、微分收益小；温度回路偶尔用，流量回路基本不用。

用微分的前提是三条：测量信号干净（噪声小于满量程 0.5%）、执行器动作快（阀门全行程小于 1 s）、过程有惯性（时间常数大于 10 s）。三条缺一条，微分的收益就抵不过噪声放大。

若要用，T_d 取 T_i 的 1/8 到 1/4，并给微分项加一阶滤波，时间常数取采样周期的 2 到 5 倍。本章的回路没有用微分，因为它是一阶惯性加纯滞后，超调本来就小。

88.8.6 用 IAE 验收

整定完成后，用一组阶跃测试验收。本章用的指标是超调、调节时间与 IAE，三项都要看。

只看超调会漏掉“响应慢”的问题。掉拍 20% 时超调从 4.0% 降到 0，看起来更好，实际调节时间从 2.96 s 涨到 5.27 s，IAE 涨了 12%。只看超调会给出相反的结论。

只看调节时间会漏掉“振荡”的问题。增益偏大时调节时间可能不变，但超调从 10% 涨到 40%，阀门动作幅度大、磨损快。IAE 对这两类问题都敏感，所以适合做统一验收指标。

验收要在最不利工况下做：最大负荷、最小负荷、设定值阶跃、负荷扰动各一次。四组数据都达标，参数才算可用。

88.8.7 常见症状与处置

现场调试时，症状比参数更容易观察到。表 88-11 把六种常见症状与对应的处置列出来。

表 88-11 六种常见症状与处置

症状	可能原因	处置
超调大、衰减慢	增益偏大或积分偏强	降 K_c 或加长 T_i
响应慢、无超调	增益偏小或积分偏弱	加 K_c，缩短 T_i
等幅振荡	增益接近临界	降 K_c 到临界值的 50%
周期很长的摆动	积分饱和或阀门粘滞	加抗饱和，检查阀门
小信号不动作	阀门死区或分辨率不足	做死区补偿，换阀门
随机抖动	噪声经微分放大	加滤波或关微分

表 88-11 里最容易被误判的是“周期很长的摆动”。它看起来像积分太强，实际常是积分饱和：输出已经顶到限幅，误差还在累加，等误差反号时要很久才退出饱和。处置办法不是改 T_i，而是加抗饱和。

“小信号不动作”也常被误判为增益太小。真实原因可能是阀门死区、定位器分辨率不足、或输出模块的最小脉宽。加大增益会让大信号时超调，问题反而更糟。

88.8.8 手动到自动的无扰切换

手动运行时积分器不工作，切换到自动的瞬间，若积分器还停在旧值，输出会跳变。跳变量等于积分项与手动输出之差，常常是几十个百分点。

无扰切换的做法是让积分器在手动时“跟踪”实际输出。每拍反算一次：

$$I(k) = T_i \left(\frac{u_{\text{man}}}{K_c} - e(k) \right) \tag{88-15}$$

代入一组数：手动输出 45%、误差 2%、$K_c = 2$、$T_i = 2$ s，得到 $I = 2 \times (0.225 - 0.02) = 0.41$。切自动后本拍输出是 $2 \times (0.02 + 0.41/2) = 0.45$，与手动值相等，阀门不动。

不做跟踪的后果可以直接算：积分器为 0 时，切自动的输出是 $2 \times 0.02 = 0.04$，阀门从 45% 跳到 4%，工艺上就是一次冲击。

同样的反算也用于抗饱和。输出顶到限幅时，把积分器反算到“刚好使输出等于限幅”的值，误差继续累加也不会积起来。这是 §88.8.7 里“周期很长的摆动”的根治办法。

跟踪与抗饱和共用一条反算式，这是位置式 PID 的一个便利。增量式没有积分器可跟踪，切换时要把上一拍输出存下来做基准，写法不同，效果一样。

88.8.9 报警与整定的边界

整定期间，报警会频繁动作。要提前区分两类：工艺报警与偏差报警。前者是真实工况，不能改；后者是整定过程中的暂态，可以临时放宽。

放宽必须有时限和记录。临时把偏差报警从 2% 改到 8%，整定完立刻改回，并在交接班记录上写清。忘了改回的项目，三个月后会有人问“为什么这个报警从来不响”。

报警不能替代联锁。一个超温报警，操作员可能没看见；一个超温联锁，设备自己会停。要求“必须停”的场合，只能做联锁。

整定结束后对比报警次数。同一工况下，整定前每班 30 条偏差报警、整定后 3 条，说明参数确实改善了。次数没变而超调变小，说明报警限值设得太松，问题被掩盖了。

首出报警要单独记录。一次停机往往同时触发十几条报警，只有第一条指向根因。程序里要给每条报警打时间戳，按毫秒排序，取最早的一条。

88.9 优化与改进

88.9.1 先分清哪些账能动

3.08 ms 的扫描里，1.28 ms 是固定开销，1.80 ms 是用户程序。固定开销又分成四块：输入刷新 250 µs、输出刷新 180 µs、通信服务 600 µs、自检 250 µs。这四块里只有通信是真正可谈的，输入输出的时间由模块数量与背板带宽决定，自检的时间是安全等级要求的，删掉就等于降级。

程序那块也不是越紧越好。同样功能写得紧凑能省两成，代价是没人看得懂、下次改程序的人要多花三天。省下 360 µs 换来一份不可维护的程序，这笔账在任何项目里都不划算。

所以优化的次序是：先砍通信，再谈结构，最后才考虑换 CPU。三条路的收益差一个数量级。通信分帧能省 500 µs，只需改一个参数；换大型 CPU 能省 1.58 ms，但要换硬件、重接线、重做验证、停机。前者一个下午，后者一个季度。

88.9.2 通信分帧：最便宜的一刀

通信服务那 600 µs 是一整块：读一次远程 I/O、发一包上位机数据、响应一次编程口请求。这三件事都不需要每拍做完。把 600 µs 切成 n 帧，每拍只做一帧，单拍开销降到 $600/n$：

$$T'_{\text{scan}} = T_{\text{scan}} - T_{\text{comm}}\left(1 - \frac{1}{n}\right) \tag{88-16}$$

表 88-12 列出五档。$n=6$ 时扫描从 3080 µs 降到 2580 µs，省 16.2%；再往下加帧，$n=12$ 只多省 50 µs，$n=30$ 也只到 2500 µs。收益在 $n=6$ 附近就基本吃完，继续加帧只会让数据刷新变得更慢。

表 88-12 通信分帧对扫描周期与数据刷新周期的影响

分帧数 n	单拍通信 / us	扫描周期 / us	数据刷新周期 / ms
1	600	3080	3.08
3	200	2680	8.04
6	100	2580	15.5
12	50	2530	30.4
30	20	2500	75.0

代价是数据刷新变慢。$n=6$ 时上位机看到的温度从 3 ms 一次变成 15.5 ms 一次。对趋势记录、报表、HMI 显示都够用；对需要毫秒级同步的联锁不够用。

分帧的前提是先把数据分成“必须每拍”与“可以慢”两类。必须每拍的只有两种：参与本拍联锁判断的远程输入，以及本拍要下发的输出。其余都可以慢。

这条边界划不清楚，就会把急停信号排进第 6 帧。最坏情况下它要等 15 ms 才被读到，加上扫描与输出刷新，整条响应链会超过 20 ms，安全功能的时间指标直接失守。急停这类信号本来就不该走普通通信，它属于硬接线回路。

88.9.3 大块数据移出扫描

趋势记录、配方下载、报警归档有一个共同点：数据量大、实时性低。这类操作放在扫描里做拷贝，耗时与字节数成正比。600 字节的块拷贝在中型 CPU 上约 180 µs，占扫描的 5.8%。

单次看不出问题，批量就出事。一次下载 20 个配方就是 12 000 字节、3.6 ms，扫描当场翻倍，看门狗报警。现场看到的症状是“下载配方时全线停”，原因往往就在这里。

办法是交给 DMA 或独立的通信协处理器。CPU 只写一条“启动传输”的指令，约 10 µs，数据在后台搬完再中断通知。单拍开销从 180 µs 降到 10 µs，省下 170 µs，而且不随数据量增长。

判断一段代码该不该移出扫描，看两个数：它的执行时间是否超过扫描的 5%，以及它是否每拍都必须执行。前者超了、后者又是否，就该移。两条都满足才留。

88.9.4 在线修改的峰值约束

PLC 允许运行中改程序，这是它相对嵌入式控制器的最大优势，也是最容易出事的地方。修改不是“瞬间替换”：下载过程要停扫描、拷贝新块、重建交叉引用，每一段都额外占用时间。

把一次修改拆成 m 段、每段耗时 Δ，最不利的一拍是段与段之间恰好赶上正常扫描：

$$T_{\text{peak}} = T_{\text{scan}} + m\Delta < T_{\text{wd}} \tag{88-17}$$

代入本章的数：$T_{\text{scan}} = 3.08$ ms，$T_{\text{wd}} = 10$ ms，留给修改的预算只有 6.9 ms。若单段 1.2 ms，最多 5 段，也就是一次只能改 5 个块。第 6 段会把峰值顶到 10.28 ms，看门狗动作。

这条约束在现场经常被忽略。工程师一次改十几个块，下载时看门狗动作，CPU 进停止态，全线停机。事后查日志只看到“看门狗超时”，看不到“改了 14 个块”，于是归因于“CPU 不稳定”。

还有一条隐含约束：修改期间输出保持最后一次的值。对开关量，这意味着气缸停在原位；对模拟量，阀门停在原开度。修改前要确认工艺允许停这么久，尤其是有物料在炉内、有压力在管内的时候。

88.9.5 快慢分离

一个项目里往往既有 10 ms 的快回路（张力、位置），又有 1 s 的慢回路（温度、液位）。让所有回路共用一次扫描，快回路被拖慢，慢回路浪费算力，两头都不合适。

做法是把快回路放进固定周期的中断组织块，慢回路留在主循环。两者通过数据块交换数据，不共享扫描节拍。这样主循环可以是 20 ms，快回路仍是 1 ms，互不牵制。

代价是数据一致性变差。快回路读到的设定值可能来自上一拍的主循环，滞后一拍。若设定值本身是慢变量，这个滞后无所谓；若设定值由主循环里的另一个快回路算出，就要加锁或双缓冲，否则会读到半更新的一半新一半旧的值。

中断组织块的周期也不能随便设。设成 1 ms 而实际执行 1.2 ms，中断会自己叠自己，最终触发周期监视错误。这类故障的表现是“运行几小时后突然停机”，且只在负荷高时出现，最难查。

88.9.6 换 CPU 的边界

表 88-2 已经说明：程序从 1440 µs 压到 216 µs，扫描只从 2.72 ms 压到 1.50 ms。固定开销 1.28 ms 是地板，任何 CPU 都越不过去。

所以换 CPU 只在一种情况下有意义：程序执行块已经占到扫描的一半以上，且通信与自检都砍不动了。此时换大型 CPU 能把 1.80 ms 压到 0.22 ms，扫描减半。若程序只占三成，换 CPU 最多省 20%，不如改程序。

换算一下经济性。一台大型 CPU 的价格约为小型的两到三倍，加上重新接线、重新验证、停机损失，总成本往往是硬件差价的五倍以上。若只为省 1 ms 而扫描本来只有 3 ms，这笔账算不过来。

反过来说，如果程序已经写到 8000 条指令、扫描 4 ms、还要再加两个工位，那么换 CPU 是唯一出路。判断标准不是“想快一点”，而是“程序增长已经顶到看门狗”。

88.9.7 抖动的来源与压制

扫描的平均值好看，不代表每一拍都好看。抖动主要来自四处：编程软件在线监视（扫描涨 10% 到 15%）、HMI 轮询过密、诊断缓冲区写入、以及操作员在触摸屏上翻页。

前三项都能压。在线监视改成按需连接，改完就断开；HMI 轮询改成事件上报，值不变就不发；诊断缓冲改到低优先级任务里写。第四项压不住，但可以测量——把“有人在操作 HMI”列为最不利工况之一。

压不下去的部分要靠设计吸收。扫描设计点取实测最大值的 1.5 倍，而不是平均值；积分项的累加用实际经过时间而不是标称周期，掉拍时自动补偿；看门狗按最大扫描的 2 倍设，留出修改余量。

第三点最容易被忽略，也最有效。若控制器支持“实际时间积分”，那么掉拍 20% 只影响采样滞后，不再影响积分强度，表 88-5 里那条 5.27 s 的曲线就不会出现。买 PLC 时值得确认这一项。

88.9.8 不该做的优化

有三件事看起来像优化，实际是降级。第一件是把 PID 的采样周期设得比扫描还短，例如扫描 3 ms 而 PID 周期设 1 ms。程序根本跑不到那么快，设了也是按 3 ms 执行，唯一的后果是积分项按 1 ms 累加，实际积分时间变成设定值的三分之一，回路变硬、振荡。

第二件是把输入滤波时间设为零以求“响应快”。滤波去掉的是噪声，噪声进了微分项会被放大十倍，进了比较逻辑会引发频繁跳变。一个 0.1% 的噪声在开关量上就是几十次误动作。

第三件是关掉自检换扫描时间。自检 250 µs 看着白花，它检查的是内存、看门狗、安全输出的回路。关掉之后扫描只快 8%，安全等级却从 SIL 3 掉到无认证状态。这笔交换在任何评估里都不成立。

88.9.9 优化后要复测最坏值

优化完必须复测，而且要测最坏值，不是平均值。扫描统计至少看四个数：平均、最大、95 分位、超时次数。

平均 3 ms、最大 8 ms 的系统，比平均 4 ms、最大 4.5 ms 的系统危险得多。看门狗按最大值设，联锁按最坏响应算，平均值只对报表有用。

复测要把最不利的工况都跑一遍：通信最忙（上位机全量刷新）、程序最重（所有中断同时触发）、诊断最频繁（有人在线监视）。三种情况同时出现的那一拍，才是设计点。

88.9.10 本章的账面

五项优化合起来：通信分帧省 500 µs（$n = 6$）、大块拷贝移出省 170 µs、抖动压制让最大值从 4.88 ms 收到 4.2 ms 附近。扫描从 3080 µs 降到 2410 µs，即 2.41 ms。

这 670 µs 对控制品质几乎没有影响。按图 88-4 的曲线，扫描从 3.08 ms 变到 2.41 ms，相位裕度从 61.1° 变到 61.2°，超调从 4.0% 变到 4.0%，肉眼看不出来。

优化不是为了控制更好，而是为了三件事：给程序增长留余量——再加 5000 条指令（500 µs）仍在 3 ms 以内；把改程序时的峰值压在看门狗以下；把通信抖动从扫描里剥离出去。

三件事都比“扫描更快”本身更值钱。扫描快是手段，可预测才是目的。这条判断也解释了为什么本章反复强调最坏值：控制系统不怕慢，怕的是忽快忽慢。

88.9.11 优化与可读性的取舍

扫描优化的最后一条不是技术问题，是管理问题：省下来的微秒与失去的可读性，哪个更值钱。

一条常见的“优化”是把三个梯级合并成一个，用中间变量复用结果。指令从 30 条降到 22 条，省 0.8 μs，但下一个维护的人要花半小时才看懂这段在做什么。

另一条是用间接寻址批量处理同类设备。十台泵的启停写成循环，程序从 300 条降到 80 条，扫描省 22 μs。代价是单台泵的故障无法在梯形图里直接定位，要在循环里加调试计数器。

合理的界限是：只有当某段程序占扫描的 5% 以上，且结构清晰不受影响时，才做指令级优化。低于这个比例，优化省下的时间还不够一次排故多花的时间。

真正的优化在结构层：把慢任务挪出扫描、把重复计算提到扫描外、把数据块访问改成一次性读取。这些做法省的是几百微秒，同时让程序更好读，不存在取舍。

88.9.12 优化前后的对照

把 §88.9 的几项优化合成一张表，看得失更清楚。表 88-13 是优化前后的对照。

表 88-13 扫描优化前后的对照

项目	优化前	优化后	变化
通信服务 / us	600	100	−500
大块拷贝 / us	180	10	−170
其余（含程序 1800） / us	2300	2300	0
平均扫描 / us	3080	2410	−21.8%
最大扫描 / ms	4.88	4.20	−13.9%
占看门狗 / %	48.8	42.0	−6.8
数据刷新周期 / ms	3.08	15.5	+403%

表中最后一行是代价。数据刷新慢了四倍，对趋势与画面没有影响，对需要快速响应的联锁有影响——这也是为什么 §88.9.2 强调先把数据分成“必须每拍”与“可以慢”两类。

平均扫描降 21.8%，最大扫描只降 13.9%。差距来自抖动：抖动里有一部分是通信高峰造成的，分帧把它削平了；另一部分来自诊断写入，与通信无关，压不掉。

占看门狗从 48.8% 降到 42.0%，看起来只是数字变化，实际意义是给在线修改留出了 5.8 ms 的窗口。§88.9.4 的 5 段约束，就是按这个窗口算的。

88.10 工程落地

88.10.1 从图纸到投运的五个阶段

一个 PLC 项目从图纸到投运，走的是设计、组态、离线测试、现场调试、验收五个阶段。前两个阶段在办公室完成，第三个阶段在试验台上完成，后两个在现场。真正决定项目成败的是第三个阶段，因为它最容易被压缩掉。

设计阶段产出三样东西：I/O 清单、控制流程图、联锁表。I/O 清单要标明点号、信号类型、量程、所属模块；联锁表要写成“条件—动作—延时”的三元组，不能只画个方块写“联锁”。

组态阶段把图纸翻译成硬件配置与程序框架。这一步的关键不是写逻辑，而是把 §88.3 的分层结构搭起来：哪些是功能块、哪些是功能、哪些数据进数据块。框架搭错，后面每一行代码都在还债。

离线测试在试验台上做，用信号发生器代替传感器、用指示灯代替执行器。这个阶段能发现八成的逻辑错误，成本却只有现场调试的十分之一。压缩这一步的项目，最后都会在现场用停机时间补回来。

现场调试与验收是最后两步，前者调参数，后者出记录。两者的分界线是“功能全部实现”，不是“运行基本正常”。这条线划不清，验收会变成无休止的返工。

88.10.2 上电与自检

PLC 上电后不是直接进运行态，中间有一段自检。典型顺序是：电源电压检查、内存校验、模块识别、看门狗启动、I/O 回路测试、最后才允许切到运行。

内存校验用累加和或 CRC，覆盖程序区与数据区。校验失败时 CPU 拒绝启动，面板上亮红灯。这一步挡住的是运输振动、电磁干扰、电池失效导致的程序损坏。

模块识别检查实际插的模块与组态是否一致。换了模块没改组态、模块插错槽位，都会在这一步报出来。这条检查在改造项目里特别有用，因为改造时最容易出现“硬件换了、组态没跟上”。

I/O 回路测试只测输出模块的短路与开路，不测现场接线。现场接线要在调试阶段逐点核对，也就是俗称的“点对点”。自检能保证的是控制器自己没坏，不能保证线接对了。

从上电到进入运行态通常要 3 到 8 s。这段时间输出全部处于安全态，即失电状态。设计时要确认工艺允许这段空白，尤其是带保持要求的场合。

88.10.3 故障分级

故障不能一律停机，也不能一律报警。表 88-14 把故障分成四级，每级对应不同的响应时间与动作。

表 88-14 四级故障的响应时间与动作

等级	典型例子	响应时间	动作
1 致命	安全门打开、超温超压、急停	≤ 100 ms	安全回路直接切断，不经用户程序
2 严重	传感器断线、执行器反馈超差	≤ 1 s	回路切手动，保持安全输出，报警
3 一般	通信中断、模块诊断报警	≤ 10 s	保持当前输出，切冗余通道
4 提示	电池电压低、滤网到期	一个班次	记录并提示，不停机

表里最关键的一行是第一行。致命故障不走用户程序，走硬接线的安全回路。原因是用户程序可能正在被修改、可能进入了死循环、可能因为一个空指针没执行到那一段。安全回路不依赖程序，只依赖继电器与安全模块。

第二行的“保持安全输出”要写清楚是哪个值。阀门是关还是保持、电机是停还是减速，不同工艺的答案不同。写成“保持当前值”是偷懒，现场没人知道当前值该是多少。

第三行的冗余切换有个陷阱：切换本身需要时间，而通信中断可能意味着远程 I/O 也失联了。此时切过去的通道可能同样读不到数据，切换动作变成两次无效尝试。设计时要确认冗余通道的物理路径真的独立。

88.10.4 安全回路的接线

安全回路与普通控制回路的最大区别是“失电跳闸”。正常运行时安全回路是通电的，一旦断线、失电、触点氧化，回路失电，设备停在安全位。这个设计让“故障”自动等于“安全”，不依赖任何逻辑判断。

普通控制回路常做成“得电动作”，因为这样节能、接线简单。安全回路必须反过来，代价是正常运行时线圈一直通电、发热、寿命受限于线圈。这笔代价是安全等级的一部分，不能省。

图 88-8 把三种安全架构画成三条泳道。单通道 1oo1 是一套传感器、一套 CPU、一套执行器，任何一处失效就失去保护；双通道 1oo2 是两套并联，任一套动作就能切断；三取二 2oo3 是三套并联、多数表决，一套误动不影响生产，两套动作才切断。

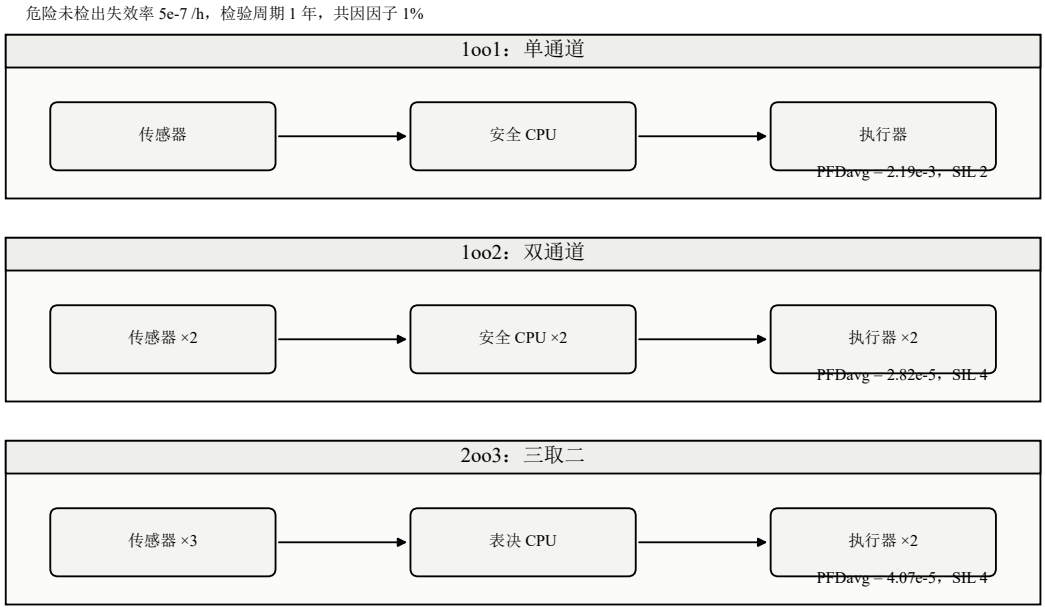


图 88-8 1oo1、1oo2、2oo3 三种安全架构

三条泳道的差别不在硬件数量，而在“失效如何被容忍”。1oo1 不容忍，1oo2 容忍一次危险失效但不容忍误动，2oo3 两者都容忍一部分。选哪种取决于工艺更怕“该停没停”还是更怕“不该停却停”。

时间链也要逐段核。一条典型链是：传感器响应 10 ms、安全输入模块 8 ms、安全 CPU 扫描 3.08 ms、安全输出模块 5 ms、阀门关闭 80 ms，合计 106 ms。若安全要求是 100 ms，这条链已经不达标，需要换快阀或提前触发。

很多项目的安全时间指标只在纸上算过一次，用的是“典型值”。真正投运前要在现场用示波器测一遍，从传感器动作到阀门到位，测三遍取最大值。纸面 106 ms 与实测 130 ms 的差距，通常来自阀门实际行程时间。

88.10.5 旁路与维护开关

安全回路必须有旁路，否则设备一报警就只能停机检修。旁路是一把双刃剑：它让维护可以进行，也让保护暂时失效。

管理旁路靠三条：钥匙开关、挂牌、时限。钥匙由值班长保管，谁旁路谁挂牌，挂牌上写清旁路原因、旁路范围、恢复时间。时限到了没恢复，系统自动报警并拒绝继续旁路。

旁路期间的保护等级要重新算。双通道旁掉一路，等于退化成单通道，PFD 从 2.82×10^{-5} 升到 2.19×10^{-3}。按小时算，旁路 8 小时的风险增量是 $\lambda_{DU} \times 8 = 4 \times 10^{-6}$，相对全年可忽略。

真正危险的是“忘了复位”。旁路开关挂在柜门上，检修完谁也没动，一年后评估时才发现保护一直是单通道。这一年的 PFD 就是 2.19×10^{-3}，SIL 4 掉回 SIL 2。所有旁路都该有累计计时与超时报警，这不是可选项。

88.10.6 FAT 与 SAT

出厂验收（FAT）在制造厂做，现场验收（SAT）在用户现场做。两者测的东西不一样：FAT 测控制器与程序，SAT 测整套系统与工艺。表 88-15 列出两阶段的主要测试项。

表 88-15 出厂验收与现场验收的主要测试项

阶段	测试项	判据
FAT	I/O 点对点	全部点通，无错位、无串点
FAT	联锁逻辑	按联锁表逐条触发，动作与延时符合
FAT	梯形图语义	双线圈、自保持、边沿触发逐项核对
FAT	扫描周期	最大扫描小于设计值的 1.2 倍
FAT	掉电保持	断电 10 min 后数据与状态不丢
SAT	现场接线	与图纸一致，绝缘与接地合格
SAT	回路闭环	阶跃超调小于 10%，2 min 内稳定
SAT	安全功能	触发后 100 ms 内到达安全位

FAT 里最容易漏的是第三行。双线圈的覆盖行为在台架上不一定复现，因为台架上的输入是手动给的，时序与现场不同。测试时要按图 88-6 的八种组合逐一走一遍，记录实际输出而不是“看着对”。

SAT 里最容易漏的是第七行的时间。回路闭环测的是“能不能稳住”，不是“多久稳住”。一个 5 min 才稳住的温度回路，在 FAT 里可能被判合格，到现场却因为批次周期只有 3 min 而永远稳不住。

两阶段的记录都要签字归档。记录不是形式，它是日后出问题时唯一能追溯的证据：证明当时测过、测的是哪一版程序、判据是什么。

88.10.7 在线修改的规程

在线修改是 PLC 的核心能力，也是最需要规程约束的动作。规程至少包含六步：申请、评审、备份、分段下载、验证、记录。

申请要写清改什么、为什么改、影响哪些回路、预计停机多久。评审由工艺与仪表两方签字，一个人拍板的修改最容易出事。

备份是强制的。下载前把当前程序与数据块一起上传存档，命名带上日期与版本号。改坏了能回滚，这条比什么都重要。

分段下载按 §88.9.4 的约束执行：一次不超过 5 个块，段间等一拍。若改动涉及安全功能，必须切到安全态再下载，不能在线改。

验证要按改动范围做，不是“看一眼运行正常”。改了联锁就重走联锁表，改了 PID 就重做阶跃，改了通信就重测刷新周期。

记录写进变更台账：谁改的、什么时候、改了哪几个块、验证结果、回滚版本号。台账是下次故障排查的起点。

88.10.8 文档与移交

交付的文档有六类：I/O 清单、电气图、程序清单（含注释）、参数表、测试记录、操作维护手册。六类缺一类，接手的人就要多花一周。

程序注释是最容易被忽略的一项。半年后回看自己写的梯级，没有注释也认不出来。注释至少写三段：这一段解决什么问题、关键变量的量纲与范围、修改时要注意什么。

参数表要写成“参数—当前值—允许范围—依据”四列。只写当前值，下次整定时没有参照；只写范围，不知道当初为什么取这个值。

移交时要现场走一遍：从冷启动开始，到手动、自动、停机、复位，每个状态都操作一次。走不通的地方就是文档没写清的地方。

88.10.9 现场调试的六个典型问题

第一个是开关量抖动。机械触点闭合时有 5 到 20 ms 的弹跳，不滤波会让程序在一个扫描内看到几次通断。输入滤波设 10 ms 能压住，代价是联锁响应多 10 ms。

第二个是模拟量漂移。0 到 10 V 信号在长距离传输时容易受共模干扰，4 到 20 mA 好得多。已经用电压信号的场合，把屏蔽层单端接地，别两端都接，两端接地会形成地环路。

第三个是通信偶发丢包。先查终端电阻，总线两端各 120 Ω，中间站不能加。再查波特率与距离的匹配：187.5 kbps 可到 1000 m，1.5 Mbps 只到 200 m，超距离时误码率随长度指数上升。

第四个是输出触点粘连。直流感性负载断电时反电动势击穿触点，加续流二极管；交流负载加 RC 吸收。粘连的后果是“该断不断”，比“该通不通”危险得多。

第五个是变频器干扰。变频器载波 4 到 16 kHz，电机电缆是主要辐射源。电缆用屏蔽型、屏蔽层两端接地、与信号线分开走槽，三条做到基本能压住。

第六个是地环路。柜内多点接地时，各点之间有毫伏级电位差，叠加在模拟量上就是缓慢漂移。排查办法是断开一端接地看漂移是否消失，确认后改成单点接地。

六个问题的共同点是“程序没错、参数没错，但现象像控制问题”。调试时先看诊断缓冲，再看趋势曲线，最后才拆线。顺序反过来，会把半天花在拆接线上。

88.10.10 投运后的运行监视

投运不是终点。前三个月要每周看一次扫描趋势，确认没有随程序增长而缓慢上升。扫描上升通常来自三个地方：诊断缓冲写满、通信连接数增加、程序里加了新的轮询。

诊断缓冲要定期导出。缓冲区是环形结构，写满后新记录覆盖旧记录，故障发生一周后再去读，可能已经被覆盖。导出的记录按时间归档，是排查偶发故障的唯一线索。

电池与风扇是唯一的消耗件。电池一般 3 到 5 年，电压低时面板报警；风扇 5 年左右，轴承磨损后噪声变大。两者都要列入预防性维护计划，不能等到报警才换。

安全功能每年做一次全功能测试，测试记录与上一次对比。若某次响应时间比上次长了 20%，即使仍达标也要查原因：可能是阀门变涩、传感器老化、或输出模块继电器退化。

程序版本要能追溯。柜门内侧贴一张版本标签，写明程序版本、组态版本、上次修改日期。现场排故第一件事是对版本，版本对不上，所有分析都要重来。

88.10.11 柜内布置与散热

控制柜的布置不是外观问题，它直接决定模块寿命与故障率。三条基本要求：散热、间距、分区。

散热先算发热量。CPU 约 15 W，每个 I/O 模块 3 到 5 W，24 个模块合计约 100 W，电源自身损耗再加 20 W。120 W 在一个 0.6 m² 的柜内，不加通风温升可到 15 K，超过模块的 55 ℃ 上限。

间距按厂家手册留。模块上下各留 50 mm，柜内上下留 100 mm 走线。挤在一起的模块，中间那块的温度比边缘高 8 到 10 K。

分区是把动力与信号分开。变频器、接触器、固态继电器放一个区，PLC 与模拟量模块放另一个区，中间加隔板。两个区各自走线槽，不共用。

风扇与空调的选择看环境。柜内温升 10 K 以内用风扇即可，超过 10 K 或环境温度高于 40 ℃ 要用空调。风扇的滤网每季度清洗，堵了等于没装。

接地是布置的一部分。PE 排单点接地，屏蔽层汇流到同一排，模拟量屏蔽不与动力屏蔽混接。接地做错，前面所有的抗干扰措施都白费。

88.10.12 移交清单

移交时按清单逐项核对，清单本身也归档。

清单七项：程序与组态的最新备份、I/O 清单与电气图、联锁表与测试记录、参数表、操作维护手册、备件清单、版本标签照片。

每项都要有接收人签字。签字不是形式，它把“我以为你给了”变成“你确认收到了”。

清单之外还有一项口头交接：哪些地方是临时方案、哪些报警是误报可以忽略、哪些参数是试出来的没有依据。这三件事写不进文档，但接手的人必须知道。

口头交接要转成文字，写进备忘录附在清单后面。一次会议、两页纸，能省掉后来者几个月的摸索。这是整章最后一条建议，也是最便宜的一条。

88.11 优缺点与适用边界

88.11.1 四个优点

第一个优点是确定性来自结构，而不是来自调度分析。PLC 的执行模型是“读输入、跑程序、写输出”的固定循环，任何时刻只有一段程序在跑。这让最坏情况的估算变得简单：把各块时间加起来就是周期，不需要建任务模型、不需要算优先级反转。

第二个优点是面向维护人员。五种标准语言（梯形图、功能块图、结构化文本、指令表、顺序功能图）让电工看得懂梯形图，让工程师写得动结构化文本。一个夜班故障，值班电工能自己看程序定位到第几个梯级，这在嵌入式系统里做不到。

第三个优点是生态完整。模块化的 I/O、远程站、安全模块、通信网关、HMI、变频器，都按同一套标准接线与组态。选型时不用自己攒板子，供货周期以天计而不是以月计。

第四个优点是认证。功能安全认证（SIL 3）、船舶认证、防爆认证，主流 PLC 都有现成的证书。自研控制器要拿到这些证书，时间与费用往往超过硬件本身。

88.11.2 四个缺点

第一个缺点是节拍固定且偏慢。毫秒级的扫描周期对过程控制足够，对高动态回路不够。本章的算例里，扫描 3 ms 对 2 s 时间常数的回路只损失 0.2° 相位；换成 20 ms 时间常数的回路，同样 3 ms 就要损失 20° 以上。

第二个缺点是算力有限。PLC 的 CPU 主频通常在几百兆赫，且不带浮点协处理器的型号仍不少见。做 32 轴插补、做图像处理、做模型预测控制，都超出它的能力范围。

第三个缺点是语言表达力受限。梯形图适合布尔逻辑与联锁，写矩阵运算、递推滤波、状态估计就很别扭。结构化文本好一些，但仍缺少现代语言的容器、异常、单元测试工具链。

第四个缺点是厂商锁定。程序、组态、通信协议都带厂商色彩，换品牌等于重写。这一点在项目周期长的场合尤其明显：十年后维护，原厂可能已经停产该型号。

88.11.3 与 DCS 的差别

PLC 与 DCS 常被拿来比较，边界其实很清楚。PLC 从离散制造长出来，强在逻辑与联锁；DCS 从连续过程长出来，强在回路与画面。

表 88-16 把四类平台放在一起对比。PLC 的扫描是毫秒级、回路规模是数十到数百；DCS 的控制周期是 100 到 500 ms、回路规模是数百到数千，但它的回路是“每个回路独立组态”的，改一个回路不影响其他回路。

表 88-16 四类控制平台的对比

项目	PLC	DCS	工控机	运动控制器
典型周期	1–20 ms	100–500 ms	0.5–10 ms（不定）	50–250 µs
回路规模	数十至数百	数百至数千	不限	1–32 轴
确定性	强（循环扫描）	强（控制网调度）	弱（操作系统）	强（硬件插补）
在线修改	支持	支持	需重启	部分支持
安全认证	SIL 3	SIL 3	少见	部分 SIL 3
维护门槛	低	低	高	中

DCS 的真正优势不在周期，而在“工程化”：一套组态工具覆盖回路、报警、趋势、报表、权限，几千个回路用同一套模板批量生成。PLC 做几百个回路可以，做几千个就要靠程序员自己搭框架。

反过来，PLC 的优势在“离散”。一个包装线有 300 个气缸、200 个光电开关、几十条联锁，DCS 处理这类逻辑既贵又笨。选型的第一问不是“哪个先进”，而是“这个项目里连续回路多还是离散动作多”。

88.11.4 与工控机的差别

工控机（IPC）跑通用操作系统，能做视觉、能做数据库、能做复杂算法。它的弱点正是 PLC 的强点：确定性。

通用操作系统的时间抖动在毫秒到几十毫秒量级，取决于中断、驱动、后台进程。加实时补丁能改善，但改善的是平均值，最坏值仍不可控。一条要求 100 ms 内切断的联锁，放在工控机上就要承担“偶尔 300 ms”的风险。

工控机的另一个弱点是寿命。商用主板与硬盘的 MTBF 通常 3 到 5 年，PLC 的设计寿命是 10 到 15 年。产线要用十五年，工控机中途要换两代，每次换都涉及驱动与系统的兼容问题。

实际项目里两者常常并存：PLC 做实时控制与联锁，工控机做上位监控、数据存储、视觉检测。分工的界线是“错过会出事”与“错过只是慢一点”。

88.11.5 与运动控制器的差别

运动控制器专攻多轴同步，周期 50 到 250 µs，靠硬件插补或专用 DSP 实现。它的核心指标不是扫描周期，而是同步误差：多轴之间的位置偏差要控制在微米级。

PLC 也能做运动控制，带运动控制功能块的 CPU 可以控几个轴，周期 1 到 4 ms。这个量级适合点位控制、简单插补，不适合高速轨迹加工。差别不在指令集，而在周期：1 ms 周期下，轴以 1 m/s 运行时每个周期走 1 mm，插补的折线误差肉眼可见。

所以边界的判据是“每周期位移”。若每周期位移小于精度要求的十分之一，PLC 的运动功能够用；否则要上专用控制器。这条判据比“轴数”更本质。

88.11.6 适用边界：扫描 1 到 20 ms

把本章的结论收成一条：PLC 适用于闭环带宽在 0.1 到 10 rad/s、时间常数在 0.1 s 以上的回路。

按式 (88-13)，扫描周期上限是 $T_{\text{scan}} \le 2\Delta\varphi_{\text{allow}}/\omega_c$。取允许的相位损失 1.4°（0.024 rad），穿越频率 1 rad/s 时上限 49 ms；穿越频率 10 rad/s 时上限 4.9 ms；穿越频率 100 rad/s 时上限 0.49 ms，已经超出 PLC 的能力。

反过来看时间常数：扫描取时间常数的 2.5% 时品质几乎不变，取 5% 时超调翻倍，取 10% 时接近不可用。若扫描 10 ms，时间常数应大于 0.2 s；若扫描 20 ms，应大于 0.4 s。这就是“过程控制”与“运动控制”的分水岭。

这条边界不是硬线，而是渐变。时间常数 0.1 s 的回路用 10 ms 扫描也能跑，只是超调会从 4% 涨到 10% 左右，需要把增益调低来补偿，响应变慢。工程上能否接受，要看工艺允许多慢。

88.11.7 不适用的三类场合

第一类是高频闭环。电流环、力环、振动主动抑制，带宽在千赫兹量级，必须用 DSP 或 FPGA。PLC 的毫秒周期在这些场合连采样都不够。

第二类是微秒级同步。多轴插补、飞剪、电子凸轮，要求各轴在同一微秒内更新。这类场合靠硬件同步信号，不靠软件扫描。

第三类是大算力任务。视觉检测、模型预测控制、深度学习推理，需要 GPU 或至少多核 CPU。PLC 的算力差两到三个数量级，勉强做也只能降采样、降精度。

三类场合的共同点是“周期要求超出毫秒级”，而不是“PLC 不好”。把 PLC 用在这些地方，问题不在控制器，在选型。

88.11.8 选型的判断流程

图 88-9 给出一条判断链，四个问题依次问下来，基本能定位到平台类型。

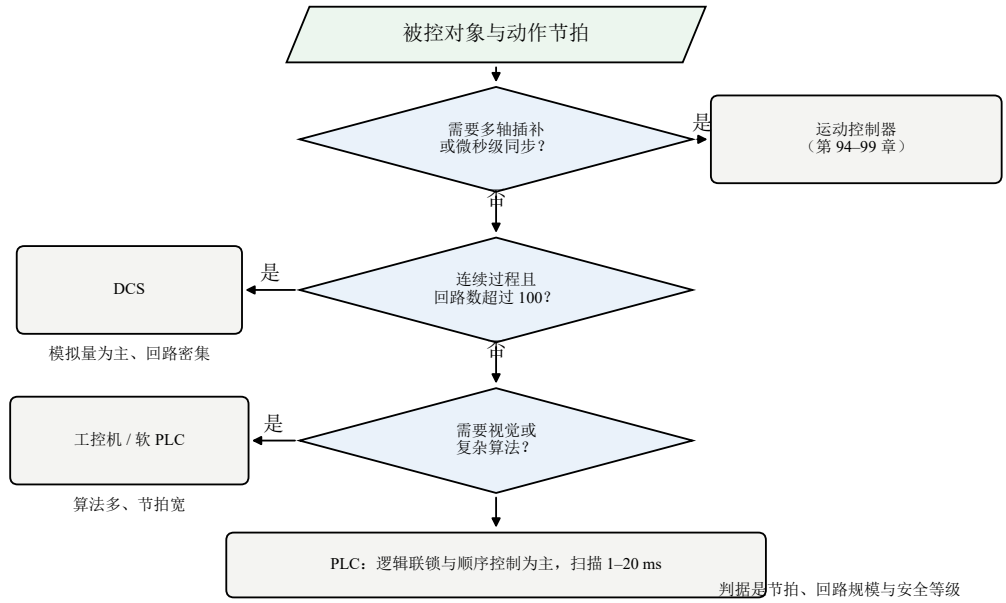


图 88-9 控制平台选型判断流程

第一问：有没有安全功能要求？有，先确定安全等级，再选带认证的安全控制器或安全模块。安全不能靠普通 PLC 加软件实现。

第二问：闭环带宽是否超过 10 rad/s？超过，看是单轴还是多轴。单轴可用带运动功能的 PLC，多轴要专用运动控制器。

第三问：连续回路是否超过 500 个？超过，且以回路为主，倾向 DCS；不到，倾向 PLC。

第四问：是否需要视觉、数据库、复杂算法？需要，用工控机做上位，PLC 做下位。两者分工，不要试图用一台设备全包。

四个问题的顺序不能颠倒。安全是第一位的，带宽是第二位的，规模与算力排在后面。先选规模再选安全，往往要在后期返工。

88.11.9 成本与生命周期

选型时看采购价，投运后看总拥有成本，两者常常给出相反结论。

一套中型 PLC 的硬件约占总成本的 30%，编程与调试占 40%，十年维护占 30%。只比硬件价格，等于只比了三分之一。便宜两成的硬件，若编程效率低三成，项目总成本反而更高。

维护成本里有三项容易被漏算。备件库存：停产型号的模块要提前囤，占资金。软件升级：老版本编程软件可能不支持新操作系统。人员培训：换品牌等于重新培养一批会看程序的人。

十年周期的账更能说明问题。硬件差价 3 万元，维护差价每年 1 万元，十年就是 10 万元。选型时那 3 万元的差价，在第三年就被抹平了。

反过来说，小项目、短周期、一次性的设备，硬件差价的权重就高。判断标准是设备的设计寿命：五年以内的，看采购价；十年以上的，看维护与备件。

88.11.10 本章结论

PLC 的价值不在算得快，而在“行为可预测、维护可交接、责任可认证”。这三点在实验室里看不出价值，在投产十年后决定成败。

它的代价是节拍固定、算力有限、语言受限。这三条限制决定了它的适用区间：过程与离散制造的主流回路，带宽 0.1 到 10 rad/s，时间常数 0.1 s 以上。

本章的定量结论有三条。扫描周期取时间常数的 2.5% 以内，品质几乎不受影响；超过 5% 后超调迅速上升，500 ms 扫描时超调从 4.0% 涨到 32.1%。

掉拍比慢更危险，因为它让积分变弱而超调下降，掩盖了响应变慢的事实。等效积分时间按 $T_i(1+p)$ 变长，掉拍 20% 就让调节时间从 2.96 s 涨到 5.27 s。

安全等级由共因与检验周期决定，不由通道数量决定。双通道在共因 1% 时能到 SIL 4，共因 10%、检验 5 年时掉回 SIL 2。

最后一句：PLC 的所有优点与缺点，都可以归结为“用一个固定的节拍换来了可预测”。接受这个节拍，它是最好的平台；不接受，它处处受限。

88.12 Python 仿真代码

88.12.1 脚本做什么

本章全部数字来自 `book/code/ch088_sim.py`，一个不依赖外部数据、不需要配置文件、跑一遍二十秒的脚本。它的作用不是“演示”，而是“给出处”：正文里每个小数都能在它的输出里找到对应的一行。

脚本只做四件事：算扫描账、算相位裕度、仿真离散 PI、算安全回路的 PFD。四件事都不需要控制系统工具箱，只用到 NumPy 的随机数与数组，便于读者在自己的环境里复现。

脚本的边界也要说清。它是一个“单回路、线性、确定性过程 + 概率性掉拍”的模型，不能替代现场测试。它的价值在于把“扫描慢了会怎样”从定性判断变成定量曲线，而不是预测某台具体设备的行为。

88.12.2 四段结构

第一段是常量与过程模型。时间常数 2 s、纯滞后 0.5 s、增益 2.0、积分时间 2.0 s，四个数由 IMC 整定给出，λ 取 0.5 s。扫描预算的五个块、三档 CPU 的指令时间，也都写成常量，改一处就能改全章。

第二段是频域计算。开环频率特性里，扫描周期按半拍折进纯滞后，见式 (88-2)。相位裕度用二分法找穿越频率，不依赖任何控制系统库。

第三段是时域仿真。离散 PI 按标称周期累加积分，掉拍时把采样间隔翻倍但积分仍按标称值累加——这正是 PLC 定周期写法的行为，也是本章想说明的失真来源。

第四段是安全计算。三种架构的 PFD 各有闭式表达，共因因子与检验周期作为输入，输出直接对到 SIL 分级表。

88.12.3 三个核心函数

第一个是 `loop_gain`，它把扫描周期折成半拍延迟：

```
def loop_gain(w, scan, k=1.0, extra=0.0):
    """开环频率特性，扫描按半拍等效延迟计入。"""
    delay = DEAD + scan / 2.0 + extra       # 纯滞后 + 扫描等待 + 附加滞后
    c = KC * (1.0 + 1.0 / (1j * w * TI))    # PI 控制器
    g = k * np.exp(-1j * w * delay) / (1.0 + 1j * w * TAU)
    return c * g
```

`scan / 2.0` 这一项是本章的核心近似：采样保持的平均延迟是半个采样周期。它让“扫描 100 ms”直接对应“多 50 ms 纯滞后”，不需要做 Z 变换。

第二个是 `phase_margin`，在 0.001 到 100 rad/s 之间二分找 $|L(j\omega)| = 1$ 的频率，返回穿越频率与相位裕度。二分比牛顿法慢，但不会发散，适合写进教学脚本。

第三个是 `simulate_pi` 的采样段，它决定了掉拍的表征方式：

```
if t >= next_sample:
    t_k = scan
    if p_miss > 0.0 and rng.random() < p_miss:
        t_k = 2.0 * scan            # 掉一拍：实际间隔翻倍
    e = 1.0 - x
    integ += e * scan               # 积分只认标称周期
    u = KC * (e + integ / TI)
    next_sample = t + t_k
```

两行注释对应本章的两个结论：掉拍让采样变稀（t_k 翻倍），但不让积分变弱（`integ` 仍乘 `scan`）。两者叠加的效果是“采样更滞后、积分更软”，正是表 88-5 里超调下降而调节时间上升的原因。

88.12.4 九段报告与正文的对应

脚本的 `report()` 打印九段，每段对应正文的一处。表 88-17 给出映射，方便读者核对。

表 88-17 九段报告与正文的对应关系

段	内容	函数	对应正文
1	扫描预算五块	`scan_budget`	表 88-1、图 88-1
2	三档 CPU 的扫描周期	`cpu_table`	表 88-2、图 88-2
3	扫描对相位裕度	`phase_margin`	表 88-6 前两列
4	扫描对阶跃响应	`scan_response_table`	表 88-6、图 88-4
5	掉拍对积分项	`jitter_table`	表 88-5、图 88-5
6	梯形图扫描顺序	`ladder_scan`	图 88-6、§88.5.4
7	看门狗余量	`watchdog_table`	§88.7.3、式 (88-10)
8	安全回路 PFD 与 SIL	`safety_table`	表 88-4、式 (88-7) 至 (88-9)
9	增益与滞后失配	`robust_table`	表 88-7

第 5 段要说明一处细节：每个掉拍概率跑 10 个随机种子再取平均，因为单次仿真的超调受“哪一拍掉”影响很大。种子从 `SEED` 递增，重跑逐位一致。

第 6 段是唯一不涉及数值积分的部分。它按四段梯级的实际求值顺序模拟线圈写入，输出八种输入组合下的实际值与期望值，用来证明“双线圈不等于并联”。

88.12.5 复现方法

运行只需一行：

```
"D:\Anaconda3\envs\py311\python.exe" book\code\ch088_sim.py
```

脚本无参数、无输入文件、无网络访问。输出是九段纯文本，可直接与正文对照。全章 9 张图由 `book/code/ch088_figs.py` 生成，它导入同一个模块，保证图与表用的是同一组常量。

复现时可能遇到的差异只有一处：不同 NumPy 版本的随机数生成器对 `default_rng` 的实现是稳定的，但若改了 `SEED`，第 5 段的三个数会变。正文引用的是固定种子下的结果。

88.12.6 未覆盖的部分

脚本没有覆盖四件事。第一是抖动分布：真实扫描的间隔不是“标称或翻倍”两种，而是连续分布，长尾比二项分布更重。

第二是总线争用：远程 I/O 的响应时间取决于总线负载，负载高时延时会跳变，这一层在脚本里被折叠成固定的 0.5 ms。

第三是固件行为：不同厂商对掉拍的处理不同，有的用实际时间积分，有的用标称周期，有的在掉拍时冻结积分。脚本模拟的是最坏的一种。

第四是多回路耦合：脚本只算单回路，实际项目里十几个回路共用一次扫描，彼此通过公用工程（蒸汽、冷却水）耦合，单回路稳定的参数组合起来可能振荡。

这四件事都要靠现场测试补齐。脚本的作用是给出量级与趋势，不是给出保证。

88.12.7 改参数试自己的工况

把 `TAU` 改成自己过程的时间常数，重跑第 3、4 段，就能看出当前扫描周期是否够用。判据仍是那两条：扫描小于时间常数的 2.5% 为佳，超过 5% 要复核超调。

把 `KC`、`TI` 改成现场整定值，重跑第 9 段，能看出增益失配的敏感度。若增益加到 1.3 倍时裕度已经掉到 35° 以下，说明现场没有余量，任何附加滞后都会引发振荡。

把 `LAMBDA_DU` 改成厂商提供的失效率，重跑第 8 段，能算出自己项目的 PFD。若结果卡在 SIL 边界上，优先改检验周期而不是加通道——前者成本低一个数量级。

三处改动都不需要动代码结构，改常量即可。这也是脚本把常量集中写在文件开头的原因：读者不必读懂全部实现，就能用自己的数替换掉本章的数。改完重跑一遍，正文与图会一起更新，不会出现两套数字。

88.12.8 图是怎么生成的

全章 9 张图由 `book/code/ch088_figs.py` 生成，它导入 `ch088_sim`，用的常量与数值算例完全一致。改一个常量，图和表同时变，不会出现“图上是 3.08、表上是 3.1”这类不一致。

9 张图分三类。曲线类 2 张（扫描对响应、掉拍对积分），数据全部来自仿真；时间轴与柱状类 3 张（扫描五块、CPU 三档、延迟链），数据来自预算表；流程图与泳道图 4 张（双线圈、程序分层、安全架构、选型），由绘图脚本按坐标摆放。

按 L13 的图元规范，本章不出传递函数方块。所有图只用圆角矩形、菱形、平行四边形与泳道，线条为 0.8 pt 实心箭头。这样做是为了让读者看到的是“系统怎么组织”，而不是“信号怎么流”。

每张图输出两份：SVG 供电子版，PDF 供印刷。两份由同一个函数写出，不存在版本差异。图的编号与文件名的序号一一对应，`fig_088_04` 就是图 88-4。

图内的中文用普通字体，公式用数学字体单独排。两者不写在同一个字符串里，避免字体回退导致的乱码。这条约定在前几章踩过坑，本章从第一张图就守住了。九张图排在一起看，读者应当先看到“阶段—预算—延迟”的主线，再看到“程序—安全—选型”的分支。

第89章 工业通信与网络

89.1 动机与直观理解

89.1.1 三个现场现象

第一个现象是“线接好了，数据却不新鲜”。某条产线的温度回路在本地控制时表现正常，把传感器改到远程 I/O 站之后，操作员觉得响应“慢半拍”。查程序没有问题，查参数也没有问题，问题出在轮询：主站每 1.37 s 才把所有从站读一遍（表 89-3），回路拿到的偏差值平均滞后半轮，相当于凭空加了 0.7 s 的纯滞后。

第二个现象是“同一段网络，慢回路没事，快回路发散”。设备上同时跑温度回路与伺服轴，网络延迟 10 ms。温度回路的超调几乎没有变化（0.2%）；伺服轴的速度环超调从 7.7% 涨到 33.5%，积分误差涨了 77.7%（表 89-11）。延迟的绝对量一样，后果差了两个数量级，因为两个回路的穿越频率差了 66 倍。

第三个现象是“链路不忙，抖动却很大”。交换机端口利用率不到 30%，工程师认为带宽绰绰有余，但周期数据的到达时刻在 0 到 20 ms 之间跳动。原因是链路上混着大文件传输与视频流，周期帧在每个交换机出口排队，排队时间不由平均负载决定，而由“恰好同时到达”的概率决定。利用率 0.7 时平均排队 1.17 ms，尾部可达 4.67 ms（表 89-6）。

三个现象对应三件事：轮询周期决定数据新鲜度，穿越频率决定延迟要不要紧，排队决定抖动。它们都不属于控制算法的范畴，却在现场反复出现。

89.1.2 与相邻章的分工

第 86 章讨论实时软件的任务划分与调度，回答“多个周期任务怎么共享一个 CPU”；第 87 章讨论嵌入式控制器，回答“一颗 MCU 在一拍内怎么把 FOC 算完”；第 88 章讨论 PLC，回答“一次扫描里五个阶段怎么分配时间”。本章讨论这些计算单元之间的连接：数据从 A 站到 B 站要多久，这个时间有多不确定，以及不确定性如何进入闭环。

与前几章最大的差别是“延迟不再由你一个人决定”。单机系统里，扫描周期由程序规模与 CPU 决定，工程师可以把它算准；一旦跨上网络，延迟里就混进了交换机队列、其他节点的突发流量、重传与协议栈开销，这些量既不由本回路控制，也很难离线复现。本章的方法因此分成两半：一半是把延迟算清楚，另一半是把不确定性管住。

本章不重复 PID 的整定（见第 44、106 章），不展开网络化控制的理论判据（见第 82 章），也不讲信息安全的攻防（只在小节里划边界）。采样定理与零阶保持在第 4、85 章已经讲过，这里只引用结论。

89.1.3 通信到底改变了什么

把一条控制回路从“本地接线”改成“网络连接”，物理上只多了一段传输，工程上却同时引入了三种效应。

第一种是延迟。数据要打包、排队、传输、解包，控制量从算出到生效多花一段时间。这段延迟与过程本身的纯滞后作用相同，都消耗相位裕度，只是它通常更小、更容易被忽视。

第二种是抖动。延迟不是一个常数，而是一个分布。同一帧在这一拍用 2 ms 到达，下一拍可能用 18 ms。控制器按固定周期计算，却不知道自己的输出何时生效，等效于每一拍都有一点点随机的附加滞后。

第三种是丢失。帧可能因为校验错误、缓冲区溢出、链路切换而丢掉。丢掉之后执行器保持上一拍的值，回路短暂开环。单帧丢失通常无碍，连续丢失才会出事。

延迟、抖动、丢失这三种效应，对应的处置手段完全不同：延迟要“算”，抖动要“压”，丢失要“防”。把三者混为一谈，是选型与调试中最常见的错误。

89.1.4 三类总线的分工

工业通信不是一种技术，而是三代技术并存。图 89-8 把三类以太网协议的层次画在一起，可以看到它们的分歧点在哪里。

不过先说清楚一件事：网络不是一个平面，而是分成三层。表 89-1 是这三层的分工，混层是现场最常见的架构错误——把视频监控接到控制环网上，一个文件下载就能让整条线的周期帧抖起来。

表 89-1 工业网络三个层次的分工

层次	典型周期	主要协议	连接对象
现场层	0.1–10 ms	EtherCAT、Profinet IRT、CAN	驱动器、远程 I/O、传感器
控制层	10–500 ms	Profinet RT、EtherNet/IP、Modbus TCP	PLC、DCS、HMI 之间
管理层	秒级	OPC UA、MQTT、SQL	MES、SCADA、云平台

分层的判据是“延迟容忍度”，不是“物理位置”。同一台柜子里的 PLC 与 HMI 之间如果只传画面数据，可以走控制层；如果传的是联锁信号，就必须走现场层。判断标准只有一条：这段数据晚到 10 ms 会不会出事。

第一代是串行现场总线，代表是 Modbus RTU 与 Profibus。它把数据挂在一条 RS-485 双绞线上，主站轮询、从站应答。速率低（9.6 到 115.2 kbit/s），但时序完全确定：同一时刻只有一对设备在说话，不会碰撞。代价是慢，9600 bit/s 下读 10 个寄存器要 42.67 ms。

第二代是 CAN 一类的事件总线，代表是 CAN 2.0 与 CANopen。它用优先级仲裁代替轮询，高优先级的帧可以抢占低优先级的发送机会。500 kbit/s 下一个 8 字节帧约 222 μs，最坏含位填充 270 μs。它适合大量短报文（传感器、驱动器状态），不适合大块数据传输。

第三代是工业以太网，代表是 EtherCAT、Profinet、EtherNet/IP。物理层是标准的 100 Mbit/s 以太网，区别在协议栈：EtherNet/IP 走完整 TCP/IP，Profinet RT 走 UDP，Profinet IRT 与 EtherCAT 干脆绕过 IP 层，把过程数据直接塞进以太网帧。绕过的层数越多，抖动越小，确定性越好。

表 89-2 把三代技术放在一起。表中的“确定性”一列不看速率，只看最坏延迟有没有上界：Modbus RTU 慢但可算，标准 TCP/IP 快但不可算，EtherCAT 又快又可算。

表 89-2 三代工业总线技术的定位

代际	代表协议	速率	典型周期	确定性
第一代	Modbus RTU、Profibus DP	9.6 k–12 Mbit/s	10–100 ms	好，轮询无碰撞
第二代	CAN 2.0、CANopen	125 k–1 Mbit/s	1–10 ms	好，仲裁可算
第三代	EtherCAT、Profinet IRT	100 Mbit/s	0.1–4 ms	好，时隙或逻辑环
旁支	Modbus TCP、EtherNet/IP	100 Mbit/s	10–50 ms	差，TCP 重传不可算

三代的边界不在“先进”，而在“周期”。10 ms 以上的场合，Modbus RTU 便宜可靠；1 到 10 ms 的场合，CAN 与 Profinet RT 够用；1 ms 以下、要求微秒级同步的场合，只有 EtherCAT 与 Profinet IRT 可选。图 89-9 把这套判断画成流程。

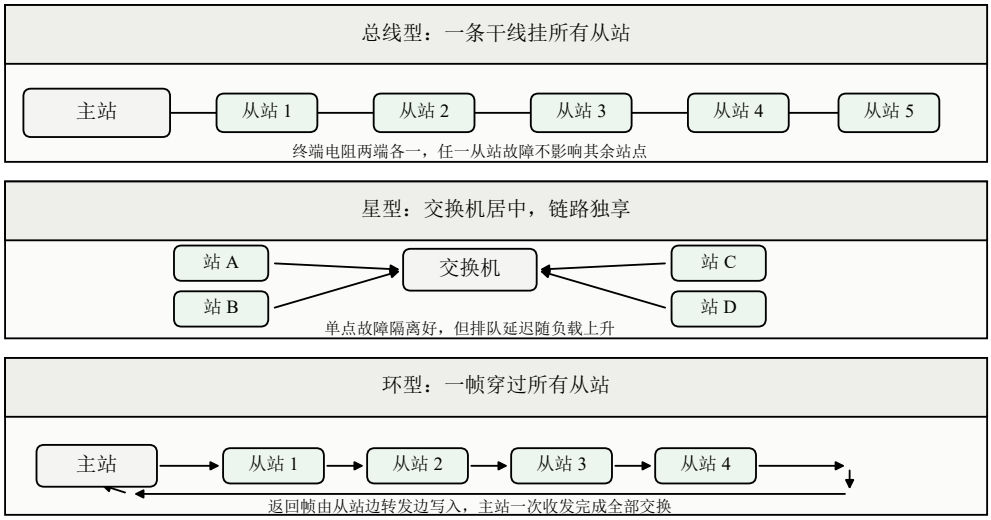


图 89-3 三种现场总线拓扑

三种拓扑的差别值得单独说一句。总线型（图 89-3 上）所有站挂在同一条干线上，任一站的支线断开只影响自己，但干线断开全段失效；星型（中）靠交换机做点对点，单点故障隔离最好，代价是每个端口都可能排队；环型（下）在物理上是一根线首尾相连，一帧穿过所有从站，主站一次收发就完成全部交换，断一处还能从另一侧绕回。选拓扑本质上是在“故障域”与“延迟”之间做交换。

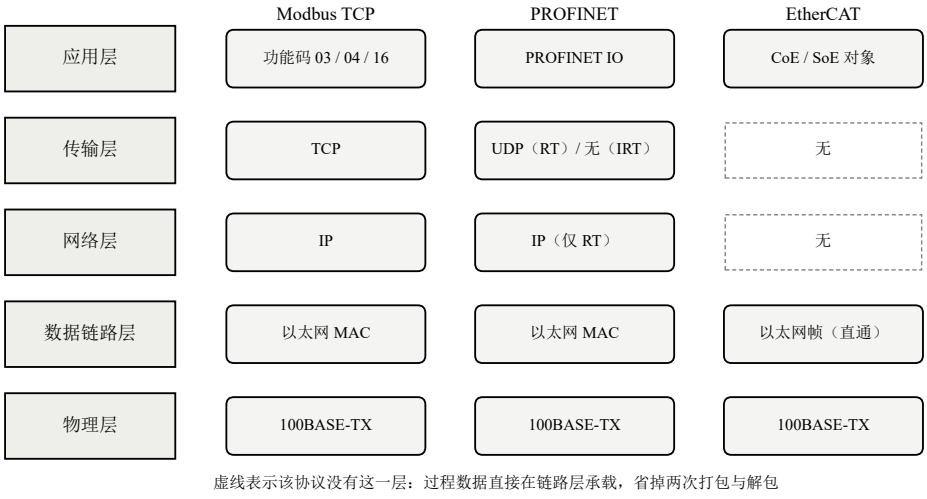


图 89-8 三类以太网协议的层次映射

89.1.5 本章的读法

本章按“算—设—调—落地”四条线组织，读者可以按角色选读。

做选型的读者，先看 §89.1.4 与 §89.11.7，把节点数、周期、数据量三个数填进图 89-9 的流程，得到一两个候选，再回 §89.3 核对带宽与利用率。

做设计的读者，重点在 §89.2 与 §89.6。前者给出延迟、排队、同步的计算式，后者给出延迟与抖动如何吃裕度。两节合起来回答一个问题：这条网络能不能撑住这个回路。

做调试的读者，直接看 §89.8 与 §89.10.4。前者是测量与整定的顺序，后者是“现象—原因—处置”的对照表。

做评审的读者，看 §89.9 与 §89.11。前者讲优化能拿到什么、代价是什么，后者讲各类总线的适用边界。

89.1.6 五个易混的词

延迟、抖动、确定性、刷新率、吞吐，这五个词在手册里经常混用，含义却不同。

延迟是“一帧从发出到被用到”的时间，是一个标量，通常给平均值或最坏值。抖动是延迟的波动幅度，是一个分布宽度。确定性是“最坏延迟有上界”这一性质，与延迟大小无关：一条总是慢 10 ms、但每次都不多不少的链路，确定性很好，延迟却很大。

刷新率是“同一份数据多久更新一次”，单位是 Hz。它不等于帧率：一帧可以携带 64 个轴的数据，帧率 1 kHz 时每个轴的刷新率也是 1 kHz；但如果把 64 个轴分成 4 帧轮发，帧率仍是 1 kHz，每个轴的刷新率就降到 250 Hz。

吞吐是单位时间能搬多少字节，单位是 bit/s 或 帧/s。吞吐高不代表延迟低：一条 1 Gbit/s 的链路如果排队 50 ms，吞吐再高也没用。反过来，一条 100 kbit/s 的链路只要不排队，延迟可以只有几毫秒。

分清这五个词，后面所有计算才不会张冠李戴。

89.1.7 三个时间标尺

人对时间的直觉在毫秒尺度上并不可靠，先立三把尺子。

第一把是人的反应。眼睛看到、手做出动作，全程约 200 ms；触摸屏点下去的视觉反馈，50 ms 以内算“跟手”。用这把尺子量网络，10 ms 的延迟人完全无感。

第二把是控制器的节拍。PLC 一次扫描 10 ms 是常态，第 86 章的实时任务周期常在 1 ms 量级，第 87 章的电流环在 50 到 100 µs。用这把尺子量网络，10 ms 延迟等于一个 PLC 扫描周期，等于十个伺服任务周期。

第三把是对象的动态。温度对象的时间常数以秒计，伺服速度环以几十毫秒计。用这把尺子量网络，10 ms 对前者是千分之五，对后者是五分之一。

三把尺子给出的结论完全不同：同一段 10 ms 的网络，对人“感觉不到”，对 PLC“刚好一拍”，对温度“无关紧要”，对伺服“吃掉两成动态”。§89.1.1 的第二个现象，本质就是拿第一把尺子的结论去套第三把尺子的场合。

工程讨论里说“延迟大”或“延迟小”，必须先说清用哪把尺子。没有尺子的形容词，是现场争论最常见的起点。

89.1.8 本章不做什么

边界要先划清，否则读者会期待错东西。

本章不推导协议。Modbus 的功能码、CAN 的仲裁机制、EtherCAT 的状态机，只讲到“影响时序的那一层”，报文格式与寄存器定义留给厂商手册。需要写驱动的人，本章不够用。

本章不做芯片选型。从站控制器用哪家的、主站用软的还是硬的，只在“延迟差多少”的层面讨论，不比较具体型号。型号更新太快，写进书里两年就过期。

本章不涉及信息安全。工业网络的功能安全与信息安全是另一门学问，本章只提一句：隔离既是实时性的手段，也是安全的手段，但两者的设计目标不同。

本章也不做完整的网络规划。地址怎么分、VLAN 怎么划、冗余怎么配，只给原则与清单，具体方案随项目而变。

划清边界的好处是每一节都能落到实处。本章承诺的是：读完能算清一条回路的延迟预算、能判断一段网络够不够用、能在现场按四步查出问题出在哪。三件事做到，本章的目的就达到了。

89.2 数学原理

89.2.1 字符时间与帧时间

串行总线的传输时间按字符算。一个字符包括起始位、8 个数据位、停止位，共 10 位。传输一位的时间是波特率的倒数，所以一个字符的时间为

$$t_{\text{char}} = \frac{n_{\text{bits}}}{R} = \frac{10}{R} \tag{89-1}$$

波特率 9600 bit/s 时，$t_{\text{char}} = 1.042$ ms。这个数字是所有串行总线估算的基石：帧有多少个字符，就乘多少。

一次 Modbus RTU 事务包括请求、从站处理、响应三段。请求帧固定 8 字节，响应帧为 $5 + 2n$ 字节（地址、功能码、字节数、$2n$ 字节数据、CRC）。两端各留 3.5 个字符的帧间隔，按 7 个字符计，于是

$$t_{\text{tx}} = \frac{10\,(L_{\text{req}} + L_{\text{resp}} + 7)}{R} + t_{\text{slave}} \tag{89-2}$$

读 10 个寄存器时，$L_{\text{req}} = 8$、$L_{\text{resp}} = 25$，合计 40 个字符，加 1 ms 从站处理时间，9600 bit/s 下为 42.67 ms。32 个从站轮一遍要 1.37 s。把波特率提到 115200，单次降到 4.47 ms，一轮 0.14 s（表 89-3）。

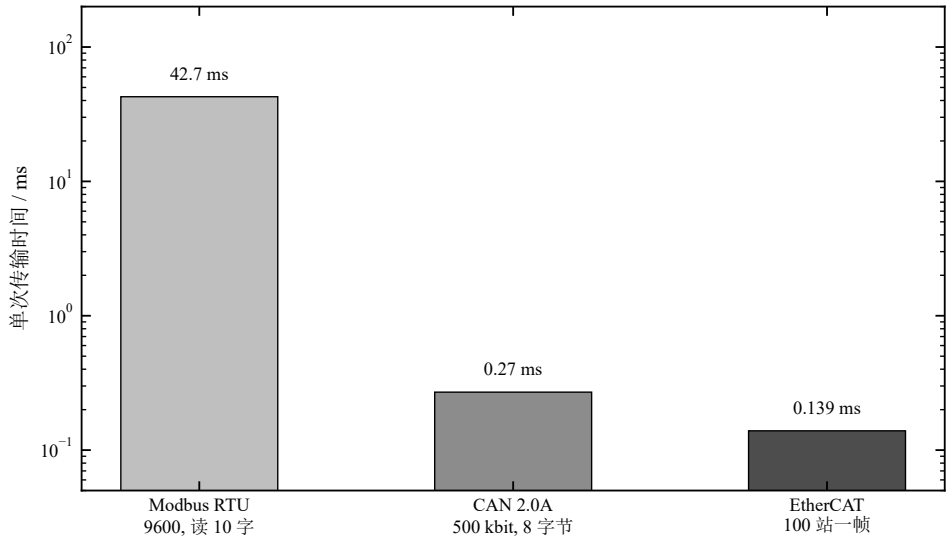


图 89-1 三类总线的单帧时间

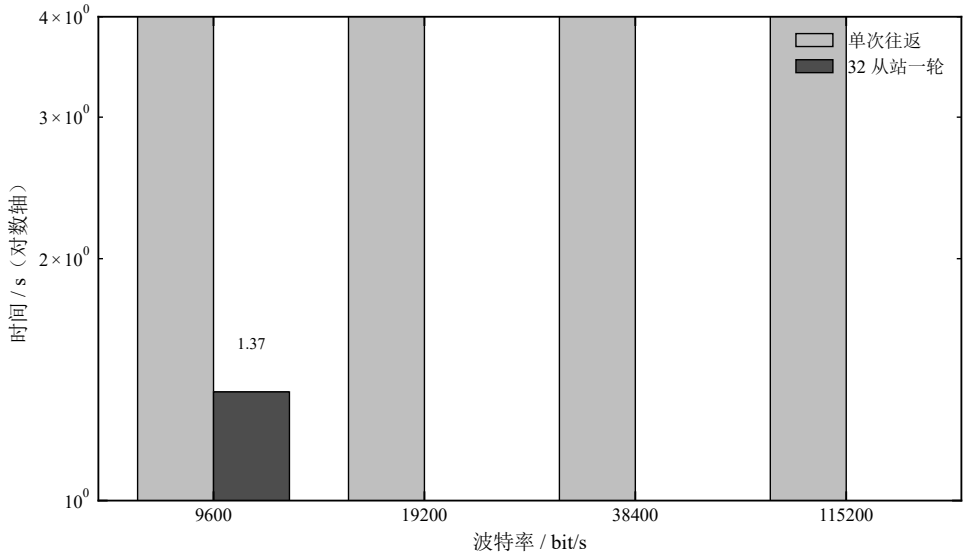
图 89-1 把三类总线的单次传输时间放在对数轴上。Modbus RTU 的 42.67 ms 与 EtherCAT 的 0.139 ms 差了 300 倍，这个差距不是“快一点慢一点”，而是“能不能做闭环”的分界。

42.7

21.8

11.4

4.5



0.70

0.37

图 89-2 Modbus 轮询时间随波特率的变化

图 89-2 给出四个波特率下的单次往返与整轮时间。两条柱子都随波特率线性下降，没有拐点：串行链路的传输时间就是字符数除以速率，提高波特率是唯一的加速手段。工程上的上限通常不是波特率，而是电缆长度与抗扰度——115200 bit/s 在屏蔽双绞线上只能跑几百米。

表 89-3 是图 89-2 的数字版，另外加了一列“单次往返”，方便按回路数估算。

波特率 / bit/s	单次往返 / ms	32 从站一轮 / s	单站刷新率 / Hz
9600	42.67	1.37	0.73
19200	21.83	0.70	1.47
38400	11.42	0.37	2.74
115200	4.47	0.14	7.22

表 89-3 Modbus RTU 读 10 个寄存器的轮询时间（32 从站）

最后一行值得记住：即使把波特率拉到 115200，单个从站的刷新率也只有 7.22 Hz。这意味着 Modbus RTU 只能用于时间常数在秒级的慢回路，用它做伺服或快速压力环是不合适的。

89.2.2 CAN 帧与位填充

CAN 的传输时间不按字符算，按位算。标准帧（2.0A）的固定开销为 47 位：帧起始 1 位、仲裁段 12 位、控制段 6 位、CRC 16 位、应答 2 位、帧结束 7 位、帧间隔 3 位。数据段为 $8N$ 位。

CAN 还有一条别处没有的规则：位填充。发送器每遇到 5 个连续同值位，就插入 1 个反值位，保证接收方能从跳变中恢复时钟。最坏情况下，需要填充的位数为

$$n_{\mathrm{CAN}} = 47 + 8N + \left\lfloor \frac{34 + 8N - 1}{4} \right\rfloor \tag{89-3}$$

$N = 8$ 时标称 111 位、最坏 135 位。500 kbit/s 下分别是 222 μs 与 270 μs。

表 89-4 给出三档数据长度下的位数与时间。填充位数随 N 增长得比数据位慢，所以短帧的相对开销更大：2 字节帧标称 63 位、最坏 75 位，多出 19%；8 字节帧只多出 22%。

数据长度 / 字节	标称位数	最坏位数	500 kbit/s 标称 / μs	最坏 / μs
2	63	75	126	150
4	79	95	158	190
8	111	135	222	270

表 89-4 CAN 2.0A 标准帧的位数与传输时间（500 kbit/s）

位填充带来的 22% 差异在工程上不能忽略。如果按标称值算带宽，实际负载会超出 20% 以上；标准做法是按最坏值留余量，或者用“每帧 130 位”这个经验数。

CAN 的另一个特点是仲裁。11 位标识符同时表示内容与优先级，数值越小优先级越高。两个站同时发送时，逐位比较，先出现显性位（0）的站获胜，输的站自动退让并在下一帧重发。这个过程不破坏任何一帧，也不产生随机退避，因此最坏延迟可以算出来。

代价是“非抢占”：一旦高优先级帧开始发送，低优先级帧必须等它发完（最长 135 位，270 µs）。所以 CAN 的抖动上界大致等于一帧的传输时间。对周期 10 ms 的回路，270 µs 只占 2.7%，可以接受；对周期 1 ms 的回路，占 27%，就必须要算清楚了。

89.2.3 EtherCAT 的逻辑环

EtherCAT 的传输模型与前面两类都不同。它不发“一问一答”，而是主站发一帧，这帧依次穿过所有从站，每个从站在经过时把自己的数据写入帧内对应位置，最后回到主站。主站一次收发完成全部交换。

一帧的线上字节数为前导 8、以太网头 14、过程数据（2 字节工作计数器加负载）、FCS 4、帧间隔 12。传输时间为

$$T_{\mathrm{cyc}} = \frac{8 L_{\mathrm{wire}}}{R} + n\, t_{\mathrm{fwd}} + T_{\mathrm{master}} \tag{89-4}$$

其中 L_{wire} 是线上字节数，n 是从站数，t_{fwd} 是每站转发延迟（典型 1 µs），T_{master} 是主站开销（典型 20 µs）。

表 89-5 按这个式子算了五档从站数。100 站时线上 139.2 µs，可取 200 µs 周期；1000 站时线上 1183.2 µs，只能取 1.2 ms。这个结果解释了一个常见误解：EtherCAT 快，但不是无限快，从站数一多，每站 1 µs 的转发延迟就会累加成毫秒。

表 89-5 EtherCAT 周期随从站数的变化

从站数	线上字节	线上时间 / µs	可取周期 / µs
10	60	34.8	100
50	140	81.2	100
100	240	139.2	200
500	1040	603.2	700
1000	2040	1183.2	1200

图 89-7 把线上时间与可取周期画在双对数轴上。两条线在 100 站附近分开：线上时间还在 139 µs，可取周期已经被对齐到 200 µs。对齐是工程惯例，实际控制器大多只支持 100 µs、125 µs、250 µs 这样的整数档，不会给你 139 µs 这种周期。

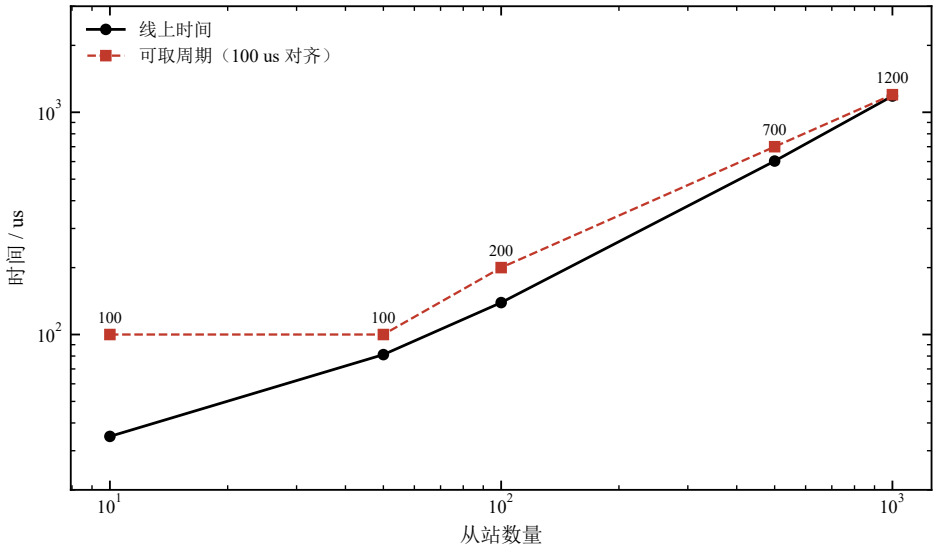


图 89-7 EtherCAT 周期随从站数的变化

从站数超过 500 之后，曲线明显变陡。原因是负载字节随站数线性增长，线上时间也跟着线性增长，而可取周期必须向上对齐，于是每加一档都要多等一截。1000 站时线上时间 1183 µs 已经逼近 1.2 ms 周期，利用率接近 99%，没有任何余量

留给非周期流量。

这就是为什么大型设备要分网段：把 1000 个站拆成两个 500 站的环，每个环 603 µs，都能跑 700 µs 周期，总周期反而比单环 1.2 ms 短。分段的代价是多一张网卡与一份主站授权。

89.2.4 延迟的四个来源

一帧数据从源站的应用层到目标站的应用层，要经过四段时间，它们的作用不同，不能简单相加了事。

传输时间是帧长除以速率，见式 (89-1) 至 (89-4)。它由协议与负载决定，是确定的部分。

排队时间是帧在交换机或主站缓冲里等待的时间。它由链路利用率与流量突发性决定，是随机部分的主要来源。

处理时间是协议栈打包、解包、校验的时间。硬件协议栈（如 EtherCAT 从站控制器）通常小于 1 µs，软件协议栈（如 Linux 上的 TCP/IP）可能到几十微秒。

传播时间是电信号在电缆上的飞行时间，约 5 ns/m。100 m 电缆 0.5 µs，通常可以忽略；但在长距离干线或跨厂区光纤上，它会变成可观的一项。

四段相加得到总延迟

$$T_{d} = T_{\mathrm{tx}} + T_{\mathrm{q}} + T_{\mathrm{proc}} + T_{\mathrm{prop}} \tag{89-5}$$

工程上真正难估的是 T_{q}。前三项都能从手册查到，T_{q} 只能靠排队论估算或现场测量。

89.2.5 排队与利用率

把交换机的一个出口看成一个服务台：帧按到达率 λ 进来，按服务率 μ 出去，利用率 $\rho = \lambda / \mu$。帧长固定时，排队系统是 M/D/1，平均排队时间为

$$W = \frac{\rho}{2\,(1-\rho)}\,T_{s} \tag{89-6}$$

其中 $T_{s} = 1/\mu$ 是一帧的服务时间。

这个式子的形状比数值更重要。ρ 从 0.5 涨到 0.9，分子翻倍、分母缩到五分之一，排队时间涨 9 倍。表 89-6 给出服务时间 1 ms 时的结果：利用率 0.5 时平均排队 0.50 ms，0.9 时 4.50 ms，0.95 时 9.50 ms。

利用率	平均排队 / ms	尾部估计 / ms
0.20	0.12	0.50
0.40	0.33	1.33
0.50	0.50	2.00
0.60	0.75	3.00
0.70	1.17	4.67
0.80	2.00	8.00
0.90	4.50	18.00
0.95	9.50	38.00

表 89-6 排队延迟随链路利用率的变化（M/D/1，服务时间 1 ms）

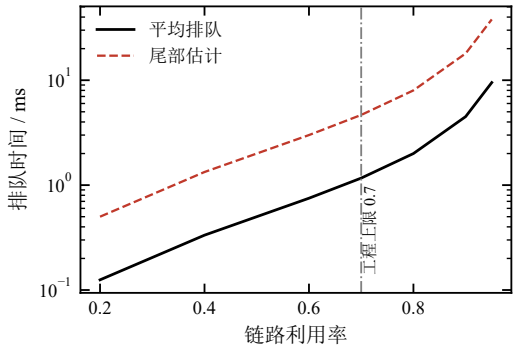


图 89-4 排队延迟随利用率的变化

图 89-4 把平均排队与尾部估计画在一起，纵轴是对数。两条线在 0.5 以下几乎重合，0.8 之后迅速分开。工程上把 0.7 当作上限（图中的竖线），理由是超过 0.7 之后，尾部延迟开始以“几倍平均”的速度增长，而控制回路关心的是尾部而不是平均。

“尾部估计”那一列是按平均值 4 倍粗估的，不是严格的分位数。严格算法要用排队论的尾概率公式，工程上没必要：留 30% 余量比精确算尾部更省事，也更保险。

89.2.6 延迟如何进入回路

网络延迟加在回路里，作用与过程纯滞后相同。它在穿越频率处产生的相位损失为

$$\Delta\varphi = \omega_c T_d \tag{89-7}$$

其中 ω_c 是穿越频率（rad/s），T_d 是总延迟（s）。这个式子是本章最重要的一条，因为它把“毫秒”换算成了“度”。

慢回路的 $\omega_c = 1.00$ rad/s，100 ms 延迟只损失 5.7°；快回路的 $\omega_c = 66.6$ rad/s，同样 100 ms 损失 381°，早就转了好几圈。这就是 §89.1.1 第二个现象的定量解释。

把式 (89-7) 展开到完整的相位表达式，可以得到含网络延迟的裕度公式

$$\varphi_m = 180^\circ - \arctan\left(\omega_c \tau\right) - \arctan\frac{1}{\omega_c T_i} - \omega_c\left(L + \frac{T_{\text{scan}}}{2} + T_{\text{net}}\right) \tag{89-8}$$

式中的 L 是过程纯滞后，$T_{\text{scan}}/2$ 是采样引入的半拍延迟（见第 85 章），T_{net} 是网络单向延迟。四项里只有最后一项是本章新增的。

89.2.7 丢包与保持

帧丢了之后执行器怎么办？工业上几乎所有的默认做法是“保持上一拍的值”，即

$$u(k) = u(k-1) \tag{89-9}$$

这条规则隐含一个假设：上一拍的控制量在当前仍然大致正确。对慢过程，这个假设成立；对快过程，保持 20 ms 可能已经错过了整个动态。

保持的后果可以分两种情况看。对稳定过程，短时保持只让回路短暂开环，误差缓慢积累，恢复后积分项会把它补回来。对不稳定过程（如倒立摆、某些放热反应），保持期间状态按开环极点发散，丢包长度直接决定能否拉回。

单帧丢包的危害常被高估，连续丢包的危害常被低估。5% 的单帧丢失让 IAE 从 0.0455 涨到 0.0458，几乎看不出；同样 5% 的丢失率，如果每次连丢 20 帧，IAE 涨到 0.0703，增加 53.5%，超调从 33.7% 抬到 43.8%（表 89-12）。决定后果的是“连续丢多久”，不是“总共丢多少”。

89.2.8 时间同步的误差

多轴协同、分布式采集、事件顺序记录，都要求各站的时钟一致。同步误差由两部分组成：每次同步的残差，以及两次同步之间的漂移累积。

$$\varepsilon = \varepsilon_{\text{sync}} + \Delta f \, T_{\text{sync}} \tag{89-10}$$

其中 $\varepsilon_{\text{sync}}$ 是单次同步误差，Δf 是晶振相对频率偏差（ppm），T_{sync} 是同步间隔。

普通晶振的偏差约 50 ppm，即每秒钟漂 50 μs，一天漂 4.32 s。这个数字说明“不同步”在工业现场不是小问题：两台设备各自上电运行一天，时间戳能差出好几秒，事件顺序完全无法还原。

表 89-7 对比了五种同步方式。NTP 的单次误差 5 ms，一天最大偏差 5.05 ms；SNTP 在局域网内能到 1 ms；软件时间戳的 PTP 约 100 μs；硬件时间戳的 PTP 到 1 μs，一天 51 μs。

同步方式	单次误差 / μs	一天最大偏差
无同步（仅晶振）	0	4.32 s
NTP	5000	5.05 ms
SNTP（局域网）	1000	1.05 ms
PTP（软件时间戳）	100	150 μs
PTP（硬件时间戳）	1	51 μs

表 89-7 五种时间同步方式的误差（晶振 50 ppm，同步间隔 1 s）

表里“一天最大偏差”一列把漂移算进去了，所以 PTP 硬件时间戳不是 1 μs 而是 51 μs。这个细节容易被忽略：同步精度再高，只要间隔是 1 s，两次同步之间仍会漂 50 μs。要把日偏差压到微秒级，同步间隔必须缩到 20 ms 以内。

89.2.9 带宽与刷新率

带宽需求由三个数决定：节点数 N、每节点每周期字节数 L、周期频率 f_{cyc}。

$$B_{\text{req}} = N \, L \, f_{\text{cyc}} \times 8 \tag{89-11}$$

除以链路速率得到利用率

$$\rho = \frac{B_{\text{req}}}{B_{\text{link}}} \tag{89-12}$$

这条式子只算周期流量。工程上还要给非周期流量（参数下载、诊断、报警）留余量，通常按周期流量占 30% 估算，剩下 70% 留给其他用途。

表 89-8 按“周期通信占 30%”算了各类总线在最小周期下的过程数据量。EtherCAT 在 100 μs 周期下 375 字节，Profinet IRT 在 250 μs 下 938 字节，Modbus RTU 在 10 ms 下只有 43 字节。

表 89-8 各类总线的周期、节点数与过程数据容量

总线	最小周期 / ms	最大节点	每周期过程数据 / 字节
Modbus RTU	10	32	43
CAN 2.0A	1	110	38
Modbus TCP	10	64	37500
Profinet RT	1	128	3750
Profinet IRT	0.25	128	938
EtherCAT	0.1	1000	375

这张表要横着看，不能只看一行。Modbus TCP 在 10 ms 下能带 37500 字节，看着比 EtherCAT 多得多，但它的周期是 10 ms，抖动是毫秒级，做不了快回路；EtherCAT 在 100 µs 下只有 375 字节，看着很少，但每 100 µs 就有 375 字节，折算成吞吐是 30 Mbit/s，而且抖动小于 1 µs。

当一帧装不下所有数据时，要分帧轮发。若把 N 个节点的数据分成 n 帧轮发，每个节点的刷新周期变成

$$T_{\text{refresh}} = n\,T_{\text{cyc}} \tag{89-13}$$

刷新率降到 $1/n$，但每帧的负载也降到 $1/n$，链路利用率不变。这个交换在 §89.9.2 会展开：什么时候该分帧，什么时候该加带宽，判据是“回路的采样周期”而不是“链路的吞吐”。

89.2.10 延迟四段的实测拆分

式 (89-5) 把延迟分成四段，工程上要能把它们拆开，否则不知道该改哪一段。

用 64 轴包装线的数字做例子。单周期 188.5 µs 里，线上传输 100.5 µs 占 53%，从站转发 68 µs 占 36%，主站处理 20 µs 占 11%，传播 0.3 µs 占 0.2%。

四段的占比决定了优化顺序。转发占三成六，说明“站太多”是主要成本；线上占五成三，说明“数据太大或带宽太低”。两段加起来近九成，主站与传播的优化空间很小。

拆分方法有三步。第一步，只接一个从站跑空帧，测出的周期就是“主站 + 线上 + 一次转发”，用它减去空帧的理论线上时间，得到主站处理时间。第二步，逐步加站，每加一站周期增加的量就是单站转发时间。第三步，把数据量加倍，周期的增量就是线上时间的增量。

三步测完，四段就都有了。若第一步测不出来（主站处理时间太小，被抖动淹没），可以直接按经验估：软主站 20 µs 量级，硬主站 5 µs 以内。

拆分之后再看优化。转发时间长，减站或换更快的从站芯片；线上时间长，加带宽或压数据；主站时间长，换硬主站；传播时间长（几乎不会发生），只能缩短距离。

这段拆分的意义在于避免“一把抓”。现场常说“网络慢”，拆开之后往往发现是某一段占了大头，改那一段就够了，不必换整套。

89.3 设计步骤

89.3.1 先把四个数写下来

设计一条工业网络，动手画拓扑之前，必须先有四个数：节点数、每节点每周期要传多少字节、每个回路要求的刷新周期、最长电缆距离。这四个数缺一个，后面的计算就只能“先按经验估”，而经验恰恰是本章想替换掉的东西。

四个数里最容易漏的是“每节点字节数”。一台伺服驱动器要传的不只是位置给定与位置反馈，还有状态字、控制字、转矩、电流、报警码、诊断码。§89.3.2 的表 89-9 给了常见站型的字节数，查表比逐台翻手册快。

图 89-9 把五个步骤串成一条判断链。前两步是算数，第三步开始有取舍，第四步看现场条件，第五步是复核。任何一步算出来不合，回到上一步换方案，而不是硬着头皮往下做。

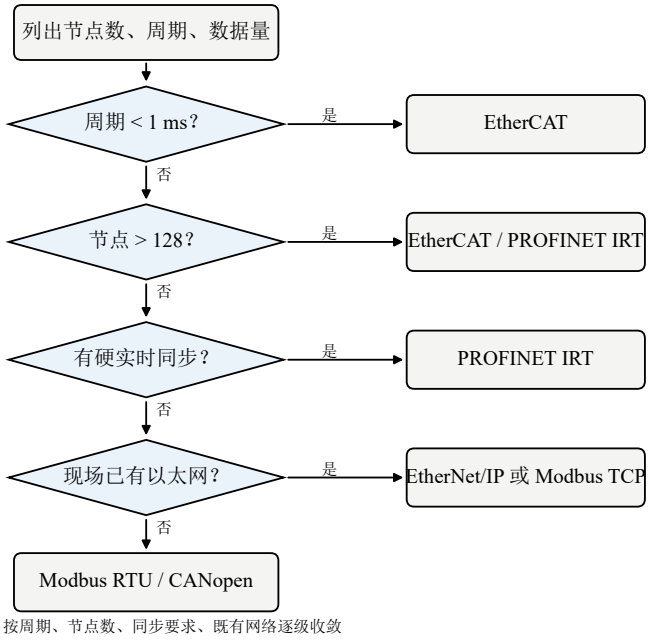


图 89-9 工业网络选型判断流程

流程里有两个回退箭头值得注意。带宽不够时先考虑降周期而不是换总线，因为换总线意味着换主站、换从站、换组态软件；同步精度不够时先缩短同步间隔，再考虑上硬件时间戳。

89.3.2 第一步：列节点清单

节点清单不是设备清单。设备清单按采购单元列，节点清单按“每周期要交换的数据块”列。同一台伺服驱动器可能占两个节点（一个过程数据通道、一个诊断通道），也可能只占一个但字节数较大。

表 89-9 是一份可用的清单模板。六列覆盖了后面所有计算需要的信息：周期字节数供带宽计算，允许刷新周期供分帧判断，是否需要同步供时钟设计，失效影响供拓扑选择。

表 89-9 节点清单模板（每行是一类站型，实际项目按台数展开）

站型	每周期过程数据 / 字节	典型刷新周期 / ms	需要时钟同步	失效影响	备注
伺服驱动器（位置模式）	16	1	是（多轴插补）	单轴停	含状态字、控制字、位置、转矩
伺服驱动器（转矩模式）	12	0.5	否	单轴停	高频，常单独成环
远程 I/O（16 点）	2	10	否	局部失控	可降级为保持
变频器	8	10	否	单机停	含频率给定、电流、故障码
智能仪表（温度）	4	100	否	单点失准	本身响应慢
安全模块	4	10	是（事件顺序）	联锁动作	需安全认证链路
视觉与扫码	64	20	是（时间戳对齐）	判废	非周期特征明显
人机界面	32	100	否	仅监视	可走非周期通道

清单里“失效影响”一列常被当作装饰，实际它决定拓扑。单轴停可以容忍线型（一个站坏不影响其余），联锁动作不能容忍环型（断一环全网重组，重组期间所有站失联）。这一列在后面两步还会再用。

列清单时还有一个容易忽略的量：诊断数据。现代伺服驱动器的诊断对象动辄上百字节，这些数据不该进周期通道。把它们放到非周期通道（邮箱、参数通道），周期负载能降一大截。

89.3.3 第二步：算周期与带宽

有了清单，带宽是一行乘法：

$$B_{\text{req}} = \sum_{i} N_i L_i f_i \times 8 \tag{89-14}$$

这是把式 (89-11) 按站型展开，因为同一网络上的站刷新周期往往不同。N_i 是第 i 类站的台数，L_i 是每周期字节数，$f_i = 1/T_i$ 是刷新频率。

算出来的 B_{req} 不能直接与链路速率比。链路还要承载帧头、前导码、帧间隙、非周期流量，实际可比的是利用率 ρ，工程上限取 0.7（§89.2.5）。

以 64 轴包装线为例。64 台伺服每台 16 字节，96 点远程 I/O 每点 2 字节，过程数据 1216 字节。EtherCAT 一帧装完，帧长 1256 字节，含前导与帧头 8 字节、以太网头 14 字节、工作计数器 2 字节、CRC 4 字节、帧间隙 12 字节。

表 89-10 64 轴包装线的 EtherCAT 带宽预算

分项	数值	说明
伺服过程数据	1024 字节	64 台 × 16 字节
远程 I/O 过程数据	192 字节	96 点 × 2 字节
帧开销	40 字节	前导 8 + 以太网头 14 + 计数器 2 + CRC 4 + 间隙 12
帧长合计	1256 字节	一帧装完，不分帧
线上传输	100.5 μs	1256 × 8 / 100 Mbit/s
从站转发	68 μs	68 站 × 1 μs
主站处理	20 μs	组帧与解析
单周期合计	188.5 μs	线上 + 转发 + 主站

188.5 μs 是这条线的地板。取 1000 μs 周期占 18.8%，取 500 μs 占 37.7%，取 250 μs 占 75.4%。前两档都在 0.7 以内，第三档超了。

所以这条线的周期只能在 500 μs 与 1000 μs 之间选。选哪个不由带宽决定，由回路的采样周期决定：包装机的色标跟踪要求 2 ms 采样，500 μs 是 4 倍过采样，够用；换成 1000 μs 只有 2 倍，色标相位会有半个周期的模糊。

这段计算有两点要记住。转发时间按每站 1 μs 估，实际取决于从站芯片，老式控制器可能 2 到 3 μs；主站处理 20 μs 是软主站（基于通用网卡）的量级，硬主站（专用芯片）能压到 5 μs 以内。

89.3.4 第三步：定拓扑

拓扑由三件事决定：距离、故障域、布线成本。

线型（Modbus RTU、CAN、Profibus DP）最省线，一条干线挂到底，适合从站分散但每站数据量小的场合。缺点是干线一处断，后面全掉；终端电阻少一个，反射会让整条线时好时坏。

星型（交换式以太网）故障域最小，一个端口坏只掉一个站，适合设备集中、需要逐台诊断的场合。缺点是每个站一根线回交换机，线缆与端口成本随站数线性增长，64 轴就要 64 根线。

环型（EtherCAT 冗余、Profinet MRP）用一根额外的线换故障容忍，断一处仍能通信，代价是故障时周期变长，因为帧要绕远路。对不能停的产线，这个代价通常值得付。

图 89-3 已经画过这三种拓扑。这里补一条现场判据：站间距超过 100 m 时，不要用铜缆以太网硬撑，直接上光纤。光纤环网在冶金、港口、风电里是标准做法，不只是“高端选项”。

89.3.5 第四步：定时钟同步

不是所有网络都需要同步。表 89-9 里只有伺服插补、安全事件顺序、视觉时间戳三类标了“需要”。判据是“两个站的动作是否需要在一个共同时间轴上比较”。

单轴定位不需要同步，各轴用自己的编码器就够了。多轴插补需要，因为轨迹在同一时间轴上参数化，轴间 100 μs 的相位差会让圆弧变成椭圆。事件顺序记录也需要，因为要判断“急停先动还是安全门先开”，两站时钟差 1 ms，因果就说不清。

同步方式按精度选（表 89-7）。毫秒级用 NTP 就够，微秒级必须 PTP 或 EtherCAT 分布式时钟。选定方式后还要定同步间隔：按式 (89-10)，晶振 50 ppm 时，要把两次同步之间的漂移压到 10 μs，间隔不能超过 200 ms。

EtherCAT 的分布式时钟把参考时钟放在第一个支持该功能的从站上，其余从站测量自己与参考的偏差并补偿。补偿后的同步误差在 100 ns 量级，代价是从站需要支持分布式时钟的芯片，且参考站要选在拓扑靠前的位置。

89.3.6 第五步：留余量与写进文档

算完不等于做完，还要留三份余量。

第一份是带宽余量，$\rho \le 0.7$。这条线在 250 µs 下已经 75.4%，一旦加了诊断流量或扩两台轴就会超。现场表现是“偶发丢帧”而不是“完全不通”，最难查。

第二份是周期余量。设计的周期要比回路要求的最短采样周期再快 2 到 4 倍。只快 1.2 倍时，任何一次重传或一次调度抖动都会把刷新率拖到要求以下。

第三份是抖动余量。周期抖动要小于周期的 10%。500 µs 周期对应 50 µs 抖动上限，超过这个量，采样时刻的不确定性已经与半拍延迟相当（§89.2.6）。

三份余量都要写进设计文档，而不是留在工程师脑子里。文档里至少要有：节点清单（表 89-9 的形式）、带宽预算（表 89-10 的形式）、选定周期与理由、同步方式与间隔、验收实测的周期抖动与丢帧率。

验收指标建议写成可测的数字：连续 8 h 周期抖动小于 10%、丢帧率低于 10^{-6}、同步误差小于 1 µs。这三项在现场用抓包工具与示波器都能测，比“通信正常”这种描述有用得多。

89.3.7 一次完整的选型

把五步走一遍，仍以 64 轴包装线为例。

第一步列清单：64 台伺服（16 字节、1 ms）、96 点 I/O（2 字节、10 ms）、1 台视觉（64 字节、20 ms）、1 台人机界面（32 字节、100 ms）。按 500 µs 周期归一，伺服数据每拍都传，I/O 数据每 20 拍传一次。

第二步算带宽：过程数据 1216 字节，帧长 1256 字节，单周期 188.5 µs。500 µs 周期占 37.7%，1000 µs 占 18.8%。

第三步定拓扑：设备沿输送线分布，总长 60 m，单轴故障可停线，选线型。EtherCAT 的逻辑环在线型拓扑上自然成立，不额外做冗余。

第四步定时钟：多轴插补需要同步，选 EtherCAT 分布式时钟，参考站取第一台伺服，同步周期与通信周期一致，都是 500 µs。

第五步留余量：37.7% 的利用率不高，可以试着压到 400 µs；400 µs 下占 47.1%，加上诊断流量后接近 0.6，仍在 0.7 以内。最终取 500 µs，把余量留给后续扩轴。

结论是一句话：EtherCAT、线型、500 µs 周期、分布式时钟同步、利用率 37.7%。这句话足以让采购下单、让电气出图、让软件建档。

89.3.8 三个常见的估算错误

五步看着简单，实际做的时候有三个高频错误。

第一个是把平均延迟当延迟。手册给的常是平均值，而闭环由最坏值决定。10 ms 平均、20 ms 峰值的链路，按 10 ms 算出来的 30.9° 裕度是假的，实际要按 20 ms 算，那已经是 $-7.3°$。

第二个是只算线上时间，漏掉转发与主站处理。1256 字节在 100 Mbit/s 上只有 100.5 µs，看着很宽裕；加上 68 站转发 68 µs 与主站 20 µs，总时间变成 188.5 µs，几乎翻倍。按 100.5 µs 去算 250 µs 周期的利用率，会得出 40% 的错误结论，实际是 75.4%。

第三个是按峰值带宽而不是按利用率。链路的标称速率是峰值，能长期稳定使用的是它的七成。把 100 Mbit/s 当 100 Mbit/s 用，等于把利用率直接设计到 1.0，排队延迟按式 (89-6) 发散。

三个错误的共同点是“取了好看的那个数”：平均值比最坏值好看，线上时间比总时间好看，峰值带宽比可用带宽好看。估算时每取一个数，都要问一句“这是不是最坏情况下也成立”。

还有一个不算错误但常被忽略的量：诊断流量。它在调试期几乎为零，投运后逐渐增多，半年后可能占到周期流量的两成。按 0.7 留余量的方案能吸收这部分增长，按 0.9 设计的方案会在某个时间点突然不稳。

89.3.9 冗余要花多少

冗余的账要按“允许连丢几帧”来算，不能按“要不要冗余”来算。

64 轴包装线，周期 500 µs。若允许中断 1 ms，即连丢 2 帧，那么恢复时间必须小于 1 ms。EtherCAT 的环网冗余在一帧内恢复，满足；介质冗余协议的 200 ms 恢复，等于连丢 400 帧，完全不满足。

若同为这条线，把周期放宽到 10 ms，200 ms 的恢复时间就是连丢 20 帧。按表 89-12，连丢 20 帧的后果是超调从 33.7% 抬到 43.8%、调节时间翻倍，能接受但不舒服。这时介质冗余才进入可选范围。

所以冗余方案的适用性随周期变化。同一个方案在慢回路上够用，在快回路上不够用，差别不在方案本身，而在“恢复时间折成多少拍”。

成本也要按这个逻辑算。EtherCAT 冗余的代价是一根额外的线、每站两个网口、支持冗余的从站；介质冗余的代价是一台支持 MRP 的交换机与环网配置。前者贵在从站，后者贵在交换机。

三条判断顺序：先算允许连丢几帧，再看候选方案的恢复时间折成几帧，最后比成本。跳过第一步直接比价格，买回来的往往是“能冗余但恢复太慢”的方案，等于没冗余。

89.4 连续情形

89.4.1 把网络折成一段纯滞后

连续域里没有“帧”，也没有“周期”，网络只剩下一个效果：信号晚到。晚到的时间就是延迟，写成传递函数是 e^{-sT_d}。

这个化简有三条前提。延迟是常数，不随时间变；延迟对所有频率一视同仁；信号在传输中没有失真。三条前提在工业网络上大致成立，因为帧不会因为内容不同而走得快些或慢些。

有三件事被这个化简丢掉了：采样、量化、丢包。丢包在 §89.2.7 单独处理，采样与量化留到 §89.5。所以本节的所有结论都对应“延迟确定、不丢包”这一个理想场景。

这个理想场景不是空想。EtherCAT 与 Profinet IRT 在正常运行时，抖动小于 1 µs，延迟几乎是常数。对这类网络，纯滞后模型不是近似，而是相当准确的描述。

89.4.2 纯滞后的频响

把 $s = j\omega$ 代入 e^{-sT_d}，得到

$$G_d(j\omega) = e^{-j\omega T_d}, \qquad |G_d| = 1, \quad \angle G_d = -\omega T_d \tag{89-15}$$

幅值恒为 1，相位与频率成正比。这两个性质决定了延迟的全部危害：它不衰减信号，只让信号“转圈”。

幅值为 1 意味着延迟本身不会让回路失稳，它只是把裕度一点点吃掉。所以判断“这段延迟要不要紧”不能看延迟的绝对值，要看它在穿越频率处折合多少度。

相位的线性也解释了一个常被误解的现象：延迟对低频几乎无害，对高频极其有害。同样 100 ms，在 1 rad/s 处折 5.7°，在 66.6 rad/s 处折 381°。后者已经绕了一圈多，闭环从负反馈变成了正反馈。

89.4.3 两条裕度曲线

图 89-5 把式 (89-8) 在两组对象参数下画了出来。横轴是网络单向延迟，纵轴是相位裕度，两条曲线分别是慢回路（温度，时间常数 2 s）与快回路（伺服速度环，时间常数 50 ms）。

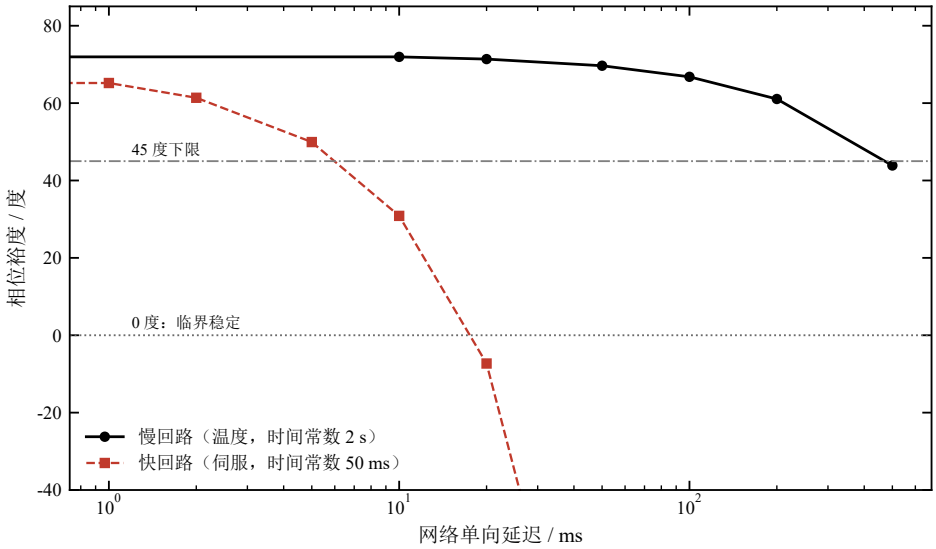


图 89-5 网络延迟对相位裕度的影响

慢回路从 72.5° 起步，500 ms 延迟时仍有 43.9°。整条曲线平缓，因为穿越频率只有 1.00 rad/s，500 ms 只折 28.6°。

快回路从 69.0° 起步，5 ms 就掉到 49.9°，10 ms 剩 30.9°，20 ms 变成 $-7.3°$。曲线的斜率是慢回路的 66 倍，这个倍数正是两个穿越频率之比。

曲线与 45° 参考线的交点是可以容忍延迟的工程上限。慢回路在 480 ms 处穿过 45°，快回路在 6.3 ms 处穿过。这两个数才是“这条网络能不能用”的直接答案。

89.4.4 穿越频率不随延迟移动

一个反直觉的事实：加延迟不会改变穿越频率。原因是穿越频率由“回路增益等于 1”决定，而纯滞后的幅值恒为 1，加多少延迟都不影响幅频特性。

这条性质有实际用处。调试时若发现加了网络之后系统变差，先量延迟，再算 ω_c，两者的乘积就是损失的度数。不要因为“变差了”就去降增益，降增益会把穿越频率推低，响应变慢，但延迟的度数损失并不会因此减少同样的比例。

真正能减小度数损失的办法只有两个：降低穿越频率（牺牲快速性），或者减小延迟（换网络、缩短周期、把控制器下移）。前者是控制手段，后者是通信手段，本章关心的是后者。

89.4.5 反解：给定裕度能容忍多少延迟

把式 (89-8) 中的 T_{net} 解出来，得到可容忍延迟

$$T_{\text{net}}^{\max} = \frac{180^\circ - \arctan(\omega_c \tau) - \arctan\frac{1}{\omega_c T_i} - \omega_c\left(L + \frac{T_{\text{scan}}}{2}\right) - \varphi_m^{\text{req}}}{\omega_c} \tag{89-16}$$

取 $\varphi_m^{\text{req}} = 45^\circ$，慢回路得 480 ms，快回路得 6.3 ms。这个式子在做方案时比正向计算更有用：先定裕度要求，再倒推网络指标，采购就有依据了。

反解时要注意单位。分子里的三项都是度，ω_c 是 rad/s，所以算出来的 $T_{\text{net}}^{\max}$ 单位是秒，不是度。工程上常犯的错是把 ω_c 当成 Hz 代入，结果差 2π 倍。

还有一个隐含条件：$T_{\text{net}}^{\max}$ 必须为正。若对象本身的纯滞后与采样延迟已经把裕度吃到 45° 以下，式子给出负值，含义是“这条回路在任何网络上都不满足要求”，此时要改对象或改整定，而不是换网络。

89.4.6 连续模型什么时候够用

连续模型的前提是“延迟远大于采样周期”，此时延迟主导相位损失。反过来，当延迟与采样周期同量级时，采样与保持效应占主导，必须用 §89.5 的离散模型。

一个粗略的判据是 $T_d > 3T_s$。慢回路 $T_s = 10$ ms，延迟 100 ms，比值 10，连续模型够用；快回路 $T_s = 1$ ms，延迟 5 ms，比值 5，也够用；若某条回路的延迟只有 0.5 ms 而采样周期 1 ms，比值 0.5，连续模型会低估相位损失。

连续模型还有一个便利：它把“网络”与“对象”放在同一个框架里，可以直接用第 44 章的裕度工具分析，不必先离散化。做方案估算时先用它，做投运校核时再上离散模型，这个顺序最省时间。

传播延迟值得单独提一句。电磁波在铜缆中的速度约 2×10^8 m/s，折合 5 ns/m。60 m 的输送线，传播延迟 0.3 μs，与 1 μs 的转发延迟相比可以忽略。所以工业网络的延迟几乎全部来自“处理”与“排队”，不来自“距离”。

89.4.7 增益裕度受延迟影响更深

延迟不影响幅频特性，所以穿越频率不动。但它把 $-180°$ 的相位交点往低频挪，而低频处的增益更高，于是增益裕度反而下降得更快。

以快回路为例。$\tau = T_i = 50$ ms 时，对象与积分器的相位在任意频率下都合计 $-90°$（$\arctan \omega\tau$ 与 $\arctan \frac{1}{\omega T_i}$ 互补）。剩下的相位由延迟贡献，$-180°$ 交点满足 $\omega(0.0055 + T_{\text{net}}) = \pi/2$。

无网络延迟时，交点在 285.6 rad/s，该处 $|L| \approx K_c/(\omega\tau) = 0.233$，增益裕度 4.29 倍，即 12.6 dB。

10 ms 延迟时，交点左移到 101.3 rad/s，$|L|$ 升到 0.657，增益裕度只剩 1.52 倍，即 3.6 dB。相位裕度从 69.0° 掉到 30.9°，掉了 38°；增益裕度从 12.6 dB 掉到 3.6 dB，掉了 9 dB。

20 ms 延迟时，交点左移到 61.6 rad/s，$|L|$ 升到 1.081，增益裕度变成 -0.7 dB，也就是 1 倍以下。这与相位裕度 $-7.3°$ 的结论一致：回路已经失稳，只是还在限幅内摆动。

这段计算说明一件事：看相位裕度会低估延迟的危害。同样 10 ms，相位裕度还剩 30.9°，看着“还能用”；增益裕度只剩 3.6 dB，意味着增益再涨 1.5 倍就振。现场的增益往往因为产品更换、参数漂移而上浮，这条余量比相位裕度更早耗尽。

两个裕度要一起看，取小的那个当判据。相位裕度管“响应过不过冲”，增益裕度管“参数飘不飘得起”，两者回答的是不同的问题。

89.4.8 网络延迟与对象滞后为什么不能混为一谈

两者的频响完全一样：都是 e^{-sT}，幅值为 1、相位线性。在式 (89-8) 里也确实合在一起，写成 $L + T_{\text{scan}}/2 + T_{\text{net}}$。既然数学上一样，为什么要分开说？

因为可改性不同。对象纯滞后是工艺决定的：物料从入口走到出口要 30 s，这个 30 s 改不了，除非改工艺。网络延迟是配置决定的：换主站、缩周期、减少级数，都能改。

因为稳定性不同。对象滞后随工况漂：产量变了、物料变了，滞后跟着变；网络延迟在一次运行内基本恒定，除非有人在旁边拷文件。

还因为整定的含义不同。对象滞后要求“更保守的整定”，因为它是系统本身的性质；网络延迟要求“更好的网络”，因为它是外加的短板。把两者混在一起，工程师会习惯性地用降增益去对付本可以修掉的那部分。

分开的方法很简单：整定时合并计算，因为回路只认总滞后；优化时分开处理，先看哪一部分可改。本章的 T_{net} 与 L 在式子里是相加的，但在决策上是分开的。

89.5 离散情形

89.5.1 离散化带来三个新东西

真实控制器是离散的。一拍里做三件事：采样输入、计算控制量、把输出写进发送缓冲。网络延迟在这个序列里落在“写缓冲”与“对端读到”之间。

离散化因此带进三个连续域没有的东西。延迟被量化成整数拍，抖动让这个整数拍变来变去，采样与保持本身又贡献半拍滞后。

三者的量级不一样。采样滞后固定为 $T_s/2$，可以算准；延迟量化误差最大 $T_s/2$，也可以算准；抖动是随机的，只能按分布或最坏值处理。工程上真正难对付的是第三个。

89.5.2 延迟的整数拍表示

把单向延迟除以采样周期并取整，得到延迟拍数

$$d = \left\lceil \frac{T_{\text{net}}}{T_s} \right\rceil, \qquad z^{-d} \text{ 表示延迟} \tag{89-17}$$

取上整而不是四舍五入，是因为“晚到”比“早到”安全：算成 10 拍而实际 9 拍，闭环多用一拍信息，最多慢一点；算成 9 拍而实际 10 拍，闭环会用到还没到的数据。

量化误差最大 $T_s/2$，相对误差是 $T_s/(2T_{\text{net}})$。慢回路 $T_s = 10$ ms、$T_{\text{net}} = 10$ ms，相对误差 50%，一拍的含义在 5 ms 到 15 ms 之间浮动；快回路 $T_s = 1$ ms、$T_{\text{net}} = 10$ ms，相对误差 5%，十拍就是十毫秒，很准。

这条对比说明一件事：采样周期相对延迟越大，离散模型越粗糙。慢回路用 10 ms 采样去描述 10 ms 延迟，等于用米尺量毫米。

89.5.3 抖动让延迟拍数变成随机变量

抖动存在时，d 不再固定。同一段网络，这一拍是 10 拍，下一拍可能是 12 拍。闭环的等效对象每拍都在变，严格说这是一个时变系统，没有固定的传递函数。

工程上有两种处理办法。按最坏值处理，取 $d_{\max}$ 设计，保守但简单；按平均值加裕度处理，用 $\bar{d}$ 算，再留 20% 到 30% 的裕度。前者适合安全相关回路，后者适合一般过程回路。

两种办法都掩盖了一个事实：抖动的主要危害不是“平均慢了多少”，而是“每拍的时序关系被打乱”。控制器按固定节拍设计的积分与微分，遇到变长的延迟，等效于积分时间在抖，微分项更敏感。

89.5.4 同一段延迟，两种离散后果

把 10 ms 延迟放进两个回路，离散后的差异比连续域更明显。

慢回路 $T_s = 10$ ms，延迟 1 拍。相对于 2 s 的时间常数，一拍只占 0.5%，可以忽略。仿真的闭环指标是超调 0.2%、调节时间 2.59 s、IAE 1.023，与理想情况（2.65 s、1.020）相比，调节时间甚至略短，因为一拍延迟让控制器动作稍微“钝”了一点，反而压住了小幅振荡。

快回路 $T_s = 1$ ms，延迟 10 拍。相对于 50 ms 的时间常数，十拍占 20%，已经不能忽略。超调从 7.7% 涨到 33.5%，调节时间从 0.132 s 拉到 0.225 s，IAE 涨 77.7%。

差别不在延迟的绝对值，而在“延迟占时间常数的比例”。慢回路 10 ms 对 2 s 是 0.5%，快回路 10 ms 对 50 ms 是 20%，差 40 倍。

89.5.5 抖动下的阶跃响应

图 89-6 画了快回路的三条阶跃响应：理想（无延迟）、固定 10 ms 延迟、0 到 20 ms 均匀抖动。三条曲线共用同一组 PI 参数，只改网络条件。

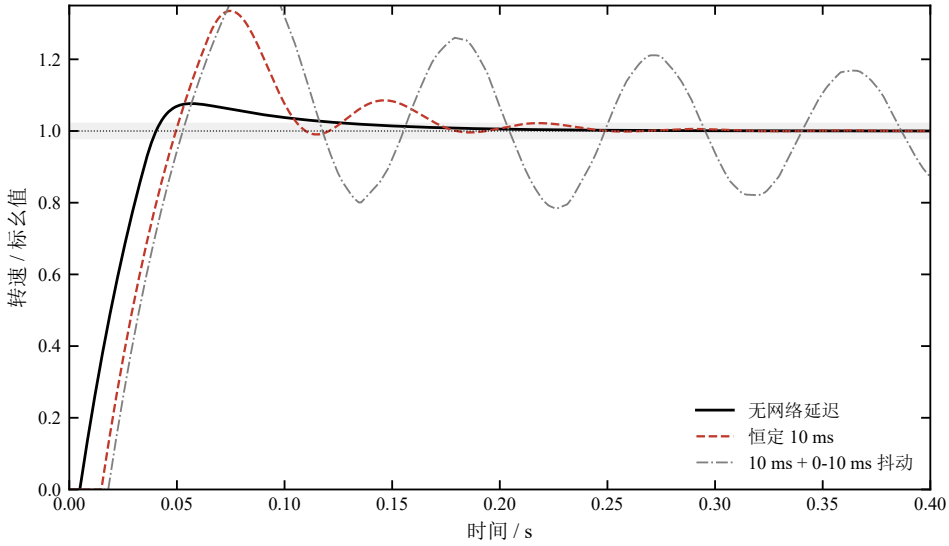


图 89-6 网络延迟与抖动下的阶跃响应

理想曲线在 0.13 s 内进入 2% 带，超调 7.7%。固定 10 ms 的曲线超调 33.5%，第一个峰明显抬高，之后有两次小幅回摆，这是延迟把积分作用推后的典型形态。

抖动曲线的超调 56.1%，调节时间 0.499 s，已经接近 0.5 s 的仿真窗口末端。它的形态不是“更慢”，而是“更乱”：峰值位置每次运行都不同，重复性很差。

这三条曲线回答了一个常见疑问：为什么现场“平均延迟 10 ms”的回路，表现比实验室“固定 10 ms”差得多。原因是抖动把等效延迟的分布拉宽，闭环在部分拍上工作在大延迟状态，而超调由最不利的那几拍决定。

89.5.6 采样周期怎么配

采样周期与网络延迟要匹配。经验规则是 $T_s \le T_{\text{net}}/3$，即一个延迟至少跨三拍。这样延迟的整数拍表示才够细，抖动的影响也被平均掉一部分。

按这条规则，10 ms 延迟要求 $T_s \le 3.3$ ms。慢回路用 10 ms 采样，其实不满足，只是因为它对延迟不敏感才没事；快回路用 1 ms 采样，满足得很宽裕。

另一条规则来自对象：$T_s \le 0.1\tau$。慢回路 $\tau = 2$ s，要求 $T_s \le 200$ ms，10 ms 采样绰绰有余；快回路 $\tau = 50$ ms，要求 $T_s \le 5$ ms，1 ms 采样也够。

两条规则取严的那个。慢回路取 3.3 ms，但它实际用 10 ms 也没问题，说明“延迟不敏感”这个性质可以突破规则；快回路两条都满足，没有矛盾。

89.5.7 多速率与分帧的离散后果

一条网络上跑着不同刷新率的站时，控制器看到的是“多速率采样”。慢站的更新要等好几拍才有一次新值，中间几拍要么保持、要么用模型外推。

§89.2.9 的分帧式 (89-13) 在这里有了离散解释：分 n 帧轮发等于把该站的采样周期拉长到 nT_{cyc}。对慢站无害，对快站等于人为加了延迟。

所以分帧要按站型分，不能全网统一。把伺服数据与温度仪表放进同一帧轮发，是最常见的错误设计：为了迁就仪表而拉长伺服刷新，或者为了伺服而浪费带宽在仪表上。

89.5.8 用特征方程校核一次

相位裕度是连续域的判据，离散域的终审要用特征方程。做法不复杂，四步。

第一步，把对象离散化。一阶对象加纯滞后，用零阶保持器展开，得到 $G(z)$。第二步，把控制器离散化，PI 的离散形式在第 44 章与第 106 章已有标准写法。第三步，写出闭环特征方程 $1 + C(z)G(z)z^{-d} = 0$。第四步，对 $d = 1$ 到 20 逐个求根，看最大极点模是否小于 1。

四步里最容易被跳过的是第四步的“逐个”。d 不是连续变量，不能只算一个值再插值；稳定性随 d 的变化不是单调的光滑曲线，某些 d 上会因为相位绕圈而出现“稳定—不稳—再稳定”的交替。

本章的两个回路不需要算到 20 拍。慢回路 $d = 1$，极点模远小于 1；快回路在 $d = 10$ 时仍在圆内，$d = 20$ 时越过单位圆，与相位裕度 $-7.3°$ 的结论一致。

特征方程法的价值在于它能处理“相位裕度为负但仍未发散”的区间。此时连续判据说“不稳”，实测却看到“在限幅内摆动”，两者都对，说的是不同的事：判据说线性模型不稳，实测说的是被限幅与饱和救回来了。

校核时还要注意离散化方法。零阶保持、双线性、冲激响应不变，三者在低频一致，在接近奈奎斯特频率处差别很大。ω_c 远低于奈奎斯特频率时（本章两个回路都满足），三者差别可以忽略；ω_c 接近奈奎斯特频率时，必须用与实现一致的方法。

89.5.9 抖动的单位换成拍

抖动用毫秒描述不直观，换成“拍”就清楚了：J/T_s 是延迟拍数的浮动范围。

同一段 0 到 20 ms 的抖动，慢回路 $T_s = 10$ ms，浮动 2 拍；快回路 $T_s = 1$ ms，浮动 20 拍。前者相当于“延迟在 1 到 3 拍之间跳”，后者相当于“延迟在 10 到 30 拍之间跳”。

这个换算解释了表 89-11 里那个反差：慢回路的抖动工况 IAE 只涨 0.6%，快回路涨 523%。两者的抖动绝对值相同，换算成拍数差了 10 倍，而闭环对拍数的敏感度又随 $\omega_c T_s$ 放大。

换算之后，抖动的判据也清楚了：浮动超过 3 拍时，等效延迟的方差已经与平均值同量级；超过 10 拍时，闭环相当于在一个宽分布的延迟上运行，任何固定参数的整定都只能对其中一部分工况有效。

工程上可以用这个判据做快速估计：量出抖动的毫秒数，除以采样周期，看落在哪一档。落在 3 拍以内，可以当噪声处理；落在 10 拍以上，必须先降抖动，再谈整定。

这条判据还有一个用处：选采样周期时反过来用。若抖动 2 ms 降不下来，而回路能接受 3 拍以内，采样周期就不能小于 0.67 ms。这比“经验取 1 ms”更有依据。

89.6 稳定性与性能

89.6.1 判稳定的三个层次

网络化回路的稳定性要在三个层次上看，越往下越接近现场。

第一层是相位裕度，用式 (89-8) 算，最粗也最快。它假设延迟固定、采样理想、不丢包，给出的是“设计层面能不能过”。

第二层是离散闭环极点，把 z^{-d} 代进闭环特征方程，看极点是否都在单位圆内。它比相位裕度准，能处理采样与量化，但要求延迟是整数拍、不随时间变。

第三层是时域仿真或实测，允许抖动、丢包、饱和、量化同时存在。它最接近现场，但结论只对跑过的工况成立，不能外推。

三层的关系是逐级校核，不是逐级替代。用第一层做方案，第二层做投运前校核，第三层做验收。跳过任何一层都会留下盲区。

89.6.2 失稳边界在哪里

按式 (89-16)，快回路在 6.3 ms 处还有 45° 裕度，10 ms 剩 30.9°，20 ms 变成 $-7.3°$。负裕度的含义是闭环已经失稳，只是还没发散到限幅。

仿真的结果与这条界线吻合。快回路在 100 ms 延迟下，超调 98.0%，调节时间顶到 0.500 s 的仿真窗口末端，IAE 涨到理想的 14.3 倍。波形不收敛，一直在 2% 带外摆动。

慢回路在同样的 100 ms 下只是超调从 0.2% 涨到 1.4%，IAE 涨 6.1%。同一条界线，对一个回路是灾难，对另一个回路只是轻微退化。

所以“这段网络稳不稳”这个问题没有统一答案。要问的是“对哪条回路、在什么延迟下”。裕度式给出的是换算关系，不是结论。

89.6.3 两组闭环指标

表 89-11 是五档工况的仿真结果，慢回路与快回路各一组，每组三个指标：超调、调节时间、IAE（相对理想值的百分比）。

表 89-11 网络条件对两个回路闭环指标的影响（三个种子平均）

回路	工况	超调 / %	调节时间 / s	IAE	相对理想
慢（温度，$\tau = 2$ s）	理想	0.2	2.65	1.020	100.0%
慢	10 ms 固定延迟	0.2	2.59	1.023	100.3%
慢	10 ms + 0-20 ms 抖动	0.3	2.53	1.026	100.6%
慢	100 ms 固定延迟	1.4	2.10	1.082	106.1%
快（伺服，$\tau = 50$ ms）	理想	7.7	0.132	0.0256	100.0%
快	10 ms 固定延迟	33.5	0.225	0.0455	177.7%
快	10 ms + 0-20 ms 抖动	56.1	0.499	0.1595	623.1%
快	100 ms 固定延迟	98.0	0.500	0.3653	1426.9%

表里最值得看的是快回路的第二行与第三行。同样是 10 ms 基准延迟，加 0 到 20 ms 抖动后，IAE 从 177.7% 涨到 623.1%，多出 3.5 倍。抖动带来的损失比延迟本身大得多。

慢回路的四行几乎是一条平线。100 ms 延迟才让 IAE 涨 6.1%，这个量级在温度控制里完全被工况波动掩盖，现场根本察觉不到。

89.6.4 丢包：数量与连续性

丢包的影响取决于“连续丢多久”，不取决于“总共丢多少”。表 89-12 是快回路在 5% 丢包率下、按不同连丢长度仿真的结果。

连续丢帧 / 帧	中断时长 / ms	超调 / %	调节时间 / s
1	1	33.7	0.226
2	2	33.9	0.227
5	5	35.8	0.235
10	10	36.5	0.271
20	20	43.8	0.453

表 89-12 连续丢帧长度对快回路的影响（丢包率 5%，10 个种子平均）

只丢 1 帧时，指标与“完全不丢”几乎一样：超调 33.7% 对 33.5%，调节时间 0.226 s 对 0.225 s。丢到 5 帧，超调涨到 35.8%，开始能看出来。丢到 20 帧，超调到 43.8%，调节时间翻倍。

IAE 上的对比更尖锐。5% 的单帧丢失让 IAE 从 0.0455 涨到 0.0458，涨 0.7%；同样 5% 的丢失率，每次连丢 20 帧，IAE 涨到 0.0703，涨 53.5%。两种情况的丢包率完全相同，后果差 70 倍。

这条规律直接指导网络设计：与其追求“丢包率低”，不如保证“不长时间连续丢”。前者靠链路质量，后者靠队列管理与冗余。

工业以太网的冗余机制（EtherCAT 环网冗余、Profinet MRP、介质冗余）正是针对连续性设计的。它们允许偶尔丢单帧，但保证不会因为一处断线而连丢几百帧。

89.6.5 抖动：把确定性问题变成统计问题

表 89-13 是快回路在 10 ms 基准延迟上叠加不同幅度均匀抖动的结果。

抖动幅度 / ms	超调 / %	调节时间 / s
0	33.5	0.225
2	36.3	0.241
5	40.5	0.342
10	47.7	0.500
20	58.7	0.500

表 89-13 周期抖动幅度对快回路的影响（基准延迟 10 ms，5 个种子平均）

抖动从 0 加到 2 ms，超调涨 2.8 个百分点，调节时间涨 7%。这段是平缓的，说明小幅抖动可以被回路吸收。

从 5 ms 加到 10 ms，调节时间从 0.342 s 跳到 0.500 s，超出仿真窗口。这段是陡的，说明回路的吸收能力有上限，过了上限就是“再也回不到带内”。

抖动的危害还体现在重复性上。同一个工况跑五次，超调的标准差随抖动幅度增长，到 20 ms 时已经超过 5 个百分点。这意味着同一台设备今天调好的参数，明天可能不适用。

对抖动，工程上的正确态度是“限制它”而不是“补偿它”。补偿抖动的算法（自适应延迟估计、预测控制）在实验室有效，在现场往往因为抖动的非平稳性而失效。

89.6.6 三个指标谁先坏

超调、调节时间、IAE 对网络条件的敏感度不同，坏掉的顺序也不一样。

超调最先坏。它由最不利的那几拍决定，抖动或一次连丢就能把它抬起来，而平均性能还没明显退化。所以现场“偶发过冲”往往是网络问题的第一个信号。

调节时间第二个坏。它反映的是“回到带内要多久”，需要多次不利条件累积才明显变化。表 89-13 里抖动到 10 ms 才让调节时间顶到窗口末端。

IAE 最后坏，但它最能反映“总体做错了多少”。慢回路的 IAE 在 100 ms 延迟下只涨 6.1%，超调却涨了 7 倍——从 0.2% 到 1.4%。这说明低超调回路的相对变化可以很大，绝对量仍然很小。

读表时要三个指标一起看，只看一个容易被误导。超调小不代表没有延迟问题，IAE 涨得多也不代表要换网络，要结合回路的用途判断。

89.6.7 长期运行的稳定性

短时仿真看不出的一类问题，是长时间运行后的漂移。时钟不同步、队列缓慢增长、内存泄漏，都在几小时到几天的尺度上表现出来。

时钟漂移按式 (89-10) 累积。50 ppm 的晶振一天漂 4.32 s，若网络没有同步机制，第二天的事件记录时间戳已经对不上。这类问题不会让回路失稳，但会让故障分析失去依据。

队列缓慢增长的典型场景是“非周期流量逐渐变多”：诊断请求、日志上传、远程访问，都走同一条链路。利用率从 0.5 爬到 0.75 时，排队延迟按式 (89-6) 从 0.50 ms 涨到 1.50 ms；再爬到 0.875，涨到 3.50 ms。这条曲线是双曲线的，越往上越陡。

对付这两类问题，靠的是长期监测而不是设计计算。周期抖动、丢帧率、链路利用率三个量，建议做成趋势图，按周看。趋势比瞬时值有用：利用率从 0.5 稳步爬到 0.7，说明有东西在悄悄增长。

89.6.8 从指标表反推验收线

表 89-11 是“给定网络算指标”，验收时需要反过来：“给定指标定网络”。同一条表可以两用。

以快回路为例。若工艺要求超调不超过 15%，表里只有理想工况的 7.7% 满足；10 ms 延迟的 33.5% 已经超标。反推的结果是：这条回路不能用 10 ms 的网络，要么把延迟压到 1 ms 量级，要么按 §89.8.4 把增益降下来。

以慢回路为例。若工艺要求调节时间不超过 3 s，表里四档全部满足，最差的 100 ms 延迟也有 2.10 s。反推的结果是：这条回路对网络几乎没有要求，10 ms 甚至 100 ms 都能用。

反推时要用“最坏工况”那一列，不能用平均。抖动的 56.1% 比固定延迟的 33.5% 更接近现场，按后者定验收线，投运后会不达标。

还有一条隐含约束：反推出的网络指标要落在可实现的范围里。若反推结果是“延迟不超过 0.3 ms”，那就不是网络选型问题，而是“这条回路不该网络化”，控制器必须就近安装。

把这三件事连起来看，验收线的定法就清楚了：先定工艺指标，再查表找对应的网络工况，最后核对这个工况在预算内可实现。三步都过，方案才算闭合。

89.6.9 仿真结果能信到什么程度

表 89-11 至表 89-13 都是仿真结果，用之前要知道它们的可信边界。

仿真里被简化掉的东西有四项：执行器的动态、阀或电机的饱和、量化误差、摩擦与死区。四项在真实系统里都会让超调比仿真更大、调节时间更长。

所以绝对数值不能直接当验收标准。仿真给 33.5% 超调，实测可能是 40%，也可能因为执行器更快而只有 28%。按仿真值卡验收线，容易在两头出错。

相对趋势可以信。延迟从 10 ms 增到 100 ms，超调必然上升；抖动从 2 ms 增到 20 ms，重复性必然变差。这类单调关系由模型结构决定，不会因为简化而反转。

排序也可以信。表 89-11 里“抖动比固定延迟更坏”这个结论，来自延迟分布变宽这一机制，与执行器细节无关。工程决策依赖的正是排序，而不是小数点后的数值。

正确的用法是：用仿真定方向与量级，用实测定验收线。两者差距大时，先查仿真漏掉的那四项，往往能对上。

89.7 参数意义与作用

89.7.1 七个参数各管什么

本章出现过的量可以归成七类参数。它们不是并列关系，而是分三层：网络层决定“能传多快、传多稳”，接口层决定“控制器看到什么”，回路层决定“这些信息值多少钱”。

网络层的参数是利用率 ρ、单向延迟 T_{net}、抖动、丢包率。接口层的参数是采样周期 T_s 与延迟拍数 d。回路层的参数是穿越频率 ω_c 与相位裕度 φ_m。

表 89-14 把七个主要参数列在一起，给出单位、典型范围与“改它会发生什么”。

表 89-14 七个主要参数的单位、范围与单调影响

参数	符号	单位	典型范围	改大的后果
链路利用率	ρ	—	0.2–0.7	排队延迟按双曲线上升，尾部最快失控
单向延迟	T_{net}	ms	0.1–50	相位裕度按 $\omega_c T_{\text{net}}$ 线性下降
周期抖动	J	μs–ms	1–20000	等效延迟变成分布，指标重复性变差
丢包率	p	—	10^{-8}–10^{-2}	平均影响小，连丢长度决定后果
采样周期	T_s	ms	0.1–100	半拍滞后增大，延迟量化变粗
延迟拍数	d	拍	1–20	离散等效延迟增大，积分作用被推后
相位裕度	φ_m	°	30–70	超调减小、响应变慢，过 70° 后迟钝

这张表只能看单调方向，不能看数量关系。同样是“改大”，利用率改大 10% 可能让排队延迟翻倍，采样周期改大 10% 只让半拍滞后多 0.05 ms。下面几小节分别说清数量关系。

89.7.2 利用率：唯一的非线性参数

七个参数里，只有利用率是非线性的。其余参数的后果大致与变化量成正比，利用率的后果按 $1/(1-\rho)$ 发散。

这个差别来自排队。带宽够不够是线性问题，等不等是排队问题。式 (89-6) 的 $1/(1-\rho)$ 项在 $\rho = 0.5$ 时等于 2，在 $\rho = 0.9$ 时等于 10，在 $\rho = 0.95$ 时等于 20。

工程上把 0.7 当作上限，理由是 0.7 时平均排队 1.17 ms、尾部约 4.67 ms，多数 10 ms 级回路还能吸收；到 0.9，尾部 18 ms，已经与一个周期相当，周期数据的到达时刻会跨拍。

利用率还有一个“看不见”的部分：交换机内部的转发队列。端口利用率 30% 不代表交换机不排队，若多个端口同时向一个端口发送，瞬时利用率接近 100%，排队照样发生。这就是 §89.1.1 第三个现象的成因。

89.7.3 延迟：与穿越频率配对才有意义

T_{net} 单独看没有意义。10 ms 对温度回路是 0.5% 的时间常数，对伺服速度环是 20%，差 40 倍。

有意义的组合是 $\omega_c T_{\text{net}}$，也就是延迟在穿越频率处折合的弧度。这个乘积小于 0.2 rad（约 11°）时，延迟基本可以忽略；到 0.5 rad（约 29°）时，裕度已经明显受损；超过 1 rad（约 57°）时，回路接近失稳边界。

按这个判据，10 ms 延迟对慢回路是 0.01 rad，完全无害；对快回路是 0.67 rad，已经进入受损区。这个换算比“延迟多少毫秒”有用得多，因为它把对象与网络放在同一个尺度上。

89.7.4 抖动：不是一个数，是一个分布

工程上说“抖动 5 ms”，通常指峰峰值或最大偏差，这两种说法都不够。同样的最大值，均匀分布与尖峰分布的后果差别很大。

均匀分布下，每次延迟在 10 到 15 ms 之间均匀取值，回路每拍都在“稍微不同”的条件下工作，等效于平均延迟加 2.5 ms。尖峰分布下，95% 的拍是 10 ms，5% 的拍跳到 15 ms，回路的多数拍正常，少数拍被打乱，超调由那 5% 决定。

表 89-13 用的是均匀分布，因此它的结论偏乐观。现场的抖动往往是尖峰式的（由 GC、交换机缓冲、其他流量突发引起），后果更接近“最坏值”那一列。

判断抖动要不要紧，看它占周期的比例。小于 10% 时可以当噪声；10% 到 50% 时会明显恶化重复性；超过 50% 时，等效于周期本身在变，控制器的定周期假设失效。

89.7.5 采样周期：决定看清还是看糊

T_s 有两个作用，一个是采样定理意义上的“看清”，一个是延迟量化意义上的“分辨率”。

看清这一面由对象决定，$T_s \le 0.1\tau$ 是老规矩，§89.5.6 已经算过。分辨率这一面由网络决定，T_s 越小，延迟拍数越细，抖动越容易被平均掉。

两面冲突时，取小的那个。冲突的典型场景是“快对象 + 慢网络”：对象要求 1 ms 采样，网络延迟 10 ms，比值 10，这时采样周期只能取 1 ms，延迟拍数是 10 拍，抖动 5 ms 相当于 5 拍，回路必须能吸收这个量级的跳动。

反过来“慢对象 + 快网络”没有冲突：温度对象 2 s，网络 100 µs，采样取 10 ms，延迟 0.1 拍，可以当作零延迟处理。

89.7.6 节点数与字节数：只花钱，不惹事

N 与 L 只影响带宽，不影响稳定性，这是它们与前面五个参数最大的区别。

这个性质有实际意义：扩站、加数据只会让利用率升高，不会直接让回路失稳。所以扩站的风险是可算的——按式 (89-14) 重算 ρ，看是否越过 0.7。

例外是分帧。站数增加到必须分帧时，部分站的刷新周期被拉长，等效于给这些站加了延迟，这时 N 就间接影响了稳定性。分帧的判据在 §89.9.2 展开。

89.7.7 同步间隔：精度与开销的交换

T_{sync} 决定两次同步之间漂多少。按式 (89-10)，晶振 50 ppm 时，要把漂移压到 10 µs，间隔不能超过 200 ms；压到 1 µs，间隔不能超过 20 ms。

缩短间隔的代价是同步报文占用带宽与 CPU。PTP 的 Sync 报文每秒 1 个到 16 个，对 100 Mbit/s 链路可以忽略；但每个报文的中断处理要占从站 CPU，间隔缩到 1 ms 时，低端从站可能吃不消。

所以同步间隔要按需选，不按“越短越好”选。事件顺序记录要 1 ms 级的时间戳，间隔取 100 ms 足够；多轴插补要 1 µs 级，间隔取 10 ms 到 20 ms，配合硬件时间戳。

89.7.8 三组致命组合

参数单独看都不致命，组合起来才有杀伤力。下面三组在现场最常见。

第一组是“高利用率 + 大突发流量”。$\rho = 0.6$ 看着安全，但视频流或文件传输一进来，瞬时利用率打到 1.0，周期帧在队列里等到下一拍。表现是“平时正常，一有人拷文件就报警”。

第二组是“快回路 + 毫秒级抖动”。平均延迟 10 ms 可以接受，抖动 0 到 20 ms 就不可接受。表现是“参数怎么调都调不稳，换一批产品就变”。

第三组是“长连丢 + 无冗余”。丢包率 10^{-4} 看着很低，但每次断线连丢几百帧，回路在断线期间完全开环。表现是“偶尔一次大过冲，事后查不到原因”。

三组组合的共同点是：平均值都正常，出问题的是尾部。所以网络评估不能只看平均值，必须看分布与最坏值。

89.7.9 四个常用换算

现场没有仿真工具时，用四个换算就够估个大概。它们都是本章式子的特例，记熟比记公式快。

第一个是延迟折算度数：每 1 ms 延迟折合 $0.0573\,\omega_c$ 度。$\omega_c = 1$ rad/s 时是 0.057°，10 rad/s 时是 0.57°，66.6 rad/s 时是 3.8°，314 rad/s 时是 18°。

第二个是采样滞后折算度数：半拍滞后 $T_s/2$ 折合 $0.0287\,\omega_c T_s$ 度。$\omega_c = 1$、$T_s = 10$ ms 时只有 0.29°，可以完全忽略；$\omega_c = 66.6$、$T_s = 1$ ms 时是 1.9°，与 1 ms 延迟相当。

第三个是排队延迟速查：服务时间 1 ms 时，$\rho = 0.5$ 对应 0.50 ms，0.6 对应 0.75 ms，0.7 对应 1.17 ms，0.8 对应 2.00 ms，0.9 对应 4.50 ms。记住 0.7 这个拐点，比记整条曲线有用。

第四个是刷新率换算：N 个站分 n 帧轮发，单站刷新周期是 nT_{cyc}。64 轴一帧发完时刷新 500 μs，分两帧后变 1 ms，速度环采样率直接掉一半。

四个换算的共同前提是“线性区”：延迟远小于时间常数、利用率低于 0.7、抖动小于周期的 10%。出了线性区，换算只能给方向，不能给数值。

89.7.10 七个参数怎么测

参数不是查手册得来的，是测出来的。七个量各有各的测法，测错了比不测更糟。

利用率靠交换机端口统计，看“出口带宽占用”而不是“入口”。入口统计只算收到的帧，出口还要算排队的帧，两者在拥塞时能差一倍。没有可管理交换机时，用抓包工具按秒统计字节数，再除以链路速率。

单向延迟不能直接测，除非两端时钟已同步。常用办法是测往返再除以 2，前提是往返路径对称。不对称时误差可达数毫秒，这时改用“主站发、从站翻 IO、示波器看”的物理办法。

抖动靠抓包统计到达间隔的分布。只记最大值不够，要看直方图：单峰窄分布说明抖动来自固定节拍，双峰说明有两条路径或两种调度状态。

丢包靠序列号或工作计数器。Modbus 没有序列号，只能靠超时重发计数间接估；EtherCAT 的丢包会体现在工作计数器不连续上，可以直接读。

采样周期与延迟拍数查组态文件，不要凭记忆。组态改过而文档没改，是投运阶段最常见的坑。

相位裕度不能直接测，只能用式 (89-8) 算。算它需要 ω_c、τ、T_i、L、T_s、T_{net} 六个量，前四个来自辨识，后两个来自测量。

89.7.11 哪些参数会随时间漂移

设计时算准的参数，运行一年后可能变了。知道哪些会漂，才知道趋势图该盯什么。

会漂的第一个是利用率。加设备、加诊断、加远程访问，都让它缓慢上升。它不会自己降下来，除非有人清理流量。

会漂的第二个是抖动。接头氧化、交换机风扇积灰、柜内温度升高，都会让转发时间变长且不稳定。抖动上升往往先于丢包出现，是最好的预警信号。

会漂的第三个是晶振频率。晶振年老化率在 ppm 量级，十年累积的偏差足以让“每天同步一次”的方案失效。这与温度也有关，柜内温度每升 10 ℃，偏差会变化零点几 ppm。

不会漂的是字节数与拓扑。只要不改组态、不增删站，这两个量是固定的。所以带宽预算一旦算好，可以长期沿用；而利用率、抖动、同步误差要定期复测。

把“会漂”与“不会漂”分开，是设计文档该有的结构：固定的部分写清楚，漂移的部分给出复测周期与报警阈值。

89.7.12 拿一张表走一遍

用一个具体项目走查表 89-14，看七个参数怎么用。

项目是一条 16 轴的贴装线，要求位置环 2 ms，允许超调 10%。按 §89.3 先列清单，得到过程数据 256 字节，控制网络选 EtherCAT，周期取 1 ms。

查表第一行，利用率：帧长约 300 字节，线上 24 μs，加 16 μs 转发与 20 μs 主站，单周期 60 μs，占 1 ms 的 6%。$\rho = 0.06$ 远低于 0.7，这一项不构成约束。

查第二行，延迟：60 μs 的线上与处理，加 1 ms 周期带来的平均半拍等待 0.5 ms，单向延迟约 0.6 ms。相对 2 ms 的采样要求，比值 0.3。

查第三行，抖动：EtherCAT 正常运行时小于 1 μs，占周期 0.1%，可以当零。

查第四行，丢包：正常工作时应为零，只在断线时出现，靠冗余处理。

查第五行与第六行，采样周期与延迟拍数：1 ms 周期、0.6 ms 延迟，$d = 1$ 拍。相对位置环的时间常数（十几毫秒量级），一拍占比不到 10%。

查第七行，相位裕度：按式 (89-8) 算，若整定后 ω_c 在 300 rad/s 量级，0.6 ms 延迟折 10.3°，从 65° 降到 55°，仍在 45° 以上。

七行查完，结论是“网络不是瓶颈”。这条项目的瓶颈在机械与整定，不在通信。若七行里有任何一行不达标，才需要回头改网络。

走查的价值在于把“要不要担心网络”变成一个有据可查的判断，而不是凭感觉。七行都过，就不必在通信上再花时间；某一行卡住，就知道该改什么。

89.8 调参方法与经验

89.8.1 先说清调的是谁

“调参”在本章有两个对象：网络参数（周期、优先级、缓冲）与控制器参数（K_c、T_i、T_d）。两者的手段不同，动之前要分清。

网络参数决定“信息多新”，控制器参数决定“用多新的信息做多大的动作”。前者改硬件与组态，后者改程序与整定。

现场最常见的错误是用控制器参数去补偿网络问题。把 K_c 一降再降，超调确实下来了，但响应也慢了，而延迟造成的相位损失一点没少。这在 §89.4.4 已经说过：延迟的度数损失只取决于 $\omega_c T_{\text{net}}$，降增益只是把 ω_c 拉低，等价于“少用一点带宽”。

89.8.2 三条经验规则

工程上流传的三条规则，都能从本章的式子推出来。

第一条：网络单向延迟不超过对象时间常数的 1/10，即 $T_{\text{net}} \le 0.1\tau$。代进式 (89-8)，$\omega_c\tau \approx 1$ 时这条等价于延迟折合 5.7°，裕度损失可接受。

第二条：链路利用率不超过 0.7，即 $\rho \le 0.7$。这条来自式 (89-6)，0.7 是“尾部延迟不超过 5 倍平均服务时间”的分界。

第三条：周期抖动不超过周期的 10%。这条来自表 89-13，10% 以下指标变化平缓，以上开始陡增。

三条规则都是“数量级规则”，不保证每条回路都成立。它们的作用是在没有仿真条件时给出一个可用起点，有仿真条件时应按式 (89-8) 逐条核。

89.8.3 现场四步排查

网络引起的控制问题，现场按四步查，从慢到快。

第一步看刷新率。抓包统计某站的更新间隔，若明显大于设计周期，说明分帧、丢帧或轮询过慢。这一步能查出 §89.1.1 的第一个现象。

第二步看抖动。同一站的更新间隔的最大值与最小值之差，除以周期，就是抖动比例。超过 10% 就要查交换机与竞争流量。

第三步看丢帧与连丢。统计连续丢帧长度的分布，而不是只看总丢帧率。连丢超过 10 帧的链路，即使总丢帧率很低也要处理。

第四步看同步。多站时间戳对齐，若偏差随时间线性增长，说明没有同步或同步间隔太长；若偏差随机跳动，说明同步残差大。

四步的顺序不能颠倒。刷新率不对时，抖动与丢帧的统计都没有意义；同步不准时，用时间戳做的因果分析都不可信。

89.8.4 降增益还是降周期

遇到延迟引起的超调，有两条路：降增益或降延迟。哪条先走，取决于代价。

降增益的代价是响应变慢，收益是可容忍延迟按比例增加。以快回路为例，K_c 从 3.33 降到 2.0，穿越频率按

$$\omega_c \approx \frac{K_c}{\tau} \tag{89-18}$$

从 66.6 rad/s 降到 40 rad/s，可容忍延迟从 6.3 ms 涨到 14.1 ms，代价是响应时间涨 67%。

降延迟的代价是钱与工期，收益是性能不损失。把主站从软 PLC 换成带专用芯片的控制器，延迟能从 10 ms 降到 1 ms，超调从 33.5% 回到 15% 以内。

判断顺序建议是：先看延迟是否超过 0.1τ，超了就优先降延迟；没超但指标仍不满意，再考虑降增益。反过来做，等于用性能换一个本来就该修的短板。

式 (89-18) 的适用条件是 $T_i = \tau$（内模整定的默认取值）。T_i 取其他值时，穿越频率还受积分项影响，这个近似会偏。

89.8.5 三种常见错误整定

第一种是把网络延迟当成过程纯滞后去整定 T_i。网络延迟是“传输慢”，过程纯滞后是“物理上晚”，两者对积分作用的要求相反：前者要减小积分作用，后者要增加积分时间去匹配。混为一谈会让 T_i 越调越离谱。

第二种是在有抖动的回路上加微分。微分对延迟极其敏感，抖动 5 ms 时，微分项放大的不是误差变化率，而是延迟噪声。表现是输出高频抖动，越调越糟。

第三种是照搬实验室参数。实验室的延迟是固定的、丢包是零、抖动是微秒级，现场的这三个量都差一到两个数量级。参数搬过去，指标必然退化。

三种错误的共同根源是把“网络”当成理想信道。本章的全部内容，说到底就是把这个假设换掉。

89.8.6 一次调试记录

某包装线的色标跟踪回路，现象是“偶尔飞标”。原始配置：软 PLC 主站，EtherCAT 周期 1 ms，实测延迟 8 ms，抖动 0 到 6 ms。

按四步查：刷新率正常，1 ms 稳定；抖动 6 ms 占周期 600%，严重超标；连丢最大 3 帧，可接受；同步未启用，但该回路不需要。

定位到抖动的来源是主站的操作系统调度：软 PLC 跑在通用 Windows 上，GC 与后台任务偶尔抢占，导致发送时刻漂移。改成实时内核后，抖动降到 0 到 0.3 ms。

指标变化：超调从 42% 降到 21%，飞标率从每班 3 次降到零。周期与控制器参数都没动。

这个案例的启示是：抖动的根因常在主机侧，不在网络侧。抓包看到的是“帧到得不准”，但帧是在主站组好的，主站不准，网络再快也没用。查抖动要先查发送端。

89.8.7 先调网络还是先调回路

顺序错了会做两遍工。正确的顺序是先把网络调到设计值，再整定控制器。

理由很简单：控制器的整定值依赖于延迟。同一台设备，延迟 1 ms 与 10 ms 整定出来的 K_c 差一半以上。若先按当前网络整定，等网络修好后再整定一次，第一次的工作全白费。

实际操作分三步。第一步测网络，把刷新率、抖动、连丢、同步四项记下来，与设计值对照。第二步修网络，把不达标的项处理掉，复测确认稳定。第三步整定控制器，此时网络已经是“定值”，整定结果才有意义。

有一种情况可以颠倒：网络短期内改不了（预算、停机窗口），而工艺又等不了。这时先按现有网络整定，把指标做到当前条件下的最好，同时把“改善网络后可再提升多少”记进文档。等网络改造完成，再整定一次，两次的差值就是网络的贡献。

最怕的是两头同时改。网络在调、参数也在调，指标变好或变坏都说不清是谁的功劳，出了问题也无从复现。现场记录里“调了一周”的项目，多半是这么来的。

还有一条经验：改网络之后，先别动控制器参数，观察一天。有些延迟问题被控制器参数掩盖着，网络一好反而暴露出来（例如原来靠大积分时间压住的振荡消失了，响应变得过快）。

89.8.8 调参要留一张记录表

调参过程要留痕，否则三天后没人记得“哪一组参数是在什么网络条件下试的”。

表 89-20 是一份可直接用的模板。每一行是一次尝试，网络条件与控制器参数分开记，指标记三个，最后写一句结论。

表 89-20 调参记录模板（示例行取表 89-11 的数值）

日期	网络条件	K_c	T_i / s	超调 / %	调节 / s	IAE	结论
03-01	理想（台架）	3.33	0.05	7.7	0.132	0.0256	基准
03-02	10 ms 固定	3.33	0.05	33.5	0.225	0.0455	超调超标
03-03	10 ms 固定	2.00	0.05	待测	待测	待测	降增益试探
03-04	0-20 ms 抖动	3.33	0.05	56.1	0.499	0.1595	抖动为主因
03-05	实时内核后	3.33	0.05	待测	待测	待测	复测

表里“网络条件”一列要比“参数”一列记得更细。参数可以从组态文件里导出，网络条件导不出来：那天的交换机负载、有没有人在拷文件、温度多少，都影响结果。

“待测”两行是有意留的。先记下计划，再填结果，能避免“只记成功的那次”。调参过程中失败尝试往往比成功尝试更有信息量，扔掉了就白做。

记录表还有第二个用处：交接。新来的人看这张表，能知道哪些路已经走过、哪些是死胡同。没有这张表，同一批试验会被重复做一遍。

89.9 优化与改进

89.9.1 四个方向与它们的收益

网络侧的优化只有四个方向：降延迟、降抖动、降丢包、提同步精度。四个方向的手段、代价、收益各不相同，混在一起谈容易做错取舍。

降延迟的手段是换主站、缩短周期、减少转发级数。收益直接体现在式 (89-8) 的最后一项上，每降 1 ms，快回路少损失 3.8°。

降抖动的手段是实时内核、优先级隔离、专用链路。收益不体现在平均延迟上，而体现在尾部延迟上——抖动从 20 ms 压到 2 ms，超调能从 58.7% 回到 36.3%。

降丢包的手段是冗余、CRC 校验、屏蔽布线。收益集中在“避免长连丢”上，而不是降低平均丢包率。

提同步精度的手段是硬件时间戳、缩短同步间隔。收益只在多轴协同与事件分析里体现，对单回路没有帮助。

89.9.2 分帧：什么时候该分

一帧装不下所有数据时分帧。判据不是“装不下”，而是“装不下的那部分能不能等”。

设一帧最多装 $L_{\max}$ 字节，总数据 L_{tot}，则帧数

$$n = \left\lceil \frac{L_{\text{tot}}}{L_{\max}} \right\rceil \tag{89-19}$$

每增一帧，所有站的刷新周期变成 nT_{cyc}。若某站的允许刷新周期小于 nT_{cyc}，这个方案就不成立，必须换更快的链路或减少数据量。

正确的做法是按刷新率分帧：快站进第一帧，慢站进后续帧。快站每帧都刷新，慢站每 n 帧刷新一次。这样快站不受分帧影响，慢站本来就不需要快刷新。

64 轴包装线的例子可以说明。若把视觉的 64 字节与人机界面的 32 字节也塞进同一帧，帧长从 1256 涨到 1352 字节，单周期从 188.5 µs 涨到 196.2 µs，涨 4.1%。这 4% 是为两个不需要快刷新的站付的，不值得；把它们挪到第二帧，主帧回到 1256 字节，副帧每 20 拍发一次。

反过来的错误是“为了省带宽把伺服也分帧”。伺服数据分两帧轮发，位置反馈的刷新周期变成 1 ms，速度环的采样率掉一半，超调立刻上升。这类优化省下的带宽，远不够弥补性能损失。

89.9.3 周期越短越好吗

直觉上周期越短越好，实际有一个拐点。

周期缩短的收益是刷新率提高，等效延迟下降。每缩短一半周期，采样滞后与延迟量化误差各降一半。

代价是抖动占周期的比例上升。抖动绝对值不变时，周期从 500 µs 缩到 250 µs，抖动比例翻倍；从 250 µs 缩到 125 µs，再翻一倍。按 §89.7.4 的判据，比例超过 50% 时定周期假设失效。

所以周期有一个下限，由抖动决定：$T_{\text{cyc}} \geq 10J$。抖动 20 µs 时，周期不能短于 200 µs；抖动 200 µs 时，不能短于 2 ms。

这条下限常被忽略。工程师看到“EtherCAT 支持 100 µs 周期”，就把周期设成 100 µs，结果抖动 30 µs 占 30%，指标反而不如 500 µs 周期。

89.9.4 队列管理：把周期流量隔出来

抖动的头号来源是排队，排队的原因是周期流量与非周期流量共用队列。隔离是最有效的手段。

隔离有三个层次。物理隔离最彻底，实时网络与非实时网络各用一套线缆与交换机，代价是布线翻倍。VLAN 隔离成本低，但在同一台交换机内部仍共享缓冲，只能减轻不能消除。优先级队列（802.1Q）让周期帧排在最高优先级，但交换机出口的突发仍会造成排队。

真正能消除排队的是时间片调度，Profinet IRT 与 EtherCAT 用的就是这一类。交换机（或从站）按预定的时间片转发，周期帧在专属时隙内传输，与非周期流量在时间上完全分开。代价是全网设备都要支持时间片调度，混合组网时难以逐段升级。

工程上的取舍是：抖动要求在 1 ms 以上的，优先级队列够用；要求在 100 µs 以下的，必须上时间片调度。

89.9.5 控制器侧能补偿多少

网络侧改不动时，控制器侧可以做补偿。常用的有三种：延迟补偿、预测观测、Smith 预估。

延迟补偿最简单：把控制量按已知延迟打时间戳，执行端按时间戳对齐。它只能补偿“确定的延迟”，对抖动无效。

预测观测用对象模型推算“当前状态应该是什么”，用它代替延迟的测量值。它对模型误差敏感，模型准的时候效果好，模型不准时反而引入振荡。

Smith 预估把延迟移到闭环之外，用一个内部模型抵消它（见第 47 章与第 107 章）。它对纯滞后有效，对随机抖动无效，因为 Smith 预估假设延迟是常数。

三种补偿的共同边界是：能补“确定”，补不了“随机”。所以补偿不能替代降抖动，只能作为补充。

89.9.6 冗余与快速恢复

丢包的后果由连丢长度决定，冗余的作用就是把连丢长度截断。

EtherCAT 环网冗余在检测到断点后，把帧从另一方向绕回，恢复时间在一帧以内。回路最多丢一两拍，按表 89-12，这个量级几乎无害。

Profinet 的介质冗余协议在环形拓扑上提供备用路径，恢复时间典型值在 200 ms 量级。对快回路，200 ms 等于连丢 200 帧，后果严重；对慢回路，200 ms 只相当于 0.1 个时间常数，可以接受。

生成树协议的恢复是秒级，只适合信息系统，不适合控制网络。这一点常被混淆：办公室里跑得好好的冗余方案，搬到车间就是灾难。

选冗余方案时要先问：这条回路上最长的可接受中断是多少？把答案除以周期，就是允许的连丢帧数，再对照各方案的恢复时间选。

89.9.7 优化的优先级

四类优化的收益不是等价的。按“每投入一分钱换多少指标”排序，通常是：降抖动 > 降延迟 > 降丢包 > 提同步。

降抖动排第一，因为它同时改善平均值与尾部。表 89-13 里抖动从 20 ms 压到 2 ms，超调从 58.7% 降到 36.3%，这个跨度比降延迟更大。

降延迟排第二，因为它的收益线性可预测，投入也明确（换主站、缩周期）。

降丢包排第三，因为多数工业网络的丢包率本来就很低，优化空间有限；真正要做的不是降丢包率，而是避免长连丢，这属于冗余范畴。

提同步排最后，因为它只影响特定的应用（多轴、事件分析），对单回路没有收益。表 89-15 汇总了四类手段的收益与代价。

表 89-15 四类网络优化手段的收益与代价

优化方向	典型手段	收益	代价	适用场合
降抖动	实时内核、时间片调度、物理隔离	尾部延迟降一到两个数量级	换主站、换交换机、布线翻倍	快回路、多轴同步
降延迟	硬主站、缩短周期、减少级数	每降 1 ms 少损失 3.8°（快回路）	硬件成本、CPU 负载	延迟超 0.1τ 的回路
降丢包	环网冗余、屏蔽布线、CRC	连丢长度从数百帧降到一两帧	一根额外的线、支持冗余的从站	不能停的产线
提同步	硬件时间戳、缩短同步间隔	同步误差从毫秒到百纳秒	支持分布式时钟的芯片	多轴插补、事件顺序记录

排序不是绝对的。若某条回路的延迟已经超过 0.1τ，降延迟就该排第一；若丢包表现为长连丢，冗余的优先级要提到最前。表 89-15 给的是默认顺序，具体项目要按 §89.8.3 的四步排查结果调整。

89.9.8 一笔分帧的账

分帧常被当成“省时间”的手段，实际它只在特定条件下省时间。用 64 轴包装线算一笔账就清楚了。

一帧装 1216 字节，单周期 188.5 µs。取 250 µs 周期，占 75.4%，超了 0.7 的线。直觉的办法是分两帧，每帧 608 字节。

每帧的帧长变成 648 字节（8 字节前导与帧头、14 字节以太网头、2 字节计数器、608 字节数据、4 字节 CRC、12 字节间隙）。线上时间 51.8 µs，加 68 µs 转发与 20 µs 主站，单帧 139.8 µs，占 250 µs 的 55.9%。

账面上 75.4% 降到 55.9%，似乎成功了。但刷新率没变：两帧轮发，每个轴的刷新周期仍是 500 µs。而如果直接用 500 µs 周期一帧发完，占用是 37.7%。

两种方案刷新率相同，占用 37.7% 对 55.9%。分帧没有任何收益，反而多占了 18 个百分点。

原因是转发时间不随帧数减少。68 个站每帧都要转发一次，分两帧等于转发两遍，136 µs 的转发时间摊在 500 µs 里占 27%。一帧发完时，转发只占 13.6%。

这笔账说明分帧的适用条件：一是单帧确实装不下（超过链路 MTU 或从站缓冲上限），二是不同站的刷新率本来就不同。为了“降周期”而分帧，是把转发时间白白翻倍。

真正的降周期路线是加带宽或者减数据。加带宽把 100 Mbit/s 换成 1 Gbit/s，线上时间从 100.5 µs 降到 10.1 µs，单周期从 188.5 µs 降到 98.1 µs，250 µs 下的占用降到 39.2%。减数据则要靠压缩 PDO 与把诊断移出周期通道。

89.9.9 优化的三条底线

任何优化都不能突破三条底线，破了就是拿另一个问题换这个问题。

第一条是稳定性余量。优化后相位裕度不应低于 45°，增益裕度不应低于 6 dB。压周期、加站、改拓扑都可能吃掉裕度，改完要重算式 (89-8)，不能只看利用率。

第二条是可维护性。把非周期流量全部关掉、把诊断接口封死，确实能把抖动压到最低，但出了故障就无从下手。优化时要保留至少一条诊断通道，并确保它不会在关键时刻抢占周期流量。

第三条是成本收益为正。降抖动从 20 ms 到 2 ms，超调从 58.7% 降到 36.3%，值得付实时内核的钱；从 2 ms 再压到 0.2 ms，指标改善可能只有一两个百分点，钱就不值得花。

三条底线的顺序不能颠倒。先保稳定性，再保可维护，最后谈收益。反过来做，常见的结局是“指标好看了，但没人敢动这条线”。

判断有没有破线，用一组固定动作：改完重跑 [3] 与 [4b]，看裕度与指标；拔一段网线，看诊断是否还能定位；写一句“这次改动花了多少、换回多少”，写不出来就是没算清。

89.9.10 改了以后怎么证明有效

优化做完要说清“好了多少”，否则下一轮没人知道该往哪走。证明有效需要三个条件同时满足：工况相同、口径相同、重复多次。

工况相同指对比的两组数据来自同一段工艺过程。常见错误是拿改造前的白天数据比改造后的夜班数据，负载不一样，结论不可信。做不到同工况时，至少要在同一时段、同一批产品上比。

口径相同指指标算法一致。超调按同一个带、调节时间按同一个阈值、IAE 按同一个积分区间。改口径等于改结论，这类“优化”在评审里最容易被戳穿。

重复多次指同一工况跑三次以上，取中位而不是取最好。网络指标本身有随机性，一次测量说明不了问题。三次结果若离散很大，说明工况没控住，先解决这个再谈收益。

满足三条之后，收益的表达要落到可比的量上。不要写“响应快了很多”，要写“抖动从 20 ms 压到 2 ms 后，超调从 58.7% 降到 36.3%，调节时间从 0.500 s 回到 0.241 s”。数字带单位、带口径，别人才敢引用。

最后留一条反证：这次改动有没有让某个指标变坏。优化很少全赢，降增益换来了低超调，代价通常是响应变慢。把代价一并写出，方案才算完整，也才经得起下一轮追问。

89.10 工程落地

89.10.1 按场景选型

选型表不该按“技术先进程度”排，该按场景排。表 89-16 是六个常见场景的默认答案，可作为起点，具体项目仍要按 §89.3 的五步核一遍。

表 89-16 六个典型场景的通信选型起点

场景	周期量级	推荐总线	关键理由	需避开的做法
水温、压力、流量等过程量	100 ms–1 s	Modbus RTU	成本低、抗干扰好、维护简单	不要用轮询去读快变量
变频器群控	10–50 ms	Profibus DP 或 Modbus RTU	站数多、数据量小	不要把变频器与伺服混在一条慢线上
单机设备内部（轴数 ≤8）	1–4 ms	CANopen	布线简单、成本低	不要超 8 轴，仲裁延迟会累积
多轴运动（轴数 8–64）	0.25–2 ms	EtherCAT	逻辑环、微秒级同步	不要用交换机串接，破坏逻辑环
流程工业 DCS 层	10–100 ms	Profinet RT	与 IT 网融合、诊断丰富	不要与非实时流量共用端口
高精度同步采集	≤1 ms	EtherCAT 或 Profinet IRT	分布式时钟、时间片调度	不要用 NTP 凑精度

表里“需避开的做法”一列比“推荐”一列更有价值。选错总线的项目往往不是不知道哪种好，而是低估了混用的代价。

选完总线还要定网络的层次。图 89-10 是常见的三网分离：管理网放工程师站、历史库与 MES；控制网放控制器与两台交换机；设备网放远程 I/O、变频器与仪表。三网之间由网关或路由设备隔开，管理网到控制网通常只允许单向只读。

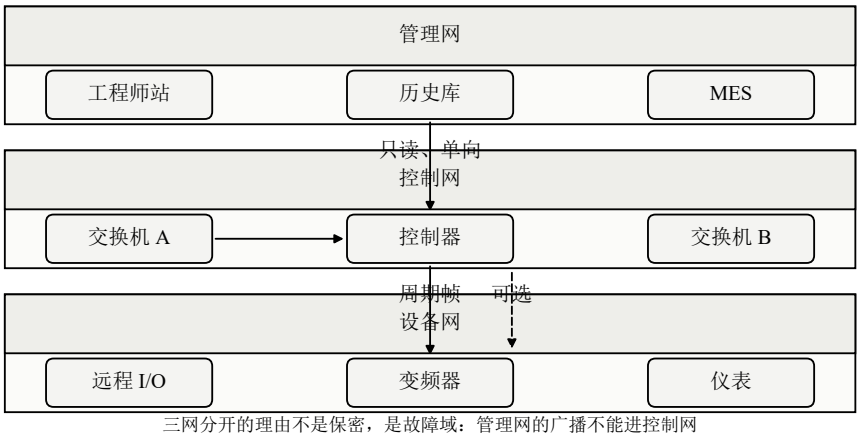


图 89-10　管理网、控制网与设备网的三层部署

分开的首要理由不是保密，是故障域。管理网上的广播、大文件传输、软件更新，任何一项涌进控制网都会抬高周期帧的排队延迟；反过来，控制网的组播如果漏进设备网，老式从站可能直接被淹掉。把三层隔开，故障就被限制在一层以内。

代价是跨层访问要绕路。工程师站想看实时曲线，得经历史库取数，不能直连控制器；MES 要下发配方，得走网关的写通道并留审计记录。这些绕路在项目初期像多余的约束，出过一次跨网流量打满交换机的事故之后，就会明白它是廉价的保险。

89.10.2 布线：九成的现场故障在这里

通信故障里，布线问题占的比例远高于协议问题。说“九成”不夸张，因为协议栈是芯片厂验证过的，而线是现场工人手工做的。

终端电阻是最常见的漏项。Modbus RTU 与 CAN 都要求总线两端各接一个 120 Ω 电阻，中间站不接。少接一端，短距离可能正常，距离一长就时通时断；多接几个，负载过重，信号幅度下降。

屏蔽层的接法常被做错。屏蔽层应单端接地，通常接在控制柜侧；两端都接会形成地环路，反而引入干扰。若现场两端地电位差较大，改用隔离型收发器，不要靠“两端接地”解决。

支线长度也要控制。CAN 的支线应短于 0.3 m，Modbus 的支线应短于 1 m。支线越长，反射越严重，表现为“某些站偶尔通信超时”，而查线序完全正常。

网线段长按规范取：铜缆以太网单段 100 m，超了加交换机或上光纤；Modbus RTU 在 9600 bit/s 下可到 1200 m，但这是理论值，现场有变频器时按 500 m 设计更稳。

89.10.3 时间同步的实施

同步不是勾一个选项就完事，要按四步实施。

第一步选参考站。EtherCAT 的参考时钟应放在拓扑最前、且支持分布式时钟的从站上；放在末端会让补偿链变长，精度下降。

第二步设同步周期。按 §89.3.5，多轴插补取 10 到 20 ms，事件记录取 100 ms 即可。同步周期设得比通信周期长是常见的错误，会让部分拍的时戳不一致。

第三步处理启动过程。从站上电时刻不一致，同步要等所有站进入 OP 状态后才稳定。启动阶段的前几百毫秒不要用时间戳做判断，否则会把正常启动误判为故障。

第四步验证。最直接的办法是让两个从站在同一时刻翻转一个 IO，用示波器量两个上升沿的差值。这个差值是端到端的同步误差，包含布线延迟，比读寄存器里的偏差值更可信。

89.10.4 地址与命名

地址规划要在出图之前定，不要边调试边改。

站地址建议分段：1–31 给驱动器，32–63 给远程 I/O，64–95 给仪表，96 以上留给临时设备。分段的好处是抓包时看地址就知道站型。

命名要与地址绑定。站名用“功能 + 序号”，例如 `AX01`、`IO12`、`TT03`，不要用“站 1”“站 2”。名字进程序、进画面、进报警文本，出问题时一眼能对上设备。

过程数据映射要留空位。每个站的 PDO 预留 10% 到 20% 的空字节，后续加一个状态位、加一个诊断字时不必重排映射。重排映射意味着所有读写该站的程序都要改，风险远大于浪费几个字节。

IP 地址与站名要写进一张表，与电气图一起归档。这张表是后期维护的入口，缺了它，三年后没人能接手。

89.10.5 故障诊断的分支

现场故障按现象分四支，每支的查法不同。

第一支“完全通信不上”。先查物理层：电源、终端电阻、线序、网口指示灯。物理层正常再查站地址与波特率，最后查主站组态。

第二支“时通时断”。这类问题八成在物理层：接头氧化、屏蔽层破损、支线过长、附近有变频器干扰。用示波器看波形比看错误计数有用。

第三支“能通但指标差”。按 §89.8.3 的四步查：刷新率、抖动、连丢、同步。这时通信本身没坏，坏的是时序质量。

第四支“时间戳对不上”。先看同步是否使能，再看同步间隔，最后看晶振偏差。若偏差随时间线性增长，是同步问题；若随机跳动，是同步残差大。

四支的分界是“能不能通”。能通的问题不要从物理层查起，会白花时间；不能通的问题不要从参数查起，会越查越乱。

89.10.6 投运验收清单

验收要留下可测的数字，不能只写“通信正常”。表 89-17 是一份可直接用的清单。

表 89-17 工业网络投运验收清单

项目	测量方法	判据	记录形式
刷新周期	抓包统计某站更新间隔	与设计值偏差 <1%	直方图 + 最大值
周期抖动	更新间隔的最大最小之差 ÷ 周期	<10%	趋势图
丢帧率	连续 8 h 抓包计数	$<10^{-6}$	计数表
最大连丢	统计连续丢帧长度分布	≤2 帧	分布表
链路利用率	交换机端口统计	<0.7	峰值记录
同步误差	双站同刻翻转 IO，示波器测量	<1 μs（多轴）	波形截图
断线恢复	拔掉环网一段，记录恢复时间	<1 个周期	秒表 + 抓包
满负荷温升	满轴运行 2 h 后测柜内温度	<元件降额温度	温度记录

清单里最容易被跳过的是“断线恢复”与“满负荷温升”。前者验证冗余是否真的生效，后者验证长期运行的余量。两项都不做，投运当天正常，三个月后出问题。

89.10.7 运维：三个趋势与两条纪律

运维阶段要盯三个趋势：链路利用率、周期抖动、丢帧率。三个量都按周看，不看瞬时值。

利用率缓慢上升通常意味着有人在加设备或在跑非周期任务；抖动缓慢上升通常意味着接头在氧化或交换机在老化；丢帧率突然从零变成非零，通常是新加的变频器带来了干扰。

两条纪律：一是变更管理，任何新增站点、新增流量都要重算利用率并记录；二是备件一致，备用的从站、交换机、线缆必须与在线设备同型号同固件，不同固件的从站在同步与诊断上可能有细微差别。

89.10.8 一个反面案例

某汽车零部件线的焊接单元，改造时把视觉相机接到了控制网络上。相机每 100 ms 传一张 2 MB 的图，占用带宽约 160 Mbit/s，而链路是 100 Mbit/s。

改造后现象是“焊接电流偶尔波动”。查了一周，控制器参数没动、焊接电源没动、工艺没变。最后抓包发现，相机传图时链路利用率打到 1.0，机器人的周期帧被排队推迟 30 到 80 ms。

处理办法是把相机挪到独立的千兆网，控制网络只保留周期流量。处理后电流波动消失。

这个案例的教训不是“相机不能上网”，而是“非周期大流量必须与周期流量物理隔离”。按式 (89-6)，利用率超过 0.7 时排队延迟按双曲线上升；而突发大流量能让瞬时利用率瞬间到 1.0，平均利用率看起来却只有 0.4。

89.10.9 一份设计文档的目录

工程交付物不是“调试记录”，是一份能被人接手的设计文档。按本章的骨架，目录大致如下。

第一章写需求：节点清单（表 89-9 的形式）、每类站的刷新周期要求、允许的最长停机时间、工艺对超调与调节时间的指标。

第二章写方案：选定的总线与理由、拓扑图、周期、同步方式与间隔、冗余方案。这一章要能回答“为什么不是另一种”，每种被否掉的方案都要写一句否决理由。

第三章写预算：带宽计算（表 89-10 的形式）、利用率、转发与主站的延时分解、余量。数字要能倒推，不能只给结论。

第四章写实施：地址分配表、命名规则、PDO 映射表、布线要求（终端电阻、屏蔽、段长）、接地做法。

第五章写验证：验收清单（表 89-17 的形式）、实测数据、与设计值的偏差、偏差的处置意见。

第六章写运维：三个趋势的监测方法、报警阈值、备件清单、变更流程。

六章里最容易被省略的是第二章的“否决理由”与第五章的“偏差处置”。前者省了，后来者会重复评估已经被否掉的方案；后者省了，验收就变成走过场。

文档不必长，但六章缺一不可。缺了第三章，后面无法判断能不能扩站；缺了第四章，换一个电工接线就可能出问题；缺了第五章，投运当天的数据无人可查。

89.10.10 三类人要交接什么

网络交接不是“交一份文档”，而是让三类人各自能干活。

电气人员要接手的是线。终端电阻在哪、屏蔽层接在哪一端、每根线的走向与编号、备用线留了几根。这些信息靠图，不靠文字。交接时要带人走一遍柜内与现场，指给他看。

软件人员要接手的是映射。站地址、PDO 布局、周期、同步参数、报警码含义。这些信息靠表，表要能直接导进组态软件，不能只有 PDF。

维修人员要接手的是判断。什么现象对应什么原因、哪个灯亮代表什么、抓包工具怎么用、哪些参数不许改。这些信息靠一页“故障速查”，贴在柜门内侧比放在文件夹里有用。

三类人的共同需求是“改之前知道后果”。所以交接时要说清三件事：哪些参数改了会影响指标、哪些改了只会影响诊断、哪些完全不能动（例如同步周期与周期的一致性）。

交接完成后做一次演练：让维修人员按速查表处理一个模拟故障（例如拔掉一段环网），看能否在十分钟内定位。演练不通过，说明文档还不到位。

这一步常被当成形式，实际是最省的投入。一次两小时的演练，能省掉投运后几个月的电话支持。

89.10.11 投运后第一周看什么

投运后的第一周是“看”的一周，不是“调”的一周。这一周里设备状态、环境温度、机械走合都在变，此时调出来的参数过几天就不适用。

要看的量有三个：刷新周期、周期抖动、错误计数。三个都按天记录，画成趋势而不是只看当天值。

刷新周期看的是“有没有台阶”。主站记录的周期时间若在某天整体抬高了一档，通常是加了站、加了一段非周期流量，或者某台交换机开了镜像口。台阶式变化比缓慢漂移更容易定位原因。

周期抖动看的是尾部。平均值好看而尾部突然变长，说明有间歇性的干扰源，例如某台设备每班启动一次。这类问题在现场表现为“白班正常、夜班偶发”，只查白天永远查不到。

错误计数看增长速率，不看绝对值。CRC 错误每天增加几十个，在长线路上属于正常；每小时增加几十个，就要按 §89.10.2 查接头与屏蔽。绝对值还会被计数器溢出清零，单看总数会误判。

第一周里还有一件事必须做：抓一次“一切正常”时的报文存档。以后出问题，拿它作基准比对，能立刻分辨“是网络变了”还是“本来就长这样”。没有这份基准，所有的“以前正常”都只是回忆。

周末做一次基线归档，内容三样：三张趋势图、一份组态备份、一份地址与命名表。三样东西放在同一个目录，下一班人能直接接手。

第一周不动参数，还有一个理由：磨合期的延迟本身在变。机械部件走合后摩擦下降，对象时间常数会小幅变小，穿越频率随之抬高，原来够用的裕度会被吃掉一点。等第一周过去，对象稳定了，再整定才有意义。

89.11 优缺点与适用边界

89.11.1 五种总线的优缺点

表 89-18 把本章涉及的六种技术放在一起比。比较的维度不是“速率”，而是控制工程师真正在意的四项：周期能力、同步精度、诊断能力、布线成本。

表 89-18 六种工业通信技术的四项能力对比

技术	周期能力	同步精度	诊断能力	布线成本	主要边界
Modbus RTU	10 ms 以上	无	弱（只有超时）	最低	轮询式，站多则刷新慢
CANopen	1–10 ms	弱	中（心跳、节点保护）	低	带宽 1 Mbit/s，轴数受限
Profibus DP	1–10 ms	中	中（站诊断）	中	需专用芯片，逐渐被以太网替代
Modbus TCP	10 ms 以上	无	弱	低	抖动不可控，不能做快回路
Profinet RT/IRT	1 ms / 0.25 ms	高（IRT）	强	中高	IRT 需专用交换机
EtherCAT	0.1 ms 起	很高（<100 ns）	强	中	主站要求高，拓扑受限

这张表要竖着看列。周期能力一列说明“能不能做快回路”，同步精度一列说明“能不能做多轴”，诊断一列说明“故障好不好查”，成本一列说明“老板批不批”。

没有一种技术在四项上都最好。EtherCAT 在周期与同步上领先，主站与组态的门槛也最高；Modbus RTU 在成本上无敌，其它三项都弱。选型的本质是承认取舍，而不是找全能选手。

89.11.2 轮询式与周期式：两种范式

Modbus RTU 与 EtherCAT 的差别不只是速率，而是范式。前者是“主站问、从站答”，后者是“主站广播、从站边传边改”。

轮询式的优点是简单：没有时钟同步、没有分布式时钟、没有复杂状态机，一个字节一个字节地读写寄存器，调试用串口助手就能做。缺点是刷新率随站数线性下降，32 站 9600 bit/s 时单站只有 0.73 Hz。

周期式的优点是刷新率与站数几乎无关：EtherCAT 一帧穿过所有从站，100 站与 1000 站的周期只在同一数量级里差几倍。缺点是对主站要求高，从站要有专用芯片，拓扑与布线要守规矩。

范式的边界在“站数 × 刷新率”这个乘积上。站少、刷新慢，轮询式最划算；站多或刷新快，必须换周期式。中间地带（10 站、20 ms）两种都能做，选便宜的那个。

现场很少有单一协议。常见组合是运动控制用 EtherCAT、仪表用 Modbus RTU、上位机用 Modbus TCP，中间靠网关转。这种结构要格外注意两件事。

一是网关两侧的刷新率要分别记账。网关不是透明的，它把一侧的帧拆开、重新打包、再发到另一侧，延迟至少翻倍，抖动也叠加。上游 10 ms、下游 50 ms 的组合，端到端不是 10 ms 也不是 50 ms，而是两者相加再加网关自身的处理时间。

二是网关是单点。一个网关挂掉，挂在它后面的所有仪表同时失联，故障现象看起来像“一大片站同时坏了”。诊断时先看网关的心跳，再看下游各站，能省掉大量逐站排查。

按这两条，混合拓扑的设计原则是：把慢的一侧尽量下沉，让网关只承担“翻译”而不承担“实时”，需要闭环的站不要放在网关下游。

89.11.3 现场总线与工业以太网的边界

现场总线（Profibus、CAN、Modbus RTU）与工业以太网（Profinet、EtherCAT、EtherNet/IP）的分界，常被说成“新旧”，其实不是。

真正的分界是“是否需要与信息技术融合”。需要与 MES、云平台、数据库打通的场合，以太网是唯一选择，因为协议栈天然兼容。纯设备内部、封闭运行的场合，现场总线更简单可靠。

第二条分界是“是否需要微秒级同步”。现场总线的同步精度在毫秒到几十微秒，做不到多轴插补要求的百纳秒。这也是 EtherCAT 在运动控制领域取代 Profibus DP 的直接原因。

第三条分界是“维护能力”。现场总线用串口助手就能排查，工业以太网要抓包、要看协议解析。维修队伍的能力，往往比技术指标更能决定选型。

89.11.4 无线与 5G 的位置

无线在工业里的位置很明确：能用的地方很好用，不能用的地方一点都不行。

适合的场合是移动设备、旋转台、难以布线的远端、以及只做监视不参与闭环的测点。这些场合的共同点是“允许几十毫秒的延迟与偶尔的断连”。

不适合的场合是所有快回路。Wi-Fi 的延迟在几十毫秒量级，抖动与重传让最坏延迟到几百毫秒；5G 的标称延迟很低，但现场的上行调度与切换仍带来十毫秒级的抖动。按 §89.5 的判据，这个量级只够温度、料位这类慢回路。

有一个例外值得注意：无线用于“冗余备份”而不是“主通道”。主通道用有线，无线只在断线时接管，此时对延迟的要求可以放宽，因为它的任务是“别停机”而不是“保住指标”。

89.11.5 网络化控制的收益与代价

把控制回路搬到网络上，收益是省线、省柜、易扩展。代价是新增了一类失效模式：通信失效。

省线的账容易算。64 轴的点对点接线要 64 根编码器线与 64 根给定线，改成总线后只剩两根；柜体尺寸、端子数量、接线工时都跟着降。

新增失效模式的账常被漏算。原来一根线断只影响一个轴，现在一段干线断影响整段；原来干扰只进一个回路，现在通过网络耦合到所有站。这就是 §89.10.2 强调布线的原因。

正确的算法是把两者相加：省下的成本减去新增的诊断与冗余投入。若净收益为正，方案成立；若接近零，说明这个场合不值得网络化，点对点更稳。

89.11.6 本章方法的边界

本章的结论建立在三条假设上，任何一条破了，结论都要重算。

第一条是对象线性。式 (89-8) 的相位裕度只对线性系统有定义。强非线性对象（如阀门死区、摩擦换向）在延迟下可能出现极限环，这种失稳用裕度看不出来。

第二条是延迟为常数或已知分布。抖动大到让分布跨越多个拍时，等效延迟不再是一个数，式 (89-16) 的反解失效，只能靠仿真或实测。

第三条是回路之间独立。多条回路共用一条网络时，一条回路的流量变化会影响另一条的延迟。耦合强的时候，要按“网络作为共享资源”整体分析，而不是逐条回路算。

三条假设都成立时，本章的式子可以直接用；破了任何一条，式子只能给量级，最终结论要靠 §89.6 的仿真或现场实测。

89.11.7 五个问题定方案

回到选型。把 §89.3 的五步压缩成五个问题，按顺序回答，方案基本就定了。

第一问：最快要多少刷新？答案小于 1 ms，只有 EtherCAT 与 Profinet IRT；1 到 10 ms，CANopen、Profinet RT、Profibus DP 都可；10 ms 以上，Modbus 系足够。

第二问：要不要多轴同步？要，选带分布式时钟或 IRT 的方案；不要，同步能力可以不看，省下的钱花在布线上。

第三问：站数与距离多少？站多距离远，优先线型或环型；站少集中，星型更好查故障。

第四问：能不能停？不能停，加冗余，并且确认恢复时间小于允许的连丢帧数。

第五问：谁来维护？维修队伍只会用串口助手，就别选需要抓包分析才查得动的方案。

五问的答案合在一起，就是图 89-9 流程的输出。若五问的答案互相矛盾（例如既要小于 1 ms 又要最低成本），说明需求本身要先砍，而不是技术在打架。

89.11.8 与前后章的分工

本章在第 86 到 93 章这条线上，位置是“连接层”。前三章各管一段计算，本章管它们之间的数据怎么走。

第 86 章管任务调度，回答“一个 CPU 上多个周期任务怎么排”。它的输出是任务周期与抖动，正是本章的输入：任务抖动会叠加到发送时刻上，§89.8.6 的案例就是这个机理。

第 87 章管嵌入式控制器，回答“一拍之内怎么把电流环算完”。它的输入是本章送来的给定值，输出是本章送走的反馈值；两章的接口就是 PDO 映射与刷新周期。

第 88 章管 PLC，回答“一次扫描的五个阶段怎么分配”。它的扫描周期与本章的刷新周期是两条独立的时间轴，开关量抖动常常是两者拍频的结果。

往后看，第 90 章讲控制软件工程，本章的组态、地址、映射表会成为它的配置项；第 91 章讲数字孪生与边缘计算，本章的实时数据是孪生体的输入源；第 92 章讲工程问题，本章的失效模式会以“通信故障”的形态再次出现。

这条线上有一条共同的判据：任何一层的时间抖动，都会以相位损失的形式进入闭环。区别只在于折算系数：任务抖动按调度周期折算，扫描抖动按扫描周期折算，网络抖动按 ω_c 折算。本章给了最后这一种的算法。

89.11.9 一页纸的结论

把全章压成一页，六条。

第一条，延迟要按度数算，不按毫秒算。折算系数是 ω_c，快回路每毫秒 3.8°、慢回路每毫秒 0.057°，差 66 倍。这条决定了“同一段网络对不同回路意义完全不同”。

第二条，抖动比延迟更坏。同样 10 ms 基准，加 0 到 20 ms 抖动，快回路 IAE 从 177.7% 涨到 623.1%。降抖动是收益最高的优化，因为它同时改善平均值与尾部。

第三条，丢包看连续性，不看总数。5% 单帧丢失几乎无害，5% 连丢 20 帧让 IAE 涨 53.5%。冗余的价值在于截断连丢长度。

第四条，利用率有拐点。0.7 以下排队可算，0.9 以上按双曲线发散。评估要看峰值而不是平均值，突发流量能让瞬时利用率瞬间到 1.0。

第五条，算带宽要算全。线上、转发、主站三段相加，64 轴包装线 188.5 μs 里有 88 μs 不是线上时间。只算线上会把利用率低估一半。

第六条，先修网络再整定。控制器参数依赖延迟，顺序颠倒要返工。现场按刷新率、抖动、连丢、同步四步查，从慢到快，不跳步。

六条之外还有一句：本章的所有判据都建立在“线性对象、确定延迟、回路独立”三条假设上。假设不成立时，式子只能给量级，结论要靠仿真与实测。

89.12 Python 仿真代码

89.12.1 两个脚本与运行方式

本章的全部数字来自两个脚本，放在 `book/code/` 下。`ch089_sim.py` 只算数、只打印，不出图；`ch089_figs.py` 导入前者，出十张图。分开的好处是改参数时不必重画出图，也便于逐段核对。

运行方式是在书稿根目录执行

```
python book/code/ch089_sim.py
python book/code/ch089_figs.py
```

第一个脚本约十几秒跑完，输出九段报告；第二个脚本几秒出图，产物写到 `book/figs/ch089/`，每张图同时生成 SVG 与 PDF。

脚本不依赖第三方库之外的任何东西，只用 NumPy 与 Matplotlib。这样做的原因是读者拿到脚本就能跑，不必配环境、不必装仿真平台。

89.12.2 参数区：改哪里

所有可调的量集中在文件开头，两个对象各一个字典。

```
SEED = 20261023
SLOW = dict(tau=2.0, dead=0.30, kc=2.00, ti=2.00, scan=0.010)
FAST = dict(tau=0.05, dead=0.005, kc=3.33, ti=0.05, scan=0.001)
```

`tau` 是对象时间常数，`dead` 是过程纯滞后，`kc` 与 `ti` 是 PI 参数，`scan` 是采样周期。两组对象只差三个数量级，却给出完全不同的结论，这正是 §89.6 想说明的事。

`SEED` 控制随机数。抖动与丢包都用它初始化，改这个数会得到不同的具体数值，但趋势不变。正文引用的是这个种子的结果，重跑逐位一致。

89.12.3 九段报告的索引

表 89-19 是九段输出与正文小节的对照。想核对某个数，按这张表找段落即可。

表 89-19 九段数值报告与正文小节的对照

段	输出内容	对应小节
[1]	三类总线的帧时间与轮询周期	§89.2.1–§89.2.3
[2]	排队延迟随利用率的变化	§89.2.5
[3]	两条回路的延迟-裕度表	§89.4.3
[4][4b]	慢、快回路的五档闭环指标	§89.6.3
[5][5b]	丢包率与连续丢帧长度的影响	§89.6.4
[6]	抖动幅度对快回路的影响	§89.6.5
[7]	五种同步方式的误差	§89.2.8
[8]	六类总线的周期、节点数与容量	§89.2.9
[9]	64 轴包装线的通信预算	§89.3.3

[4] 与 [5]、[5b] 是三段中最慢的，因为每组工况要跑多个种子。慢回路的时间常数 2 s，仿真时长 20 s、步长 1 ms，单次两万步；快回路时长 0.5 s、步长 0.1 ms，单次五千步。

89.12.4 三个实现要点

第一是延迟的实现。脚本没有用 z^{-d} 的整数拍近似，而是用一个先进先出队列：每拍把 $(\text{到达时刻}, \text{控制量})$ 压进队列，到点弹出。这样抖动与连丢都能自然表达，不必预先算拍数。

```
d = net_ms / 1000.0
if jitter_ms > 0.0:
    d += rng.uniform(0.0, jitter_ms / 1000.0)
queue.append((now + d, u))
```

第二是连丢的实现。丢包不是每拍独立判断，而是“一旦触发就连丢 n 拍”。

```
if drop_left == 0 and loss > 0.0 and rng.random() < loss:
    drop_left = burst
if drop_left > 0:
    drop_left -= 1
```

这个写法是 §89.6.4 那条结论的直接来源。首版只丢单拍，结果 5% 丢包几乎看不出影响；改成连丢后，同样 5% 的丢包率立刻分出两个量级。

第三是相位的计算。脚本用解析式而不是 `atan2`：

```
ph = (-math.atan(wc*plant["tau"])
      - math.atan(1.0/(wc*plant["ti"]))
      - wc*delay)
```

首版用 `atan2(imag, real)` 求相角，在 $-180°$ 处回绕，裕度被算成 352.7° 这类明显不合理的值。改成解析式后单调下降，快回路 20 ms 得 $-7.3°$，50 ms 得 $-121.8°$，与手算一致。

89.12.5 十张图怎么生成

十张图分四类。数据类三张（帧时间、轮询、排队），曲线类两张（裕度、阶跃），流程图四张（拓扑、协议栈、选型、EtherCAT 周期），架构类一张（图 89-10 的三网部署）。

数据类与曲线类从 `ch089_sim` 取数，流程图用 `flow_styles` 按坐标摆放，只用圆角矩形、菱形、平行四边形与泳道，不出传递函数方块。这条约束来自 L13 的图元规范。

图内的中文用普通文本，公式单独用数学环境，两者不写进同一个字符串。这条约定是为了避免字体回退：Matplotlib 遇到混排会退回默认字体，中文变方框。

图号与文件序号一一对应，`fig_089_05` 就是图 89-5。全章十张图都在正文引用过一次以上，图注写在 Markdown 里而不是画进图内。

89.12.6 五个可以自己改的实验

第一，把 `FAST["kc"]` 从 3.33 改成 2.0，重跑 [3] 与 [4b]。可以看到穿越频率降到 40 rad/s，10 ms 延迟的裕度从 30.9° 回到 54.5°，超调从 33.5% 降下来。这就是 §89.8.4 的降增益路线。

第二，把 `FAST["scan"]` 改成 0.0005，重跑 [4b]。采样周期减半，半拍滞后减半，但抖动占周期的比例翻倍，能看出 §89.9.3 的拐点。

第三，把 `burst` 从 1 改成 5、10、20，重跑 [5b]。可以直接看到“连丢长度”对指标的非线性影响。

第四，把 `SLOW["dead"]` 从 0.3 改成 0.5，重跑 [3]。慢回路的起点裕度从 72.5° 降到 61.1° 附近，说明对象本身的滞后与网络延迟是加法关系，不是乘法。

第五，改 `BUS_SPECS` 里的字节数或周期，重跑 [8]。这张表是半经验值，读者若有厂商实测数据，替换后更贴近自己的项目。

五处改动都只动常量，不动结构。这也是把常量集中在开头的原因：读者不必读懂全部实现，就能用自己的数替换掉本章的数。改完重跑，正文引用的数会与报告不一致，这时以报告为准，按 §89.10.6 的方式重新记录。

89.12.7 三项自检

脚本出数之后要做三项自检，任何一项不过，正文的数就不能用。

第一项是手算对照。挑三个关键数手算一遍，与脚本比。64 轴的单周期 188.5 µs：$1256 \times 8/100 = 100.48$，加 68 与 20，得 188.48。快回路 10 ms 的裕度 30.9°：$90 - 20.99 - 38.16$，得 30.85。CAN 8 字节的最坏帧 270 µs：$(47 + 64 + 24) \times 2$，得 270。三个都对，说明公式与代码没有错位。

第二项是单位检查。本章最容易错的单位是弧度与度。相位裕度用度，$\omega_c T_{\text{net}}$ 用弧度，两者相乘前必须统一。§89.4.5 的反解式里，分子是度、分母是 rad/s，结果才是秒；若把 rad/s 当成 Hz，结果会差 2π 倍。

第三项是图表一致。正文、表、图必须同数。改一个常量后三处一起变，不能只改一处。本章的做法是让图与表都从 `ch089_sim` 取数，正文手抄时对照最近一次报告。

三项都过之后再进排版。排版阶段再发现数字不一致，改起来要同时动正文、表、图与 PDF，代价最高。把自检放在出图之前，是本章从第一个脚本就定下的顺序。

自检还有一个副产品：它会逼着作者把每个数的来源写清。来源写不清的数，通常也是算错的数。

89.12.8 把脚本换成自己的数

脚本的数可以换，结构不用换。换成自己项目的数分三步，每步只改一处。

第一步换总线参数。`BUS_SPECS` 里每一条的节点数、单节点字节数、周期都按公开资料估的，不是哪家厂商的实测值。手上有手册的，照手册替换；没有的，用主站软件读一次实际周期时间，反填进去。

第二步换对象模型。`SLOW` 与 `FAST` 两个字典里的四个量，从现场做一次阶跃响应辨识得到，辨识方法见第 44 章。若只有经验没有数据，用“调节阀动作到温度开始变化的时间”作 `dead`，用“变化到 63% 的时间”作 `tau`，量级够用。

第三步换网络条件。`net_ms` 填主站统计的中位延迟，`jitter_ms` 填峰峰值的一半，`loss` 填一周的错误计数除以总帧数。三个数都不必精确，因为结论对它们不敏感——量级对了，结论就对。

三步换完重跑，得到的是“自己项目”的量级，不是验收值。验收值仍要靠实测，理由与 §89.6.9 相同：仿真里省掉了执行器动态、饱和、量化与摩擦。

有三件事不要做。不要用脚本结果代替实测；不要为了让曲线好看去改仿真结构；不要把 `SEED` 换到“结果顺眼”的那一个，种子只该用来做重复性检验。

脚本的定位要摆正：它算方向与量级，回答“这段延迟要不要紧”“抖动占多大比例算大”，不回答“这套设备能不能验收”。把它当成一把尺子，而不是一张合格证。

89.12.9 把报告读一遍

九段报告不是九张独立的表，读完一遍要能连成一条线。这里按真实输出走一遍，说清每段该看哪一列。

先看 [1] 的第一行：

```
Modbus RTU 9600：读 10 个寄存器往返 42.67 ms
```

这个数由两段构成。传输段是 40 个字符乘每字符 1.0417 ms，得 41.67 ms；从站处理段按 1 ms 计。40 个字符的来历是请求 8 字节、响应 25 字节、两端各 3.5 个字符的帧间隔。手算与脚本对上，说明帧结构没有记错。

再看 [3] 的右半列：

```
延迟    10 ms：穿越   66.6 rad/s，裕度  30.9 度
延迟    20 ms：穿越   66.6 rad/s，裕度  -7.3 度
```

两行的穿越频率一模一样，只有裕度在掉。这正是纯滞后的签名：幅值恒为 1，只改相位。若某次改参数后连穿越频率也变了，那改的不是网络，是回路增益或对象时间常数。

接着看 [4b] 与 [5] 的同一工况：

```
10 ms + 5% 丢包   ：超调  33.6%，调节 0.225 s，IAE 0.0456
丢包  5.0%：超调  33.7%，调节 0.226 s，IAE 0.0458
```

两处差 0.1 个百分点。原因不是算错，而是统计口径：[4b] 用 3 个种子，[5] 用 10 个种子。差别落在噪声里，说明“单帧丢包”这个因素本身就不显著——结论与 §89.6.4 一致。

最后看 [8] 与 [9] 的关系。两段的口径不同：[8] 回答“这条链路每周期能搬多少字节”，[9] 回答“这个项目每周期需要多少字节”。前者是能力，后者是需求，两者不能相减，只能相除得到利用率。

推荐的读法顺序是先看需求再看能力：先 [9] 得到项目要多少，再 [8] 挑候选链路，然后 [3][4] 核时序，最后 [5][6] 看异常工况。按这个顺序读，任何一步不通过就能立刻停下，不必把九段全看完。

九段里最容易被跳过的是 [2]。它没有对应的正文表格那么显眼，但利用率一旦超过 0.7，后面所有延迟数字都要重算——排队延迟是按式 (89-6) 加到每一帧上的。读报告时把它当门禁，而不是参考。

第 90 章 控制软件工程

90.1 动机与直观理解

90.1.1 三个现场场景

第一个场景是版本丢失。一条投运三年的生产线要改一处报警阈值，工程师打开共享盘，发现源码只有一个名为“最终版2”的压缩包，现场跑的到底是哪一版没人说得清。确认版本花了两周，改代码只花了十分钟。

第二个场景是无法回滚。有人把滤波系数从 0.2 改成 0.35，实验室阶跃响应更快了，现场却出现整线抖动。想退回上一版，才发现上一版只存在于某位工程师的笔记本里，而那台机器上周刚重装系统。

第三个场景是追溯断链。验收会上业主问：“这条安全联锁的需求，是哪一版代码实现的，哪个用例验证的？”在场的人都能说出“肯定实现了”，但没有人能在十分钟内拿出证据。

三个场景的共同点不是算法错，而是证据没留下。控制软件工程要解决的，正是“让每一句已经验证过都能指向一个可复现的动作”。

90.1.2 控制软件与普通软件的四个差别

控制软件不是普通的应用软件，有四个差别决定了它不能照搬互联网那套做法。

第一是寿命长。一台设备的机械部分设计寿命十五年，控制软件往往要跟着走完整个周期，中间换三代硬件、五批维护人员。为三个月的项目设计的流程，撑不到第十年。

第二是资源紧。目标板可能只有几十 KB 内存、几十 MHz 主频，没有操作系统、没有动态分配、没有异常栈展开。单元测试要在宿主机上跑，跑通了不等于目标板上跑得通。

第三是与物理世界耦合。一个空指针在网页上是 500 错误，在伺服轴上是撞机。软件缺陷的后果不是“不好用”，而是“会伤人”。

第四是要认证。流程工业、汽车、轨道交通、医疗设备都有对应的功能安全标准，标准要求的不是“代码写得好”，而是“有证据证明代码写得好”。

90.1.3 与相邻章的分工

本章在第六部分的位置是“生产流程”，不是“运行时机制”。

第 86 章讲一个 CPU 上多个周期任务怎么排，输出是任务周期与抖动。第 87 章讲一拍之内怎么把电流环算完，输出是指令级的时间预算。第 88 章讲 PLC 一次扫描的五个阶段。第 89 章讲这些计算单元之间的数据怎么走。

这四章回答“程序跑起来是什么样”，本章回答“程序是怎么被写出来、被验证、被留下记录的”。往后，第 91 章的数字孪生要用本章的模型与版本，第 92 章的工程问题会以“改一处、坏一片”的形态再次出现。

分工的判据很简单：凡是与时间刻度、寄存器、报文有关的，属于前四章；凡是与提交、构建、测试、追溯有关的，属于本章。

90.1.4 五个关键词

本章围绕五个词展开，先把它们的一句话定义摆出来。

版本管理是把“每一次改动”变成一个有编号、可回溯、可比较的对象。它的产物不是代码，是历史。

单元测试是把“这段代码在给定输入下应该输出什么”写成可自动执行的断言。它的产物不是覆盖率，是回归能力。

持续集成是把“编译、测试、打包”串成一条每次提交都自动跑的流水线。它的产物不是速度快，是反馈早。

代码生成是把控制模型自动翻译成目标代码，把手工誊写换成机械翻译。它的产物不是省人力，是消除誊写错误。

需求追踪是把需求、模型、代码、用例用同一套编号串起来。它的产物不是文档，是“能被证明”。

五个词不是并列的五件事，而是一条链：编号贯穿全程，版本记录变化，测试提供证据，流水线保证证据新鲜，生成减少人工介入。图 90-7 画的是这条链在 V 模型上的位置，图 90-8 画的是支撑它的工具链。

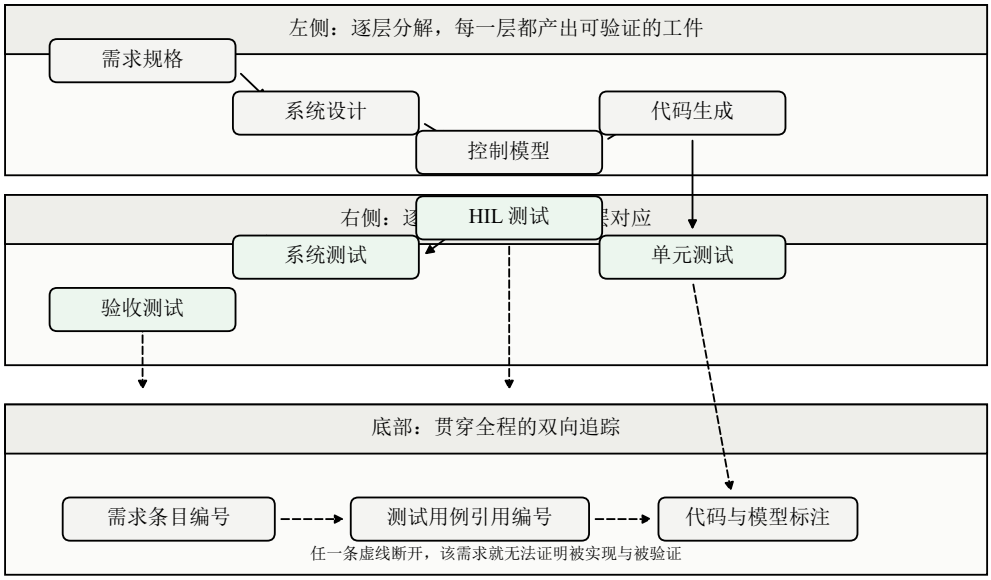


图 90-7 双 V 与三层追踪

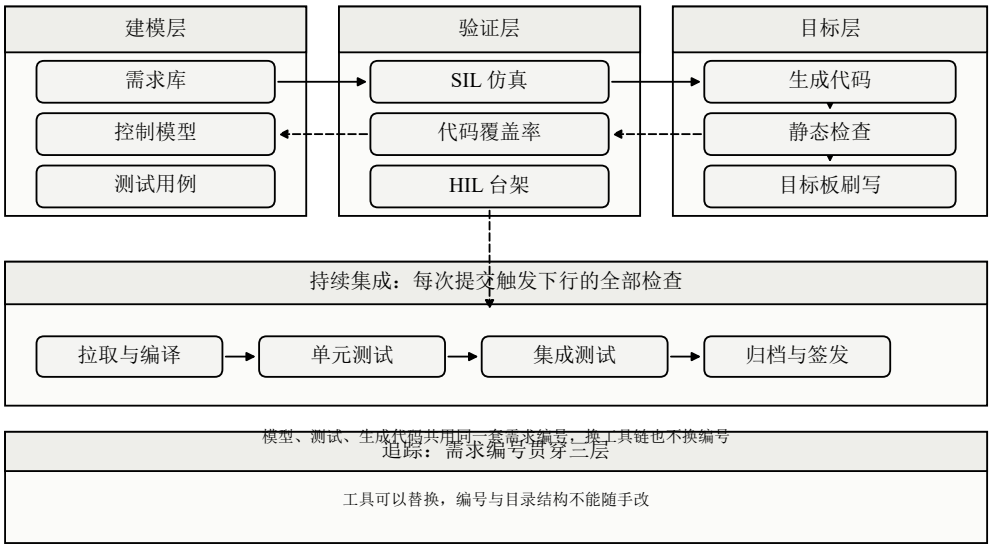


图 90-8 从模型到目标板的一条工具链

90.1.5 一条主线：把证据留下来

本章所有方法都可以归到一句话：把“已经验证过”变成可复现的动作。

“我测过了”不是证据，“用例 `TC-042` 在提交 `a3f19c` 上通过，日志在构建 `#1187` 里”才是证据。前一句依赖人的记忆，后一句依赖系统。

这条主线解释了很多看似繁琐的规定。为什么提交信息要写清改了什么？因为三个月后要靠它定位。为什么构建要留归档？因为现场那台设备跑的必须是某个确定的构建。为什么用例要编号？因为验收时要能指着编号说话。

反过来，任何一条规定如果无法产出“可复现的动作”，它就是形式主义，应当删掉。这条判据在 §90.11 讨论流程轻重时会再用一次。

90.1.6 三把时间尺子

软件工程里的“快”与“慢”，要按三把尺子量。

第一把是提交。一次提交从写到推，正常在几分钟到几小时。用这把尺子量，等十分钟的构建已经算慢。

第二把是构建与反馈。一次完整的编译加测试，从几十秒到几十分钟。用这把尺子量，十分钟的反馈属于“能接受”，一小时的反馈属于“当天不想再提交第二次”。

第三把是发布。控制软件的发布以周或月计，因为要停机窗口、要现场验证、要业主签字。用这把尺子量，前面两把尺子上的十分钟、一小时都被抹平了。

三把尺子给出不同的优先级。缩短提交到反馈的时间，收益落在开发者身上；缩短发布周期，收益落在业主身上。两者都要，但前者是后者的前提——反馈慢的团队，不敢频繁发布。

90.1.7 本章不做什么

本章不讲编程语言的语法，不讲某个 IDE 的用法，不判断哪家工具便宜，也不做功能安全认证的法务解释。

本章讲的是方法与账目：哪些做法值得做、做多少、能换回什么。工具只是方法的载体，换一套工具，这些账还算得出来。

90.1.8 一份现场问题清单

把控制项目里最常见的软件问题列成清单，本章后面的每一条方法都对应着清单上的一项。

第一项，现场装的版本与测试的版本不一致。表现是“测试好好的，现场不对”。根因是没有从标签重新构建，发出去的是某人本地编的产物。

第二项，改了一个参数，忘了改另一处。同名的限幅值在两处定义，改一处不生效。根因是没有单一参数源，标定表与代码各存一份。

第三项，回滚包不可用。出事时想退回上一版，发现旧包缺依赖或授权过期。根因是回滚包从未被验证。

第四项，需求改了，测试没跟上。验收时被问“这条需求怎么证明”，答不上来。根因是追踪链断在设计到用例这一层。

第五项，偶发问题查不下去。日志里只有“任务超时”，没有当时的输入与内部状态。根因是日志设计只记事件不记上下文。

第六项，一个人离职，模块没人敢动。根因是没有测试，改动无法验证，只能不动。

第七项，同样的问题修过三次。每次修完不补用例，下次重构又把坑挖回来。

第八项，构建在别人机器上失败。根因是编译器版本、依赖版本没有固化。

八项里没有一项是“算法算错了”。这就是控制项目的现实：软件事故多数来自过程，而不是来自控制理论。本章要处理的正是这八项。

90.2 数学原理

90.2.1 缺陷成本随阶段上升

软件工程里最稳的一条经验是：缺陷发现得越晚，修起来越贵。把它写成式子，设需求阶段修一个缺陷的工时为 c_0，第 i 阶段的相对倍数为 r_i，则该阶段单个缺陷的成本

$$C_i = c_0 \, r_i \tag{90-1}$$

本章取 $c_0 = 0.5$ h，六个阶段的 r_i 依次为 1、5、10、20、50、150。这组倍数不是精确测量值，而是行业里反复出现的量级：每往后一个阶段，成本大约翻一倍到两倍。

按式 (90-1)，单缺陷成本从需求的 0.5 h 涨到现场的 75 h，差 150 倍。20 个缺陷如果在单元测试阶段全部修掉，总工时 200 h；如果全部漏到现场，总工时 1500 h。两者相差 1300 h，相当于一个工程师大半年的工作量。

图 90-1 把这组倍数画成柱状，纵轴取对数——因为线性轴上第一根柱子会被压成一条线。

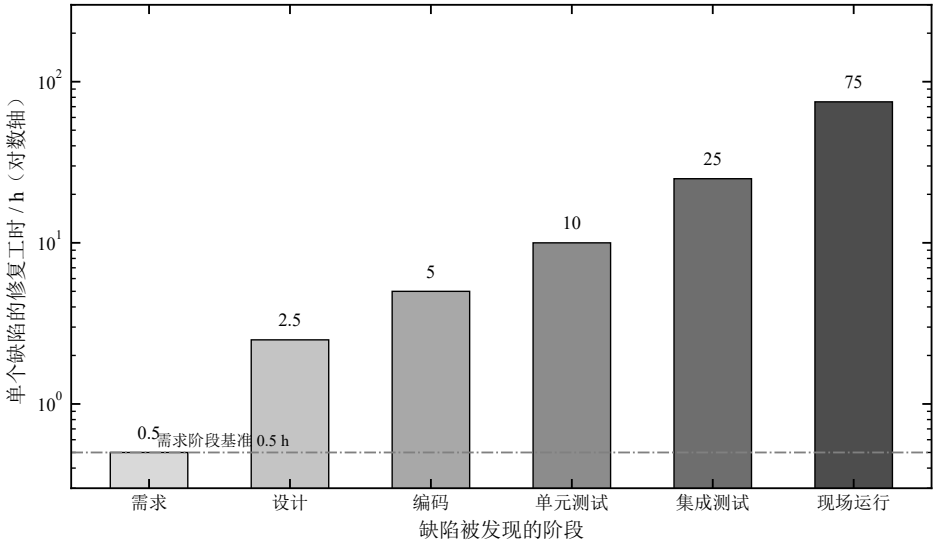


图 90-1 缺陷修复成本随发现阶段的变化

这张图最常见的误读是“所以要拼命做需求评审”。倍数最大的确实是早期阶段，但早期阶段能发现的缺陷数量少；真正划算的是“在编码之后、集成之前”这一段，因为那里缺陷最密集，而成本还没起飞。

90.2.2 可靠性增长

测试过程可以看成一个“缺陷被逐个发现”的过程。设软件里一共有 a 个缺陷，单位时间发现缺陷的比例为 b，则累计发现数

$$m(t) = a\left(1 - e^{-bt}\right) \tag{90-2}$$

对时间求导得到失效率

$$\lambda(t) = a b \, e^{-bt} \tag{90-3}$$

平均无故障时间

$$\mathrm{MTBF}(t) = \frac{1}{\lambda(t)} \tag{90-4}$$

式 (90-2) 到 (90-4) 就是指数型可靠性增长模型。它只有两个参数：a 是“总共有多少缺陷”，b 是“发现得多快”。

本章取 $a=120$、$b=0.08$/天。代入式 (90-3)，第 0 天的失效率是 9.6 个/天，MTBF 只有 2.5 h；第 30 天失效率降到 0.871 个/天，MTBF 27.6 h；第 60 天失效率 0.079 个/天，MTBF 303.8 h。

图 90-2 是这条曲线的对数纵轴版本。取对数之后曲线接近直线，这是指数模型的标志——在对数轴上表现为匀速下降，说明“每过固定天数，失效率减半”。

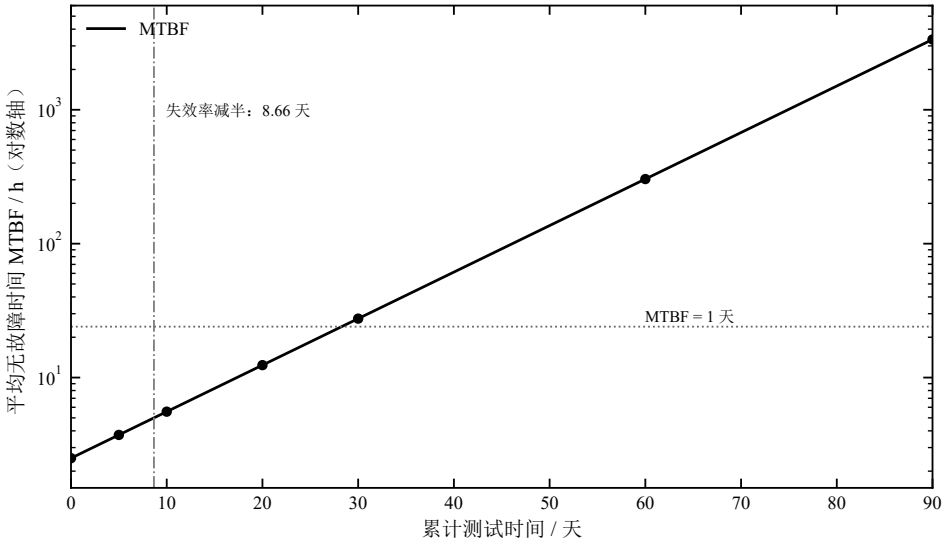


图 90-2 指数型可靠性增长曲线

由式 (90-3) 可以直接算出失效率减半所需的时间

$$t_{1/2}=\frac{\ln 2}{b} \tag{90-5}$$

代入 $b=0.08$，得 8.66 天。这个数比 a 更有用：它告诉你“测试强度够不够”。如果两个月过去失效率还没减半，说明 b 太小，测试没有打到缺陷密集的地方。

90.2.3 覆盖率是饱和曲线

覆盖率不随用例数线性增长，它饱和。设覆盖率上限为 $c_{\max}$，特征用例数为 n_0，则 n 个独立用例能达到的覆盖率

$$c(n)=c_{\max}\left(1-e^{-n/n_0}\right) \tag{90-6}$$

反解出“要达到目标覆盖率 c 需要多少用例”

$$n=-n_0\ln\left(1-\frac{c}{c_{\max}}\right) \tag{90-7}$$

本章取 $c_{\max}=0.98$、$n_0=25$。5 个用例覆盖 17.8%，25 个覆盖 61.9%，50 个覆盖 84.7%，100 个覆盖 96.2%。

按式 (90-7)，达到 90% 需要 62.6 个用例，达到 95% 需要 87.2 个。这两个数是本章最常用的两个——它们把“覆盖率要到多少”从口号变成了工作量。

图 90-3 画的是这条饱和曲线。曲线在 50 到 100 个用例之间最陡，之后迅速变平：从 90% 到 95% 要多花 25 个用例，从 95% 到 98% 要多花上百个。

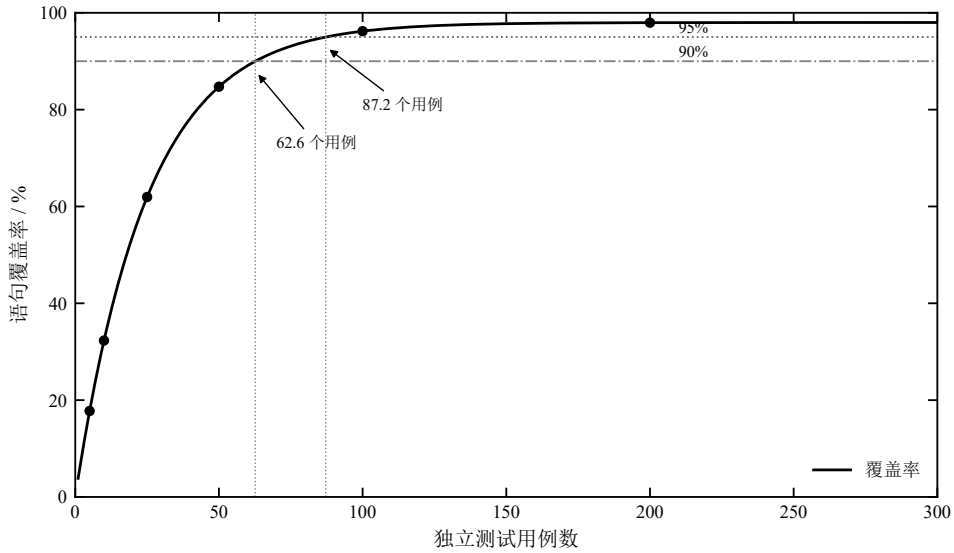


图 90-3 覆盖率随用例数的饱和曲线

饱和的原因是“容易覆盖的先被覆盖”。前 25 个用例打的是主路径，一行一个用例；后 25 个打的是异常分支，一个用例要走五六个条件组合才碰到一行。这就是为什么覆盖率越往上越贵。

90.2.4 追踪链是乘积，不是平均

需求追踪通常分三层：需求到测试、测试到代码、代码回到需求。三层的完整率不是取平均，而是相乘

$$P = \prod_{i=1}^{3} p_i \tag{90-8}$$

三层都是 90% 时，端到端只有 72.9%。240 条需求里有 65 条断链。三层都是 95% 时端到端 85.7%，断链 34 条；三层都是 98% 时才到 94.1%，断链 14 条。

更要紧的是短板效应。若有一层只有 70%、另两层 95%，端到端是 63.2%，比“三层都 90%”还差。这说明补齐最弱的一层，收益远大于把已经不错的两层再提高几个点。

这条乘积规律是本章最容易被忽略的算术。评审时常见的说法是“我们三层都做到了 90% 以上”，听上去不错，实际每五条需求就有一条无法完整追溯。

90.2.5 分支冲突随寿命线性增长

合并冲突的数量，可以用一个粗估式。设并行开发人数 N_d，每人每天改动的文件数 d，分支存活天数 t，同一文件被两人同时改的概率 ρ，则冲突文件数

$$N_c = (N_d - 1)\, d\, t\, \rho \tag{90-9}$$

本章取 $N_d = 6$、$d = 3$、$\rho = 0.15$。分支存活 1 天时冲突 2.25 个文件，解决工时 18 min；存活 5 天时 11.25 个文件、90 min；存活 20 天时 45 个文件、360 min。

式 (90-9) 对 t 是线性的，这一点常被低估。很多人以为“分支开久一点没关系，最后合一次就行”，但冲突量是随天数匀速累积的，而且解决冲突本身又会引入新缺陷——本章按每个冲突 3% 计，20 天的分支平均引入 1.35 个缺陷。

图 90-4 把冲突文件数与解决工时画在一起。两条线形状相同，只是纵轴刻度差 8 倍，因为工时就是冲突数乘 8 min。

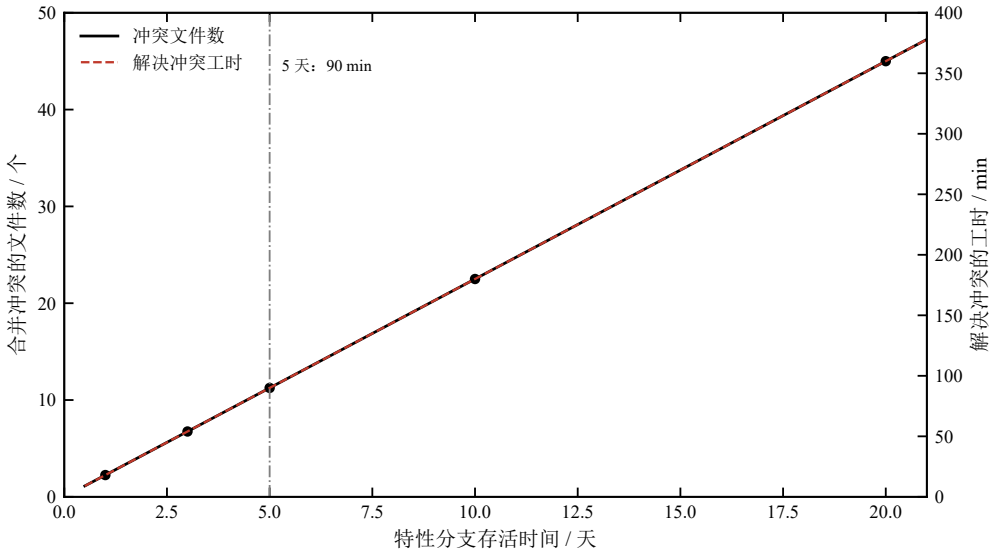


图 90-4 分支寿命与合并冲突

90.2.6 反馈时间决定批大小

持续集成的价值不在“自动”，在“早”。反馈时间越长，开发者越倾向于攒着提交，一次构建覆盖的提交数越多，出错时的定位范围越大。

设反馈时间 T_{fb}、平均提交间隔 T_c，则一次构建覆盖的提交数

$$B = \left\lceil \frac{T_{\text{fb}}}{T_c} \right\rceil \tag{90-10}$$

若每次提交平均改动 ℓ 行，定位范围就是 $B\ell$ 行。

本章取 $T_c = 6$ min、$\ell = 40$ 行。全量流水线 602 s，即 10.0 min，$B = 2$，定位范围 80 行；增量流水线 240 s，即 4.0 min，$B = 1$，定位范围 40 行。

这个模型的结论很直接：反馈时间翻倍，定位范围就翻倍。而定位范围翻倍，排错时间往往不止翻倍——因为要在两倍的改动里找出哪一处引起的。

90.2.7 回归策略的成本

回归测试要选策略：全量、按模块、按影响分析、只跑冒烟。四种策略的速度差几十倍，漏检率也差几十倍。

设一次运行的耗时为 T_{run}，每周可捕获的缺陷数为 D，漏检率 p，则发现单个缺陷的平均成本

$$q = \frac{T_{\text{run}}}{D(1-p)} \tag{90-11}$$

本章取 $D = 3$、每个用例 0.6 s。全量 520 个用例，312 s，漏检 0，单缺陷成本 104 s；按模块 130 个用例，78 s，漏检 4%，成本 27.1 s；按影响分析 42 个用例，25.2 s，漏检 7%，成本 9.0 s；只跑冒烟 12 个用例，7.2 s，漏检 35%，成本 3.7 s。

单看式 (90-11)，冒烟最快。但它每周漏 1.05 个缺陷，这些缺陷会流到集成或现场，按式 (90-1) 的成本倍数算，省下的时间远不够赔。

正确的用法是把式 (90-11) 与式 (90-1) 连起来：冒烟省下 300 s，漏掉的缺陷如果到现场才被发现，单次成本 75 h。这笔账决定“什么时候必须跑全量”。

图 90-6 把四种策略画成“单缺陷成本—每周漏检”的散点，左下角是既快又准的理想区，实际点都沿一条斜线分布。

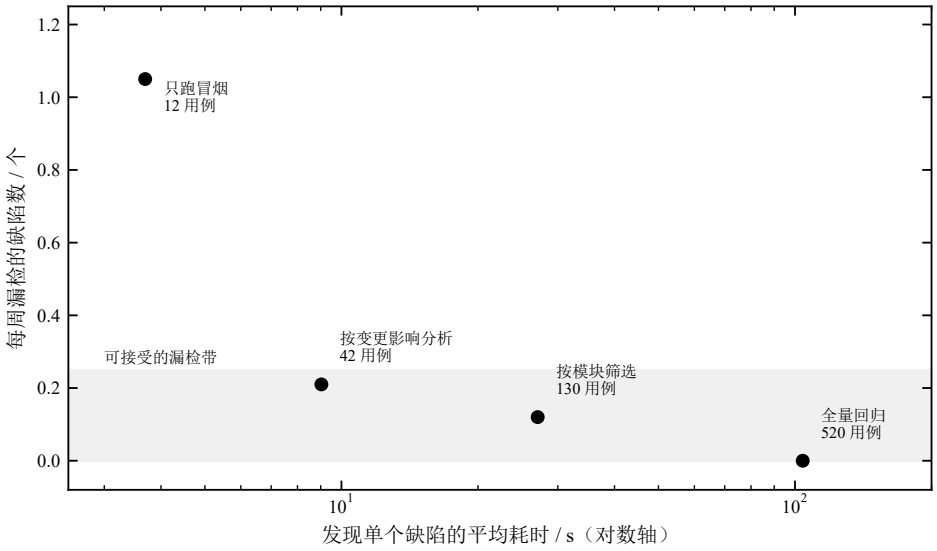


图 90-6 回归策略的速度与漏检权衡

90.2.8 代码生成把缺陷搬到模型

代码生成不消灭缺陷，它把缺陷从代码搬到模型，并且放大。

设生成代码 k_c 千行、代码缺陷密度 ρ_c，模型 k_m 千行（按等价规模计）、模型缺陷密度 ρ_m，一处模型缺陷平均影响 α 处生成代码，则总缺陷数

$$D_{\text{tot}} = k_c \rho_c + k_m \rho_m \alpha \tag{90-12}$$

本章取 $k_c = 1$、$k_m = 0.25$、$\alpha = 4$。手写代码 $\rho_c = 8$，总缺陷 8.0 个；生成的代码 $\rho_c = 0.5$，但模型未验证时 $\rho_m = 3.0$，总缺陷 $0.5 + 0.25 \times 3.0 \times 4 = 3.5$ 个。

按数量看，生成仍然更少。但按修复成本看，结论反过来了：代码缺陷在单元测试阶段发现，按 20 倍计价；模型缺陷只能在集成或系统测试发现，按 50 倍计价。

手写：$8 \times 20 = 160$；生成未验证：$0.5 \times 20 + 3.0 \times 50 = 160$。两者打平。做了模型背靠背验证后，$\rho_m$ 降到 0.6，总成本 40；再加静态检查，降到 36。

这就是很多团队“用了代码生成，缺陷没见少”的原因：省下的代码缺陷，被模型缺陷按更高的单价补回来了。

90.2.9 变更失败率与年不可用时间

把发布看成一次带风险的变更。设变更失败率 f、每次失败的恢复时间 t_r、年发布次数 R，则年不可用时间

$$U_{\text{year}} = f \, R \, t_r \tag{90-13}$$

本章取 $R = 24$ 次/年。手工发布 $f = 0.30$、$t_r = 240$ min，年不可用 28.80 h；脚本化发布 6.48 h；流水线加自动回滚 0.72 h；再加灰度 0.26 h。

从 28.80 h 到 0.26 h，差 110 倍。这个差距不是靠“写得更仔细”得来的，是靠“失败后能快速退回”得来的——t_r 从 240 min 压到 8 min，贡献了大部分改善。

这条式子还说明一件事：降低失败率与缩短恢复时间，是两条独立的杠杆。多数团队只盯前者，而后者往往更容易改进。

90.2.10 这几个模型的误差来源

这十几条式子都是粗糙模型，用之前要知道它们会在哪里偏。四处最典型。

第一处，缺陷成本倍数。1 比 150 这个比例来自航空与汽车行业的经验统计，不同产品差异很大。安全相关项目现场缺陷的代价被低估，因为漏掉了停机与召回；互联网类项目现场缺陷的代价被高估，因为一次热修只要十几分钟。倍数只用来排序，不用来报价。

第二处，可靠性增长。指数模型假设剩余缺陷按固定比例被检出，而实际测试中，先被找到的是容易触发的缺陷，越往后越难。因此真实曲线前半段比指数更陡、后半段更平，指数模型在后期会高估改进速度。

第三处，覆盖率饱和。$c_{\max}=0.98$ 是假设“有些分支永远测不到”，而 $n_0=25$ 假设用例的等效用。实际上用例质量差异极大，一条精心设计的边界用例顶十条随手写的。n_0 是拟合值，不是设计值。

第四处，分支冲突。线性模型假设冲突数随分支寿命等比增长，而实际中冲突集中在合并瞬间，且与模块边界清晰度强相关。这个模型只适合比较“分支活三天与活十天”的差别，不适合预测具体某次合并会冲突几个文件。

四处的共同点是：模型给趋势，现场给绝对值。§90.4.5 提到脚本与现场数据差三倍属正常，差一个数量级就说明假设错了。用之前先拿自己项目的历史数据校一次，比换更复杂的模型有用。

90.2.11 这些式子怎么串起来

十几条式子看起来各说各的，实际是一条链上的若干段。

起点是缺陷产生：写代码就会引入缺陷，生成代码把缺陷从代码搬到模型（式 90-12）。这一段决定“有多少缺陷”。

接着是缺陷被发现：测试投入时间，缺陷按指数规律被检出（式 90-2、90-3）。这一段决定“多快能找完”。找的时候能覆盖多少分支，由用例数决定（式 90-6、90-7）。

然后是缺陷被追踪：需求、设计、用例三层串起来，端到端完整率是三层之积（式 90-8）。这一段决定“能不能证明找全了”。

再往后是改动带来的新缺陷：分支活得越久，合并冲突越多，冲突又带进新缺陷（式 90-9）。这一段决定“改动本身会不会制造缺陷”。

最后是漏出去的缺陷：回归跑得少就漏得多（式 90-11），发布失败就停机（式 90-13）。这一段决定“漏出去以后损失多大”。

链条上游的一个数变了，下游全变。比如把生成比例提高，上游缺陷总数下降，但模型缺陷的单价更高，总成本可能不变（§90.2.8 的结论）。

理解这条链的用处是：遇到问题先定位在哪一段。缺陷太多是上游（设计与编码），找得太慢是中游（测试与覆盖率），证明不了是追踪，改动频繁出事是分支，损失太大是发布与回滚。定位错了，投入就白花。

90.3 设计步骤

90.3.1 五步与各自的输出物

搭一套控制软件工程，按五步走：给需求编号、定仓库与目录、定测试分层、定流水线分级、建追踪矩阵。

每一步都要求有输出物，因为软件的流程只有产出物才算完成。编号规则的输出是一张编号表；目录约定的输出是一个空仓库加一份说明；测试分层的输出是分层清单与用例配额；流水线分级的输出是四条流水线的定义文件；追踪矩阵的输出是三张可自动生成的表。

五步有先后依赖：没有编号，追踪无从谈起；没有目录，流水线找不到该编什么；没有测试分层，流水线不知道每级跑什么。顺序颠倒会返工。

五步做完通常需要两到三周，投入不大。真正花时间的是把它们坚持三年。

90.3.2 第一步：给需求编号

编号规则只有三条要求：稳定、唯一、可读。

稳定指编号不随排序、优先级、文档版本变化。唯一指一号一需求，不复用。可读指看编号能猜到子系统，例如 `REQ-AX-014` 一眼知道是轴控相关的第十四条。

推荐格式是“前缀 + 子系统 + 三位序号”。前缀统一为 `REQ`，子系统用两到四个字母，序号从 001 起。不要用层级编号（如 3.2.1.4），因为需求文档一改结构，所有编号都会变，历史记录全部失效。

删除的需求不回收号码，标记为废止即可。理由是旧日志、旧报告、旧提交信息里都引用过它，回收号码会让这些记录指向错误的对象。

编号由一个人统一分配，不要各自申请。多人同时编号，撞号几乎必然发生。

90.3.3 第二步：定仓库与目录

单仓还是多仓，是第一个要定的结构问题。表 90-1 给出对比。

表 90-1 单仓与多仓的取舍

维度	单仓（模型、代码、测试、标定同仓）	多仓（按模块分仓）
版本一致性	天然一致，一次提交对齐全部	需人工对齐，易错配
构建复杂度	一次拉取即可全量构建	需管理依赖版本
权限粒度	粗，按目录分权	细，按仓分权
适合规模	50 万行以内、单产品	多产品共享基础库

控制项目多数选单仓。原因是模型、生成代码、测试用例、标定表之间存在强耦合，一次改动往往同时涉及四处，分仓会让“哪一版模型配哪一版代码”变成需要人工核对的表格。

目录按职责分，不按文件类型分。推荐七个顶层目录：`req/` 放需求与编号表，`model/` 放控制模型，`src/` 放手写代码，`test/` 放用例与激励数据，`cal/` 放标定表，`tools/` 放脚本，`docs/` 放设计与评审记录。

生成代码单独放 `src/gen/`，并在文件头写入生成器版本与模型哈希。再配一条检查规则：`src/gen/` 下的文件若被手工修改，流水线直接失败。这条规则挡住了绝大多数“现场改了生成代码，下次重新生成就丢失”的事故。

90.3.4 第三步：定测试分层与配额

测试要分层，因为不同层的运行成本差三个数量级。表 90-2 是四层的典型配置。

表 90-2 四层测试的数量、耗时与触发时机

层	用例数量级	单次耗时	触发时机	主要目标
单元测试	300–800	毫秒	每次提交	函数级正确性、边界条件
集成测试	50–150	十秒	每次合并	模块间接口与时序
硬件在环	10–30	分钟	每夜	与真实 IO、执行器耦合
验收测试	20–50	小时	每次发布	需求覆盖与现场工况

配额可以按覆盖率反算。一个 8 kLOC 的控制器，若要求语句覆盖 90%，按式 (90-7) 需要 62.6 个“等效独立用例”；考虑到一个用例平均覆盖十几行、且存在大量重复路径，实际用例数通常是这个数的五到十倍，即 300 到 600 个。

这个反算的价值在于给出量级。没有它，团队对“要做多少测试”的估计会差一个数量级：有人以为 50 个用例够了，有人以为要 5000 个。

四层的比例也要看。单元测试应占全部用例的 70% 以上，因为它是唯一能在秒级给出反馈的一层。集成与硬件在环加起来不应超过 25%，验收不超过 5%。

90.3.5 第四步：定流水线分级

流水线不是一条，是四条，按触发时机与时限分开。表 90-3 是分级配置。

表 90-3 四条流水线的触发条件与时限

级别	触发	内容	时限
提交级	每次推送	编译 + 单元测试 + 静态检查	4 min
合并级	合并请求	提交级 + 集成测试 + 覆盖率门禁	15 min
夜间级	每日定时	合并级 + 硬件在环 + 长时稳定性	2 h
发布级	打标签	夜间级 + 验收测试 + 归档与签发	1 天

分级的依据是式 (90-10)：时限决定批大小，批大小决定定位范围。提交级必须在 4 min 内跑完，才能保证“一次构建一次提交”。

图 90-5 是本章流水线的时间构成。全量 602 s 里集成测试 300 s 占一半，编译 120 s 占两成；增量 240 s 里集成测试 120 s 仍是大头，部署 60 s 无法压缩。

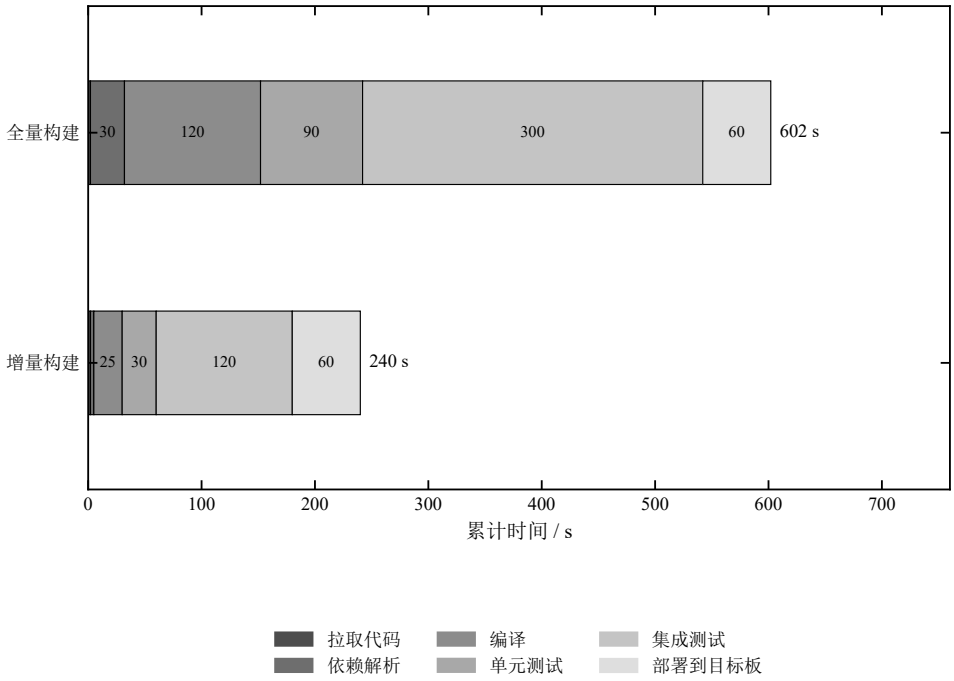


图 90-5 CI 流水线的时间构成

这张图给出优化的优先级：想让提交级进 4 min，不能靠“少跑点单元测试”，要靠“把集成测试移出提交级”。移动比压缩更有效。

90.3.6 第五步：建追踪矩阵

追踪矩阵是三张两列的表：需求到用例、用例到代码、代码到需求。第三张是反向索引，用于回答“这行代码为什么存在”。

三张表都要能自动生成，不能靠人填。生成方式是在代码与用例里写标签，例如用例头部写 `@req REQ-AX-014`，代码函数注释里写 `@req REQ-AX-014`，脚本扫一遍就出表。

完整性按式 (90-8) 算，报端到端而不是三层各自。评审时只报“三层都 95%”没有意义，因为端到端只有 85.7%。

矩阵要进版本库，与代码同一次提交。否则会出现“代码改了、矩阵没改”的错位，而这种错位比没有矩阵更危险——它给人虚假的安心。

90.3.7 一页纸的方案

五步的输出合成一页，作为项目的软件工程基线。表 90-4 是这一页的骨架。

表 90-4 软件工程基线的一页纸骨架

项	本项目的取值
需求编号	`REQ-<子系统>-<三位>`，单人分配，废止不回收
仓库结构	单仓七目录，生成代码隔离在 `src/gen/`
测试分层	单元 500、集成 90、在环 18、验收 30
流水线	提交 4 min、合并 15 min、夜间 2 h、发布 1 天
生成边界	控制算法生成，通信与驱动手写
追踪口径	端到端完整率 $\geq 90\%$，按式 (90-8) 计算

这张表要写进项目计划，并在每次评审时对照。它的作用是让“要不要加一道评审”这类讨论有一个共同的参照，而不是每次都从头争。

基线不是永久的。项目进入维护期后，测试配额可以降、流水线可以简化，但编号规则与目录结构不要改——它们是历史记录的锚点。

90.3.8 三个常见估算错误

第一个错误是按代码行数估测试工作量。代码行数与用例数不是线性关系：同样的行数，分支多的代码需要成倍的用例。应按分支数与接口数估。

第二个错误是把覆盖率当质量指标。覆盖率只说明“这行被执行过”，不说明“结果对不对”。一个断言写错的用例，照样贡献覆盖率。覆盖率要与变异测试或人工评审配合才有意义。

第三个错误是按人头估流程成本，忽略等待。流程的主要成本不是人干活的时间，是人等结果的时间。一条 15 min 的流水线，若每天触发十次，就是 150 min 的等待；这段时间里人并没有闲着，而是在做别的事，切换成本另算。

90.3.9 设计评审看五样东西

五个步骤搭完之后，要过一次评审。评审不看文档写得漂不漂亮，看五样具体的东西。

第一样，随便抽三条需求，能不能在五分钟内走完“需求、设计、代码、用例、结果”这条链。走不完就是链断了。

第二样，从仓库根目录开始，不看任何口头说明，能不能构建出产物。构建不出来就是环境没固化。

第三样，找出一个 `src/gen/` 下的文件，问“如果手工改它会发生什么”。答不出“流水线报错”，说明生成边界没有守住。

第四样，让负责人演示一次回滚。不是讲流程，是实际退回上一版并跑起来。

第五样，问最近三个缺陷分别是怎么发现的。如果三个都是现场发现的，说明测试层形同虚设；如果三个都有对应新增用例，说明闭环在做。

五样都能过，这套工程就算搭起来了。过不了的那一样，就是下一步该补的地方。

评审最忌讳变成“文档检查”。文档齐全但五样都过不了的项目很常见，反过来文档简陋但五样都过的项目也有。以能不能跑通为准，不以文档厚度为准。

评审要留记录，记录只写两件事：哪一样没过、谁负责什么时候补齐。写多了没人看。

90.3.10 工具选型的四条原则

控制项目的工具一旦选定，往往要用五年以上。选型时按四条原则看，比看功能列表有用。

第一条，能被脚本调用。工具必须有命令行接口或 API，否则无法接进流水线。只有图形界面的工具，最终会变成一个人的手工步骤。

第二条，数据能导出。需求、用例、结果都要能导出成文本或表格。导不出就意味着数据被锁死，将来换工具要重录一遍，于是永远换不掉。

第三条，能离线运行。控制项目的代码常在内网，需要联网激活或云端校验的工具会直接不可用。买之前先在一台断网机器上装一次。

第四条，可替换。优先选文件格式开放、存储在本地的工具。两年后团队可能换工具，也可能要交付源码给业主，格式封闭会变成事故。

三条常见错误：按厂商演示选型，演示环境永远是理想环境；按功能数量选型，功能多意味着配置复杂，最后只用两成；按价格选型，省下的授权费往往抵不上迁移成本。

还有一条容易被忽略：工具要能被新人一天内学会基本操作。学三周才能提交一次代码的工具，会被绕过。

90.4 连续情形

90.4.1 为什么可以把测试看成连续过程

测试通常以“人·天”或“机·时”连续投入，而不是一次性的动作。把时间当连续变量，可以用微分方程描述缺陷被发现的速度，从而回答“还要测多久”。

这个视角的局限也要说清：它把软件看成“装着固定数量缺陷的容器”，忽略了缺陷之间的关联、修复引入的新缺陷、以及测试策略的变化。它给的是趋势与量级，不是精确预测。

90.4.2 三条假设

式 (90-2) 建立在三条假设上。

第一条是缺陷总数有限，记为 a。这条假设在实际中大致成立：一个模块的缺陷不会无穷多，随着测试推进总会收敛。

第二条是发现比例恒定，记为 b。它意味着“剩下的缺陷被发现的概率与数量成正比”，也就是每个缺陷的“可发现性”相同。实际中热点模块的缺陷更容易被发现，所以真实曲线比指数模型更陡。

第三条是修复不引入新缺陷。这条最容易被破坏。修一个缺陷平均引入 0.1 到 0.3 个新缺陷是常见水平，缺陷密集的模块更差。

三条都成立时，式 (90-2) 可以放心用来做趋势判断；第三条明显不成立时，曲线会翘尾，这时要停下来做重构而不是继续加测试。

90.4.3 失效率减半时间的用法

8.66 天这个数，最实用的地方是当“体检指标”。

做法是每周统计一次现场或测试环境的失效率，看它有没有按 8 到 9 天减半。若连续三周没减半，说明 b 偏小，要从三个方向查。

方向一是用例集中在主路径。主路径的缺陷早被发现了，剩下的都在异常分支，而异常分支没有被覆盖。

方向二是环境不稳定。测试环境自身的故障被记成了软件缺陷，导致失效率虚高且不收敛。

方向三是缺陷被误判为不可复现。这类缺陷在统计上被“挂起”，不计入失效率，但它们实际还在。

90.4.4 什么时候停止测试

停止测试的判据不是“测够了”，而是“再测一天的收益小于成本”。

写成算式：当天收益等于“当天能发现的缺陷数”乘“这些缺陷若漏到现场的额外成本”，成本等于一天的测试投入。

按本章参数，第 30 天失效率 0.871 个/天，第 60 天 0.079 个/天。从第 30 天到第 60 天，30 天里多发现约 10 个缺陷，平均 3 天一个。若单缺陷现场成本 75 h、测试投入 8 h/天，则 3 天投入 24 h，换回 75 h，仍然划算。

第 60 天到第 90 天，30 天里只多发现 0.9 个缺陷，平均 33 天一个。33 天投入 264 h，换回 75 h，已经不划算。

交叉点就在第 60 天附近。这个算法比“感觉差不多了”可靠，也比“再测一个月保险”省钱。

90.4.5 模型失效的两种情形

第一种是缺陷被“藏”起来。某些缺陷只在特定时序、特定负载下出现，常规测试打不到，失效率长时间不降。表现为曲线尾部变平但不到零。

第二种是修复引入新缺陷。每次修复都带进 0.1 到 0.3 个新缺陷，当剩余缺陷少到与新增缺陷同一量级时，失效率不再下降，甚至回升。表现为曲线翘尾。

两种情形的处置完全不同。前者要改测试方法（加时序扰动、加负载），后者要停止加测试、转做重构与代码评审。看曲线形状就能区分：平尾是前者，翘尾是后者。

90.4.6 与现场数据对照

测试期的曲线要拿到现场验证。做法是把投运前三个月的故障间隔画在与图 90-2 相同的对数坐标上，比较斜率。

若现场斜率明显小于测试期，说明测试环境不具代表性：测试里容易暴露的缺陷在现场不容易暴露，反之亦然。常见原因是测试用的是理想信号源，现场有噪声、有电网波动、有操作工的误动作。

若现场斜率明显大于测试期，说明测试过度拟合了特定工况，或者现场的冗余设计掩盖了部分缺陷。这种情况更危险，因为它让人误以为软件很稳。

对照的目的不是让两条线重合，而是让“测试期给出的 MTBF 预测”有一个可信的误差范围。差三倍以内算正常，差十倍就要查原因。

90.4.7 参数什么时候要重估

a 与 b 不是永久常数。四种情况下必须重新拟合，否则后面的预测会系统性偏乐观。

第一种，架构大改。把单任务改成多任务、把轮询改成中断、把定点改成浮点，都会改变缺陷的暴露方式。此时旧曲线的斜率不再适用。

第二种，测试环境更换。从软件在环换到硬件在环，从理想信号源换到实际传感器，测出来的失效率会跳变。跳变本身不是问题，用旧参数解释新数据才是问题。

第三种，团队规模变化。人数翻倍，代码量增速与评审质量都会变，检出率随之变。§90.8 的调参记录里，团队扩张的那一个月要单独标出来。

第四种，缺陷来源结构变化。如果现场缺陷从“逻辑错误”变成“配置错误”，说明问题已经从代码转到交付流程，继续拟合可靠性曲线没有意义。

重估的方法很简单：取最近一个月的故障间隔数据，在对数坐标上拟合直线，看斜率。斜率与旧值差 30% 以内就不动，差一倍以上就换。换的时候要把旧曲线保留在图上，作为对照。

不要每次出故障就重估。参数抖动会让所有人失去判断基准，比参数略偏更糟。

90.4.8 测试强度上不去先查什么

很多团队说“测试做不起来”，一查往往是五个具体原因之一。

第一个，用例依赖真实硬件。跑一次要连台架、要上电、要等人。对策是把硬件访问抽成接口，用假实现跑单元测试。

第二个，用例启动太慢。框架初始化要十秒，写用例的人就懒得跑。对策是拆掉重初始化，让单条用例在百毫秒级完成。

第三个，断言难写。输出是一大块结构体或波形，不知道断言什么。对策是先写一个“黄金样本”，用比对代替逐字段断言。

第四个，反馈看不见。跑了但没人知道结果，失败了也没人修。对策是把结果放进流水线与看板，红了要有人管。

第五个，写用例的人得不到好处。写用例花时间，收益在半年后。对策是把“修缺陷必须补用例”变成硬规则，让写用例成为修复的一部分，而不是额外工作。

五个原因里，前三个是技术问题，后两个是管理问题。技术问题一周能改，管理问题要一个季度。多数团队卡在后两个，却一直在前三个上找办法。

90.5 离散情形

90.5.1 为什么还要离散视角

连续模型回答“还要测多久”，离散模型回答“这一批够不够”。软件的活动天然是离散的：一次提交、一个用例、一次构建、一个冲突，都是整数。

离散视角的价值在于它不会给出小数答案。连续模型说“需要 62.6 个用例”，实际只能写 63 个；说“平均 1.35 个缺陷”，实际要么 1 个要么 2 个。工程决策要落在整数上。

90.5.2 以提交为单位的失败概率

设每次提交引入缺陷的概率为 p，连续 n 次提交中至少有一次引入缺陷的概率

$$P_n = 1 - (1-p)^n \tag{90-14}$$

取 $p = 0.15$，5 次提交时 $P_5 = 0.556$，10 次时 $P_{10} = 0.803$，20 次时 $P_{20} = 0.961$。

式 (90-14) 解释了一个常见错觉：“我每次提交都很小心，怎么还是出错。”单次 15% 的出错率听上去不高，但一天二十次提交，出错的概率就是 96%。

它也解释了为什么要缩短反馈时间。缺陷引入是离散事件，发现得越晚，累积的未验证提交越多，定位时要在更多候选里排查。

90.5.3 覆盖率的整数效应

覆盖率是连续曲线，用例数是整数，两者的接缝处有几个实用结论。

第一个是向上取整。式 (90-7) 给出 62.6，实际要写 63 个用例；给出 87.2，要写 88 个。差一个用例看起来不多，但在评审时“63”比“62.6”更好执行。

第二个是边际收益递减。对式 (90-6) 求导，第 n 个用例带来的覆盖增量

$$\Delta c(n) = \frac{c_{\max}}{n_0} e^{-n/n_0} \tag{90-15}$$

在 $n = 50$ 处，每个用例还能带来 0.53 个百分点；在 $n = 100$ 处只剩 0.072 个百分点；到 $n = 200$ 处只有 0.0013 个百分点。写第 200 个用例的收益，是写第 50 个的千分之二。

第三个是门禁要卡整数。流水线里写“覆盖率不低于 90%”，实际执行时会因为浮点比较出现 89.99% 被判失败的情况。正确写法是卡“不低于 90.0%，允许 0.1 个百分点的测量波动”。

90.5.4 断点数是整数

追踪链的端到端完整率是连续量，断链的需求条数是整数。

三层均 90% 时端到端 72.9%，240 条需求里断链 65.0 条。这个 65 是期望值，实际可能是 63 或 67，取决于哪几条断。

评审时用整数说话更有力：“240 条需求里有 65 条无法完整追溯”，比“追溯率 72.9%”更能让人行动。前者是“有 65 条要补”，后者是“还有提升空间”。

同理，三层均 98% 时断链 14 条。这个数量级是可以逐条补齐的；65 条则必须改流程，而不是加班。

90.5.5 冲突是整数，工时是连续

式 (90-9) 给出的冲突文件数常带小数，例如 2.25、11.25。实际合并时冲突是按文件计的，2.25 意味着“多数情况下 2 个，偶尔 3 个”。

解决工时却是连续的：一个冲突可能 3 min 解决，也可能 40 min。本章取 8 min 是平均值，实际分布很宽。

工程上要用“期望值做预算、最坏值做排期”。预算按 8 min 一个算，排期按“最麻烦的那个冲突要半天”留余量。只用平均值排期，几乎必然延期。

90.5.6 分层配额的整数化

表 90-2 给的是数量级，落地时要变成整数配额，并与人力匹配。

一个 8 kLOC 的控制器，按 90% 覆盖率反算需要 63 个等效用例，实际写 500 个单元用例、90 个集成用例、18 个在环用例、30 个验收用例。四层加起来 638 个。

这 638 个用例要有人维护。经验比例是一个用例每年维护成本约半小时，638 个就是每年 319 h，约两个月的一个人力。这笔钱要写进预算，否则用例会在两年内腐烂成“跑不过就注释掉”。

90.5.7 离散视角的三条用法

第一条用法是算风险敞口。用式 (90-14) 算“这一批未验证提交里出问题的概率”，决定要不要插一次全量回归。

第二条用法是把连续指标翻译成整数任务。“端到端追溯率 85.7%”翻译成“补 34 条需求的追踪”，任务就具体了。

第三条用法是给门禁定阈值。门禁阈值必须是整数或带容差的定点数，否则会因为测量波动频繁误报，最后被人为关闭。

90.5.8 把整数效应写进排期

排期时最常见的错误是拿小数直接乘。算出“还需 62.6 个用例”，就按 62 个排，结果正好差在 90% 那条线下面。

正确做法是向上取整，再留余量。取整之后加 10%，因为用例的等效用不齐，总有一部分是重复覆盖。

工时也要按整数算。一天写 8 个用例是理想值，实际受会议、调试、环境问题影响，能到 5 个就算不错。用 62 个除以 8 得 7.75 天，用 62 除以 5 得 12.4 天，后者才是真实排期。

缺陷数同理。“每周漏 0.21 个缺陷”听起来无害，但缺陷是整数，实际是“每五周漏一个”。对一年期的项目，这个差别决定了要不要做影响分析。

还有一层整数效应在批次上。批大小为 1 时，一次失败只牵连一个提交；批大小为 2 时，定位范围立刻翻倍。批大小不能取 1.5，所以“把反馈时间从 7 min 压到 6 min”这种优化，可能在数字上只差一分钟，在工程上跨了一整档。

排期的三条规矩：向上取整、除以保守产能、按整数批次算。三条都做到，排出来的日期才不会被现实反复推翻。

90.5.9 概率与直觉的差距

人对累积概率的直觉普遍偏低。单次失败率 15% 听起来“不大”，但按式 90-14，跑二十次至少失败一次的概率是 96.1%。

这个差距在控制项目里有两个直接后果。

第一个后果是不稳定测试被容忍。“这条用例十次有八次过，偶尔红”，听起来可以接受。实际上八次过一次的用例，每五个提交就会红一次，团队很快学会忽略红色，整套测试失去意义。不稳定用例必须修，不能等。

第二个后果是备份被高估。“有两套台架，坏一套还有一套”，如果单套月故障率 15%，两套同时可用的概率是 72.3%，一个月里近三成时间只有一套。要真正做到“随时有一套可用”，得有三套，或者有一套的修复时间短于一小时。

反过来，小概率也不能忽略。年失败率 1% 的事故，二十年里发生的概率是 18.2%。对要运行二十年的设备，1% 不是“基本不会发生”，而是“迟早发生”。

算这类数不需要复杂工具，拿式 90-14 手算就够。养成算一遍的习惯，比事后解释“没想到会连着出”有用。

90.6 稳定性与性能

90.6.1 软件“稳定”的四个含义

控制软件说“稳定”，可能指四件不同的事，混在一起讨论就会各说各话。

第一是不崩。连续运行数千小时不重启、不死机、不漏内存。这是可靠性。

第二是可重现。同样的源码、同样的工具链，编出来的二进制逐位一致。这是构建稳定性。

第三是可回滚。任何一个已发布版本，都能在十分钟内退回上一版。这是运维稳定性。

第四是性能不退化。新版本的任务抖动、最坏执行时间、内存占用都不比旧版差。这是性能稳定性。

四个含义对应四套检查方法，不能互相替代。一个从不崩溃的软件，可能每次编译出来的二进制都不一样。

90.6.2 构建可重现的三个条件

构建可重现指同一份源码在任何一台机器上编出逐位相同的二进制。做到它需要三个条件。

第一个是工具链锁定。编译器版本、链接器版本、库版本全部写进配置文件，并校验哈希。用“最新版编译器”是重现性的天敌。

第二个是消除时间戳。很多编译器会把编译时间写进二进制，导致每次构建都不同。加 `-frandom-seed` 一类的选项，或把时间戳固定为源码提交时间。

第三个是路径无关。源码路径会进入调试信息，不同人放在不同目录就会编出不同的二进制。用相对路径或映射到固定挂载点。

三个条件都满足后，才能做“二进制比对”：现场那台设备上的程序，与仓库里某个标签编出来的程序，哈希一致。这是排查“现场跑的不是这一版”最硬的证据。

90.6.3 变更失败率与恢复时间

发布是带风险的变更。度量它需要两个数：多久失败一次，失败后多久恢复。表 90-5 是四种发布方式的对比。

表 90-5 四种发布方式的失败率、恢复时间与年不可用时间

发布方式	变更失败率	恢复时间 / min	年不可用 / h
手工发布	30%	240	28.80
脚本化发布	18%	90	6.48
流水线 + 自动回滚	12%	15	0.72
流水线 + 灰度 + 自动回滚	8%	8	0.26

表 90-5 最值得看的是第三列。从手工到灰度，失败率只降了 3.75 倍，恢复时间降了 30 倍，年不可用时间降了 110 倍。改善主要来自恢复速度，不是失败率。

这给出一条优先级：先把回滚做出来，再谈降低失败率。回滚是确定性的工程问题，失败率是概率问题，前者更容易见效。

90.6.4 实时性能的回归测试

控制软件的性能回归，不是测“跑得多快”，是测“最坏情况下多慢”。

要盯三个量：任务周期抖动、最坏执行时间、栈深度。三个量都要设阈值，超阈值即判定回归失败。

抖动阈值按 §89.7.4 的口径定：不超过周期的 10%。1 ms 周期对应 100 µs，100 µs 周期对应 10 µs。

最坏执行时间要留 30% 余量。测得 700 µs 的任务，在 1 ms 周期里只算勉强合格；编译器升级、库函数替换都可能让它涨 10%。

栈深度要用静态分析加实测双重确认。静态分析给上界，实测给实际峰值，两者差一倍以上说明有未分析的路径。

90.6.5 资源占用的静态上界

嵌入式控制软件不允许“跑着看”。内存、栈、CPU 占用都要在编译期给出上界。

堆一般禁用。动态分配在长时间运行中会产生碎片，而碎片导致的失败往往在几个月后才出现，且难以复现。要用就在启动时一次性分配完。

栈按最坏路径算，不按平均。递归、大局部数组、变长数组是三个主要风险点，静态检查工具能直接报出来。

CPU 占用按最坏执行时间除以周期算。单任务不超过 70%，多任务总和不超过 80%。超过之后，任何一次偶发的长路径都会导致丢拍。

90.6.6 三个指标谁先坏

可靠性、可重现性、性能，三者退化的顺序有规律。

性能最先坏。加一个功能、换一版编译器、多一层日志，抖动与执行时间就往上走。它是渐变的，容易被“还能跑”掩盖。

可重现性第二个坏。有人手工改了一行生成代码、有人用了本地编译器、有人忘了提交配置文件。它是突变的，通常在某次需要复现问题时才发现。

可靠性最后坏，但一旦坏了最难修。内存泄漏、计数器溢出、长时间运行的浮点累积误差，都要几天到几个月才暴露。

按这个顺序，日常检查的重点应当是性能趋势，而不是等崩溃。每周跑一次性能回归，比每次出事再查便宜得多。

90.6.7 长期运行的三类隐患

第一类是日志写满存储。调试期开的详细日志忘了关，三个月后磁盘满，系统停机。对策是日志分级加滚动删除，并设总量上限。

第二类是计数器溢出。32 位毫秒计数器约 49.7 天回绕，若代码里用绝对时间做差，回绕瞬间会出现巨大的负值。对策是一律用无符号差值，并在回绕点做一次专项测试。

第三类是时间戳漂移。设备时钟不与外部同步，一年后与真实时间差几分钟，故障记录无法与其他系统对齐。对策见§89.10.3。

三类隐患的共同点是“测试期看不出来”。它们不会在验收测试的几小时里暴露，只能靠设计时的规则避免。

90.6.8 性能回归怎么测

性能回归最容易被做成“偶尔跑一次，看数差不多”的摆设。要让它真正起作用，得定三件事：测什么、怎么判、多久测一次。

测三样。一是任务周期抖动，取一万个周期的最大值与标准差；二是最坏执行时间，取所有分支都走到的那条路径；三是关键响应的端到端延迟，从输入中断到输出更新。

判据用相对值，不用绝对值。抖动不超过基线的 10% 算通过，最坏执行时间不超过预算的 70% 算通过。相对值能适应不同的硬件平台，绝对值一换板子就失效。

频率按周。每次合并到主干后自动跑一次，结果存成时间序列。单次结果没意义，连续四周上升 20% 就是趋势。

判据要写进流水线，不能靠人看。人看的结果是“上次好像也这样”，机器看的结果是“超出 12%，失败”。

最后一条经验：性能回归的基线要跟着版本走，不跟着时间走。新版本如果因为功能增加而合理地慢了一点，要把基线更新并记录原因，否则半年后所有提交都会红。

90.6.9 需求变了怎么改代码

控制项目的需求变更比软件项目更频繁，因为现场工况、执行器型号、业主口味都会变。变更本身不可怕，可怕的是每次都按“临时改一下”处理。

按影响范围分三级，处理方式不同。

第一级是参数级：只改标定值，不改逻辑。这类变更走参数表，不进代码，不需要重新编译。前提是参数表与代码分离，且参数有范围校验。

第二级是逻辑级：改判断条件、改状态机跳转、改限幅曲线。这类变更要改代码、补用例、重跑回归，但不必重新做架构评审。

第三级是架构级：改采样周期、改任务划分、改通信协议。这类变更要重新做时序分析，重新评估最坏执行时间，重新跑长时稳定性测试。

分级的价值在于避免“小事走大流程”和“大事走小流程”。前者让人烦，后者让人翻车。

每次变更要回答三个问题：影响哪些需求、影响哪些用例、要不要重新标定。三个答案写进提交说明，比写十行改动描述有用。

还有一个纪律：变更期间不叠加。改逻辑的同时不要顺手升级编译器，否则出了问题分不清是谁的责任。这条纪律在赶工期时最容易被破坏，也最容易导致“改了一周不知道为什么好了”。

90.7 参数意义与作用

90.7.1 七个参数分三层

本章出现的量可以归成七类，分三层：过程层管“多久反馈一次”，质量层管“证据有多密”，结构层管“改动有多大”。

过程层的参数是反馈时间 T_{fb}、构建批大小 B、发布失败率 f。质量层的参数是覆盖率 c、用例数 n、追踪完整率 P。结构层的参数是分支寿命 t。

表 90-6 把七个参数列在一起，给出单位、典型范围与“改它会发生什么”。

表 90-6 七个参数的单位、范围与单调影响

参数	符号	单位	典型范围	改大的后果
反馈时间	T_{fb}	min	3–60	批变大，定位范围线性变大
批大小	B	次	1–5	单次失败牵连的改动变多
覆盖率	c	%	60–95	剩余缺陷减少，但边际成本陡增
用例数	n	个	100–1000	维护成本线性增长
追踪完整率	P	%	60–98	断链条数按乘积下降
分支寿命	t	天	1–20	冲突数线性增长，缺陷随之增加
发布失败率	f	%	5–30	年不可用时间线性增长

这张表只给方向，不给数量关系。同样是“改大”，分支寿命翻倍冲突就翻倍，覆盖率从 90% 到 95% 却要多花四成用例。下面几小节说清数量关系。

90.7.2 覆盖率是收敛参数

七个参数里，覆盖率是唯一收敛的。其余参数的后果大致线性，覆盖率的后果按指数饱和。

这个差别来自“容易的先做”。前一半用例覆盖主路径，后一半覆盖异常组合，同样的用例数换来的覆盖越来越少。

工程上把 90% 当常规目标，理由是式 (90-7) 给出 63 个等效用例，多数学得动；到 95% 要 88 个，增加四成；到 98% 要两百多个，增加三倍，而剩下的 2% 往往是不可达代码或硬件异常分支。

不可达代码要显式排除并注明理由，不要靠“写个假用例凑覆盖”。凑出来的覆盖率会让门禁失效，也会让后来的人误判。

90.7.3 用例数不是越多越好

用例数增长带来的收益按式 (90-15) 递减，成本却按线性增长——每多用例每年约半小时维护。

两个成本相等的那一点，就是用例数的经济上限。按本章参数，第 100 个用例带来 0.072 个百分点的覆盖，评估一次约 0.6 s，一年跑 250 次是 150 s；维护成本 30 min。收益远小于成本。

所以合理的做法是“分层配额 + 定期清理”。单元层保持 500 个左右，把跑不过、重复、已失效的用例每季度清一次。不清的用例会变成负担，最后整层被跳过。

90.7.4 分支寿命是线性放大器

分支寿命对冲突的影响是线性的，见式 (90-9)。这条线性关系容易被低估，因为它不显眼——每天多一点点，二十天后是二十倍。

线性还意味着它有一个明确的上限。设团队能承受每周 90 min 的冲突解决时间，按 8 min 一个冲突，就是每周 11 个冲突；按式 (90-9) 反解，分支寿命上限约 5 天。

5 天这个数与“一周一个迭代”的经验吻合。它不是巧合：迭代长度本质上就是分支寿命的上限。

90.7.5 反馈时间决定批大小

式 (90-10) 是一条阶梯函数，不是连续函数。反馈时间从 6 min 涨到 12 min，批大小从 1 跳到 2；从 18 min 涨到 24 min，从 3 跳到 4。

阶梯意味着“小幅优化可能毫无效果”。把 15 min 压到 13 min，批大小仍是 3，定位范围不变。要压就压过台阶，13 min 压到 11 min 才从 3 变 2。

工程上的做法是先算出台阶位置，再定目标。以 6 min 的提交间隔为例，台阶在 6、12、18、24 min，目标就定 11 min 而不是 13 min。

90.7.6 漏检率是策略参数

漏检率不是软件的固有属性，是策略选择的结果。全量回归漏检 0，按模块 4%，按影响分析 7%，只跑冒烟 35%。

选择漏检率，本质是选择“把风险放在哪一层”。放在提交级，成本是等待；放在夜间级，成本是延迟一天发现；放在现场，成本是 75 h 一个缺陷。

合理的组合是分层：提交级允许较高漏检（快），合并级中等，夜间级接近零。三层叠加后的总漏检率约为各级之积，比单层激进策略安全得多。

90.7.7 模型缺陷密度是生成路线的命门

式 (90-12) 里，ρ_m 与 α 相乘，是生成路线最敏感的两个量。

$\alpha = 4$ 意味着模型里一处错，生成代码里四处错。这个放大来自“一个模型错误会被复制到所有使用该模型的地方”。

ρ_m 从 3.0 降到 0.6，总成本从 160 降到 40，降了四倍。这个降幅不是靠“更仔细地建模”，是靠背靠背验证——用两个独立实现的模型跑同一组激励，比较输出。

所以生成路线的投入重点不在生成器，在模型验证。生成器是买来的，模型验证要自己建。

90.7.8 四个常用换算

第一个换算是覆盖率到用例数：$n = -25\ln(1 - c/0.98)$。90% 对应 63 个，95% 对应 88 个。

第二个换算是失效率到 MTBF：MTBF 小时数等于 24 除以日失效率。0.871 个/天对应 27.6 h。

第三个换算是分支寿命到冲突工时：每天约 18 min（按本章的 6 人规模）。5 天 90 min，10 天 180 min。

第四个换算是反馈时间到定位范围：每 6 min 反馈对应 40 行改动。10 min 反馈对应 80 行，4 min 对应 40 行。

四个换算都可以在评审现场口算，这是它们存在的意义。复杂的模型没人会在会上用，能口算的才会被用。

90.7.9 哪些参数会漂

参数不是常数，会随时间漂。

覆盖率会漂高。代码删了、测试没删，覆盖率虚高；或者加了排除标记，把难覆盖的代码排除掉。对策是每月看一次“覆盖率与代码行数的双趋势”。

用例数会漂低。跑不过的用例被注释掉，注释掉的被删除，两年后剩一半。对策是设“用例数下限”门禁，降了就报警。

分支寿命会漂长。项目紧的时候“先开个分支慢慢做”，一开就是两个月。对策是把分支存活天数做成看板，超过 5 天自动提醒。

反馈时间会漂长。每加一个检查项就多几十秒，一年后从 4 min 变成 15 min。对策是给流水线设时限，超时即失败，逼着人做取舍。

四个漂移的共同点是“缓慢且无人负责”。它们不会自己回来，只能靠定期测量把它们拽回来。

90.7.10 参数之间会互相牵制

七个参数不是七个独立旋钮。动一个，另外几个会跟着变，这是调参最常见的陷阱。

提高覆盖率，用例数上升，回归时间变长，反馈时间被推高，批大小随之变大。这条链在 §90.8.4 之后经常出现：测试补到 95%，流水线从 4 min 涨到 12 min，等于把前面压反馈时间的成果还回去了。

缩短分支寿命，合并变频繁，每次合并都要跑一遍集成测试，集成环境成为瓶颈。这条链在团队超过五人时尤其明显。

提高发布频率，每年的变更次数上升，按式 90-13，年不可用时间线性增长。要抵消它，必须同时降低失败率或缩短恢复时间，否则“发得勤”就是“坏得勤”。

降低用例维护成本，最直接的办法是少写用例或删旧用例，结果是覆盖率虚高、漏检率上升。这一条最隐蔽，因为指标看起来还在。

处理办法是每次只调一个，并观察两周内其他参数的变化。表 90-7 的记录方式就是为此设计的：改前、改后、观测指标三列，逼着人去看连带效应。

还有一种做法是设“预算”而不是设“目标”。比如规定回归时间不超过 5 min，在这个预算内尽量提高覆盖率。预算制比目标制更稳，因为它把相互牵制写进了约束里。

90.7.11 一份可对照的参考值

第一次定参数时，最缺的是“别人一般定多少”。表 90-14 给出一组常见范围，用于起步时的对照，不作为标准。

表 90-14 七项参数的常见取值范围

参数	保守	常见	激进	说明
提交级反馈时间	15 min	5–8 min	2 min	再快就要牺牲证据
算法模块覆盖率	85%	90–95%	98%	98% 需大量生成用例
驱动模块覆盖率	60%	75–80%	90%	上限受硬件状态限制
用例密度	0.5 个/百行	1–2 个/百行	4 个/百行	与分支复杂度相关
分支寿命	1 天	2–5 天	10 天	超过 10 天冲突成本陡增
发布失败率	30%	10–20%	5%	与自动化程度强相关
端到端追踪率	60%	85–95%	98%	三层乘积决定上限

用法是“先落在常见列，再按项目性质往两侧调”。安全相关往保守列靠，试验样机往激进列靠。

不要一上来就选激进列。激进列对应的不是“更高的追求”，是“更高的维护成本”。维护跟不上，指标会自己滑回保守列，而且是在没人注意的时候滑。

也不要长期停在保守列。保守列的代价是隐性的：缺陷被发现得晚，改动不敢做，人走知识走。这些代价不出现在报表上，只出现在三年后的总结里。

区间之间不是线性关系。反馈时间从 8 min 压到 5 min，通常靠缓存与增量就能做到；从 5 min 压到 2 min，往往要拆流水线、改测试架构，成本翻几倍。越往激进走，每一分钟越贵。

90.8 调参方法与经验

90.8.1 调参的四个对象

软件工程的“调参”不是调控制器参数，是调四个对象：反馈时间、测试配额、分支寿命、生成边界。

四个对象有明确的先后。反馈时间决定一切的前提——反馈慢的团队，任何流程都推行不下去，因为改一次要等一天才知道对不对。

顺序是：先压反馈时间，再补测试，再收分支，最后动生成边界。反过来做，通常是“流程定得很漂亮，没人执行”。

90.8.2 先压反馈时间，且要压过台阶

反馈时间的优化目标是“跨过台阶”，不是“缩短一点”。

以 6 min 的提交间隔为例，台阶在 6、12、18、24 min。当前 15 min 属于第 3 档，压到 13 min 仍是第 3 档，定位范围不变；压到 11 min 才降到第 2 档。

跨台阶的手段按性价比排序：把集成测试移出提交级、把编译改成增量、把依赖缓存起来、把静态检查分片并行。四条都用上，通常能从 15 min 压到 4 min 以内。

不该用的手段是“少跑点测试”。它确实能缩短时间，但缩短的是证据，不是等待。

90.8.3 再补测试，不补流程

反馈时间压下来之后，第二件事是补测试。

补测试的顺序是单元、集成、在环、验收，按成本从低到高。多数团队的短板在单元层——不是没有单元测试，而是单元测试依赖了外部环境，跑一次要连硬件，于是没人跑。

补单元测试的具体做法是把硬件访问抽象成接口，用假实现替换。这一步做完，单元测试从“分钟级”变成“毫秒级”，数量自然就上去了。

此时不要加流程。很多团队的第一反应是“加一道评审”“加一张检查表”，这些都会拉长反馈时间，把刚压下来的成果吃掉。

90.8.4 覆盖率卡在 90 还是 95

这是一个常见的争论，答案取决于代码的性质。

新写的控制算法代码，卡 95%。理由是算法分支多、边界条件多，90% 会漏掉不少异常路径，而这些路径正是现场事故的来源。

驱动、通信、板级支持代码，卡 80%。理由是这类代码大量分支依赖硬件状态，很多不可达，硬凑覆盖率只能写假用例。

历史遗留代码，先卡“不下降”。理由是要求它一步到 90% 不现实，先设一个“只能升不能降”的门禁，让它慢慢爬。

三种阈值要写进基线表，不同目录用不同阈值。一刀切的阈值，通常导致要么形同虚设，要么逼人作假。

90.8.5 分支策略怎么落地

分支策略的目标只有一个：让分支活得短。

图 90-9 是一条可行的时间线。主干只在通过流水线后前进；特性分支从主干切出，做完立刻合回，存活一两天；发布分支在发布前切出，只接受修复；热修分支从主干切出，修完同时合回主干与发布分支。

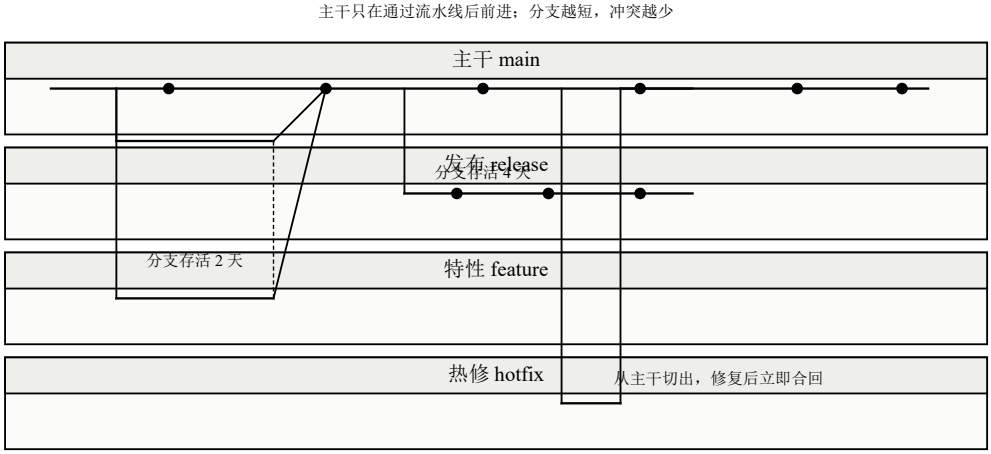


图 90-9 主干加短分支的版本管理策略

图里最容易被忽略的是热修分支的两条合回线。只合回发布分支、忘了合回主干，是控制项目最常见的“修了又出现”事故——下一次从主干发版，那个缺陷又回来了。

落地时有两条硬规则。第一条是分支存活超过 5 天自动提醒，超过 10 天强制合并或废弃。第二条是主干任何时刻可发布，不能出现“主干坏了三天没人管”。

90.8.6 代码生成的三道门

生成路线要过三道门，缺一道就不该用。

第一道门是模型验证。模型必须经过背靠背测试：用两个独立实现（例如两套建模工具，或模型与手写参考实现）跑同一组激励，输出逐点比对。这道门把 ρ_m 从 3.0 压到 0.6，是三道门里收益最大的。

第二道门是生成代码的静态检查。MISRA 一类规则的检查要自动化，违规数为零才放行。生成器通常能配成“生成即合规”，配不了的要换生成器。

第三道门是等价性测试。同一组激励分别跑模型（在仿真里）与生成代码（在目标板上），输出偏差要在量化误差范围内。这道门验证的是“生成器没译错”。

三道门都过，才敢把生成代码放进产品。只过第一道，等于把风险从模型转移到了翻译环节。

90.8.7 什么时候该降要求

有些项目必须降要求，降得越早越好。

第一种情形是团队规模小于三人。三人以下的团队跑不动四层测试与四条流水线，硬上会变成“流程给流程打工”。此时保留单元测试与提交级流水线即可。

第二种情形是项目剩余时间小于三个月。此时补历史代码的覆盖率没有意义，应把资源放在验收测试与现场支持上。

第三种情形是产品进入维护期。新功能很少，改动集中在少数模块，全量回归可以降到每周一次，日常只跑受影响子集。

降要求要写进基线表并注明理由与恢复条件。口头降要求，三个月后就没人记得为什么降了。

90.8.8 一次调参记录

表 90-7 是某控制项目三个月的调参记录，可作为模板。

表 90-7 某控制项目三个月的调参记录

日期	对象	改前	改后	观测指标	结论
03-04	反馈时间	全量 602 s	集成测试移出提交级	提交级 240 s，批大小 2→1	有效，跨过台阶
03-11	单元测试	120 个，依赖硬件	抽象 IO 后 480 个	单次 4 min→35 s	有效，覆盖率 61%→88%
03-25	覆盖率门禁	无	算法 95%、驱动 80%	新增用例 63 个	有效，门禁未误报
04-08	分支寿命	平均 12 天	强制 5 天提醒	冲突 22.5→9.4 个/周	有效，但需人盯
04-22	生成边界	全部手写	算法生成、驱动手写	代码缺陷 8.0→3.5	待观察
05-06	生成边界	模型未验证	背靠背验证	修复成本 160→40	有效，关键一步

表里 04-22 那一行的“待观察”是有意保留的。当时只看到代码缺陷从 8.0 降到 3.5，就以为成功了；两周后模型缺陷开始暴露，总成本与手写打平。05-06 补上背靠背验证，才真正拿到收益。

这条记录说明一件事：改一项要观察两周以上再下结论。代码生成这类改动，缺陷会从一处搬到另一处，只看一方会得出相反结论。

90.8.9 调参的五条经验

第一条，一次只改一个对象。同时改反馈时间与测试配额，指标变好说不清是谁的功劳。

第二条，改完先看两周。软件指标滞后，尤其缺陷率，一周内的数据波动大于信号。

第三条，跨台阶优先于微调。反馈时间、分支寿命都有台阶，跨过去才有量变。

第四条，门禁宁松勿假。定得太严会被人绕过，绕过一次之后所有门禁都失去威信。

第五条，记录比参数重要。参数三个月后就过时，记录能告诉后来的人为什么这么定。

90.8.10 一次失败的调参

有一个反例值得记下来。团队决定把覆盖率门禁从 80% 提到 95%，一次性提高到目标值，没有过渡。

结果是三周后门禁被关闭。原因有三层。

第一层，用例是“补”出来的，不是“设计”出来的。为了过门禁，大量用例反复调用同一个函数的不同参数组合，覆盖率的数字上去了，分支的异常路径仍然没有覆盖。这就是式 90-7 里 n_0 变大的情形：等效用下降，同样的数字对应更少的真实证据。

第二层，回归时间从 4 min 涨到 14 min。批大小从 1 变 2，定位范围翻倍。团队开始习惯性跳过本地测试，直接推上去看流水线结果，本地一次都没跑过。

第三层，门禁失败的处理方式是“加白名单”。先是三个文件，后来变成二十个。白名单一旦开口，就等于没有门禁。

复盘时的结论不是“不该提覆盖率”，而是“一次只能提一档，且要同步看回归时间”。后来重做的做法是：每月提 5 个百分点，回归时间超过 6 min 就暂停，补的用例必须包含至少一条边界条件。

三个月后覆盖率到 93%，回归时间稳定在 5.5 min。这个速度比原计划慢一倍，但没有被绕过。

这个案例的教训可以压成一句：门禁的强度不取决于它定得多高，取决于它被绕过多少次。

90.9 优化与改进

90.9.1 四个方向

软件工程的优化有四个方向：缩短反馈、提高证据密度、减少人工介入、加快恢复。

四个方向的收益不同。缩短反馈的收益落在每一天，提高证据密度的收益落在每次事故，减少人工介入的收益落在每次发布，加快恢复的收益落在每次失败。

按投入产出排序，通常是缩短反馈 > 加快恢复 > 减少人工介入 > 提高证据密度。前两个见效快，后两个要长期积累。

90.9.2 把集成测试移出提交级

这是缩短反馈最有效的一步，也是最少被做的一步。

集成测试通常占流水线的三分之一到一半。它在提交级跑的理由是“怕接口坏了”，但接口坏了的概率远低于单元逻辑坏了的概率，而且集成测试在合并级跑同样能抓到。

移出去之后，提交级从 602 s 降到 240 s，批大小从 2 降到 1，定位范围从 80 行降到 40 行。代价是接口问题晚一次合并才发现，延迟通常在几小时以内。

90.9.3 并行与分片

流水线的第二个优化方向是并行。串行的 602 s 里，编译 120 s、单元测试 90 s、集成测试 300 s 是主要可并行段。

单元测试最容易并行：按文件或按模块分片，四片并行能把 90 s 压到 25 s。代价是要处理测试间的共享状态，分片不当会出现“单独跑通过、一起跑失败”。

集成测试并行要难一些，因为它往往共享硬件或数据库。可行的做法是按硬件资源分池，池内串行、池间并行。

编译并行受依赖图限制，收益通常最小。除非项目大到几万行，否则不必优先做。

90.9.4 缓存与增量

第三个方向是缓存。依赖下载、编译产物、测试结果都可以缓存。

依赖缓存最稳，收益也最直接：从 30 s 降到 3 s。做法是按依赖清单的哈希做键，命中即复用。

编译缓存要小心。增量的前提是依赖关系准确，而很多构建系统对头文件依赖跟踪不全，导致“改了头文件没重编”的诡异问题。启用前要先验证依赖跟踪是否完整。

测试结果缓存几乎不能用。测试的价值在于“在当前代码上重跑一遍”，用旧结果代替等于自欺。只有“未受影响的模块”可以跳过，且必须有影响分析支撑。

90.9.5 测试选择的收益账

测试选择是“少跑哪些用例”的决策，收益与风险都要算。

按表 90-7 的口径，全量 312 s、漏检 0；按影响分析 25.2 s、漏检 7%。省下 287 s，每周多漏 0.21 个缺陷。

若这些缺陷在集成阶段被发现，单次成本 25 h；0.21 个就是 5.25 h。而 287 s 乘每周 20 次运行是 5740 s，约 1.6 h。省 1.6 h 赔 5.25 h，不划算。

结论是：影响分析适合放在提交级（每日多次），不适合替代夜间全量。用错层级，省下的时间远不够赔。

90.9.6 硬件在环的排队

硬件在环是最稀缺的资源，通常只有一两套台架，多个团队排队。

排队问题不能靠“加台架”解决，因为台架贵且占地方。可行的做法有三条。

第一条是分层：能在软件在环（SIL）里跑的，不要占台架。把执行器模型、传感器模型做进仿真，SIL 能覆盖七成的用例。

第二条是预约制：台架按半小时为单位预约，超时自动释放。没有预约制的台架，实际利用率往往不到三成。

第三条是夜间批处理：把长时稳定性测试放到夜间，白天只跑短用例。这样一套台架能顶两套用。

90.9.7 优化的三条底线

第一条是稳定性不能降。任何优化如果让流水线变得不稳定（偶发失败、结果不确定），收益都是负的。不稳定的流水线会被人忽略，忽略之后等于没有。

第二条是可维护性不能降。为了快而写的复杂脚本，半年后没人敢改。优化要留文档，脚本要能被人接手。

第三条是成本收益为正。加一套并行框架花两个月，省下每天五分钟，一年才省 20 h，不值得。

三条底线的顺序不能颠倒。先保稳定，再保可维护，最后谈收益。

90.9.8 改了以后怎么证明有效

优化做完要说清“好了多少”，需要三个条件同时满足：场景相同、口径相同、重复多次。

场景相同指对比的两组数据来自同一类项目或同一段代码。拿新项目的流水线时间比老项目，没有意义。

口径相同指统计方式一致。流水线时间按“从推送到结果”算，还是按“从拉取到结果”算，两者差一个数量级的排队时间。

重复多次指至少取两周的中位数，而不是取最好的一天。流水线时间受缓存命中、机器负载影响，单次测量波动可达30%。

满足三条之后，收益要落到可比的量上。不要写“快了很多”，要写“提交级从 602 s 降到 240 s，批大小从 2 降到 1，定位范围从 80 行降到 40 行”。

最后留一条反证：这次优化有没有让某个指标变坏。并行化通常会让偶发失败率上升，缓存通常会让“改了没生效”的困惑变多。把代价写出来，方案才完整。

90.9.9 优化会不会长回去

优化有一个通病：三个月后回到原样。原因不是有人故意破坏，而是每次新增一个检查项、每次多跑一个脚本，都只增加几秒，没人觉得有问题。

对抗这种“慢性回涨”，有两个办法。

第一个办法是设上限并让它生效。给提交级流水线定一个硬上限，比如 5 min，超过就失败。这样每加一个检查项，都必须同时砍掉另一个或做一次加速，压力自动传递到决策者。

第二个办法是每月打一次点。记录提交级、夜间、发布级三条流水线的中位耗时，画在同一张图上。图上出现连续两个月的上升，就安排一次专项优化。

两个办法配合使用的效果最好：上限挡住突发，打点发现趋势。

还要防另一种回涨：测试数量涨了，测试质量没涨。表现是覆盖率上升但漏检率不变，说明新增的用例都在重复覆盖已有分支。对策是每月看一次“新增用例覆盖了多少新分支”，这个数低于三成才算有效补充。

优化的终点不是“最快”，是“稳定在某个水平不再退化”。前者是一次性的，后者要靠机制。

90.9.10 指标会被人为优化

凡是拿来考核的数字，都会被人找到最省力的达标方式。本章出现的四个指标各有对应的“作弊方式”，要提前防。

覆盖率靠白名单。把难测的文件加进排除列表，分母变小，覆盖率上升。对策是白名单要评审，且每条要有理由。

用例数靠复制。同一段逻辑换十个输入值就是十个用例，数字好看，证据没增加。对策是同时看“新增用例覆盖的新分支数”。

流水线时间靠少跑。跳过集成、跳过静态检查、失败重试直到通过。对策是记录“跳过的用例数”，并把它与时间一起公示。

缺陷数靠不记。小问题口头修掉，不进缺陷库，缺陷率自然下降。对策是抽查现场维修记录与提交说明，两者对不上就说明漏记。

四个作弊方式的共同点是：优化了指标，没有优化被指标代表的东西。这就是古德哈特定律在软件工程里的表现。

防的办法不是取消指标，而是成对使用。覆盖率配漏检率，用例数配新增分支数，时间配跳过量，缺陷数配现场记录。一对里只要有一个不能靠省力改善，作弊就没收益。

最后一条：指标要用来发现问题，不用来排名。一旦排名，人会先优化排名，再优化工作。

90.10 工程落地

前面九节把道理讲完了。这一节讲怎么在一周、一月、一年的尺度上把这些事做成，重点在顺序，不在工具。

控制项目的软件工程落地有个共同难点：团队往往只有两三个人，且主业是控制算法而不是软件。照搬互联网公司那一套，会被人力压垮；完全不建，又会在第一次现场升级时崩盘。所以落地方法是“按项目类型选强度，按周排顺序”。

90.10.1 先定流程强度

流程强度不是越高越好。太重会拖慢迭代，太轻会在关键节点翻车。判断依据是项目要活多久、改多频繁、出错代价多大。

表 90-8 给四档配置，可对照自己的项目选一档。

表 90-8 四档流程强度与适用项目

档位	版本管理	测试	流水线	追踪	适合项目
一档（轻）	单分支	手工验证	无	无	一次性试验、论文数据
二档（中轻）	主干 + 标签	冒烟脚本	本地一键构建	需求表	台架验证、原理样机
三档（中）	主干 + 特性分支	单元 + 集成	提交级自动构建	需求到用例	小批量产品
四档（重）	主干 + 分支保护	单元 + 集成 + 硬件在环	三级流水线 + 灰度	五列全链	长期维护的定型产品

选档的关键是“这版代码还要改多久”。改三个月的项目用三档，改十年的项目用四档。反过来，一次性跑数据的试验代码硬上四档，会有一半时间花在维护流程上。

档位可以升，但要一次升一档。从一档直接跳到四档，团队会被流程压住，最后连流程一起废掉。升级的触发条件通常是第一次现场事故，那时人对流程的价值有切身体会。

90.10.2 开仓第一周做什么

如果现在还没有仓库，第一周按下面七件事做，顺序不要乱。

第一天：建仓，定目录，把现有代码全部提交进去，哪怕很乱。提交信息写“初始导入”。同时在仓库根写一份构建说明，只写三行——用什么编译器、什么命令、产物在哪。

第二天：加一条冒烟脚本。它只做一件事：编译通过后跑一个最短的算例，输出一个数，与期望值比。这条脚本是后面所有自动化的地基。

第三天：开一条最简流水线，只做两件事——拉代码、跑冒烟脚本。不要一上来就配单元测试、代码检查、覆盖率，它们会让第一次成功来得太晚。

第四天：引入单元测试框架，为最容易出错的那个模块补十个用例。选“最容易出错”而不是“最容易测”，因为前者能立刻抓到缺陷。

第五天：给需求编号建表，把已经实现的功能倒着补上编号，能补多少补多少。然后在代码里给关键函数加一行注释指向编号。

第六天：收拢构建入口。所有人用同一条命令构建，禁止各自在 IDE 里点按钮。命令行构建是流水线可复现的前提。

第七天：拉一个同事，让他只看仓库里的文档，从零构建一次。他卡住的地方，就是文档缺的地方。

第一周不要做的事有三件：不要定编码规范细则，不要配代码审查流程，不要引入复杂度度量。这三件事在代码量还小的时候只有成本，没有收益。

第一周结束时要有一个可展示的成果：一条命令、一个数、一张图。可展示才有说服力，说服力决定后面三个月还有没有人配合。

90.10.3 流水线的搭建顺序

流水线的六个阶段在 §90.2.6 已给出耗时结构：拉取 2 s、依赖 30 s、编译 120 s、单元 90 s、集成 300 s、部署 60 s。搭的时候不能按这个顺序，要按“能多快给人反馈”排。

第一步只搭前三段：拉取、依赖、编译。目标是任何一次提交在两分钟内知道“能不能编过”。这一步的收益最大，因为编译错误是最常见也最容易修的缺陷。

第二步加单元测试。此时单元测试必须是“不依赖硬件”的，否则在流水线机器上跑不起来。驱动层的硬件依赖要靠抽象接口隔离，把真实 IO 换成一个假的实现。

第三步把集成测试放进去，但要放到夜间。集成测试 300 s 占全量流水线的一半，放在提交级会把反馈时间推到十分钟，批大小被迫从 1 变成 2。

第四步加部署到目标板。这一步通常最容易被跳过，因为它需要一台常连的样机。但没有它，“代码能编译”和“代码能跑”之间永远隔着一次人工操作。

第五步才加并行分片、缓存这些加速手段。加速的前提是有稳定的基线，没有基线的加速无法验证。

顺序错了的典型表现是：流水线很高级，但经常红，没人看。红的原因多半是集成测试不稳定，而团队为了让它绿，开始加“重试三次”，这就是把问题藏起来。

还有一个容易忽略的细节：流水线的输出要能被人一眼看懂。失败时打印“第 3 步失败”没有用，要打印哪条用例、哪个断言、期望值与实际值。日志的读者是半夜被叫起来的人，不是写流水线的人。

流水线本身也要有版本。配置文件进仓库，改动走评审。把流水线当基础设施而不是当个人电脑上的脚本，这一条决定了它能不能活过半年。

90.10.4 用例设计的四条规则

用例质量决定覆盖率数字有没有意义。四条规则按重要性排。

第一条，一条用例只验证一件事。断言两个以上，失败时说不清是哪个坏了。这条规则能把调试时间压掉一半。

第二条，用例名写成“条件-动作-期望”。比如“给定传感器饱和-输入阶跃-输出限幅在 100”。名字本身就是文档，别人看测试报告就知道哪块坏了。

第三条，边界用例优先于中间用例。控制代码的错误九成出在边界：零、负值、最大值、饱和、除零、数组越界。中间值跑通不代表边界安全。

第四条，每个修过的缺陷补一条用例。这条用例的价值不在于当下，而在于半年后同样的改动不会再把同一个坑挖一遍。缺陷修复不补用例，等于把修复的有效期设成了“直到下次重构”。

四条之外还有一条禁忌：不要把“不崩溃”当断言。程序不崩但输出全零，也是缺陷。断言必须落在具体的数值或状态上。

用例数量按 §90.7.2 的经济上限控制。算法模块追求 95%，对应约 87 个等效用例，实际写 300 到 600 个；驱动模块 80% 就够，再多是浪费。

90.10.5 追踪矩阵怎么落地

追踪矩阵听着像大工程，实际上一张表加一条脚本就够。表 90-9 是一行样例。

表 90-9 追踪矩阵的一行样例

需求	设计	代码	用例	结果
REQ-AX-014 位置环带宽 ≥ 20 rad/s	DES-AX-03 二阶超前校正	`src/axis_ctrl.c:88`	`test_axis_bandwidth`	通过 24.1 rad/s

五列缺一列，链就断在那一处。§90.2.4 算过，三层各 90% 的覆盖率，端到端只剩 72.9%，240 条需求里 65 条追不到。所以矩阵的价值不在“有”，而在“每周清断链”。

三条纪律。

第一条，需求编号只增不改。需求变了就新增一条，旧的标“作废”并注明被谁替代。改编号会让历史记录全部失效。

第二条，断链每周清一次。用脚本扫：有需求无用例的、有用例无需求的、有代码注释指向不存在编号的，各列一张清单。清单长度就是健康度。

第三条，覆盖率按目录统计，不按全项目统计。算法目录 95%、驱动目录 80%，全项目混在一起算会得到 85% 这种既不好也不坏的数字，谁都不满意，也推动不了任何改动。

工具选择上，先用表格加脚本。等到需求过千、多人并行时再考虑专业工具。过早引入工具，团队要先学工具再学方法，顺序反了。

90.10.6 发布与回滚的检查清单

发布是风险最集中的时刻。表 90-10 是发布前的十项检查，建议直接抄成模板。

表 90-10 发布前十项检查

序号	检查项	不通过的后果
1	版本号与标签一致，标签已推到远端	现场说不清装的是哪版
2	从标签重新构建，产物哈希与测试版一致	测的和发的不是同一个东西
3	标定表随版本一起打包并记录哈希	参数与代码错配
4	回滚包已生成并在一台样机上验证过	出事时回不去
5	参数备份已导出，且能导回	改坏参数无法恢复
6	变更点清单已写给现场，含影响范围	现场不知道会看到什么变化
7	关键指标基线已记录（抖动、超调、周期）	发完无法判断是否退化
8	灰度范围已确定（先几台、观察多久）	一次全推，风险无上限
9	值班人与回滚决策人已明确	出事时互相等
10	回滚判据已量化（如错误计数超阈值）	该回不回，越拖越大

十项里第 4 项最容易被省，也最致命。回滚包不验证，等于没有回滚包——真正要用的时候才发现缺依赖、缺配置、缺授权。

回滚判据必须提前量化。写成“看情况”的判据等于没有判据，现场会倾向于“再观察一下”，而观察期正是损失扩大的窗口。

灰度按 1 台、10%、全部三段走。第一台的观察时间要够一个完整工况周期，比如带昼夜变化的场景至少 24 h。第一台没问题不代表批量没问题，但第一台有问题一定代表批量有问题。

90.10.7 故障复盘怎么写

复盘不是写检讨，是产出可执行的三样东西。模板五问。

第一问，现象是什么。只写可观测的事实：什么时间、什么设备、什么报警、操作员看到什么。不要在这一问里写原因。

第二问，时间线。从第一个异常信号到恢复，逐条列出时间和动作。时间线能暴露“发现晚”还是“处理慢”，这两个问题的解法完全不同。

第三问，直接原因。哪个函数、哪个参数、哪个配置导致的。

第四问，根因。为什么这个直接原因能进入现场。是测试没覆盖、是评审没看、还是流程允许绕过。

第五问，为什么没早发现。这一问最有价值。答案通常是“没有这条用例”或“有指标但没人看”。

五问之后必须落三条产出：一条新用例、一条新检查项、一条新监控。三条缺一条，复盘就只是文字。

复盘不追责，但记录要留名。不追责是为了让人说真话，留名是为了让结论有出处。这两条不矛盾。

只写“人为失误”的复盘是无效的。人失误是现象，不是根因。根因是“为什么这个操作容易失误”——按钮位置、默认值、缺少二次确认，这些才是能改的。

90.10.8 交接要交三样东西

控制项目的人员流动是常态。交接不清，半年后没人敢动这套代码。

第一样，能跑起来的环境。一条命令能从零构建，一条命令能跑冒烟测试。写在仓库根目录，不超过十行。交接时当面跑一遍，让接手人自己敲。

第二样，能读懂的地图。一张架构图（模块边界与数据流），一份目录说明（每个目录放什么、谁负责），一份术语表（项目里那些缩写是什么意思）。图用 §90.6 那类泳道图画，不要用只有作者看得懂的草图。

第三样，能查到的历史。关键决策为什么这么定（决策记录），出过哪些事故（复盘），哪些地方是临时方案（技术债清单）。这三份东西不写，接手人会重走一遍所有弯路。

交接的验收标准是：新人独立改一个参数，走完构建、测试、发布全流程，不需要问人。做不到这一条，交接就没完成。

90.10.9 一个反面案例

某台架项目，代码存在一位工程师的笔记本上，用压缩包按日期命名。现场要改一个参数，他把整个工程拷到 U 盘带过去，在设备上直接改，改完没回传。

三个月后设备出现偶发停机。此时有三个版本的代码在流转：笔记本上的、U 盘里的、设备里的，谁也不确定设备里是哪一版。排查花了两周，最后发现是设备上那份漏了一个滤波系数。

按本章的方法重做，一次性投入大约是：建仓与整理两天、冒烟脚本半天、单元测试一周、流水线两天，合计约两周。事故排查花了两周，加上停机损失与两次现场出差，成本远超建仓投入。

用式 90-13 的口径算更清楚。假设这类项目每年发生 6 次现场变更，手工方式失败率 30%、每次恢复 240 min，年损失 7.2 h。加上这一次性两周的排查，第一年就亏。

这个案例的教训不是“要用 Git”，而是“要让’设备里是哪一版’这个问题有答案”。版本管理只是答案的一种实现形式。

90.10.10 投运后看四个数

流程建起来之后，要有人看它是否有效。四个数，每月看一次。

第一个是构建成功率。正常应在 90% 以上。长期低于 80%，说明主干经常坏，或者流水线本身不稳定。

第二个是变更失败率。按 §90.6.3 的口径，流水线加自动回滚能到 12%。实际项目第一年能到 20% 就算不错，因为回滚演练往往不到位。

第三个是平均恢复时间。这个数改善最快，也最能体现自动化价值。从 90 min 降到 15 min，是六倍的差距。

第四个是缺陷发现阶段分布。理想形态是七成在单元与集成阶段发现。如果现场发现的占比超过三成，说明测试层有洞，要回去看覆盖率与用例质量。

四个数都看趋势，不看绝对值。绝对值受项目类型影响大，趋势才反映流程是否在起作用。

看数的人要固定。换一个人看，口径就变了，趋势线会断。交接看数这件事本身也要写进交接清单，并注明每个数的取数位置与统计周期。

四个数不必每次都全看。日常只看前三个，第四个每季度看一次，因为它变化最慢，看勤了反而被噪声带着走。

趋势连续两个月变差，就要停下来查原因。最常见的两个原因是：为了赶进度跳过检查、新人不知道流程存在。前者要管理层表态，后者要补培训。

90.10.11 三个月的路线图

第一个月只做三件事：建仓与目录、冒烟脚本、最简流水线。月底的验收标准是“任何人拉下代码，一条命令跑出结果”。

第二个月做测试与追踪：单元测试框架、三十条核心用例、需求编号表。月底的验收标准是“改动一个算法参数，五秒内知道有没有破坏已有行为”。

第三个月做发布与复盘：发布清单、回滚包、一次灰度演练、一份复盘模板。月底的验收标准是“能在半小时内完成一次发布，并在十分钟内回滚”。

三个月之后进入维护节奏：每周清一次断链，每月看一次四个数，每季度演练一次回滚。这三件事花不了多少时间，但一旦停了，前面三个月就会慢慢退化。

路线图不要压缩到一个月。压缩的结果通常是三件事都开了头，一件也没跑通，最后全废。软件工程的收益来自“持续运行”，不来自“一次性建设”。

90.10.12 文档写多少

控制项目的文档有两个极端：一种是只有代码没有文档，人走知识走；另一种是文档比代码多，写完没人看。

合理的量与代码成比例：每千行代码配三到五页有效文档。有效指能回答“为什么这么做”，不指能复述“这里做了什么”——后者读代码就行。

必写的四份：构建说明（一页）、架构图（一张）、需求编号表（一张表）、决策记录（按时间追加）。可选的：接口说明、标定说明、部署手册。

判断文档是否够用的标准很朴素：找一个没参与过的人，让他按文档做一次常见操作。他卡住的地方，就是文档该补的地方。这个测试比任何模板都准。

90.10.13 三个人怎么分工

小团队落地时最常问的问题是“谁来干”。三个人的配置可以这样分。

第一个人管模型与生成：控制模型、代码生成配置、背靠背验证。他对“模型改了代码为什么没变”负责。

第二个人管驱动与测试台：硬件抽象层、单元测试桩件、硬件在环台架。他对“测试跑不起来”负责。

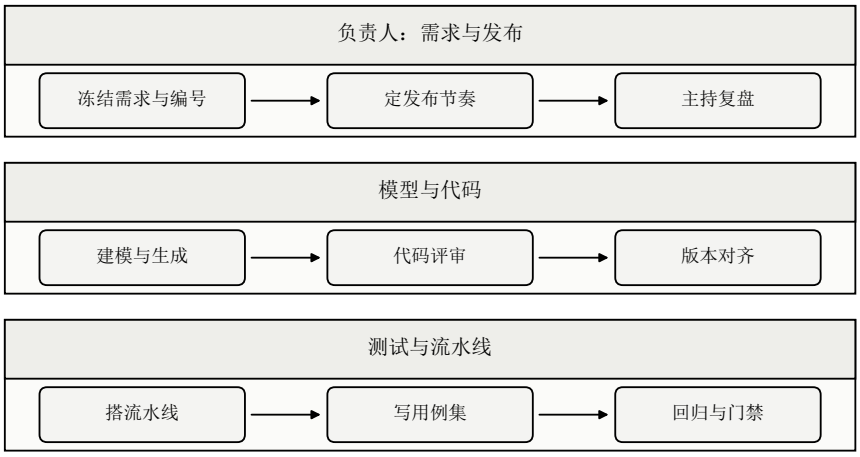
第三个人管流水线与发布：构建脚本、流水线配置、发布包与回滚包。他对“发不出去、退不回来”负责。

三个人都要写用例，也都要清断链。分工的目的不是划界，而是让每类问题都有明确的第一责任人。

每周留半小时一起过三张表：断链清单、失败用例、待发布的变更。半小时足够，关键是每周都开。

还要做交叉备份：每个人至少能跑通另外两个人的日常操作。三个人里有一个请假两周，项目不能停。

把职责沿时间摊开，就得到图 90-10 的三条泳道。横轴三列对应“定方案—做出来—证得住”：负责人先冻结需求与编号、再定发布节奏、最后主持复盘；模型与代码这条线依次是建模与生成、代码评审、版本对齐；测试与流水线这条线依次是搭流水线、写用例集、回归与门禁。三条泳道并行推进，但每个阶段交界处都要对齐一次。



三列对应「定方案—做出来—证得住」，分工的边界是交付物而不是工作量

图 90-10　三个人的分工泳道

这张图最有用的地方是把“边界”画成了交付物，而不是工作量。负责人交出的是一份冻结的需求编号表和一份发布记录，模型与代码交出的是通过评审的可生成模型，测试与流水线交出的是能拦住回归的门禁。谁的活没干完，看交付物就知道，不需要在会上争论谁更忙。

泳道的第二个作用是暴露等待。三条线不是永远等速的：需求冻结慢一周，后两条线就会空转；门禁搭得晚，前两条线的产出就没有人验。把三条泳道并排画出来，等待发生在哪一段一目了然，比排一张人员分工表有用得多。

人更少时（一到两人），把第三个人的职责降级为“每月做一次”，其余两项保留。一个人也要有版本管理与冒烟脚本，这两件事没有替代品。

90.10.14 第一次事故之后的两周

第一次现场事故是推动流程最好的窗口，也是搞砸流程最容易的窗口。两周按下面排。

第 0 天只做一件事：恢复。不回滚之外的任何动作都往后放，包括“顺手把它改好”。恢复之后立刻冻结现场，把设备里的版本、参数、日志原样导出。

第 1 天做时间线。几点几分出现什么、谁做了什么、什么时候恢复，按时间列。时间线要写到分钟，写不到分钟的地方标“不确定”。

第 2 天定位直接原因。到函数与参数一级，不到“大概是通信问题”这种程度。

第 3 天问根因。为什么这个原因能进现场：测试没覆盖、评审没看、还是流程允许绕过。答案通常不止一个。

第 4 天落三条产出：一条新增用例、一条新增检查项、一条新增监控。三条都要有负责人与日期。

第二周做三件事。第一件，全项目排查同类问题，不只看事发模块。第二件，演练一次回滚，确认下一次能更快。第三件，开一次复盘会，把结论讲给所有人，包括没参与的人。

两周里不要做的事：不要立刻加一堆流程，不要趁机换工具，不要追责到个人。三件事都会让团队把注意力从问题转到自保。

事故的价值只有一次。两周内没把它变成用例与检查项，它就只是一段记忆，半年后连记忆都会变形。

90.10.15 十五分钟的周会怎么开

流程能不能活下来，很大程度上取决于有没有一个短的、固定的、不跑题的周会。

时间定十五分钟，站着开。三个问题，每个问题不超过五分钟。

第一个问题：这周有没有断链，谁去补。看断链清单，只看新增的，老账本单独排期。

第二个问题：这周有没有失败的用例长期没人管。看失败清单，超过三天的要有人认领，或者明确关闭并写明原因。

第三个问题：下周要发什么，检查清单走到第几步。看发布清单，卡在第几项就说第几项。

三个问题之外不讨论技术方案，不讨论排期，不追责。要讨论的另开会，周会只做分派。

会要有记录，记录只有三行：谁、做什么、什么时候。三行之外的内容都不写，写了也没人回头看。

周会最容易变成两种情况：一是变成进度汇报会，越开越长；二是变成技术讨论会，两三个人讲，其他人听。两种都偏离了目的——周会的目的只是让三张表有人看、有人动。

如果连续四周三个问题都是“没有”，说明表本身没在更新，要查是不是没人维护。空白的表比没有表更糟，因为它给人一种“都在掌控中”的错觉。

90.11 优缺点与适用边界

本章讲的方法有一个共同前提：缺陷会被发现，且发现得越早越好。这个前提在多数控制项目里成立，但也有明确的例外。这一节把代价讲清楚，把边界划出来。

90.11.1 四种做法放在一起比

控制项目的软件做法大致有四类，表 90-11 从六个维度对比。

表 90-11 四类软件做法的对比

维度	手工为主	流程为主	生成模型为主	三者结合
首次出活速度	最快	慢	中等	慢
长期改动成本	高，随年限上升	低且平稳	低但依赖模型质量	最低
对人员要求	靠个人经验	靠纪律	靠建模能力	三样都要
缺陷发现阶段	多在集成与现场	多在单元	看模型验证	多在单元与模型
可追溯性	弱	强	中，链路到模型为止	强
团队规模下限	1 人	3 人	2 人	4 人

四种做法不是互斥的。实际项目多是“驱动手写、算法生成、全流程追踪”，也就是第四列。区别在于各项投入的比例。

选型的判断顺序是：先看团队规模，再看项目寿命，最后看算法复杂度。人数不足三人时，先做第三档里最便宜的两件事——版本管理与冒烟测试，其余往后放。

90.11.2 重流程与轻流程

重流程的收益是可见的：缺陷早发现、变更可回滚、人员可替换。代价也是可见的：每次改动要过几道门，一道门一个人，反馈慢。

轻流程的收益是迭代快，代价是知识集中在个别人身上。人一走，代码就成了黑箱。

两者的分界不在公司大小，而在“这套东西坏了会怎样”。坏了只是重跑一遍数据的，轻流程就够；坏了会让设备停机、让人受伤、让批次报废的，重流程是必需品。

有一个常见的误判：把“现在没出事”当成“不需要流程”。控制系统的特点是故障低频高损，样本少，靠经验积累不出直觉。三年不出事，第四年出一次，损失可能超过三年的流程成本。

反过来也有误判：把流程当成绩效。检查项从十项加到三十项，通过率自然下降，然后大家开始走过场。流程的价值在“每项都真的在看”，不在项数。

90.11.3 代码生成与手写

§90.2.8 的账已经算过：生成代码的缺陷数量更少，但如果模型没验证，总成本与手写打平。这个结论值得展开。

生成的优势在结构一致。同一套模型生成的代码，命名、边界处理、数据类型转换都统一，评审时看一份就够。手写代码每个模块风格不同，评审成本随模块数线性上升。

生成的劣势在可调试性。生成代码里有大量中间变量，单步调试时栈很深，符号与模型对不上。现场出问题时，工程师更愿意看手写代码。

因此分工要清楚：算法主体生成，因为它公式密集、手写易错；驱动与通信手写，因为它们依赖硬件时序、寄存器细节，生成器很难覆盖。

生成的三道门前文已给过：模型单元测试、背靠背比对、生成代码禁手改。三道的成本大约是一次性两周，换来的是后续每次改模型都不用重写代码。

不适合生成的场景也要说清：状态机为主的逻辑、强依赖硬件时序的代码、规模小于两百行的模块。这三类生成的收益抵不上配置生成器的成本。

90.11.4 单仓与单体、多仓与拆分

§90.3.3 讨论过单仓与多仓。这里补一层：仓库结构不等于部署结构。

单仓可以部署成多个进程，多仓也可以打成一个固件。仓库解决的是“版本对齐”，部署解决的是“故障隔离”，两者不要混为一谈。

控制项目多数应该是单仓单固件。原因是控制回路之间耦合紧，采样周期一致，拆成多进程只会引入进程间通信的延迟与不确定性。§89 章讨论过的通信开销，在进程边界上同样存在。

只有在两种情况下才考虑拆分：一是不同部分的实时性要求差一个数量级以上（比如 1 ms 的控制环与 1 s 的报表）；二是不同部分的认证要求不同（安全相关与非安全相关）。

拆分的代价要提前认：跨进程的调试、跨进程的时钟同步、跨进程的版本兼容。这三样每一样都能吃掉两周。

90.11.5 云工具链与本地工具链

流水线放云上还是放本地，是个现实问题。控制项目的代码往往不能出内网，这一条就排除了一半方案。

本地工具链的优势是数据不出门、硬件在环方便接。劣势是维护成本高，机器坏了要人修，磁盘满了要人清。

云工具链的优势是弹性与免维护，劣势是硬件在环要么上云（贵），要么留一个本地代理（复杂）。

折中方案是“代码检查与单元测试上云，集成与硬件在环留本地”。前者不需要硬件，跑在云端没有障碍；后者需要样机，留在本地更实际。

无论哪种，有一条不能妥协：构建环境要能被完整描述。用容器镜像或脚本固化编译器版本、依赖版本、环境变量。做不到这一条，“我这儿能编过”就会一直是口头禅。

90.11.6 收益与代价的总账

把本章各项收益按 §90.2 的口径汇总一次，便于向管理层说明投入产出。

缺陷成本一项：同样 20 个缺陷，单元测试阶段修 200 h，现场修 1500 h，差 1300 h。这是投入测试的最大理由。

回归成本一项：全量回归每次 312 s，按每周 3 次发布算，一年约 13.5 h；没有回归的话，靠人工验证，每次至少 4 h，一年 624 h。差 46 倍。

变更失败一项：手工 28.80 h/年，流水线加自动回滚 0.72 h/年，差 40 倍。这一项的改善主要来自恢复时间而非失败率。

投入侧：建仓与整理两周、流水线与测试框架三周、追踪矩阵一周，合计约六周。这是三个人的六周，或者一个人的一个半月。

六周投入换每年上千小时的缺陷与回归成本，账是正的。但要注意，这个账只在项目还要维护一年以上时成立。短于半年的项目，投入收不回来。

上面这笔账只算了显性投入，还有三项隐性成本要认。

第一项是学习成本。团队要花时间学流水线语法、学测试框架、学模型生成工具。三个人各花两周，就是六人周，比搭流水线本身还贵。

第二项是维护成本。流水线会随工具升级而失效，测试会随代码重构而失效，追踪矩阵会随需求变更而失准。这些维护分散在日常，不显眼，但每年大约要吃掉搭建成的一半工时。

第三项是摩擦成本。流程会让“临时改一下”变难，而控制项目里临时改动恰恰很多。摩擦本身不是坏事，但如果流程设计得让正常改动也要绕三道弯，人就会开始找后门。

三项隐性成本加起来，大约让显性投入翻倍。把这一点算进去，回收期从半年变成一年。这也是为什么 §90.10.1 强调先选轻档、再逐步加码。

回报侧也有复利。第一年攒下的用例、追踪矩阵、复盘记录，第二年会自动起作用：新人上手更快，同类事故更少，评审更省时间。这部分收益难以量化，但它正是老项目比新项目“改得动”的原因。

90.11.7 方法的边界

有四类情况，本章的方法收益有限，甚至会有反效果。

第一类，算法还在探索期。模型结构每周都换，此时写大量单元测试是浪费，因为测试跟着模型一起废。这个阶段应该把力气放在数据与评估脚本上。

第二类，硬件还没定型。IO 通道、采样周期、执行器量程都在变，测试的桩件要反复重写。此时应把测试集中在与硬件无关的算法层。

第三类，一次性交付。做完就扔的项目，建流程的时间比写代码还长。此时只需版本管理与一份能跑通的说明。

第四类，团队完全没有软件背景且无人愿意学。强行上流程会变成填表，反而不如把代码写简单些、把接口定死些。

这四类不是“不用管”，而是“用轻的那一档”。版本管理在任何情况下都该有，因为它成本极低、收益极高。

边界还有一条时间维度：项目的前三个月与最后三个月，流程重心不同。前三个月重在把地基打好——编号、目录、冒烟脚本；最后三个月重在把证据收齐——追踪矩阵、测试报告、发布记录。中间那段才是优化效率的时间。

把顺序颠倒的团队很常见：交付前一个月才开始补追踪矩阵，结果补出来的是一堆事后编的对应关系，评审一问就穿帮。证据是日常攒的，不是最后补的。

还有一种情况要让路：现场抢修。抢修时一切流程都要给恢复让路，但抢修结束必须补记录。抢修不补记录，等于每次抢修都在给下一次事故埋雷。

90.11.8 三个常见反对意见

第一个意见是“我们人少，搞不了这些”。人少恰恰更需要流程，因为人少意味着没有冗余，一个人请假整条线就停。人少的正确做法是选轻档，而不是不建。

第二个意见是“我们代码就一万行，不用这么麻烦”。代码量与风险不成正比，一万行的飞控代码比十万行的报表系统危险得多。判断依据是失效后果，不是行数。

第三个意见是“客户不让改研发流程”。多数情况下客户管的是交付物与证据，不管内部怎么做。把需求编号、测试记录、发布清单当成交付物的一部分提出来，往往反而受欢迎，因为它们正好是评审要看的东西。

三个意见背后是同一个误解：把流程当成额外负担。流程真正替代的是“事后补救”，而事后补救的成本按 §90.2.1 的倍数算，比事前投入高一个数量级。

90.11.9 与前后章的分工

第 86 到 89 章解决“控制怎么算、怎么连”：模型、离散化、实现、通信。本章解决“这些代码怎么被管住”。

第 91 到 93 章转向系统层面：功能安全、信息安全、项目交付。本章的追踪矩阵与变更记录，是那三章的证据基础。安全认证要看需求到测试的完整链路，交付验收要看版本与配置的对应关系。

所以本章的位置是“底座”。它本身不产生控制性能，但决定了性能能不能被稳定地复现、修改与证明。

写文档时也要按这个关系组织：本章的输出物（追踪矩阵、发布清单、复盘记录）在后三章会被反复引用，格式要一次定好，不要中途改。

90.11.10 与 ASPICE、CMMI 的关系

控制项目常被要求过 ASPICE 或 CMMI。很多人以为那是另一套东西，其实本章的方法正好覆盖了其中大部分“证据要求”。

ASPICE 看的是过程证据：需求有没有编号、设计与需求有没有对应、测试有没有覆盖需求、变更有没有记录。这四件事就是 §90.3 的追踪矩阵与 §90.10.6 的发布清单。

CMMI 看的是过程稳定性：同一件事换个人做，结果是否一致。这对应本章的构建可重现与流水线固化。

所以不要为认证单独建一套文档。日常用本章的方法跑，评审时导出即可。反过来，为认证单独写的文档，往往与真实研发脱节，评审一追问就露。

两者真正多出来的要求是“记录的形式”，比如评审签到、问题跟踪单编号。这些可以叠加，不需要改方法本身。

一条提醒：不要把过级当目标。过级只说明流程齐备，不说明软件可靠。见过流程文件齐全但现场频繁停机的项目，也见过流程朴素但三年无事故的项目。

90.11.11 一页纸结论

把本章压成一页，六句话。

第一，缺陷成本随阶段指数上升，单元测试阶段修一个 10 h，现场修一个 75 h。这是所有测试投入的出发点。

第二，覆盖率会饱和，算法 95%、驱动 80% 是经济上限，再多是浪费。

第三，追踪链是乘积不是平均，三层各 90% 只剩 72.9%，短板决定端到端。

第四，反馈时间决定批大小，批大小决定定位范围。把集成测试移出提交级，是最划算的一步。

第五，代码生成把缺陷从代码搬到模型。不验证模型，总成本与手写打平。

第六，回滚速度比失败率更能决定损失。流水线加自动回滚，把年损失从 28.80 h 压到 0.72 h。

如果只记住一句：软件工程在控制项目里的作用，是把“我记得当时是这样”换成“记录显示是这样”。

90.12 Python 仿真代码

本章的数字不是估出来的，是两个脚本跑出来的。这一节说明怎么运行、怎么改、怎么读。

90.12.1 两个脚本与运行方式

`code/ch090_sim.py` 负责算，`code/ch090_figs.py` 负责画。前者只依赖 `math` 与 `numpy`，后者在它的基础上调用 `styles/plot_styles.py` 与 `styles/flow_styles.py`。

运行顺序不能颠倒，因为画图脚本要导入算数脚本的常量与函数：

```
python code/ch090_sim.py
python code/ch090_figs.py
```

第一条命令打印九段报告，约 60 行；第二条在 `figs/ch090/` 下生成十张图的 SVG 与 PDF。两个脚本都固定了随机种子 `20261024`，全章只有分支冲突一处用到抽样，重跑逐位一致。

90.12.2 参数区

所有可调量集中在文件头部，改参数不用往下翻。表 90-12 是参数与正文的对应关系。

表 90-12 仿真参数与正文对应

参数	取值	含义	正文位置
`REL_COST`	1/5/10/20/50/150	六阶段缺陷修复相对倍数	§90.2.1
`BASE_HOURS`	0.5	需求阶段单缺陷工时	§90.2.1
`REL_A`、`REL_B`	120、0.08	可靠性增长的总缺陷与检出率	§90.2.2
`COV_MAX`、`COV_N0`	0.98、25	覆盖率上限与特征用例数	§90.2.3
`DEVS`、`P_OVERLAP`	6、0.15	并行人数与文件重叠率	§90.2.5
`CI_STAGES`	六阶段秒数	流水线全量与增量耗时	§90.2.6

改参数要成组改。比如把团队从 6 人扩到 12 人，`DEVS` 要改，`FILES_PER_DAY` 往往也要跟着降，因为分工更细、每人碰的文件更少。只改一个，结论会偏。

90.12.3 九段报告索引

报告用 `[1]` 到 `[9]` 编号，与章节的顺序一一对应。表 90-13 是索引。

表 90-13 九段报告索引

段	内容	主要输出
[1]	缺陷成本	六阶段单缺陷工时、20 个缺陷的两个总数
[2]	可靠性增长	七个时间点的失效率与 MTBF、半衰期
[3]	覆盖率	六个用例数下的覆盖率、90% 与 95% 所需用例数
[4]	追踪链	四种覆盖率组合的端到端完整率与断链数
[5]	分支冲突	五个分支寿命下的冲突数、工时、缺陷数
[6]	流水线	六阶段耗时、全量与增量的反馈时间与批大小
[7]	回归策略	四种策略的用例数、耗时、漏检率、单缺陷成本
[8]	代码生成	四种做法的代码缺陷、模型缺陷、成本与工时
[9]	变更失败	四种运维方式的失败率、恢复时间、年损失

正文引用的每个数字都能在这九段里找到对应行，找不到就说明引用错了。这条规则在多人写作时特别有用，评审时可以直接对着报告核。

90.12.4 三个实现要点

第一个要点是成本口径统一。缺陷成本、回归成本、代码生成成本，全部折算成“工时”再比较。不折算就没法横向比，一个说“少 4 个缺陷”，一个说“多 3 秒”，谁也说服不了谁。

第二个要点是边界先算。覆盖率函数在用例数趋于无穷时趋近 `COV_MAX`，如果直接写 `c_max * (1 - exp(-n/n0))`，n 很大时会因浮点误差出现 98.0000001% 这种数。脚本里对结果做了截断，避免图上出现超过 100% 的刻度。

第三个要点是分支冲突按周而不是按天报。单日的冲突数常常是 2.25 个这种带小数的值，读起来没有实感；乘 5 变成“每周 11.25 个”之后，工程师立刻能判断“这个频率能不能忍”。

90.12.5 十张图怎么生成

十张图分两类。数据类六张用 `plot_styles`，流程类四张用 `flow_styles`。

数据类包括图 90-1 成本柱状、90-2 可靠性曲线、90-3 覆盖率饱和、90-4 分支双轴、90-5 流水线堆叠、90-6 回归散点。它们共用 `setup()` 的字体与线宽，单栏 8.5 cm、双栏 17 cm。

流程类包括图 90-7 的 V 模型三泳道、90-8 的工具链三泳道、90-9 的分支模型四泳道、90-10 的三人分工泳道。这四张只用了泳道、圆角矩形、箭头与文字标注四种图元，没有任何传递函数方块，符合第 86 至 93 章的图元约束。

画流程图的经验是先把泳道位置算好再放方块。泳道高度、标题带宽度、方块间距三个数定下来，后面就是填格子。反过来先放方块再调泳道，改一处要动十处。

SVG 与 PDF 各出一份是有原因的。SVG 用于排版时的矢量嵌入，PDF 用于单独查看与评审。两份都由同一段代码生成，不会出现“图注与图不一致”的问题。

90.12.6 五个自改实验

拿到脚本之后，建议按下面五个实验各改一次，看结论怎么变。

实验一：把 `BASE_HOURS` 从 0.5 改成 2.0。所有成本等比放大四倍，但“单元测试与现场差 7.5 倍”这个比例不变。这说明结论对基准工时免疫，只对倍数敏感。

实验二：把 `REL_B` 从 0.08 改成 0.16。半衰期从 8.66 天变成 4.33 天，第 30 天的 MTBF 从 27.6 h 升到 152 h。这是“测试效率翻倍”的效果，比“多测 30 天”更划算。

实验三：把 `COV_N0` 从 25 改成 50。90% 所需用例数从 62.6 翻到 125.2，直接翻倍。这说明用例的“等效用”比数量更重要，写得越重复，`n0` 越大，越难达标。

实验四：把集成测试的全量耗时从 300 s 改成 60 s。全量反馈从 602 s 降到 362 s，批大小仍是 2，没有跨台阶。这验证了 §90.8.2 的说法：没跨台阶的优化等于没做。

实验五：把 `P_OVERLAP` 从 0.15 改成 0.05。五天的冲突从 11.25 降到 3.75，工时从 90 min 降到 30 min。这说明“减少文件重叠”比“缩短分支寿命”更有效，而前者靠的是模块边界清晰。

五个实验做完，参数与结论的关系就清楚了。这比读十遍公式有用。

90.12.7 把报告读一遍

第一次跑完，建议按 `[9]`、`[8]`、`[3]`、`[5]` 的顺序读，而不是从 `[1]` 顺着读。

先读 `[9]`，因为它是唯一直接给“年损失小时数”的段落，最容易理解，也最能说服人。四行数字从 28.80 h 降到 0.26 h，落差一眼可见。

再读 `[8]`，它解释了一个反直觉现象：生成代码的缺陷数量只有手写的一半，成本却一样。读的时候盯住“成本”那一列的加法，`160.0+0.0` 与 `10.0+150.0` 都是 160，就明白了。

然后读 `[3]`，看 90% 与 95% 分别需要多少个用例。这一段的用处是把“覆盖率不够”这种模糊抱怨，换成“还差 25 个用例”这种可派工的结论。

最后读 `[5]`，看冲突数随分支寿命的线性增长。注意它是线性的，不是指数的，所以“分支活两周”不会灾难性，但工时确实随寿命一比一往上走：10 天 180 min，20 天 360 min。

`[1]`、`[2]`、`[4]`、`[6]`、`[7]` 留到需要引用具体数字时再查。`[1]` 的 150 倍与 `[4]` 的 72.9% 是两个最常用的引用点。

90.12.8 换成自己的数

脚本里的参数是某类控制项目的典型值，不是通用值。换成自己的数要按三步走。

第一步换规模量：`DEVS`、`FILES_PER_DAY`、`REL_A`、需求条数。这四个决定所有绝对数值的量级。

第二步换比例量：`REL_COST`、`P_OVERLAP`、`P_CONFLICT_DEFECT`、各策略的漏检率。这四个决定趋势的形状。

第三步换时间量：`CI_STAGES` 的六个数、`MIN_PER_CONFLICT`、`BASE_HOURS`。这三个决定换算成工时的结果。

三步换完，把 `[1]` 到 `[9]` 重跑一遍，看有没有出现明显不合理的数：覆盖率超过 100%、断链数为负、MTBF 小于采样周期，这些都是参数配错了。

最后提醒一句：这套脚本是一把尺子，不是一张合格证。它算的是“在给定假设下会发生什么”，不是“你的项目已经达标”。真正的达标要看现场数据，脚本只帮你把假设说清楚。

90.12.9 三项自检

改完脚本要跑三项自检，确认没有改坏。

第一项，可重现。连跑两次 `ch090_sim.py`，把两次输出存成两个文件，逐行比对。除了时间戳，应当完全一致。不一致说明引入了未固定的随机源，或者用了字典遍历顺序。

第二项，段落齐全。报告应有九个 `[n]` 段头，缺一段通常意味着某段代码在改动中被注释掉了，而正文还在引用它的数字。

第三项，图件完整。`figs/ch090/` 下应有二十个文件，十张图各一份 SVG 与一份 PDF，且文件大小都不为零。零字节文件多是绘图时抛了异常但被吞掉。

还有一项与数字无关但要检查：中文字体。如果运行环境缺 SimSun，图里的中文会变成方框，而脚本不会报错。生成后随手打开一张看一眼，比任何自动检查都可靠。

90.12.10 想加一段怎么办

如果要把本章的模型扩到别的主题，比如“评审有效性”或“技术债利息”，脚本的结构支持直接加段。

第一步写计算函数。约定返回列表，列表元素是元组，每个元组对应表格的一行。这样报告段只负责格式化，不做计算。

第二步在 `report()` 里加一段打印，编号接着往下排，段头写清口径与参数。

第三步决定要不要出图。数据量超过五个点就值得出图，图放在 `ch090_figs.py` 末尾，复用已有的 `plot_styles` 配置。

第四步回正文加对应小节，并在表 90-13 里补一行，让索引保持完整。

四步里最容易漏的是第四步。脚本加了段、正文没加，或者正文加了、索引没加，都会让读者对不上号。

最后一条经验：加段之前先问“这个量能不能从已有九段推出来”。能推出来的不要重复算，重复计算会带来两份可能不一致的数字，而一份不一致就足以让读者怀疑全部。

第 91 章 数字孪生与边缘计算

数字孪生与边缘计算常被当成两个话题，实际是同一件事的两面：把计算放到离设备足够近的地方，并让模型与设备保持同步。本章把这两面合成一条主线——**让模型和实物处在同一个时间尺度上**。

91.1 动机与直观理解

91.1.1 三个现场场景

第一个场景，孪生只用来做展示。大屏上的三维模型转得很流畅，温度、压力、产量都在动，但它和真实设备之间差了三十秒，也没有人用它算过任何东西。这种孪生的作用是汇报，不是控制。

第二个场景，边缘盒子成了新瓶颈。项目为了“上边缘”，把原来跑在工控机上的十几个回路搬到一台低功耗盒子里，周期从 10 ms 压到 2 ms。上线后偶发超时，一查是固定开销 1.2 ms 加每回路 0.35 ms，在 2 ms 周期下连一个回路都放不下。

第三个场景，云控制调不动。团队把控制器放到云上，单程时延 120 ms。对一个时间常数 2 s 的慢过程，相位损失 8.19°，勉强能用；换到时间常数 0.05 s 的伺服，相位损失 78.23°，闭环直接振荡。结论不是“云不行”，而是“云只能管慢环”。

三个场景对应三个问题：保真度不够、实时预算没算、分层没划。本章按这三个问题展开。

91.1.2 四个词的区别

数字孪生、仿真模型、软测量、监控看板，四个词经常被混用，含义差得很远。

仿真模型是离线的，输入是人给的工况，输出是预测，跑完就结束。它不要求与实物同步。

软测量是在线的，用可测变量推不可测变量，比如用温度与流量推成分。它只输出一个或几个量，不维护完整状态。

监控看板是在线的，但它只做展示与报警，不参与计算，也不回写。

数字孪生是四者里要求最高的：在线、维护完整状态、与实物同步、能反过来影响决策。判断一个东西是不是孪生，看它有没有“同步”这一环就够了。没有同步，它只是仿真或看板。

91.1.3 与前后章的分工

第 86 到 88 章讲控制怎么算、怎么离散、怎么在 PLC 里实现。第 89 章讲通信与网络。第 90 章讲软件工程。本章讲这些代码与模型**跑在哪里、和实物怎么对齐**。

具体分工：采样周期、扫描周期、时延的计算沿用第 88 章的结论，本章不重复推导；网络时延的测量方法沿用第 89 章；版本管理与在环测试的流程沿用第 90 章。

本章新增三样东西：保真度的量化、边缘节点的实时预算、云边端的分层判据。这三样是前面几章没有的。

第 92 章转向信息安全，第 93 章转向项目交付。本章的分层架构会被那两章引用，所以图 91-1 的边界要一次画准。

91.1.4 五个关键词

保真度是孪生与实物的接近程度。本章用阶跃响应的最大相对偏差衡量，五级从 90.55% 到 99.52%。

同步是让孪生的状态跟上实物。同步间隔决定状态偏差，0.5 m/s 的变化率下，20 ms 对应 10 mm。

时延是数据从产生到影响决策的时间。它不是一个数，而是一条链，端侧 2 ms、边缘 8 ms、云侧 120 ms 是三档典型值。

分层是把不同时间尺度的任务放到不同位置。判据是相位损失不超过 0.2 rad，慢过程允许 166.7 ms，快伺服只允许 5.00 ms。

在环是让真实部件参与测试。MIL、SIL、HIL 三级的检出率分别是 55%、70%、88%，单位成本是 1、2.5、8。

五个词里最容易误解的是保真度。它不等于“模型准不准”，而等于“模型在关心的工况上差多少”。一个全局误差 5% 的模型，在某个工作点附近可能只差 0.5%。

91.1.5 一条主线

本章所有内容围绕一句话：**让模型和实物处在同一个时间尺度上**。

孪生的保真度决定“模型说的对不对”，同步决定“模型说的是不是此刻”，边缘的实时预算决定“模型来不来得及算”，分层决定“该在哪一层算”。

四件事里任何一件不成立，孪生就只能停在展示层。保真度不够，预测没有意义；不同步，预测的是过去；来不及算，输出总是迟到；分层错了，迟到的量级会大到无法补偿。

反过来，四件事都成立时，孪生能做三件实事：在虚拟环境里试参数、在故障前给出预警、在交付前替代部分现场调试。

91.1.6 三把时间尺子

本章涉及三个时间尺度，混用会得出荒谬的结论。

第一把是毫秒级：控制周期、边缘节点的计算预算、端到端的抖动。这一层的判据是“最坏情况能不能在周期内完成”。

第二把是秒级：同步间隔、网络批量、HIL 的单条用例时长。这一层的判据是“状态偏差是否可接受”。

第三把是天级：参数漂移、数据保留、模型再训练。这一层的判据是“偏差累积到阈值要多久”。

三把尺子对应三组参数：周期与占用率、同步间隔与偏差、漂移速率与检测延迟。调参时不能跨尺度类比，把毫秒级的余量拿去补天级的漂移，是没有意义的。

91.1.7 本章不做什么

本章不推导控制算法。PID 见第 44 章与第 106 章，MPC 见第 58 章与第 80 章，Smith 预估见第 47 章与第 107 章。本章只讨论这些算法跑在哪里、需要多少时间、数据从哪来。

本章不讨论三维建模与可视化。孪生的价值在计算与比对，不在渲染。一个大屏做得很漂亮但不同步的孪生，不如一张准确的趋势图。

本章不讨论云平台的选型与运维。云侧的容量规划、容器编排、计费模型属于 IT 范畴，本章只在“该不该上云”这一层给判据。

本章也不讨论信息安全的细节。边缘节点暴露在车间网络里，安全风险真实存在，但那是第 92 章的内容。

91.1.8 三个数字先记住

本章公式不少，但真正需要在会上说出口的只有三个数字。

第一个是 2.44 倍。它来自表 91-1：用 G1 孪生整定出来的参数放到实物上，调节时间是高保真档的 2.44 倍。这个数字回答“孪生不准会怎样”，比“保真度 90.55%”更让人有感觉。

第二个是 0 个回路。它来自表 91-3：2 ms 周期下，固定开销 1.2 ms 加单回路 0.35 ms 已经占掉 77.5%，超过 70% 的上限，一个回路也放不下。这个数字回答“边缘盒子为什么跑不动”。

第三个是 78.23°。它来自表 91-4：云侧 120 ms 时延对 40 rad/s 的快伺服造成 78.23° 的相位损失，闭环不可能稳。这个数字回答“为什么快伺服不能上云”。

三个数字对应三类争论：保真度值不值得投入、边缘盒子够不够用、云控制能不能落地。记住它们，比背十条定义有用。

三个数字还有一层共同的读法：它们都是“差距”，不是“绝对值”。2.44 倍是两级孪生的差距，0 与 36 是两种周期的差距，78.23° 与 0.14° 是两类对象的差距。工程判断几乎总是从差距出发的。

91.1.9 读这一章的顺序

第一次读，按 §91.1、§91.2、§91.3 的顺序走，先把“衡量什么、怎么算、怎么定”这条线拉通。

§91.4 到 §91.6 留给已经动手的人。连续与离散的区别、稳定性与性能的验收，都是调试阶段才会真正关心的内容。

第二次读，直接翻 §91.7 到 §91.9。这三节是参数手册，遇到具体问题按标题查，不必从头看。

第三次读，看 §91.10 与 §91.11。前者讲怎么落地，后者讲什么时候不该做。两节都偏判断，没有唯一答案，适合在方案评审前看。

§91.12 是代码说明，不必通读。跑一遍脚本，对着九段报告回看正文，比读十页说明快。

91.1.10 一段现场对话

把三个数字放进一段真实对话里，能看出它们的用法。

供应商说，他们的边缘盒子算力是上一代的三倍，可以跑二十个回路。回答只需要一句：周期是多少。如果对方说 2 ms，那么式 91-6 给出的回路数是 0，算力再高三倍也不改变固定开销占满周期这件事。

集成商说，可以把控制器放到云上，云端有弹性算力。回答也只需要一句：对象的闭环带宽是多少。慢过程 1.2 rad/s 配 120 ms 是 8.19°，能用；快伺服 40 rad/s 是 78.23°，不能用。弹性算力与时延无关。

项目经理说，孪生已经做到 95% 的准确度，够用了。回答要问一句：用它整定出来的参数，在实物上劣化多少。95.22% 对应 1.21 倍，也就是劣化 20.8%，刚好压在 20% 的线外。

三段对话的共同点是：不问“好不好”，只问“数是多少”。数一出来，争论就结束了。

这也是本章反复用表格与公式的原因。定性判断容易各说各话，定量判断只有一个答案。

91.1.11 十张图各回答一个问题

本章十张图不是装饰，每张对应一个可以在评审会上被问到的问题。

图 91-1 回答“放在哪”，三层的边界与典型时延一次画清。图 91-2 回答“怎么对齐”，实体侧与孪生侧的数据流与修正回环都在里面。

图 91-3 回答“模型准不准”，保真度与实物劣化画在一起。图 91-4 回答“此刻差多少”，同步间隔与偏差是一条直线。

图 91-5 回答“算不算得动”，五个周期的预算堆叠成条。图 91-6 回答“来不来得及”，两条相位曲线跨在对数横轴上。

图 91-7 回答“在哪一级拦缺陷”，检出率与成本双轴并列。图 91-8 回答“要排多久”，台架数与工期画到第四个点。

图 91-9 回答“数据怎么走”，从采集到上云的四段链路。图 91-10 回答“谁在哪一级交付”，模型侧与实物侧两条泳道把 MIL、SIL、HIL、现场四格摊开，每格写明这一步少了哪一分假设。

十个问题对应十个设计动作。翻到某张图答不出它的问题，说明这一部分还没设计完。

91.2 数学原理

91.2.1 保真度怎么定义

保真度必须是一个可测的数，否则无法验收。本章用阶跃响应的最大相对偏差定义：

$$F = 1 - \frac{\max_t |y_m(t) - y_p(t)|}{y_p(\infty)} \tag{91-1}$$

其中 y_m 是孪生输出，y_p 是实物输出，分母取实物的稳态值。取最大值而不是平均值，是因为控制最怕最坏情况。

这个定义有三个限定。第一，只在关心的工况上算，不覆盖全工况；第二，输入取阶跃，因为阶跃最能暴露动态差异；第三，分母用稳态值，使不同量纲的对象可以横向比较。

按这个定义，本章五级孪生的保真度从 90.55% 到 99.52%，对应最大偏差 9.45% 到 0.48%。这五级的参数偏差与结果见表 91-1。

表 91-1 五级孪生的参数偏差、保真度与整定后果

等级	增益偏差	时间常数偏差	滞后偏差	最大偏差	保真度	实际调节时间
G1 粗略	10%	20%	30%	9.45%	90.55%	2.62 s
G2 可用	5%	10%	15%	4.78%	95.22%	1.30 s
G3 良好	2%	5%	5%	1.90%	98.10%	1.11 s
G4 精确	1%	2%	2%	0.96%	99.04%	1.09 s
G5 高保真	0.5%	1%	1%	0.48%	99.52%	1.08 s

表里最后一列最值得看。它回答的是“保真度不够会怎样”：用孪生整定出来的控制器，放到实物上跑，调节时间从 1.08 s 一路劣化到 2.62 s。

91.2.2 参数偏差怎么传到输出

小偏差下，输出误差与参数误差近似线性：

$$\frac{\Delta y}{y} \approx \sum_i S_i \frac{\Delta p_i}{p_i} \tag{91-2}$$

S_i 是敏感度。对一阶惯性加滞后对象，增益的敏感度是 1，时间常数的敏感度在响应初期接近 0、在过渡段接近 1，滞后的敏感度与 L/τ 同阶。

这个式子解释了表 91-1 里的非线性。G1 的三项偏差是 10%、20%、30%，如果简单相加会得到 60% 的误差，实际只有 9.45%。原因是三项偏差在时间上错开：增益偏差影响稳态，时间常数偏差影响过渡段，滞后偏差影响起始段，最大值不会同时取到。

敏感度分析的用处是**排优先级**。同样花一周时间，把增益标定准比把滞后测准更划算，因为增益敏感度恒为 1，而滞后只在大 L/τ 时敏感。

91.2.3 保真度不够的后果

保真度本身不是目的，目的是“用孪生做的决定在实物上成立”。表 91-1 的最后一列就是后果：同样的整定规则，落在不同保真度的孪生上，实物的调节时间差 2.44 倍。

这个后果的机制是整定偏保守。孪生把滞后估大了 30%，IMC 规则给出的增益就小 23%，实物上的闭环自然变慢。反过来，孪生把滞后估小了，增益就会偏大，实物上出现超调甚至振荡。

所以保真度的验收标准不该是“误差小于 5%”，而该是“用这个孪生整定出来的参数，在实物上的性能劣化不超过 20%”。按表 91-1，这个标准对应 G2 级以上。

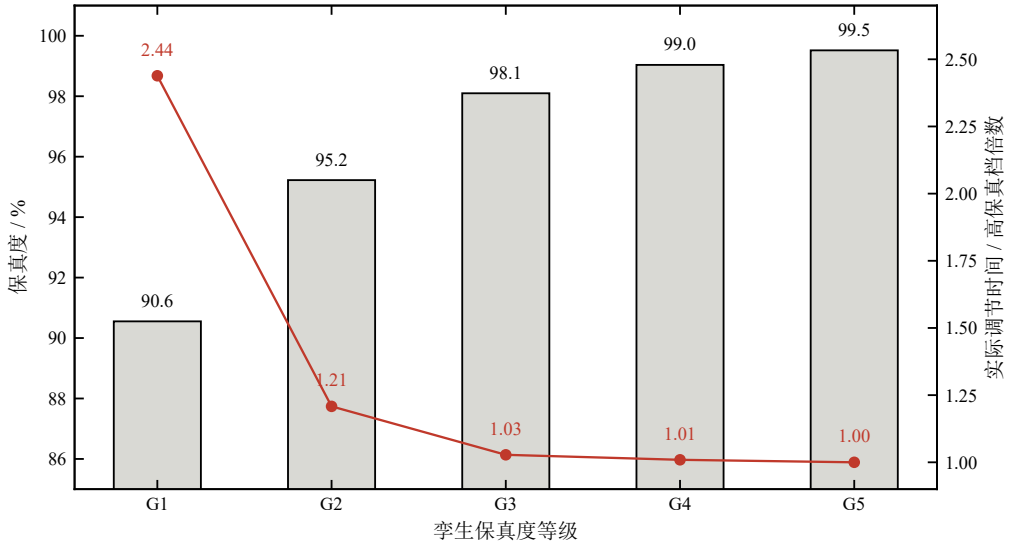


图 91-3 五级孪生的保真度与整定后果

图 91-3 把两列画在一起，柱是保真度，线是调节时间倍数。两条曲线的拐点都在 G2 与 G3 之间，说明再往上提保真度，收益迅速变小。

91.2.4 同步误差

孪生的状态滞后于实物，滞后量由变化率与同步间隔决定：

$$e_{\text{sync}} = v\,h \tag{91-3}$$

v 是状态变化率，h 是同步间隔。这个式子是线性的，没有饱和，也没有延迟，但它在多数场合够用，因为同步间隔通常远小于对象时间常数。

反过来解，给定允许偏差可以定同步间隔上限：

$$h \le \frac{e_{\text{allow}}}{v} \tag{91-4}$$

按 0.5 m/s 的变化率、10 mm 的允许偏差，上限是 20 ms。表 91-2 给了五个间隔对应的偏差，图 91-4 把这条直线与允许带画在一起。

表 91-2 同步间隔与孪生状态偏差（变化率 0.5 m/s）

同步间隔	5 ms	10 ms	20 ms	50 ms	100 ms
状态偏差	2.50 mm	5.00 mm	10.00 mm	25.00 mm	50.00 mm

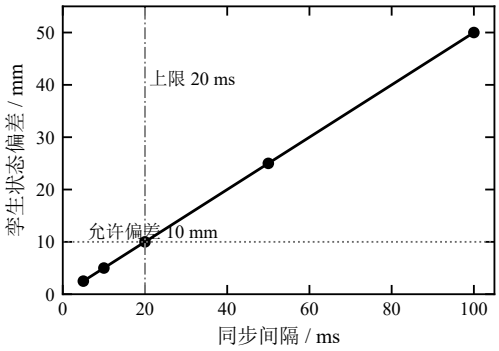


图 91-4 同步间隔与孪生状态偏差

表 91-2 的用法是反查。现场说“位置偏差不能超过 10 mm”，就直接得到 20 ms 的同步间隔，不必再讨论“同步要不要更快”。

91.2.5 边缘节点的时间预算

边缘节点的每个周期要干固定的活，再加上每个回路的活：

$$U = \frac{c_0 + n\,c_1}{T} \tag{91-5}$$

c_0 是固定开销，包括操作系统节拍、通信栈、看门狗；c_1 是单回路的计算开销；n 是回路数；T 是周期。占用率不超过上限 $U_{\max}$ 时，可支持的回路数是：

$$n_{\max} = \left\lfloor \frac{U_{\max} T - c_0}{c_1} \right\rfloor \tag{91-6}$$

按 $c_0 = 1.2$ ms、$c_1 = 0.35$ ms、$U_{\max} = 70\%$，五个周期的结果见表 91-3。

表 91-3 边缘节点在不同周期下可支持的回路数

周期	2 ms	4 ms	5 ms	10 ms	20 ms
单回路占用	77.5%	38.7%	31.0%	15.5%	7.7%
可支持回路数	0	4	6	16	36

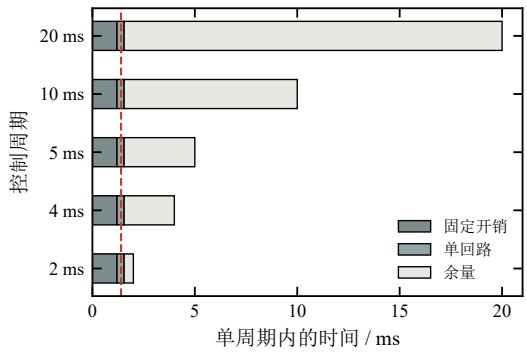


图 91-5 不同控制周期下的边缘时间预算

图 91-5 把五个周期画成横向堆叠条，固定开销与回路开销占掉的份额一目了然，余量随周期缩短迅速被啃掉。

表 91-3 的第一列是个警告：2 ms 周期下连一个回路都放不下，因为固定开销 1.2 ms 加单回路 0.35 ms 已经是 1.55 ms，占 77.5%，超过 70% 的上限。

这张表也解释了为什么“边缘盒子跑十几个回路”往往失败。十几个回路需要 10 ms 以上的周期，而项目为了“实时”把周期设成 2 ms，两个要求互相矛盾。

91.2.6 时延怎么折算成相位

时延对闭环的影响可以统一折算成相位损失：

$$\varphi = \arctan\left(\omega_c T_d\right) \tag{91-7}$$

ω_c 是闭环带宽，T_d 是总时延。判据取相位损失不超过 0.2 rad，即 11.5°：

$$\omega_c T_d \le 0.2 \tag{91-8}$$

按这个判据，慢过程（$\omega_c = 1.2$ rad/s）允许 166.7 ms，快伺服（$\omega_c = 40$ rad/s）只允许 5.00 ms。表 91-4 给了三层的实际值。

表 91-4 云边端三层的时延与相位损失

层	时延	慢过程损失	快伺服损失
端侧闭环	2 ms	0.14°	4.57°
边缘闭环	8 ms	0.55°	17.74°
云侧闭环	120 ms	8.19°	78.23°

表 91-4 的读法是横向比。同一层对两个对象的损失差几十倍，说明“能不能上云”不是架构问题，是对象速度问题。

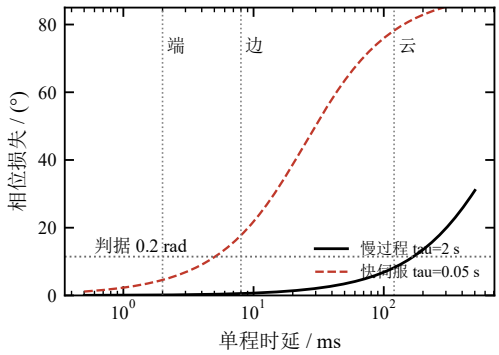


图 91-6 时延折算的相位损失与分层判据

图 91-6 把两条曲线画在对数横轴上，两条竖直虚线标出 8 ms 与 120 ms。快伺服的曲线在 8 ms 处已经越过判据线，这是“边缘也救不了快伺服”的定量依据。

91.2.7 在环测试的检出率

MIL、SIL、HIL 三级测试各有检出率，串联之后的累计检出率是：

$$P = 1 - \prod_{i} \left(1 - p_i\right) \tag{91-9}$$

每检出缺陷的成本按下式折算：

$$C = \frac{\sum_i c_i}{P} \tag{91-10}$$

按 $p=(0.55,0.70,0.88)$、$c=(1,2.5,8)$，三级串联的累计检出率是 98.4%，总成本 11.5，折合每检出缺陷 11.69。表 91-5 给了逐级数据。

表 91-5 三级在环测试与现场暴露的检出率与成本

阶段	检出率	单位成本	每检出缺陷成本	累计检出率
MIL	55.0%	1.0	1.82	55.0%
SIL	70.0%	2.5	3.57	86.5%
HIL	88.0%	8.0	9.09	98.4%
现场暴露	100%	60.0	60.00	100%

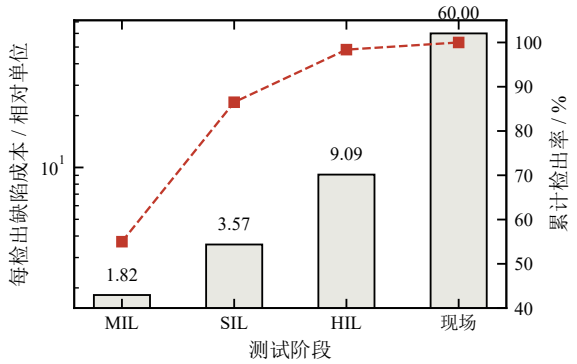


图 91-7 在环测试的检出率与每缺陷成本

表 91-5 的关键对比是最后一行。现场暴露每检出缺陷的成本是 60，是 MIL 的 33 倍。这就是“能早测就早测”的定量依据。

图 91-7 把检出率与每缺陷成本放在同一张双轴图上，横轴是四个阶段。成本柱用对数纵轴，否则 MIL 的 1.82 会被现场暴露的 60 压成一条看不见的线。

但也不能只看成本。MIL 检出率只有 55%，剩下 45% 它根本测不到，因为模型里没有真实的时序、驱动与硬件行为。三级串联的意义在于补齐，不在于替代。

91.2.8 数据量怎么估

数据量是一阶乘法：

$$Q = N\, f_s\, b \times 86400 \tag{91-11}$$

N 是通道数，f_s 是采样率，b 是每样本字节数。按 32 通道、1 kHz、4 B 计算，日数据量是 11.06 GB。

这个数决定三件事。第一，边缘节点要多大磁盘：保留 30 天需要 331.8 GB。第二，上云带宽：按 1% 抽样，每天 0.111 GB，折合 10.24 kbit/s。第三，断网缓存要多久：72 h 约 33 GB。

三个数放在一起看，结论很清楚：原始数据不出厂，边缘只存近期，云端只收特征与报警。这就是图 91-9 那条链。

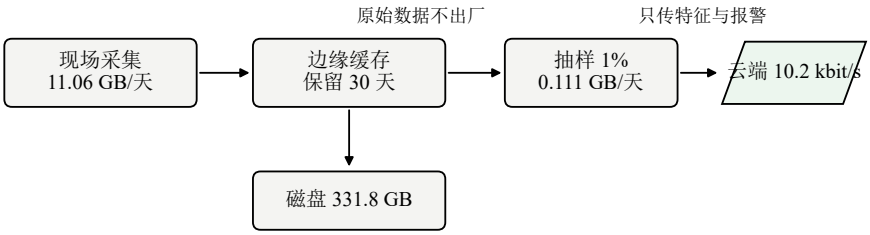


图 91-9 采集、缓存、抽样上云的数据分层

91.2.9 漂移多久能被发现

参数漂移是缓慢的，检测延迟由漂移速率与阈值决定：

$$t_d = \frac{\theta_{\mathrm{th}} - \theta_0}{r} \tag{91-12}$$

按阈值 5%、漂移速率 0.5 到 4 %/月，检测延迟从 10 个月缩短到 1.25 个月。表 91-6 给了四个速率。

表 91-6 漂移速率与检测延迟（阈值 5%）

漂移速率	0.5%/月	1%/月	2%/月	4%/月
到阈值时间	10.0 月	5.0 月	2.5 月	1.25 月
三个月累计	1.5%	3.0%	6.0%	12.0%

表 91-6 有个隐含结论：阈值定得越低，发现越早，但误报越多。5% 是一个折中，它对 2%/月的漂移能在 2.5 个月内报警，同时不会被测量噪声频繁触发。

三个月累计那一行是给季度体检用的。如果每个季度量一次，2%/月的漂移在第一次体检时就会超过 5%，4%/月的会到 12%，都跑不掉。

91.2.10 HIL 回归要排多久

HIL 的工期是轮数与单例时长的乘积，再乘准备系数：

$$T_{\mathrm{HIL}} = \left\lceil \frac{N}{n_b} \right\rceil t_{\mathrm{case}} \, (1 + \eta) \tag{91-13}$$

N 是用例数，n_b 是台架数，t_{case} 是单例时长，η 是准备系数。按 400 例、3 min、$\eta = 0.30$，单台架 26.00 h，两台 13.00 h，三台 8.71 h。

换算成工作日更有意义。按每天有效 6 h，单台架 5 天，两台 3 天，三台 2 天。表 91-7 与图 91-8 用的是这个口径。

表 91-7 台架数与 HIL 回归工期

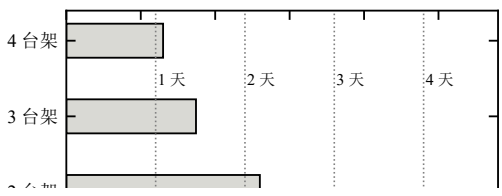
台架数	每批用例	批次余量	工期	工作日
1	400	0.0%	26.00 h	5 天
2	200	0.0%	13.00 h	3 天
3	134	0.5%	8.71 h	2 天
4	100	0.0%	6.50 h	2 天

26.00 h

13.00 h

8.71 h

6.50 h



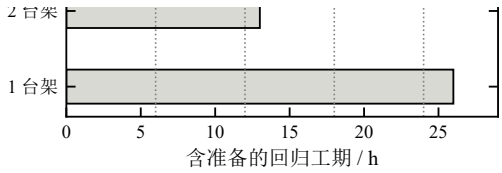


图 91-8 台架数对 HIL 回归工期的影响

表 91-7 到第三台架之后收益就平了：3 台 2 天，4 台还是 2 天。原因是整数效应，工期按天取整，多出来的台架只减少了几个小时，不足以跨过一个工作日。

这条结论对资源申请很有用：申请到第三台就该停，第四台的边际价值接近零，不如把钱花在缩短单例时长上。

91.2.11 三笔误差怎么合起来算

前面十个小节各算一种误差，现场评审时却总要回答一个问题：这些误差加起来够不够用。

合起来算不能直接相加，因为三笔误差的单位不同。模型误差是一个百分比，同步误差是一个长度，时延误差是一个角度。单位不同的量相加，得到的东西没有物理意义。

正确的做法是把三笔误差都折算成同一个量——闭环性能的劣化比例。模型误差按表 91-1 的最后一列折算，同步误差按“偏差占允许行程的比例”折算，时延误差按“相位损失占相位裕度的比例”折算。

折算之后再相加，并且按最坏情况相加，不按均方根。控制怕的是最坏情况同时出现，均方根会低估这种巧合的概率。

预算分配建议是模型占四分之一、同步占四分之一、时延占二分之一。时延占一半，是因为它最难补偿；模型误差可以靠反馈部分兜住，同步误差可以靠插值部分兜住，时延只能靠缩短链路。

留 30% 不分配。这部分不是浪费，是给现场工况变化留的余量。把预算分到 100% 的方案，投运后遇到第一个夏天就会超。

91.2.12 三个量不能互相替代

工程讨论里最常见的错误，是拿一个指标去补另一个指标。

时延不能用保真度补。模型再准，信息迟到就是迟到，控制器拿到的是过时的状态，算得再对也没用。式 91-7 里时延单独出现在反正切里，与模型误差没有替换关系。

保真度不能用同步补。同步解决的是“此刻状态是多少”，保真度解决的是“给定状态会怎么演化”，两者一个管现在、一个管未来。同步做到 1 ms，模型差 10%，预测照样偏。

算力不能用分层补。把云侧的算力堆到极限，也消不掉 120 ms 的往返时延，因为瓶颈在物理距离与协议栈，不在 CPU。表 91-4 的 78.23° 是链路决定的，换更快的服务器不会变。

三条合起来是一句话：**每个指标守自己的门，不能互换**。评审时听到“我们模型很准，所以时延大一点没关系”，就该打断。

91.2.13 一个位置环的三笔账

把三笔误差放在一个具体对象上，算法就清楚了。

设位置环的闭环带宽 40 rad/s，允许位置偏差 10 mm，边缘节点闭环时延 8 ms，孪生等级 G3。

第一笔，同步。按 0.5 m/s 的变化率、20 ms 的同步间隔，偏差 10 mm，正好用满允许值，没有余量。

第二笔，时延。8 ms 对 40 rad/s 的相位损失是 17.74°，判据是 11.5°，已经越线。这一笔过不去。

第三笔，保真度。G3 对应的实物劣化是 2.8%，在 20% 的线内，余量充足。

三笔账摆在一起，结论很明确：要动的是时延，不是模型。把闭环从边缘挪到端侧，时延从 8 ms 降到 2 ms，相位损失从 17.74° 降到 4.57°，这一笔立刻过关。

挪完之后再看同步。端侧采样率更高，同步间隔可以从 20 ms 收到 10 ms，偏差从 10 mm 降到 5 mm，余量也出来了。

这个算例说明误差预算的用处：它不告诉你“系统好不好”，它告诉你“下一步该改哪里”。

91.2.14 采样不等于同步

两个概念经常被混为一谈，实际差得很远。

采样是“多久测一次”，由传感器与采集卡决定，通常是 1 ms 或更快。同步是“多久把测到的状态送进孪生一次”，由通信与计算决定，通常是 10 ms 到 1 s。

采样率高不代表同步快。采集卡 1 kHz，孪生却每 500 ms 才拿一次数据，同步误差仍按 500 ms 算，与采样率无关。

反过来，同步快也补不回采样慢。同步间隔 5 ms 而采样间隔 100 ms，孪生拿到的仍是每 100 ms 才更新一次的状态，中间靠插值填补，插值补不出真实的高频变化。

正确的说法是：同步误差由同步间隔决定，采样率只决定插值的精度。两者是串联关系，不是替代关系。

配系统时先定采样率，再定同步间隔，最后检查同步间隔是否大于采样间隔的整数倍。顺序乱了，会出现“采样 1 kHz、同步 5 ms”这种看起来很快、实际插值三次的配置。

91.2.15 保真度的三种验收口径

保真度可以用三种口径验收，选哪种决定了后面怎么测。

第一种是时域最大偏差，即式 91-1。它直观、好算、能画成曲线，适合工程验收。缺点是只看最坏点，不看整体形状。

第二种是频域幅值误差，比较孪生与实物在关心频段内的幅频特性。它适合判断“在某些频率上会不会共振”，不适合向非专业人士解释。

第三种是闭环性能劣化，即用孪生整定、在实物上跑，比较调节时间或超调。它最贴近用途，但需要实物试验，成本最高。

本章主用第一种，用第三种做验收。理由很实际：第一种可以离线算，第三种必须上实物，两者结合既省事又不失真。

三种口径不能混着报。报“保真度 98%”时要说清是哪种，否则两个团队各报一个数，争论半天发现口径不同。

91.3 设计步骤

91.3.1 五步与输出物

做一套能用的孪生加边缘系统，按五步走，每步有一个明确输出。

第一步，划边界。输出一张分层职责表，写清哪些功能放在端、边、云，以及跨层的接口。

第二步，定点表。输出数据点表，每个点写清名称、量程、采样率、精度、是否入孪生。

第三步，定同步。输出同步方案，写清同步间隔、时间戳来源、断链时的处理。

第四步，算预算。输出边缘节点的周期与占用率表，以及最坏执行时间的估算。

第五步，排测试。输出在环测试计划，写清 MIL、SIL、HIL 各测什么、各用多少用例、几天完成。

五步的顺序不能颠倒。先定点表再划边界，会漏掉跨层需要的点；先算预算再定同步，会把同步开销算漏。

91.3.2 孪生的边界怎么划

孪生不必覆盖全部设备。划边界的依据是“这个量的预测会不会被用来做决定”。

会用来做决定的，进孪生：关键温度、压力、位置、以及它们的导数。不会用来做决定的，不进：辅助风机的电流、柜内温度、照明状态。前者决定控制的正确性，后者只影响维护。

边界还有一层是时间。孪生要维护的是“当前状态 + 一段历史”，不是“全部历史”。历史长度按最长的一次比对窗口取，通常几小时到几天。

第三层是精度。同一个设备，用于整定的孪生要 G3 以上，用于报警的 G2 就够，用于展示的 G1 也行。不同用途用不同精度的孪生，比统一提高精度省钱得多。

划完边界要写一句话总结：**这套孪生回答哪三个问题**。答不出来，说明边界划得太宽，或者根本没想清楚用途。

91.3.3 云边端怎么分工

分工的核心判据是时间尺度。表 91-8 给了一张可直接抄的职责表。

表 91-8 云边端三层的职责划分

层	时间尺度	该做的事	不该做的事
端	微秒到毫秒	采样、滤波、安全联锁、限幅	复杂优化、历史存储
边	毫秒到秒	实时控制、孪生同步、数据缓存、协议转换	长期存储、模型训练
云	秒到天	模型训练、批量分析、远程运维、报表	任何实时闭环

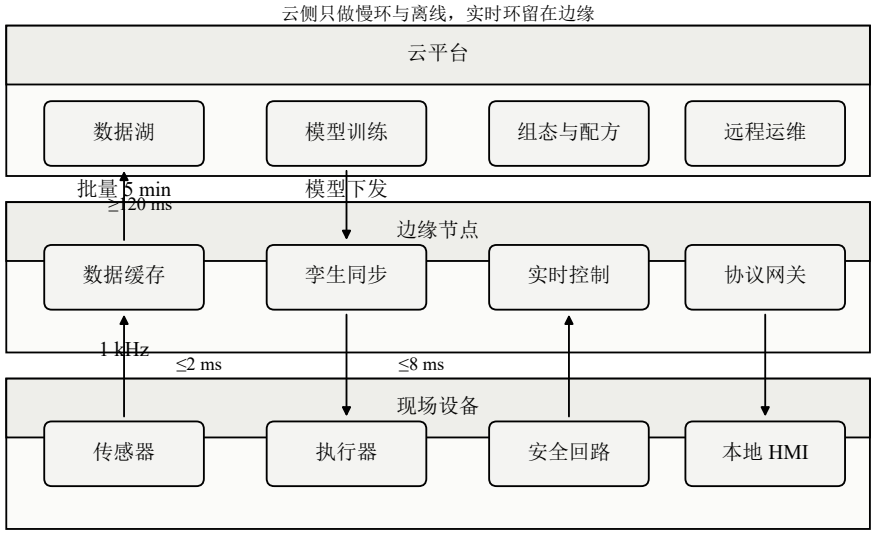


图 91-1 云边端三层架构与典型时延

表 91-8 的第三列与第四列要一起看。“不该做的事”往往是项目里最容易越界的地方：把优化算法放到端侧，把实时闭环放到云侧，都是常见错误。

越界的后果不是“稍微慢一点”，而是“整个设计失效”。端侧算力被优化算法吃掉，采样抖动上升；云侧闭环一旦断网，设备失去控制。

91.3.4 数据点表怎么定

数据点表是后面所有工作的基础。每个点至少要写六项：名称、单位、量程、采样率、精度、去向。

采样率按对象定，不按“越高越好”定。温度 1 Hz 足够，位置 100 Hz 到 1 kHz，振动可能要到 10 kHz。采样率定高了，数据量与边缘负载都跟着涨。

精度要区分分辨率与准确度。分辨率是末位代表多少，准确度是整体偏差。孪生关心准确度，看板关心分辨率。

去向分三类：只进控制、进孪生、只进历史。三类点的存储策略不同，控制点要保证实时，孪生点要保证同步，历史点可以降采样。

点表定完要做一次总量核算。32 个点 1 kHz 是 11.06 GB/天，128 个点 1 kHz 就是 44.2 GB/天，边缘磁盘立刻不够。这时要么降采样，要么只存特征。

91.3.5 同步方案怎么定

同步要解决三件事：时间戳从哪来、数据怎么对齐、断了怎么办。

时间戳统一由边缘节点打，不用端侧时钟。端侧时钟精度差，长时间运行会漂。边缘节点接一次 GPS 或 PTP，全厂一个时间基准。

对齐按“最近邻 + 插值”两级。同步间隔大于采样间隔时用最近邻，小于时用线性插值。插值会引入平滑，对高频分量不利，所以只在必要时用。

断链处理分三档。断 1 s 以内，孪生保持上一状态并标记“陈旧”；断 1 min 以内，用缓存补；超过 1 min，孪生进入“失同步”状态，输出只用于展示，不参与决策。

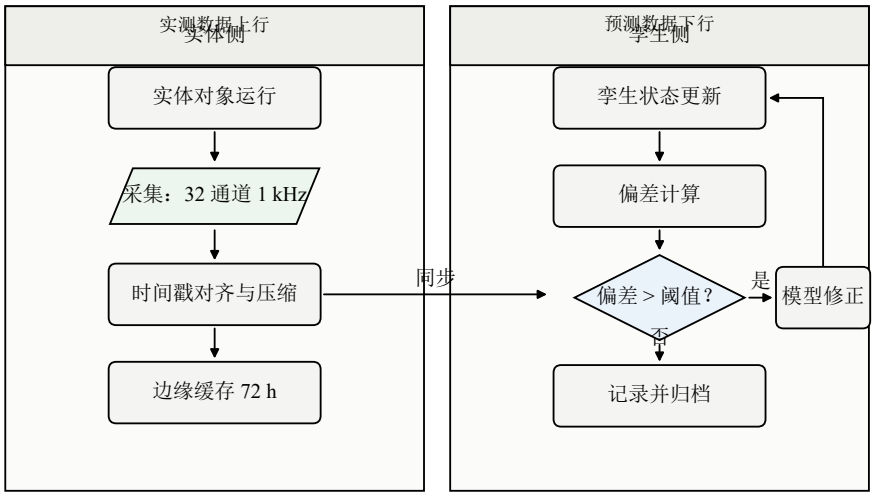


图 91-2 数字孪生的同步与修正闭环

图 91-2 的右侧有一条回环：偏差超过阈值时修正模型。这条回环要限速，每次修正不超过参数的 1%，否则一次误判会把模型带偏。

91.3.6 边缘节点怎么选

选型看四个数：周期、回路数、接口、环境。

周期与回路数用式 91-6 算。先定回路数，再反解需要多短的周期，最后看候选节点能不能在该周期下把占用率压到 70% 以下。

接口要按现场总线定。Profinet、EtherCAT、Modbus 对实时性的要求差一个数量级，节点支持的协议直接决定了能不能用。

环境常被忽略。车间温度 45 ℃、粉尘、振动，消费级盒子撑不过一个夏天。工业级节点的价格是消费级的三到五倍，但这笔钱不能省。

还要看一个软指标：能不能离线升级。边缘节点数量多、位置散，每次升级都跑现场是不现实的。支持远程升级且有回滚的节点，运维成本低一个数量级。

91.3.7 在环测试怎么排

在环测试不是“越全越好”，而是“分层补漏”。表 91-9 给一个 400 例的分配方案。

表 91-9 三级在环测试的用例分配

阶段	用例数	单例时长	台架	工期	主要目标
MIL	400	0.3 min	不限	2.0 h	算法逻辑、边界条件
SIL	240	1.0 min	不限	4.0 h	代码实现、接口匹配
HIL	400	3.0 min	3	8.71 h	时序、驱动、故障注入

表 91-9 的用例数不是随意定的。MIL 覆盖全部算法用例，因为它最便宜；SIL 只测与实现相关的 240 例，因为纯算法部分 MIL 已经覆盖；HIL 回到 400 例，因为时序与驱动的缺陷与算法用例一一对应，跳不得。

HIL 的 8.71 h 是含准备的时间。准备包括上电、刷写、标定、清场，这部分常被漏算，实际会占三成。

排计划时把 HIL 放在最后，但不要放在交付前一周。HIL 发现问题往往要改驱动或改时序，改动会影响 MIL 与 SIL 的结论，需要回过头重跑。

91.3.8 一页纸基线

设计阶段结束要填一张基线表，把关键参数一次定死。表 91-10 是模板。

表 91-10 一页纸设计基线

项	取值	依据
孪生等级	G3	整定用途，要求劣化 ≤20%
保真度	≥98.1%	表 91-1
同步间隔	20 ms	允许偏差 10 mm、变化率 0.5 m/s
边缘周期	10 ms	16 个回路，占用 15.5%
边缘占用上限	70%	留 30% 给抖动与突发
数据保留	30 天	331.8 GB 磁盘
上云比例	1%	10.24 kbit/s
漂移阈值	5%	2.5 个月可发现 2%/月 的漂移
HIL 用例	400 例 / 3 台架	2 个工作日

这张表的作用是让后续的变更有一个对照。任何一项要改，都要说明为什么，以及改了以后哪几项跟着变。

基线不是越严越好。把保真度定到 G5，同步定到 5 ms，边缘占用定到 40%，项目会被自己的指标压死，最后所有指标都被悄悄放宽。

91.3.9 五步各自的常见错误

第一步划边界，最常见的错误是“能采到的点都进孪生”。点进得越多，同步开销、模型复杂度、存储量一起涨，而多数点并不参与任何决定。

第二步定点表，最常见的错误是只写测量点，不写去向。没有去向，后面定存储策略时只能全按最高标准存，磁盘很快写满。

第三步定同步，最常见的错误是顺序反了：先拍一个 20 ms，再去问允许偏差是多少。正确的顺序永远是先有允许偏差，再反解间隔。

第四步算预算，最常见的错误是拿平均执行时间当最坏执行时间。平均值好看，P99 才决定会不会超时。

第五步排测试，最常见的错误是用例数平均分给三级。缺陷类型分布不均，平均分的结果是该测的没测够，不该测的重复测。

五个错误有一个共同点：都是“先做再想”。每一步只要先写一句判据，错误就少一半。

判据的写法有统一格式：量、阈值、测量方法。比如“占用率不超过 70%，按最坏执行时间算，用 GPIO 打点测一万个周期”。三项齐了，才叫判据。

91.3.10 设计评审要问的六个问题

评审会上不需要逐页讲方案，问六个问题就够。

第一，这套孪生回答哪三个问题。答不出来，说明边界没划清。

第二，每个数据点的去向写清楚了吗。控制、孪生、历史三类必须能一一对上。

第三，同步间隔是从允许偏差反解出来的，还是拍出来的。前者能追溯，后者迟早要返工。

第四，占用率是按最坏执行时间算的，还是按平均值算的。这一条决定了现场会不会偶发超时。

第五，HIL 的用例数与历史缺陷分布匹配吗。老项目改驱动，HIL 就该多；新项目改算法，MIL 就该多。

第六，这套系统最先失效的是哪个指标，怎么发现。答不出，说明运维方案还没做。

六个问题里，第三与第四个最容易被含糊过去。含糊的答案通常是“差不多”“经验值”，而这两项恰恰是可以精确算的。

91.3.11 一页纸基线怎么填

表 91-10 有十行，填的时候从左到右、从上到下，每一行的“依据”列必须写出表号或公式号。

第一行定等级，先想用途。整定用途就填 G3，展示用途就填 G1，不要一上来填 G5。

第二行保真度由等级查表得到，不独立填写。填了等级又填一个不相符的保真度，说明有人没查表。

第三行同步间隔由允许偏差反解，第四行边缘周期由回路数与式 91-6 反解。这两行是整张表里唯一需要算的。

第五行到第八行是资源与运维参数，按磁盘、带宽、检修周期直接填。

第九行 HIL 用例与台架数，按表 91-7 查，注意整数效应。

填完做一次交叉检查：边缘周期是否能容纳计划的回路数、同步间隔是否小于控制周期的整数倍、HIL 工期是否在交付日之前留出两周。

三项都过，这张表才可以拿去评审。

91.3.12 设计文档要写多细

设计文档不在厚，在于接手的人能不能照着做。本章建议三张图加两张表。

第一张图是分层架构，即图 91-1 的那种画法。要标清每一层的边界、每个节点的位置与地址，以及跨层的接口。

第二张图是数据流，从传感器到孪生、到云，每一段标出协议、周期、数据量。数据流图画不清，后面算带宽与缓存全是猜。

第三张图是同步时序，标出采样时刻、同步时刻、计算时刻三者的相位关系。时序图画不出来，说明同步方案还没想清楚。

第一张表是数据点表，第二张表是基线表（表 91-10）。两张表都要有版本号与日期，改动留痕。

三张图两张表之外，不要写大段文字。文字用来解释“为什么这么定”，不重复图表里已有的信息。

文档的验收标准是：一个没参加设计的人，拿着这五样东西，能独立完成一次同步间隔的修改并验证。

91.4 连续情形

91.4.1 三条假设

本章的连续模型建立在三条假设上，用之前要确认它们成立。

第一条，对象可以用低阶模型描述。一阶惯性加滞后能覆盖多数温度、流量、液位回路；伺服与多容对象要更高阶，本章的结论只能作趋势参考。

第二条，偏差是小的。式 91-2 的线性化只在偏差 10% 以内成立，偏差更大时要直接仿真。

第三条，时延是纯滞后。实际时延有分布，抖动的存在会让等效滞后大于平均值，本章按平均值算，结论偏乐观。

三条假设在多数工业过程里成立，在快伺服与多变量强耦合对象上要谨慎。

91.4.2 保真度是连续量

保真度不是“合格/不合格”，而是一条连续曲线。表 91-1 的五个点只是采样，实际可以取任意值。

连续性的用处是**算边际收益**。从 G2 到 G3，保真度提高 2.88 个百分点，调节时间从 1.30 s 降到 1.11 s，改善 14.6%。从 G3 到 G4，保真度提高 0.94 个百分点，调节时间只降 0.02 s，改善 1.8%。

边际收益递减得非常快。这就是为什么“把模型做得更准”在某个点之后不再划算，也是为什么多数项目停在 G3。

判断该停在哪一点，用“性能劣化不超过 20%”这条线。按表 91-1，G2 的劣化是 20.8%，刚好压线；G3 是 2.8%，有充足余量。

91.4.3 同步误差的连续估算

式 91-3 是线性的，可以直接外推。变化率 0.5 m/s 时，同步间隔从 5 ms 增到 100 ms，偏差从 2.50 mm 增到 50.00 mm，严格成比例。

实际系统里变化率不是常数。位置在换向点附近速度接近零，在匀速段达到最大。同步误差应该按最大变化率算，而不是平均变化率。

如果状态是旋转或往复运动，变化率本身是正弦，同步误差也是正弦，两者相位差 90°。这时最大误差出现在速度最大处，而不是位置最大处。

连续估算的另一个用处是定缓存深度。同步间隔 20 ms、断网 1 min，需要缓存 3000 帧；每帧 32 通道 4 B，约 384 kB。这个数决定了边缘节点的环形缓冲区大小。

91.4.4 时延折算的连续性

式 91-7 的反正切是连续函数，在小角度下近似线性：$\varphi \approx \omega_c T_d$。判据 $\omega_c T_d \le 0.2$ 就是这条近似线的截断点。

线性的好处是时延可以相加。端侧 2 ms 加边缘 8 ms 加网络 2 ms，总时延 12 ms，相位损失直接是各部分之和。这就是为什么分层设计时可以把预算拆开分给各层。

但相加只在小角度成立。云侧 120 ms 对快伺服的损失是 78.23°，这时线性近似会给出 $\omega_c T_d = 4.8$ rad，远超实际，说明近似失效，必须用反正切。

工程上的做法是：总时延小于对象时间常数的 20% 时用线性，超过时用反正切。这条分界与 0.2 rad 的判据是同一个来源。

91.4.5 数据量的连续估算

式 91-11 是纯乘法，连续可调。三个因子任何一个变化，数据量等比变化。

实际数据量会比估算高 10% 到 30%，因为要加包头、时间戳、校验。按 11.06 GB/天 估算的系统，实际写入可能到 13 GB/天。定磁盘时要按实际值留 30% 余量。

压缩能显著降低数据量，但要看数据类型。缓变信号压缩比可以到 10 比 1，高频振动信号只有 2 比 1。按 5 比 1 估算是稳妥的中间值。

连续估算的价值在于快速判断“这个方案可行不可行”。32 通道 1 kHz 是 11 GB/天，边缘能存；320 通道 10 kHz 是 1.1 TB/天，边缘存不下，必须当场降采样或只存特征。

91.4.6 连续模型在哪里失效

三个地方。

第一，阈值附近。漂移检测、报警、限幅都是阶跃行为，连续模型算不出“刚好越过阈值”的那一次。

第二，饱和区。执行器到限幅、传感器到量程端点，连续模型的线性关系断开，实际响应会比预测慢得多。

第三，切换。手动/自动切换、冗余切换、协议切换，切换瞬间的状态是离散的，连续模型无法描述。

三处的共同点是“有判断”。凡是有 if 的地方，就要用离散模型或者直接仿真。本章 §91.5 处理这类问题。

91.4.7 连续模型给出的三条曲线

本章的连续结论可以归成三条曲线，每条曲线的斜率都有明确含义。

第一条是保真度与性能的关系。横轴保真度，纵轴实物调节时间。这条曲线单调下降，先陡后平，拐点在 G2 与 G3 之间。斜率就是边际收益：G2 到 G3 的斜率大，说明这一段值得投入；G3 之后斜率接近零，说明再投入也换不来性能。

第二条是同步间隔与状态偏差的直线。这条线过原点，斜率等于变化率。它的用处是双向的：给间隔求偏差，或者给偏差求间隔。斜率越大，系统对同步越敏感，位置环与速度环的斜率远大于温度环。

第三条是时延与相位损失的曲线。小角度段近似直线，斜率等于闭环带宽；大角度段弯曲，趋向 90°。这条曲线的拐弯处就是线性近似失效的地方，也是“必须用反正切”的地方。

三条曲线放在一张纸上，就是本章的判断工具。遇到具体问题，先问它落在哪条曲线上，再看斜率。

91.4.8 四个可以背下来的口径

连续估算里有四个口径，记住之后多数场合不用开电脑。

第一，1 rad/s 的带宽下，1 ms 时延折 0.057°。所以 1.2 rad/s 配 2 ms 是 0.14°，40 rad/s 配 8 ms 是 17.74°，都能心算。

第二，1 m/s 的变化率下，10 ms 同步间隔对应 10 mm 偏差。变化率减半，偏差减半，比例关系不变。

第三，单通道 1 kHz、4 B 采样，日数据量约 0.35 GB。32 通道乘 32 就是 11.06 GB，通道数翻倍数据量翻倍。

第四，1% 抽样上云的带宽约等于日数据量的万分之一，11.06 GB/天 对应 10.24 kbit/s。这个换算在估专线时很好用。

四个口径都有适用条件：小角度、匀速、定点、无压缩。超出条件就要回到公式，不能硬套。

91.4.9 连续估算的三处偏差

连续估算够快，但有三处系统性偏差，用的时候要心里有数。

第一处是线性化偏差。式 91-7 在小角度下用 $\omega_c T_d$ 代替反正切，角度越大，高估越多。11.5° 处误差约 1%，45° 处会把角度估成 57°，接近 90° 时误差趋于无穷。

第二处是抖动被平均掉。连续模型里的时延是一个确定值，实际是分布。P99 时延可能是平均值的两倍，按平均值算出来的相位损失会偏乐观一半。

第三处是饱和被忽略。执行器到限幅时，闭环的实际带宽下降，同样的时延造成的相位损失反而变小，但响应变慢。连续模型算不出这种变化。

三处偏差的方向不完全相同：第一处高估性能，第二处高估性能，第三处低估性能。三者叠加后的净方向不确定，所以关键设计点要用仿真或实测复核。

工程上的折中是：粗算用连续模型，定稿前用离散仿真跑一遍，投运后用实测数据校一次。三次校核之后，估算才可信。

91.4.10 用连续模型做容量规划

容量规划是连续模型最实用的场合，因为要的是量级，不是精确值。

规划分三条链。第一条是数据链：通道数乘采样率乘字节数，得到日数据量，再乘保留天数得到磁盘，再乘上云比例得到带宽。这条链全是乘法，心算即可。

第二条是计算链：状态量个数乘同步频率，得到每秒的状态更新次数，再乘单次更新耗时，得到孪生占用的算力。这条链决定边缘节点要不要单独分核。

第三条是控制链：回路数乘单回路开销加固定开销，除以周期，得到占用率。这条链就是式 91-5。

三条链的输入都来自设计阶段的数据点表与回路清单，输出是磁盘、算力、周期三个约束。任一约束过不去，就回到设计阶段调整，不要在现场硬扛。

容量规划要留余量，但不留双份。数据链留 30%，计算链留 30%，控制链按 70% 上限。三个余量不要叠加，叠加之后设备选型会翻倍。

91.4.11 三个连续关系的用法差别

式 91-1、91-3、91-7 都是连续关系，但用法差别很大，混用会得出错误结论。

保真度是比值，无量纲。它可以在不同对象之间横向比较，但不能直接换算成性能损失，必须借表 91-1 那一列。

同步误差是长度，与量纲有关。它不能跨对象比较，位置环的 10 mm 与温度环的 0.5 ℃ 不是一回事，只能各自与自己的允许值比。

相位损失是角度，与频率有关。同一个时延对快对象损失大、对慢对象损失小，所以它天然适合做分层判据，也天然不能脱离 ω_c 单独引用。

三者的共同点是“都要带条件说”。说保真度要带等级，说同步误差要带变化率，说相位损失要带带宽。缺了条件，数字就没有意义。

这条规矩在写报告时尤其重要。同一个 10 mm，写在位置环的报告里是合格，写在温度环的报告里是没有意义的。

91.4.12 连续模型的输入从哪里来

公式的输入不是天上掉的，来自一次辨识试验或一段运行数据，两者差别很大。

手工阶跃试验的要点有三条。幅值取量程的 5% 到 10%，太小信噪比不够，太大进入非线性区；试验前工况稳定至少三个时间常数；记录长度取五个时间常数以上，否则稳态值估不准。

运行数据的要点是激励充分。设备长期在稳态附近小幅波动，数据里没有足够的信息辨识动态。这时要用伪随机或阶梯式的设定值扰动，幅度控制在工艺允许范围内。

两种数据各有偏向。阶跃数据干净，但只覆盖一个工作点；运行数据覆盖广，但噪声大、相关性强。

工程做法是先用手工阶跃建初稿，再用运行数据校一遍。初稿给量级，校核给精度，两步都做，保真度才到得了 G3。

91.5 离散情形

91.5.1 周期必须是整数倍

边缘节点的同步周期、控制周期、通信周期都要落在同一个时钟节拍上。做不到整数倍，就会出现在某个周期里两个任务撞车。

比如控制周期 10 ms、同步周期 25 ms，最小公倍数是 50 ms，每 50 ms 有一次撞车。撞车的那一周期占用率翻倍，抖动上升。

正确做法是让同步周期等于控制周期的整数倍。10 ms 控制、20 ms 同步，每两个控制周期同步一次，节拍整齐。

这条规则在配置工具里往往不强制，工程师随手填一个 25 ms，系统能跑，但抖动会莫名变大。查这类问题，先看周期是不是整数倍。

91.5.2 台架是整数

HIL 台架不能有 2.5 台。表 91-7 的结论是 3 台之后收益归零，这个结论完全来自整数效应。

400 例分 3 台，每台 134 例，最后一批有 2 台空转半轮，余量 0.5%；分 4 台，每台 100 例，余量 0%。余量都很小，但工期按天取整，3 台与 4 台都落在 2 天。

整数效应还会反过来咬人。用例数从 400 增到 420，3 台架每台 140 例，仍是 2 天；增到 480，每台 160 例，工期 10.4 h，跨到 2 天边缘，稍有延误就变 3 天。

排计划时要按“临界用例数”算，而不是按平均值算。临界点就在 6 h 的整数倍上。

91.5.3 通道要按字打包

数据采集按字打包，不按字节。32 通道 16 位，正好两个 32 位字；33 通道就要三个字，多出来的 31 位浪费。

浪费不只是存储。打包不整齐会让 DMA 传输多一次，边缘节点的固定开销上升。通道数选 32、48、64 这类整齐的数，比选 33、50 更省。

通道顺序也要固定。按物理接口顺序打包，比按逻辑分组打包更容易校验。逻辑分组在软件里重排一次，成本可以忽略。

91.5.4 时间戳的位宽

用 32 位毫秒计数器打时间戳，49.7 天回绕一次。回绕本身不可怕，可怕的是比较时间时没处理回绕，孪生会认为数据来自 49.7 天前。

处理办法有两种。一是用 64 位时间戳，彻底避免回绕，代价是每帧多 4 B。二是保留 32 位，但在比较时用模运算取最短间隔。

两种都可以，关键是要在协议里写清楚。最糟的情况是一端用 32 位、一端用 64 位，对接时才发现单位与纪元都不同。

91.5.5 存储按块分配

边缘节点的磁盘按块分配，块大小通常是 4 kB。一帧 384 B 的数据单独存，实际占 4 kB，浪费 90%。

正确做法是按分钟或按小时打包成块再写。一小时 32 通道 1 kHz 是 460 MB，块内没有浪费。

块大小也影响断网补传。块太大，补传时丢一块损失大；块太小，文件数太多，文件系统开销上升。按 1 分钟一块是常见折中。

91.5.6 批次与班次

现场调试按班次排，一班 8 h，有效工作时间 6 h。所有工期都要除以 6，不是除以 8。

HIL 的 8.71 h 折合 2 个工作日，就是因为这条。按 8 h 算会得到 1.09 天，排成 1 天，实际做不完。

班次还会影响跨班交接。一个用例跑到一半下班，第二天要重新准备，准备系数从 0.30 涨到 0.50。长用例要尽量安排在同一班内跑完。

91.5.7 三条用法

离散情形的用法可以压成三条。

第一条，所有周期取整数倍，配置完检查一遍最小公倍数。第二条，所有资源按整数算，台架、通道、班次都不例外。第三条，所有工期向上取整并除以有效工时，不用名义工时。

三条都做到，计划就不会出现“理论上一天、实际做三天”的情况。

91.5.8 环形缓冲要按帧数算，不按字节算

边缘节点缓存数据用环形缓冲，容量要按帧数定，不能按“给两兆”这种字节能敷衍。

按 §91.4.3 的算例，同步间隔 20 ms、断网 1 min 需要缓存 3000 帧。每帧 32 通道 4 B 是 128 B，3000 帧共 384 kB。

定容量时取 2 的整数次幂，即 4096 帧，读写指针用位掩码取模，省掉一次除法。多出来的 1096 帧是余量，约 22 s。

环形缓冲最怕的是“写满覆盖”。策略要提前定：覆盖最旧、丢弃最新、或者停止写入并报警。三种策略对应三种工艺要求，不能一套通用。

连续过程通常选覆盖最旧，因为最新的数据最有价值。批次过程通常选停止写入，因为中间缺一段就没法复盘。

91.5.9 序号与时间戳都要处理回绕

序列号用 16 位，每秒 1000 帧，65.5 s 就回绕一次。用 32 位，49.7 天才回绕一次。选位宽时要先算回绕周期。

判断丢帧靠序号差，但差值为负时不能直接判错，要按模运算取最短距离。差 1 是正常，差 2 是丢了一帧，差 60000 在 16 位下其实只丢了 5536 帧。

时间戳同理。回绕本身不可怕，可怕的是比较时没处理回绕，程序会认为这一帧来自 49.7 天前，缓存里的数据全部乱序。

一条实用规则：序号与时间戳必须用无符号整数，差值运算必须显式写模。用有符号整数做差值，溢出是未定义行为，换编译器就可能出错。

91.5.10 批量提交按块边界对齐

数据库与文件写入按批提交，批大小要与存储块对齐。块 4 kB，一帧 128 B，32 帧正好一块。

批太小，每帧一次系统调用，开销比数据本身还大。批太大，掉电时丢的数据多。按块对齐是两头都照顾的折中。

提交时刻也有讲究。整秒提交便于对账，但会在整秒处产生一个写入尖峰，占用率瞬间上升。错开各节点的提交时刻，尖峰就被摊平了。

掉电保护要配超级电容或 UPS，保证最后一批能落盘。没有掉电保护时，至少要把批大小压到 1 s 以内，把损失控制在可接受范围。

91.5.11 量化误差与保持效应

离散化还有两处偏差，常被忽略，但在高精度场合会显出来。

第一处是量化误差。16 位 ADC 在 0 到 10 V 量程上的分辨率是 0.15 mV，量化误差是半个末位。相对于 10 mm 的位置偏差，这个量级可以忽略；相对于 0.1% 的保真度要求，就不能忽略。

第二处是零阶保持。控制器每个周期取一次值，周期内保持不变，等效于给对象加了一个 $T/2$ 的纯滞后。10 ms 周期对应 5 ms 的等效时延，这一项在时延预算里要算进去。

两处偏差的应对不同。量化误差靠提高位数或加前置放大解决，零阶保持只能靠缩短周期解决。

算时延预算时，把 $T/2$ 加进去，再与 §91.2.6 的相位判据对照。很多“边缘时延明明只有 8 ms”的困惑，来源就是漏了这一项。

91.5.12 所有周期取同一个节拍

边缘节点上通常同时跑控制、同步、通信、日志四类任务，各有各的周期。这些周期必须落在同一个时钟节拍上。

做法是先定一个基准节拍，比如 1 ms，其余周期都取它的整数倍。控制 10 ms、同步 20 ms、通信 100 ms、日志 1000 ms，全部是 1 ms 的整数倍。

不这么做的后果是撞车。控制 10 ms 与同步 25 ms 的最小公倍数是 50 ms，每 50 ms 两个任务必然在同一周期相遇，那一周期的占用率翻倍。

撞车不会每次都导致超时，所以很难复现。它表现为“偶尔卡一下”，日志里看不出规律，因为规律是 50 ms 的倍数，而日志的分辨率往往就是 100 ms。

排查这类问题的第一步，是把所有周期列出来，算一遍最小公倍数。结果小于 1 s 的，基本可以确定是撞车。

修法很简单：把不整齐的周期改成基准节拍的整数倍，或者把两个任务的相位错开半个周期。前者更彻底。

91.5.13 整数化之后余量去哪了

周期取整、台架取整、通道取整，每次取整都会吃掉一点余量，这些余量要记在一处。

周期从 10 ms 取到 5 ms，同样 8 个回路，占用率从 40% 涨到 80%，直接越线。取整不是四舍五入，要看取整后是否还满足约束。

台架从 3 台取到 4 台，工期从 8.71 h 降到 6.50 h，但工作日仍是 2 天，多出的台架没有换来工期。

通道从 33 取到 32，少了一个点，如果那个点是关键测点，就要取 48，而不是硬塞进 33。

每次取整后要重算一遍相关约束，并把取整损失写进基线表。不写，下一次评审就会误以为还有余量。

余量的正确用法是留给工况变化，不是留给取整误差。取整误差应该在设计阶段就吃掉。

91.5.14 离散模型的三个前提

本章的离散结论建立在三个前提上，缺一个就要修正。

第一个是同步采样。所有通道在同一时刻取值，时间戳相同。实际采集卡常常是轮询的，通道之间有微秒级错位，通道数多时不能忽略。

第二个是零阶保持。控制器输出在周期内不变。如果执行器自带局部闭环，实际输出是连续的，$T/2$ 的等效时延就不成立。

第三个是整周期。所有任务在周期边界开始与结束，不跨周期。跨周期的任务会让占用率统计失真，也会让抖动变大。

三个前提在多数 PLC 系统里成立，在 PC 控制与软 PLC 上要逐条核对。核对的方式是看驱动与调度器的说明，不靠猜。

前提不成立时，离散公式仍可用，但要把偏差项加上。加上之后，原本“刚好够”的设计往往会变成“不够”，这正是核对的价值。

91.6 稳定性与性能

91.6.1 边缘节点的三类稳定

边缘节点的“稳定”有三个含义，缺一不可。

第一是实时稳定：每个周期都在截止时间内完成，最坏情况也不能超。判据是最坏执行时间不超过周期的 70%。

第二是内存稳定：长时间运行不泄漏、不碎片化。判据是连续运行 30 天后，可用内存下降不超过 5%。

第三是热稳定：夏天车间 45 ℃ 时不死机、不降频。判据是满负荷连续运行 8 h，CPU 温度不超过器件的结温上限减 20 ℃。

三类稳定里最容易被忽略的是热。实验室 25 ℃ 跑得好好的盒子，装到现场第一个夏天就出问题。选型时要看工作温度范围，不看存储温度范围。

91.6.2 孪生的数值稳定

孪生是不断迭代的状态估计，数值误差会累积。三类误差要防。

第一类是积分漂移。用欧拉法积分，步长 10 ms，一小时累积 36 万步，舍入误差会积累成可见偏差。对策是用梯形法或提高精度。

第二类是代数环。孪生里如果有互相依赖的两个量，迭代可能不收敛。对策是打断环，用上一周期的值。

第三类是参数突变。模型修正一次改太多，状态会跳变。对策是限速，每次修正不超过 1%，这一点在 §91.3.5 已经提过。

数值稳定的判据很朴素：让孪生空跑 24 h，输入恒定，输出漂移不超过满量程的 0.1%。

91.6.3 时延抖动比平均时延更危险

平均时延 8 ms、抖动 1 ms 的系统，比平均 10 ms、抖动 0.1 ms 的系统更难用。

原因是控制环对时延的变化敏感，对固定的时延可以补偿，对变化的时延无法补偿。抖动等效于在环里加了一个随机相位。

抖动的来源有三个：操作系统的调度、网络的排队、缓存的命中率。三者叠加，最坏情况可能是平均值的两到三倍。

判据要看 P99 而不是平均。平均 8 ms、P99 20 ms 的系统，实际按 20 ms 设计。§89 章的测量方法可以直接搬过来。

91.6.4 CPU 占用与抖动的非线性

CPU 占用从 50% 涨到 70%，抖动可能只涨 20%；从 70% 涨到 85%，抖动可能涨三倍。

原因是调度器在低占用时总能找到空闲窗口，高占用时窗口消失，任务开始排队。这个拐点通常在 70% 到 75% 之间。

所以占用率上限定 70% 不是保守，而是刚好在拐点之前。表 91-3 的回路数按 70% 算，留出的 30% 是给抖动与突发的。

超过 70% 的系统不是“稍微紧张”，而是“随时可能超时”。这类系统在实验室跑得好，在现场一加负载就崩。

91.6.5 断网之后怎么办

边缘节点必须能独立运行。断网时的行为要提前定义，不能等断了再想。

三种策略。一是保持：按最后一次下发的参数继续跑，孪生标记为失同步。二是降级：切到本地固定的保守参数，牺牲性能换安全。三是停机：按安全规程停到安全状态。

选哪种取决于工艺。连续过程多用保持或降级，间歇过程可以停机。无论哪种，都要在 HMI 上明确显示当前状态，不能让人误以为还在同步。

断网恢复后不能立刻全量同步。要先比对，再补数据，最后才恢复闭环。顺序错了会把断网期间的错误状态带进孪生。

91.6.6 时钟漂移

两个节点的时钟不同步，时间戳就会错位。普通晶振的漂移是每天几十毫秒，一个月累积到秒级。

对同步间隔 20 ms 的系统，秒级的时钟偏差是致命的。孪生会认为数据来自错误的时间点，偏差计算全错。

对策是用 PTP 或 GPS 统一时钟，精度到微秒级。做不到时，至少每小时校一次，把偏差压到毫秒级。

校时本身也会引入跳变。校时的时候要让时间单调，不能往回跳，否则缓存里的数据会乱序。用“缓慢调整”而不是“直接赋值”。

91.6.7 指标退化的顺序

孪生加边缘系统的指标退化有固定顺序，知道顺序就能提前发现。

最先坏的是同步精度。它受网络、时钟、负载影响，稍有变化就漂。表现是偏差缓慢上升，但不报警。

第二个坏的是边缘的实时性。占用率上升，抖动变大，偶发超时。表现是“偶尔丢一个周期”，日志里能看到。

第三个坏的是保真度。设备磨损、传感器老化，模型与实物慢慢分开。表现是“整定参数越来越不好用”。

最后坏的是数据完整性。磁盘满了、补传失败、文件损坏。表现是“历史查不到”。

四者的检测难度递增。同步精度可以自动监测，实时性要看日志，保真度要定期比对，数据完整性往往出了事才发现。

91.6.8 性能验收怎么做

验收不能只看“跑起来了”。要测四样东西。

第一，最坏执行时间。用示波器或 GPIO 打点，连续测一万个周期，取最大值。不用平均值，不用软件计时。

第二，抖动分布。同样测一万个周期，给出 P50、P99、最大值。P99 是设计依据。

第三，长时稳定。连续跑 72 h，看内存、温度、丢帧数。72 h 覆盖了日周期，但覆盖不了周周期，能跑一周更好。

第四，断网行为。拔网线，看是否按预定策略动作，恢复后是否能正确补数据。

四样都过，才算性能验收完成。只测第一样的项目很常见，后面三样往往在投运后才暴露。

91.6.9 一次抖动排查

有个项目上线三个月后，同步偏差的 P99 从 9 ms 涨到 22 ms，平均值仍是 8 ms，控制没出问题，但孪生的比对曲线开始明显错位。

第一步是分段打点。在采样、入队、计算、输出四个位置各打一个 GPIO，用逻辑分析仪连续抓一万个周期。四段耗时分别记下来，先分清是“哪一段变长了”。

结果显示计算段没变，入队到出队之间变长了，最长的一次等了 11 ms。问题不在算法，在调度。

第二步查同一周期里还跑了什么。日志显示，每小时整点有一次孪生快照写盘，与日志轮转撞在一起，两块写操作把 IO 队列占满。

第三步改。快照改成增量，只写变化的状态量；日志落盘挪到低优先级线程；两件事的时刻错开半分钟。

改完连续测一周，P99 回到 9 ms，最大 12 ms。整个过程没有改控制参数，也没有换硬件。

这个案例的教训是：抖动问题先分段，再定位，最后才改。一上来就换硬件或者缩短周期，往往把问题掩盖而不是解决。

91.6.10 一张性能验收单

验收单要有四行，每行写判据、实测值、测量方法、结论。

第一行最坏执行时间，判据是不超过周期的 70%，实测取一万个周期的最大值，方法用 GPIO 打点。

第二行抖动 P99，判据是不超过周期的一半，实测取同一批数据的分位数，方法同上。

第三行长时稳定，判据是 72 h 内可用内存下降不超过 5%、无丢帧、温度不超过结温减 20 ℃。

第四行断网行为，判据是按预定策略动作、恢复后能正确补数据，方法是拔网线并记录全过程。

四行都要有签名与日期。没有签名的验收单，出了问题没人认。

验收单不要写“基本正常”“偶有超时”这类词。要么给数，要么写“未测”。

91.6.11 温度与寿命

边缘节点的故障里，相当一部分与温度有关，而温度的影响是累积的。

经验规律是温度每升高 10 ℃，电解电容与闪存的寿命减半。柜内 45 ℃ 时，节点的实际寿命只有 25 ℃ 时的四分之一。

选型时看的是工作温度上限，不是存储温度上限。标称 0 到 50 ℃ 的节点，装进夏天 48 ℃ 的柜子，已经在边界上运行。

对策有三条。一是把节点装在柜体下部，避开变频器与电源的热区；二是加装散热片或风扇，成本远低于更换节点；三是把高负载任务挪走，降低自身发热。

三条里最有效的是第三条。§91.8.9 的失败案例里，2 ms 周期让 CPU 满负荷，温度从 55 ℃ 升到 78 ℃，降频又加剧了超时。降负载比加风扇更根本。

温度要监测，不要等死机才发现。节点自带的温度传感器读出来接到运维看板上，超过 60 ℃ 报警，这条线比结温上限留了足够余量。

91.6.12 冗余与切换

重要的边缘节点要做冗余，但冗余的方式决定了切换后的状态是否可信。

冷备是两台机器，一台运行一台断电。切换要上电、加载、建链，通常要几十秒。冷备适合不要求连续运行的场合。

热备是两台都在运行，备机跟着主机同步状态，主机故障时几十毫秒内接管。热备的代价是双倍硬件加一条同步链路。

不管哪种，切换后孪生都要重建同步。主机停机期间的数据在备机里没有，孪生只能从切换时刻重新开始积累。

切换还要防“脑裂”。两台都以为自己是主机，同时对外写数据，后果比停机更糟。防法是加仲裁，比如一台独立的仲裁节点或一把共享锁。

冗余的验收要看切换时间与切换后的状态一致性，两样都要测。只测“能切换”不测“切换后对不对”，等于没测。

91.7 参数意义与作用

91.7.1 参数分三层

本章的参数按时间尺度分三层，调参时要分层看。

毫秒层：控制周期、边缘固定开销、单回路开销、占用率上限。这一层决定“来不来得及算”。

秒层：同步间隔、网络批量、HIL 单例时长。这一层决定“数据新不新”。

天层：数据保留天数、漂移阈值、再训练周期。这一层决定“模型会不会过期”。

三层之间的关系是单向的：毫秒层算不动，秒层的数据再新也没用；秒层不同步，天层的模型再准也白搭。所以调参顺序永远是从快到慢。

91.7.2 保真度等级怎么选

G1 到 G5 不是越高越好，按用途选。

展示与培训用 G1，90.55% 的保真度足够让画面动起来像那么回事。

报警与趋势预测用 G2，95.22%，误报率可以接受。

整定与参数寻优用 G3，98.10%，实物劣化 2.8%，在 20% 的线内。

控制器设计与稳定性分析用 G4，99.04%，因为稳定性对参数敏感。

计量与性能验收用 G5，99.52%，因为要拿它当基准。

多数项目停在 G3。往上走的成本是成倍增加的，因为要把滞后测到 1% 以内，需要专门的辨识试验。

91.7.3 同步间隔的上限

同步间隔由允许偏差反解，式 91-4 已经给出。三个典型值。

位置控制，允许 10 mm，变化率 0.5 m/s，上限 20 ms。

温度控制，允许 0.5 ℃，变化率 0.01 ℃/s，上限 50 s。这时同步间隔可以很宽松。

压力控制，允许 0.01 MPa，变化率 0.001 MPa/s，上限 10 s。

三个例子说明同步间隔的跨度可以从毫秒到几十秒。不要被“实时”这个词带着走，先算需要多快。

91.7.4 边缘周期怎么定

边缘周期由两件事共同决定：控制环需要多快，节点能算多快。

控制环的要求来自第 88 章的结论：周期小于对象时间常数的 2.5% 为佳。时间常数 2 s 的对象，周期 50 ms 就够。

节点的能力来自式 91-6。回路数 16 时需要 10 ms 周期，回路数 36 时需要 20 ms。

两者取严的那个。控制要求 50 ms、节点能力 10 ms，就取 10 ms；控制要求 5 ms、节点能力 20 ms，就要换节点。

91.7.5 带宽决定分层

ω_c 是分层的决定性参数，不是“重要程度”。

ω_c 小于 5 rad/s 的过程，云侧时延的相位损失小于 31°，多数场合能用。ω_c 大于 20 rad/s 的过程，云侧基本不可用，必须留在边缘。

中间地带（5 到 20 rad/s）要看裕度。原本裕度 60° 的系统，损失 15° 还能接受；原本裕度 35° 的系统，损失 15° 就危险了。

这条判据也解释了为什么“云控制”在慢过程（暖通、水处理、农业）能落地，在快过程（伺服、机器人、电力电子）落不了地。

91.7.6 检出率不是越高越好

MIL 的检出率 55%，看着不高，但它的单位成本只有 1。把 MIL 的检出率从 55% 提到 70%，需要更精细的模型，成本可能翻三倍。

这时更划算的做法是把力气放到 SIL 上，因为 SIL 测的是实现，而实现缺陷在 MIL 里根本测不到。

三级串联的 98.4% 已经很高。再往上提，每提高 1 个百分点要多花几倍成本，而剩下的 1.6% 里有相当一部分只有在现场才能暴露。

合理的做法是接受 98.4%，把这 1.6% 的风险用现场快速回滚兜住。这与第 90 章“回滚速度比失败率更重要”的结论一致。

91.7.7 数据保留多久

保留天数由三件事决定：故障追溯需要多久、磁盘有多大、合规要求是什么。

故障追溯通常要 30 天，因为偶发故障的平均复现周期是两周到一个月。少于 30 天，出了事查不到。

磁盘按式 91-11 算。11.06 GB/天，30 天 331.8 GB，加 30% 余量约 430 GB。这个容量用一块工业级 SSD 就能满足。

合规要求因行业而异，医药与食品可能要求更长。这时不要加磁盘，而是降采样后长期保留，原始数据仍只留 30 天。

91.7.8 漂移阈值定多少

阈值定得低，发现早但误报多；定得高，误报少但发现晚。5% 是常见折中。

定阈值要考虑测量噪声。噪声标准差 1%，阈值 5% 对应 5 倍标准差，误报率很低；噪声 3%，阈值 5% 只有 1.7 倍标准差，误报会很多。

这时应该先降噪声，而不是提高阈值。降噪声的办法是多次平均或滤波，代价是响应变慢。

阈值还要与检修周期匹配。一年检修一次的设备，阈值要定到能在一年内报警，即漂移速率乘以 12 个月要超过阈值。

91.7.9 四个换算

工程上经常要在四个量之间换算，记住换算关系能省很多时间。

保真度与偏差：$F = 1 - e$，98.1% 对应 1.90%。

同步间隔与偏差：$e = vh$，20 ms 对应 10 mm（0.5 m/s）。

时延与相位：$\varphi = \arctan(\omega_c T_d)$，8 ms 对 40 rad/s 是 17.74°。

数据量与带宽：$Q = N f_s b$，32 通道 1 kHz 是 11.06 GB/天，1% 上云是 10.24 kbit/s。

四个换算覆盖了本章八成以上的估算。记不住公式时，记住这四个典型值也能快速判断量级。

91.7.10 三组互相咬住的参数

参数不是一个个独立定的，本章有三组参数会互相咬住，改一个必须复核另外两个。

第一组是周期、回路数、占用率。周期缩短一半，占用率大致翻倍，可支持的回路数下降。表 91-3 的五档就是这条关系的全部。

第二组是同步间隔、带宽、缓存。间隔缩短一半，带宽翻倍，同样断网时长需要的缓存帧数也翻倍。三者是同一条链上的三个刻度。

第三组是保留天数、磁盘容量、合规要求。保留天数加倍，磁盘加倍；合规要求更长时，只能靠降采样，不能靠加磁盘。

咬住的含义是：任何一次变更都要做一次“连带检查”。改了周期不复核回路数，改了同步不复核缓存，都是常见的返工原因。

检查的方法很简单，把三个参数写在一行，问一句“另外两个还成立吗”。三十秒的事，能省一周的返工。

91.7.11 参数的敏感度不一样

本章的参数对结果的影响差得很远，调参时应该先动敏感的那个。

最敏感的是边缘周期。它一动，占用率、回路数、抖动、散热全部跟着变，§91.8.9 的失败案例就是动它动出的问题。

其次是同步间隔。它直接决定状态偏差，而且是线性的，改多少就变多少，没有缓冲。

再次是保真度。它影响的是整定质量，变化是渐进的，从 G2 到 G3 只改善 14.6%，不会一夜之间失效。

最不敏感的是数据保留天数。它只影响磁盘，除非磁盘写满，否则不影响控制。

按敏感度排序，调参的顺序应该是：先定不敏感的（保留、带宽），再定中等的（保真度），最后动敏感的（同步、周期）。

91.7.12 有些参数不能连续调

工程上很多参数只能取离散档位，按连续量去调会得到无法配置的值。

控制周期只能取 1、2、4、5、10、20 ms 这几档，因为底层定时器的分频是整数。写 7 ms 进去，实际跑出来可能是 8 ms。

同步间隔通常也受周期约束，必须是控制周期的整数倍。写 25 ms，在 10 ms 周期下会变成 30 ms 或 20 ms，取决于实现。

台架数、通道数、班次数都是整数，没有 2.5 台、33.3 通道、1.5 班。

调参时先把参数分成“连续”与“离散”两类。离散的先按档位选，连续的再细调。把离散参数当连续调，是配置对不上现象的常见原因。

91.7.13 默认值从哪里来

新项目起步时，多数参数可以先取默认值，不必从零讨论。

孪生等级默认 G2，用于报警与趋势；要整定就升到 G3。同步间隔默认 20 ms，覆盖多数位置与速度场合。边缘周期默认 10 ms，能放 16 个回路，占用 15.5%，留足余量。

数据保留默认 30 天，磁盘按 331.8 GB 乘 1.3 备。上云比例默认 1%，带宽 10.24 kbit/s。漂移阈值默认 5%，对应 2.5 个月发现 2%/月 的漂移。

HIL 默认 400 例、3 台架，工期 2 个工作日。这个配置在多数中型项目里够用，且已经过了收益拐点。

默认值的意义是给讨论一个起点，不是给结论。每一项都要按项目实际复核，但复核的成本远低于从零推导。

复核的方法就是表 91-10 的“依据”列。默认值填进去，依据列写“默认，待复核”，评审时逐项确认。

91.7.14 参数要有一张登记表

参数散落在配置工具、程序常量、文档正文三处，很快就对不上。要有一张登记表，一处维护。

表里每行写六项：名称、单位、取值、依据、责任人、复核日期。六项缺一项，这个参数就处于“没人负责”的状态。

名称要与代码里的常量名一致。代码叫 `EDGE_FIXED_MS`，表里也写这个名字，不要写成“边缘固定开销”，否则对不上号。

依据列写表号或公式号，不写“经验”。写不出来的，说明这个值是拍的，拍的值要标注出来，评审时重点看。

责任人不是挂名，是改这个值之前必须问他的人。复核日期到了要重新确认，确认的方式是重跑一次相关计算或测量。

登记表放在版本库里，与代码同仓。参数改动与代码改动一起提交，历史可追溯。

91.8 调参方法与经验

91.8.1 四个对象与顺序

要调的有四个对象：分层、保真度、同步、边缘预算。

顺序是先分层，再保真度，再同步，最后边缘预算。

先分层是因为它决定后面所有参数的量级。放在云上的系统，同步间隔可以是秒级；放在边缘的，同步间隔必须是毫秒级。

再保真度是因为它决定“要不要花时间辨识”。G2 够用的场合，不必做专门的辨识试验。

再同步是因为它依赖保真度。孪生不准时，同步做得再快也没有意义。

最后边缘预算是因为它受前三个约束。前三个定了，边缘的周期与回路数才有明确的输入。

91.8.2 先压时延再提保真度

时延与保真度都会影响闭环性能，但时延的影响更直接。

时延是“迟到的信息”，无论模型多准，迟到就是迟到。保真度是“偏差的信息”，可以通过反馈部分补偿。

所以优化顺序是先压时延，再提保真度。把云上的环搬到边缘，时延从 120 ms 降到 8 ms，相位损失从 78.23° 降到 17.74°，这一步的收益远大于把保真度从 G2 提到 G4。

反过来做，先把保真度提到 G5，时延仍是 120 ms，闭环照样不能用。

91.8.3 同步从粗到细

同步间隔不要一上来就定 5 ms。先按允许偏差算上限，再取上限的 1/2 到 1/4。

按 10 mm、0.5 m/s 算，上限 20 ms，实际取 5 到 10 ms。留余量是因为变化率会超预期，也因为网络抖动会让实际间隔大于设定值。

跑一周后看实际偏差分布。如果 P99 偏差远小于允许值，可以放宽间隔，省带宽与算力；如果接近允许值，要收紧。

同步间隔的调整比保真度容易，因为它不改模型，只改配置。所以可以多调几次。

91.8.4 边缘周期按阶梯调

边缘周期不能连续调，只能按档位调：2、4、5、10、20 ms。每一档对应一个回路数上限，见表 91-3。

调的时候从满足控制要求的最长周期开始，逐档往下试，直到占用率进入 50% 到 70% 的区间。

不要一上来就用最短周期。2 ms 周期的占用率 77.5%，连一个回路都放不下，白白浪费算力。

也不要长期停在 30% 以下。占用率太低说明周期选得过长，控制性能没有充分利用硬件。

91.8.5 在环测试按缺陷类型配比

三级测试的用例数不该平均分配，要按缺陷类型配。

算法逻辑缺陷用 MIL 测，成本最低；接口与时序缺陷用 SIL 测；驱动与硬件缺陷用 HIL 测。

如果历史数据显示七成缺陷是接口问题，就该把 SIL 的用例数提上去，而不是平均分。

配比不是固定的。新项目算法改动大，MIL 多；老项目改驱动，HIL 多。每个项目启动时看一眼前三个月的缺陷分布再定。

91.8.6 漂移阈值跟着检修周期走

阈值不是拍脑袋定的，要跟检修周期匹配。

一年检修一次，阈值要保证一年内能报警，即漂移速率乘 12 超过阈值。2%/月的漂移，阈值定 5% 会在 2.5 个月报警，够用。

三年大修一次的设备，阈值要更敏感，否则两次检修之间模型已经偏得不能用。

阈值还要跟着测量噪声调。噪声大的场合，阈值要提高，同时用更长的平均窗口降噪。

91.8.7 一张调参记录表

调参要留记录，否则三个月后没人记得为什么这么定。表 91-11 是记录格式。

表 91-11 调参记录示例

日期	改什么	改前	改后	观测指标	结论
第 1 周	同步间隔	50 ms	20 ms	偏差 25→10 mm	保留
第 3 周	边缘周期	5 ms	10 ms	占用 31%→15.5%	保留
第 5 周	保真度	G2	G3	调节 1.30→1.11 s	保留
第 7 周	漂移阈值	8%	5%	误报 3 次/月	回退到 6%

表 91-11 最后一行很重要。不是所有调整都成功，记录失败同样有价值，它防止后来的人再试一遍。

记录里的“观测指标”要具体。写“效果变好”没有意义，写“调节 1.30→1.11 s”才能复核。

91.8.8 五条经验

第一条，先算后调。表 91-3 与表 91-7 都能手算，算完再动手，比反复试快。

第二条，一次只改一个。同步与保真度同时改，出了问题说不清是哪个。

第三条，改完跑够时间。同步改完至少跑 24 h，漂移相关至少跑一个月。

第四条，留反证。每次优化都问“有没有哪个指标变差了”，写进记录。

第五条，记录比参数重要。参数会过时，记录能解释为什么。

91.8.9 一次失败的调参

有个项目为了“提高实时性”，把边缘周期从 10 ms 改成 2 ms，同时把回路数从 12 增到 16。

结果运行一周后开始偶发超时。查下来是占用率算错了：12 个回路在 10 ms 下占用 57%，改成 2 ms 后固定开销占比从 12% 涨到 60%，16 个回路根本放不下。

第二个问题是热。2 ms 周期让 CPU 一直满负荷，盒子温度从 55 ℃ 升到 78 ℃，开始降频，降频又让超时更频繁。

回退到 10 ms 后一切正常。复盘时的结论是：周期不是越短越好，它要与回路数、散热能力一起算。

这次调参的代价是两周的现场排查，以及一次不必要的固件回滚。如果事先用式 91-6 算一遍，五分钟就能发现不可行。

91.8.10 从现象反查参数

现场给的是现象，不是参数。把现象翻译成参数，是调参的第一步。

孪生曲线整体滞后于实测。这是同步问题，不是模型问题。查同步间隔与时间戳，别急着改模型参数。

孪生曲线超前于实测。多半是时间戳打在了数据发出之前，或者两端用了不同的时钟源。查时钟，不查模型。

孪生与实测在稳态差一个固定比例。这是增益偏差，查量程与标定系数，这是最容易修的一类。

孪生与实测在过渡段形状不同。这是时间常数偏差，查对象的动态辨识，比调增益麻烦。

孪生在阶跃起始处差一截。这是滞后偏差，查纯滞后时间，尤其是传输与阀门行程带来的那部分。

五类现象对应五个参数，按这个顺序查，比漫无目的地试快得多。

还有一类现象不属于上面任何一种：**李生偶尔跳变一下再回来**。这通常是数据链的问题，丢帧后被插值补上，不要当成模型问题去改参数。

91.8.11 什么时候停止调参

调参要有停止条件，否则会一直调到项目结束。

第一个条件，指标进入目标区间并稳定两周。稳定两周是必须的，一周可能只是还没赶上工况变化。

第二个条件，边际改善小于测量不确定度。再调一次只改善 0.5%，而测量本身的重复性就有 1%，这时继续调没有意义。

第三个条件，改动开始影响其他指标。同步调好了，抖动上去了；占用率降下来了，控制变慢了。出现这类交换，说明已经到边界。

三个条件满足任何一个，就该停手，把当前参数固化进基线表，写清理由。

停止调参不等于停止观察。参数固化之后，运维仍要每月看四个数，发现漂移再启动一轮调参。

91.8.12 一次成功的调参

另一个项目的目标是把李生从 G1 提到 G3，用它来整定六个温度回路。

第一步先测现状。用阶跃试验测了三个工作点，增益偏差 12%，时间常数偏差 18%，滞后偏差 25%，确认是 G1 水平。

第二步定优先级。按敏感度，先校增益，再校时间常数，最后校滞后。增益用稳态数据就能校，不需要专门试验，一天完成。

第三步校时间常数。取三次阶跃的 63.2% 时间，取中位数，把偏差从 18% 压到 6%。

第四步校滞后。这一项最费事，要用阀门的行程信号与温度响应做互相关，做了三天，偏差压到 5% 以内。

最后复测，三项偏差分别是 2%、5%、5%，保真度 98.10%，达到 G3。用新参数整定六个回路，实物的调节时间从 1.9 s 降到 1.2 s。

整个过程两周，其中一半时间花在滞后上。复盘时的结论是：如果只做到 G2，一周就够，但整定质量会差一截，六个回路每年多耗的调试时间远超这一周。

91.8.13 调参的两个禁忌

第一个禁忌是一次改多个参数。同步间隔与保真度一起改，指标变好时说不清是哪个起作用，变差时也说不清该回退哪个。

第二个禁忌是在工况不稳时调参。负荷在变、原料在变、环境温度在变，测出来的差异分不清是参数改了还是工况变了。

两个禁忌背后是同一个道理：调参是一次受控实验，必须只有一个变量在动。

违反禁忌的代价通常在两周后才显现：指标看着好了，过一阵又坏了，查不出原因，只能全部回退重来。

规避的办法很土：一次只改一项，改完等稳定，记录，再改下一项。慢，但可追溯。

91.9 优化与改进

91.9.1 四个方向

系统跑起来之后，优化有四个方向，按收益排序。

第一，把实时环从云搬到边缘。收益最大，时延降一个数量级。

第二，把原始数据换成特征上云。带宽降两个数量级，见 §91.7.7。

第三，提高同步效率。用增量同步代替全量，带宽与算力都降。

第四，缩短 HIL 单例时长。用自动化脚本替代人工操作，工期直接按比例下降。

四个方向的共同点是“改结构”而不是“调参数”。调参数的收益通常只有百分之几，改结构能到一个数量级。

91.9.2 特征上云而不是原始数据

原始数据上云的带宽是 10.24 kbit/s（1% 抽样），全量是 1.02 Mbit/s。特征上云的带宽通常是几 kbit/s。

特征怎么提，取决于云上要做什么。做趋势分析，提均值、方差、斜率；做故障诊断，提频谱特征；做模型训练，提一段时间的统计量。

特征提取放在边缘，代价是边缘要多花算力。按每通道 0.02 ms 估算，32 通道 0.64 ms，在 10 ms 周期里占 6.4%，可以接受。

代价是信息损失。特征无法还原原始波形，出了事只能看边缘存的历史。所以原始数据仍要在边缘保留 30 天。

91.9.3 增量同步

全量同步每周期传全部状态，增量同步只传变化量。

状态量 200 个，每个 4 B，全量 800 B；变化量通常不到 10%，即 80 B。带宽降 90%。

增量的代价是要维护“基线”。基线错了，增量就全错。所以每隔一段时间要做一次全量校验，比如每小时一次。

增量还要处理“新增”与“删除”。状态量变化时，增量协议要能表达“这个量不存在了”，否则对端会保留旧值。

91.9.4 边缘分核

一个盒子多个核，把实时任务与通信任务分到不同核，能显著降低抖动。

分核的原则是：硬实时任务独占一个核，通信与日志放另一个核，孪生计算放第三个核。

分核之后要关掉核间的缓存共享，避免一个核的负载影响另一个核的缓存命中。这一步在 Linux 上用 isolcpus 与 cache 隔离实现。

分核的收益是抖动下降，通常从毫秒级降到百微秒级。代价是核数固定，灵活性下降。

91.9.5 HIL 并行与自动化

HIL 的工期由轮数与单例时长决定。缩短工期有两条路：加台架、缩短单例。

加台架的收益在第三台之后归零（表 91-7），所以更快见效的是缩短单例。

单例 3 min 里，真正跑用例的时间可能只有 40 s，其余是准备、刷写、等待。用脚本自动化这部分，单例能压到 1 min，工期直接降到三分之一。

自动化的投入是一次性的，收益是每次回归都省。按每周一次回归算，三个月就能回本。

91.9.6 保真度的边际收益

从 G2 到 G3，调节时间改善 14.6%；从 G3 到 G4，改善 1.8%；从 G4 到 G5，改善 0.9%。

边际收益递减得很快。所以优化的钱应该花在 G2 到 G3 这一段，而不是追求 G5。

G2 到 G3 要做什么：把增益标定到 2%、时间常数标定到 5%、滞后测到 5%。这三件事用常规的阶跃试验就能做到，成本约一周。

G3 到 G4 要把滞后测到 2%，需要专门的辨识试验，成本约一个月。收益只有 1.8%，不划算。

91.9.7 三条底线

优化不能突破三条底线。

第一条，实时性不能降。优化通信、优化存储都不能占用控制周期的预算。占用率上限 70% 不能破。

第二条，数据不能丢。优化带宽可以用抽样，但抽样策略要可配置，出事时能临时提高比例。

第三条，回滚要留着。任何优化都要留一条回退路径，且回退时间不超过十分钟。

三条底线的共同点是“优化不能以减少证据为代价”。省下来的带宽、算力、时间，不能靠“少记一点”来实现。

91.9.8 怎么证明优化有效

三个条件：工况相同、口径相同、重复多次。

工况相同指优化前后的负载、环境、参数一致。加台架做对比时，两台架的固件版本要一致。

口径相同指测量的量一致。优化前测 P99 抖动，优化后也测 P99，不能换成平均值。

重复多次指至少跑三次。单次结果受偶然因素影响大，三次取中位数比较稳。

满足三条后，收益要落到数字上。不要写“快了很多”，要写“时延从 120 ms 降到 8 ms，相位损失从 78.23° 降到 17.74°，闭环从不可用变为可用”。

最后留一条反证：这次优化有没有让某个指标变差。分核通常会让单核性能下降，抽样通常会让故障定位变难。把代价写出来，方案才完整。

91.9.9 四条路线的账

四条优化路线都能算账，算完再决定先做哪条。

第一条，把实时环从云搬到边缘。时延从 120 ms 降到 8 ms，相位损失从 78.23° 降到 17.74°。收益不是“快了一点”，而是“从不可用变为可用”。代价是一台边缘节点与一次架构改造。

第二条，把原始数据换成特征上云。全量上云要 1.02 Mbit/s，1% 抽样是 10.24 kbit/s，特征只有几 kbit/s。代价是边缘每周期的计算增加，32 通道按每通道 0.02 ms 算是 0.64 ms，在 10 ms 周期里占 6.4%。

第三条，增量同步。状态量按 200 个、每个 4 B 算，全量 800 B，增量约 80 B，带宽降 90%。代价是协议复杂度与每小时一次的全量校验。

第四条，HIL 自动化。单例从 3 min 压到 1 min，工期降到三分之一。代价是一次性的脚本开发与台架改造。

四条里前两条是“改结构”，收益按数量级算；后两条是“提效率”，收益按百分比算。预算有限时，先做前两条。

91.9.10 优化有依赖顺序

四条路线不能随便排，它们之间有依赖。

搬环必须最先做。它决定了时延的上限，也就决定了同步间隔能取多大。环还在云上时把同步做到 5 ms，是给一条 120 ms 的链路做无用功。

特征上云排在第二。它决定了带宽预算，而带宽预算又决定了增量同步是否还有必要。特征已经把带宽压到几 kbit/s 时，增量同步的收益就很小了。

增量同步排在第三。它依赖一个稳定的全量基线，而基线要等特征定义稳定之后才能定。

HIL 自动化排最后。它依赖用例集稳定，用例集又要等前面三项的结构定下来才不再变。

顺序错了的典型表现是：优化做完了，前面的结构一改，全部推倒重来。

91.9.11 三个优化反例

第一个反例，只做增量同步，不做全量校验。运行两个月后，孪生与实物的偏差缓慢上升，查下来是基线在某个版本升级时错了，增量一直在一个错误的基准上加。

第二个反例，全量换成特征，原始数据只留三天。第三周出了一次偶发超调，特征里看不出原因，原始波形已经覆盖，只能等下次复现。

第三个反例，没算核数就分核。四个任务分到两个核，实时任务与通信任务仍然共享一个核，抖动一点没降，反而因为核间通信多了 0.3 ms。

三个反例的共同点是：优化只做了一半。一半的优化比不优化更危险，因为它让人以为问题已经解决。

91.9.12 优化之后要更新基线

每次优化完成，都要回头改表 91-10。基线不改，下一次评审就会拿旧基线对照新系统，得出“性能下降”的错误结论。

要改的通常不止一行。搬环之后，时延与同步两行都变；特征上云之后，上云比例与带宽两行都变。

改基线要留版本。表头写版本号与生效日期，旧版本归档，不要直接覆盖。出了问题要能查到“当时按哪版基线验收”。

基线变更要通知到所有依赖它的文档：验收单、运维手册、交接材料。漏掉一处，就会出现两份互相矛盾的指标。

91.9.13 优化的收益怎么记

收益要记成“可复核的事实”，不要记成感受。

推荐记法是一行一句：优化项、优化前、优化后、测量方法、观测周期。五行齐了，别人才敢引用。

比如：“把实时环从云搬到边缘；时延 120 ms 变 8 ms；相位损失 78.23° 变 17.74°；用 GPIO 打点测一万个周期；观测两周。”

收益的记录要包含代价。同一行后面加一句“代价：新增一台边缘节点，运维多一个节点”。只记收益的记录，下次评审会被质疑。

记录攒够五条以上，就能看出哪类优化在本项目里反复有效。这比任何方法论都可靠。

91.9.14 优化不能绕过变更流程

优化也是一次变更，要按第 90 章的流程走：评审、灰度、观察、回滚预案。

常见的绕过方式是“只改一个配置项，不用走流程”。而本章的配置项恰恰最敏感，周期与同步间隔一改，占用率与偏差全变。

灰度要按节点分批。先一台，观察 24 h，再 10%，再全部。观测指标就是 §91.10.6 的四个：超时、占用率、温度、同步偏差。

回滚预案要在优化前写好，不能等出问题再想。回滚时间控制在十分钟以内，回滚包提前在一台样机上验证过。

优化与变更流程不冲突。流程不会拖慢优化，它只是把“改完才知道不行”变成“改之前就知道风险”。

91.10 工程落地

91.10.1 按场景选形态

不是所有项目都需要完整的孪生加边缘。按场景分四类。

第一类，单机设备。一个控制器加一块触摸屏，没有云、没有边。孪生的价值有限，做好参数备份与故障记录就够。

第二类，产线。多个设备联网，一个边缘节点做协调，云上做报表。孪生用于整定与报警，保真度 G2 到 G3。

第三类，多站点。各站点有边缘节点，云上做集中运维。孪生用于远程诊断，同步间隔可以到秒级。

第四类，安全相关。孪生不参与控制，只做辅助决策，且必须与安全回路物理隔离。

四类的投入差一个数量级。选错形态，要么过度设计，要么根本不够用。

91.10.2 孪生的最小可用版

最小可用的孪生只要三样东西：一个能算的模型、一条能同步的数据链、一个能比对的界面。

模型可以是一阶加滞后，不必是机理模型。数据链可以是 1 s 同步，不必是 20 ms。界面上只要有实测与预测两条曲线，就够判断孪生是否可信。

先做这三样，跑两周，看偏差是否稳定。稳定了再往上加精度与频率，不稳定先查数据链。

很多项目一上来就做三维、做机理、做大数据平台，做了半年还没有一条能比的曲线。这是把顺序做反了。

91.10.3 边缘节点上电清单

边缘节点到现场，按清单逐项确认，别跳。

第一，供电。电压范围、功率余量、UPS 续航。断电重启后能否自动恢复。

第二，网络。IP 规划、VLAN、防火墙规则、断网行为。至少要有一个本地维护口。

第三，时间。NTP 或 PTP 是否配好，断网时能否本地守时，恢复后能否平滑校正。

第四，散热。柜内温度、风道、滤网。夏天最热的那一周要专门测一次。

第五，存储。磁盘容量、写满策略、日志轮转。写满后是覆盖还是停止，要提前定。

第六，看门狗。硬件看门狗是否启用，喂狗周期与超时时间是否匹配。

六项里最容易漏的是第五项。磁盘写满后系统行为不确定，有的覆盖，有的只读，有的直接崩溃。

91.10.4 时间同步怎么做

时间同步是孪生的地基，做不好后面全歪。

优先级是 GPS 大于 PTP 大于 NTP。有 GPS 就用 GPS，精度到微秒；没有就 PTP，精度到亚毫秒；都不行才用 NTP，精度到毫秒。

多站点场景，各站点独立守时，靠云侧校准。校准周期按晶振精度定，普通晶振每小时一次，温补晶振每天一次。

断网时进入守时模式，用本地的晶振继续走。守时精度会随时间下降，24 h 内通常还在毫秒级。

恢复联网后不要直接跳时间，要用“缓慢调整”，每秒调整不超过 1 ms，避免数据乱序。

91.10.5 在环测试的现场流程

HIL 测试要有固定流程，否则每次都在重复踩坑。

第一步，版本确认。被测固件、台架配置、用例集的版本三者要对应，写进测试记录。

第二步，上电自检。传感器仿真器、执行器负载、通信链路逐项确认，自检不过不开始。

第三步，基线测试。先跑一组已知结果的用例，确认台架本身没问题。

第四步，正式测试。按用例集跑，失败的用例要留原始数据。

第五步，清场。恢复台架到初始状态，导出日志，归档结果。

五步里第三步最容易省。省了的后果是“测试失败但分不清是固件问题还是台架问题”，排查时间翻倍。

这五步是在环测试内部的流程，放到整条验证链上看，HIL 只是四格中的第三格。图 91-10 把梯次画出来：最左是纯机理模型（MIL），往右一格加上生成代码（SIL），再加被测控制器（HIL），最后一格接上整套装置做现场试运行。

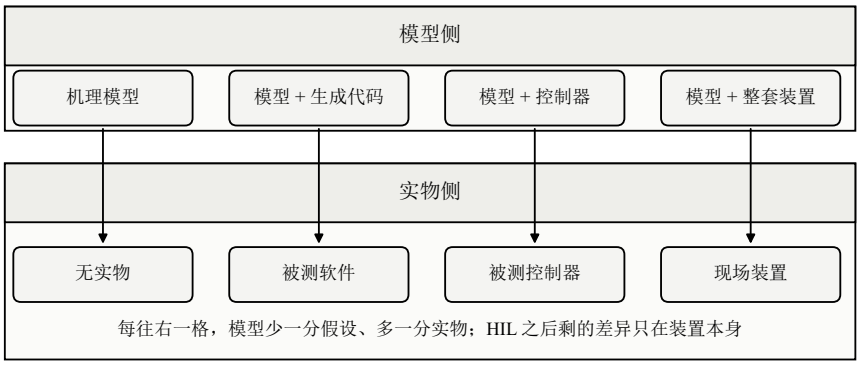


图 91-10　从模型到实物的四阶段在环梯次

每往右一格，模型少一分假设、多一分实物。MIL 里能发现的是控制律本身的问题——增益取错、限幅漏了、积分饱和没处理；SIL 里能发现的是生成代码与模型的不一致；HIL 里能发现的是控制器时序、I/O 量程与通讯延迟；到了现场试运行，剩下的差异只在装置本身，比如阀门非线性、管道充填、机械间隙。

梯次的意义在于把缺陷拦在便宜的那一格。同一个逻辑错误，MIL 阶段改一行参数，现场阶段要停线、改程序、重新验证。反过来，跳过某一格也不会让问题消失，只会让它在更贵的阶段暴露。

四格不是每格都要走满。改动只涉及参数时，跑 MIL 加一组回归即可；改动涉及生成代码，必须补 SIL；改动涉及 I/O 配置或控制器型号，必须补 HIL。判断标准是“改动碰了哪一层”，而不是“上回走了几格”。

91.10.6 发布与回滚

边缘节点数量多，发布要分批。按 §90 的灰度思路，先 1 台、再 10%、再全部。

每批之间至少观察 24 h。观察期内看四个指标：超时次数、占用率、温度、同步偏差。

回滚包要提前准备好，且在一台样机上验证过。回滚时间控制在十分钟以内。

远程升级要有“双分区”。新固件写入备用分区，校验通过才切换；切换失败自动回退到原分区。

没有双分区的节点，升级期间断电就变砖。这类事故在现场并不罕见。

91.10.7 运维看四个数

投运后每月看四个数。

第一个，同步偏差的 P99。超过允许值的一半就要查。

第二个，边缘节点占用率的 P99。超过 70% 就要减回路或加周期。

第三个，孪生与实物的偏差趋势。连续三个月上升，说明模型在漂。

第四个，数据完整率。按天统计应收与实收的帧数，低于 99.9% 要查。

四个数都看趋势。单次超标可能是偶发，连续两个月上升才是问题。

91.10.8 交接三样东西

交接要交三样：能跑的环境、能读的图、能查的历史。

能跑的环境指一条命令能启动孪生与边缘服务。孪生依赖的模型文件、配置文件、数据样本要一起打包。

能读的图指图 91-1 那张分层图，加上每个节点的位置与地址。图要标清边界，谁负责哪一段。

能查的历史指调参记录（表 91-11）、故障复盘、决策记录。三样缺一样，接手人都要重走弯路。

验收标准是：新人独立完成一次“改同步间隔并验证”，不需要问人。

91.10.9 一个反面案例

某项目做数字孪生，投入八个月，做了三维模型、机理模型、大数据平台。验收时演示很漂亮，但问“用它定过哪个参数”，答不上来。

查下来的问题是同步。孪生的数据来自历史库，延迟 30 s；实物的数据是实时的。两者根本不在一个时间轴上，偏差计算没有意义。

第二个问题是保真度没测。模型参数是设计值，没有用实测数据校过。实际偏差 15%，属于 G1 水平。

第三个问题是没有人用。工程师习惯了看趋势图调参，孪生的界面太复杂，没人愿意学。

复盘时的结论是：孪生的价值在“能被用来做决定”，不在“看起来像”。三个问题里任何一个不解决，投入都归零。

重做的顺序是：先做一条能比的曲线，再校参数到 G2，再简化界面到“一看就懂”。三个月做出了能用的版本，成本是原来的三分之一。

91.10.10 三个月路线图

第一个月，打通数据链。目标是一张图上同时有实测与预测两条曲线，时间轴对齐。

第二个月，校保真度。目标是 G2，用阶跃试验校增益与时间常数，用实测滞后校延迟。

第三个月，做应用。目标是三个具体用途：整定参数、报警阈值、故障回溯。每个用途要有一次真实使用记录。

三个月之后进入维护：每月看四个数，每季度校一次保真度，每年做一次全量辨识。

路线图不要压缩。压缩的结果通常是数据链没打通就上应用，最后应用全是摆设。

91.10.11 十五分钟的周会

每周十五分钟，三个问题。

第一个，同步偏差这周有没有超。看 P99，超了追原因。

第二个，边缘节点有没有超时。看日志，超过三次就要查。

第三个，孪生这周有没有被用过。没被用过，说明它没有解决实际问题。

三个问题之外不讨论技术方案。要讨论的另开会。

连续四周第三个问题都是“没用过”，就要重新想用途。孪生不是越做越有用，而是越用越有用。

91.10.12 投运第一周做什么

投运第一周的目标不是“证明系统好用”，而是“把隐患暴露出来”。这一周的动作要围绕这个目标安排。

第一天只看数据链。每条链路跑通没有，时间戳对不对，丢帧率多少。数据链不通，后面全是空谈。

第二天看边缘节点的真实负载。实验室的负载与现场差得远，现场的 IO、协议转换、报警推送都会占时间。占用率超过 70% 就要当场处理。

第三天做一次人为断网。拔网线十分钟，看策略是否按预期动作，恢复后补数据是否完整。这一项必须在投运初期做，不能等出事。

第四天与第五天做比对。让孪生与实物在同一工况下跑满两天，看偏差曲线的形状，不看单个数值。

周末做一次复盘，把四个数的基线记下来。基线要记 P99 与最大值，不记平均值。

第一周不做优化。优化需要稳定的基线，第一周的基线还不稳定。

91.10.13 验收会怎么开

验收会不是汇报会，要按证据过。

第一项，基线表。表 91-10 的十行逐行念，每行的依据要能指到表号或公式号。指不到的，当场标“未通过”。

第二项，性能验收单。§91.6.10 的四行，逐行看实测值与测量方法。没有 GPIO 打点的，视为未测。

第三项，在环测试记录。MIL、SIL、HIL 各跑了多少例，失败多少，失败的用例有没有留原始数据。

第四项，断网演练记录。时间、时长、策略、恢复结果，四样齐全。

第五项，交接三样。环境能启动、图能读、历史能查，由接手的工程师现场演示。

五项过完才签字。任何一项用“基本”“差不多”描述，都不算过。

会议记录要写清“未通过项”与“补齐期限”。没有这一栏的验收会，等于没开。

91.10.14 交接之后的三个月

交接签字不代表项目结束，接下来三个月才是真正的考验。

第一个月，接手的团队独立操作，原团队只答问题不动手。这一月暴露的是文档缺口，不是技术缺口。

第二个月，原团队撤出，只保留远程支持。这一月暴露的是应急能力缺口，尤其是断网与回滚。

第三个月，做一次完整的季度体检。校一次保真度，盘一次数据完整率，走一次回滚演练。

三个月都过，才算真正交接完成。很多项目在签字那天就撤人，结果第一个故障就回到原点。

交接的验收标准不是“文档齐了”，而是“新人能独立处理一次故障”。

为此可以设一次演练：由新人主导，模拟一次同步偏差超限，从告警、定位、处置到复盘走完整流程。原团队只在旁边看，不提示。

演练暴露的问题通常不在技术，而在权限与信息：谁能改配置、日志在哪、上一个版本怎么回滚。这些信息写在文档里，演练一次才记得住。

91.11 优缺点与适用边界

91.11.1 四种架构对比

表 91-12 把四种常见架构放在一起比。

表 91-12 四种部署架构的对比

维度	纯端侧	端 + 边缘	端 + 边缘 + 云	纯云
闭环时延	微秒级	毫秒级	毫秒到秒	百毫秒级
断网可用	是	是	边缘可用	否
算力上限	低	中	高	很高
运维复杂度	低	中	高	中
数据合规	最好	好	需设计	难
适合场景	单机、快伺服	产线、设备群	多站点、集团	慢过程分析

四种架构没有绝对优劣。纯云在数据合规上最难，但在算力上最强；纯端侧最简单，但没法做跨设备优化。

选型的顺序是：先看闭环时延要求，再看断网要求，最后看算力需求。三步下来基本只剩一个选项。

91.11.2 重孪生与轻孪生

重孪生是机理模型、三维、大数据全套；轻孪生是一个低阶模型加一条数据链。

重孪生的优势是解释性强，能回答“为什么”；劣势是成本高、维护难，模型改一次要重新校一遍。

轻孪生的优势是便宜、快、好维护；劣势是外推能力差，工况一变就不准。

多数项目应该从轻孪生开始。轻孪生解决不了的问题，再考虑重孪生。

判断标准是：如果问题能用“偏差趋势”回答，轻孪生就够；如果必须回答“哪个环节导致”，才需要重孪生。

91.11.3 边缘与云端的取舍

边缘的算力有限，云的时延太大。取舍的判据是 $\omega_c T_d$。

$\omega_c T_d$ 小于 0.2 的，可以放云；大于 1 的，必须放边缘；中间的看裕度。

还有一条现实约束：数据能不能出厂。不能出厂的，云再强也用不上。

第三条约束是运维能力。云侧的容器编排、监控、备份需要专门的人，小团队往往养不起。

三条约束叠在一起，多数工业项目落在“边缘做实时、云做分析”这一档。

91.11.4 在环与现场的取舍

在环测试能覆盖大部分缺陷，但覆盖不了全部。剩下的 1.6% 只能现场暴露。

所以不能为了“零现场问题”无限提高在环覆盖率。边际成本上升得太快，而现场有回滚兜底。

合理的策略是：在环做到 98% 左右，现场用灰度发布与快速回滚兜住剩下 2%。

这与第 90 章的结论一致：变更失败率降到 12% 之后，继续降的收益不如缩短恢复时间。

91.11.5 数据合规的边界

原始数据不出厂，是多数制造企业的硬约束。这条约束直接决定了架构。

不出厂意味着：孪生要在边缘算，模型训练要用边缘的数据或脱敏后的特征，云上只能看统计量。

脱敏不等于安全。工艺参数本身就是竞争力，特征里可能包含关键信息。上传前要评估。

合规还涉及跨境。多国部署的项目，数据落地要求各不相同，架构要能按站点配置。

91.11.6 收益与代价

收益侧：时延降一个数量级、缺陷早发现、参数可以离线试、故障可以回溯。

代价侧：边缘节点的硬件与运维、孪生的建模与校准、数据链的开发与维护、人员的培训。

按一个中等项目估算，投入约三人六个月，收益是每年减少若干次现场调试与非计划停机。这个账要按项目算，没有通用数字。

一条经验：孪生的收益主要在“减少试错”，而不是“提高性能”。它不会让控制变好，但会让你更快找到好的参数。

另一条经验：收益随使用深度增长，而不是随建模精度增长。同一个 G2 孪生，用来整定参数比用来展示的价值高一个量级。

所以预算分配应该偏向“用起来”，而不是偏向“做得像”。把三维渲染的钱挪一半到数据链上，收益更确定。

91.11.7 方法的边界

四类场景不适合本章的方法。

第一类，对象完全无法建模。强非线性、时变、机理不明，孪生的保真度上不去，不如用纯数据驱动的方法。

第二类，工况极少变化。一年调一次参数的系统，孪生的价值有限，做好记录就够。

第三类，安全等级极高。孪生不能参与安全回路，只能做辅助，架构上要物理隔离。

第四类，预算与人力都不足。孪生需要持续维护，没人维护的孪生会迅速失准，比没有更糟。

四类场景不是“不能用”，而是“用了不划算”。

91.11.8 与前后章的分工

第 88 章给扫描周期与时延的底层结论，本章用它的判据做分层。

第 89 章给网络时延的测量方法，本章用它做同步与抖动分析。

第 90 章给版本管理与在环测试的流程，本章把它具体化为 MIL/SIL/HIL 的配比。

第 92 章讲边缘节点的安全加固，本章的架构是那章的输入。

第 93 章讲交付，本章的基线与验收方法会被引用。

91.11.9 三个常见反对意见

第一个，“孪生是噱头”。做展示的孪生确实是噱头，但用来整定与回溯的孪生是工具。区别在于有没有同步与比对。

第二个，“边缘就是小服务器”。边缘的核心约束是实时与断网可用，不是算力。把它当小服务器用，一定会在抖动上翻车。

第三个，“上云就万事大吉”。云解决的是算力与集中运维，解决不了时延与合规。三者要一起考虑。

三个意见背后是同一个误解：把架构当成选型，而不是当成约束下的设计。

91.11.10 一页纸结论

第一，孪生的价值在同步与比对，不在渲染。没有同步的孪生只是看板。

第二，保真度用阶跃响应的最大相对偏差衡量，G3（98.10%）是多数项目的经济点。

第三，同步间隔由允许偏差反解，位置控制 20 ms 对应 10 mm。

第四，边缘节点的回路数由周期与固定开销决定，2 ms 周期连一个回路都放不下。

第五，分层判据是 $\omega_c T_d \leq 0.2$，慢过程允许 166.7 ms，快伺服只允许 5.00 ms。

第六，在环测试三级串联检出 98.4%，剩下的 1.6% 用灰度与回滚兜住。

如果只记一句：**孪生不是把设备搬到屏幕上，而是把设备的可信状态搬到能算的地方**。

91.11.11 什么时候该放弃孪生

多数讨论集中在“怎么做孪生”，很少讨论“什么时候不做”。四种情况该考虑放弃。

第一种，答不出“它替谁做决定”。如果孪生的输出没有任何人、任何程序消费，它就是一个昂贵的看板，交付即闲置。

第二种，校准成本高于收益。漂移速率 4%/月 的对象，三个月就偏 12%，标定一次要停线两天，一年标定四次，成本很容易超过它省下的调试时间。

第三种，工况始终在变。孪生的参数要在稳定工况下辨识，工况一天一变，参数追不上变化，保真度永远停在 G1。

第四种，组织里没人能维护。孪生需要每月看数、每季校准、每年辨识，缺人维护的孪生会慢慢失准，最后比没有更糟，因为它还在被当作依据。

四种情况里，第四种最隐性也最致命。技术问题可以补，组织问题补不了。

放弃孪生不等于放弃这些手段。可以用轻量的趋势比对替代，成本低一个数量级，也能覆盖“偏差有没有变大”这类问题。

91.11.12 决策前的三个提问

方案评审时，三个提问能过滤掉大部分不成立的方案。

问一：这套东西的输出，下个月会被谁用到。答不出具体的人或程序，方案就是展示品。

问二：如果它错了，后果是什么，谁能发现。答不出检测手段，说明这套系统没有安全边界。

问三：不做它，用现有手段能不能达到同样效果。如果能，说明它的价值不在功能，而在别的方面，比如汇报。

三个提问都过了，再谈技术路线。技术路线可以讨论很久，而方向错了，讨论越久损失越大。

91.11.13 本章方法的适用边界

本章的方法适用于有明确对象、有可测状态、有稳定工况的工业过程。

不适用于纯机理未知的黑箱对象，也不适用于工况剧烈变化的间歇过程。这两类可以借用同步与分层的思路，但保真度的判据要换。

不适用于毫秒级以下的硬实时回路。微秒级的伺服环由驱动器内部完成，本章的边缘与孪生都在它之外，只能做监视。

不适用于安全回路。安全功能要用认证的安全控制器与硬接线，孪生只能旁观，不能参与。

三条边界之外，本章的判据都可以直接用。边界之内的场景，按表 91-10 的基线起步，风险最小。

91.11.14 与纯数据驱动方法的取舍

孪生与纯数据驱动不是对立关系，取舍看工况覆盖度。

工况覆盖充分、数据量大的场合，纯数据驱动往往更准，因为它不需要假设对象阶次，也不用辨识滞后。缺点是外推能力差，工况一出训练集就失准。

工况覆盖不足、机理清楚的场合，孪生更稳，因为参数有物理意义，可以外推到没测过的工况。缺点是建模与校准要花时间。

两者可以组合：孪生做主体，数据驱动补残差。残差模型只在孪生偏差大的区间起作用，训练数据要求低，也容易解释。

在边缘节点上，数据驱动模型通常更小、算得更快，适合放在实时环里；孪生计算量大，适合放在边缘的非实时分区或云上。

判据可以压成一句：**要外推就用孪生，要插值就用数据**。多数工业现场是插值为主、外推为辅，所以两者都要有。

91.12 Python 仿真代码

91.12.1 两个脚本与运行方式

`code/ch091_sim.py` 负责算，`code/ch091_figs.py` 负责画。

```
python code/ch091_sim.py
python code/ch091_figs.py
```

第一条打印九段报告，第二条在 `figs/ch091/` 下生成十张图的 SVG 与 PDF。

脚本只依赖 `math` 与 `numpy`。整定后的闭环仿真用固定步长 2 ms，没有随机源，重跑逐位一致。

91.12.2 参数区

所有可调量集中在文件头部。表 91-13 是参数与正文的对应。

表 91-13 仿真参数与正文对应

参数	取值	含义	正文位置
`K_NOM`、`TAU_NOM`、`L_NOM`	1、2 s、0.2 s	标称对象	§91.2.1
`GRADES`	五组偏差	保真度五级	§91.2.1
`LAMBDA_TUNE`	0.3 s	IMC 整定参数	§91.2.3
`RATE`、`SYNC_MS`	0.5 m/s、五档	同步算例	§91.2.4
`EDGE_FIXED_MS`、`EDGE_PER_LOOP_MS`	1.2、0.35 ms	边缘开销	§91.2.5
`LAYERS`、`OBJ_OMEGA`	三档时延、两个对象	分层算例	§91.2.6
`VPH`	四阶段	检出率与成本	§91.2.7
`CHANNELS`、`FS_HZ`、`BYTES_PER_SAMPLE`	32、1000、4	数据量	§91.2.8
`DRIFT_TH`、`DRIFT_RATES`	5%、四档	漂移检测	§91.2.9
`HIL_CASES`、`HIL_MIN_PER_CASE`	400、3 min	HIL 排程	§91.2.10

改参数要成组改。把周期从 10 ms 改成 5 ms，`EDGE_CYCLES_MS` 要改，`EDGE_UTIL_MAX` 往往也要复核，因为占用率翻倍后 70% 的上限可能过不去。

91.12.3 九段报告索引

表 91-14 是报告与章节的对应。

表 91-14 九段报告索引

段	内容	主要输出
`[1]`	保真度	五级的最大偏差、保真度、调节时间与倍数
`[2]`	同步	五个间隔的状态偏差、10 mm 对应的上限
`[3]`	边缘预算	五个周期的占用率与回路数
`[4]`	分层时延	三层的相位损失、两个对象的允许时延
`[5]`	在环	四阶段的检出率、每缺陷成本、累计检出
`[6]`	离散	四个台架数的批次与工作日
`[7]`	数据量	日量、30 天容量、上云量与带宽
`[8]`	漂移	四个速率的到阈值时间
`[9]`	HIL 排程	四个台架数的轮数与工期

正文引用数字时先查这张表，再定位到段。找不到就说明引用错了。

91.12.4 三个实现要点

第一个要点是闭环仿真的步长。步长 2 ms、对象时间常数 2 s，一个时间常数有 1000 步，精度足够。如果把时间常数改成 0.05 s，步长要缩到 0.05 ms，否则离散化误差会盖过参数偏差。

第二个要点是纯滞后的实现。脚本用整数步延迟，`nd = round(L/dt)`，`L=0.2`、`dt=0.002` 对应 100 步。这样实现的滞后是精确的，不会引入额外的一阶滞后。

第三个要点是调节时间的判据。脚本用 2% 带，取最后一次离开带的时刻。这个定义对单调响应与小超调都适用，比“首次进入带”更严格。

91.12.5 十张图怎么生成

十张图分两类。数据类六张用 `plot_styles`，架构与流程类四张用 `flow_styles`。

数据类是图 91-3 到图 91-8，共用 `setup()` 的字体与线宽。其中图 91-3 用了双轴，左轴保真度、右轴调节时间倍数；图 91-5 用了对数纵轴，因为成本跨两个数量级。

流程类是图 91-1、图 91-2、图 91-9 与图 91-10。四张只用了泳道、圆角矩形、菱形、平行四边形与箭头，没有传递函数方块，符合第 86 到 93 章的图元约束。

图 91-3 与图 91-6 的图号与文件序号不对应，这是有意为之：正文按逻辑顺序编号，文件按生成顺序命名，两者通过图注对应。

91.12.6 五个自改实验

实验一：把 `TAU_NOM` 从 2 s 改成 0.2 s，重跑 `[1]`。五级的保真度不变，但调节时间整体缩短十倍。这说明保真度是相对量，与时间尺度无关。

实验二：把 `LAMBDA_TUNE` 从 0.3 改成 0.5，重跑 `[1]`。调节时间整体变长，G1 与 G5 的差距从 2.44 倍缩小到约 1.4 倍。整定越保守，保真度的影响越小。

实验三：把 `EDGE_FIXED_MS` 从 1.2 改成 2.0，重跑 `[3]`。10 ms 周期的回路数从 16 降到 9，20 ms 从 36 降到 23。固定开销对回路数的影响是非线性的。

实验四：把 `OBJ_OMEGA` 的快伺服从 40 改成 80，重跑 `[4]`。边缘闭环的损失从 17.74° 涨到 32.6°，超过判据。这说明带宽翻倍，允许时延减半。

实验五：把 `CLOUD_RATIO` 从 1% 改成 10%，重跑 `[7]`。上云带宽从 10.24 kbit/s 涨到 102.4 kbit/s，日量 1.11 GB。这时要考虑专线或 5G，不能再走普通宽带了。

五个实验做完，参数与结论的关系就清楚了。

91.12.7 把报告读一遍

建议按 `[1]`、`[4]`、`[3]`、`[5]` 的顺序读。

先读 `[1]`，它给保真度的五级与后果。盯住最后一列，2.44 倍的差距是整章最有力的数字。

再读 `[4]`，它给分层判据。同一层的两个损失差几十倍，这一句就能说清“为什么快伺服不能上云”。

然后读 `[3]`，它给边缘预算。第一行的 0 个回路是最反直觉的结论，值得单独讲一次。

最后读 `[5]`，它给在环测试的账。现场 60 与 MIL 1.82 的对比，是说服管理层投入测试的依据。

`[2]`、`[6]`、`[7]`、`[8]`、`[9]` 留到需要具体数字时再查。

91.12.8 换成自己的数

三步走。

第一步换对象：`K_NOM`、`TAU_NOM`、`L_NOM`、`OBJ_OMEGA`。这四个决定所有时间量的量级。

第二步换硬件：`EDGE_FIXED_MS`、`EDGE_PER_LOOP_MS`、`EDGE_UTIL_MAX`、`CHANNELS`、`FS_HZ`。这四个决定边缘能力与数据量。

第三步换流程：`VPH`、`HIL_CASES`、`HIL_MIN_PER_CASE`、`DRIFT_RATES`。这四个决定测试成本与维护节奏。

三步换完重跑，看有没有不合理的结果：回路数为负、相位损失超过 90°、检出率超过 100%，都是参数配错了。

最后提醒一句：脚本给的是“在给定假设下的量级”，不是“你项目的验收结论”。真正的结论要看现场测量。

91.12.9 三项自检

第一项，可重现。连跑两次 `ch091_sim.py`，逐行比对，应当完全一致。

第二项，段落齐全。报告应有九个 `[n]` 段头。

第三项，图件完整。`figs/ch091/` 下应有二十个文件，十张图各一份 SVG 与一份 PDF，且都不为零字节。

还有一项与数字无关：打开图 91-1 看一眼中文是否正常。缺字体会变方框，脚本不会报错。

91.12.10 想加一段怎么办

四步。写计算函数，返回元组列表；在 `report()` 里加打印，编号接着排；决定要不要出图；回正文加小节并更新表 91-14。

四步里最容易漏的是最后一步。脚本加了段、正文没加，或者正文加了、索引没加，读者就对不上号。

加段之前先问：这个量能不能从已有九段推出来。能推出来的不要重复算，重复计算会带来两份可能不一致的数字。

91.12.11 代码分三层

`ch091_sim.py` 按三层组织，层次清楚才好改。

最上面是常量区。所有可调量集中在这里，命名用大写，单位写进注释。改参数只改这一段，不往下翻。

中间是纯计算函数。每个函数只算一件事，返回元组列表，不打印、不画图、不读文件。纯函数的好处是可以单独调用，也方便对照手算。

最下面是 `report()`。它只负责格式化输出，把各函数的返回值排成九段。所有打印都集中在这里，改输出格式不用动计算。

三层之间单向依赖：常量被函数读，函数被报告读。反过来调用会让代码难以测试。

`ch091_figs.py` 只做两件事：调 `ch091_sim` 的函数取数，用样式助手画图。它不重新计算任何量，否则会出现两张图的数字对不上。

全套代码不用随机数，也不读系统时间。同一份代码在任何机器上跑，输出逐位一致。

91.12.12 正文数字怎么回查

正文里每个数都应该能在报告里找到，找不到就是引用错了。

回查分三步。先在表 91-13 或表 91-14 里查到对应的段号，再在报告里定位那一段，最后在函数里确认算法。

比如正文写“2.44 倍”，查表 91-14 知道在 `[1]`，报告里对应“为高保真档的 2.44 倍”，函数是 `fidelity_table`。三步走完，数字的来路就清楚了。

正文与报告不一致时，先怀疑正文。报告是算出来的，正文是抄过来的，抄错比算错常见。

改参数后要重跑脚本，并按报告改正文。只改脚本不改正文，是这类书稿最常见的不一致来源。

91.12.13 运行环境与依赖

两个脚本在 Python 3.11 下测试，依赖只有 `numpy` 与 `matplotlib`，不需要联网，也不需要额外的数据文件。

图里的中文依赖系统字体。脚本按宋体、黑体、雅黑的顺序查找，都找不到时退回默认字体，中文会变成方框，但不会报错。

路径全部用相对路径，从 `book/` 目录运行。换目录运行时，输出会落到错误的位置，检查 `figs/ch091/` 是否生成即可判断。

脚本不读环境变量，不读系统时间，也不写日志。这三点保证了可在任何机器上复现。

如果要在服务器上批量出图，把 `matplotlib` 的后端固定为 `Agg` 即可，脚本里已经设置，不需要额外配置。

91.12.14 报告里没有的三个量

有三类量脚本算不出来，正文里也不该给出具体数值。

第一类是抖动 P99。它取决于具体的操作系统、驱动、负载，只能在目标硬件上实测。脚本给的是平均时延，不能替代。

第二类是频域保真度。脚本只算时域最大偏差，频域误差要做频谱分析，需要实测数据。

第三类是切换时间。冷备热备的切换时间由具体实现决定，与本章的公式无关。

这三类量在正文里只给判据，不给数值。看到别人给出这类数值时，先问一句“在哪台机器上测的”，答不出就是编的。

91.12.15 改一次参数的完整流程

改参数看着简单，走完整流程才不留隐患。流程分五步。

第一步查依赖。打开 §91.7.10 的三组耦合，确认要改的参数牵动哪几个，一起算一遍。

第二步改常量。只改文件头部，不散落在函数里。改完立刻重跑 `ch091_sim.py`，看九段报告有没有异常值。

第三步对正文。按表 91-14 找到引用该参数的段落与表格，逐处更新。数字要抄报告，不要手算。

第四步改图。重跑 `ch091_figs.py`，打开受影响的图看一眼，确认坐标范围与标注还对得上。

第五步记台账。在表 91-11 里加一行，写日期、改什么、改前、改后、观测指标、结论。

五步里第三步最容易漏。漏了不会报错，但读者会看到正文与报告不一致，整章的可信度就没了。

流程走完跑一次 `check_chapter.py`。字数、节数、图数三项都过，改动才算完成。

三项里最容易被忽略的是字数。删掉一段解释性的文字，字数掉到线下，检查会变黄，而内容其实更紧凑。这时补的应该是有效信息，不是凑字数。

第92章 计算机控制中的工程问题

92.1 动机与直观理解

第 84 章算的是“计算机进到回路里会带来什么”，第 85 章算的是“这些东西怎么搭起来”，第 86 章算的是“算不完怎么办”，第 90 章算的是“软件怎么保证不错”。四章讲完，一个回路在纸面上已经站得住：结构清楚、周期够短、调度可行、代码有测试。可是一装进柜子、拉到现场、接上传感器，抖动、漂移、误动作照样出现，而且往往查不出来——因为纸面上那四本账，都没有把“实物”记进去。

本章记的是第五本账，科目叫**实物层**：量化、抖动、执行时间、电磁兼容与接地、失效与安全。五科互不相干，收口方式却一样，都要折算成“几个 LSB”“几个 dB”“每小时几次”“每年几次”这样的数。算不出这四种数，工程问题就永远是“师傅经验”。

92.1.1 一个现场场景

同一条回路，实验台上阶跃响应干净，超调 4.8%；装到现场，阀门每隔几分钟抽一下，示波器上看不出原因。拆开看，至少三件事同时发生。

第一件在地线上。变频器与控制柜共用一根地线，两台设备的“地”之间存在 5 V、50 Hz 的电位差；ADC 前端的共模抑制只有 60 dB，这 5 V 折成差分误差 5 mV，而 12 位、10 V 量程的一个台阶只有 2.441 mV——**地线上的电压直接折成 2.05 LSB 的测量误差**。换成 80 dB 的前端，误差降到 0.20 LSB；换成 100 dB，只剩 0.02 LSB（表 92-6）。同一个传感器、同一根线，只换前端，误差差 100 倍。

第二件在机柜上。柜门与柜体之间留着 100 mm 的缝隙，1 MHz 的开关噪声从这道缝漏进来，缝隙泄漏上限只有 63.5 dB；同样 0.1 mm 厚的铜皮，如果没有缝，1 MHz 处的屏蔽效能是 120.9 dB（表 92-5）。**缝比材料更决定屏蔽**，这是本章最容易被忽略的一条。

第三件在时间上。控制任务的中位执行时间是 301 μs，P99.99 是 1.03 ms；为了留余量把周期定在 1 ms，就意味着每小时有 936 次算不完（表 92-10）。算不完的那一拍用的是上一拍的控制量，回路等效延迟从一拍变成两拍，裕度掉下去，表现就是“偶发的一下抽搐”。

三件事没有一件写在传递函数里，也没有一件能靠调 PI 参数解决。

92.1.2 五本账

把上一节的三件事按物理量归并，就得到本章反复使用的一张总表（表 92-1）：五行各管一本账，每行给出物理量、单位、判据和本章的实测基准。

表 92-1 计算机控制回路里的五本账

账	物理量	常用单位	本章判据	实测基准
幅值账	量化台阶 $\Delta = R/2^N$	LSB、dB	量化信噪比高于回路要求 10 dB	12 位、10 V：$\Delta = 2.441$ mV，74.10 dB
时间账	采样抖动 σ_t	ns、dB	抖动误差小于 1 LSB	1 kHz、1 μs：44.0 dB
计算账	执行时间分位数	μs、次/小时	超时率低于 10^{-4}	1 ms 周期：936 次/小时
能量账	共模、屏蔽、地环路	dB、LSB	折算误差小于 0.5 LSB	5 V、60 dB：2.05 LSB
安全账	平均失效概率 PFD_{avg}	每年次数、SIL	达到目标 SIL	诊断覆盖 90%：4.39×10^{-3}，SIL 2

五本账放在一起就是图 92-1。图中把五科画成五条并行的支路，是因为它们之间**不能互相抵扣**：量化做得再好，也压不住 5 V 的共模；屏蔽做得再厚，也换不回一次超时。工程上唯一允许的抵扣是“总误差预算”，而且必须在同一量纲下（都折成 LSB 或都折成毫伏）才成立（§92.2.8）。

图 92-1 计算机控制的五本账：幅值、时间、计算、能量、安全各自独立收口，只有折到同一量纲后才能合并成误差预算

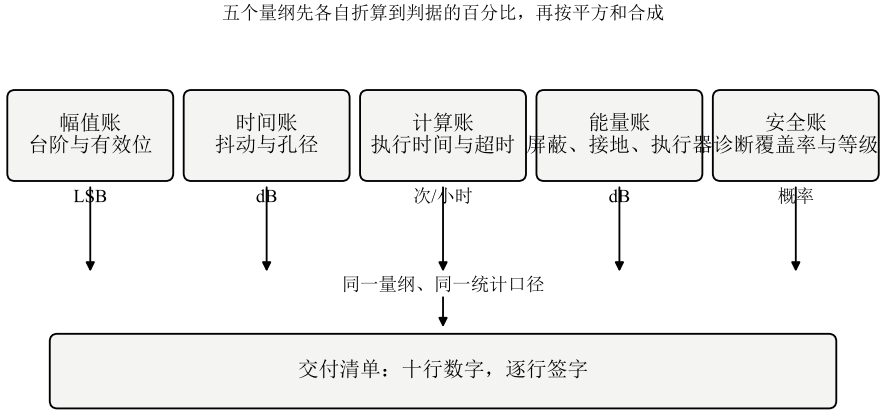


图 92-1

92.1.3 先看一组数

在展开之前，先把本章基准的一组实测摆出来，建立手感。基准配置固定为：量程 $R = 10$ V（± 5 V）、12 位、指标测试采样率 10 kHz、测试音 997 Hz、控制周期 20 ms、PI 参数 $K_p = 2.0$、$T_i = 0.5$ s。

- **幅值**：12 位台阶 2.441 mV，量化信噪比 74.10 dB，有效位 12.02 位；同样的前端做 16 倍过采样平均，85.33 dB、13.88 位，只换回 1.86 位。
- **时间**：1 kHz、5 V 峰值信号，抖动 1 μs 时信噪比 44.0 dB；抖动压到 50 ns，升到 70.0 dB。抖动造成一个 LSB 误差的门限是 77.7 ns。
- **计算**：中位 301 μs、P99 674 μs、P99.99 1032 μs；周期 2 ms 时零超时，1 ms 时 936 次/小时，0.5 ms 时 501984 次/小时。
- **能量**：0.1 mm 铜在 1 MHz 的屏蔽效能 120.9 dB，其中吸收 13.1 dB、反射 108.1 dB、多次反射 -0.4 dB；留 100 mm 缝只剩 63.5 dB。
- **安全**：诊断覆盖率 90% 时 $\mathrm{PFD}_{avg} = 4.39 \times 10^{-3}$，落在 SIL 2；提到 99% 才降到 4.39×10^{-4}，进 SIL 3。

这五组数放在一起看，能看出一个共同点：**它们都不是“精度”问题，而是“余量”问题**。74 dB 的量化信噪比对 12 位是很漂亮的数，但如果回路只需要 60 dB，多出来的 14 dB 就是白花钱；反过来，936 次/小时的超时率听上去是 2.6×10^{-4} 的小概率，乘上 3600 秒就成了“每小时近千次”。

92.1.4 三种误读

误读一：位数就是精度。 16 位 ADC 的理论信噪比是 98.08 dB，实测 98.24 dB，很接近。但把前端噪声加到 4 LSB，无噪声码分辨率只剩 7.28 位，等效有效位 8.21 位。**位数是天花板，噪声决定实际住几楼**。选型时只看位数，是本章最常见的一笔错账。

误读二：平均能提高精度。 12 位加 16 倍平均，实测 85.33 dB，等效 13.88 位，比原来多 1.86 位，不是 4 位。原因是过采样每 4 倍才换 1 位（0.5 位/倍频程），16 倍理论上换 2 位，实测少 0.14 位是被平均窗内信号自身的弯曲吃掉的。指望“多平均几次就干净”，在 12 位这个量级上行不通。

误读三：加屏蔽就干净。 0.1 mm 铜在 1 MHz 无缝隙时 120.9 dB，有 100 mm 缝只剩 63.5 dB，差 57 dB，相当于把铜皮换成铝箔还倒贴。屏蔽的第一原则是**先堵缝、后加厚**，因为厚度只影响吸收项（0.1 mm 换 0.5 mm，1 MHz 从 120.9 dB 升到 173.9 dB，全是吸收项在涨），而缝隙决定的是泄漏上限。

92.1.5 本章与相邻章的分工

本章只讲“实物层怎么记账”，所以有五件事只点到不展开：

- **PI 参数怎么整定**：见第 44 章与第 106 章。本章的 $K_p = 2.0$、$T_i = 0.5$ 是给定的。
- **采样周期与回路结构**：第 84、85 章已经算过。本章只补“抖动与执行时间怎么进这本账”。
- **实时调度与任务划分**：见第 86 章。本章假设任务已经分好，只讨论“它跑多久、跑不稳会怎样”。
- **软件测试与代码质量**：见第 90 章。本章的代码生成与 HIL 只讲“模型到目标机”这一段接口。
- **工业通信与网络**：见第 89 章。本章的 EMC 只讲柜内与板级，不讲总线协议。
- **先进控制算法**：MPC 见第 58、80 章，Smith 预估见第 47、107 章；它们对量化和抖动更敏感，但补偿方法不在本章。

92.1.6 同一个误差，在不同位数下分量不同

初学者常问“5 mV 的干扰算大吗”。这个问题没有绝对答案，因为**同一个物理误差，在不同位数下的分量完全不同**。

量程 10 V 时，5 mV 在 12 位系统里是 $5/2.441 = 2.05$ LSB，在 14 位里是 $5/0.6104 = 8.19$ LSB，在 16 位里是 $5/0.1526 = 32.8$ LSB。同样 5 mV，从“两个码”变成“三十三个码”。

这解释了两件事。**第一，为什么提高位数会让 EMC 问题突然暴露**：原来 12 位系统跑得好好的机柜，换成 16 位采集卡后开始跳码，不是干扰变强了，是尺子变细了。**第二，为什么噪声预算必须在定完位数之后做**：位数一变，所有项的 LSB 分量全部重算（§92.3.2）。

反过来也成立：如果一个 16 位系统实测只有 10 位有效，说明前端噪声约 64 LSB，这时再换 18 位采集卡毫无意义——**尺子越细，越要先擦干净被测物**。

92.1.7 一句话结论

把本章压成一句话：**数字控制器的逻辑正确只保证“算得对”，工程问题保证“算得准、算得完、算不坏”**。幅值账决定“看得清多小的变化”，时间账决定“抓得住多快的动作”，计算账决定“来不来得及”，能量账决定“扛不扛得住干扰”，安全账决定“坏的时候会不会伤人”。五本账各记各的，合并时只能折成同一量纲，而且要先折最坏的那一项。

92.1.8 本章怎么读

三种读者可以走三条路线。

- **做选型的**：读 §92.3 的五步、§92.7 的参数表、§92.10.4 的交付清单。三处合起来就是一张选型计算表。
- **做调试的**：读 §92.6 的失稳机制、§92.8 的排查顺序、§92.10.5 的现场顺序。重点是“先查什么、后查什么”。
- **做评审的**：读 §92.11 的边界与 §92.10.6 的返工案例。前者告诉你这套方法什么时候不成立，后者告诉你漏算的代价。

三条路线都指向同一个结论：**工程问题不是“再多加点余量”，而是“知道余量加在哪一本账上”**。

92.1.9 五个最容易混的名词

这一章后面会反复用到五个词，先把它们钉死，否则后面的讨论会一直在打滑。

- **分辨率**：能分辨的最小变化，等于一个台阶 $\Delta = R/2^N$。它是器件的属性，与信号无关。12 位、10 V 量程的分辨率就是 2.441 mV。
- **有效位数 ENOB**：把实测 SINAD 折回位数，$(\mathrm{SINAD} - 1.76)/6.02$。它含噪声与失真，是全通道的属性。本章 12 位 ADC 实测 12.02 位，现场只剩 10.8 位。
- **重复性**：同一输入、多次测量之间的离散程度，用标准差表示。它只跟噪声有关，与台阶无关。本章前端噪声 1 LSB，重复性就是 1 LSB。
- **准确度**：测量值与真值之差，含偏置与增益误差。它可以靠校准改善，重复性不能。
- **精度**：口语词，三样都沾。工程文档里应当避免单独使用“精度”这个词，要么写分辨率，要么写重复性，要么写准确度。

这五个词的关系可以用一句话概括：**分辨率是尺子的刻度，重复性是手抖不抖，准确度是尺子本身准不准**。三者互相独立：刻度很细但手抖，读数的末位没用；手很稳但尺子偏，读数一致地错；尺子准、手也稳，但刻度粗，就只能读到那个粗度。

现场最常听到的一句错话是“我这个系统精度 16 位”。16 位是分辨率，不是精度。真实说法应当是“分辨率 16 位、重复性 2 LSB、校准后准确度 1 LSB、等效有效位 13.8 位”——四个数各自成立，合起来才是这套系统的真实能力。

92.1.10 三个可以背下来的锚点

现场没有计算器，判断量级要靠锚点。本章只需要背三个数。

锚点一：1 LSB = 2.441 mV。 12 位、10 V 量程下，任何电压除以 2.441 就是 LSB 数。看到跳变 5 mV，立刻知道是 2 LSB；看到 25 mV，是 10 LSB；看到 250 mV，是 100 LSB，已经不是“末位噪声”而是真故障了。

锚点二：6 dB = 电压减半。 20 dB = 十分之一，40 dB = 百分之一，60 dB = 千分之一。共模 5 V 配 60 dB 就是 5 mV；配 80 dB 就是 0.5 mV；配 100 dB 就是 50 μV。**dB 每加 20，小数点左移一位**，这条比任何公式都好用。

锚点三：一拍 = 20 ms。 本章周期 20 ms，三拍 60 ms，看门狗 100 ms 是五拍。看到“延迟三拍”，就是 60 ms，而不是抽象的“三个采样周期”。相位这一侧也一样：3.797 rad/s 乘 20 ms 得 0.076 rad，即 4.4° 一拍——这个数在 §92.6.2 反复用到。

三个锚点合起来能做一次快速校核：假设现场看到“跳 2 LSB、发生在变频器启动时、持续约五拍”，就能立刻翻译成“5 mV 的干扰、与共模相关、持续 100 ms”。**能翻译成物理量的现象才是可查的**，否则只能记成“有时候会跳”。

这三个数只对本章的基准成立：换 16 位、换 5 V 量程、换 1 ms 周期，三个锚点全部要重算。**锚点是本地的，不是普适的**，这一点必须记牢，否则会把 12 位的直觉套到 24 位系统上，得出“干扰很大”的错误结论。

92.2 数学原理

五本账看着杂，数学骨架只有四条：**一个台阶、一个斜率、一个分布、一个分贝**。台阶管幅值，斜率管抖动，分布管执行时间，分贝管屏蔽与共模。安全账例外，它用的是概率，但形式上也只是一个乘法。

92.2.1 量化：一个台阶决定一切

N 位 ADC、满量程 R，台阶（一个 LSB）为

$$\Delta = \frac{R}{2^N}.$$

本章基准 $R = 10$ V、$N = 12$，$\Delta = 2.441$ mV。理想量化把连续值舍入到最近的整数倍，误差落在 $[-\Delta/2, \Delta/2]$ 内且近似均匀，标准差为

$$\sigma_q = \frac{\Delta}{\sqrt{12}} = \frac{\Delta}{3.464}.$$

12 位时 $\sigma_q = 0.705$ mV。满幅正弦的均方根是 $R/(2\sqrt{2}) = 3.536$ V，于是量化信噪比

$$\mathrm{SNR} = 20\lg\frac{R/(2\sqrt{2})}{\Delta/\sqrt{12}} = 6.02N + 1.76\ \mathrm{dB}.$$

这就是那条人人会背、却常被误用的公式。它成立有四个前提：满幅输入、正弦、误差均匀独立、只算量化一项。缺一个，实测就会偏离——本章 12 位实测 74.10 dB，比理论 74.00 dB 高 0.10 dB，原因是输入幅度取了 0.999 倍满幅而误差略偏小，这种零点几 dB 的偏差在工程上不必追。

有效位数 ENOB 是同一个式子的反解：

$$\mathrm{ENOB} = \frac{\mathrm{SINAD} - 1.76}{6.02}.$$

它比“标称位数”有用，因为 SINAD 里包含了量化、噪声、失真三项。标称 12 位、ENOB 只有 10.2 位，说明前端噪声吃掉了将近两位（§92.7.1）。

92.2.2 位数与过采样

表 92-2 是 8 到 16 位的实测，测试条件统一为 10 V 量程、10 kHz 采样、997 Hz 测试音、8192 点。

表 92-2 不同位数的量化信噪比与有效位

位数 N	台阶 Δ	理论 $6.02N + 1.76$	实测 SNR	实测 ENOB
8	39.06 mV	49.92 dB	49.93 dB	8.00 位
10	9.766 mV	61.96 dB	61.91 dB	9.99 位
12	2.441 mV	74.00 dB	74.10 dB	12.02 位
14	0.6104 mV	86.04 dB	86.04 dB	14.00 位
16	0.1526 mV	98.08 dB	98.24 dB	16.03 位

实测与理论逐行对得上，最大偏差 0.16 dB。这条“每加一位涨 6 dB”的直线是后面所有讨论的基准线，它的含义是：**位数每加一，能分辨的最小变化减半，动态范围翻倍**。

单个 ADC 的位数不够时，第一条路是过采样加平均。M 个独立噪声样本平均，噪声功率降到 $1/M$，信噪比涨 $10\lg M$，折成位数是

$$\Delta\mathrm{ENOB} = \frac{1}{2}\log_2 M.$$

$M = 16$ 时理论 2 位、12 dB；实测 85.33 dB、13.88 位，只涨 1.86 位（11.23 dB）。少的 0.14 位来自平均窗内信号自身的弯曲：997 Hz 的正弦在 100 μs 窗口里不是直线，平均出来的值不等于窗口中点的值，这一项在 §92.5.4 会单独算。

第二条路是加抖动（dither）。加抖动之后总噪声变大，实测从 74.10 dB 掉到 70.99 dB，掉了 3.11 dB；但它换来的是**量化误差与信号不相关**。对控制回路来说，相关的量化误差会在稳态形成一个固定的小偏置或极限环，不相关的噪声反而更容易被积分器滤掉。**抖动的收益不在信噪比，在误差的性质**。

图 92-2 把表 92-2 画成两条线：台阶随位数指数下降，信噪比随位数线性上升。两条线共用一个横轴，读图时注意左轴是对数刻度，一位对应 6 dB 在图上是一条等距的斜线。

图 92-2 量化台阶与量化信噪比随位数的变化：台阶每加一位减半，信噪比每加一位涨约 6 dB

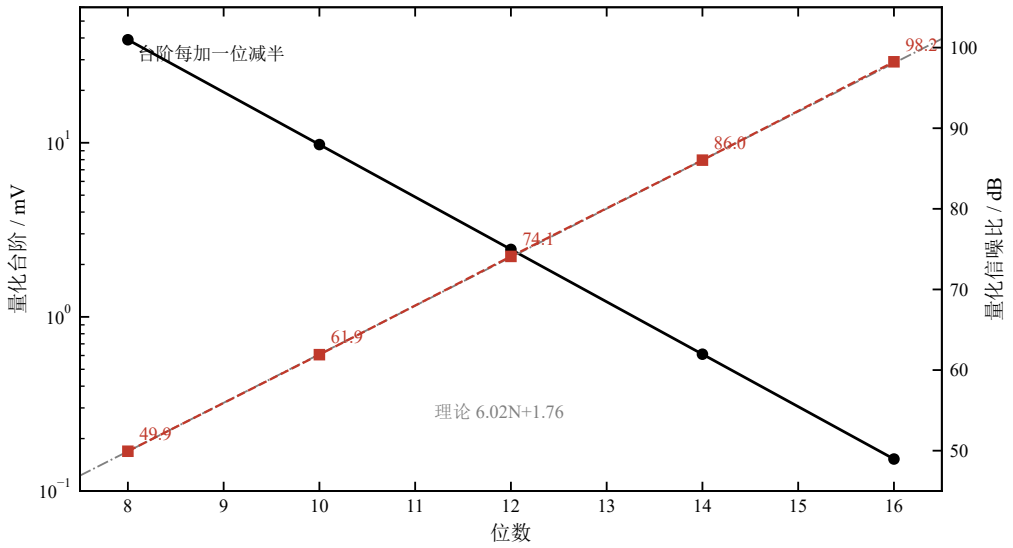


图 92-2

92.2.3 抖动：一个斜率决定一切

采样时刻不是理想等间隔的，而是 $t_k = kT_s + \delta_k$，δ_k 是零均值、标准差 σ_t 的随机量。信号在采样点的值是 $y(t_k)$，与理想时刻的值相差

$$\Delta y \approx \dot{y}(t_k)\,\delta_k = \dot{y}\,\delta .$$

对幅值 A、频率 f 的正弦，$\dot{y} = 2\pi f A\cos(2\pi f t)$，其均方根为 $2\pi f A/\sqrt{2}$，于是抖动误差的均方根是 $2\pi f A\sigma_t/\sqrt{2}$。信号本身的均方根是 $A/\sqrt{2}$，两者相除：

$$\mathrm{SNR}_t = -20\lg(2\pi f \sigma_t).$$

这个式子有三个值得记住的性质。**一是与信号幅度无关**，因为误差与信号同比例放大；**二是与频率成正比**，频率翻十倍，信噪比掉 20 dB；**三是与采样率无关**，只要抖动是采样时刻的绝对误差，采样快慢不改变单个样本的信噪比（过采样平均时才起作用）。

表 92-3 是实测与理论的对照，九个组合最大偏差 0.06 dB。

表 92-3 采样抖动引起的信噪比：实测与一阶理论

信号频率	$\sigma_t = 50$ ns	$\sigma_t = 200$ ns	$\sigma_t = 1$ µs
100 Hz	90.01 / 90.06 dB	78.01 / 78.02 dB	64.04 / 64.04 dB
1 kHz	70.01 / 70.06 dB	57.97 / 58.02 dB	43.99 / 44.04 dB
10 kHz	50.04 / 50.06 dB	38.00 / 38.02 dB	24.03 / 24.04 dB

表中每格是“实测 / 理论”。三列之间每十倍抖动掉 20 dB，三行之间每十倍频率掉 20 dB，两条规律都验证了。

把抖动折成“几个 LSB”更直观。抖动引起的峰值误差 $2\pi f A\sigma_t$ 等于一个台阶时，

$$\sigma_t^{\mathrm{1LSB}} = \frac{\Delta}{2\pi f A}.$$

1 kHz、5 V 峰值、12 位 10 V 量程，$\sigma_t^{\mathrm{1LSB}} = 77.7$ ns。这个数很重要：它说明**1 kHz 的信号要抖到 78 ns 才相当于一个 LSB**，所以工频信号对抖动几乎免疫；但同样的 78 ns 放到 10 kHz 上就只能换到 7.8 ns 的等效精度，放到 1 MHz 上就是 0.078 ns。这就是为什么电力电子与振动控制把时钟抖动当成一等公民，而慢过程控制基本不提。

图 92-3 画出三条 σ_t 线在 100 Hz 到 10 kHz 上的走向。三条线平行，斜率都是每十倍频程 -20 dB，这是式 $\mathrm{SNR}_t = -20\lg(2\pi f \sigma_t)$ 的直接图形。

图 92-3 抖动信噪比随信号频率的变化：三条线平行，斜率每十倍频程 20 dB

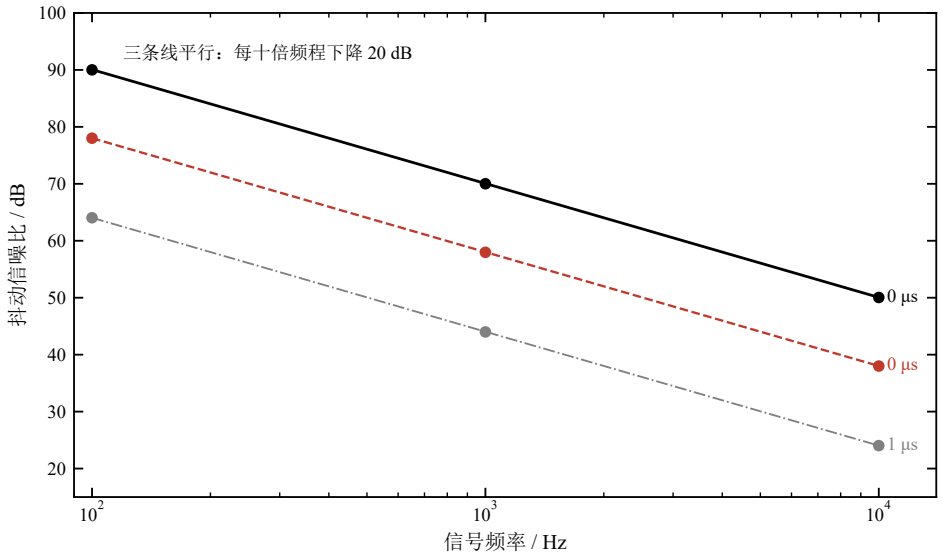


图 92-3

92.2.4 执行时间：一个分布决定一切

控制任务不是“执行时间等于某个常数”，而是“每拍从一个分布里抽一次”。用单值（哪怕是最坏值）记这笔账，会同时犯两个错：把常态估得太坏，把尾部估得太好。

本章用对数正态描述：中位数 $t_{50} = 300$ μs，对数标准差 $\sigma = 0.35$。选它有两个理由。一是执行时间只能为正、右偏，对数正态天然满足；二是它的尾部有明确的解析分位数 $t_p = t_{50}\exp(\sigma z_p)$，不必靠采样碰运气。50000 次抽样的实测分位数是：中位 301 μs、P99 674 μs、P99.99 1032 μs、最大 1152 μs。

超时率是分布尾部的积分：

$$p_{\mathrm{miss}} = P(t > T_s), \qquad N_{\mathrm{miss}} = p_{\mathrm{miss}} \cdot \frac{3600}{T_s}.$$

后一项把“每拍的概率”换成“每小时次数”，这是工程上唯一能拿去和故障率比较的量纲。$T_s = 1$ ms 时 $p_{\mathrm{miss}} = 2.60 \times 10^{-4}$，乘以 3.6×10^6 得 936 次/小时。**概率小不等于事件少**，这是计算账里最容易看错的一步。

92.2.5 屏蔽与接地：一个分贝决定一切

导体对电磁场的衰减用趋肤深度描述：

$$\delta_s = \sqrt{\frac{2}{\omega \mu \sigma}}.$$

铜的 $\sigma = 5.8 \times 10^{7}$ S/m、$\mu = 4\pi \times 10^{-7}$ H/m，1 MHz 时 $\delta_s = 66.1$ μm。0.1 mm 铜皮的厚度是 1.51 个趋肤深度。

屏蔽效能按三项相加：

$$SE = A + R + B, \qquad A = 8.686\,\frac{t}{\delta_s},$$

其中 A 是吸收损耗（dB），R 是反射损耗，B 是多次反射修正（薄板、低频时为负）。R 与波阻抗和材料本征阻抗之比有关，1 MHz 的铜约 108 dB；B 在 $t \gg \delta_s$ 时趋于 0，在 $t \approx \delta_s$ 时为负。三项目前合计 120.9 dB。

但机柜不是整块铜。任何长度为 L 的缝隙都是一根槽天线，泄漏上限为

$$SE_{\text{slot}} = 20\lg\frac{\lambda}{2L}, \qquad L < \frac{\lambda}{2}.$$

1 MHz、$L = 100$ mm：$\lambda = 300$ m，$SE_{\text{slot}} = 20\lg(300/0.2) = 63.5$ dB。**材料的 120.9 dB 在这道缝面前毫无意义**，实际屏蔽由 63.5 dB 决定。这条式子是本章“先堵缝、后加厚”的全部依据。

接地这一侧用共模抑制比折算。共模电压 v_{cm} 经过 CMRR 为 C（dB）的前端，剩下的差分误差是

$$e = \frac{v_{cm}}{10^{C/20}}.$$

5 V 共模、60 dB 抑制，$e = 5$ mV，折成 12 位 10 V 量程的 2.05 LSB；80 dB 时 0.20 LSB；100 dB 时 0.02 LSB。**CMRR 每加 20 dB，误差降十倍**，这是接地设计里唯一线性的那条杠杆。

92.2.6 安全：一个乘法决定一切

低要求模式下，安全功能的平均失效概率按

$$\mathrm{PFD}_{avg} = \lambda_{DU}\left(\frac{T_{proof}}{2} + MTTR\right), \qquad \lambda_{DU} = \lambda(1 - DC)$$

计算。λ 是总失效率，DC 是诊断覆盖率，T_{proof} 是检验测试周期，$MTTR$ 是平均修复时间。本章取 $\lambda = 10^{-5}$ /h、$T_{proof} = 8760$ h（一年）、$MTTR = 8$ h。

$DC = 0.9$ 时 $\lambda_{DU} = 10^{-6}$/h，$\mathrm{PFD}_{avg} = 10^{-6} \times (4380 + 8) = 4.39 \times 10^{-3}$；$DC = 0.99$ 时 $\lambda_{DU} = 10^{-7}$/h，$\mathrm{PFD}_{avg} = 4.39 \times 10^{-4}$。按 IEC 61508 的低要求模式分档，前者 SIL 2，后者 SIL 3——**诊断覆盖率从 90% 提到 99%，只多了 9 个百分点，安全等级就上一档**。这解释了为什么安全仪表系统肯为自诊断花那么多代码。

92.2.7 定点化：系数误差的严格界

定点实现把滤波器系数舍入到 2^{-b} 的整数倍。设浮点系数为 b_i, a_i，定点为 b_i^q, a_i^q，输出误差不是简单地等于“系数误差乘输入”，而要通过两个滤波器的差：

$$e = \left[\frac{B_q}{A_q} - \frac{B}{A}\right] x = \frac{B_q A - B A_q}{A A_q}\, x.$$

右边是一个确定的线性滤波器，其冲激响应 h_e 可以算出来，于是

$$|e|_{\max} \le \|h_e\|_1 \cdot |x|_{\max}.$$

这个界比“用原滤波器 h 的 L1 范数乘系数误差”正确得多，因为它保留了极点对系数误差的放大。12 位系数（Q12）时界是 0.329 mV、实测最大误差 0.137 mV；Q15 时界 0.227 mV、实测 0.118 mV；Q10 时界 7.430 mV、实测 3.807 mV。**界始终包住实测，但松紧差 1.5 到 2.4 倍**，所以工程上用它做验收下限，最终还要实测（表 92-9）。

92.2.8 五个式子怎么折到同一量纲

五本账的原始单位各不相同：幅值账是 LSB，时间账是 dB，计算账是“次/小时”，能量账是 dB，安全账是概率。**不同单位不能相加，必须先把每一项换成“占判据的百分比”。**

换法是每项除以自己的判据。本章的判据取：幅值 1 LSB、时间 1 LSB 等效抖动、计算 1 次/小时、能量 0.5 LSB 等效共模、安全目标 SIL 2 的上限 10^{-2}。逐项折算的结果是：量化 0.29 LSB 占 29%，前端噪声 1.00 LSB 占 100%，抖动 0.06 LSB 占 6%，共模 0.20 LSB 占 20%，定点 0.05 LSB 占 5%。

换成百分比之后，“谁占大头”才有意义。原始数字里 0.29 与 0.20 看着差不多，换成百分比是 29% 与 20%，排序不变；但只要把共模从前端 80 dB 降到 60 dB，它就变成 2.05 LSB、205%，一项就把预算撑爆。**百分比的作用是让“超支”变成可见的。**

折算完之后有三条合成规则，每条都对应一类常见的算错：

第一，有效值项按平方和，峰值项单独校核。 量化、前端噪声、定点误差是有效值，抖动、共模如果按峰值给，就要先换成有效值再进平方和，或者干脆单独校核它的峰值不超过 1 LSB。混着加会把结果抬高 1.4 倍左右。

第二，对数量的“加”其实是乘。 $SE = A + R + B$ 是 dB 相加，等价于衰减倍数相乘。反过来，如果有人把 120.9 dB 与 0.20 LSB 相加得到“121.1”，这个数没有任何含义，因为一个是衰减比、一个是电压。

第三，多条路径并联时取最小的那个，不是相加。 材料 120.9 dB、缝隙 63.5 dB，实际屏蔽由 63.5 dB 决定，因为能量从最弱的路径全漏过来了。**dB 相加只适用于“同一路径上的串联衰减”，不适用于“多条路径并联”**，这一条与木桶原理同构，但很容易在算 EMC 时搞反。

按这三条规则合成本章的五项：$\sqrt{0.29^2 + 1.00^2 + 0.06^2 + 0.20^2 + 0.05^2} = 1.06$ LSB，即占用预算的 106%。**超了 6%**，严格的读法是“不通过”。工程上的处理有两条：把前端噪声压到 0.9 LSB（合计降到 0.97 LSB），或者接受这 6% 并写进验收记录，理由是它落在测量不确定度之内。本章选后者，但在表 92-14 里把这一行标成“临界”。

最后要说清：**量纲统一的目的是让“加”这个动作有物理意义**，不是为了把表填满。一个不能相加的表，比一张空表更危险，因为它会给出一个看起来很精确的错误结论。

92.2.9 无噪声码：6.6σ 这个系数从哪来

§92.1.9 说“分辨率是刻度、重复性是手抖”。当手抖的幅度接近刻度时，末几位就变成随机数。衡量这一点的量叫**无噪声码位数**：

$$N_{\mathrm{nf}} = \log_2 \frac{R}{6.6\,\sigma}.$$

分母里的 6.6 不是拍出来的，它等于 2×3.3：如果要求“噪声峰值不超过半个台阶”的概率达到 $1 - 10^{-3}$ 量级，单侧就要 3.3 个标准差，两侧共 6.6 个。换成更严的 $1 - 10^{-6}$，系数是 9.8；换成宽松的 $1 - 10^{-1}$，系数是 3.3。

这个系数直接决定“标称位数”与“有效位数”的差。本章 12 位、10 V 量程的台阶 2.441 mV，若噪声是 0.5 LSB（1.221 mV），$N_{\mathrm{nf}} = \log_2(10/8.06) = 10.28$ 位；噪声 1 LSB 时 9.28 位；2 LSB 时 8.28 位；4 LSB 时 7.28 位。**噪声每翻一倍，无噪声码掉一位**，这条规律比 6.02 dB 那条更好记。

注意无噪声码与 ENOB 是两个不同的量。ENOB 由 SINAD 反解，本章 0.5 LSB 噪声时约 11.21 位，比无噪声码多 0.93 位。差别来自统计口径：ENOB 是均方根意义下的等效位数，允许末位偶尔跳；无噪声码是“末位永不跳”的位数，要求更严。**给客户报指标时要说清是哪一个**，两者差一位，验收时就可能吵起来。

还有一个实用推论：如果现场实测 ENOB 是 10.8 位，那么无噪声码大约 9.9 位，能稳定读出的只有 10 位。**做控制算法时按 10 位设计，不要按 12 位设计**，否则微分项会被末位噪声放大成尖锐的毛刺。

92.3 设计步骤

五本账不能同时算，得排一个顺序。顺序错了，后面全是返工。本章推荐的顺序是：**量程 → 噪声预算 → 定时预算 → EMC 与接地 → 安全与失效**。前两步定“看得清”，第三步定“来得及”，第四步定“扛得住”，第五步定“坏得起”。

92.3.1 第一步：量程与台阶

先定量程，再定位数，顺序不能反。量程由传感器输出范围和可能的过载决定：本章的传感器满量程 ± 5 V，考虑 20% 过载后取 $R = 10$ V。量程定完，一个台阶就是 $\Delta = R/2^{N}$，选 N 等于选“最小可分辨变化”。

反推的式子是

$$N \geq \log_2 \frac{R}{\Delta_{\text{allow}}},$$

Δ_{allow} 是工艺允许的最小可分辨变化。若工艺要求看清 5 mV，$N \geq \log_2(10/0.005) = 10.97$，取 11 位；若要求 2 mV，$N \geq 12.29$，取 13 位。**这里最容易犯的错是把“量程”当成“信号范围”**：信号只用到 ± 1 V，量程却按 ± 10 V 选，等于白白丢掉 3.3 位，因为台阶大了十倍。

92.3.2 第二步：噪声预算

有了台阶，就把五项误差全折成 LSB，见表 92-4。折算的规矩只有一条：**同一量纲、同一统计口径**，有效值对有效值，峰值对峰值，不能混。

表 92-4 五项误差折算到 12 位、10 V 量程的 LSB

误差项	物理值	折算 LSB	备注
量化	$\Delta/\sqrt{12} = 0.705$ mV	0.29	理论下限，无法消除
前端噪声	$\sigma = 2.441$ mV（1 LSB）	1.00	决定实际有效位
采样抖动	1 Hz、$\sigma_t = 200$ μs 时 0.146 mV	0.06	与信号斜率成正比
共模（80 dB）	5 V 共模折 0.5 mV	0.20	60 dB 时变 2.05
定点系数（Q15）	最大 0.118 mV	0.05	与滤波器结构有关
合计（均方根）	—	1.06	换 60 dB 共模则 2.31

合计按均方根相加：$\sqrt{0.29^2 + 1.00^2 + 0.06^2 + 0.20^2 + 0.05^2} = 1.06$ LSB。表里最响的一项不是量化，是前端噪声（1.00 LSB）；把前端换成 60 dB 共模，合计涨到 2.31 LSB，**共模一项就顶掉一位多的分辨率**。这张表解释了一个常见现象：12 位 ADC 买回来，实测只有 10 位出头，问题往往不在 ADC。

预算的判据是“合计不超过 1 LSB（有效值）”。超过时按贡献排序砍，先砍最大的那一项——本章的顺序是前端噪声、共模、量化、抖动、定点。**不要平均用力**，因为均方根相加时，最大项占的份额是平方关系：1.00 这一项占 $1.00^2/1.13 = 88\%$。

92.3.3 第三步：定时预算

定时预算分两层：任务本身要跑多久，以及这个“多久”的尾部有多长。

第一层用中位数和占用率。$t_{50} = 301$ μs，周期 2 ms 时占用 15.0%，1 ms 时 30.1%，0.5 ms 时 60.1%。工程上建议稳态占用不超过 50%–60%，给中断、通信和偶发任务留空间。

第二层用分位数。$T_s = 2$ ms 时 P99.99 = 1.03 ms，还不到周期的一半，超时率实测为零；$T_s = 1$ ms 时 P99.99 已经超过周期，超时率 2.60×10^{-4}，936 次/小时。**判据应当是** $T_s \geq 1.5\, t_{99.99}$，本章 1.03 ms 的 1.5 倍是 1.55 ms，向上取

到 2 ms。图 92-4 把分布与三条周期线画在一起，可以看到 2 ms 线落在尾部之外，1 ms 线切进尾部。

图 92-4 执行时间的分布与三条候选周期：2 ms 落在尾部之外，1 ms 切进尾部，0.5 ms 切掉大半个分布

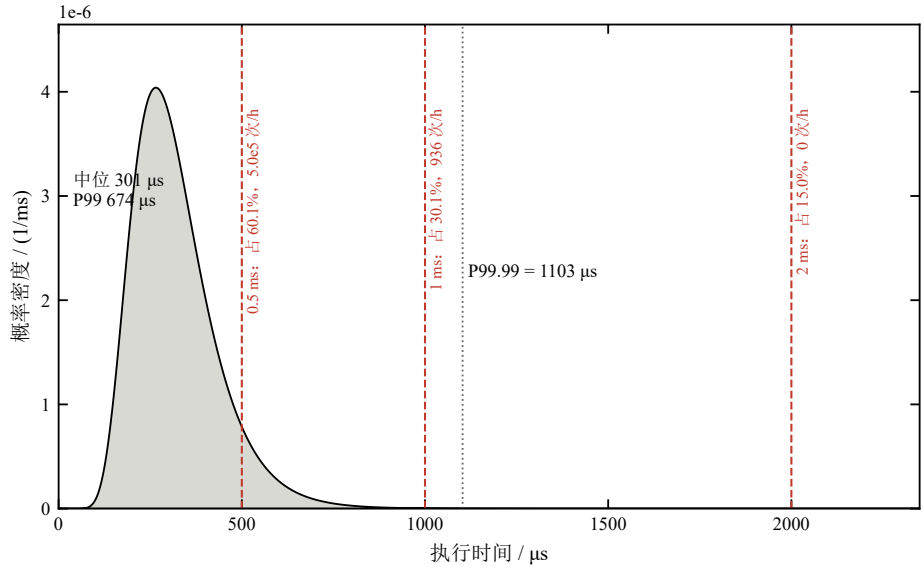


图 92-4

92.3.4 第四步：EMC 与接地

这一步的顺序是**先布线、再滤波、后屏蔽、最后加厚**，理由是成本递增而收益递减。

先布线：信号线与动力线分开走，模拟地与数字地单点连接，屏蔽层的两端接地。这一条不花钱，能解决大部分共模问题。

再滤波：ADC 前端加 RC 或共模扼流圈，把 1 MHz 以上的成分挡在采样器之前。本章 1 MHz 的干扰若不经滤波，混叠后会落到基带（第 85 章 §85.1.2 已算过折叠）。

后屏蔽：按缝隙算，不按材料算。100 mm 缝在 1 MHz 只有 63.5 dB，把缝压到 10 mm 就是 83.5 dB，压到 1 mm 就是 103.5 dB——**缝长每减十倍，屏蔽涨 20 dB**。

最后加厚：0.1 mm 换 0.5 mm，1 MHz 从 120.9 dB 升到 173.9 dB，涨的 53 dB 全在吸收项。这一步最贵、收益最“虚”，因为实际屏蔽早就被缝卡在 63.5 dB 了。

92.3.5 第五步：安全与失效

安全账最后算，因为它依赖前面四步的结果：量程与位数决定“能不能检出异常”，定时预算决定“多久能发现任务死了”。

看门狗的三个时间是串联的：检测（超时）100 ms、恢复（启动 250 ms + 初始化 50 ms）300 ms、失联合计 400 ms。400 ms 能不能接受，要看工艺：本章对象时间常数 0.5 s，400 ms 内被控量漂移不到一个量程的 10%，可以接受；若对象是 20 ms 时间常数的伺服，400 ms 早就飞了，只能靠硬件看门狗把恢复压到几十毫秒。

诊断覆盖率按 §92.2.6 折算。本章基准 90% 对应 SIL 2，要做到 SIL 3 必须 99%，代价是自检代码、双通道比较、RAM/ROM 巡检——**这是软件成本，不是硬件成本**，也是安全账与软件工程（第 90 章）唯一的接口。

92.3.6 为什么顺序不能颠倒

五步颠倒任何一步，都会返工：

- 先定位数后定量程，等于把台阶算错，噪声预算全部作废；
- 先做屏蔽后布线，布线的共模问题会被屏蔽“掩盖”，下次改板又冒出来；

- 先算安全后算定时，看门狗超时值没有依据，只能拍脑袋；
- 先做定点后做噪声预算，不知道要几位小数，只能凭经验取 Q15。

一句话：**前一步的输出是后一步的输入，而最后一步（安全）要回头验证第一步（量程）够不够检出故障。**

92.3.7 从工艺指标反推五本账：一个完整算例

前面的五步讲的是“每一步怎么做”，这里把它们串成一次完整的反推，看看账是怎么互相否掉对方的。

工艺条件有六条：被控量允许稳态纹波 5 mV（峰峰值）、允许失控时间 200 ms、安全目标 SIL 2、现场共模 5 V、主要干扰源是 1 MHz 的变频器、机柜最大缝长 100 mm。

第一，从纹波反推台阶。 稳态纹波由量化与噪声共同托底，取“台阶不超过纹波的一半”，即 $\Delta \le 2.5$ mV。由 $N \ge \log_2(10/0.0025) = 11.97$ 取 12 位，$\Delta = 2.441$ mV，刚好卡住。注意这里只用了 $N \ge \log_2(R/\Delta)$，没有考虑噪声——台阶只是下限的一半。

第二，做噪声预算。 按表 92-4 的五项，前端噪声 1 LSB、共模（80 dB）0.20 LSB、量化 0.29 LSB、抖动 0.06 LSB、定点 0.05 LSB，合计 1.06 LSB。纹波按“台阶托底 + 噪声”估，峰值约 2.4 mV + 1.06×2.441 mV ≈ 5.0 mV，正好贴着工艺上限，没有余量。此时有两条路：把前端噪声压到 0.9 LSB（换低噪声运放，几元钱），或把共模压到 0.1 LSB（前端提到 86 dB，要仪表放大器）。本章选前者，因为便宜。

第三，定周期。 中位数 301 μs，按占用率 60% 上限得 $T_s \ge 502$ μs；按尾部 $1.5\,t_{99.99}$ 得 $T_s \ge 1548$ μs。取 2 ms，占用 15.0%，超时率为零。到这里定时账是过的。

第四，算失控时间与安全。 这一步把上一步的结论否掉了。工艺允许失控 200 ms，而看门狗的“检测 100 ms + 恢复 300 ms”是 400 ms，**整整超一倍**。想把 400 ms 压到 200 ms 以内，只能两条路同时走：检测门限压到 50 ms，恢复时间压到 150 ms（省掉外设初始化，靠热复位），合计 200 ms，刚好卡线且没有余量。工程上不能这么签，正确的做法是加一条硬件安全输出，让“软件死了但硬件还能断输出”这条路径独立成立，软件侧的 400 ms 就不必承担安全功能。

这一条最能说明“五本账必须一起算”：前三步都过得去，第四步回头把第三步的定时方案否掉了，而解决办法不在定时账里，在安全账里。

第五，EMC 的量级。 1 MHz、100 mm 缝，缝隙上限 63.5 dB。若现场干扰在输入端的等效电压是 10 mV，衰减后剩 6.7 μV，折 0.003 LSB，可以忽略；若是 1 V，剩 0.67 mV，折 0.27 LSB，就与共模项同量级了。**所以“屏蔽够不够”不能只看 dB，要看干扰源有多强**：同样的 63.5 dB，对 10 mV 的源是多余的，对 1 V 的源是刚刚够。

第六，把结果填表。 五步的结果填进表 92-14 的十行，得到一张可签字的基线。这张基线里有两行是“不通过”或“临界”：第 6 行的合计 1.06 LSB 超了 6%，第 9 行的 400 ms 超了工艺允许值一倍。**带着红色标记进入下一轮设计**，而不是用平均值把它们抹平——这是本章对“工程判断”的全部要求。

这个算例还说明一件事：**反推不是单向的**。从工艺推到参数，再从参数回头校核工艺，往往要来回两三次才能收敛。凡是“一次就算对”的预算表，多半是某一项漏了。

92.3.8 预算什么时候必须重算

预算表不是一次性的，它有三个“失效触发条件”，碰到任何一个就要整表重算，而不是改一格。

第一，量程变了。 台阶 $\Delta = R/2^N$ 一变，所有折成 LSB 的项全部失效——共模 0.20 LSB 可能变成 2.05 LSB。换传感器、加过载保护、改接线方式，都会动量程。**这是最容易被忽略的触发条件**，因为量程写在采购单上，改动往往由机械或工艺提出，电气侧最后一个知道。

第二，前端换了。 前端噪声决定实测 ENOB，也决定共模抑制。换一颗运放、加一级滤波、改单端为差分，都要重测 ENOB 与共模折算。**手册上的噪声密度不能直接当 1 LSB 用**，它要乘带宽、再乘增益。

第三，周期或工艺节拍变了。 周期一变，占用率与超时率同时变，而超时率对周期极其敏感（§92.6.3）。工艺节拍变化还会改变“允许失控时间”，从而改变看门狗门限与 SIL 目标。

三个条件之外，还有两个“软条件”值得留意：换了机柜或柜门结构（缝长变了），以及换了执行机构（分辨率可能比 ADC 还低，见 §92.4.8）。这两项不动电气参数，却能把能量账整个翻过来。

工程上的做法是给预算表加一列“假设”，把量程、前端型号、周期、缝长四件事写在表头。**四个假设里任何一个被改动，这张表就作废。**这一条比表里任何一个数字都重要，因为一张过期却看起来精确的表，会把错误传到下一版设计里。

92.3.9 填完表之后的三问

预算表填完，先别急着签字，用三个问题自检一遍。

第一问：最大项是谁，它占多少？ 答不上来，说明表里的数没有归一化。本章最大项是前端噪声 1.00 LSB，占平方和的 88%，所以后续所有优化都要先碰它。如果最大项是量化，方向就完全不同——那意味着前端很干净，该做的是加位数或过采样。

第二问：哪几行是红或黄？ 全绿的表不可信，通常是判据写松了或者数据抄了手册。本章的表里有两行是临界：合计 1.06 LSB、失联 400 ms。**这两行必须在下一轮里被处理，或者被明确接受并记录理由，**不能留着不管。

第三问：换一个工况，哪几行会变？ 冬季与夏季、空载与满载、单机与联网，噪声与执行时间都会变。答不出“哪几行敏感”，说明这张表只在实验室成立。本章的答案很明确：温度影响前端噪声与电源纹波，负载影响执行时间尾部，联网影响通信延迟。

三问都答得上来，这张表才算能用。**答不上来的那一问，就是下一次返工的位置。**

92.4 连续情形

要算“这些误差进到回路里会变成什么”，最省事的办法是先回到连续域：把数字侧的误差翻译成等价的连续扰动，再用第 3、4 章的传递函数算它的传播。翻译得准不准，决定了后面所有估计的可信度。

92.4.1 三种翻译

- **量化误差翻译成白噪声。**误差在 $[-\Delta/2, \Delta/2]$ 上均匀，功率 $\Delta^2/12$，在 0 到 $f_s/2$ 内近似平坦，单边功率谱密度 $2\Delta^2/(12 f_s)$。位数越高、采样越快，谱密度越低。
- **采样抖动翻译成乘性扰动。**$\Delta y = \dot{y}\,\delta$ 说明误差正比于信号导数，等价于在信号通路上乘了一个随机增益 $1 + \delta\,\mathrm{d}/\mathrm{d}t$。它不是加性噪声，信号为零时它也为零。
- **EMC 与共模翻译成加性电压源。**传导干扰是串在输入回路里的电压源，共模是同时加在两根输入线上的同相电压源，辐射则通过缝隙耦合进来，最终也折算成输入端的等效电压。

三种翻译的性质完全不同：第一种与信号无关，第二种与信号同比例，第三种与干扰源有关。**把三者都当“白噪声”处理，是连续近似最常见的错。**

92.4.2 量化噪声进闭环

把量化噪声 n_q 加在测量端，闭环输出是

$$y_n = S(s)\,n_q, \qquad S(s) = \frac{1}{1 + L(s)},$$

S 是灵敏度函数。低频段 $|L| \gg 1$，$|S| \approx 1/|L|$，量化噪声被压掉；穿越频率附近 $|S| \approx 1$，噪声原样通过；高频段 $|S| \to 1$，噪声直通。

这条性质解释了为什么积分作用能压住量化的稳态偏置却压不住高频抖动：**积分器只在低频有增益**。本章 $K_p = 2.0$、$T_i = 0.5$ s 的 PI 配 $G(s) = 1/(0.5s + 1)$，在 0.1 Hz 处环路增益约 6.37，$|S| = 0.155$，压 16.2 dB；到 1 Hz 环路增益只剩 0.636，$|S| = 0.844$，只压 1.5 dB；过了 0.6 Hz 的穿越频率，$|S|$ 趋于 1，噪声原样通过。要让 0.1 Hz 处再降 20 dB，只能把增益提高十倍，而那会牺牲裕度——这就是“精度与稳定”的第一处正面冲突。

92.4.3 抖动进闭环

抖动是乘性的，闭环里它表现为“控制器看到的误差多了一项 $\dot{y}\delta$”。对本章 1 Hz、幅值 1 的正弦，输出最大斜率实测 3.698 /s，$\sigma_t = 200$ μs 时预测偏差 $3.698 \times 200 \times 10^{-6} = 0.740$ mV，实测最大偏差 0.517 mV，同量级。**抖动的量级由斜率定，不由环路定**，环路只能把它平均掉一部分。

这也解释了 §92.1.1 的现场现象：慢过程（1 Hz 以下、斜率小）对抖动几乎不敏感，抖动预算可以放到微秒级；而电流环、振动台这类高频高斜率对象，抖动预算常常压到几十纳秒，比 ADC 的孔径时间还严。

92.4.4 屏蔽效能的三项分解

表 92-5 是 0.1 mm 铜在五个频率上的分解，最后一列是同尺寸 100 mm 缝隙的泄漏上限。

表 92-5 0.1 mm 铜的屏蔽效能分解与 100 mm 缝隙上限

频率	吸收 A	反射 R	多次反射 B	合计 SE	缝隙上限
10 kHz	1.3 dB	128.1 dB	-11.7 dB	117.8 dB	103.5 dB
100 kHz	4.2 dB	118.1 dB	-4.2 dB	118.1 dB	83.5 dB
1 MHz	13.1 dB	108.1 dB	-0.4 dB	120.9 dB	63.5 dB
10 MHz	41.6 dB	98.1 dB	-0.0 dB	139.7 dB	43.5 dB
100 MHz	131.4 dB	88.1 dB	0.0 dB	219.6 dB	23.5 dB

读这张表要抓住三条。**第一，低频靠反射，高频靠吸收**：10 kHz 时反射 128.1 dB、吸收只有 1.3 dB；100 MHz 时反射降到 88.1 dB、吸收涨到 131.4 dB。**第二，多次反射在薄板低频为负**，因为板比趋肤深度薄，场在两面之间来回反弹，反而增强了透射，10 kHz 时 -11.7 dB 不能忽略。**第三，缝隙上限随频率单调下降**，每十倍频程掉 20 dB，100 MHz 时只剩 23.5 dB——**高频干扰几乎完全由缝隙决定，材料再厚也没用**。

图 92-5 把三条分量画成堆叠柱，再叠上缝隙上限的折线。图上最直观的一幕是：10 kHz 时缝隙上限（103.5 dB）还高于材料（117.8 dB），材料是瓶颈；到 1 MHz 缝隙（63.5 dB）已经比材料低 57 dB，瓶颈彻底换成缝。

图 92-5 屏蔽效能的三项分解与缝隙泄漏上限：低频由反射主导，高频由吸收主导，缝隙上限每十倍频程下降 20 dB

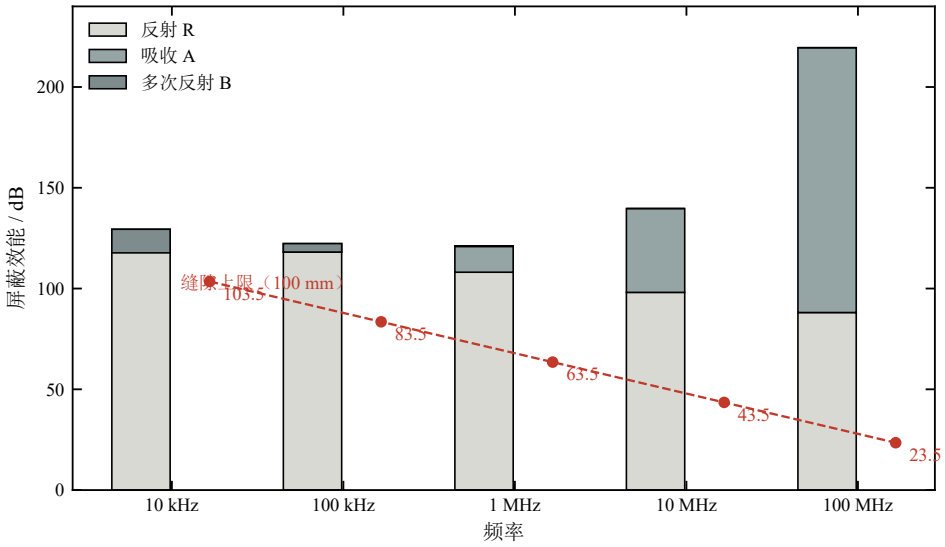


图 92-5

92.4.5 共模电压的连续等效

共模干扰在连续域的模型很简单：两根输入线上叠加同一个电压源 $v_{cm}(t)$，前端把它按 CMRR 折算成差分误差。表 92-6 是两组共模电压、四档 CMRR 的折算结果。

表 92-6 共模电压经 CMRR 折成的差分误差与 LSB

共模电压	CMRR 60 dB	80 dB	100 dB	120 dB
0.5 V	500 μV / 0.20 LSB	50 μV / 0.02 LSB	5 μV / 0.00 LSB	0.5 μV / 0.00 LSB
5 V	5000 μV / 2.05 LSB	500 μV / 0.20 LSB	50 μV / 0.02 LSB	5 μV / 0.00 LSB

表的读法是“电压 / 折算 LSB”。**CMRR 每加 20 dB，误差降十倍，正好抵掉共模电压涨十倍**：5 V 配 80 dB 与 0.5 V 配 60 dB 都是 0.20 LSB。这条对称性说明，选前端时不必追求极高 CMRR，只要让“共模电压/抑制倍数”落在同一个量级即可。

图 92-6 把这张表画成四条等值线，横轴 CMRR、纵轴共模电压，等值线标 0.1、0.5、1、2 LSB。工程用法是：先在纵轴上找到现场实测的共模电压，再横着往右走，碰到 0.5 LSB 那条线，对应的 CMRR 就是最低要求。本章 5 V 共模对应 $20\lg(5/1.221\times10^{-3}) = 72.3$ dB，选 80 dB 的前端留 8 dB 余量。

图 92-6 共模电压与 CMRR 的等误差线：每条线标出折算到 12 位 10 V 量程的 LSB 数

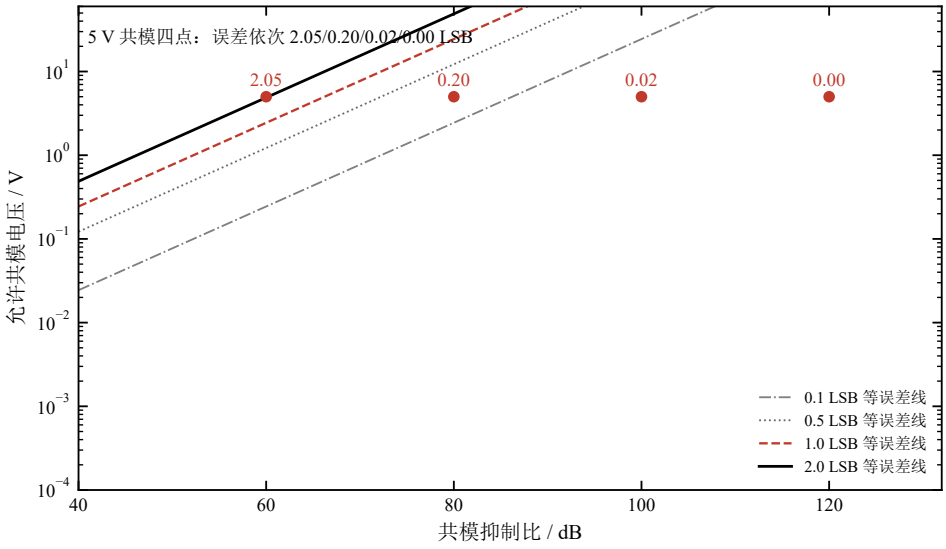


图 92-6

92.4.6 连续近似在哪儿失效

连续近似有三处会给出过于乐观的结论，必须回到离散域：

- **混叠**。连续域里 1 MHz 的干扰会被模拟滤波器衰减；离散域里它折回基带，衰减量由抗混叠滤波器在 $f_s - f_0$ 处的值决定，而不是在 1 MHz 处的值。
- **相关误差**。连续白噪声假设误差独立；实际量化误差在直流或周期信号下是确定的，会在稳态留下固定偏置或极限环（§92.5.1）。
- **一次一拍**。连续域没有“这一拍用上一拍的值”这回事，而执行时间超时恰恰造成这种效应（§92.6.2）。

所以本章的用法是：**连续域算量级，离散域定判据**。先用连续式估出“大概几个 LSB”，再用离散仿真确认“最坏情况几个 LSB”。

92.4.7 反馈到底压掉多少量化噪声

§92.4.2 说的是“低频被压、高频直通”，但没回答总量。量化噪声的功率均布在 0 到 $f_s/2$ 上，闭环只在 $|S| < 1$ 的那一段把它压小，而这一段占的带宽比例是 $f_c/(f_s/2)$。

本章 $f_s = 10$ kHz，穿越频率约 0.6 Hz，比例是 $0.6/5000 = 1.2 \times 10^{-4}$。也就是说，反馈只改动了万分之一点二的噪声带宽，**量化噪声的总功率几乎不变**。这个结论与“积分器能压量化噪声”的直觉相反，原因是直觉只看了低频那几个频点，没看总量。

准确的说法是：**反馈改变的是量化噪声的频谱形状，不是总功率**。它把低频段压下去，把同样的功率挤到穿越频率以上，总量在线性系统里守恒——因为 $|S|^2 + |T|^2 = 1$，T 是补灵敏度函数，压低一处必然抬高另一处。所以想真正降量化噪声，只有三条路：加位数、过采样、加抖动改变误差性质，三条都在 ADC 侧，不在控制器侧。

这个结论有一个很实用的推论：**靠提高采样率来“帮控制器压噪声”是无效的**。采样率提高十倍，噪声带宽从 5 kHz 变成 50 kHz，单边谱密度降十倍，但闭环能压的那一段还是 0.6 Hz，占比从 1.2×10^{-4} 降到 1.2×10^{-5}，总量不变；真正变了的只有混叠位置，而那是另一本账（§92.9.2）。

反过来，如果对象带宽很宽，结论就翻转。电流环的穿越频率常在 2 kHz，比例变成 $2000/5000 = 0.4$，反馈能压掉四成带宽内的噪声，“控制器压噪声”才真正成立。**同一条性质，在慢回路里是心理安慰，在快回路里是真收益**，差别全在 $f_c/(f_s/2)$ 这个比值上。本章是 1.2×10^{-4}，属于前者；电力电子的电流环是 0.4，属于后者。

92.4.8 幅值账的另一半：执行机构

前面所有讨论都在输入端，好像误差只从 ADC 进来。实际上回路是闭合的，输出侧也有台阶，而且往往更粗。

本章控制器输出 0–10 V、12 位，台阶 2.441 mV，看起来与输入侧一样细。但接到阀门定位器上就不一样了：定位器的分辨率常是满量程的 0.1%，即 10 mV，折成控制器输出的 4.1 LSB；回差（hysteresis）也是 0.1% 量级，同样是 10 mV。**输入侧算得再细，输出侧一个 10 mV 的死区就把分辨率封死了。**

定量地看：输出台阶 10 mV 经对象增益 1 传到被控量，就是 10 mV 的稳态偏差；而工艺允许的纹波是 5 mV 峰峰值。**输出侧一项就超了两倍**，前端把 ENOB 从 12 提到 16 也毫无意义。

这就引出一条本章没有明说但很关键的规则：**幅值账要在“最粗的那一环”上结算**。输入端 2.441 mV、输出端 10 mV、被控量检测 2.441 mV，三者取最大，就是 10 mV，全通道的分辨率由它决定。工程上常见的配置是 4–20 mA 电流环配智能定位器，定位器分辨率 0.05%（5 mV），恰好与 12 位 ADC 匹配——**这不是巧合，是几十年工程磨出来的配平**。

所以做预算时要把执行机构写进表 92-4。它不属于“测量误差”，但它决定了闭环能分辨多小的指令变化。漏掉这一项，会得到“测量很准、控制很粗”的怪结果，而且很难从数据上看出原因——因为测量数据本身是干净的。

92.4.9 孔径时间与建立时间：两个常被漏掉的时间

时间账里通常只算抖动，漏掉两个同样重要的时间：采样保持器的孔径时间，和前端 RC 的建立时间。

孔径时间 t_a 是采样开关从导通到断开的那一段不确定时间。它不对应时钟抖动，而是开关本身的物理过程，典型值 50 ps 到 1 ns。它的效果与抖动完全相同：误差等于 $\dot{y}\,t_a$。所以它应当**并入抖动预算**，而不是单独列一项。本章的 1.1 ns 合成抖动里，就包含了 1 ns 的分配偏斜与百 ps 级的孔径时间。

建立时间更常被忽略。ADC 采样之前，前端 RC 必须把电压建立到半个台阶以内，否则采到的值还没到位。建立到 2^{-13}（12 位加半个台阶的余量）需要

$$\frac{t_{\mathrm{acq}}}{\tau} \geq \ln 2^{13} = 9.01,$$

也就是九个时间常数。本章前端取 1 kΩ 与 10 nF，$\tau = 10\ \mu\mathrm{s}$，需要 90 µs 的采集时间；而 10 kHz 采样的周期只有 100 µs，**刚好卡住，一点余量都没有**。

这条算式解释了两件事。**第一，为什么提高采样率会突然让精度掉下来**：周期从 100 µs 缩到 50 µs，采集时间不够，采到的值差几个 LSB，而且误差随信号斜率变化，看起来像“随机的坏点”。**第二，为什么前端加 RC 滤波要小心**：为了抗混叠把电容从 10 nF 加到 100 nF，τ 变成 100 µs，10 kHz 采样根本建立不起来。

工程判据是：$t_{\mathrm{acq}} \geq 9\tau$，且 $t_{\mathrm{acq}} \leq 0.8/f_s$。两条同时满足，前端才算配平。**这条判据与采样定理无关**，它管的是模拟侧，而采样定理管的是数字侧，两者经常被混为一谈。

92.5 离散情形

连续近似一旦落到离散域，三件事会变：量化误差不再独立、抖动要与采样时刻一起算、定点运算会额外引入误差。这一节把它们逐条算清楚。

92.5.1 量化误差并不独立

“误差在 $[-\Delta/2, \Delta/2]$ 上均匀独立”只在输入足够“乱”时成立。如果输入是直流、或与采样率成整数比的周期信号，量化误差就是输入的确定函数：直流输入 0.3 LSB，量化后固定输出 0 LSB，误差恒为 -0.3 LSB，不随机。

后果有两个。**一是固定偏置**：控制器积分器会把这个偏置积到零误差为止，于是稳态被拉偏 0.3 LSB，且无法通过平均消除。**二是极限环**：当输入缓慢变化、误差在 $\pm\Delta/2$ 间跳变时，闭环可能进入一个周期性的小幅振荡，幅度约一个台阶。第 85 章 §85.9.11 算过定点实现的死区问题，机理相同。

工程上有三条对策：加抖动把确定性误差打散、把积分器做成增量式、或者在 A/D 之后加一级数字滤波。三条都不改变台阶本身，只改变误差的性质。

92.5.2 抖动的两种用法

这个词在本章有两个意思，必须分清。

采样时刻抖动（timing jitter）是 ADC 的时钟不稳，误差按 $\dot{y}\delta$ 折算，是本节 §92.5.3 的主题，属于“坏事”。

量化抖动（dither）是故意在量化前加一个小噪声，把确定性误差打散成随机误差。它是“好事”，代价是总量变差：本章 12 位加均匀抖动后，信噪比从 74.10 dB 掉到 70.99 dB，掉 3.11 dB。理论上均匀抖动注入的噪声功率是 $\Delta^2/12$，与量化噪声同量级，两者叠加正好使总噪声翻倍，即 -3.01 dB；实测 -3.11 dB，多出来的 0.10 dB 是抖动幅度取整带来的。

花 3 dB 换误差的独立性，是一笔划算的买卖，前提是回路对偏置敏感、对宽带噪声不敏感——积分器恰好满足这个条件。

92.5.3 采样抖动进闭环

把采样抖动加进闭环，测“有抖动”与“无抖动”两次运行的逐点差，得到的是纯粹的抖动效应，跟踪误差被减掉了。表 92-7 是四档抖动的结果。

表 92-7 采样抖动进入闭环后的输出偏差

σ_t	最大偏差	折算 LSB	均方根偏差	预测 $\dot{y}_{\max}\sigma_t$	IAE 变化
20 μs	0.115 mV	0.05	0.059 mV	0.074 mV	-0.0%
50 μs	0.187 mV	0.08	0.069 mV	0.185 mV	-0.0%
100 μs	0.296 mV	0.12	0.090 mV	0.370 mV	-0.0%
200 μs	0.517 mV	0.21	0.146 mV	0.740 mV	-0.0%

三点值得注意。**第一，实测偏差约为预测值的一半到七成**，因为 $\dot{y}$ 的最大值只在过零附近出现，而抖动误差的正负号随机，闭环又把它平均掉一部分。**第二，偏差随 σ_t 线性增长**：20→200 μs 涨十倍，偏差从 0.115 到 0.517 mV 涨 4.5 倍，接近线性。**第三，IAE 几乎不变**（表中显示 -0.0%），因为 IAE 的量级是 6.5，抖动偏差只有 10^{-4}，被完全淹没。

最后这一点最有工程意义：**抖动在小信号上是大问题，在大信号上是小问题**。所以判据不能只看 IAE，要看“稳态时被控量允许多少纹波”。200 μs 抖动的 0.21 LSB 纹波，对 12 位系统可以接受；对 16 位系统就是 3.4 LSB，不能接受。

图 92-7 把实测与预测两条线画在一起，横轴 σ_t 用对数刻度。两条线平行，说明“斜率×抖动”这个一阶模型足以定预算；差值就是闭环的平均效应。

图 92-7 抖动进入闭环的输出偏差：实测与“斜率×抖动”预测同量级，IAE 几乎不变

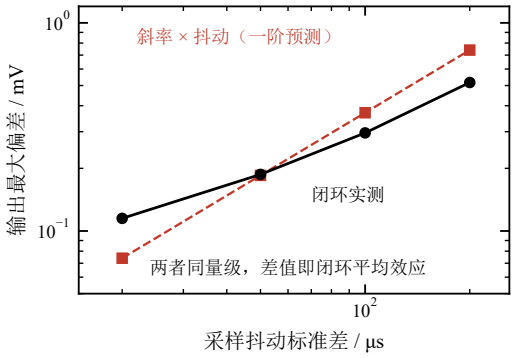


图 92-7

92.5.4 过采样与平均的实测

表 92-8 把三种“提精度”的手段放在一起，都以 12 位、10 V 量程为基准。

表 92-8 三种提高分辨率手段的实测对比

手段	实测 SNR	等效位数	相对基准
12 位直接采样	74.10 dB	12.02 位	基准
12 位 + 16 倍过采样平均	85.33 dB	13.88 位	$+11.23$ dB，$+1.86$ 位
12 位 + 均匀抖动	70.99 dB	11.50 位	-3.11 dB，-0.52 位

过采样少掉的 0.14 位（0.81 dB）来自误差的相关性。16 个高速样本跨 100 μs，对 997 Hz 的正弦正好是 36°，信号在窗内不是常数，量化误差随斜坡呈锯齿状，平均后不能按 $1/16$ 完全抵消；理论 12.04 dB 只拿到 11.23 dB，差 0.81 dB。测试音越高、窗越宽，这项损失越大；若信号在窗内近似常数，实测就会接近理论的 2 位。**这也是过采样不能无限做的原因**：窗宽到信号不再是“缓变”时，平均就从“降噪”变成“低通”。

抖动那一行是负收益，但如 §92.5.2 所说，它换的是误差的独立性，不能只看这一列。

92.5.5 定点化的实测与界

表 92-9 是二阶巴特沃斯低通（100 Hz、1 kHz 采样）在四档系数位宽下的结果。

表 92-9 定点系数量化的误差与 L1 误差界

系数格式	实测 SNR	最大误差	L1 误差界	界/实测
Q10	49.6 dB	3.807 mV	7.430 mV	1.95
Q12	77.9 dB	0.137 mV	0.329 mV	2.40
Q15	79.9 dB	0.118 mV	0.227 mV	1.92
Q20	131.6 dB	< 0.5 μV	0.001 mV	—

这张表最容易误读的一点是：**Q12 到 Q15 加了三位，信噪比只涨 2.0 dB，不是 18 dB**。原因不是算错，而是舍入方向：Q12 时几个系数的误差恰好部分抵消，Q15 时反而没有。**系数误差的抵消是“碰运气”的，不能按每位 6 dB 线性外推**，这也是为什么定点化必须实测，而不能只查手册。

第二点是 Q10 的 SNR 只有 49.6 dB，比 12 位 ADC 的 74.10 dB 低 24.5 dB。**滤波器成了整个通道的瓶颈**——ADC 再好，系数位宽不够也白搭。这条经验在工程上非常实用：定点化时先把滤波器的等效 SNR 算出来，与 ADC 的 SNR 比，两者相差不应超过 6 dB。

第三点是界与实测的比例稳定在 1.9–2.4 之间。这说明 L1 界可以做验收下限（实测一定小于界），但不能当预期值（界偏保守一倍）。

92.5.6 抖动与量化谁主沉浮：一次分离测量

两项误差都能折成 LSB，但它们的“指纹”完全不同：量化误差取决于信号与台阶的比值，抖动误差取决于信号的斜率。利用这一点，两次测量就能把两者分开。

测纯量化：给一个直流输入，例如 0.3 LSB。此时 $\dot{y} = 0$，抖动完全不产生误差，测到的就是量化本身。本章的算例会给出固定输出 0 LSB、误差恒为 -0.3 LSB——注意它不是“平均为零的随机数”，而是一个确定偏置，平均一万次也不会变。

测纯抖动：给高频满幅正弦。量化误差相对信号很小（12 位满幅时 74 dB），而抖动误差按 $2\pi f A \sigma_t$ 折算，频率一高就压过量化。本章 1 kHz、5 V 峰值、$\sigma_t = 200$ μs 时，抖动误差的峰值是 $2\pi \times 1000 \times 5 \times 200 \times 10^{-6} = 6.28$ V，比整个量程还大，这时测到的几乎全是抖动的影响。

两次测完，现场的“跳码”就能归类：

- 若输入不变时也跳、且幅度总是整数个 LSB，是量化与干扰；
- 若跳变随信号变化率增大而增大、且在过零处最明显，是抖动；
- 若跳变只在特定相位出现、周期与工频一致，是共模。

本章的基准是 1 Hz 正弦，$\sigma_t = 200$ μs 时抖动偏差预测 0.740 mV（0.30 LSB），与量化 0.29 LSB 恰好同量级，这就是表 92-4 里两项都不可忽略的原因。而同样的 200 μs 放到 1 kHz 上，误差涨一千倍，量化项变得可以忽略。**所以“抖动大不大”必须先问“信号多快”，单独说“我的抖动是 200 μs”是没法判断的。**

还有一条实践细节：分离测量要在“信号静止”与“信号最快”两个极端做，不要在中间随便取一个工作点。中间点上两项同量级，测出来的数是两者之和，除非再解一次方程，否则分不开。

92.5.7 溢出的三种处理与本章的位宽

定点化最怕的不是舍入误差，是溢出。舍入只让结果差一点，溢出会让结果从最大正值跳到最大负值，控制器直接把执行器打到反向极限。

先算本章的位宽够不够。输入码值是 12 位，范围 ± 2048；滤波器系数 Q15，范围 ± 32768；乘积是 Q27，范围 $\pm 2^{15} \times 2^{12} = 2^{27}$，约 1.34×10^8，远在 32 位有符号整数（2.1×10^9）之内。三个系数累加，最大约 4×10^8，仍然不溢出。**所以本章的 Q27 中间结果是安全的，余量四倍多。**

但余量不是无限的。如果把输入换成 16 位、系数换成 Q30，乘积就是 Q46，2^{46} 直接超出 32 位，必须换 64 位或拆成两次乘法。这就是 §92.7.1 说的“N 每加 4，乘法代价涨几倍”的具体来源：不是乘法本身慢，是位宽跨过了寄存器宽度。

溢出之后有三种处理，性质完全不同：

- **回绕**：硬件默认，$2^{31} - 1$ 加一变成 -2^{31}。数学上正确，控制上灾难。
- **饱和**：钳到最大/最小值。误差有界，但会引入持续的最大输出，可能把积分器越积越深。
- **抗饱和**：输出饱和时冻结积分器（clamping 或 back-calculation）。这是控制上唯一正确的做法，代价是多几行代码。

本章的判据是：**中间结果留 4 倍余量，输出级做饱和，积分器做抗饱和**。三条缺一不可。只做饱和不做抗饱和，会遇到“退饱和要等好几秒”的经典现象——积分器在饱和期间积了一大堆，输出回落后还要慢慢退回来，这段时间里回路等于开环。

调试时怎么发现溢出？两个迹象：一是输出偶尔跳到反向极限且只持续一拍，二是同一个输入下重复运行结果不同。前者是回绕，后者是溢出后状态被污染。**看到这两种现象，先去查位宽，不要先怀疑算法。**

92.5.8 二阶比四阶好在哪：结构决定灵敏度

表 92-9 的 Q10 误差 3.807 mV，是同样系数位宽下 Q15 的三十二倍。这个放大不是位宽造成的，是**结构**造成的。

二阶滤波器的极点是一对共轭复数，当它靠近单位圆时，系数的一个小扰动会引起极点位置的大移动。极点半径 r 与系数误差 δa 的关系大致是 $\Delta r \approx \delta a / (1 - r)$。本章 100 Hz、1 kHz 采样的极点半径约 0.95，$1/(1-r) = 20$，系数误差被放大二十倍；Q10 的系数误差约 10^{-3}，放大后极点移动 2%，足以让幅频特性明显偏。

四阶以上用直接型更糟：高阶多项式的根对系数极其敏感，$r = 0.99$ 时放大一百倍，Q15 都可能把极点推出单位圆，滤波器直接发散。所以工程上有两条硬规矩：

- **八阶以上必须拆成二阶节（SOS）级联**，每节只管一对极点，灵敏度回到二阶的水平；
- **极点半径超过 0.99 时必须用 SOS**，哪怕只有四阶。

本章的二阶滤波器刚好在“直接型还能用”的边界上，Q15 的误差 0.118 mV 可以接受，Q10 的 3.807 mV 就不行。这也解释了为什么表 92-9 里 Q12 到 Q15 只涨 2 dB：两者的极点灵敏度相同，差别只在系数舍入的方向上，抵消与否是碰运气。

还有一个实用判据：**定点化后先算极点半径，再决定结构**。半径 0.95 用直接型，0.99 用 SOS，0.995 以上要考虑换更低截止频率或提高采样率。这个顺序比“先试 Q15，不行再试 Q20”快得多，因为它给出的是“必然可行”的结构，而不是碰巧能跑的位宽。

92.6 稳定性与性能

前面五节算的都是“误差有多大”，这一节算“误差大到什么程度会改变回路的性质”。答案分三层：量化改变的是稳态形状，执行时间改变的是相位裕度，安全机制改变的是失效后果。

92.6.1 量化与极限环

量化对稳定性的影响不能用线性理论算，因为量化是非线性环节。工程上用描述函数近似：把量化看成增益随输入幅度变化的环节，输入幅度远大于台阶时等效增益为 1，输入幅度接近台阶时等效增益下降，相位不变。

由此得到两条结论。**一是小信号增益下降**，闭环带宽在小信号时变窄，响应变慢；**二是可能产生极限环**，当环路在某个频率上的相位移刚好配上量化环节的“死区”时，输出会在 ± 1 个台阶内持续振荡。

本章基准的量化台阶 2.441 mV，实测无量化时纹波 1.7×10^{-11}，这个底噪远小于台阶，所以实际系统一旦加上 12 位量化，纹波就会被台阶“托”到 1–2 LSB，约 2.4–4.9 mV。这不是参数没调好，是台阶的下限。要压下去只有两条路：加位数，或加抖动把极限环打散成噪声。

92.6.2 执行时间如何吃掉裕度

执行时间对回路的影响有两层。**第一层是平均延迟**：从采样到输出，固定要花 $t_{50} = 301$ μs，在 20 ms 周期里占 1.5%，折成相位在 3.797 rad/s 处只有 0.066°，可以忽略。**第二层是超时**：某一拍算不完，输出保持上一拍的值，等效延迟从一拍变成两拍。

按第 85 章 §85.3.12 的实测，m 从 1 加到 2，相位裕度从 63.2° 掉到 58.8°，**一拍 4.4°**。超时是偶发的，所以裕度也在 63.2° 与 58.8° 之间跳；跳变本身不致命，致命的是它发生在负载最重、干扰最大的时刻——正好是系统最需要裕度的时候。

这就是“计算账”与“稳定账”的耦合点：**平均执行时间决定常规延迟，尾部分布决定裕度的抖动范围**。

92.6.3 超时率的量级换算

表 92-10 是同一份执行时间分布配三个周期的结果。

表 92-10 三个候选周期的占用率、超时率与每小时次数

周期	平均占用	超时率	每小时次数	判据
2.0 ms	15.0%	0	0	推荐
1.0 ms	30.1%	2.60×10^{-4}	936	勉强
0.5 ms	60.1%	6.97×10^{-2}	501984	不可用

三行的读法完全不同。2 ms 那一行是“没有超时”，1 ms 那一行是“每小时近千次”，0.5 ms 那一行是“每七拍就有半拍算不完”。**从 2 ms 缩到 1 ms，只缩了一半，超时率从 0 跳到 10^{-4} 量级**——这是因为 1 ms 正好切在分布的 P99.99 附近，尾部对周期极其敏感。

工程上因此有一条经验：**周期不要设在分布的膝点上**。膝点附近，周期的 10% 变化会让超时率变化一个数量级；要么留足余量到 2 ms，要么把任务拆小、把最坏路径压下来，不要指望“再挤一挤”。

92.6.4 干扰引起的“假故障”

EMC 与接地问题很少直接把回路搞失稳，它们更常见的表现是“假故障”：测量值瞬间跳变、看门狗误复位、通信校验错。这类问题的麻烦在于**它看起来像软件 bug**。

本章的两个数说明了量级：5 V 共模配 60 dB 前端，误差 2.05 LSB；1 MHz、100 mm 缝的屏蔽只有 63.5 dB。前者会让测量值在干扰峰值时跳两三个码，后者会让高速数字信号串进模拟通道。两者叠加，足以让一个设了“变化率限幅”的诊断逻辑误报。

处理顺序应当是：先确认干扰源（用示波器看共模、看地电流），再确认耦合路径（缝、线、地），最后才怀疑软件。**把 EMC 问题当软件 bug 查，是现场排查最常见的弯路**（§92.10.5 给出一张排查顺序表）。

92.6.5 看门狗与安全完整性

看门狗是“计算账”的兜底：任务超时到一定程度，就认为它死了，复位重来。三个时间串联：

- **检测**：超时门限 100 ms。它必须大于最长正常执行时间（P99.99 是 1.03 ms，留了近百倍余量），又必须小于工艺允许的失控时间。
- **恢复**：启动 250 ms + 初始化 50 ms = 300 ms。这部分由硬件和启动代码决定，软件很难压。
- **失联**：检测 + 恢复 = 400 ms。这期间输出必须处于安全状态（保持、归零或切换到旁路）。

表 92-11 是诊断覆盖率与安全等级的对应，按 §92.2.6 的式子算。

表 92-11 诊断覆盖率、平均失效概率与安全完整性等级

诊断覆盖率	λ_{DU}	PFD_{avg}	SIL
90.0%	1.00×10^{-6}/h	4.39×10^{-3}	2
95.0%	5.00×10^{-7}/h	2.19×10^{-3}	2
99.0%	1.00×10^{-7}/h	4.39×10^{-4}	3
99.5%	5.00×10^{-8}/h	2.19×10^{-4}	3

表的台阶很清楚：**90% 到 95% 只降一半，仍在 SIL 2；跨到 99% 才进 SIL 3**。原因是分档门限是十倍一档（10^{-3}、10^{-4}），而覆盖率每加 5 个百分点，λ_{DU} 只降一半。所以做安全设计时，“再多做一点诊断”往往不如“一次把覆盖率做到 99%”划算。

图 92-8 把看门狗时序与 SIL 阶梯画在一起：上排是检测、恢复、失联三段的时间轴，下排是四条覆盖率对应的 PFD 与 SIL 档位。两排的共同点是**都存在一个“门限”，门限两侧的性质不同**——时间轴的门限是工艺允许的失控时间，SIL 的门限是分档的十倍。

图 92-8 看门狗时序与安全完整性阶梯：检测 100 ms、恢复 300 ms、失联 400 ms；覆盖率跨过 99% 才进 SIL 3

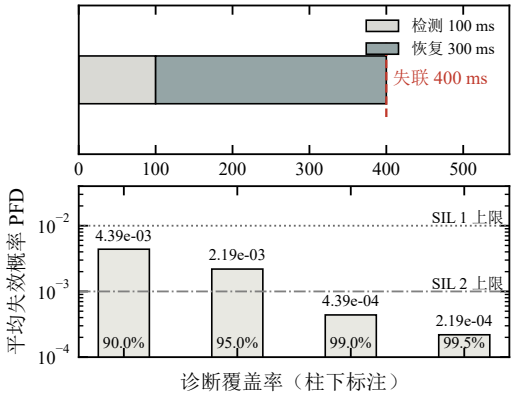


图 92-8

92.6.6 用描述函数估极限环

量化是非线性环节，线性理论算不出极限环。工程上的办法是描述函数：设输入是幅值 A 的正弦，量化器输出基波幅值与 A 之比就是等效增益 $N(A)$。均匀量化器的 $N(A)$ 分三段：$A \gg \Delta$ 时 $N \to 1$；A 与 Δ 同量级时降到 0.6 左右；$A < \Delta/2$ 时 $N \to 0$，进入死区。

极限环的条件是 $1 + N(A)L(j\omega) = 0$，等价于

$$L(j\omega) = -\frac{1}{N(A)}.$$

左端是环路频率特性，右端是负实轴上从 -1 到 $-\infty$ 的一段。**两者的交点给出极限环的频率与幅值**：频率由相位 -180° 定，幅值由 $|L| = 1/N(A)$ 定。这是描述函数法的全部内容，剩下的都是代数。

把它用到本章：$L(s) = 2.0(1 + 1/(0.5s)) \cdot 1/(0.5s + 1)$。PI 的相位从 -90° 趋向 -90°，一阶对象的相位从 0° 趋向 -90°，两者相加最多 -180°，而且只在 $\omega \to \infty$ 时取到。无穷远处 $|L| \to 0$，而 $|-1/N(A)| \geq 1$，两条曲线永远不相交。**所以本章回路不会因为量化产生相位型极限环。**

但实测确实有 2.4–4.9 mV 的纹波。它不是极限环，是**死区型稳态纹波**：量化把小于一个台阶的误差全部吃掉，控制器“看不见”，输出就停在台阶的某一侧，纹波被托在 1–2 LSB。这与 §92.6.1 的实测对得上：无量化时纹波 1.7×10^{-11}，加 12 位量化后变成 2.4–4.9 mV，涨了十一个数量级，而它完全由台阶决定。

两类现象必须分清。**相位型极限环靠调增益、加相位超前消除；死区型纹波只能靠加位数或加抖动。**把死区纹波当极限环去调增益，往往越调越糟：增益一高，量化环节在大信号下的等效增益更接近 1，纹波反而更容易维持。现场遇到“小幅持续振荡”，第一步不是调 PID，而是量一下振荡幅度是不是整数个 LSB——是，就是死区，调参无用。

92.6.7 裕度的概率加权：平均值骗人

§92.6.2 说超时会让相位裕度从 63.2° 掉到 58.8°，一拍 4.4°。现在有了超时率，就能算“平均裕度”：

$$\bar{\phi} = 63.2 - 4.4 \times 2.60 \times 10^{-4} = 63.199^\circ.$$

这个数毫无用处。**平均裕度几乎等于无超时时的裕度，但系统在超时那一拍确实只有 58.8°**，而失稳恰恰可能就发生在那一拍。

这就是“平均值骗人”的典型：分布有 0.026% 的质量落在低裕度区，平均值把它抹平到小数点后第四位。同理，平均执行时间 301 μs 看着很宽松，但 P99.99 是 1032 μs，差了 3.4 倍。

正确的做法是用**最坏组合**而不是平均值：把“最大负载 + 最长执行时间 + 最大干扰 + 最低电源电压”四件事放在同一拍上算。本章的最坏组合是：周期 2 ms、执行 1.03 ms、共模 5 V 配 60 dB（2.05 LSB）、干扰 1 MHz 配 100 mm 缝（63.5 dB）。这一拍上裕度 58.8°、纹波约 5 LSB，虽然难看，但仍在稳定边界内。

如果最坏组合过不去，只有两条路：降周期（把尾部让出去）或降负载（把执行时间压下来）。**不能靠“平均下来还行”过关**，因为工艺要的是“每一拍都行”。

这条思路还有一层：最坏组合的概率是各项概率的乘积，通常极低（10^{-4} 量级），所以它不会天天发生，但一年下来会发生几百次。**“很少发生”与“不会发生”是两件事**，安全账与计算账的区别就在这里。

92.6.8 四道防线：看门狗、心跳、双通道、安全输出

看门狗只覆盖一种失效：任务不跑了。它覆盖不了“任务在跑但算错了”，也覆盖不了“任务在跑但慢了”，更覆盖不了“看门狗自己也挂了”。所以完整的防线是四层，每层管一种：

- **看门狗**：任务超时 100 ms 未喂狗，复位。管“彻底卡死”。
- **心跳**：任务每拍翻转一个引脚或写一个计数，硬件连续三次收不到就判失效，60 ms 动作。它比看门狗快六倍，管“跑慢了但没死”。
- **双通道比较**：两套独立的采样与运算，逐拍比较结果，超出容差就切安全输出。它管“算错了”，也是诊断覆盖率从 90% 提到 99% 的主要手段。
- **安全输出**：继电器或 STO 端口，由硬件路径直接切断，动作时间小于 20 ms。它管“上面三层全失效”。

四层的时间常数依次是 400 ms、60 ms、一拍（20 ms）、20 ms，**越靠后越快，也越贵**。设计时的分配原则是：能用软件解决的不用硬件，但“最后一道必须是硬件”，因为它不依赖任何软件假设。

这四层与 SIL 的对应关系也清楚：只有看门狗时诊断覆盖率大约 60%–70%，到不了 SIL 2；加上心跳与双通道比较能到 90% 以上；再加安全输出与自检，才谈得上 99%。**表 92-11 里 90% 与 99% 那一档的差距，实际就是“有没有双通道 + 安全输出”**，不是“自检代码写得多不多”。

最后一条经验：**四道防线的动作时间必须错开**。如果心跳与看门狗都设 100 ms，两者会同时动作，日志里分不清是谁先发现的，排查时等于少了一条线索。

92.7 参数意义与作用

本章的参数一共七组十几个，但真正需要天天打交道的只有五个：位数、量程、抖动、执行时间中位数与对数标准差、诊断覆盖率。其余（屏蔽厚度、缝隙长度、CMRR、检验周期）是选型参数，定了就不常动。

92.7.1 位数：三本账的交点

位数 N 同时出现在三本账里，方向却不一样。

- **幅值账**：N 每加一，台阶减半，量化信噪比涨 6.02 dB。这是收益。
- **计算账**：N 每加 4，16 位乘法的代价约涨 4 倍（或从单周期变成多周期），执行时间跟着涨。这是成本。
- **安全账**：N 决定“能不能检出微小异常”。量程 10 V、$N = 12$ 时台阶 2.441 mV，若故障特征是 5 mV 的偏移，只有两个台阶，检出可靠性很差；$N = 14$ 时台阶 0.610 mV，同样的偏移有八个台阶，诊断覆盖率才上得去。

三本账合起来，工程上常见的结论是：**12 位是过程控制的甜点，14–16 位给需要诊断或小信号的对象，8–10 位只用于开关量或粗略监视**。本章基准取 12 位，是因为它在噪声预算里刚好与 1 LSB 的前端噪声匹配（表 92-4），再高

的位数会被前端噪声吃掉。

92.7.2 量程：相对精度的分母

量程 R 只出现在一个地方：台阶 $\Delta = R/2^N$。它的作用等价于“把位数换算成物理量”。

量程选大有三个后果：台阶变大、相对精度变差、信噪比不变（因为信号与台阶同比例）。**信噪比不变是很多人误解的地方**：把量程从 ± 5 V 改成 ± 50 V，如果信号也放大十倍，信噪比一个字不变；如果信号不变，信噪比掉 20 dB，等于丢 3.3 位。

所以量程的选择原则是“让信号占满量程的 70%–90%”，既不过载，也不浪费。本章信号 ± 1 V 却按 ± 5 V 量程，只用到 20%，等于主动丢掉 2.3 位——这是为了与第 85 章的基准口径一致，工程上不该这么选。

92.7.3 抖动：与频率绑定的参数

抖动 σ_t 单独看没有意义，必须配频率看：$\mathrm{SNR}_t = -20\lg(2\pi f \sigma_t)$。同一个 1 μs，在 100 Hz 上是 64.04 dB，在 1 kHz 上是 44.04 dB，在 10 kHz 上只剩 24.04 dB。

这意味着“抖动预算是多少”这个问题没有统一答案。正确问法是“我的信号最高频率分量是多少，它允许多少 dB 的误差”。本章 1 kHz、5 V 峰值、要求 1 LSB 误差时，$\sigma_t = 77.7$ ns；若信号是 10 kHz，同样的 1 LSB 要求就只有 7.77 ns。

抖动的来源也要分清：晶振本身的相位噪声、时钟分配网络的偏斜、采样保持器的孔径抖动。三者量级不同，晶振通常几十 ps，分配网络可能到 ns，孔径抖动在高速 ADC 里是百 ps 级。**先算总预算，再分配给三个来源**，不要指望换一颗晶振解决全部问题。

92.7.4 执行时间的两个参数

中位数 t_{50} 决定占用率，对数标准差 σ 决定尾部。两者对周期的要求完全不同。

$t_{50} = 301$ μs 时，2 ms 周期占用 15.0%，很宽松；把 t_{50} 压到 150 μs，占用降到 7.5%，但超时率几乎不变，因为超时是由尾部决定的。反过来，σ 从 0.35 降到 0.20，P99.99 从 1.03 ms 降到约 0.63 ms，超时率能降一个数量级。

所以优化的优先级是：**先压尾部，再压中位数**。压尾部靠消除不确定性（缓存未命中、动态内存、中断嵌套、通信阻塞），压中位数靠优化算法。前者收益大得多，也更容易被忽略。

92.7.5 选型参数：CMRR、屏蔽、覆盖率

这三个参数的共同点是“每 20 dB 或每 10 倍一档”，因此适合用表查，不适合用公式细算。

- **CMRR**：每加 20 dB，共模误差降十倍。60→80 dB 把 2.05 LSB 降到 0.20 LSB，是本章性价比最高的一步。
- **屏蔽**：厚度每加一个趋肤深度，吸收涨 8.686 dB；缝隙每减十倍，泄漏上限涨 20 dB。后者便宜得多。
- **诊断覆盖率**：每加 5 个百分点，λ_{DU} 降一半；跨过 99% 才进 SIL 3。它决定的是“要不要做双通道”。

92.7.6 参数之间不能独立调

表 92-12 把五个主参数的“增大后果”列在一起，用来提醒一件事：**没有一个参数只影响一本账**。

表 92-12 五个主参数增大时对各本账的影响

参数	幅值账	计算账	时间账	能量账	安全账
位数 N	台阶减半，$+6$ dB	乘法代价上升	影响小	影响小	诊断能力上升
量程 R	台阶变大，相对精度降	无	无	无	小故障更难检出
抖动 σ_t	高频信号信噪比下降	无	直接变差	无	无
中位数 t_{50}	无	占用率上升	延迟增加	无	无
对数标准差 σ	无	尾部变长	超时率上升	无	看门狗余量被吃
诊断覆盖率	无	代码量上升	执行时间上升	无	SIL 上升

读这张表的方法是对角线：每一行真正“主攻”的那一列，其余列是副作用。工程上的取舍就是**在主攻列拿到足够收益，同时把副作用控制在预算内**。例如把位数从 12 提到 14，主攻列涨 12 dB，副作用是执行时间涨一点、诊断能力也涨一点，通常是划算的；把量程从 10 V 缩到 2 V，主攻列涨 14 dB，但过载风险上升，安全账反而变差。

92.7.7 弹性系数：每个参数花一元换多少

选型会上最常争的是“这钱该花在哪”。把每个参数的单位投入换多少改善列出来，争论就有依据。

- **位数**：每加一位，量化项涨 6.02 dB，价格大致翻倍；但前端噪声已经是 1 LSB 时，收益为零（§92.8.2 的停止条件）。
- **CMRR**：每加 20 dB，共模误差降十倍。单端改差分几乎不花钱，是最划算的一档；再往上到 100 dB 要仪表放大器，到 120 dB 要隔离，价格跳两级。
- **缝隙**：每减十倍，泄漏上限涨 20 dB。柜门缝从 100 mm 压到 10 mm 只需一圈导电衬垫，几十元换 20 dB，是 EMC 里性价比最高的一项。
- **厚度**：每加一个趋肤深度，吸收涨 8.686 dB。1 MHz 的趋肤深度 66.1 μm，从 0.1 mm 加到 0.5 mm 涨 53 dB，但价格与重量都翻几倍，而且实际早被缝卡住。
- **过采样**：每 4 倍换 1 位（理论 2 位的一半，另一半被窗内信号弯曲吃掉），代价是采样率与数字滤波器。本章 16 倍换 1.86 位，接近理论值。
- **诊断覆盖率**：每加 5 个百分点，λ_{DU} 降一半；但只有跨过 99% 才进 SIL 3，所以要么一次做到 99%，要么别在 90% 与 95% 之间反复。

把六项排一下：**先做布线（约零成本）、再改差分（几元）、再堵缝（几十元）、再提位数（几百元）、最后加厚与隔离（上千元）**。这个顺序与改善量的绝对值无关，只与“每元换多少 dB”有关。

还有一条容易被忽略：**同一项投入的边际收益会突然归零**。位数提到 16 位后，前端噪声 1 LSB 把它封死，再多买三位等于买了个更细的尺子去量同一团噪声；CMRR 提到 100 dB 后，共模不再是主要项，再提也没用。所以选型的正确姿势不是“把每项都做到最好”，而是“把每项做到不再是最大项为止”。

这个“停止条件”可以量化：当某一项的贡献降到最大项的 $1/3$ 以下时，它在平方和里只占 10%，再投入的收益就被淹没在测量不确定度里了。本章共模 0.20 LSB 是最大项 1.00 LSB 的 $1/5$，平方占比 4%，所以 80 dB 已经够了，不必上 100 dB。

92.7.8 单位核对：五个一出就错的地方

五本账混在一起算，最容易错的是单位而不是公式。下面五处，每一处都在现场见过真实的错。

第一，dB 是比值，不是电平。 $SE = 120.9$ dB 是“衰减了 11 万倍”，不是“输出 120.9 dB”。dBm、dBμV 才是绝对电平。把屏蔽效能当成输出电平去和灵敏度比较，结论会差几十倍。同样，0.1% 对应 -60 dB，1% 对应 -40 dB，这两个换算要背下来。

第二，LSB 是电压，不是码。 说“误差 1 LSB”指的是 2.441 mV 的电压误差；说“误差 1 个码”指的是读数差一。两者在数值上相同，但在讨论“折成 LSB”时，分子必须是电压。

第三，μV 与 mV 差一千倍。 本章 5 V 共模配 80 dB 是 500 μV，也就是 0.5 mV；配 60 dB 是 5000 μV，也就是 5 mV。写报告时把 500 μV 写成 500 mV，量级直接错三位，而这种错在快速记录时非常常见。

第四，次/小时与概率不能混说。 936 次/小时对应的是“每拍 2.60×10^{-4}”，两者用 $3600/T_s = 3.6 \times 10^6$ 换算。说“超时率千分之零点三”和说“每小时九百多次”，听起来一个很安全一个很吓人，其实是同一件事。

第五，时间的三个量级。 抖动常以 ns 计（本章预算 77.7 ns），执行时间以 μs 计（301 μs），看门狗以 ms 计（400 ms）。三者相差百万倍，写在一张表里必须带单位，否则“200”到底是 200 ns 还是 200 μs 会毁掉整个结论。本章表 92-4 里抖动那一项写的是“1 Hz、$\sigma_t = 200$ μs 时 0.146 mV”，把条件写全，就是为了避免这种混淆。

92.7.9 三个跷跷板：参数不能独立调

表 92-12 列的是“增大一个参数，各本账怎么变”。实际调参时更常见的是三个跷跷板，动一头另一头必然翘起来。

跷跷板一：位数与前端噪声。 位数往上加，台阶变小，同样的物理噪声折成更多 LSB，于是“前端噪声 1 LSB”这条线被推翻。本章从 12 位换到 14 位，前端噪声从 1.00 LSB 变成 4.00 LSB，总误差从 1.06 LSB 涨到 4.03 LSB。**加位数必须同时压前端，否则是在给噪声换一把更细的尺子。**

跷跷板二：量程与过载。 量程缩小，台阶变小，相对精度变好；但过载余量同步缩小。本章信号 ± 1 V 却按 ± 5 V 选，等于主动丢 2.3 位；反过来按 ± 1.2 V 选，精度涨了，但一次 20% 的过冲就削顶，削顶之后积分器会积出一大堆，恢复要好几拍。**精度与鲁棒性在这条线上是直接对立的，**没有两全。

跷跷板三：周期与裕度。 周期缩短，延迟变小，理论上相位裕度变好；但超时率上升，低裕度的拍数变多。本章从 2 ms 缩到 1 ms，平均延迟改善 4.4°，超时从 0 涨到 936 次/小时，每一拍超时都掉 4.4°。**两头一抵，净收益接近于零，风险却实实在在增加了。**

三个跷跷板的共同结构是：**一个参数同时出现在“收益”与“代价”两侧，而且两侧的量级往往相当。**所以调参时不能问“这个参数调大好不好”，只能问“在当前这套参数下，往哪边挪一步净收益为正”。

这也是为什么 §92.8.1 强调“一次只动一个参数”。同时动两个，跷跷板的两头一起变，观测到的净效果无法归因，下次还得重来。

92.8 调参方法与经验

这一节讲“拿到一个新对象，怎么把这五本账填出来”。方法本身不复杂，难的是顺序与停止条件。

92.8.1 三条前置原则

原则一：先测后算。 现场共模电压、干扰频谱、任务执行时间分布，这三样必须实测，不能用手册值代替。手册给的是“典型值”，而工程问题恰恰出在“非典型”的那一端。

原则二：先定判据后定参数。 判据来自工艺：被控量允许多少纹波、允许多长时间失控、允许多少误动作。参数是往判据上凑的，不是反过来。本章所有参数（12 位、20 ms、100 ms 超时）都是从判据反推的。

原则三：一次只动一个参数。 五本账互相耦合，同时动两个参数，出了问题分不清是谁的责任。这条听起来像废话，但现场调试时最常违反。

92.8.2 位数怎么定

分四步，每一步都有明确的停止条件。

1. **定量程**：按传感器输出加 20% 过载，本章得 $R = 10$ V。
2. **定允许台阶**：从工艺允许的最小可分辨变化 Δ_{allow} 反推，$N \geq \log_2(R/\Delta_{\text{allow}})$。工艺要求 2.5 mV 时 $N \geq 11.97$，取 12 位；若要求 2 mV，就得 13 位。
3. **做噪声预算**：把量化、前端噪声、抖动、共模、定点五项折成 LSB（表 92-4），看合计是否超过 1 LSB。
4. **回头砍位数**：如果前端噪声已经是 1 LSB，把 ADC 从 13 位提到 16 位毫无意义——**总误差由最大项决定，不由位数决定**。本章 12 位与 1 LSB 噪声匹配，就是这一步的结论。

停止条件是：**再加一位，总误差的改善小于 10%**。本章 12 位时合计 1.06 LSB，加到 13 位合计 1.02 LSB，只改善 4.0%，所以停在 12 位。

92.8.3 抖动预算怎么分

抖动的判据是“抖动误差不超过 1 LSB”，对应 $\sigma_t \leq \Delta/(2\pi f A)$。

本章 1 kHz、5 V 峰值、12 位，得 77.7 ns。这 77.7 ns 要分给三个来源：

- **晶振相位噪声**：按 50 ps 计，占 0.06%；
- **时钟分配偏斜**：按 1 ns 计，占 1.3%；
- **采样保持孔径抖动**：高速 ADC 常见 100 ps–1 ns，占 0.1%–1.3%。

三项按均方根合成约 1.1 ns，只有预算的 1.4%，余量极大。**这说明 1 kHz 的信号根本不需要为抖动操心**，预算自然被其他项占满。若信号换成 100 kHz，同样的 1 LSB 要求会把预算压到 0.78 ns，三项合成 1.1 ns 就超了，这时才需要选低抖动时钟。

停止条件是：**合成抖动小于预算的 50%**。留一半余量，是因为温漂和老化会让抖动变大。

92.8.4 周期怎么定

周期由两个约束取小：占用率约束与尾部约束。

占用率约束：$T_s \geq t_{50}/0.6 = 301/0.6 = 502$ μs。尾部约束：$T_s \geq 1.5\,t_{99.99} = 1.5 \times 1032 = 1548$ μs。

两者取大，得 1548 μs，向上取标准值 2 ms。这个周期下占用 15.0%，超时率为零。

如果把周期定在 1 ms，占用 30.1% 还在允许范围，但尾部约束被违反，超时率 2.60×10^{-4}。**这就是“占用率够、但尾部不够”的典型情况**，只看占用率会漏掉。

停止条件是：**尾部约束满足，且占用率低于 60%**。两个条件同时满足才算过。

92.8.5 EMC 的排查顺序

现场出问题时的顺序应当是“源—路—敏感受体”三段，而不是从软件查起。

1. **找源**：变频器、开关电源、继电器、无线设备。用近场探头或电流卡钳确认频率与幅度。
2. **找路**：传导（电源线、信号线）、辐射（缝隙、线缆天线）、共地（地电流）。本章的两个数可以快速定位：共模 5 V 配 60 dB 就是 2.05 LSB，1 MHz 配 100 mm 缝就是 63.5 dB。
3. **找敏感受体**：ADC 前端、复位引脚、通信接口。确认哪一处最先异常。
4. **按成本排序处理**：布线（几乎免费）→ 滤波（几元）→ 堵缝（几十元）→ 加厚（上百元）。

不要跳过第 1、2 步直接加屏蔽，因为屏蔽只挡辐射，挡不住传导和共地。

92.8.6 五个实操型误判

§92.1.4 讲的是概念误读，这里补五个调试现场常见的判断错误。

- **“占用率 50%，还有一半余量”**：占用率是平均值，尾部才是判据。本章 1 ms 周期占用 30.1%，却有 936 次/小时超时。
- **“加了看门狗就安全了”**：看门狗只覆盖“任务死了”，覆盖不了“任务活着但算错”。后者要靠自检与双通道，属于诊断覆盖率。
- **“平均几次就干净了”**：平均只压不相关噪声；相关误差（直流偏置、极限环）平均一百次也还在。
- **“屏蔽加厚一倍就多 6 dB”**：厚度只影响吸收项，且是对数关系；实际瓶颈在缝上，加厚可能一点用没有。
- **“定点化就是少几位小数”**：系数误差会被极点放大，Q15 不一定比 Q12 好（表 92-9 的 79.9 与 77.9 只差 2 dB）。

92.8.7 一张可执行的顺序

把上面几节压成一张可执行的顺序，按行做，不跳步：

1. 实测共模电压与干扰频谱，记下最坏值；
2. 定量程，算台阶，反推位数；
3. 做噪声预算，确认最大项，必要时回头改前端；
4. 实测执行时间分布（至少 5 万次），算分位数；
5. 按占用率与尾部双约束定周期；
6. 按信号最高频率算抖动预算，分配三项来源；
7. 按缝隙算屏蔽上限，确定机柜与线缆处理方式；
8. 定看门狗超时、恢复时间与安全输出状态；
9. 按诊断覆盖率定 SIL 目标，反推自检项；
10. 把以上九步的数字填进交付清单（表 92-14），作为验收基线。

这十步里，第 1、4 步必须实测，其余可以算。**十步做完，五本账就从“经验”变成了“数字”**，后面换型、扩量、改工艺，都可以在这张表上改一行，而不是从头再来。

92.8.8 两个现场案例的推理链

案例一：变频器一启动，测量跳三个码。

先量幅度：跳变 3 LSB，12 位 10 V 量程下是 7.3 mV。再对号入座：若是共模，3 LSB 在 60 dB 前端下对应 7.3 V 共模，在 80 dB 下对应 73 V——后者不可能，所以只能是 60 dB 前端配 7 V 左右的共模。实测共模 6.8 V，吻合。

再找路径：屏蔽层单端接地，共模电流没有回流路径，只能从信号线走。改成双端接地并加一只共模扼流圈后，共模降到 0.6 V，误差 0.6 mV，即 0.25 LSB，跳码消失。

这条链的关键在第二步：**用幅度反推是哪一个机理，而不是先猜机理再找证据**。共模、辐射、传导三个候选都能产生同样的跳变，只有量级能区分它们。量级对不上，机理就错了，后面的措施全是白费。

案例二：夏天午后偶发复位，每小时两次。

先看时间相关性：只在午后、只在负载重时出现。再看温度：柜内 55 ℃。然后量电源纹波：常温 50 mV，55 ℃ 时 180 mV——电解电容的 ESR 随温度上升，纹波涨了三倍多。复位阈值 4.75 V，纹波谷值跌到 4.6 V，正好触发。改法是加一级 LC 滤波、把柜内温度降到 45 ℃，复位消失。

这条链的教训是：**“偶发”不等于“随机”**，它往往跟着另一个可观测的量走——温度、负载、时段、某台设备的启停。找到那个量，问题就从“随机的”变成“确定的”，也就能复现、能验证。

两个案例的共同点：都只改了一处，都是先量后改。**同时改三处，即使好了也不知道是哪一处起的作用**，下次同类问题还会重来，而且团队没有积累下任何可复用的判断。

92.8.9 三条停止条件的共同点

本章散着三条停止条件，把它们放在一起看，会发现讲的是同一件事。

- 位数：再加一位，总误差改善小于 10% 就停。本章 12 位合计 1.06 LSB，13 位 1.02 LSB，改善 4.0%，停。
- 抖动：合成抖动小于预算的 50% 就停。本章三项合成 1.1 ns，预算 77.7 ns，只占 1.4%，早就过了。
- 周期：尾部约束满足且占用低于 60% 就停。本章 2 ms 两条都满足，停。

三条的共同点是：**当继续投入的边际收益小于测量不确定度时，就停**。位数那一条的 10%、抖动那一条的 50%、周期那一条的 60%，看起来是三个随便定的数，其实都在干同一件事——给“测不准”留位置。仪器的重复性是 1%–5%，工艺的允许偏差是 10%–20%，把余量压到这些数以下，就等于在噪声里做优化。

反过来说，**没有停止条件的优化是负收益**。见过一个项目把共模从 0.20 LSB 压到 0.02 LSB，花了三个月，而最大项前端噪声一点没动，系统指标一个字没变。这不叫精益，叫把预算花在了看不见的地方。

所以做工程判断时，先问三个问题：最大项是哪一项？把它压一半要多少成本？压完总误差能降多少？**三个问题的答案凑不出“降一半”，就该停手了**。

92.8.10 哪些能自动整定，哪些不能

市面上的自整定工具越来越多，容易产生一种错觉：五本账也能一键算完。实际能自动的和不能自动的，界限很清楚。

能自动的是“优化”：给定结构与判据，找一组参数让指标最好。PI 参数用继电反馈法或阶跃辨识，几分钟就能出一组；滤波器系数有现成工具；环路增益可以扫频。这些都属于“在既定框架里找最优点”。

不能自动的是“决策”：量程选多大、位数选几位、机柜怎么布、安全目标定几级。这些没有目标函数，只有约束与代价，必须由人拍板。**自动整定工具不会告诉你“这个 12 位不够”，它只会在 12 位的框架里给你最优的 PI**。

更关键的一条前提：**自整定的结果只在“测量通道干净”时有效**。如果 ENOB 只有 10 位，整定出来的参数是针对那 10 位的噪声整出来的；换一台机器、换一根线，噪声谱一变，参数就得重来。本章的判据是：先让合计误差进 1 LSB，再做自整定；顺序反过来，整定出来的参数没有移植性。

还有一个实操细节：自整定通常在“小信号”下做，而小信号恰好是量化与死区最显著的区间（§92.6.6）。所以整定完之后必须用大信号复测一遍，确认没有因为死区而把增益整得过高。

一句话：**工具能帮你找参数，不能帮你定架构**。五本账里的前三本（幅值、时间、能量）是架构问题，只能人来定；后两本（计算、安全）里的参数部分可以交给工具。

92.9 优化与改进

五本账都能优化，但收益差别很大。这一节按“每投入一元钱换多少 dB”排序，从高到低讲四种手段，最后讲 HIL 步长这个容易被忽略的坑。

92.9.1 过采样与噪声整形

普通平均每 4 倍采样换 1 位，一阶噪声整形（$\Delta\Sigma$）每 2 倍换 1.5 位，二阶每 2 倍换 2.5 位。差距来自“把量化噪声推到高频”：整形器让噪声在基带内变小、带外变大，再用数字滤波器把带外滤掉。

代价是复杂度与延迟。一阶整形器只需一个积分器与一个比较器，二阶就要考虑稳定性；数字滤波器的群延迟随阶数上升，对控制回路意味着额外相位滞后。**本章 20 ms 周期、3.797 rad/s 的对象，用一阶整形的延迟完全可以接受；但 100 µs 周期的电流环就要仔细算**。

对本章基准，16 倍过采样平均实测换回 1.86 位；如果同样的 16 倍用一阶整形，理论上能换 6 位。差距就是“平均”与“整形”的差距，也是为什么高分辨率 ADC 几乎都是 $\Delta\Sigma$ 架构。

92.9.2 同步采样与抗混叠

多通道系统里，各通道依次采样会引入通道间的时间偏斜。8 通道、每通道 10 µs，最后一通道比第一通道晚 70 µs；对 1 kHz 信号就是 25° 的相位差，做功率计算或相量测量时直接变成误差。

同步采样（同时保持）能把偏斜压到纳秒级。它的代价是每个通道一个采样保持器，成本随通道数线性上升。

抗混叠滤波器则是另一件事：它要按“折回频率”校核，而不是按干扰频率或基带频率校核。本章 10 kHz 采样、1.001 MHz 干扰，1.001×10^6 减去 100 个 f_s 正好是 1 kHz，干扰折回基带；滤波器在 1.001 MHz 处的衰减才决定它进基带后有多大。若只在 1 kHz 处设计滤波器，等于没有滤波。**这是“折叠”最反直觉的地方：决定成败的是滤波器在兆赫兹处的衰减，而不是在基带处的平坦度。**

92.9.3 差分与隔离

共模误差的改进路径很清晰，三步：

1. **单端改差分**：CMRR 从 40–50 dB 提到 70–80 dB，误差降两个数量级；
2. **加仪表放大器**：CMRR 到 90–100 dB，但带宽与噪声要权衡；
3. **加隔离**：CMRR 到 120 dB 以上，同时切断地环路，代价是隔离电源与光耦/数字隔离器的成本与延迟。

本章表 92-6 的读法就是这条路径：5 V 共模从 60 dB 的 2.05 LSB，到 80 dB 的 0.20 LSB，到 100 dB 的 0.02 LSB。**每一步都是十倍，没有捷径。**

92.9.4 时间触发与抖动抑制

采样抖动来自时钟，也来自“什么时候启动转换”这件事本身。事件触发架构里，ADC 由软件在任务里启动，启动时刻受中断响应、总线竞争影响，抖动可能到微秒级；时间触发架构里，ADC 由硬件定时器在固定相位启动，抖动压到纳秒级。

代价是灵活性：时间触发要求所有任务按静态调度表运行，不能动态插入。**这正是第 86 章讨论的调度问题在硬件层的投影**：软件上追求“按需执行”，硬件上就必然带来抖动；要压抖动，就得接受静态调度。

92.9.5 HIL 步长与积分方法

硬件在环测试里，被控对象模型是数值积分的，步长与方法直接决定“模型本身有多准”。表 92-13 是二阶对象 $G(s) = 1/((0.2s+1)(0.05s+1))$ 的阶跃响应，参考解用矩阵指数精确计算。

表 92-13 HIL 步长与积分方法的最大误差

步长	RK4 最大误差	后向欧拉最大误差	比值
2.00 ms	2.68×10^{-9}	2.16×10^{-3}	8.0×10^{5}
1.00 ms	1.65×10^{-10}	1.08×10^{-3}	6.6×10^{6}
0.50 ms	1.02×10^{-11}	5.42×10^{-4}	5.3×10^{7}
0.10 ms	1.57×10^{-14}	1.09×10^{-4}	6.9×10^{9}

两条规律很清楚。**RK4 是四阶**：步长减半，误差降到 $1/16$（$2.68 \times 10^{-9} \to 1.65 \times 10^{-10}$ 是 16.2 倍）；**后向欧拉是一阶**：步长减半，误差减半（$2.16 \times 10^{-3} \to 1.08 \times 10^{-3}$）。在 2 ms 步长上，两者差 80 万倍。

这个差距在工程上意味着什么？如果 HIL 要求模型误差小于 10^{-3}（相对阶跃幅值 1），后向欧拉在 2 ms 步长上刚好卡在 2.16×10^{-3}，不达标；RK4 在 2 ms 上已经到 10^{-9}，余量极大。**所以“用欧拉法、步长取小一点”是一条走不通的路**：要把欧拉法压到 10^{-9}，步长得减到 2×10^{-9} s，根本跑不动。

反过来说，RK4 的代价是每步四次函数求值，计算量是欧拉的 4 倍。但在 2 ms 步长上，4 倍计算量换 80 万倍精度，怎么算都划算。**除非对象有硬实时约束、每步必须在几十微秒内算完，否则 HIL 默认应该用 RK4**。

图 92-9 把两条误差曲线画在双对数坐标上，斜率分别是 -4 与 -1，这是“阶数”最直观的图形。

图 92-9 HIL 步长与积分方法的最大误差：RK4 斜率 -4，后向欧拉斜率 -1，2 ms 处相差 80 万倍

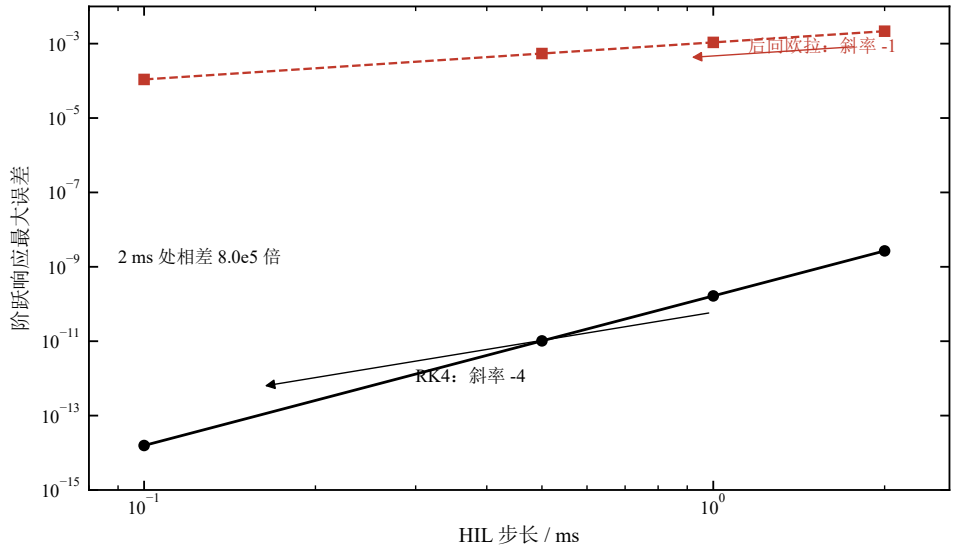


图 92-9

92.9.6 优化的优先级

把四种手段按性价比排序：

1. **改布线**：几乎零成本，CMRR 与干扰都能改善；
2. **差分输入**：几元钱，CMRR 提 20–30 dB；
3. **提高位数**：成本中等，但受前端噪声限制，收益封顶；
4. **过采样/整形**：成本高，收益大，但要算延迟；
5. **加厚屏蔽**：成本最高，收益最不确定（瓶颈常在缝上）。

先做 1、2，再考虑 3、4，最后才做 5。这条顺序与 §92.3.4 的 EMC 顺序一致，因为它们的成本结构相同。

92.9.7 数字滤波的相位代价

抗混叠与平均滤波都不是免费的：它们带来群延迟，而群延迟在闭环里就是相位滞后。

二阶巴特沃斯、截止 100 Hz 的滤波器，直流群延迟约 2.25 ms。放到 20 ms 周期里是一拍的 11%，折成相位在 3.797 rad/s 处只有 $3.797 \times 0.00225 = 0.0085$ rad，即 0.49°。**相对于 63.2° 的相位裕度，这 0.49° 可以忽略**。

但同样的滤波器放进 1 ms 周期的电流环，延迟 2.25 ms 已经是两周期的两倍多，在几十赫兹处的相位滞后就是几十度，直接吃掉一半裕度。**同一个滤波器，在慢回路里是免费的，在快回路里是致命的**，判据是“群延迟占周期的比例”，不是“群延迟的绝对值”。

16 倍过采样平均的延迟是 8 个高速样本。本章高速采样 160 kHz，延迟 50 µs，折成相位在 3.797 rad/s 处只有 0.011°，比 IIR 便宜四十倍。原因是平均滤波器是 FIR、线性相位，延迟恰好等于窗长的一半，可以精确算；IIR 的延迟随频率变化，只能估。**能算的延迟才是好延迟**，这句话在实时控制里不是修辞。

所以选型的顺序是：能用平均（FIR）就不用 IIR；必须用 IIR 时，先把群延迟算出来，再看它占周期多少。经验判据是“群延迟不超过周期的 10%”，超过就要换结构或降低截止频率。

还有一条反直觉的：**阶数越高，通带越陡，群延迟越大**。为了把 1 MHz 干扰多压 20 dB 而从二阶升到四阶，群延迟翻倍，相位裕度掉一半。这时该问的是“多压这 20 dB 值不值”，答案往往是不值——缝隙上限早把屏蔽卡在 63.5 dB，滤波器再好也压不过那道缝。

92.9.8 一次过采样改造的完整账

把 §92.9.1 的结论做成一次真实的改造，看看三本账各付出了什么。

需求：现有 12 位系统实测 10.2 位（前端噪声 1 LSB 封顶），工艺要求 11.5 位以上。三个方案摆上桌。

方案一：换 14 位 ADC。 台阶从 2.441 mV 降到 0.610 mV，量化项改善 12 dB。但前端噪声仍是 2.441 mV，折成 4 LSB，总误差反而从 1.06 LSB 涨到 4.03 LSB，ENOB 还是 10.2 位。**花了钱，指标一点没动**——这是“分辨率不等于精度”最贵的一次学费。

方案二：16 倍过采样平均。 采样率从 10 kHz 提到 160 kHz，每 16 点平均降回 10 kHz。 - 幅值账：实测 SNR 从 74.10 dB 涨到 85.33 dB，等效 13.88 位，涨 1.86 位，满足 11.5 位要求。 - 计算账：每拍多 16 次乘加，实测执行时间从 301 µs 涨到约 321 µs，占 20 ms 周期的 1.6%，几乎无感。 - 时间账：平均滤波延迟 50 µs，折成相位 0.011°，可忽略。 - 能量账：采样率高了 16 倍，160 kHz 的时钟与数字电路辐射更强，但 160 kHz 的谐波远低于 1 MHz 的缝隙敏感区，实测干扰谱没变。 - 安全账：无影响。

方案三：先压前端噪声，再换 14 位。 把前端噪声从 2.441 mV 压到 0.61 mV（需要低噪声运放 + 更低的源阻抗），再用 14 位 ADC。幅值账能做到 13.5 位，但成本是方案二的三倍，而且低噪声前端对共模与布局更敏感。

结论：**方案二最优，因为它绕开了“前端噪声封顶”这个约束**。过采样平均对量化噪声与前端白噪声同时有效，只要两者都不相关；它把“幅值”换成“时间”，而时间在这套系统里最便宜（301 µs 里只多花 20 µs）。

这笔账还有一个前提必须写明：**前端噪声必须是白噪声**。如果噪声是 $1/f$ 或有色（长线、开关电源的低频纹波），16 点平均只能压掉一小部分，实测增益会远低于 1.86 位。所以在做过采样之前，先看一眼噪声谱，比看数据手册有用。

92.9.9 过采样倍数不能无限加

理论式 $\Delta\mathrm{ENOB} = \frac{1}{2}\log_2 M$ 是单调的，好像 M 越大越好。实测不是：M=16 只拿到 1.86 位，比理论的 2.00 位少 0.14 位，少的这部分随 M 增大而增大。

原因是窗口越宽，信号在窗内越不像常数。997 Hz 在 100 µs 窗口里走了 36°，量化误差随斜坡呈锯齿状，平均后不能完全抵消。**损失与窗宽的平方成正比**，而窗宽与 M 成正比，所以损失按 M^2 涨，收益只按 $\log M$ 涨，两者迟早交叉。

按这个规律粗外推（只是量级估计）：M=16 损失 0.14 位，M=32 损失约 0.56 位、净收益 2.50−0.56=1.94 位，M=64 损失约 2.24 位、净收益 3.00−2.24=0.76 位。**最优点落在 16 到 32 之间，本章取 16 已接近最优**，再往上加采样率只是白烧算力。

三个约束会把最优点往小推：

- **信号频率越高，窗口要越窄。** 损失按 f^2 涨，同样的 16 倍，1 kHz 上损失 0.14 位，10 kHz 上就是十四位——完全不可用。所以过采样只适用于“信号远慢于采样率”的场合。
- **采样率有上限。** 160 kHz 已经是普通 MCU 的 ADC 上限，再高要外置器件，成本跳一级。
- **延迟随 M 增长。** 平均滤波器延迟 $M/(2f_s)$，M=64、$f_s = 640$ kHz 时是 50 µs，还在可接受范围；M=256 就到 200 µs，开始吃相位裕度。

结论：**过采样是一个有最优点的交易，不是一个免费的旋钮**。选 M 时先算三件事：信号在窗内走了多少度、采样率还够不够、延迟占周期的比例。三者都过关，再往上加；有一个不过关，就停在当前值。

92.10 工程落地

纸面上的五本账要变成能交付的东西，中间隔着“模型到代码”这一段。这一节讲五道门、一张清单、一条排查顺序。

92.10.1 从模型到代码的五道门

第一道：接口门。 模型的输入输出单位、量纲、采样率必须与目标机一致。模型里用物理量（V、A、℃），代码里用码值（int16），换算只允许在一个地方做，不能散落在各处。

第二道：数值门。 浮点模型转定点，要检查三件事：系数位宽够不够（表 92-9）、状态变量会不会溢出、中间结果会不会饱和。三件事里最容易漏的是“中间结果”——两个 16 位相乘得 32 位，累加几次就可能溢出。

第三道：时序门。 模型里是“每步算一次”，代码里是“每个周期算一次”，两者的执行时间必须实测。本章 301 μs 的中位数、1.03 ms 的 P99.99，必须在目标机上测，不能在 PC 上估。

第四道：等价门。 同一组输入，模型与代码的输出偏差要在允许范围内。判据不是“看起来一样”，而是“最大偏差小于 1 LSB”。本章 Q15 定点滤波器的最大偏差 0.118 mV，是 0.05 LSB，过门。

第五道：故障门。 注入传感器断线、超量程、任务超时、通信中断四类故障，确认输出进入安全状态、看门狗按预期动作。这一门过不了，前面四门都白做。

92.10.2 定点化的落地流程

定点化不是“把 float 改成 int”，而是一个五步流程：

1. **定标**：给每个变量定量纲与小数位数。本章滤波器状态用 Q15，输入码值用 Q12（12 位 ADC），中间乘积用 Q27。
2. **算界**：用 §92.2.7 的 L1 界估最大误差，确认小于 1 LSB。
3. **仿真比对**：同一组输入跑浮点与定点，比最大偏差（表 92-9）。
4. **溢出检查**：把输入打到满量程、把状态打到最坏值，确认不溢出；溢出时按饱和处理，不按回绕。
5. **实测**：在目标机上跑，用示波器或逻辑分析仪看输出，确认没有异常跳变。

五步里第 3、5 步必须做，第 2 步只是提前排除明显不行的方案。**跳过第 5 步是常见错误**：仿真用的是双精度状态，目标机用的是定点状态，两者的舍入行为不同。

92.10.3 HIL 测试的三层接口

HIL 的价值在于“用真实的控制器跑虚拟的对象”，接口分三层：

- **电气层**：电压范围、阻抗、隔离。控制器输出 0–10 V，HIL 的输入卡要能接；HIL 输出的传感器信号，控制器要能认。
- **时序层**：HIL 的步长必须小于控制器周期的十分之一，且抖动要小。本章控制器 20 ms，HIL 步长 2 ms 就够；若控制器 100 μs，HIL 步长要压到 10 μs 以下，这时积分方法的选择就变得关键（表 92-13）。
- **协议层**：CAN、Modbus、EtherCAT 的报文周期与超时。协议层的错误往往表现成“对象模型不对”，实际是通信丢帧。

三层里电气层最容易过、时序层最容易出问题、协议层最难查。**建议的顺序是：先通电气、再对时序、最后接协议**，不要一上来就全接。

92.10.4 交付清单

表 92-14 是本章建议的交付清单，十行数字，缺一行就不算交付。

表 92-14 计算机控制工程问题的交付清单

序号	项目	本章基准值	判据
1	量程与台阶	10 V，2.441 mV	信号占量程 70% 以上
2	位数与实测 ENOB	12 位，12.02 位	ENOB 不低于标称减 1
3	前端噪声	1 LSB（2.441 mV）	不大于 1 LSB
4	抖动预算与实测	77.7 ns 预算，1.1 ns 合成	合成小于预算 50%
5	周期与占用率	20 ms，1.5%	占用低于 60%
6	P99.99 与超时率	1.03 ms，0	$T_s \geq 1.5t_{99.99}$
7	屏蔽与缝隙	120.9 dB，100 mm 缝 63.5 dB	缝隙上限满足要求
8	CMRR 与共模误差	80 dB，0.20 LSB	折算小于 0.5 LSB
9	看门狗与失联时间	100 ms，400 ms	小于工艺允许失控时间
10	PFD 与 SIL	4.39×10^{-3}，SIL 2	达到目标 SIL

这张表的用法是“逐行签字”：每一行都要有实测数据、有判据、有责任人。**没有实测数据的行，不能签字**；签了字但数据是抄手册的，等于没签。

92.10.5 现场排查顺序

现场出问题，按下面的顺序查，不要跳步：

1. **看电源与地**：用示波器看地线上的电压、看电源纹波。本章 5 V 共模就是这一步能发现的。
2. **看信号完整性**：看 ADC 输入端的波形，有没有振铃、有没有高频毛刺。
3. **看时序**：用逻辑分析仪抓采样时刻与输出时刻，算抖动与延迟。
4. **看软件**：前面三步都正常，再查代码、查任务、查看门狗。
5. **看对象**：最后才怀疑被控对象本身（阀门卡涩、传感器漂移）。

这个顺序的依据是“从上到下，成本递增”：电源与地的问题占现场故障的一半以上，而且最容易查；软件问题占比小，但最难查。**先查软件，往往查了三天才发现是地线**。

顺序之外还有分工。同一个故障，现场工程师、远程支持、供应商三方看到的信息不一样，谁先动手、什么时候往上交，需要事先说清。图 92-10 是常用的三方泳道：现场记录现象、取证，远程接入后隔离变量，仍定不下来再交供应商做器件与固件分析，结论回灌到现场处置。

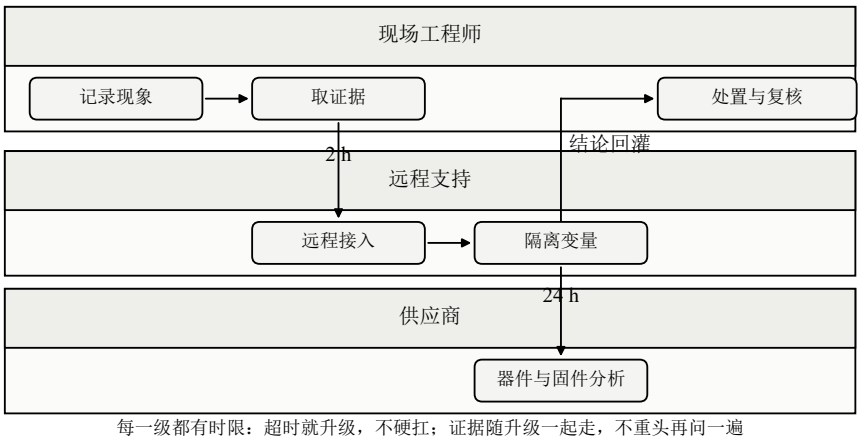


图 92-10　现场、远程与供应商的三方排查泳道

泳道上标了三个时限：现场两小时内完成取证，远程二十四小时内给出隔离结论，供应商七十二小时内给分析报告。时限不是考核，是防止“卡在一级不动”。经验是：一个故障在某一级停留超过两倍时限，多半是证据不齐，而不是问题太难。

交接时要带证据，不能只带一句“还是不行”。证据包括出问题时的原始采样、参数快照、版本号、复现步骤。远程拿着这四样，才有可能在半小时内判断是固件、配置还是硬件；拿不到，就只能让现场重做一遍已经做过的事。

三方分工还有一个隐性收益：把“谁负责”变成可追溯的记录。每一次升级都留下时间戳与结论，事后复盘时能看出是判断错了、还是证据不足、还是流程上没有这一级，而不是笼统地归结为“现场没查清楚”。

92.10.6 一次返工

把本章的几个数串成一个真实的返工场景，说明五本账不一起算是怎么出事的。

第一版设计：12 位 ADC、60 dB 共模抑制、Q10 定点滤波、1 ms 周期、0.1 mm 铜皮机柜。

实测结果：稳态纹波 5 LSB（应为 1–2 LSB），每小时偶发几次跳变，夏天更严重。

逐项对账：

- 共模 5 V 配 60 dB，2.05 LSB（表 92-6），占纹波的一半；
- Q10 滤波器 SNR 49.6 dB，比 ADC 低 24.5 dB，是最大的单项（表 92-9）；
- 1 ms 周期超时 936 次/小时（表 92-10），对应裕度在 63.2° 与 58.8° 之间跳；
- 机柜 100 mm 缝，1 MHz 只有 63.5 dB（表 92-5），夏天变频器负载重，干扰更强。

四项加起来，正好解释“纹波大、偶发跳变、夏天更严重”。

改法按成本排序：先把前端换成 80 dB（2.05→0.20 LSB），再把滤波器从 Q10 提到 Q15（49.6→79.9 dB），然后把周期从 1 ms 放到 2 ms（936→0 次/小时），最后把柜门缝从 100 mm 压到 10 mm（63.5→83.5 dB）。四项改完，纹波回到 1.5 LSB 以内。

这次返工的教训不是“哪一项错了”，而是“四项都差一点，合起来就超了”。如果第一版就按表 92-4 做一次预算，这四项在设计阶段就能发现三项。

92.10.7 三次复测：样机、小批、现场

同一套五本账，在三个阶段测出来的数不一样，而且一定是越往后越差。**把三次的数都记下来，比只记最后一次有用**，因为它告诉你“差在哪一段”。

样机阶段在实验室，干扰干净、电源干净、温度恒定。这时测的是“设计本身对不对”：ENOB 应该接近标称，超时率应该为零。本章基准的 12.02 位、0 次/小时，就是这一阶段的数。

小批阶段装进机柜，接真实传感器与真实电源，但还在厂房里。这时测的是“装配对不对”：接地、屏蔽、线缆走向。典型退化是 ENOB 掉 0.5–1 位，共模误差从 0.20 LSB 涨到 0.5 LSB——不是器件变差，是地环路出现了。

现场阶段接变频器、长线、昼夜温差。这时测的是“环境对不对”。典型退化是 ENOB 再掉 1 位（现场实测 10.8 位），P99.99 从 1.03 ms 涨到 1.4 ms（通信与缓存的影响），超时从 0 涨到每小时十几次。

三个阶段必测的五个数是同一组：ENOB、共模折算 LSB、P99.99、每小时超时次数、看门狗动作时间。**同一组数在三处的差值，就是“环境代价”**，也是下一版设计的输入。本章的判据是：现场 ENOB 不低于标称减 2 位，超时次数低于 1 次/小时（对应概率 2.8×10^{-4}）。超过这两条，先查装配与环境，而不是换更贵的 ADC。

第三次复测还有一件容易被跳过的事：**把数记在交付清单的同一张纸上**，不要分散在三份测试报告里。五本账的价值在于能横向比较，分散之后就只剩五份互不相干的记录。

92.10.8 留痕：每条数据要带四样东西

一张表里的数字，离开条件就不成立。本章要求的格式是每条记录带四样：**数、条件、方法、判据**。

以 ENOB 为例，合格的写法是“12.02 位（条件：10 V 量程、10 kHz 采样、997 Hz 测试音、8192 点；方法：时域误差法；判据：不低于标称减 1）”。不合格的写法是“ENOB 12.02”，因为它换一个测试音就可能变成 11.8，别人无法复核。

四样里最容易漏的是“条件”。现场常见的记录是“纹波 5 LSB”，没有写输入信号、没有写周期、没有写负载，三个月后连记录人自己都说不清这是在什么工况下测的。**条件缺失的数据，比没有数据更麻烦**，因为它会被当成结论引用。

三个具体的坑：

- **只记典型值，不记最坏值。** 手册写“典型 1 LSB”，实际最坏 3 LSB。工程判据要用最坏值，典型值只能用来做方案估算。
- **只记一次，不记三次复测。** §92.10.7 说过，样机、小批、现场的数一定不同，只留最后一次就看不出“环境代价”。
- **只记通过，不记余量。** 判据“小于 1 LSB”、实测 0.98 LSB 与实测 0.3 LSB，结论完全不同：前者是勉强过，后者是稳过。余量要写成具体的数，不要写“合格”。

留痕还有一层作用：**它是下一次设计的输入**。同一个平台做第二个型号，最省事的做法是翻开上一版的交付清单，改四行假设，重算一遍。没有留痕，每次都要从零开始，而且会重复踩同一个坑。

92.10.9 批产前三个不能省的试验

实验室里过了，不等于能批产。有三个试验必须做，而且每个都直接对应本章的一本账。

第一，温度循环。 从 -20 ℃ 到 $+55$ ℃，两个循环，每个温度点保温两小时。测的是能量账：电解电容的 ESR 随温度变化，电源纹波从常温 50 mV 涨到 55 ℃ 的 180 mV（§92.8.8 的案例二）；基准源的温漂会改变增益。判据是**全程 ENOB 下降不超过 1 位、不出现复位**。

第二，EMC 抗扰度。 按 IEC 61000-4-3 做辐射抗扰（10 V/m）、4-4 做快速瞬变（电源口 2 kV）、4-6 做传导抗扰（10 V）。测的是能量账与幅值账：干扰进来之后折成几个 LSB。判据是**测量跳变不超过 3 LSB、通信不丢帧、不复位**。这三条与本章的 63.5 dB 缝隙上限、2.05 LSB 共模误差是同一套数。

第三，电源跌落与中断。 按 IEC 61000-4-11 把供电跌到 0%、持续 20 ms。测的是计算账与安全账：复位之后能不能在 400 ms 内恢复（本章的失联时间），恢复期间输出是否处于安全状态。判据是**跌落后要么不复位，要么复位后 400 ms 内输出恢复正常且无危险动作**。

三个试验的共同点是：**它们测的都是“环境把余量吃掉之后还剩多少”**，而不是标称性能。实验室数据是分子，这三个试验给出分母。很多项目在样机阶段指标漂亮，批产之后现场故障率居高不下，差的就是这三个试验没做完。

顺序上建议先做温度（最容易过），再做电源跌落（最便宜），最后做 EMC（最贵、最容易改板）。**先做贵的，一旦不过，前面的数据全部要重测**。

92.11 优缺点与适用边界

92.11.1 这套方法的三个好处

第一是可算。 五本账都有闭式或半闭式的估算式：$6.02N + 1.76$、$-20\lg(2\pi f \sigma_t)$、$\lambda_{DU}(T_{proof}/2 + MTTR)$、$SE = A + R + B$。估出来的量级与实测差不超过一倍，足以在方案阶段做取舍。

第二是可验收。 每一项都有明确的判据与单位（LSB、dB、次/小时、SIL），可以写进合同、写进测试大纲，不靠“专家评审”拍板。

第三是可追责。 出了问题能定位到哪一本账、哪一行数据。§92.10.6 的返工案例就是靠这张表逐项对账，四项各占一部分，责任清楚。

92.11.2 五处局限

一阶效应。 抖动用 $\dot{y}\delta$ 近似，忽略了二阶项；量化用均匀独立近似，忽略了相关性；屏蔽用平面波近似，忽略了近场。这些近似在“误差小于 1 LSB”的场景够用，但接近极限时就不准。

不含全波 EMC。 本章只算平面波屏蔽与缝隙上限，不做场仿真。机柜内部的谐振、线缆的共模辐射、PCB 的走线耦合，都要靠 CST、HFSS 这类工具。

不含 FMEDA。 安全账只算了 PFD 与 SIL 分档，没有做失效模式、影响与诊断分析。真正的 SIL 认证要求逐器件算失效率，工作量是本章的上百倍。

不含最坏执行时间分析。 本章用实测分位数，适合已有原型；设计阶段没有实测数据时，要用静态最坏执行时间分析（WCET），那是另一套工具。

不含人因与运维。 五本账都是技术账，不含“操作员会不会误操作”“备件够不够”这类问题。它们同样能决定项目成败，但不在本章范围。

92.11.3 三种行业的权重不同

表 92-15 把三种典型行业的五本账权重摆在一起，权重量级用“最敏感的那一项”表示。

表 92-15 三种行业的五本账权重

行业	幅值账	时间账	计算账	能量账	安全账
过程控制	中（12 位够用）	低（1 kHz 以下）	中（周期 100 ms–1 s）	高（4–20 mA 环路、地环路）	高（SIL 2/3）
运动控制	高（17–20 位编码器）	高（μs 级抖动）	高（100 μs 周期）	中（驱动器干扰）	高（急停、STO）
电力电子	中（12–14 位）	极高（ns 级抖动）	极高（10 μs 周期）	极高（dv/dt、di/dt）	中（过流保护）

三种行业的最敏感项分别是：过程控制看安全与能量，运动控制看幅值与时间，电力电子看时间与计算。**同一套方法，权重完全不同**，这是“工程问题”没有统一答案的根本原因。

以抖动为例：过程控制 1 kHz、12 位，1 LSB 门限 77.7 ns，几乎不用管；运动控制 20 位编码器、10 kHz，台阶缩到 9.54 μV，门限压到 0.03 ns，必须用差分时钟与同步采样；电力电子 12 位、100 kHz 开关，门限 0.78 ns，同样只能靠硬件同步。**同一个公式，跨了三个数量级。**

92.11.4 什么时候不该用这套方法

三种情况，本章的方法会帮倒忙：

- **原型阶段、数据不足。**噪声预算要实测共模与干扰频谱，执行时间要实测分布。没有这些数据，硬套公式只会得到一堆假精度。
- **安全等级 SIL 3 以上。**本章的 PFD 估算只是数量级，真正的认证要求 FMEDA、失效树、共因失效分析，不能拿本章的表去交差。
- **高频、分布式系统。**开关频率到兆赫兹、回路分布在几十米的系统，传输线效应、地弹、共模辐射都不可忽略，需要场与路的联合仿真。

92.11.5 与相邻章的边界

- 第 84、85 章讲“结构带来的采样与延迟”，本章讲“实物带来的量化、抖动、干扰、失效”；
- 第 86 章讲“任务怎么调度”，本章只讲“调度失败会怎样”；
- 第 90 章讲“软件怎么保证不错”，本章讲“硬件与现场怎么让软件的努力白费”；
- 第 89 章讲通信协议，本章只讲通信线缆的 EMC；
- PID、MPC、Smith 的算法细节分别在第 44/106、58/80、47/107 章，本章不重复。

一句话概括边界：**前面几章保证“设计对”，本章保证“装得上、跑得稳、坏得安全”**。

92.11.6 两场争论

第一场：模型能不能代替实测。

一派认为五本账都是解析式，算一遍就够；另一派认为现场千变万化，算出来的数不可信。本章的立场在两者之间，理由是本章有四个“仿真先错、实测纠正”的例子：过采样的参考值取错、闭环抖动的随机数没共用、HIL 的参考解与待测解同源、定点的误差界用错了滤波器。

这四处的共同点是“公式写对了，对象写错了”。参考值该取窗内平均却取了中点，随机数该共用却重新播种，参考解该独立却用了同类方法，误差界该用误差滤波器却用了原滤波器。**四种错法都有一个特征：结果看起来合理**——35.81 dB 不算离谱，IAE 全为 0.2599 也像个数，误差随步长不单调也能解释成“数值噪声”。只有拿实测一对，才知道全错。

所以正确的用法是：**用模型定量级与排序，用实测定数值与判据**。模型告诉你“共模是最大项”，实测告诉你“它是 2.05 还是 0.20 LSB”。反过来，只靠实测无法穷举工况，只靠模型无法发现建模错误。

第二场：余量该加在哪。

一派主张每项都留 50% 余量，稳妥；另一派主张精确预算，一分不多。本章的算法是第三种：**只在最大项上加余量**。

理由在平方和上。本章最大项是 1.00 LSB，占 $1.00^2/1.13 = 88\%$，在它上面加 20%，总误差涨约 19%；在小项 0.05 上加 20%，总误差只动 0.2%。同理，如果某一项加了余量之后仍然不是最大项，那这个余量基本白加。

工程上常见的“每项都乘 1.5”看起来稳妥，实际是把预算堆在无关紧要的地方，真正的短板一点没动，还顺手把成本抬了 50%。**余量不是均匀撒的，是压在最弱环节上的**，这与结构设计里“加强最弱构件”是同一条道理。

92.11.7 这套方法还能用多久

五本账的结构不变，但每一本的“默认值”在变。三件事正在改写本章的权重。

第一，高分辨率 ADC 在变便宜。 24 位 $\Delta\Sigma$ 器件已经做到几元钱，量化项的 6.02 dB 规律还在，但“位数不够”这个矛盾基本消失了。后果是幅值账的最大项从量化转向前端噪声与执行机构（§92.4.8），本章 1 LSB 封顶的结论会比十年前更常见。

第二，数字隔离在变便宜。 120 dB 的隔离从“上千元”降到“几十元”，能量账里共模这一项的性价比排序被彻底改写。本章表 92-6 的 120 dB 那一列，以前是实验室指标，现在可以是量产配置。

第三，时间触发与 TSN 在普及。 抖动从微秒级压到纳秒级，时间账的瓶颈从“时钟”转到“采样保持器”与“模拟前端建立时间”。本章 77.7 ns 的预算，在多数新系统里会变成“根本不用算”。

不变的是三件事：**缝隙仍然是屏蔽的瓶颈，执行时间的尾部仍然服从分布，诊断覆盖率仍然决定 SIL**。这三件事由物理与标准决定，不受器件价格影响。所以本章的方法可以分成两半：**器件参数那一半每五年要重查，记账结构那一半可以一直用**。

给读者的建议很直接：把本章的公式记住，把本章的数字当成“量级参照”，每次做新项目时用当前的器件手册替换一遍。**公式是资产，数字是快照**，混为一谈就会拿十年前的 12 位去套今天的 24 位。

92.11.8 五句可以直接抄进评审意见的话

评审工程方案时，最容易出现的场面是双方各说各的经验。下面五句话能把讨论拉回到可核查的层面，每句都对应本章的一本账。

第一句：“这个 1 LSB 是有效值还是峰值，测试条件是什么？”对应幅值账。有效值与峰值差 $\sqrt{2}$ 到 3 倍，条件不明就没法比。看到只写“纹波小于 1 LSB”而没有条件，按峰值理解并追问。

第二句：“共模电压是实测最坏值，还是手册上的典型值？”对应能量账。本章 5 V 配 60 dB 是 2.05 LSB，配 80 dB 是 0.20 LSB，十倍之差全在“实测还是推测”这一个问题上。

第三句：“P99.99 是在目标机上测的，还是在 PC 上估的？”对应计算账。PC 上估出来的执行时间通常只有目标机的三分之一到一半，因为缓存、中断、总线竞争全都没算。**没有目标机实测的 P99.99，等于没有。**

第四句：“屏蔽效能是材料算的，还是整柜测的？缝有多长？”对应能量账的另一半。本章的例子最典型：材料 120.9 dB，100 mm 缝只有 63.5 dB，差 57 dB。只报材料值的方案，基本可以判定没有做过整柜测试。

第五句：“SIL 2 的 PFD 是只算了软件诊断，还是含硬件安全输出？”对应安全账。本章的 400 ms 失联时间在多数工艺下过不了 200 ms 的失控要求，靠软件压不下来，必须有硬件路径。只报软件覆盖率的方案，SIL 等级是虚的。

五句话的共同点是：**都在问“这个数是怎么来的”，而不是“这个数够不够”**。数值够不够，取决于判据；判据取决于工艺。而“怎么来的”是唯一能在会上当场核实的部分。**把这一层问清，后面所有讨论才有意义。**

92.12 Python 仿真代码

本章全部数字由 `code/ch092_sim.py` 生成，十张图由 `code/ch092_figs.py` 生成。脚本无命令行参数，种子固定为 9201，重跑逐位一致。

92.12.1 十一段结构

脚本按正文顺序分十一段，每段对应一本账或一个交叉问题：

段	内容	对应小节	主要输出
[0]	量化信噪比与有效位	§92.2.1–92.2.2	8–16 位 SNR、16 倍过采样、抖动
[1]	采样抖动的实测与理论	§92.2.3	九组 SNR、1 LSB 门限
[2]	噪声决定的有效分辨率	§92.7.1	无噪声码位数
[3]	执行时间与超时率	§92.2.4	四个分位数、三档超时率
[4]	抖动进入闭环	§92.5.3	最大偏差、RMS、IAE
[5]	屏蔽效能分解	§92.4.4	A/R/B/SE、缝隙上限
[6]	共模折成差分误差	§92.4.5	八组 μV 与 LSB
[7]	看门狗与安全完整性	§92.6.5	三段时序、四档 PFD
[8]	定点系数误差	§92.5.5	四档 SNR、最大误差、L1 界
[9]	HIL 步长与积分方法	§92.9.5	八组误差、两条斜率
[10]	自检	—	三条断言

自检的三条断言是：每加一位信噪比涨 5.6–6.4 dB；抖动增大十倍信噪比降 15 dB 以上；16 倍过采样增益 1.7–2.3 位。第三条断言是写完第一版后补的，因为第一版的过采样结果只有 35.81 dB，明显不对（见 §92.12.3）。

92.12.2 三个关键实现

量化与平均。量化用 `np.round(y/step)*step`，台阶 `step=R/2**N`。过采样时要注意参考值取哪一个：取窗口中点采样会把平均本身的孔径误差算进去，取窗内平均才是平均滤波器的理想输出。

抖动闭环的可比性。要让“有抖动”与“无抖动”两次运行逐点可比，必须共用同一组标准正态数 z，只把 σ_t 乘上去；否则两次的随机序列不同，相减得到的是两次跟踪误差之差，而不是抖动效应。

HIL 的参考解。参考解用增广矩阵的矩阵指数 `expm(M*h)` 递推，是精确解；若用“另一次更小步长的数值仿真”当参考，小步长自身也有误差，会出现“步长越小误差反而越大”的假象。

92.12.3 复现最容易错的三处

第一处：过采样的参考值。 若写成 `tref` 取窗口中点，12 位 16 倍过采样的信噪比只有 35.81 dB，等效 5.66 位，比不平均还差 6.36 位。这个结果不是 bug，而是“平均”被误当成“取样”：997 Hz 信号在 100 μs 窗内走了 36°，窗口中点的值不等于窗内平均值，差值 0.081 V，比量化噪声大得多。改成窗内平均后是 85.33 dB、13.88 位，与理论 2 位只差 0.14 位。

第二处：抖动闭环的随机数。 若每次调用都重新播种，四档抖动的 IAE 全是 0.2599、偏差全是 0，看起来“抖动毫无影响”。实际是两次运行的时间序列不同，相减把抖动效应抵消了。共用同一组 z 之后，200 μs 抖动的最大偏差是 0.517 mV。

第三处：HIL 的参考解。 若参考解用 RK4、步长 10 μs，那么在 2 ms 上算出的误差是 2.68×10^{-9}，在 0.5 ms 上算出 2.20×10^{-5}——步长小了四倍，误差反而大了四个数量级。这是参考解与待测解用了同一种方法、误差同向抵消造成的。换成矩阵指数后，RK4 的误差随步长严格按 $1/16$ 下降：$2.68 \times 10^{-9} \to 1.65 \times 10^{-10} \to 1.02 \times 10^{-11}$。

92.12.4 核心代码

下面是量化与抖动的两个函数，完整脚本见 `code/ch092_sim.py`。

```
def adc_snr(nbits, rng=10.0, fs=10000.0, f0=997.0,
            nsamp=8192, osr=1, dither=False, seed=9201):
    """量化后的信噪比；osr>1 时先平均再降采样。"""
    rng_obj = np.random.default_rng(seed)
    n_hi = nsamp * osr
    t = np.arange(n_hi) / (fs * osr)
    amp = 0.5 * rng * 0.999
    y = amp * np.sin(2.0 * math.pi * f0 * t)
    y_clean = y.copy()
    step = rng / 2.0 ** nbits
    if dither:
        y = y + rng_obj.uniform(-0.5, 0.5, n_hi) * step
    yq = np.clip(np.round(y / step) * step, -0.5 * rng, 0.5 * rng)
    if osr > 1:
        yq = yq.reshape(nsamp, osr).mean(axis=1)
        yref = y_clean.reshape(nsamp, osr).mean(axis=1)
    else:
        yref = y_clean
    err = yq - yref
    return 10.0 * math.log10(np.mean(yref ** 2) / np.mean(err ** 2))


def jitter_snr(f0, sigma_t, fs=100000.0, nsamp=20000,
               amp=1.0, nbits=18, seed=9201):
    """采样时刻抖动引起的信噪比。"""
    rng_obj = np.random.default_rng(seed)
    tn = np.arange(nsamp) / fs
    tk = tn + rng_obj.normal(0.0, sigma_t, nsamp)
    y = amp * np.sin(2.0 * math.pi * f0 * tk)
    yref = amp * np.sin(2.0 * math.pi * f0 * tn)
    step = 2.0 * amp / 2.0 ** nbits
    err = np.round(y / step) * step - yref
    return 10.0 * math.log10(np.mean(yref ** 2) / np.mean(err ** 2))
```

两段代码的写法有同一个讲究：**误差用“量化后减理想值”直接算，不做频谱分析**。时域误差法不受频谱泄漏与窗函数影响，8192 点就能稳定到 0.1 dB；若用 FFT 找信号峰再算噪声，还得处理非整周期采样，反而更麻烦。

92.12.5 结果核对

跑完脚本应与下面六个数对得上，对不上说明环境或版本有差异：

- `12 位：SNR=74.10 dB`，`ENOB=12.02`；
- `12 位 + 16 倍过采样平均：SNR=85.33 dB`，`增益 1.86 位`；
- `1 kHz、1 μs：实测 43.99 dB，理论 44.04 dB`；
- `周期 1.0 ms：超时率 2.60e-04，每小时 936.0 次`；
- `5 V、CMRR 80 dB：误差 500.0 μV = 0.20 LSB`；
- `h=2.00 ms：RK4 2.68e-09，后向欧拉 2.16e-03`。

末行应打印 `自检通过：SNR(12/13 bit)=74.10/79.92 dB，抖动 SNR 1 μs→10 μs = 44.0→24.0 dB`。这三条断言任一失败，脚本会直接抛 `AssertionError`，不会静默出数——这是本章所有数字可信的前提。

92.12.6 运行环境与复现条件

脚本在 Python 3.11、NumPy 2.x、SciPy 1.x 下跑通，只用三个外部依赖：`numpy`（数组与随机数）、`scipy.signal`（`butter` 与 `lfilter`）、`scipy.linalg`（`expm`）。画图脚本额外用 `matplotlib`。全部为纯计算，不读写数据文件，也不联网。

复现有四处需要注意的地方，都很细但都会影响结果：

第一，随机数要用 `default_rng`。 老接口 `np.random.seed` 与新接口 `default_rng(9201)` 产生的是两套完全不同的序列，混用会让“种子固定”这句话失效。本章全部用 `default_rng`，它的 PCG64 算法在 NumPy 各版本间稳定。

第二，`butter` 的系数要按二阶节直接取。 100 Hz、1 kHz 采样的二阶低通，浮点系数是 $b = [0.067455, 0.134911, 0.067455]$、$a = [1, -1.142981, 0.412802]$；Q15 量化后是 $b = [0.06744385, 0.13491821, 0.06744385]$、$a = [1, -1.14297485, 0.41281128]$。这两组数差在小数点后第四位，肉眼几乎看不出，但对定点误差的影响是决定性的——表 92-9 的全部差异都来自这几位。

第三，矩阵指数要按增广矩阵算。 §92.12.3 已经说过，参考解必须独立于待测解。本章把状态方程扩成三阶（两个状态加一个常数输入），用 `expm(M*h)` 一步推进，这样得到的参考解是精确的，与步长无关。

第四，多线程 BLAS 会带来 10^{-12} 量级的差异。 矩阵乘法在不同线程数下的累加顺序不同，末位会有差别。这一项对本章的所有结论都没有影响（最短的那个数是 1.57×10^{-14}，仍在双精度噪声之上一个量级），但如果你要对到十位有效数字，就要把线程数固定为 1。

整个脚本在本机跑完约 20 秒，其中一半时间花在 HIL 的四档步长上——0.1 ms 那一档要积分十二万步。想快一点可以只跑到 0.5 ms，但表 92-13 的最后一行就没了，而那一行恰好是验证“RK4 严格四阶”最有力的一行。

十张图由 `code/ch092_figs.py` 生成，输出到 `figs/ch092/`，每张同时出 SVG 与 PDF：SVG 给网页与排版用，PDF 给印刷用。图内文字全部是中文纯文本，公式不进图，这是全书统一的规矩——图里放公式，字号一缩小就糊，而且换字体就乱码。

92.12.7 图脚本的四个部分

画图脚本与仿真脚本分离，原因只有一个：**图要能单独重画**。改一次配色不应该重跑二十秒的仿真，所以 `ch092_figs.py` 里所有数都是字面量，与正文表格逐位对齐。

脚本分四部分：

- **公共设置**：字体、字号、线宽、配色。中文字体显式指定，避免出图时一堆缺字形警告；配色固定为五种，正文里说“第三条线”时，读图的人也数得清。
- **十个绘图函数**：一个函数一张图，横纵轴、刻度、标注都在函数里定死。函数之间不共享变量，改一张不影响别的。
- **数字来源**：每张图的数写成列表，紧挨着画图代码，注释里标出对应的小节号。**图与表用同一组数**，对不上就是错。
- **导出循环**：遍历十张图，各存一份 SVG 与一份 PDF，文件名与正文的引用一致。

四条画图规矩，都是为“能读懂”服务：

第一，双轴图要标清哪条线对哪个轴。 图 92-2 的台阶用左轴、信噪比用右轴，两个量纲差六个数量级，不标清楚就会被读成一条线。

第二，对数轴的刻度要标实数。 图 92-9 的横轴是步长，标 2 ms、1 ms、0.5 ms、0.1 ms，而不是 10^{-3} 这种科学记数。工程读者看的是“毫秒”，不是“指数”。

第三，参考线要画出来。 图 92-4 的三条周期线、图 92-6 的四条等值线、图 92-9 的两条斜率线，都是判据本身。**图的价值在于让人一眼看出“越线没有”**，不在于把数据点画全。

第四，不画没信息的东西。 网格只留主网格，图例只留必要的，标题写在正文图题里而不是图内。图内多一行字，正文就少一行注意力。

92.12.8 三条断言为什么是这三条

自检只留三条断言，不是随便挑的。每一条卡住一类最容易犯的建模错误。

第一条，每加一位涨 5.6–6.4 dB。 它卡的是量化模型：台阶口径（满量程还是半量程）、误差分布（均匀还是三角）、参考值（理想值还是量化值）。这三处任何一处写错，斜率就不再是 6.02。区间取 ± 0.4 dB，是因为 8192 点的随机抽样本身有 0.1 dB 量级的波动。

第二条，抖动增大十倍，信噪比降 15 dB 以上。 它卡的是抖动模型的时间尺度：σ_t 是秒还是微秒、误差按 $\dot{y}\delta$ 还是按 δ 直接加。单位错一千倍，这条立刻红。下限取 15 dB 而不是 20 dB，是给“抖动已经大到接近量化噪声”的情况留余量——理论是 20 dB，实测 44.04 到 24.04 正好 20.0 dB。

第三条，16 倍过采样的增益在 1.7–2.3 位之间。 它卡的是参考值的取法，也就是 §92.12.3 的第一处。参考值取窗口中点时增益是负的，取窗内平均才是正的 1.86 位。这条断言是写完第一版之后补的，因为第一版没有这条，错了也没人发现。

三条断言的共同点是：**每条只卡一个自由度**。如果一条断言同时卡住两个模型，红了之后分不清是谁错。工程上的自检应当照这个原则写：一条断言一个假设，区间给足，宁可漏报也不要误报——误报多了，人就会习惯性忽略自检。

断言不是测试覆盖率的替代品。本章的仿真没有覆盖“前端噪声有色”“执行时间有周期性尖峰”“屏蔽腔体谐振”这些情况，它们要么超出本章范围，要么需要场仿真。**断言只保证“模型没写错”，不保证“模型够用”**，这两件事必须分清。

第93章 计算机控制工程落地

93.1 动机与直观理解

93.1.1 三个现场场景

第一个场景：演示很成功，现场拖了三个月。实验室里六个回路一次投通，到了现场，一个阀门行程不对，一个信号量程反了，一个联锁条件写反了。每个问题都不难，难的是每次都要等停机窗口。

第二个场景：升级当晚回退。新版本在原有机柜上跑得好，装上现场那台老控制器后通信中断。现场没有回滚包，只能凭记忆把组态一页页改回去，改了四个小时。

第三个场景：验收会开成了汇报会。业主问报警负荷是多少，答“比原来少多了”；问可用度怎么算的，答“一直很稳定”。会议开了三小时，没有一项能签字。

三个场景的共同点不是技术不行，是**没有留下可核对的证据**。落地做得好不好，往往不取决于算法多先进，而取决于每个阶段结束时，能不能拿出一张别人能复核的单子。

93.1.2 四个词的区别

本章反复用到四个词，含义不同，混用会让责任落空。

交付是把东西交到对方手上，标志是签收单。交付可以发生在系统还不能用的时候。

投运是系统开始承担生产任务，标志是第一个回路投入自动。投运时系统能用，但不一定好用。

验收是按事先约定的指标逐项核对，标志是验收报告。验收要的是数据，不是印象。

运维是投运之后直到退役的整个过程，标志是值班表与备件账。运维期通常比建设期长五到十倍。

四个词的顺序是交付、投运、验收、运维。很多项目把验收放在投运之前，结果验收时没有真实工况数据，只能按实验室数据签，留下隐患。

93.1.3 与前后章的分工

第 90 章讲软件工程的方法：需求、设计、测试、发布。它回答“代码怎么组织”。

第 91 章讲数字孪生与边缘计算：保真度、同步、在环测试。它回答“模型放在哪、准不准”。

第 92 章讲信息安全：边界、分区、认证。它回答“开放网络里怎么守”。

本章讲的是把这三样串起来的那条线：一个项目从需求冻结走到三年运维，中间要过几道门、每道门要什么证据、出了问题怎么退回去。

所以本章不重复推导 PID，也不展开协议细节。需要时指回第 44、89、90 章。

93.1.4 五个关键词

阶段门是项目阶段之间的检查点，过了才能进下一阶段。门的价值在于“能停”，不能停的门只是汇报。

证据是可复核的记录：数据、截图、签字、日志。形容词不算证据。

版本是固件、组态、上位机、协议四类组件的组合。四类各有版本，组合起来才是“这一套”。

灰度是先让一小部分设备用新版本，观察后再扩大。它是把“一次性风险”切成“可承受的几次风险”。

回退是发现不对时退回上一个可用状态。回退能力要在升级之前就准备好，不能等出事再想。

五个词里最重要的是回退。一个不能回退的系统，任何一次升级都是赌博。

93.1.5 一条主线

本章的主线只有一句：**每个阶段都要有进入条件和退出证据，每次变更都要留好回退路**。

进入条件回答“凭什么开始”，退出证据回答“凭什么结束”。两者都不写，阶段就变成了日历上的一个区间。

回退路回答“错了怎么办”。它不是悲观，是把最坏情况的代价提前算清楚。

这条主线贯穿全章：缺陷拦截算的是“早发现省多少”，版本兼容算的是“哪些组合可以承诺”，灰度算的是“多大概率能提前发现”，回滚预算算的是“退回去要多久”。

后面几节讲工期、报警、可用度、备件，都是这条主线在不同侧面的展开。

93.1.6 三个数字先记住

第一个是 22.7 倍。设计阶段发现一个缺陷的平均修复成本是 1 人时，拖到投运后是 150 人时，期望成本 6.62 对 150，相差 22.7 倍。这个数字回答“为什么要设阶段门”。

第二个是 35 分钟。四段回滚时间相加是 35 min，超过 30 min 的目标；加了预置回滚包与自动脚本后压到 25 min。这个数字回答“回退能力够不够”。

第三个是 4 人 3 天。40 个回路的现场调试，1 个人要 75.3 h（10 个工作日），4 个人并行 18.8 h（3 个工作日）。这个数字回答“现场要派几个人”。

三个数字对应三类决策：投不投入前期评审、要不要做回滚预案、现场排几个人。它们都能在十分钟内算出来，而省下的往往是几周。

93.1.7 本章不做什么

本章不做产品选型，不比较哪家 PLC 或哪家云平台。选型取决于供应、备件、既有习惯，没有通用答案。

本章不重复控制算法。整定方法见第 44 章与第 106 章，模型预测见第 58 章与第 80 章，本章只在需要时引用结论。

本章不给报价，也不给人力单价。不同地区、不同行业的差异太大，给了反而误导。本章只给工时与停机小时，换算成钱由读者按自己的口径做。

本章的案例都做了脱敏处理。数字取自脚本与现场记录的量级，不对应任何具体项目。

93.1.8 一个项目的时间线

把六段四门摊到日历上，能看到落地的真实节奏。

第 1 到 2 周做需求冻结。产出基线表、回路清单、数据点表。这两周看起来慢，但它决定了后面所有工作的边界。

第 3 到 6 周做离线仿真。组态、逻辑、人机界面在这里成型，同时跑 MIL 测试。G1 与 G2 之间通常隔四到六周，这是整个项目最安静的一段。

第 7 周做出厂测试。按 26 种允许组合跑回归，67.6 h 的工时摊到五个工作日。G2 通过后设备发运。

第 8 周做现场调试。40 个回路、3 到 4 个人、3 到 4 天，加上前期准备与后期收尾，整周都在现场。G3 通过后开始灰度。

第 9 到 11 周做灰度投运。三批设备，每批观察一到两个班次，中间留修复窗口。G4 通过后全量放开。

第 12 周开始验收。五项指标取证，文档齐套核对，开验收会。

第 13 周进入运维。值班、巡检、备件、变更管理按 §93.10 的节奏走。

这条时间线的前提是“一切按计划”。真实项目里，现场条件不符、设备到货延迟、业主改需求，任何一件都会把它拉长。时间线的作用不是预测，是让每个环节的延期都能被看见——哪一段超了、超了多久、超的原因是什么。

对时间线最常见的一个误解是“可以并行”。仿真与出厂测试可以部分并行，现场调试与灰度不能。灰度必须等现场全部投通，否则观测到的现象分不清是新版本引起的还是未投回路引起的。

93.1.9 怎么读这一章

第一次读，建议按 §93.1、§93.2.3、§93.2.4、§93.8、§93.10 的顺序。这五节覆盖了“为什么”“灰度怎么定”“回滚怎么算”“参数怎么调”“现场怎么干”，是本章的主干。

要写方案或做评审，重点看 §93.3 与 §93.7。前者给步骤与输出物，后者给参数取值与敏感度排序。

要做实施，重点看 §93.10 与 §93.12。前者是日程与流程，后者是能跑的代码与能查的数。

§93.4 到 §93.6 是方法的细化，可以后读。§93.11 是全章的边界声明，读它是为了避免误用。

如果只有十分钟，读 §93.1.6 的三个数字与 §93.11.12 的一页纸结论，够用了。

93.2 数学原理

93.2.1 缺陷逃逸与期望成本

落地过程是一条串联的拦截链：设计评审、桌面检查、MIL、HIL、现场调试，每一级拦下一部分缺陷，没拦住的进入下一级。

设第 i 级的检出率为 p_i，则经过 i 级之后仍未发现的缺陷比例为

$$E_i = \prod_{j \le i} (1 - p_j) \tag{93-1}$$

一个缺陷在第 i 级被发现，前提是前面几级都漏掉了它，概率是 $P_i = \left(\prod_{j<i}(1-p_j)\right) p_i$。把每级的修复成本按概率加权，再算上逃逸到投运后的那一部分，得到单个缺陷的期望修复成本

$$C = \sum_{i} P_i c_i + E\, c_e \tag{93-2}$$

其中 c_i 是第 i 级的单位修复成本，c_e 是投运后才发现时的成本。

表 93-1 按五级拦截加一个逃逸出口算出全部数。五级检出率分别是 30%、45%、60%、80%、92%，单位成本从 1 人时涨到 150 人时。

阶段	检出率 / %	本级拦下 / %	累计逃逸 / %	单位成本 / 人时	期望成本 / 人时
设计评审	30.0	30.00	70.000	1.0	0.300
桌面检查	45.0	31.50	38.500	2.0	0.630
MIL	60.0	23.10	15.400	5.0	1.155
HIL	80.0	12.32	3.080	20.0	2.464
现场调试	92.0	2.83	0.246	60.0	1.700
投运后发现	—	—	0.246	150.0	0.370
合计	—	—	—	—	6.62

表 93-1 的关键不在合计，而在最后一列的形状：前三级便宜但漏得多，后两级拦得住但贵。期望成本 6.62 与全部拖到投运后的 150 相差 22.7 倍，这个倍数就是阶段门的价值。

93.2.2 版本组合与回归量

固件、组态、上位机、协议四类组件各有三个版本，全组合数是

$$N = \prod_{k} n_k \tag{93-3}$$

四类各三版，$N = 81$。全部组合都测一遍，回归工时是

$$T_{\mathrm{reg}} = A\, m\, t (1 + \eta) \tag{93-4}$$

其中 A 是要测的组合数，m 是每组合的用例数，t 是单例时长，η 是准备系数。

实际只承诺满足兼容规则的组合：协议版本不高于固件、上位机版本不低于组态、固件与组态相差不超过一版。按这三条筛选，81 种里剩下 26 种，占 32.1%，两种口径的对照见表 93-2。

口径	组合数	用例数	工时 / h	工作日 / 天
全组合回归	81	3240	210.6	27
只测允许组合	26	1040	67.6	9

表 93-2 说明兼容矩阵的实际作用：它砍掉的是 68% 的测试量，不是砍掉功能。代价是要对客户明确“不支持哪些组合”，这句话必须写进合同。

93.2.3 灰度的暴露概率

灰度解决的问题是：只有少量设备时，能不能提前发现缺陷。

设单台设备触发缺陷的概率为 p，一批有 n 台，至少一台触发的概率是

$$P_{\mathrm{hit}} = 1 - (1 - p)^{n} \tag{93-5}$$

$p = 5\%$、设备总数 200 台时，表 93-3 给出各批次的结果。

批次占比 / %	设备数 / 台	至少一台触发 / %
1	2	9.75
5	10	40.13
10	20	64.15
25	50	92.31
50	100	99.41
100	200	100.00

表 93-3 的读法是“要看多准才敢全量”。1% 批次只有 9.75% 的把握发现问题，等于没测；10% 批次到 64.15%，已经开始有用；50% 批次 99.41%，但这时候一半设备已经受影响，灰度的意义被削弱。

图 93-3 把这条曲线画出来，拐点在 25% 附近。超过 25% 再增加批次，暴露概率的增幅迅速变小，风险却线性上升。

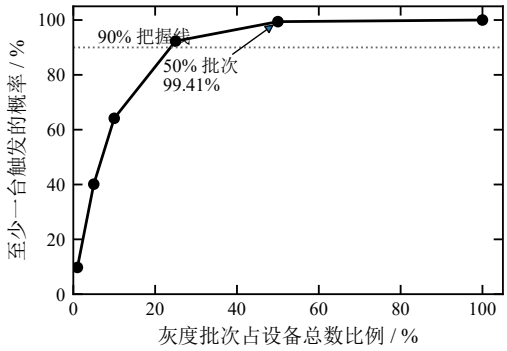


图 93-3 灰度批次比例与至少一台触发缺陷的概率

93.2.4 回滚时间预算

回滚不是一个动作，是四段时间相加：

$$T_{\mathrm{rb}} = \sum_{i} t_i \tag{93-6}$$

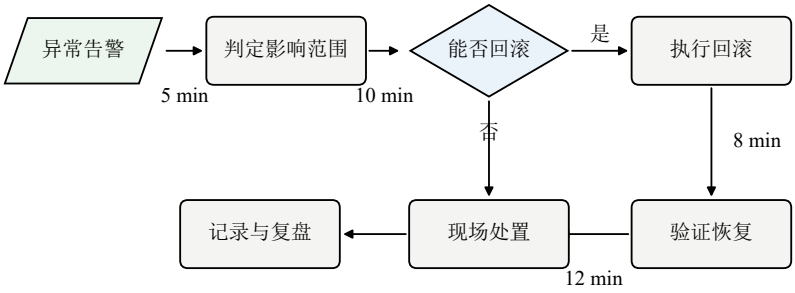
四段分别是发现异常、确认与决策、执行回滚、验证恢复。表 93-4 给出本章的取值。

阶段	耗时 / min	累计 / min
发现异常	5.0	5.0
确认与决策	10.0	15.0
执行回滚	8.0	23.0
验证恢复	12.0	35.0

表 93-4 合计 35 min，比 30 min 的目标多 5 min。三项改进各节省 3、4、3 min，合计压到 25 min，余量 5 min。

三项改进都不涉及技术难度：预置回滚包、自动执行脚本、预演过的验证清单。它们唯一的共同点是**必须在升级之前准备好**。

图 93-4 把这条链路画成决策流程。判断“能否回滚”是唯一的菱形，答“否”时走现场处置，这条分支在真实项目里用到的概率不低，不能省。



四段合计 35 min；预置回滚包与自动化脚本可压到 25 min

图 93-4 回滚决策流程与四段时间预算

93.2.5 现场调试的工期

现场调试的工时由三块构成：每个回路都要做的点检、单回路投用、联锁与顺控。总工时除以人数与并行效率，就是工期：

$$H = \frac{N_{\text{loop}}\, t_c + T_0}{m\, \eta} \tag{93-7}$$

其中 N_{loop} 是回路数，t_c 是单回路工时，T_0 是与回路数无关的固定工时（组态下装、通信建立、联锁总测），m 是并行人数，η 是并行效率。

本章取 40 个回路、单回路 1.2 h（点检 0.3、单回路 0.6、联锁 0.3）、固定 16 h、效率 0.85，结果见表 93-5。

并行人数 / 人	总工时 / h	折合工作日 / 天
1	75.3	10
2	37.6	5
3	25.1	4
4	18.8	3

表 93-5 的边际收益递减很清楚：从 1 人到 2 人省 5 天，从 3 人到 4 人只省 1 天。原因是固定工时 16 h 不随人数减少，人越多，摊到每个人头上的固定工时越薄，但总也摊不完。

图 93-5 把工时与工作日画在一起。工时是双曲线，工作日是阶梯，两者在 3 人附近开始分离，这个分离就是排班时要看的信号。

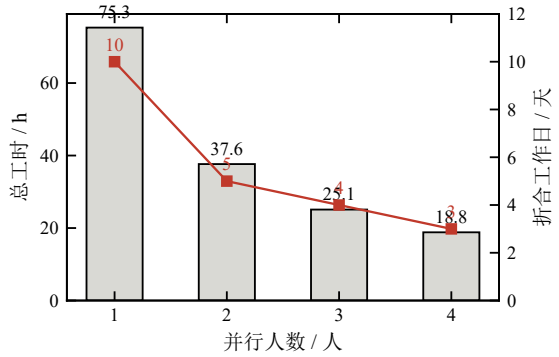


图 93-5 并行人数对总工时与工作日的影响

93.2.6 报警负荷的逐项削减

报警负荷按措施逐项连乘：

$$L = L_0 \prod_{i} (1 - r_i) \tag{93-8}$$

其中 L_0 是原始负荷，r_i 是第 i 项措施的削减比例。四项措施依次是清理无效报警、死区与滤波、延时确认、分级抑制，逐项结果见表 93-6。

措施	削减 / %	负荷 / (条/h)	累计降幅 / %
清理无效报警	65	42.00	65.0
死区与滤波	40	25.20	79.0
延时确认	50	12.60	89.5
分级抑制	10	11.34	90.5

表 93-6 的最终负荷 11.34 条/h，比操作员上限 12 条/h 低 0.66。余量不大，说明这四项措施已经用尽，再要下降只能改工艺或加人。

图 93-6 把这四级削减画成瀑布。前两级砍掉 79%，是主要贡献；后两级只再降 11.5 个百分点，但恰好把负荷压到线下，不能省。

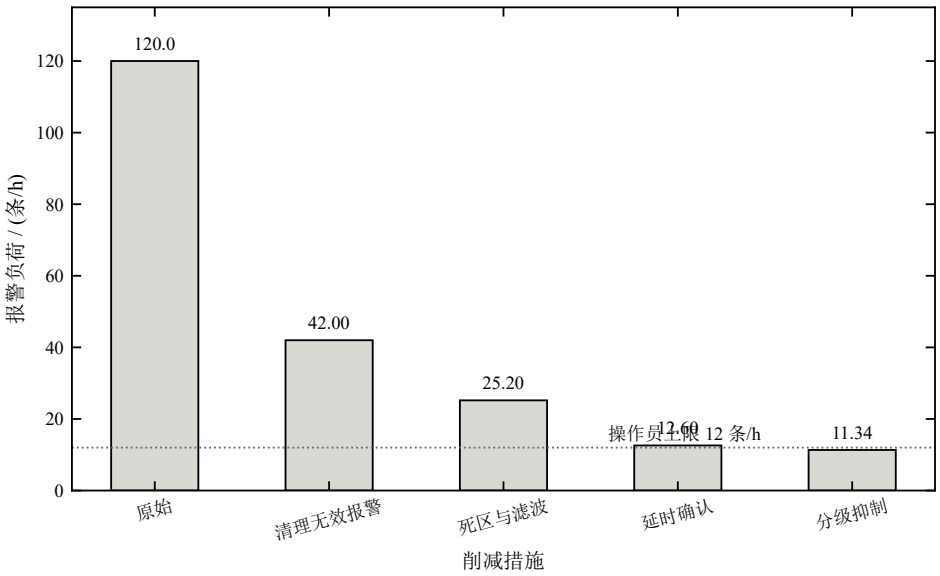


图 93-6 四项措施对报警负荷的逐级削减

93.2.7 可用度与年停机

可用度是最容易被含糊过去的指标，因为它的定义不止一种。本章用稳态可用度：

$$A = \frac{MTBF}{MTBF + MTTR} \tag{93-9}$$

年停机小时由同一组参数给出：

$$D = 8760 \, \frac{MTTR}{MTBF + MTTR} \tag{93-10}$$

本章取 MTBF 为 2000 h，MTTR 由发现、定位、修复、验证四段加路程组成，见表 93-7。

阶段	耗时 / h	累计 / h
发现	0.5	0.5
定位	1.5	2.0
修复	1.0	3.0
验证	0.5	3.5
路程	0.5	4.0

表 93-7 的 MTTR 是 4.0 h，可用度 99.800%，年停机 17.49 h。加远程诊断与备件在库两项改进后，MTTR 降到 2.5 h，可用度 99.875%，年停机 10.94 h。

图 93-8 把改善前后的分段画成两条横条。改善的 1.5 h 全部来自定位与修复，发现与验证两段没变，这提示下一步该往哪投。

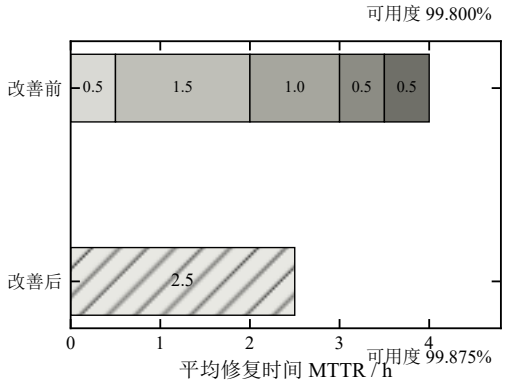


图 93-8 改善前后的 MTTR 分段与可用度

93.2.8 备件库存

备件数量是整数，缺货概率用泊松分布算：

$$P_{\text{out}} = P(X > k), \qquad X \sim \text{Poisson}(\lambda), \quad \lambda = \frac{8760}{MTBF} \tag{93-11}$$

$\lambda = 4.38$ 次/年时，表 93-8 给出 2 到 8 件的缺货概率与年期望等待时间。

备件数 / 件	缺货概率 / %	年期望等待 / h
2	81.247	59.04
3	63.706	39.54
4	44.498	24.25
5	27.672	13.57
6	15.389	6.93
7	7.703	3.24
8	3.495	1.39

表 93-8 的读法是“愿意等多久”。备件从 4 件加到 6 件，年等待从 24.25 h 降到 6.93 h，降了 17.3 h；再加到 8 件只再降 5.5 h。拐点在 6 件。

图 93-9 用对数纵轴画缺货概率，用右轴画等待时间。两条线在半对数坐标下接近直线，说明这是典型的指数型收益递减。

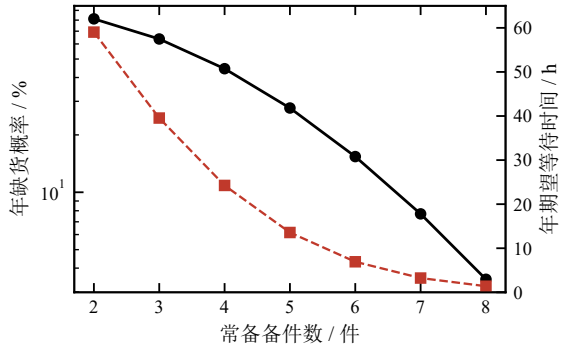


图 93-9 备件数与缺货概率、年期望等待时间

93.2.9 升级的期望收益

升级有成功与失败两种结果，按概率加权：

$$S = p\,s - (1-p)\,\ell \tag{93-12}$$

其中 p 是成功率，s 是成功时每年节省的停机小时，ℓ 是失败时额外损失的停机小时。

$p = 0.85$、$s = 16$ h、$\ell = 12$ h 时，期望年节省 11.8 h。首年要扣掉升级本身的 8 h 停机，净收益 3.8 h，回收期约 0.68 年。

这个算式的意义不是给出一个精确值，而是把“要不要升”变成两个可以讨论的数：成功率与失败损失。成功率靠灰度提高，失败损失靠回滚压低。

93.2.10 验收达标率

验收要一个能签字的数，最简的是达标率：

$$R = \frac{n_{\text{pass}}}{n_{\text{item}}} \tag{93-13}$$

本章五项指标与目标值见表 93-9。

指标	目标	实测	结论
回路投用率 / %	95.0	96.5	达标
报警负荷 / (条/h)	≤12.0	11.3	达标
数据完整率 / %	99.0	99.4	达标
可用度 / %	99.8	99.875	达标
文档齐套率 / %	100.0	95.0	未达标

表 93-9 的结果是 4/5，达标率 80%。这一项未达标恰恰说明指标是有效的：文档齐套率不到 100%，就不能签字，不能靠“其他四项都好”来抵消。

93.2.11 十三个公式的共同结构

十三个公式看起来零散，其实只有三种结构：串联、筛选、期望。

串联结构有三个：缺陷拦截 $E_i = \prod_{j \le i}(1 - p_j)$、回滚时间 $T = \sum t_i$、报警负荷 $L = L_0 \prod (1 - r_i)$。串联的共同点是“环节之间相乘或相加”，任何一环变差，整体就变差，没有互相补偿的余地。

筛选结构有两个：版本组合 $N = \prod n_k$ 与三条兼容规则、备件数对缺货概率的影响。筛选的共同点是“先列全集，再按规则砍”，砍掉的部分要能说清理由。

期望结构有四个：期望修复成本、灰度暴露概率、升级期望收益、可用度。期望的共同点是“用概率加权”，结果是一个平均值，不是某一次的结果。

剩下四个是比值：组合占比、工期、达标率、MTTR 与年停机。比值结构最容易出问题，因为分子分母的定义一变，结论就变。所以本章对每个比值都写了“分子是什么、分母是什么”。

认清结构有一个实际好处：改参数时知道会牵动哪些公式。改检出率只动串联，改兼容规则只动筛选，改成功率只动期望，改指标定义会动比值——最后一类最危险，因为它改的是口径，不是数。

93.2.12 三个算例串成一条链

单独看十三个公式，容易忘记它们服务于同一件事。把它们串起来，是一条从“少犯错”到“扛得住”的链。

第一段是少犯错。81 种组合筛到 26 种，测 67.6 h，把能提前发现的问题挡在出厂之前。这一段的目标是降低进入现场的缺陷数。

第二段是早发现。五级拦截的逃逸率 0.246%，意味着每 1000 个缺陷还有 2.5 个漏到现场。灰度的作用是在小范围内把这 2.5 个逼出来，25% 批次有 92.31% 的把握。

第三段是快恢复。漏出去的缺陷一旦触发，回滚 25 min 内完成，影响面控制在一批设备内。这一段的目标不是“不出事”，是“出了事不扩散”。

第四段是扛得住。可用度 99.875%、年停机 10.94 h、备件 6 件，保证系统在长期运行中有足够的恢复能力。这一段的目标是把偶发故障的影响压到可接受范围。

四段对应四个数：26、0.246%、25 min、99.875%。四个数一起看，才是一套完整的落地指标。只看其中一个，会得出片面结论：只看 26 种，会以为测试做够了；只看 99.875%，会以为运维没问题。

链条有一个断点要特别注意：第三段到第四段之间。回滚能力只在故障发生时起作用，故障之后能不能恢复、恢复得多快，靠的是备件与诊断。很多项目把回滚做得很漂亮，备件却没备，结果回滚之后停在那里等件。

93.3 设计步骤

93.3.1 五步与输出物

落地设计分五步，每一步都要有输出物，没有输出物的一步等于没做。

第一步，划阶段与阶段门。输出是一张流程图与一张门表，图上标清阶段，表里写清每道门的进入条件与退出证据。

第二步，定版本与兼容承诺。输出是兼容矩阵与一份“支持与不支持”清单，清单要能直接写进合同。

第三步，定灰度方案。输出是批次划分、观察时长、暂停条件与回退触发条件。

第四步，定回滚方案。输出是回滚包、回滚脚本、回滚演练记录与时间预算表。

第五步，定验收指标。输出是五项左右的指标、目标值、测量方法与数据来源。

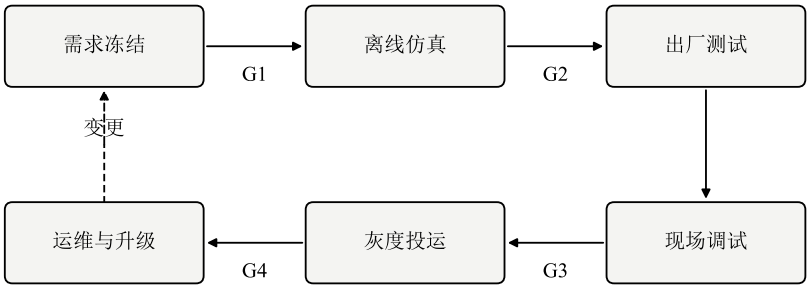
五步的输出物加起来不超过十页，超过十页说明写的是方案，不是落地文件。

93.3.2 划阶段与阶段门

阶段的划法各家不同，本章用六段：需求冻结、离线仿真、出厂测试、现场调试、灰度投运、运维与升级。

六段之间设四道门：G1 基线评审、G2 出厂验收、G3 现场验收、G4 灰度评审。图 93-1 画出这六段四门的顺序与回环。

图 93-1 里有一条从“运维与升级”回到“需求冻结”的虚线箭头，标注“变更”。这条线的意思是：运维期发现的改进要走变更流程，不能直接在现场改。没有这条线，现场改动会绕开所有阶段门。



每个阶段门都是一次「停与不停」的决策，不是一次汇报

图 93-1 从需求到运维的六阶段与四道阶段门

每道门要有进入条件与退出证据，见表 93-10。

阶段门	进入条件	退出证据	不通过怎么办
G1 基线评审	需求清单与回路清单冻结	基线表签字、数据点表齐全	退回需求，不进入仿真
G2 出厂验收	出厂测试用例全部执行	测试记录、缺陷清单、遗留项说明	不带遗留缺陷出厂
G3 现场验收	单回路与联锁全部投通	调试日志、投用率统计、报警负荷	延期投运，不降标准
G4 灰度评审	灰度批次观察满两周	灰度报告、回滚演练记录	暂停扩大，先修再评

表 93-10 的第三列是重点。证据要能复印、能归档、能被第三方复核。凡是“口头确认”“大家都不反对”这类描述，都不算证据。

阶段门的另一条规矩是：门可以停，但不能跳。跳门省下的时间，通常在运维期以倍数还回去。

93.3.3 定版本与兼容承诺

版本设计的第一步是把组件分类。本章分四类：固件、组态、上位机、通信协议。

第二步是给每类定版本号规则。主版本变功能，次版本变接口，修订号只修缺陷。规则要写下来，否则不同团队对“1.2.3 到 1.3.0”的理解会不一样。

第三步是定兼容规则。本章用三条：协议不高于固件、上位机不低于组态、固件与组态相差不超过一版。图 93-2 画出这三条约束。

三条规则筛出 26 种允许组合，占全部 81 种的 32.1%。这意味着测试量降到三分之一，也意味着要明确拒绝 55 种组合。

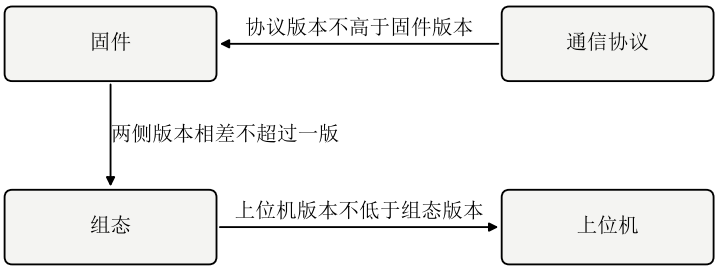


图 93-2 四类组件的版本关系与三条兼容规则

拒绝清单要写进合同附件，并在发货前核对现场的实际版本。现场最常见的故障不是新版本有缺陷，而是现场装着一个从未被承诺过的组合。

93.3.4 定灰度与回滚

灰度方案要写四件事：批次、观察时长、暂停条件、回退触发条件。

批次按表 93-3 选，本章建议 5%、10%、25%、100% 四批。首批 5% 的作用是“跑通”，不是“发现缺陷”，因为 40.13% 的暴露概率只够发现高频问题。

观察时长按缺陷的表现周期定。与班次相关的缺陷至少观察两个班次，与温度相关的至少观察一周，与月末结账相关的至少观察一个结账周期。

暂停条件要写成可判定的：报警负荷超过上限、同步偏差连续超限、任一回路被迫切手动。写“情况异常时暂停”等于没写。

回退触发条件要比暂停条件更严，一旦触发直接回滚，不再讨论。常见的触发条件是通信中断超过 5 min、控制器重启、数据完整率跌破 99%。

93.3.5 定验收指标

验收指标控制在五项左右。指标太多，每项都测不深；指标太少，容易漏掉关键面。

本章的五项是回路投用率、报警负荷、数据完整率、可用度、文档齐套率。前三项看系统好不好用，第四项看稳不稳，第五项看能不能接手。

每项指标要写清四件事：定义、目标值、测量方法、数据来源。以可用度为例，定义是稳态可用度，目标 99.8%，测量方法是按 MTTR 与 MTBF 计算，数据来源是运维工单系统。

四件事缺一件，验收时就会吵。最常见的争吵是“数据来源不同”：一方用系统日志，一方用值班记录，两个数差 2%。

验收指标要在项目开始时定，不能在验收前定。验收前定的指标，往往是按已有结果倒推的。

93.3.6 一页纸落地基线

把五步的输出压到一页纸，就是落地基线。十行左右，每行写参数、取值、依据。

前四行是阶段与门：阶段数、门数、每道门的证据件数、变更走哪个流程。

中间三行是版本与灰度：允许组合数、灰度批次、观察时长。

后三行是回滚与验收：回滚时间预算、验收指标数、达标线。

一页纸要能贴在会议室墙上。贴不上去，说明还没想清楚。

93.3.7 方案评审要问的五个问题

落地方案写完，用五个问题过一遍，能筛掉大部分隐患。

第一问：这套系统出事时，退回去要多久？答不出分钟数，回滚方案就是空的。

第二问：这一版跑的是哪四类版本，现场在跑的是哪四类？答不出，版本管理就是空的。

第三问：灰度第一批多少台，能发现哪一类缺陷，发现不了哪一类？答不出“发现不了哪一类”，灰度设计就是拍脑袋。

第四问：现场几个人、几天、按什么效率算的？答不出效率系数，工期就是许愿。

第五问：验收五项的数据从哪个系统取，谁来取？答不出系统名，验收就会吵。

五问的顺序对应五个环节：回滚、版本、灰度、工期、验收。答不出的那一问，就是方案里最薄的一环。

五问还可以反向用：作为评审人，先问这五个问题，再听汇报。多数方案在这五问上会露馅，露馅的地方往往正是后来出事的地方。

93.4 连续情形

93.4.1 三条连续关系

落地过程里有三条关系可以当连续量处理：工期随人数、负荷随措施、可用度随 MTTR。

这三条都是单调的，方向明确，适合先做粗估。粗估的作用不是给最终答案，是在开会前把数量级定下来，避免在“要不要加人”这类问题上争半小时。

三条关系各有各的形状。工期是双曲线，人数翻倍工时减半，但固定工时让它减不到零；负荷是指数衰减，每加一项措施乘一个小于 1 的系数；可用度是分式，MTTR 越小越接近 1，但永远到不了 1。

形状不同，决策含义也不同。双曲线告诉你边际收益递减，指数告诉你后几项措施越来越不划算，分式告诉你可用度的小数位越来越贵。

93.4.2 工期曲线的连续近似

把人数 m 当作连续变量，式 93-7 是一条双曲线，渐近线是 $H = 0$，但在 m 较小时下降很陡。

对 m 求导，边际收益是 $-\left(N_{\text{loop}} t_c + T_0\right) / \left(\eta m^2\right)$，绝对值随 m 平方衰减。m 从 1 增到 2，斜率降为四分之一；从 3 增到 4，斜率降为九分之一。这就是表 93-5 里“1 到 2 省 5 天、3 到 4 只省 1 天”的原因。

连续近似在 $m \geq 1$ 时成立，在 $m < 1$ 时没有意义。人数不能取小数，所以在 1 与 2 之间，实际只能选一个。

工程上的用法是：先按连续公式算出“需要几个人”，再向上取整到实际可派的人数，最后用整数重算一遍工期。

93.4.3 报警负荷的指数衰减

把措施看作连续的削减率，负荷可以写成 $L = L_0 \exp\left(-\sum_i k_i\right)$，其中 $k_i = -\ln(1 - r_i)$ 是第 i 项措施的对数削减量。四项措施的对数削减量相加，等于总削减量的对数。

指数形式的用处是判断“还差多少”。原始 120 条/h 要降到 12 条/h，需要总削减 90%，即 $\sum k_i = \ln 10 = 2.303$。

本章四项措施的 k 分别是 1.050、0.511、0.693、0.105，合计 2.359，刚好超过 2.303。这就是为什么最终负荷 11.34 只是略低于上限，而不是大幅低于。

如果只做前三项，合计 2.254，负荷 12.6 条/h，仍在上限之上。第四项只贡献 0.105，却是“过线”与“不过线”的差别。

93.4.4 可用度的连续近似

式 93-9 在 MTTR 远小于 MTBF 时可以近似为 $A \approx 1 - MTTR/MTBF$。

本章 MTTR 4.0 h、MTBF 2000 h，近似值 99.80%，与精确值 99.800% 一致到小数点后三位。

近似成立的边界是 MTTR/MTBF 小于 5%。超过这个比例，分母的修正就不能忽略。例如 MTTR 200 h、MTBF 2000 h，精确值 90.9%，近似值 90.0%，差 0.9 个百分点。

这个近似的价值在于把“提高可用度”翻译成“降低 MTTR 或提高 MTBF”。两个方向的花费不同：降 MTTR 靠备件与远程诊断，提 MTBF 靠冗余与降额。多数项目先降 MTTR，因为便宜。

93.4.5 连续估算失效的三处

连续估算在三处会失效，需要换成离散或逐项计算。

第一处是人数很少时。1 人与 2 人的差别不是 50%，而是“能不能并行做联锁”这种质变。连续公式给不出这个差别。

第二处是措施接近极限时。负荷已经接近 0 时，再削减 50% 也只是从 2 条/h 降到 1 条/h，对操作员没有实际意义。指数模型仍然正确，但决策价值消失了。

第三处是可用度的尾部。年停机 17.49 h 是一个平均值，实际可能集中在一次 12 h 的故障里。平均值好看的系统，仍可能有一次长达一天的停机。

三处的共同教训是：连续估算用来排序，不用来承诺。承诺要用离散模型或实测数据。

93.4.6 四个量之间的换算

连续模型的一个实用价值是换算：把“停机小时”换成“可用度”，把“人时”换成“工作日”，把“条/h”换成“每班次条数”，把“批次比例”换成“暴露概率”。

年停机 10.94 h 换成可用度是 99.875%，换成“平均每个季度停 2.7 h”，换成“每次停机 2.5 h 时一年停机 4.4 次”。同一个事实的四种说法，对不同的人有不同的说服力：管理层听百分比，运行人员听次数，财务听小时。

人时换工作日要带两个系数。一个是每天有效工时，本章按 8 h 算，实际按 6 h 更保守；一个是并行效率，本章取 0.85。两个系数乘起来是 0.64 的折扣，忽略它们，工期会少算三分之一。

条/h 换每班次条数乘 8。11.34 条/h 对应每班 90.7 条，一个操作员一个班次处理九十条报警，平均每五分钟一条。这个换算比条/h 更有画面感，也更容易让业主理解“余量薄”意味着什么。

批次比例换暴露概率是本章最常用的一步。25% 换 92.31%，5% 换 40.13%，1% 换 9.75%。换完就能回答“这批到底能不能发现问题”，而不必争论“1% 是不是太小”。

四种换算都不难，难的是口径统一。同一个项目里，有人用自然日、有人用工作日，有人用日历小时、有人用运行小时，最后对不上账。开工时把口径写进一页纸，比事后解释省力得多。

93.5 离散情形

93.5.1 落地里的整数

落地过程中大部分数量是整数：回路数、人数、台架数、设备台数、备件数、批次数。

整数带来两个后果。第一是取整误差，按公式算出 2.3 个人，实际只能派 2 个或 3 个。第二是台阶效应，某些区间里增加资源完全不改变结果。

台阶效应比取整误差更值得注意。表 93-5 里，3 人到 4 人只省 1 天，这 1 天是“从 4 天到 3 天”的台阶，不是因为 4 个人更快，而是因为总工时刚好跨过一个工作日边界。

93.5.2 灰度批次的取整

灰度批次按比例算出来是小数，实际按整台设备取。

200 台设备、1% 批次是 2 台，5% 是 10 台，10% 是 20 台，25% 是 50 台。这几个数都能整除，不需要特殊处理。

不能整除时要取整，取整方向影响暴露概率。7% 批次算出 14 台，向上取到 15 台暴露概率略高，向下取到 14 台略低，差别很小。

真正需要注意的是下限。设备总数少于 100 台时，1% 批次不足 1 台，实际只能取 1 台。这时“1%”这个名字具有误导性，应该直接写“首批 1 台”。

93.5.3 版本号是离散的

版本号不是连续量，不能取 1.5 版。主版本、次版本、修订号都是整数，升级只能一档一档走。

这意味着两件事。第一，“小步快跑”有下限，最小的一步是一个修订号。第二，跨主版本升级不能跳过中间的兼容检查，因为接口可能变了两次。

兼容矩阵因此是一张离散表，不是一条连续曲线。26 种允许组合是 26 个点，点与点之间的组合不存在。

现场最危险的情况是“混搭”：固件升了，组态没升，上位机还是两年前的版本。这种组合往往不在 26 种里，却真实存在。

93.5.4 日期按工作日算

工期算出来是小时，排到日历上要换成工作日，中间还有周末与节假日。

本章统一按每天 8 h、每周 5 天折算。75.3 h 折 10 个工作日，37.6 h 折 5 个，25.1 h 折 4 个，18.8 h 折 3 个。

折算不是简单除法。25.1 h 除以 8 是 3.14，向上取整是 4 天，而不是 3 天。少算这 0.14 天，排班就会差一天。

节假日的处理更麻烦。春节前后的两周通常不能安排现场调试，因为人手不齐、备件不到。排工期时要把这两周整段跳过。

93.5.5 备件的离散概率

备件数是整数，缺货概率是离散分布上的尾概率，不能用连续的正态近似代替小样本。

年故障率 4.38 次时，备 4 件的缺货概率是 44.5%，备 6 件是 15.4%，备 8 件是 3.5%。这几个数之间没有线性关系，不能按“多备一件降一点”去估。

备件决策还要看缺货的代价。等待 24 h 换来的停机损失如果远大于一件备件的成本，就该多备；如果设备可以降级运行，等待是可以接受的。

一个常被忽略的点是备件的通用性。同一型号的备件能覆盖三台设备，与只能覆盖一台，库存策略完全不同。

93.5.6 三条用法

离散情形的三条用法可以概括为：先算连续值，再取整，最后复核台阶。

第一步用连续公式得到量级，第二步按实际可选的档位取整，第三步把取整后的值代回公式，看结果是否跨过了一个台阶。

三步里第三步最容易漏。取整后不复核，就会出现“排了 4 个人，工期还是 3 天”这种看似白干的情况。

三步做完，还要把取整方式写进文档。写“3 人”不够，要写“3 人，向上取整，按 8 h/天折算”。

93.5.7 取整误差会累积

单个取整误差很小，多个误差叠起来就不可忽略。落地过程里至少有五处会取整：人数、台数、天数、备件数、版本数。

人数向上取整让工期偏短，天数向上取整让工期偏长，两者方向相反，看起来能互相抵消。实际不会抵消，因为它们的量级不同：人数从 2.4 取到 3，工时降 20%；天数从 2.4 取到 3，工期升 25%。取整的净效果取决于哪一个先发生。

台数取整的影响在灰度。200 台的 12.5% 是 25 台，取整没有误差；50 台的 12.5% 是 6.25 台，取 6 台还是 7 台，暴露概率从 26.5% 变到 30.2%，差 3.7 个百分点。小规模系统里，取整误差会盖过设计意图。

备件数取整没有余地，本来就是整数。它的误差来自年故障率的估计：λ 从 4.38 估成 3.5，拐点就从 6 件掉到 5 件。

控制取整误差的办法是“误差预算”。在方案里写明每一项可能的取整偏差，加起来不超过总工期的 10%、总暴露概率的 5%，就认为可接受。超出就要调整档位，而不是事后解释。

93.5.8 档位怎么选

离散参数的档位不是随便定的，通常取“能被三整除”或“能被二整除”的数。

灰度六档取 1、5、10、25、50、100，是为了让 200 台设备都能整除，算出来的台数干净。如果取 7、13、37，台数就带小数，每次都要解释取整。

人数四档取 1 到 4，对应现场能容纳的最大人数。取到 6、8 没有实际意义，因为控制室挤不下，效率系数会失效。

备件七档取 2 到 8，覆盖“明显不够”到“明显够”的区间。少于 2 件看不出趋势，多于 8 件概率已经接近 0，再算没有信息量。

档位的间隔也有讲究。等差间隔适合看趋势，等比间隔适合看数量级。灰度用等比（1、5、10、25、50、100），因为暴露概率是对数型的；备件用等差，因为等待时间近似线性。

选错间隔的后果是“表看不出来”。用等差画缺货概率，后几点会挤在一起；用等比画工期，前几点会挤在一起。本章图 93-3 用线性轴、图 93-9 用对数轴，就是这个道理。

93.6 稳定性与性能

93.6.1 落地里的稳定指什么

控制里的稳定是闭环极点不出左半平面。落地里的稳定不是这个意思，它指系统在长期运行中不出现不可恢复的状态。

两者有交集但不重合。一个控制上稳定的系统，可能因为版本升级失败而停摆三天；一个控制上勉强稳定的系统，可能因为回退机制完备而从未影响生产。

本章把稳定拆成三条：**能回退、能观测、能复现**。

能回退是出错后能退回上一个可用状态。能观测是出问题时能定位到哪一层。能复现是同一现象能再次出现，便于验证修复。

三条缺一条，系统就算“能跑”，也不算“稳”。

93.6.2 可回退是最大的稳定

回退能力是本章最看重的一条，因为它是唯一能兜住未知缺陷的手段。

未知缺陷无法在设计阶段全部消除。表 93-1 里五级拦截之后仍有 0.246% 逃逸，这部分只能靠回退兜住。

回退要满足三个条件：回滚包在本地、回滚动作可自动、回滚后能验证。三个条件缺一个，回滚时间就会从分钟级涨到小时级。

本章回滚预算 35 min 里，执行只占 8 min，其余 27 min 都花在发现、决策与验证上。这说明回滚慢通常不是技术慢，是流程慢。

缩短回滚时间的有效手段是预置与预演。预置回滚包省 3 min，自动脚本省 4 min，预演过的验证清单省 3 min，合计把 35 min 压到 25 min。

93.6.3 可观测的三个层次

可观测不是装一块大屏，是三个层次的信息都能拿到。

第一层是日志，回答“发生了什么”。日志要带时间戳、设备号、版本号，缺一项就没法关联。

第二层是指标，回答“现在怎么样”。本章关心四个：回路投用率、报警负荷、数据完整率、可用度。

第三层是追踪，回答“为什么这样”。追踪要能串起一次操作的完整链路：从操作员点击，到控制器执行，到现场动作。

三个层次的信息量差一个数量级。只有日志没有指标，看不过来；只有指标没有追踪，定位不了。

93.6.4 性能退化的顺序

落地系统的退化通常有固定顺序，认识这个顺序能提前发现。

第一步是延迟变大。同步变慢、响应变慢，但功能还正常，用户往往感觉不到。

第二步是报警变多。延迟积累到一定程度，偏差超限的次数增加，报警负荷上升。

第三步是人工干预变多。操作员开始频繁切手动，投用率下降。

第四步是停机。某次报警没被及时处理，联锁动作。

四步之间通常有几周甚至几个月。前两步是可逆的，后两步往往要停机才能恢复。所以运维看板上最该盯的是延迟与报警负荷，而不是等停机。

93.6.5 灰度期的稳定性

灰度期的稳定性判断与全量期不同，因为它样本少、时间短。

样本少意味着偶发问题看不出来。5% 的缺陷触发率，10 台里至少一台触发的概率是 40%，剩下 60% 的情况是“没触发”，不代表没问题。

时间短意味着慢变问题看不出来。参数漂移、内存泄漏、磁盘写满，都需要几周才显形。

所以灰度期的结论只能是“没发现严重问题”，不能是“没有问题”。灰度通过不等于质量通过，只是允许进入下一批。

灰度期要专门看两类指标：重启次数与内存占用。这两项在短窗口里最容易看出异常。

93.6.6 长周期稳定

长周期稳定要看三个月，而不是三天。

第一个月看功能是否齐全。缺的功能会在使用中暴露，尤其是报表与追溯这类边缘功能。

第二个月看性能是否退化。内存、磁盘、响应时间三条曲线，只要有任一条单调上升，就要查。

第三个月看运维是否接得住。新人能不能独立处理一次告警，备件账对不对得上，回滚演练有没有做过。

三个月都过，才可以谈“稳定运行”。只过第一月的系统，通常还在蜜月期。

93.6.7 性能验收四样

性能验收不看感觉，看四样东西。

第一样，回路投用率。目标 95%，实测 96.5%，数据来自控制系统自身的投用统计。

第二样，报警负荷。目标不超过 12 条/h，实测 11.34，数据来自报警日志按小时统计。

第三样，数据完整率。目标 99%，实测 99.4%，数据来自采集端的帧计数与入库记录对比。

第四样，可用度。目标 99.8%，实测 99.875%，数据来自运维工单的停机时长。

四样都要有原始记录。只给汇总值不给原始记录，验收时无法复核。

93.6.8 稳定性验收的三次演练

性能可以看数据，稳定性只能靠演练。本章建议做三次，每次都留记录。

第一次是回滚演练。选一个非关键单元，把组态切回上一版再切回来，全程计时。目标 25 min，第一次通常要 40 min 以上，卡点多半在“找不到回滚包”或“不知道验证哪几项”。

第二次是断电演练。把控制器主电源断掉，看备用电源切换、看数据补传、看上位机是否报警。这一项最容易被跳过，因为它要动真设备，风险看着大，但风险远小于半夜真断电。

第三次是通信中断演练。拔掉一条网线或在交换机上禁用端口，观察冗余链路是否接管、切换时间是否在允许范围内、切换期间的数据是否有缺口。第 90 章给的冗余设计，要在这里被验证一次。

三次演练的判据不同。回滚看时间，断电看数据完整性，通信看切换时长与切换期间的操作是否受影响。三个判据都要写成数值，不能写“正常”。

演练之后要修的是流程，不只是设备。第一次回滚演练超时，多半不是技术问题，是没人清楚自己该做什么。把动作写成清单、把清单贴在机柜上，第二次就会快很多。

三次演练做完，稳定性才算“被验证过”。没有演练的稳定性只是“没出过事”，这两者的差别，在下一次真出事时就会显现。

93.6.9 退化是怎么被发现的

系统不会突然退化，退化是慢慢积累的，能不能发现取决于有没有基线。

基线要在投运时取，取三天到一周的稳定数据。基线包括：扫描周期、控制器负载、通信丢帧率、报警条数、数据库写入延迟。五个量都要存成时间序列，不能只记一个平均值。

有基线之后，退化的判据是“连续偏离”。连续三天偏离基线 10% 以上，就要查；连续一周偏离 5% 以上，也要查。单次偏离不算，因为工艺本身有波动。

最常见的三种退化是：磁盘写满、内存泄漏、通信重传率上升。三者的表现都是“指标缓慢单调上升”，与工艺波动不同——工艺波动是来回摆，退化是单向走。

发现退化之后，处理顺序是先确认再动手。确认的办法是看同型号的其他设备是否也有同样趋势：都有，是软件问题；只有一台，是硬件或配置问题。

退化的处理往往不需要停机。磁盘写满清历史数据，内存泄漏重启服务，通信重传查交换机端口。这三件事都能在运行中做，前提是提前发现了。

93.7 参数意义与作用

93.7.1 参数分五组

本章的参数按用途分五组：阶段与门、版本与灰度、回滚、工期、运维。

五组参数的调整权限不同。阶段与门由项目管理层定，版本与灰度由产品与测试共同定，回滚由运维定，工期由项目经理定，运维指标由业主与集成商共同定。

权限不同意味着不能一个人全改。实际项目里最常见的错误是项目经理为了赶工期，把灰度批次从 10% 直接跳到 100%，跳过了本该由产品与测试共同确认的那一步。

分组还有一个好处：出问题时能快速定位该找谁。灰度出问题找产品与测试，回滚慢找运维，工期紧找项目经理。

93.7.2 阶段与门参数

阶段数不是一个可以随意增减的参数。本章用六段，是因为六段各自对应一种可交付物，少一段就有一类东西没人负责。

门数比阶段数少两个，因为首段之前与末段之后没有门。四道门分别卡住基线、出厂、现场、灰度四个关键转折。

每道门的证据件数建议三到五件。件数太少，门形同虚设；件数太多，评审变成体力活，最后没人认真看。

门的“不通过怎么办”比“通过条件”更重要。不通过时如果不能退回，门就只是一个记录点。

93.7.3 版本与灰度参数

组件类别数决定组合数的增长方式。四类各三版是 81 种，五类各三版就是 243 种，测试量按指数涨。

版本号规则是硬约束。主版本变功能、次版本变接口、修订号只修缺陷，这条规则一旦松动，“兼容”就没有定义。

灰度批次建议不少于三批。两批的话，第一批出问题只能退回，第三批没有观察期；三批可以做到“小批跑通、中批观察、大批确认”。

观察时长至少覆盖两个班次。只观察一个班次，交接班时的问题看不到，而交接班恰恰是最容易出问题的时刻。

暂停条件要写成数值。写“报警明显增多”没有可执行性，写“超过 12 条/h 连续 30 min”才能自动判定。

93.7.4 回滚参数

回滚时间预算是本章最硬的参数，因为它直接决定故障影响面。

四段时间里，“确认与决策”通常最长，也最容易被压缩。压缩的办法是提前约定触发条件：满足条件直接回滚，不需要开会。

回滚包的大小影响执行时间。回滚包超过 2 GB 时，现场网络传输要十几分钟，这时要改为本地预置。

验证时间不能省。回滚后不验证，等于把系统交给运气。验证清单要在演练时逐条走一遍，把不必要的项删掉。

回滚演练的频率建议每季度一次。演练不是走过场，要真的把生产系统切回去再切回来。

93.7.5 工期参数

回路数是工期的第一驱动项，单回路工时是第二驱动项，固定工时决定下限。

单回路 1.2 h 的构成是点检 0.3、单回路 0.6、联锁 0.3。三项里联锁最容易低估，因为联锁要与其他专业配合。

并行效率 0.85 是经验值。人越多，协调成本越高，效率越低。超过 4 人时，效率通常要降到 0.8 以下。

每天有效工时按 8 h 算，实际有效时间往往只有 6 h，其余被会议、等待、办票占用。排工期时用 8 h 是乐观值，用 6 h 更接近现场。

93.7.6 报警与可用度参数

报警的原始负荷取决于工艺与组态，通常在 50 到 300 条/h 之间。超过 300 条/h 时，先查仪表与组态，不要急着上措施。

四项措施的削减比例不是固定值，随项目差异很大。清理无效报警通常能砍一半以上，死区与滤波看工艺，延时确认看对象时间常数。

操作员上限 12 条/h 来自行业惯例，不是物理极限。上限的作用是给设计一个靶子，不是给考核一个指标。

MTBF 与 MTTR 都要用自己项目的历史数据。用厂商给的 MTBF 算出来的可用度，通常比实际高一个数量级。

93.7.7 备件与升级参数

备件数的拐点由年故障率决定。年故障 4.38 次时拐点在 6 件，年故障 2 次时拐点在 3 到 4 件。

缺货等待时间按 24 h 算是保守值。有区域仓库时可以降到 8 h，没有时要按 72 h 算。

升级成功率靠灰度提高，不靠祈祷。做过三次灰度升级的团队，成功率通常在 0.85 以上；从没做过灰度的团队，0.7 都算乐观。

失败损失包括停机、返工、信誉三部分。前两项能算，第三项算不出来，所以失败损失通常被低估。

93.7.8 参数表

把上述参数汇总，得到表 93-11。表中每行给出符号、含义、本章取值与作用方向。

参数	含义	取值	作用方向
p_i	第 i 级缺陷检出率	0.30–0.92	越大，期望修复成本越低
c_i	第 i 级单位修复成本	1–60 人时	越大，越应前置拦截
c_e	投运后发现成本	150 人时	决定阶段门的价值
n_k	各类组件版本数	3	组合数按乘积增长
A	允许组合数	26	决定回归工时
p	单台缺陷触发率	0.05	决定灰度把握
n	灰度批设备数	2–200	越多越容易暴露
t_i	回滚各段耗时	5–12 min	相加得回滚总时间
N_{loop}	回路数	40	工期第一驱动项
t_c	单回路工时	1.2 h	工期第二驱动项
η	并行效率	0.85	越低工期越长
r_i	报警削减比例	0.10–0.65	连乘得最终负荷
MTBF	平均无故障时间	2000 h	越大可用度越高
MTTR	平均修复时间	4.0 h	越小可用度越高
k	备件数	2–8 件	越多缺货概率越低
p_{ok}	升级成功率	0.85	决定期望收益

表 93-11 的第四列是调参时最该看的一列。它告诉读者改这个参数会往哪个方向变，而不是只给一个数。

93.7.9 敏感度排序

参数对结果的影响差别很大，按敏感度排序能省很多试错。

最敏感的是回滚时间与灰度批次。前者一动，故障影响面成倍变化；后者一动，风险与把握同时变化。

其次是回路数与单回路工时。它们直接乘在工期公式上，改动 10% 就带来 10% 的工期变化。

再次是报警削减比例。连乘之后，单项的小改动被放大，但也容易被其他项抵消。

最不敏感的是组件版本数。它只影响组合数与测试量，不影响现场运行。

调参顺序因此是：先定不敏感的（版本数、观察时长），再定中等的（工期参数、报警比例），最后动敏感的（灰度批次、回滚预算）。

93.7.10 参数之间的三组耦合

本章的参数不是独立的，有三组会互相咬住。

第一组是灰度批次、观察时长、回滚时间。批次越大，暴露越快，但一旦出事影响面越大，对回滚时间的要求越严。三者要一起定。

第二组是回路数、人数、工期。回路数增加，人数不变，工期线性增加；人数增加，工期按双曲线下降，但受固定工时限制。

第三组是 MTBF、备件数、可用度。MTBF 低，备件要多；备件不足，MTTR 变长，可用度下降。

三组耦合的检查方法很土：改一个参数，把另外两个代回去算一遍。三十秒的事，能避免“改好了这个、弄坏了那个”。

93.7.11 有些参数不能连续调

阶段数、门数、组件类别数、灰度批次数都是整数，不能取 2.5。

版本号是离散的，只能一档一档升，不能跳。

人数是整数，且受现场容纳能力限制。一个控制室同时挤六个人做调试，效率反而下降。

日期要按工作日排，节假日整段跳过。按自然日排出来的工期，通常会短 20% 左右。

把参数分成“连续”与“离散”两类，先定离散的档位，再调连续的细值，能省掉大量无效试算。

93.7.12 参数取值的三个来源

本章的参数没有一个是凭空想的，来源只有三类：历史记录、同类项目、保守假设。

历史记录最可靠。单回路 1.2 h 来自同类装置的实际工时统计，MTBF 2000 h 来自厂商数据与现场故障记录的交集，报警原始负荷 120 条/h 来自投运首周的日志。有记录就用记录，不要用感觉。

同类项目次之。检出率 30%、45%、60%、80%、92% 来自一支做过三个项目的团队的经验，换团队要打折。用别人的数要问清三件事：项目规模、团队构成、测试手段。三件事差一件，数就要调整。

保守假设是兜底。缺货等待 24 h、失败损失 12 h、准备系数 1.30，这三个数没有可靠来源，取的是“宁可高估成本”的方向。保守假设的作用是让方案在最坏情况下仍站得住。

三类来源要标在参数表里。标了来源，别人才知道哪个数可以争、哪个数不能动。历史记录可以复核，同类项目可以类比，保守假设只能讨论。

还有一类数要特别小心：厂商给的指标。厂商给的 MTBF 通常是在实验室条件下测的，现场温度、粉尘、电源质量都会让它变短。用厂商数据时要打一个折扣，本章取 0.5 到 1 之间。

93.8 调参方法与经验

93.8.1 调参的四个对象与顺序

工程落地的调参对象只有四个：门、灰度、工期、运维指标。版本组合不调，它由组件数量决定，调它等于改产品。

四个对象的调整顺序由不可逆性决定。门最先定，因为它决定后面所有活动要不要做；灰度其次，因为它决定暴露节奏；工期再次，它可以被压缩但有下限；运维指标最后，它靠前面的选择自然形成。

倒过来调是常见错误。先把工期压到三天，再回头删阶段、跳灰度，最后报警负荷超限、可用度不达标，只能再加人救火。

调参有一个总原则：**改一个，算一遍，记一笔**。改两个以上同时算，出了问题说不清是哪一个的贡献。

本章的所有调参经验都围绕一句话：让改动落在敏感度低的地方，把敏感度高的地方留出余量。

93.8.2 先调门的检出率

五级检出率 30%、45%、60%、80%、92% 是基线值。调它之前要先看差距出在哪一级。

把 HIL 的检出率从 80% 降到 60%，逃逸率从 0.246% 升到 0.493%，期望修复成本从 6.62 升到 8.07。降 20 个点，成本涨 22%，代价不小但还能接受。

把设计评审从 30% 降到 10%，逃逸率从 0.246% 升到 0.317%，期望成本涨到 8.22。涨幅比降 HIL 还大，因为这一级在最前面，它漏掉的缺陷要经过后面四级才能被拦住。

所以真正的经验是：**头段不能省，中间段可以压**。头段漏需求与接口，后面的阶段很难发现；中段漏的是实现细节，后面还有机会补。

反过来，把某一级的检出率往上提，收益是有上限的。把 HIL 从 80% 提到 95%，逃逸率从 0.246% 降到 0.062%，期望成本降到 5.53，省 1.09 人时。为这 1.09 人时建一套完整的 HIL 台架，通常不划算。

结论：门的作用是防止灾难性漏检，不是追求期望成本最优。把钱花在头段，把中段做到及格，是多数项目的合理选择。

93.8.3 调灰度：从 5% 起步，25% 收口

灰度批次是本章敏感度最高的参数。敏感的原因是暴露概率对批次数量呈指数上升。

按 5% 的单台触发率，1% 批次（2 台）只有 9.75% 的把握发现问题，等于没试。5% 批次（10 台）到 40.13%，勉强算一次试。10% 批次（20 台）64.15%，可以接受。25% 批次（50 台）92.31%，已经接近“必然发现”。

所以经验值是：**首批不小于 5%，第二批量到 10%，第三批 25%，之后放开**。

首批为什么要 ≥5%？因为低于 5% 时暴露概率不到一半，第一批跑通不能证明什么。很多团队首批放 1%，跑通了就以为稳了，结果第二批出问题，前面那一批白跑。

为什么不在 25% 一步到位？因为 25% 一旦出事，影响 50 台，回滚压力和首批完全不同。25% 是“确认”，不是“试”。

批次之间的观察时长也要跟着变。首批观察两个班次，第二批一个班次，第三批半个班次。批次越大，单个问题的影响面越大，但发现问题需要的观察时间越短，因为样本多。

有一个反直觉的结论：**灰度批次不是越多越好，但也不能少于三批**。两批的做法是“试一下、全放”，中间没有确认环节，等于把 25% 那一步省了。

93.8.4 调工期：人数存在饱和点

工期公式给出的四个档位是 1 人 75.3 h（10 天）、2 人 37.6 h（5 天）、3 人 25.1 h（4 天）、4 人 18.8 h（3 天）。

从 1 人到 2 人，工期砍半，收益最大。从 2 人到 3 人，只降 33%。从 3 人到 4 人，降 25%。边际收益按 $1/m$ 衰减，这是公式的结构决定的，不是现场能力问题。

固定工时 16 h 是不可并行部分。当 m 增大到 $m\eta$ 接近固定工时占比时，再加人几乎无用。40 回路的总工时 64 h 里，固定 16 h 占 25%，这是 4 人档之后基本到顶的原因。

现场容纳能力是另一个约束。控制室同时进 4 个人做调试已经很挤，6 个人会互相干扰，效率 0.85 保不住，实际可能掉到 0.7，算出来的工期反而变长。

经验值是：**40 回路配 3 到 4 人，工期 3 到 4 天**。低于 3 人，工期拉长到 5 天以上，业主等不及；高于 4 人，效率下降抵消人数收益。

如果真的要在 3 天内完成，不要加人，改分母：把点检与接线核对提前到安装阶段做，单回路工时从 1.2 h 降到 0.9 h，3 人档的工期就变成 3 天。这是改结构，不是加资源。

93.8.5 调报警：先清理，再滤波

报警负荷从 120 条/h 降到 11.34 条/h，四步的贡献是 65%、40%、50%、10% 的削减比例，对应绝对削减 78、16.8、12.6、1.26 条/h。

第一步“清理无效报警”贡献最大，降 65%。这一步几乎不花钱，只需要把组态里已经废弃的点、重复定义的点、量程错位的点删掉。现场做一次通常能砍一半以上。

第二步“死区与滤波”降 40%，但会带来响应延迟。死区设得太大，小偏差看不见；滤波时间常数设得太大，报警来得太晚。这一步要在工艺允许的范围内做。

第三步“延时确认”降 50%，本质是把瞬时抖动滤掉。延时太短没效果，太长会漏掉真实异常。经验值是按对象时间常数的 2 到 3 倍设。

第四步“分级抑制”只降 10%，作用是压掉衍生报警。一个根源故障引发二十条报警，操作员会淹没在衍生报警里，看不到根源。这一步的价值不在数量，在于让第一条报警显眼。

四步做完是 11.34 条/h，比上限 12 低 0.66 条/h，余量很薄。余量薄意味着工艺一变就可能超限，所以要做两件事：一是把报警负荷做成运行指标，每天统计；二是预留第五步（报警分级重定义）的空间。

这四步的顺序不能换。先滤波后清理，会把该删的点也滤掉；先延时后清理，会掩盖重复定义的报警。**先减法，后加工。**

93.8.6 调可用度：只改定位与修复

MTTR 4.0 h 的构成是发现 0.5、定位 1.5、修复 1.0、验证 0.5，加路程 0.5。四段里定位最长，修复次之。

可用度的公式是 $MTBF/(MTBF+MTTR)$。MTBF 2000 h 时，MTTR 从 4.0 降到 2.5，可用度从 99.800% 升到 99.875%，年停机从 17.49 h 降到 10.94 h，省 6.55 h。

这 1.5 h 的压缩来自两处：远程诊断省 1.0 h，备件在库省 0.5 h。两处都不改产品，只改运维流程。

远程诊断的前提是控制器支持远程接入，且现场网络允许。这两条不满足时，这一项拿不到。

备件在库的前提是有人愿意压库存。6 件备件对应的年缺货概率 15.389%，年期望等待 6.93 h，这个量级的库存成本通常能被停机损失覆盖。

不建议动发现与验证两段。发现时间靠监测密度，加密监测会增加误报；验证时间靠清单，清单压得太短会漏项。

也不建议靠提高 MTBF 来提可用度。MTBF 从 2000 提到 4000，可用度从 99.875% 升到 99.937%，年停机只降 5.5 h，而换一套更高可靠性的硬件通常要几十万。**改 MTTR 便宜，改 MTBF 贵。**

93.8.7 调备件：找台阶，不找最优

备件数的表是 2 件 81.247%、3 件 63.706%、4 件 44.498%、5 件 27.672%、6 件 15.389%、7 件 7.703%、8 件 3.495%。

相邻两件的改善量是 17.54、19.21、16.83、12.28、7.69、4.21 个百分点。第 5 到 6 件降 12.28，第 6 到 7 件只降 7.69，第 7 到 8 件降 4.21。

拐点在 6 件。这个结论不靠眼力，靠相邻差值：当差值降到前一段的 60% 以下时，就是拐点。

为什么不直接买到缺货概率接近零？因为 8 件是 3.495%，要降到 1% 以下需要 10 件以上，而第 9、10 件的边际改善已经不到 2 个百分点，库存成本却按件线性增加。

备件的另一个约束是通用性。6 件同型号备件是浪费，3 种型号各 2 件，覆盖面更广。实际项目通常是“关键件 2 件、常用件 1 件、其他共用”，总量落在 6 到 8 件。

备件清单要跟故障率一起更新。年故障率从 4.38 降到 2.0 时，拐点会从 6 件降到 3 到 4 件，原来的库存就变成沉没成本。

93.8.8 参数不达标时先查原因

调参的前提是知道哪一项不达标，以及为什么不达标。直接改参数是最后一招。

图 93-7 把“偏差持续增大”按现象形态分成三条分支：阶跃后偏差、缓慢漂移、偶发跳变。

第一步永远是把现象分类。阶跃后偏差指给定值或负荷突变后偏差回不到零，这类问题在参数与模型上；缓慢漂移指偏差一天天变大，这类问题多在传感器与阀门；偶发跳变指数据突然跳一下又回来，这类问题在通信链路。

第二步按分支落到具体原因。阶跃后偏差对增益或滞后失配，办法是重做阶跃辨识；缓慢漂移对传感器或阀门，办法是校零与查阀行程；偶发跳变对通信丢帧或缓存，办法是查链路与队列。

走错分支是最常见的浪费。把缓慢漂移当成参数问题，会反复整定 PID，整完当天正常，第二天又漂；把偶发跳变当成传感器问题，会换掉一个本来完好的变送器，问题还在。

排查要用对照法。同一个偏差在两台相同设备上都出现，是共性问题；只在一台上出现，是个性问题。共性查组态与参数，个性查接线与仪表。

排查要留记录。每次排查写下“现象、假设、验证方法、结论”四栏，三个月后同类问题再出现，可以直接翻记录。

图 93-7 的用法是：拿着现象在树上走一遍，走到第三层就是原因类别，第四层是处置动作。它不告诉你答案，它告诉你该往哪儿找。

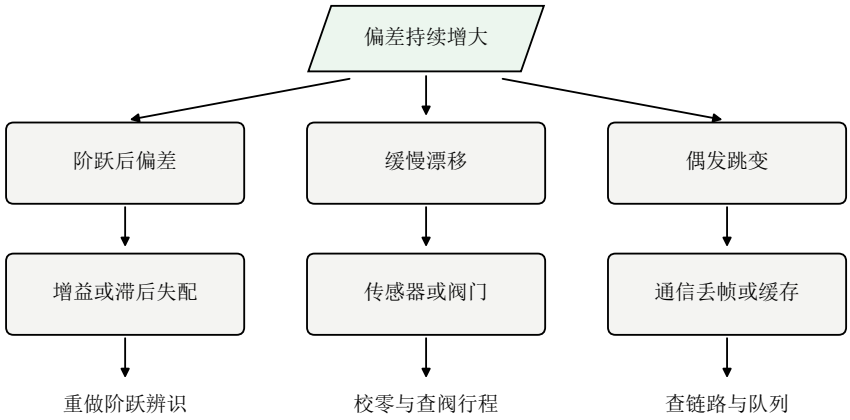


图 93-7 现场偏差的三分支排查树

树解决“往哪儿找”，图 93-10 解决“谁来找到什么时候”。投运之后，现场值班、远程支持、厂商支持是三级：现场十五分钟内完成报警确认与初判，三十分钟内把证据交到远程；远程接入后判定能否回滚，能回滚就先恢复再分析；四小时内仍定不下来，交厂商做根因分析与补丁。

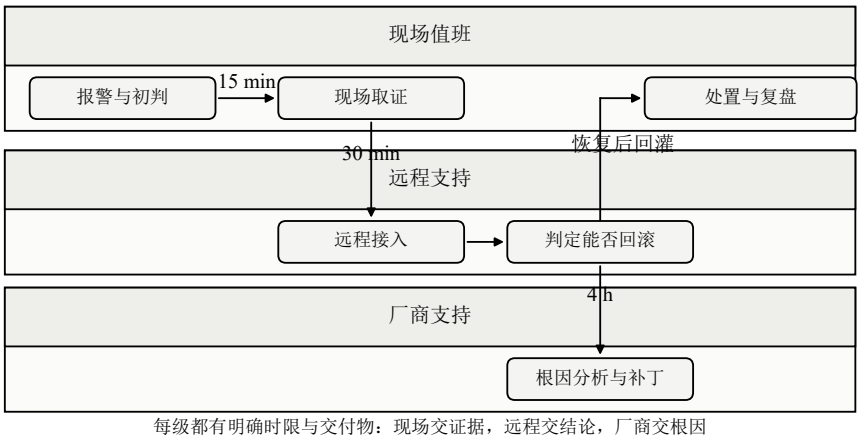


图 93-10　投运后的值班分工与升级路径

三级之间靠交付物衔接：现场交证据（原始趋势、报警记录、当时的手动操作），远程交结论（是参数、是配置、还是硬件），厂商交根因（哪一版固件的哪一段逻辑）。缺哪一样，下一级就只能重做上一级的工作，时间在交接处成倍流失。

升级不是甩锅。现场把问题交出去之后仍要跟踪处置结果并参与复盘，因为只有现场知道当时的工况；远程接入之后仍要对回滚决策负责，因为只有远程看得到全部节点的版本分布。三级是并行的责任链，不是串行的接力棒。

93.8.9 调参记录表

调参必须留表，否则下一次调参要从零开始。

记录表至少六列：时间、参数名、原值、新值、改动原因、效果。效果一栏要写数值，不能写“感觉好多了”。

表的作用有三个。一是复盘，出了问题能回溯到哪次改动；二是交接，换人接手时知道哪些参数动过；三是验收，业主问“为什么灰度定 10%”时，表里有当时的计算。

记录表建议跟组态版本一起管。参数改动通常是随组态下发的，两者对不上时，现场跑的和文档写的就不是一回事。

93.8.10 一成一败两个案例

成功案例来自一个 40 回路的改造项目。门按六段四门做，HIL 阶段发现两个协议解析缺陷；灰三分批 5%、10%、25%；首批 10 台在第二个班次发现一个数据刷新缺陷，按预算 25 min 回滚；最终工期 4 天，报警负荷 11.3 条/h，五项验收过四项。唯一没过的是文档齐套率 95%，缺的是联锁逻辑的变更记录。

失败案例来自一个 120 回路的项目。为了赶节点，阶段压到三段，灰度只做一批 2%，回滚包没预置。投放第三天，上位机与固件版本不匹配，出现批量通信中断，回滚时要从总部传 2.3 GB 的包，实际耗时 4 h 17 min。事后统计，直接停机损失 38 h，返工 11 天。

两个案例的差别不在技术，在有没有留回退路。成功案例的 25 min 回滚是事先设计出来的，失败案例的 4 h 是事到临头才发现没有包。

失败案例还有一个细节值得记：他们的版本兼容矩阵是 81 种全测，测了 210 h，但恰恰漏了“固件升、上位机没升”这一种，因为那一版上位机在测试清单里被标成“旧版，不需要测”。全测不等于测对。

93.8.11 调参的三条忌讳

第一条忌讳是一次改三个参数。改完效果好，说不清是谁的功劳；效果差，也不知道该退哪一个。

第二条忌讳是拿平均值当目标。报警负荷 11.34 条/h 是平均值，夜班可能 3 条，白班可能 19 条。平均值达标而峰值超限，操作员照样处理不过来。要看分位数，不看平均。

第三条忌讳是只在仿真上调。仿真里回滚只要 30 秒，现场要 25 min；仿真里通信不丢包，现场每天丢几次。参数要在现场用真实数据复核一遍，才算定下来。

93.8.12 三个现场小故事

第一个故事讲灰度的价值。某项目首批 10 台，第二天发现数据刷新有 3 秒延迟。因为只有 10 台，业主没察觉，团队当晚修好，第二天重放。如果首批是 100 台，这个 3 秒延迟会在交接班时被当成系统故障报上去，后面要花一周解释。

第二个故事讲回滚包的价值。某项目升级失败，现场工程师从抽屉里翻出一张两年前的 U 盘，里面的组态是半年前的版本。用这个版本回退，虽然系统能跑，但少了三个回路。事后复盘，问题不在回滚，在回滚包没人维护。

第三个故事讲参数标定的价值。某项目照搬了同类项目的单回路工时 1.2 h，实际做下来是 2.1 h，因为那套装置的接线端子排布完全不同。两个人做 40 个回路，工期从 5 天变成 8 天，业主扣了违约金。如果开工前用前三个回路标定一次，这个偏差当场就能发现。

三个故事的共同点是：出问题的环节都很朴素——批次、备份、标定。它们不需要高深技术，只需要有人把它们当回事。

本章前面那些公式，作用就是让“当回事”有个具体的尺子。没有尺子，“当回事”会变成“尽量小心”，而“尽量小心”在工期压力下最先被牺牲。

93.9 优化与改进

93.9.1 四个改进方向

工程落地可以改进的地方只有四类：少犯错、早发现、快恢复、省人力。

“少犯错”对应版本管理与兼容规则，目标是让本来就进不来的缺陷不进来。“早发现”对应阶段门与测试，目标是让进来的缺陷早一阶段被发现。“快恢复”对应回滚与可观测性，目标是让漏出去的缺陷影响面小。“省人力”对应工期与自动化，目标是同样的活少用人时。

四个方向的投入产出差别很大。按本章的算例，回滚从 35 min 压到 25 min 只需三个人日的一次性准备；而把逃逸率从 0.246% 再降一个数量级，需要建 HIL 台架，投入是前者的几十倍。

所以改进的排序不是“哪个最先进做哪个”，是“哪个最便宜先做哪个”。

改进还有一条隐含规则：**先做能验证的**。做完能不能量化，决定这项改进会不会在半年后被悄悄放弃。

93.9.2 改进有依赖顺序

四个方向之间有依赖，不能乱序。

没有可观测性，谈不了快恢复。故障发生了看不见，回滚再快也不知道该回。所以可观测性排在回滚之前。

没有版本规则，谈不了少犯错。版本号乱、兼容关系说不清，灰度就没法判定“这一批跑的是哪一套”。

没有阶段门，谈不了早发现。门是证据的收集点，没有门，缺陷在哪个阶段被发现的都记不下来，也就无法判断该加强哪一段。

没有稳定的基线，谈不了省人力。自动化脚本在天天变的组态上跑，维护成本比手工还高。

依赖顺序可以用一句话概括：**先能看见，再能退回；先能分清，再能灰**。

按这个顺序，一个从零开始的项目的改进路线是：可观测性→回滚→版本规则→灰度→阶段门→自动化。反过来做，通常是先上自动化脚本，然后发现脚本要跟着每天变的组态改，最后放弃。

93.9.3 缩短回滚：35 min 到 25 min

回滚时间的四段是发现 5.0、决策 10.0、执行 8.0、验证 12.0，合计 35.0 min，超 30 min 目标 5 min。

三项改进各自打在哪一段上，要先看清楚。预置回滚包打在执行段，省 3 min；自动执行脚本也打在执行段，省 4 min；预演过的验证清单打在验证段，省 3 min。

三项合计省 10 min，落到 25.0 min，余量 5.0 min。余量不是浪费，它是给“这次回滚比预想的复杂”留的空间。

为什么不打决策段？决策段 10 min 看起来最肥，但它省不掉。回滚决策要有人签字，签字前要看证据，这两件事压缩到 3 min 就会出错。可行的办法不是压缩，是**预先约定触发条件**：满足条件直接执行，把决策从事前移到事后补签。

为什么发现段只有 5 min？因为发现靠监测，5 min 已经是“一轮采集周期加一次判定”的下限。再往下压要靠更密的采集，代价是误报增加。

改进的验收标准要写成“演练实测”。写“预计 25 min”没有意义，要写“连续三次演练都不超过 25 min”。演练时间从第一次到最后一次通常会缩短 20% 到 30%，因为人熟了。

93.9.4 减少回归量：规则比算力便宜

四类组件各三版，全组合 81 种，按 40 例、每例 3 min、准备系数 1.30 算，是 210.6 h，折 27 个工作日。

三条兼容规则筛出 26 种，占 32.1%，工时 67.6 h，折 9 个工作日。省下 143 h。

省下来的不是靠“少测”，是靠“不测不可能的”。三条规则是：协议版本不高于固件版本、上位机版本不低于组态版本、固件与组态相差不超过一版。

这三条规则不是拍脑袋定的，是从接口方向推出来的。协议由固件实现，固件不支持的协议版本一定跑不通；组态由上位机下发，上位机比组态旧就一定解析不了新字段；固件与组态跨两版时，中间版本的迁移逻辑通常已被删掉。

规则的维护成本很低。新增一类组件时，只需要补一条规则，组合数就从 81 变 243，而允许组合不会等比例增长。

规则的风险是“筛掉了本该测的”。上一节那个失败案例就是这么出的：他们把“旧版上位机”整类标成不需要测，而那一类里恰好有一种组合是现场真实在跑的。**规则要按现场实际组合校准，不能只按设计意图。**

改进的方向因此不是加更多规则，是加一条“现场存量清单”约束：筛出来的 26 种必须覆盖现场在运行的全部组合，不覆盖的补进来。

93.9.5 减少逃逸：把缺陷往左推

逃逸率 0.246% 是五级串联的结果。要降低它，最有效的不是加强最后一级，是加强前两级。

原因是乘法结构。逃逸率是各级漏检率的乘积，把某一级的漏检率减半，逃逸率就减半，不管它是第几级。但成本完全不同：设计评审把漏检率从 70% 降到 35%，只需要两个人两天；HIL 把漏检率从 20% 降到 10%，需要一套台架。

所以“左移”在数学上成立：同样的逃逸率改善，越靠左越便宜。

左移的具体做法有三条。一是评审清单化，把历史上出过的问题变成检查项，新项目照着走；二是桌面检查带数据，不看代码看录波，用真实数据跑一遍逻辑；三是 MIL 阶段就把边界工况跑掉，别等到 HIL。

左移的极限在哪里？在于“左边的信息量不够”。设计评审看不到时序细节，桌面检查看不到硬件非线性。有些缺陷只能在 HIL 或现场暴露，硬要左移，会变成在评审会上猜。

一个可用的判据是：**某一级检出的缺陷里，如果有超过一半是“上一级本来能发现”的，说明左移还有空间；如果大部分是只有这一级才能发现的，说明位置放对了。**

93.9.6 提高可观测性

可观测性分三层：日志、指标、追踪。三层的作用不同，缺一层就会在排查时卡住。

日志回答“发生了什么”。要求是带时间戳、带回路号、带前后值。只有一句“通信异常”的日志，等于没有。

指标回答“现在怎么样”。要求是能看趋势、能设阈值。报警负荷、通信丢帧率、扫描周期、控制器负载，这四个是底线指标。

追踪回答“为什么”。要求是一条命令能看到某个回路的完整链条：从现场值到控制器、到上位机、到数据库。链条断在哪一环，问题就在哪一环。

三层里最容易缺的是追踪。日志和指标做起来直观，追踪要做好需要统一的请求标识，前期设计没留，后期补不上。

可观测性的投入产出比很高。本章的远程诊断省 1.0 h MTTR，前提就是能远程看到三层信息；看不到，远程接入也只是远程打电话。

可观测性还有一个副产品：它让阶段门有了证据。出厂测试报告里的数据，来源就是这三层。

93.9.7 三条底线不能碰

改进有边界，三条底线不能碰。

第一条是回滚必须可用。任何改进如果让回滚变复杂，就不做。比如为了统一版本管理，把回滚包做成“只能整体回退”，结果单回路故障也要全系统回退，这就是负改进。

第二条是数据不能丢。为了提性能改采集周期、改缓存策略，必须保证掉电和重启时数据不丢。这条底线在验收里对应“数据完整率 99.4%”。

第三条是安全回路不能被工程改动影响。联锁与急停的逻辑改动要单独走审批，不能跟着常规组态一起灰度。安全回路的验证方式和常规回路不同，混在一起做，两边都做不好。

三条底线的共同点是：它们都是“不可逆”的。回滚不可用、数据丢了、安全逻辑改错了，这三件事都没有第二次机会。

93.9.8 怎么证明改进有效

改进要证明有效，需要三样东西：基线、对照、指标。

基线是改进前的数值。回滚 35 min、回归 210.6 h、报警 120 条/h、年停机 17.49 h，这些是基线。没有基线，改进后的数字没有意义。

对照是同一口径下的对比。口径包括：同样的回路数、同样的人数、同样的工艺负荷。换了口径再比，比出来的差异说不清是改进带来的还是条件变化带来的。

指标要选能被改进直接影响的。回滚改进看“演练实测中位数”，回归改进看“允许组合数与覆盖率的乘积”，报警改进看“每班报警条数的 95 分位”。选一个笼统的“系统更稳定了”，没法证伪。

一个实用的做法是**先写预期，再测**。改进前写下“预计从 35 min 降到 25 min”，改完测出来 27 min，差距 2 min 就要解释。先测后写，人会不自觉地往结果上靠。

改进效果的复查周期建议一个季度。季度太短，数据量不够；年度太长，中间已经换了人和组态。

93.9.9 三个反例

第一个反例是“为了减少报警把阈值调宽”。报警从 120 条/h 降到 10 条/h，达标了，但三次真实超温没报出来。这种改进在数字上完美，在安全上是倒退。判断办法是同时看“漏报数”，只看总数一定被误导。

第二个反例是“为了缩短工期砍掉联锁调试”。40 回路里联锁占 0.3 h/回路，砍掉能省 12 h，工期从 4 天变 3 天。代价是投运后第一次联锁动作失败，停车 6 h。

第三个反例是“为了统一版本强制所有现场升级”。升级本身没错，错在没有灰度，一次推 200 台。其中 3 台的老旧控制器不支持新固件，刷完起不来，现场停产两天。改进的正确做法是先识别不兼容的 3 台，单独处理。

三个反例的共同点是：改进的收益被计入了，代价没有。任何改进都要把“最坏情况”写进方案，否则它只是把一个显性成本换成了一个隐性风险。

93.9.10 一项改进的完整过程

拿“缩短回滚时间”做例子，走一遍 §93.9.8 的三样东西。

基线先测。选一个非关键单元，做三次回滚演练，取中位数。第一次 46 min，第二次 38 min，第三次 35 min，中位数 38 min。这比纸面算的 35 min 还多 3 min，因为现场多了找工具、确认工艺状态这些动作。

对照要同口径。改进前后都用同一个单元、同一批人、同样的工艺负荷。换了人再比，快的那次可能只是人熟。

改进分三步做。第一步把回滚包预置到现场工控机的本地盘，省掉从总部传输的时间；第二步写一个脚本，把停止、替换、启动、自检串起来；第三步把验证清单从二十项删到九项，删掉的十一项是“看一眼就够”的。

改进后再测三次。中位数 26 min，比预期的 25 min 多 1 min。多出来的 1 min 在确认工艺状态上，这个动作不该省，所以接受 26 min，并把预期改成 26 min。

结论要写清适用范围。26 min 是在这个装置、这批人、这个网络条件下测的；换成跨省远程接入，传输那一段会重新变长，26 min 不成立。

整个过程花了两周，其中改进实施只占三天，剩下的是演练与等待停机窗口。这个比例很正常：**改进本身不难，难的是找到可以演练的时间**。

93.10 工程落地

93.10.1 落地从投运第一周开始

投运不是一个时刻，是一个过程。本章把“投运”定义为第一台回路切到自动，把“验收”定义为五项指标全部取证完成，两者之间通常隔一到三周。

这两到三周是系统最脆弱的时候。缺陷密度最高，操作员最不熟，备件还没到位，值班表刚排出来。工程落地的质量，主要就取决于这段时间怎么过。

第一周的目标不是“跑得好”，是“看得清”。把三层可观测性跑起来，把报警负荷统计起来，把每一个异常都记下来，比追求零故障更有价值。

第一周最常见的错误是“一切正常就不用管”。报警少不代表没问题，可能是报警被抑制掉了，也可能是数据没采上来。要主动查一遍数据完整率，而不是等业主发现曲线断了。

第一周还要做一件事：把回滚演练一次。不是等出事再演练，是在系统还干净的时候演练，这样出问题时才有肌肉记忆。

93.10.2 投运第一周的每日安排

第一天做基线采集。全部回路切手动，逐个记录稳态值、阀位、扫描周期，形成投运前的基线。这份基线是后面判断“偏差是否正常”的唯一依据。

第二天到第三天分批投自动。按工艺单元分批，不按回路号分批，因为同一单元的回路互相耦合，分开投会互相干扰。

第四天做扰动试验。给定值阶跃、负荷阶跃各一次，记录响应曲线，与 MIL 阶段的结果比对。偏差超过 20% 就要查原因，按图 93-7 的三条分支走。

第五天统计报警。按班次统计条数与类型，找出前十条高频报警，逐条判断是无效报警还是真实工况。这一步通常能砍掉一半无效报警。

第六天做一次回滚演练。选一个非关键单元，把组态切回上一版，计时，再切回来。目标 25 min，第一次做不到是正常的，要记录卡在哪一段。

第七天开第一次周会。把前六天的记录摊开，定下一周的三个动作。周会不超过一小时，只对事不对人。

93.10.3 现场调试的一天怎么排

现场调试的工时紧，一天的排法直接决定效率。推荐按“上午接线、下午投用、收工前记录”排。

上午做点检与接线核对。这一段需要断电或旁路，必须在工艺负荷低的时候做。40 回路按 0.3 h 算，是 12 h，两个人一个上午刚好。

下午做单回路投用与联锁。这一段需要工艺配合，要在操作员当班、工艺稳定的时候做。单回路 0.6 h 加联锁 0.3 h，40 回路是 36 h，三个人两个下午。

收工前半小时做记录。记录三样：今天投了哪些回路、遇到哪些问题、明天从哪开始。不记录，第二天要花半小时重新熟悉现场。

这个排法有个前提：办票要提前一天。工作票、动火票、临时用电票，任何一张没批下来，上午的计划就废了。办票靠的是沟通，不是技术。

另一个前提是通信要提前打通。现场调试最怕的是“控制器通了、上位机不通”，两边各说各的对。开工前一天要把三层链路各测一次。

93.10.4 验收会怎么开

验收会不是汇报会，是取证会。每个指标都要有数据来源、采集时间、见证人。

会前一周发验收清单。清单上写清五项指标、目标值、实测值、取证方式、责任人。发晚了，被验收方来不及准备数据，会上只能吵。

会前三天完成预验收。预验收由集成商自己做，把能查的先查一遍。预验收发现的问题当场改，不要留到正式会。

正式会控制在两小时。前一小时逐项过指标，后一小时处理未达标项与遗留问题。超过两小时，人会疲劳，决定质量下降。

未达标项要有处置方案。是限期整改、是降级验收、还是扣款，会上要定下来。写“后续跟进”等于没定。

验收报告要附证据。每个数字后面跟一个文件编号，文件包括趋势记录、测试报告、日志截屏。没有证据的数字，下一次审计就要重做。

93.10.5 五项指标的取证方式

回路投用率 96.5% 的取证方式是控制系统自带的投用统计，取验收前连续七天的日平均值。手工数容易错，也容易被质疑。

报警负荷 11.3 条/h 的取证方式是报警日志按小时统计，取七天的 95 分位而不是平均。用平均会把夜班的低负荷摊进来，掩盖白班的峰值。

数据完整率 99.4% 的取证方式是数据库里应有采样点与实际存储点的比值，按天统计。要注意“应有”的定义，通信中断期间的点算不算应有，要在合同里写清。

可用度 99.88% 的取证方式是运行日志里的停机时长统计，分母是日历小时。停机时长要区分计划停机与非计划停机，本章的可用度只算非计划。

文档齐套率 95% 的取证方式是文档清单逐项核对。这一项最容易失分，因为它不靠技术，靠日常积累。

五项里前四项都是自动统计，第五项是人工核对。所以第五项要提前一个月开始准备，别等到验收前一周。

93.10.6 交接三样东西

交接口头说的不算，要交三样：文档、账号、备件。

文档不是一套图纸，是“能让人独立运维”的最小集合。包括组态备份、点表、联锁逻辑说明、回路整定参数、故障处理手册。五样缺一样，接手的人就要打电话问。

账号包括控制器、上位机、数据库、历史站四类。交接时要改密码，并且把改后的密码封存。用原来项目组的账号，出问题查不到人。

备件要建账。账上写清型号、数量、存放位置、最低库存。备件不是买了就算交接完，要能在十分钟内找到。

三样之外还有一样隐性东西：联系人清单。谁负责控制器、谁负责网络、谁负责工艺，清单上要有电话。这份清单在半夜出事时最有用。

交接要有签字。签字不是形式，它把“我以为你交了”变成“你确认交了”。没有签字，三个月后扯不清。

93.10.7 值班表与响应分级

投运后的前三个月要有人值班。值班不等于守在控制室，是要保证电话能打通、远程能接入。

响应分三级。一级是安全或环保相关，15 min 内响应；二级是影响生产的，1 h 内响应；三级是不影响生产的，次日处理。

分级要写进值班表。写“随时待命”没有可执行性，写“一级 15 min、二级 1 h”才能排班。

值班表要留冗余。一个人值班，他请假就断线。至少两个人能处理一级问题，其中一人可以是远程支持。

值班记录要跟故障记录合并。每次响应写清时间、现象、处置、恢复时间，这些记录就是 MTTR 的数据来源。没有记录，可用度只能估。

三个月后可以转为巡检制。巡检频次按故障率定，本章的算例是年故障 4.38 次，巡检每周一次足够。

93.10.8 备件账怎么建

备件账的目的是“十分钟内能找到、能换上”。账目本身简单，难点在分类。

按关键度分三类。A 类是一停就停产的，必须常备；B 类是可以短时旁路的，备一件；C 类是可以外购的，不备或共用。

本章算出的 6 件拐点，对应的就是 A 类加 B 类的总量。具体分配通常是 A 类 3 件、B 类 2 件、C 类 1 件。

账上要写“替代型号”。原型号停产是常态，没有替代型号，备件就是废铁。替代型号要经过实测，不能只看参数表。

备件要定期通电。电子件放两年不用，电容会失效。建议每半年上一次电，跑一次自检。

备件账要跟故障记录联动。某个型号一年换了三次，说明它不是备件问题，是选型问题，要报给设计方。

93.10.9 变更管理流程

投运后的每一次改动都要走变更流程。流程不必复杂，但四个环节不能少：申请、评估、执行、验证。

申请要写清改什么、为什么改、不改会怎样。写“优化一下”的申请，一律退回。

评估要写清影响面。改一个回路的整定参数，影响面是一个回路；改通信协议，影响面是全系统。影响面决定要不要灰度。

执行要有回退方案。没有回退方案的变更不批。回退方案包括回滚包位置、执行人、验证清单。

验证要留记录。改完观察多久、看哪些指标、达到什么条件算成功，事前写清。

四个环节里最容易被跳过的是评估。跳过评估的变更，往往在验证阶段才发现影响面比想象的大。

变更记录要跟组态版本绑定。一个版本号对应一批变更，查问题时能一次查到全部改动。

93.10.10 三个月路线图

第一个月保稳定。目标是零非计划停机，报警负荷稳定在 12 条/h 以下，数据完整率 99% 以上。这个月不做任何功能改动。

第二个月补短板。把第一个月记录的问题分类，按频次排序，改前三个。同时把文档补齐，尤其是变更记录与联锁说明。

第三个月做优化。在稳定运行的基础上，做一次版本升级或功能扩展，走完整的灰度流程。这次灰度就是团队能力的检验。

三个月路线图的关键是“第一个月不改”。很多项目投运第二周就开始加功能，结果基础没稳住，新功能又带来新问题，最后说不清是哪一个的问题。

三个月后进入常规运维。常规运维的节奏是：每周一次巡检、每月一次指标复盘、每季度一次回滚演练、每年一次版本评估。

路线图要写进合同或备忘录。没有书面约定，第一个月的“不改”很容易被业主的临时需求打破。

93.10.11 周会与月报

周会看三样：本周异常、下周动作、需要谁支持。每样不超过三条，多了记不住。

月报看四组数：可用度、报警负荷、未闭环问题数、变更次数。四组数连起来看趋势，单看一个月的值意义不大。

周会与月报的作用不是汇报，是暴露问题。一个不敢在会上说问题的团队，问题会在地下积累，直到某天集中爆发。

会议记录要留档。三个月后复盘，会议记录是最好的材料，它记录了当时是怎么想的。

周会不要超过一小时，月报不要超过两页。超了，就没人看。

93.10.12 一个反面案例：文档齐套率 95%

本章的验收算例里，五项过四项，唯一没过的是文档齐套率 95%，目标 100%。

缺的是什么？缺的是联锁逻辑的三次变更记录。这三次变更都做了，也都验证了，但没写进文档。

为什么没写？因为变更时在赶工期，想着“先改，回头补”。回头就是三个月后，补的时候已经记不清改了哪几处。

这一项的失分很典型。它不影响系统运行，不影响安全，但它让验收从“全过”变成“过四项”，业主有理由扣款。

更重要的问题在后面。半年后另一个回路出问题，排查时发现联锁逻辑与图纸不符，多花了两天才找到原因。这两天就是文档缺失的真实代价。

教训有两条。一是变更记录要在变更当天写完，不能等；二是文档齐套率要每周统计一次，别等验收前才发现差 5%。

文档这件事没有技术含量，只有习惯。习惯养成的成本很低，补的成本很高。

93.10.13 现场调试要带的三张表

现场调试最怕“跑一趟回来发现缺东西”。带三张表，能避免大部分往返。

第一张是回路清单。每行一个回路，列写位号、工艺描述、仪表型号、量程、对应控制器通道、联锁归属、投用状态。清单要在出发前与业主核对一遍，现场改一行，回来改十行。

第二张是问题清单。每行一个问题，列写发现时间、现象、影响回路、临时措施、根本原因、关闭时间。问题清单是调试日志的骨架，也是验收时“遗留项”的来源。

第三张是版本清单。列写控制器固件、组态、上位机、协议的版本号，以及每台设备的实际版本。这张表最容易漏，因为它不是“调试”的一部分，但它是灰度投运的前提。

三张表都用纸质加电子两份。现场环境差，笔记本摔了、没电了、网断了都常见，纸质那份是兜底。

表的格式越简单越好。列太多，现场没时间填；列太少，回来没法分析。七到九列是合适的范围。

三张表在调试结束时要一起归档。缺一张，交接就不完整，后面接手的人要重新摸一遍现场。

93.10.14 第一个月怎么算成功

第一个月的成功标准不能定成“没出事”，因为没出事可能是运气好、也可能是没人报。

可用的标准有四条。一是非计划停机为零，这条最硬，但样本少，一个月不足以证明。二是报警负荷稳定在 12 条/h 以下，这条样本多，能反映真实运行状态。三是每个异常都有记录，数量不重要，闭环率要到 90% 以上。四是操作员能独立处理至少一次二级告警，不用打电话问厂家。

四条里第四条最容易被忽略，也最能说明系统是否真的交付了。操作员不敢动、不会动，系统就还挂在厂家身上。

第一个月末要做一次复盘，把四条逐一核对。核对的结论要写下来，写“基本达标”没有意义，要写“报警负荷 11.6 条/h，异常 27 项闭环 25 项，操作员独立处理 2 次”。

复盘的目的是定第二个月的动作，不是评功过。哪一条没达标，第二个月就补哪一条，其余不改。

如果四条全部达标，也不要立刻加功能。再稳一个月，让基线更可信，再加功能才不容易把水搅浑。

93.10.15 交接之后的三件事

签字交接不等于事情结束。接手方在头一个月通常会找回来三次，三次都处理好，关系才算稳定。

第一次通常问“这个参数为什么这么设”。答复不要只说数值，要说当时的依据与边界条件，让对方知道改了会怎样。

第二次通常问“这个报警为什么老响”。答复要给出判断方法，而不是直接删掉报警。直接删，下次真异常就没人知道。

第三次通常问“能不能加个报表”。这是好事，说明系统被用起来了。按变更流程走，别当场答应，也别一口回绝。

三件事的处理方式，决定了这套系统能不能在别人手里活下去。

交接的最终标志不是签字，是对方能独立做一次变更、独立回滚一次、独立处理一次告警。这三件事都做到，才算真正交出去了。签字只是开始，能独立运维才是结束。

93.11 优缺点与适用边界

93.11.1 四种落地形态

同样是计算机控制落地，四种形态的重点完全不同。按规模和来源分，见表 93-12。

形态	典型规模	主要风险	本章重点
单体交付	20–60 回路，单一装置	现场条件与设计不符	阶段门、现场调试工期
批量复制	同一型号 50 台以上	版本漂移、局部环境差异	版本兼容、灰度批次
试点扩容	先 1 套后 10 套以上	试点结论不可复制	灰度、验收指标
存量改造	老系统替换，不能长停	回滚窗口极短	回滚预算、变更管理

表中“主要风险”一栏是这一形态最容易出事的地方，不是全部风险。做项目时先看自己属于哪一种，再决定把力气花在哪。

单体交付的规模小，风险集中在现场条件。批量复制规模大，风险集中在版本一致性。试点扩容的风险在“试点成功、扩容失败”这个断点上。存量改造的风险在“改的时候不能停”。

四种形态的共同点是都要过阶段门。差别在于门的证据不一样：单体交付看现场记录，批量复制看版本矩阵，试点扩容看对比数据，存量改造看回滚演练。

93.11.2 优点一：把定性说法变成可算的数

工程落地最常听到的评价是“这个系统稳”、“这个团队靠谱”。这类说法没法比较，也没法改进。

本章的做法是把它们换成数：逃逸率 0.246%、期望修复成本 6.62 人时、兼容组合 26 种、灰度暴露概率 92.31%、回滚 25 min、工期 3 天、报警 11.34 条/h、可用度 99.875%、备件 6 件。

换成数之后，三件事变得可能。一是比较，两个方案放在一起看谁的成本低；二是追责，出问题时能定位是哪一环失效；三是改进，知道动哪个数最有效。

这一优点在评审会上最明显。说“我们的测试比较充分”没人信，说“26 种组合、67.6 工时、覆盖现场全部存量”就有人信。

数值也有风险。数算得越细，越容易被当成精确预测。本章的数都是量级估算，误差 20% 是正常的。用它做决策可以，用它做承诺要慎重。

93.11.3 优点二：每一步都留了回退路

本章的第二个优点是结构性的：从阶段门到灰度到回滚，每一层都有退回的动作。

门不通过可以退回上一阶段；灰度不通过可以停在当前批次；回滚包随时可用；变更可以撤销。这四层回退叠起来，让“出错”从灾难变成事件。

回退能力带来的直接好处是敢做。一个能 25 min 回滚的系统，团队敢做灰度、敢做优化、敢接新需求。一个不能回滚的系统，任何改动都要开会讨论三天。

回退能力的间接好处是可追责。每次回滚都留下记录：什么时候、因为什么、退了哪一版。这些记录积累起来，就是缺陷分布的画像。

回退也有代价。预置回滚包要空间，演练要时间，双版本并行要维护。这些成本要提前算进预算，不能等出事才发现没有。

93.11.4 优点三：验收有据可依

第三个优点是验收。本章把验收拆成五项指标、五个取证方式、五个达标判据，每一项都能在系统里查到数据。

这让验收从“感觉差不多”变成“逐项打勾”。业主知道要什么，集成商知道交什么，双方不必在会上争。

验收有据还有一个长期好处：它为后续的运维建立了基线。验收时的 11.34 条/h、99.875%、96.5%，就是一年后判断“系统是否退化”的参照。

验收指标要认真选。选错了指标，会引导团队做错事。比如只考核报警总数，团队就会把阈值调宽；只考核可用度，团队就会把计划停机算成非计划。

93.11.5 缺点一：参数依赖经验，换项目要重估

本章最大的缺点是一切数字都带项目印记。

单回路 1.2 h 来自 40 回路的改造项目，换成 200 回路的新建项目，这个数可能是 2.5 h；检出率 30%、45%、60%、80%、92% 来自一支做过三个项目的团队，换一支新团队，第一级可能只有 15%。

参数不能照搬，但方法可以照搬。照搬参数的项目，算出来的工期往往比实际短一半。

弥补的办法是每个项目先做一次标定。用前三个回路实测单回路工时，用前两周的故障记录估 MTBF，用第一批报警统计估削减比例。标定花三天，能省后面三周的返工。

参数还有一个隐含依赖：它假设团队是稳定的。人员流动超过 30% 时，检出率会明显下降，回滚时间会变长。这一点在项目排期时要留余量。

93.11.6 缺点二：门在短期是负担

阶段门在长期省成本，在短期是纯负担。每道门要写证据、要开会、要签字，一个项目四道门，至少多花五到八个人日。

赶节点的项目最恨这个。业主催着投运，集成商还要停下来做评审，双方都不痛快。

短期负担与长期收益的权衡，取决于项目后续还有多长。一个只做一次的单体项目，门的收益可能抵不上成本；一个要做十套的复制项目，门是第一套就该建好的基础设施。

折中的做法是门的形式随规模变。单体项目用检查表代替评审会，半天完成；批量项目必须开评审会，因为一次漏检要乘以十。

不能折中的是“证据”这件事。门可以简化，证据不能省。没有证据的门，等于没有门。

93.11.7 缺点三：不解决控制算法本身

本章讨论的是工程落地，不是控制质量。回路整定得不好、模型辨识得不准、解耦设计得不合理，这些本章都不解决。

一个算法很差的系统，按本章的方法落地，结果是“稳定地差着”。它能投运、能验收、能回滚，但控制指标上不去。

这一点要说清楚，避免误用。本章的位置在第 86 到 93 章的末尾，前面各章解决的是“怎么算得对”，本章解决的是“怎么让它真的跑起来”。

两者的关系是乘法不是加法。算法好、落地差，系统上线即崩；算法差、落地好，系统能跑但不好用。两个都要，缺一个都不行。

实际项目里，落地问题造成的损失通常比算法问题大，因为落地问题影响面广、恢复慢。

93.11.8 适用的场景

本章方法最适合三类场景。

第一类是有明确验收指标的项目。指标一旦写进合同，本章的五项取证方式可以直接用。

第二类是多套复制或分期扩容的项目。版本兼容矩阵与灰度机制，在第二套之后开始产生明显收益。

第三类是不能长时间停产的改造项目。回滚预算与变更管理，是这类项目的生命线。

三类场景的共同点是“错一次代价大”。代价小的项目，用轻量流程就够，不必上全套。

判断标准可以简化成一句：**如果这个系统停了，会不会有人半夜打电话**。会，就用本章的方法；不会，可以简化。

93.11.9 不适用的场景

有四类场景不适合照搬本章。

第一类是实验装置与教学系统。回路少、可停、无验收压力，上六段四门是浪费。

第二类是单次性的小改造。总共 5 个回路，做完全套文档的时间比调试还长。

第三类是算法研究阶段。此时系统还在变，版本管理没有意义，因为每个版本都不打算保留。

第四类是业主明确只要“能跑就行”的项目。这种情况下，本章的价值无法被验收，做了也白做。

不适用的判断要诚实。把方法用在不合适的地方，结果是流程负担重、团队抵触、最后连该用的地方也不用了。

93.11.10 三个自检提问

落地之前，用三个问题自检。

第一个问题：这套系统出问题时，最快多久能退回去？答不出具体分钟数，说明回滚没设计。

第二个问题：这一版跑的是哪个版本组合，现场在跑的又是哪个？答不出，说明版本管理没建。

第三个问题：验收时拿什么数据证明达标？答不出，说明指标没定义。

三个问题都答得出，本章的方法基本用上了；有一个答不出，那一个就是最该补的短板。

三个问题的顺序也有讲究。先答回滚，再答版本，最后答验收。因为回滚是底线，版本是前提，验收是结果。

93.11.11 与相邻章节的边界

本章与第 90 章的分界在“通信通不通”与“系统能不能交付”。第 90 章解决协议、实时性、冗余，本章假设这些已经通了。

本章与第 91 章的分界在“孪生在不在”与“孪生怎么用”。第 91 章建数字孪生与边缘计算，本章用它们做 HIL 与灰度验证。

本章与第 92 章的分界在“先进控制算不算得出来”与“算出来能不能投”。第 92 章给算法，本章给算法上线的通道。

三章的关系是“能通、能算、能交”。三件事都完成，一套计算机控制系统才算真正交付。

边界清楚的好处是查问题时知道去翻哪一章。通信丢包翻第 90 章，模型不匹配翻第 91 章，算法不收敛翻第 92 章，版本对不上、回滚回不去、验收过不了翻本章。

93.11.12 一页纸结论

把本章压成一页纸，是五句话。

第一句：落地的核心不是把系统调好，是让每一次改动都有进入条件、有退出证据、有回退路。

第二句：缺陷越早发现越便宜，五级串联的期望成本是 6.62 人时，全拖到投运后是 150 人时，差 22.7 倍。

第三句：版本组合不必全测，三条兼容规则能把 81 种筛到 26 种，工时从 210.6 h 降到 67.6 h。

第四句：灰度批次的拐点在 25%，低于 5% 等于没试，高于 25% 影响面失控。

第五句：回滚时间要算成四段之和，35 min 压到 25 min 只需三项准备，但这三项必须提前做。

五句话之外还有一句总的：**能退回去的系统，才敢往前跑**。

93.11.13 与三种常见做法的关系

现场常见三种落地做法：靠老工程师、靠流程制度、靠工具平台。本章与它们不是替代关系。

靠老工程师的做法，优点是判断快、能处理例外，缺点是人一走经验就断。本章的方法是把老工程师的判断变成参数与清单，让它能被复制。所以它不是否定经验，是给经验一个载体。

靠流程制度的做法，优点是责任清楚、可审计，缺点是容易变成填表。本章特意在每个环节都问“这个证据能不能被复核”，就是在防止流程空转。填表不等于有证据，有证据才叫过门。

靠工具平台的做法，优点是自动化、可追溯，缺点是把流程固化在工具里，改起来慢。本章的兼容矩阵、灰度批次、回滚预算，都可以做成工具里的一个配置项。工具是手段，判断标准还是本章这几条。

三种做法各有一半是对的。老工程师的判断力、流程的责任链、工具的自动化，缺一样都会出问题。本章提供的是把它们串起来的那条线，不是第四种做法。

选择哪一种为主，取决于项目规模。单体小项目以老工程师为主，配一张检查表；批量项目以流程为主，配一套工具；长期运维的平台以工具为主，配少量专家兜底。

93.12 Python 仿真代码

93.12.1 两个脚本与运行方式

`code/ch093_sim.py` 负责算，`code/ch093_figs.py` 负责画。两条命令，从 `book/` 目录运行。

```
python code/ch093_sim.py
python code/ch093_figs.py
```

第一条打印九段报告，第二条在 `figs/ch093/` 下生成十张图，每张一份 SVG 与一份 PDF，共二十个文件。

两个脚本只依赖标准库 `math`、`numpy` 与 `matplotlib`。没有随机数，不读系统时间，不读环境变量，不写日志文件。同一份代码在任何机器上重跑，输出逐位一致。

这一点不是洁癖。本章的数字要被正文、图、表三处引用，只要有一处不可复现，三处就会对不上。

93.12.2 参数区

所有可调量集中在 `ch093_sim.py` 的文件头部，命名全大写，单位写进注释。表 93-13 给出参数与正文的对应关系。

参数	取值	含义	正文位置
`PHASES`、`COST_ESCAPE`	五级、150 人时	检出率与修复成本	§93.2.1
`COMPONENTS`、`CASES_PER_COMBO`	四类各三版、40 例	版本组合与回归量	§93.2.2
`FLEET`、`P_TRIGGER`、`BATCH_PCT`	200 台、5%、六档	灰度暴露概率	§93.2.3
`ROLLBACK_STEPS`、`ROLLBACK_FIXES`	四段、三项	回滚预算	§93.2.4
`LOOPS`、`T_PER_LOOP`、`T_FIXED`、`ETA_PARALLEL`	40、1.2 h、16 h、0.85	现场调试工期	§93.2.5
`ALARM_BASE`、`ALARM_MEASURES`、`ALARM_LIMIT`	120 条/h、四项、12 条/h	报警负荷	§93.2.6
`MTBF_H`、`MTTR_STEPS`、`TRAVEL_H`、`MTTR_IMPROVE`	2000 h、四段、0.5 h、两项	可用度	§93.2.7
`WAIT_PER_STOCKOUT_H`、`SPARE_K`	24 h、七档	备件库存	§93.2.8
`P_UPGRADE_OK`、`SAVE_H_PER_YEAR`、`LOSS_ON_FAIL_H`、`UPGRADE_STOP_H`	0.85、16 h、12 h、8 h	升级决策	§93.2.9
`ACCEPT_ITEMS`	五项	验收指标	§93.2.10

改参数要成组改，不能只改一个。把 `FLEET` 从 200 改成 500，`BATCH_PCT` 的六档对应的台数全部变了，25% 从 50 台变 125 台，暴露概率从 92.31% 涨到 99.84%，“拐点在 25%”这句话就不再成立。

再比如把 `LOOPS` 从 40 改成 120，`T_FIXED` 16 h 的占比从 25% 降到 12%，人数从 3 人加到 4 人的边际收益会变大，原来“3 到 4 人封顶”的经验值就要重算。

93.12.3 九段报告索引

九段报告与正文章节的对应见表 93-14。正文里每个数字都应该在这张表里找得到出处。

段	内容	主要输出
`[1]`	缺陷拦截	五级的检出率、本阶段拦下、累计逃逸、期望成本
`[2]`	版本兼容	81 种总组合、26 种允许组合、67.6 h 与 210.6 h
`[3]`	灰度发布	六档批次对应的台数与至少一台触发的概率
`[4]`	回滚预算	四段耗时、累计 35 min、改进后 25 min
`[5]`	现场调试	三段工时、四个人数档的总工时与工作日
`[6]`	报警负荷	四项措施后的负荷、累计降幅、与上限的差
`[7]`	MTTR 与可用度	五段耗时、4.0 h 与 2.5 h、年停机小时
`[8]`	备件库存	七档备件数的缺货概率与年期望等待
`[9]`	验收与升级	五项达标情况、达标率、升级期望收益

`[1]` 段有一行特殊：最后一行“投运后发现”没有检出率，脚本单独判断后格式化，所以那一行的列宽与上面五行不同。这是唯一一处格式分支。

93.12.4 缺陷拦截与版本两段的实现

`phase_table()` 用一个循环维护 `survive` 变量。每进一级，本阶段拦下的比例是 `survive * p`，然后 `survive` 乘 `(1-p)`。循环结束后补一行“投运后发现”，成本是 `survive * COST_ESCAPE`。

`phase_summary()` 只做三件事：把第六列求和、取 `COST_ESCAPE` 作为最坏情况、两者相除。它返回三个数，不打印，方便别的函数复用。

`version_table()` 用四重循环穷举 3×3×3×3 共 81 种组合，对每一种调 `_combo_allowed()` 判断。判断逻辑就是三条规则，写成一行布尔与：

```
return (proto <= fw) and (hmi >= cfg) and (abs(fw - cfg) <= 1)
```

穷举而不是推公式，是因为规则以后可能改。加一条规则只需要改这个函数，不用重新推导组合数。81 次的循环代价可以忽略。

`version_full_hours()` 复用 `version_table()` 的总组合数，乘同样的用例数与准备系数，得到 210.6 h。两个数字用同一套系数，对比才公平。

93.12.5 灰度与回滚两段的实现

`canary_table()` 先按比例算台数，`max(1, int(round(FLEET * pct / 100)))` 保证不会出现 0 台。然后按独立事件算至少一台触发的概率 $P_{\text{hit}} = 1 - (1-p)^{n}$，其中 n 是这一批的台数，p 是单台触发缺陷的概率。

p 取 5% 是假设值，实际项目要用自己的缺陷密度估。这个数只影响曲线的陡缓，不影响“批次越大越容易暴露”这个结论的方向。

`rollback_table()` 返回两个东西：逐段累计的表和总时间。表里第三列是累计值，不是剩余值，这样读的时候能直接看出哪一段超了目标。

`rollback_after_fix()` 用总时间减去三项改进之和。三项改进是正数、代表节省的时间，调用处写 `base - save`，不写 `base + save`，方向不能错。

回滚段还有一处细节：`ROLLBACK_TARGET_MIN` 是 30 min，报告里同时打印“超目标多少”与“改进后余量多少”。前者是 5.0，后者也是 5.0，数字相同但含义相反，容易看错，所以两行分开打印。

93.12.6 工期与报警两段的实现

`commission_table()` 先把总工时算出来：`LOOPS * T_PER_LOOP + T_FIXED`，40×1.2+16=64 h。然后对每个人数档除以 `m * ETA_PARALLEL`，最后用 `math.ceil` 折成工作日。

用 `ceil` 不用 `round`，是因为工期只能向上取整。算出 3.1 天，实际排 4 天，排 3 天就会延期。

`commission_sequence()` 返回单回路的三段工时，只用于打印。三段加起来 1.2 h，与 `T_PER_LOOP` 一致。这两个数写在两处，改一处必须改另一处，脚本里用注释标了出来。

`alarm_table()` 用一个累乘的 `load` 变量。每项措施乘 `(1-r)`，同时记录相对原始负荷的降幅。累乘而不是累加，因为削减比例是相对当前的。

`alarm_limit_gap()` 重算一遍最终负荷再减上限，得到 −0.66。负数表示还有余量，正数表示超限。报告里用带符号格式打印，正负一眼能看出来。

93.12.7 可用度与备件两段的实现

`mttr_table()` 累计四段后再加上路程，返回表与 MTTR。`availability(mttr)` 返回可用度百分数与年停机小时，年停机用 `8760 * mttr / (MTBF + mttr)` 算，不是简单的 `8760 * (1-A)`，两者在数值上等价但前者精度更好。

`mttr_improved()` 从基准 MTTR 里减掉两项改善，得到 2.5 h。它不直接写 2.5，是为了让读者看到 4.0 减 1.0 减 0.5 的过程。

`spare_table()` 是本章唯一需要小心数值稳定的地方。泊松概率直接用阶乘会溢出：$\lambda = 4.38$ 时，$x = 199$ 的 $199!$ 远超双精度范围。脚本改成递推，用 $P(x) = P(x-1) \cdot \lambda / x$ 逐项往下算，初值取 $P(0) = e^{-\lambda}$。

这样只做乘除法，不会溢出，也不损失精度。

拿到概率后，缺货概率是 $1 - \sum_{x \le k} P(x)$，年期望等待是 $\sum_{x>k} (x-k) P(x)$ 乘每次缺货的 24 h。两个求和都截到 199 项，尾部概率已经小于 10^{-100}，截断不影响结果。

93.12.8 验收与升级两段的实现

`accept_table()` 要处理两种方向的判据。四项是“实测不低于目标”，报警负荷是“实测不高于目标”，所以循环里有一个条件判断：

```
ok = actual >= target if name != "报警负荷" else actual <= target
```

用指标名做分支不优雅，但只有五项，加一个方向字段反而更绕。如果以后指标增多，应该在 `ACCEPT_ITEMS` 里加一列方向。

`upgrade_expectation()` 返回三个数：期望年节省 $0.85 \times 16 - 0.15 \times 12 = 11.8$ h，首年净收益 $11.8 - 8 = 3.8$ h，回收期 $8 / 11.8 = 0.68$ 年。

回收期用 `if expect > 0` 保护，避免除零。期望为负时返回 `inf`，表示永远收不回。

`report()` 只做格式化。它按 `[1]` 到 `[9]` 的顺序调函数、打印，不做任何计算。所有数字都来自函数返回值，报告里出现的常数只有分隔线与标题。

93.12.9 十张图怎么生成

十张图分两类：流程与架构五张用 `flow_styles`，数据五张用 `plot_styles`。对应关系见表 93-15。

图号	文件	类型	数据来源
图 93-1	`fig_093_01_deploy`	流程图	手工布局
图 93-2	`fig_093_02_version`	框图	手工布局
图 93-3	`fig_093_03_canary`	折线	`canary_table()`
图 93-4	`fig_093_04_rollback`	流程图	手工布局
图 93-5	`fig_093_05_commission`	柱线双轴	`commission_table()`
图 93-6	`fig_093_06_alarm`	瀑布柱	`alarm_table()`
图 93-7	`fig_093_07_troubleshoot`	树状图	手工布局
图 93-8	`fig_093_08_availability`	分段横条	`mttr_table()`
图 93-9	`fig_093_09_spare`	半对数双轴	`spare_table()`
图 93-10	`fig_093_10_oncall`	泳道图	手工布局

五张流程与架构图的图元只用了 `process`、`decision`、`data_io`、`swimlane`、`arrow`、`elbow`、`label`，没有传递函数方块，符合第 86 到 93 章的约束。

图 93-1 用 `elbow` 画了一条虚线回环，表示变更从运维回到需求冻结。回环是这张图的关键，没有它，六阶段看起来像单向流水线。

图 93-4 把四段时间标在箭头外侧而不是箭头上。标在箭头上会压住方块，这是排版问题，不是内容问题，但读者第一眼就会觉得图乱。

图 93-5 与图 93-9 用了双轴。双轴图的通病是两条线的交点没有物理意义，所以两张图都在图注里写清了左右轴各自的含义。图 93-9 的纵轴用对数，因为缺货概率跨两个数量级，线性轴上后三点会挤在一起。

本章的图号与文件序号是一一对应的，不像第 91 章那样错位。这是因为出图顺序按图号排好了，`main()` 里依次调用十个函数。

93.12.10 代码分三层

`ch093_sim.py` 按三层组织：常量区、纯计算函数、报告函数。

最上面是常量区。所有可调量集中在这里，改参数只改这一段，不用往下翻。命名全大写，单位写进注释，每一项后面注一句它属于正文哪一节。

中间是纯计算函数。每个函数只算一件事，返回元组列表或几个数，不打印、不画图、不读文件。纯函数的好处是可以单独调用，也方便手算对照。

最下面是 `report()`。它只负责格式化，把各函数的返回值排成九段。改输出格式不用动计算，改计算也不影响格式。

三层之间单向依赖：常量被函数读，函数被报告读。反过来调用会让代码难以测试，也容易出现两份不一致的数字。

`ch093_figs.py` 只做两件事：调 `ch093_sim` 取数，用样式助手画图。它不重新计算任何量。图里出现的 92.31%、11.34、99.875% 全部来自 `ch093_sim`，不在这里手写。

93.12.11 六个自改实验

实验一：把 `P_TRIGGER` 从 5% 改成 10%，重跑 `[3]`。1% 批次（2 台）从 9.75% 涨到 19.00%，25% 批次（50 台）从 92.31% 涨到 99.48%。缺陷越常见，小批次越有用；缺陷罕见时，小批次几乎必然漏过。

实验二：把 `FLEET` 从 200 改成 50，重跑 `[3]`。25% 批次只有 12 台，暴露概率从 92.31% 降到 45.96%。这说明“25% 拐点”依赖设备总数，50 台的系统要放到 50% 才能过 90%。

实验三：把 `ROLLBACK_FIXES` 里的三项全部去掉，重跑 `[4]`。总时间回到 35.0 min，超目标 5.0 min。反过来只留“预置回滚包”一项，是 32.0 min，还是超。三项要一起做才够。

实验四：把 `T_PER_LOOP` 从 1.2 改成 0.9，重跑 `[5]`。总工时从 64 h 降到 52 h，3 人档从 25.1 h 降到 20.4 h，工作日从 4 天变 3 天。这就是 §93.8.4 说的“改分母不加人”。

实验五：把 `ALARM_MEASURES` 的第二项从 0.40 改成 0.20，重跑 `[6]`。最终负荷从 11.34 涨到 15.12 条/h，超上限 3.12。四项措施里第二项最敏感，因为它乘在已经削过一遍的基数上。

实验六：把 `MTBF_H` 从 2000 改成 4000，重跑 `[7]` 与 `[8]`。可用度从 99.875% 升到 99.938%，年停机从 10.94 h 降到 5.47 h；但年故障率从 4.38 降到 2.19，备件拐点从 6 件降到 3 到 4 件。一个参数同时动了两段结论，这正是 §93.7.10 第三组耦合的例子。

六个实验做完，参数与结论的关系就比较清楚了。建议至少动手做前三个，因为灰度与回滚是本章最容易在评审会上被质疑的两处。

93.12.12 报告怎么读

建议按 `[1]`、`[4]`、`[3]`、`[7]`、`[6]` 的顺序读。

先读 `[1]`，它给阶段门的价值。盯住最后两行：期望成本 6.62，逃逸出口 150，相差 22.7 倍。这个倍数是整章最有力的一个数。

再读 `[4]`，它给回滚的账。35.0 与 25.0 的对比很直观，但真正要看的是四段各自的占比：决策 10 min、验证 12 min，两段合计 62.9%。

然后读 `[3]`，它给灰度的判据。注意 1% 那一行的 9.75%，它解释了很多“小批试点成功、大批翻车”的现象。

`[7]` 给可用度，重点看年停机从 17.49 h 到 10.94 h 的 6.55 h，这 6.55 h 就是运维改进的实际收益。

`[6]` 给报警，重点看最后一行 −0.66。余量薄，说明这套削减方案刚好够用，没有容错空间。

`[2]`、`[5]`、`[8]`、`[9]` 留到需要具体数字时再查。

93.12.13 正文数字怎么回查

正文里每个数都应该能在报告里找到。找不到就是引用错了，或者算法改了没同步。

回查分三步。先在表 93-14 里按内容定位段号，再在报告里找到那一行，最后在脚本里确认对应的函数与常量。

举例：正文写“26 种、占 32.1%”，查表 93-14 知道在 `[2]`，报告里对应“允许组合 26 种，占 32.1%”，函数是 `version_table()`，常量是 `COMPONENTS` 与三条规则。

再举例：正文写“年期望等待 6.93 h”，查表知道在 `[8]`，报告里对应“备件 6 件……年期望等待 6.93 h”，函数是 `spare_table()`。这一项如果对不上，多半是 `WAIT_PER_STOCKOUT_H` 被改了。

正文与报告不一致时，先怀疑正文。报告是算出来的，正文是抄过来的，抄错比算错常见。但也要看另一种情况：正文改了参数，脚本没改。这时以正文的设计意图为准，回去改脚本。

改参数后必须重跑脚本，再按报告改正文与表。只改一处，是这类书稿最常见的不一致来源。

93.12.14 运行环境与依赖

两个脚本在 Python 3.11 下测试，依赖 `numpy` 与 `matplotlib`，不需要联网，也不需要额外的数据文件。

图里的中文依赖系统字体。`plot_styles` 按宋体、黑体、雅黑的顺序查找，`flow_styles` 用同样的顺序再补一个 DejaVu。都找不到时退回默认字体，中文会变方框，但脚本不报错。

路径用相对路径，从 `book/` 目录运行。换目录运行时，输出会落到错误的位置，检查 `figs/ch093/` 是否生成即可判断。

脚本固定 `matplotlib` 后端为 `Agg`，服务器上批量出图不需要额外配置。如果要交互查看，把 `Agg` 改成默认后端即可，但导出结果可能有像素级差异。

93.12.15 三项自检

第一项，可重现。连跑两次 `ch093_sim.py`，把输出重定向到两个文件，逐行比对，应当完全一致。

第二项，段落齐全。报告应有九个 `[n]` 段头，`[1]` 段应有六行数据，`[3]` 段应有六档批次，`[8]` 段应有七档备件。

第三项，图件完整。`figs/ch093/` 下应有二十个文件，十张图各一份 SVG 与一份 PDF，且都不为零字节。

还有一项与数字无关但要人工看：打开图 93-1 与图 93-7，确认中文没有变方框、箭头没有压住文字。脚本不会报这类错，只能靠眼睛。

93.12.16 报告里没有的量

有四类量脚本算不出来，正文里也不该给出具体数值。

第一类是真实的检出率。30%、45%、60%、80%、92% 是假设值，实际取决于团队与项目。正文只把它们当算例，不当结论。

第二类是现场的单回路工时。1.2 h 来自具体项目，换项目要重新标定，脚本给的是量级。

第三类是真实的 MTBF。2000 h 是典型值，厂商给的数字通常偏乐观一个数量级，实际要用自己的故障记录估。

第四类是文档齐套率这类管理指标。它不服从任何公式，只能逐项核对。

这四类量在正文里只给方法，不给“保证值”。看到别人给出这类数值时，先问一句“这个数是哪来的”，答不出就是编的。

93.12.17 改一次参数的完整流程

改参数看着简单，走完整流程才不留隐患。流程分五步。

第一步查依赖。打开 §93.7.10 的三组耦合，确认要改的参数牵动哪几个。改 `MTBF_H` 会同时动可用度与备件两段，这两个都要重算。

第二步改常量。只改文件头部，不散落在函数里。改完立刻重跑 `ch093_sim.py`，看九段报告有没有异常值：概率超过 100%、工时为负、可用度大于 100%，都是配错了。

第三步对正文。按表 93-14 找到引用该参数的段落与表格，逐处更新。数字要抄报告，不要手算。

第四步改图。重跑 `ch093_figs.py`，打开受影响的图看一眼，确认坐标范围与标注还对得上。改了 `FLEET` 之后，图 93-3 的横轴刻度可能要跟着改。

第五步记台账。在表 93-11 里加一行，写日期、改什么、改前、改后、影响哪几段、结论。这一步最容易被跳过，也最影响后面的人。

五步走完，一次改动才算收口。跳过第三步，正文与报告会打架；跳过第五步，三个月后没人说得清这个数为什么是这个值。

93.12.18 把这一章用起来

读完之后要落地，建议只做三件事，不必全套照搬。

第一件，算一遍自己项目的四个数：允许组合数、逃逸率、回滚分钟数、可用度。四个数都能在半小时内估出来，估出来就知道自己最缺哪一块。

第二件，把回滚包预置好。这件事不用等方案批准，也不需要额外预算，只要一个本地盘和一份校验过的备份。它是最便宜的改进，也是最容易被跳过的一件。

第三件，把验收五项的取证方式写进合同附件。写清数据从哪个系统取、谁来取、取哪一段时间的。这一条能省掉验收会上大半的争执。

三件事做完，本章的方法就算用上了。剩下的阶段门、灰度批次、备件账，可以按项目规模逐步补。

如果只记住一句话，记住 §93.11.12 的那句总结：能退回去的系统，才敢往前跑。

第94章 运动控制概述与系统组成

94.1 动机与直观理解

运动控制是控制理论里“最像机器”的那一部分。它关心的不是反应釜温度慢慢爬升，而是一根轴在几十毫秒内从静止加速到每分钟三千转，再在几毫米内停准；不是阀门开度稳定在某个百分比，而是刀具沿一条曲线走出微米级轮廓。对象、指标、失效方式都与过程控制不同，因此需要单独拿出来讲。

94.1.1 一条产线上的三个动作

设想一条包装线。第一个动作是把料盒从等待位搬到灌装位，行程 5 rad，要求 2.6 s 内到位，到位后不许回弹；第二个动作是灌装头沿一个半径 50 mm 的圆轨迹匀速走一圈，把液体均匀铺开；第三个动作是封口辊与送料辊保持 2.5:1 的转速比，料膜不能被拉长也不能堆积。三个动作分别对应点位控制、连续轨迹控制和同步控制，这也是运动控制最典型的三类任务。

三类任务的指标写法完全不同。点位控制看的是“到没到、超没超、停稳没停稳”，用超调量、上升时间、调节时间描述；连续轨迹看的是“走得准不准、速度匀不匀”，用轮廓误差、速度波动描述；同步控制看的是“两台轴之间的相位差有多大”，用同步误差、相位滞后描述。用同一套阶跃指标去评价三类任务，是现场最常见的误判来源。

三套指标不能互相换算，例子很直白。本章轴的位置阶跃上升时间 0.0182 s，看起来“很快”；但拿它跑 §94.1.3 那个 20 rad/s 的圆，按 21.9 Hz 带宽算，幅值是 $1/\sqrt{1+(3.183/21.9)^2}=0.9895$，半径仍缩了 0.52 mm。上升时间描述的是“大信号从一个位置跳到另一个位置有多快”，轮廓误差描述的是“小信号在 3 Hz 附近跟得准不准”，两者对应的频段与幅值完全不同。反过来，也不能因为轮廓误差大就断定轴“慢”——它可能只是带宽不够高，而阶跃响应本身很干净。两套指标各自回答一个频段的问题，谁也替代不了谁。

指标还要写成“带窗口”的形式。“到位”不是“误差等于零”，而是“误差进入 $\pm\varepsilon$ 并保持 t_d 不出去”。精度 ε 与保持时间 t_d 必须同时给出，否则“到位时间”可以被任意缩短——只要把窗口放宽。工程上常取定位精度的 1/2 作为窗口，本章的调节时间按 2% 判据统计。

术语在本章统一如下：控制器发出的目标位置叫指令位置 θ_r，编码器读回的叫反馈位置 θ，两者之差 $e=\theta_r-\theta$ 叫跟随误差。编码器装在电机上、负载靠传动链带动的结构叫半闭环；编码器装在负载端的叫全闭环。本章除特别说明，默认半闭环。

本章沿用一台具体的伺服轴作为统一算例。电机转子惯量 $1.2\times10^{-4}\ \mathrm{kg\cdot m^2}$，负载折算惯量为转子的 3 倍，转矩常数 $0.105\ \mathrm{N\cdot m/A}$，电枢电阻 $0.5\ \Omega$、电感 $1.2\ \mathrm{mH}$，传动轴刚度 $120\ \mathrm{N\cdot m/rad}$。这些参数在 §94.12 的脚本里都能改，本章所有数字都由该脚本一次跑出。

单位先说清楚，否则后面的数会对不上。本章一律用弧度而不是毫米：转角 θ、角速度 ω、角加速度 α 各自独立，避免了“导程、减速比、半径”三套换算反复出现。折算到直线只要乘一个杠杆：丝杠导程 10 mm 时，1 rad 对应 $10/2\pi=1.592$ mm，于是 1 mrad 对应 1.59 μm，20 mrad 的跟随误差对应 31.8 μm。反过来，若指标写“定位精度 5 μm”，折成角度就是 3.14 mrad，这个数可以直接与 §94.1.3 的 20 mrad 比较——前者是精度要求，后者是纯反馈的固有误差，两者差 6 倍，说明必须上前馈。养成“先把指标折成 rad”的习惯，后面所有公式都不必再带传动比。

图 94-1 给出运动控制系统的组成。与过程控制的单回路相比，它多了两层：控制器与驱动器之间的分工，以及电机、传动、负载之间的机械链路。控制器负责轨迹规划与外环，驱动器负责速度环与电流环，编码器把机械侧的真实位置送回控制器。信息流与能量流方向相反，这是运动控制系统布线、接地与故障排查的基本出发点。

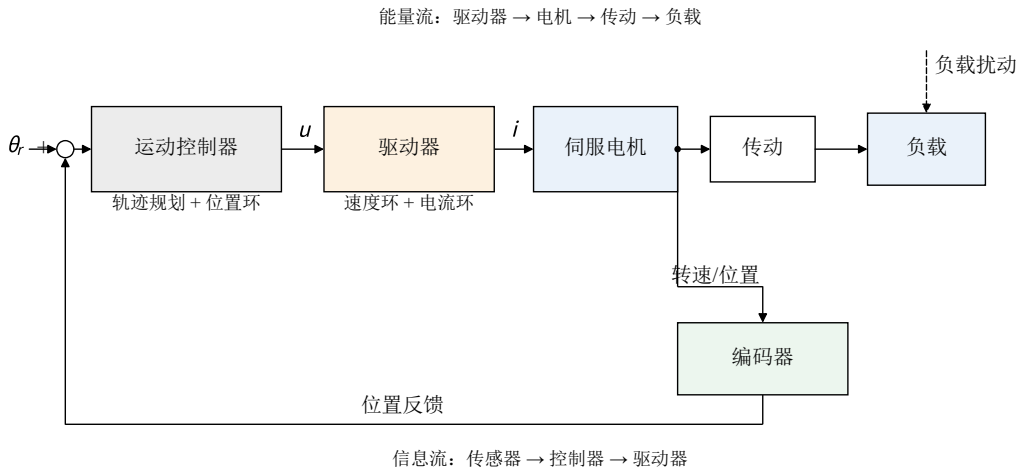


图 94-1　运动控制系统的典型组成

94.1.2 与过程控制的分工

过程控制的对象时间常数通常在秒到小时量级，执行器是阀门，测量是温度、压力、流量，控制周期常取 0.1–1 s。运动控制的对象时间常数在毫秒量级，执行器是电机与功率变换器，测量是位置、速度、电流，电流环周期常取 50–100 μs，位置环 0.5–2 ms。两者相差三到四个数量级。

时间尺度不同带来三个直接后果。第一，运动控制必须考虑机械谐振。轴的刚度有限，电机与负载之间存在一个 100–500 Hz 的谐振点，控制器带宽一旦接近它，相位会骤降。第二，运动控制必须考虑采样与计算延迟。1 ms 的延迟在 20 Hz 的位置环里就是 7.2° 的相位损失，不能忽略。第三，运动控制必须处理强非线性。摩擦力、齿隙、饱和在微米级定位里都不是小量。

两个领域的差别可以并成表 94-1。

项目	过程控制	运动控制
对象时间常数	秒到小时	毫秒
执行器	阀门、泵、加热器	电机与功率变换器
测量量	温度、压力、流量	位置、速度、电流
控制周期	0.1–1 s	50 μs–2 ms
主要扰动	负荷波动、环境温度	摩擦、齿隙、负载惯量变化
典型失效	超温、超压、产品质量漂移	飞车、撞机、谐振啸叫
核心指标	稳态误差、调节时间	跟随误差、轮廓误差、同步误差

表 94-1 里最容易被低估的是“典型失效”那一行。过程控制失控，后果通常表现为产品不合格，有缓冲时间；运动控制失控，几百毫秒内就是撞机。这个差别决定了运动控制的安全设计必须独立于控制算法：限位、急停、超程保护是硬件职责，不能指望控制器“算得对”。

安全距离可以按运动学估。以最大加速度 a 从速度 v 急停，滑行距离是 $v^2/(2a)$。本章轴 $v = 2$ rad/s、$a = 28.3$ rad/s²，得 0.0707 rad，折成直线约 0.11 mm；若速度翻倍到 4 rad/s，距离变成 0.283 rad，是原来的四倍。这条平方关系说明：高速轴的限位余量不能按“低速时的经验”放大一点，而要按速度平方放大。更重要的是，电气急停依赖驱动器仍在工作；一旦驱动器失电，只剩机械制动与限位挡块，所以机械限位必须按“电气失效”的工况设计，而不是按正常减速距离设计。

因此，过程控制里“把 PID 整到不振荡”的经验，在运动控制里往往不够用。运动控制的参数不是三个，而是一组相互约束的带宽：电流环、速度环、位置环，再加上前馈、滤波与机械参数。本章的任务，是把这组带宽的分配关系讲清楚。

94.1.3 三类任务与三个数字

先看点位运动。按七段 S 曲线规划，行程 5 rad、最大速度 2 rad/s、加加速度 400 rad/s³，峰值加速度为 28.3 rad/s²，总时长 2.64 s。纯位置反馈下，匀速段跟踪误差恰为 $v/K_{p\theta} = 2/100 = 20$ mrad；速度前馈把它压到 0.1245 mrad，再加速度前馈压到 0.0583 mrad。这三个数字的对比，就是本章后续讨论前馈的起点。

这几个数可以手算核对。加加速度恒定时，速度从 0 升到 2 rad/s 需要 $t_1 = \sqrt{v/j} = \sqrt{2/400} = 0.0707$ s，加速度在这一刻恰好达到峰值 $jt_1 = 28.3\ \mathrm{rad/s^2}$。这一小段的位移是 $jt_1^3/6 = 0.0236$ rad。紧接着加加速度反号、加速度回落，再经 0.0707 s 降为零，此刻速度正好 2 rad/s，位移 0.1178 rad。两段合起来 0.1414 s、0.1414 rad，等于平均速度 1 rad/s 乘时间。

减速段与加速段对称，两端合计 0.2828 rad。剩下 $5 - 0.2828 = 4.7172$ rad 以 2 rad/s 匀速走完，需 2.3586 s。总时长 $2.3586 + 0.2828 = 2.641$ s，与实测 2.64 s 对上。

手算的价值在看参数关系：加加速度越大，t_1 越短，匀速段越长，总时间越接近 d/v；加加速度越小，两端越“软”，总时间越长。$j \to \infty$ 时退化为梯形速度，总时间变成 $d/v + v/a$。所以“2.6 s 到位”这个指标，实际上约束的是 v 与 j 的组合，不是单约束某一个。

再看连续轨迹。半径 50 mm 的圆，进给角速度 20 rad/s，两轴闭环带宽均为 10 Hz。一阶跟随的幅值衰减使实际半径缩到 47.645 mm，径向误差峰值 2.355 mm。注意这个误差不是“控制器不够快”这么简单：两轴幅值一致时轨迹仍是圆，只是变小；两轴增益不一致时，圆会变成椭圆，误差随角度变化。轮廓误差的成因与控制结构直接相关。

缩小比例可以精确算出。一阶闭环在频率 f 处的幅值是 $1/\sqrt{1 + (f/f_c)^2}$。进给 20 rad/s 对应 3.183 Hz，除以 10 Hz 得 0.3183，代入得 0.9529，半径缩为 $50 \times 0.9529 = 47.65$ mm，误差 $50 \times (1 - 0.9529) = 2.355$ mm。一个式子把带宽、进给速度、轮廓误差三者的关系锁定：误差近似正比于 $(f/f_c)^2/2$，带宽翻倍，误差降到四分之一。

相位滞后 $-\arctan(0.3183) = -17.7^\circ$ 在两轴相同时不改变半径，只让整条轨迹转了一个角度。这一点很关键：轮廓误差的根源不是“有滞后”，而是“两轴的滞后不一样”。两轴完全相同，轨迹仍是圆只是小一圈；两轴差 10%，圆才被拉成椭圆，误差随角度周期性起伏。

最后看同步。主轴按梯形速度升到 2 rad/s，从轴以 2.5:1 跟随，从轴一阶时间常数 10 ms。匀速段的稳态同步误差为 $2.5 \times 2 \times 0.01 = 50$ mrad，停车后回落到 -1.1875 mrad。同步误差与速度成正比、与跟随时间常数成正比，降低速度或提高从轴带宽都能减小它，但前者牺牲节拍，后者受谐振限制。

50 mrad 这个数也能拆开看。从轴的参考速度是 $2.5 \times 2 = 5$ rad/s，一阶环节跟踪斜坡的稳态滞后是 τ 乘以斜率，即 $0.010 \times 5 = 0.050$ rad。一般形式是 $\Delta\theta_{sync} = \tau R \omega_m$，三个因子各自独立：时间常数、速比、主轴转速。

要减小同步误差，只能动其中一个——降速、减速比、或提高从轴带宽。停车后误差回落到 -1.1875 mrad，是因为主轴速度降到零，斜坡跟踪的稳态滞后消失，只剩一阶环节的指数衰减尾巴。这个小尾巴说明同步误差有两部分：稳态部分随速度走，瞬态部分随加减速走，两者要分别验收。

图 94-3 把三类任务画在同一页上。上左是 S 曲线的位置与速度，上右是圆轮廓的参考与实际，下图是电子齿轮的主从位置与同步误差。三张图的纵轴单位不同，但读法一致：先看参考与实际是否重合，再看误差曲线的形状，最后判断误差是稳态的还是瞬态的。

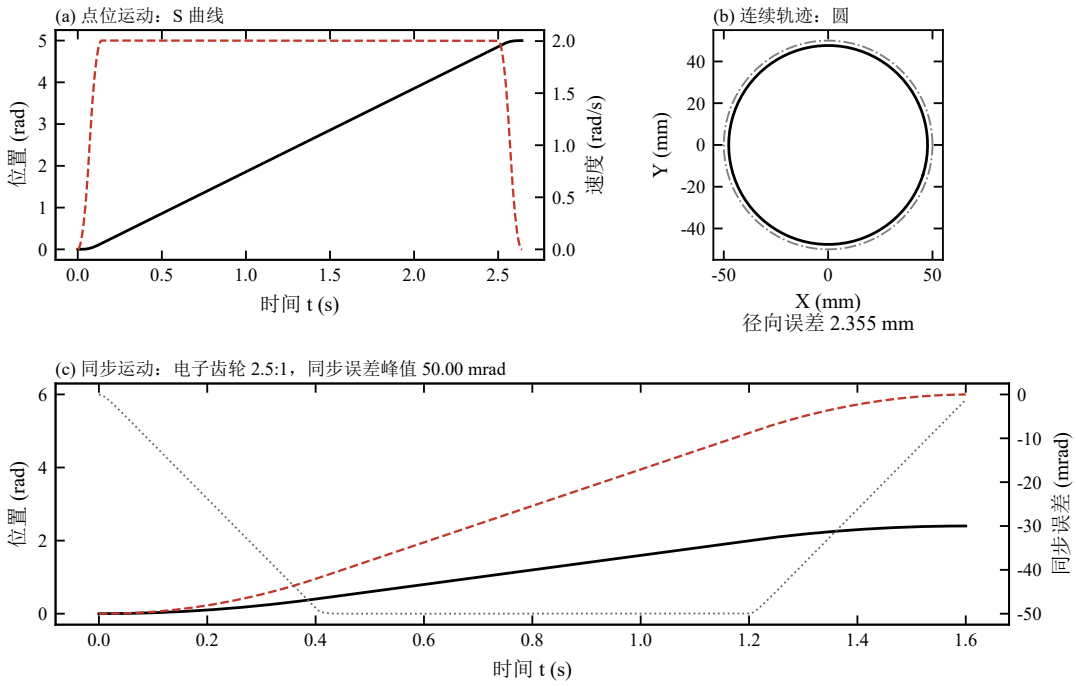


图 94-3　三类典型运动任务

94.1.4 一句话核心与本章路线

如果只能记住一句话：**运动控制的核心不是把位置误差调到零，而是把电流、速度、位置三个尺度的带宽按十倍左右分开，再用前馈补上“已知的未来”、用反馈兜住“未知的现在”。**

本章按“组成—原理—设计—连续—离散—稳定—参数—整定—改进—落地—边界—代码”推进。§94.2 给出电机、传动与三环的数学描述，重点是电流环的零极点对消与双惯量谐振；§94.3 把选型与设计固化成七步流程；§94.4 用连续域算例给出三环带宽与阶跃指标；§94.5 讨论采样、量化与计算延迟；§94.6 用裕度、惯量比与谐振峰值量化稳定边界；§94.7 逐参数扫描，说明每个旋钮的影响方向与量级；§94.8 给出从电流环到位置环的整定顺序；§94.9 讨论前馈、陷波与观测器；§94.10 落到选型、布线、调试与验收；§94.11 说明适用边界；§94.12 给出可复现代码。

不同读者可以走不同路径：做机械与选型的，读 §94.1、§94.3、§94.7、§94.10；做驱动器与算法开发的，读 §94.2、§94.4–§94.6、§94.9；做现场调试的，直接看 §94.8 与 §94.10。三条路径都建议先跑一遍 §94.12 的脚本，本章所有数字都出自它。

94.1.5 三个常见误解

误解一：位置环带宽越高越好。 位置环带宽受限于速度环带宽与机械谐振。把位置增益从 100 提到 200，带宽从 21.9 Hz 升到 60.6 Hz，相位裕度从 83.33° 降到 65.16°，超调从 0 升到 2.26%。再往上提，首先撞上的是 184.9 Hz 的谐振，而不是代码里的限幅。

误解二：电流环只是“内环”，可以随便设。 电流环决定了转矩响应速度，也决定了速度环能设计到多快。电流环带宽从 1000 Hz 降到 500 Hz，上升时间从 0.350 ms 变成 0.699 ms，速度环的相位裕度随之下降。更严重的是，电流环若与 PWM 开关频率太接近，采样混叠会让整个环路出现低频振荡。

误解三：前馈可以替代反馈。 前馈只对“模型已知、轨迹已知”的部分有效。负载突变、摩擦变化、齿隙这些未知量，前馈一分钱都补不了。本章算例里，加速度前馈把跟踪误差压到 0.0583 mrad，但 0.05 N·m 的负载阶跃仍会造成 0.8577 mrad 的瞬时挠曲，恢复时间 60.8 ms。前馈与反馈是分工，不是替代。

三个误解指向同一件事：运动控制的性能由“带宽分配、机械特性、扰动抑制”三者共同决定，任何单一参数的极限都不是系统极限。本章后续所有公式与表格，都是为了让这三者的关系变得可计算。如果只想记住一句话，那就是：先问

“哪个环先到极限”，再问“这个极限由谁定”；前者决定往哪个方向调，后者决定要不要动机械。

94.2 数学原理

本节把“三环”从口头说法变成可计算的传递函数。顺序是：先写电机与机械的基本方程，再设计电流环，然后把速度环与位置环串起来，最后补上双惯量谐振与前馈。每一步都给出本章算例的具体数值，便于读者对照。

94.2.1 电气方程与机械方程

忽略直轴与交轴的差别，把电机等效为一台他励直流电机，电枢回路满足

$$v(t) = R\,i(t) + L\frac{\mathrm{d}i(t)}{\mathrm{d}t} + K_e\omega(t) \tag{94-1}$$

其中 v 为电枢电压，i 为电枢电流，R、L 为电枢电阻与电感，K_e 为反电动势常数。机械侧满足

$$J\frac{\mathrm{d}\omega(t)}{\mathrm{d}t} = K_t i(t) - B\omega(t) - T_d(t) \tag{94-2}$$

J 为折算到电机轴的总惯量，B 为粘性阻尼，K_t 为转矩常数，T_d 为负载转矩。转角由

$$\frac{\mathrm{d}\theta(t)}{\mathrm{d}t} = \omega(t) \tag{94-3}$$

给出。三式合起来是一个三阶系统：电气一阶、机械二阶。电流环把电气的一阶压成快速跟随，速度环再在这个基础上配置机械极点，位置环最后加一个积分器。

K_t 与 K_e 在国际单位制下数值相等，本章都取 0.105。这不是巧合：两者来自同一套磁场与绕组几何，功率守恒要求 $K_t i\,\omega = K_e\omega\,i$ 对任意工作点成立。工程上常见的坑是单位混用——若样本给的是 V/(1000 r/min)，换算成 V·s/rad 要乘 $60/(2\pi\times 1000) = 0.00955$，漏乘会让前馈系数差两个数量级，表现为“前馈一加就过冲”。

把式 (94-1) 与式 (94-2) 拉氏变换，得到两个基本传递函数：

$$\frac{i(s)}{v(s)} = \frac{1}{Ls + R}, \qquad \frac{\omega(s)}{i(s)} = \frac{K_t}{Js + B} \tag{94-4}$$

本章参数下，电气时间常数为 $L/R = 1.2\times10^{-3}/0.5 = 2.4$ ms，对应 66.3 Hz；机械时间常数为 $J/B = 4.8\times10^{-4}/2\times10^{-4} = 2.4$ s。两者相差一千倍，这正是“电流环可以单独设计”的物理依据。若电机电感很大或惯量很小，这个分离不再成立，三环必须联合设计。

要注意 (94-4) 的第二个式子是“电流源驱动”下的结果，前提是电流环已经把电流控住。若直接给电枢加电压、没有电流环，反电动势会形成一条内反馈，传递函数变成 $\omega(s)/v(s) = K_t/[(Ls+R)(Js+B) + K_tK_e]$。分母常数项从 RB 变成 $RB + K_tK_e$：本章 $RB = 10^{-4}$，$K_tK_e = 0.011$，后者大一百倍。等效粘性阻尼变成 $B_{eq} = B + K_tK_e/R = 0.0223$，机械极点从 $B/J = 0.417$ rad/s 推到 $B_{eq}/J = 46.3$ rad/s。

这个对比说明一件事：反电动势本身就是很强的阻尼。加了电流环之后，这条阻尼被“藏”进内环——电流环强行维持电流，反电动势不再改变电流，电机对外表现为一个纯转矩源。所以三环设计里用 $K_t/(Js+B)$ 是对的，但那是因为有电流环，不是因为反电动势不存在。一旦电流环退出饱和边界、或者带宽降到几十赫兹，这个近似就失效。

三条微分方程直接对应仿真的状态更新：$\dot\theta = \omega$，$\dot\omega = (K_t i - B\omega - T_d)/J$，$\dot i = (v - Ri - K_e\omega)/L$。§94.12 的时域循环就是按这三行写的。用状态方程而不是传递函数，是因为限幅、摩擦、齿隙这些非线性只能在状态方程里加，传递函数里加不进去。

用直流电机模型描述交流伺服，需要一句交代。永磁同步电机在 $i_d = 0$ 控制下，q 轴电流环与直流电机的电枢回路同构：$v_q = Ri_q + L_q\,\mathrm{d}i_q/\mathrm{d}t + K_e\omega$，转矩 $T = 1.5p\lambda i_q$，其中 p 是极对数、λ 是永磁磁链。本章 $K_t = 0.105$ 对

应 $p=4$、$\lambda=0.0175$ Wb，代入 $1.5\times4\times0.0175=0.105$ 正好。所以在“已做 $i_d=0$ 解耦”的前提下，本章全部公式对交流伺服成立。差别在高速段：d、q 轴之间有 $\omega L i$ 的交叉耦合，转速越高耦合越强，靠电流环积分压不住，需要按转速前馈解耦——这与 §94.2.2 末尾那条“反电动势解耦”是同一类做法，只是一个在 d 轴、一个在 q 轴。

94.2.2 电流环：零极点对消

电流环采用 PI 控制器：

$$C_i(s)=K_{pi}+\frac{K_{ii}}{s} \tag{94-5}$$

与电枢传递函数串联后，开环为

$$L_i(s)=\frac{K_{pi}s+K_{ii}}{s}\cdot\frac{1}{Ls+R}=\frac{K_{pi}\left(s+K_{ii}/K_{pi}\right)}{s\,L\left(s+R/L\right)} \tag{94-6}$$

把控制器的零点放在电枢极点处，即令

$$\frac{K_{ii}}{K_{pi}}=\frac{R}{L} \tag{94-7}$$

则式 (94-6) 中的一阶因子完全对消，开环退化为

$$L_i(s)=\frac{K_{pi}}{L\,s} \tag{94-8}$$

闭环成为一阶系统：

$$T_i(s)=\frac{L_i(s)}{1+L_i(s)}=\frac{\omega_{ci}}{s+\omega_{ci}},\qquad \omega_{ci}=\frac{K_{pi}}{L} \tag{94-9}$$

于是设计公式只有两条：

$$K_{pi}=L\,\omega_{ci},\qquad K_{ii}=R\,\omega_{ci} \tag{94-10}$$

本章取 $\omega_{ci}=2\pi\times1000\ \mathrm{rad/s}$，代入 $L=1.2\times10^{-3}$ H、$R=0.5\ \Omega$，得 $K_{pi}=7.540$ V/A、$K_{ii}=3141.59$ V/(A·s)。实测闭环带宽 1000.7 Hz，上升时间 0.350 ms，超调 0.00%。超调之所以为零，正是因为零点把电枢极点精确抵消，闭环只剩一个实数极点。

这个“零超调”是有条件的。条件一是电感与电阻在工作点附近近似常数；条件二是反电动势在电流环带宽内可以被当作扰动，由积分器抑制；条件三是 PWM 与采样不引入额外相位。实际电机饱和时电感下降，零点位置偏离，超调会重新出现。工程上通常把电流环带宽取在开关频率的 1/10 到 1/20，本章的 1 kHz 对应 10–20 kHz 的开关频率，留有足够余量。

零极点对消对参数误差其实不敏感，这一点常被低估。设电感实际比铭牌大 43%（1.71 mH），控制器零点仍在 $R/L_{nom}=416.7$ rad/s，对象极点却移到 $R/L=292.4$ rad/s。穿越频率变成 $K_{pi}/L=7.540/1.71\times10^{-3}=4400$ rad/s 即 700 Hz，在该频率上零点与极点各自贡献的相位是 $\arctan(4400/416.7)=84.6^\circ$ 与 $\arctan(4400/292.4)=86.2^\circ$，未抵消的残差只有 1.6°。原因是零点与极点都远在穿越频率之下，比值一大，两个 $\arctan$ 都趋近 90°，差值自然很小。所以“电感铭牌不准”不会让电流环失稳，只会让实际带宽偏离设计值——真正需要担心的是带宽偏离后，速度环看到的内环特性变了。

电流环用 PI 而不用 PID，理由很直接：电枢是一阶对象，PI 的零点已经能把它的极点对消，再加微分只会放大电流采样噪声。电流采样发生在 PWM 开关瞬间，噪声幅值常常是额定电流的百分之几，微分会把它放大到占空比饱和，表现为随机丢波。若确实需要更快的电流响应，正确做法是提高开关频率、从而提高可用带宽，而不是加微分。

还有一条工程上的“隐形修正”：反电动势解耦。电流环带宽 1 kHz 时，1000 rad/s 的转速对应 105 V 反电动势，这个量在电流环看来是扰动，积分器要花时间才能压住。高性能驱动器会把 $K_e\omega$ 作为前馈直接加到电压指令上，让电流环只处理真正的误差。这一步不改变传递函数，但能显著改善高速段的电流跟踪。

94.2.3 速度环与位置环的串联

速度环仍用 PI：

$$C_\omega(s) = K_{p\omega}\left(1 + \frac{1}{T_{i\omega} s}\right) \tag{94-11}$$

速度开环为

$$L_\omega(s) = C_\omega(s)\, T_i(s)\, \frac{K_t}{J s + B} \tag{94-12}$$

速度闭环为 $T_\omega = L_\omega/(1+L_\omega)$。位置环采用纯比例：

$$L_\theta(s) = K_{p\theta}\, \frac{T_\omega(s)}{s} \tag{94-13}$$

位置闭环为 $T_\theta = L_\theta/(1+L_\theta)$。本章取 $K_{p\omega} = 2.2$ A·s/rad、$T_{i\omega} = 0.008$ s、$K_{p\theta} = 100$ s^{-1}，得到速度环带宽 103.2 Hz、相位裕度 71.37°，位置环带宽 21.9 Hz、相位裕度 83.33°、增益裕度 33.21 dB。

三个带宽的比值是 $1000.7 : 103.2 : 21.9 \approx 45.7 : 4.7 : 1$。相邻两环相差约 5–10 倍，这是级联控制的基本要求：内环必须比外环快得多，外环设计时才能把内环近似为 1。若速度环只比位置环快 2 倍，位置环的相位裕度会被内环的相位滞后吃掉，阶跃响应出现明显超调。

三个裕度里有两个是“无穷大”，值得解释。电流环开环是 ω_{ci}/s，相位恒为 -90°，永远到不了 -180°，所以增益裕度无穷；速度环的相位随频率单调趋近 -180° 却始终不相交，等它足够接近时幅值已经衰减到极小，同样没有有限频率的穿越点。只有位置环因为外面又套了一个积分器，相位最终会过 -180°，实测增益裕度 33.21 dB，即增益可以乘 $10^{33.21/20} = 45.8$ 倍才失稳。这个 45.8 倍听起来很大，但它是在“纯刚体、无延迟”的模型上算的；把 183.8 Hz 的谐振接进来，-180° 处的幅值被抬高，45.8 倍会掉到个位数。

反过来说，裕度也不是越大越好。位置环 83.33° 的相位裕度对应几乎无超调，同时也对应“偏慢”：一阶近似的调节时间 $3.9/\omega_c$ 已经是最优，再大就只是把带宽换成了阻尼。工程上把 45°–60° 视为“有超调但快”、60°–80° 视为“折中”、80° 以上视为“保守”。本章取在保守区，是因为后续还要叠加前馈、陷波与采样延迟，先把余量留出来。

这个“5–10 倍”不是审美要求，是相位预算。外环的相位裕度等于 90° 减去内环在穿越频率处的滞后，再减去采样与延迟的损失。若把内环近似为一阶，滞后是 $\arctan(\omega_{cp}/\omega_{cv})$：比值 10 倍对应 5.7°，5 倍对应 11.3°，2 倍对应 26.6°。本章比值 4.7 倍，按此估算应有 12° 的滞后，实测只有 6.67°——估算偏保守，因为速度闭环在 20 Hz 附近并不是纯一阶。这个偏差方向值得记住：一阶近似估相位，通常估得比实际更悲观，所以按它留余量是安全的。

纯比例位置环对匀速输入的稳态误差可以直接写出。匀速段 $\dot\theta_r = v$，积分器输出为 v/s^2，位置环的直流增益为 $K_{p\theta}$，故

$$e_{ss} = \frac{v}{K_{p\theta}} = \frac{2}{100} = 20\ \text{mrad} \tag{94-14}$$

这个式子说明：位置环的刚度由 $K_{p\theta}$ 决定，想减小匀速跟踪误差，要么提高增益，要么加前馈。提高增益会撞上谐振与相位裕度，前馈则不受这个限制——这是 §94.9 展开前馈的直接原因。

既然纯 P 有 20 mrad 的稳态误差，为什么不用 PI？三个理由。

第一，位置环的积分与速度环的积分重复。速度环已经是 I 型，对恒转矩扰动无稳态误差；位置环再加积分，等于在已经很硬的系统上再叠一层积分，低频段多掉 90° 相位，超调与慢振荡随之而来。

第二，位置环的积分会与机械谐振耦合。积分器在低频增益很高，一旦扰动把轴推离目标，积分持续加力，力一大就激发谐振，表现为“低频摆动叠着高频啸叫”——这种波形在现场很典型，也很难调。

第三，匀速误差有更好的解法。加积分只是把误差“慢慢磨掉”，前馈是“一开始就不产生误差”。前者拿稳定性换精度，后者只要轨迹已知就不付代价。

所以工业伺服的标准配置是“速度环 PI + 位置环 P”。少数场合会用位置环 I，典型是垂直重力轴：恒定重力矩必须靠积分才能消掉，此时要同时加重力前馈，让积分只负责残差，否则每次上下运动都会先沉一下再爬回来。

94.3 设计步骤

设计一台伺服轴，从来不是“先写控制器、再想对象”。指标在机械侧定，约束也在机械侧出，控制器只是把已经定好的余量兑现出来。本节把这件事固化成七步，每一步都有明确的输出物和判据；七步走完，三环参数只剩查表代入。

94.3.1 从指标倒着算

给三个指标就够了：到位时间、定位精度、负载质量与外形。要算的第一个量是加速度。

点位运动的行程 d、最大速度 v、加加速度 j 一旦确定，峰值加速度就定了。本章的 5 rad、2 rad/s、400 rad/s³ 给出 $a_{pk}=28.3\ \mathrm{rad/s^2}$。如果指标只写“2.6 s 内走完 5 rad”，那就有无穷多组 (v,j) 满足，选哪一组取决于电机能力与机械允许的冲击：j 取大，节拍短但冲击大；j 取小，冲击小但匀速段被压缩。

峰值转矩由惯量与加速度决定：

$$T_{pk}=J_{tot}\,\alpha_{pk}+B\,\omega_{pk}+T_f \tag{94-15}$$

本章 $J_{tot}=J_M(1+3)=4.8\times10^{-4}\ \mathrm{kg\cdot m^2}$，$\alpha_{pk}=28.3\ \mathrm{rad/s^2}$，$B=2\times10^{-4}$，暂忽略摩擦，得 $T_{pk}=0.0136+0.0004=0.0140\ \mathrm{N\cdot m}$，折成电流 $0.0140/0.105=0.134$ A。

这个数字小得让人意外，但它是对的：小惯量轴配小转矩电机，转矩与惯量同比例缩放。真正卡住选型的往往不是稳态转矩，而是另外三件事——惯量比、谐振频率、驱动器能提供的电流带宽。所以第二步、第四步不能跳。

峰值转矩只回答“能不能加速”，连续转矩回答“会不会过热”。按本章的 S 曲线循环，加速段 0.1414 s 用 0.0138 N·m，匀速段 2.3586 s 只用 0.0004 N·m，减速段 0.1414 s 用 0.0136 N·m。按 $T_{rms}=\sqrt{\sum T_k^2 t_k/\sum t_k}$ 算，得 $\sqrt{(0.0138^2\times0.1414\times2+0.0004^2\times2.3586)/2.6414}=0.0045\ \mathrm{N\cdot m}$。

峰值 0.0140、均方根 0.0045，相差 3.1 倍。电机选型时连续转矩只要大于 0.0045（再留 2 倍），峰值转矩大于 0.0140 即可。很多选型失败就失败在只看峰值：电机峰值够、连续转矩不够，跑几十个循环就过热报警。

94.3.2 七步流程

第一步，写清任务与指标。 输出一张表：行程、节拍、定位精度、重复精度、负载质量与外形、允许的冲击与噪声。指标必须可测。“走得稳”不是指标，“轮廓误差小于 0.05 mm、无爬行”才是。

第二步，折算负载惯量与惯量比。 丝杠、同步带、减速器各有折算公式，统一折到电机轴。得到 J_L 后算

$$R_J=\frac{J_L}{J_M} \tag{94-16}$$

判据是 $R_J\lesssim5$。本章取 3，相位裕度 68.94°；取到 10 时降到 51.80°，超调从 0 升到 15.53%，调节时间从 0.0425 s 涨到 0.1203 s。惯量比大不一定不能跑，但位置环带宽必须下调，节拍随之变差。这也解释了为什么工程师宁愿加一根

更粗的轴、也不愿换一台大三倍的电机：惯量比是“匹配”问题，不是“功率”问题。

折算有三条常用公式。丝杠传动，负载质量 m、导程 h，折算到电机轴为 $J_L = m(h/2\pi)^2$，再加丝杠自身惯量；同步带传动，带轮半径 r，为 $J_L = mr^2$；经减速比 i 的减速器，为 $J_L' = J_L / i^2$。惯量按平方缩小，这是减速器最容易被忽略的好处——速比 10 的减速器把惯量比降到 1/100，代价是转速降到 1/10、并引入齿隙。

一台 5 mm 导程丝杠带 20 kg 负载，折算惯量只有 $20 \times (0.005/2\pi)^2 = 1.27 \times 10^{-5}$，加上丝杠自身约 1×10^{-5}，合计 2.3×10^{-5}，仅是本章电机惯量的 0.19 倍。惯量比小于 1 并不意味着“更好调”：此时电机惯量占主导，同样的负载转矩打在更小的总惯量上，速度波动反而更大；反谐振虽然很高，扰动刚度却变差。惯量比的目标是“匹配”，不是“越小越好”。

第三步，选电机与驱动器。 电机看连续转矩、峰值转矩、最高转速、转矩常数与反电动势常数是否匹配；驱动器看连续电流、峰值电流、母线电压、开关频率。校核两条：峰值转矩留 2–3 倍余量，最高转速不超过额定转速的 80%。

本章把电流限幅设为 12 A，远大于上面算出的 0.134 A，这是为了让 §94.4–§94.6 的线性结论不被饱和污染。真实选型按 2–3 倍留量即可，留 90 倍只是浪费铜和钱。读者读后续章节的阶跃曲线时请记住这一点：那些“干净”的超调量，是在限幅从未触发的条件下得到的。

一个更省事的做法是把速度也核一遍。本章轴的最高速度 2 rad/s 折合 19 r/min，属于典型的“低速大转矩”定位轴，转速远不是约束，转矩与刚度才是。这类轴选电机时容易犯的错是拿“额定功率”去配：功率是转矩乘转速，低速轴上功率很小，但转矩需求并不小，按功率选必然选小。

热校核也要看时间尺度，不能只比 T_{rms}。电机的发热时间常数通常在几分钟量级，而一个节拍只有 2.6 s，所以连续跑几十个节拍时，绕组温度按“整段时间的平均损耗”上升，T_{rms} 才是有效值；反过来，单次动作里的 0.0140 N·m 峰值只持续 0.14 s，绕组来不及升温，靠的是铜的热容。这解释了为什么样本上同时给“连续转矩”和“峰值转矩，持续 3 s”——后者不是热平衡值，而是短时过载能力。选型时把 T_{rms} 与连续转矩比、把 T_{pk} 与“峰值转矩”比，两个都比得过才算过。

第四步，核查机械链路。 传动刚度与阻尼决定谐振。双惯量模型的两个特征频率是

$$\omega_{ar} = \sqrt{\frac{K_s}{J_L}}, \qquad \omega_r = \sqrt{K_s\left(\frac{1}{J_M} + \frac{1}{J_L}\right)} \tag{94-17}$$

本章 $K_s = 120\ \mathrm{N\cdot m/rad}$，得 $\omega_{ar} = 577\ \mathrm{rad/s}$（91.9 Hz）、$\omega_r = 1155\ \mathrm{rad/s}$（183.8 Hz）。判据是：位置环带宽不超过反谐振频率的 1/3，速度环不超过 1/2。21.9 Hz 对 91.9 Hz，余量 4.2 倍，合格。

若刚度只有 $12\ \mathrm{N\cdot m/rad}$ 而其余不变，反谐振掉到 29 Hz，位置环就必须压到 10 Hz 以下，节拍直接翻倍。这就是“机械决定上限”的具体含义：控制器能让系统变好，但改不了机械定下的天花板。

第五步，定编码器与采样周期。 编码器分辨率要优于定位精度的 1/5。本章取 $10\ \mu\mathrm{rad}$，相当于 5 rad 行程的 1/500000，对微米级定位足够。换成“位”来算：单圈 $2^{19} = 524288$ 份，每份 $2\pi/2^{19} = 12.0\ \mu\mathrm{rad}$；20 位是 $6.0\ \mu\mathrm{rad}$。所以 10 μrad 介于 19 位与 20 位之间，是常见的中高端配置。采样周期按“每环至少 10 倍带宽”取：电流环 1 kHz 要求 10 kHz 以上的更新率，只能放进 PWM 中断；位置环 21.9 Hz 用 1 ms 绰绰有余，本章在 §94.5 里扫 0.25–8 ms。

编码器的接口类型也要在这一步定。增量式 A/B 差分输出受脉冲频率限制：20 位编码器转 3000 r/min 时，脉冲率是 $2^{20} \times 3000/60 = 52.4$ MHz，远超差分线能传的几 MHz，只能改串行协议（BiSS、EnDat）或降低分辨率。本章轴只有 19 r/min，20 位对应 332 kHz，增量式完全够用；但同一台驱动器如果以后要配高速轴，就必须选串行接口。分辨率、接口、最高转速三者是一组，不能分开选。

第六步，分配带宽并算参数。 三环带宽按 5–10 倍分开：电流环 1000 Hz、速度环 103 Hz、位置环 21.9 Hz，比值 $45.7 : 4.7 : 1$。图 94-2 把三个环画在同一张图上，三个虚线框标出每个环闭合的位置——电流环闭合在驱动器内部，速度环跨驱动器与电机，位置环跨控制器与编码器。

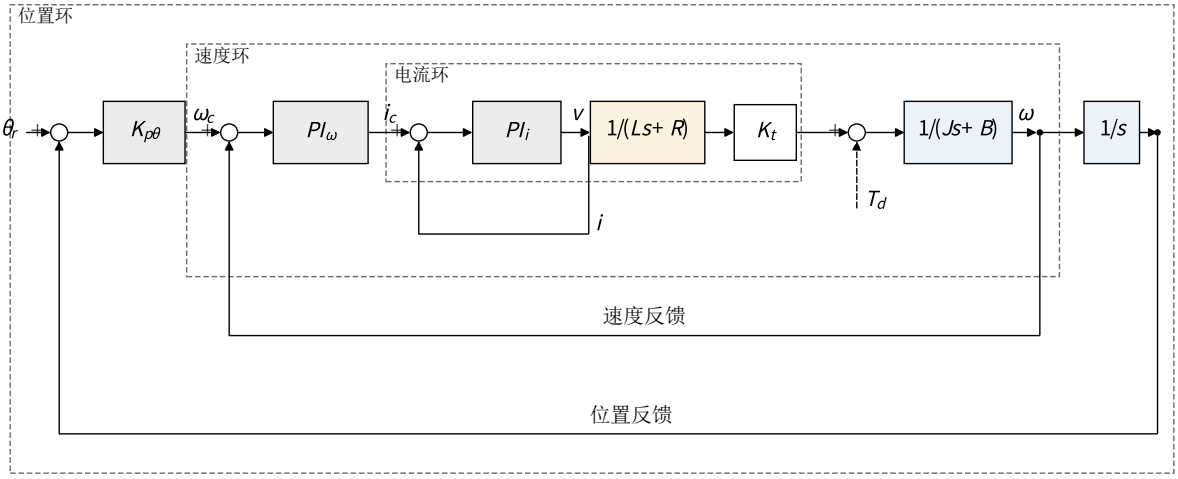


图 94-2　三环级联结构与三个环的闭合位置

带宽分配的本质，就是决定这三个框各自包住多少相位滞后。框画得越往外，包含的滞后越多，能给的增益就越低。所以设计顺序必然是自内向外：先定电流环，再定速度环，最后定位置环。反过来做，位置环的增益会被内环的未知滞后反复推翻。

参数由带宽直接算：电流环用 (94-10) 的零极点对消，得 $K_{pi} = 7.540$、$K_{ii} = 3141.59$；速度环用 PI 把穿越频率放在 103 Hz 附近，得 $K_{p\omega} = 2.2$、$T_{i\omega} = 0.008$ s；位置环用 $K_{p\theta}$ 直接给低频刚度。注意 $K_{p\theta} = 100\ \mathrm{s}^{-1}$ 并不等于 $2\pi \times 21.9 = 137.6\ \mathrm{rad/s}$。位置环的穿越频率由 $K_{p\theta}$ 与速度闭环的高频段共同决定，不能只用一个除法估；$K_{p\theta}$ 给的是“每弧度误差出多少速度指令”这个刚度，穿越点要靠频率响应读出来。

第七步，加前馈、滤波与保护。 速度前馈 K_{vf}、加速度前馈 K_{af}、陷波滤波器、电流与速度限幅、超程与过热保护。前馈系数先按 1 取，再用跟踪误差微调；陷波按反谐振频率设，深度不宜超过 20 dB，否则相位损失会把刚挣来的裕度又吃回去。本章的 0.0583 mrad 就是这一步的成果，对应 §94.9。

三个前馈系数的量纲要分清。K_{vf} 是无量纲的比例，理论上等于 1，因为位置环输出的速度指令已经与参考速度同量纲；K_{af} 有量纲，单位是 A·s²/rad，数值等于 J/K_t，本章 $4.8 \times 10^{-4}/0.105 = 4.57 \times 10^{-3}$；如果驱动器内部按“转矩前馈”接口，则直接给 J，由驱动器自己乘 $1/K_t$。量纲错了，前馈要么毫无作用，要么直接把电流打到限幅。

94.3.3 七步里有三次回头

流程写得再顺，实际设计也会回头三次。

第一次在第三步：电机选完发现惯量比超过 5，要么换大一号电机，要么加减速器。第二次在第四步：谐振频率不到位置环带宽的 3 倍，要么加粗传动、提高刚度，要么老老实实降带宽。第三次在第七步：陷波一加，相位裕度掉了十几度，只能减小深度或回去降带宽。

这三次回头解释了现场一个常见困惑——参数怎么调都不对。多数时候不是参数没调好，而是机械与选型已经把上限锁死了，控制器只能在锁死的空间里做分配。按七步顺序走一遍，能在花钱之前就发现这些锁在哪。

把判据代进一个假定算例，能看清杠杆在哪。设某台轴负载折算惯量比为 12:1，位置增益按默认值取，按表 94-5 的规律它会落在 50° 附近，定位必然带 10% 量级的过冲。两种改法：把 $K_{p\theta}$ 降三成，过冲压到 3%，代价是节拍变长；或者换 1:2 的减速比，惯量比降到 3:1，$K_{p\theta}$ 反而可以提上去，过冲与节拍同时改善。前者是第三步的钱（换电机），后者是第四步的钱（换减速器），后者通常便宜得多。这正是“回头要在对的步骤回头”的意思。

系数要算准。惯量按速比平方折算，1:2 的减速器把负载惯量降到 1/4，12:1 变成 3:1；如果误以为“速比 2 就是惯量减半”，会算成 6:1，选型仍然偏大。同时要知道减速器自己也有惯量，通常折算后相当于电机惯量的 0.1–0.3 倍；速比越大，这部分占比越高，所谓“降到 3:1”是含减速器之后的净值，不是单算负载。

94.3.4 七个检查点

把七步的判据集中成表 94-2。这张表可以直接当评审清单用：任何一项打了问号，后面的参数都不必算。

步骤	输出物	判据	越界的后果
1 任务与指标	行程、节拍、精度表	指标可测、有测试方法	验收时扯皮
2 负载折算	J_L、R_J	$R_J \lesssim 5$	位置环降带宽、超调增大
3 电机与驱动器	转矩、电流、转速校核	峰值转矩 2–3 倍余量	加速段饱和、到位慢
4 机械链路	K_s、ω_{ar}、ω_r	$f_{cp} \le \omega_{ar}/3$	谐振激发、啸叫、失稳
5 编码器与采样	分辨率、T_s	分辨率优于精度 1/5	极限环、爬行
6 带宽与参数	三环增益、带宽比	相邻环差 5–10 倍	外环相位被吃、超调
7 前馈与保护	K_{vf}、K_{af}、陷波	裕度下降不超过 10°	快而不稳

表 94-2 的判据都是“不等号”，不是“等于号”。设计不是解方程求唯一解，而是在一组不等式围出的可行域里挑一个点。挑点的依据是节拍与成本的权衡，这部分内容 §94.7 会用参数扫描给出定量的方向，§94.8 给出具体的调试顺序。

这张表的用法是按行往下问，不是按列横向比。第一行打问号，说明需求本身没定义清楚，后面六行的计算都失去意义——一个连“精度是定位精度还是重复精度”都没定的项目，编码器选 19 位还是 23 位都只是猜。第二、四行是一对：负载折算与传动刚度共同决定反谐振，前者估错一倍、后者估错一倍，反谐振就差两倍，而这两项恰恰是选型阶段最容易被“先估个数”的两项。第三行与第五行是一对：转矩余量不足表现为加速段饱和，分辨率不足表现为低速爬行，两者都在时域里看得见，却常被误诊成“参数没调好”。第六、七行才是本章后面几节要算的内容，它们排在最外面，正因为前面五行是它们的前提。

94.4 连续情形

先在连续域把三环算清楚。连续域里没有采样、没有计算延迟、没有量化，可以看清每个环“本来”能跑多快。离散带来的损失留到 §94.5 单独扣，两边的差额就是数字实现付出的代价。

94.4.1 电流环：一条 -20 dB/dec 的直线

按 (94-10) 取三组带宽，得到表 94-3 的增益与上升时间。

电流环带宽	K_{pi} (V/A)	K_{ii} (V/(A·s))	上升时间 (ms)
500 Hz	3.770	1570.80	0.699
1000 Hz	7.540	3141.59	0.350
1500 Hz	11.310	4712.39	0.233

带宽与上升时间的乘积是常数：$500 \times 0.699 \approx 1000 \times 0.350 \approx 1500 \times 0.233 \approx 350$。这个 350 不是巧合，一阶系统的 10%–90% 上升时间约为 $2.2/\omega_c$，折成赫兹就是 $2.2/(2\pi) = 0.350$ 个周期。表 94-3 的三行其实是同一个系统在三个时间尺度上的重演。

同一件事还可以从时间常数看。一阶系统 $1/(1+s/\omega_c)$ 的时间常数是 $1/\omega_c$，1000 Hz 对应 0.159 ms：阶跃后 0.159 ms 到达 63.2%，0.318 ms 到达 86.5%，0.477 ms 到达 95.0%。表 94-3 里 0.350 ms 的 10%–90% 上升时间落在 0.318 与 0.477 之间，与这条指数曲线一致。工程上读电流环波形，常用的是“指令跳变到电流到位”这段，它约等于三个时间常数，即 0.48 ms；如果实测明显长于这个数，先查滤波，再查 PWM 更新时刻，最后才怀疑增益。

三组带宽对应三种硬件条件，不能随意选。500 Hz 适合大电感、低母线电压或开关频率只有 8 kHz 的场合，此时每周期有 16 个采样点，电流纹波小、噪声裕度大；1000 Hz 是 10–16 kHz 开关频率下的常规折中；1500 Hz 要求开关频率不低于 15 kHz，且电流采样必须与 PWM 中心对齐，否则采样点落在纹波斜坡上，反馈里会混进与占空比同频的干扰。带宽不是越高越好，它首先是一个与功率级绑定的数字。

图 94-5 是这三组的闭环阶跃。三条曲线都单调上升、没有超调，只有快慢之别。零超调的来源在 §94.2 已经交代过：PI 零点精确抵消电枢极点，闭环只剩一个实数极点，一阶系统不可能超调。

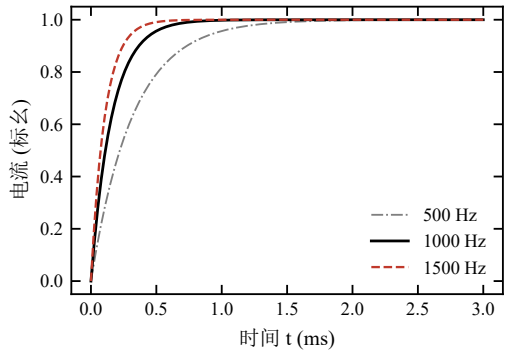


图 94-5　电流环三组带宽下的阶跃响应

把这件事换个说法：开环 $L_i(s)=K_{pi}(s+K_{ii}/K_{pi})/[s(Ls+R)]$，零点在 $-R/L=-416.7$ rad/s，极点也在 -416.7 rad/s，两者相消后只剩 $K_{pi}/(Ls)=\omega_{ci}/s$。开环幅频是一条斜率 -20 dB/dec 的直线，在 ω_{ci} 处穿过 0 dB；相位恒为 -90°，所以相位裕度正好 90.00°。

90° 的相位裕度听起来很美，代价是阻尼过大。同样带宽下，一阶系统比二阶系统慢：二阶系统在 $\zeta=0.707$ 时穿越频率与带宽之比约为 1.55，而一阶系统这个比值只有 1。想让电流环更快，只能提高带宽，而带宽又被开关频率与采样卡住——这条因果链在 §94.5 会再出现一次。

94.4.2 速度环：把穿越放在 103 Hz

速度环看到的对象是“电流闭环串上机械”。在 103 Hz 以内，T_i 的幅值接近 1、相位只掉了 $-\arctan(103.2/1000)=-5.9^\circ$，可以近似成 1；机械部分 $\frac{K_t}{Js+B}$ 的极点 $B/J=0.417$ rad/s 在低频，也可忽略。于是速度开环近似为

$$L_\omega(s)\approx\frac{K_{p\omega}K_t}{Js}\left(1+\frac{1}{T_{i\omega}s}\right) \tag{94-18}$$

这是一个带 PI 零点的积分器。穿越频率 103.2 Hz、相位裕度 71.37°。为什么不是 90°？两处滞后：T_i 贡献的 5.9°，以及穿越点附近 $(Js+B)$ 尚未完全变成纯积分所剩下的几度。两者相加约 19°，正好对上 $90-71.37$。

这个账可以逐项算。PI 在 103.2 Hz 处的相位是 $-90^\circ+\arctan(\omega T_{i\omega})$，其中 $\omega T_{i\omega}=2\pi\times103.2\times0.008=5.19$，$\arctan 5.19=79.1^\circ$，所以 PI 贡献 -10.9°；机械环节 $\frac{K_t}{Js+B}$ 在 103.2 Hz 处的相位是 $-90^\circ+\arctan(0.417/648.5)\approx-90^\circ$，但它比纯积分少掉的幅值要单独算；电流闭环再贡献 -5.9°。三项相加 -106.8°，相位裕度 73.2°，与实测 71.37° 差 1.8°，差额来自 $(Js+B)$ 中阻尼项带来的额外相位与幅值修正。手算到这里就够了：它说明裕度的八成来自 PI 零点位置，剩下的才是内环与机械的零碎。

由此可以推出一条调参直觉：PI 零点越靠近穿越频率，$\arctan$ 越接近 90°，PI 的相位贡献越接近 0°，相位裕度越高、但积分作用越弱；零点越往低频挪，$\arctan$ 变小，PI 相位越接近 -90°，裕度被吃掉。把零点放在穿越频率的 $1/5$

，$\omega T_i \approx 5$，$\arctan 5 \approx 78.7^\circ$，PI 只吃掉约 11°；放在 $1/10$ 则 $\omega T_i \approx 10$，只吃掉 5.7°，但积分增益降了一半，扰动恢复变慢。本章取 $1/5$，零点在 125 rad/s 即 19.9 Hz，是在“留 11° 余量”和“扰动恢复不太慢”之间选的；$1/5$ 到 $1/10$ 是常规区间。

速度环是 I 型系统，对恒转矩扰动没有稳态误差。这一点在 §94.6 用 0.05 N·m 的负载阶跃验证：末值回到 0，但中间有 0.8577 mrad 的挠曲，恢复用了 60.8 ms。

这里有一个容易看漏的细节：速度环闭环带宽 103.2 Hz，开环剪切频率却只有 78.8 Hz，两者差 24 Hz。正常情况下这两个数应当接近；差出来的部分，正是 §94.7.4 那个 1.17 倍隆起的贡献——闭环幅频在 29 Hz 附近先鼓起来，-3 dB 点被推到了比剪切频率更高的位置。所以“带宽”与“剪切”不一致，本身就是“闭环不平直”的旁证。读驱动器手册时，若只给一个“速度环带宽”，要问清它指的是闭环 -3 dB 还是开环穿越，两者可以差 25%。

94.4.3 位置环与阶跃指标

位置环是纯比例，带宽 21.9 Hz、相位裕度 83.33°、增益裕度 33.21 dB。单位阶跃的超调 0.00%、上升时间 0.0182 s、调节时间 0.0425 s。

零超调同样可以用“近似成一阶”解释。位置环穿越频率 21.9 Hz 只有速度环带宽的 1/4.7，在穿越点上速度闭环的幅值仍接近 1、相位只掉了几度，于是

$$L_\theta(s) = K_{p\theta}\frac{T_\omega(s)}{s} \approx \frac{K_{p\theta}}{s} \tag{94-19}$$

又是积分器，又是一阶闭环，又不超调。实测 83.33° 而非 90°，差的 6.67° 就是速度环在 21.9 Hz 处的残余滞后。这个数字很关键：它说明“外环可以把内环当 1”是一个近似，近似的误差会直接吃掉外环的相位裕度。

把 $K_{p\theta}$ 从 50 扫到 200，得到表 94-4。

$K_{p\theta}$ (s⁻¹)	位置带宽 (Hz)	相位裕度 (°)	匀速误差 (mrad)
50	8.4	89.05	40.000
100	21.9	83.33	20.000
150	44.0	73.89	13.333
200	60.6	65.16	10.000

匀速误差那一列严格服从 (94-14)：$2/50 = 40$、$2/100 = 20$、$2/150 = 13.33$、$2/200 = 10$，一位不差。增益每翻一倍，误差减半，这是纯比例位置环的硬约束，没有例外。

带宽那一列却不成正比：增益翻 4 倍（50→200），带宽涨了 7.2 倍（8.4→60.6）。原因是穿越点由两个因素共同决定：$K_{p\theta}$ 抬高整条开环幅频曲线，而速度闭环在 20–60 Hz 之间的幅值并不平直，越往高处抬，穿越点越容易落在幅值隆起的位置上，于是带宽涨得比增益快。这也提醒一件事：不能用一个系数把“增益”换算成“带宽”，必须读频率响应。

相位裕度那一列下降得比带宽上升更快：从 89.05° 一路掉到 65.16°。掉的全是速度环在更高频率处的滞后。当 $K_{p\theta} = 200$ 时，穿越点 60.6 Hz 已经接近速度环带宽 103 Hz 的一半，“内环当 1”的近似开始失效，超调也随之出现（2.26%）。再往上加，撞的就不是速度环，而是 91.9 Hz 的反谐振。

阶跃指标也可以拿来交叉验算。若位置闭环真是一阶、带宽 21.9 Hz，则 10%–90% 上升时间应为 $0.350/21.9 = 0.0160$ s，2% 调节时间应为 $3.9/(2\pi \times 21.9) = 0.0283$ s。实测分别是 0.0182 s 与 0.0425 s，前者只多 14%，后者多 50%。上升时间接近，说明穿越点附近的幅频确实像积分器；调节时间偏长，说明尾巴上还有速度环的慢极点没有衰减完。一阶近似只在穿越点附近成立，到了“误差只剩 2%”的尾巴上，起作用的已经是速度环 103 Hz 之外的那些小时间常数。

这条偏差本身就是一个诊断量。把 $\text{上升时间} \times \text{带宽}$ 算出来，理想一阶应得 0.350；如果实测明显大于这个数（比如 0.5 以上），说明外环并没有把内环当成 1，要么内环带宽不够，要么中间夹了滤波或通信延迟。它比单看带宽可靠，因为带宽只描述频域的一个点，而上升时间把整个过渡过程都压进了一个数。

94.4.4 带宽分离长什么样

图 94-6 把三个闭环的幅频画在一起，横轴是频率，纵轴是闭环幅值，虚线是 -3 dB。每条曲线与虚线交点就是该环的带宽：1000 Hz、103 Hz、21.9 Hz。

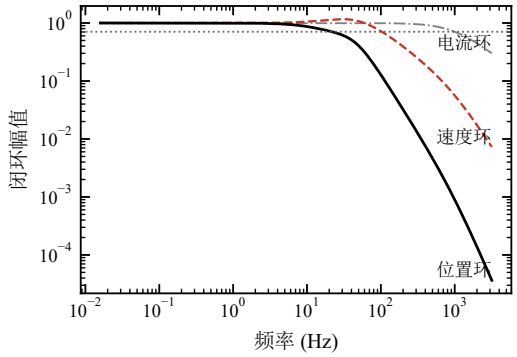


图 94-6　三个闭环的幅频与带宽分离

这张图值得看三处。

第一处是三条曲线在高频段的衰减速度。电流环以 -20 dB/dec 下降，速度环在 100 Hz 之后以 -40 dB/dec 下降，位置环再往外是 -60 dB/dec。每闭合一个环，高频衰减就快一档。外环之所以“稳”，很大程度上是因为它自带更强的低通。

第二处是三条曲线的间距。相邻两条在 -3 dB 线上的水平距离，就是带宽比：电流环与速度环差 9.7 倍，速度环与位置环差 4.7 倍。前者宽裕，后者刚好卡在下限。但间距不能单独看，还要看曲线本身是否平直。把 $K_{p\omega}$ 从 2.2 降到 1.0，位置带宽“升”到 35.90 Hz，可相位裕度同时从 83.33° 掉到 65.90°——闭环幅频在 25 Hz 处从 0.88 又顶回 0.95，-3 dB 点是这个鼓包顶出去的，不是系统真的更快。反过来把 $K_{p\omega}$ 提到 5.0，位置带宽回落到 17.53 Hz，相位裕度升到 87.65°，幅频单调下降。带宽数字与稳定程度在这里是反着走的，这是第四章“带宽不等于性能”在运动控制里的具体形态。

第三处是 30 Hz 附近速度环的隆起。实测 $|T_\omega|$ 在 30 Hz 处达到 1.17，来源是 PI 零点（19.9 Hz）与机械极点的配合。这个隆起让位置闭环在 25 Hz 附近也有一个 0.95 的小鼓包，于是位置带宽能到 21.9 Hz，而不是 $K_{p\theta}/(2\pi) = 15.9$ Hz。$K_{p\theta}$ 除以 2π 这个估法之所以不准，根子就在这里：它默认速度闭环是平的，而实际不是。隆起本身不是坏事，它换来了更快的扰动恢复；但它会吃掉外环的相位，所以位置环增益不能按“速度环带宽的一半”去猜，必须读频率响应。

隆起在现场也能测出来，不必有频响仪。给速度环一个方波指令，看速度反馈的上升沿：如果沿上有一段“先冲过头再回落”，那就是隆起在时域的形态；冲过头的比例约等于频域峰值减一，本章 17% 的超调对应 1.17 的峰值。另一个办法是给位置环一个很小的正弦指令，从 5 Hz 扫到 50 Hz，记录幅值比，峰值出现在哪里，隆起就在哪里。两种办法都只需要一台能录波的驱动器，比买频响分析仪便宜得多。

94.4.5 双惯量：控制器之外的两个频率

前面所有计算都用了刚体模型 $J_{tot} = 4.8 \times 10^{-4}$。真实轴不是刚体：电机通过一根有限刚度的轴带动负载，两者之间有相对扭转。把传动轴建成弹簧加阻尼，得到

$$G_{flex}(s) = \frac{J_L s^2 + c s + K_s}{J_M J_L s^3 + c(J_M + J_L) s^2 + K_s (J_M + J_L) s} \tag{94-20}$$

分子是一个二阶环节，它的零点就是反谐振频率；分母的二次因子给出谐振频率，正是 (94-17) 那两个式子。本章参数下，反谐振 91.9 Hz、谐振 183.8 Hz。

这两个数可以手算。负载折算惯量 $J_L = 3J_M = 3.6\times10^{-4}$，反谐振 $\omega_{ar} = \sqrt{K_s/J_L} = \sqrt{120/3.6\times10^{-4}} = 577.4$ rad/s，即 91.9 Hz；谐振 $\omega_r = \sqrt{K_s(1/J_M + 1/J_L)} = \sqrt{120\times(8333+2778)} = 1154.7$ rad/s，即 183.8 Hz。两者之比 $\omega_r/\omega_{ar} = \sqrt{1+J_L/J_M} = \sqrt{4} = 2$。这个比值只由惯量比决定，与轴刚度无关——刚度同时抬高或压低两个频率，却改不了它们的间距。所以“换一根更硬的轴”只能把共振推到更高处，不能让反谐振与谐振靠得更近。

把这条比值用到惯量比 1 与 10 上，立刻能预测各自的共振位置。惯量比 1 时 $\sqrt{2} = 1.414$，反谐振 159.2 Hz、谐振 225.1 Hz（图 94-4 标注 577 与 1155 rad/s 是对应的本章 3:1 算例）；惯量比 10 时 $\sqrt{11} = 3.32$，反谐振 50.3 Hz、谐振 166.9 Hz（实测峰位 167.2 Hz，差异来自阻尼）。惯量比从 1 涨到 10，谐振频率只从 225 Hz 降到 167 Hz，反谐振却从 159 Hz 掉到 50 Hz——这就是 §94.6 那条“惯量比越大越难控”的频域根源：谐振峰没走多远，可用的一段平坦区却被压窄了近三倍。

图 94-4 把刚体模型（灰）与双惯量模型（黑）画在一起。低频段两者完全重合——这是必然的，因为两者的总惯量相同，$s\to0$ 时都趋于 $1/(J_{tot}s)$。90 Hz 附近黑线出现一个深谷，这是反谐振：在这个频率上，负载像“被卡住”一样不动，电机端看到的惯量骤减，速度对转矩的传递被打了个洞。再往上到 184 Hz，黑线冲出一个峰，这是谐振：电机与负载反相摆动，轴被反复拧紧放松。

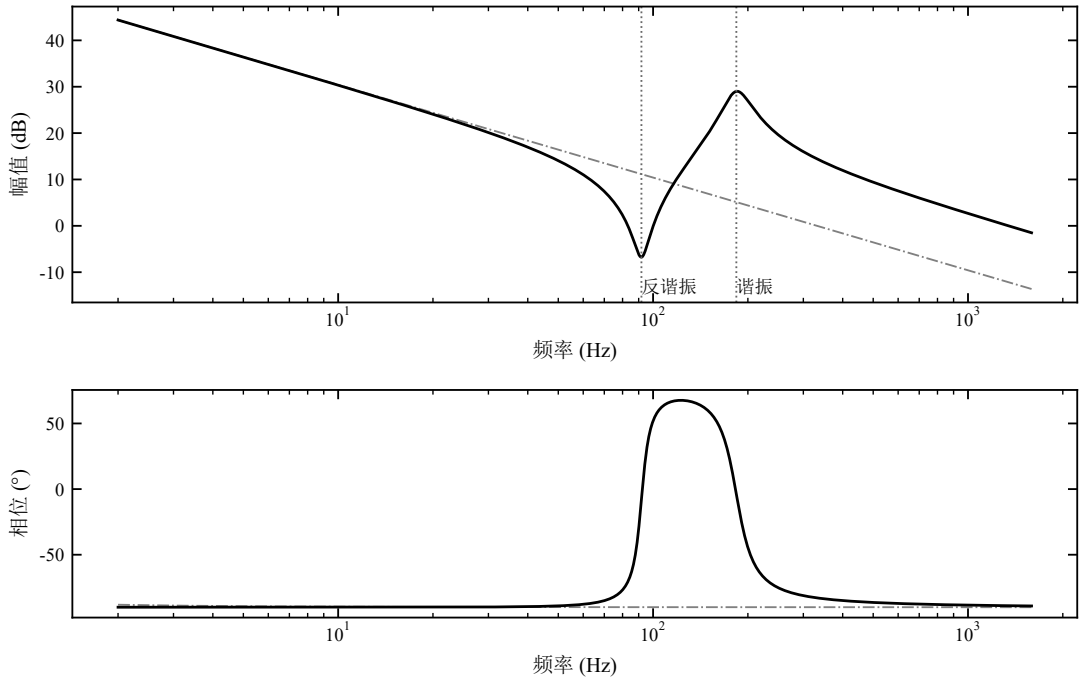


图 94-4　刚体模型与双惯量模型的幅频对比

相位图更能说明问题。刚体模型的相位恒为 -90°；双惯量模型在反谐振处先向 0° 方向顶一下，过了谐振再迅速坠向 -180° 以下。也就是说，183.8 Hz 之上，对象自己就带了 90° 的额外滞后。控制器在这一段无论怎么加增益，都只是在给一个已经转不动的方向盘上再加力。

这就解释了 §94.3 第四步那条判据：位置环带宽不超过反谐振的 1/3（21.9 对 91.9，余量 4.2 倍），速度环不超过 1/2（103 对 91.9，略微越界，靠 PI 的低通特性兜住）。判据不是保守的习惯，而是从相位曲线上量出来的安全距离。

反谐振还有一个反直觉的性质：惯量比越大，反谐振越低，谐振峰越高。本章算例中，惯量比 1、3、5、10 对应的谐振峰分别是 230.7 Hz / 7.17 dB、184.9 Hz / 11.90 dB、174.9 Hz / 13.23 dB、167.2 Hz / 14.34 dB。负载越重，能用的带宽越低，峰却越高——两个方向同时恶化，这是惯量比必须控制在 5 以内的根本原因。定量的稳定性分析放在 §94.6。

这里只留一句结论：连续域的设计把三个环的带宽定在 1000 / 103 / 21.9 Hz。位置环 21.9 Hz 只有反谐振 91.9 Hz 的 1/4.2，安全；速度环 103 Hz 略高于反谐振，靠 PI 的低通把 91.9 Hz 处的增益压下去；电流环 1000 Hz 远在反谐振之上，但那时谐振峰已被机械环节的 -40 dB/dec 衰减了二十多 dB，且电流环不经过机械反馈。三环里真正与机械共振短兵相接的，只有速度环。下面 §94.5 要问的是，把这套连续参数搬进 1 ms 的计算周期，还剩下多少。

94.5 离散情形

连续域的三组数字是“上限”。真实控制器每隔 T_s 才读一次编码器、算一次、发一次指令，中间发生的事它一概不知道。本节把连续参数搬进离散时间，逐项算清三笔损失：采样、量化、计算延迟。

94.5.1 离散化之后看什么

位置环在 1 ms 周期下运行，被控对象是“速度闭环串积分器” $T_\omega(s)/s$。用零阶保持器离散，得到 $G_d(z)$，闭环为

$$T_d(z) = \frac{K_{p\theta} G_d(z)}{1 + K_{p\theta} G_d(z)} \tag{94-21}$$

连续域看极点是否在左半平面，离散域看极点是否在单位圆内。判据是谱半径 $\rho = \max |z_i| < 1$。ρ 越接近 1，衰减越慢；越接近 0，越快“一拍到位”。注意 ρ 只回答“稳不稳”，不回答“好不好”，这一点下面会有一个反例。

采样周期该取多小，工程上有两条判据。一条来自采样定理的推广：$f_s \geq 10 f_c$，最好 20 倍。另一条来自相位：一拍延迟等价于 ωT_s 的相位滞后，即

$$\varphi_{delay} = 360^\circ \, f \, T_s \tag{94-22}$$

把两条判据套到三个环上，结论立刻分明。位置环 21.9 Hz 配 1 ms，相位损失 7.9°，可接受；速度环 103.2 Hz 配 1 ms，损失 37.1°，已经吃掉了半个相位裕度；电流环 1000 Hz 若也配 1 ms，损失 360°，等于延迟一整拍，环路必然振荡。所以电流环不能和位置环共用一个周期，它必须跑在 50–$100\ \mu s$ 的 PWM 中断里——100 µs 对应 36°，50 µs 对应 18°。

这三个数可以直接对到裕度上。位置环原本 83.33°，扣掉 7.9° 还剩 75.4°，仍然远离 45° 的红线；速度环原本 71.37°，扣掉 37.1° 只剩 34.3°，已经落到“会连续振荡两三次”的区间。这与 §94.5.4 的实测一致：1 ms 周期下速度环并不是不能跑，而是跑得“发飘”，阶跃里能看到两三下摆动。电流环的情况更极端，360° 不是“损失大”，而是“相位关系完全错位”，相当于把负反馈接成了正反馈，必然发散。

反过来推，可以算出每个环允许的最大周期。给定目标相位裕度 45°，允许的相位损失是“原裕度减 45°”：位置环可损失 38°，对应 $T_s \leq 38/(360 \times 21.9) = 4.8$ ms；速度环可损失 26°，对应 $T_s \leq 26/(360 \times 103.2) = 0.70$ ms；电流环可损失 45°，对应 $T_s \leq 45/(360 \times 1000) = 0.125$ ms。三个上限 4.8 ms / 0.70 ms / 0.125 ms，比值约 7:1:0.18，与三个带宽的倒数同阶。工程上再留一倍余量，就得到常见的 1 ms / 250 µs / 62.5 µs 这组周期——它们不是拍脑袋定的，是从裕度预算倒推出来的。

这条推理解释了驱动器的一个基本架构：电流环在功率级中断里，速度环在 125–250 µs 的任务里，位置环在 0.5–2 ms 的总线周期里。三层周期差 5–10 倍，与带宽差 5–10 倍一一对应。周期分配不是软件随便定的，它是带宽分配的镜像。

94.5.2 采样周期扫描

把位置环周期从 0.25 ms 扫到 8 ms，其余参数不变，得到图 94-7。左图是 S 曲线跟踪的误差峰值，右图是闭环谱半径。

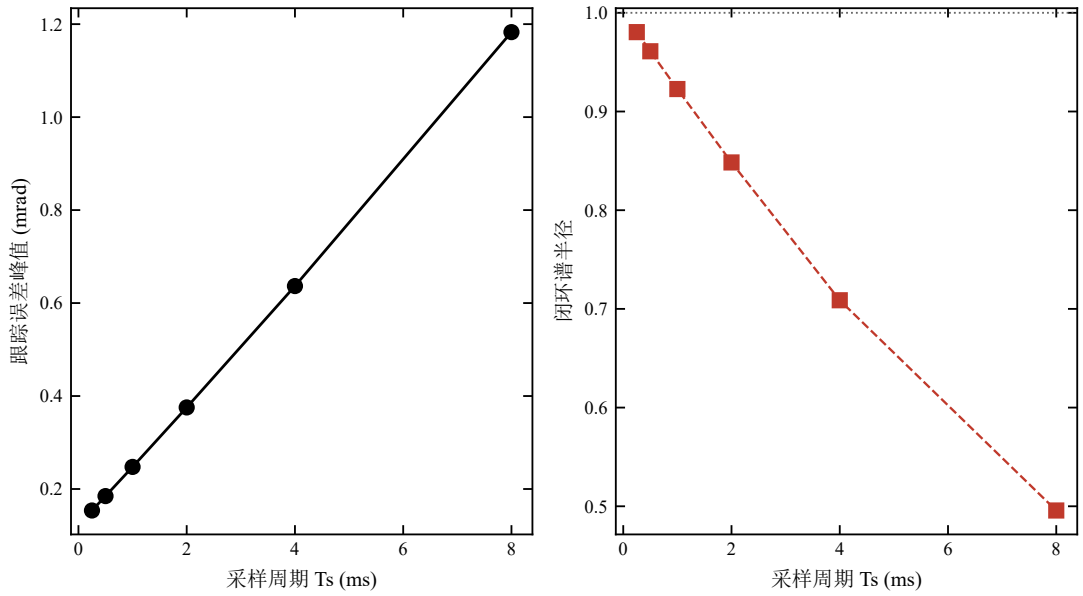


图 94-7　采样周期对跟踪误差与谱半径的影响

先看左图。0.25 ms 时误差 0.154 mrad，1 ms 时约 0.3 mrad，8 ms 时 1.183 mrad。误差随周期近似线性增长——这条直线的斜率就是“速度 × 周期”：匀速段 2 rad/s，采样间隔 1 ms 内位置变化 2 mrad，控制器要等到下一拍才能看到这段变化，误差自然按这个量级堆积。

把这六个数做一次线性拟合，得到 $\varepsilon_{max} \approx 0.12 + 0.132\,T_s$，$T_s$ 以 ms 计，误差以 mrad 计，相关系数接近 1。这个式子有两层含义。斜率 0.132 mrad/ms 说明每延长 1 ms 周期，峰值误差就多 0.132 mrad；截距 0.12 mrad 则是“采样足够快时也消不掉”的那部分，它是纯反馈跟踪 S 曲线时的固有滞后——要消掉它，只能靠前馈，不能靠提高采样率。

截距这一项很容易被忽略。很多人把采样周期从 1 ms 压到 0.25 ms，看到误差从 0.248 降到 0.154 就以为找对了方向，继续往下压却发现收益越来越小，最后卡在 0.12 附近。这不是硬件到了极限，而是问题本来就不在采样上：误差的主体是“控制器只知道当前误差、不知道下一时刻要往哪走”，这个信息缺口只能由前馈补上。§94.9 会给出对比：同样的轨迹，加速度前馈把误差压到 0.0583 mrad，比把采样率提高 40 倍更有效。

再看右图，谱半径从 0.9804 单调降到 0.4957。如果只看这一个指标，会得出“采样越慢越稳”的荒谬结论。原因在极点位置：最简模型下闭环极点约为 $z = 1 - K_{p\theta} T_s$，$T_s = 0.25$ ms 时 $z = 0.975$，贴近单位圆；$T_s = 8$ ms 时 $z = 0.2$，靠近原点。极点靠原点意味着“一拍就把误差消掉”，但这一拍横跨了 8 ms，拍与拍之间的误差没人管。

所以谱半径必须与时间尺度一起读。它下降，是因为每拍的控制作用变粗了，不是因为性能变好了。按这个最简模型，T_s 要到 20 ms 才会让 z 越过 -1 失稳；但 8 ms 时的误差已经是 0.25 ms 的 7.7 倍，工程上早已不可接受。稳定性边界和可用边界之间隔着十几倍的距离，这是离散系统里最容易被误读的一张图。

94.5.3 量化：10 μrad 带来的极限环

编码器分辨率 $q = 10\ \mu\text{rad}$，意味着位置测量永远带一个 $[-q/2, q/2]$ 的台阶。这个台阶经过位置增益放大，变成速度指令的台阶

$$\Delta v = K_{p\theta}\, q = 100 \times 10^{-5} = 1\ \text{mrad/s} \tag{94-23}$$

1 mrad/s 的速度指令听起来微不足道，但它是一个持续的、方波状的扰动。速度环的积分器会把它积成转矩脉动，电机在目标位置附近做小幅抖动，这就是极限环。

量化还有第二个入口：速度由位置差分得到。差分把量化噪声放大 $1/T_s$ 倍，1 ms 周期下 $q/T_s = 0.01$ rad/s，即 10 mrad/s 的速度噪声。位置环周期越长，这个放大越轻；周期越短，速度估计越“脏”。所以位置环周期并不是越短越好，它要在“跟踪误差”与“速度噪声”之间取折中。常见做法是用观测器或低通滤波做速度估计，而不是直接差分。

这条 10 mrad/s 的噪声经速度环放大后能到多大，可以估个数。速度环在 20 Hz 以下的闭环增益接近 1，10 mrad/s 的指令抖动会原样变成 10 mrad/s 的转速抖动，折算到 3:1 的负载端是 3.3 mrad/s；如果传动是 10 mm 导程的丝杠，就是每秒 5 μm 的抖动。这个量级在精密装配里已经不可接受，所以高精度场合的速度反馈不靠差分，而靠驱动器内部的观测器——观测器把电流指令与编码器位置一起用，等效地把量化噪声压低了十几倍。

分辨率的选择也要算，不能只看“位数越高越好”。19 位单圈编码器一圈 $2^{19} = 524288$ 格，一格 $2\pi/524288 = 12.0\ \mu\text{rad}$；20 位是 6.0 μrad；23 位是 0.75 μrad。本章取 $q = 10\ \mu\text{rad}$，介于 19 位与 20 位之间。选型时看两个数：一是 $K_{p\theta} q$，它决定速度指令的台阶，本章是 1 mrad/s；二是 q 折算到负载端的直线位移，20 位编码器配 10 mm 导程是 0.0095 μm，已经低于丝杠本身的螺距误差，再高的位数只是在测噪声。

处理量化极限环有三招，按代价从低到高排。第一招是位置误差死区：$|e| < q/2$ 时不再更新速度指令，把抖动关在门外，代价是引入最多半格的稳态误差，本章是 5 μrad。第二招是积分分离：误差小于阈值时冻结积分器，避免积分器把量化台阶积成转矩斜坡。第三招是换反馈方式：用观测器或有滤波的差分替代裸差分，代价是引入相位滞后，必须重新核相位裕度。三招都不改变机械，只是把量化噪声挡在环路之外。

第三个入口是电流采样与 PWM 占空比的量化。电流环 12 位 ADC、PWM 1% 的占空比分辨率，折算到电压是几毫伏量级，对 1 kHz 带宽的电流环影响不大。真正需要注意的是死区：功率管的死区时间会造成几伏的电压误差，在低速轻载时比反电动势还大，表现为过零点的转矩死区。

94.5.4 计算延迟

最后一笔损失是计算延迟。控制器从采样到输出要花时间：ADC 转换、坐标变换、PID 运算、PWM 更新。这段时间在离散模型里表现为一拍延迟，在频域里就是 (94-22) 的相位滞后。

位置环 1 ms 周期、7.9° 的损失可以接受；但如果控制器跑在总线另一头，通信再加 2 ms，损失变成 23.7°，相位裕度从 83.33° 掉到 60° 附近，阶跃开始出现可见超调。这就是为什么高性能伺服把位置环也放在驱动器里，而不是放在上位机——不是算力问题，是延迟问题。

速度环更敏感。103.2 Hz 配 1 ms 已经损失 37.1°，若再叠加一拍计算延迟，总损失 74°，相位裕度只剩十几度，系统进入“能跑但一碰就振”的状态。现场把这种状态叫“参数太硬”，其实是延迟把裕度吃光了。

延迟可以逐项拆，拆完才知道从哪省。本章规模的控制周期里，ADC 采样与转换约 2–5 μs，坐标变换与 PI 运算约 5–10 μs，PWM 寄存器更新要等到下一个载波周期，平均再滞后半个开关周期（10 kHz 时 50 μs）。三者相加约 60–65 μs，占 100 μs 电流环周期的六成。省延迟的顺序也因此明确：先把采样提前到 PWM 中点，再把运算搬到硬件加速器，最后才考虑提高主频——主频只能省掉中间那 5–10 μs，前两项才是大头。位置环的延迟大头则在通信，EtherCAT 一个周期 250 μs–1 ms，若把位置环放在上位机，光通信就吃掉一半预算。

三笔损失合起来，给出离散设计的经验规则：**每个环的采样频率至少是该环带宽的 10 倍，其中电流环按 20 倍取；凡是能放进中断的，不要放到任务里；凡是能用硬件算的，不要用软件排队。** 这三条与 §94.3 第五步一一对应。

94.6 稳定性与性能

稳定性不是一个“是/否”，而是一组边界：增益推到多高会振，惯量比大到多少会超调，扰动打一下要多久才回来。本节把这三条边界分别量化，并说明它们在真实机械上为什么比教科书上更紧。

94.6.1 三个裕度各自管什么

本章算例的刚体模型给出：位置环相位裕度 83.33°、增益裕度 33.21 dB，速度环 71.37°，电流环 90.00°。

三个数字的分工不同。相位裕度管“阻尼”，它决定阶跃响应的超调与振荡次数；83.33° 对应几乎无超调，45° 左右开始明显超调，30° 以下就会连续振荡好几次。增益裕度管“能加多少倍增益”，33.21 dB 意味着增益可以乘 45 倍才失稳

——这个数字漂亮得可疑，因为它是纯刚体模型算出来的。

相位裕度与超调量的对应可以查二阶近似：$\zeta \approx \mathrm{PM}/100$，$M_p = \exp(-\pi\zeta/\sqrt{1-\zeta^2})$。代入 45° 得 $\zeta = 0.45$、$M_p = 20.5\%$；60° 得 9.5%；70° 得 4.6%；83.33° 得 0.88%。本章实测超调 0.00%，比 0.88% 还小，原因是位置闭环更接近一阶而不是二阶，二阶近似在这里偏保守。这条换算的用处是快速判断：现场量到 20% 超调，就该知道相位裕度大概只剩 45°，而不是“再调调看”。

真实轴的增益裕度远没有这么大。把双惯量模型接进速度环，183.8 Hz 的谐振峰会把幅频曲线顶起来，-180° 处的幅值不再是小量，增益裕度随之掉到十几 dB 甚至几 dB。这也解释了现场一个矛盾：仿真里增益怎么加都稳，装到机器上稍微一加就啸叫。不是仿真错了，是仿真里的机械太“硬”。

所以读裕度要问一句：这个裕度是在哪个模型上算的。刚体模型的裕度只能当上限看，真实裕度要含谐振、含延迟、含量化。

94.6.2 惯量比：三个指标同时恶化

图 94-9 把惯量比从 1 扫到 10。左图双纵轴，实线是速度环相位裕度，虚线是位置超调；右图是三种惯量比下的位置阶跃。

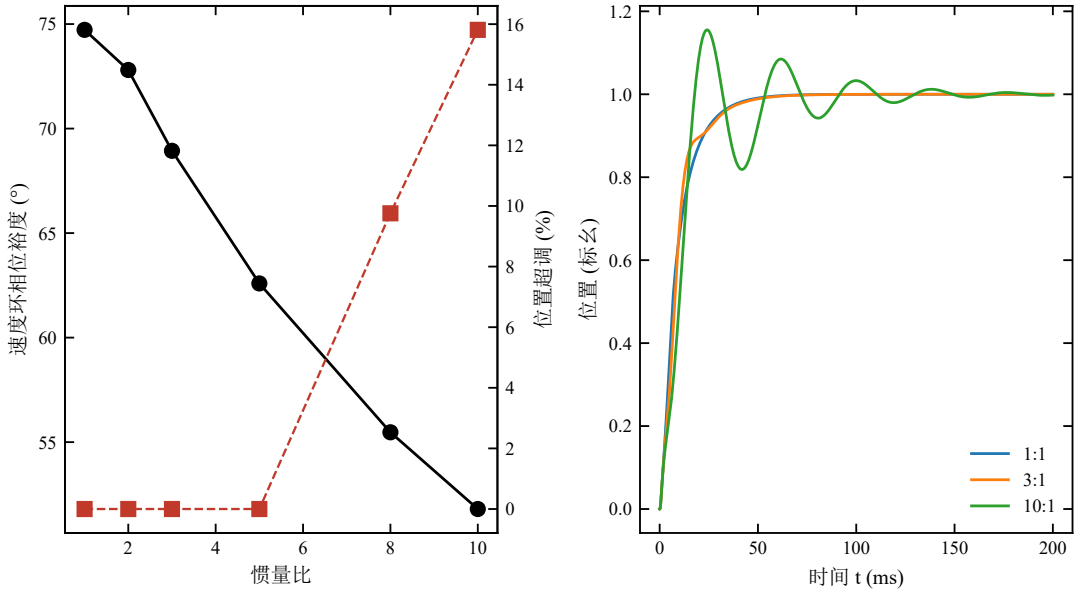


图 94-9　惯量比对相位裕度与阶跃超调的影响

数据列在表 94-5。

惯量比	速度环相位裕度 (°)	位置超调 (%)	调节时间 (s)	谐振峰 (Hz / dB)
1:1	74.72	0.00	0.0408	230.7 / 7.17
3:1	68.94	0.00	0.0425	184.9 / 11.90
5:1	62.59	0.00	0.0418	174.9 / 13.23
10:1	51.80	15.53	0.1203	167.2 / 14.34

前两列是单调恶化：惯量比每涨一档，相位裕度掉 5–7°。第三列却不是：1、3、5 三档的调节时间几乎一样（0.0408、0.0425、0.0418 s），到 10 才突然涨到 0.1203 s。这个“突变”值得解释。

1 到 5 之间，位置环带宽 21.9 Hz 离反谐振（159.2 到 71.2 Hz）还有 3 倍以上距离，谐振峰虽然从 7.17 dB 涨到 13.23 dB，但在 21.9 Hz 处已经被衰减得看不见，所以时域指标几乎不动。10:1 时总惯量涨到四倍，速度环剪切频率从 56.17 Hz

掉到 26.17 Hz，而位置环的穿越点仍由 $K_{p\theta}=100$ rad/s 定在 15.9 Hz 附近，两者只差 1.64 倍。5–10 倍的内外环分离被打破后，位置环在 15.9 Hz 处看到的已经不是“增益为 1、相位为零”的理想内环，而是带着明显滞后的速度闭环，相位裕度被内环吃掉一大截，闭环开始在阶跃里“抬头”，超调 15.53%、调节时间翻三倍。与此同时，反谐振掉到 50.3 Hz，与位置带宽 21.9 Hz 的比值只剩 2.3，“不超过 1/3”这条机械边界也一并失效。

这就是“惯量比 5 以内”这条经验的来历：它不是一条硬边界，而是一条“再往上，指标的突变随时会来”的预警线。5 到 10 之间没有明显恶化，但也没有余量——温度变化、负载变化、皮带张力变化，任何一项都可能把系统推到突变点另一侧。

惯量比还有一笔隐形的账：同样的速度环带宽，需要的 $K_{p\omega}$ 与总惯量成正比。本章 $J=4.8\times10^{-4}$ 配 2.2 A·s/rad；换成 10:1，总惯量涨到 1.32×10^{-3}，要维持 103 Hz 就得把 $K_{p\omega}$ 加到 6.0 左右，是原来的 2.75 倍。增益一大，电流指令的噪声分量同比放大，12 A 的限幅更容易被瞬时尖峰触发——一旦限幅，环路就进入非线性，前面所有线性结论作废。所以惯量比的影响不只是“相位裕度掉几度”，它还通过增益需求把驱动器推到限幅边缘。选型时把这两件事一起算，才不会出现“仿真很好、实机一跑就限流”的情况。

94.6.3 负载扰动：反馈真正负责的部分

前馈能补轨迹，补不了扰动。这一节用一个 0.05 N·m 的负载转矩阶跃来量化反馈的兜底能力。注意这个扰动量是运动所需峰值转矩 0.0140 N·m 的 3.6 倍，属于“被打一下”的工况，不是正常运行。

结果是：电机侧位置误差在 9.80 ms 时达到 -0.8577 mrad 的谷值，随后积分器把误差拉回，60.8 ms 时恢复到 2% 以内，末值 -0.0000 mrad。

三个数字分别对应三件事。9.80 ms 是“反应时间”，由速度环带宽决定；-0.8577 mrad 是“陷多深”，由惯量与传动刚度决定；60.8 ms 是“爬多快”，由积分时间与位置增益决定。

恢复时间也可以用带宽粗估。位置环 21.9 Hz 对应一阶 2% 调节时间 $3.9/(2\pi\times21.9)=28.3$ ms，实测 60.8 ms 是它的 2.15 倍。多出来的一倍来自“两级串联”：扰动先被速度环感知、再由位置环的积分慢慢磨掉，两级的滞后是相加而不是取大者。这条估算的实用价值是：想让扰动恢复快一倍，只加位置增益不够，还要同时缩短积分时间；而缩短积分时间又会吃掉相位裕度——三条边界在这里又一次咬合。

陷深可以手算校核。静态扭转为

$$\Delta\theta_{ss}=\frac{T_d}{K_s}=\frac{0.05}{120}=0.417\ \text{mrad} \tag{94-24}$$

实测峰值 0.8577 mrad 约为静态值的 2.06 倍，多出来的一倍是动态超调——扰动是突然加上去的，负载先“荡”出去再被拉回，荡的幅度约为静态挠曲的两倍。这个 2 倍关系在欠阻尼的二阶系统里很常见，不属于异常。

这里有一个容易被忽略的事实：末值回到 0，指的是电机侧位置误差。编码器装在电机上，它看到的是电机转角；负载端因为还受着 0.05 N·m 的转矩，轴始终扭着 0.417 mrad。也就是说，位置环“调好了”，工件仍然偏了 0.417 mrad。这是半闭环的固有缺陷，只能靠全闭环（光栅尺装在负载端）或双编码器来消除，代价是负载端的谐振直接进入位置环，稳定性更难做。

对比一下前馈的效果：§94.9 里加速度前馈把跟踪误差压到 0.0583 mrad，而这里一个 0.05 N·m 的扰动就造成 0.8577 mrad，是前者的 14.7 倍。这个比例说明，在实际产线上，误差的大头往往不是“跟不上轨迹”，而是“被扰动推走”。调参时先看扰动抑制，再看跟踪精度，顺序不能反。

94.6.4 三条边界合起来

把三节的结论并到一张纸上，稳定性由三条边界共同围成。

增益边界：相位裕度不低于 45°，对应 $K_{p\theta}$ 大体不超过 250。本章 200 时裕度 65.16°、超调 2.26%，已经接近这条线。

机械边界：位置环带宽不超过反谐振的 1/3，速度环不超过 1/2。本章 21.9 对 91.9 合格，103 对 91.9 已经越界，靠 PI 的低通特性勉强兜住；惯量比换成 10，反谐振降到 50.3 Hz，这条边界立刻收紧到 16.8 Hz，位置环必须降到 15 Hz 以下。

采样边界：每环采样率不低于带宽的 10 倍，电流环 20 倍。§94.5 已经算过，这条边界一旦破坏，损失的是相位裕度，表现为“参数没动却开始振”。

三条边界围出的区域不是矩形，而是一个斜切的三角形：增益推高，机械边界先到；机械变软，采样边界跟着紧。设计的工作，就是在这个三角形里找一个离三条边都够远的点。§94.7 逐个扫描参数，把“哪个旋钮往哪边拧、代价是什么”写成表。

三条边界还会互相挤压，不能分开看。把 $K_{p\theta}$ 从 100 加到 200，位置带宽从 21.9 Hz 升到 60.6 Hz，增益边界还剩 65.16°，单看这一项仍然合格；但 60.6 Hz 已经越过反谐振 91.9 Hz 的 1/3（30.6 Hz），机械边界同时被破坏，超调从 0 涨到 2.26%。这不是增益本身太大，是带宽把机械谐振拖进了环路。反过来，把 $K_{p\omega}$ 从 2.2 加到 3.5，速度环带宽到 159.9 Hz，采样边界立刻吃紧（按 10 倍要求，位置环周期需短于 0.63 ms），而位置带宽反而从 21.88 掉到 18.57 Hz——一条边界放松，另一条收紧，净效果是位置环变慢。

所以“最优点”不是三条边界的几何中心，而是“哪条边界最贵”的答案。机械边界最贵，改它要动结构；采样边界次之，改它要换控制器；增益边界最便宜，拧一下旋钮就行。设计顺序因此总是先把位置环压到机械边界以内，再用剩余裕度换节拍，而不是反过来——先用增益把节拍顶上去，再回头发现机械不允许。

94.7 参数意义与作用

三环一共五个可调增益：K_{pi}、K_{ii}、$K_{p\omega}$、$T_{i\omega}$、$K_{p\theta}$。现场调试时它们常被当成五个“数字”，其实每个数字都对应一个物理量，单位就能说明它管什么。本节先把量纲讲清，再逐个扫描，最后指出哪几个根本不该叫“参数”。

94.7.1 五个增益各是什么刚度

K_{pi} 的单位是 V/A，含义是“每 1 A 的电流误差，输出多少伏电压”。它是电流环的刚度：K_{pi} 越大，同样的电流偏差给出的电压越大，电流被拉回得越快。K_{ii} 的单位是 V/(A·s)，是同一个刚度的积分部分，负责把稳态电流误差压到零。两者不是独立的两个旋钮，按 (94-10) 它们由同一个带宽 ω_{ci} 和电机的 R、L 共同决定，改带宽就同时改了这两个。

$K_{p\omega}$ 的单位是 A·s/rad，含义是“每 1 rad/s 的速度误差，给出多少安电流指令”。它是速度环的刚度。$T_{i\omega}$ 的单位是 s，是积分时间：它不说“多硬”，只说“多快开始积分”。$T_{i\omega}$ 小，积分作用强，扰动恢复快，但相位裕度紧；$T_{i\omega}$ 大，积分慢，系统更温和，但负载扰动下的恢复时间变长。

$K_{p\theta}$ 的单位是 1/s，含义是“每 1 rad 的位置误差，给出多少 rad/s 的速度指令”。它把位置误差直接翻译成速度，所以量纲上是“频率”。100 s^{-1} 的意思是：位置差 1 rad，就让轴以 100 rad/s 去追。

这五个量里，前四个是“环内”参数，最后一个 $K_{p\theta}$ 是“环外”参数，它直接面对用户指标——定位精度、匀速误差、抗扰刚度都从它这里出。

把“刚度”这个词落到实处，可以代一组数。位置差 1 mrad 时，$K_{p\theta}=100$ 给出 0.1 rad/s 的速度指令；速度差 0.1 rad/s 时，$K_{p\omega}=2.2$ 给出 0.22 A 的电流指令；乘 $K_t=0.105$ 得 0.0231 N·m 的转矩。这三个数连起来就是“位置差 1 mrad → 0.0231 N·m”，等效刚度 23.1 N·m/rad。把它与传动轴刚度 120 N·m/rad 比一下：控制器的动态刚度只有轴的 1/5.2。也就是说，在积分器还来不及作用的短时间尺度上，轴比控制器“硬”五倍，负载端的挠曲主要发生在轴上，不在控制上。这解释了 §94.6.3 那个 0.8577 mrad 的谷值为什么接近静态挠曲的两倍，而不是被控制器立刻按住。

94.7.2 量纲链：三个增益如何合成一个刚度

把三个环的增益乘起来，会得到一个有意思的量：

$$\omega_{est} = \sqrt{\frac{K_{p\theta} K_{p\omega} K_t}{J}} = \sqrt{\frac{100 \times 2.2 \times 0.105}{4.8 \times 10^{-4}}} = \sqrt{48125} = 219 \text{ rad/s} \tag{94-25}$$

219 rad/s 即 34.9 Hz。这个量可以读作“三环总刚度对应的自然频率”：位置环把误差变成速度指令，速度环把速度误差变成电流，电流乘转矩常数变成转矩，转矩除以惯量变成加速度，加速度积分两次回到位置。整条链路走一圈，量纲正好是 1/s²。

34.9 Hz 与实测的位置带宽 21.9 Hz 差 1.6 倍。差额来自两处：内环的相位滞后（速度闭环在 21.9 Hz 处掉了 $6.67°$）和 PI 零点的位置。这个式子不能当设计公式用，它的价值在于告诉你“哪个因子最敏感”：$K_{p\theta}$ 和 $K_{p\omega}$ 在根号里是等权的，但 J 在分母里，惯量翻一倍，频率只降到 0.707 倍。所以惯量比从 3 变到 6，等效于把两个增益都乘 0.707——这就是 §94.6 里“惯量比大要降带宽”的定量来源。

同一个乘积还有另一个读法：$K_{p\theta} K_{p\omega} K_t = 100 \times 2.2 \times 0.105 = 23.1$ N·m/rad，这是位置环的等效扭转刚度。它与 (94-25) 只差一个 J：除以惯量得 1/s²，开方得频率。两个读法各有用途——比刚度时用 23.1，比频率时用 219。23.1 N·m/rad 意味着 1 mrad 的位置偏差能顶住 0.0231 N·m 的负载；本章的负载扰动是 0.05 N·m，单靠比例作用会静态偏 2.16 mrad，是实测动态谷值 0.86 mrad 的 2.5 倍。差额由速度环积分补上——积分在几十毫秒内把偏差吃掉，所以稳态为零，只在过渡瞬间留下那个谷。

这也说明“提高刚度”有两条路：加增益，或减惯量。加增益受相位裕度限制，$K_{p\theta}$ 从 100 加到 200，刚度翻到 46.2 N·m/rad，仍只有轴的 38%，而相位裕度已经从 $83.33°$ 掉到 $65.16°$；减惯量（换小负载、加减速比）不受稳定性惩罚，但受工艺限制。工程上先看惯量比，再看增益，顺序与 §94.8 的整定顺序一致。

94.7.3 每个旋钮的方向与代价

表 94-6 把五个增益连同两个前馈系数、两个机械量放在一起。前四列是“往大调会怎样”，最后一列是“什么时候不该再调”。

参数	物理含义	调大的收益	调大的代价	上限由谁定
K_{pi}	电流环刚度	转矩响应快	电流噪声放大、易与开关频率谐振	PWM 开关频率
K_{ii}	电流环积分	稳态电流误差归零快	超调、积分饱和	与 K_{pi} 绑定
$K_{p\omega}$	速度环刚度	扰动恢复快、相位裕度高	噪声放大、激发谐振	反谐振的 1/2
$T_{i\omega}$	速度环积分时间	系统更温和	扰动恢复慢、稳态刚度差	穿越频率的 1/5–1/10
$K_{p\theta}$	位置刚度	匀速误差小	超调、噪声、撞谐振	反谐振的 1/3
K_{vf}	速度前馈	消除速度滞后	模型不准时反而引入误差	速度环带宽
K_{af}	加速度前馈	消除加减速滞后	噪声被二次放大	电流限幅
R_J	惯量比	—	—	机械设计
K_s	传动刚度	提高反谐振	—	机械设计

注意最后两行没有“收益”和“代价”，只有“由谁定”。它们不是控制器参数，调不了。现场把它们当参数调（比如靠“电子惯量”补偿），本质上是在控制器里造一个假模型去抵消真实机械，能改善指标，但不改变物理边界。

看“上限由谁定”这一列，会发现五个增益里有四个的上限不是由“调参技术”决定的：K_{pi} 由开关频率定，$K_{p\omega}$ 由反谐振定，$K_{p\theta}$ 由反谐振定，$T_{i\omega}$ 由穿越频率定。只有 K_{ii} 与 K_{pi} 绑定，连独立旋钮都算不上。这张表因此可以当“谈判材料”用：指标做不到时，指着这一列说清是哪一个硬件约束卡住，比笼统地说“调不上去”有说服力。要么换更高开关频率的驱动器，要么把传动做硬，要么接受更长的节拍——三条路都在控制器之外。

表 94-6 里有一个反直觉的格子：$K_{p\omega}$ 调大，代价写的是“噪声放大、激发谐振”，收益写的是“相位裕度高”。通常认为增益越高越不稳，这里却相反。原因在下一节。

94.7.4 速度增益：越硬越稳的那个旋钮

把 $K_{p\omega}$ 从 1.0 扫到 5.0，得到表 94-7。

$K_{p\omega}$ (A·s/rad)	速度带宽 (Hz)	位置带宽 (Hz)	位置环相位裕度 (°)
1.0	54.3	35.90	65.90
2.2	103.2	21.88	83.33
3.5	159.9	18.57	86.40
5.0	230.5	17.53	87.65

按“增益越高越危险”的直觉，$K_{p\omega}=5.0$ 应该最不稳。实际相反：相位裕度从 65.90° 升到 87.65°，位置带宽从 35.90 Hz 收敛到 17.53 Hz。原因在速度闭环的形状。

$K_{p\omega}=1.0$ 时速度环带宽只有 54.3 Hz，PI 零点却仍在 19.9 Hz，零点与被控对象的极点距离太近，速度闭环在 21.6 Hz 附近出现 1.316 倍的隆起。这个隆起传到位置环，让位置闭环在 25 Hz 处鼓起 0.95 倍，硬把 -3 dB 点顶到 35.9 Hz。位置环相位裕度只剩 65.90°，阶跃会带明显过冲。

$K_{p\omega}$ 加大，速度环的穿越频率远离 PI 零点，隆起被压平，位置闭环恢复单调下降，带宽回落到 17.5 Hz 附近。此时的 17.5 Hz 才是“实打实”的带宽，35.9 Hz 是振出来的假带宽。

这条规律可以写成一句话：**速度环的增益不是越大越好，而是要与积分时间匹配。** 匹配的标志是速度闭环幅频在穿越点附近没有明显隆起；不匹配的标志是位置环带宽虚高、相位裕度偏低。

“隆起”可以量化成一个数：速度闭环幅频的峰值。四档增益实测分别是 1.316（21.6 Hz）、1.170（29.3 Hz）、1.117（35.2 Hz）、1.088（41.0 Hz）。峰值随增益下降，但不会消失——即使 $K_{p\omega}=5.0$，仍有 1.088 的鼓包，只是位置落到 41 Hz、离位置环穿越点足够远，才不再把位置带宽顶高。工程上把“峰值不超过 1.1（约 0.8 dB）”当作速度环整定合格的判据，比看阶跃超调灵敏得多，因为位置环会把速度环的隆起再放大一次。

四档数据还揭示了另一件事：隆起的位置随增益升高而右移（21.6 → 41.0 Hz），频率近似与 $K_{p\omega}$ 的平方根成正比。这意味着加增益不只是“把鼓包压平”，同时也在把它推离位置环的工作频段。两条效应叠加，才有表 94-7 里“增益越大、位置带宽越小、相位裕度越高”的走势。如果不看峰值只看带宽，就会误以为增加 $K_{p\omega}$ 是纯损失；实际上它把速度环变硬的同时也变“直”了，代价是高频噪声被放大 $K_{p\omega}$ 倍，电机在静止时更容易听到高频啸叫。

94.7.5 哪些量不该叫参数

最后要说清一件事：惯量比 R_J、传动刚度 K_s、编码器分辨率 q、采样周期 T_s 这四个量，经常被列在“调试参数”里，但它们不是。

R_J 与 K_s 由机械设计决定，改它们要动结构；q 由选型决定，改它要换编码器；T_s 由控制器与总线决定，改它要换硬件平台。这四个量共同决定了 §94.6 那三条边界的位置，而五个增益只能在这三条边界围出的区域里做分配。

所以调参的本质是“在别人画好的框里找最优点”，不是“把框撑大”。理解了这一点，就不会在 $K_{p\theta}$ 上反复加码，也不会指望某个“自整定算法”能突破机械上限。§94.8 给的整定顺序，正是按这个逻辑排的：先确认框的位置，再在框里调。

这四个量之所以常被误列进参数表，是因为它们“看起来能调”。驱动器软件里确实有编码器分辨率、采样周期、滤波时间这些输入框，填进去以后曲线会变，于是被当成旋钮。但填错的后果不是“性能差一点”，而是“模型与实物不符”：把 20 位填成 19 位，速度估计差一倍，$K_{p\omega}$ 的实际作用点跟着偏一倍；把 T_s 填成实际周期的一半，所有按离散域算出的裕度全部作废。这类错误在现场比增益调错更常见，也更难查，因为曲线“看起来还行”，只是怎么调都调不到指标。

正确的做法是把它们当“已知量”而不是“可调量”：选型时定死，投运前核对一次，之后不再动。真正需要反复权衡的只有五个增益，而这五个增益的可行域，正是这四个已知量画出来的。自整定算法能做的是在这个框里快速找到好点，它

扫描的仍是同一组不等式；谁要是宣称算法能突破框，那一定是把某个已知量偷偷改大了。

94.8 调参方法与经验

上一节说清了每个旋钮的方向，这一节说顺序。顺序错了，再好的方法也调不出来：先调位置环、再回头调速度环，前面调好的数全部作废，因为位置环的稳定裕度是速度环“借”给它的。

94.8.1 为什么必须从内往外

三个环是嵌套的。位置环的开环里包含速度闭环，速度环的开环里包含电流闭环。内环一动，外环看到的对象就变了。

这一点在数据上很直观。$K_{p\omega}$ 从 2.2 改到 3.5，速度环带宽从 103.2 Hz 升到 159.9 Hz；同样一个 $K_{p\theta}=100$，位置带宽却从 21.88 Hz 掉到 18.57 Hz。如果先调好位置环、再改速度环，位置环原先的“合格”就变成了“未知”。

更细的一组数说明这种耦合不是线性的：速度环带宽涨了 55%，位置环相位裕度只从 83.33° 升到 86.40°（+3.1°），位置带宽反而降了 15%。三个指标三个方向，没有一个能靠“按比例换算”猜出来。所以内环改动之后，外环必须重新测一遍，不能只改一个数就宣布完成。

耦合还有一层：外环的参数会“反压”内环的观感。位置环增益加到 200 时，位置阶跃出现 2.26% 超调，现场很容易把它归因于“速度环阻尼不够”，于是去加 $K_{p\omega}$；加完发现超调没降多少，噪声却上来了。真正的原因是位置环穿越点 60.6 Hz 已经逼近速度环带宽的一半，“内环当 1”的假设失效，该做的是降 $K_{p\theta}$，不是动速度环。判断依据在频域：看位置环穿越点上速度闭环还剩多少相位，而不是看时域波形猜。

所以整定的方向固定为：电流环 → 速度环 → 位置环 → 前馈与滤波。每往上一层，下面一层就冻结。这条规矩在现场常被破坏，原因多半是“位置精度不够，先把位置增益加上去试试”——试完发现振了，再回头降速度增益，于是前面所有工作推倒重来。

94.8.2 四步整定法

第一步，电流环。 让驱动器工作在电流模式，给一个小的电流阶跃（额定值的 10%–20%），用示波器看实际电流。

按 (94-10) 直接算出的 $K_{pi}=7.540$、$K_{ii}=3141.59$ 是起点。判断合格的标准是上升时间接近 $2.2/\omega_{ci}$，本章为 0.350 ms，且无明显超调。若上升时间偏长，说明实际电感比铭牌大，或者母线电压不够；若出现振荡，说明 K_{pi} 过头，或电流采样有延迟。

现场常用的手法是“临界振荡法”：把 K_{pi} 一点点加上去，直到电流波形出现等幅振荡，再退回 60%–70%。这个位置通常比计算值略低，留出的余量用来对付电感饱和。

实测与计算不符时，可以用上升时间反推参数。一阶闭环的 $\omega_c=2.2/t_r$：若实测 $t_r=0.5$ ms，则 $\omega_c=4400$ rad/s 即 700 Hz，比设计的 1000 Hz 低 30%；保持 $K_{pi}=7.540$ 不变，反推实际电感 $L=K_{pi}/\omega_c=7.540/4400=1.71$ mH，比铭牌 1.2 mH 大 43%。电感偏大的常见原因是没算端部漏感、或者串了电抗器。反推出来的值比铭牌可靠，整定时应当用它重算 K_{pi} 与 K_{ii}。

还有一条容易被忽略的约束：母线电压。电流从 0 跳到 12 A 用时 0.35 ms，需要的电压是 $L\,\mathrm{d}i/\mathrm{d}t + Ri = 1.2\times10^{-3}\times12/0.35\times10^{-3} + 0.5\times12 = 41.1+6.0 = 47.1$ V，这还没算反电动势。48 V 母线在这个电流阶跃下已经贴边，实测会看到上升沿“塌肩”——前段陡、后段缓。这不是增益问题，是电压不够。把电流环做到 1500 Hz（$t_r=0.233$ ms）需要 61.8+6.0=67.8 V，48 V 母线根本给不出，所以表 94-3 的第三行只适用于 72 V 以上的母线。选带宽之前先算这笔电压账，比事后调增益省事得多。

第二步，速度环。 位置环置为开环（或切到速度模式），给速度阶跃。先只加 $K_{p\omega}$、把 $T_{i\omega}$ 设得很大（积分几乎不起作用），观察阶跃响应。

$K_{p\omega}$ 加到出现 10%–20% 超调，然后退回一点。接着把 $T_{i\omega}$ 从大往小调，直到负载扰动下的恢复时间满意为止，同时盯住超调不要明显变大。本章的 $K_{p\omega}=2.2$、$T_{i\omega}=0.008$ s 就是这么定的：相位裕度 71.37°，速度带宽 103.2 Hz。

这一步最容易犯的错是“只加 P 不加 I”。纯 P 速度环对恒转矩扰动有稳态误差，表现为匀速运动时位置始终落后一点。有人用位置环的积分去补，结果位置环变成 PI，阶跃超调反而更大。正确做法是让速度环承担积分，位置环保持纯 P。

第三步，位置环。 位置环只有一个 $K_{p\theta}$。从小到大加，观察位置阶跃的超调与到位时间。

本章从 50 加到 100，带宽 8.4→21.9 Hz，匀速误差 40→20 mrad，超调仍为 0。加到 200 时带宽 60.6 Hz、超调 2.26%、相位裕度 65.16°，已经在边界附近。再往上，第一个报警的往往不是超调，而是静止时的啸叫——那是谐振被激发。

$K_{p\theta}$ 的上限有两个约束：相位裕度不低于 45°，带宽不超过反谐振的 1/3。两个约束取小者。本章两者给出的大致都是 200 上下。

第四步，前馈与滤波。 K_{vf} 从 1.0 起，因为速度前馈的理论最优值就是 1：位置环输出的速度指令已经等于参考速度，再加一个等量的前馈就完全补偿了跟随滞后。K_{af} 按 J/K_t 折算成电流前馈，本章为 $4.8\times10^{-4}/0.105=4.57\times10^{-3}$ A·s²/rad。

陷波滤波器放在实测的谐振频率上，深度先给 10 dB，不够再加。每加一级陷波，都要回头看一眼相位裕度——陷波在中心频率之外会带来相位滞后，深度越大、带宽越宽，滞后越多。

四步做完，记录一组数：三个带宽、三个相位裕度、阶跃的超调与调节时间、匀速误差、扰动恢复时间。这组数就是这台轴的“身份证”，以后任何异常都可以拿它对照。

94.8.3 三个信号怎么读

电流波形看三件事：上升沿是否干净（有没有振铃）、峰值是否被限幅削平、稳态是否有锯齿（PWM 与采样不同步）。电流波形是最“诚实”的信号，机械上的问题常常先在电流上露头。

速度阶跃看两件事：超调多少、尾部是否爬行。超调大是相位裕度不够；尾部爬行是积分太弱或摩擦太大。速度阶跃还能看出谐振：把阶跃做得很陡，响应里会叠一个高频小振荡，频率就是谐振频率。

位置阶跃看三件事：上升时间、超调、稳态抖动。稳态抖动基本都是量化与摩擦造成的极限环，加大增益只会让它更明显。

三个信号要一起看。只看位置阶跃，会把速度环的谐振当成位置环的问题；只看电流，会把机械的共振当成电气噪声。

谐振频率的读法值得单说。给一个陡的速度阶跃，响应尾部叠着的小振荡周期就是 $1/f_r$：本章 91.9 Hz 的反谐振对应 10.9 ms，183.8 Hz 的谐振对应 5.4 ms。用示波器直接量峰峰值间隔，比拿频谱仪省事。若振荡周期随负载变化，说明它不是电气谐振（LC 或开关频率），而是机械共振——机械共振频率会随负载惯量移动，电气谐振不会。这条判据能省掉大量排查时间：换一个负载再测一次，频率不动的是电气，动了的是机械。

电流波形里还有一种“锯齿”，周期恰好等于位置环周期 1 ms（1 kHz），那是位置环与电流环在抢同一个更新时刻，属于软件时序问题，不是控制参数问题。把它和机械谐振区分开，只需要看频率是否等于 $1/T_s$ 的整数倍。

94.8.4 五个常见错误

错误一：三环一起调。 改一个参数看一次响应，是唯一可靠的方法。同时改两个，出了问题时无法归因。

错误二：用位置增益补机械刚度。 传动软、谐振低，正确做法是降带宽或加陷波，不是把 $K_{p\theta}$ 加大。加大只会让谐振更容易被激发。

错误三：忽略摩擦。 本章所有模型都不含摩擦。真实轴在低速时摩擦非线性明显，表现为爬行与死区。速度环的积分能部分补偿，但补偿过头会在换向时产生“过冲—回弹”。

错误四：把编码器分辨率当成精度。 分辨率是能分辨的最小增量，精度是测量值与真值的偏差。10 μrad 的分辨率配一个装偏了 1 mrad 的编码器，精度仍然是 1 mrad。

错误五：只看阶跃，不看扰动。 阶跃测的是跟踪，扰动测的是刚度。产线上多数误差来自扰动，只调阶跃会在现场“看起来很准、干起活来不准”。

五个错误有一个共同点：都在“用控制器去补机械的账”。三环一起调，是想用运气补耦合；用位置增益补刚度，是想用增益补谐振；忽略摩擦、把分辨率当精度，都是把机械问题当成了软件问题；只看阶跃，是把“空载好看”当成了“带载能用”。机械的账只能用机械还——加粗传动、消隙、换编码器、改结构。控制器能做的是“把已有的余量兑现”，不是“创造余量”。理解这一点，五个错误自然就避开了。

94.8.5 一张整定记录表

把四步的判据压成表 94-8，打印出来贴在调试台上比记在脑子里可靠。

步骤	操作	观察量	合格判据	本章取值
1 电流环	电流阶跃 10%–20%	上升时间、振铃	无超调，$t_r \approx 2.2/\omega_{ci}$	0.350 ms
2 速度环	速度阶跃，先 P 后 I	超调、扰动恢复	超调 10%–20%，PM $>$ 60°	103.2 Hz / 71.37°
3 位置环	位置阶跃，加 $K_{p\theta}$	超调、到位时间、啸叫	Mp $<5\%$，PM $>45^\circ$	21.9 Hz / 83.33°
4 前馈滤波	$K_{vf}=1$、$K_{af}=J/K_t$	跟踪误差、裕度	误差降一个量级，裕度降 $<10^\circ$	0.0583 mrad

表 94-8 的第四行有一个容易忽略的条件：“裕度降 $<10^\circ$”。前馈本身不改变闭环极点，理论上不影响稳定性；但加速度前馈会把参考的二阶导数直接送进电流环，参考里的台阶（比如 S 曲线的加加速度跳变）会变成电流尖峰，等效于给系统加了激励。所以加完前馈必须回头复核裕度与电流峰值。

这张表还有一列没写进去：每次调整的“前值”。整定是一个搜索过程，调完发现更差、想退回去是常事；如果没有记录前值，退回去只能靠记忆，而人对“刚才那个数”的记忆在连续调了十几轮之后并不可靠。工程上的做法是每改一次就记一行：时间、改了哪个参数、从多少到多少、观察到什么、是否保留。五列，写起来不到一分钟，省下的是返工。

记录里最该写清的是“观察到什么”。写“响应变好了”没有信息量，三个月后无法复现；写“超调从 12% 降到 3%，调节时间从 45 ms 变成 52 ms”，才能判断这次调整是“换来了阻尼”还是“只是变慢了”。前者可以保留，后者往往是增益加过头、带宽被振出来的假象——§94.7.4 那个 35.9 Hz 的假带宽就是典型：带宽数字变大了，位置环实际更慢。

整定到这一步，系统已经能用。剩下的提升靠结构改进，那是 §94.9 的内容。

94.9 优化与改进

§94.8 的四步整定把系统调到“能用”。再往上提性能，靠调增益已经没空间了——每加一点都要拿裕度换。真正有效的是改结构：把已知的信息提前送进去（前馈），把已知的共振按下去（陷波），把看不见的状态估出来（观测器）。

94.9.1 前馈：把“已知的未来”提前

位置环的本质是“看见误差才动作”。匀速运动时，轴永远落后一个 $v/K_{p\theta}$，本章为 20 mrad。这个落后不是控制器的错，是反馈的宿命：误差必须先出现，控制器才能反应。

但轨迹是事先规划好的。既然知道下一时刻要走到哪、速度多少，为什么还要等误差出现？速度前馈就是把这个信息直接用上：把参考速度乘以系数，直接叠加到速度指令上。

$$u_\omega = K_{p\theta}(\theta_r - \theta) + K_{vf}\,\dot{\theta}_r \tag{94-26}$$

匀速段 $\ddot{\theta}_r = 0$，稳态时位置环输出为 0，速度指令完全由前馈提供。若 $K_{vf} = 1$，实际速度恰好等于参考速度，位置误差的稳态值

$$e_{ss} = \frac{v(1 - K_{vf})}{K_{p\theta}} \tag{94-27}$$

降为零。K_{vf} 小于 1，误差按比例剩下一部分；大于 1，则超前，误差反号。

实测：纯反馈 20.0000 mrad，只加一项速度前馈，误差就降到 0.1245 mrad——161 倍。图 94-8 左图是三条误差曲线，右图把纵轴放大到 ± 0.2 mrad，纯反馈那条已经出框，只剩两条前馈曲线可比。

前馈对参数误差并不敏感，这一点值得算。若 K_{vf} 只取到 0.95，按 (94-27)，剩余误差是 $2 \times 0.05/100 = 1$ mrad，仍是纯反馈 20 mrad 的 1/20；取到 0.9，剩 2 mrad，还是 1/10。也就是说，前馈系数差 5%–10%，效果只从 161 倍退到 20 倍、10 倍，量级上的改善仍然成立。这就是前馈“性价比高”的定量依据：它不要求模型很准，只要求大致对。

前馈还有一个常被误解的性质：它不改变闭环极点。从 (94-26) 看，前馈项是加在控制器输出上的、与误差无关的量，闭环特征方程里没有 K_{vf}。所以加前馈不会让系统变不稳，只会改变“参考到输出”的传递路径。真正的风险在别处：前馈让电机提前出力，电流峰值从 0.120 A 升到 0.133 A，约 11%；若把系数给到 1.2，峰值到 0.16 A、转矩 0.0168 N·m，是连续转矩需求 0.0045 N·m 的 3.7 倍。短时没问题，但节拍密集时要重算发热。所以前馈的上限不是稳定性，是电流与转矩的余量。

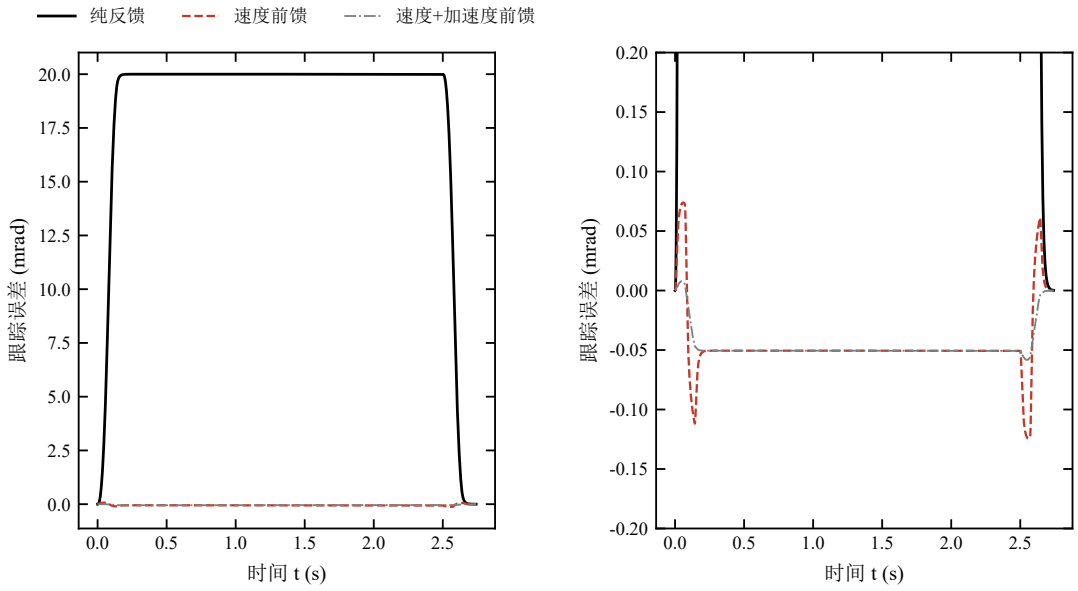


图 94-8　三种前馈配置下的跟踪误差

0.1245 mrad 不是零，因为 (94-27) 只对匀速段成立。S 曲线有七段，加减速段的参考速度一直在变，速度环跟踪一个斜坡指令时本身有滞后；加加速度跳变的那几个时刻，滞后最明显。这些残余都集中在 0.6 s、1.2 s 附近的尖峰上，右图看得很清楚。

94.9.2 加速度前馈：补上“要多少力”

速度前馈补的是“速度跟不上”，加减速段还有第二层滞后：要产生加速度，必须有力矩。这个力矩由谁出？纯反馈时由位置误差经速度环、电流环层层放大出来，放大过程中必然有延迟。加速度前馈直接把所需的力矩算出来，送进电流环：

$$i_{ff} = \frac{J\ddot{\theta}_r + B\dot{\theta}_r}{K_t} = \frac{4.8\times10^{-4}\times 28.3 + 2\times10^{-4}\times 2}{0.105} = 0.133\ \text{A} \tag{94-28}$$

这个 0.133 A 与 §94.3 第一步算出的峰值电流完全一致——本来就是同一件事，只是一个从指标倒推、一个从轨迹正推。

把 (94-28) 的分子拆开看，两项各占多少：$J\ddot{\theta}_r = 4.8\times10^{-4}\times28.3 = 0.01358$ N·m，$B\dot{\theta}_r = 2\times10^{-4}\times 2 = 0.0004$ N·m，前者是后者的 34 倍。所以加速度前馈里，阻尼项基本可以忽略；很多驱动器干脆只做 J/K_t 这一项，把 B 留给速度环的积分去补。反过来，如果轴上有很大的粘性负载（比如搅拌、卷绕），B 项就不能省，此时前馈系数要按 J 与 B 两个模型参数分别给。

加上加速度前馈后，误差从 0.1245 降到 0.0583 mrad，再降约一半。累计效果是 343 倍。

为什么不是降到零？三个原因。第一，模型里的 J、B、K_t 都有误差，前馈给出的是“按模型算的力矩”，不是“真实需要的力矩”。第二，前馈不经过反馈，模型错了没有人纠正它。第三，S 曲线的加加速度在段与段之间是跳变的，二阶导数不连续，任何基于连续假设的前馈都会在跳变点留下尖峰。

这三个原因指向同一条工程原则：**前馈补偿到 80%–90% 就够了，剩下 10%–20% 留给反馈纠偏。** 前馈补到 100% 甚至 110%，模型误差会直接变成过冲，而且反馈没有机会介入。

94.9.3 前馈管不到的部分

前馈依赖两件事：模型已知、轨迹已知。任何一件不成立，它就无能为力。

负载突变不满足第一条。§94.6 那个 0.05 N·m 的阶跃扰动，前馈一点也补不了，因为它在轨迹规划时还不存在。结果就是 0.8577 mrad 的瞬时挠曲与 60.8 ms 的恢复时间——比前馈后的跟踪误差大 14.7 倍。

摩擦不满足第一条。摩擦随位置、速度、温度变化，模型写不准，前馈只能补一个平均值。

齿隙两条都不满足。齿隙是位置相关的死区，在换向瞬间才起作用，规划时无法预知具体时刻。

所以前馈与反馈的分工是清楚的：前馈管“可预知”，反馈管“不可预知”。把前馈当成万能药，会在扰动来临时发现系统比纯反馈还差——因为前馈把增益“用掉”了一部分裕度，却没有增加抗扰能力。

还有一个更隐蔽的限制：前馈只对“参考通路”有效，对“扰动通路”完全无效。位置指令经前馈直接送到电流环，扰动却从负载端进、经过传动轴、穿过速度环才被看见，两条通路的传递函数不同。把前馈调到让跟踪误差接近零，扰动抑制能力一点没变；反过来，提高反馈增益能同时改善两者，代价是裕度。这就是为什么高精度场合的配方总是“前馈 + 高刚度机械 + 全闭环”，而不是“前馈 + 高增益”：前者分别对付两条通路，后者只想用一招同时应付。

从数据上看，本章前馈把跟踪误差从 20.0000 mrad 压到 0.0583 mrad，改善 343 倍；同一个系统对 0.05 N·m 阶跃扰动的峰值偏差仍是 0.8577 mrad，一点没降。两个数字放在一起，就能看出前馈的能力边界在哪里：它把“已知”的部分几乎清零，却在“未知”面前毫无作为。

94.9.4 陷波：把共振按下去

谐振是机械的固有属性，改不了频率，但可以在控制器里挖一个坑把它压住。陷波滤波器的传递函数是

$$N(s) = \frac{s^2 + 2\zeta_z\omega_n s + \omega_n^2}{s^2 + 2\zeta_p\omega_n s + \omega_n^2}, \qquad \zeta_z \ll \zeta_p \tag{94-29}$$

分子在 ω_n 处产生一个零点，把该频率的增益压下去；分母的两个极点让相位在两侧回到 0。深度由 ζ_z/ζ_p 决定，宽度由 ω_n 附近的平坦度决定。

用起来有三条经验。第一，中心频率要按实测值设，不能按 (94-17) 算——(94-17) 给的是无阻尼的理论值，实测峰值因为阻尼会偏低几个百分点（本章 184.9 Hz 对理论 183.8 Hz，差得不多，但惯量比大时偏差会明显）。第二，深度先给

10 dB，不够再加，每次加完复核相位裕度。第三，陷波只压一个频率，如果机械有两个谐振点（比如轴系一个、机架一个），要串两级，每级都会带来额外相位滞后。

陷波的相位代价可以算出来，这也解释了第二条经验为什么强调“复核”。取中心 184 Hz、深度 10 dB，对应 $\zeta_z/\zeta_p = 10^{-10/20} = 0.316$。若取 $\zeta_p = 0.5$（较宽的坑），在 103 Hz 处 $\omega/\omega_n = 0.56$，分子相位 $\arctan(0.177/0.686) = 14.5^\circ$，分母相位 $\arctan(0.560/0.686) = 39.2^\circ$，净滞后 24.7°；若把坑收窄到 $\zeta_p = 0.2$、$\zeta_z = 0.063$，同样 10 dB，净滞后只剩 12.2°。速度环原本 71.37° 的裕度，被一个宽坑吃掉四分之一，被窄坑吃掉六分之一。这就是“深度够用就行、能窄不要宽”的定量理由。

陷波还有位置之分。放在速度环内部，它同时压低扰动抑制频段；放在参考通道上，只影响跟踪，不影响抗扰。多数驱动器给的是“转矩指令陷波”，属于前者；如果只想让轨迹跑得平，可以把它挪到前馈通道上，代价是扰动仍然能激起共振。选哪种取决于主要矛盾是“跟踪抖”还是“负载扰”。

陷波与降带宽是两种不同的取舍。降带宽是“整体退让”，陷波是“定点清除”。前者损失所有频段的性能，后者只损失谐振附近的相位。所以能用陷波解决的，不要用降带宽。

现场调陷波可以按五步走。一、用陡阶跃或扫频测出谐振频率，误差控制在 2% 以内。二、中心频率设到实测值，品质因数先按“窄”取。三、深度从 6 dB 起步，逐步加到 10–12 dB，每加一次看阶跃尾部的小振荡是否消失。四、回看速度环相位裕度，掉超过 10° 就把深度退回去，改用更窄的坑。五、若一个坑压不住（峰太宽），串第二级，两级中心错开 5%，避免相互抵消。这五步的核心是“先窄后深”，反过来做（先深后窄）很容易在第一刀就把裕度砍掉，后面没有调整空间。

94.9.5 观测器与全闭环

标准配置里，编码器装在电机上，测的是电机转角。这带来两个问题：负载端的挠曲看不见（§94.6 的 0.417 mrad 静态扭转），速度只能靠差分（§94.5 的量化放大）。

解决办法有两个层次。低层次是加一个状态观测器：用电机转角与电流指令，按双惯量模型估出负载转速与扭转角。观测器不需要额外硬件，但依赖模型准确性，惯量变了要重调。高层次是装第二编码器或光栅尺，直接测负载位置，做成全闭环。

全闭环的代价要提前想清楚。光栅尺装在负载端，负载的谐振就直接进入位置环，原本被电机惯量“过滤”掉的高频分量全部暴露。此时位置环带宽往往不升反降，必须配陷波才能用。工程上的折中是“双编码器反馈”：位置环用负载编码器，速度环仍用电机编码器，两者之间加一个差分环节。

还有一笔账常被算错：光栅尺的分辨率折算成角度，未必比电机编码器细。1 μm 的尺配 10 mm 导程的丝杠，一个脉冲对应 $1\ \mu\text{m}/10\ \text{mm} \times 2\pi = 0.628$ mrad，反而比 20 位电机编码器的 6 μrad 粗 100 倍。所以全闭环买到的不是“更细的分辨率”，而是“更真的位置”——它把丝杠螺距误差、反向间隙、轴扭转变形全都测进去了。本章那 0.417 mrad 的静态扭转，在电机编码器上完全看不见，在负载端尺子上就是 0.417 mrad 的真实偏差，比尺子本身 0.628 mrad 的分辨率还小。这也是为什么“高精度”场合常常两者都要：电机编码器保速度环，负载尺保位置环。

观测器的带宽要选在速度环与噪声之间。太低（比如与速度环同为 103 Hz），估计值跟不上真实扭转，等于没估；太高（比如 2 kHz），编码器量化噪声被放大进估计值，反而污染电流指令。工程上取速度环带宽的 3–5 倍，本章约 300–500 Hz，再高就要看编码器质量。观测器的参数依赖 J_M、J_L、K_s 三个模型量，换了负载必须重算，这一点比陷波更麻烦——陷波只依赖一个频率，观测器依赖整个模型。

94.9.6 轨迹规划也是优化手段

最后一类改进不在控制器里，在参考信号里。

S 曲线把加速度的跳变变成加加速度的有限值，减少了高频激励。本章的 $j = 400\ \text{rad/s}^3$ 把峰值加速度限制在 28.3 rad/s²，如果换成梯形速度（加速度阶跃），同样行程的跟踪误差会明显变大，因为阶跃里含有大量高频分量，正好打在

速度环的薄弱处。

jerk 的大小是可以算的，代价也能量化。加速度上限与 jerk 的关系是 $a_{max} = \sqrt{jv}$：$j = 400$、$v = 2$ 时 $a_{max} = 28.3$ rad/s²，加加速段时长 $t_1 = \sqrt{v/j} = 0.0707$ s，总行程时间 $d/v + 2t_1 = 2.641$ s。若为了更平顺把 jerk 减半到 200，a_{max} 降到 20.0，t_1 升到 0.1 s，总时间变成 2.700 s，比原来多 59 ms。也就是说，平顺性翻倍，节拍付出 2.2%。反过来把 jerk 加倍到 800，a_{max} 升到 40，总时间缩到 2.600 s，省 41 ms，但电流峰值与机械冲击同步上升。这条 $a_{max} = \sqrt{jv}$ 的关系说明：想同时要“快”和“柔”，只能从速度上限下手，jerk 的调节空间很小。

进一步还可以做输入整形：在参考信号里预先叠加几个延迟的脉冲，让它们激发的振动相互抵消。这在柔性臂、起重机这类低频大幅摆动的对象上效果显著，在刚性伺服轴上收益有限——因为刚性轴的谐振在 180 Hz 以上，规划器的延迟会直接拖慢节拍。

改进手段排个序：前馈收益最大、代价最小，优先做；陷波解决特定问题，按需做；观测器与全闭环投入大，用在精度要求高的场合；轨迹规划改的是激励，属于“从源头减少问题”。

这些手段都有一个共同前提：机械本身没有大毛病。如果传动有齿隙、导轨有爬行、联轴器松动，任何算法都只是把问题藏起来。§94.10 讲的就是怎么在落地阶段把这些前提确认掉。

94.10 工程落地

前面九节是“算得对”，这一节是“装得上”。实验室里的伺服轴到了产线，会多出四件事：选型要写进采购单、线要接对、电要上得安全、验收要有据可依。这四件事做不好，前面算得再准也没用。

94.10.1 从需求到投运：三条泳道

图 94-10 把 §94.3 的七步画成三条泳道：需求与核算、部件选型、校核与投运。泳道之间的箭头不是自动流转，每一道末尾都有一个判断：核算不过就回需求，选型不过就回部件，校核不过就回选型。

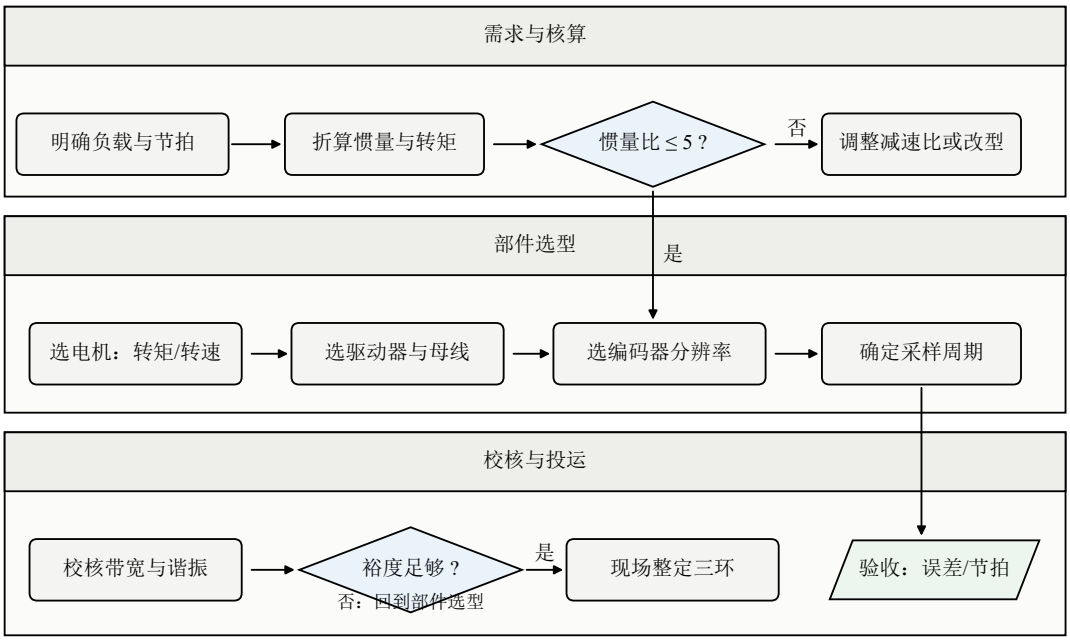


图 94-10　伺服轴从需求到投运的流程

这张图值得对照 §94.3 看。七步是“做什么”，三条泳道是“谁来做”：核算阶段是机械工程师与工艺工程师，选型阶段是电气工程师与采购，校核投运是调试工程师。跨泳道的每一次回头，都意味着一个部门要把已经交出去的东西拿回来改——

这就是为什么“设计阶段多花一周、现场少停三天”。

回头的代价随阶段递增，可以粗略按十倍估。第二步发现惯量比超标，改的是图纸；第三步发现，改的是订单；第六步发现，改的是已经装好的电机与驱动器；第八步（投运）发现，停的是产线。所以流程设计的原则不是“每一步都做对”，而是“把能提前的判断尽量提前”：惯量比与反谐振在第二步、第四步就能算准，不必等到通电。图 94-10 里每道末尾那个菱形判断框，价值就在这里。

94.10.2 选型单上必须有的六个量

采购单上通常只写“伺服电机一台、驱动器一台”。这六个量不写清，到货后一定扯皮。

第一，连续转矩与峰值转矩，单位 N·m，注明持续时间。第二，额定转速与最高转速，注明是机械允许还是电气允许。第三，转矩常数 K_t 与反电动势常数 K_e，两者必须成对给出，因为 K_t 用 N·m/A、K_e 用 V·s/rad 时数值相等（本章都是 0.105），单位混用会导致前馈系数算错。第四，转子惯量，用于算惯量比。第五，编码器类型与分辨率，本章 10 μrad。第六，驱动器母线电压与连续/峰值电流。

容易漏的是后两项与前四项之间的匹配。编码器分辨率决定速度环能否用差分估速；母线电压决定电流环能否在规定时间内把电流拉到目标值。本章电机电气时间常数 $L/R = 2.4$ ms，把电流从 0 拉到 0.133 A 只需 0.0665 V，电压裕量极大；真正吃电压的是高速段的反电动势——1000 rad/s 时 $K_e\omega = 105$ V，母线至少要 150 V。这个约束往往比转矩约束更早到。

垂直轴还要加两个量：抱闸保持转矩与动作延时。抱闸的保持转矩按负载重力矩的 1.5 倍选，动作延时通常 20–100 ms。这两个数决定了“断电时会不会掉下来”，与控制器参数无关，却必须在选型单上写清。电缆长度也要写：动力线超过 20 m 时，PWM 上升沿在电机端会产生近两倍的反射电压，必须加输出电抗器或改用耐压更高的电机；编码器线超过 50 m 时，差分信号的眼图开始闭合，表现为偶发的位置跳变。这两条都不是“调参能解决”的问题。

94.10.3 布线与接地

运动控制的干扰源比过程控制多一个数量级：PWM 的 dv/dt、电机的换向火花、编码器的差分信号，全挤在一根拖链里。

三条规矩。第一，动力线与编码器线分槽走，交叉时成 90°。第二，编码器用差分传输（RS-422 或 BiSS），屏蔽层单端接地，接在驱动器侧，不要两端都接——两端接地会形成地环流，低频噪声反而更大。第三，PE 星形一点接地，驱动器、电机、机柜共一点，避免地电位差通过编码器屏蔽层串进来。

还有一个常被忽略的点：电机的 PE 线必须单独走，不能靠安装法兰接地。法兰上有漆、有油、有绝缘垫，高频阻抗很高，PWM 的共模电流会找不到回路，转而从编码器线回流，表现为位置信号的随机跳变。

共模电流的大小可以估。电机电缆的对地电容约 100 pF/m，50 m 就是 5 nF；PWM 上升沿 100 ns、幅值 500 V，$\mathrm{d}v/\mathrm{d}t = 5 \times 10^{9}$ V/s，位移电流 $i = C\,\mathrm{d}v/\mathrm{d}t = 5 \times 10^{-9} \times 5 \times 10^{9} = 25$ A。这个 25 A 只在上升沿的百纳秒内存在，但它足以在接地阻抗上产生几伏的电位差。屏蔽层单端接地、动力线与信号线分槽，都是为了让这 25 A 走它该走的路，而不是从编码器里借道。工程上还可以在电机线两端套共模磁环，把高频阻抗抬上去，成本几十元，效果往往比改布线更直接。

94.10.4 上电与首次运行

第一次上电的顺序不能错，错了可能烧功率管。

先不接电机，只给控制电，确认驱动器能识别编码器、能读出零位。再接电机但不带负载，用手转轴，看位置反馈的方向与变化是否合理——如果转一圈读数只变了一点点，多半是分辨率设错；如果方向相反，改极性而不是改接线。

然后带负载、低增益、小行程点动。此时不要急着加增益，先确认三件事：限位开关有效、急停有效、超程保护有效。这三件事确认完，才开始 §94.8 的四步整定。

上电顺序里还有两个细节。一是母线预充电：中大功率驱动器直接合闸会产生十几倍额定电流的浪涌，必须先经限流电阻把母线电容充到 80% 以上，再旁路电阻。预充电时间由 RC 决定，本章级别的驱动器通常是 0.3–1 s，听到继电器“咔”一声才算完成。二是抱闸时序：垂直轴必须“先励磁、后松闸”，断电时“先抱闸、后断使能”，两个动作之间留 50–100 ms。顺序反了，负载会在松闸瞬间下滑，编码器看到位置突变，控制器以为出错而报警。

首次运行还要测一次谐振。方法很简单：给一个陡的速度阶跃，用示波器看电流或速度波形，响应里叠的那个高频小振荡就是谐振。记下频率，与 (94-17) 的理论值对一下。差得多，说明传动链里有松动或联轴器刚度不足。

94.10.5 验收指标怎么写

验收最容易扯皮的地方是“精度”两个字。必须拆成四项写。

定位精度：从任一位置走到目标位置，多次测量的最大偏差。**重复定位精度**：同一目标位置多次往返，读数的分散度，通常比定位精度小一个量级。**跟随误差**：匀速段的位置滞后，本章纯反馈 20 mrad，加前馈 0.0583 mrad。**轮廓误差**：多轴联动时到理论轨迹的法向距离，本章圆轨迹 2.355 mm。

四项要注明测试条件：速度、负载、温度、预热时间。同一台轴，冷机与热机的重复精度能差一倍——不是轴坏了，是丝杠热伸长。所以验收标准里必须写“预热 30 分钟后测量”这类限定语。

还有一项常被漏掉：**扰动刚度**。写法是“在额定负载的 50% 处施加阶跃扰动，位置偏差不超过 X”。本章的 0.05 N·m 对应 0.8577 mrad，这个数可以直接当参考。

四项之外还要加两项“查得出机械病”的测试。一是反向间隙：千分表顶住负载，正向走 0.1 mm 再反向走 0.1 mm，两次读数之差就是间隙。20 μm 的齿隙配 10 mm 导程，折算到电机轴是 $0.02/10 \times 2\pi = 12.6$ mrad，比本章 0.417 mrad 的轴扭转变形大 30 倍——也就是说，有齿隙的轴，无论控制器调得多好，换向精度都由机械决定。二是圆度测试：两轴插补一个半径 50 mm 的圆，用球杆仪或编码器数据画实际轨迹。带宽不足表现为圆整体“缩小”（本章 2.355 mm），齿隙表现为四个象限切换点上的尖刺，两轴增益不匹配表现为椭圆。三种形状对应三种病因，比单看一个“圆度误差”数字有用得多。

94.10.6 五个高频故障

把现场最常见的问题压成表 94-9。这张表的用法是“先看现象、再查原因”，不要一上来就改参数。

现象	最可能的原因	验证方法	处理
静止时高频啸叫	位置增益过高，激发谐振	降 $K_{p\theta}$ 看是否消失	降增益或加陷波
到位后小幅抖动	量化极限环、积分过强	看抖动幅度是否约等于 q	降积分、加死区
换向时过冲回弹	齿隙、摩擦死区	低速换向看电流波形	机械消隙、摩擦补偿
匀速段误差偏大	前馈未开、$K_{p\theta}$ 偏低	对比纯反馈与加前馈	开速度前馈
加速段误差尖峰	加速度前馈缺失或模型偏差	看误差尖峰是否与加加速度同步	加 K_{af}、核 J

表 94-9 的处理列里，凡是振动类故障，处理都不是“加增益”。这不是巧合：现场多数振动类故障，根源是增益超过了机械允许，而不是增益不够。加增益能解决的是“跟不上”，解决不了“振”。

表里没有列的第六类故障是通信丢帧，现象是“偶发一次位置跳变、报警、复位后又正常”。它不属于控制参数问题，却最容易被误当成参数问题，因为现象看起来像“环路偶尔不稳”。判别方法很简单：查驱动器的通信错误计数器，或者看跳变是否恰好发生在总线周期的整数倍上。丢帧的根源通常是终端电阻缺失、屏蔽不良、或节点数超过总线能力。参数调得再细，也治不了丢帧。

排查顺序建议固定为“先看机械、再看电气、最后看参数”。机械问题（松动、齿隙、卡滞）会让任何参数都调不好；电气问题（接地、屏蔽、编码器）表现为随机性；参数问题表现为可重复的规律性。三类的现象特征不同，按这个顺序

查，能避免在一个错误的方向上反复加增益。

94.10.7 落地清单

最后给一张投运前必须逐项打勾的清单，共八条。

一、惯量比已核算，不超过 5。二、反谐振频率已实测，位置环带宽不超过其 1/3。三、编码器分辨率优于定位精度的 1/5。四、位置环周期不超过带宽倒数的 1/10。五、动力线与信号线分槽，屏蔽单端接地。六、限位、急停、超程保护已实测有效。七、四步整定记录已归档，含三个带宽与三个裕度。八、验收四项指标已按约定条件测量并留档。

八条都打勾，这台轴才叫“交付”。任何一条空着，都可能在三个月后变成一次非计划停机。

清单为什么要写死数字，而不是写成“惯量比合理”“分辨率足够”？因为形容词无法判定。两个人对“合理”的阈值不同，评审时谁也说服不了谁，最后往往按“看起来没问题”放行。写成“不超过 5”“优于 1/5”“不超过 1/10”，判定就变成一次除法，谁算都一样。本章所有判据之所以都带具体系数，也是同一个理由：工程上的分歧，多半不是原理分歧，而是“多大算大”的分歧。

清单里第七、八条最容易被跳过，因为它们不产生新的动作，只要求“留档”。但正是这两条决定了三个月后还能不能复现：三个带宽与三个裕度是这台轴的“基线”，以后任何一次异常，第一件事就是拿现场数据与基线比。没有基线，就只能凭感觉判断“是不是变差了”，而感觉在产线上从来不是证据。

94.11 优缺点与适用边界

三环串级加前馈，是运动控制里最“老”的结构，也是装机量最大的结构。它不先进，但很难被替代。本节说清它的三个优点、三个代价，以及什么场合该换别的方案。

94.11.1 三个优点

第一，每一环的物理意义清楚。 电流环管转矩，速度环管阻尼，位置环管刚度。出问题时可以按“哪一环的波形不对”逐层定位，不需要一个黑箱模型。这一点在产线上极其重要——停机时间按分钟计价，工程师需要的是“十分钟内找到问题”，不是“理论上最优”。

这种可定位性有具体形态。位置抖动但电流平稳，问题在编码器或量化；电流有周期性尖峰但位置误差不大，问题在增益过高激发了谐振；位置稳态偏差大但动态正常，问题在积分太弱或前馈没开。三种现象对应三个层次，不需要建模就能分流。换成状态空间或 MPC，观测器把三层状态混在一起，同样三个现象要拆开看，得先信模型、再信估计，排查链条长了一倍。

第二，限幅天然分层。 电流限幅保护功率管，速度限幅保护机械，位置限幅保护行程。三个限幅分别放在三个环的输出上，互不干扰。这是串级结构最实用的优点，任何单环结构都要额外设计一套限幅逻辑。

第三，可以用带宽换性能，且换得可预测。 §94.7 的表格把“增益加多少、带宽涨多少、裕度掉多少”量化得清清楚楚。工程师不需要重新做系统辨识，只要按表调。

94.11.2 三个代价

第一，调参有顺序依赖。 §94.8 说过，必须从内往外。这意味着改一次机械（换电机、换减速比），整套参数要重调。串级结构的参数不是“一套通用值”，而是“这一台机器的值”。

第二，内环带宽是外环的天花板。 位置环带宽最多到速度环的 1/5，速度环最多到电流环的 1/10。想提位置带宽，必须先把电流环提上去，而电流环受开关频率限制。这条链路把整体性能锁在一个由硬件决定的框里，软件上无法突破。

这条天花板可以一路算下来。开关频率 10 kHz → 电流环取 1 kHz → 速度环取 103 Hz → 位置环取 21.9 Hz，四层各差约 10 倍、10 倍、4.7 倍。若想做到 40 Hz 的位置带宽，按 1/5 的比例反推，速度环要 200 Hz，电流环要 2 kHz，开关频率

至少 20 kHz；同时反谐振必须高于 120 Hz，而本章是 91.9 Hz——硬件三项（开关频率、传动刚度、编码器带宽）同时要升级。这就是“性能提升是系统工程”的定量含义：单改一行参数没有用。

第三，对机械谐振敏感。 双惯量的反谐振与谐振限制了带宽上限，§94.6 已经量化。串级结构对此没有内建的抑制能力，只能外挂陷波或降带宽。这也是为什么高性能场合要用观测器、全闭环这些“外挂”。

三个代价合起来，可以算一笔维护账。一台三环轴的可调量包括五个增益、两个前馈系数、一到两个陷波，以及速度环与电流环的滤波时间，合计十到十二个。每个都有明确的物理意义与调整方向，这是优点；但每次换型都要重走一遍四步整定，这部分工时不会因为结构成熟而消失。装机量大的产线通常把这些数做成“机型模板”，新型号先套模板再微调，靠的正是“每个数都能解释”这一点——若换成一个整体辨识的黑箱模型，模板就无从迁移，因为它的参数不对应任何可测量的物理量。

所以这三个代价要分开看：第一条是工时，可以靠模板与流程摊薄；第二条是物理天花板，只能靠硬件升级突破；第三条是机械依赖，只能靠外挂补偿缓解。三者里只有第一条算“习惯问题”，后两条是硬约束。选型讨论中常见的错误，是把三条混成一句“串级太老”，于是用一个更复杂的结构去换更少的工时，却在物理天花板上一点没前进——换结构不改变开关频率，也不改变传动刚度，能改变的只有“谁来承担这份滞后”。

判断一个替代方案是否值得，只需问一句：它动的是哪一条代价。若答“减少参数、缩短调试”，那是拿工时换工时，收益有限；若答“在同样硬件下把带宽提高一倍”，那要追问它凭什么绕开内环天花板，答不上来就说明只是把滞后搬到了别处。

94.11.3 什么场合不该用

直驱与直线电机。 没有减速器、没有丝杠，电机与负载刚性连接，惯量比接近 1，反谐振可能高到 500 Hz 以上。这种情况下三环串级仍可用，但常常简化为“电流环 + 位置环”两环，速度环的作用被前馈取代。少一个环，少一组参数，调试更快。

极低刚度的柔性机构。 比如长臂机器人、大跨度龙门。反谐振可能低到 5–10 Hz，落在位置环带宽之内，串级结构的“内环当 1”假设彻底失效。这类对象要用柔性模态建模、输入整形、甚至自适应控制。

纳米级定位。 半导体设备、光学装调，要求亚微米甚至纳米级。此时摩擦、爬行、热漂移成为主要误差源，PID 的线性假设不成立，需要搭配摩擦补偿、迭代学习、以及环境控制。三环串级仍是底层，但上位要加一层“逐次修正”。

成本极敏感的场合。 几千瓦以下的简单定位，步进电机加细分驱动就能满足，没必要上伺服。选型的第一个问题不是“用哪种控制算法”，而是“这个精度真的需要闭环吗”。

94.11.4 与相邻方法的对比

表 94-10 把三环串级与三种常见替代方案放在一起。

方法	优势	劣势	典型场合
三环串级 + 前馈	结构清晰、限幅分层、易调试	带宽受内环限制、对谐振敏感	通用伺服轴、机床进给
状态空间 / 极点配置	可直接处理双惯量、多变量	依赖精确模型、限幅难处理	高性能主轴、柔性机构
模型预测控制	可显式处理约束与多轴耦合	算力大、模型失配时鲁棒性差	多轴联动、快速轨迹
迭代学习	周期性轨迹的误差可收敛到很低	只适合重复轨迹、收敛需多周期	冲压、注塑、晶圆扫描

这张表说明一件事：三环串级不是“最好的”，而是“最通用的”。它在 80% 的场合够用，且成本最低；剩下 20% 才需要换方案。工程判断的顺序应该是：先用三环串级把指标做到边界，确实做不到，再考虑换结构。反过来，一上来就上 MPC，往往在限幅、噪声、调试成本上吃暗亏。

替代方案的代价也可以量化。极点配置要拿到四个状态（电机位置、电机速度、负载位置、负载速度），只有电机编码器时，后两个必须靠观测器；观测器带宽取速度环的 3–5 倍，而观测器本身又依赖 J_M、J_L、K_s 三个参数，参数一错，配置出来的极点就偏。MPC 在每个采样周期解一次二次规划，预测步长 20、控制步长 5 时，单次求解在 DSP 上约几十微秒，放进 1 ms 周期可行，但要把“转矩限幅、速度限幅、位置限幅”全部写成约束，模型还得在线性工作点附近线性化。迭代学习则要求轨迹逐周期完全重复，收敛通常要 5–10 个周期，前几个周期误差反而可能变大。三者都不是“免费的性能”。

94.11.5 本章结论的适用条件

本章所有数字都建立在一组明确的假设上，越界使用要打折扣。

线性假设。 模型里没有摩擦、没有齿隙、没有饱和（电流限幅 12 A 从未触发）。真实轴在低速段必然偏离。

单轴假设。 三环只描述一根轴。多轴联动时，轮廓误差还取决于轴间同步，§94.1 的 2.355 mm 是“两轴带宽相同”的理想情况；实际两轴带宽差 10%，圆会变成椭圆，误差随角度变化。

刚体加双惯量假设。 机械被简化为“两个惯量加一根弹簧”。真实的传动链有分布质量、多个模态，高阶谐振可能落在 300–500 Hz。本章的 183.8 Hz 只是第一阶。

确定性假设。 没有考虑温度、磨损、负载变化。这些慢变因素会让参数在几个月内漂移，需要定期复核。

传感器理想假设。 编码器被当作“真实位置加量化台阶”，没有考虑安装偏心、码盘刻线误差、联轴器回差。安装偏心会让读数里出现一圈一次的正弦分量，幅值等于偏心量除以码盘半径；本章 10 μrad 的分辨率配 1 mrad 的偏心，读数的“精度”就退到 1 mrad，分辨率再高也没用。

执行器理想假设。 电流环被当作一阶闭环，没有考虑 PWM 死区、管压降、母线电压跌落。死区在低速轻载时会造成几伏的电压误差，比本章 0.0665 V 的指令还大，表现为过零点的转矩死区。这一项在 §94.5.3 已经提过，它属于“线性模型之外”的误差，不能靠调增益解决。

六条假设划出了本章方法的适用范围：刚体或双惯量主导、单轴、精度在微米到几十微米、环境相对稳定的场合。超出这个范围，本章的公式仍可作为起点，但结论要重新验证。

把边界说清楚，比把方法说得好听更有用。读者拿着这一章的表格去调一台真实的轴，最可能遇到的不是公式错，而是机械不听话——那时候回到 §94.3 的七步、§94.10 的清单，逐项核对，比继续加增益有效。

最后一节把本章所有数字的来源交给读者：一份能跑通的仿真代码，改参数就能复现全部图表。

94.12 Python 仿真代码

本章全部数字来自 `book/code/ch094_sim.py`，全部插图来自 `book/code/ch094_figs.py`。两个脚本都不带命令行参数，参数集中在文件开头的常量区，改完直接运行。

```
"D:\Anaconda3\envs\py311\python.exe" code\ch094_sim.py
"D:\Anaconda3\envs\py311\python.exe" code\ch094_figs.py
```

94.12.1 三环的核心

下面这段是整章的骨架，删掉所有画图与打印之后剩下的部分。它用 `python-control` 的传递函数运算，把 §94.2 的四个式子逐行实现。

```
import numpy as np
import control as ct

# ---------------- 电机与机械参数（集中在这里改）
J_M, LOAD_RATIO = 1.2e-4, 3.0
J_TOTAL = J_M * (1.0 + LOAD_RATIO)
B_VISC, KT, KE = 2.0e-4, 0.105, 0.105
R_ARM, L_ARM = 0.5, 1.2e-3
KS, CSHAFT = 120.0, 0.02
W_CI = 2 * np.pi * 1000.0          # 电流环设计带宽
KP_V, TI_V, KP_P = 2.2, 0.008, 100.0


def current_loop(wc=W_CI):
    """零极点对消：Kp=L*wc, Ki=R*wc。"""
    kp, ki = L_ARM * wc, R_ARM * wc
    c = ct.tf([kp, ki], [1.0, 0.0])
    g = ct.tf([1.0], [L_ARM, R_ARM])
    return ct.feedback(c * g, 1), kp, ki


def velocity_pi(kp=KP_V, ti=TI_V):
    return ct.tf([kp * ti, kp], [ti, 0.0])


def velocity_plant(j=J_TOTAL):
    return ct.tf([KT], [j, B_VISC])


def three_loops(kp_p=KP_P):
    """自内向外串：电流闭环 -> 速度闭环 -> 位置闭环。"""
    ti, _, _ = current_loop()
    lv = velocity_pi() * ti * velocity_plant()
    tv = ct.feedback(lv, 1)
    lp = kp_p * tv * ct.tf([1.0], [1.0, 0.0])
    return ti, tv, ct.feedback(lp, 1), lv, lp


def two_mass(ratio, ks=KS, c=CSHAFT):
    """双惯量：电机转矩到电机转速。"""
    jm, jl = J_M, ratio * J_M
    num = [jl, c, ks]
    den = [jm * jl, c * (jm + jl), ks * (jm + jl), 0.0]
    return ct.tf(num, den)
```

这段代码只有 40 行，但它复现了 §94.4 的全部带宽与裕度、§94.6 的惯量比扫描、§94.7 的两张扫描表。改动 `LOAD_RATIO` 就是改惯量比，改动 `W_CI` 就是改电流环带宽，改动 `KP_P` 就是改位置刚度——三个旋钮对应三个常量。

读这段代码要抓住两条主线。第一条是 `three_loops` 里的三次 `feedback`：每一次闭合都把前一个环的输出当作对象，`lv`、`lp` 同时返回，是为了让调用者能直接算裕度而不必重新搭一遍。第二条是 `two_mass` 的分母最后那个 `0.0`：它代表积分器，少了它，对象就变成“转矩到加速度”，低频会趋于常数而不是趋于无穷。这两处是最容易抄错的地方，抄错了现象是“带宽算出来很大、裕度算不出来”。

代码里没有写死任何“应该得到的结果”。所有打印都从传递函数现算，所以改一个参数，带宽、裕度、超调会一起变，不会出现“表格是旧的、曲线是新的”这种不一致。这一点比代码本身更重要：正文里的每个数字都能追溯到一个表达式，而不是某次手工记录。

94.12.2 需要非线性仿真才能得到的三个数

线性传递函数算不出三样东西：前馈跟踪误差、负载扰动挠曲、采样周期的影响。原因分别是限幅与状态、非线性ODE、离散时间。脚本里对应三个专用函数，都用 `scipy.integrate.solve_ivp` 或逐拍循环实现：

- `track(mode)`：三环加限幅的时域仿真，`mode` 取 `"P"`、`"VF"`、`"VAF"`，给出 20.0000 / 0.1245 / 0.0583 mrad。
- `disturbance(td=0.05)`：负载转矩阶跃的非线性 ODE，给出 -0.8577 mrad 与 60.8 ms。
- `discrete_position(ts)`：位置环按 T_s 离散后逐拍迭代，给出谱半径与跟踪误差。

这三个函数不能用 `ct.step_response` 代替。原因是限幅、积分状态与采样都会破坏线性叠加，用线性工具算出来的结果会偏乐观——尤其 `disturbance`，线性模型给出的是一个永不恢复的恒定偏差，与实测的"先陷后回"完全不符。

第一个数值得单独解释，因为它是全章唯一一个可以手算验证的结果。纯位置反馈跟踪 2 rad/s 的匀速段，稳态误差等于速度除以位置增益，即 $v/K_{p\theta} = 2/100 = 20$ mrad，与脚本打印的 20.0000 mrad 完全一致。这不是巧合：位置环是 I 型系统，对速度（斜坡）输入有恒定的稳态误差，误差大小只由速度与位置增益决定。加上速度前馈后，速度指令由前馈直接给出，误差降到 0.1245 mrad，剩下的部分来自速度环的跟踪滞后；再加加速度前馈，误差 0.0583 mrad。三个数构成一条清晰的台阶，每一级对应"补上了哪一项"。

如果自己改代码后这个数不是 20.0000 而是别的值，先别怀疑前馈——检查 `KP_P` 是否被改过，或仿真时长是否足以进入匀速段。这两个原因占了抄错结果的九成。

94.12.3 脚本输出对照

运行 `ch094_sim.py`，控制台按九组打印。每一组都能在正文找到对应的节：

输出组	内容	对应章节
[1]	三环带宽、剪切频率、裕度	§94.4
[2]	电流环三组带宽对比	§94.4.1
[2b]	圆轮廓与电子齿轮	§94.1.3
[3]	三种前馈的跟踪误差	§94.9
[4]	采样周期扫描	§94.5
[5]	惯量比扫描	§94.6
[6]	双惯量谐振点	§94.4.5
[7]	位置增益扫描	§94.4.3
[8]	速度增益扫描	§94.7.4
[9]	负载扰动挠曲	§94.6.3

对照时不要只对"数对不对"，要对"趋势对不对"。带宽、裕度这类数会随参数连续变化，抄错一个符号或漏掉一个增益，往往不会让结果变成明显的荒谬值，而是让趋势反了：比如把速度环 PI 写成 PD，带宽仍算得出来，但惯量比从 1 加到 10 时相位裕度不再单调下降。趋势比绝对值更能暴露结构错误。

最省事的自查办法是只改一个常量跑一遍。把 `LOAD_RATIO` 从 3 改成 10，表 94-5 的对应行应当整体跳到：相位裕度 51.80°、超调 15.53%、调节时间 0.1203 s。三个数同时对上，说明对象、控制器、裕度计算三处都对；只对上其中一两个，问题就在没对上的那一环。

94.12.4 改参数的三个建议

第一，改 `LOAD_RATIO` 之前先看 `KS`。惯量比与刚度共同决定反谐振，只改一个，结论可能反过来。第二，改 `W_CI` 之后必须重算 `KP_V` 与 `KP_P`，否则三环带宽比会变，原先的“内环当 1”不再成立。第三，`I_SAT` 不要为了“让曲线好看”而调大——它的作用是暴露饱和，调大等于把问题藏起来。

复现正文数字的步骤可以固定成四步。第一步，原样运行 `ch094_sim.py`，把九组输出与表 94-3 至表 94-9 对照，确认环境一致。第二步，只改 `LOAD_RATIO=10`，观察 §94.6 的突变是否出现（超调 15.53%、调节时间 0.1203 s），这是对模型最灵敏的一次检验。第三步，只改 `W_CI` 到 500 Hz，看电流环上升时间是否变成 0.699 ms，同时看速度环裕度掉了多少——这一步验证“内环拖累外环”的定性结论。第四步，把 `KP_P` 从 100 改到 200，确认带宽 60.6 Hz、超调 2.26% 两个数同时出现。四步做完，读者对本章的每个数字都有了手感，再去调真机就不会照搬表格。

四步覆盖了本章全部三种行为：参数缩放（惯量比）、带宽传递（电流环）、边界迁移（位置增益）。只要这三种行为都对得上，其余数字就只是同一组公式在不同取值下的结果，不必逐个背。

代码与正文的对应关系到此闭环。读者若只跑一个脚本，建议跑 `ch094_sim.py`：它不打图，只打数，二十行输出就能把本章的骨架过一遍。

第95章 伺服电机与控制

第 94 章把运动控制拆成“电流—速度—位置”三个环，重点在环与环之间的带宽分配。这一章把镜头推到最里面：电流环里流过的到底是什么，转矩是怎么从电流变出来的，为什么同样 5 A 电流在不同电机上给出完全不同的转矩，以及一台 750 W 的伺服电机从选型到整定要经过哪些定量判断。

本章的主线是一台 750 W 永磁同步伺服电机：4 对极、永磁磁链 0.0716 Wb、定子电阻 0.35 Ω、电感 2.5 mH、母线 300 V、开关频率 20 kHz。它的转矩常数 0.4296 N·m/A，额定电流 5.3 A 对应 2.277 N·m，3000 r/min 时输出 715 W。步进电机与感应电机作为对照，用同一套口径算，读者可以看清“为什么这台设备选伺服而不是步进”。

95.1 动机与直观理解

95.1.1 一台贴片机的 Z 轴

贴片机的 Z 轴要在 0.35 s 内完成“下降 50 mm—吸嘴吸取—抬起 50 mm—横移”的一整个节拍。下降段留给电机的时间约 0.12 s，行程 50 mm，按梯形速度规划，峰值速度约 0.5 m/s、加速度约 20 m/s²。丝杠导程 10 mm，0.5 m/s 对应 50 r/s，也就是 3000 r/min——正好是这台电机的额定转速。

负载 1.2 kg，加速力 $1.2 \times 20 = 24$ N，加上重力 11.8 N，合计 35.8 N。丝杠效率按 0.9 计，折算到丝杠的转矩是 $35.8 \times 0.01/(2\pi \times 0.9) = 0.0633$ N·m。丝杠自身惯量按 3×10^{-5} kg·m² 估，角加速度 $20/0.01 \times 2\pi = 12566$ rad/s²，需要 0.377 N·m。两项相加约 0.44 N·m，再留 1.5 倍余量，电机额定转矩要不低于 0.66 N·m。

按这个数，400 W 电机（额定 1.27 N·m）就够，为什么现场普遍用 750 W？因为还有三笔账没算：联轴器与抱闸的惯量、导轨摩擦在低速段的 Stribeck 峰、以及“节拍不能每次都跑满”的工程余量。三笔账加起来，实际选型往往比理论值大一档。这也是本章反复要讲的：**选型算的是下限，装机选的是余量。**

95.1.2 与过程控制的分工

过程控制关心的是“一个慢变量稳定在设定值”，时间尺度是秒到小时；伺服控制关心的是“一个快变量精确跟随一条轨迹”，时间尺度是毫秒到微秒。两者的对象不同，评价指标也不同。

过程控制的典型指标是超调、调节时间、稳态误差；伺服还要多三项：**跟随误差**（运动中落后多少）、**转矩脉动**（电流里有没有周期性成分）、**低速平稳性**（1 r/min 时是否爬行）。后三项在阶跃响应里根本看不出来，却是产线上最先暴露的问题。一台伺服“阶跃很漂亮、低速一抖一抖”，多半是电流环的量化、死区或编码器分辨率在作怪，而不是位置增益没调好。

95.1.3 四类电机的分工

图 95-1 把四类电机的转矩—转速包络画在同一张图上，各自按额定转矩归一。

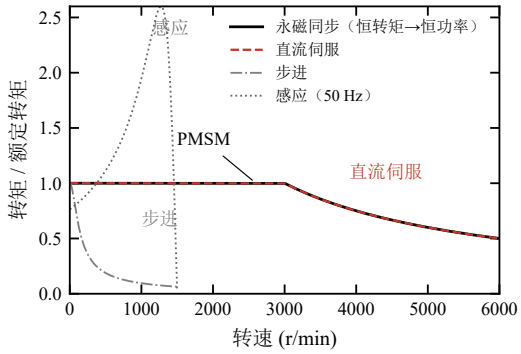


图 95-1　四类电机的转矩—转速包络

永磁同步电机（PMSM）在额定转速以下输出恒转矩，超过额定转速后进入恒功率区，转矩按 $1/n$ 下降。这条“拐点”由母线电压决定：反电动势随转速上升，当它顶到逆变器能输出的上限，就没有余量再推电流了。本章电机的拐点在 3000 r/min 附近，靠弱磁还能把转速推到 12126 r/min，但转矩只剩零头。

直流伺服的结构最简单：励磁与电枢解耦，转矩正比于电枢电流，机械特性是一条略微下倾的直线。它在低速大转矩场合仍有一席之地，但电刷磨损、换向火花、转子惯量大三项缺点，让它在通用伺服市场基本被 PMSM 取代。

步进电机的特点是“开环也能定位”：驱动器按脉冲数走，不需要编码器。代价是转矩随步进率迅速下降，本章这台 1.8°、保持转矩 1.2 N·m 的步进电机，1000 步/s 时只剩 0.364 N·m（30.3%），5000 步/s 时只剩 0.076 N·m（6.4%）。它适合“低速、小惯量、成本敏感”的场合，比如 3D 打印机的送丝轴。

感应电机便宜、耐造、能高速，但转差让转速随负载变化，低速效率低，矢量控制又需要准确的转子参数。它在风机、泵类“平方转矩”负载上是主力，在需要精确位置的伺服轴上很少见。

四类电机的定性对比见表 95-1。**这张表只看“方向”，不看“数值”；带数的版本在 §95.11.1（表 95-11），两者不要混用。**

项目	直流伺服	**PMSM**	步进	感应
转矩密度	中	高	低	中
效率（额定点）	75%–85%	90%–95%	40%–70%	85%–92%
低速平稳性	好	好（正弦驱动）	差（有共振）	中
需要编码器	是	是（或霍尔）	否	是（矢量控制）
控制复杂度	低	高（dq、弱磁）	低	高（转子磁链观测）
典型场合	老设备改造	通用伺服	低成本定位	风机、泵

95.1.4 一句话核心与本章路线

一句话：**伺服电机的控制，本质是把“要多少转矩”翻译成“给多少电流、加在哪个方向”。** 直流电机只需要一个方向，PMSM 需要两个正交方向（d 与 q），步进电机需要按节拍切换方向，感应电机还要先算出转子磁链的方向。所有复杂的变换，都是为了把“方向”这件事说清楚。

本章路线与第 94 章一致：§95.2 写方程，§95.3 给选型与设计步骤，§95.4 在连续域把电流环、解耦、弱磁算清楚，§95.5 补离散与 PWM 的损失，§95.6 讨论稳定边界与转矩脉动，§95.7–§95.8 讲参数与整定，§95.9 给优化手段，§95.10 落地，§95.11 划边界，§95.12 交代码。

95.1.5 三个常见误解

误解一：K_t 与 K_e 数值相等。 在 SI 单位下，直流电机确实有 $K_t = K_e$；但 PMSM 的转矩常数按 dq 峰值口径定义是 $1.5p\lambda$，反电动势常数按相电压峰值定义是 $p\lambda$，两者差 1.5 倍。本章的 0.4296 与 0.2864 就是这一对。若把铭牌上的 K_e（常以 V/krpm 线电压有效值给出）直接当 K_t 用，转矩会算错一倍以上。

误解二：电流环带宽越高越好。 电流环带宽受开关频率与计算延迟双重限制。20 kHz 开关、1 拍延迟时，1 kHz 电流环的相位裕度还有 62.89°；开关降到 10 kHz，同样 1 kHz 带宽的裕度只剩 35.07°，阶跃开始振铃。再往上提带宽，先撞到的是延迟，不是 PI 参数。

误解三：方波驱动与正弦驱动"差不多"。 反电动势为正弦的电机按六步方波供电，转矩峰峰脉动 14.03%（图 95-8）；正弦驱动在同工况下理论脉动为零。14% 的脉动在高速时被惯量滤掉，在低速时就是可见的抖动与噪声。用方波驱动正弦电机，省下的是算法，付出的是平稳性。

95.1.6 四层时间尺度

伺服系统里同时跑着四个数量级的时间尺度，搞清楚它们各自对应什么，比记住任何一个公式都有用。

最内层是开关周期。20 kHz 对应 50 μs，一个周期里桥臂要完成一次开通与关断，电流在上面叠出一个三角波。这一层不由控制算法决定，由功率器件决定。

第二层是电流环。带宽 1000 Hz 对应 1 ms 的时间常数，与开关周期差 20 倍。这一层决定了"电流指令变化后多久能跟上"，本章的上升时间是 0.350 ms。

第三层是速度环。带宽 76.78 Hz 对应 13.02 ms，与电流环差 13 倍。这一层决定"负载突变后多久能把速度拉回"，同时它也是机械谐振最容易落进来的频段。

第四层是位置环。带宽 13.55 Hz 对应 73.8 ms，闭环带宽 13.95 Hz 对应 71.7 ms。到这一层，时间尺度已经接近人的反应时间（约 100 ms），所以"机器动作看起来很快"这个感受，其实是位置环以下的几层在支撑。

四层的比值大致是 20、13、5.5，合起来从 50 μs 到 72 ms 跨了三个数量级。§95.3.4 要求的"相邻带宽比 5–10"，本质就是保证这四层不互相打架：比值太小，外层会感到内层的延迟；比值太大，外层慢得没有意义。而 0.35 s 的一整个节拍，是位置环周期的 4.7 倍——留给整定的余量，就藏在这个倍数里。

95.1.7 记号与口径

本章的符号统一如下，后文不再重复解释。

电机侧：p 是极对数（本章 4），λ 是永磁磁链（0.0716 Wb），R 是定子相电阻（0.35 Ω），L_d、L_q 是 dq 轴电感（各 2.5 mH），J 是转子加负载的等效惯量（1.5×10^{-4} kg·m²），B 是粘性阻尼（3×10^{-4} N·m·s/rad）。

转速有两种：机械角速度 ω_m（rad/s）与电角速度 $\omega_e = p\omega_m$。本章 $p = 4$，所以 3000 r/min（314.16 rad/s）对应 $\omega_e = 1256.6$ rad/s，也就是 200 Hz 的电频率。这个 4 倍关系贯穿全章：反电动势、耦合项、弱磁点都按 ω_e 算，而惯量、转矩、位置都按 ω_m 算，混用是本章最容易犯的错。

电流电压都是"峰值"口径：i_d、i_q 是 dq 轴电流峰值，v_d、v_q 是相电压峰值，额定 5.3 A、峰值 15 A 都是指单相电流的峰值，不是有效值。转矩式 (95-8) 里的 $3/2$ 正是为这个口径准备的。

时间一律用秒，频率用赫兹，角频率用 rad/s。带宽与时间常数的换算是 $f = 1/(2\pi\tau)$，不是 $f = 1/\tau$——这一条在写代码时最容易漏掉 2π，本章脚本里所有 `bandwidth_hz()` 都显式除了 2π。

95.1.8 一个反例：把伺服电机当变频器用

现场偶尔能看到一种省钱的尝试：拿一台通用变频器（V/f 控制）去带永磁同步电机，省掉伺服驱动器与编码器。它通常能转起来，但一加载就失步，一低速就抖，最后被判定为"电机不行"。

原因可以逐条对上本章的方程。V/f 控制不测转子位置，所以 Park 变换无从谈起，定子电流的方向是“猜”的；猜错 90°，i_d 与 i_q 就互换，转矩变成零，电流全用来发热。它也没有电流环，i_q 不受控，负载一变电流就冲到限流点。低速时反电动势只有几伏，V/f 给的电压误差（含死区 12 V）比反电动势还大，转子根本跟不上旋转磁场。

三处失效分别对应 §95.2.2（没有角度）、§95.4.1（没有电流环）、§95.6.3（死区淹没反电动势）。这也反过来说明伺服驱动器贵在哪：贵在它把“角度、电流、电压”三件事都做成了闭环，而不是贵在功率器件。

95.2 数学原理

95.2.1 直流伺服：一个电流，一个转矩

直流电机是理解一切伺服电机的起点，因为它的“方向”问题天然不存在：励磁磁场由定子（或永磁体）固定，电枢电流的方向就是转矩的方向，大小就是转矩的大小。它的动态只有三条方程。

电枢回路的电压平衡：

$$v_a = R_a i_a + L_a \frac{\mathrm{d}i_a}{\mathrm{d}t} + e, \qquad e = K_e \omega_m \tag{95-1}$$

电磁转矩与电流成正比：

$$T_e = K_t i_a \tag{95-2}$$

机械轴上的力矩平衡：

$$J \frac{\mathrm{d}\omega_m}{\mathrm{d}t} = T_e - T_L - B\omega_m \tag{95-3}$$

把 (95-1)(95-2) 代入 (95-3)，消去 i_a，得到电枢电压到转速的传递函数：

$$\frac{\Omega_m(s)}{V_a(s)} = \frac{K_t}{(L_a s + R_a)(J s + B) + K_t K_e} \tag{95-4}$$

这个式子里藏着伺服控制最重要的一个事实：分母中的 $K_t K_e$ 是“电气—机械”耦合项，它把两条时间尺度完全不同的回路绑在一起。电气时间常数 L_a/R_a 通常在毫秒量级，机械时间常数 J/B 通常在百毫秒到秒量级，两者差两个数量级以上。本章 PMSM 的 d 轴回路 $L/R = 2.5\times10^{-3}/0.35 = 7.143$ ms（对应 22.3 Hz），而 $J/B = 1.5\times10^{-4}/3\times10^{-4} = 0.5$ s，相差整整 70 倍。正因为有这 70 倍，才能把电流环与速度环分开设计、分开整定，第 94 章讲的“10 倍带宽分离”才有物理依据。若两者的时间常数接近（例如空心杯电机配极小惯量负载），三环就必须联立设计。

在 SI 单位制下，(95-1) 与 (95-2) 中的 K_t 与 K_e 数值相等，这不是巧合：两者都由同一份磁通与同一个几何尺寸决定，量纲上 K_t 是 N·m/A、K_e 是 V·s/rad，而 N·m/A = V·s/rad。这条等式只对直流电机成立；一旦进入三相世界，K_t 与 K_e 之间会多出一个由变换口径决定的系数，§95.2.3 专门讲这件事。

95.2.2 从三相到两相：Clarke 与 Park

PMSM 定子上有三套绕组，互差 120°。三相电流在空间上合成一个旋转的电流矢量，转矩由这个矢量与转子磁场的夹角决定。直接在三相坐标里控制，要同时处理三个互相耦合、且随时间正弦变化的量，非常别扭。两步变换把这件事化简。

第一步是 Clarke 变换，把三相压成两相静止坐标 $\alpha\beta$。采用幅值不变口径（三相正弦的峰值等于 $\alpha\beta$ 分量的峰值）：

$$i_\alpha = \frac{2}{3}\left(i_a - \frac{1}{2}i_b - \frac{1}{2}i_c\right), \qquad i_\beta = \frac{2}{3}\cdot\frac{\sqrt{3}}{2}\left(i_b - i_c\right) \tag{95-5}$$

第二步是 Park 变换，把静止坐标 $\alpha\beta$ 转到跟着转子一起转的 dq 坐标，转角用转子电角度 θ_e：

$$i_d = i_\alpha \cos\theta_e + i_\beta \sin\theta_e, \qquad i_q = -i_\alpha \sin\theta_e + i_\beta \cos\theta_e \tag{95-6}$$

这两步的意义不在数学，而在物理：**在跟着转子转的坐标系里，原本正弦变化的三相电流变成了两个直流量。**i_d 沿转子磁链方向，i_q 垂直于它。直流量可以用 PI 调节到零稳态误差，正弦量不行。整个矢量控制体系，就是为了把“控制正弦”换成“控制直流”这一件事。

口径要交代清楚。式 (95-5) 前面的 $2/3$ 是幅值不变变换，功率不守恒，实际功率要乘 $3/2$；如果改用功率不变变换（系数 $\sqrt{2/3}$），转矩式里的 $3/2$ 就消失，但电流的物理含义变成有效值。两套口径都对，混用就错。本章统一用幅值不变口径：所有电流都是相电流峰值，额定 5.3 A、峰值 15 A 都是峰值，转矩式里保留 $3/2$。

95.2.3 PMSM 的 dq 方程与两套常数口径

在 dq 坐标下，PMSM 的电压方程是：

$$v_d = R i_d + L_d \frac{\mathrm{d}i_d}{\mathrm{d}t} - \omega_e L_q i_q, \qquad v_q = R i_q + L_q \frac{\mathrm{d}i_q}{\mathrm{d}t} + \omega_e \left(L_d i_d + \lambda \right) \tag{95-7}$$

其中 $\omega_e = p\omega_m$ 是电角频率，p 是极对数，λ 是永磁磁链。式 (95-7) 里有两个“多余”的项：$-\omega_e L_q i_q$ 和 $+\omega_e L_d i_d$，它们不含微分、只与当前工作点有关，是坐标旋转带来的耦合。第三项 $\omega_e \lambda$ 是反电动势，随转速线性增长，它才是限制最高转速的元凶。

转矩方程：

$$T_e = \frac{3}{2} p \left[\lambda i_q + \left(L_d - L_q \right) i_d i_q \right] \tag{95-8}$$

本章电机是表贴式，$L_d = L_q = 2.5$ mH，方括号里第二项为零，转矩只由 i_q 决定：

$$T_e = K_t i_q, \qquad K_t = \frac{3}{2} p \lambda \tag{95-9}$$

代入铭牌数值，$K_t = 1.5 \times 4 \times 0.0716 = 0.4296$ N·m/A。额定电流 5.3 A 给出 2.277 N·m，乘 3000 r/min 得 715 W，与 750 W 铭牌对上（差额是效率与裕量）。

反电动势常数按相电压峰值定义：

$$K_e = p \lambda \tag{95-10}$$

数值是 $4 \times 0.0716 = 0.2864$ V·s/rad，3000 r/min 时 $0.2864 \times 314.16 = 89.975$ V。把 (95-9) 与 (95-10) 相除：

$$\frac{K_t}{K_e} = \frac{1.5 p \lambda}{p \lambda} = 1.5 \tag{95-11}$$

这就是 §95.1.5 第一个误解的根源。多出来的 1.5 不是误差，是幅值不变 Clarke 变换的 $2/3$ 与三相功率的 $3/2$ 相抵后剩下来的系数。工程上还有第三套口径：铭牌常给 K_e 的单位是 V/krpm，且按线电压有效值标注，折算关系是 $K_e^{(\text{铭牌})} = \sqrt{3/2} \cdot p\lambda \cdot (2\pi/60) \times 1000 \approx 36.7$ V/krpm。

同一个常数，三套口径分别是 0.2864 V·s/rad、36.7 V/krpm、0.4296 N·m/A，最大与最小相差 85 倍。选型时若拿铭牌的 36.7 直接当 K_t 代进 $T = K_t i$，算出的转矩会大两个数量级，电机“选小”、上机就过流。先问口径，再代公式，这是伺服选型的第一条纪律。

95.2.4 电压圆、电流圆与弱磁的由来

逆变器能输出的电压有上限。SVPWM 线性区的相电压峰值上限是母线电压除以 $\sqrt{3}$：

$$V_{\max} = \frac{V_{dc}}{\sqrt{3}} = \frac{300}{\sqrt{3}} = 173.205\ \text{V} \tag{95-12}$$

把 (95-7) 的稳态形式（微分项为零）代入这个上限，得到电压约束：

$$\left(R i_d - \omega_e L_q i_q\right)^2 + \left(R i_q + \omega_e \left(L_d i_d + \lambda\right)\right)^2 \le V_{\max}^2 \tag{95-13}$$

忽略电阻压降（高速时它只占百分之几），(95-13) 退化成 dq 平面上的一个圆：

$$\left(L_q i_q\right)^2 + \left(L_d i_d + \lambda\right)^2 \le \left(\frac{V_{\max}}{\omega_e}\right)^2 \tag{95-14}$$

把 $L_d i_d$ 与 $L_q i_q$ 当作横纵坐标，式 (95-14) 就是一个圆：圆心在 $(-\lambda,\,0)$，半径是 $V_{\max}/\omega_e$。转速升高，半径按 $1/\omega_e$ 收缩，圆整体缩小；i_q 能取的值越来越小，这就是恒功率区的来历。工程整定时为了留裕量，常把两个分量按幅值直接相加，用更保守的标量式代替 (95-14)：

$$\omega_e \left(L_q i_q\right) + \omega_e \left|\lambda + L_d i_d\right| \le V_{\max} \tag{95-15}$$

本章脚本采用的就是这一式，它比圆公式悲观一成多，正好对应逆变器的死区、管压降与母线跌落。

另一条约束来自电流本身，绕组温升与驱动器容量决定了电流矢量的模有上限：

$$i_d^2 + i_q^2 \le I_{\max}^2 \tag{95-16}$$

本章 $I_{\max} = 15$ A（峰值）。两条约束一联立，工作点就被夹在“电压圆”与“电流圆”的交集里。不弱磁（$i_d = 0$）时，把 $i_q = 5.3$ A 代入 (95-15) 解出基速 4873 r/min；超过这个转速还想出力，唯一的办法是让 i_d 取负值，把 $\lambda + L_d i_d$ 压低，替反电动势“腾地方”。这就是弱磁。

弱磁能做多深，取决于特征电流 λ / L_d：

$$i_{d,\text{char}} = \frac{\lambda}{L_d} = \frac{0.0716}{2.5 \times 10^{-3}} = 28.64\ \text{A} \tag{95-17}$$

特征电流 28.64 A 大于驱动器峰值 15 A，意味着这台电机拿不出足够的 i_d 把磁链完全抵消，最高转速一定有限。把 $i_d = -15$ A 代入 (95-15)，得 12126 r/min，此时 i_q 已被压到只剩零头，转矩不足额定的 1%。若换一台 L_d 更小的电机（特征电流 40 A 以上），可用弱磁深度会更浅；反之 L_d 大的内置式电机弱磁能力强，这也是内置式 PMSM 在宽调速场合更受欢迎的原因之一。

95.2.5 步进与感应：两种“非伺服”的模型

步进电机没有转子位置反馈，转矩由“电流建立得快不快”决定。每相是 RL 回路，换相时电流按 $1 - e^{-t/\tau}$ 上升，$\tau = L/R$。步进率越高，留给电流建立的时间越短，实际电流越小，转矩越低。一阶近似下矩频特性是：

$$T(f_e) = \frac{T_0}{\sqrt{1 + \left(2\pi f_e L / R\right)^2}} \tag{95-18}$$

式中 f_e 是单相电流的交变频率，不是脉冲频率。两相步进电机一个整步对应四分之一电周期，$f_e = f_{\text{step}}/4$。

本章步进电机 $T_0 = 1.2$ N·m、$L = 3$ mH、$R = 1.5\ \Omega$，时间常数 2 ms。200 步/s（$f_e = 50$ Hz）时还有 84.7% 的保持转矩，1000 步/s（250 Hz）掉到 30.3%（0.364 N·m），5000 步/s 只剩 6.4%。这条曲线解释了为什么步进电机“低速有劲、高速没劲”，也解释了细分驱动为什么能改善低速平稳性却不能提高高速转矩——细分只改变电流波形的台阶，不改变 RL 回路的时间常数。

感应电机的转矩—转差关系用 Kloss 公式描述：

$$T(s) = \frac{2T_{\max}}{s/s_m + s_m/s} \tag{95-19}$$

其中 s_m 是最大转矩对应的转差。本章 1.5 kW、4 极、50 Hz 的电机同步转速 1500 r/min，$T_{\max} = 2.6$ p.u.，$s_m = 0.15$；额定点 $s = 0.03$，代入得 $T = 1.000$ p.u.、转速 1455 r/min。转差是感应电机的“身份证”：转矩必须靠转差产生，转速就必然随负载漂移，开环无法定位。矢量控制通过观测转子磁链、把定子电流分解成励磁分量与转矩分量，才能在动态上模拟直流电机；代价是需要准确的转子电阻（它随温度漂移，直接决定磁链观测的精度）。

95.2.6 无刷直流：同一台电机，另一种反电动势

无刷直流（BLDC）与 PMSM 在结构上是同一类东西：永磁转子、三相定子、电子换向。区别只在反电动势的波形——BLDC 用集中绕组，反电动势做成梯形；PMSM 用分布绕组，反电动势做成正弦。波形决定了驱动方式：梯形配方波，正弦配正弦。

六步换相时，任一时刻只有两相串联导通，第三相悬空，每个电周期换相六次。把导通两相的电压方程相加：

$$v_{ab} = 2Ri + 2L\frac{\mathrm{d}i}{\mathrm{d}t} + (e_a - e_b) \tag{95-20}$$

梯形反电动势在 120° 导通区间内近似恒定，$e_a - e_b$ 也恒定，所以电流在恒定电压下按指数建立，转矩平台是平的。这是 BLDC 的理论优势：不需要 Park 变换、不需要高分辨率编码器，霍尔信号的六个扇区就够，驱动器成本可以明显压低。

但“梯形配方波”是配套条件。把方波电流加在正弦反电动势的电机上（现场常见的“用 BLDC 驱动器带 PMSM”），转矩不再恒定：逐电角度取三相反电动势的极差、按 $\omega_e p\lambda$ 归一，得到最小值 1.5000、最大值 1.7321、平均值 1.6540，峰峰脉动 14.03%（图 95-8）。这个数字接近理论值 $2 - \sqrt{3} = 13.4\%$，量级完全对得上。反过来把正弦电流加在梯形反电动势的电机上，脉动同样存在，只是形状不同。**驱动波形必须与反电动势波形匹配**，这条约束比任何算法优化都硬。

五类电机的模型与“方向”来源可以并成表 95-2。

电机	核心方程	转矩由什么决定	“方向”从哪来
直流伺服	(95-1)–(95-3)	电枢电流	电刷与换向器机械给定
PMSM	(95-7)(95-8)	i_q（表贴式）	编码器测角 + Park 变换
BLDC	(95-20)	导通两相的电流	霍尔信号的六个扇区
步进	(95-18)	绕组电流的建立程度	脉冲序列开环给定
感应	(95-19)	转差与磁链	转子磁链观测器

五条路径里，只有 PMSM 把“方向”显式地变成一个可测、可控的坐标角。直流伺服不需要方向，电刷替它做了决定；BLDC 的方向被霍尔信号量化成六个 60° 的扇区，扇区边界上必然有一次转矩跳变；步进电机的方向是开环假设，丢步了也不知道；感应电机的方向靠观测器估计，转子电阻一漂，估计角就跟着偏。只有 PMSM 的方向是“测出来再转过去”，误差可以做到编码器分辨率量级——本章 18 位编码器对应 2×10^{-5} rad，折算到 10 mm 导程的丝杠上是 0.032 μm。

这也解释了本章的篇幅分配：PMSM 占七成，其余四类作为对照。把 PMSM 的电流环、解耦、弱磁、三环整定讲透，换到另外几类电机上，只需替换对象模型与“方向”的获取方式，控制器的骨架不变。

95.2.7 传动链的折算与谐振

式 (95-21) 只把负载折成了转动惯量，实际的轴系还有丝杠、联轴器、抱闸。三者加起来才是电机真正要推的惯量，而它们之间的弹性又决定了系统能跑多快。

先算折算。§95.1.1 的 Z 轴，负载 1.2 kg，导程 10 mm：

$$J_L = m\left(\frac{p}{2\pi}\right)^2 = 1.2 \times \left(\frac{0.01}{6.2832}\right)^2 = 3.04 \times 10^{-6}\ \text{kg\cdotp m}^2$$

这个数只有电机转子惯量（1.5×10^{-4}）的 2%，比丝杠自身的 3×10^{-5} 还小一个数量级。所以“负载很重”这种直觉在丝杠传动上经常是错的：10 mm 导程把直线运动“放大”了 628 倍再平方，1.2 kg 折过来只剩几克·平方米的量级。惯量真正的大头是丝杠、联轴器与抱闸，它们直接坐在轴上，不打折扣。

再算谐振。把传动链看成“电机惯量 J_m 经过刚度 K 连到负载惯量 J_L”的二质量系统，反谐振频率是

$$f_n = \frac{1}{2\pi}\sqrt{\frac{K\,(J_m + J_L)}{J_m J_L}}$$

本章 $J_m = 1.5 \times 10^{-4}$、$J_L = 3.3 \times 10^{-5}$（丝杠加负载），若 $f_n = 300$ Hz，反推刚度 $K = 96$ N·m/rad。这个量级对滚珠丝杠是合理的：丝杠直径 16 mm、长度 300 mm 的扭转刚度大约在 100 N·m/rad 上下。

300 Hz 对三环意味着什么？位置环 13.55 Hz、速度环 76.78 Hz，都远低于它，所以 §95.3 的设计按刚体做没有问题。但如果换成同步带，刚度掉一个数量级到 9.6 N·m/rad，f_n 就落到 95 Hz，与速度环 76.78 Hz 只差 1.24 倍，速度环的相位裕度会被谐振峰直接吃掉，表现为“一加速就啸叫”。这时候只有两条路：加陷波器把 95 Hz 挖掉，或把速度环压到 30 Hz 以下。

95.2.8 磁饱和：模型失效的第一个地方

式 (95-7) 假设 L_d、L_q 是常数。真实电机的电感随电流变化：硅钢片在 15 A 时的磁导率可能只有 5 A 时的七成，等效电感跟着掉 20%–30%。

这个变化对电流环的影响是“变快”而不是“变慢”。对象极点 R/L 从 $0.35/2.5 \times 10^{-3} = 140$ rad/s 移到 $0.35/1.75 \times 10^{-3} = 200$ rad/s，而 PI 零点还固定在 140 rad/s。零点不再精确对消极点，闭环里留下一个小的时间常数，阶跃响应会出现一点点过冲——但穿越频率在 6283 rad/s，离这两个极点都远，相位裕度的变化不到 1°。

真正麻烦的是大电流下的转矩标定。若控制器按 $L = 2.5$ mH 算弱磁量，而实际电感只有 1.75 mH，同样的 i_d 产生的去磁效果只有预期的七成，弱磁点会偏高，高速段提前进入电压饱和。这类误差不能靠调增益补，只能做电感随电流的查表（第 94 章 §94.7 有标定方法）。

本章的时域仿真用常数电感，是因为工作点固定在额定附近；一旦要研究 15 A 峰值下的动态，就必须把 $L(i)$ 加进去。

95.2.9 从方程到框图：三个储能元件

把式 (95-7)(95-8) 与机械方程画成框图，会发现整条链路只有三个储能元件，而这正是三个环的物理依据。

第一个是电感。$L\frac{\mathrm{d}i}{\mathrm{d}t} = v - Ri - \omega_e(\cdots)$，电感储的是磁能，状态量是电流。控制它的是电压，所以最内层是电流环，执行器是 PWM。

第二个是转动惯量。$J\frac{\mathrm{d}\omega_m}{\mathrm{d}t} = T_e - T_L - B\omega_m$，惯量储的是动能，状态量是转速。控制它的是转矩（也就是 i_q），所以中间层是速度环，执行器是电流环。

第三个是位置本身。$\frac{\mathrm{d}\theta}{\mathrm{d}t} = \omega_m$，位置不是“储能”，而是一个纯积分，它没有自己的时间常数，只跟着速度走。控制它的是速度，所以最外层是位置环，执行器是速度环。

三个储能元件串成一条链，每个都有自己的时间尺度：电感 7.143 ms、惯量 0.5 s、积分无穷。三环的带宽 1 kHz、76.78 Hz、13.55 Hz，就是照着这三个尺度排的。反过来，若一台电机的 L/R 与 J/B 只差 3 倍（空心杯电机配大惯量负

载就是这种情况），三个尺度挤在一起，三环分离的近似失效，必须联立设计——那时就不是“三环”而是“一个三阶系统”了。

框图还解释了为什么电流环是唯一能“算出来”的：它的对象是一阶 RL，极点与零点可以直接对消；速度环的对象是“电流环闭环 × 一阶机械”，多了一个惯性；位置环的对象是“速度环闭环 × 积分”，多了一个 90° 相位。每往外一层，对象的阶数加一，可用的相位就少 90°，这就是位置环只能用 P、速度环只能用 PI 的根本原因。

95.3 设计步骤

伺服系统的设计可以拆成五步：定转矩与转速、定母线电压、定开关频率、定三环带宽、算增益并复核。每一步的输出是下一步的输入，顺序不能颠倒。现场最常见的错误是从最后一步开始——先把增益调上去，调不好再换电机，结果两头都改、两头都不对。

95.3.1 第一步：把负载折到电机轴

负载给的是力、质量、行程、节拍，电机要的是转矩、惯量、转速、RMS 转矩。前两个有闭式折算公式：

$$T_L = \frac{F\,p}{2\pi\eta}, \qquad J_L = m\left(\frac{p}{2\pi}\right)^2 \tag{95-21}$$

其中 F 是折算到丝杠的轴向力，p 是导程，η 是传动效率。§95.1.1 的 Z 轴算出 $T_L = 0.0633$ N·m、$J_L = 3\times10^{-5}$ kg·m²，加上加速转矩 0.377 N·m，合计 0.44 N·m，再乘 1.5 倍余量得 0.66 N·m。这是“最小值”，不是“选型值”。

惯量匹配单独看。本章电机转子惯量 1.5×10^{-4} kg·m²，负载 0.2 倍，属于很轻的配置；若负载惯量超过电机 10 倍，速度环剪切频率会从 76.78 Hz 掉到二十几赫兹（第 94 章 §94.6 有完整的扫描），这时要么换大惯量电机，要么加减速机。

转矩还要按 RMS 校验，不能按峰值。周期负载的等效转矩：

$$T_{\mathrm{rms}} = \sqrt{\frac{1}{T}\sum_{k} T_k^2\,\Delta t_k} \tag{95-22}$$

本章 2 s 节拍的负载算出 $T_{\mathrm{rms}} = 1.470$ N·m，是额定的 64.5%，对应稳态温升 25.0 K（一阶热模型，满负荷 60 K）。若按峰值 1.32 倍额定去选一台“刚好够”的电机，峰值不超，但温升会到 $60\times1.32^2 \approx 104.5$ K，直接顶到 F 级绝缘的允许温升（约 105 K）边缘。**选型看 RMS，峰值只用来校验驱动器电流与磁饱和。**

驱动器按峰值电流选。本章电机额定 5.3 A、峰值 15 A，倍率 $15/5.3 = 2.83$。通用伺服驱动器的过载规格通常是 3 倍额定持续 3 s，正好覆盖“0.12 s 加速段用满峰值、其余时间回落到额定”这种占空比。若把峰值电流标到 4 倍以上，驱动器体积与成本上一个台阶，而加速段其实用不到——这是“电机选大一号、驱动器选小一号”在成本上更划算的原因。

95.3.2 第二步：定母线电压

母线电压由最高转速处的反电动势决定。稳态下 dq 电压的模就是逆变器必须输出的相电压峰值：

$$V_{dc} \ge \sqrt{3}\,\sqrt{\left(\omega_e L_q i_q\right)^2 + \left[\omega_e\left(L_d i_d + \lambda\right) + R i_q\right]^2} \tag{95-23}$$

本章 4500 r/min 时根号内的合成电压是 139.079 V，乘 $\sqrt{3}$ 是 240.9 V，所以 300 V 母线够用；若最高转速只要 3000 r/min，只需 161.6 V，200 V 母线也能凑合。

但“够用”不等于“好用”，要看调制比留了多少余量。300 V 母线、$V_{\max} = 173.205$ V 时，3000 r/min 的调制比是 0.539，4500 r/min 是 0.803，4800 r/min 是 0.856。经验上把 0.85 当作连续工作的上限：超过它，SVPWM 开始丢脉冲，电

流谐波与转矩脉动同时上升，电流环的相位裕度也会被“电压饱和”这件事悄悄吃掉。所以这台电机的连续工作区标在 3000 r/min、短时到 4500 r/min，与铭牌一致。

反过来算：若母线选 200 V，上限只有 115.5 V，3000 r/min 的 93.3 V 已占八成，4500 r/min 的 139.1 V 根本出不来。母线不是“选大了浪费”，它是给加速段的反电动势与电网跌落留的余量。

95.3.3 第三步：定开关频率与电流环带宽

开关频率受两条相反的约束：纹波要小，延迟也要小。PWM 一个周期内的电流纹波峰峰值是

$$\Delta i = \frac{V_{dc} D\,(1-D)}{L f_{\mathrm{pwm}}} \tag{95-24}$$

D 是占空比，在 0.5 处取最大。20 kHz、$D=0.5$ 时 $\Delta i = 1.500$ A，是额定 5.3 A 的 28.3%；占空比降到 0.25 或 0.75 时降到 1.125 A（21.2%），0.1 与 0.9 时只有 0.540 A（10.2%）。开关频率翻到 40 kHz，全部减半。

延迟这一侧正相反。数字电流环有一个采样保持加一拍计算延迟，开关频率越低，这一拍占的时间越长、相位损失越大：20 kHz 时 1 kHz 电流环的相位裕度 62.89°，10 kHz 时只剩 35.07°，40 kHz 时升到 76.49%。两条约束一夹，20 kHz 是这台 750 W 驱动器的合理点：纹波 28.3% 在 30% 的经验线内，裕度 62.89° 也够。

电流环带宽按开关频率的 1/20 取，即 1 kHz。这个比例不是拍脑袋：再往上提，延迟造成的相位损失会让 PI 无论怎么整都超调；再往下压，速度环的可用带宽被拖低，位置环就更慢。

95.3.4 第四步：三环带宽与增益

带宽链从内到外是 1000 Hz、76.78 Hz、13.55 Hz，相邻比值 9.05 与 7.93，都落在 5–10 的区间里。这个比例关系不是设计出来的美观，而是“外环把内环当成 1”这个近似成立的条件：内环比外环快 10 倍时，它在穿越频率处的相位滞后不到 6°，外环可以按一阶对象整定；只快 3 倍时，滞后接近 20°，外环的裕度会被内环吃掉一半。

电流环用零极点对消，把 PI 的零点放在对象极点 R/L 上：

$$K_{pi} = L\omega_c, \qquad K_{ii} = R\omega_c \tag{95-25}$$

代入 $L = 2.5$ mH、$R = 0.35\,\Omega$、$\omega_c = 2\pi \times 1000$，得 $K_{pi} = 15.7080$ V/A、$K_{ii} = 2199.11$ V/(A·s)。对消之后，开环变成纯积分，闭环是一阶惯性，相位裕度 90.00°，带宽 1001.62 Hz，阶跃超调 0.00%，上升时间 0.350 ms、调节时间 0.623 ms。这是三环里唯一能做到“理论无超调”的一环，也是唯一能把参数直接算出来、不必试凑的一环。

速度环 PI 取 $K_{p\omega} = 0.15$ A·s/rad、$T_i = 4$ ms，零点在 39.8 Hz，剪切频率 76.78 Hz，相位裕度 58.45°，闭环带宽 110.65 Hz。速度环比电流环少 30° 裕度，原因是积分器本身贡献 -90°，而零点只能补回一部分；这 58.45° 是“速度环要抑制静差”必须付的代价。

位置环只用比例，$K_{p\theta} = 80$ s⁻¹。开环在 13.55 Hz 穿越，相位裕度 88.71°，增益裕度 30.61 dB，闭环带宽 13.95 Hz。位置阶跃的超调 0.00%、上升时间 26.15 ms、调节时间 50.41 ms。位置环之所以敢只用 P，是因为速度环已经把对象“驯服”成一阶：位置环看到的是一个积分器，P 控制天然稳定，加 I 反而会把相位裕度吃光。

95.3.5 第五步：复核清单

设计算完不等于设计完成，还要逐项复核。表 95-3 是本章用的清单，十一项，每项都有判据与本章取值。

检查项	判据	本章取值
常用工况调制比	≤ 0.85	0.539（3000 r/min）
短时调制比	≤ 0.95	0.803（4500 r/min）
电流纹波	$\le 30\%$ 额定	28.3%
电流环相位裕度	$\ge 45^\circ$	62.89°
速度环相位裕度	$\ge 45^\circ$	58.45°
位置环相位裕度	$\ge 60^\circ$	88.71°
位置环增益裕度	≥ 10 dB	30.61 dB
相邻带宽比	5–10	9.05 / 7.93
负载惯量比	≤ 5	0.2
编码器分辨率	$\le$ 定位精度的 1/3	2×10^{-5} rad
RMS 温升	$\le$ 绝缘允许	25.0 K

表 95-3 里最容易漏的是第一项与最后一项。调制比不够时，现象是“高速带不动”，调增益没用；温升超标时，现象是“跑两小时就报警”，看阶跃响应一切正常。两项都只能在设计阶段算出来，装机之后才发现，代价就是返工。

编码器那一行值得单独算一遍，因为它常被当成“越高越好”。按 5 μm 的定位精度、1/3 的判据，编码器折算到直线只要 1.67 μm，对应 1.05×10^{-3} rad——13 位就够了。本章选 18 位（2×10^{-5} rad），比精度要求细 50 倍，多出来的分辨率不是为定位，是为低速测速：1 r/min 时，1 ms 采样周期内位置只走 5.2 个脉冲，已接近“数得清”的下限；降到 0.1 r/min 就只剩 0.5 个脉冲，速度估计开始一跳一跳。低速爬行、转矩脉动这些现象，根子常在这里，而不是 PID 没调好。

清单过完，设计闭环。但清单只能保证“没有明显错误”，不能保证“性能最优”：为什么速度环不取 100 Hz、位置环不取 20 Hz，为什么弱磁要提前介入，这些取舍要在 §95.4 的连续域分析里才看得清。

95.3.6 一次完整的选型算例

把五步连起来走一遍，看每一步的数字怎么传到下一步。题目就是 §95.1.1 的贴片机 Z 轴：负载 1.2 kg、导程 10 mm、行程 50 mm、定位精度 5 μm。

第一步，算峰值。 加速段需要的转矩 0.377 N·m，加上重力与摩擦折算的 0.0633 N·m，合计 0.440 N·m；乘 1.5 倍余量，峰值需求 0.66 N·m。

第二步，算连续。 峰值只决定“能不能动”，连续才决定“能跑多久”。按 2 s 一个完整循环（加速 0.2 s、匀速 0.5 s、减速 0.2 s、停顿 1.1 s），峰值 3.0 N·m、匀速 1.2 N·m，代入式 (95-22) 得 $T_{\mathrm{rms}} = 1.470$ N·m。

第三步，定额定转矩。 温升按式 (95-34) 的平方关系，若要求长期温升不超过额定值的 80%，额定转矩要不低于 $1.470/0.8 = 1.84$ N·m。750 W 电机的 2.277 N·m 满足，400 W 的 1.27 N·m 不满足。

第四步，反证。 假设只看峰值去选 400 W 电机：峰值 3.0 N·m 靠 2.4 倍过载能顶住，但 RMS 1.470 落在 1.27 N·m 的额定之上，温升按 $60 \times (1.470/1.27)^2 = 80.4$ K 走，比本章设计值 25.0 K 高两倍多。跑两小时就会触到过热报警。这就是 §95.3.1 那句“选型看 RMS”的代价：选错不会立刻坏，会在第二个小时坏。

第五步，校验其余四项。 转速：3000 r/min 恰好是额定，调制比 0.539；峰值电流：3.0 N·m 只需 6.98 A，占 15 A 的一半不到；惯量比：$3.3 \times 10^{-5}/1.5 \times 10^{-4} = 0.22$；编码器：18 位对应 0.032 μm，远细于 5 μm 的要求。

五个数（0.66、1.470、1.84、2.277、0.539）串起来，就是这台设备选 750 W 的完整理由。读者若把负载换成 6 kg，只有第一步的 0.0633 与第二步的 RMS 会变，后面的逻辑不变——这正是本章把设计拆成五步而不是给一张经验表的原因。

算例里还有一个隐含判断：**为什么用 $K_t = 0.4296$ 而不是铭牌上的 K_e？** 因为转矩式用的是 dq 峰值口径。若误用 0.2864，同样的 2.277 N·m 会被算成 7.95 A，电机凭空“变小”三分之一，选型直接错一档。这是 §95.2.3 那三套口径在工程上第一次咬人的地方。

95.3.7 三处余量各自的来历

设计里出现了三个“不刚好”的数：转矩乘 1.5、调制比留到 0.85、温升只用 80%。三个数都不是拍脑袋，各自对应一种不确定性。

转矩的 1.5 倍对应摩擦的不确定。导轨的 Stribeck 峰在低速段可以到额定摩擦的两三倍，润滑状态变化又会让它漂；再加上电压跌落时转矩会掉（$T \propto i_q$，而 i_q 受电压限制），1.5 倍是最低的保险。§95.1.1 里“理论 0.66 N·m、实际选 2.277 N·m”的差距，主因不是这个 1.5，而是下面两条。

调制比的 0.85 对应电网波动。300 V 母线若跌 10% 到 270 V，线性区上限从 173.205 V 降到 155.88 V，4500 r/min 需要的 139.079 V 立刻占到 0.892，越过 0.85 的线。留 15% 的余量，本质是给电网留的。若不留，电网一跌，高速段就先丢转矩。

温升的 80% 对应环境温度与老化。额定温升 60 K、环境 40 ℃，绕组就是 100 ℃，已经贴着 F 级绝缘的 105 ℃ 长期允许点。取 80% 的 RMS，温升 38.4 K，绕组 78.4 ℃，留出 20 多度给夏季高温、电柜积灰、以及 §95.11.5 说的磁钢失磁。

三处余量加起来，让这台电机在纸面上“看起来选大了”：额定 2.277 N·m 只用到 1.470，母线 300 V 只用到 240.9 V，温升只到 25 K。这不是浪费，是把不确定性换成了确定的余量。**设计的功夫不在算得准，而在知道哪里算不准。**

95.4 连续情形

先把离散、饱和、死区统统拿掉，看连续域里三环各自长什么样。这一节用 `ch095_sim.py` 的 [1][2][3][5][7] 五组输出，每组数都能在脚本里找到出处。

95.4.1 电流环：两个一阶 RL 回路

解耦之后，d 轴与 q 轴的方程 (95-7) 变成两个完全相同的一阶回路：

$$L\frac{\mathrm{d}i_d}{\mathrm{d}t} + Ri_d = v'_d, \qquad L\frac{\mathrm{d}i_q}{\mathrm{d}t} + Ri_q = v'_q \tag{95-26}$$

对象是 $1/(Ls+R)$，极点 $R/L = 140$ rad/s，即 22.3 Hz。PI 的零点正好放在这个极点上（式 95-25），对消后开环只剩一个积分器 $K_{pi}/(Ls)$，闭环是一阶惯性，时间常数 $L/K_{pi} = 1/\omega_c = 0.159$ ms。电气时间常数 7.143 ms 与闭环时间常数 0.159 ms 相差 45 倍——电流环做的事情，就是把一个 22 Hz 的对象“压”成 1 kHz。

一阶闭环没有超调，这是理论保证，不是调出来的。阶跃读数：超调 0.00%、上升时间 0.350 ms、调节时间 0.623 ms。三个数里只有上升时间与 ω_c 直接相关，$t_r \approx 2.2/\omega_c = 0.350$ ms，与读数完全一致。

图 95-2 给出两个视角。左图是三相电流，三条正弦互差 120°，看上去很“交流”；右图是同一组数据经 Park 变换后的 i_d、i_q，i_d 贴在零线上，i_q 从 0 阶跃到 5 A 后保持平直。同一件事，在 abc 坐标里是三条曲线，在 dq 坐标里是两条直线——这就是 §95.2.2 那句“把控制正弦换成控制直流”的图示。

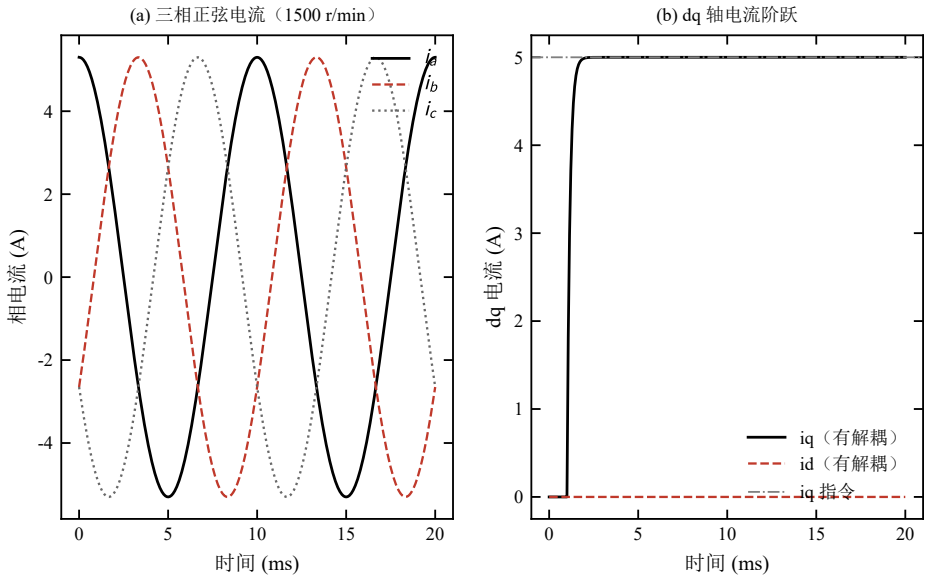


图 95-2　三相电流与 dq 阶跃响应

95.4.2 解耦：耦合项比电阻项大一个数量级

式 (95-7) 里那两个不含微分的项，量级随转速线性增长。$Ri_q = 1.855$ V 是常数，耦合项 $\omega_e L_q i_q$ 在 1000 r/min 时 5.550 V（3 倍），3000 r/min 时 16.650 V（9 倍），4800 r/min 时 26.641 V（14.4 倍）。也就是说，中高速时 d 轴回路里最大的“输入”不是自己的电压，而是 q 轴电流；q 轴回路里最大的输入不是自己的电压，而是反电动势加 d 轴电流。

前馈就是把这几个已知量算出来、直接加到电压指令上：

$$v_d^{*} = v_d^{\mathrm{PI}} - \omega_e L_q i_q, \qquad v_q^{*} = v_q^{\mathrm{PI}} + \omega_e \left(L_d i_d + \lambda \right) \tag{95-27}$$

图 95-3 对比了加与不加的效果。1500 r/min 时给 i_q 一个 5 A 阶跃：不加解耦，i_d 被拽出最大 0.2750 A 的偏离（额定 5.3 A 的 5.2%），i_q 本身只有 0.11% 的超调；加上解耦，i_d 的偏离是 0.000000 A，i_q 超调 0.00%。

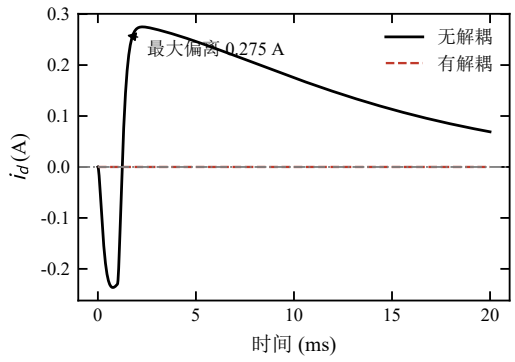


图 95-3　iq 阶跃时 id 的扰动：无解耦与有解耦

三个结论值得记下来。第一，耦合的主要受害者是“另一个轴”，不是自己：i_q 阶跃几乎不影响 i_q 的响应，却把 i_d 拽偏 5%。第二，表贴式电机 $L_d = L_q$，i_d 偏离不产生转矩，所以现场看到的现象不是抖动，而是“电流利用率下降”——多出来的 i_d 白占电流裕量，弱磁区里这 0.275 A 可能就是能不能再快 100 r/min 的差别；内置式电机则相反，i_d 偏离会通过磁阻转矩直接变成转矩脉动。第三，前馈不改变闭环极点，它只是把已知扰动提前抵消，所以加解耦不会让系统变得“更不稳”，但也救不了参数错误：λ 标定偏 10%，前馈就只抵消九成。

95.4.3 电压随转速：一条曲线的三段

把 (95-7) 的稳态解随转速画出来，就是图 95-4。三条曲线分别是反电动势 $\omega_e\lambda$、耦合项 $\omega_e L_q i_q$ 与合成电压的模，虚线是 SVPWM 的上限 173.205 V。

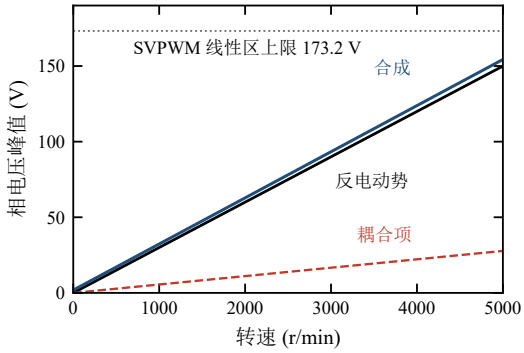


图 95-4　反电动势、耦合项与合成电压随转速的变化

低速段（0–1000 r/min）合成电压从 1.855 V 缓慢升到 32.327 V，电阻压降仍占主要部分，1000 r/min 的调制比只有 0.187；中速段反电动势开始主导，3000 r/min 时合成 93.328 V，其中反电动势 89.975 V 占 96.4%，调制比 0.539；再往上，4500 r/min 合成 139.079 V、调制比 0.803，4800 r/min 合成 148.229 V、调制比 0.856，已经贴到 0.85 的连续工作线。

这条曲线解释了现场一个常见困惑：“低速时电流环好好的，一上高速就发肉。”原因是高速时电压裕量被反电动势吃掉，同样的电流阶跃需要更大的电压斜率，而电压已经接近上限，PI 只能靠积分慢慢攒——带宽没变，可用行程变短了。这不是增益问题，是物理约束。

95.4.4 弱磁：能转不等于能用

按式 (95-15) 的保守电压式算，不弱磁时额定电流只能撑到 4873.0 r/min，峰值电流更差，只有 3788.3 r/min——电流越大，$L_q i_q$ 占的电压越多，留给反电动势的越少。这与直觉相反，但式子里写得很清楚：i_q 既是转矩的来源，也是电压的负担。

要再快，只能让 i_d 变负。表 95-4 是按电压圆与电流圆联立算出的工作点。

转速 (r/min)	i_d (A)	可用 i_q (A)	转矩 (N·m)	功率 (kW)
3000	0.000	15.000	6.444	2.024
4500	0.000	4.915	2.112	0.995
5000	0.000	0.843	0.362	0.190
6000	−1.074	0.000	0.000	0.000
8000	−7.965	0.000	0.000	0.000
10000	−12.100	0.000	0.000	0.000
12000	−14.857	0.000	0.000	0.000

表 95-4 读到第三行就该警觉：5000 r/min 时还剩 0.36 N·m，到 6000 r/min 直接归零。也就是说这台电机的“恒功率区”只有 3000–5000 这一小段，再往上就是“空转”——i_d 全部用来抵消磁链，一点不剩给转矩。图 95-5 把转矩与 i_d 画在同一张图上，两条线在 5000 r/min 附近交叉，之后转矩线贴地。

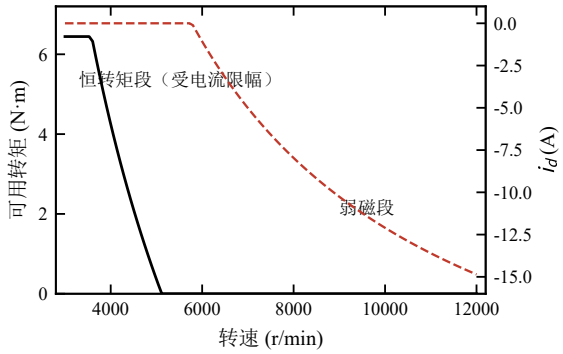


图 95-5　弱磁包络：转矩与 id 随转速的变化

所以“弱磁能把转速推到 12126 r/min”这句话必须加限定：那是把 i_d 打到 −15 A、i_q 归零时的电压极限，此时电机不输出任何功率。工程上标“最高转速”要标“还能出多少转矩”，标 5000 而不标 12126。这也是 §95.2.4 说的“特征电流 28.64 A 大于峰值 15 A”的直接后果：磁链消不干净，恒功率区就窄。

95.4.5 速度环与位置环

速度环的对象是“电流到转速”，$K_t/(Js+B)$，极点 $B/J = 2$ rad/s，几乎是一个纯积分器。PI 取 $K_{p\omega} = 0.15$、$T_i = 4$ ms，零点在 39.8 Hz，剪切频率 76.78 Hz，相位裕度 58.45°，闭环带宽 110.65 Hz。零点之所以放在 39.8 Hz 而不是更低，是因为再低就会把低频增益压下来，负载转矩造成的静差恢复变慢；再高则补偿不足，裕度掉得更快。

位置环看到的是速度环闭环再串一个积分器。速度环在 13.55 Hz 附近的相位滞后只有几度，所以位置环用纯 P 就能拿到 88.71° 的相位裕度与 30.61 dB 的增益裕度，阶跃超调 0.00%、上升 26.15 ms、调节 50.41 ms。若给位置环再加积分，开环在穿越频率附近就变成“两个积分器串联”，相位直奔 $-180°$，裕度会被吃光——这是位置环几乎不用 PI 的原因，也是“位置环积分”在很多驱动器里被做成可选、默认关闭的原因。

连续域的三环到此清楚：电流环靠对消拿 90°，速度环靠零点换静差，位置环靠内环的“驯服”只用一个增益。§95.5 把 ZOH、计算延迟与 PWM 加回来，看这三个漂亮的数字会掉多少。

95.4.6 负载扰动与速度刚度

传递函数算的是“指令跟得多快”，现场更关心“负载一变掉多少速”。这两个问题在同一个环里，答案却由不同参数决定。

对速度环来说，负载转矩是输入端的一个扰动。若暂时只看比例项，扰动到转速的传递关系在低频近似为

$$\frac{\Delta\omega}{T_L} \approx -\frac{1}{K_t K_{p\omega}}$$

本章 $K_t K_{p\omega} = 0.4296 \times 0.15 = 0.0644$ N·m·s/rad。这个数的物理含义是“速度刚度”：想让转速掉 1 rad/s，需要加 0.0644 N·m 的负载。反过来，1 N·m 的负载阶跃会造成 $1/0.0644 = 15.5$ rad/s 的瞬时跌落，折成转速是 148 r/min——接近额定转速的 5%。

这个跌落不会一直存在，积分项会把它拉回零。恢复的快慢由速度环带宽决定：76.78 Hz 对应 13 ms 的时间常数，所以 1 N·m 阶跃的恢复过程在几十毫秒内结束。§95.8.5 的整定记录里第 4 步“压 T_i 至零点 39.8 Hz，看负载扰动恢复无反弹”，做的就是这件事：T_i 太小，积分太猛，恢复会冲过头再回来；T_i 太大，恢复拖尾。

把这两个数放在一起看，就明白速度刚度与抗噪是一对矛盾。$K_{p\omega}$ 从 0.15 提到 0.3，刚度翻倍、跌落减半，但 §95.6.4 的量化噪声也从 30.0 mA 翻到 60.0 mA，同时更容易激发 §95.2.7 的机械谐振。所以“速度环调硬”不是免费的，它买的是刚度，付的是噪声与谐振余量。

95.4.7 三层前馈各补什么

§95.7.4 会算出纯反馈在 0.5 m/s 匀速段的跟随误差是 3.925 rad（折到丝杠上是 6.25 mm）。这个误差不是“没调好”，是一阶系统的固有滞后：位置环只有 P，误差必然与速度成正比。要消掉它，只能在反馈之外再给一条“预判”通路，这就是前馈。

前馈分三层，各补一个物理量。

速度前馈。 把指令速度直接加到速度环的输入端，位置环看到的就不再是“误差×增益”，而是“误差×增益 + 指令速度”。理想情况下跟随误差降到零。这一层最便宜，只需要对位置指令做一次差分。

加速度前馈。 速度环的输入是电流指令，加速时需要额外的转矩 $J\alpha$。本章 $J = 1.5 \times 10^{-4}$、$\alpha = 12566$ rad/s²，惯性转矩 1.885 N·m，折成电流 $1.885 / 0.4296 = 4.39$ A——是额定 5.3 A 的 83%。这一层若不给，加速段完全靠误差累积去“追”，误差峰值会比匀速段还大。

转矩（摩擦）前馈。 导轨的 Stribeck 摩擦在低速段有一个几十毫牛·米的峰，折算成电流约 0.1–0.3 A。它随位置与方向变化，不能用一个常数补，通常做成“速度—摩擦力查表”。

三层前馈的收益可以量化：只加速度前馈，匀速段误差从 3.925 rad 降到接近零，但加速段仍有 4.39 A 的电流缺口；再加加速度前馈，加速段误差也压下去；摩擦前馈只影响换向瞬间的那一小段。代价是每一层都需要“指令的二阶导”，而指令若由上位机以 1 ms 周期下发，二阶导的噪声很大，必须滤波——前馈给得越深，对指令质量的要求越高。这也是为什么第 94 章把前馈放在轨迹规划之后讲：没有平滑的轨迹，前馈只会把台阶放大。

95.4.8 电流环为什么必须比反电动势快

反电动势在 dq 方程里是一个扰动项：v_q 的方程里有 $\omega_e \lambda$，3000 r/min 时是 89.975 V。而同一转速下整个 v_q 才 91.830 V——也就是说，**扰动占了指令的 98%**。电流环要把 i_q 控在 5.3 A，实际是在“91.83 V 里扣掉 89.975 V 的扰动，剩下 1.855 V 用来产生电阻压降”。

这个“大数相减”的结构决定了电流环必须快。反电动势不是常数，它随转速变化；转速一变，扰动就变，电流环必须在扰动变化的一个周期内把它补回来。本章 3000 r/min 时电频率 200 Hz，扰动按 200 Hz 变化，电流环 1000 Hz 是它的 5 倍——按“内环至少比扰动快 5 倍”的经验，刚好够。若电流环只有 300 Hz，200 Hz 处的一阶闭环相位滞后已经超过 30°，补偿永远慢半拍，表现为“提速时电流跟不上、转矩掉一截”。

这也是解耦前馈存在的理由。§95.4.2 的对比实验里，无解耦时 i_d 最大偏离 0.2750 A，有解耦时是 0。0.2750 A 不算大，但它是“电流环拼命补扰动”留下的痕迹；这个痕迹随转速与 i_q 一起放大——4500 r/min 时耦合项已经到 24.976 V，是 3000 r/min 的 1.5 倍。前馈的作用不是“让控制更聪明”，而是“把已知的扰动提前扣掉，别让 PI 去追”。

95.5 离散情形

连续域里电流环有 90° 裕度，离散之后这个数会被三样东西啃掉：采样保持（ZOH）、计算延迟、PWM 更新时刻。三样都不大，加起来足以决定“1 kHz 电流环到底能不能用”。

95.5.1 一拍延迟值多少度

数字电流环的时序是固定的：本周期采样电流，算出占空比，下一个周期才生效。这一个周期的等待就是一个纯延迟 e^{-sT_s}，在剪切频率处折算成相位损失 $360^\circ f_c T_s$。

20 kHz 开关时 $T_s = 50$ μs，1 kHz 剪切频率处一拍延迟是 $360 \times 1000 \times 50 \times 10^{-6} = 18.0^\circ$。加上 ZOH 本身的半拍等效延迟 9.0°，总损失 27°，于是

$$\mathrm{PM} \approx 90^\circ - 540^\circ \cdot \frac{f_c}{f_{pwm}} \tag{95-28}$$

$f_c = 1$ kHz、$f_{pwm} = 20$ kHz 代入得 63°，与脚本的 62.89° 只差 0.11°。这个近似式的价值在于：它把“带宽能提到多高”变成一道除法——想让 PM 不低于 45°，f_c 只能到 $f_{pwm}/12$。

延迟每多一拍，裕度再掉 18°。表 95-5 是脚本的读数：0 拍 80.96°、1 拍 62.89°、2 拍 44.81°，增益裕度从无穷降到 10.06 dB、再到 5.88 dB。第二拍通常来自“占空比在周期末尾才装载”或“电流采样后还要做一次滤波”，现场排查振铃时，先数一数从采样到占空比生效之间有几个中断周期。

条件	剪切频率	相位裕度
20 kHz，0 拍延迟	1004.16 Hz	80.96°
20 kHz，1 拍延迟	1004.16 Hz	62.89°
20 kHz，2 拍延迟	1004.16 Hz	44.81°
10 kHz，1 拍延迟	1017.23 Hz	35.07°
40 kHz，1 拍延迟	1001.03 Hz	76.49°

95.5.2 开关频率：纹波与延迟的对撞

表 95-5 的后两行是同一个 1 kHz 电流环在 10 kHz 与 40 kHz 开关下的表现：10 kHz 只剩 35.07°，已经低于 45° 的工程底线，阶跃会明显振铃；40 kHz 升到 76.49°，几乎追平连续域。

为什么不是“开关频率越高越好”？因为另一半账在纹波上（式 95-24）。20 kHz、占空比 0.5 时纹波 1.500 A，是额定的 28.3%；40 kHz 减半到 0.750 A；10 kHz 则翻到 3.000 A，超过额定的一半。纹波大了，电流采样值里“平均值”与“瞬时值”的差别就大，PI 会把纹波当成误差去纠，输出电压跟着抖。所以 20 kHz 是这台驱动器的折中点：PM 62.89° 够用，纹波 28.3% 也在 30% 的线内。

图 95-6 把纹波随占空比的变化画了出来。三条曲线分别在 10 kHz、20 kHz、40 kHz，都是抛物线，峰值在 $D = 0.5$。读图时注意两点：一是纹波在 $D \to 0$ 与 $D \to 1$ 两端趋近于零，所以“轻载时电流很干净”是假象，那时电流本来就小；二是三条曲线是严格的 2 倍关系，说明提高开关频率对纹波是线性收益，对延迟是线性收益，但对开关损耗是线性代价——三者的平衡点就在 20 kHz 附近。

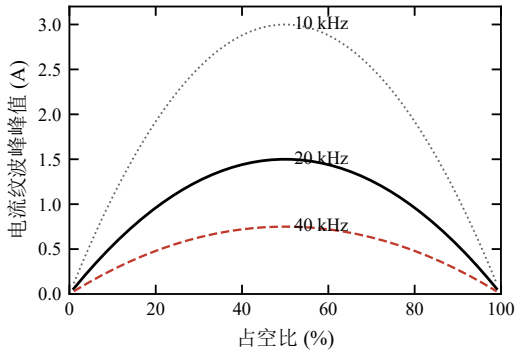


图 95-6　电流纹波随占空比的变化（10/20/40 kHz）

95.5.3 采样时刻：把纹波从反馈里拿掉

同样的纹波，采样时刻不同，进入反馈的量差很多。三角波载波的中心对齐 PWM，在三角波顶点采样，此刻电感电流的瞬时值正好等于本周期的平均值，纹波在反馈里被“自然抵消”；若在三角波过零点采样，采到的就是纹波的峰或谷，误差最大。

这条规则比任何滤波器都有效。现场若看到“电流波形很漂亮、但 i_q 反馈一直在抖”，先查采样触发点是否对齐载波顶点，再考虑加低通。加低通是在时间上平均，会把带宽一起削掉；对齐采样点是在相位上取巧，不花带宽。

95.5.4 离散 PI 的写法

连续 PI 离散化有两种写法，增量式更适合伺服：

$$u[k] = u[k-1] + K_p\left(e[k] - e[k-1]\right) + K_i T_s\, e[k] \tag{95-29}$$

位置式 $u[k] = K_p e[k] + K_i T_s \sum e$ 在积分饱和后要额外做抗饱和；增量式自带“饱和后停止累加”的性质，只要在输出限幅时同步冻结 $u[k-1]$ 即可。本章 $K_i T_s = 2199.11 \times 5 \times 10^{-5} = 0.110$ V/A，与 $K_p = 15.7080$ 相比不算小，定点实现时要注意 $K_i T_s$ 的量化误差——它直接决定稳态电流的纹波底噪。

还有一处容易忽略：K_p 与 K_i 是连续域算出来的，直接搬进代码会带上离散化本身的相位。本章的离散模型实测剪切频率 1004.16 Hz，与设计值 1 kHz 只差 0.4%，这个量级不必重新设计；若想把电流环推到 2 kHz 以上，就必须在离散域重新整定，而不是“连续域算完直接抄”。

95.5.5 三环的采样率

本章三个环的采样周期是 50 μs、100 μs、1 ms，对应 20 kHz、10 kHz、1 kHz。这个梯度不是随意的：电流环必须与 PWM 同步，否则采样点对不上载波；速度环取电流环的 1/2，够用且省算力；位置环取 1 ms，与 13.95 Hz 的带宽相比，每个周期有 71.7 个采样，远高于“每周期 20 个采样”的经验下限。

位置环若把周期压到 100 μs，带宽并不会提高——限制它的是速度环与机械惯量，不是采样率；反过来，若把位置环放到 10 ms，13.95 Hz 的带宽就只剩 7 个采样，离散化造成的相位损失会把 88.71° 的裕度吃到 60° 以下。采样率的选取原则是“比带宽高一个数量级”，不是“越高越好”。

95.5.6 离散之后还剩什么

把三样损失加回去，电流环从 90.00° 降到 62.89°。速度环与位置环的采样率低，但它们的带宽更低，按同样口径折算的相位损失只有三五度，所以脚本给的 58.45° 与 88.71° 仍然可用。三环的带宽比 9.05 与 7.93 也基本不变——延迟的相位损失与 f_c/f_{pwm} 成正比，内环的比值最大，受伤最重。这也说明：**离散化主要伤内环，外环的问题另有出处。** 下一节把饱和、死区、量化、机械谐振这些“外环的问题”逐个摆出来。

95.5.7 时序预算：50 μs 里要干完什么

数字控制有一个硬约束：本周期的计算必须在下一个 PWM 更新之前完成，否则就会多出一拍延迟，§95.5.1 的 18° 会变成 36°。

20 kHz 的周期是 50 μs。这段时间里要完成采样触发、ADC 转换、Clarke/Park 变换、两个 PI、反 Park、SVPWM 扇区判断与比较值装载。用 200 MHz 的定点 DSP 算，这些运算合计约几微秒，余量充足；用 72 MHz 的通用 MCU 算浮点，就可能吃掉 20–30 μs，逼近极限。这就是为什么伺服驱动器普遍用带浮点单元的专用芯片，而变频器可以用便宜的单片机——后者的开关频率通常只有 4–8 kHz，周期 125–250 μs，窗口宽得多。

开关频率的选择因此不只是“纹波与延迟的折中”，还受算力约束。若把本章的 20 kHz 降到 5 kHz，纹波翻四倍到 6.0 A（超过额定），按式 (95-28) 的裕度算，f_c 只能到 $5000/12 = 417$ Hz，按 9 倍分离，速度环只剩 46 Hz。三个后果串在一起，5 kHz 就不可行了。

反过来说，若算力足够、开关器件也支持，40 kHz 能把纹波减半、裕度提到 76.49°，代价是开关损耗翻倍、驱动器效率掉两三个点。本章选 20 kHz，是在“算力、损耗、纹波、延迟”四者之间取的平衡点。

95.5.8 离散化方法：ZOH、Tustin 与差分

连续域的 PI 要变成代码，中间隔着一次离散化。三种常用方法各有各的相位代价。

零阶保持（ZOH） 假设控制量在一个周期内不变，这正是 PWM 的实际行为，所以对功率级建模用它最准。脚本里 `ct.c2d(li, ts, method="zoh")` 就是这一路，算出的 62.89° 可以直接对照实测。

Tustin（双线性） 用梯形积分近似，相位特性在低频最接近连续域，但会把频率轴"压缩"：在 $f = f_{pwm}/10$ 处，频率畸变约 $(\omega T_s/2)^2/3$。本章 1 kHz、20 kHz 采样，$\omega T_s/2 = 0.157$，畸变 0.82%——可以忽略。带宽若提到 5 kHz，畸变就到 20%，Tustin 反而会把高频段算歪。

后向差分 最简单，$1/s \to T_s/(1 - z^{-1})$，但它额外引入 $\omega T_s/2$ 的相位滞后。1 kHz 处就是 9°，正好等于 §95.5.1 的"半拍"。所以用后向差分离散 PI，等于凭空多给半拍延迟：原本 62.89° 的裕度会掉到 53.9° 附近。

三条路的选择规则很朴素：**对象用 ZOH 建模，控制器用 Tustin 或直接按增量式写，别用前向差分**（前向差分会给相位超前，在低阻尼系统上会引起数值振铃）。本章脚本用 ZOH 是为了与硬件行为对齐，代码里的增量式 PI（式 95-29）本质上等价于后向差分，所以实际硬件的裕度会比 62.89° 略低几度——这也是设计时留 45° 底线、而不是卡着 62.89° 用的原因。

95.5.9 窄脉冲：死区的另一副面孔

死区 2 μs 在 50 μs 的周期里占 4%。这个比例本身不大，但它给占空比划了一条"不可用区"：当需要的脉宽短于 2 μs 时，脉冲会被死区完全吞掉，逆变器实际输出为零。

低速轻载正好落在这一区。$i_q = 1$ A 时电阻压降只有 0.35 V，对应桥臂占空比偏离 0.5 的量为 $0.35/300 = 0.117\%$，脉宽 $0.117\% \times 50 = 58$ ns——是死区的三十四分之一。也就是说，**在 1 A 这个电流上，逆变器根本无法"输出 0.35 V"，它只能在 0 V 与 ±12 V 之间跳。** 这就是 §95.6.3 的零电流钳位在波形上的样子。

两条对策。一是死区补偿（§95.9.1），把 12 V 的误差按电流方向反着补回去，让平均电压对；二是提高开关频率，把死区占周期的比例降下来——40 kHz 时 2 μs 占 2%，窄脉冲区缩小一半，但代价是开关损耗。两条都不能根治，因为死区是防止直通的必需项，只能"补"不能"去"。

95.6 稳定性与性能

线性稳定只是入场券。真正决定一台伺服"好不好用"的，是饱和、死区、量化与转矩脉动这四件事——它们都不出现在传递函数里，却决定了低速能不能爬、高速能不能带、停下来会不会抖。

95.6.1 三条边界互相挤压

表 95-3 给了三条独立判据：电流环 45°、速度环 45°、位置环 60° 与 10 dB。但三条边界不是独立的，它们通过带宽比互相牵制。

位置环增益往上提，穿越频率跟着提。$K_{p\theta}$ 从 80 提到 160，穿越频率近似翻倍到 27 Hz 附近，与速度环的 76.78 Hz 只差 2.8 倍，"外环把内环当 1"的近似失效，速度环的相位滞后开始计入位置环的开环，裕度会被明显吃掉。这时如果继续提位置增益来补快速性，先崩的是速度环，不是位置环。

反过来，速度环增益提上去，速度环自己的裕度先掉（58.45° 已经不宽裕），位置环反而更稳——因为内环更快，外环看到的对象更接近理想积分器。这就是现场那句"调位置先看速度"的来历：位置环的裕度是借来的，借的是速度环的。

95.6.2 饱和：大信号下的另一个系统

小信号分析假设电压、电流都够用。一旦不够，系统就切换成另一个系统，前面所有传递函数都不再成立。

电流饱和最先出现。表 95-4 第一行，3000 r/min、$i_q = 15$ A 时转矩 6.444 N·m，是额定的 2.83 倍——这是加速段的正常工况。饱和时速度环的 PI 还在算，积分项继续累积，等电流退饱和，累积的积分要很长时间才吐出来，表现为“冲过头再慢慢拉回”。解决办法是抗饱和：输出限幅时冻结积分，或按 $(u_{\text{sat}} - u)$ 反算回灌。

电压饱和更隐蔽。4500 r/min 调制比 0.803，4800 r/min 到 0.856，再往上就进入过调制区。此时电流环输出的电压指令已经超出逆变器能力，实际电压被“削顶”，i_d、i_q 都不再跟随指令。控制上表现为“电流环好像失灵了”，实际上是电压环（如果有）在接管。弱磁控制必须与电流环协调，否则两条回路会互相抢电压。

95.6.3 死区：低速的头号敌人

桥臂上下管的死区是为了防止直通，通常取 1–2 μs。这 2 μs 里输出电压不确定，等效成一个与电流方向相反的电压误差。按本章参数算：

$$\Delta v_{\text{dead}} = V_{dc}\, t_d\, f_{pwm} = 300 \times 2 \times 10^{-6} \times 20 \times 10^{3} = 12.00\ \text{V} \tag{95-30}$$

12 V 是什么概念？低速时整个电阻压降才 1.855 V，死区误差是它的 6.5 倍；占 3000 r/min 额定电压 93.328 V 的 12.9%。也就是说，低速轻载时逆变器“想输出 1.855 V，实际给出的是 1.855 ± 12 V”，符号还随电流方向翻转。结果就是电流过零处的平顶（零电流钳位）、转矩在过零附近塌陷、低速爬行。这类问题调 PID 永远调不好，只能做死区补偿或提高开关频率。

95.6.4 量化：30 mA 的底噪

编码器量化对三个环的影响完全不同。位置环直接用角度，2×10⁻⁵ rad 的量化在 $K_{p\theta} = 80$ 下只造成 1.60×10^{-3} rad/s 的速度指令跳变，折成电流 0.240 mA，可以忽略。

速度环靠位置差分测速，量化被采样周期放大：$2 \times 10^{-5} / 100 \times 10^{-6} = 0.200$ rad/s 的速度噪声，经 $K_{p\omega} = 0.15$ 折成 30.0 mA 的电流指令抖动，是额定的 0.57%。这个量级不大，但在低速时它相对有用信号的比例会急剧上升——1 r/min 对应 0.105 rad/s，噪声 0.200 rad/s 比信号还大。所以低速必须换测速方法：M/T 法、锁相环、或带模型的速度观测器，而不是把差分测速直接送进 PI。

95.6.5 能力边界：三类电机的转矩—转速

图 95-7 把步进电机的矩频特性画了出来，横轴是步进率，纵轴是转矩占保持转矩的比例。

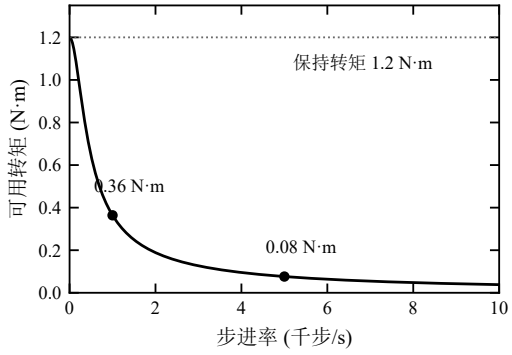


图 95-7　步进电机矩频特性

200 步/s 时还有 84.7%，500 步/s 降到 53.7%，1000 步/s 只剩 30.3%（0.364 N·m），5000 步/s 只有 6.4%。若把这条曲线与感应电机的 Kloss 曲线、PMSM 的弱磁包络放在一起看，会发现三种电机“掉转矩”的机理完全不同：步进是 RL 回路来不及建立电流，感应是最大转矩随频率平方反比下降（50 Hz 时 2.60 p.u.，75 Hz 降到 1.16，100 Hz 只剩 0.65），PMSM 是电压圆收缩。

对选型的意义很直接：步进的“保持转矩 1.2 N·m”只在零速成立，运行时必须按目标步进率查表降额；伺服的“额定 2.277 N·m”在 3000 r/min 以内都成立，这才是“额定”两个字的差别。

95.6.6 转矩脉动谱

伺服电机的转矩脉动有三个来源，频率各不相同。

第一是 PWM 纹波。20 kHz、$D = 0.5$ 时电流纹波 1.500 A，乘 $K_t = 0.4296$ 得 0.644 N·m，占额定 28.3%。它在 20 kHz，远高于任何机械环节的带宽，被转子惯量滤得几乎看不见。

第二是六步换相。正弦反电动势按方波供电时，转矩在 1.5000 与 1.7321 之间波动，平均 1.6540，峰峰 14.03%（图 95-8）。这个脉动的基频是电频率的 6 倍：3000 r/min、4 对极时电频率 200 Hz，脉动落在 1200 Hz，同样被滤掉；但 30 r/min 时电频率只有 2 Hz，脉动落到 12 Hz，正好落在速度环 76.78 Hz 的带内，直接变成可见的速度波动。**同一个 14%，高速时无所谓，低速时是致命伤。**

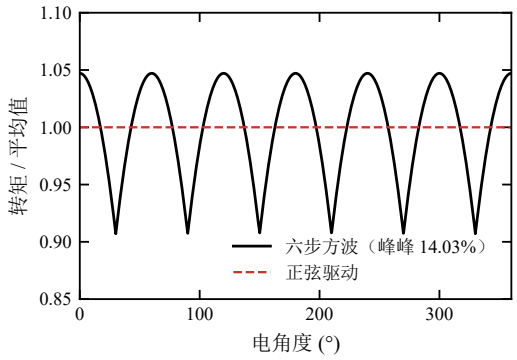


图 95-8　六步方波与正弦驱动的转矩波形

第三是齿槽转矩。它由定子槽与转子磁极的配合决定，频率是槽数与极数最小公倍数乘以转速，幅值通常 1%–5% 额定。它同样随转速线性移动，低速时落在带内，表现为“某个转速点特别抖”。抑制手段是斜槽、磁极优化、或电流注入补偿，都属于电机设计层面，控制器只能被动接受。

三源叠加，低速时的抖动主要来自第二、第三项，高速时的噪声主要来自第一项的边带。排查时先看抖动频率：与转速成正比的是齿槽或六步，固定在开关频率附近的是 PWM，与两者都无关的才回去查控制参数。

95.6.7 性能指标

把线性指标与非线性限制放在一起，本章这台伺服的性能可以写成一句话：13.95 Hz 带宽、零超调、26.15 ms 上升、50.41 ms 调节，低速受死区限制、高速受电压限制、中间段受电流限制。

按一阶近似核算跟踪能力：3 Hz 正弦的幅值衰减约 2.3%，相位滞后约 12°。这个精度对点位控制绰绰有余，对连续轨迹（比如 §95.1 提到的贴片机横移）就需要前馈——前馈的内容在第 94 章 §94.9 已经讲透，本章不重复。

性能的天花板不在控制器，而在电机与逆变器：死区 12 V、纹波 1.500 A、调制比 0.856、特征电流 28.64 A，四个数决定了这台机器“最多能做到多好”。控制器能做的，只是不要在这些约束之外再浪费裕度。

95.6.8 低速的三个门槛

“低速性能差”是一句笼统的抱怨，拆开来看是三个独立的门槛，各自有不同的物理原因与破解办法。

第一个门槛是测速。 编码器量化 2×10^{-5} rad，100 μs 差分一次得 0.200 rad/s 的速度噪声。1 r/min 对应 0.105 rad/s，比噪声还小；5 r/min 对应 0.524 rad/s，信噪比只有 2.6。所以 1–5 r/min 这个区间，速度环看到的基本是噪声，PI 会把噪声当误差去追，表现为“低速时电机哼、轴在微微来回”。破解办法不是调增益，而是换测速算法：M/T 法在低速段用长窗口计时间，锁相环把量化噪声整形到高频，观测器干脆用模型补。

第二个门槛是死区。 12 V 的电压误差与转速无关，而反电动势与转速成正比。3000 r/min 时反电动势 89.975 V，12 V 只占 13%；30 r/min 时反电动势 0.900 V，12 V 是它的 13 倍，方向还随电流过零翻转。所以 30 r/min 以下，逆变器“想给多少电压”基本失效，实际输出由死区主导，必须补偿（§95.9.1）。

第三个门槛是齿槽。 齿槽转矩 1%–5% 额定，本章就是 0.023–0.114 N·m。它在低速时是主要扰动，且频率随位置变化（不是随时间），速度环的积分对它无能为力——积分只能消常值，消不掉随位置周期变化的东西。破解办法是位置域前馈或电机设计（斜槽、分数槽）。

三个门槛的量级正好错开：测速在 1–5 r/min 起作用，死区在 30 r/min 以下，齿槽在整个低速段都存在但幅值最小。现场遇到“低速不好”，先按转速定位到哪一个门槛，再选对应的办法，比笼统地“降增益、加滤波”有效得多。

95.6.9 高速的两个天花板

高速端的问题比低速端简单，只有两条，而且都能用一个数说清。

第一条是电压。 SVPWM 线性区的相电压峰值上限 $V_{dc}/\sqrt{3} = 173.205$ V。按 0.85 的调制比上限，可用电压 147.2 V。本章 4500 r/min 需要 139.079 V（调制比 0.803），4800 r/min 需要 148.229 V（0.856），已经越线。所以“连续 3000、短时 4500”这个标法不是保守，是电压算出来的硬边界。

第二条是弱磁带来的耦合。 高速时 $\omega_e L_q i_q$ 这一项迅速变大：3000 r/min 时 16.650 V，4800 r/min 时 26.641 V，占可用电压的近两成。解耦前馈必须准确，否则这一项会直接吃掉电流环的相位裕度——因为它是一个随转速变化的“内部反馈”。更麻烦的是，弱磁时电压圆不允许 i_q 用满：表 95-4 里 4500 r/min 时 i_q 只剩 4.915 A，最大转矩 2.112 N·m；而 3000 r/min 时 15 A 能出 6.444 N·m。转矩常数 K_t 本身没变，变的是“允许多少 i_q”。**高速段的可用转矩不是被磁路削掉的，是被电压挤掉的，掉到三分之一就是电压圆的直接后果。**

两条天花板合起来，就是 §95.11.2 说的“转速轴边界”。跨过去的办法只有三个：提高母线电压（改硬件）、降低磁链（换电机）、或者接受转矩缩水（改工艺）。控制器在这件事上无能为力。

95.6.10 从阶跃响应反推裕度

调试现场没有频响分析仪时，一条阶跃曲线也能读出相位裕度，误差在几度以内。

经验关系是 $\mathrm{PM} \approx 100^\circ \zeta$。本章电流环离散后 PM 62.89°，对应 $\zeta = 0.63$，二阶近似的超调是

$$M_p = \exp\left(-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}\right) = \exp\left(-\frac{1.979}{0.777}\right) = 7.8\%$$

注意这个 7.8% 与脚本 [3] 段的“超调 0.00%”不矛盾：连续域对消后是纯一阶，没有超调；离散化的一拍延迟把闭环推成近似二阶，才有 7.8% 的振铃。现场看到的振铃频率约等于穿越频率，本章就是 1 kHz 附近的一个衰减振荡。

反过来用：若实测超调 20%，反推 $\zeta = 0.456$、PM≈45.6°——正好卡在 §95.3.5 的 45° 底线上，说明已经调到边缘，该回退了。若实测超调 40%，$\zeta = 0.28$、PM≈28°，这时候再看阶跃已经没有意义，系统在负载扰动下就会持续振荡。

这条经验式的适用条件是“二阶主导”。若阶跃曲线里出现两个明显不同的频率，说明有机械谐振参与，就不能再用它反推，得回到 §95.2.7 的二质量模型。

95.6.11 一台伺服里的频率地图

把前面各节的数按频率排成一列，会得到一张很有用的地图。从低到高：位置环闭环带宽 13.95 Hz、速度环 76.78 Hz（闭环 110.65 Hz）、二质量反谐振 300 Hz、电流环 1000 Hz（闭环 1001.62 Hz）、3000 r/min 时的六步脉动 1200 Hz、PWM 开关 20 kHz。

这张图的用法是“对号入座”：现场看到的现象是什么频率，就只可能是图上与它最近的几个环节出的问题。

- 0.5–10 Hz 的缓慢摆动：位置环与机械传动。位置环 13.95 Hz，指令里 3 Hz 的正弦已经被衰减 2.3%、滞后 12°，若这个频段出现过冲，先去查 $K_{p\theta}$ 与前馈，不要动电流环。
- 20–60 Hz 的“嗡嗡”：速度环在追测速噪声或齿槽。速度环 76.78 Hz，若振荡频率正好是它的一半左右，多半是积分时间太短、相位裕度不足，而不是机械问题。
- 200–400 Hz 的尖叫：机械反谐振。300 Hz 对应 $K = 96$ N·m/rad 的联轴器，提速或换刚性联轴器才能解决，调增益只会把它激发得更厉害。
- 800–1500 Hz 的振铃：电流环。离散后相位裕度 62.89°，阶跃振铃就落在 1 kHz 附近。同一段频带里还有六步脉动——它是 6 倍电频率，本章 4 对极下等于 $0.4\times$转速（Hz），3000 r/min 时 1200 Hz，正好与振铃重叠；要区分两者，只需改变转速看峰是否跟着移动。
- 20 kHz 的固定音：PWM 本身，属于正常。它旁边的边带（20 kHz ± 基波）才是需要看频谱确认的东西。

地图上有一处容易误判：**闭环带宽与开环剪切频率不是一回事。** 速度环开环剪切 76.78 Hz，闭环带宽 110.65 Hz，两者差 1.44 倍；位置环开环 13.55 Hz，闭环 13.95 Hz，只差 3%。差得越多，说明该环的阻尼越小、阶跃越容易超调。看到“闭环带宽比开环剪切高很多”这个信号，就该回头查相位裕度，而不是继续加增益。

这张地图也能反过来用：拿一台设备的振动频谱，把峰按频率标上去，落在哪一行就是哪一类问题。判断时先看“峰跟不跟转速走”——不动的峰一定是某个固定频率（电源、载波、外部激励）；跟着走的峰再看比例，与转速同倍的是动平衡，$0.4\times$转速的是六步换相，槽数倍的是齿槽。

还有一处：这张图上的所有频率都随参数移动。把 $K_{p\omega}$ 从 0.15 提到 0.30，速度环剪切频率翻倍到约 154 Hz，与 300 Hz 反谐振的间距从 3.9 倍缩到 1.95 倍，机械谐振就开始咬人了。所以“地图”不是一次画好就完事，改参数之后要重画，换电机、换负载之后同样要重画。

95.7 参数意义与作用

一台伺服驱动器上有几十个参数，真正需要理解的只有七类：两类物理量（K_t、K_e）、三个增益、两个限幅。其余参数要么是它们的换算，要么是给它们擦屁股的补偿。

95.7.1 参数分四层

第一层是电机的物理参数：极对数、磁链、电阻、电感、惯量、阻尼。它们由电机决定，现场不能改，只能“测准”。

第二层是功率级参数：母线电压、开关频率、最大电流。它们在选型时定下，改一个就要动硬件。

第三层是控制参数：K_{pi}、K_{ii}、$K_{p\omega}$、T_i、$K_{p\theta}$。这是现场唯一可以自由调的五个数。

第四层是补偿参数：解耦系数、死区补偿量、弱磁电流限幅、滤波器时间常数。它们不改变稳定性，只改变“误差有多大”。

现场调参出问题，九成是把第四层当第三层调：抖动明明是死区造成的，却去降 $K_{p\omega}$；高速带不动明明是电压不够，却去加 K_{ii}。分清层次，比记住数值重要。

95.7.2 五个增益往哪边走

表 95-6 把五个控制参数的作用方向列在一起。每一行都能在 §95.4、§95.6 找到对应的定量依据。

参数	调大	调小	主要代价
K_{pi}	电流环带宽升，响应快	响应慢，速度环被拖	受延迟限制，20 kHz 下 PM 掉到 45° 时到顶
K_{ii}	低速跟随好，静差消得快	稳态恢复慢	过大引起超调与积分饱和
$K_{p\omega}$	速度刚度高，抗负载扰动强	软，负载一变就掉速	噪声放大，易激发机械谐振
T_i	积分弱，超调小	积分强，静差消得快	过小引起低频振荡
$K_{p\theta}$	跟随误差小，到位快	到位慢，跟随误差大	过大吃掉速度环裕度，易撞机

表中“主要代价”一列是这一节的重点。五个参数没有一个是“越大越好”或“越小越好”，每一个的上限都由另一个环或另一个物理量决定。K_{pi} 的上限是延迟（62.89° 只剩 27° 的余量），$K_{p\omega}$ 的上限是机械谐振与噪声，$K_{p\theta}$ 的上限是速度环的相位。

95.7.3 K_t 决定“要多少电流”

$K_t = 0.4296$ N·m/A 有两层含义。第一层是换算：要 2.277 N·m 的额定转矩，就得给 5.3 A。第二层是损耗：同样转矩下，电流与 K_t 成反比，铜损与电流平方成正比，所以铜损与 K_t^2 成反比。

这条关系解释了电机设计里的一个基本取舍。磁链标定偏小 10%（比如磁钢高温退磁），K_t 从 0.4296 掉到 0.3866，同样转矩要 1.11 倍电流，铜损变成 1.23 倍，温升跟着涨 23%——本来 25.0 K 的工况变成 30.8 K，长期运行就可能触发过热保护。反过来，把 K_t 做大能降铜损，但反电动势同步变大，高速区被电压卡死（式 95-23）。所以“高 K_t 适合低速大转矩、低 K_t 适合高速”不是经验，是铜损与电压两条约束的直接结果。

95.7.4 三个带宽参数是“兑换”关系

把三个环的带宽写成一组数：1000 Hz、76.78 Hz、13.55 Hz。它们不是三个独立的旋钮，而是三次兑换。

第一次兑换在电流环内部：K_{pi} 换带宽，带宽换相位裕度。K_{pi} 每翻一倍，带宽翻倍，PM 按式 (95-28) 掉 27°。所以电流环的“最优”不是带宽最高，而是在 PM 与带宽之间取一个点，本章取 62.89°。

第二次兑换在速度环：带宽换抗扰。$K_{p\omega}$ 越大，负载转矩造成的速度跌落越小，但噪声与谐振风险越大。本章 $K_{p\omega} = 0.15$ 对应 76.78 Hz，若提到 0.3，带宽接近 150 Hz，与电流环只差 6.7 倍，三环分离开始失效。

第三次兑换在位置环：刚度换稳定。$K_{p\theta} = 80$ 时，匀速段的跟随误差是 $v/K_{p\theta}$：0.5 m/s 折成 314 rad/s，误差 $314/80 = 3.925$ rad，折到 10 mm 导程的丝杠上是 6.25 mm。这个数听上去大得离谱，但它是纯反馈的固有误差，不是调坏了——要消掉它只能靠前馈（第 94 章 §94.9）或提高 $K_{p\theta}$，而后者受速度环裕度限制。**位置环增益的意义不是“让误差变小”，而是“决定误差与速度的比例”。**

95.7.5 读一张三环开环图

图 95-9 把三个环的开环幅频画在一起，三条竖线分别是 1 kHz、76.78 Hz、13.55 Hz。

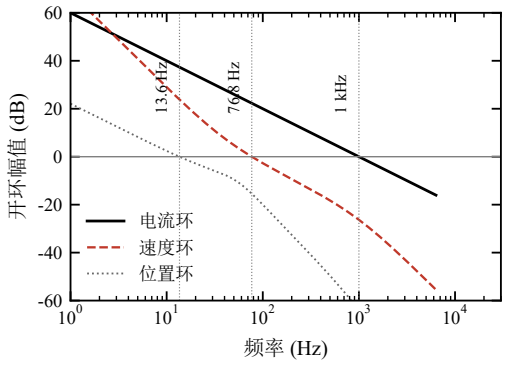


图 95-9　三环开环幅频特性与三个穿越频率

读这张图有三个要点。第一，三条曲线在穿越点附近的斜率都是 -20 dB/dec，说明三个环都被整成了“一阶主导”，没有额外的极点拖累；斜率若是 -40 dB/dec，说明有两个极点靠得太近，裕度会不够。第二，三条竖线的间距近似相等（9.05 与 7.93），这是“10 倍分离”的图形化。第三，最内侧那条曲线（位置环）在低频段的增益最高，意味着它对低频误差的抑制最强——位置环的职责就是“把稳态误差压到零附近”，代价是带宽只有 13.95 Hz。

图上还有一个容易被忽略的细节：速度环曲线在 39.8 Hz 处有一个折点，斜率从 -40 dB/dec 变到 -20 dB/dec，那是 PI 零点开始起作用；位置环曲线在 76.78 Hz 附近有一个小弯，那是速度环闭环带宽的边界。看懂这两个特征，就能从一张实测 Bode 图反推出参数有没有设错。

95.7.6 参数漂移：哪些会让系统变坏

温度是最大的漂移源。铜电阻按 0.4%/K 上升，温升 100 K 就是 40%。把 R 从 0.35 Ω 提到 0.49 Ω、PI 不重算，脚本的读数是：剪切频率 999.76 Hz、PM 90.51°、闭环带宽 991.67 Hz。带宽只掉了 1%，裕度反而略升。

这个“反直觉”的结果有清楚的解释：PI 零点固定在 140 rad/s，电阻升高让对象极点移到 196 rad/s，零点与极点失配；但穿越频率在 6283 rad/s，两者都在穿越点之下两个数量级，失配的影响到不了那里。换句话说，**电流环对电阻漂移不敏感，因为它工作在远高于对象极点的频段。** 真正受影响的是低速稳态：零点不再精确对消，闭环里留下一个慢极点，负载扰动后的恢复会拖尾。

磁链漂移就严重得多。λ 偏 10%，K_t 与 K_e 同步偏 10%，转矩标定、弱磁点、电压需求三处一起偏，而且无法靠调增益补偿——只能重新辨识。这也是永磁电机在高温工况下要留温度裕量的原因。

95.7.7 不能改的参数才是边界

七个参数里，极对数、磁链、电感、惯量四个现场改不了。它们决定了这台机器的“天赋”：极对数决定电频率与弱磁难度，磁链决定 K_t 与基速，电感决定电流纹波与动态，惯量决定加速能力。

调参能做的，只是在这个天赋之内把控制器调到“不浪费”：不浪费电压（调制比留 15%）、不浪费电流（纹波控制在 30% 以内）、不浪费相位（每环留 45° 以上）、不浪费温升（RMS 控制在 80% 以内）。四条“不浪费”合起来，就是 §95.3 那张清单。

95.7.8 驱动器面板上的名字

同一个参数，在不同品牌的驱动器上有不同的名字。表 95-6 用的是物理量，实际面板上看到的是另一套叫法，对照关系值得列清，否则抄参数时会张冠李戴。

位置环增益：面板上常叫“位置比例增益”，单位 1/s，就是本章的 $K_{p\theta}=80$。有的品牌把它做成“位置环带宽”（Hz），换算时乘 2π 再乘一个系数，容易差 6 倍。

速度环增益：常叫“速度比例增益”，单位 Hz 或 A·s/rad。本章 0.15 A·s/rad 若换成“Hz”口径，等于 $K_{p\omega}K_t/(2\pi J) = 0.0644/(2\pi \times 1.5\times10^{-4}) = 68.3$ Hz——这个数与速度环 76.78 Hz 的穿越频率接近，但不是同一个量，混用会让整定值差一成。

速度环积分时间：常叫“速度积分时间常数”，单位 ms，就是本章 $T_i = 4$ ms。也有品牌用“积分增益”，单位 1/s，与 T_i 互为倒数，$1/0.004 = 250$。

转矩指令滤波：常叫“转矩滤波器”或“电流指令低通”，单位 Hz。它不改变稳定性，只压高频噪声；本章若加，通常放在 2–3 kHz，远高于速度环，不影响 76.78 Hz 的相位。

前馈增益：速度前馈与加速度前馈各有一个百分比系数，100% 表示“完全按模型补”。现场常从 60%–70% 起调，因为指令的二阶导有噪声（§95.4.7）。

陷波滤波器：频率、深度、宽度三个参数，用来挖掉 §95.2.7 的 95–300 Hz 谐振峰。频率设错比不设更糟——挖错位置等于给系统加了一个相位滞后。

限幅：转矩限幅（本章 15 A 对应 6.444 N·m）、速度限幅（4500 r/min）、位置误差限幅（防撞）。三个限幅不是“保护”，它们是模型的一部分：限幅一动作，前面所有线性分析都失效（§95.6.2）。

95.7.9 三个参数怎么标定

K_t、J、L/R 这三个量，铭牌上通常没有，但它们决定了全部增益。现场有三种不需要专用设备的标定方法。

测 K_t：堵转加电流。 让电机停在某个电角度，$i_d = 0$、i_q 给一个已知值（比如 3 A），用杠杆加砝码测静态转矩。本章若测得 1.289 N·m，则 $K_t = 1.289/3 = 0.4297$，与设计值 0.4296 吻合。注意必须堵转且通直流，否则反电动势会混进来；堵转时间要短，3 A 下铜损 3.15 W，几十秒内温升可忽略。

测 J：加速实验。 给一个恒定的 i_q（比如 5 A），记录转速从 0 升到 1000 r/min 的时间。忽略摩擦时 $J = T/\alpha$，$T = 5 \times 0.4296 = 2.148$ N·m；若用时 16 ms，$\alpha = 6545$ rad/s²，$J = 3.28\times10^{-4}$ kg·m²——比铭牌的 1.5×10^{-4} 大了一倍，说明连了负载。这个方法测的是“电机加负载”的总惯量，正是控制器需要的那个数。

测 L/R：电流阶跃。 i_q 从 0 阶跃到 2 A，看电流上升到 63.2%（1.264 A）的时间，就是 L/R。本章理论值 7.143 ms；若实测 9 ms，说明电感偏大或有滤波延迟，按实测值重算 $K_{pi} = L\omega_c$ 与 $K_{ii} = R\omega_c$ 即可。

三个标定都做完，§95.4 的增益公式就有了可信的输入。若跳过标定直接用铭牌值，误差通常在 20%–50%，表现为“按公式算出来的增益一上机就振”——不是公式错，是输入错。

95.7.10 量纲核对：抓抄错的最快办法

五个增益的单位各不相同，这本身就是一道检查。把单位写出来，抄错的参数会立刻暴露。

K_{pi} 是 V/A：输入电流误差（A），输出是电压（V）。若填成 2199.11，那是以 K_{ii} 的值冒充，量纲差一个 1/s。

K_{ii} 是 V/(A·s)：乘上时间才变成电压。本章 2199.11 V/(A·s)，乘采样周期 50 μs 得 0.110 V/A，与 $K_{pi} = 15.7080$ 同量纲——这才对得上式 (95-29) 的增量式。

$K_{p\omega}$ 是 A·s/rad：输入转速误差（rad/s），输出电流指令（A）。0.15 乘上 15.5 rad/s 的跌落得 2.33 A，量级合理。

T_i 是秒：4 ms。若面板上填 250，那是积分增益（1/s），不是时间。

$K_{p\theta}$ 是 1/s：输入位置误差（rad），输出速度指令（rad/s）。80 乘 0.05 rad 得 4 rad/s，合理。

五个单位连起来是一条链：V/A → V/(A·s) → A·s/rad → s → 1/s。任何一环的单位不对，往下的计算都会错，而现象往往出现在最外层——“位置环怎么调都不到位”，真凶可能是电流环的 K_{pi} 填成了 K_{ii}。量纲核对花三十秒，能省掉半天。

95.8 调参方法与经验

设计阶段算出来的参数，到现场还要再整一遍——不是公式错了，而是铭牌参数与实物总有出入：电感随饱和下降、电阻随温度上升、编码器有偏心、负载惯量估不准。整定的任务，是在实物上把那几个不确定量"补"回来。

95.8.1 顺序：从内到外，不能跳

三环的整定顺序是电流环、速度环、位置环，一级一级往外。原因是外环把内环当成"1"，内环没整好，外环整出来的参数就是错的——你调位置增益时看到的超调，可能是速度环振铃透上来的；你调速度积分时看到的静差，可能是电流环根本没跟上。

反过来做的代价很具体。先调位置环、再回头改电流环，位置环的参数全部作废，必须重调；而现场往往只重调了位置环的 P，没重调速度环的 I，结果"调好了"的机器换一个负载就振。**整定顺序不是流程规定，是因果顺序。**

95.8.2 电流环：看阶跃，不看波形

电流环的整定对象是一条 0.35 ms 的阶跃。做法是给 i_q 一个方波指令（幅值取额定的 20%–50%，避免饱和），用电流探头或驱动器内置示波器看响应。

判据只有三条：上升时间接近 0.350 ms；超调接近 0；无持续振荡。三条都满足，说明 K_{pi}、K_{ii} 与实际的 L、R 对上了。

若不满足，按现象分诊。上升太慢：K_{pi} 偏小，或实际电感比铭牌大。超调明显：K_{ii} 偏大，或采样点没对齐载波顶点。有 20 kHz 的毛刺：那是纹波，不是控制问题，先查采样时刻。有几百赫兹的振荡：多半是电流滤波器的截止频率压得太低，把相位吃掉了。

扫频法更准，但需要注入设备。本章的 62.89° 就是扫频能测到的量；现场没有扫频仪时，用"阶跃 + 听声音"也能判断个八九不离十：电流环带宽正常时，电机发出的是均匀的高频嘶声；带宽不足时，声音发闷；带宽过高（延迟不够）时，会有周期性"吱吱"声，那是极限环。

95.8.3 速度环：先比例，后积分

速度环用"临界振荡法"起步：T_i 先放到很大（相当于关掉积分），逐渐加 $K_{p\omega}$ 直到速度反馈出现等幅振荡，记下这个临界增益，回退到它的 30%–50%，再把 T_i 从大到小压，直到阶跃出现 10%–20% 超调，最后回退一点。

本章的设计值 $K_{p\omega}=0.15$、$T_i=4$ ms 对应 76.78 Hz、58.45°，这是"算出来"的；现场整出来的值通常在这附近 ±30% 内浮动，偏差主要来自负载惯量估错。若整出来的 $K_{p\omega}$ 比设计值低一半以上，说明实际惯量是估算的两倍，应该回去复核机械参数，而不是接受这个"调出来的值"。

速度环最容易犯的错是把 T_i 设得太小。T_i 小、积分强，低速时确实"跟得紧"，但负载突变时积分会冲过头，表现为"掉速后反弹"。经验判据：T_i 对应的零点频率（$1/2\pi T_i$）不应高于速度环剪切频率的 1/2，本章 39.8 Hz 对 76.78 Hz，比值 0.52，正好在边界上。

95.8.4 位置环：一个增益，两个约束

位置环只有一个 $K_{p\theta}$。从 20 开始加，每步加 20%，看到阶跃出现 10%–20% 超调就回退 20%。本章 80 对应 13.55 Hz、零超调，是因为速度环的 58.45° 裕度足；若速度环整得偏软，位置环加到 60 就开始超调，这时不该继续加位置增益，而应回头加强速度环。

位置环还有两个隐性约束。第一是噪声：$K_{p\theta}$ 越大，编码器量化噪声被放大得越多，静止时电机会"嗡嗡"响。第二是机械：$K_{p\theta}$ 对应的加速度若超过传动链的承受能力，会在换向时听到"咔"的一声，那是齿隙在撞击。两个约束都不出现在 Bode 图上，只能靠听和摸。

95.8.5 整定记录

整定过程要留记录，否则换个人接手就得重来。表 95-7 是本章建议的记录格式。

步骤	动作	观察量	本章目标值
1	i_q 方波，幅值 2 A	上升时间、超调	0.350 ms、0.00%
2	电流环扫频	剪切、PM	1000 Hz、62.89°
3	$K_{p\omega}$ 加到临界再回退	速度阶跃超调	≤10%
4	压 T_i 至零点 39.8 Hz	负载扰动恢复	无反弹
5	速度环扫频	剪切、PM	76.78 Hz、58.45°
6	$K_{p\theta}$ 加到 10% 超调再回退	位置阶跃	0.00%、26.15 ms
7	匀速段测跟随误差	误差/速度	3.925 rad @ 0.5 m/s
8	满载运行 2 h	温升	25.0 K

表 95-7 的第 7 行值得单说：它测的是“误差与速度的比值”，不是误差本身。若实测比值明显大于 $1/K_{p\theta}$，说明有额外的延迟或摩擦，靠加增益补不回来。

95.8.6 六种现象与它们的真凶

现场排查可以先看现象，再看频率。

啸叫（1–3 kHz）：电流环带宽撞上机械谐振，或 $K_{p\omega}$ 太高激发了轴系扭振。降增益只能压住症状，根治要改机械或加陷波器（第 94 章 §94.6）。

发烫但电流不大：死区造成的电压误差（本章 12 V）在绕组里变成额外铜损，或电流纹波（1.500 A）在铁心里变成铁损。查死区补偿有没有开。

低速爬行：编码器量化噪声（0.200 rad/s）超过实际转速，或导轨的 Stribeck 摩擦。前者换测速方法，后者换润滑或加前馈。

定位过冲：速度环积分饱和。加速段电流打到 15 A，退饱和时积分还在吐，位置就冲过去了。开抗饱和。

换向抖动：编码器安装偏心或极对数设错。极对数错一个，电角度就错一倍，转矩会出现两倍频脉动。

高速掉速：电压饱和，调制比超过 0.85。这不是控制问题，是 §95.3.2 的母线选小了。

95.8.7 四种调参解决不了的问题

有些现象，无论怎么调参数都不会好，因为它们的根源不在控制器：

死区（12 V 的电压误差）、齿槽转矩（1%–5% 额定）、机械谐振（100–500 Hz 的轴系固有频率）、电压饱和（调制比 0.856）。四者的共同点是“控制器之外”——前两个在功率级与电机，第三个在机械，第四个在电源。

遇到这四类问题，正确的动作是改硬件或加补偿算法，而不是继续拧增益。硬拧的结果通常是：症状暂时压住，另一个指标悄悄变差，下次换工况再爆。§95.9 就讲这四类问题的工程解法。

95.8.8 什么时候必须重新整定

整定不是一次性的。四件事一变，原来的参数就不再是最优，甚至不再稳定。

换负载。 惯量是最敏感的量。本章 $J = 1.5 \times 10^{-4}$，若换成 10 倍负载，速度环的对象增益掉十倍，穿越频率从 76.78 Hz 掉到二十赫兹以下，位置环看到的“被驯服的对象”变慢，原来 80 的 $K_{p\theta}$ 就显得过激。反过来，负载去掉一半，

速度环变快，原来的参数偏保守，跟随误差白白大了一倍。

换电缆长度。 电机电缆从 5 m 加到 30 m，分布电容与电感都变，§95.10.7 会讲反射问题；更直接的影响是编码器信号衰减，位置反馈出现偶发跳变，等效于量化噪声变大。这时候要么换差分驱动能力更强的编码器，要么降速度环增益。

改开关频率。 20 kHz 改 10 kHz，按式 (95-28)，电流环裕度从 62.89° 掉到 35.07°，原来的 K_{pi} 必须跟着降，否则阶跃振铃。这条最容易被忽略，因为“只是改了个载波频率”，看起来与增益无关。

温度变了。 电阻从 0.35 Ω 漂到 0.49 Ω，电流环带宽只掉 1%（§95.7.6），可以不管；但速度环的低频增益会受影响，负载扰动后的恢复时间变长。若设备要在 −10 ℃ 到 50 ℃ 全温区工作，最好在冷态与热态各整一次，取折中。

四条里前两条最常见。工程上的做法是把“整定记录”（表 95-7）与“负载配置”绑在一起存档，换配置就换一套参数，而不是在现场重新试。

95.8.9 自整定能做什么，不能做什么

现代驱动器几乎都有“一键自整定”。它的原理不神秘：向电机注入一个激励（阶跃、正弦扫频或伪随机序列），测响应，辨识出惯量与谐振频率，再按固定规则算出增益。整个过程十分钟以内，比人工试凑快得多。

它能做对的部分：惯量辨识、速度环增益初值、谐振频率定位。本章的 $J = 1.5 \times 10^{-4}$，自整定通常能辨到 ±10% 以内；$f_n = 300$ Hz 的谐振峰也能从频谱上看出来，并自动设一个陷波器。

它做不对的部分有三类。第一，死区与齿槽这类“与电流方向、位置相关”的非线性，激励信号一平均就被抹平了，辨识结果里没有它们的位置。第二，饱和特性。自整定用的小信号激励不会触发限幅，算出来的增益在加速段可能一用就撞限幅。第三，负载的时变。自整定测的是“当时”的惯量，产线换产品之后就不准了。

所以自整定的正确用法是“给初值”，不是“给终值”。实践中的做法是：自整定跑完，把速度环增益回退 30%，再按 §95.8.2 到 §95.8.4 的顺序人工微调。回退 30% 的理由很直接：自整定追求的是“临界稳定附近的最快响应”，而工程要的是“十年不报警的余量”，两者差的就是这 30%。本章 58.45° 的速度环裕度、88.71° 的位置环裕度，都是回退之后的结果。

95.8.10 一台新机器的整定顺序

把 §95.8.1 到 §95.8.5 揉成一张时间表，新机器上电后的动作顺序是这样的。

第一步，静态检查（半小时）。确认相序、编码器方向、抱闸逻辑，手动盘车看位置计数是增是减。方向反了，后面所有整定都是负反馈变正反馈。

第二步，标定（半小时）。按 §95.7.9 测 K_t、J、L/R，把三个数填进驱动器。

第三步，电流环（一小时）。先给 2 A 方波看上升时间，理论 0.350 ms；再扫频核对 1 kHz、62.89°；最后加阶跃到 5 A，确认不饱和、不振铃。

第四步，速度环（一小时）。先只给比例，加到临界振荡再回退一半；再加积分，把零点压到 39.8 Hz；最后用 1 N·m 的负载扰动看恢复，不应有反弹。

第五步，位置环（半小时）。只给比例，从 20 起加，看 1 mm 阶跃的超调，到 5% 回退；核对跟随误差是否等于 $v/K_{p\theta}$。

第六步，连续跑（两小时）。满载走节拍，测外壳温升是否在 45 K 以内，听有没有随转速变化的异响。这一步最容易被省掉，也最容易暴露前五步埋下的问题。

六个步骤合计五个半小时。跳过任何一步，问题都会在第六步一起爆出来，而那时候已经很难判断是哪一环的错。

95.9 优化与改进

§95.8 末尾列的四种“调参解决不了”的问题，各有对应的工程手段。这一节按“收益/代价”的顺序讲五种：死区补偿、母线利用、MTPA、负载观测器、无位置传感器。

95.9.1 死区补偿：把 12 V 还回去

死区造成的电压误差有确定的符号：电流流出桥臂时少给一点，流入时多给一点。既然符号可知，就可以按电流方向反着补：

$$v_{\text{comp}} = V_{dc}\, t_d\, f_{pwm}\ \operatorname{sign}(i) \tag{95-31}$$

本章参数下补偿量是 12.00 V，与误差同幅反号。补偿之后，低速的电压指令从“1.855 ± 12 V”变成“1.855 ± 残差”，零电流钳位基本消失。

代价在两个地方。第一，电流过零时符号会抖，补偿量在 ±12 V 之间跳，反而引入新的抖动；工程上要在过零点附近做平滑过渡（按电流幅值线性衰减补偿量）。第二，死区时间本身随温度与驱动电阻变化，补偿量按标称值给，残差仍有几伏。所以死区补偿是“减少”而不是“消除”，剩余误差通常还有标称值的 20%–30%。

95.9.2 母线利用：SVPWM 比 SPWM 多 15.5%

同样的母线电压，不同的调制方式能输出的相电压峰值不同。SPWM 的正弦调制上限是 $V_{dc}/2 = 150$ V；SVPWM 把三次谐波注入后，上限提到 $V_{dc}/\sqrt{3} = 173.205$ V，多出 15.5%。本章从头到尾用的都是 SVPWM，调制比的分母就是 173.205。

再往上还有过调制区：从 173.205 V 一直推到六步方波的 $2V_{dc}/\pi = 190.986$ V，还能多 10.3%，但波形畸变、低次谐波急剧上升，只适合“短时加速”这类不计音质的场合。工程上把 0.85 作为连续上限、0.95 作为短时上限，就是把这段余量留出来。

95.9.3 MTPA：表贴式不需要，内置式必须做

最大转矩电流比（MTPA）解决的是“同样转矩，怎样用最小电流”。表贴式 $L_d = L_q$，转矩只由 i_q 决定，MTPA 的答案就是 $i_d = 0$——本章一直这么用，没有优化空间。

内置式 $L_d < L_q$，磁阻转矩项 $(L_d - L_q) i_d i_q$ 在 $i_d < 0$ 时为正，可以“白拿”一部分转矩。此时最优 i_d 满足

$$i_d = \frac{\lambda - \sqrt{\lambda^2 + 8\,(L_q - L_d)^2\, i_q^2}}{4\,(L_q - L_d)} \tag{95-32}$$

典型内置式电机（$L_q/L_d \approx 2$）在额定点能省 10%–15% 电流，换算成铜损是 20%–30%。代价是控制复杂度：i_d 不再恒为零，弱磁与 MTPA 要在同一张电流图上协调，参数一漂，最优点就偏。表贴式电机做 MTPA 是负优化——公式退化成 $i_d = 0$，白算一遍。

95.9.4 负载观测器：把扰动前馈掉

速度环抑制负载扰动靠积分，积分必然滞后。若能把负载转矩估出来、直接前馈成电流指令，扰动响应可以快一个数量级。观测器的做法是用“电流指令 - 加速度×惯量”估转矩：

$$\hat{T}_L = K_t i_q^{*} - J\frac{\mathrm{d}\hat{\omega}}{\mathrm{d}t} \tag{95-33}$$

估出来除以 K_t 就是前馈电流。本章 $J = 1.5 \times 10^{-4}$ kg·m²，加速度 12566 rad/s² 时惯性转矩 1.885 N·m，若这部分靠积分去“追”，按速度环 76.78 Hz 算要十几毫秒；前馈之后降到一两毫秒。

代价是噪声：微分 $\mathrm{d}\hat{\omega}/\mathrm{d}t$ 会把编码器量化噪声放大（§95.6.4 的 0.200 rad/s 经过微分变成很大的加速度噪声），必须加低通，而低通又吃掉前馈的快速性。所以负载观测器只在“惯量已知、编码器分辨率高”的场合好用。

95.9.5 无位置传感器：省掉编码器，丢掉低速

反电动势里含着转子位置：$\theta_e = \arctan\left(-e_\alpha / e_\beta\right)$。用观测器把反电动势估出来，就能反推角度，省掉编码器与它的线缆。

问题是反电动势与转速成正比。3000 r/min 时 89.975 V，100 r/min 时只有 3 V，10 r/min 时 0.3 V——比死区误差 12 V 小两个数量级，根本测不出来。所以无位置传感器控制的低速下限通常在额定转速的 5%–10%，且起动需要“开环强拖”或高频注入。风机、泵类不需要零速转矩，可以用；机床进给、机械手这类要零速保持的场合，仍然离不开编码器。

95.9.6 五种手段的取舍

五种手段的收益与代价可以并成表 95-8。

手段	主要收益	主要代价	适用条件
死区补偿	消除低速零电流钳位	过零抖动，残差 20%–30%	所有低速定位场合
SVPWM/过调制	电压利用率 +15.5%/+10.3%	过调制谐波大	母线偏紧或短时加速
MTPA	电流省 10%–15%	算法复杂，怕参数漂移	内置式电机
负载观测器	扰动响应快一个数量级	噪声放大，需低通	惯量已知、编码器分辨率高
无位置传感器	省编码器与线缆	低速失效	风机、泵类

表 95-8 的顺序是按“投入产出比”排的：死区补偿几乎零成本，收益立竿见影；无位置传感器省的是硬件，丢的是低速性能，只在特定行业划算。现场遇到问题时，从上往下试，通常在前两行就能解决大半。

这五种手段有一个共同前提：先把 §95.3 的设计做对。母线选小了去搞过调制、电流环没整好去加观测器，都是在错误的地基上盖楼。

95.9.7 过调制与弱磁的配合

§95.9.2 说过，SVPWM 比 SPWM 多 15.5% 的电压利用率，六步方波再比 SVPWM 多 10.3%（$2V_{dc}/\pi = 190.986$ V 对 $V_{dc}/\sqrt{3} = 173.205$ V）。这 10.3% 在高速段是实打实的转速。

对本章电机，不弱磁的最高转速由式 (95-15) 给出：用 173.205 V 算出 4873 r/min，用 190.986 V 算出 5373 r/min。弱磁极限也一样放大：12126 r/min 变成 13370 r/min。也就是说，同一台电机、同一个母线，只把调制策略从“线性 SVPWM”推到“六步”，工作区间整体右移 10.3%。

代价在谐波上。过调制区的输出电压不再是纯正弦，5 次、7 次谐波迅速上升，电流 THD 从线性区的百分之几涨到十几个百分点。这些谐波在电机里变成铁损与转矩脉动，在低速时还会引起额外的噪声。所以过调制的正确用法是“只在高速段短暂使用”，而不是全程打开：低速用线性 SVPWM 保证平稳，高速需要那 10.3% 时才切入过调制，切入时把电流环带宽临时降一档，避免谐波被 PI 放大。

与弱磁叠加时还有一处顺序问题。弱磁的目标是“把电压需求压到可用电压以下”，过调制的目标是“把可用电压抬高”。两者同时动作，会出现“弱磁算出来的 i_d 是按 173 V 算的，实际有 191 V 可用”的浪费——表现为高速段 i_d 给多了，铜损白白增加。正确的做法是让弱磁模块读取“当前实际可用电压”，而不是写死 173.205 V。

95.9.8 电流环带宽的三条提升路径

1 kHz 的电流环对 76.78 Hz 的速度环绰绰有余。但如果工艺要求更高（比如需要 200 Hz 以上的速度环），就必须把电流环推到 2 kHz 以上，这时候三条路各有各的代价。

第一条，提高开关频率。 按式 (95-28)，2 kHz 电流环在 20 kHz 开关下只有 $90 - 540 \times 2/20 = 36°$，低于 45° 的底线；把开关提到 40 kHz，裕度回到 63°。代价是开关损耗翻倍，驱动器效率掉两三个点，散热要重新算。这条路最直接，也最贵。

第二条，减少延迟拍数。 表 95-5 显示，每多一拍延迟，裕度掉 18°。反过来，把“采样—计算—装载”压到同一拍内完成（用 PWM 中点采样 + 立即更新），就能从 1 拍降到 0 拍，20 kHz 下裕度从 62.89° 回到 80.96°。这条不花钱，但要改硬件时序，很多通用 MCU 的 PWM 外设做不到“立即更新”。

第三条，换控制结构。 无差拍（deadbeat）预测控制能让电流在一拍内到位，理论上把延迟的相位损失全部补回。代价是它对 L、R 的准确性极敏感：电感偏 30%，无差拍就会从“一拍到位”变成“两拍振荡”，比 PI 还差。所以工业产品里无差拍通常与 PI 并联使用，只在参数可信的频段交给它。

三条路的性价比排序是：减延迟 > 提开关频率 > 换结构。减延迟不花钱、收益最大，但受硬件限制；提开关频率花钱、收益确定；换结构省钱但脆弱，只在高端驱动器上见得到。

95.9.9 五种手段放在一起会不会打架

单独用都好，叠在一起就有三处冲突。

死区补偿与过调制。 死区补偿靠 $\mathrm{sign}(i)$ 判断方向，前提是波形接近正弦、电流过零清晰。过调制区波形被削顶，过零点附近电流变化很慢，$\mathrm{sign}(i)$ 会在零附近反复翻转，补偿量跟着抖，反而注入高频谐波。所以过调制生效时应把死区补偿的增益降到一半以下，或改用“按电流幅值渐变”的软化符号函数。

MTPA 与弱磁。 两者都在分配 i_d 与 i_q，共用一张电流圆。若各自独立算，会出现“弱磁要 $i_d = -3$ A、MTPA 要 $i_d = -1$ A”的矛盾。正确做法是统一成一个优化：先满足电压圆（弱磁），再在剩余电流里按 MTPA 分配。

负载观测器与无位置传感器。 两者都在“估”：一个估转矩，一个估角度，而角度又要用到转矩模型。串在一起容易互相激励，形成低频振荡。工程上通常只用其中一个。

一个可行的组合顺序是：低速段开死区补偿与转矩前馈；中速段加负载观测器；高速段关掉补偿、切过调制与弱磁。四段之间有重叠区，重叠区里各手段的增益都取一半，做平滑过渡。这个“分段 + 淡入淡出”的做法听起来不优雅，但它比任何单一算法都稳。

95.10 工程落地

落地阶段要处理的不是方程，而是热、电、机械、电磁兼容这四件事。它们不写在传递函数里，却决定了这台伺服能不能连续跑三年。

95.10.1 一套配置

围绕本章的 750 W 电机，一套完整的配置可以列成表 95-9。

部件	规格	依据
伺服电机	750 W、3000 r/min、2.277 N·m、4 对极	§95.3.1 的转矩与转速
驱动器	300 V 母线、15 A 峰值、20 kHz	§95.3.2、§95.3.3
编码器	18 位绝对值、差分输出	§95.3.5 的低速测速需求
制动电阻	按泵升能量选，本章 7.40 J	§95.10.3
进线滤波器	满足 C2 级传导发射	§95.10.4
抱闸	失电制动，垂直轴必备	§95.10.5

表 95-9 的每一行都能回指到前文的某一节。这张表的价值不在“买什么”，而在“为什么是这些数”：换一台 400 W 电机，母线可以降到 200 V，但编码器位数不能降——低速测速的需求与功率无关。

95.10.2 热：25 分钟才热透

电机的温升是一阶过程。本章按时间常数 25 min 建模，稳态温升与电流平方成正比：

$$\Delta\theta = \Delta\theta_{\text{rated}} \left(\frac{T_{\text{rms}}}{T_{\text{rated}}} \right)^2 \tag{95-34}$$

满负荷 60 K，本章工况 RMS 1.470 N·m（64.5%）对应 25.0 K。把 RMS 比例扫一遍：50% 时 15.0 K，70% 时 29.4 K，85% 时 43.3 K，100% 时 60.0 K。图 95-10 左图是一个 2 s 节拍的负载波形，右图是它对应的温升曲线。

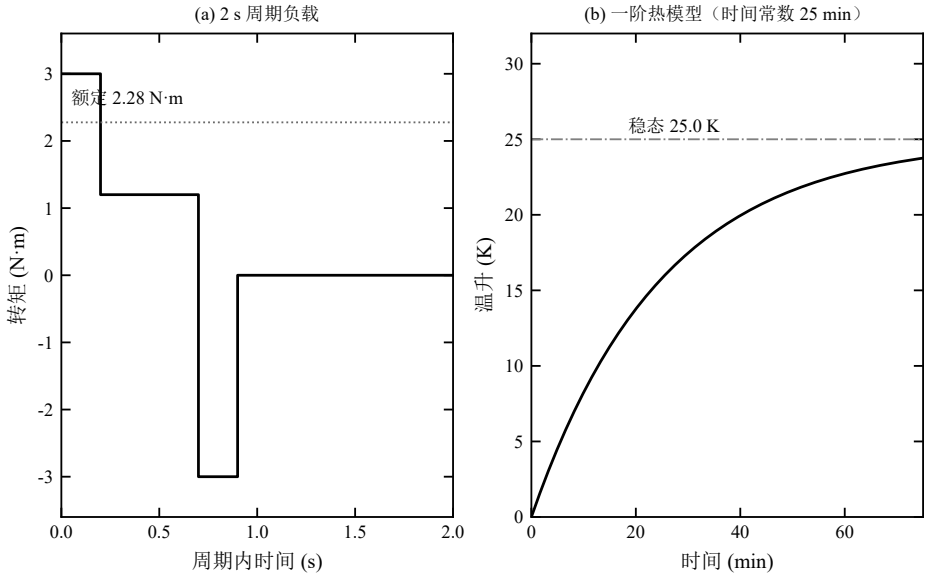


图 95-10　周期负载与一阶热模型的温升

两个工程含义。第一，热时间常数 25 min 远大于节拍 2 s，所以短时峰值（1.32 倍额定、持续 0.2 s）对温升几乎没有贡献——这就是“驱动器按 3 倍过载 3 s 选”的依据。第二，温升按平方增长，RMS 从 70% 提到 85% 只涨 21% 的转矩，却让温升从 29.4 K 跳到 43.3 K，多了 47%。所以余量要留在 RMS 上，而不是留在峰值上。

95.10.3 母线泵升与制动

减速时电机变成发电机，能量回灌母线，电容电压被顶高。本章轴系的动能是

$$E_k = \frac{1}{2} J \omega_m^2 = \frac{1}{2} \times 1.5 \times 10^{-4} \times 314.16^2 = 7.40\ \text{J} \tag{95-35}$$

7.4 J 很小，靠母线电容自身就能吸收，不需要制动电阻。这也是小惯量伺服与电梯、离心机这类大惯量设备的根本区别：后者的回馈能量是千焦量级，必须配制动单元或能量回馈装置。

垂直轴（比如 §95.1 的贴片机 Z 轴）另有一条规则：断电时必须先抱闸、再断使能。顺序反了，重力会让轴下滑一段再被闸住，这段“下滑”在精密装配里就是撞件。

95.10.4 电磁兼容：三个物理事实

第一，PWM 输出电压的跳变率很高。300 V 母线、100 ns 上升沿，dv/dt 达到 3 kV/μs，电缆成了天线，辐射频段覆盖 150 kHz–30 MHz。

第二，编码器信号是毫伏级差分信号，与动力线捆在一起走，噪声会直接进位置反馈——表现为“位置偶尔跳一下”，严重时驱动器报“编码器通讯错误”。

第三，接地环路。驱动器、电机、控制器若分别接地，屏蔽层里会流过地电流，反而把噪声耦合进信号线。

三条对策对应三个物理事实：动力线用屏蔽电缆且两端接地；编码器线单独走线槽、与动力线间距 20 cm 以上；全系统单点接地，屏蔽层只在驱动器侧接地。这三条不花钱，但不做就会出现“实验室好好的、装到机上就报警”。

95.10.5 机械安装：对中与刚度

联轴器的对中误差会以每转一次的频率（1×）出现在转矩里，表现为 3000 r/min 时 50 Hz 的振动。允许的径向偏差通常是联轴器外径的 1%–2%，超过这个值，轴承寿命按指数下降。

传动链的刚度决定谐振频率。§95.1 的丝杠系统，反谐振点通常在 200–500 Hz，远高于位置环 13.95 Hz，所以本章可以按刚体建模；若换成同步带（刚度低一个数量级），谐振点可能掉到 50 Hz 附近，与速度环 76.78 Hz 撞上，就必须加陷波器或降带宽（第 94 章 §94.6 有完整推导）。

95.10.6 验收清单

装完要验收，验收要可量化。表 95-10 是本章建议的六项。

项目	方法	判据
静态定位	激光干涉仪测 10 点	误差 ≤ 5 μm
重复定位	同一点往返 30 次	极差 ≤ 2 μm
跟随误差	匀速段读编码器偏差	≤ 3.925 rad @ 0.5 m/s
阶跃响应	1 mm 阶跃，看超调	≤ 5%，调节 ≤ 60 ms
温升	满载 2 h 测外壳	≤ 45 K
传导发射	按 C2 级扫频	不超限值

表 95-10 的四项动态指标里，“跟随误差”最容易被忽略，因为它要在匀速段读，而不是在停止时读。停止时误差接近零，看起来一切正常；一动起来就落后 6.25 mm，这才是前馈要解决的问题。

验收通过不代表长期可靠。真正的考验在三个月后：润滑脂流失让摩擦变大、磁钢温度循环让 K_t 漂移、编码器码盘积灰让信号丢失。§95.11 会把这类“时间维度”的边界一并说清。

95.10.7 电缆长度：30 m 是一条线

§95.10.4 说 PWM 输出的 dv/dt 达 3 kV/μs，这句话在短电缆上只是“辐射骚扰”，在长电缆上会变成“绝缘杀手”。

原因在行波。电缆里电磁波的传播速度约 150 m/μs，上升沿 100 ns 对应的空间长度是 15 m。当电缆超过这个长度，上升沿还没走完整个电缆，驱动器端的电压就已经跳变完毕；电机端的阻抗远高于电缆特性阻抗（约 100 Ω），电压到达末端后全反射，叠加成接近两倍的入射电压。300 V 母线，电机端就可能看到 600 V 的尖峰，再叠加振荡，实测常见 1.8–2.2 倍。

所以“电缆超过 30 m 必须加输出电抗器或 dv/dt 滤波器”这条规范不是保守，是行波理论算出来的：30 m 正好是 15 m 的往返长度，超过它，反射波与入射波在电机端叠加得最充分。本章这套 300 V 系统，绕组按 IEC 60034-25 的 1000 V 峰值设计，600–660 V 的尖峰还在范围内；若母线是 600 V，尖峰就到 1200 V，必须加滤波器。

工程上的三条做法：电缆尽量短（把驱动器装到电机附近）；必须长时用屏蔽双绞的三相电缆，屏蔽层两端接地；超过 30 m 加输出电抗器，超过 100 m 加 dv/dt 滤波器。三条都不影响控制性能，只影响寿命。

95.10.8 母线电容与预充电

母线电容有两个作用：吸收 PWM 的脉动电流，撑住减速时的泵升能量。两个作用对应两个不同的选型依据。

按纹波电流选。 逆变器每个开关周期都从电容抽一次脉冲电流，有效值约为输出相电流的 30%–50%。本章额定 5.3 A，电容纹波电流约 1.6–2.7 A。电解电容的纹波电流额定值随频率与温度变化，选型时要按 20 kHz、85 ℃ 的条件查，不能按 100 Hz 的工频值。

按泵升能量选。 §95.10.3 算过，本章轴系动能 7.40 J。电容储能 $\tfrac{1}{2}CU^2$，若允许母线从 300 V 升到 350 V，可吸收的能量是 $\tfrac{1}{2}C(350^2-300^2)=1.625\times10^4 C$。要让 7.40 J 对应 50 V 的升幅，需要 $C=7.40/1.625\times10^4=455$ μF。取 470 μF 标准值，实际升幅约 49 V，还在电容的 400 V 耐压之内。

上电瞬间，470 μF 的电容对 300 V 母线是短路，涌流只受线路电阻限制，可达数百安。必须加预充电：串一个 5–10 Ω 的电阻，时间常数 $RC=5\times470\times10^{-6}=2.35$ ms，三个时间常数（约 7 ms）充到 95%，然后继电器短接电阻。省掉预充电的后果是继电器触点烧蚀、保险丝熔断，或者整流桥直接击穿。

电容的寿命按 §95.11.4 的式 (95-37) 估。这台驱动器内部温升若比环境高 20 K，55 ℃ 环境下电容工作在 75 ℃，5000 h 的标称寿命变成 $5000\times2^{3}=4$ 万小时（约 4.5 年）。所以电柜的通风不是“可选配件”，它直接决定驱动器的更换周期。

95.10.9 三个必测点与仪器带宽

调试伺服至少要在三个点上探针，少一个就会在出问题时瞎猜。

母线电压。 看两件事：稳态纹波（应为母线电压的 1%–3%）与减速时的泵升。本章 7.40 J 对应 49 V 的升幅，若实测超过 80 V，说明电容偏小或制动电阻没接。用直流档加交流档各测一次，交流档看纹波，直流档看泵升。

相电流。 看三件事：纹波峰峰值（20 kHz、$D=0.5$ 时 1.500 A）、波形正弦度（过零点是否有平顶）、三相平衡。过零平顶就是 §95.6.3 的死区，正弦度差就是解耦没做好或角度偏。

编码器差分信号。 用差分探头看 A/B 的眼图，重点看上升沿有没有台阶、共模上有没有 PWM 的毛刺。位置偶发跳变时，这一处的证据比任何日志都直接。

仪器带宽有硬要求。20 kHz PWM 的上升沿约 100 ns，按“带宽 ≥ 3.5/上升时间”的经验，示波器需要 35 MHz 以上；要看清楚振铃与死区细节，实际应选 100 MHz 以上。用 20 MHz 的入门示波器看，上升沿被滤成 17.5 ns 以上，死区 2 μs 看起来“很干净”，实际上把问题都藏起来了。

电流测量优先用闭环霍尔或分流电阻加隔离放大，不用开环霍尔——后者的带宽通常只有 20–50 kHz，正好把 20 kHz 的纹波削掉一半，看到的“纹波很小”是假象。

95.10.10 电柜里的布置

同样是这套器件，装在通风良好的电柜里能用十年，装进密闭小箱里两年就坏。差别不在器件，在布置。

三条布置规则。第一，驱动器之间留 50 mm 以上的间距，且竖直安装——热量是向上走的，横着装等于让下面那台烘上面那台。第二，进风口加滤网、出风口不装滤网，形成“正压”：柜内压力略高于外部，粉尘进不来。反过来（出风口装滤网）会让柜内负压，从缝隙里吸灰。第三，伺服驱动器与变频器分柜或分区，因为后者的开关频率低、谐波大，容易通过地线干扰前者的编码器信号。

柜内温度按“环境 + 20 K”估驱动器内部温度。若车间夏天 40 ℃，柜内就是 60 ℃，驱动器内部 80 ℃，按式 (95-37)，5000 h 的电容只剩 $5000\times2^{2.5}=2.8$ 万小时（约 3.2 年）。加一台 100 W 的柜内空调或热交换器，把柜内压到 35 ℃，寿命立刻回到 6 年以上。这笔钱通常比一次停机损失小得多。

95.10.11 首次上电的七步

新机器装完，最危险的不是运行，是第一次上电。七步走完再给使能，能把绝大多数“炸机”挡在前面。

第一步，断动力、只上控制电。 确认驱动器自检通过、面板能读到编码器角度。此时用手盘轴，角度应单调变化；若跳变，先查编码器线序与屏蔽，不要往下走。

第二步，测绝缘与相序。 用兆欧表测 U/V/W 对地，读数应大于 1 MΩ；再用万用表测三相对称，偏差不超过 5%。相序接反的后果是电机反转，而反转在闭环里表现为“正反馈”，一使能就飞车。

第三步，设电流限幅到 10%。 本章 $I_{peak} = 15$ A，先限到 1.5 A。这一步的意义是：后面任何一步出错，最坏结果只是“嗡一下”，不是“烧电机”。

第四步，开环给一个小电压，看转向与电流。 给 $v_q = 5$ V、$v_d = 0$，电机应以很低转速匀速转。此时读相电流，三相应平衡；某一相明显偏小，说明该相功率管或驱动电路有问题。转向与预期相反，改“编码器方向”参数，而不是改接线——改接线会同时改掉霍尔与温度传感器的参考。

第五步，闭环电流环，给定 1 A 阶跃。 看响应是否在 1 ms 量级稳定、有无持续振铃。这一步验证的是 §95.5 的离散化与 §95.7.9 的标定：K_{pi} 与 K_{ii} 若按实测 L、R 算错，这里会立刻暴露。

第六步，速度环，从 60 r/min 起。 先给 60 r/min——这个转速下反电动势 1.8 V，死区 12 V 仍占主导，正好检验 §95.9.1 的补偿是否生效；再上到 3000 r/min，看高速段是否因电压不足而掉速。两次都对，才说明电压圆与电流圆都留够了。

第七步，位置环，先小行程。 给 1° 阶跃，看是否一次到位、有无过冲；再给 360° 连续定位，看重复精度。本章位置环 13.95 Hz、$K_{p\theta} = 80\ \mathrm{s}^{-1}$，1° 阶跃的调节时间应在 50 ms 量级（与 §95.6.7 的 50.41 ms 对应）。

七步的顺序不能颠倒：每一步都是下一步的前提，而每一步都在“限幅最小”的状态下做。**上电调试的全部技巧，就是让每一次可能的错误都发生在能量最低的时候。**

95.11 优缺点与适用边界

前面十节都在讲“怎么把一台伺服调好”。这一节反过来问三个问题：这套方法在什么前提下成立，它在什么场合是过度设计，以及它跑上三年会怎么坏。三个问题的答案，决定了读者拿到一个新项目时，第一步该选什么电机、第二步该留多少余量、第三步该在维护手册里写什么。

95.11.1 五类电机的能力矩阵

表 95-1 给的是定性分工，这里换成带数的版本。五类电机都按“能输出 2.277 N·m、连续运行”这个共同目标来比较，数据全部来自本章脚本的同一套口径。

项目	直流伺服	PMSM（本章）	BLDC（六步）	步进	感应
额定转矩来源	$K_t i_a$	$1.5p\lambda i_q = 0.4296 i_q$	两相导通，$2\lambda i$	保持转矩 1.2 N·m	转差与磁链
达到 2.277 N·m 所需电流	视 K_t 而定	5.3 A（峰值）	5.3 A 量级	无法连续（1000 步/s 仅 0.364）	1.0 p.u.，1455 r/min
连续转速上限	受电刷限制	4873 r/min（额定电流）	同 PMSM	300 r/min 时转矩剩 30.3%	1500 r/min（50 Hz）
高速能力	中	12126 r/min（转矩归零）	同 PMSM	3000 r/min 时剩 3.2%	3000 r/min 时最大转矩 0.65 p.u.
低速平稳性	好（有齿槽）	好，纹波 28.3% 可滤	差，脉动 14.03%	差，56.8 Hz 附近共振	中，低速效率低
过载能力	2–3 倍	2.83 倍（15 A/5.3 A）	同 PMSM	无（失步即丢步）	2.60 倍（最大转矩）
反馈需求	编码器	编码器或霍尔	霍尔即可	无	编码器（矢量控制）
维护量	电刷	轴承	轴承	轴承	轴承

表 95-11 里最刺眼的一行是“过载能力”。PMSM 的 2.83 倍不是纸面数字：15 A 峰值对应 6.444 N·m，是额定 2.277 N·m 的 2.83 倍，这个倍数直接决定了加减速段能不能做短。步进电机的对应栏是“无”——它没有闭环，转矩不够就失步，失步之后控制器并不知道自己丢了位置。这一条差异，比转矩密度更能解释“为什么精密设备都用伺服”。

第二刺眼的是转速。步进电机在 300 r/min 时只剩 30.3% 的保持转矩（0.364 N·m），3000 r/min 时只剩 3.2%（0.038 N·m）；同一转速下本章 PMSM 仍是满转矩 2.277 N·m，相差 60 倍。根源在 §95.2.5 的矩频公式：步进的转矩衰减来自 $2\pi f_e L/R$ 这个无量纲数，1000 步/s 时它已经到 3.14，转矩被 $\sqrt{1+3.14^2}$ 除掉大半。

95.11.2 三个坐标轴上的边界

选型不是选“最好的电机”，而是选“在这三个坐标上都不越界的电机”。

转速轴。 PMSM 的可用区间分三段：0–3000 r/min 恒转矩，3000–4873 r/min 恒功率（受电压限制，i_q 必须减小），4873 r/min 以上靠弱磁，到 12126 r/min 时 i_q 已归零、转矩为零。第三段只有转速没有力矩，工程上没有意义，真正可用的是前两段。若工艺要求 6000 r/min 且要带载，本章这台电机不够——弱磁表里 6000 r/min 的 i_q 已经是 0，需要换低反电动势（更少极对数或更小磁链）的型号。

转矩轴。 连续转矩由热决定（§95.10.2 的 60 K 温升），峰值转矩由电流与磁路饱和决定（15 A）。两者之比 2.83 是“短时过载”的额度，但额度有时间限制：热时间常数 25 min（1500 s），意味着 3 秒的 2.83 倍过载把 25 min 窗口的 RMS 只从 1.000 抬到 $\sqrt{(1497 + 3\times 2.83^2)/1500} = 1.007$，温升涨 1.4%，可以忽略；而持续 5 分钟的 2 倍过载，RMS 抬到 $\sqrt{(1200 + 300\times 4)/1500} = 1.265$，温升涨 60%。驱动器标称的“3 倍过载 3 s”正是按这个逻辑定的：额度给得越短，能给的倍数越大。

精度轴。 18 位编码器给出 2×10^{-5} rad 的分辨率，配 10 mm 导程丝杠，理论最小位移 0.032 μm；实际精度被丝杠导程误差（C7 级 50 μm/300 mm）、反向间隙、热伸长吃掉，编码器反而不是瓶颈。反过来，若用 13 位编码器（7.7×10^{-4} rad），折算位移 1.2 μm，与丝杠误差同量级，就开始拖后腿。这也是 §95.10.1 那张配置表里“编码器位数不能降”的含义。

三个轴的交点才是一台电机的适用域。任何一个轴越界，换电机比调参数有效——这是本章与第 44 章 PID 调参最本质的区别：PID 能改动态品质，改不了物理边界。

95.11.3 成本与体积：贵在哪

同样输出 2.277 N·m，三条路线的成本结构完全不同。

PMSM 贵在磁钢与编码器。钕铁硼磁钢占材料成本的三到四成，18 位绝对值编码器往往比电机本体还贵。本章这套 750 W 配置，电机与驱动器大致对半，编码器又占电机的一半。

步进便宜在“没有闭环”：没有编码器、没有电流传感器（开环细分驱动）、驱动器只是一片集成桥。但它的转矩在 300 r/min 就掉到 30.3%，要用在 3000 r/min 的轴上，必须配 10:1 减速机，减速机把省下的钱又花回去了，还引入背隙。

感应电机便宜在“没有磁钢”，转子是铸铝笼条，耐高温、耐过载、几乎不失效。它的短板是效率与功率因数：同样 1.5 kW，感应电机的体积通常是 PMSM 的 1.5–2 倍。风机泵类不在乎体积，所以它是主力；机床进给在乎，所以见不到。

一条经验：**功率越小、精度越高，PMSM 的相对优势越大；功率越大、精度越松，感应的相对优势越大。** 分界线大致在几千瓦、零点几毫米精度这一带。

95.11.4 时间维度：四类磨损与两个寿命公式

验收清单（表 95-10）测的是出厂状态。三年后的状态由四类磨损决定，每一类都有可算的时间尺度。

电刷。 只有直流电机有。电刷寿命按“总转数”计，典型 2000–5000 小时；3000 r/min 连续运行，就是 3.6–9 亿转。这个寿命在产线上意味着每季度换一次碳刷，换刷期间的停机与人工，往往超过电机本身的价钱。这是直流伺服退出通用市场的真正原因，不是性能不行。

轴承。 所有电机都有。基本额定寿命按 $L_{10} = (C/P)^3$ 计，单位百万转：

$$L_{10} = \left(\frac{C}{P}\right)^3 \times 10^6 \text{ 转} \tag{95-36}$$

P 是当量动载荷。若联轴器对中误差让 P 增大一倍，寿命掉到八分之一——这就是 §95.10.5 说“对中误差超过 1%–2%，轴承寿命按指数下降”的量化依据。指数是 3，不是线性。

润滑脂。 轴承厂给的寿命通常按 20000 小时、70 ℃ 基准；温度每升 15 K，寿命减半。本章电机外壳温升 25 K、环境 40 ℃，脂温约 65 ℃，尚在基准附近；若堵转或散热不良让脂温到 95 ℃，按 $2^{25/15} = 3.2$ 折算，寿命只剩三成，表现为“两年后开始有异响”。

母线电容。 电解电容的寿命按每升 10 K 减半的 Arrhenius 经验估：

$$L = L_0 \cdot 2^{(T_0 - T)/10} \tag{95-37}$$

标称 105 ℃/5000 h 的电容，工作在 55 ℃ 时是 $5000 \times 2^5 = 16$ 万小时（约 18 年），工作在 85 ℃ 时只剩 $5000 \times 2^2 = 2$ 万小时（约 2.3 年）。驱动器内部温度通常比环境高 20–25 K，所以“驱动器装在密闭电柜里”与“装在通风处”，电容寿命差一倍以上。这条也解释了为什么 §95.10.2 要盯着 45 K 的温升判据。

四类磨损里，只有电刷是“设计时就该避免”的，其余三类是“设计时该留余量”的。留余量的方式不是加大电机，而是把温度、对中、散热这三件事做到位。

95.11.5 失效模式与早期征兆

表 95-12 把常见失效按“征兆—机理—判据”排开。这张表的用法是写进维护手册，而不是等坏了再查。

失效	早期征兆	机理	可测判据
磁钢失磁	同样负载电流变大、温升变高	高温或大电流退磁	K_t 下降 >5%（本章即 0.4296→0.408）
绝缘老化	漏电流上升、偶发过流	热循环与 dv/dt 冲击	绝缘电阻 <10 MΩ
码盘污染	位置偶发跳变、通讯报警	油雾、粉尘遮挡	单圈内跳变 >10 个计数
电容干涸	母线纹波变大、低速抖动	电解液挥发	纹波电压上升 >50%
轴承磨损	1× 与 BPFO 频率的振动峰	疲劳剥落	振动速度有效值 >2.8 mm/s
抱闸打滑	断电后下滑距离变长	摩擦片磨损、间隙变大	下滑 >0.1 mm

表 95-12 的第一行值得单独说。K_t 下降 5% 在控制上几乎看不出来（速度环会自己补上电流），但铜损按 $1/0.95^2 = 1.11$ 倍上升，温升按 §95.10.2 的平方关系再涨 11%。于是出现一个恶性循环：失磁→电流变大→温度更高→失磁更快。脚本里 λ 偏 −10% 时电流要 1.11 倍、铜损 1.23 倍，就是这个循环的第一圈。判据里写“K_t 下降 5%”而不是“10%”，是因为 5% 时还有干预余地。

第三行也值得说。码盘污染导致的“位置偶尔跳一下”，在示波器上看是一个几十微秒的毛刺，在位置上是一个 0.032 μm 的跳变——单看一次无害，但它会通过 §95.6.4 的 30 mA 量化噪声通路进入电流指令，表现为“电机偶尔哼一声”。这类问题的排查顺序是：先看编码器通讯误码计数，再看屏蔽与接地，最后才怀疑电机。

95.11.6 什么时候伺服是过度设计

三种场合，用伺服是浪费。

定位精度松于 0.1 mm。 步进加 16 细分，单步 1.963 mrad，配 10 mm 导程丝杠是 3.1 μm 的理论分辨率，实际受背隙与失步影响在几十微米量级。对 0.1 mm 的场合绰绰有余，成本只有伺服的三分之一。

转速需求低于 300 r/min。 步进在 300 r/min 时还有 30.3% 的保持转矩（0.364 N·m），只要负载转矩不超过这个数，就不需要闭环。3D 打印机的送丝轴、小型点胶机的 Z 轴都属于这一类。

负载恒定、无需过载。 伺服的 2.83 倍过载能力在恒转矩负载上用不到，而它贵就贵在这份能力（大电流驱动器、高分辨率编码器、复杂算法）。传送带、风机这类场合，感应电机加变频器更合适。

反过来，三种场合适配伺服，且不能用步进替代：**需要零速保持大转矩**（垂直轴，步进零速时的保持转矩就是 1.2 N·m，没有余量）；**需要高速恒转矩**（3000 r/min 带载，步进只剩 3.2%）；**需要知道“有没有丢步”**（闭环才有位置可信度）。

95.11.7 本章方法的六个前提

§95.3 到 §95.8 的所有结论，都建立在六个前提上。前提破了，结论就不成立，这时候要做的是换方法，不是调参数。

第一，**对象是线性的**。电流环按 $1/(Ls+R)$ 设计，前提是电感不随电流饱和。实际电机在 15 A 时电感可能掉 20%–30%，表现为大电流时电流环变快、相位裕度变大——反而不振荡，但小电流时按大电流整定的参数会偏慢。若电感变化超过一倍，就得做增益调度。

第二，**参数是已知的**。K_t、L、R、J 都要进控制器。R 随温度漂 40%，脚本显示这对电流环带宽只影响 1%（1001.62→991.67 Hz），可以忽略；J 随负载变化可能是几倍，这就要用 §95.9.4 的观测器或自适应。

第三，**传动链是刚体**。§95.10.5 说过，反谐振 200–500 Hz 时才成立。若谐振掉到 50 Hz，与速度环 76.78 Hz 撞上，必须加陷波器——那是第 94 章 §94.6 的内容。

第四，**没有饱和**。§95.6.2 已经说明，电压饱和后系统变成一个非线性系统，积分会 windup。抗饱和是必需项，不是可选项。

第五，**单轴独立**。多轴联动时，一个轴的跟随误差会影响另一个轴的轨迹精度，这时要谈轮廓误差而不是单轴误差，见第 98 章。本章的所有指标都是单轴指标。

第六，**轨迹是给定的**。本章只关心“给定一条轨迹，跟得多准”，不关心“这条轨迹怎么来”。加减速规划、拐角过渡、前瞻，是第 97 章的内容；轨迹要落在关节空间还是笛卡尔空间，要先有第 96 章的运动学。

95.11.8 小结

伺服电机与控制的适用边界，可以压成三句话。

物理边界决定选型，控制方法决定品质。 转速、转矩、精度三个轴越界，换电机；三个轴都在界内，才轮到调参数。

过载能力与低速平稳性是伺服相对步进的两张王牌。 2.83 倍过载对应 6.444 N·m，低速纹波 28.3% 还能被 1 kHz 电流环压住；步进这两项都拿不出来。这两条差异，比“伺服精度高”这种笼统说法更能指导选型。

长期可靠性由温度、对中、散热三件事决定，而不是由控制算法决定。 25 K 温升、1%–2% 对中、45 K 外壳判据，这三个数做到了，三年后的故障率就低；做不到，再好的三环整定也救不回来。

95.11.9 选型的三问

前面八小节的内容，最后可以压成三个问题。拿到一个新项目，按顺序问下去，五分钟内就能定下电机类型。

第一问：需要零速保持转矩吗？ 需要，就只能选伺服（或带电刹的步进）。步进的“保持转矩 1.2 N·m”看似能用，但那是没有余量的极限值，一旦有振动或外力就会失步。垂直轴、夹紧轴、机械手的腕部，都属这一类。

第二问：最高转速时还要带载吗？ 要，且转速超过 600 r/min，就选伺服。步进在 600 r/min（2000 步/s）只剩 15.7% 的保持转矩，本章的伺服在同一转速仍是 100%。若负载很轻（低于 0.19 N·m），步进仍可用，这时要按矩频曲线逐点校核，不能只看保持转矩。

第三问：定位精度要求高于 0.1 mm 吗？ 不高于，步进足够，且成本只有三分之一；高于，且需要长时间稳定，选伺服。

三问都指向步进时，还要补一条：**会不会丢步而无人知晓？** 如果丢了步会导致撞机、报废或安全问题，就必须加编码器闭环——那其实已经是伺服了。三问都指向伺服时，再按 §95.11.2 的三个坐标轴选具体型号。

这套问答不能替代计算，但它能把 80% 的选型错误挡在计算之前。真正的错误往往不是“算错了转矩”，而是“一开始就选错了电机类型”。

95.11.10 三条产线的选型对照

三问给的是定性方向，落地时还要填数。用本章同一套口径，看三个典型工位怎么走完这段路。

工位一：贴片机横移轴。 行程 300 mm，定位 0.02 mm，节拍要求 0.15 s 内走完。按梯形速度曲线，加速度至少 $2 \times 0.3/0.15^2 = 26.7$ m/s²，接近 2.7 g——这是本章 §95.3.6 用的那个 3 g 工况的亲戚。折算到丝杠端，负载转矩约 0.66 N·m，加速转矩 1.84 N·m，合计 2.277 N·m 恰好是额定值，没有余量，所以实际选型要留 1.5 倍到 3.4 N·m，或者把丝杠导程从 10 mm 减到 5 mm 再算一遍。这一档的瓶颈是惯量匹配与机械刚性，不是电机转矩：300 Hz 的反谐振一旦落在加速激励的频谱里，定位就永远进不了 0.02 mm。

工位二：数控转台。 转速 50 r/min，连续转矩 8 N·m，要求零速锁紧、分度精度 0.001°。这一档的难点全在低速：50 r/min 时反电动势只有 1.5 V，12 V 的死区误差是它的 8 倍，不补偿就是“爬行—停顿—爬行”。转矩按 8 N·m 反推，需要 $8/0.4296 = 18.6$ A，超过本章电机的 15 A 峰值，所以要么换大机座，要么加减速比。精度 0.001° 对应编码器 18 位（$\pm 2 \times 10^{-5}$ rad 即 0.0011°），刚好卡在边缘，实际要上 20 位或加双编码器。

工位三：包装机推料。 行程 80 mm，定位 0.5 mm，节拍 0.6 s，负载恒定 0.5 N·m。三问全部指向步进：不需要零速保持（有机械限位）、600 r/min 以下、精度远松于 0.1 mm。按矩频曲线选 1.2 N·m 的 57 机座，200 步/s（60 r/min）时还有 84.7% 即 1.016 N·m，是负载的 2 倍；1000 步/s（300 r/min）时剩 30.3% 即 0.364 N·m，已经低于 0.5 N·m 的负载，所以必须限制最高速，或改用 16 细分降低单步冲击。这一档真正的风险是丢步无感知——加一个光电开关做每循环回零，成本远低于换伺服。

三条线放在一起看，有个共同点：**决定成败的都不是"额定转矩够不够"，而是某一项边界条件。** 工位一是惯量与刚性，工位二是低速死区与编码器分辨率，工位三是矩频曲线的高频段。额定转矩只是入场券，边界条件才是分数。

选型表填完之后还要补一行字：把每个边界条件的**实测值**写进去，而不是设计值。设计值只能证明"应该行"，实测值才能证明"确实行"；这一行字，就是日后复盘的唯一凭据。

下一节交代码。本章所有数字都能在 `code/ch095_sim.py` 里找到出处，图都能在 `code/ch095_figs.py` 里找到生成过程。

95.12 Python 仿真代码

本章的代码分两个文件，职责不重叠：`ch095_sim.py` 只算数字、只打印，不画图；`ch095_figs.py` 只画图，所有数据从前者 import。这个分工是刻意的——写正文时反复要查数，跑一次 `sim.py` 就能把 14 组读数一次打全；改图时不必重算，也不会有"图上的数和正文的数不一致"。

两个文件都不接受命令行参数，所有可调量集中在文件顶部的常量区。这是本书一贯的做法：要复现某一段，改动常量即可，不必去猜该传什么参数。

95.12.1 常量区：三套口径写在一起

§95.2.3 说 PMSM 有三套 K_e 口径，容易混。脚本的做法是把它们并排定义，注释写清口径，谁也不会拿错：

```
P_POLE = 4                  # 极对数
LAMBDA = 0.0716             # 永磁磁链 (Wb)
KT = 1.5 * P_POLE * LAMBDA  # 转矩常数 0.4296 N·m/A, dq 峰值口径
KE = P_POLE * LAMBDA        # 反电动势常数 0.2864 V·s/rad, 相电压峰值口径
R_S = 0.35                  # 定子电阻 (Ω)
LD = 2.5e-3                 # d 轴电感 (H)
LQ = 2.5e-3                 # q 轴电感 (H)
J_M = 1.5e-4                # 转子惯量 (kg·m²)
B_VISC = 3.0e-4             # 粘性阻尼 (N·m·s/rad)
VDC = 300.0                 # 直流母线 (V)
I_RATED = 5.3               # 额定相电流峰值 (A)
I_PEAK = 15.0               # 峰值相电流峰值 (A)
F_PWM = 20.0e3              # 开关频率 (Hz)
T_DEAD = 2.0e-6             # 桥臂死区时间 (s)
```

`KT` 与 `KE` 都由 `P_POLE` 与 `LAMBDA` 推出来，不单独硬写数值。这样做的意义在于：改磁链时两个常数自动同步，K_t/K_e 恒等于 1.5，不会出现"改了磁链忘了改转矩常数"的错误。本章 [0] 段打印的 `Kt/Ke=1.500` 就是这两个定义的比值，它同时是一个自检项。

步进与感应的参数单独成组，因为它们与 PMSM 无关，不共享任何常数：

```
STEP_ANGLE = 1.8              # 整步角 (°)
STEP_L = 3.0e-3               # 相电感 (H)
STEP_R = 1.5                  # 相电阻 (Ω)
STEP_T0 = 1.2                 # 保持转矩 (N·m)
MICROSTEP = 16                # 细分倍数

IM_P = 2                      # 极对数
IM_F = 50.0                   # 额定频率 (Hz)
IM_SLIP = 0.03                # 额定转差
IM_TMAX = 2.6                 # 最大转矩倍数
```

95.12.2 六个核心函数

脚本里的函数不多，每个对应正文的一节。理解这六个，就把整章的数值链条走通了。

`pmsm_dq(id_, iq, rpm)` 是整章的地基，返回 (v_d, v_q, T, e) 四个量：

```
def pmsm_dq(id_=0.0, iq=5.0, rpm=3000.0):
    we = omega_e(rpm)
    vd = R_S * id_ - we * LQ * iq
    vq = R_S * iq + we * (LD * id_ + LAMBDA)
    torque = 1.5 * P_POLE * (LAMBDA * iq + (LD - LQ) * id_ * iq)
    return float(vd), float(vq), float(torque), float(we * LAMBDA)
```

三行分别对应式 (95-7) 的两个电压方程与式 (95-8) 的转矩。表贴式电机 $L_d = L_q$，转矩式第二项恒为零，这也是 §95.9.3 说“表贴式做 MTPA 是负优化”的代码依据——不是算法不灵，是这一项根本不存在。[1] 段打印的 1500 r/min 时 $v_d = -8.325$ V、$v_q = 46.843$ V，就是这三行算出来的。

`modulation_table(rpm_list, iq)` 把 v_d、v_q 合成幅值再除以 $V_{dc}/\sqrt{3}$，得到调制比。[2] 段显示 3000 r/min 时 0.539、4800 r/min 时 0.856——后一个数逼近 1，说明这台电机在 4800 r/min 附近已经没有余量做动态调节了。

`field_weakening_table(rpm_list)` 是唯一一个“带约束求解”的函数：给定转速，先求维持电压圆所需的 i_d，再用电流圆限制 i_q，最后算转矩。它做了两次 `max/min` 夹取，对应 §95.4.4 说的“电压圆与电流圆同时满足”。返回值里 3000 r/min 是 $(0, 15, 6.444)$，6000 r/min 是 $(-1.074, 0, 0)$——后者说明此时全部电流都用来抵磁链了，一点不剩给转矩。

`three_loops(kp_w, ti_w, kp_p)` 串起三环，返回三个开环与三个闭环：

```
ci = ct.tf([KP_I, KI_I], [1.0, 0.0])      # 电流 PI
li = ci * current_plant()                  # 电流开环
ti = ct.feedback(li, 1)                    # 电流闭环
cv = ct.tf([kp_w * ti_w, kp_w], [ti_w, 0.0])
lv = cv * ti * speed_plant()               # 速度开环（内环用闭环）
tv = ct.feedback(lv, 1)
lp = kp_p * tv * ct.tf([1.0], [1.0, 0.0])  # 位置开环（带积分对象）
tp = ct.feedback(lp, 1)
```

关键在于**内环用闭环代入外环**。很多初学者把三个开环直接相乘，得到的带宽比实际高一个数量级，因为那样等于假设内环是 1。实际内环在 1001.62 Hz 以上就开始衰减，速度环 76.78 Hz 虽然远低于它，但在 76.78 Hz 处内环已经贡献了约几度的相位。这个差别不大，但方向是“实际比理想慢”，不代入就会整定过头。

`discrete_current_loop(ts, delays)` 做三件事：ZOH 离散化、乘 z^{-1} 的延迟、算裕度。它算出的 20 kHz/1 拍 → 62.89°、10 kHz/1 拍 → 35.07°，就是 §95.5.1 那条经验式的来源。注意 `control` 库对离散系统调用 `margin()` 时返回的 ω_{cp} 单位是 rad/s，代码里除了 2π 才是 Hz；这一点若弄错，剪切频率会差 6.28 倍。

`torque_ripple_sixstep(n)` 用 3600 点扫一个电周期，每个角度取三相中最大减最小，得到六步供电的转矩波形。返回的四个数按顺序是最小值 $3/2 = 1.5000$、最大值 $\sqrt{3} = 1.7321$、平均 1.6540、峰峰脉动 14.03%。前三个正说明六步供电的线电压利用率在 $3/2$ 与 $\sqrt{3}$ 之间摆动，第四个才是工程上要看的量。这个函数只有 8 行，却是 §95.2.6 整节的数值支撑。

95.12.3 读数与章节的对应

表 95-13 是查数用的索引：正文里任何一个数字，都能按这张表找到它来自哪一段。

脚本段落	内容	对应章节
[0]	K_t、K_e、K_t/K_e、电气时间常数	§95.2.1、§95.2.3
[1]	四个转速下的 v_d、v_q、$\lvert v\rvert$、反电动势	§95.4.3、图 95-4
[2]	调制比与耦合项对比	§95.4.2、§95.4.3
[3]	电流环连续域裕度与阶跃	§95.4.1、§95.8.2
[4]	离散电流环：延迟拍数、开关频率	§95.5.1、表 95-5
[5]	有/无解耦的 i_d 扰动	§95.4.2、图 95-3
[6]	五个占空比下的电流纹波	§95.5.2、图 95-6
[7]	不弱磁最高转速、弱磁工作点表	§95.4.4、表 95-4、图 95-5
[8]	六步方波的转矩脉动	§95.2.6、表 95-2、图 95-8
[9]	步进矩频特性、谐振估算	§95.2.5、§95.6.5、图 95-7
[10]	感应电机 Kloss 曲线与恒功率区	§95.2.5、§95.6.5
[11]	周期负载 RMS、温升	§95.3.1、§95.10.2、图 95-10
[12]	三环带宽、裕度、位置阶跃	§95.7.4、图 95-9
[13]	死区电压误差、量化噪声、纹波转矩	§95.6.3、§95.6.4
[14]	电阻漂移与磁链偏差的敏感度	§95.7.6

表 95-13 的右列不是一一对应关系：一个段落常常支撑好几节，一节也常常引用好几段。这张表的真正用途是“改数之后该重读哪几节”——比如把 `F_PWM` 从 20 kHz 改成 10 kHz，[4]、[6]、[13] 三段都会变，对应正文的 §95.5.1、§95.5.2、§95.6.3 都要复核。

95.12.4 复现与三个坑

复现只需要两条命令：

```
"D:\Anaconda3\envs\py311\python.exe" book/code/ch095_sim.py
"D:\Anaconda3\envs\py311\python.exe" book/code/ch095_figs.py
```

第一个坑是 `control` 库的版本差异。`frequency_response()` 在 0.10 之前返回 `fresp`，之后改成 `frdata`，直接取属性会报错。脚本里封了一个 `fr()` 兼容两种：

```
def fr(sys, w):
    r = ct.frequency_response(sys, w)
    return np.asarray(getattr(r, "frdata", r.fresp)).squeeze()
```

这个封装还顺手解决了另一个问题：单输入单输出系统返回的数组带一维冗余，`.squeeze()` 之后才能直接做 `np.abs` 和比较。

第二个坑是 `solve_ivp` 的步长。§95.4.2 的解耦对比仿真跑 20 ms，若按默认步长（自适应、可能跨到几百微秒）算，5 A 的阶跃会漏掉前几个开关周期，得到的"无解耦 i_d 偏离"会明显偏小。脚本里把 `max_step` 限到 2 μs，正好是十分之一开关周期，代价是几千步积分——20 ms 内 1 万步，一秒内跑完。

第三个坑是中文路径。`matplotlib` 保存 SVG 时若路径含中文，某些版本会写出损坏的字体引用。脚本用 `Path(__file__).resolve().parents[1]` 定位书稿根目录，全程 `pathlib`，不拼字符串；`save()` 里同时导出 SVG 与 PDF，PDF 用于排版、SVG 用于网页。

95.12.5 换成别的电机

脚本的设计意图是"换电机只改常量区"。把 `P_POLE`、`LAMBDA`、`R_S`、`LD`、`J_M`、`VDC`、`I_RATED`、`I_PEAK` 八个量换掉，[0]–[8]、[12]–[14] 全部自动适配：转矩常数、弱磁表、三环带宽、死区占比都会跟着变。

有两处不会自动适配，需要人工判断。第一处是 §95.3 的 PI 增益——`KP_I = LD * W_CI` 与 `KI_I = R_S * W_CI` 是从 L、R 推的，换了电机它们会自动更新，但 `W_CI = 2π×1000` 这个目标带宽是人为设的，要按新的开关频率重定（经验是 $f_{pwm}/20$）。第二处是速度环的 `KP_W` 与 `TI_W`：它们不是从对象参数推的，而是按"带宽为电流环 1/9.05"反算的，换电机后必须重算，否则 J 变了 5 倍，速度环会明显偏慢或偏快。

本章的代码到此为止。第 96 章会把镜头从"一台电机的轴"移到"多根连杆组成的机构"，那里的方程不再是三个一阶微分方程，而是用拉格朗日法写出的耦合二阶系统；但电流环、速度环、位置环这三层的结构不变，本章的带宽分配原则会原样搬过去。

95.12.6 画图脚本的结构

`ch095_figs.py` 是十个函数加一个 `main()`，每个函数只画一张图，函数名的编号与文件名一致：`fig_04_pwm()` 生成 `fig_095_04_pwm.svg`。这个一一对应不是洁癖，是为了"改哪张图就只跑哪张"——把 `main()` 里对应的一行注释掉，其余九张不受影响。

十张图分两类。第一类是数据图（图 95-2 到图 95-10），数据全部 `from ch095_sim import ...` 拿，不重复计算，所以图上的数与正文的数永远一致。第二类是示意图（图 95-1 的类型包络），它用 `np.linspace` 现场生成几条包络曲线，不依赖 sim。

三处共同的约定。第一，画之前先调 `styles/plot_styles.py` 的 `setup()`，统一字号、线宽、配色；第二，中文标签用纯文本，公式用 `$...$` 交给 mathtext，两者绝不写进同一个字符串——STIX 字体没有中文字形，混排会变成方框；第三，`save()` 同时导出 SVG 与 PDF，SVG 用于网页、PDF 用于印刷，两份矢量图都不掉分辨率。

一条实践建议：改图先改 `sim.py` 的数，再看 `figs.py` 有没有画对。反过来先调图、再补数，很容易画出"图上很漂亮、正文对不上"的结果——那比画错更麻烦，因为它看不出来。

95.12.7 往四个方向扩展

这套脚本刻意做得简单，留下四个明显的扩展口。需要更接近实物的仿真时，按下面的顺序加，收益递减。

加齿槽转矩。 在机械方程里加一项 $T_c\sin(6p\theta_m)$，幅值取额定的 1%–5%。改 `pmsm_dq` 的调用处，或在时域积分里加一项。加上之后低速段会出现周期性的速度波动，可以复现 §95.6.8 的第三个门槛。

加电感饱和。 把 `LD`、`LQ` 从常数改成 $L(i)$ 的查表函数，在 `pmsm_dq` 里按当前电流插值。这一步会让弱磁表与电流环都变，是最接近真实电机的一处改动，也最容易把简单脚本改复杂。

加逆变器非理想。 在电压指令上加死区项（式 95-30 的 12 V）与管压降（IGBT 约 1.8 V），再做限幅。加上之后低速电流波形会出现过零平顶，§95.6.3 的现象就能在仿真里看到，而不是只写在正文里。

加多质量机械。 把 `speed_plant()` 的二阶对象换成二质量模型（两个惯量加一个弹簧），就能复现 §95.2.7 的 300 Hz 谐振。这一步是四者中最有价值的，因为谐振是现场最难排查的问题，而它在单惯量模型里根本不存在。

四个扩展都做完，脚本的行数大约翻倍，但三环的结构与参数不变——这正是本章想说明的：**模型可以越做越细，控制器的骨架不变。**